

TRADEO FABLE MAX AUDIT PACK

A single-file, self-contained LLM audit dossier for Asier to upload to Fable 5 Max / Anthropic when the whole repo cannot be uploaded.

> Canonical source: ``tmp/tradeo-wave4a-merge-100211``, branch ``main``, synchronized with ``origin/main`` at ``0ad200f5d6e8082138c88f0404d464544a7fc652``. ``/home/vboxuser/tradeo`` was not used as source because it was on a working branch.

Table Of Contents

- ? Fable Max Instructions
- ? Source Snapshot And Scope
- ? Filtered Project Tree
- ? Executive Overview For External Auditor
- ? System Architecture At A Glance
- ? End-To-End Flows
- ? Recent Remediation And Wave Ledger
- ? Configuration / Environment Surface
- ? Data Models And Contracts
- ? Research Pipeline Review Map
- ? Lab / Paper / Director Gates Review Map
- ? Execution Safety Review Map
- ? Backtester And Data Provider Review Map
- ? Audit Bridge Review Map
- ? Target Completion Spec: PIT/Delisting Nasdaq Data Link
- ? Known Open Gaps And Desired Expert Review
- ? Test Coverage Map
- ? Python Public API Inventory
- ? Critical Full Source Appendix
- ? Relevant Tests Full Source Appendix
- ? Key Docs And Remediation Full Text Appendix
- ? Audit Bridge Full Text Appendix
- ? Secondary File Summaries And Risks
- ? Final Instructions For Fable Max

Fable Max Instructions

You are Fable 5 Max / Anthropic acting as an adversarial external technical auditor for Tradeo. Review this pack as if the repository cannot be uploaded separately. Do not give a shallow architecture summary. Produce findings that can change engineering decisions.

Required review tasks:

- Trace the Research -> Lab -> Director -> Production/Fox Hunter -> IBKR path end-to-end. Identify any place where a pattern can advance without the evidence the docs claim.
- Trace canonical outcomes and costs from ``RewardRiskAnalyzer``, ``Backtester``, shadow observations, lab paper fills, and audit export. Look for same-bar, gap, skipped-signal, NaN, cost, and sample-count inconsistencies.
- Audit data integrity: point-in-time behavior, universe survivorship, delisting handling, adjusted feed assumptions, partial/incomplete bars, timezone/session gaps, duplicate/unsorted bars, and IBKR pacing/failure semantics.
- Audit statistical validity: multiple testing, global trial count, DSR, CSCV/PBO, nested outer PBO, OOS splits, effective sample weighting, regime calibration, concentration, and rediscovery for legacy patterns.
- Audit execution safety: live/paper gates, kill switch, reconciliation, order-state transitions, max notional, ADV caps, short enablement, human approvals, and whether frontend/API paths can bypass service invariants.
- Audit the audit bridge: CSV/JSON contract, exporter, validator, Director gate, phases 14-17, fill provenance, event/trade/fill reconciliation, independence labels, and secret/account redaction.
- Compare documentation/remediation claims to code and tests. Mark stale claims, overclaims, and claims that rely on pending operator/API validation.
- Review tests for meaningful coverage. Identify tests that assert implementation details without proving the claimed invariant, and missing tests for high-risk paths.
- Review operational config/env defaults. Confirm safety defaults are fail-closed and identify any value that would be dangerous if copied from examples.
- Review frontend/API integration only enough to identify auth, routing, stale display, and operator-misleading states.

Required output format:

- Start with a concise verdict: ``APPROVED``, ``APPROVED_WITH_RESERVATIONS``, ``QUARANTINE``, or ``REJECT_FOR_PRODUCTION``, with one paragraph why.
- Then a findings table with columns: ``id``, ``severity``, ``confidence``, ``area``, ``file/path``, ``line_or_block``, ``evidence``, ``impact``, ``recommended_fix``, ``tests_to_add``.
- Severity must be one of: ``CRITICAL``, ``HIGH``, ``MEDIUM``, ``LOW``, ``OBSERVATION``.
- For every ``CRITICAL`` or ``HIGH``, include a reproducible trace: `inputs/state -> function/path -> wrong or risky output``.
- Include a separate ``Overclaim/Stale Documentation`` section.
- Include a separate ``Open Gaps That Are Honest, Not Bugs`` section.
- Include a separate ``Questions for Asier`` section only for decisions that cannot be answered from code.
- Include a ``Suggested Patch Plan`` ordered by risk reduction.

Non-assumption contract:

- Do not assume PIT/delisting vendor data is implemented. It is not.
- Do not assume Nasdaq Data Link API validation has occurred. It has not.
- Do not assume real-time microstructure feeds exist. Current code marks ``microstructure_feed = none_available``.
- Do not assume Optuna ran. The code supports a conditional path, but this snapshot states Optuna is not installed and exhaustive deterministic selection is the active path.
- Do not assume live trading readiness or demonstrated edge. The system is safety-gated and evidence-seeking, not production-proven.
- Do not infer API results, paper fills, account balances, or secrets not present in this pack.

Source Snapshot And Scope

field	value
canonical root	/tmp/tradeo-wave4a-merge-100211
branch	main
HEAD	0ad200f5d6e8082138c88f0404d464544a7fc652
upstream	origin/main
upstream HEAD	0ad200f5d6e8082138c88f0404d464544a7fc652
generation UTC	2026-06-11T17:51:26.435807+00:00
git status before/around generation	(clean before dossier generation; this report file may now be untracked)

Correction note: /home/vboxuser/tradeo was intentionally NOT used as source content because it was on branch feat/wave4-nested-optuna-pbo-20260611. This pack uses /tmp/tradeo-wave4a-merge-100211, main synchronized with origin/main at 0ad200f.

Secret handling: real ``.env``, local credential files, account IDs and API keys are not embedded. ``.env.example`` is treated as a template path/config reference, not as secret evidence. High-confidence key/account patterns are redacted in embedded text.

High-confidence secret/account scan notes (values redacted if embedded):

? backend/tradeo/services/openai_supervisor.py: matched high-confidence secret/account pattern ``sk-[A-Za-z0-9_\\-]{8,}``; values redacted if embedded

Inclusion / Exclusion Policy

- ? Included in full: core settings, DB models, schemas, data provider, IBKR provider, backtester, Director gates, execution/risk/reconciliation, research engines/gates, lab/entry scanner implementations, relevant tests, key docs/remediation, and audit bridge contract/export/validate/gate files.
- ? Summarized only: secondary routers/services/frontend/ops/config files not central to the audit path.
- ? Excluded from tree and content: ``.git``, ``.venv``, ``.node_modules``, caches, generated frontend lockfile content, reports generated by this task, data/artifacts, and secret-like paths.
- ? No application code or tests were modified by this pack generation.

group	files	source lines	bytes
all git-tracked files	230	49074	1985399
full critical code files	61	19107	822884
full relevant tests	23	7792	282259
full key docs	24	2963	106357
full remediation docs	18	1752	109450
full audit bridge files	14	7491	283870

Filtered Project Tree

Generated from ``git ls-files``, excluding dependency/generated/cache/data/artifact/report/secret paths. ``.frontend/package-lock.json`` is intentionally omitted as generated lockfile content, though the docs mention its existence for reproducible builds.

```
??? .dockerignore
??? .env.example
??? .gitignore
??? CLAW_IMPLEMENTATION_PROMPT.md
??? CLAW_RESEARCH_LAB_INTEGRATION_PROMPT.md
??? docker-compose.audit.yml
??? docker-compose.yml
??? Makefile
??? README.md
??? TRADEO_INTRADAY_SMALLCAP_EXPANSION.md
??? .github
?   ??? workflows
?     ??? ci.yml
?     ??? director-audit.yml
??? backend
?   ??? Dockerfile
?   ??? pyproject.toml
?   ??? tradeo
?     ??? __init__.py
?     ??? main.py
?     ??? schemas.py
?     ??? agents
?       ?   ??? audit_agent.py
?       ?   ??? base.py
?       ?   ??? data_collector.py
?       ?   ??? improvement_agent.py
?       ?   ??? pattern_discovery_lab_agent.py
?       ?   ??? pattern_hunter.py
?       ?   ??? risk_agent.py
?       ?   ??? supervisor_agent.py
?       ?   ??? technical_context.py
```

```

?      ??? core
?      ?      ??? config.py
?      ?      ??? security.py
?      ??? db
?      ?      ??? init_db.py
?      ?      ??? models.py
?      ?      ??? session.py
?      ??? modules
?      ?      ??? __init__.py
?      ?      ??? fox_hunter
?      ?      ?      ??? __init__.py
?      ?      ?      ??? production_manifest.py
?      ?      ?      ??? scanner.py
?      ?      ??? laboratory
?      ?      ?      ??? __init__.py
?      ?      ?      ??? diagnostics.py
?      ?      ?      ??? paper_observations.py
?      ?      ?      ??? scanner.py
?      ?      ??? research
?      ?      ?      ??? __init__.py
?      ?      ??? shared
?      ?      ??? __init__.py
?      ?      ??? entry_scanner.py
?      ??? research
?      ?      ??? __init__.py
?      ?      ??? active_learning.py
?      ?      ??? adversarial_research.py
?      ?      ??? autonomous_research_director.py
?      ?      ??? causal_invariance.py
?      ?      ??? cluster_research_engine.py
?      ?      ??? determinism.py
?      ?      ??? foundation_teacher.py
?      ?      ??? global_experiment_registry.py
?      ?      ??? hypothesis_engine.py
?      ?      ??? market_replay.py
?      ?      ??? nested_optimization.py
?      ?      ??? novel_pattern_matcher.py
?      ?      ??? novel_pattern_registry.py
?      ?      ??? pattern_embedding_engine.py
?      ?      ??? quant_validation.py
?      ?      ??? rediscovery_readiness.py
?      ?      ??? research_director.py
?      ?      ??? research_memory_graph.py
?      ?      ??? reward_risk_analyzer.py
?      ?      ??? sequential_tests.py
?      ?      ??? types.py
?      ?      ??? validation_gate.py
?      ?      ??? window_sampler.py
?      ??? routers
?      ?      ??? __init__.py
?      ?      ??? backtests.py
?      ?      ??? dashboard.py
?      ?      ??? fox_hunter.py
?      ?      ??? health.py
?      ?      ??? ibkr.py
?      ?      ??? laboratory.py
?      ?      ??? reports.py
?      ?      ??? research.py
?      ?      ??? risk.py
?      ?      ??? scan.py
?      ?      ??? self_improvement.py
?      ?      ??? signals.py
?      ??? services
?      ?      ??? backtester.py
?      ?      ??? data_provider.py
?      ?      ??? data_quality.py
?      ?      ??? director_review_gate.py
?      ?      ??? effective_sample.py
?      ?      ??? entry_variants.py
?      ?      ??? evidence.py
?      ?      ??? execution_quality.py
?      ?      ??? ibkr_broker.py
?      ?      ??? ibkr_data_provider.py
?      ?      ??? implementation_shortfall.py
?      ?      ??? lab_diagnostics.py
?      ?      ??? lab_paper_observations.py
?      ?      ??? market_regime.py
?      ?      ??? market_session.py
?      ?      ??? metrics.py
?      ?      ??? ml_scorer.py
?      ?      ??? module_dashboard.py
?      ?      ??? openai_supervisor.py
?      ?      ??? opportunity_ranking.py
?      ?      ??? order_outcomes.py
?      ?      ??? paper_broker.py
?      ?      ??? pattern_detector.py
?      ?      ??? pattern_entry_scanner.py
?      ?      ??? pattern_health_monitor.py
?      ?      ??? production_manifest.py
?      ?      ??? provider_factory.py
?      ?      ??? reconciliation.py
?      ?      ??? reports.py
?      ?      ??? risk_manager.py
?      ?      ??? runtime_status.py
?      ?      ??? scanner.py
?      ?      ??? self_improvement.py

```

```

?     ?   ??? signal_quality.py
?     ?   ??? state_policy.py
?     ?   ??? strategy_config.py
?     ?   ??? supervisor.py
?     ?   ??? system_controls.py
?     ?   ??? technical_indicators.py
?     ?   ??? universe_snapshot.py
?     ?   ??? vision_scorer.py
?     ?   ??? watchdog.py
?     ??? tasks
?     ?   ??? __init__.py
?     ?   ??? worker.py
?     ??? tests
?         ??? conftest.py
?         ??? fixtures.py
?         ??? test_agent_c_remediation.py
?         ??? test_audit_hardening.py
?         ??? test_autonomous_research_director.py
?         ??? test_autonomous_research_lab.py
?         ??? test_backtester_non_trade_accounting.py
?         ??? test_data_provider.py
?         ??? test_data_quality.py
?         ??? test_director_review_gate.py
?         ??? test_discovery_determinism.py
?         ??? test_domain_module_facades.py
?         ??? test_effective_sample_weights.py
?         ??? test_execution_quality_audit.py
?         ??? test_execution_state_transitions.py
?         ??? test_ibkr_data_provider.py
?         ??? test_lab_diagnostics.py
?         ??? test_market_regime.py
?         ??? test_market_session.py
?         ??? test_module_dashboard.py
?         ??? test_nested_optimization.py
?         ??? test_novel_pattern_registry.py
?         ??? test_pattern_detector.py
?         ??? test_pattern_discovery_lab.py
?         ??? test_pattern_entry_scanner.py
?         ??? test_quant_validation.py
?         ??? test_quant_validation_integration.py
?         ??? test_rediscovery_readiness.py
?         ??? test_regime_outcome_calibration.py
?         ??? test_remediation_docs_traceability.py
?         ??? test_research_lab_fox_lifecycle.py
?         ??? test_reward_risk_analyzer.py
?         ??? test_risk.py
?         ??? test_runtime_status.py
?         ??? test_shadow_canonical_outcome_parity.py
?         ??? test_watchdog.py
??? config
?     ??? strategy_cup_v0.json
??? docs
?     ??? agent_design.md
?     ??? api_review_contract.md
?     ??? architecture.md
?     ??? audit_2026_06_10_implementation_report.md
?     ??? autonomous_research_director.md
?     ??? autonomous_research_lab.md
?     ??? deployment.md
?     ??? fable5_tradeo_upgrade_2026_06_10.md
?     ??? ibkr_operational_guide.md
?     ??? lab_research_operating_loop.md
?     ??? laboratory_entry_signal_engine.md
?     ??? module_boundaries.md
?     ??? openclaw.md
?     ??? pattern_discovery_lab.md
?     ??? post_audit_r1_r6_tracking_2026_06_10.md
?     ??? risk_policy.md
?     ??? remediation
?     ?     ??? agent_a_data_universe_repro_2026_06_10.md
?     ?     ??? agent_b_representation_matching_parity_2026_06_10.md
?     ?     ??? agent_c_outcomes_costs_backtester_2026_06_10.md
?     ?     ??? agent_d_execution_director_monitoring_2026_06_10.md
?     ?     ??? agent_e2_intraday_incremental_cache_2026_06_11.md
?     ?     ??? agent_e_data_regime_gap_2026_06_11.md
?     ?     ??? agent_f_execution_state_gap_2026_06_11.md
?     ?     ??? agent_g_audit_final_gap_2026_06_11.md
?     ?     ??? agent_h_shadow_canonical_outcome_parity_2026_06_11.md
?     ?     ??? agent_i_regime_outcome_calibration_2026_06_11.md
?     ?     ??? agent_j_non_trade_accounting_2026_06_11.md
?     ?     ??? agent_k_discovery_determinism_2026_06_11.md
?     ?     ??? agent_l_rediscovery_readiness_2026_06_11.md
?     ?     ??? tradeo_12_phase_compliance_matrix_2026_06_10.md
?     ?     ??? tradeo_12_phase_final_report_2026_06_10.md
?     ?     ??? tradeo_12_phase_final_report_2026_06_11.md
?     ?     ??? wave4_a_director_skip_evidence_2026_06_11.md
?     ?     ??? wave4_d_nested_optuna_pbo_2026_06_11.md
?     ??? research
?         ??? director_pattern_search_hardening.md
?         ??? quant_validation_core_2026_06_10.md
?         ??? research_hardening_p1_p2_2026_06_10.md
?         ??? wave4_rediscovery_runbook.md
??? frontend
?     ??? Dockerfile
?     ??? next-env.d.ts
?     ??? next.config.mjs

```

```

?   ??? package.json
?   ??? tsconfig.json
?   ??? app
?   ?   ??? globals.css
?   ?   ??? layout.tsx
?   ?   ??? page.tsx
?   ?   ??? api
?   ?       ??? tradeo
?   ?       ??? [...path]
?   ?       ??? route.ts
?   ??? lib
?   ?   ??? api.ts
?   ??? public
?   ??? .gitkeep
??? ops
?   ??? openclaw
?   ?   ??? OPENCLAW_TASKS.md
?   ??? scripts
?   ?   ??? backup_tradeo.sh
?   ?   ??? deploy_ubuntu.sh
?   ??? systemd
?   ??? tradeo.service
??? research
?   ??? audit_bridge
?   ??? AUDIT_CONTRACT.md
?   ??? director_gate.py
?   ??? export_audit_package.py
?   ??? README.md
?   ??? requirements.txt
?   ??? run_director_audit.py
?   ??? SKILL.md
?   ??? validate_audit_package.py
?   ??? agents
?   ?   ??? director_weekly_auditor
?   ?   ?   ??? SKILL.md
?   ?   ??? internal_daily_auditor
?   ?   ?   ??? SKILL.md
?   ??? decisions
?   ?   ??? DECISIONS.md
?   ??? schedules
?   ?   ??? PERIODIC_TASKS.md
?   ??? state
?   ?   ??? process_improvements.md
?   ??? task_queue
?   ??? pending
?   ??? 2026-06-07_claw_merge_redeploy_director_audit_automation.md
?   ??? 2026-06-07_ib_paper_patterns_director_gate_and_research_fixes.md
??? scripts
    ??? research_forever.sh

```

Executive Overview For External Auditor

Tradeo is a personal automated trading research and execution-safety system. Its stated philosophy is research-first and paper-first: discover patterns, validate them statistically, observe or paper-test them, require Director evidence, then only allow production/fox/live paths through explicit gates. The codebase is Python/FastAPI with SQLAlchemy models, a Next frontend, an IBKR-only market data/execution integration surface, and a standalone `research/audit\_bridge` for exporting external audit packages.

The most important audit question is whether claims about evidence, point-in-time data, execution realism, and production readiness are actually enforced by code and tests. This pack intentionally preserves contradictory/stale remediation notes where they exist so Fable can distinguish historical status from current snapshot truth.

Key honest boundaries in this snapshot:

- ? PIT/delisting vendor data is still open and blocked on data/API validation; current code has mitigations and flags, not a completed licensed vendor integration.
- ? Nasdaq Data Link is a target provider idea, not validated evidence in this snapshot.
- ? Real-time microstructure feeds are not integrated; execution quality code marks `microstructure\_feed = none\_available` and docs keep richer feeds open.
- ? Optuna is conditionally supported but not installed/pinned in this snapshot; Wave4-D uses deterministic exhaustive inner selection when the grid is fully enumerable.
- ? Live trading is heavily gated and off by default; the existence of broker code is not evidence of live-readiness or edge.
- ? Legacy patterns may lack Wave4 metadata until a real rediscovery run refreshes them; Wave4-C provides readiness/flagging, not fake backfill.

System Architecture At A Glance

Primary subsystems:

- ? Backend API: `backend/tradeo/main.py` mounts routers for scan, signals, risk, backtests, research, laboratory, IBKR, reports, health, dashboard and self-improvement.
- ? Persistence: `backend/tradeo/db/models.py` stores signals, trades, audit logs, discovered patterns, research runs, current matches, lab cycles and manifests.
- ? Data layer: `data\_provider.py` normalizes OHLCV, caches bars, supports incremental refresh, and wraps IBKR through `IBKRHistoricalDataProvider` via `provider\_factory.py`.

? Research: `PatternDiscoveryLabAgent` orchestrates windows, embeddings, clustering, reward/risk metrics, global trial counts, ValidationGate, registry upserts and reports.

? Laboratory: shared entry scanner and laboratory paper observation modules create shadow observations, paper order records, near misses and diagnostics without granting production authority.

? Director gates: `DirectorReviewGate` evaluates lab evidence and `DirectorProductionGate` evaluates production-readiness packets; these are distinct from research validation.

? Execution safety: `RiskManager`, `IBKRBroker`, `ExecutionReconciler`, `system\_controls`, `execution\_quality`, and evidence/effective-sample helpers guard order submission and auditability.

? Audit bridge: `research/audit\_bridge` exports/validates package artifacts and applies an external Director gate with explicit CSV/JSON contracts.

End-To-End Flow: Research Candidate To Production Review

Expected chain:

1. Market data enters through IBKR-only provider settings and is normalized/cached with manifests and data-quality checks.
2. `WindowSampler` and `PatternEmbeddingEngine` produce candidate windows/features available at the decision time.
3. `ClusterResearchEngine` discovers clusters/candidates with null baselines, adjusted p-values, DSR/CSCV/PBO-related metadata, regime metadata and global trial accounting.
4. `RewardRiskAnalyzer` computes canonical triple-barrier outcomes and RR metrics, explicitly separating skipped/non-trade signals after Agent J.
5. `ValidationGate` decides whether a research candidate has enough statistical evidence to be persisted/advanced.
6. `NovelPatternRegistry` stores discovered patterns; legacy rows may need Wave4-C rediscovery refresh for embedding/cluster/regime metadata.
7. Laboratory matching/entry scanner creates shadow observations or paper orders; lab mode must not silently become live execution.
8. Lab fills/observations accumulate evidence, effective sample weights and execution-quality metadata.
9. `DirectorReviewGate` can mark a pattern for review, but this is not production approval. It surfaces research skip-rate evidence after Wave4-A.
10. `DirectorProductionGate` requires stronger evidence and manifest checks before production status. Fox Hunter/live paths should consume only production patterns with valid manifests and runtime safety gates.
11. IBKR order submission remains guarded by trading mode, read-only flag, live arm phrase, kill switch, human approval, allowed symbols and notional/risk caps.

Audit this chain for any alternate path, stale status, missing metadata, or API/frontend route that can bypass a service-level invariant.

End-To-End Flow: Audit Bridge

The audit bridge is a parallel external-audit surface. It exports a package with pattern catalog, events, trades, fills, metrics and docs. Important contract sections 14-17 are `pattern\_events.csv`, `trades.csv`, `fills.csv`, and `metrics\_summary.json`; the Director weekly auditor also uses phases 14-17 for process audit, blocking criteria, Candidate-A criteria and severity scale. Fable should verify whether exporter and validator enforce these contracts and whether reconstructed or simulated evidence can be mistaken for broker-provenance fills.

Recent Remediation And Wave Ledger

This section is an explicit index of recent remediation notes requested by Asier. Treat documentation claims as audit evidence only after checking the code blocks and tests embedded later.

| item | area | implemented/evidence claim | remaining gap / caution |

| --- | --- | --- | --- |

| Agent A | Data/universe/repro | Deterministic OHLCV cache, manifest, universe snapshot flags, survivorship cap. | PIT/delisting vendor data still absent. |

| Agent B | Representation/matching parity | Embedding contract persisted, cluster signature/concentration diagnostics, ambiguity metadata. | Historical rows need rediscovery; HDBSCAN/DTW deferred. |

| Agent C | Outcomes/costs/backtester | Canonical triple-barrier outcome shared with backtester, tiered costs, RR grid reduction, anti-overfit guards. | Initial non-trade accounting gap later closed by J/Wave4-A. |

| Agent D | Execution/director/monitoring | Microstructure auditability markers, ADV and family caps, shortfall monitoring, sequential gate defaults. | Richer real-time microstructure provider remains open. |

| Agent E/E2 | Data/regime/intraday cache | PIT-stable SPY regime labels, incremental daily and intraday cache refresh, source hashes. | Live IBKR adjusted-feed validation and RTH session modeling remain limited. |

| Agent F | Execution state/effective sample | Order-state tests and persisted effective-sample weights. | Review binding logic and edge cases in Director code/tests. |

| Agent G | Audit final/docs traceability | Consolidated final report and doc path traceability tests. | Prose accuracy still requires human/LLM review. |

| Agent H | Shadow canonical parity | Shadow exits delegate to canonical outcome; closed trades persist canonical outcome block. | Shadow costs remain OR by design. |

| Agent I | Regime outcome calibration | Benchmark-regime outcome buckets and disabled-by-default hard gate. | Existing DB patterns need rediscovery; Director enable decision pending. |

| Agent J | Non-trade accounting | Skipped/invalid/no-data signals removed from traded arrays; skip counts/rate exposed. | Director surfacing was later Wave4-A. |

Wave4-A	Director skip-rate evidence	DirectorReviewGate and production packet surface stored research skip accounting; evidence only.	Hard skip-rate ceiling remains policy decision.
Wave4-B / Agent K	Discovery determinism	Canonical JSON/hash and deterministic report ordering/in-process harness.	Cross-environment bit-equality not claimed.
Wave4-C / Agent L	Rediscovery readiness	Audits/flags legacy rows lacking Wave4 metadata; runbook for bounded real rediscovery.	Does not populate production metadata by itself.
Wave4-D	Nested optimization + outer PBO	Mandatory nested PBO hardening in SelfImprovement; exhaustive deterministic inner selection active.	Optuna not installed/pinned; complement folds are PBO measure, not causal WF.
Phases 14-17 audit bridge	Contract/process phases	Pattern events, trades, fills, metrics summary; and weekly auditor process/blocking/Candidate-A/severity phases.	Must reconcile exporter/validator/Director behavior against these contracts.
Microstructure	Prior integration status	Signals/entry quality carry auditable liquidity/ATR/volume/extension/regime/rejection markers.	No richer real-time feed provider integrated; do not treat as complete microstructure model.
PIT/delisting	Open	Flags and survivorship honesty exist.	Vendor data and Nasdaq trial pending.

Explicit Spanish Checkpoint Requested By Asier

? `fases 14-17`: interpreted and documented as audit-bridge contract/process phases, specifically `pattern\_events.csv`, `trades.csv`, `fills.csv`, `metrics\_summary.json`, plus weekly Director auditor phases for process audit, blocking criteria, Candidate-A criteria and severity scale. They are not treated as separate completed feature branches.
 ? `Wave4-A/B/C/D`: included explicitly: Wave4-A Director skip-rate evidence; Wave4-B/Agent K discovery determinism; Wave4-C/Agent L rediscovery readiness; Wave4-D nested optimization plus outer-fold PBO.
 ? `microstructure ya integrado anterior`: represented honestly as prior auditability integration (liquidity/ATR/volume/extension/regime/rejection markers and `microstructure\_feed` evidence fields), not as a real-time microstructure provider.
 ? `PIT/delisting pendiente`: still open and blocked on vendor data; current code only provides flags, caps and honesty markers.
 ? `Nasdaq API trial pendiente`: no real Nasdaq Data Link API trial/result is included or claimed; the target spec is pending validation and must not be treated as completed evidence.

Configuration / Environment Surface

Critical env/config themes to audit:

? `Settings` in `backend/tradeo/core/config.py` is the authoritative config contract and is embedded in full source.
 ? `.env.example` is not embedded in full to avoid encouraging credential copying, but its path is listed and config values are summarized by `Settings` source.
 ? Live defaults should remain disabled: `live\_trading\_enabled=false`, `ibkr\_readonly=true`, kill switch behavior active, and market data provider constrained to IBKR.
 ? Research knobs include RR levels, CSCV/PBO, nested optimization, regime windows, incremental cache, PIT/survivorship flags, and data-quality thresholds.
 ? Audit bridge env handling should redact admin password, account IDs and IBKR connection values where exported.
 ? Docker/ops scripts may contain placeholder credentials (`tradeo`, `change-me`) that are acceptable only for local dev and should not be exposed.

path	lines	kind	summary / first lines
.env.example	201	text	# Tradeo local/VPN
configuration TRADEO_APP_NAME=Tradeo TRADEO_ENVIRONMENT=local TRADEO_DATABASE_URL=postgresql+ps			
ycopg://tradeo:tradeo@db:5432/tradeo TRADEO_REDIS_URL=redis://redis:6379/0 TRADEO_SECRET_KEY=replace-with-a-lo			
ng-random-string TRADEO_ADMIN_USERNAME=admin TRADEO_ADMIN_PASSWORD=replace-this-password TRADEO			
_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000 #			
Trading mode. Default must remain paper until the full validation gate is			
passed. TRADEO_TRADING_MODE=paper TRADEO_LIVE_TRADING_ENABLED=false TRADEO_LIVE_TRADING_CON			
FIRMATION_VALUE= TRADEO_KILL_SWITCH_ENABLED=false #			
Account and risk policy TRADEO_INITIAL_CAPITAL_USD=3000			
backend/pyproject.toml	45	toml	[project] name = "tradeo-backend" version = "0.1.0" description = "Tradeo automated
pattern research, paper trading and supervised execution backend" requires-python = ">=3.11" dependencies = [			
"fastapi">=0.115.0", "uvicorn[standard]>=0.30.0", "pydantic">=2.7.0", "pydantic-settings">=2.4.0", 			
"sqlalchemy">=2.0.0", "psycopg[binary]>=3.2.0", "pandas">=2.2.0", "numpy">=1.26.0", "matplotlib">=3.8.0", 			
"scikit-learn">=1.5.0", "python-dotenv">=1.0.1", "openai">=1.90.0",			
docker-compose.yml	94	yaml	services: db: image: postgres:16-alpine container_name: tradeo-db restart:
unless-stopped environment: POSTGRES_USER: tradeo POSTGRES_PASSWORD: tradeo 			
POSTGRES_DB: tradeo volumes: - tradeo_postgres:/var/lib/postgresql/data healthcheck: test:			
["CMD-SHELL", "pg_isready -U tradeo -d tradeo"] interval: 10s timeout: 5s retries: 5 redis:			
docker-compose.audit.yml	8	yaml	services: backend: volumes: - ./research:/app/research:ro
worker: volumes: - ./research:/app/research:ro			
Makefile	52	text	-include .env export PHONY: setup up down logs ps test scan report self-improve discover-patterns
match-discovered-patterns current-matches research-runs research-director research-director-latest setup: cp -n			
.env.example .env \\| true mkdir -p reports artifacts up: docker compose up -d --build down: docker			
compose down logs: docker compose logs -f --tail=200 			
config/strategy_cup_v0.json	25	json	{ "name": "cup_v0", "pattern": "mid_small_cap_cup_breakout", "description":
 "Technical cup/base breakout detector for USA mid/small-cap equities. Paper-first; 4R minimum target.",
 "target_r_multiple":

```

4.0,<br> "min_bars": 45,<br> "max_bars": 210,<br> "min_depth": 0.10,<br> "max_depth": 0.42,<br> "rim_tolerance": 0.12,<br>
"max_handle_depth": 0.18,<br> "breakout_buffer": 0.002,<br> "min_breakout_volume_ratio": 1.12,<br> "min_confidence": 0.68,<br>
"min_composite_score": 0.70,<br> "max_holding_bars": 30,<br> "promotion_gate": {<br> "min_historical_trades": 40, |
| ops/openclaw/OPENCLAW_TASKS.md | 52 | markdown | Tareas programables para OpenClaw; Tarea diaria de salud; Tarea semanal
de laboratorio; Tarea manual de escaneo |
| ops/scripts/backup_tradeo.sh | 7 | bash | #!/usr/bin/env bash<br>set -euo pipefail<br>cd "$(dirname "$0")/../../<br>mkdir -p
backups<br>archive="backups/tradeo_backup_$(date +%Y%m%d_%H%M%S).tar.gz"<br>tar -czf "$archive" reports data config .env
docker-compose.yml README.md docs \\\ true<br>echo "$archive" |
| ops/scripts/deploy_ubuntu.sh | 21 | bash | #!/usr/bin/env bash<br>set -euo pipefail<br><br>cd "$(dirname "$0")/../../<br><br>if !
command -v docker >/dev/null 2>&1; then<br> echo "Docker no está instalado. Instalando docker.io y compose plugin..."<br> sudo
apt-get update<br> sudo apt-get install -y docker.io docker-compose-plugin<br> sudo usermod -aG docker "$USER" \\\ true<br> echo
"Si tu usuario acaba de entrar al grupo docker, cierra sesión y vuelve a entrar."<br>fi<br><br>mkdir -p reports artifacts<br>if [ ! -f .env ];
then<br> cp .env.example .env<br> echo "Se ha creado .env. Edita TRADEO_ADMIN_PASSWORD y TRADEO_SECRET_KEY antes
de exponer la web."<br>fi |
| ops/systemd/tradeo.service | 16 | ini | [Unit]<br>Description=Tradeo research
stack<br>Requires=docker.service<br>After=docker.service
network-online.target<br>Wants=network-online.target<br><br>[Service]<br>Type=oneshot<br>WorkingDirectory=/home/vboxuser/trad
eo<br>ExecStart=/usr/bin/docker
compose up -d<br>ExecStop=/usr/bin/docker compose
stop<br>RemainAfterExit=yes<br>TimeoutStartSec=0<br><br>[Install]<br>WantedBy=multi-user.target |
|.github/workflows/ci.yml | 45 | yaml | name: ci<br><br>on: <br> push: <br> branches: [main]<br> pull_request: <br>
workflow_dispatch: <br><br>jobs: <br> backend: <br> name: backend lint + tests<br> runs-on: ubuntu-latest<br> defaults: <br>
run: <br> working-directory: backend<br> steps: <br> - uses: actions/checkout@v4<br> - uses: actions/setup-python@v5 |
|.github/workflows/director-audit.yml | 42 | yaml | name: Director audit bridge<br><br>on: <br> pull_request: <br> paths: <br> -
"research/audit_bridge/**"<br> - ".github/workflows/director-audit.yml"<br> schedule: <br> - cron: "35 21 * * *"<br>
workflow_dispatch: <br><br>jobs: <br> audit-bridge: <br> runs-on: ubuntu-latest<br> steps: <br> - uses:
actions/checkout@v4<br><br> - uses: actions/setup-python@v5 |

```

Data Models And Contracts

Audit focus: SQLAlchemy persistence (`db/models.py`), Pydantic request/response contracts (`schemas.py`), audit bridge CSV/JSON contracts, evidence enums, and JSON metadata blocks that function as de facto schemas. Check whether every persisted metadata shape has a version, tests, and downstream validation.

- ? Discovered patterns carry metrics/evidence in JSON blobs; schema drift is a primary risk.
- ? Trades/signals/lab cycles/audit logs are central for reconstructing evidence and should reconcile with audit export.
- ? PIT fields exist as flags/metadata but not as completed vendor-backed facts.

Research Pipeline Review Map

- ? Start in `PatternDiscoveryLabAgent.run` and trace into window sampling, embeddings, clustering, reward/risk, validation gate, global experiment registry, and registry upsert.
- ? Check that all features used for matching/research are available at decision time and that incomplete bars are dropped consistently.
- ? Review null baselines, adjusted p-values, global/candidate trial counts, DSR, CSCV/PBO, nested PBO, and deterministic content hashes.
- ? Review Wave4-C rediscovery readiness: flags are not metadata; only real rediscovery can populate legacy rows.

Lab / Paper / Director Gates Review Map

- ? Trace shared entry scanner in lab mode and fox/production mode. Verify no live IBKR path is reachable from laboratory shadow observations.
- ? Review evidence type transitions: shadow signal -> paper order -> paper fill -> live fill, and whether provenance/timestamps are required before effective-sample credit.
- ? Review DirectorReviewGate versus DirectorProductionGate: review eligibility is not production approval.
- ? Review Wave4-A skip-rate evidence: it is visible evidence only, not a blocker.

Execution Safety Review Map

- ? RiskManager should bound position sizing, max notional, daily loss, ADV participation, pattern family concentration, and shorts.
- ? IBKRBroker should fail closed unless runtime flags, read-only setting, live arm, human approval, symbol allowlist and bracket validity are satisfied.
- ? Reconciliation should trigger kill switch on confirmed DB/IBKR divergence while treating pending orders carefully.
- ? ExecutionQuality must not imply a real microstructure feed where none exists.

Backtester And Data Provider Review Map

- ? Backtester and RewardRiskAnalyzer should share canonical outcome semantics and cost model. Verify skipped invalid/no-data events are excluded from traded arrays and surfaced as skip accounting.
- ? Data provider should never silently use synthetic/non-IBKR data under production-like settings.

? Incremental refresh must not cement partial bars, must handle overlap mismatch, and must not pretend PIT/delists are present.
? IBKR provider has external failure modes: no bars, blocking connection, pacing, adjusted feed semantics, timezone, account privacy.

Audit Bridge Review Map

? Review `AUDIT\_CONTRACT.md`, `SKILL.md`, exporter, validator, and standalone director gate as a single contract.
? Phases 14-17: `pattern\_events.csv` must prove available data cutoff $\leq$ decision $\leq$ entry; `trades.csv` must reconcile costs/net PnL; `fills.csv` must distinguish simulated vs broker fills; `metrics\_summary.json` must reconcile aggregate metrics and sample counts.
? Director weekly auditor phases 14-17 define process audit, blocking criteria, Candidate-A criteria, and severity scale; use these to judge the whole system.
? Check redaction/account handling and whether package validation catches all dangerous overclaims.

Target Completion Spec: PIT/Delists Nasdaq Data Link

STATUS: pending real API validation; do not treat as completed evidence.

This section is intentionally written as a target implementation/completion specification for the next agent or external auditor. It is not evidence that the current snapshot has licensed point-in-time/delisting data. It exists because the audit request needs Fable Max to review the desired contract as if it were the next implementation target, while clearly separating target design from completed code.

Objective

Replace the current honest-but-survivorship-biased universe limitation with a licensed, point-in-time universe and delisting-aware data layer backed by Nasdaq Data Link or an equivalent vendor. The goal is to let Research and Backtester evaluate historical candidates against the symbols that were tradable at the decision date, including delisted symbols and terminal outcomes.

Current Snapshot Reality

? `universe\_snapshot.py` and remediation docs explicitly mark `delisting\_data\_available = false`.
? The compliance matrix keeps PIT/delisting vendor data open and blocked on data.
? README mentions Nasdaq Data Link as a possible provider, but no API trial evidence is present.
? This dossier contains no API key and no live API results.
? The current survivorship cap to `lab\_watchlist` or configured universe is a mitigation, not a substitute for PIT data.

Target Data Contracts

Required normalized tables/artifacts:

- `pit\_universe\_membership`: symbol, vendor_symbol, stable security id, exchange, security_type, start_date, end_date, delisted, delisting_date, delisting_reason, source_dataset, source_asof, source_hash.
- `symbol\_mapping\_history`: stable_id, symbol, vendor_symbol, effective_from, effective_to, corporate_action_type, notes.
- `delisting\_events`: stable_id, symbol, delisting_date, last_trade_date, terminal_price, cash_distribution, reason, source_dataset, source_hash.
- `pit\_price\_adjustment\_metadata`: stable_id, symbol, bar_date, adjustment_factor, split_factor, dividend_amount, raw_close, adjusted_close, source_hash.

Target Config

```
nasdaq_data_link_enabled: bool = False
nasdaq_data_link_api_key: SecretStr | None = None
nasdaq_data_link_base_url: str = "https://data.nasdaq.com/api/v3"
nasdaq_data_link_timeout_seconds: int = 20
nasdaq_data_link_rate_limit_per_minute: int = 60
universe_pit_vendor: str = "nasdaq_data_link"
universe_pit_cache_dir: Path = Path("data/pit_universe")
universe_require_pit_for_research: bool = True
universe_allow_survivorship_fallback: bool = False
universe_delisting_terminal_return_policy: Literal["vendor_total_return", "terminal_price", "cash_distribution", "block_if_unknown"] = "block_if_unknown"
```

Safety defaults should be fail-closed for research claims: if PIT is required and the vendor/cache is missing, discovery should block or explicitly mark all outputs `survivorship\_biased=true` and keep Director promotion blocked.

Target Implementation Sketch

```
class NasdaqDataLinkClient:
    def __init__(self, api_key: SecretStr, base_url: str, timeout: float): ...
    def fetch_table(self, dataset: str, *, filters: dict[str, str], cursor: str | None = None) -> VendorPage: ...
    def iter_table(self, dataset: str, filters: dict[str, str]) -> Iterator[dict[str, Any]]: ...

class PITUniverseProvider:
    def membership_asof(self, asof_date: date, universe_rule: UniverseRule) -> PITUniverseSnapshot: ...
```

```
def symbol_identity(self, symbol: str, asof_date: date) -> SymbolIdentity: ...
def delisting_event(self, stable_id: str) -> DelistingEvent | None: ...
def validate_coverage(self, start: date, end: date, exchanges: set[str]) -> CoverageReport: ...
```

Integration points: `universe\_snapshot.py`, `PatternDiscoveryLabAgent`, `DataProvider`, `RewardRiskAnalyzer`, `Backtester`, `DirectorReviewGate`, and `audit\_bridge/export\_audit\_package.py`.

Target Tests

1. `test\_pit\_universe\_excludes\_future\_listings`.
2. `test\_pit\_universe\_includes\_delisted\_symbol\_before\_delisting`.
3. `test\_recycled\_ticker\_does\_not\_join\_future\_identity`.
4. `test\_discovery\_blocks\_when\_pit\_required\_but\_vendor\_missing`.
5. `test\_backtester\_terminal\_delisting\_policy\_block\_if\_unknown`.
6. `test\_audit\_package\_exports\_pit\_artifacts`.
7. `test\_no\_api\_key\_in\_logs\_or\_reports`.
8. `test\_vendor\_cache\_hash\_changes\_on\_payload\_change`.
9. `test\_pit\_snapshot\_reproducible\_from\_cache`.
10. `test\_director\_blocks\_survivorship\_biased\_candidate`.

Go / No-Go Criteria

Go only when a real Nasdaq Data Link trial or licensed equivalent has been executed outside this dossier; dataset codes, response schemas, pagination, rate limits and coverage are documented from real calls; at least one known delisted symbol validates end-to-end; no secrets leak; sanitized fixtures cover tests; and one operator-run integration check validates live API connectivity separately.

No-go if the vendor lacks stable identifiers or delisting data, the implementation backfills today's active symbols, credentials leak, or research remains production-reviewable while claiming unbiased history without PIT coverage.

Known Open Gaps And Desired Expert Review

- ? PIT/delisting vendor data and Nasdaq Data Link validation are pending; ask whether target contracts are sufficient and what minimum vendor fields are non-negotiable.
- ? Survivorship mitigation is not equivalent to unbiased historical research; review whether current caps are strict enough to prevent overclaiming.
- ? Regime gate ships disabled and existing DB patterns need rediscovery; review whether Director should block patterns without calibration buckets.
- ? Microstructure remains audit markers without a real feed; review whether current execution-quality evidence is enough for paper/live progression.
- ? Optuna path is inactive and unpinned; review whether deterministic exhaustive search is appropriate for current candidate space and how to gate future larger spaces.
- ? Audit bridge exporter is large and central; review fill provenance, independence labels, account redaction, and whether reconstructed evidence can slip into broker-fill fields.
- ? No Alembic migration tree is visible; review schema evolution risk and how JSON metadata contracts are versioned/tested.
- ? IBKR calls may be synchronous/blocking in API paths; review event-loop blocking, timeouts, and operator UX failure modes.
- ? Frontend is monolithic and may present status in ways operators can misread; audit only where it can cause unsafe decisions.
- ? Test counts in docs are historical; this pack generation did not rerun pytest, so Fable should distinguish source tests from executed evidence.

Test Coverage Map

This map links high-risk surfaces to embedded tests. Passing counts are taken from remediation docs where noted; this generator did not rerun the backend suite.

surface	embedded tests	what to inspect
---	---	---
Data provider / incremental cache / PIT flags	`test_data_provider.py`, `test_ibkr_data_provider.py`, `test_data_quality.py`	Partial-bar exclusion, refresh lineage, delisting flag honesty, IBKR adapter behavior.
Reward/risk + non-trade accounting	`test_reward_risk_analyzer.py`, `test_backtester_non_trade_accounting.py`, `test_agent_c_remediation.py`	Canonical triple-barrier behavior, cost accounting, skipped signal exclusion.
Director gates / evidence	`test_director_review_gate.py`, `test_effective_sample_weights.py`	Paper fill thresholds, effective sample evidence, skip-rate evidence, production packet.
Execution safety	`test_execution_state_transitions.py`, `test_execution_quality_audit.py`, `test_pattern_entry_scanner.py`	Order states, kill switch, lab/fox separation, missing microstructure declaration.
Research determinism / rediscovery	`test_discovery_determinism.py`, `test_rediscovery_readiness.py`, `test_pattern_discovery_lab.py`	Bit-for-bit in-process hash, legacy metadata flags, discovery report behavior.
Quant/stat validation	`test_quant_validation.py`, `test_quant_validation_integration.py`, `test_nested_optimization.py`, `test_regime_outcome_calibration.py`	DSR/WRC/SPA/PBO, nested outer PBO, regime outcome calibration.
Audit bridge	`test_audit_hardening.py`, `test_remediation_docs_traceability.py`	Exporter/validator contract, fill provenance, event counts, doc path traceability.

Python Public API Inventory

Generated from AST for all tracked Python files. For critical files, the full source appears later.

path	lines	role / docstring	public classes	public functions	routes	risk heuristics
backend/tradeo/__init__.py	1	No module docstring; infer role from symbols/imports.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/audit_agent.py	14	No module docstring; infer role from symbols/imports.	AuditAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/base.py	19	No module docstring; infer role from symbols/imports.	AgentResult:8, Agent:15	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/data_collector.py	16	No module docstring; infer role from symbols/imports.	DataCollectorAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/improvement_agent.py	14	No module docstring; infer role from symbols/imports.	ImprovementAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/pattern_discovery_lab_agent.py	869	Orchestrates discovery runs, persistence, global trial counts, reports, and ValidationGate. Audit end-to-end research reproducibility.	PatternDiscoveryLabAgent:39	?	?	broad `except Exception` present; wall-clock timestamp usage; IBKR/broker/data dependency
backend/tradeo/agents/pattern_hunter.py	21	No module docstring; infer role from symbols/imports.	PatternHunterAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/risk_agent.py	14	No module docstring; infer role from symbols/imports.	RiskAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/supervisor_agent.py	19	No module docstring; infer role from symbols/imports.	SupervisorAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/technical_context.py	22	No module docstring; infer role from symbols/imports.	TechnicalContextAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/core/config.py	386	Central Pydantic settings and validation contract. Audit env defaults, fail-closed trading settings, IBKR-only provider enforcement, and anti-overfit knobs.	Settings:13	get_settings:385	?	IBKR/broker/data dependency
backend/tradeo/core/security.py	35	No module docstring; infer role from symbols/imports.	?	require_admin:12	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/db/init_db.py	136	No module docstring; infer role from symbols/imports.	?	init_db:12, seed_db:129	?	JSON metadata contract; schema drift risk
backend/tradeo/db/models.py	337	SQLAlchemy persistence schema. Audit state transitions, JSON evidence fields, enum-like strings, and migration risk because there is no Alembic migration tree in this snapshot.	SignalStatus:24, TradeStatus:34, Signal:40, Trade:67, EquityPoint:93, PatternMetric:103, StrategyVersion:118, AuditLog:131	utcnow:15, json_type:19	?	wall-clock timestamp usage; JSON metadata contract; schema drift risk
backend/tradeo/db/session.py	24	No module docstring; infer role from symbols/imports.	Base:15	get_db:19	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/main.py	63	No module docstring; infer role from symbols/imports.	?	lifespan:28, create_app:38	?	IBKR/broker/data dependency
backend/tradeo/modules/__init__.py	2	Domain modules for Tradeo research, paper validation and production hunting.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/fox_hunter/__init__.py	2	FoxHunter domain: production pattern monitoring and gated live execution.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/fox_hunter/production_manifest.py	254	No module docstring; infer role from symbols/imports.	?	build_production_manifest:16, production_manifest_hash:47, production_manifest_status:53, production_manifest_for_pattern:113, production_manifest_is_active:117, production_manifest_summary:125	?	wall-clock timestamp usage
backend/tradeo/modules/fox_hunter/scanner.py	47	No module docstring; infer role from symbols/imports.	FoxHunterScanner:14	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/laboratory/__init__.py	2	Laboratory domain: paper validation of Research patterns.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/laboratory/diagnostics.py	606	No module docstring; infer role from symbols/imports.	?	laboratory_diagnostics:20	?	wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk
backend/tradeo/modules/laboratory/paper_observations.py	706	Shadow observation lifecycle. Audit canonical exit parity, pending/closed transitions, no-order invariant, and open-gap handling.	MarketDataFetch:40, FallbackFrame:46, LabPaperObservationService:53	?	?	broad `except Exception` present; wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk
backend/tradeo/modules/laboratory/scanner.py	47	No module docstring; infer role from symbols/imports.	LaboratoryScanner:14	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/research/__init__.py	7	Research domain facade. The implementation remains in ``tradeo.research`` because it is already a cohesive package. This namespace marks it as one of the three top-level Tradeo domains alongside Laboratory and FoxHunter.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/shared/__init__.py	2	Shared operational primitives used by Tradeo domain modules.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/shared/entry_scanner.py	1341	Actual PatternEntryScanner implementation behind service facade. Audit lab/fox mode separation, shadow/paper/live guardrails, near-miss recording, IBKR submission gates, and evidence metadata.	PatternEntryScannerSafetyError:52, PatternEntryScanner:57	?	?	broad `except Exception` present; wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk
backend/tradeo/research/__init__.py	5	Research laboratory modules for discovering novel market patterns. The research package is				

deliberately isolated from order execution. It can create LAB candidates and review artifacts, but it cannot route trades to brokers | ? | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/active_learning.py | 146 | No module docstring; infer role from symbols/imports. | ActiveLearningAgenda:10 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/adversarial_research.py | 332 | No module docstring; infer role from symbols/imports. | AdversarialResearchEngine:14 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/autonomous_research_director.py | 747 | No module docstring; infer role from symbols/imports. | ResearchDirector:16 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/research/causal_invariance.py | 237 | No module docstring; infer role from symbols/imports. | CausalInvariantTester:16 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/cluster_research_engine.py | 1914 | Discovery clustering engine. Audit feature construction, KMeans assumptions, null baselines, overfitting controls, regime/outcome metadata, and deterministic seeds. | ClusterResearchEngine:37 | ? | ? | RNG path; verify seed/control |

| backend/tradeo/research/determinism.py | 116 | Deterministic serialization and content hashing for research artifacts. The discovery pipeline must be bit-for-bit reproducible for identical inputs, config and seeds. Wall-clock timestamps, database run ids and filesystem | ? | canonical_payload:50, canonical_json:78, content_hash:88, candidate_content_payload:92, discovery_content_hash:114 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/foundation_teacher.py | 262 | No module docstring; infer role from symbols/imports. | FoundationChartTeacher:13 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/global_experiment_registry.py | 268 | No module docstring; infer role from symbols/imports. | GlobalExperimentRegistry:20 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/research/hypothesis_engine.py | 296 | No module docstring; infer role from symbols/imports. | HypothesisEngine:12 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/market_replay.py | 306 | No module docstring; infer role from symbols/imports. | MarketReplayConfig:13, MarketReplayEngine:20 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/nested_optimization.py | 186 | Wave4-D nested optimization/PBO report. Audit pass criteria and limitations of complement train folds vs causal walk-forward. | ? | optuna_available:41, nested_optimization_report:81 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/novel_pattern_matcher.py | 866 | Live/current matcher for discovered patterns. Audit feature parity, threshold/ambiguity logic, regime fit, and incomplete-bar exclusions. | NovelPatternMatcher:35 | ? | ? | broad `except Exception` present; wall-clock timestamp usage |

| backend/tradeo/research/novel_pattern_registry.py | 306 | No module docstring; infer role from symbols/imports. | NovelPatternRegistry:22 | ? | ? | broad `except Exception` present |

| backend/tradeo/research/pattern_embedding_engine.py | 601 | Embedding contract used for research/lab parity. Audit contract versioning and feature availability at decision time. | PatternEmbeddingEngine:13 | ? | ? | broad `except Exception` present |

| backend/tradeo/research/quant_validation.py | 798 | Quant/stat validation core. Audit purged walk-forward, CSCV/PBO, DSR, WRC/SPA approximations, and multiple-testing ledger. | ? | select_nonoverlapping_events:83, average_uniqueness_weights:118, triple_barrier_outcome:159, tiered_round_trip_cost_r:303, stationary_bootstrap_ci:341, newey_west_tstat:382, profit_factor:411, permutation_pvalue:430, benjamini_hochberg:459, expected_max_sharpe:485 | ? | RNG path; verify seed/control; TODO/FIXME present |

| backend/tradeo/research/rediscovery_readiness.py | 274 | Wave4-C readiness tool. Audit that it flags only and never fabricates metadata. | PatternReadiness:50 | evaluate_pattern_metrics:73, audit_patterns:102, apply_rediscovery_flags:126, build_manifest:168, run_readiness:222, main:241 | ? | wall-clock timestamp usage |

| backend/tradeo/research/research_director.py | 466 | No module docstring; infer role from symbols/imports. | ResearchDirector:21 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/research/research_memory_graph.py | 320 | No module docstring; infer role from symbols/imports. | ResearchMemoryGraph:16 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/research/reward_risk_analyzer.py | 303 | Canonical reward/risk and skipped-signal accounting. Audit triple barrier semantics, cost model, RR metrics, and sample counts. | RewardRiskAnalyzer:12 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/sequential_tests.py | 165 | Sequential evidence tests for the Director review loop (informe §4.7). Pure functions, numpy + stdlib only. They turn "n=10 lab trades" from a pseudo-decision into a review trigger backed by three orthogonal checks: - `` | ? | posterior_probability_edge:35, sprt_inferiority:77, ks_two_sample:137 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/types.py | 189 | No module docstring; infer role from symbols/imports. | ForwardOutcome:12, WindowSample:79, ClusterCandidate:93, HypothesisPackage:132 | freeze_json:112, thaw_json:123 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/validation_gate.py | 415 | Pattern validation thresholds. Audit min samples, OOS, adjusted p-values, PF/expectancy, and gate semantics. | ValidationGate:10 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/research/window_sampler.py | 286 | No module docstring; infer role from symbols/imports. | WindowSampler:15 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/__init__.py | 1 | No module docstring; infer role from symbols/imports. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/backtests.py | 21 | No module docstring; infer role from symbols/imports. | ? | run_backtest:16 | run_backtest: @router.post('/run', response_model=BacktestMetrics) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/dashboard.py | 40 | No module docstring; infer role from symbols/imports. | ? | summary:16 | summary: @router.get('/summary', response_model=DashboardSummary) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/fox_hunter.py | 51 | No module docstring; infer role from symbols/imports. | ? | fox_hunter_status:17, fox_hunter_overview:25, scan_fox_hunter:33 | fox_hunter_status: @router.get('/status')
fox_hunter_overview: @router.get('/overview')
scan_fox_hunter: @router.post('/scan', response_model=PatternEntryScanResponse) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/health.py | 65 | No module docstring; infer role from symbols/imports. | ? | health:16, health_notice:28, deep_health:51, ibkr_health:64 | health: @router.get('/health')
health_notice: @router.get('/health/notice')
deep_health: @router.get('/health/deep')
ibkr_health: @router.get('/health/ibkr') | IBKR/broker/data dependency |

| backend/tradeo/routers/ibkr.py | 92 | No module docstring; infer role from symbols/imports. | IBKRSignalOrderRequest:18 | ibkr_health:23, ibkr_account:29, ibkr_positions:37, ibkr_open_orders:45, ibkr_preview_signal_order:53, ibkr_submit_signal_bracket:70 | ibkr_health: @router.get('/health')
ibkr_account: @router.get('/account')
ibkr_positions: @router.get('/positions')
ibkr_open_orders: @router.get('/open-orders')
ibkr_preview_signal_order: @router.post('/signals/{signal_id}/preview')
ibkr_submit_signal_bracket: @router.post('/signals/{signal_id}/submit-bracket', response_model=TradeOut) | broad `except Exception` present; IBKR/broker/data dependency |

| backend/tradeo/routers/laboratory.py | 61 | No module docstring; infer role from symbols/imports. | ? | laboratory_status:18, laboratory_overview:26, laboratory_diagnostics_view:34, scan_laboratory:43 | laboratory_status: @router.get('/status')
laboratory_overview: @router.get('/overview')
laboratory_diagnostics_view: @router.get('/diagnostics', response_model=LabDiagnosticsResponse)
scan_laboratory: @router.post('/scan', response_model=PatternEntryScanResponse) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/reports.py | 23 | No module docstring; infer role from symbols/imports. | ? | generate_report:14, latest_report:19 | generate_report: @router.post('/generate')
latest_report: @router.get('/latest') | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/research.py | 225 | No module docstring; infer role from symbols/imports. | ? | run_discovery:35, list_discovered_patterns:44, get_discovered_pattern:54, get_discovered_pattern_examples:71, get_discovered_pattern_metrics:88, get_discovered_pattern_rr_metrics:106, list_confirmation_queue:132, match_current_patterns:147, list_current_matches:165, list_discovery_runs:187 | run_discovery: @router.post('/run-discovery', response_model=DiscoveryRunResponse)
list_discovered_patterns: @router.get('/discovered-patterns', response_model=list[DiscoveredPatternOut])
get_discovered_pattern: @router.get('/discovered-patterns/{pattern_id}', response_model=DiscoveredPatternDetailOut)
get_discovered_pattern_examples: @router.get('/discovered-patterns/{pattern_id}/examples', response_model=list[DiscoveredPatternExampleOut])
get_discovered_pattern_metrics: @router.get('/discovered-patterns/{pattern_id}/metrics', response_model=list[DiscoveredPatternMetricOut])
get_discovered_pattern_rr_metrics: @router.get('/discovered-patterns/{pattern_id}/rr-metrics') | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/risk.py | 37 | No module docstring; infer role from symbols/imports. | ? | set_kill_switch:16 | set_kill_switch: @router.post('/kill-switch') | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/scan.py | 21 | No module docstring; infer role from symbols/imports. | ? | run_scan:15 | run_scan: @router.post("", response_model=ScanResponse) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/self_improvement.py | 20 | No module docstring; infer role from symbols/imports. | ? | run_self_improvement:15 | run_self_improvement: @router.post('/run', response_model=SelfImprovementResponse) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/routers/signals.py | 47 | No module docstring; infer role from symbols/imports. | ? | list_signals:16, human_approve_signal:21, paper_execute_signal:39 | list_signals: @router.get("", response_model=list[SignalOut])
human_approve_signal: @router.post('/{signal_id}/human-approve', response_model=SignalOut)
paper_execute_signal: @router.post('/{signal_id}/paper-execute', response_model=TradeOut) | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/schemas.py | 465 | Pydantic API schemas. Audit request/response boundaries and whether backend fields used by services are surfaced consistently. | PatternFeatures:9, PatternCandidate:22, RiskDecision:40, SupervisorDecision:50, SignalOut:60, TradeOut:84, EquityPointOut:103, PatternMetricOut:112 | ? | ? | JSON metadata contract; schema drift risk |

| backend/tradeo/services/backtester.py | 162 | Simple backtest engine with canonical outcome and non-trade accounting. Audit same-bar assumptions, costs, skipped signals, and parity with research analyzer. | SimulatedTrade:15, Backtester:28 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/data_provider.py | 399 | Market data normalization/cache/incremental refresh layer. Audit PIT behavior, partial-bar exclusion, manifest lineage, data quality, and IBKR adjusted feed assumptions. | MarketDataProvider:17, CachedMarketDataProvider:67 | load_universe:44, pick_symbols:57, detect_unadjusted_splits:376 | ? | wall-clock timestamp usage; IBKR/broker/data dependency |

| backend/tradeo/services/data_quality.py | 149 | Per-symbol OHLCV quality assessment for research-grade scanning. `normalize\_ohlc`/`validate\_ohlc` reject structurally broken bars (NaN, negative prices, high < low). This module covers the next layer: data that is well | DataQualityReport:29 | assess_ohlc_quality:74, assess_ohlc_quality_from_settings:134 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/director_review_gate.py | 1275 | Director review and production gate logic. Audit evidence thresholds, effective samples, skip-rate evidence, blockers, and whether recommendations can be mistaken for approvals. | DirectorReviewGate:37, DirectorProductionGate:806 | ? | ? | broad `except Exception` present; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/services/effective_sample.py | 118 | Effective-sample weights for Director paper-fill evidence (informe §4.7). Paper fills that share the same symbol and trading day are not independent observations of the edge: a burst of five AAPL fills on one afternoon i | ? | trade_cluster_key:44, effective_sample_summary:50, persist_effective_sample_weights:87 | ? | wall-clock timestamp usage; JSON metadata contract; schema drift risk |

| backend/tradeo/services/entry_variants.py | 297 | No module docstring; infer role from symbols/imports. | ? | build_entry_audit_context:13, classify_regime:29, build_entry_variants:61 | ? | wall-clock timestamp usage |

| backend/tradeo/services/evidence.py | 379 | No module docstring; infer role from symbols/imports. | EvidenceType:24, EvidenceQuality:33, FillProvenance:38 | evidence_type_for_metadata:109, evidence_quality_for_metadata:154, fill_provenance_for_metadata:162, evidence_type_from_metadata:172, evidence_quality_from_metadata:193, evidence_metadata_with_stored_columns:197, with_evidence_metadata:211, is_director_review_paper_fill_evidence:232, is_paper_order_or_fill_evidence:248, is_strong_fill_evidence:276 | ? | IBKR/broker/data dependency |

| backend/tradeo/services/execution_quality.py | 215 | Execution-quality audit helpers. Important because it declares absence of real microstructure feed and should not be overstated. | ? | trade_execution_quality:76, persist_execution_quality:151,

pattern_execution_quality_summary:182 | ? | wall-clock timestamp usage; JSON metadata contract; schema drift risk |

| backend/tradeo/services/ibkr_broker.py | 454 | No module docstring; infer role from symbols/imports. | IBKRSafetyError:17, IBKRBroker:38 | ? | ? | broad `except Exception` present; wall-clock timestamp usage; RNG path; verify seed/control; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/services/ibkr_data_provider.py | 156 | IBKR historical data adapter. Audit connection lifecycle, blocking calls, account privacy, bar conversion, and error semantics. | IBKRHistoricalDataProvider:80 | inspect_ibkr_connection:116 | ? | broad `except Exception` present; RNG path; verify seed/control; IBKR/broker/data dependency |

| backend/tradeo/services/implementation_shortfall.py | 114 | Implementation shortfall / slippage_R per trade and per pattern (informe §4.6). For every closed trade with a real broker fill we compare the achieved fills against the theoretical prices the Research labeler assumes: sh | ? | trade_slippage_r:42, pattern_slippage_summary:94 | ? | JSON metadata contract; schema drift risk |

| backend/tradeo/services/lab_diagnostics.py | 4 | Backward-compatible imports for Laboratory diagnostics. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/lab_paper_observations.py | 4 | Backward-compatible imports for Laboratory paper observations. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/market_regime.py | 270 | PIT benchmark regime labeling. Audit strict SMA/vol tercile construction, insufficient-history honesty, source hashes, and no future data. | MarketRegimeService:182 | regime_table:24, regime_at:70, unlabeled_regime_key:117, regime_keys_for_dates:121, period_to_months:149, required_benchmark_bars:170 | ? | broad `except Exception` present |

| backend/tradeo/services/market_session.py | 106 | No module docstring; infer role from symbols/imports. | ? | is_us_equity_regular_session_open:11, market_session_status:24, is_nyse_holiday:47, holiday_name:51 | ? | wall-clock timestamp usage |

| backend/tradeo/services/metrics.py | 35 | No module docstring; infer role from symbols/imports. | ? | refresh_pattern_metrics:9 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/ml_scorer.py | 50 | No module docstring; infer role from symbols/imports. | FeatureScorer:10 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/module_dashboard.py | 535 | No module docstring; infer role from symbols/imports. | ? | module_overview:24 | ? | wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/services/openai_supervisor.py | 84 | No module docstring; infer role from symbols/imports. | OpenAISupervisor:11 | ? | ? | broad `except Exception` present |

| backend/tradeo/services/opportunity_ranking.py | 164 | No module docstring; infer role from symbols/imports. | ? | rank_entry_matches:8 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/order_outcomes.py | 59 | No module docstring; infer role from symbols/imports. | ? | classify_order_failure:9, mark_signal_order_failure:52 | ? | wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/services/paper_broker.py | 58 | No module docstring; infer role from symbols/imports. | PaperBroker:11 | ? | ? | wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/services/pattern_detector.py | 270 | No module docstring; infer role from symbols/imports. | CupDetectorParams:17, CupPatternDetector:32 | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/pattern_entry_scanner.py | 8 | Backward-compatible imports for the shared entry scanner. New code should import from ``tradeo.modules.shared.entry\_scanner`` or through the Laboratory/FoxHunter domain facades. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/pattern_health_monitor.py | 216 | Post-promotion pattern decay detection with CUSUM (informe §4.8). For every pattern in PRODUCTION or DIRECTOR_REVIEW we run a two-sided CUSUM over the chronological series of realized R per trade, standardized against th | PatternHealthMonitor:46 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/services/production_manifest.py | 4 | Backward-compatible imports for FoxHunter production manifests. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/provider_factory.py | 22 | No module docstring; infer role from symbols/imports. | ? | get_market_data_provider:8 | ? | IBKR/broker/data dependency |

| backend/tradeo/services/reconciliation.py | 266 | DB to IBKR reconciliation and automatic kill switch. Audit divergence handling, pending orders, zero quantities, and fail-closed behavior. | ReconciliationService:79 | ? | ? | broad `except Exception` present; wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/services/reports.py | 165 | No module docstring; infer role from symbols/imports. | ReportService:14 | ? | ? | wall-clock timestamp usage; JSON metadata contract; schema drift risk |

| backend/tradeo/services/risk_manager.py | 171 | Sizing and portfolio risk controls. Audit ADV participation, pattern-family exposure, daily loss, short enablement, and default limits. | AccountState:14, RiskManager:22 | ? | ? | wall-clock timestamp usage; JSON metadata contract; schema drift risk |

| backend/tradeo/services/runtime_status.py | 100 | No module docstring; infer role from symbols/imports. | ? | write_worker_heartbeat:14, worker_runtime_status:27, write_entry_scan_status:47, entry_scan_status:70 | ? | wall-clock timestamp usage |

| backend/tradeo/services/scanner.py | 90 | No module docstring; infer role from symbols/imports. | MarketScanner:17 | ? | ? | broad `except Exception` present |

| backend/tradeo/services/self_improvement.py | 330 | Candidate mutation and anti-overfit gate. Audit CSCV/PBO, plateau guard, nested optimization integration, and lab candidate creation conditions. | SelfImprovementEngine:24 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/services/signal_quality.py | 227 | No module docstring; infer role from symbols/imports. | ? | build_entry_quality:9, build_signal_snapshot:99 | ? | wall-clock timestamp usage |

| backend/tradeo/services/state_policy.py | 26 | No module docstring; infer role from symbols/imports. | ? | is_legacy_promotion_state:23 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/strategy_config.py | 19 | No module docstring; infer role from symbols/imports. | ? | load_strategy_config:8 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/supervisor.py | 102 | No module docstring; infer role from symbols/imports. | TradeSupervisor:12 | ? | ? | JSON metadata contract; schema drift risk |

| backend/tradeo/services/system_controls.py | 101 | Runtime kill switch persisted in the database (informe \$4.5). TRADEO_KILL_SWITCH_ENABLED is read once at startup, so an automatic safety trigger could never make it bite without a container restart. This module adds a DB | ? | runtime_kill_switch:33, runtime_kill_switch_active:37, activate_runtime_kill_switch:42, deactivate_runtime_kill_switch:77 | ? | wall-clock timestamp usage |

| backend/tradeo/services/technical_indicators.py | 89 | No module docstring; infer role from symbols/imports. | ? | sma:7, atr:11, max_drawdown_pct:22, normalize_ohlc:31, validate_ohlc:55 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/universe_snapshot.py | 208 | No module docstring; infer role from symbols/imports. | UniverseSnapshotService:20 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/services/vision_scorer.py | 75 | No module docstring; infer role from symbols/imports. | VisionGeometryScorer:11 | render_pattern_chart:46 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/services/watchdog.py | 110 | No module docstring; infer role from symbols/imports. | SystemWatchdog:14 | ? | ? | wall-clock timestamp usage |

| backend/tradeo/tasks/__init__.py | 1 | No module docstring; infer role from symbols/imports. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tasks/worker.py | 374 | No module docstring; infer role from symbols/imports. | ? | scan_job:30, report_job:45, self_improvement_job:56, discovery_job:67, novel_match_job:117, research_director_job:138, laboratory_entry_job:156, fox_hunter_entry_job:175, reconciliation_job:192, pattern_health_job:212 | ? | broad `except Exception` present; blocking `time.sleep` present; wall-clock timestamp usage; IBKR/broker/data dependency |

| backend/tradeo/tests/conftest.py | 23 | Test-session environment defaults. Settings path properties (reports, artifacts, market data cache, universe snapshots) create their directories on first access. The shipped defaults live under ``/app`` which only exists | ? | ? | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/fixtures.py | 28 | No module docstring; infer role from symbols/imports. | ? | fixture_ohlc:7 | ? | wall-clock timestamp usage; RNG path; verify seed/control |

| backend/tradeo/tests/test_agent_c_remediation.py | 70 | No module docstring; infer role from symbols/imports. | ? | test_backtester_uses_canonical_both_touch_stop_first:27, test_backtester_gap_past_target_entry_is_skipped:39, test_self_improvement_blocks_when_pbo_unavailable:48 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/test_audit_hardening.py | 1012 | No module docstring; infer role from symbols/imports. | ? | test_validation_gate_never_promotes_research_metric_to_paper_candidate:18, test_director_gate_blocks_zero_trade_promotions_and_unproven_oos_fit:65, test_export_filters_fills_to_exported_paper_trade_ids:129, test_export_excludes_shadow_observations_from_paper_trades:170, test_export_marks_real_paper_fills_with_evidence_contract:216, test_export_rejects_closed_paper_trade_without_broker_provenance:263, test_export_leaves_operational_metrics_empty_without_closed_lab_trades:303, test_export_metrics_reconstruct_independent_sample_counts_from_event_rows:335, test_export_groups_closed_lab_trade_performance_by_regime_and_entry_variant:351, test_audit_validator_accepts_manifest_gate_buckets_and_reported_duplicates:389 | ? | subprocess usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/tests/test_autonomous_research_director.py | 133 | No module docstring; infer role from symbols/imports. | ? | session_factory:14, test_research_director_enriches_patterns_and_writes_artifacts:98, test_research_director_latest_artifact_is_reusable_json:123 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/test_autonomous_research_lab.py | 281 | No module docstring; infer role from symbols/imports. | ? | test_market_replay_penalizes_latency_and_partial_fills:118, test_validation_gate_rejects_adversarial_hard_failure:134, test_research_director_writes_memory_graph_papers_and_agenda:179, test_hypothesis_engine_emits_immutable_package:252 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/test_backtester_non_trade_accounting.py | 76 | No module docstring; infer role from symbols/imports. | _Params:13, _StubDetector:17, _StubProvider:40 | test_gap_prefiltered_signal_counts_as_skipped_not_trade:57, test_executed_signal_counts_as_trade_with_zero_skips:68 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/test_data_provider.py | 487 | No module docstring; infer role from symbols/imports. | _RecordingUpstream:123, FakeIB:440 | test_settings_reject_synthetic_market_data:19, test_settings_reject_non_ibkr_market_data_provider:24, test_market_data_factory_returns_ibkr_provider:29, test_market_data_factory_wraps_ibkr_with_cache:44, test_cached_market_data_provider_persists_manifest:74, test_incremental_refresh_appends_missing_tail:172, test_incremental_refresh_full_refetch_on_overlap_mismatch:202, test_incremental_refresh_disabled_keeps_cache_untouched:227, test_incremental_refresh_skips_fresh_cache:245, test_intraday_incremental_refresh_appends_missing_tail:263 | ? | wall-clock timestamp usage; IBKR/broker/data dependency |

| backend/tradeo/tests/test_data_quality.py | 124 | No module docstring; infer role from symbols/imports. | ? | test_clean_fixture_is_research_grade:17, test_empty_frame_is_rejected:25, test_insufficient_bars_flagged:31, test_zero_volume_run_flagged:37, test_stale_close_run_flagged:45, test_calendar_gap_flagged_for_daily_bars:53, test_calendar_gap_ignored_for_intraday:62, test_suspect_split_jump_flagged:69, test_downward_split_jump_flagged_symmetrically:77, test_report_to_dict_is_json_friendly:84 | ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/test_director_review_gate.py | 661 | No module docstring; infer role from symbols/imports. | ? | session_factory:22, add_pattern:28, add_closed_lab_trade:46, production_contract_metrics:119, test_director_review_gate_waits_for_ten_closed_lab_trades:149, test_director_production_gate_blocks_missing_nested_replay_contract:169, test_director_review_gate_marks_candidate_after_ten_closed_lab_trades:192, test_director_review_gate_blocks_until_effective_lab_trade_minimum:214, test_director_review_gate_blocks_low_diversity_lab_results:232, test_director_review_gate_ignores_shadow_and_near_miss_no_order_observations:247 | ? | IBKR/broker/data dependency; JSON metadata contract; schema drift risk |

| backend/tradeo/tests/test_discovery_determinism.py | 155 | Bit-for-bit determinism contract for the discovery/research path. Runs the real ClusterResearchEngine twice over identical synthetic fixtures and asserts the full candidate payload (metrics included) is byte-identical un | ? | test_canonical_payload_is_order_and_type_insensitive:91, test_content_hash_excludes_volatile_keys:97,

test_canonical_json_maps_non_finite_to_null:109, test_discovery_is_bit_for_bit_deterministic_across_runs:114,
test_discovery_is_independent_of_input_sample_order:125, test_discovery_report_content_hash_is_stable_across_run_ids:134 | ? |
RNG path; verify seed/control |
| backend/tradeo/tests/test_domain_module_facades.py | 53 | No module docstring; infer role from symbols/imports. | ? |
test_laboratory_scanner_uses_laboratory_module:7, test_fox_hunter_scanner_uses_fox_hunter_module:31 | ? | no obvious heuristic
risk; still review logic/tests |
| backend/tradeo/tests/test_effective_sample_weights.py | 228 | Persisted effective-sample weights for Director paper fills (informe
\$4.7). | ? | session_factory:28, add_pattern:34, add_paper_fill:52,
test_same_symbol_same_day_cluster_counts_as_one_effective_sample:115,
test_mixed_clusters_report_expected_n_eff_and_kish:134, test_independent_fills_keep_full_effective_sample:150,
test_gate_blocks_concentrated_fills_even_above_raw_count_minimum:169,
test_gate_persists_per_trade_weights_in_trade_metadata:189, test_gate_accepts_diversified_effective_sample:210 | ? |
IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/tests/test_execution_quality_audit.py | 255 | Entry-quality execution audit tests (informe \$4.3 fallback path). No
real-time microstructure feed exists, so the auditable surface is the broker fill record itself: fill price vs theoretical entry, submit->fill
latency | ? | make_trade:22, test_long_fill_above_theoretical_entry_is_positive_slippage:53,
test_short_fill_below_theoretical_entry_is_positive_slippage:64, test_better_than_theoretical_fill_reports_negative_slippage:75,
test_time_to_fill_from_submitted_at_and_fill_time:84, test_time_to_fill_falls_back_to_broker_execution_time:98,
test_negative_fill_latency_is_rejected_not_reported:112, test_commission_bps_relative_to_fill_notional:126,
test_estimated_vs_realized_compares_pretrade_estimates:137, test_data_basis_markers_declare_missing_microstructure_feed:151 |
? | JSON metadata contract; schema drift risk |
| backend/tradeo/tests/test_execution_state_transitions.py | 738 | Explicit order-state transition and reconciliation tests (informe \$4.5).
Covers the trade state machine end to end: - signal -> trade transitions in the paper broker; - lab shadow observation lifecycle (open ->
closed / | FakeBroker:104, FakeProvider:185 | session_factory:46, add_signal:52, add_open_trade:75, audit_actions:127,
test_paper_broker_opens_trade_and_marks_signal_executed:136, test_paper_broker_rejects_non_executable_signal_states:159,
test_paper_broker_rejects_zero_quantity:170, make_frame:197, add_shadow_observation:204, observation_service:233 | ? |
IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/tests/test_ibkr_data_provider.py | 28 | No module docstring; infer role from symbols/imports. | DummyIB:9 |
test_ibkr_provider_sets_event_loop_before_thread_import:14 | ? | IBKR/broker/data dependency |
| backend/tradeo/tests/test_lab_diagnostics.py | 278 | No module docstring; infer role from symbols/imports. | ? | session_factory:23,
add_pattern:29, add_closed_paper_trade:55, add_closed_shadow_observation:112,
test_laboratory_diagnostics_combines_candidates_rejections_and_paper_history:165 | ? | IBKR/broker/data dependency; JSON
metadata contract; schema drift risk |
| backend/tradeo/tests/test_market_regime.py | 169 | No module docstring; infer role from symbols/imports. | ? |
test_trend_regime_bull_and_bear:37, test_trend_requires_full_sma_window:50, test_vol_tercile_high_after_calm_history:59,
test_vol_tercile_low_after_turbulent_history:67, test_regime_labels_are_point_in_time_stable:74,
test_regime_at_respects_as_of_cutoff:94, test_regime_at_empty_frame_is_honest:104,
test_regime_key_combines_trend_and_vol:112, test_market_regime_service_uses_provider_and_settings:127,
test_market_regime_service_survives_provider_failure:151 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_market_session.py | 32 | No module docstring; infer role from symbols/imports. | ? |
test_us_equity_regular_session_is_open_during_weekday_rth:12, test_us_equity_regular_session_is_closed_after_hours:16,
test_market_session_status_reports_closed_state:20, test_market_session_closes_on_nyse_holiday:27 | ? | no obvious heuristic risk;
still review logic/tests |
| backend/tradeo/tests/test_module_dashboard.py | 414 | No module docstring; infer role from symbols/imports. | ? | session_factory:13,
test_legacy_ibkr_paper_research_trades_are_laboratory_history:19, test_module_overview_exposes_dashboard_data_scope:67,
test_laboratory_overview_api_exposes_default_dashboard_data_scope:81,
test_legacy_ibkr_paper_smoke_order_is_laboratory_history:93, test_cancelled_paper_trade_is_signal_status_not_operation:139,
test_closed_trade_keeps_signal_executed_and_operation_closed:186,
test_unsubmitted_signal_keeps_approval_status_for_module_dashboard:241,
test_approved_signal_without_trade_is_retryable_for_module_dashboard:290,
test_expired_signal_is_discarded_not_retried_for_module_dashboard:321 | ? | IBKR/broker/data dependency; JSON metadata contract;
schema drift risk |
| backend/tradeo/tests/test_nested_optimization.py | 185 | No module docstring; infer role from symbols/imports. | ? |
test_overfit_specialists_are_blocked:43, test_persistent_edge_passes:56, test_consistent_loser_fails_outer_score_gate:68,
test_insufficient_periods_blocks:76, test_insufficient_variants_blocks:83, test_non_finite_values_block:89,
test_report_is_deterministic:97, test_fallback_contract_reports_optuna_availability:103,
test_optuna_inner_search_is_seeded_and_deterministic:111, test_engine_nested_guard_blocks_specialist_overfit:138 | ? | RNG path;
verify seed/control |
| backend/tradeo/tests/test_novel_pattern_registry.py | 123 | No module docstring; infer role from symbols/imports. | ? |
session_factory:14, test_registry_deduces_similar_centroids:52, test_matcher_scales_legacy_centroid_prefix:77,
test_registry_persists_confirmation_lifecycle:89, test_registry_blocks_legacy_promotion_states:113 | ? | no obvious heuristic risk; still
review logic/tests |
| backend/tradeo/tests/test_pattern_detector.py | 11 | No module docstring; infer role from symbols/imports. | ? |
test_detector_runs_without_crash:8 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_pattern_discovery_lab.py | 689 | No module docstring; infer role from symbols/imports. | ? |
test_window_sampler_generates_forward_labeled_samples:62, test_window_sampler_adds_benchmark_relative_strength:94,
test_cluster_engine_returns_candidates_or_empty_without_crashing:119, test_cluster_engine_fits_scaler_on_train_only:152,
test_cluster_signature_records_medoid_and_concentration:203, test_cluster_engine_selects_best_rr_from_train_only:224,
test_validation_gate_rejects_underpowered_candidate:247, test_validation_gate_allows_edge_below_4r_as_watchlist:264,
test_validation_gate_rejects_high_adjusted_null_p_value:303, test_validation_gate_rejects_failed_reality_check_and_edge_decay:346
| ? | no obvious heuristic risk; still review logic/tests |

| backend/tradeo/tests/test_pattern_entry_scanner.py | 1307 | No module docstring; infer role from symbols/imports. |
 FixtureProvider:32, StaticProvider:43, NoBarsProvider:51, StaticMatcher:56 | session_factory:71, add_pattern:77,
 add_shadow_observation:115, match_payload:164, near_miss_volume_payload:199, scanner:244,
 test_laboratory_scanner_creates_paper_signal_for_validated_lab_pattern:266,
 test_laboratory_matcher_records_feature_parity_and_ambiguity_ratio:324,
 test_laboratory_volume_near_miss_opens_shadow_observation_without_ibkr:381,
 test_laboratory_near_miss_hard_blocker_does_not_open_shadow_observation:462 | ? | IBKR/broker/data dependency; JSON
 metadata contract; schema drift risk |
 | backend/tradeo/tests/test_quant_validation.py | 346 | No module docstring; infer role from symbols/imports. | ? |
 test_both_barriers_same_bar_resolves_to_stop:43, test_gap_open_through_stop_fills_at_open_worse_than_stop:56,
 test_gap_open_through_target_fills_at_target_not_open:68, test_entry_bar_gap_through_stop_is_skipped:81,
 test_entry_bar_gap_past_target_is_skipped:92, test_time_stop_exits_at_close_of_last_bar:103, test_short_side_is_symmetric:117,
 test_mfe_mae_are_recorded_in_r:134, test_round_trip_cost_reduces_r:145, test_select_nonoverlapping_keeps_disjoint_events:167 | ? |
 | RNG path; verify seed/control |
 | backend/tradeo/tests/test_quant_validation_integration.py | 388 | No module docstring; infer role from symbols/imports. |
 _PatternStub:309 | test_engine_quant_validation_dedups_overlapping_occurrences:108,
 test_engine_quant_validation_keeps_disjoint_symbols:120, test_engine_match_tau_from_cluster_dispersion:130,
 test_run_level_inference_applies_fdr_and_family_dsr:159, test_validation_gate_rejects_low_n_eff_and_failed_fdr:177,
 test_validation_gate_caps_lab_candidate_without_dsr:197, test_validation_gate_caps_lab_candidate_with_survivorship_bias:226,
 test_matcher_drops_live_daily_bar_during_session:282, test_matcher_keeps_bar_after_close_and_yesterday_bars:290,
 test_matcher_ignores_intraday_frames:303 | ? | RNG path; verify seed/control; JSON metadata contract; schema drift risk |
 | backend/tradeo/tests/test_rediscovery_readiness.py | 158 | No module docstring; infer role from symbols/imports. | ? |
 session_factory:21, modern_metrics:27, add_pattern:44, test_legacy_pattern_without_fields_is_flagged:52,
 test_modern_pattern_with_all_fields_is_ok:66, test_outdated_embedding_contract_is_flagged_as_stale:78,
 test_dry_run_manifest_is_honest_and_mutates_nothing:91, test_apply_flags_marks_legacy_but_does_not_fake_metadata:111,
 test_flag_cleared_after_real_metadata_arrives:128, test_manifest_written_to_disk_and_truncation_reported:147 | ? | no obvious
 heuristic risk; still review logic/tests |
 | backend/tradeo/tests/test_regime_outcome_calibration.py | 262 | No module docstring; infer role from symbols/imports. | ? |
 test_regime_keys_for_dates_is_point_in_time:72, test_regime_keys_for_dates_without_table_is_unlabeled:88,
 test_period_to_months_parses_provider_periods:94, test_history_table_covers_period_plus_warmup_and_records_config:103,
 test_benchmark_regime_outcomes_buckets_by_pit_regime:141, test_benchmark_regime_outcomes_unavailable_without_table:174,
 test_pattern_regime_fit_calibrated_positive_bucket:211, test_pattern_regime_fit_calibrated_negative_bucket_is_hard_gate_eligible:225,
 test_pattern_regime_fit_falls_back_when_bucket_too_small:237,
 test_pattern_regime_fit_falls_back_for_unlabeled_benchmark_regime:248 | ? | no obvious heuristic risk; still review logic/tests |
 | backend/tradeo/tests/test_remediation_docs_traceability.py | 129 | Traceability checks for the 12-phase remediation documentation.
 Every remediation report under ``docs/remediation`` lists the files it changed. These tests assert that those references still resolve to
 real files in the | ? | test_remediation_docs_directory_present_or_skipped:50, test_files_changed_references_resolve_to_real_files:56,
 test_compliance_matrix_covers_all_agents:86, test_every_agent_report_is_reflected_in_matrix:96,
 test_audit_export_exposes_independent_sample_fields:122 | ? | no obvious heuristic risk; still review logic/tests |
 | backend/tradeo/tests/test_research_lab_fox_lifecycle.py | 285 | No module docstring; infer role from symbols/imports. |
 FixtureProvider:26 | session_factory:43, settings:49, add_research_approved_pattern:71, close_lab_trade:127,
 test_research_to_lab_to_director_to_fox_lifecycle:174 | ? | IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
 | backend/tradeo/tests/test_reward_risk_analyzer.py | 142 | No module docstring; infer role from symbols/imports. | ? |
 test_target_and_stop_same_bar_counts_stop_first:51, test_expectancy_profit_factor_and_drawdown:59,
 test_best_rr_uses_edge_score_not_highest_rr:71, test_simulation_subtracts_execution_cost_r:82,
 test_cost_multiplier_stresses_result_r:97, test_canonical_outcome_skips_entry_gap_past_target:103,
 test_skipped_signals_do_not_dilute_traded_aggregates:112, test_all_skipped_signals_yield_zero_sample_count:133 | ? | no obvious
 heuristic risk; still review logic/tests |
 | backend/tradeo/tests/test_risk.py | 134 | No module docstring; infer role from symbols/imports. | ? |
 test_position_size_respects_30_usd_risk_budget:15, test_rejects_reward_risk_below_four:34,
 test_position_size_respects_adv_participation_cap:53, test_rejects_when_pattern_family_position_cap_reached:76 | ? | wall-clock
 timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
 | backend/tradeo/tests/test_runtime_status.py | 36 | No module docstring; infer role from symbols/imports. | ? |
 test_entry_scan_status_accumulates_symbols:7, test_entry_scan_status_keeps_market_closed_reason:19 | ? | no obvious heuristic
 risk; still review logic/tests |
 | backend/tradeo/tests/test_shadow_canonical_outcome_parity.py | 301 | ShadowTracker canonical-outcome parity tests (informe §6).
 Shadow observation exits must use the same triple-barrier fill rules as the backtester and Research RR simulation
 ('triple_barrier_outcome'): - a bar that opens | FakeProvider:46 | session_factory:40, make_frame:54, bar:61,
 add_shadow_observation:65, service:119, test_gap_through_stop_on_first_bar_fills_at_open:126,
 test_gap_through_stop_on_later_bar_fills_at_open:142, test_gap_past_target_fills_at_target_not_open:160,
 test_intrabar_stop_and_target_resolves_to_stop:178, test_short_side_gap_through_stop_fills_at_open:193 | ? | IBKR/broker/data
 dependency; JSON metadata contract; schema drift risk |
 | backend/tradeo/tests/test_watchdog.py | 35 | No module docstring; infer role from symbols/imports. | ? |
 test_watchdog_closes_stale_running_discovery_run:13 | ? | wall-clock timestamp usage |
 | research/audit_bridge/director_gate.py | 921 | Standalone audit-bridge Director gate. Audit blockers, severity thresholds, and
 independence/sample count handling. | GateInputsError:81 | main:85, load_rows:137, read_csv:151, read_csv_optional:159,
 evaluate:165, scientific_contract_status:387, count_blank:502, count_rows_missing_any:506, has_active_blocker_text:514,
 research_metrics_in_operational_columns:519 | ? | wall-clock timestamp usage; JSON metadata contract; schema drift risk |
 | research/audit_bridge/export_audit_package.py | 2373 | External audit package exporter. Audit evidence reconstruction, secret
 redaction, fill provenance, count reconciliation, and contract adherence. | ? | main:371, read_env:491, api_get:505,
 api_get_optional:512, run_git:519, read_universe_symbols:523, write_csv:530, csv_value:538, pattern_id:548, normalize_ts:552 | ? |

broad `except Exception` present; wall-clock timestamp usage; subprocess usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| research/audit_bridge/run_director_audit.py | 409 | No module docstring; infer role from symbols/imports. | CommandRun:23 | main:32, parse_args:124, build_audit_id:136, default_api_url:141, collect_compose_status:147, compose_ps_command:181, parse_compose_ps_json:192, health_from_status:230, run_exporter:239, run_subprocess:288 | ? | broad `except Exception` present; wall-clock timestamp usage; subprocess usage |
| research/audit_bridge/validate_audit_package.py | 599 | No module docstring; infer role from symbols/imports. | ? | main:160, validate_package:179, validation_report:222, load_json_silent:245, director_gate_status_value:254, director_gate_promotion_allowed:271, load_json:282, check_manifest:292, check_director_gate_result:333, read_csv:370 | ? | broad `except Exception` present; IBKR/broker/data dependency |

Critical Full Source Appendix

Full source for critical backend/research/execution files. These are not summarized in place; the complete code is included for external audit.

Full Source: `backend/tradeo/main.py`

`backend/tradeo/main.py` ? 63 lines, 1841 bytes, sha256 prefix `7ffe9ea256bef3ba`

Public surface summary before full source:

```
? async function `lifespan` line 28`  
? function `create_app` line 38`
```

Risk hints: IBKR/broker/data dependency.

```
from __future__ import annotations  
  
from contextlib import asynccontextmanager  
  
from fastapi import FastAPI  
from fastapi.middleware.cors import CORSMiddleware  
  
from tradeo.core.config import get_settings  
from tradeo.db.init_db import init_db, seed_db  
from tradeo.db.session import SessionLocal  
from tradeo.routers import (  
    backtests,  
    dashboard,  
    fox_hunter,  
    health,  
    ibkr,  
    laboratory,  
    reports,  
    research,  
    risk,  
    scan,  
    self_improvement,  
    signals,  
)  
  
@asynccontextmanager  
async def lifespan(app: FastAPI):  
    init_db()  
    db = SessionLocal()  
    try:  
        seed_db(db)  
    finally:  
        db.close()  
    yield  
  
def create_app() -> FastAPI:  
    settings = get_settings()  
    app = FastAPI(title=settings.app_name, version="0.1.0", lifespan=lifespan)  
    app.add_middleware(  
        CORSMiddleware,  
        allow_origins=settings.cors_origins_list,  
        allow_credentials=True,  
        allow_methods=["*"],  
        allow_headers=["*"],  
    )  
    app.include_router(health.router, prefix=settings.api_prefix)  
    app.include_router(ibkr.router, prefix=settings.api_prefix)  
    app.include_router(laboratory.router, prefix=settings.api_prefix)  
    app.include_router(fox_hunter.router, prefix=settings.api_prefix)  
    app.include_router(dashboard.router, prefix=settings.api_prefix)  
    app.include_router(scan.router, prefix=settings.api_prefix)  
    app.include_router(signals.router, prefix=settings.api_prefix)  
    app.include_router(backtests.router, prefix=settings.api_prefix)  
    app.include_router(risk.router, prefix=settings.api_prefix)  
    app.include_router(reports.router, prefix=settings.api_prefix)  
    app.include_router(research.router, prefix=settings.api_prefix)  
    app.include_router(self_improvement.router, prefix=settings.api_prefix)  
    return app
```

```
app = create_app()
```

Full Source: `backend/tradeo/core/config.py`

`backend/tradeo/core/config.py` ? 386 lines, 16544 bytes, sha256 prefix `643a2abe853a4816`

Audit relevance: Central Pydantic settings and validation contract. Audit env defaults, fail-closed trading settings, IBKR-only provider enforcement, and anti-overfit knobs.

Public surface summary before full source:

? class `Settings` line 13 methods: `ibkr_allowed_symbol_set`, `only_ibkr_market_data`, `synthetic_market_data_disabled`, `pct_bounds`, `cors_origins_list`, `discovery_window_size_list`, `discovery_forward_bar_list`, `discovery_rr_level_list`, `discovery_cost_stress_multiplier_list`, `reports_path`, `artifacts_path`, `market_data_cache_path`

? function `get\_settings` line 385`

Risk hints: IBKR/broker/data dependency.

```
from __future__ import annotations

from functools import lru_cache
from pathlib import Path
from typing import Literal

from pydantic import field_validator
from pydantic_settings import BaseSettings, SettingsConfigDict

TradingMode = Literal["research", "paper", "live"]

class Settings(BaseSettings):
    """Runtime settings for Tradeo.

    The defaults are deliberately conservative: paper mode, no live orders, no options,
    no margin and conservative reward/risk requirements.
    """

    model_config = SettingsConfigDict(env_file=".env", env_prefix="TRADEO_", extra="ignore")

    app_name: str = "Tradeo"
    environment: str = "local"
    api_prefix: str = "/api"
    cors_origins: str = "http://localhost:3000,http://127.0.0.1:3000"

    database_url: str = "postgresql+psycopg://tradeo:tradeo@db:5432/tradeo"
    redis_url: str = "redis://redis:6379/0"

    secret_key: str = "change-me-before-live"
    admin_username: str = "admin"
    admin_password: str = "change-me"

    trading_mode: TradingMode = "paper"
    live_trading_enabled: bool = False
    live_trading_confirmation_phrase: str = "I_ACCEPT_LIVE_MARKET_RISK"
    live_trading_confirmation_value: str = ""
    kill_switch_enabled: bool = False

    initial_capital_usd: float = 3000.0
    risk_per_trade_pct: float = 0.01
    daily_loss_limit_pct: float = 0.02
    monthly_loss_limit_pct: float = 0.08
    max_open_positions: int = 4
    max_gross_exposure_pct: float = 0.95
    min_reward_risk: float = 4.0
    preferred_reward_risk: float = 3.0
    premium_reward_risk: float = 4.0
    max_position_value_pct: float = 0.45
    max_adv_participation_pct: float = 0.005
    max_open_positions_per_pattern_family: int = 2

    allow longs: bool = True
    allow shorts: bool = True
    allow_options: bool = False
    allow_margin: bool = False

    market_data_provider: str = "ibkr"
    allow_synthetic_market_data: bool = False
    market_data_cache_enabled: bool = True
    market_data_cache_dir: str = "/app/data/ohlcv_cache"
    market_data_adjusted: bool = True
    market_data_what_to_show: str = "ADJUSTED_LAST"
    # Incremental daily-cache refresh: append only the missing tail after
    # verifying an overlap window against cached bars; any mismatch (e.g.
    # dividend/split re-adjustment) forces an honest full refetch.
    market_data_incremental_enabled: bool = True
    market_data_incremental_overlapBars: int = 5
    market_data_incremental_min_gap_days: int = 1
    market_data_incremental_max_gap_days: int = 45
    # Intraday cache refresh reuses the same overlap-verified tail append.
    # Gap thresholds are bar-relative (min) and wall-clock (max) because a
```

```

# weekend gap on a 5m cache is normal while a 5-day-old intraday cache
# is cheaper to refetch in full than to stitch.
market_data_incremental_intraday_enabled: bool = True
market_data_incremental_intraday_min_gap_bars: int = 2
market_data_incremental_intraday_max_gap_days: int = 5
universe_file: str = "/app/data/universe-us_mid-small.csv"
universe_snapshot_monthly: bool = True
universe_snapshot_dir: str = "/app/data/universe_snapshots"
universe_point_in_time_available: bool = False
# No licensed delisting/PIT membership vendor is wired yet; snapshots stay
# honest via survivorship flags until this flips with a real data source.
universe_delisting_data_available: bool = False
market_regime_benchmark_symbol: str = "SPY"
market_regime_sma_window: int = 200
market_regime_vol_window: int = 20
market_regime_vol_tercile_lookback: int = 252
# Calibrated regime gate (3.8): minimum research-labeled outcomes in the
# current benchmark bucket before the calibrated fit replaces heuristics,
# and whether a calibrated-negative bucket hard-blocks new matches.
market_regime_outcome_min_samples: int = 12
market_regime_hard_gate_enabled: bool = False
survivorship_cap_state: str = "lab_watchlist"
strategy_config_file: str = "/app/config/strategy_cup_v0.json"
reports_dir: str = "/app/reports"
artifacts_dir: str = "/app/artifacts"

scan_period: str = "2y"
scan_interval: str = "1d"
scan_limit_default: int = 50
min_avg_dollar_volume: float = 5_000_000.0
min_price: float = 2.0
max_atr_pct: float = 0.14

# Per-symbol OHLCV quality gate applied before pattern detection. Symbols
# with halted/illiquid bars, frozen feeds, unadjusted splits or calendar
# holes are skipped and recorded in AuditLog instead of being sampled.
data_quality_filter_enabled: bool = True
data_quality_min_bars: int = 60
data_quality_max_zero_volume_pct: float = 0.15
data_quality_max_stale_close_run: int = 8
data_quality_max_single_gap_business_days: int = 5
data_quality_max_bar_return_ratio: float = 4.0

# Novel pattern discovery laboratory. This never routes orders; it creates
# LAB candidates and compact review artifacts for supervisor/API review.
discovery_enabled: bool = True
discovery_scheduler_enabled: bool = True
# >= 1440 means "one run per completed daily bar": the worker switches from an
# interval to a post-close cron trigger. Intraday reruns over the same daily
# bar only inflate N_trials without adding information (P0-8).
discovery_scan_minutes: int = 1440
discovery_post_close_hour_utc: int = 22
discovery_post_close_minute_utc: int = 15
discovery_period: str = "5y"
discovery_interval: str = "1d"
discovery_limit_default: int = 80
discovery_window_sizes: str = "20,50,100,200"
discovery_forward_bars: str = "5,10,20"
discovery_rr_levels: str = "2.5,4.0"
discovery_stride: int = 3
discovery_max_total_windows: int = 12000
discovery_max_windows_per_symbol: int = 450
discovery_min_cluster_size: int = 60
discovery_max_clusters_per_window: int = 12
discovery_min_samples: int = 100
# Gate over the effective sample size (sum of average-uniqueness weights of
# deduplicated occurrences). Raw overlapping windows overstate evidence by
# 4-10x (P0-1); n_eff is what the statistics actually see.
discovery_min_effective_samples: float = 60.0
# Run-level Benjamini-Hochberg FDR over the permutation p-values of every
# cluster evaluated in the run, accepted and rejected alike (P0-2).
discovery_fdr_q: float = 0.05
# Family-deflated Sharpe gate for lab_candidate. Uses N_trials accumulated in
# the global experiment registry so mining more makes the bar rise.
discovery_min_dsr: float = 0.95
discovery_min_symbols: int = 8
discovery_min_years: int = 2
discovery_min_reward_risk: float = 2.5
discovery_candidate_reward_risk: float = 3.0
discovery_premium_reward_risk: float = 4.0
discovery_max_drawdown_r: float = 12.0
unvalidated_pattern_min_reward_risk: float = 4.0
discovery_min_profit_factor: float = 1.8
discovery_min_expectancy_r: float = 0.25
discovery_min_stability_score: float = 0.45
discovery_max_adjusted_p_value: float = 0.25
discovery_confirmation_min_samples: int = 50
discovery_confirmation_min_profit_factor: float = 1.6
discovery_confirmation_min_expectancy_r: float = 0.20
discovery_out_of_sample_pct: float = 0.25
discovery_walk_forward_folds: int = 4
discovery_walk_forward_embargo_samples: int = 5
discovery_min_walk_forward_positive_rate: float = 0.50
discovery_min_expectancy_ci_low: float = 0.0
discovery_max_overfit_score: float = 0.65
discovery_cost_stress_multipliers: str = "1.0,2.0,3.0"

```

```

discovery_required_cost_stress_multiplier: float = 2.0
discovery_store_rejected: bool = True
discovery_report_top_n: int = 20
discovery_match_enabled: bool = True
discovery_match_scan_minutes: int = 30
discovery_match_symbol_limit: int = 80
discovery_match_max_patterns: int = 25
discovery_match_similarity_threshold: float = 0.45
# Never match against a live (incomplete) daily bar: in-session bars carry
# partial volume and provisional high/low, which breaks Research<->Lab
# feature parity (P0-3). The matcher drops today's bar before the close.
discovery_match_complete_bars_only: bool = True
# Per-pattern similarity threshold derived from the intra-cluster similarity
# distribution (tau = this percentile of member distances to the centroid).
# The global similarity threshold remains as a safety floor only (P0-4).
discovery_match_per_pattern_threshold: bool = True
discovery_match_tau_percentile: float = 92.5
discovery_match_max_results: int = 100
discovery_registry_similarity_threshold: float = 0.96
research_director_enabled: bool = True
research_director_interval_minutes: int = 180
research_director_pattern_limit: int = 120
entry_gate_enabled: bool = True
entry_min_score: float = 0.50
entry_min_quality_score: float = 0.60
entry_min_regime_score: float = 0.45
entry_min_volume_ratio: float = 1.05
entry_max_extension_atr: float = 2.75
entry_cooldown_minutes: int = 60
entry_variant_max_per_pattern_symbol: int = 3
entry_exploration_rate: float = 0.15

# Laboratory scans validated Research patterns in paper mode. Paper order
# submission is enabled by default, but still passes through entry/risk,
# paper-mode, kill-switch, live-armed and IBKR live-port safety gates.
laboratory_scanner_enabled: bool = True
laboratory_scan_minutes: int = 5
laboratory_symbol_limit: int = 80
laboratory_max_patterns: int = 25
laboratory_similarity_threshold: float = 0.45
laboratory_store_signals: bool = True
laboratory_auto_submit_paper_orders: bool = True
laboratory_market_hours_only: bool = True

# Fox Hunter scans production patterns. Live order submission requires both
# this explicit switch and the existing live_armed safety gate.
fox_hunter_enabled: bool = True
fox_hunter_scan_minutes: int = 1
fox_hunter_symbol_limit: int = 80
fox_hunter_max_patterns: int = 25
fox_hunter_similarity_threshold: float = 0.50
fox_hunter_store_signals: bool = True
fox_hunter_auto_submit_live_orders: bool = False
fox_hunter_market_hours_only: bool = True

# Director sequential evaluation (informe §4.7). n=10 closed lab trades is
# only the review trigger; the decision additionally uses a Bayesian
# posterior shrunk toward the Research expectancy, an SPRT fast-kill and a
# KS lab-vs-research distribution check.
director_sequential_evaluation_enabled: bool = True
director_posterior_min_probability: float = 0.80
director_posterior_min_edge_r: float = 0.10
director_sprt_alpha: float = 0.05
director_sprt_beta: float = 0.20
director_min_eff_trades: int = 25
director_min_symbols: int = 8
director_min_days: int = 10
# Implementation shortfall gate (informe §4.6): median slippage_R over real
# broker fills must stay below this for the pattern's edge to count as
# executable.
director_max_median_slippage_r: float = 0.10
director_min_slippage_samples: int = 5

# DB <-> IBKR reconciliation (informe §4.5). Confirmed divergence between
# open trades in the DB and broker positions/open orders activates the
# runtime kill switch automatically. Broker connection failures never do.
reconciliation_enabled: bool = True
reconciliation_interval_minutes: int = 30
reconciliation_auto_kill_switch: bool = True

# Pattern health monitor (informe §4.8): CUSUM over realized R per trade
# vs the Research expectancy for PRODUCTION/DIRECTOR_REVIEW patterns. A
# downward trigger marks the pattern drift_status='decaying' and the
# matcher stops generating new signals for it until re-validation.
health_monitor_enabled: bool = True
health_monitor_interval_minutes: int = 60
health_monitor_min_trades: int = 8
health_monitor_cusum_k: float = 0.5
health_monitor_cusum_h: float = 4.0
health_monitor_shortfall_cusum_k: float = 0.025
health_monitor_shortfall_cusum_h: float = 0.20
health_monitor_block_decaying: bool = True

openai_api_key: str | None = None
openai_supervisor_model: str = "gpt-5.5-pro"
openai_supervisor_enabled: bool = False

```

```

ibkr_host: str = "host.docker.internal"
ibkr_port: int = 7497
ibkr_client_id: int = 17
ibkr_account: str | None = None
ibkr_readonly: bool = True
ibkr_connect_timeout_seconds: float = 8.0
ibkr_order_timeout_seconds: float = 20.0
ibkr_max_order_value_usd: float = 1500.0
ibkr_allow_market_orders: bool = False
ibkr_allowed_symbols: str = ""

@property
def ibkr_allowed_symbol_set(self) -> set[str]:
    return {s.strip().upper() for s in self.ibkr_allowed_symbols.split(",") if s.strip()}

scheduler_enabled: bool = True
scheduler_scan_minutes: int = 15
scheduler_report_hour_utc: int = 22

watchdog_enabled: bool = True
watchdog_interval_minutes: int = 5
watchdog_stale_discovery_minutes: int = 30
watchdog_close_stale_discovery_runs: bool = True

self_improvement_max_trials: int = 80
self_improvement_max_pbo: float = 0.10
self_improvement_min_pbo_blocks: int = 16
self_improvement_plateau_pf_fraction: float = 0.80
# Wave4-D nested optimization (fail-closed: disabling it blocks acceptance).
self_improvement_nested_enabled: bool = True
self_improvement_nested_outer_folds: int = 5
self_improvement_nested_inner_trials: int = 64
self_improvement_nested_max_pbo: float = 0.10
self_improvement_nested_seed: int = 17
self_improvement_nested_use_optuna: bool = True

@field_validator("market_data_provider")
@classmethod
def only_ibkr_market_data(cls, value: str) -> str:
    if value.lower() != "ibkr":
        raise ValueError("Tradeo only permits IBKR market data; non-IBKR providers are disabled")
    return "ibkr"

@field_validator("allow_synthetic_market_data")
@classmethod
def synthetic_market_data_disabled(cls, value: bool) -> bool:
    if value:
        raise ValueError("Synthetic market data is forbidden")
    return False

@field_validator("risk_per_trade_pct", "daily_loss_limit_pct", "monthly_loss_limit_pct")
@classmethod
def pct_bounds(cls, value: float) -> float:
    if value <= 0 or value > 0.5:
        raise ValueError("risk percentages must be between 0 and 0.5")
    return value

@property
def cors_origins_list(self) -> list[str]:
    return [origin.strip() for origin in self.cors_origins.split(",") if origin.strip()]

@property
def discovery_window_size_list(self) -> list[int]:
    return [int(x.strip()) for x in self.discovery_window_sizes.split(",") if x.strip()]

@property
def discovery_forward_bar_list(self) -> list[int]:
    return [int(x.strip()) for x in self.discovery_forwardBars.split(",") if x.strip()]

@property
def discovery_rr_level_list(self) -> list[float]:
    values = sorted({float(x.strip()) for x in self.discovery_rr_levels.split(",") if x.strip()})
    return values or [2.5, 4.0]

@property
def discovery_cost_stress_multiplier_list(self) -> list[float]:
    values = sorted({float(x.strip()) for x in self.discovery_cost_stress_multipliers.split(",") if x.strip()})
    return values or [1.0, 2.0, 3.0]

@property
def reports_path(self) -> Path:
    path = Path(self.reports_dir)
    path.mkdir(parents=True, exist_ok=True)
    return path

@property
def artifacts_path(self) -> Path:
    path = Path(self.artifacts_dir)
    path.mkdir(parents=True, exist_ok=True)
    return path

@property
def market_data_cache_path(self) -> Path:
    path = Path(self.market_data_cache_dir)

```

```

        path.mkdir(parents=True, exist_ok=True)
        return path

@property
def universe_snapshot_path(self) -> Path:
    path = Path(self.universe_snapshot_dir)
    path.mkdir(parents=True, exist_ok=True)
    return path

@property
def account_risk_usd(self) -> float:
    return round(self.initial_capital_usd * self.risk_per_trade_pct, 2)

@property
def live_armed(self) -> bool:
    return (
        self.trading_mode == "live"
        and self.live_trading_enabled
        and self.live_trading_confirmation_value == self.live_trading_confirmation_phrase
        and not self.kill_switch_enabled
    )

@lru_cache
def get_settings() -> Settings:
    return Settings()

```

Full Source: `backend/tradeo/core/security.py`

`backend/tradeo/core/security.py` ? 35 lines, 1255 bytes, sha256 prefix `95bbce787744f2c7`

Public surface summary before full source:

? function `require\_admin` line 12`

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import hmac
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials

from tradeo.core.config import Settings, get_settings

security = HTTPBasic(auto_error=False)

def require_admin(
    credentials: HTTPBasicCredentials | None = Depends(security),
    settings: Settings = Depends(get_settings),
) -> str:
    """Simple local/VPN HTTP Basic auth.

    For a public deployment, replace this with OIDC or mTLS. The intended deployment is
    private local/VPN, so this is intentionally small and auditable.
    """
    if credentials is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Authentication required",
            headers={"WWW-Authenticate": "Basic"},
        )
    username_ok = hmac.compare_digest(credentials.username, settings.admin_username)
    password_ok = hmac.compare_digest(credentials.password, settings.admin_password)
    if not (username_ok and password_ok):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials",
            headers={"WWW-Authenticate": "Basic"},
        )
    return credentials.username

```

Full Source: `backend/tradeo/db/models.py`

`backend/tradeo/db/models.py` ? 337 lines, 17646 bytes, sha256 prefix `247fd549a7457c30`

Audit relevance: SQLAlchemy persistence schema. Audit state transitions, JSON evidence fields, enum-like strings, and migration risk because there is no Alembic migration tree in this snapshot.

Public surface summary before full source:

? class `SignalStatus` line 24 methods: ?
 ? class `TradeStatus` line 34 methods: ?
 ? class `Signal` line 40 methods: ?
 ? class `Trade` line 67 methods: ?
 ? class `EquityPoint` line 93 methods: ?
 ? class `PatternMetric` line 103 methods: ?

? class `StrategyVersion` line 118 methods: ?
 ? class `AuditLog` line 131 methods: ?
 ? class `SystemControl` line 143 methods: ?
 ? class `RiskLedger` line 163 methods: ?
 ? class `DiscoveredPatternStatus` line 175 methods: ?
 ? class `DiscoveryRun` line 189 methods: ?
 ? function `utcnow` line 15`
 ? function `json\_type` line 19`

Risk hints: wall-clock timestamp usage; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from datetime import datetime, timezone
from enum import Enum
from typing import Any

from sqlalchemy import Boolean, DateTime, Enum as SAEnum, Float, ForeignKey, Integer, String, Text
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy.types import JSON

from tradeo.db.session import Base

def utcnow() -> datetime:
    return datetime.now(timezone.utc)

def json_type() -> Any:
    # PostgreSQL uses JSONB; SQLite/testing can use generic JSON.
    return JSONB().with_variant(JSON(), "sqlite")

class SignalStatus(str, Enum):
    WATCHLIST = "watchlist"
    REJECTED = "rejected"
    PAPER_APPROVED = "paper_approved"
    PENDING_HUMAN_APPROVAL = "pending_human_approval"
    LIVE_APPROVED = "live_approved"
    EXECUTED = "executed"
    EXPIRED = "expired"

class TradeStatus(str, Enum):
    OPEN = "open"
    CLOSED = "closed"
    CANCELLED = "cancelled"

class Signal(Base):
    __tablename__ = "signals"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    symbol: Mapped[str] = mapped_column(String(24), index=True)
    pattern: Mapped[str] = mapped_column(String(80), index=True)
    side: Mapped[str] = mapped_column(String(8))
    timeframe: Mapped[str] = mapped_column(String(16), default="1d")
    entry: Mapped[float] = mapped_column(Float)
    stop: Mapped[float] = mapped_column(Float)
    target: Mapped[float] = mapped_column(Float)
    reward_risk: Mapped[float] = mapped_column(Float)
    confidence: Mapped[float] = mapped_column(Float)
    composite_score: Mapped[float] = mapped_column(Float)
    risk_usd: Mapped[float] = mapped_column(Float, default=0.0)
    suggested_qty: Mapped[int] = mapped_column(Integer, default=0)
    strategy_version: Mapped[str] = mapped_column(String(80), default="cup_v0")
    status: Mapped[SignalStatus] = mapped_column(SAEnum(SignalStatus), default=SignalStatus.WATCHLIST)
    supervisor_notes: Mapped[str] = mapped_column(Text, default="")
    human_approved: Mapped[bool] = mapped_column(Boolean, default=False)
    expires_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    metadata_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)

    trades: Mapped[list["Trade"]] = relationship(back_populates="signal")

class Trade(Base):
    __tablename__ = "trades"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    signal_id: Mapped[int | None] = mapped_column(ForeignKey("signals.id"), nullable=True)
    symbol: Mapped[str] = mapped_column(String(24), index=True)
    pattern: Mapped[str] = mapped_column(String(80), index=True)
    side: Mapped[str] = mapped_column(String(8))
    qty: Mapped[int] = mapped_column(Integer)
    entry: Mapped[float] = mapped_column(Float)
    stop: Mapped[float] = mapped_column(Float)
    target: Mapped[float] = mapped_column(Float)
    status: Mapped[TradeStatus] = mapped_column(SAEnum(TradeStatus), default=TradeStatus.OPEN)
    opened_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    closed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)

```



```

exit_price: Mapped[float | None] = mapped_column(Float, nullable=True)
pnl_usd: Mapped[float] = mapped_column(Float, default=0.0)
r_multiple: Mapped[float] = mapped_column(Float, default=0.0)
broker_order_id: Mapped[str | None] = mapped_column(String(120), nullable=True)
evidence_type: Mapped[str | None] = mapped_column(String(40), nullable=True, index=True)
evidence_quality: Mapped[str | None] = mapped_column(String(40), nullable=True, index=True)
metadata_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)

signal: Mapped[Signal | None] = relationship(back_populates="trades")

class EquityPoint(Base):
    __tablename__ = "equity_points"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    timestamp: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, index=True)
    equity: Mapped[float] = mapped_column(Float)
    cash: Mapped[float] = mapped_column(Float)
    open_risk: Mapped[float] = mapped_column(Float, default=0.0)

class PatternMetric(Base):
    __tablename__ = "pattern_metrics"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    pattern: Mapped[str] = mapped_column(String(80), index=True)
    strategy_version: Mapped[str] = mapped_column(String(80), default="cup_v0")
    total_trades: Mapped[int] = mapped_column(Integer, default=0)
    win_rate: Mapped[float] = mapped_column(Float, default=0.0)
    profit_factor: Mapped[float] = mapped_column(Float, default=0.0)
    expectancy_r: Mapped[float] = mapped_column(Float, default=0.0)
    max_drawdown_pct: Mapped[float] = mapped_column(Float, default=0.0)
    avg_r_multiple: Mapped[float] = mapped_column(Float, default=0.0)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

class StrategyVersion(Base):
    __tablename__ = "strategy_versions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(120), unique=True, index=True)
    state: Mapped[str] = mapped_column(String(40), default="lab")
    parent_name: Mapped[str | None] = mapped_column(String(120), nullable=True)
    params_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
    metrics_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    promoted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)

class AuditLog(Base):
    __tablename__ = "audit_logs"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    timestamp: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, index=True)
    actor: Mapped[str] = mapped_column(String(80))
    action: Mapped[str] = mapped_column(String(120))
    entity_type: Mapped[str] = mapped_column(String(80), default="system")
    entity_id: Mapped[str] = mapped_column(String(80), default="")
    details_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)

class SystemControl(Base):
    """Runtime operational controls that must survive process restarts.

    The env-based kill switch (TRADEO_KILL_SWITCH_ENABLED) is immutable at
    runtime because Settings are cached at startup. Automatic safety triggers
    (broker reconciliation divergence) persist their state here so every order
    path can honour it immediately, without editing .env and restarting.
    """

    __tablename__ = "system_controls"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    key: Mapped[str] = mapped_column(String(80), unique=True, index=True)
    enabled: Mapped[bool] = mapped_column(Boolean, default=False)
    reason: Mapped[str] = mapped_column(Text, default="")
    actor: Mapped[str] = mapped_column(String(80), default="")
    details_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

class RiskLedger(Base):
    __tablename__ = "risk_ledger"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    day: Mapped[str] = mapped_column(String(12), index=True)
    realized_pnl: Mapped[float] = mapped_column(Float, default=0.0)
    open_risk: Mapped[float] = mapped_column(Float, default=0.0)
    halted: Mapped[bool] = mapped_column(Boolean, default=False)
    notes: Mapped[str] = mapped_column(Text, default="")
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

class DiscoveredPatternStatus(str, Enum):
    LAB = "lab"
    LAB_WATCHLIST = "lab_watchlist"

```

```

LAB_CANDIDATE = "lab_candidate"
NEEDS_CONFIRMATION = "needs_confirmation"
CONFIRMED_CANDIDATE = "confirmed_candidate"
FAILED_CONFIRMATION = "failed_confirmation"
DIRECTOR_REVIEW = "director_review"
PREMIUM_CANDIDATE = "premium_candidate"
PAPER_CANDIDATE = "paper_candidate"
PRODUCTION = "production"
REJECTED = "rejected"

```

```
class DiscoveryRun(Base):
```

```

__tablename__ = "discovery_runs"

id: Mapped[int] = mapped_column(Integer, primary_key=True)
started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, index=True)
finished_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
status: Mapped[str] = mapped_column(String(40), default="running", index=True)
symbols_scanned: Mapped[int] = mapped_column(Integer, default=0)
windows_sampled: Mapped[int] = mapped_column(Integer, default=0)
clusters_evaluated: Mapped[int] = mapped_column(Integer, default=0)
accepted_patterns: Mapped[int] = mapped_column(Integer, default=0)
rejected_patterns: Mapped[int] = mapped_column(Integer, default=0)
duration_seconds: Mapped[float] = mapped_column(Float, default=0.0)
params_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
summary_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
report_path: Mapped[str | None] = mapped_column(String(500), nullable=True)

patterns: Mapped[list["DiscoveredPattern"]] = relationship(back_populates="run")

```

```
class DiscoveredPattern(Base):
```

```

__tablename__ = "discovered_patterns"

id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
run_id: Mapped[int | None] = mapped_column(ForeignKey("discovery_runs.id"), nullable=True, index=True)
pattern_key: Mapped[str] = mapped_column(String(160), unique=True, index=True)
name: Mapped[str] = mapped_column(String(160), index=True)
status: Mapped[DiscoveredPatternStatus] = mapped_column(
    SAEnum(DiscoveredPatternStatus), default=DiscoveredPatternStatus.LAB, index=True
)
side: Mapped[str] = mapped_column(String(8), default="long")
timeframe: Mapped[str] = mapped_column(String(16), default="1d")
window_size: Mapped[int] = mapped_column(Integer, default=50)
cluster_id: Mapped[int] = mapped_column(Integer, default=0)
pattern_family_key: Mapped[str] = mapped_column(String(160), default="", index=True)
canonical_pattern_key: Mapped[str] = mapped_column(String(160), default="", index=True)
variant_key: Mapped[str] = mapped_column(String(160), default="")
variant_count: Mapped[int] = mapped_column(Integer, default=1)
drift_status: Mapped[str] = mapped_column(String(40), default="stable", index=True)
drift_score: Mapped[float] = mapped_column(Float, default=0.0)
sample_count: Mapped[int] = mapped_column(Integer, default=0)
symbol_count: Mapped[int] = mapped_column(Integer, default=0)
year_count: Mapped[int] = mapped_column(Integer, default=0)
score: Mapped[float] = mapped_column(Float, default=0.0)
reward_risk_estimate: Mapped[float] = mapped_column(Float, default=0.0)
expectancy_r: Mapped[float] = mapped_column(Float, default=0.0)
profit_factor: Mapped[float] = mapped_column(Float, default=0.0)
win_rate: Mapped[float] = mapped_column(Float, default=0.0)
avg_mfe_r: Mapped[float] = mapped_column(Float, default=0.0)
avg_mae_r: Mapped[float] = mapped_column(Float, default=0.0)
stability_score: Mapped[float] = mapped_column(Float, default=0.0)
out_of_sample_expectancy_r: Mapped[float] = mapped_column(Float, default=0.0)
out_of_sample_profit_factor: Mapped[float] = mapped_column(Float, default=0.0)
best_rr: Mapped[float] = mapped_column(Float, default=0.0)
best_tested_rr: Mapped[float] = mapped_column(Float, default=0.0)
best_expectancy_r: Mapped[float] = mapped_column(Float, default=0.0)
best_profit_factor: Mapped[float] = mapped_column(Float, default=0.0)
best_win_rate: Mapped[float] = mapped_column(Float, default=0.0)
best_max_drawdown_r: Mapped[float] = mapped_column(Float, default=0.0)
preferred_rr_passed: Mapped[bool] = mapped_column(Boolean, default=False)
premium_rr_passed: Mapped[bool] = mapped_column(Boolean, default=False)
promotion_status: Mapped[str] = mapped_column(String(40), default="rejected", index=True)
promotion_reason: Mapped[str] = mapped_column(Text, default="")
confirmation_status: Mapped[str] = mapped_column(String(40), default="", index=True)
confirmation_priority_score: Mapped[float] = mapped_column(Float, default=0.0)
confirmation_reason: Mapped[str] = mapped_column(Text, default="")
confirmation_next_action: Mapped[str] = mapped_column(Text, default="")
confirmation_attempts: Mapped[int] = mapped_column(Integer, default=0)
rr_metrics_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
rejection_reasons_json: Mapped[list[str]] = mapped_column(json_type(), default=list)
in_sample_expectancy_r: Mapped[float] = mapped_column(Float, default=0.0)
in_sample_profit_factor: Mapped[float] = mapped_column(Float, default=0.0)
out_of_sample_win_rate: Mapped[float] = mapped_column(Float, default=0.0)
out_of_sample_max_drawdown_r: Mapped[float] = mapped_column(Float, default=0.0)
validation_passed: Mapped[bool] = mapped_column(Boolean, default=False)
validation_reasons_json: Mapped[list[str]] = mapped_column(json_type(), default=list)
centroid_json: Mapped[list[float]] = mapped_column(json_type(), default=list)
metrics_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
feature_summary_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, index=True)
updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

run: Mapped[DiscoveryRun | None] = relationship(back_populates="patterns")
examples: Mapped[list["DiscoveredPatternExample"]] = relationship(
    back_populates="pattern", cascade="all, delete-orphan"
)

```

```

    )
    metric_snapshots: Mapped[list["DiscoveredPatternMetric"]] = relationship(
        back_populates="pattern", cascade="all, delete-orphan"
    )

class DiscoveredPatternExample(Base):
    __tablename__ = "discovered_pattern_examples"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    pattern_id: Mapped[int] = mapped_column(ForeignKey("discovered_patterns.id"), index=True)
    symbol: Mapped[str] = mapped_column(String(24), index=True)
    timeframe: Mapped[str] = mapped_column(String(16), default="1d")
    window_start: Mapped[str] = mapped_column(String(40), default="")
    window_end: Mapped[str] = mapped_column(String(40), default="")
    forward_end: Mapped[str] = mapped_column(String(40), default="")
    entry_price: Mapped[float] = mapped_column(Float, default=0.0)
    risk_proxy: Mapped[float] = mapped_column(Float, default=0.0)
    outcome_r: Mapped[float] = mapped_column(Float, default=0.0)
    mfe_r: Mapped[float] = mapped_column(Float, default=0.0)
    mae_r: Mapped[float] = mapped_column(Float, default=0.0)
    similarity: Mapped[float] = mapped_column(Float, default=0.0)
    kind: Mapped[str] = mapped_column(String(24), default="typical")
    chart_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
    features_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

    pattern: Mapped[DiscoveredPattern] = relationship(back_populates="examples")

class DiscoveredPatternMetric(Base):
    __tablename__ = "discovered_pattern_metrics"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    pattern_id: Mapped[int] = mapped_column(ForeignKey("discovered_patterns.id"), index=True)
    as_of: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, index=True)
    split: Mapped[str] = mapped_column(String(32), default="full")
    metrics_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)

    pattern: Mapped[DiscoveredPattern] = relationship(back_populates="metric_snapshots")

class DiscoveredPatternMatch(Base):
    __tablename__ = "discovered_pattern_matches"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    pattern_id: Mapped[int] = mapped_column(ForeignKey("discovered_patterns.id"), index=True)
    symbol: Mapped[str] = mapped_column(String(24), index=True)
    timeframe: Mapped[str] = mapped_column(String(16), default="1d")
    side: Mapped[str] = mapped_column(String(8), default="long")
    similarity: Mapped[float] = mapped_column(Float, default=0.0)
    score: Mapped[float] = mapped_column(Float, default=0.0)
    entry_price: Mapped[float] = mapped_column(Float, default=0.0)
    stop_price: Mapped[float] = mapped_column(Float, default=0.0)
    target_price: Mapped[float] = mapped_column(Float, default=0.0)
    reward_risk: Mapped[float] = mapped_column(Float, default=4.0)
    matched_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, index=True)
    window_end: Mapped[str] = mapped_column(String(40), default="")
    status: Mapped[str] = mapped_column(String(40), default="lab_watchlist", index=True)
    notes: Mapped[str] = mapped_column(Text, default="")
    chart_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)
    metrics_json: Mapped[dict[str, Any]] = mapped_column(json_type(), default=dict)

    pattern: Mapped[DiscoveredPattern] = relationship()

```

Full Source: `backend/tradeo/db/session.py`

`backend/tradeo/db/session.py` ? 24 lines, 560 bytes, sha256 prefix `b80d7ccc99749c49`

Public surface summary before full source:

? class `Base` line 15 methods: ?

? function `get\_db` line 19`

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from tradeo.core.config import get_settings

settings = get_settings()
engine = create_engine(settings.database_url, pool_pre_ping=True, future=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)

class Base(DeclarativeBase):
    pass

```

```
def get_db() -> Generator:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Full Source: `backend/tradeo/db/init\_db.py`

`backend/tradeo/db/init\_db.py` ? 136 lines, 5932 bytes, sha256 prefix `d94371f92298fdef`

Public surface summary before full source:

```
? function `init_db` line 12`
? function `seed_db` line 129`
```

Risk hints: JSON metadata contract; schema drift risk.

```
from __future__ import annotations

from sqlalchemy import inspect, text
from sqlalchemy.orm import Session

from tradeo.core.config import get_settings
from tradeo.db.models import EquityPoint, StrategyVersion
from tradeo.db.session import Base, engine
from tradeo.services.strategy_config import load_strategy_config

def init_db() -> None:
    Base.metadata.create_all(bind=engine)
    _migrate_trades_evidence_columns()
    _migrate_discovered_patterns()

def _migrate_trades_evidence_columns() -> None:
    inspector = inspect(engine)
    if "trades" not in inspector.get_table_names():
        return
    existing = {col["name"] for col in inspector.get_columns("trades")}
    columns = {
        "evidence_type": "VARCHAR(40)",
        "evidence_quality": "VARCHAR(40)",
    }
    with engine.begin() as conn:
        for name, ddl in columns.items():
            if name not in existing:
                conn.execute(text(f"ALTER TABLE trades ADD COLUMN {name} {ddl}"))
    if engine.dialect.name == "postgresql":
        conn.execute(
            text(
                "UPDATE trades "
                "SET evidence_type = NULLIF(metadata_json ->> 'evidence_type', '') "
                "WHERE evidence_type IS NULL "
                "AND NULLIF(metadata_json ->> 'evidence_type', '') IS NOT NULL"
            )
        )
        conn.execute(
            text(
                "UPDATE trades "
                "SET evidence_quality = NULLIF(metadata_json ->> 'evidence_quality', '') "
                "WHERE evidence_quality IS NULL "
                "AND NULLIF(metadata_json ->> 'evidence_quality', '') IS NOT NULL"
            )
        )
    else:
        conn.execute(
            text(
                "UPDATE trades "
                "SET evidence_type = NULLIF(json_extract(metadata_json, '$.evidence_type'), '') "
                "WHERE evidence_type IS NULL "
                "AND NULLIF(json_extract(metadata_json, '$.evidence_type'), '') IS NOT NULL"
            )
        )
        conn.execute(
            text(
                "UPDATE trades "
                "SET evidence_quality = NULLIF(json_extract(metadata_json, '$.evidence_quality'), '') "
                "WHERE evidence_quality IS NULL "
                "AND NULLIF(json_extract(metadata_json, '$.evidence_quality'), '') IS NOT NULL"
            )
        )

def _migrate_discovered_patterns() -> None:
    inspector = inspect(engine)
    if "discovered_patterns" not in inspector.get_table_names():
        return
    if engine.dialect.name == "postgresql":
        enum_values = [
            "LAB_WATCHLIST",
```

```

        "LAB_CANDIDATE",
        "NEEDS_CONFIRMATION",
        "CONFIRMED_CANDIDATE",
        "FAILED_CONFIRMATION",
        "DIRECTOR_REVIEW",
        "PREMIUM_CANDIDATE",
        "PAPER_CANDIDATE",
        "PRODUCTION",
    ]
    with engine.begin() as conn:
        for value in enum_values:
            conn.execute(
                text(
                    "DO $$ BEGIN "
                    f"ALTER TYPE discoveredpatternstatus ADD VALUE IF NOT EXISTS '{value}'; "
                    "EXCEPTION WHEN undefined_object THEN NULL; END $$;"
                )
            )
    existing = {col["name"] for col in inspector.get_columns("discovered_patterns")}
    json_type_name = "JSONB" if engine.dialect.name == "postgresql" else "JSON"
    columns = {
        "best_rr": "DOUBLE PRECISION DEFAULT 0",
        "best_tested_rr": "DOUBLE PRECISION DEFAULT 0",
        "best_expectancy_r": "DOUBLE PRECISION DEFAULT 0",
        "best_profit_factor": "DOUBLE PRECISION DEFAULT 0",
        "best_win_rate": "DOUBLE PRECISION DEFAULT 0",
        "best_max_drawdown_r": "DOUBLE PRECISION DEFAULT 0",
        "pattern_family_key": "VARCHAR(160) DEFAULT ''",
        "canonical_pattern_key": "VARCHAR(160) DEFAULT ''",
        "variant_key": "VARCHAR(160) DEFAULT ''",
        "variant_count": "INTEGER DEFAULT 1",
        "drift_status": "VARCHAR(40) DEFAULT 'stable'",
        "drift_score": "DOUBLE PRECISION DEFAULT 0",
        "preferred_rr_passed": "BOOLEAN DEFAULT FALSE",
        "premium_rr_passed": "BOOLEAN DEFAULT FALSE",
        "promotion_status": "VARCHAR(40) DEFAULT 'rejected'",
        "promotion_reason": "TEXT DEFAULT ''",
        "confirmation_status": "VARCHAR(40) DEFAULT ''",
        "confirmation_priority_score": "DOUBLE PRECISION DEFAULT 0",
        "confirmation_reason": "TEXT DEFAULT ''",
        "confirmation_next_action": "TEXT DEFAULT ''",
        "confirmation_attempts": "INTEGER DEFAULT 0",
        "rr_metrics_json": f"{json_type_name} DEFAULT " + ("{}" if engine.dialect.name == "postgresql" else "{}"),
        "rejection_reasons_json": f"{json_type_name} DEFAULT " + ("[]" if engine.dialect.name == "postgresql" else "[]"),
        "in_sample_expectancy_r": "DOUBLE PRECISION DEFAULT 0",
        "in_sample_profit_factor": "DOUBLE PRECISION DEFAULT 0",
        "out_of_sample_win_rate": "DOUBLE PRECISION DEFAULT 0",
        "out_of_sample_max_drawdown_r": "DOUBLE PRECISION DEFAULT 0",
    }
    with engine.begin() as conn:
        for name, ddl in columns.items():
            if name not in existing:
                conn.execute(text(f"ALTER TABLE discovered_patterns ADD COLUMN {name} {ddl}"))

def seed_db(db: Session) -> None:
    settings = get_settings()
    if db.query(EquityPoint).count() == 0:
        db.add(EquityPoint(equity=settings.initial_capital_usd, cash=settings.initial_capital_usd))
    if db.query(StrategyVersion).filter(StrategyVersion.name == "cup_v0").count() == 0:
        params = load_strategy_config(settings.strategy_config_file)
        db.add(StrategyVersion(name="cup_v0", state="paper", params_json=params))
    db.commit()

```

Full Source: `backend/tradeo/schemas.py`

`backend/tradeo/schemas.py` ? 465 lines, 12300 bytes, sha256 prefix `47aa545655ab948f`

Audit relevance: Pydantic API schemas. Audit request/response boundaries and whether backend fields used by services are surfaced consistently.

Public surface summary before full source:

```

? class `PatternFeatures` line 9 methods: ?
? class `PatternCandidate` line 22 methods: ?
? class `RiskDecision` line 40 methods: ?
? class `SupervisorDecision` line 50 methods: ?
? class `SignalOut` line 60 methods: ?
? class `TradeOut` line 84 methods: ?
? class `EquityPointOut` line 103 methods: ?
? class `PatternMetricOut` line 112 methods: ?
? class `DashboardSummary` line 125 methods: ?
? class `ScanRequest` line 139 methods: ?
? class `ScanResponse` line 146 methods: ?
? class `BacktestRequest` line 156 methods: ?

```

Risk hints: JSON metadata contract; schema drift risk.

from __future__ import annotations

```

from datetime import datetime
from typing import Any, Literal

from pydantic import BaseModel, Field

class PatternFeatures(BaseModel):
    cup_depth: float = 0.0
    cup_length: int = 0
    rim_symmetry: float = 0.0
    bottom_smoothness: float = 0.0
    handle_depth: float = 0.0
    volume_dryup: float = 0.0
    breakout_volume_ratio: float = 0.0
    atr_pct: float = 0.0
    trend_score: float = 0.0
    avg_dollar_volume: float = 0.0

class PatternCandidate(BaseModel):
    symbol: str
    pattern: str = "mid_small_cap_cup_breakout"
    side: Literal["long", "short"] = "long"
    timeframe: str = "1d"
    entry: float
    stop: float
    target: float
    reward_risk: float
    confidence: float
    rule_score: float
    ml_score: float
    vision_score: float
    composite_score: float
    features: dict[str, Any] = Field(default_factory=dict)
    notes: list[str] = Field(default_factory=list)

class RiskDecision(BaseModel):
    approved: bool
    suggested_qty: int = 0
    risk_usd: float = 0.0
    notional_usd: float = 0.0
    reason: str = ""
    hard_rejections: list[str] = Field(default_factory=list)
    warnings: list[str] = Field(default_factory=list)

class SupervisorDecision(BaseModel):
    approved_for_paper: bool
    eligible_for_live_review: bool
    status: str
    confidence: float
    notes: str
    risk: RiskDecision
    candidate: PatternCandidate

class SignalOut(BaseModel):
    id: int
    symbol: str
    pattern: str
    side: str
    timeframe: str
    entry: float
    stop: float
    target: float
    reward_risk: float
    confidence: float
    composite_score: float
    risk_usd: float
    suggested_qty: int
    strategy_version: str
    status: str
    supervisor_notes: str
    human_approved: bool
    created_at: datetime
    metadata_json: dict[str, Any]

    model_config = {"from_attributes": True}

class TradeOut(BaseModel):
    id: int
    symbol: str
    pattern: str
    side: str
    qty: int
    entry: float
    stop: float
    target: float
    status: str
    opened_at: datetime
    closed_at: datetime | None
    exit_price: float | None
    pnl_usd: float

```

```

    r_multiple: float

    model_config = {"from_attributes": True}

class EquityPointOut(BaseModel):
    timestamp: datetime
    equity: float
    cash: float
    open_risk: float

    model_config = {"from_attributes": True}

class PatternMetricOut(BaseModel):
    pattern: str
    strategy_version: str
    total_trades: int
    win_rate: float
    profit_factor: float
    expectancy_r: float
    max_drawdown_pct: float
    avg_r_multiple: float

    model_config = {"from_attributes": True}

class DashboardSummary(BaseModel):
    mode: str
    live_armed: bool
    kill_switch_enabled: bool
    initial_capital_usd: float
    risk_per_trade_usd: float
    min_reward_risk: float
    equity: list[EquityPointOut]
    pattern_metrics: list[PatternMetricOut]
    recent_signals: list[SignalOut]
    open_trades: list[TradeOut]
    supervisor_state: dict[str, Any]

class ScanRequest(BaseModel):
    limit: int | None = None
    period: str | None = None
    interval: str | None = None
    force_symbols: list[str] | None = None

class ScanResponse(BaseModel):
    scanned: int
    candidates: int
    stored_signals: int
    rejected: int
    data_errors: int = 0
    data_quality_skips: int = 0
    decisions: list[SupervisorDecision]

class BacktestRequest(BaseModel):
    symbols: list[str] | None = None
    period: str = "3y"
    interval: str = "1d"
    max_symbols: int = 30

class BacktestMetrics(BaseModel):
    total_trades: int
    win_rate: float
    profit_factor: float
    expectancy_r: float
    avg_r_multiple: float
    max_drawdown_pct: float
    total_signals: int = 0
    skipped_signals: int = 0
    skip_rate: float = 0.0
    trades: list[dict[str, Any]] = Field(default_factory=list)

class KillSwitchRequest(BaseModel):
    enabled: bool
    reason: str = "manual"

class SelfImprovementResponse(BaseModel):
    generated: int
    accepted_lab_candidates: int
    best_candidate: dict[str, Any] | None = None
    report_path: str | None = None

class DiscoveryRunRequest(BaseModel):
    limit: int | None = None
    symbols: list[str] | None = None
    period: str | None = None
    interval: str | None = None
    window_sizes: list[int] | None = None

```

```

forwardBars: list[int] | None = None
stride: int | None = None
max_total_windows: int | None = None
max_windows_per_symbol: int | None = None
min_cluster_size: int | None = None
max_clusters_per_window: int | None = None
store_rejected: bool | None = None

```

```

class DiscoveryRunResponse(BaseModel):
    run_id: int
    status: str
    symbols_scanned: int
    windows_sampled: int
    clusters_evaluated: int
    accepted_patterns: int
    rejected_patterns: int
    stored_patterns: int
    duration_seconds: float
    report_path: str | None = None
    top_patterns: list[dict[str, Any]] = Field(default_factory=list)
    warnings: list[str] = Field(default_factory=list)

```

```

class DiscoveredPatternOut(BaseModel):
    id: int
    pattern_key: str
    name: str
    status: str
    side: str
    timeframe: str
    window_size: int
    cluster_id: int
    pattern_family_key: str = ""
    canonical_pattern_key: str = ""
    variant_key: str = ""
    variant_count: int = 1
    drift_status: str = "stable"
    drift_score: float = 0.0
    sample_count: int
    symbol_count: int
    year_count: int
    score: float
    reward_risk_estimate: float
    expectancy_r: float
    profit_factor: float
    win_rate: float
    avg_mfe_r: float
    avg_mae_r: float
    stability_score: float
    out_of_sample_expectancy_r: float
    out_of_sample_profit_factor: float
    best_rr: float = 0.0
    best_tested_rr: float = 0.0
    best_expectancy_r: float = 0.0
    best_profit_factor: float = 0.0
    best_win_rate: float = 0.0
    best_max_drawdown_r: float = 0.0
    preferred_rr_passed: bool = False
    premium_rr_passed: bool = False
    promotion_status: str = "rejected"
    promotion_reason: str = ""
    confirmation_status: str = ""
    confirmation_priority_score: float = 0.0
    confirmation_reason: str = ""
    confirmation_next_action: str = ""
    confirmation_attempts: int = 0
    rr_metrics_json: dict[str, Any] = Field(default_factory=dict)
    rejection_reasons_json: list[str] = Field(default_factory=list)
    in_sample_expectancy_r: float = 0.0
    in_sample_profit_factor: float = 0.0
    out_of_sample_win_rate: float = 0.0
    out_of_sample_max_drawdown_r: float = 0.0
    validation_passed: bool
    validation_reasons_json: list[str]
    metrics_json: dict[str, Any]
    feature_summary_json: dict[str, Any]
    created_at: datetime

    model_config = {"from_attributes": True}

```

```

class DiscoveredPatternExampleOut(BaseModel):
    id: int
    pattern_id: int
    symbol: str
    timeframe: str
    window_start: str
    window_end: str
    forward_end: str
    entry_price: float
    risk_proxy: float
    outcome_r: float
    mfe_r: float
    mae_r: float
    similarity: float

```



```

kind: str
chart_json: dict[str, Any]
features_json: dict[str, Any]

model_config = {"from_attributes": True}

class DiscoveredPatternMetricOut(BaseModel):
    id: int
    pattern_id: int
    as_of: datetime
    split: str
    metrics_json: dict[str, Any]

    model_config = {"from_attributes": True}

class DiscoveredPatternDetailOut(DiscoveredPatternOut):
    examples: list[DiscoveredPatternExampleOut] = Field(default_factory=list)
    metric_snapshots: list[DiscoveredPatternMetricOut] = Field(default_factory=list)

class DiscoveryRunOut(BaseModel):
    id: int
    started_at: datetime
    finished_at: datetime | None
    status: str
    symbols_scanned: int
    windows_sampled: int
    clusters_evaluated: int
    accepted_patterns: int
    rejected_patterns: int
    duration_seconds: float
    params_json: dict[str, Any]
    summary_json: dict[str, Any]
    report_path: str | None

    model_config = {"from_attributes": True}

class NovelPatternMatchRequest(BaseModel):
    symbols: list[str] | None = None
    limit: int | None = None
    max_patterns: int | None = None
    similarity_threshold: float | None = None
    module: Literal["laboratory", "fox_hunter"] = "laboratory"
    store: bool = True

class NovelPatternMatchOut(BaseModel):
    id: int
    pattern_id: int
    symbol: str
    timeframe: str
    side: str
    similarity: float
    score: float
    entry_price: float
    stop_price: float
    target_price: float
    reward_risk: float
    matched_at: datetime
    window_end: str
    status: str
    notes: str
    chart_json: dict[str, Any]
    metrics_json: dict[str, Any]

    model_config = {"from_attributes": True}

class NovelPatternMatchResponse(BaseModel):
    patterns_checked: int
    symbols_checked: int
    matches: list[dict[str, Any]] = Field(default_factory=list)
    stored_matches: int
    module: str = "laboratory"
    similarity_threshold: float
    generated_at: str

class ResearchDirectorResponse(BaseModel):
    generated_at: str
    run_id: int | None = None
    patterns_reviewed: int
    hypotheses_created: int
    memory_graph: dict[str, Any]
    active_learning_agenda: list[dict[str, Any]] = Field(default_factory=list)
    director_state: dict[str, Any]
    artifacts: dict[str, str] = Field(default_factory=dict)

class PatternEntryScanRequest(BaseModel):
    symbols: list[str] | None = None
    limit: int | None = None
    max_patterns: int | None = None

```

```

similarity_threshold: float | None = None
store_signals: bool | None = None
execute_orders: bool | None = None

class PatternEntryScanResponse(BaseModel):
    module: str
    patterns_checked: int
    symbols_checked: int
    matches_found: int
    entry_variants_considered: int = 0
    signals_created: int
    orders_submitted: int
    skipped_duplicates: int
    skipped_cooldown: int = 0
    rejected_by_entry_gate: int = 0
    rejected_by_entry_quality: int = 0
    rejected_by_risk: int
    order_errors: list[dict[str, Any]] = Field(default_factory=list)
    signal_ids: list[int] = Field(default_factory=list)
    trade_ids: list[int] = Field(default_factory=list)
    paper_observations_opened: int = 0
    paper_observations_closed: int = 0
    paper_observation_trade_ids: list[int] = Field(default_factory=list)
    paper_observation_lifecycle: dict[str, Any] = Field(default_factory=dict)
    top_opportunities: list[dict[str, Any]] = Field(default_factory=list)
    store_signals: bool
    execute_orders: bool
    similarity_threshold: float
    generated_at: str

class LabPaperHistoryOut(BaseModel):
    closed_trades: int = 0
    open_trades: int = 0
    total_r: float = 0.0
    expectancy_r: float = 0.0
    win_rate: float = 0.0
    profit_factor: float = 0.0
    unique_symbols: int = 0
    unique_days: int = 0
    variant_closed_trades: int = 0
    regime_closed_trades: int = 0
    last_trade_status: str | None = None

class LabOpportunityDiagnosticOut(BaseModel):
    source: str
    status: str
    symbol: str
    pattern: str
    pattern_id: int | None = None
    side: str = ""
    timeframe: str = ""
    entry_variant_id: str = ""
    entry_variant_label: str = ""
    regime_key: str = ""
    rank: int | None = None
    rank_score: float | None = None
    score: float | None = None
    entry_score: float | None = None
    similarity: float | None = None
    opportunity_rank_components: dict[str, Any] = Field(default_factory=dict)
    entry_gate: dict[str, Any] = Field(default_factory=dict)
    entry_gate_components: list[dict[str, Any]] = Field(default_factory=list)
    entry_quality: dict[str, Any] = Field(default_factory=dict)
    regime_fit: dict[str, Any] = Field(default_factory=dict)
    rejection_stage: str | None = None
    rejection_reason: str = ""
    created_at: str = ""
    window_end: str = ""
    paper_history: LabPaperHistoryOut = Field(default_factory=LabPaperHistoryOut)
    promotion: dict[str, Any] = Field(default_factory=dict)
    missing_to_promote: list[str] = Field(default_factory=list)

class LabDiagnosticsResponse(BaseModel):
    generated_at: str
    summary: dict[str, Any]
    opportunities: list[LabOpportunityDiagnosticOut] = Field(default_factory=list)

```

Full Source: ``backend/tradeo/services/data_provider.py``

``backend/tradeo/services/data_provider.py`` ? 399 lines, 16817 bytes, sha256 prefix ``bdefd699106a6cdc``

Audit relevance: Market data normalization/cache/incremental refresh layer. Audit PIT behavior, partial-bar exclusion, manifest lineage, data quality, and IBKR adjusted feed assumptions.

Public surface summary before full source:

- ? class ``MarketDataProvider`` line 17 methods: `fetch_ohlc`
- ? class ``CachedMarketDataProvider`` line 67 methods: `fetch_ohlc`, `data_manifest`

```
? function `load_universe` line 44`
? function `pick_symbols` line 57`
? function `detect_unadjusted_splits` line 376`
```

Risk hints: wall-clock timestamp usage; IBKR/broker/data dependency.

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime
import hashlib
import json
import os
from pathlib import Path
from typing import Any, Protocol

import pandas as pd

from tradeo.core.config import get_settings
from tradeo.services.technical_indicators import normalize_ohlc

class MarketDataProvider(Protocol):
    def fetch_ohlc(self, symbol: str, period: str = "2y", interval: str = "1d") -> pd.DataFrame:
        ...

_DAILY_INTERVALS = {"1d", "1day", "1 day"}
# Bar widths for the intraday intervals the IBKR provider understands; used
# both for completeness masking and for bar-relative refresh gap thresholds.
_INTRADAY_BAR_MINUTES = {
    "1m": 1,
    "1min": 1,
    "1 min": 1,
    "5m": 5,
    "5min": 5,
    "5 mins": 5,
    "15m": 15,
    "15min": 15,
    "15 mins": 15,
    "30m": 30,
    "30min": 30,
    "30 mins": 30,
    "1h": 60,
    "60m": 60,
    "1 hour": 60,
}

def load_universe(path: str | None = None) -> pd.DataFrame:
    settings = get_settings()
    p = Path(path or settings.universe_file)
    if not p.exists():
        raise FileNotFoundError(f"Universe file not found: {p}")
    df = pd.read_csv(p)
    if "symbol" not in df.columns:
        raise ValueError("Universe CSV must include a 'symbol' column")
    df["symbol"] = df["symbol"].astype(str).str.upper().str.strip()
    df = df[df["symbol"].str.len() > 0].drop_duplicates("symbol")
    return df

def pick_symbols(limit: int | None = None, force_symbols: list[str] | None = None) -> list[str]:
    if force_symbols:
        return [s.upper().strip() for s in force_symbols if s.strip()]
    settings = get_settings()
    df = load_universe(settings.universe_file)
    n = limit or settings.scan_limit_default
    return df["symbol"].head(n).tolist()

@dataclass
class CachedMarketDataProvider:
    upstream: MarketDataProvider | None = None
    cache_dir: Path | None = None
    adjusted: bool | None = None
    what_to_show: str | None = None
    schema_version: int = 1
    incremental_enabled: bool | None = None
    incremental_overlapBars: int | None = None
    incremental_min_gap_days: int | None = None
    incremental_max_gap_days: int | None = None
    incremental_intraday_enabled: bool | None = None
    incremental_intraday_min_gapBars: int | None = None
    incremental_intraday_max_gap_days: int | None = None

    def __post_init__(self) -> None:
        if self.upstream is None:
            raise ValueError("CachedMarketDataProvider requires an explicit real-data upstream")
        settings = get_settings()
        self.cache_dir = self.cache_dir or settings.market_data_cache_path
        self.adjusted = settings.market_data_adjusted if self.adjusted is None else self.adjusted
        self.what_to_show = self.what_to_show or settings.market_data_what_to_show
        if self.incremental_enabled is None:
```

```

        self.incremental_enabled = settings.market_data_incremental_enabled
    if self.incremental_overlapBars is None:
        self.incremental_overlapBars = settings.market_data_incremental_overlapBars
    if self.incremental_min_gap_days is None:
        self.incremental_min_gap_days = settings.market_data_incremental_min_gap_days
    if self.incremental_max_gap_days is None:
        self.incremental_max_gap_days = settings.market_data_incremental_max_gap_days
    if self.incremental_intraday_enabled is None:
        self.incremental_intraday_enabled = settings.market_data_incremental_intraday_enabled
    if self.incremental_intraday_min_gapBars is None:
        self.incremental_intraday_min_gapBars = (
            settings.market_data_incremental_intraday_min_gapBars
        )
    if self.incremental_intraday_max_gap_days is None:
        self.incremental_intraday_max_gap_days = (
            settings.market_data_incremental_intraday_max_gap_days
        )
    self._cache: dict[tuple[str, str, str], pd.DataFrame] = {}

def fetch_ohlcv(self, symbol: str, period: str = "2y", interval: str = "1d") -> pd.DataFrame:
    key = (symbol.upper(), period, interval)
    if key not in self._cache:
        csv_path = self._cache_path(*key)
        if csv_path.exists():
            df = self._read_csv(csv_path)
            df = self._maybe_refresh(df, key=key, csv_path=csv_path)
        else:
            df = self._full_refetch(key, csv_path, refresh_mode="full_fetch")
        self._cache[key] = self._complete_bars_only(df)
    return self._cache[key].copy()

def _full_refetch(
    self,
    key: tuple[str, str, str],
    csv_path: Path,
    *,
    refresh_mode: str,
) -> pd.DataFrame:
    symbol, period, interval = key
    assert self.upstream is not None
    df = self.upstream.fetch_ohlcv(symbol, period=period, interval=interval)
    df = self._annotate(df, symbol=symbol, period=period, interval=interval)
    df = self._complete_bars_only(df)
    self._atomic_write(csv_path, df, key=key, refresh_mode=refresh_mode)
    return df

def _maybe_refresh(
    self,
    cached: pd.DataFrame,
    *,
    key: tuple[str, str, str],
    csv_path: Path,
) -> pd.DataFrame:
    """Append the missing tail after verifying overlap bars.

    Adjusted feeds rewrite history on splits/dividends, so an incremental
    append is only trusted when the refetched overlap matches the cached
    bars; otherwise the whole artifact is refetched. Daily gaps are
    measured in calendar days; intraday gaps in bar widths (min) and
    wall-clock days (max), since a weekend gap on a 5m cache is routine.
    """
    symbol, period, interval = key
    interval_lower = interval.lower()
    is_daily = interval_lower in _DAILY_INTERVALS
    bar_minutes = _INTRADAY_BAR_MINUTES.get(interval_lower)
    if not self.incremental_enabled or cached.empty:
        return cached
    if not is_daily and (bar_minutes is None or not self.incremental_intraday_enabled):
        return cached
    last_ts = pd.Timestamp(cached.index[-1])
    now = pd.Timestamp(datetime.now(UTC))
    overlapBars = max(1, int(self.incremental_overlapBars or 5))
    if is_daily:
        gap_days = (now.date() - last_ts.date()).days - 1
        if gap_days < int(self.incremental_min_gap_days or 1):
            return cached
        gap_too_large = gap_days > int(self.incremental_max_gap_days or 45)
        tail_days = gap_days + overlapBars * 2 + 4
    else:
        gap = now - self._as_utc(last_ts)
        min_gapBars = max(1, int(self.incremental_intraday_min_gapBars or 1))
        if gap < pd.Timedelta(minutes=bar_minutes * min_gapBars):
            return cached
        gap_too_large = gap > pd.Timedelta(
            days=int(self.incremental_intraday_max_gap_days or 5)
        )
    # +3 days keeps overlap bars in the tail across weekends/holidays.
    tail_days = int(gap.days) + 3
    if gap_too_large:
        return self._full_refetch(key, csv_path, refresh_mode="full_refetch_gap_too_large")
    assert self.upstream is not None
    tail = self.upstream.fetch_ohlcv(symbol, period=f"{tail_days}d", interval=interval)
    tail = self._annotate(tail, symbol=symbol, period=period, interval=interval)
    tail = self._complete_bars_only(tail)
    if not self._overlap_matches(cached, tail, overlapBars=overlapBars):
        return self._full_refetch(

```

```

        key, csv_path, refresh_mode="full_refetch_overlap_mismatch"
    )
    new_rows = tail[pd.DatetimeIndex(tail.index) > last_ts]
    if new_rows.empty:
        return cached
    merged = pd.concat([cached, new_rows[cached.columns.intersection(new_rows.columns)]]
    merged = merged[~merged.index.duplicated(keep="first")].sort_index()
    self._atomic_write(
        csv_path,
        merged,
        key=key,
        refresh_mode="incremental_append",
        rows_appended=int(len(new_rows)),
    )
    return merged

@staticmethod
def _overlap_matches(
    cached: pd.DataFrame,
    tail: pd.DataFrame,
    *,
    overlapBars: int,
    rel_tolerance: float = 1e-4,
) -> bool:
    common = cached.index.intersection(tail.index)
    if len(common) == 0:
        return False
    common = common.sort_values()[-overlapBars:]
    for column in ("open", "close"):
        if column not in cached.columns or column not in tail.columns:
            continue
        cached_values = cached.loc[common, column].astype(float).to_numpy()
        tail_values = tail.loc[common, column].astype(float).to_numpy()
        scale = abs(cached_values).clip(min=1e-9)
        if (abs(cached_values - tail_values) / scale > rel_tolerance).any():
            return False
    return True

def data_manifest(self, symbols: list[str] | None = None) -> dict[str, Any]:
    """Return a deterministic manifest over cached OHLCV artifacts."""
    assert self.cache_dir is not None
    requested = {s.upper() for s in symbols} if symbols else None
    entries: dict[str, Any] = {}
    for csv_path in sorted(self.cache_dir.glob("**.csv")):
        meta_path = csv_path.with_suffix(".metadata.json")
        metadata = self._read_metadata(meta_path)
        symbol = str(metadata.get("symbol") or csv_path.stem.split("_", 1)[0]).upper()
        if requested is not None and symbol not in requested:
            continue
        digest = _sha256_file(csv_path)
        entries[csv_path.stem] = {
            "symbol": symbol,
            "interval": metadata.get("interval"),
            "period": metadata.get("period"),
            "path": str(csv_path),
            "sha256": digest,
            "rows": metadata.get("rows", 0),
            "adjusted": metadata.get("adjusted", self.adjusted),
            "what_to_show": metadata.get("what_to_show", self.what_to_show),
            "bar_complete_column": "bar_complete",
            "provider_boundary": metadata.get("provider_boundary", "period_interval_fetch"),
            "incremental_fetch_supported": metadata.get("incremental_fetch_supported", False),
            "refresh_mode": metadata.get("refresh_mode", "full_fetch"),
            "last_timestamp": metadata.get("last_timestamp", ""),
        }
    payload = {
        "schema_version": self.schema_version,
        "generated_at": datetime.now(UTC).isoformat(),
        "cache_dir": str(self.cache_dir),
        "artifact_format": "canonical_csv",
        "hash_algorithm": "sha256",
        "entries": entries,
    }
    payload["manifest_hash"] = hashlib.sha256(
        json.dumps(payload["entries"], sort_keys=True, separators=(",", ":"), default=str).encode()
    ).hexdigest()
    return payload

def _cache_path(self, symbol: str, period: str, interval: str) -> Path:
    assert self.cache_dir is not None
    safe = "_".join(_safe_part(part) for part in (symbol.upper(), interval, period))
    return self.cache_dir / f"{safe}.csv"

def _annotate(self, df: pd.DataFrame, *, symbol: str, period: str, interval: str) -> pd.DataFrame:
    out = normalize_ohlc(df)
    out = out.copy()
    out["adjusted"] = bool(self.adjusted)
    out["what_to_show"] = str(self.what_to_show or "")
    out["bar_complete"] = self._bar_complete_mask(out.index, interval)
    return out

@staticmethod
def _bar_complete_mask(index: pd.Index, interval: str) -> list[bool]:
    interval_lower = interval.lower()
    if interval_lower in _DAILY_INTERVALS:
        today = datetime.now(UTC).date()

```

```

        return [pd.Timestamp(value).date() < today for value in index]
    bar_minutes = _INTRADAY_BAR_MINUTES.get(interval_lower)
    if bar_minutes is None:
        return [True] * len(index)
    now = pd.Timestamp(datetime.now(UTC))
    width = pd.Timedelta(minutes=bar_minutes)
    return [
        CachedMarketDataProvider._as_utc(value) + width <= now for value in index
    ]

@staticmethod
def _as_utc(value: Any) -> pd.Timestamp:
    # Naive timestamps are treated as UTC; IBKR intraday bars arrive tz-aware.
    ts = pd.Timestamp(value)
    return ts.tz_localize("UTC") if ts.tzinfo is None else ts.tz_convert("UTC")

@staticmethod
def _complete_bars_only(df: pd.DataFrame) -> pd.DataFrame:
    if "bar_complete" not in df.columns:
        return df
    return df[df["bar_complete"].astype(bool)].copy()

@staticmethod
def _read_csv(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path, index_col=0, parse_dates=True)
    if df.index.name == "timestamp":
        df.index.name = None
    return df

def _atomic_write(
    self,
    path: Path,
    df: pd.DataFrame,
    *,
    key: tuple[str, str, str],
    refresh_mode: str = "full_fetch",
    rows_appended: int = 0,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp_path = path.with_name(f"{path.name}.{os.getpid()}.tmp")
    meta_path = path.with_suffix(".metadata.json")
    tmp_meta = meta_path.with_name(f"{meta_path.name}.{os.getpid()}.tmp")
    interval_lower = key[2].lower()
    incremental_supported = bool(
        self.incremental_enabled
        and (
            interval_lower in _DAILY_INTERVALS
            or (
                self.incremental_intraday_enabled
                and interval_lower in _INTRADAY_BAR_MINUTES
            )
        )
    )
    try:
        writable = df.copy()
        writable.index.name = "timestamp"
        writable.to_csv(tmp_path, float_format="%.10g", date_format="%Y-%m-%dT%H:%M:%S%z")
        digest = _sha256_file(tmp_path)
        metadata = {
            "schema_version": self.schema_version,
            "symbol": key[0],
            "period": key[1],
            "interval": key[2],
            "rows": int(len(df)),
            "first_timestamp": str(df.index[0]) if len(df) else "",
            "last_timestamp": str(df.index[-1]) if len(df) else "",
            "sha256": digest,
            "adjusted": bool(self.adjusted),
            "what_to_show": str(self.what_to_show or ""),
            "bar_complete_column": "bar_complete",
            "provider_boundary": "period_interval_fetch_with_tail_merge"
            if incremental_supported
            else "period_interval_fetch",
            "incremental_fetch_supported": incremental_supported,
            "refresh_mode": refresh_mode,
            "rows_appended": int(rows_appended),
            "created_at": datetime.now(UTC).isoformat(),
        }
        tmp_meta.write_text(json.dumps(metadata, indent=2, sort_keys=True), encoding="utf-8")
        os.replace(tmp_path, path)
        os.replace(tmp_meta, meta_path)
    finally:
        for leftover in (tmp_path, tmp_meta):
            if leftover.exists():
                leftover.unlink()

@staticmethod
def _read_metadata(path: Path) -> dict[str, Any]:
    try:
        payload = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return {}
    return payload if isinstance(payload, dict) else {}

```

```

def detect_unadjusted_splits(df: pd.DataFrame, *, threshold: float = 0.35) -> list[str]:

```

```

        """Heuristic timestamps where a split-like open/previous-close jump appears."""
        if df.empty or "open" not in df or "close" not in df:
            return []
        out = normalize_ohlc(df)
        previous_close = out["close"].shift(1)
        jump = (out["open"] / previous_close - 1.0).abs()
        volume = out["volume"].astype(float)
        median_volume = volume.rolling(20, min_periods=5).median().shift(1)
        volume_not_extreme = median_volume.isna() | (volume <= median_volume * 3.0)
        timestamps = out.index[(jump > threshold) & volume_not_extreme.fillna(True)]
        return [pd.Timestamp(ts).isoformat() for ts in timestamps]

def _safe_part(value: str) -> str:
    return "".join(ch if ch.isalnum() or ch in {"-", "."} else "_" for ch in value).strip("_")

def _sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()

```

Full Source: `backend/tradeo/services/ibkr\_data\_provider.py`

`backend/tradeo/services/ibkr\_data\_provider.py` ? 156 lines, 5003 bytes, sha256 prefix `06684e6a0931dcb4`

Audit relevance: IBKR historical data adapter. Audit connection lifecycle, blocking calls, account privacy, bar conversion, and error semantics.

Public surface summary before full source:

? class `IBKRHistoricalDataProvider` line 80 methods: fetch_ohlc

? function `inspect\_ibkr\_connection` line 116`

Risk hints: broad `except Exception` present; RNG path; verify seed/control; IBKR/broker/data dependency.

```

from __future__ import annotations

import asyncio
import random
import threading
from dataclasses import dataclass
from typing import Any

import pandas as pd

from tradeo.core.config import Settings, get_settings
from tradeo.services.technical_indicators import normalize_ohlc

_event_loop_state = threading.local()

def _duration_from_period(period: str) -> str:
    p = period.lower().strip()
    if p.endswith("y"):
        return f"{int(p[:-1])} Y"
    if p.endswith("mo"):
        return f"{int(p[:-2])} M"
    if p.endswith("d"):
        return f"{int(p[:-1])} D"
    return "2 Y"

def _bar_size_from_interval(interval: str) -> str:
    mapping = {
        "1d": "1 day",
        "1wk": "1 week",
        "1h": "1 hour",
        "30m": "30 mins",
        "15m": "15 mins",
        "5m": "5 mins",
        "1m": "1 min",
    }
    return mapping.get(interval.lower(), "1 day")

def _ensure_event_loop() -> None:
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        loop = getattr(_event_loop_state, "loop", None)
        if loop is None or loop.is_closed():
            loop = asyncio.new_event_loop()
            _event_loop_state.loop = loop
        asyncio.set_event_loop(loop)

def _connect_ibkr(settings: Settings):
    _ensure_event_loop()
    from ib_insync import IB

```

```

client_ids = random.sample(range(1000, 10000), 4)
if settings.ibkr_client_id not in client_ids:
    client_ids.append(settings.ibkr_client_id)
last_exc: Exception | None = None
for client_id in client_ids:
    ib = IB()
    try:
        ib.connect(
            settings.ibkr_host,
            settings.ibkr_port,
            clientId=client_id,
            timeout=float(getattr(settings, "ibkr_connect_timeout_seconds", 8.0)),
        )
        return ib, client_id
    except Exception as exc: # noqa: BLE001
        last_exc = exc
        if ib.isConnected():
            ib.disconnect()
if last_exc:
    raise last_exc
raise ConnectionError("IBKR connection failed")

@dataclass
class IBKRHistoricalDataProvider:
    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()

    def fetch_ohlc(self, symbol: str, period: str = "2y", interval: str = "1d") -> pd.DataFrame:
        _ensure_event_loop()
        from ib_insync import Stock, util

        ib, _ = _connect_ibkr(self.settings)
        try:
            contract = Stock(symbol.upper(), "SMART", "USD")
            qualified = ib.qualifyContracts(contract)
            if not qualified:
                raise ValueError(f"IBKR could not qualify contract for {symbol}")
            contract = qualified[0]
            bars = ib.reqHistoricalData(
                contract,
                endDate="",
                durationStr=_duration_from_period(period),
                barSizeSetting=_bar_size_from_interval(interval),
                whatToShow=self.settings.market_data_what_to_show,
                useRTH=True,
                formatDate=1,
            )
            df = util.df(bars)
            if df is None or df.empty:
                raise ValueError(f"IBKR returned no bars for {symbol}")
            if "date" in df.columns:
                df = df.set_index("date")
            return normalize_ohlc(df)
        finally:
            ib.disconnect()

def inspect_ibkr_connection(settings: Settings | None = None) -> dict[str, Any]:
    """Read-only connectivity check for TWS/IB Gateway.

    This function is safe for public health endpoints: it confirms connectivity but does not
    return balances or managed account identifiers. Use the admin-only IBKR account endpoint
    for detailed account state.
    """
    settings = settings or get_settings()

    ib = None
    try:
        ib, client_id = _connect_ibkr(settings)
        server_time = ib.reqCurrentTime()
        managed_accounts = ib.managedAccounts()
        return {
            "ok": True,
            "host": settings.ibkr_host,
            "port": settings.ibkr_port,
            "client_id": client_id,
            "readonly": settings.ibkr_readonly,
            "trading_mode": settings.trading_mode,
            "live_armed": settings.live_armed,
            "server_time": server_time.isoformat(),
            "managed_accounts_count": len(managed_accounts),
            "selected_account_configured": bool(settings.ibkr_account),
            "account_summary_included": False,
        }
    except Exception as exc: # noqa: BLE001
        return {
            "ok": False,
            "host": settings.ibkr_host,
            "port": settings.ibkr_port,
            "client_id": settings.ibkr_client_id,
            "readonly": settings.ibkr_readonly,
            "trading_mode": settings.trading_mode,

```



```

        "live_armed": settings.live_armed,
        "error": str(exc),
    }
finally:
    if ib and ib.isConnected():
        ib.disconnect()

```

Full Source: `backend/tradeo/services/provider\_factory.py`

`backend/tradeo/services/provider\_factory.py` ? 22 lines, 917 bytes, sha256 prefix `ce3944a441a82087`

Public surface summary before full source:

? function `get\_market\_data\_provider` line 8`

Risk hints: IBKR/broker/data dependency.

```

from __future__ import annotations

from tradeo.core.config import get_settings
from tradeo.services.data_provider import CachedMarketDataProvider, MarketDataProvider
from tradeo.services.ibkr_data_provider import IBKRHistoricalDataProvider

def get_market_data_provider() -> MarketDataProvider:
    settings = get_settings()
    if settings.allow_synthetic_market_data:
        raise RuntimeError("Synthetic market data is forbidden")
    if settings.market_data_provider.lower() != "ibkr":
        raise RuntimeError("Tradeo only permits IBKR market data")
    upstream = IBKRHistoricalDataProvider(settings=settings)
    if not settings.market_data_cache_enabled:
        return upstream
    return CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=settings.market_data_cache_path,
        adjusted=settings.market_data_adjusted,
        what_to_show=settings.market_data_what_to_show,
    )

```

Full Source: `backend/tradeo/services/data\_quality.py`

`backend/tradeo/services/data\_quality.py` ? 149 lines, 4985 bytes, sha256 prefix `939424808eb38f3c`

Public surface summary before full source:

? class `DataQualityReport` line 29 methods: research_grade, to_dict

? function `assess\_ohlcv\_quality` line 74`

? function `assess\_ohlcv\_quality\_from\_settings` line 134`

Risk hints: no obvious heuristic risk; still review logic/tests.

```

"""Per-symbol OHLCV quality assessment for research-grade scanning.

`normalize_ohlcv`/`validate_ohlcv` reject structurally broken bars (NaN, negative
prices, high < low). This module covers the next layer: data that is well-formed
but statistically poisonous for pattern research ? halted/illiquid symbols,
frozen feeds, unadjusted splits and large calendar holes. Symbols failing these
checks should be skipped and audited, never silently sampled.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import TYPE_CHECKING

import numpy as np
import pandas as pd

if TYPE_CHECKING:
    from tradeo.core.config import Settings

ISSUE_INSUFFICIENT_BARS = "insufficient_bars"
ISSUE_ZERO_VOLUME = "excess_zero_volume_bars"
ISSUE_STALE_CLOSES = "stale_close_run"
ISSUE_CALENDAR_GAP = "calendar_gap"
ISSUE_SUSPECT_BAR_RETURN = "suspect_split_or_bad_tick"

@dataclass(frozen=True)
class DataQualityReport:
    symbol: str
    bars: int
    zero_volume_pct: float
    longest_stale_close_run: int
    max_single_gap_business_days: int
    max_bar_return_ratio: float
    issues: list[str] = field(default_factory=list)

```

```

@property
def research_grade(self) -> bool:
    return not self.issues

def to_dict(self) -> dict:
    return {
        "symbol": self.symbol,
        "bars": self.bars,
        "zero_volume_pct": round(self.zero_volume_pct, 4),
        "longest_stale_close_run": self.longest_stale_close_run,
        "max_single_gap_business_days": self.max_single_gap_business_days,
        "max_bar_return_ratio": round(self.max_bar_return_ratio, 4),
        "issues": list(self.issues),
        "research_grade": self.research_grade,
    }

def _longest_constant_run(values: pd.Series) -> int:
    if values.empty:
        return 0
    changed = values.ne(values.shift())
    run_ids = changed.cumsum()
    return int(run_ids.value_counts().max())

def _max_business_day_gap(index: pd.Index) -> int:
    if not isinstance(index, pd.DatetimeIndex) or len(index) < 2:
        return 0
    idx = index.tz_localize(None) if index.tz is not None else index
    starts = idx[:-1].to_numpy(dtype="datetime64[D]")
    ends = idx[1:].to_numpy(dtype="datetime64[D]")
    # Missing business days strictly between consecutive bars.
    gaps = np.busday_count(starts, ends) - 1
    return int(gaps.max()) if len(gaps) else 0

def assess_ohlcv_quality(
    df: pd.DataFrame,
    symbol: str = "",
    *,
    interval: str = "1d",
    min_bars: int = 60,
    max_zero_volume_pct: float = 0.15,
    max_stale_close_run: int = 8,
    max_single_gap_business_days: int = 5,
    max_bar_return_ratio: float = 4.0,
) -> DataQualityReport:
    """Assess whether a normalized OHLCV frame is fit for pattern research.

    Expects a frame already passed through ``normalize_ohlcv`` (lowercase
    columns, sorted index). Calendar-gap detection only applies to daily bars.
    """
    issues: list[str] = []
    bars = int(len(df))
    if bars == 0:
        return DataQualityReport(
            symbol=symbol,
            bars=0,
            zero_volume_pct=1.0,
            longest_stale_close_run=0,
            max_single_gap_business_days=0,
            max_bar_return_ratio=1.0,
            issues=[ISSUE_INSUFFICIENT_BARS],
        )

    zero_volume_pct = float((df["volume"] <= 0).mean())
    stale_run = _longest_constant_run(df["close"])
    gap_days = _max_business_day_gap(df.index) if interval == "1d" else 0
    ratios = (df["close"] / df["close"].shift(1)).dropna()
    if ratios.empty:
        return_ratio = 1.0
    else:
        return_ratio = float(np.maximum(ratios, 1.0 / ratios).max())

    if bars < min_bars:
        issues.append(ISSUE_INSUFFICIENT_BARS)
    if zero_volume_pct > max_zero_volume_pct:
        issues.append(ISSUE_ZERO_VOLUME)
    if stale_run > max_stale_close_run:
        issues.append(ISSUE_STALE_CLOSES)
    if gap_days > max_single_gap_business_days:
        issues.append(ISSUE_CALENDAR_GAP)
    if return_ratio > max_bar_return_ratio:
        issues.append(ISSUE_SUSPECT_BAR_RETURN)

    return DataQualityReport(
        symbol=symbol,
        bars=bars,
        zero_volume_pct=zero_volume_pct,
        longest_stale_close_run=stale_run,
        max_single_gap_business_days=gap_days,
        max_bar_return_ratio=return_ratio,
        issues=issues,
    )

```

```
def assess_ohlcw_quality_from_settings(
    df: pd.DataFrame,
    symbol: str,
    interval: str,
    settings: "Settings",
) -> DataQualityReport:
    return assess_ohlcw_quality(
        df,
        symbol,
        interval=interval,
        min_bars=settings.data_quality_min_bars,
        max_zero_volume_pct=settings.data_quality_max_zero_volume_pct,
        max_stale_close_run=settings.data_quality_max_stale_close_run,
        max_single_gap_business_days=settings.data_quality_max_single_gap_business_days,
        max_bar_return_ratio=settings.data_quality_max_bar_return_ratio,
    )
```

Full Source: `backend/tradeo/services/universe\_snapshot.py`

`backend/tradeo/services/universe\_snapshot.py` ? 208 lines, 8161 bytes, sha256 prefix `953996d02dbfc3e8`

Public surface summary before full source:

? class `UniverseSnapshotService` line 20 methods: build_monthly_snapshot, latest_snapshot, symbols_for_month

Risk hints: wall-clock timestamp usage.

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import date, datetime, UTC
import hashlib
import json
import os
from pathlib import Path
from typing import Any

import pandas as pd

from tradeo.core.config import Settings, get_settings
from tradeo.services.data_provider import load_universe

ALLOWED_US_EXCHANGES = {"NYSE", "NASDAQ", "AMEX", "ARCA", "NYSEARCA"}

@dataclass(slots=True)
class UniverseSnapshotService:
    """Forward point-in-time universe snapshots.

    Historical delisting-aware data needs a licensed source. Until that exists,
    snapshots built by this service are honest from their creation month forward
    and carry `survivorship_biased=true` for backtests that look before the first
    snapshot.
    """

    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()

    def build_monthly_snapshot(
        self,
        as_of: date | datetime | str | None = None,
        *,
        universe: pd.DataFrame | None = None,
    ) -> dict[str, Any]:
        settings = self.settings
        assert settings is not None
        snapshot_date = _coerce_date(as_of)
        month_key = snapshot_date.strftime("%Y-%m")
        raw = universe.copy() if universe is not None else load_universe(settings.universe_file)
        filtered, rules = self._eligible(raw)
        filtered = filtered.sort_values("symbol").drop_duplicates("symbol").reset_index(drop=True)
        filtered["snapshot_month"] = month_key
        filtered["snapshot_as_of"] = snapshot_date.isoformat()
        filtered["point_in_time"] = bool(settings.universe_point_in_time_available)
        filtered["survivorship_biased"] = not bool(settings.universe_point_in_time_available)

        path = settings.universe_snapshot_path / f"{month_key}.csv"
        metadata_path = path.with_suffix(".metadata.json")
        self._atomic_write(path, filtered)
        digest = _sha256_file(path)
        metadata = {
            "schema_version": 1,
            "snapshot_month": month_key,
            "snapshot_as_of": snapshot_date.isoformat(),
            "source_universe_file": settings.universe_file,
            "path": str(path),
            "sha256": digest,
            "row_count": int(len(filtered)),
            "symbols": filtered["symbol"].tolist(),
            "eligibility_rules": rules,
            "point_in_time": bool(settings.universe_point_in_time_available),
        }
```

```

        "survivorship_biased": not bool(settings.universe_point_in_time_available),
        "delisting_data_available": bool(settings.universe_delisting_data_available),
        "content_hash": _content_hash(
            month_key=month_key,
            as_of=snapshot_date.isoformat(),
            symbols=filtered["symbol"].tolist(),
            rules=rules,
        ),
        "built_at": datetime.now(UTC).isoformat(),
    }
    self._write_json_atomic(metadata_path, metadata)
    return metadata

def latest_snapshot(self) -> dict[str, Any] | None:
    settings = self.settings
    assert settings is not None
    paths = sorted(settings.universe_snapshot_path.glob("*.metadata.json"))
    if not paths:
        return None
    try:
        payload = json.loads(paths[-1].read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return None
    return payload if isinstance(payload, dict) else None

def symbols_for_month(self, as_of: date | datetime | str) -> list[str]:
    settings = self.settings
    assert settings is not None
    month_key = _coerce_date(as_of).strftime("%Y-%m")
    path = settings.universe_snapshot_path / f"{month_key}.csv"
    if not path.exists():
        return []
    df = pd.read_csv(path)
    if "symbol" not in df:
        return []
    return df["symbol"].astype(str).str.upper().tolist()

def _eligible(self, universe: pd.DataFrame) -> tuple[pd.DataFrame, dict[str, Any]]:
    settings = self.settings
    assert settings is not None
    df = universe.copy()
    if "symbol" not in df.columns:
        raise ValueError("Universe snapshot requires a 'symbol' column")
    df["symbol"] = df["symbol"].astype(str).str.upper().str.strip()
    df = df[df["symbol"].str.len() > 0].copy()
    rules: dict[str, Any] = {
        "min_price": settings.min_price,
        "min_avg_dollar_volume": settings.min_avg_dollar_volume,
        "allowed_exchanges": sorted(ALLOWED_US_EXCHANGES),
        "price_column": "",
        "dollar_volume_column": "",
        "exchange_column": "",
        "missing_filter_columns": [],
    }
    price_col = _first_column(df, ("price", "last_price", "close", "adj_close"))
    if price_col:
        rules["price_column"] = price_col
        df = df[pd.to_numeric(df[price_col], errors="coerce") >= settings.min_price]
    else:
        rules["missing_filter_columns"].append("price")
    dollar_col = _first_column(
        df,
        ("avg_dollar_volume", "average_dollar_volume", "dollar_volume", "adv_usd"),
    )
    if dollar_col:
        rules["dollar_volume_column"] = dollar_col
        df = df[pd.to_numeric(df[dollar_col], errors="coerce") >= settings.min_avg_dollar_volume]
    else:
        rules["missing_filter_columns"].append("avg_dollar_volume")
    exchange_col = _first_column(df, ("exchange", "primary_exchange", "listing_exchange"))
    if exchange_col:
        rules["exchange_column"] = exchange_col
        exchanges = df[exchange_col].astype(str).str.upper().str.strip()
        df = df[exchanges.isin(ALLOWED_US_EXCHANGES)]
    else:
        rules["missing_filter_columns"].append("exchange")
    return df, rules

@staticmethod
def _atomic_write(path: Path, df: pd.DataFrame) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp_path = path.with_name(f"{path.name}.{os.getpid()}.tmp")
    try:
        df.to_csv(tmp_path, index=False)
        os.replace(tmp_path, path)
    finally:
        if tmp_path.exists():
            tmp_path.unlink()

@staticmethod
def _write_json_atomic(path: Path, payload: dict[str, Any]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp_path = path.with_name(f"{path.name}.{os.getpid()}.tmp")
    try:
        tmp_path.write_text(json.dumps(payload, indent=2, sort_keys=True), encoding="utf-8")
        os.replace(tmp_path, path)

```

```

        finally:
            if tmp_path.exists():
                tmp_path.unlink()

def _coerce_date(value: date | datetime | str | None) -> date:
    if value is None:
        return datetime.now(UTC).date()
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    return datetime.fromisoformat(value).date()

def _first_column(df: pd.DataFrame, candidates: tuple[str, ...]) -> str:
    columns = {str(column).lower(): str(column) for column in df.columns}
    for candidate in candidates:
        if candidate in columns:
            return columns[candidate]
    return ""

def _content_hash(*, month_key: str, as_of: str, symbols: list[str], rules: dict[str, Any]) -> str:
    """Deterministic snapshot identity: same inputs -> same hash across rebuilds.

    Excludes `built_at` and absolute paths so bit-for-bit reproducibility can
    be asserted without caring when or where the snapshot was rebuilt.
    """
    payload = {
        "snapshot_month": month_key,
        "snapshot_as_of": as_of,
        "symbols": symbols,
        "eligibility_rules": rules,
    }
    return hashlib.sha256(
        json.dumps(payload, sort_keys=True, separators=(",", ":"), default=str).encode()
    ).hexdigest()

def _sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()

```

Full Source: `backend/tradeo/services/backtester.py`

`backend/tradeo/services/backtester.py` ? 162 lines, 6444 bytes, sha256 prefix `183a6fe8e92b1bd7`

Audit relevance: Simple backtest engine with canonical outcome and non-trade accounting. Audit same-bar assumptions, costs, skipped signals, and parity with research analyzer.

Public surface summary before full source:

? class `SimulatedTrade` line 15 methods: ?

? class `Backtester` line 28 methods: run

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from dataclasses import dataclass
import numpy as np
import pandas as pd

from tradeo.schemas import BacktestMetrics, PatternCandidate
from tradeo.research.quant_validation import tiered_round_trip_cost_r, triple_barrier_outcome
from tradeo.services.data_provider import MarketDataProvider
from tradeo.services.pattern_detector import CupPatternDetector
from tradeo.services.technical_indicators import max_drawdown_pct

@dataclass
class SimulatedTrade:
    symbol: str
    entry_date: str
    exit_date: str
    side: str
    entry: float
    stop: float
    target: float
    exit: float
    r_multiple: float
    outcome: str

class Backtester:
    def __init__(
        self,
        provider: MarketDataProvider | None = None,

```

```

        detector: CupPatternDetector | None = None,
        starting_equity: float = 3000.0,
        risk_per_trade_pct: float = 0.01,
    ) -> None:
        if provider is None:
            from tradeo.services.provider_factory import get_market_data_provider

            provider = get_market_data_provider()
        self.provider = provider
        self.detector = detector or CupPatternDetector()
        self.starting_equity = starting_equity
        self.risk_per_trade_pct = risk_per_trade_pct

def run(self, symbols: list[str], period: str = "3y", interval: str = "1d") -> BacktestMetrics:
    trades: list[SimulatedTrade] = []
    equity_curve = [self.starting_equity]
    equity = self.starting_equity
    total_signals = 0
    skipped_signals = 0
    for symbol in symbols:
        df = self.provider.fetch_ohlc(symbol, period=period, interval=interval)
        if len(df) < 260:
            continue
        symbol_trades, symbol_signals, symbol_skipped = self._run_symbol(symbol, df, equity_curve, equity)
        trades.extend(symbol_trades)
        total_signals += symbol_signals
        skipped_signals += symbol_skipped
        equity = equity_curve[-1]
    r_values = [t.r_multiple for t in trades]
    wins = [r for r in r_values if r > 0]
    losses = [r for r in r_values if r < 0]
    total = len(trades)
    profit_factor = float(sum(wins) / abs(sum(losses))) if losses else float(sum(wins) or 0.0)
    metrics = BacktestMetrics(
        total_trades=total,
        win_rate=round(len(wins) / total, 4) if total else 0.0,
        profit_factor=round(profit_factor, 4),
        expectancy_r=round(float(np.mean(r_values)), 4) if r_values else 0.0,
        avg_r_multiple=round(float(np.mean(r_values)), 4) if r_values else 0.0,
        max_drawdown_pct=round(max_drawdown_pct(equity_curve), 4),
        total_signals=total_signals,
        skipped_signals=skipped_signals,
        skip_rate=round(skipped_signals / total_signals, 4) if total_signals else 0.0,
        trades=[t.__dict__ for t in trades[:500]],
    )
    return metrics

def _run_symbol(
    self, symbol: str, df: pd.DataFrame, equity_curve: list[float], equity: float
) -> tuple[list[SimulatedTrade], int, int]:
    """Simulate one symbol; returns (trades, signals detected, signals skipped).

    Skipped signals are detected candidates that never become trades (entry
    gap pre-filter or canonical gap-entry skip) and must not enter trade
    aggregates.
    """
    trades: list[SimulatedTrade] = []
    signals = 0
    skipped = 0
    i = 230
    max_holding = self.detector.params.max_holdingBars
    while i < len(df) - max_holding - 1:
        lookback = df.iloc[: i + 1]
        candidate = self.detector.detect(symbol, lookback)
        if candidate is None:
            i += 1
            continue
        signals += 1
        # Enter next bar near open. Skip if next open gaps beyond 1R against setup.
        next_bar = df.iloc[i + 1]
        entry = float(next_bar["open"])
        if abs(entry - candidate.entry) / candidate.entry > 0.08:
            skipped += 1
            i += 1
            continue
        simulated = self._simulate_exit(symbol, df.iloc[i + 1 : i + 1 + max_holding], candidate, entry)
        if simulated is None:
            skipped += 1
            i += 1
            continue
        trades.append(simulated)
        equity += equity * self.risk_per_trade_pct * simulated.r_multiple
        equity_curve.append(equity)
        i += max_holding
    return trades, signals, skipped

def _simulate_exit(
    self, symbol: str, future: pd.DataFrame, candidate: PatternCandidate, entry: float
) -> SimulatedTrade | None:
    stop = candidate.stop
    risk_per_share = max(0.01, entry - stop)
    target = candidate.target
    cost_r = tiered_round_trip_cost_r(
        entry_price=entry,
        risk_per_share=risk_per_share,
        avg_dollar_volume=float(candidate.features.get("avg_dollar_volume", 0.0)),
    )

```

```

)
out = triple_barrier_outcome(
    [candidate.entry, *future["open"].astype(float).tolist()],
    [candidate.entry, *future["high"].astype(float).tolist()],
    [candidate.entry, *future["low"].astype(float).tolist()],
    [candidate.entry, *future["close"].astype(float).tolist()],
    signal_index=0,
    side=1,
    stop_price=float(stop),
    target_price=float(target),
    max_bars=len(future),
    gap_entry_policy="skip",
    round_trip_cost_R=cost_r,
)
if out["status"] != "ok":
    # skipped (gap entry) / invalid / no_data: non-trade, excluded from aggregates.
    return None
exit_idx = int(out["exit_index"] or 1) - 1
exit_idx = max(0, min(exit_idx, len(future) - 1))
exit_price = float(out["exit_price"])
outcome = str(out["reason"])
exit_date = str(future.index[exit_idx].date())
r_multiple = float(out["R"])
return SimulatedTrade(
    symbol=symbol,
    entry_date=str(future.index[0].date()),
    exit_date=exit_date,
    side="long",
    entry=round(entry, 2),
    stop=round(stop, 2),
    target=round(target, 2),
    exit=round(exit_price, 2),
    r_multiple=round(float(r_multiple), 4),
    outcome=outcome,
)

```

Full Source: `backend/tradeo/services/director\_review\_gate.py`

`backend/tradeo/services/director\_review\_gate.py` ? 1275 lines, 55088 bytes, sha256 prefix `bb58132baf6cb19a`

Audit relevance: Director review and production gate logic. Audit evidence thresholds, effective samples, skip-rate evidence, blockers, and whether recommendations can be mistaken for approvals.

Public surface summary before full source:

- ? class `DirectorReviewGate` line 37 methods: from_settings, refresh
- ? class `DirectorProductionGate` line 806 methods: evaluate_pattern, approve_pattern

Risk hints: broad `except Exception` present; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Callable

import numpy as np
from sqlalchemy.orm import Session, joinedload

from tradeo.core.config import Settings
from tradeo.db.models import AuditLog, DiscoveredPattern, DiscoveredPatternStatus, Trade, TradeStatus
from tradeo.research.sequential_tests import (
    ks_two_sample,
    posterior_probability_edge,
    sprt_inferiority,
)
from tradeo.services.effective_sample import (
    effective_sample_summary,
    persist_effective_sample_weights,
)
from tradeo.services.execution_quality import (
    pattern_execution_quality_summary,
    persist_execution_quality,
)
from tradeo.services.implementation_shortfall import pattern_slippage_summary
from tradeo.services.evidence import (
    PAPER_FILL_EVIDENCE_TYPES,
    evidence_metadata_with_stored_columns,
    evidence_quality_from_metadata,
    evidence_type_from_metadata,
    is_director_review_paper_fill_evidence,
)
from tradeo.modules.fox_hunter.production_manifest import build_production_manifest
from tradeo.services.state_policy import REVIEW_TRIGGER_STATES

@dataclass(slots=True)
class DirectorReviewGate:
    min_closed_lab_trades: int = 10
    min_lab_symbols: int = 3
    min_lab_trading_days: int = 3
    max_lab_drawdown_r: float = 4.0
    min_lab_profit_factor: float = 1.2

```

```

min_lab_expectancy_r: float = 0.0
min_baseline_edge_r: float = 0.05
min_research_expectancy_ratio: float = 0.5
# Sequential evaluation (informe §4.7): n=10 stays a *review trigger*; the
# decision additionally uses a Bayesian posterior shrunk toward Research,
# an SPRT fast-kill and a KS mechanism-change check.
sequential_evaluation_enabled: bool = True
posterior_min_probability: float = 0.80
posterior_min_edge_r: float = 0.10
sprt_alpha: float = 0.05
sprt_beta: float = 0.20
ks_mechanism_change_pvalue: float = 0.01
# Implementation shortfall gate (informe §4.6): a research edge that costs
# more than this median slippage_R to execute is not an executable edge.
max_median_slippage_r: float = 0.10
min_slippage_samples: int = 5
min_effective_lab_trades: int = 25
# Skip-rate evidence (Wave4-A): research/backtest skip accounting is
# surfaced as Director evidence and a warning only ? never a blocker, so
# gate behavior is unchanged. A high skip_rate means reported expectancy
# excludes many non-trades and executable coverage must be reviewed.
skip_rate_warning_threshold: float = 0.25

@classmethod
def from_settings(cls, settings: Settings) -> "DirectorReviewGate":
    return cls(
        min_lab_symbols=settings.director_min_symbols,
        min_lab_trading_days=settings.director_min_days,
        sequential_evaluation_enabled=settings.director_sequential_evaluation_enabled,
        posterior_min_probability=settings.director_posterior_min_probability,
        posterior_min_edge_r=settings.director_posterior_min_edge_r,
        sprt_alpha=settings.director_sprt_alpha,
        sprt_beta=settings.director_sprt_beta,
        max_median_slippage_r=settings.director_max_median_slippage_r,
        min_slippage_samples=settings.director_min_slippage_samples,
        min_effective_lab_trades=settings.director_min_eff_trades,
    )

def refresh(self, db: Session) -> dict[str, Any]:
    patterns = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.validation_passed.is_(True))
        .filter(DiscoveredPattern.status.in_(list(REVIEW_TRIGGER_STATES)))
        .all()
    )
    closed_trades = (
        db.query(Trade)
        .options(joinedload(Trade.signal))
        .filter(Trade.status == TradeStatus.CLOSED)
        .all()
    )
    marked: list[dict[str, Any]] = []
    blocked: list[dict[str, Any]] = []
    for pattern in patterns:
        trades = self._closed_lab_trades_for_pattern(closed_trades, pattern.id)
        baseline_trades = self._baseline_lab_trades_for_pattern(closed_trades, pattern.id)
        excluded_trades = self._excluded_lab_trades_for_pattern(closed_trades, pattern.id)
        metrics = self._store_lab_execution_metrics(
            pattern,
            trades,
            baseline_trades=baseline_trades,
            excluded_trades=excluded_trades,
        )
        blockers = list(metrics["promotion_blockers"])
        if blockers:
            blocked.append(
                {
                    "pattern_id": pattern.id,
                    "name": pattern.name,
                    "promotion_blockers": blockers,
                    "lab_execution": metrics,
                }
            )
        continue
    previous_status = (
        pattern.status.value if hasattr(pattern.status, "value") else str(pattern.status)
    )
    pattern.status = DiscoveredPatternStatus.DIRECTOR_REVIEW
    pattern.promotion_status = DiscoveredPatternStatus.DIRECTOR_REVIEW.value
    pattern.promotion_reason = (
        "Director review trigger reached, not production approval: "
        f"{len(trades)} normal IBKR paper fills >= {self.min_closed_lab_trades}"
    )
    db.add(
        AuditLog(
            actor="director_review_gate",
            action="pattern_marked_for_director_review",
            entity_type="discovered_pattern",
            entity_id=str(pattern.id),
            details_json={
                "pattern_id": pattern.id,
                "previous_status": previous_status,
                "new_status": DiscoveredPatternStatus.DIRECTOR_REVIEW.value,
                "lab_execution": metrics,
            },
        ),
    )

```



```

    )
    marked.append({"pattern_id": pattern.id, "name": pattern.name, "lab_execution": metrics})
db.commit()
return {
    "min_closed_lab_trades": self.min_closed_lab_trades,
    "review_trigger_min_closed_lab_trades": self.min_closed_lab_trades,
    "gate_scope": "director_review_trigger_not_production_approval",
    "patterns_checked": len(patterns),
    "marked_for_director_review": len(marked),
    "marked": marked,
    "blocked": blocked,
}

@staticmethod
def _closed_lab_trades_for_pattern(trades: list[Trade], pattern_id: int) -> list[Trade]:
    matched: list[Trade] = []
    for trade in trades:
        signal = trade.signal
        if signal is None:
            continue
        pattern = DirectorReviewGate._lab_pattern_id_for_trade(trade)
        if pattern is None:
            continue
        if pattern == pattern_id and DirectorReviewGate._counts_as_paper_fill(trade):
            matched.append(trade)
    return matched

@staticmethod
def _excluded_lab_trades_for_pattern(trades: list[Trade], pattern_id: int) -> list[Trade]:
    excluded: list[Trade] = []
    for trade in trades:
        pattern = DirectorReviewGate._lab_pattern_id_for_trade(trade)
        if pattern == pattern_id and not DirectorReviewGate._counts_as_paper_fill(trade):
            excluded.append(trade)
    return excluded

@staticmethod
def _lab_pattern_id_for_trade(trade: Trade) -> int | None:
    signal = trade.signal
    if signal is None:
        return None
    metadata = signal.metadata_json or {}
    if metadata.get("entry_module") != "laboratory":
        return None
    try:
        return int(metadata.get("pattern_id") or 0)
    except (TypeError, ValueError):
        return None

@staticmethod
def _counts_as_paper_fill(trade: Trade) -> bool:
    metadata = evidence_metadata_with_stored_columns(
        trade.metadata_json or {},
        evidence_type=trade.evidence_type,
        evidence_quality=trade.evidence_quality,
    )
    signal_metadata = trade.signal.metadata_json if trade.signal is not None else {}
    return is_director_review_paper_fill_evidence(
        metadata,
        signal_metadata=signal_metadata,
        trade_status=DirectorReviewGate._status_value(trade.status),
        broker_order_id=trade.broker_order_id,
    )

@staticmethod
def _status_value(status: Any) -> str:
    return str(status.value if hasattr(status, "value") else status)

@classmethod
def _baseline_lab_trades_for_pattern(cls, trades: list[Trade], pattern_id: int) -> list[Trade]:
    baseline: list[Trade] = []
    for trade in trades:
        lab_pattern_id = cls._lab_pattern_id_for_trade(trade)
        if (
            lab_pattern_id is not None
            and lab_pattern_id != pattern_id
            and cls._counts_as_paper_fill(trade)
        ):
            baseline.append(trade)
    return baseline

def _store_lab_execution_metrics(
    self,
    pattern: DiscoveredPattern,
    trades: list[Trade],
    *,
    baseline_trades: list[Trade],
    excluded_trades: list[Trade],
) -> dict[str, Any]:
    r_values = [float(trade.r_multiple or 0.0) for trade in trades]
    wins = [r for r in r_values if r > 0]
    losses = [abs(r) for r in r_values if r < 0]
    profit = sum(wins)
    loss = sum(losses)
    expectancy = sum(r_values) / len(r_values) if r_values else 0.0
    profit_factor = profit / loss if loss > 0 else profit if profit > 0 else 0.0

```

```

win_rate = len(wins) / len(r_values) if r_values else 0.0
unique_symbols = len({trade.symbol for trade in trades})
unique_days = len(
    {
        (trade.closed_at or trade.opened_at).date().isoformat()
        for trade in trades
        if trade.closed_at or trade.opened_at
    }
)
max_drawdown = _max_drawdown_r(r_values)
baseline_r_values = [float(trade.r_multiple or 0.0) for trade in baseline_trades]
baseline_expectancy = (
    sum(baseline_r_values) / len(baseline_r_values) if baseline_r_values else 0.0
)
research_expectancy = float(pattern.best_expectancy_r or pattern.expectancy_r or 0.0)
research_profit_factor = float(pattern.best_profit_factor or pattern.profit_factor or 0.0)
research_r_values = self._research_r_values(pattern)
effective_sample = effective_sample_summary(trades)
persist_effective_sample_weights(trades, effective_sample)
effective_trades = float(effective_sample["n_eff"])
slippage = pattern_slippage_summary(trades)
execution_quality = pattern_execution_quality_summary(trades)
persist_execution_quality(trades)
sequential = self._sequential_evaluation(
    lab_r=r_values,
    research_r=research_r_values,
    research_expectancy=research_expectancy,
)
skip_accounting = self._research_skip_accounting(pattern)
skip_rate_warning = bool(
    skip_accounting.get("available")
    and float(skip_accounting.get("skip_rate") or 0.0) >= self.skip_rate_warning_threshold
)
blockers = self._promotion_blockers(
    closed_trades=len(trades),
    effective_trades=effective_trades,
    unique_symbols=unique_symbols,
    unique_days=unique_days,
    max_drawdown=max_drawdown,
    expectancy=expectancy,
    profit_factor=profit_factor,
    baseline_expectancy=baseline_expectancy,
    research_expectancy=research_expectancy,
    slippage=slippage,
    sequential=sequential,
)
metrics = {
    "gate_scope": "director_review_trigger",
    "not_production_approval": True,
    "production_gate_required": "DirectorProductionGate",
    "evidence_count_policy": (
        "Only normal ibkr_paper_fill evidence counts. Shadow, near-miss, "
        "no_ibkr_order and degraded fallback rows are excluded."
    ),
    "closed_lab_trades": len(trades),
    "paper_fill_trades": len(trades),
    "effective_lab_trades": round(effective_trades, 4),
    "min_effective_lab_trades": self.min_effective_lab_trades,
    "effective_lab_trades_note": (
        "Weighted effective sample: each normal IBKR paper fill weighs "
        "1/cluster_size for its (symbol, trading_day) cluster, so correlated "
        "same-symbol same-day fills count as one sample. Per-trade weights are "
        "persisted in trade.metadata_json.effective_sample."
    ),
    "effective_sample": effective_sample,
    "director_review_trigger_trades": len(trades),
    "min_closed_lab_trades": self.min_closed_lab_trades,
    "review_trigger_min_closed_lab_trades": self.min_closed_lab_trades,
    "excluded_lab_evidence_trades": len(excluded_trades),
    "excluded_evidence_type_counts": self._evidence_type_counts(excluded_trades),
    "excluded_evidence_quality_counts": self._evidence_quality_counts(excluded_trades),
    "trades_remaining_for_director_review": max(
        0, self.min_closed_lab_trades - len(trades)
    ),
    "unique_lab_symbols": unique_symbols,
    "min_lab_symbols": self.min_lab_symbols,
    "unique_lab_days": unique_days,
    "min_lab_trading_days": self.min_lab_trading_days,
    "max_lab_drawdown_r": round(max_drawdown, 4),
    "max_allowed_lab_drawdown_r": self.max_lab_drawdown_r,
    "eligible_for_director_review": not blockers,
    "promotion_blockers": blockers,
    "lab_expectancy_r": round(expectancy, 4),
    "lab_profit_factor": round(profit_factor, 4),
    "lab_win_rate": round(win_rate, 4),
    "baseline_expectancy_r": round(baseline_expectancy, 4),
    "baseline_delta_r": round(expectancy - baseline_expectancy, 4),
    "min_baseline_edge_r": self.min_baseline_edge_r,
    "research_expectancy_r": round(research_expectancy, 4),
    "research_profit_factor": round(research_profit_factor, 4),
    "expectancy_delta_r": round(expectancy - research_expectancy, 4),
    "profit_factor_delta": round(profit_factor - research_profit_factor, 4),
    "implementation_shortfall": {
        key: value for key, value in slippage.items() if key != "per_trade"
    },
    "max_median_slippage_r": self.max_median_slippage_r,
}

```

```

"execution_quality": {
    key: value
    for key, value in execution_quality.items()
    if key != "per_trade"
},
"execution_quality_note": (
    "Diagnostic only ? not a promotion blocker. Per-trade reports are "
    "persisted in trade.metadata_json.execution_quality. No real-time "
    "microstructure feed backs these numbers (informe §4.3, no provider).",
),
"sequential_evaluation": sequential,
"research_skip_accounting": skip_accounting,
"skip_rate_warning_threshold": self.skip_rate_warning_threshold,
"research_skip_rate_warning": skip_rate_warning,
"research_skip_accounting_note": (
    "Evidence only ? never a promotion blocker. Skipped signals are "
    "true non-trades: research expectancy/profit factor exclude them, "
    "so a high skip_rate means the edge was measured on fewer "
    "executable signals than were generated."
),
"by_entry_variant": self._bucket_metrics(
    trades,
    lambda trade: str(((trade.signal.metadata_json or {}) if trade.signal else {}).get("entry_variant_id") or "unknown"),
),
"by_regime": self._bucket_metrics(
    trades,
    lambda trade: str(
        (((trade.signal.metadata_json or {}) if trade.signal else {}).get("regime") or {}).get("regime_key")
        or "unknown"
    ),
),
)
metrics["by_entry_variant_empty_reason"] = self._empty_bucket_reason(
    trades,
    "closed lab trades with signal.metadata_json.entry_variant_id",
)
metrics["by_regime_empty_reason"] = self._empty_bucket_reason(
    trades,
    "closed lab trades with signal.metadata_json.regime.regime_key",
)
metrics["best_entry_variant"] = self._best_bucket(metrics["by_entry_variant"])
metrics["worst_entry_variant"] = self._worst_bucket(metrics["by_entry_variant"])
metrics["best_regime"] = self._best_bucket(metrics["by_regime"])
metrics["worst_regime"] = self._worst_bucket(metrics["by_regime"])
metrics["director_recommendations"] = self._director_recommendations(
    trades=trades,
    blockers=blockers,
    expectancy=expectancy,
    profit_factor=profit_factor,
    baseline_expectancy=baseline_expectancy,
    research_expectancy=research_expectancy,
    by_entry_variant=metrics["by_entry_variant"],
    by_regime=metrics["by_regime"],
    skip_accounting=skip_accounting,
    skip_rate_warning=skip_rate_warning,
)
pattern.metrics_json = {(pattern.metrics_json or {}), "lab_execution": metrics}
return metrics

@staticmethod
def _evidence_type_counts(trades: list[Trade]) -> dict[str, int]:
    counts: dict[str, int] = {}
    for trade in trades:
        signal_metadata = trade.signal.metadata_json if trade.signal is not None else {}
        metadata = evidence_metadata_with_stored_columns(
            trade.metadata_json or {},
            evidence_type=trade.evidence_type,
            evidence_quality=trade.evidence_quality,
        )
        evidence_type = (
            evidence_type_from_metadata(
                metadata,
                status=DirectorReviewGate._status_value(trade.status),
                signal_metadata=signal_metadata,
                broker_order_id=trade.broker_order_id,
            )
            or "unknown"
        )
        counts[evidence_type] = counts.get(evidence_type, 0) + 1
    return counts

@staticmethod
def _evidence_quality_counts(trades: list[Trade]) -> dict[str, int]:
    counts: dict[str, int] = {}
    for trade in trades:
        metadata = evidence_metadata_with_stored_columns(
            trade.metadata_json or {},
            evidence_type=trade.evidence_type,
            evidence_quality=trade.evidence_quality,
        )
        quality = evidence_quality_from_metadata(metadata)
        counts[quality] = counts.get(quality, 0) + 1
    return counts

@staticmethod
def _bucket_metrics(trades: list[Trade], key_fn: Callable[[Trade], str]) -> dict[str, dict[str, Any]]:

```

```

buckets: dict[str, list[float]] = {}
for trade in trades:
    buckets.setdefault(key_fn(trade), []).append(float(trade.r_multiple or 0.0))
result: dict[str, dict[str, Any]] = {}
for key, values in sorted(buckets.items()):
    wins = [value for value in values if value > 0]
    losses = [abs(value) for value in values if value < 0]
    loss = sum(losses)
    result[key] = {
        "closed_trades": len(values),
        "expectancy_r": round(sum(values) / len(values), 4) if values else 0.0,
        "win_rate": round(len(wins) / len(values), 4) if values else 0.0,
        "profit_factor": round(sum(wins) / loss, 4) if loss > 0 else round(sum(wins), 4),
    }
return result

@staticmethod
def _empty_bucket_reason(trades: list[Trade], required_data: str) -> str:
    if trades:
        return ""
    return (
        "no_closed_lab_trades: bucket performance is empty because there are no closed "
        f"normal IBKR paper fill evidence rows yet; missing {required_data}."
    )

@staticmethod
def _best_bucket(buckets: dict[str, dict[str, Any]]) -> dict[str, Any]:
    if not buckets:
        return {}
    key, metrics = max(
        buckets.items(),
        key=lambda item: (
            float(item[1].get("expectancy_r", 0.0) or 0.0),
            float(item[1].get("profit_factor", 0.0) or 0.0),
            int(item[1].get("closed_trades", 0) or 0),
        ),
    )
    return {"key": key, **metrics}

@staticmethod
def _worst_bucket(buckets: dict[str, dict[str, Any]]) -> dict[str, Any]:
    if not buckets:
        return {}
    key, metrics = min(
        buckets.items(),
        key=lambda item: (
            float(item[1].get("expectancy_r", 0.0) or 0.0),
            float(item[1].get("profit_factor", 0.0) or 0.0),
            int(item[1].get("closed_trades", 0) or 0),
        ),
    )
    return {"key": key, **metrics}

def _director_recommendations(
    self,
    *,
    trades: list[Trade],
    blockers: list[str],
    expectancy: float,
    profit_factor: float,
    baseline_expectancy: float,
    research_expectancy: float,
    by_entry_variant: dict[str, dict[str, Any]],
    by_regime: dict[str, dict[str, Any]],
    skip_accounting: dict[str, Any] | None = None,
    skip_rate_warning: bool = False,
) -> list[dict[str, Any]]:
    recommendations: list[dict[str, Any]] = []
    if skip_rate_warning and skip_accounting:
        recommendations.append(
            {
                "action": "review_research_skip_rate",
                "priority": "high",
                "skip_rate": float(skip_accounting.get("skip_rate") or 0.0),
                "signal_count": int(skip_accounting.get("signal_count") or 0),
                "skipped_count": int(skip_accounting.get("skipped_count") or 0),
                "skip_reason_counts": skip_accounting.get("skip_reason_counts") or {},
                "reason": (
                    "research metrics report a high non-trade skip_rate; reported "
                    "expectancy and profit factor exclude skipped signals, so verify "
                    "executable signal coverage before relying on this evidence"
                ),
            },
        )
    trades_remaining = max(0, self.min_closed_lab_trades - len(trades))
    if trades_remaining:
        recommendations.append(
            {
                "action": "collect_closed_lab_trades",
                "priority": "high",
                "trades_remaining": trades_remaining,
                "reason": (
                    f"needs {trades_remaining} more normal IBKR paper fills before "
                    "Director review can rely on execution evidence; this is not production approval"
                ),
            },
        )

```

```

    )
    if not trades:
        recommendations.append(
            {
                "action": "keep_research_only",
                "priority": "high",
                "reason": (
                    "no closed_lab_trades normal IBKR paper fill evidence exists, so by_regime "
                    "and by_entry_variant performance cannot be ranked yet"
                ),
            },
        )
    )
    return recommendations
    if any(blocker.startswith("unique_lab_symbols_below_") for blocker in blockers) or any(
        blocker.startswith("unique_lab_days_below_") for blocker in blockers
    ):
        recommendations.append(
            {
                "action": "diversify_paper_validation",
                "priority": "medium",
                "reason": "paper evidence is too concentrated by symbol or day",
            },
        )
    if expectancy <= baseline_expectancy + self.min_baseline_edge_r:
        recommendations.append(
            {
                "action": "compare_against_baseline_before_promotion",
                "priority": "high",
                "reason": "lab expectancy does not clear the configured baseline edge",
            },
        )
    if research_expectancy > 0 and expectancy < research_expectancy * self.min_research_expectancy_ratio:
        recommendations.append(
            {
                "action": "freeze_and_explain_research_decay",
                "priority": "high",
                "reason": "paper expectancy degraded materially versus research expectancy",
            },
        )
    if profit_factor < self.min_lab_profit_factor:
        recommendations.append(
            {
                "action": "refine_or_reject_profit_factor",
                "priority": "high",
                "reason": "paper profit factor is below Director threshold",
            },
        )
    best_variant = self._best_bucket(by_entry_variant)
    worst_variant = self._worst_bucket(by_entry_variant)
    if best_variant and worst_variant and best_variant.get("key") != worst_variant.get("key"):
        recommendations.append(
            {
                "action": "prioritize_entry_variant",
                "priority": "medium",
                "best_entry_variant": best_variant.get("key"),
                "worst_entry_variant": worst_variant.get("key"),
                "reason": "entry variants have materially different paper outcomes",
            },
        )
    best_regime = self._best_bucket(by_regime)
    worst_regime = self._worst_bucket(by_regime)
    if best_regime and worst_regime and best_regime.get("key") != worst_regime.get("key"):
        recommendations.append(
            {
                "action": "gate_by_regime_until_confirmed",
                "priority": "medium",
                "best_regime": best_regime.get("key"),
                "worst_regime": worst_regime.get("key"),
                "reason": "paper performance differs by market regime",
            },
        )
    if not recommendations:
        recommendations.append(
            {
                "action": "prepare_director_packet",
                "priority": "medium",
                "reason": (
                    "minimum review-trigger paper evidence is present; this is not production "
                    "approval and still requires Director audit"
                ),
            },
        )
    )
    return recommendations

def _promotion_blockers(
    self,
    *,
    closed_trades: int,
    effective_trades: float | None = None,
    unique_symbols: int,
    unique_days: int,
    max_drawdown: float,
    expectancy: float,
    profit_factor: float,
    baseline_expectancy: float,
    research_expectancy: float,

```

```

        slippage: dict[str, Any] | None = None,
        sequential: dict[str, Any] | None = None,
    ) -> list[str]:
        blockers: list[str] = []
        if closed_trades < self.min_closed_lab_trades:
            blockers.append(f"closed_lab_trades_below_{self.min_closed_lab_trades}")
        effective = float(closed_trades if effective_trades is None else effective_trades)
        if effective < self.min_effective_lab_trades:
            blockers.append(f"effective_lab_trades_below_{self.min_effective_lab_trades}")
        if unique_symbols < self.min_lab_symbols:
            blockers.append(f"unique_lab_symbols_below_{self.min_lab_symbols}")
        if unique_days < self.min_lab_trading_days:
            blockers.append(f"unique_lab_days_below_{self.min_lab_trading_days}")
        if max_drawdown > self.max_lab_drawdown_r:
            blockers.append(f"lab_drawdown_above_{self.max_lab_drawdown_r}r")
        if expectancy < self.min_lab_expectancy_r:
            blockers.append(f"lab_expectancy_below_{self.min_lab_expectancy_r}r")
        if profit_factor < self.min_lab_profit_factor:
            blockers.append(f"lab_profit_factor_below_{self.min_lab_profit_factor}")
        if expectancy < baseline_expectancy + self.min_baseline_edge_r:
            blockers.append("lab_expectancy_not_above_baseline")
        if research_expectancy > 0 and expectancy < research_expectancy * self.min_research_expectancy_ratio:
            blockers.append("lab_expectancy_degraded_vs_research")
        if slippage and slippage.get("count", 0) >= self.min_slippage_samples:
            median = slippage.get("median_slippage_r")
            if median is not None and float(median) > self.max_median_slippage_r:
                blockers.append(f"median_slippage_above_{self.max_median_slippage_r}r")
        if sequential and self.sequential_evaluation_enabled:
            sprt = sequential.get("sprt") or {}
            if sprt.get("decision") == "no_edge":
                blockers.append("sprt_concludes_no_edge")
            posterior = sequential.get("posterior") or {}
            probability = posterior.get("probability_edge")
            if (
                closed_trades >= self.min_closed_lab_trades
                and probability is not None
                and np.isfinite(probability)
                and float(probability) < self.posterior_min_probability
            ):
                blockers.append(
                    f"posterior_probability_below_{self.posterior_min_probability}"
                )
            ks = sequential.get("ks") or {}
            ks_p = ks.get("p_value")
            if (
                ks_p is not None
                and np.isfinite(ks_p)
                and float(ks_p) < self.ks_mechanism_change_pvalue
            ):
                blockers.append("lab_research_r_distribution_mismatch")
        return blockers

```

@staticmethod

```

def _normalized_skip_evidence(data: Any) -> dict[str, Any] | None:
    """Normalize research (`signal_count`/'skipped_count') or backtest
    (`total_signals`/'skipped_signals') skip accounting; None if absent."""
    if not isinstance(data, dict) or "skip_rate" not in data:
        return None
    signal_count = data.get("signal_count", data.get("total_signals"))
    skipped_count = data.get("skipped_count", data.get("skipped_signals"))
    if signal_count is None and skipped_count is None:
        return None
    evidence: dict[str, Any] = {
        "skip_rate": float(data.get("skip_rate") or 0.0),
        "signal_count": int(signal_count or 0),
        "skipped_count": int(skipped_count or 0),
    }
    reasons = data.get("skip_reason_counts")
    if isinstance(reasons, dict):
        evidence["skip_reason_counts"] = {
            str(reason): int(count or 0) for reason, count in reasons.items()
        }
    return evidence

```

@classmethod

```

def _research_skip_accounting(cls, pattern: DiscoveredPattern) -> dict[str, Any]:
    """Locate stored skip accounting for Director evidence.

    Reads what research/backtest runs already persisted on the pattern;
    never derives or invents numbers when the run predates skip accounting.
    """
    metrics = pattern.metrics_json or {}
    best_rr_key = f"{{float(pattern.best_rr or 0.0):g}}"
    rr_metric_sources = (
        ("pattern.rr_metrics_json", pattern.rr_metrics_json or {}),
        ("pattern.metrics_json.rr_metrics", metrics.get("rr_metrics") or {}),
    )
    for source, rr_metrics in rr_metric_sources:
        if not isinstance(rr_metrics, dict):
            continue
        keys = [key for key in (best_rr_key,) if key in rr_metrics]
        keys.extend(key for key in sorted(rr_metrics) if key not in keys)
        for key in keys:
            evidence = cls._normalized_skip_evidence(rr_metrics.get(key))
            if evidence is not None:
                return {

```

```

        "available": True,
        "source": f"{source}[{key}]",
        "rr_key": key,
        **evidence,
    }
    for key in ("backtest", "lab_backtest", "backtest_summary"):
        evidence = cls._normalized_skip_evidence(metrics.get(key))
        if evidence is not None:
            return {
                "available": True,
                "source": f"pattern.metrics_json.{key}",
                **evidence,
            }
    evidence = cls._normalized_skip_evidence(metrics)
    if evidence is not None:
        return {"available": True, "source": "pattern.metrics_json", **evidence}
    return {
        "available": False,
        "reason": (
            "no skip accounting (skip_rate + signal/skipped counts) found in "
            "pattern.rr_metrics_json or pattern.metrics_json; the research run "
            "likely predates non-trade accounting"
        ),
    }
}

@staticmethod
def _research_r_values(pattern: DiscoveredPattern) -> list[float]:
    """Research-side R sample for distribution and dispersion estimates."""
    try:
        examples = pattern.examples or []
    except Exception: # noqa: BLE001 - detached instance without session
        return []
    return [
        float(example.outcome_r)
        for example in examples
        if example.outcome_r is not None
    ]

def _sequential_evaluation(
    self,
    *,
    lab_r: list[float],
    research_r: list[float],
    research_expectancy: float,
) -> dict[str, Any]:
    """Posterior + SPRT + KS package for the Director decision (§4.7)."""
    if not self.sequential_evaluation_enabled:
        return {"enabled": False}
    research_sd = (
        float(np.std(np.asarray(research_r), ddof=1)) if len(research_r) >= 5 else 0.0
    )
    lab_sd = float(np.std(np.asarray(lab_r), ddof=1)) if len(lab_r) >= 3 else 0.0
    sigma = research_sd if research_sd > 0 else (lab_sd if lab_sd > 0 else 1.0)
    prior_sd = max(sigma / 2.0, 0.05)
    posterior = posterior_probability_edge(
        lab_r,
        prior_mean=research_expectancy,
        prior_sd=prior_sd,
        min_edge_r=self.posterior_min_edge_r,
    )
    sprt = sprt_inferiority(
        lab_r,
        research_mean=research_expectancy,
        sigma=sigma,
        alpha=self.sprt_alpha,
        beta=self.sprt_beta,
    )
    ks = (
        ks_two_sample(lab_r, research_r)
        if len(lab_r) >= 10 and len(research_r) >= 10
        else {
            "n_a": len(lab_r),
            "n_b": len(research_r),
            "statistic": None,
            "p_value": None,
            "skipped_reason": "needs >=10 lab and >=10 research R values",
        }
    )
    return _json_safe(
        {
            "enabled": True,
            "sigma_r": round(sigma, 4),
            "research_r_count": len(research_r),
            "posterior": posterior,
            "sprt": sprt,
            "ks": ks,
        }
    )

```

```
@dataclass(slots=True)
```

```
class DirectorProductionGate:
```

```
    """Hard production approval gate.
```

```

    DirectorReviewGate is only a review trigger. This gate is the first runtime
    path allowed to create a production manifest.

```

```
"""
```

```
min_paper_fills: int = 30
min_fill_symbols: int = 3
min_fill_trading_days: int = 10
max_drawdown_r: float = 4.0
min_profit_factor: float = 1.3
min_expectancy_r: float = 0.05
required_edge_claim: str = "NO_DEMOSTRADO"
```

```
def evaluate_pattern(
    self,
    pattern: DiscoveredPattern,
    closed_trades: list[Trade],
) -> dict[str, Any]:
    fills = [
        trade
        for trade in closed_trades
        if DirectorReviewGate._lab_pattern_id_for_trade(trade) == pattern.id
        and DirectorReviewGate._counts_as_paper_fill(trade)
    ]
    r_values = [float(trade.r_multiple or 0.0) for trade in fills]
    wins = [r for r in r_values if r > 0]
    losses = [abs(r) for r in r_values if r < 0]
    loss = sum(losses)
    profit_factor = sum(wins) / loss if loss > 0 else sum(wins) if wins else 0.0
    expectancy = sum(r_values) / len(r_values) if r_values else 0.0
    unique_symbols = len({trade.symbol for trade in fills})
    unique_days = len(
        {
            (trade.closed_at or trade.opened_at).date().isoformat()
            for trade in fills
            if trade.closed_at or trade.opened_at
        }
    )
    max_drawdown = _max_drawdown_r(r_values)
    blockers: list[str] = []
    if len(fills) < self.min_paper_fills:
        blockers.append(f"ibkr_paper_fills_below_{self.min_paper_fills}")
    if unique_symbols < self.min_fill_symbols:
        blockers.append(f"fill_symbols_below_{self.min_fill_symbols}")
    if unique_days < self.min_fill_trading_days:
        blockers.append(f"fill_days_below_{self.min_fill_trading_days}")
    if max_drawdown > self.max_drawdown_r:
        blockers.append(f"paper_fill_drawdown_above_{self.max_drawdown_r}r")
    if expectancy < self.min_expectancy_r:
        blockers.append(f"paper_fill_expectancy_below_{self.min_expectancy_r}r")
    if profit_factor < self.min_profit_factor:
        blockers.append(f"paper_fill_profit_factor_below_{self.min_profit_factor}")
    scientific_contracts = self._scientific_contracts(pattern, fills)
    blockers.extend(scientific_contracts["blockers"])
    return {
        "gate_scope": "director_production_gate",
        "approved_for_production": not blockers,
        "blockers": blockers,
        "scientific_contracts": scientific_contracts,
        "ibkr_paper_fills": len(fills),
        "min_paper_fills": self.min_paper_fills,
        "unique_fill_symbols": unique_symbols,
        "min_fill_symbols": self.min_fill_symbols,
        "unique_fill_days": unique_days,
        "min_fill_trading_days": self.min_fill_trading_days,
        "paper_fill_expectancy_r": round(expectancy, 4),
        "paper_fill_profit_factor": round(profit_factor, 4),
        "paper_fill_max_drawdown_r": round(max_drawdown, 4),
        "evidence_types_accepted": sorted(PAPER_FILL_EVIDENCE_TYPES),
        # Evidence only (Wave4-A): research/backtest skip accounting is
        # surfaced for the Director packet, never used as a blocker here.
        "research_skip_accounting": DirectorReviewGate._research_skip_accounting(pattern),
    }
```

```
def _scientific_contracts(
    self,
    pattern: DiscoveredPattern,
    fills: list[Trade],
) -> dict[str, Any]:
    metrics = pattern.metrics_json or {}
    blockers: list[str] = []

    nested = self._first_dict(
        metrics,
        ("nested_discovery_replay",),
        ("research_hypothesis", "nested_discovery_replay"),
        ("research_hypothesis_package", "nested_discovery_replay"),
    )
    nested_state = self._nested_replay_state(nested)
    if not nested_state["present"]:
        blockers.append("nested_discovery_replay_missing")
    else:
        if not nested_state["implemented"]:
            blockers.append("nested_discovery_replay_not_implemented")
        if not nested_state["passed"]:
            blockers.append("nested_discovery_replay_not_passed")
        if nested_state["blocking"]:
            blockers.append("nested_discovery_replay_blocking")
```



```

director_gate = self._first_dict(
    metrics,
    ("director_gate",),
    ("director_gate_result",),
    ("audit_gate",),
    ("audit_package", "director_gate"),
)
director_status = self._first_string(
    metrics,
    ("director_gate_status",),
    ("audit_gate_status",),
    ("director_gate", "status"),
    ("director_gate_result", "status"),
    ("audit_gate", "status"),
    ("audit_package", "director_gate_status"),
).lower()
if not director_status and director_gate:
    director_status = str(director_gate.get("director_gate_status") or "").strip().lower()
director_passed = director_status == "passed" or _truthy(director_gate.get("passed") if director_gate else None)
if not director_passed:
    blockers.append("director_audit_gate_not_passed" if director_status else "director_audit_gate_missing")

active_blockers = self._active_contract_blockers(metrics, director_gate)
if active_blockers:
    blockers.append("active_director_blockers_present")

event_ledger_hash = self._first_string(
    metrics,
    ("event_ledger_sha256",),
    ("event_ledger_hash",),
    ("research_hypothesis", "event_ledger_hash"),
    ("research_hypothesis_package", "event_ledger_hash"),
)
if not event_ledger_hash:
    blockers.append("event_ledger_hash_missing")

evidence_packet = self._first_dict(
    metrics,
    ("production_evidence_packet",),
    ("evidence_packet",),
    ("director_packet",),
    ("audit_package", "evidence_packet"),
)
evidence_packet_ref = self._evidence_packet_ref(evidence_packet)
evidence_packet_hash = self._first_string(
    metrics,
    ("evidence_packet_hash",),
    ("production_evidence_packet_hash",),
    ("audit_package_hash",),
    ("production_evidence_packet", "hash"),
    ("production_evidence_packet", "sha256"),
    ("production_evidence_packet", "packet_hash"),
    ("evidence_packet", "hash"),
    ("evidence_packet", "sha256"),
    ("evidence_packet", "packet_hash"),
)
if not evidence_packet_ref:
    blockers.append("evidence_packet_ref_missing")
if not evidence_packet_hash:
    blockers.append("evidence_packet_hash_missing")

provenance_state = self._execution_provenance_state(metrics, fills)
if not provenance_state["costs_reconciled"]:
    blockers.append("cost_provenance_not_reconciled")
if not provenance_state["slippage_reconciled"]:
    blockers.append("slippage_provenance_not_reconciled")
if not provenance_state["fills_reconciled"]:
    blockers.append("fill_provenance_not_reconciled")

drift_status = self._first_string(
    metrics,
    ("drift_status",),
    ("research_hypothesis", "drift_status"),
    ("research_hypothesis_package", "drift_status"),
) or str(getattr(pattern, "drift_status", "") or "")
if drift_status.strip().lower() in {"degrading", "regressing", "deteriorating"}:
    blockers.append("drift_status_degrading")

edge_claim = self._first_string(
    metrics,
    ("edge_claim",),
    ("research_hypothesis", "edge_claim"),
    ("research_hypothesis_package", "edge_claim"),
    ("global_experiment_registry", "edge_claim"),
)
if not edge_claim:
    blockers.append("edge_claim_missing")
elif edge_claim != self.required_edge_claim:
    blockers.append("edge_claim_inflated")

registry = self._first_dict(metrics, ("global_experiment_registry",))
registry_hash = self._first_string(
    metrics,
    ("global_experiment_registry", "registry_hash"),
    ("global_experiment_registry", "hash"),
    ("global_experiment_registry", "sha256"),
)

```

```

)
registry_run_manifest_hash = self._first_string(
    metrics,
    ("global_experiment_registry", "run_manifest_hash"),
    ("global_experiment_registry", "latest_run_manifest_hash"),
)
registry_chain_valid = (
    True
    if not registry or "hash_chain_valid" not in registry
    else _truthy(registry.get("hash_chain_valid"))
)
if not registry_hash:
    blockers.append("global_experiment_registry_hash_missing")
if not registry_run_manifest_hash:
    blockers.append("global_experiment_registry_run_manifest_hash_missing")
if not registry_chain_valid:
    blockers.append("global_experiment_registry_hash_chain_invalid")

return {
    "blockers": blockers,
    "nested_discovery_replay": nested_state,
    "director_gate_status": director_status or "missing",
    "director_gate_passed": director_passed,
    "active_blockers": active_blockers,
    "event_ledger_hash": event_ledger_hash,
    "evidence_packet": {
        "ref": evidence_packet_ref,
        "hash": evidence_packet_hash,
    },
    "execution_provenance": provenance_state,
    "drift_status": drift_status or "unknown",
    "edge_claim": edge_claim,
    "global_experiment_registry": {
        "path": registry.get("path") if registry else None,
        "registry_hash": registry_hash,
        "previous_registry_hash": registry.get("previous_registry_hash") if registry else None,
        "run_manifest_hash": registry_run_manifest_hash,
        "hash_chain_valid": registry_chain_valid,
    },
}

@staticmethod
def _nested_replay_state(replay: dict[str, Any]) -> dict[str, Any]:
    if not replay:
        return {
            "present": False,
            "implemented": False,
            "passed": False,
            "blocking": False,
            "status": "missing",
        }
    status = str(replay.get("status") or "").strip().lower()
    passed_statuses = {"passed", "pass", "ok", "complete", "completed", "succeeded", "success"}
    return {
        "present": True,
        "implemented": _truthy(replay.get("implemented")),
        "passed": _truthy(replay.get("passed")) or status in passed_statuses,
        "blocking": _truthy(replay.get("blocking")),
        "status": status or "unknown",
    }

@classmethod
def _execution_provenance_state(
    cls,
    metrics: dict[str, Any],
    fills: list[Trade],
) -> dict[str, Any]:
    provenance = cls._first_dict(
        metrics,
        ("execution_provenance",),
        ("cost_slippage_fill_provenance",),
        ("production_execution_provenance",),
    )
    if provenance:
        costs = _truthy(
            provenance.get("costs_reconciled")
            if "costs_reconciled" in provenance
            else provenance.get("cost_provenance_reconciled")
        )
        slippage = _truthy(
            provenance.get("slippage_reconciled")
            if "slippage_reconciled" in provenance
            else provenance.get("slippage_provenance_reconciled")
        )
        fill_state = _truthy(
            provenance.get("fills_reconciled")
            if "fills_reconciled" in provenance
            else provenance.get("fill_provenance_reconciled")
        )
    return {
        "source": "pattern.metrics_json",
        "costs_reconciled": costs,
        "slippage_reconciled": slippage,
        "fills_reconciled": fill_state,
    }

```

```

trade_count = len(fills)
cost_rows = 0
slippage_rows = 0
fill_rows = 0
for trade in fills:
    metadata = trade.metadata_json or {}
    if (
        _truthy(metadata.get("cost_provenance_reconciled"))
        or (
            metadata.get("commission") is not None
            and metadata.get("commission_source")
            and metadata.get("estimated_spread_cost") is not None
            and metadata.get("spread_cost_source")
        )
    ):
        cost_rows += 1
    if _truthy(metadata.get("slippage_provenance_reconciled")) or (
        metadata.get("estimated_slippage") is not None and metadata.get("slippage_source")
    ):
        slippage_rows += 1
    if _truthy(metadata.get("fill_provenance_reconciled")) or (
        trade.broker_order_id
        or metadata.get("broker_order_id")
        or metadata.get("parent_order_id")
        or metadata.get("fill_id_hash")
        or metadata.get("entry_fill_time")
    ):
        fill_rows += 1
return {
    "source": "trade.metadata_json",
    "costs_reconciled": trade_count > 0 and cost_rows == trade_count,
    "slippage_reconciled": trade_count > 0 and slippage_rows == trade_count,
    "fills_reconciled": trade_count > 0 and fill_rows == trade_count,
    "trade_count": trade_count,
    "cost_rows": cost_rows,
    "slippage_rows": slippage_rows,
    "fill_rows": fill_rows,
}

@staticmethod
def _active_contract_blockers(metrics: dict[str, Any], director_gate: dict[str, Any]) -> list[str]:
    values: list[Any] = []
    for key in ("active_blockers", "promotion_blockers", "director_blockers"):
        if key in metrics:
            values.append(metrics.get(key))
    if director_gate:
        values.append(director_gate.get("blockers"))
        summary = director_gate.get("summary")
        if isinstance(summary, dict):
            values.append(summary.get("blockers"))
    lab_execution = metrics.get("lab_execution")
    if isinstance(lab_execution, dict):
        values.append(lab_execution.get("promotion_blockers"))
    blockers: list[str] = []
    for value in values:
        if isinstance(value, list):
            blockers.extend(str(item) for item in value if str(item).strip())
        elif isinstance(value, str) and value.strip():
            blockers.append(value.strip())
    return blockers

@staticmethod
def _evidence_packet_ref(packet: dict[str, Any]) -> str:
    if not packet:
        return ""
    for key in ("id", "packet_id", "uri", "path"):
        value = str(packet.get(key) or "").strip()
        if value:
            return value
    return ""

@staticmethod
def _first_dict(metrics: dict[str, Any], *paths: tuple[str, ...]) -> dict[str, Any]:
    for path in paths:
        value: Any = metrics
        for key in path:
            value = value.get(key) if isinstance(value, dict) else None
        if isinstance(value, dict) and value:
            return value
    return {}

@staticmethod
def _first_string(metrics: dict[str, Any], *paths: tuple[str, ...]) -> str:
    for path in paths:
        value: Any = metrics
        for key in path:
            value = value.get(key) if isinstance(value, dict) else None
        text = str(value or "").strip()
        if text:
            return text
    return ""

def approve_pattern(
    self,
    db: Session,
    *,

```

```

        pattern: DiscoveredPattern,
        reviewer: str = "director",
    ) -> dict[str, Any]:
        closed_trades = (
            db.query(Trade)
            .options(joinedload(Trade.signal))
            .filter(Trade.status == TradeStatus.CLOSED)
            .all()
        )
        decision = self.evaluate_pattern(pattern, closed_trades)
        if not bool(decision["approved_for_production"]):
            pattern.metrics_json = {
                **(pattern.metrics_json or {}),
                "production_gate": decision,
            }
            db.add(pattern)
            db.commit()
            return decision
        manifest = build_production_manifest(
            pattern,
            reviewer=reviewer,
            evidence_packet=decision,
        )
        previous_status = (
            pattern.status.value if hasattr(pattern.status, "value") else str(pattern.status)
        )
        pattern.status = DiscoveredPatternStatus.PRODUCTION
        pattern.promotion_status = DiscoveredPatternStatus.PRODUCTION.value
        pattern.promotion_reason = "DirectorProductionGate approved normal IBKR paper fill evidence"
        pattern.metrics_json = {
            **(pattern.metrics_json or {}),
            "production_gate": decision,
            "production_manifest": manifest,
        }
        db.add(
            AuditLog(
                actor="director_production_gate",
                action="pattern_production_manifest_approved",
                entity_type="discovered_pattern",
                entity_id=str(pattern.id),
                details_json={
                    "previous_status": previous_status,
                    "new_status": DiscoveredPatternStatus.PRODUCTION.value,
                    "production_gate": decision,
                    "production_manifest": manifest,
                },
            )
        )
        db.add(pattern)
        db.commit()
        return {"decision": decision, "production_manifest": manifest}

def _json_safe(value: Any) -> Any:
    """Replace non-finite floats with None so metrics survive JSONB storage."""
    if isinstance(value, dict):
        return {key: _json_safe(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [_json_safe(item) for item in value]
    if isinstance(value, (float, np.floating)):
        return float(value) if np.isfinite(value) else None
    if isinstance(value, np.integer):
        return int(value)
    return value

def _max_drawdown_r(r_values: list[float]) -> float:
    peak = 0.0
    cumulative = 0.0
    max_drawdown = 0.0
    for value in r_values:
        cumulative += value
        peak = max(peak, cumulative)
        max_drawdown = max(max_drawdown, peak - cumulative)
    return max_drawdown

def _truthy(value: Any) -> bool:
    if isinstance(value, bool):
        return value
    return str(value or "").strip().lower() in {"true", "1", "yes", "y", "ok", "passed", "pass"}

```

Full Source: ``backend/tradeo/services/effective_sample.py``

``backend/tradeo/services/effective_sample.py`` ? 118 lines, 4425 bytes, sha256 prefix ``17c52ae0553b554c``

Public surface summary before full source:

```

? function `trade_cluster_key` line 44`
? function `effective_sample_summary` line 50`
? function `persist_effective_sample_weights` line 87`

```

Risk hints: wall-clock timestamp usage; JSON metadata contract; schema drift risk.

```
"""Effective-sample weights for Director paper-fill evidence (informe §4.7).
```

Paper fills that share the same symbol and trading day are not independent observations of the edge: a burst of five AAPL fills on one afternoon is much closer to one sample than to five. The Director gates therefore need an *effective* sample size, not a raw fill count.

The weighting scheme is deliberately simple so it can be audited by hand:

- Each closed paper fill belongs to a cluster keyed by `(symbol, trading day)` where the day comes from `closed\_at` (falling back to `opened\_at`).
- Each fill weighs $1 / \text{cluster_size}$, so every cluster contributes exactly one effective sample regardless of how many fills landed in it.
- `n_eff` is the sum of weights, which equals the number of distinct clusters, and is the number gates compare against their minimums. The Kish effective sample size is reported alongside as a secondary diagnostic of weight inequality only (with equal weights it returns the raw count), so it is always $\geq n_eff$ and never used as the binding number.

Weights are persisted on each trade's `metadata_json["effective_sample"]` and the full per-trade breakdown is stored in the pattern's `lab_execution` metrics, so any `n_eff` used by a gate decision can be reproduced from the stored rows alone.

```
"""
```

```
from __future__ import annotations
```

```
from datetime import datetime, timezone
from typing import Any, Iterable
```

```
from tradeo.db.models import Trade
```

```
EFFECTIVE_SAMPLE_METHOD = "inverse_symbol_day_cluster_size_v1"
```

```
__all__ = [
    "EFFECTIVE_SAMPLE_METHOD",
    "trade_cluster_key",
    "effective_sample_summary",
    "persist_effective_sample_weights",
]
```

```
def trade_cluster_key(trade: Trade) -> str:
    moment = trade.closed_at or trade.opened_at
    day = moment.date().isoformat() if moment is not None else "unknown"
    return f"{{trade.symbol or ''}.upper()}|{{day}}"
```

```
def effective_sample_summary(trades: Iterable[Trade]) -> dict[str, Any]:
    """Compute auditable per-trade weights and the resulting n_eff."""
    trades = list(trades)
    cluster_sizes: dict[str, int] = {}
    for trade in trades:
        key = trade_cluster_key(trade)
        cluster_sizes[key] = cluster_sizes.get(key, 0) + 1
    weights: list[dict[str, Any]] = []
    weight_values: list[float] = []
    for trade in trades:
        key = trade_cluster_key(trade)
        size = cluster_sizes[key]
        weight = 1.0 / size
        weight_values.append(weight)
        weights.append(
            {
                "trade_id": trade.id,
                "symbol": trade.symbol,
                "cluster_key": key,
                "cluster_size": size,
                "weight": round(weight, 6),
            }
        )
    total = sum(weight_values)
    sum_sq = sum(value * value for value in weight_values)
    kish = (total * total) / sum_sq if sum_sq > 0 else 0.0
    return {
        "method": EFFECTIVE_SAMPLE_METHOD,
        "n_trades": len(trades),
        "n_eff": round(total, 6),
        "kish_n_eff": round(kish, 6),
        "cluster_count": len(cluster_sizes),
        "cluster_sizes": dict(sorted(cluster_sizes.items())),
        "weights": weights,
    }
```

```
def persist_effective_sample_weights(
    trades: Iterable[Trade],
    summary: dict[str, Any],
) -> int:
    """Store each trade's weight in its metadata_json. Returns rows touched.

    The weight depends on the cluster composition at evaluation time, so the
    stored entry records the method and timestamp that produced it.
    """
```

```

by_trade_id = {entry["trade_id"]: entry for entry in summary.get("weights", [])}
now = datetime.now(timezone.utc).isoformat()
touched = 0
for trade in trades:
    entry = by_trade_id.get(trade.id)
    if entry is None:
        continue
    record = {
        "method": summary["method"],
        "cluster_key": entry["cluster_key"],
        "cluster_size": entry["cluster_size"],
        "weight": entry["weight"],
        "computed_at": now,
    }
    existing = (trade.metadata_json or {}).get("effective_sample")
    if existing and all(
        existing.get(key) == record[key]
        for key in ("method", "cluster_key", "cluster_size", "weight")
    ):
        continue
    trade.metadata_json = {**(trade.metadata_json or {}), "effective_sample": record}
    touched += 1
return touched

```

Full Source: `backend/tradeo/services/evidence.py`

`backend/tradeo/services/evidence.py` ? 379 lines, 12979 bytes, sha256 prefix `a60c99fa54cc39a1`

Public surface summary before full source:

```

? class `EvidenceType` line 24 methods: ?
? class `EvidenceQuality` line 33 methods: ?
? class `FillProvenance` line 38 methods: ?

? function `evidence_type_for_metadata` line 109`
? function `evidence_quality_for_metadata` line 154`
? function `fill_provenance_for_metadata` line 162`
? function `evidence_type_from_metadata` line 172`
? function `evidence_quality_from_metadata` line 193`
? function `evidence_metadata_with_stored_columns` line 197`
? function `with_evidence_metadata` line 211`
? function `is_director_review_paper_fill_evidence` line 232`
? function `is_paper_order_or_fill_evidence` line 248`
? function `is_strong_fill_evidence` line 276`
? function `evidence_type_counts` line 304`
? function `evidence_quality_counts` line 317`

```

Risk hints: IBKR/broker/data dependency.

```

from __future__ import annotations

from enum import Enum
from typing import Any, Mapping

EVIDENCE_NEAR_MISS_SHADOW = "near_miss_shadow"
EVIDENCE_SHADOW_NO_ORDER = "shadow_no_order"
EVIDENCE_IBKR_PAPER_ORDER = "ibkr_paper_order"
EVIDENCE_IBKR_PAPER_FILL = "ibkr_paper_fill"
EVIDENCE_LIVE_ORDER = "live_order"
EVIDENCE_LIVE_FILL = "live_fill"

FILL_PROVENANCE_BROKER_EXECUTION = "broker_execution"
FILL_PROVENANCE_BROKER_STATEMENT_IMPORT = "broker_statement_import"
FILL_PROVENANCE_SIMULATED_CLOSE = "simulated_close"
FILL_PROVENANCE_SHADOW_CLOSE = "shadow_close"

EVIDENCE_QUALITY_STANDARD = "standard"
EVIDENCE_QUALITY_DEGRADED = "degraded"

LAB_SHADOW_EXECUTION_MODE = "lab_shadow_observation"

class EvidenceType(str, Enum):
    NEAR_MISS_SHADOW = EVIDENCE_NEAR_MISS_SHADOW
    SHADOW_NO_ORDER = EVIDENCE_SHADOW_NO_ORDER
    IBKR_PAPER_ORDER = EVIDENCE_IBKR_PAPER_ORDER
    IBKR_PAPER_FILL = EVIDENCE_IBKR_PAPER_FILL
    LIVE_ORDER = EVIDENCE_LIVE_ORDER
    LIVE_FILL = EVIDENCE_LIVE_FILL

class EvidenceQuality(str, Enum):
    NORMAL = EVIDENCE_QUALITY_STANDARD
    DEGRADED = EVIDENCE_QUALITY_DEGRADED

class FillProvenance(str, Enum):
    BROKER_EXECUTION = FILL_PROVENANCE_BROKER_EXECUTION

```

```

BROKER_STATEMENT_IMPORT = FILL_PROVENANCE_BROKER_STATEMENT_IMPORT
SIMULATED_CLOSE = FILL_PROVENANCE_SIMULATED_CLOSE
SHADOW_CLOSE = FILL_PROVENANCE_SHADOW_CLOSE

VALID_EVIDENCE_TYPES = {
    EVIDENCE_NEAR_MISS_SHADOW,
    EVIDENCE_SHADOW_NO_ORDER,
    EVIDENCE_IBKR_PAPER_ORDER,
    EVIDENCE_IBKR_PAPER_FILL,
    EVIDENCE_LIVE_ORDER,
    EVIDENCE_LIVE_FILL,
}
PAPER_FILL_EVIDENCE_TYPES = {EVIDENCE_IBKR_PAPER_FILL}
SHADOW_EVIDENCE_TYPES = {EVIDENCE_NEAR_MISS_SHADOW, EVIDENCE_SHADOW_NO_ORDER}
FILL_EVIDENCE_TYPES = {EVIDENCE_IBKR_PAPER_FILL, EVIDENCE_LIVE_FILL}
REAL_FILL_PROVENANCE = {
    FILL_PROVENANCE_BROKER_EXECUTION,
    FILL_PROVENANCE_BROKER_STATEMENT_IMPORT,
}
VALID_FILL_PROVENANCE = {
    FILL_PROVENANCE_BROKER_EXECUTION,
    FILL_PROVENANCE_BROKER_STATEMENT_IMPORT,
    FILL_PROVENANCE_SIMULATED_CLOSE,
    FILL_PROVENANCE_SHADOW_CLOSE,
}
STRONG_EVIDENCE_QUALITY = {EVIDENCE_QUALITY_STANDARD, "normal"}

FILL_ID_KEYS = (
    "fill_id",
    "broker_fill_id",
    "ib_fill_id",
    "execution_id",
    "exec_id",
    "ib_exec_id",
    "broker_execution_id",
    "execution_hash",
    "broker_execution_hash",
    "ib_execution_hash",
    "fill_id_hash",
)
BROKER_TIMESTAMP_KEYS = (
    "broker_execution_time",
    "ib_execution_time",
    "broker_executed_at",
    "execution_time",
    "executed_at",
    "fill_time",
    "broker_fill_time",
    "statement_execution_time",
)
COMMISSION_KEYS = (
    "commission",
    "commission_usd",
    "broker_commission",
    "ib_commission",
    "fees",
    "other_fees",
)
EVIDENCE_NESTED_METADATA_KEYS = (
    "broker_execution",
    "execution",
    "execution_report",
    "execution_observation",
    "broker_statement",
    "statement_row",
)

def evidence_type_for_metadata(
    metadata: Mapping[str, Any] | None,
    *,
    trade_status: Any = None,
) -> str | None:
    """Return explicit or legacy-inferred evidence type for a signal/trade."""
    metadata = metadata or {}
    explicit = str(metadata.get("evidence_type") or "").strip()
    if explicit in VALID_EVIDENCE_TYPES:
        return _fill_type_for_closed_order(explicit, trade_status, metadata)

    if _is_near_miss_shadow(metadata):
        return EVIDENCE_NEAR_MISS_SHADOW
    if _is_no_order_shadow(metadata):
        return EVIDENCE_SHADOW_NO_ORDER

    execution_mode = str(metadata.get("execution_mode") or "").strip()
    ibkr_mode = str(metadata.get("ibkr_mode") or metadata.get("trading_mode") or "").strip()
    if execution_mode == "ibkr" and ibkr_mode == "paper":
        return (
            EVIDENCE_IBKR_PAPER_FILL
            if _can_promote_closed_order_to_fill(trade_status, metadata)
            else EVIDENCE_IBKR_PAPER_ORDER
        )
    if execution_mode == "ibkr" and ibkr_mode == "live":
        return (
            EVIDENCE_LIVE_FILL

```

```

        if _can_promote_closed_order_to_fill(trade_status, metadata)
        else EVIDENCE_LIVE_ORDER
    )
    if execution_mode == "paper":
        return (
            EVIDENCE_IBKR_PAPER_FILL
            if _can_promote_closed_order_to_fill(trade_status, metadata)
            else EVIDENCE_IBKR_PAPER_ORDER
        )
    if metadata.get("broker_order_id") or metadata.get("parent_order_id"):
        return (
            EVIDENCE_IBKR_PAPER_FILL
            if _can_promote_closed_order_to_fill(trade_status, metadata)
            else EVIDENCE_IBKR_PAPER_ORDER
        )
    return None

def evidence_quality_for_metadata(metadata: Mapping[str, Any] | None) -> str:
    metadata = metadata or {}
    quality = str(metadata.get("evidence_quality") or "").strip()
    if quality == "normal":
        return EVIDENCE_QUALITY_STANDARD
    return quality or EVIDENCE_QUALITY_STANDARD

def fill_provenance_for_metadata(metadata: Mapping[str, Any] | None) -> str | None:
    metadata = metadata or {}
    provenance = str(metadata.get("fill_provenance") or "").strip()
    if provenance in VALID_FILL_PROVENANCE:
        return provenance
    if _is_near_miss_shadow(metadata) or _is_no_order_shadow(metadata):
        return FILL_PROVENANCE_SHADOW_CLOSE
    return None

def evidence_type_from_metadata(
    metadata: Mapping[str, Any] | None,
    *,
    status: Any = None,
    trade_status: Any = None,
    signal_metadata: Mapping[str, Any] | None = None,
    broker_order_id: str | None = None,
    stored_evidence_type: str | None = None,
    stored_evidence_quality: str | None = None,
) -> str | None:
    merged = dict(signal_metadata or {})
    merged.update(dict(metadata or {}))
    if stored_evidence_type and not merged.get("evidence_type"):
        merged["evidence_type"] = stored_evidence_type
    if stored_evidence_quality and not merged.get("evidence_quality"):
        merged["evidence_quality"] = stored_evidence_quality
    if broker_order_id and not merged.get("broker_order_id"):
        merged["broker_order_id"] = broker_order_id
    return evidence_type_for_metadata(merged, trade_status=trade_status if trade_status is not None else status)

def evidence_quality_from_metadata(metadata: Mapping[str, Any] | None) -> str:
    return evidence_quality_for_metadata(metadata)

def evidence_metadata_with_stored_columns(
    metadata: Mapping[str, Any] | None,
    *,
    evidence_type: str | None = None,
    evidence_quality: str | None = None,
) -> dict[str, Any]:
    updated = dict(metadata or {})
    if evidence_type and not updated.get("evidence_type"):
        updated["evidence_type"] = evidence_type
    if evidence_quality and not updated.get("evidence_quality"):
        updated["evidence_quality"] = evidence_quality
    return updated

def with_evidence_metadata(
    metadata: Mapping[str, Any] | None,
    *,
    evidence_type: str | None = None,
    evidence_quality: str | None = None,
    fill_provenance: str | None = None,
    trade_status: Any = None,
    degradation_reason: str | None = None,
) -> dict[str, Any]:
    updated = dict(metadata or {})
    resolved_type = evidence_type or evidence_type_for_metadata(updated, trade_status=trade_status)
    if resolved_type:
        updated["evidence_type"] = resolved_type
    updated["evidence_quality"] = evidence_quality or evidence_quality_for_metadata(updated)
    if fill_provenance:
        updated["fill_provenance"] = fill_provenance
    if degradation_reason:
        updated["evidence_degradation_reason"] = degradation_reason
    return updated

```



```

def is_director_review_paper_fill_evidence(
    metadata: Mapping[str, Any] | None,
    *,
    trade_status: Any = None,
    signal_metadata: Mapping[str, Any] | None = None,
    broker_order_id: str | None = None,
) -> bool:
    return is_strong_fill_evidence(
        metadata,
        trade_status=trade_status,
        signal_metadata=signal_metadata,
        broker_order_id=broker_order_id,
        allowed_evidence_types=PAPER_FILL_EVIDENCE_TYPES,
    )

def is_paper_order_or_fill_evidence(
    metadata: Mapping[str, Any] | None,
    *,
    trade_status: Any = None,
    signal_metadata: Mapping[str, Any] | None = None,
    broker_order_id: str | None = None,
) -> bool:
    metadata = dict(signal_metadata or {}) | dict(metadata or {})
    if broker_order_id and not metadata.get("broker_order_id"):
        metadata["broker_order_id"] = broker_order_id
    evidence_type = evidence_type_for_metadata(metadata, trade_status=trade_status)
    if evidence_type not in {EVIDENCE_IBKR_PAPER_ORDER, EVIDENCE_IBKR_PAPER_FILL}:
        return False
    if evidence_quality_for_metadata(metadata) == EVIDENCE_QUALITY_DEGRADED:
        return False
    if _is_near_miss_shadow(metadata) or _is_no_order_shadow(metadata):
        return False
    if evidence_type == EVIDENCE_IBKR_PAPER_FILL:
        return is_strong_fill_evidence(
            metadata,
            trade_status=trade_status,
            allowed_evidence_types=PAPER_FILL_EVIDENCE_TYPES,
        )
    if _is_closed_status(trade_status):
        return False
    return True

def is_strong_fill_evidence(
    metadata: Mapping[str, Any] | None,
    *,
    trade_status: Any = None,
    signal_metadata: Mapping[str, Any] | None = None,
    broker_order_id: str | None = None,
    allowed_evidence_types: set[str] | None = None,
) -> bool:
    merged = dict(signal_metadata or {}) | dict(metadata or {})
    if broker_order_id and not merged.get("broker_order_id"):
        merged["broker_order_id"] = broker_order_id
    evidence_type = evidence_type_for_metadata(merged, trade_status=trade_status)
    allowed = allowed_evidence_types or FILL_EVIDENCE_TYPES
    if evidence_type not in allowed:
        return False
    if evidence_quality_for_metadata(merged) not in STRONG_EVIDENCE_QUALITY:
        return False
    if fill_provenance_for_metadata(merged) not in REAL_FILL_PROVENANCE:
        return False
    if _is_near_miss_shadow(merged) or _is_no_order_shadow(merged):
        return False
    return (
        _has_any_metadata_value(merged, FILL_ID_KEYS)
        and _has_any_metadata_value(merged, BROKER_TIMESTAMP_KEYS)
        and _has_any_metadata_value(merged, COMMISSION_KEYS)
    )

def evidence_type_counts(
    items: list[Mapping[str, Any] | None],
    *,
    trade_statuses: list[Any] | None = None,
) -> dict[str, int]:
    counts: dict[str, int] = {}
    for index, metadata in enumerate(items):
        trade_status = trade_statuses[index] if trade_statuses is not None and index < len(trade_statuses) else None
        evidence_type = evidence_type_for_metadata(metadata, trade_status=trade_status) or "unknown"
        counts[evidence_type] = counts.get(evidence_type, 0) + 1
    return counts

def evidence_quality_counts(items: list[Mapping[str, Any] | None]) -> dict[str, int]:
    counts: dict[str, int] = {}
    for metadata in items:
        quality = evidence_quality_for_metadata(metadata)
        counts[quality] = counts.get(quality, 0) + 1
    return counts

def _is_near_miss_shadow(metadata: Mapping[str, Any]) -> bool:
    return bool(metadata.get("near_miss_shadow")) or (
        bool(metadata.get("near_miss")) and bool(metadata.get("no_ibkr_order"))
    )

```

```

)

def _is_no_order_shadow(metadata: Mapping[str, Any]) -> bool:
    return (
        bool(metadata.get("no_ibkr_order"))
        or bool(metadata.get("observation_only"))
        or str(metadata.get("execution_mode") or "") == LAB_SHADOW_EXECUTION_MODE
    )

def _fill_type_for_closed_order(
    evidence_type: str,
    trade_status: Any,
    metadata: Mapping[str, Any],
) -> str:
    if not _is_closed_status(trade_status):
        return evidence_type
    if fill_provenance_for_metadata(metadata) not in REAL_FILL_PROVENANCE:
        return evidence_type
    if evidence_type == EVIDENCE_IBKR_PAPER_ORDER:
        return EVIDENCE_IBKR_PAPER_FILL
    if evidence_type == EVIDENCE_LIVE_ORDER:
        return EVIDENCE_LIVE_FILL
    return evidence_type

def _can_promote_closed_order_to_fill(trade_status: Any, metadata: Mapping[str, Any]) -> bool:
    return _is_closed_status(trade_status) and fill_provenance_for_metadata(metadata) in REAL_FILL_PROVENANCE

def _is_closed_status(value: Any) -> bool:
    status = str(value.value if hasattr(value, "value") else value or "").strip()
    return status == "closed"

def _has_any_metadata_value(metadata: Mapping[str, Any], keys: tuple[str, ...]) -> bool:
    for source in _metadata_sources(metadata):
        for key in keys:
            value = source.get(key)
            if value is not None and str(value).strip() != "":
                return True
    return False

def _metadata_sources(metadata: Mapping[str, Any]) -> list[Mapping[str, Any]]:
    sources: list[Mapping[str, Any]] = [metadata]
    for key in EVIDENCE_NESTED_METADATA_KEYS:
        value = metadata.get(key)
        if isinstance(value, Mapping):
            sources.append(value)
    return sources

```

Full Source: `backend/tradeo/services/execution\_quality.py`

`backend/tradeo/services/execution\_quality.py` ? 215 lines, 8586 bytes, sha256 prefix `7e2bc9d23b6c26b1`

Audit relevance: Execution-quality audit helpers. Important because it declares absence of real microstructure feed and should not be overstated.

Public surface summary before full source:

```

? function `trade_execution_quality` line 76`
? function `persist_execution_quality` line 151`
? function `pattern_execution_quality_summary` line 182`

```

Risk hints: wall-clock timestamp usage; JSON metadata contract; schema drift risk.

""Entry-quality execution audit for real broker fills (informe \$4.3 fallback).

Richer real-time microstructure feeds (bid-ask, depth, order flow) remain unavailable ? there is no provider integrated, so pre-trade microstructure context cannot be captured yet. What *is* verifiable today is the post-trade record the broker already gives us: fill prices, fill/submit timestamps and commissions. This module turns that record into an auditable per-trade entry-quality report:

- ``entry\_slippage\_bps`` / ``entry\_slippage\_r``: achieved entry fill vs the signal's theoretical entry, signed so positive always means "worse than theoretical" regardless of side.
- ``time\_to\_fill\_s``: order submission -> entry fill latency, when both timestamps were recorded.
- ``commission\_bps``: commission relative to the entry fill notional.
- ``estimated\_vs\_realized``: the pre-trade slippage/spread estimates stored on the trade compared with what actually happened, so estimate quality itself becomes auditable.

Every report carries ``data\_basis`` and ``microstructure\_feed`` markers making explicit that no real-time feed backed the numbers ? when a provider lands, the method version must be bumped and the markers updated.

Shadow observations are excluded (their fills are theoretical by

```

construction); only trades with real broker fill provenance produce a report.
Reports are persisted idempotently on ``trade.metadata_json["execution_quality"]``
following the same pattern as ``effective_sample``.
"""

```

```

from __future__ import annotations

from datetime import datetime, timezone
from typing import Any, Iterable

import numpy as np

from tradeo.db.models import Trade
from tradeo.services.evidence import REAL_FILL_PROVENANCE

EXECUTION_QUALITY_METHOD = "fill_vs_signal_eod_v1"
DATA_BASIS = "broker_fill_record_eod_daily"
MICROSTRUCTURE_FEED = "none_available"

__all__ = [
    "EXECUTION_QUALITY_METHOD",
    "trade_execution_quality",
    "persist_execution_quality",
    "pattern_execution_quality_summary",
]

def _side_sign(side: str) -> int:
    return -1 if str(side or "").lower().strip() == "short" else 1

def _parse_ts(value: Any) -> datetime | None:
    if not value:
        return None
    try:
        parsed = datetime.fromisoformat(str(value))
    except (TypeError, ValueError):
        return None
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed

def _positive_float(value: Any) -> float | None:
    try:
        number = float(value)
    except (TypeError, ValueError):
        return None
    return number if number > 0 else None

def trade_execution_quality(trade: Trade) -> dict[str, Any] | None:
    """Entry-quality report for one real-fill trade, or None.

    Returns None when the trade has no real broker fill provenance or lacks
    the prices needed for an honest number (no entry fill, no theoretical
    entry). Open trades are allowed: entry quality is measurable as soon as
    the entry leg fills.
    """
    metadata = trade.metadata_json or {}
    provenance = str(metadata.get("fill_provenance") or "").strip().lower()
    if provenance not in REAL_FILL_PROVENANCE:
        return None
    theoretical_entry = _positive_float(trade.entry)
    fill_entry = _positive_float(metadata.get("entry_fill_price"))
    if theoretical_entry is None or fill_entry is None:
        return None
    sign = _side_sign(trade.side)
    shortfall = (fill_entry - theoretical_entry) * sign
    entry_slippage_bps = shortfall / theoretical_entry * 10_000.0
    risk = abs(theoretical_entry - float(trade.stop or 0.0))
    entry_slippage_r = shortfall / risk if risk > 0 else None

    submitted_at = _parse_ts(metadata.get("submitted_at"))
    fill_time = _parse_ts(
        metadata.get("entry_fill_time") or metadata.get("broker_execution_time")
    )
    time_to_fill_s: float | None = None
    if submitted_at is not None and fill_time is not None:
        delta = (fill_time - submitted_at).total_seconds()
        if delta >= 0:
            time_to_fill_s = round(delta, 3)

    commission_usd: float | None = None
    try:
        commission_usd = float(metadata.get("commission"))
    except (TypeError, ValueError):
        commission_usd = None
    notional = fill_entry * abs(int(trade.qty or 0))
    commission_bps = (
        round(abs(commission_usd) / notional * 10_000.0, 4)
        if commission_usd is not None and notional > 0
        else None
    )

    estimated_slippage = _positive_float(metadata.get("estimated_slippage"))

```

```

estimated_spread_cost = _positive_float(metadata.get("estimated_spread_cost"))
estimated_vs_realized: dict[str, Any] | None = None
if estimated_slippage is not None or estimated_spread_cost is not None:
    estimated_total = (estimated_slippage or 0.0) + (estimated_spread_cost or 0.0)
    estimated_vs_realized = {
        "estimated_slippage_usd": estimated_slippage,
        "estimated_spread_cost_usd": estimated_spread_cost,
        "realized_entry_shortfall_per_share_usd": round(shortfall, 6),
        "estimate_error_per_share_usd": round(shortfall - estimated_total, 6),
    }

return {
    "method": EXECUTION_QUALITY_METHOD,
    "data_basis": DATA_BASIS,
    "microstructure_feed": MICROSTRUCTURE_FEED,
    "trade_id": trade.id,
    "symbol": trade.symbol,
    "side": str(trade.side or ""),
    "fill_provenance": provenance,
    "theoretical_entry": round(theoretical_entry, 6),
    "entry_fill_price": round(fill_entry, 6),
    "entry_slippage_bps": round(entry_slippage_bps, 4),
    "entry_slippage_r": round(entry_slippage_r, 6) if entry_slippage_r is not None else None,
    "time_to_fill_s": time_to_fill_s,
    "commission_usd": commission_usd,
    "commission_bps": commission_bps,
    "estimated_vs_realized": estimated_vs_realized,
}

def persist_execution_quality(trades: Iterable[Trade]) -> int:
    """Store each trade's report in its metadata_json. Returns rows touched.

    Idempotent: an existing record produced by the same method from the same
    fill data is left untouched, so repeated gate evaluations do not churn
    metadata or timestamps.
    """
    now = datetime.now(timezone.utc).isoformat()
    touched = 0
    for trade in trades:
        report = trade_execution_quality(trade)
        if report is None:
            continue
        existing = (trade.metadata_json or {}).get("execution_quality")
        if existing and all(
            existing.get(key) == report[key]
            for key in (
                "method",
                "entry_fill_price",
                "entry_slippage_bps",
                "time_to_fill_s",
                "commission_bps",
            )
        ):
            continue
        record = {**report, "computed_at": now}
        trade.metadata_json = {**(trade.metadata_json or {}), "execution_quality": record}
        touched += 1
    return touched

def pattern_execution_quality_summary(trades: Iterable[Trade]) -> dict[str, Any]:
    """Distribution of entry-quality metrics across a pattern's real fills."""
    rows = [
        report
        for report in (trade_execution_quality(trade) for trade in trades)
        if report is not None
    ]
    base: dict[str, Any] = {
        "method": EXECUTION_QUALITY_METHOD,
        "data_basis": DATA_BASIS,
        "microstructure_feed": MICROSTRUCTURE_FEED,
        "count": len(rows),
        "median_entry_slippage_bps": None,
        "mean_entry_slippage_bps": None,
        "p75_entry_slippage_bps": None,
        "worst_entry_slippage_bps": None,
        "median_time_to_fill_s": None,
        "total_commission_usd": None,
        "per_trade": rows,
    }
    if not rows:
        return base
    slippage = np.asarray([row["entry_slippage_bps"] for row in rows], dtype=float)
    base["median_entry_slippage_bps"] = round(float(np.median(slippage)), 4)
    base["mean_entry_slippage_bps"] = round(float(slippage.mean()), 4)
    base["p75_entry_slippage_bps"] = round(float(np.quantile(slippage, 0.75)), 4)
    base["worst_entry_slippage_bps"] = round(float(slippage.max()), 4)
    fill_times = [row["time_to_fill_s"] for row in rows if row["time_to_fill_s"] is not None]
    if fill_times:
        base["median_time_to_fill_s"] = round(float(np.median(np.asarray(fill_times))), 3)
    commissions = [row["commission_usd"] for row in rows if row["commission_usd"] is not None]
    if commissions:
        base["total_commission_usd"] = round(float(sum(abs(c) for c in commissions)), 4)
    return base

```

Full Source: `backend/tradeo/services/implementation\_shortfall.py`

`backend/tradeo/services/implementation\_shortfall.py` ? 114 lines, 4447 bytes, sha256 prefix `a10181cab4c01213`

Public surface summary before full source:

? function `trade\_slippage\_r` line 42`
? function `pattern\_slippage\_summary` line 94`

Risk hints: JSON metadata contract; schema drift risk.

```
"""Implementation shortfall / slippage_R per trade and per pattern (informe $4.6).
```

```
For every closed trade with a real broker fill we compare the achieved fills
against the theoretical prices the Research labeler assumes:
```

```
    shortfall_entry = (fill_entry - theoretical_entry) * side_sign
    shortfall_exit = (theoretical_exit - fill_exit) * side_sign
    slippage_R      = (shortfall_entry + shortfall_exit) / risk_per_share
```

```
Theoretical entry is the signal entry price; theoretical exit depends on the
exit reason (stop -> stop price, target -> target price, time stop -> the
achieved close, which contributes zero by construction). Shadow observations
are excluded: their fills are theoretical, so their shortfall is zero by
definition and would dilute the distribution the Director gate looks at.
"""
```

```
from __future__ import annotations
```

```
from typing import Any
```

```
import numpy as np
```

```
from tradeo.db.models import Trade
```

```
from tradeo.services.evidence import REAL_FILL_PROVENANCE
```

```
STOP_EXIT_REASONS = {"stop_hit", "stop", "stop_gap", "stopped_out"}
TARGET_EXIT_REASONS = {"target_hit", "target", "target_gap"}
```

```
__all__ = ["trade_slippage_r", "pattern_slippage_summary"]
```

```
def _side_sign(side: str) -> int:
    return -1 if str(side or "").lower().strip() == "short" else 1
```

```
def _has_real_fill(trade: Trade) -> bool:
    metadata = trade.metadata_json or {}
    provenance = str(metadata.get("fill_provenance") or "").strip().lower()
    return provenance in REAL_FILL_PROVENANCE
```

```
def trade_slippage_r(trade: Trade) -> dict[str, Any] | None:
    """Slippage in R for one closed trade with real fills, or None.
```

```
Returns None when the trade has no real broker fill provenance, is not
closed, or lacks the data needed for an honest number (no fill prices,
zero risk per share).
```

```
"""
    if trade.closed_at is None and str(getattr(trade.status, "value", trade.status)) != "closed":
        return None
```

```
    if not _has_real_fill(trade):
        return None
```

```
    metadata = trade.metadata_json or {}
    theoretical_entry = float(trade.entry or 0.0)
    risk = abs(theoretical_entry - float(trade.stop or 0.0))
    if risk <= 0 or theoretical_entry <= 0:
        return None
```

```
    try:
        fill_entry = float(metadata.get("entry_fill_price") or 0.0)
    except (TypeError, ValueError):
        return None
```

```
    if fill_entry <= 0:
        return None
```

```
    fill_exit_raw = metadata.get("exit_fill_price", trade.exit_price)
```

```
    try:
        fill_exit = float(fill_exit_raw or 0.0)
    except (TypeError, ValueError):
        return None
```

```
    if fill_exit <= 0:
        return None
```

```
    exit_reason = str(metadata.get("exit_reason") or "").strip().lower()
```

```
    if exit_reason in STOP_EXIT_REASONS:
```

```
        theoretical_exit = float(trade.stop)
```

```
    elif exit_reason in TARGET_EXIT_REASONS:
```

```
        theoretical_exit = float(trade.target)
```

```
    else:
```

```
        # Time stop or unknown: the labeler assumes exit at the achieved
        # close, so the exit leg contributes no measurable shortfall.
```

```
        theoretical_exit = fill_exit
```

```
    sign = _side_sign(trade.side)
```

```
    shortfall_entry = (fill_entry - theoretical_entry) * sign
```

```

shortfall_exit = (theoretical_exit - fill_exit) * sign
slippage = (shortfall_entry + shortfall_exit) / risk
return {
    "trade_id": trade.id,
    "symbol": trade.symbol,
    "exit_reason": exit_reason or "unknown",
    "shortfall_entry_r": round(shortfall_entry / risk, 6),
    "shortfall_exit_r": round(shortfall_exit / risk, 6),
    "slippage_r": round(float(slippage), 6),
}

def pattern_slippage_summary(trades: list[Trade]) -> dict[str, Any]:
    """Distribution of slippage_R across a pattern's real-fill trades."""
    rows = [row for row in (trade_slippage_r(trade) for trade in trades) if row is not None]
    values = np.asarray([row["slippage_r"] for row in rows], dtype=float)
    if values.size == 0:
        return {
            "count": 0,
            "median_slippage_r": None,
            "mean_slippage_r": None,
            "p75_slippage_r": None,
            "worst_slippage_r": None,
            "per_trade": [],
        }
    return {
        "count": int(values.size),
        "median_slippage_r": round(float(np.median(values)), 6),
        "mean_slippage_r": round(float(values.mean()), 6),
        "p75_slippage_r": round(float(np.quantile(values, 0.75)), 6),
        "worst_slippage_r": round(float(values.max()), 6),
        "per_trade": rows,
    }

```

Full Source: ``backend/tradeo/services/order_outcomes.py``

``backend/tradeo/services/order_outcomes.py`` ? 59 lines, 2278 bytes, sha256 prefix ``f719ddaf4e4f4fdb``

Public surface summary before full source:

? function ``classify_order_failure`` line 9`
? function ``mark_signal_order_failure`` line 52`

Risk hints: wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from datetime import datetime, timezone
from typing import Any

from tradeo.db.models import Signal

def classify_order_failure(error: str) -> dict[str, Any]:
    normalized = error.lower()
    if "did not acknowledge every bracket leg" in normalized or "permid" in normalized:
        reason_code = "ibkr_bracket_not_accepted"
        reason = "IBKR did not acknowledge every bracket leg"
        next_action = "retry_order_submission"
        retryable = True
    elif "could not qualify contract" in normalized:
        reason_code = "ibkr_contract_not_qualified"
        reason = "IBKR could not qualify the contract"
        next_action = "review_symbol_then_retry"
        retryable = False
    elif "readonly=true" in normalized or "blocks order submission" in normalized:
        reason_code = "order_blocked_by_safety"
        reason = "Order submission is blocked by safety settings"
        next_action = "fix_configuration_then_retry"
        retryable = False
    elif "human approval is required" in normalized:
        reason_code = "missing_human_approval"
        reason = "Human approval is required before submission"
        next_action = "approve_then_retry"
        retryable = False
    elif "requires stop" in normalized or "notional" in normalized or "suggested_qty is zero" in normalized:
        reason_code = "invalid_order_parameters"
        reason = "Order parameters failed validation"
        next_action = "rebuild_signal"
        retryable = False
    else:
        reason_code = "broker_submission_failed"
        reason = "Broker submission failed"
        next_action = "retry_order_submission"
        retryable = True

    return {
        "status": next_action,
        "reason_code": reason_code,
        "reason": reason,
        "retryable": retryable,
        "next_action": next_action,
        "error": error,
    }

```

```

}

def mark_signal_order_failure(signal: Signal, error: str) -> dict[str, Any]:
    outcome = classify_order_failure(error)
    outcome["updated_at"] = datetime.now(timezone.utc).isoformat()
    metadata = dict(signal.metadata_json or {})
    metadata["execution_outcome"] = outcome
    metadata["last_order_error"] = error
    signal.metadata_json = metadata
    return outcome

```

Full Source: ``backend/tradeo/services/reconciliation.py``

``backend/tradeo/services/reconciliation.py`` ? 266 lines, 11091 bytes, sha256 prefix ``01d95ed7b6231917``

Audit relevance: DB to IBKR reconciliation and automatic kill switch. Audit divergence handling, pending orders, zero quantities, and fail-closed behavior.

Public surface summary before full source:

? class ``ReconciliationService`` line 79 methods: reconcile

Risk hints: broad ``except Exception`` present; wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

"""DB <-> IBKR reconciliation with automatic kill switch (informe \$4.5).

On every run we compare what the database believes is open against what IBKR reports (positions + open orders). Any confirmed divergence means the system's model of its own exposure is wrong, which is exactly the failure mode the kill switch exists for ? so divergence activates the runtime kill switch and the order paths stop until a human investigates.

Beyond symbol existence, reconciliation is explicit about **how much** and **which order**:

- ``position_qty_mismatch_at_broker``: per symbol, the signed broker position is compared against the signed quantity the DB expects (sum of open IBKR trades, shorts negative). A position larger than expected, on the wrong side, or smaller with no open orders that could still fill it, is a divergence. A smaller-than-expected position **with** pending open orders on the symbol is the normal partial-entry state, so it is recorded as a warning, not a divergence.
- ``db_order_id_missing_at_broker``: a DB trade whose symbol shows up at the broker only through open orders must find its ``broker_order_id`` among those orders (as order id, perm id, or the parent id bracket children point to); otherwise the orders the broker holds are not the orders we think we placed.
- ``partial_fill_open_order``: an open order with ``0 < filled < quantity`` is a legitimate transient state, never a divergence ? it is surfaced as a warning and audited so partial fills are observable instead of silent.

Connection failures are NOT divergences: the broker being unreachable proves nothing about state mismatch. They are audited and surfaced, never escalated to the kill switch automatically. Warnings likewise never trip the kill switch.

```

from __future__ import annotations

```

```

from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any

```

```

from loguru import logger
from sqlalchemy.orm import Session

```

```

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, Trade, TradeStatus
from tradeo.services.ibkr_broker import IBKRBroker
from tradeo.services.system_controls import (
    activate_runtime_kill_switch,
    runtime_kill_switch_active,
)

```

```

__all__ = ["ReconciliationService"]

```

```

def _is_ibkr_trade(trade: Trade) -> bool:
    metadata = trade.metadata_json or {}
    return str(metadata.get("execution_mode") or "") == "ibkr"

```

```

def _signed_qty(trade: Trade) -> float:
    sign = -1.0 if str(trade.side or "").lower().strip() == "short" else 1.0
    return sign * abs(float(trade.qty or 0))

```

```

def _order_matches_trade(order: dict[str, Any], broker_order_id: str) -> bool:
    """True when an open-order row references the trade's broker order id.

```

Bracket children carry their own ids but point back to the parent via ``parent\_order\_id``, so a trade whose parent leg already filled is still matched through its surviving child legs.

"""

```
for key in ("order_id", "perm_id", "parent_order_id"):
    value = order.get(key)
    if value is not None and str(value) == broker_order_id:
        return True
return False
```

```
@dataclass(slots=True)
class ReconciliationService:
    settings: Settings | None = None
    broker: IBKRBroker | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()
        self.broker = self.broker or IBKRBroker(self.settings)

    def reconcile(self, db: Session) -> dict[str, Any]:
        settings = self.settings
        assert settings is not None
        now = datetime.now(timezone.utc)
        open_trades = (
            db.query(Trade)
            .filter(Trade.status == TradeStatus.OPEN)
            .all()
        )
        db_ibkr_trades = [trade for trade in open_trades if _is_ibkr_trade(trade)]
        result: dict[str, Any] = {
            "ok": True,
            "checked_at": now.isoformat(),
            "db_open_ibkr_trades": len(db_ibkr_trades),
            "broker_positions": 0,
            "broker_open_orders": 0,
            "divergences": [],
            "warnings": [],
            "kill_switch_activated": False,
            "kill_switch_already_active": runtime_kill_switch_active(db),
        }
        try:
            assert self.broker is not None
            positions = self.broker.positions()
            open_orders = self.broker.open_orders()
        except Exception as exc: # noqa: BLE001
            # Unreachable broker is an availability problem, not a state
            # divergence; never auto-trip the kill switch on it.
            logger.warning("reconciliation skipped: broker unreachable: {}", exc)
            db.add(
                AuditLog(
                    actor="reconciliation",
                    action="reconciliation_broker_unreachable",
                    entity_type="system",
                    entity_id="ibkr",
                    details_json={
                        "error": f"{type(exc).__name__}: {exc}",
                        "db_open_ibkr_trades": len(db_ibkr_trades),
                    },
                )
            )
            db.commit()
            return {"**result", "ok": False, "error": f"broker_unreachable: {exc}"}

        position_qty: dict[str, float] = {}
        for row in positions:
            symbol = str(row.get("symbol") or "").upper()
            qty = float(row.get("position") or 0.0)
            if symbol and abs(qty) > 0:
                position_qty[symbol] = position_qty.get(symbol, 0.0) + qty
        position_symbols = set(position_qty)
        orders_by_symbol: dict[str, list[dict[str, Any]]] = {}
        for row in open_orders:
            symbol = str(row.get("symbol") or "").upper()
            orders_by_symbol.setdefault(symbol, []).append(row)
        order_symbols = set(orders_by_symbol)
        db_symbols = {trade.symbol.upper() for trade in db_ibkr_trades}
        result["broker_positions"] = len(position_symbols)
        result["broker_open_orders"] = len(open_orders)

        divergences: list[dict[str, Any]] = []
        warnings: list[dict[str, Any]] = []
        for trade in db_ibkr_trades:
            symbol = trade.symbol.upper()
            if symbol not in position_symbols and symbol not in order_symbols:
                divergences.append(
                    {
                        "kind": "db_open_trade_missing_at_broker",
                        "symbol": symbol,
                        "trade_id": trade.id,
                        "broker_order_id": trade.broker_order_id,
                    }
                )
        elif (
            symbol not in position_symbols
            and trade.broker_order_id
```



```

        and not any(
            _order_matches_trade(order, str(trade.broker_order_id))
            for order in orders_by_symbol.get(symbol, [])
        )
    ):
        divergences.append(
            {
                "kind": "db_order_id_missing_at_broker",
                "symbol": symbol,
                "trade_id": trade.id,
                "broker_order_id": trade.broker_order_id,
                "broker_open_order_ids": sorted(
                    str(order.get("order_id"))
                    for order in orders_by_symbol.get(symbol, [])
                    if order.get("order_id") is not None
                ),
            }
        )
    for symbol in sorted(position_symbols - db_symbols):
        divergences.append(
            {
                "kind": "broker_position_not_in_db",
                "symbol": symbol,
            }
        )

    expected_qty: dict[str, float] = {}
    for trade in db_ibkr_trades:
        symbol = trade.symbol.upper()
        expected_qty[symbol] = expected_qty.get(symbol, 0.0) + _signed_qty(trade)
    for symbol in sorted(position_symbols & set(expected_qty)):
        broker_qty = position_qty[symbol]
        db_qty = expected_qty[symbol]
        if broker_qty == db_qty:
            continue
        detail = {
            "symbol": symbol,
            "db_signed_qty": db_qty,
            "broker_signed_qty": broker_qty,
        }
        wrong_side = (broker_qty > 0) != (db_qty > 0)
        larger_than_expected = abs(broker_qty) > abs(db_qty)
        if not wrong_side and not larger_than_expected and symbol in order_symbols:
            # Smaller position with orders still working: the normal
            # partial-entry state, observable but not a divergence.
            warnings.append(
                {"kind": "position_qty_below_db_pending_orders", **detail}
            )
            continue
        divergences.append({"kind": "position_qty_mismatch_at_broker", **detail})

    for symbol in sorted(order_symbols):
        for order in orders_by_symbol[symbol]:
            try:
                filled = float(order.get("filled") or 0.0)
                quantity = float(order.get("quantity") or 0.0)
            except (TypeError, ValueError):
                continue
            if 0 < filled < quantity:
                warnings.append(
                    {
                        "kind": "partial_fill_open_order",
                        "symbol": symbol,
                        "order_id": order.get("order_id"),
                        "filled": filled,
                        "quantity": quantity,
                        "remaining": order.get("remaining"),
                    }
                )
    result["divergences"] = divergences
    result["warnings"] = warnings

    db.add(
        AuditLog(
            actor="reconciliation",
            action="reconciliation_completed",
            entity_type="system",
            entity_id="ibkr",
            details_json={
                "db_open_ibkr_trades": len(db_ibkr_trades),
                "broker_positions": sorted(position_symbols),
                "broker_open_order_count": len(open_orders),
                "divergence_count": len(divergences),
                "divergences": divergences,
                "warning_count": len(warnings),
                "warnings": warnings,
            },
        )
    )
    if divergences and settings.reconciliation_auto_kill_switch:
        activate_runtime_kill_switch(
            db,
            reason="reconciliation divergence between DB open trades and IBKR state",
            actor="reconciliation",
            details={"divergences": divergences},
        )

```

```

        result["kill_switch_activated"] = True
        logger.error(
            "reconciliation divergence: kill switch activated ({} divergences)",
            len(divergences),
        )
    else:
        db.commit()
    return result

```

Full Source: `backend/tradeo/services/risk\_manager.py`

`backend/tradeo/services/risk\_manager.py` ? 171 lines, 7490 bytes, sha256 prefix `ba9ca319a64f1517`

Audit relevance: Sizing and portfolio risk controls. Audit ADV participation, pattern-family exposure, daily loss, short enablement, and default limits.

Public surface summary before full source:

? class `AccountState` line 14 methods: ?

? class `RiskManager` line 22 methods: `account_state`, `validate_candidate`, `approve_signal_for_live`

Risk hints: wall-clock timestamp usage; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timezone

from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import RiskLedger, Signal, SignalStatus, Trade, TradeStatus
from tradeo.schemas import PatternCandidate, RiskDecision

@dataclass
class AccountState:
    equity: float
    cash: float
    open_positions: int
    open_risk: float
    realized_pnl_today: float

class RiskManager:
    """Hard risk gate for every candidate.

    With 3,000 USD starting capital and 1% risk, max risk per trade is 30 USD.
    The manager rejects candidates before any broker adapter can see them.
    """

    def __init__(self, settings: Settings | None = None) -> None:
        self.settings = settings or get_settings()

    def account_state(self, db: Session) -> AccountState:
        open_trades = db.query(Trade).filter(Trade.status == TradeStatus.OPEN).all()
        open_positions = len(open_trades)
        open_risk = sum(abs(t.entry - t.stop) * t.qty for t in open_trades)
        today = datetime.now(timezone.utc).date().isoformat()
        ledger = db.query(RiskLedger).filter(RiskLedger.day == today).one_or_none()
        realized = ledger.realized_pnl if ledger else 0.0
        # v0 uses configured capital as equity until a broker sync or paper fills update it.
        equity = self.settings.initial_capital_usd + realized
        cash = equity
        return AccountState(equity=equity, cash=cash, open_positions=open_positions, open_risk=open_risk, realized_pnl_today=realized)

    def validate_candidate(self, candidate: PatternCandidate, db: Session | None = None) -> RiskDecision:
        hard: list[str] = []
        warnings: list[str] = []
        settings = self.settings
        equity = settings.initial_capital_usd
        open_positions = 0
        realized_today = 0.0
        if db is not None:
            state = self.account_state(db)
            equity = state.equity
            open_positions = state.open_positions
            realized_today = state.realized_pnl_today

        if settings.kill_switch_enabled:
            hard.append("kill_switch_enabled")
        if candidate.side == "long" and not settings.allow longs:
            hard.append("longs_disabled")
        if candidate.side == "short" and not settings.allow shorts:
            hard.append("shorts_disabled")
        if candidate.reward_risk < settings.min_reward_risk:
            hard.append(f"reward_risk_below_{settings.min_reward_risk}")
        if open_positions >= settings.max_open_positions:
            hard.append("max_open_positions_reached")
        if realized_today <= -settings.initial_capital_usd * settings.daily_loss_limit_pct:
            hard.append("daily_loss_limit_reached")
        if db is not None:

```

```

        family_key = self._candidate_family_key(candidate)
        if family_key and self._open_family_positions(db, family_key) >= settings.max_open_positions_per_pattern_family:
            hard.append(
                f"pattern_family_position_cap_{settings.max_open_positions_per_pattern_family}_reached"
            )

    risk_per_share = abs(candidate.entry - candidate.stop)
    if risk_per_share <= 0:
        hard.append("invalid_stop_distance")
        return RiskDecision(approved=False, reason=";".join(hard), hard_rejections=hard, warnings=warnings)

    risk_budget = equity * settings.risk_per_trade_pct
    qty_by_risk = int(risk_budget // risk_per_share)
    max_position_value = equity * settings.max_position_value_pct
    qty_by_notional = int(max_position_value // candidate.entry)
    qty_caps = [qty_by_risk, qty_by_notional]
    adv_cap = self._qty_cap_by_adv(candidate)
    if adv_cap is not None:
        qty_caps.append(adv_cap)
    qty = max(0, min(qty_caps))
    risk_usd = round(qty * risk_per_share, 2)
    notional = round(qty * candidate.entry, 2)

    if qty <= 0:
        hard.append("position_size_zero")
    if notional > equity * settings.max_gross_exposure_pct and not settings.allow_margin:
        hard.append("gross_exposure_exceeds_cash_limit")
    if risk_usd > risk_budget * 1.0001:
        hard.append("risk_budget_exceeded")
    if adv_cap is not None and qty_by_risk > adv_cap:
        warnings.append(f"adv_participation_cap_reduced_qty_to_{adv_cap}")
    if candidate.features.get("avg_dollar_volume", 0) < settings.min_avg_dollar_volume:
        hard.append("liquidity_filter_failed")
    if candidate.features.get("atr_pct", 0) > settings.max_atr_pct:
        hard.append("atr_filter_failed")
    if candidate.confidence < 0.68:
        warnings.append("confidence_under_preferred_threshold")

    approved = not hard
    reason = "approved" if approved else ";".join(hard)
    return RiskDecision(
        approved=approved,
        suggested_qty=qty,
        risk_usd=risk_usd,
        notional_usd=notional,
        reason=reason,
        hard_rejections=hard,
        warnings=warnings,
    )

def approve_signal_for_live(self, signal: Signal) -> bool:
    return (
        signal.status == SignalStatus.PENDING_HUMAN_APPROVAL
        and signal.human_approved
        and self.settings.live_armed
    )

def _qty_cap_by_adv(self, candidate: PatternCandidate) -> int | None:
    avg_dollar_volume = self._safe_float(candidate.features.get("avg_dollar_volume"))
    if avg_dollar_volume <= 0 or candidate.entry <= 0:
        return None
    max_participation = max(0.0, float(self.settings.max_adv_participation_pct))
    if max_participation <= 0:
        return None
    adv_shares = avg_dollar_volume / candidate.entry
    return int(adv_shares * max_participation)

@staticmethod
def _candidate_family_key(candidate: PatternCandidate) -> str:
    for key in ("pattern_family_key", "canonical_pattern_key", "pattern_key", "pattern_id"):
        value = str(candidate.features.get(key) or "").strip()
        if value:
            return value
    return ""

@classmethod
def _open_family_positions(cls, db: Session, family_key: str) -> int:
    open_trades = db.query(Trade).filter(Trade.status == TradeStatus.OPEN).all()
    count = 0
    for trade in open_trades:
        signal_metadata = trade.signal.metadata_json if trade.signal is not None else {}
        trade_metadata = trade.metadata_json or {}
        keys = {
            str(signal_metadata.get("pattern_family_key") or "").strip(),
            str(signal_metadata.get("canonical_pattern_key") or "").strip(),
            str(signal_metadata.get("pattern_key") or "").strip(),
            str(signal_metadata.get("pattern_id") or "").strip(),
            str(trade_metadata.get("pattern_family_key") or "").strip(),
            str(trade_metadata.get("canonical_pattern_key") or "").strip(),
            str(trade_metadata.get("pattern_key") or "").strip(),
            str(trade_metadata.get("pattern_id") or "").strip(),
        }
        if family_key in keys:
            count += 1
    return count

```

```

@staticmethod
def _safe_float(value: object) -> float:
    try:
        return float(value)
    except (TypeError, ValueError):
        return 0.0

```

Full Source: `backend/tradeo/services/pattern\_entry\_scanner.py`

`backend/tradeo/services/pattern\_entry\_scanner.py` ? 8 lines, 252 bytes, sha256 prefix `73e12326dcdcf410`

```

"""Backward-compatible imports for the shared entry scanner.

New code should import from ``tradeo.modules.shared.entry_scanner`` or through
the Laboratory/FoxHunter domain facades.
"""

from tradeo.modules.shared.entry_scanner import * # noqa: F403

```

Full Source: `backend/tradeo/modules/shared/entry\_scanner.py`

`backend/tradeo/modules/shared/entry\_scanner.py` ? 1341 lines, 58232 bytes, sha256 prefix `5f3a039c47ba6b58`

Audit relevance: Actual PatternEntryScanner implementation behind service facade. Audit lab/fox mode separation, shadow/paper/live guardrails, near-miss recording, IBKR submission gates, and evidence metadata.

Public surface summary before full source:

? class `PatternEntryScannerSafetyError` line 52 methods: ?
 ? class `PatternEntryScanner` line 57 methods: scan, status

Risk hints: broad `except Exception` present; wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from typing import Any, Literal

from sqlalchemy.orm import Session, joinedload

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, DiscoveredPattern, Signal, SignalStatus, Trade, TradeStatus
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher
from tradeo.schemas import PatternCandidate
from tradeo.services.evidence import (
    EvidenceQuality,
    EvidenceType,
    evidence_metadata_with_stored_columns,
    is_director_review_paper_fill_evidence,
)
from tradeo.services.ibkr_broker import IBKRBroker
from tradeo.modules.laboratory.paper_observations import LAB_SHADOW_EXECUTION_MODE, LabPaperObservationService
from tradeo.services.market_session import market_session_status
from tradeo.services.order_outcomes import mark_signal_order_failure
from tradeo.services.opportunity_ranking import rank_entry_matches
from tradeo.modules.fox_hunter.production_manifest import production_manifest_is_active
from tradeo.services.risk_manager import RiskManager
from tradeo.services.signal_quality import build_entry_quality, build_signal_snapshot

EntryModule = Literal["laboratory", "fox_hunter"]
LAB_NEAR_MISS_VOLUME_REASONS = {"insufficient_volume", "weak_volume_confirmation"}
LAB_NEAR_MISS_HARD_REASONS = {
    "weak_trigger",
    "weak_entry_score",
    "no_operational_trigger",
    "insufficient_history",
    "regime_not_aligned",
    "regime_filter_failed",
    "excessive_extension",
    "overextended",
    "excessive_volatility",
    "volatility_filter_failed",
    "atr_filter_failed",
    "liquidity_filter_failed",
    "thin_liquidity",
    "trigger_not_confirmed",
}

LAB_NEAR_MISS_TOP_RANK_LIMIT = 3
LAB_NEAR_MISS_MIN_ENTRY_SCORE = 0.50
LAB_NEAR_MISS_MIN_RANK_SCORE = 0.45
LAB_NEAR_MISS_HIGH_RANK_SCORE = 0.60

```

```

class PatternEntryScannerSafetyError(RuntimeError):
    """Raised when a scanner tries to cross its allowed execution boundary."""

```

```

@dataclass(slots=True)
class PatternEntryScanner:
    """Operational scanner for validated Research patterns.

    Research discovers patterns. Laboratory validates them with IB paper fills.
    Fox Hunter only watches production patterns and can route live orders only
    after the existing live_armed gate is explicitly enabled.
    """

    settings: Settings | None = None
    matcher: NovelPatternMatcher | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()
        self.matcher = self.matcher or NovelPatternMatcher(settings=self.settings)

    def scan(
        self,
        db: Session,
        *,
        module: EntryModule,
        symbols: list[str] | None = None,
        limit: int | None = None,
        max_patterns: int | None = None,
        similarity_threshold: float | None = None,
        store_signals: bool | None = None,
        execute_orders: bool | None = None,
    ) -> dict[str, Any]:
        settings = self.settings
        assert settings is not None
        self._validate_module_execution(module, execute_orders=execute_orders)
        resolved = self._resolved_options(
            module,
            limit=limit,
            max_patterns=max_patterns,
            similarity_threshold=similarity_threshold,
            store_signals=store_signals,
            execute_orders=execute_orders,
        )
        observation_lifecycle = self._close_lab_observations(db, module)
        session = market_session_status()
        if self._requires_market_hours(module) and not bool(session["regular_session_open"]):
            result = self._market_closed_result(module, resolved, session)
            result["paper_observations_opened"] = 0
            result["paper_observations_closed"] = observation_lifecycle["closed_observations"]
            result["paper_observation_trade_ids"] = []
            result["shadow_no_order_observations_opened"] = 0
            result["shadow_no_order_trade_ids"] = []
            result["paper_observation_lifecycle"] = observation_lifecycle
            from tradeo.services.runtime_status import write_entry_scan_status

            write_entry_scan_status(module, result, settings)
            return result
        assert self.matcher is not None
        match_result = self.matcher.match_current(
            db,
            symbols=symbols,
            limit=resolved["limit"],
            max_patterns=resolved["max_patterns"],
            similarity_threshold=resolved["similarity_threshold"],
            module=module,
            store=True,
        )
        signals_created = 0
        orders_submitted = 0
        skipped_duplicates = 0
        skipped_cooldown = 0
        rejected_by_entry_gate = 0
        rejected_by_entry_quality = 0
        rejected_by_risk = 0
        rejected_by_production_manifest = 0
        near_miss_shadow_observations_opened = 0
        order_errors: list[dict[str, Any]] = []
        signal_ids: list[int] = []
        near_miss_signal_ids: list[int] = []
        trade_ids: list[int] = []
        paper_observation_trade_ids: list[int] = []
        near_miss_trade_ids: list[int] = []
        paper_observations_opened = 0
        shadow_no_order_observations_opened = 0
        shadow_no_order_trade_ids: list[int] = []

        all_ranked_matches = rank_entry_matches(
            match_result["matches"],
            settings=settings,
            execution_history=self._execution_history(db, module),
        )
        ranked_matches = self._select_best_variant_per_exposure(all_ranked_matches)

        for match in ranked_matches:
            if module == "fox_hunter" and not self._fox_match_has_active_manifest(db, match):
                rejected_by_production_manifest += 1
                db.add(
                    AuditLog(
                        actor=module,
                        action="entry_match_rejected_by_production_manifest",
                    )

```

```

        entity_type="discovered_pattern_match",
        entity_id=str(match.get("pattern_id", "")),
        details_json={
            "match": match,
            "reason": "active_production_manifest_required",
        },
    )
)
db.commit()
continue
entry_gate = ((match.get("metrics") or {}).get("entry_gate") or {})
if settings.entry_gate_enabled and not bool(entry_gate.get("passed", False)):
    rejected_by_entry_gate += 1
    self._audit_entry_gate_rejection(db, module=module, match=match, entry_gate=entry_gate)
    if module == "laboratory" and self._is_lab_near_miss_shadow_candidate(match, entry_gate):
        opened = self._open_near_miss_shadow_observation(
            db,
            match=match,
            resolved=resolved,
            session=session,
        )
        if opened is not None:
            signal, observation = opened
            signals_created += 1
            signal_ids.append(signal.id)
            near_miss_signal_ids.append(signal.id)
            paper_observations_opened += 1
            near_miss_shadow_observations_opened += 1
            paper_observation_trade_ids.append(observation.id)
            near_miss_trade_ids.append(observation.id)
            continue
        continue
    if self._has_active_exposure(db, match, module=module):
        skipped_duplicates += 1
        continue
    if self._has_duplicate_signal(db, match, module=module):
        skipped_duplicates += 1
        db.add(
            AuditLog(
                actor=module,
                action="entry_match_skipped_idempotent",
                entity_type="discovered_pattern_match",
                entity_id=str(match.get("pattern_id", "")),
                details_json={
                    "signal_idempotency_key": self._signal_idempotency_key(module, match),
                    "reason": "signal_already_exists_for_pattern_symbol_variant_bar",
                },
            )
        )
    db.commit()
    continue
if self._has_recent_signal(db, match, module=module):
    skipped_cooldown += 1
    db.add(
        AuditLog(
            actor=module,
            action="entry_match_skipped_by_cooldown",
            entity_type="discovered_pattern_match",
            entity_id=str(match.get("pattern_id", "")),
            details_json={
                "match": match,
                "cooldown_minutes": settings.entry_cooldown_minutes,
            },
        )
    )
    db.commit()
    continue
candidate = self._candidate_from_match(match)
risk = RiskManager(settings).validate_candidate(candidate, db)
if not risk.approved:
    rejected_by_risk += 1
    db.add(
        AuditLog(
            actor=module,
            action="entry_match_rejected_by_risk",
            entity_type="discovered_pattern_match",
            entity_id=str(match.get("pattern_id", "")),
            details_json={"match": match, "risk": risk.model_dump(mode="json")},
        )
    )
    db.commit()
    continue
entry_quality = build_entry_quality(
    match=match,
    risk=risk,
    settings=settings,
    execution_requested=bool(resolved["execute_orders"]),
    market_session=session,
)
if entry_quality["score"] < settings.entry_min_quality_score or entry_quality["flags"]:
    rejected_by_entry_quality += 1
    db.add(
        AuditLog(
            actor=module,
            action="entry_match_rejected_by_quality",
            entity_type="discovered_pattern_match",

```

```

        entity_id=str(match.get("pattern_id", "")),
        details_json={
            "match": match,
            "entry_quality": entry_quality,
            "min_quality_score": settings.entry_min_quality_score,
        },
    )
)
db.commit()
continue
if not resolved["store_signals"]:
    continue

signal = self._store_signal(
    db,
    module=module,
    match=match,
    candidate=candidate,
    risk=risk,
    execute_orders=bool(resolved["execute_orders"]),
    market_session=session,
    entry_quality=entry_quality,
)
signals_created += 1
signal_ids.append(signal.id)

if not resolved["execute_orders"]:
    if module == "laboratory":
        observation = self._open_lab_observation(
            db,
            signal=signal,
            match=match,
            risk=risk,
        )
        if observation is not None:
            paper_observations_opened += 1
            shadow_no_order_observations_opened += 1
            paper_observation_trade_ids.append(observation.id)
            shadow_no_order_trade_ids.append(observation.id)
        continue

try:
    trade = IBKRBroker(settings).submit_signal_bracket(
        db,
        signal,
        reason=self._execution_reason(module),
    )
    orders_submitted += 1
    trade_ids.append(trade.id)
except Exception as exc: # noqa: BLE001
    outcome = mark_signal_order_failure(signal, str(exc))
    order_errors.append(
        {
            "signal_id": signal.id,
            "symbol": signal.symbol,
            "error": str(exc),
            "reason_code": outcome["reason_code"],
            "retryable": outcome["retryable"],
            "next_action": outcome["next_action"],
        }
    )
    db.add(
        AuditLog(
            actor=module,
            action="entry_order_submission_failed",
            entity_type="signal",
            entity_id=str(signal.id),
            details_json={"error": str(exc), "outcome": outcome, "match": match},
        )
    )
    db.commit()

result = {
    "module": module,
    "patterns_checked": match_result["patterns_checked"],
    "symbols_checked": match_result["symbols_checked"],
    "matches_found": len(ranked_matches),
    "entry_variants_considered": len(all_ranked_matches),
    "signals_created": signals_created,
    "orders_submitted": orders_submitted,
    "skipped_duplicates": skipped_duplicates,
    "skipped_cooldown": skipped_cooldown,
    "rejected_by_entry_gate": rejected_by_entry_gate,
    "rejected_by_entry_quality": rejected_by_entry_quality,
    "rejected_by_risk": rejected_by_risk,
    "rejected_by_production_manifest": rejected_by_production_manifest,
    "near_miss_shadow_observations_opened": near_miss_shadow_observations_opened,
    "order_errors": order_errors,
    "signal_ids": signal_ids,
    "near_miss_signal_ids": near_miss_signal_ids,
    "trade_ids": trade_ids,
    "paper_observations_opened": paper_observations_opened,
    "paper_observations_closed": observation_lifecycle["closed_observations"],
    "paper_observation_trade_ids": paper_observation_trade_ids,
    "near_miss_trade_ids": near_miss_trade_ids,
    "shadow_no_order_observations_opened": shadow_no_order_observations_opened,
}

```

```

        "shadow_no_order_trade_ids": shadow_no_order_trade_ids,
        "paper_observation_lifecycle": observation_lifecycle,
        "top_opportunities": self._top_opportunities(ranked_matches),
        "store_signals": resolved["store_signals"],
        "execute_orders": resolved["execute_orders"],
        "similarity_threshold": match_result["similarity_threshold"],
        "generated_at": datetime.now(timezone.utc).isoformat(),
    }
}
from tradeo.services.runtime_status import write_entry_scan_status

write_entry_scan_status(module, result, settings)
return result

def status(self, db: Session) -> dict[str, Any]:
    from tradeo.db.models import DiscoveredPattern, DiscoveredPatternStatus
    from tradeo.services.runtime_status import entry_scan_status, worker_runtime_status

    lab_statuses = NovelPatternMatcher._statuses_for_module("laboratory")
    laboratory_patterns = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.validation_passed.is_(True))
        .filter(DiscoveredPattern.status.in_(lab_statuses))
        .count()
    )
    legacy_runtime_blocked_patterns = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.validation_passed.is_(True))
        .filter(
            DiscoveredPattern.status.in_(
                [
                    DiscoveredPatternStatus.PREMIUM_CANDIDATE,
                    DiscoveredPatternStatus.PAPER_CANDIDATE,
                ]
            )
        )
        .count()
    )
    production_patterns = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.validation_passed.is_(True))
        .filter(DiscoveredPattern.status == DiscoveredPatternStatus.PRODUCTION)
        .all()
    )
    production_manifest_patterns = [
        pattern for pattern in production_patterns if production_manifest_is_active(pattern)
    ]
    settings = self.settings
    assert settings is not None
    worker_status = worker_runtime_status(settings)
    session = market_session_status()
    market_open = bool(session["regular_session_open"])
    laboratory_market_ok = market_open or not settings.laboratory_market_hours_only
    fox_market_ok = market_open or not settings.fox_hunter_market_hours_only
    laboratory_ok = (
        settings.scheduler_enabled
        and settings.laboratory_scanner_enabled
        and bool(worker_status["ok"])
        and laboratory_market_ok
    )
    fox_hunter_ok = (
        settings.scheduler_enabled
        and settings.fox_hunter_enabled
        and bool(worker_status["ok"])
        and fox_market_ok
    )
    laboratory_state = "ok" if laboratory_ok else "market_closed" if not laboratory_market_ok else "stopped"
    fox_state = "ok" if fox_hunter_ok else "market_closed" if not fox_market_ok else "stopped"
    laboratory_entry_status = entry_scan_status("laboratory", settings)
    fox_entry_status = entry_scan_status("fox_hunter", settings)
    paper_order_safety_ok = (
        not settings.kill_switch_enabled
        and settings.trading_mode == "paper"
        and not settings.live_armed
        and int(settings.ibkr_port) not in {7496, 4001}
    )
    paper_orders_allowed = settings.laboratory_auto_submit_paper_orders and paper_order_safety_ok
    live_orders_allowed = (
        settings.fox_hunter_auto_submit_live_orders
        and settings.live_armed
        and not settings.kill_switch_enabled
    )
    return {
        "research": {"purpose": "discover_new_patterns"},
        "laboratory": {
            "purpose": "paper_validate_research_patterns",
            "enabled": settings.laboratory_scanner_enabled,
            "operational_ok": laboratory_ok,
            "state": laboratory_state,
            "market_session": session,
            "worker": worker_status,
            "auto_submit_paper_orders": settings.laboratory_auto_submit_paper_orders,
            "paper_orders_allowed": paper_orders_allowed,
            "paper_order_safety_ok": paper_order_safety_ok,
            "blocked_but_healthy": laboratory_ok and not paper_orders_allowed,
            "execution_block_reason": None
            if paper_orders_allowed
        },
        "fox_hunter": {
            "purpose": "fox_hunter_validate_research_patterns",
            "enabled": settings.fox_hunter_scanner_enabled,
            "operational_ok": fox_hunter_ok,
            "state": fox_state,
            "market_session": session,
            "worker": worker_status,
            "auto_submit_live_orders": settings.fox_hunter_auto_submit_live_orders,
            "live_orders_allowed": live_orders_allowed,
            "paper_order_safety_ok": paper_order_safety_ok,
            "blocked_but_healthy": fox_hunter_ok and not live_orders_allowed,
            "execution_block_reason": None
            if live_orders_allowed
        },
    }

```



```

        else self._paper_order_block_reason(settings, paper_order_safety_ok),
        "eligible_patterns": laboratory_patterns,
        "legacy_runtime_blocked_patterns": legacy_runtime_blocked_patterns,
        "symbols_checked": laboratory_entry_status["symbols_checked"],
        "last_symbols_checked": laboratory_entry_status["last_symbols_checked"],
    },
    "fox_hunter": {
        "purpose": "live_trade_production_patterns",
        "enabled": settings.fox_hunter_enabled,
        "operational_ok": fox_hunter_ok,
        "state": fox_state,
        "market_session": session,
        "worker": worker_status,
        "auto_submit_live_orders": settings.fox_hunter_auto_submit_live_orders,
        "live_orders_allowed": live_orders_allowed,
        "blocked_but_healthy": fox_hunter_ok and not live_orders_allowed,
        "execution_block_reason": None
        if live_orders_allowed
        else self._live_order_block_reason(settings),
        "eligible_patterns": len(production_manifest_patterns),
        "production_status_patterns": len(production_patterns),
        "production_manifest_required": True,
        "production_manifest_blocked_patterns": len(production_patterns)
        - len(production_manifest_patterns),
        "symbols_checked": fox_entry_status["symbols_checked"],
        "last_symbols_checked": fox_entry_status["last_symbols_checked"],
        "live_armed": settings.live_armed,
    }
}

@staticmethod
def _paper_order_block_reason(settings: Settings, paper_order_safety_ok: bool) -> str:
    if not settings.laboratory_auto_submit_paper_orders:
        return "paper_auto_submit_disabled"
    if settings.kill_switch_enabled:
        return "kill_switch_enabled"
    if settings.trading_mode != "paper":
        return "trading_mode_not_paper"
    if settings.live_armed:
        return "live_armed_blocks_laboratory"
    if not paper_order_safety_ok:
        return "ibkr_live_port_blocked"
    return "unknown"

@staticmethod
def _live_order_block_reason(settings: Settings) -> str:
    if not settings.fox_hunter_auto_submit_live_orders:
        return "live_auto_submit_disabled"
    if not settings.live_armed:
        return "live_armed_false"
    if settings.kill_switch_enabled:
        return "kill_switch_enabled"
    return "unknown"

def _requires_market_hours(self, module: EntryModule) -> bool:
    settings = self.settings
    assert settings is not None
    return (
        settings.laboratory_market_hours_only
        if module == "laboratory"
        else settings.fox_hunter_market_hours_only
    )

@staticmethod
def _market_closed_result(
    module: EntryModule,
    resolved: dict[str, Any],
    session: dict[str, object],
) -> dict[str, Any]:
    return {
        "module": module,
        "patterns_checked": 0,
        "symbols_checked": 0,
        "matches_found": 0,
        "entry_variants_considered": 0,
        "signals_created": 0,
        "orders_submitted": 0,
        "skipped_duplicates": 0,
        "skipped_cooldown": 0,
        "rejected_by_entry_gate": 0,
        "rejected_by_entry_quality": 0,
        "rejected_by_risk": 0,
        "rejected_by_production_manifest": 0,
        "near_miss_shadow_observations_opened": 0,
        "order_errors": [],
        "signal_ids": [],
        "near_miss_signal_ids": [],
        "trade_ids": [],
        "paper_observations_opened": 0,
        "paper_observations_closed": 0,
        "paper_observation_trade_ids": [],
        "near_miss_trade_ids": [],
        "shadow_no_order_observations_opened": 0,
        "shadow_no_order_trade_ids": [],
        "paper_observation_lifecycle": {
            "open_observations": 0,

```

```

        "closed_observations": 0,
        "closed_trade_ids": [],
        "data_errors": [],
    },
    "top_opportunities": [],
    "store_signals": resolved["store_signals"],
    "execute_orders": resolved["execute_orders"],
    "similarity_threshold": resolved["similarity_threshold"],
    "skipped_reason": session.get("state", "market_closed"),
    "market_session": session,
    "generated_at": datetime.now(timezone.utc).isoformat(),
}

def _resolved_options(self, module: EntryModule, **overrides: Any) -> dict[str, Any]:
    settings = self.settings
    assert settings is not None
    if module == "fox_hunter":
        defaults = {
            "limit": settings.fox_hunter_symbol_limit,
            "max_patterns": settings.fox_hunter_max_patterns,
            "similarity_threshold": settings.fox_hunter_similarity_threshold,
            "store_signals": settings.fox_hunter_store_signals,
            "execute_orders": settings.fox_hunter_auto_submit_live_orders,
        }
    else:
        defaults = {
            "limit": settings.laboratory_symbol_limit,
            "max_patterns": settings.laboratory_max_patterns,
            "similarity_threshold": settings.laboratory_similarity_threshold,
            "store_signals": settings.laboratory_store_signals,
            "execute_orders": settings.laboratory_auto_submit_paper_orders,
        }
    return {key: defaults[key] if value is None else value for key, value in overrides.items()}

def _validate_module_execution(
    self,
    module: EntryModule,
    *,
    execute_orders: bool | None,
) -> None:
    settings = self.settings
    assert settings is not None
    execute = execute_orders
    if execute is None:
        execute = (
            settings.fox_hunter_auto_submit_live_orders
            if module == "fox_hunter"
            else settings.laboratory_auto_submit_paper_orders
        )
    if not execute:
        return
    if settings.kill_switch_enabled:
        raise PatternEntryScannerSafetyError("kill switch blocks scanner order execution")
    if module == "laboratory":
        if settings.trading_mode != "paper" or settings.live_armed:
            raise PatternEntryScannerSafetyError(
                "Laboratory can only auto-submit orders in paper mode"
            )
        if int(settings.ibkr_port) in {7496, 4001}:
            raise PatternEntryScannerSafetyError("Laboratory refuses live IBKR ports")
        return
    if not settings.live_armed:
        raise PatternEntryScannerSafetyError(
            "Fox Hunter live execution requires live_armed=true"
        )

@staticmethod
def _fox_match_has_active_manifest(db: Session, match: dict[str, Any]) -> bool:
    try:
        pattern_id = int(match.get("pattern_id") or 0)
    except (TypeError, ValueError):
        return False
    pattern = db.get(DiscoveredPattern, pattern_id)
    if pattern is None:
        return False
    return production_manifest_is_active(pattern)

def _candidate_from_match(self, match: dict[str, Any]) -> PatternCandidate:
    metrics = match.get("metrics") or {}
    features = dict(metrics.get("features") or {})
    features["entry_variant_id"] = str(match.get("entry_variant_id") or "")
    features["regime_key"] = str((match.get("regime") or {}).get("regime_key") or "")
    for key in ("pattern_family_key", "canonical_pattern_key", "pattern_key", "pattern_id"):
        if match.get(key) is not None:
            features[key] = str(match.get(key) or "")
    return PatternCandidate(
        symbol=str(match["symbol"]),
        pattern=str(match["pattern_name"]),
        side=str(match["side"]),
        timeframe=str(match["timeframe"]),
        entry=float(match["entry_price"]),
        stop=float(match["stop_price"]),
        target=float(match["target_price"]),
        reward_risk=float(match["reward_risk"]),
        confidence=float(match["score"]),
        rule_score=float(match["similarity"]),
    )

```

```

        ml_score=float(metrics.get("pattern_score", 0.0)),
        vision_score=float(metrics.get("pattern_stability_score", 0.0)),
        composite_score=float(match.get("entry_score") or match["score"]),
        features=features,
        notes=[str(match.get("notes", ""))],
    )

    @classmethod
    def _has_active_exposure(cls, db: Session, match: dict[str, Any], *, module: EntryModule) -> bool:
        active_signals = (
            db.query(Signal)
            .filter(Signal.symbol == str(match["symbol"]))
            .filter(Signal.pattern == str(match["pattern_name"]))
            .filter(
                Signal.status.in_(
                    [
                        SignalStatus.WATCHLIST,
                        SignalStatus.PAPER_APPROVED,
                        SignalStatus.PENDING_HUMAN_APPROVAL,
                        SignalStatus.LIVE_APPROVED,
                    ]
                )
            )
            .all()
        )
        if any(cls._signal_belongs_to_module(signal, module) for signal in active_signals):
            return True
        active_trades = (
            db.query(Trade)
            .options(joinedload(Trade.signal))
            .filter(Trade.symbol == str(match["symbol"]))
            .filter(Trade.pattern == str(match["pattern_name"]))
            .filter(Trade.status == TradeStatus.OPEN)
            .all()
        )
        return any(cls._trade_belongs_to_module(trade, module) for trade in active_trades)

    @staticmethod
    def _signal_idempotency_key(module: EntryModule, match: dict[str, Any]) -> str:
        """Stable identity of a signal: same pattern, symbol, variant and bar.

        Cooldowns only guard a time window; a worker restart or a re-scan of the
        same completed bar can still emit the same economic signal twice. The
        key pins the signal to (pattern, symbol, variant, bar_ts) so re-scans of
        an unchanged bar are no-ops.
        """
        return "|".join(
            (
                str(module),
                str(match.get("pattern_id") or ""),
                str(match.get("symbol") or "").upper(),
                str(match.get("timeframe") or ""),
                str(match.get("entry_variant_id") or ""),
                str(match.get("window_end") or ""),
            )
        )

    def _has_duplicate_signal(self, db: Session, match: dict[str, Any], *, module: EntryModule) -> bool:
        if not str(match.get("window_end") or ""):
            return False
        key = self._signal_idempotency_key(module, match)
        existing = (
            db.query(Signal)
            .filter(Signal.symbol == str(match["symbol"]))
            .filter(Signal.pattern == str(match["pattern_name"]))
            .all()
        )
        for signal in existing:
            metadata = signal.metadata_json or {}
            if str(metadata.get("signal_idempotency_key") or "") == key:
                return True
        return False

    def _has_recent_signal(self, db: Session, match: dict[str, Any], *, module: EntryModule) -> bool:
        settings = self.settings
        assert settings is not None
        cooldown_minutes = max(0, int(settings.entry_cooldown_minutes))
        if cooldown_minutes <= 0:
            return False
        cutoff = datetime.now(timezone.utc) - timedelta(minutes=cooldown_minutes)
        recent_signals = (
            db.query(Signal)
            .filter(Signal.symbol == str(match["symbol"]))
            .filter(Signal.pattern == str(match["pattern_name"]))
            .filter(Signal.created_at >= cutoff)
            .all()
        )
        variant = str(match.get("entry_variant_id") or "")
        for signal in recent_signals:
            if not self._signal_belongs_to_module(signal, module):
                continue
            metadata = signal.metadata_json or {}
            previous_variant = str(metadata.get("entry_variant_id") or "")
            if not variant or not previous_variant or previous_variant == variant:
                return True
        return False

```

```

@staticmethod
def _signal_belongs_to_module(signal: Signal, module: EntryModule) -> bool:
    metadata = signal.metadata_json or {}
    entry_module = metadata.get("entry_module")
    if entry_module:
        return str(entry_module) == module
    if module == "fox_hunter":
        return signal.strategy_version.startswith("fox_hunter_")
    purpose = str(metadata.get("purpose") or "")
    return signal.strategy_version.startswith(("laboratory_", "ibkr_paper_", "ibkr_smoke_")) or (
        purpose.startswith("ibkr_paper_")
    )

@classmethod
def _trade_belongs_to_module(cls, trade: Trade, module: EntryModule) -> bool:
    if trade.signal is not None:
        return cls._signal_belongs_to_module(trade.signal, module)
    metadata = trade.metadata_json or {}
    source_module = metadata.get("entry_module") or metadata.get("source_module")
    if source_module:
        return str(source_module) == module
    reason = str(metadata.get("reason") or "")
    if module == "fox_hunter":
        return reason.startswith("fox_hunter")
    execution_mode = str(metadata.get("execution_mode") or "")
    return reason.startswith("laboratory") or execution_mode in {"paper", "lab_shadow_observation"}

@staticmethod
def _top_opportunities(matches: list[dict[str, Any]], *, limit: int = 10) -> list[dict[str, Any]]:
    return [
        {
            "rank": match.get("opportunity_rank"),
            "rank_score": match.get("opportunity_rank_score"),
            "symbol": match.get("symbol"),
            "pattern_name": match.get("pattern_name"),
            "entry_variant_id": match.get("entry_variant_id"),
            "regime_key": (match.get("regime") or {}).get("regime_key"),
            "entry_score": match.get("entry_score"),
            "similarity": match.get("similarity"),
            "entry_gate_passed": ((match.get("metrics") or {}).get("entry_gate") or {}).get("passed"),
            "entry_gate_reason": ((match.get("metrics") or {}).get("entry_gate") or {}).get("reason"),
            "entry_rejection_reasons": ((match.get("metrics") or {}).get("entry_gate") or {}).get(
                "rejection_reasons", []
            ),
            "near_miss_shadow_candidate": match.get("near_miss_shadow_candidate"),
            "history_count": (match.get("opportunity_rank_components") or {}).get("history_count"),
            "history_expectancy_r": (match.get("opportunity_rank_components") or {}).get(
                "history_expectancy_r"
            ),
            "rank_reason": match.get("opportunity_rank_reason"),
        }
        for match in matches[:limit]
    ]

@staticmethod
def _select_best_variant_per_exposure(matches: list[dict[str, Any]]) -> list[dict[str, Any]]:
    selected: dict[tuple[str, str], dict[str, Any]] = {}
    for match in matches:
        key = (str(match.get("symbol") or ""), str(match.get("pattern_name") or ""))
        current = selected.get(key)
        if current is None or float(match.get("opportunity_rank_score") or 0.0) > float(
            current.get("opportunity_rank_score") or 0.0
        ):
            selected[key] = match
    ordered = sorted(
        selected.values(),
        key=lambda item: (
            float(item.get("opportunity_rank_score") or 0.0),
            float(item.get("entry_score") or 0.0),
            float(item.get("score") or 0.0),
        ),
        reverse=True,
    )
    for index, match in enumerate(ordered, start=1):
        match["opportunity_rank"] = index
    return ordered

def _execution_history(self, db: Session, module: EntryModule) -> dict[tuple[str, ...], dict[str, float]]:
    trades = (
        db.query(Trade)
        .options(joinedload(Trade.signal))
        .filter(Trade.status == TradeStatus.CLOSED)
        .order_by(Trade.closed_at.desc(), Trade.opened_at.desc())
        .limit(500)
        .all()
    )
    by_key: dict[tuple[str, ...], list[float]] = {}
    for trade in trades:
        signal = trade.signal
        if signal is None:
            continue
        metadata = signal.metadata_json or {}
        if metadata.get("entry_module") != module:
            continue
        trade_metadata = evidence_metadata_with_stored_columns(

```

```

        trade.metadata_json or {},
        evidence_type=trade.evidence_type,
        evidence_quality=trade.evidence_quality,
    )
    if not is_director_review_paper_fill_evidence(
        trade_metadata,
        trade_status=trade.status,
        signal_metadata=metadata,
        broker_order_id=trade.broker_order_id,
    ):
        continue
    variant = str(metadata.get("entry_variant_id") or "")
    regime_key = str((metadata.get("regime") or {}).get("regime_key") or "")
    keys = [
        (trade.pattern, trade.symbol, variant, regime_key),
        (trade.pattern, "", variant, regime_key),
        (trade.pattern, trade.symbol, variant, ""),
        (trade.pattern, "", variant, ""),
        (trade.pattern, trade.symbol, "", ""),
        (trade.pattern, "", "", ""),
    ]
    for key in keys:
        by_key.setdefault(key, []).append(float(trade.r_multiple or 0.0))
    return {key: self._history_item(values) for key, values in by_key.items()}

def _open_near_miss_shadow_observation(
    self,
    db: Session,
    *,
    match: dict[str, Any],
    resolved: dict[str, Any],
    session: dict[str, Any],
) -> tuple[Signal, Trade] | None:
    settings = self.settings
    assert settings is not None
    if not resolved["store_signals"]:
        return None
    entry_gate = ((match.get("metrics") or {}).get("entry_gate") or {})
    if self._has_active_exposure(db, match, module="laboratory"):
        return None
    if self._has_duplicate_signal(db, match, module="laboratory"):
        return None
    if self._has_recent_signal(db, match, module="laboratory"):
        return None
    candidate = self._candidate_from_match(match)
    risk = RiskManager(settings).validate_candidate(candidate, db)
    if not risk.approved:
        db.add(
            AuditLog(
                actor="laboratory",
                action="near_miss_shadow_rejected_by_risk",
                entity_type="discovered_pattern_match",
                entity_id=str(match.get("pattern_id", "")),
                details_json={"match": match, "risk": risk.model_dump(mode="json")},
            )
        )
        db.commit()
        return None
    entry_quality = build_entry_quality(
        match=match,
        risk=risk,
        settings=settings,
        execution_requested=False,
        market_session=session,
    )
    if float(entry_quality.get("score") or 0.0) < 0.45:
        db.add(
            AuditLog(
                actor="laboratory",
                action="near_miss_shadow_rejected_by_quality",
                entity_type="discovered_pattern_match",
                entity_id=str(match.get("pattern_id", "")),
                details_json={
                    "match": match,
                    "entry_quality": entry_quality,
                    "min_near_miss_quality_score": 0.45,
                },
            )
        )
        db.commit()
        return None
    near_miss_reasons = self._entry_gate_reasons(entry_gate)
    match = self._mark_near_miss_shadow_match(match, near_miss_reasons)
    entry_quality = dict(entry_quality)
    entry_quality["near_miss_shadow"] = True
    entry_quality["actionable"] = False
    entry_quality["label"] = "watch"
    entry_quality["flags"] = sorted(set(list(entry_quality.get("flags") or []) + ["near_miss_shadow"]))
    signal = self._store_signal(
        db,
        module="laboratory",
        match=match,
        candidate=candidate,
        risk=risk,
        execute_orders=False,
        market_session=session,
    )

```

```

        entry_quality=entry_quality,
    )
    observation = self._open_lab_observation(db, signal=signal, match=match, risk=risk)
    if observation is None:
        return None
    db.add(
        AuditLog(
            actor="laboratory",
            action="near_miss_shadow_observation_opened",
            entity_type="trade",
            entity_id=str(observation.id),
            details_json={
                "signal_id": signal.id,
                "trade_id": observation.id,
                "match": match,
                "entry_gate": entry_gate,
                "near_miss_reasons": near_miss_reasons,
                "no_ibkr_order": True,
            },
        )
    )
    db.commit()
    return signal, observation

def _is_lab_near_miss_shadow_candidate(
    self,
    match: dict[str, Any],
    entry_gate: dict[str, Any],
) -> bool:
    settings = self.settings
    assert settings is not None
    if bool(entry_gate.get("passed", False)):
        return False
    if not str(match.get("entry_variant_id") or ""):
        return False
    if not str((match.get("regime") or {}).get("regime_key") or ""):
        return False
    trigger = str(entry_gate.get("trigger") or match.get("entry_trigger") or "")
    if trigger in {"", "no_operational_trigger", "insufficient_history"}:
        return False
    trigger_score = self._safe_float(
        entry_gate.get("trigger_score"),
        1.0 if trigger not in {"", "no_operational_trigger"} else 0.0,
    )
    if trigger_score <= 0:
        return False
    reasons = set(self._entry_gate_reasons(entry_gate))
    if not reasons or not reasons & LAB_NEAR_MISS_VOLUME_REASONS:
        return False
    if reasons & LAB_NEAR_MISS_HARD_REASONS:
        return False
    if any(reason not in LAB_NEAR_MISS_VOLUME_REASONS for reason in reasons):
        return False
    if entry_gate.get("regime_ok") is False:
        return False
    if entry_gate.get("volatility_ok") is False:
        return False
    if entry_gate.get("not_extended") is False:
        return False
    metrics = match.get("metrics") or {}
    features = metrics.get("features") or {}
    atr_pct = self._safe_float(entry_gate.get("atr_pct", features.get("atr_pct")), 0.0)
    if atr_pct > settings.max_atr_pct:
        return False
    extension_atr = self._safe_float(entry_gate.get("extension_atr"), 0.0)
    if extension_atr > settings.entry_max_extension_atr:
        return False
    rank = self._safe_int(match.get("opportunity_rank"))
    rank_score = self._safe_float(match.get("opportunity_rank_score"), 0.0)
    entry_score = self._safe_float(entry_gate.get("entry_score", match.get("entry_score")), 0.0)
    if entry_score < max(LAB_NEAR_MISS_MIN_ENTRY_SCORE, settings.entry_min_score):
        return False
    if rank_score < LAB_NEAR_MISS_MIN_RANK_SCORE:
        return False
    top_ranked = rank is not None and rank <= LAB_NEAR_MISS_TOP_RANK_LIMIT
    return top_ranked or rank_score >= LAB_NEAR_MISS_HIGH_RANK_SCORE

@staticmethod
def _entry_gate_reasons(entry_gate: dict[str, Any]) -> list[str]:
    reasons = entry_gate.get("rejection_reasons")
    collected: list[str] = []
    if isinstance(reasons, list):
        collected.extend(str(reason).strip() for reason in reasons if str(reason).strip())
    reason = str(entry_gate.get("reason") or "").strip()
    if reason and reason not in {"entry gate failed", "entry gate passed"}:
        collected.extend(part.strip() for part in reason.replace(" ", ";").split(";") if part.strip())
    if entry_gate.get("volume_confirmed") is False:
        collected.append("insufficient_volume")
    deduped: list[str] = []
    for reason_item in collected:
        if reason_item not in deduped:
            deduped.append(reason_item)
    return deduped

@staticmethod
def _safe_float(value: Any, default: float) -> float:

```

```

try:
    return float(value)
except (TypeError, ValueError):
    return default

@staticmethod
def _safe_int(value: Any) -> int | None:
    try:
        return int(value)
    except (TypeError, ValueError):
        return None

@staticmethod
def _mark_near_miss_shadow_match(match: dict[str, Any], reasons: list[str]) -> dict[str, Any]:
    enriched = dict(match)
    metrics = dict(enriched.get("metrics") or {})
    entry_gate = dict(metrics.get("entry_gate") or {})
    entry_gate["near_miss_shadow"] = True
    entry_gate["near_miss"] = True
    entry_gate["would_have_failed_entry_gate"] = True
    entry_gate["near_miss_reasons"] = reasons
    entry_gate["entry_gate_rejection_reasons"] = reasons
    metrics["entry_gate"] = entry_gate
    metrics["near_miss"] = True
    metrics["near_miss_shadow"] = True
    metrics["near_miss_reasons"] = reasons
    metrics["entry_gate_rejection_reasons"] = reasons
    metrics["would_have_failed_entry_gate"] = True
    metrics["paper_only"] = True
    metrics["no_ibkr_order"] = True
    metrics["evidence_type"] = EvidenceType.NEAR_MISS_SHADOW.value
    metrics["evidence_quality"] = EvidenceQuality.NORMAL.value
    enriched["metrics"] = metrics
    enriched["near_miss"] = True
    enriched["near_miss_shadow"] = True
    enriched["near_miss_shadow_candidate"] = True
    enriched["near_miss_type"] = "volume_only_entry_gate_shadow"
    enriched["near_miss_reasons"] = reasons
    enriched["entry_gate_rejection_reasons"] = reasons
    enriched["entry_gate_reason"] = str(entry_gate.get("reason") or ";".join(reasons))
    enriched["would_have_failed_entry_gate"] = True
    enriched["paper_only"] = True
    enriched["no_ibkr_order"] = True
    enriched["observation_only"] = True
    enriched["evidence_type"] = EvidenceType.NEAR_MISS_SHADOW.value
    enriched["evidence_quality"] = EvidenceQuality.NORMAL.value
    enriched["notes"] = (
        f"{enriched.get('notes', '')} Near-miss Lab shadow observation; "
        "entry gate failed only on soft volume confirmation."
    ).strip()
    return enriched

@staticmethod
def _audit_entry_gate_rejection(
    db: Session,
    *,
    module: EntryModule,
    match: dict[str, Any],
    entry_gate: dict[str, Any],
) -> None:
    db.add(
        AuditLog(
            actor=module,
            action="entry_match_rejected_by_entry_gate",
            entity_type="discovered_pattern_match",
            entity_id=str(match.get("pattern_id", "")),
            details_json={"match": match, "entry_gate": entry_gate},
        )
    )
    db.commit()

@staticmethod
def _history_score(values: list[float]) -> float:
    if not values:
        return 0.5
    expectancy = sum(values) / len(values)
    wins = [value for value in values if value > 0]
    losses = [abs(value) for value in values if value < 0]
    win_rate = len(wins) / len(values)
    profit_factor = (sum(wins) / sum(losses)) if losses else (sum(wins) if wins else 0.0)
    profit_factor_score = max(0.0, min(1.0, profit_factor / 3.0))
    chronological = list(reversed(values))
    equity = 0.0
    peak = 0.0
    max_drawdown = 0.0
    for value in chronological:
        equity += value
        peak = max(peak, equity)
        max_drawdown = max(max_drawdown, peak - equity)
    drawdown_score = max(0.0, min(1.0, 1.0 - max_drawdown / 6.0))
    split = max(1, min(5, len(values) // 2 or 1))
    recent = values[:split]
    older = values[split:] or recent
    recent_expectancy = sum(recent) / len(recent)
    older_expectancy = sum(older) / len(older)
    decay_score = max(0.0, min(1.0, 0.5 + (recent_expectancy - older_expectancy) / 2.0))

```

```

expectancy_score = max(0.0, min(1.0, 0.5 + expectancy / 2.0))
raw_score = (
    expectancy_score * 0.35
    + win_rate * 0.20
    + profit_factor_score * 0.20
    + drawdown_score * 0.15
    + decay_score * 0.10
)
confidence = min(1.0, len(values) / 10.0)
return round(0.5 * (1.0 - confidence) + raw_score * confidence, 6)

def _history_item(self, values: list[float]) -> dict[str, float]:
    wins = [value for value in values if value > 0]
    losses = [abs(value) for value in values if value < 0]
    profit_factor = (sum(wins) / sum(losses)) if losses else (sum(wins) if wins else 0.0)
    chronological = list(reversed(values))
    equity = 0.0
    peak = 0.0
    max_drawdown = 0.0
    for value in chronological:
        equity += value
        peak = max(peak, equity)
        max_drawdown = max(max_drawdown, peak - equity)
    split = max(1, min(5, len(values) // 2 or 1))
    recent = values[:split]
    older = values[split:] or recent
    recent_expectancy = sum(recent) / len(recent)
    older_expectancy = sum(older) / len(older)
    decay_score = max(0.0, min(1.0, 0.5 + (recent_expectancy - older_expectancy) / 2.0))
    return {
        "score": self._history_score(values),
        "count": float(len(values)),
        "expectancy_r": round(sum(values) / len(values), 6) if values else 0.0,
        "win_rate": round(len(wins) / len(values), 6) if values else 0.0,
        "profit_factor": round(profit_factor, 6),
        "max_drawdown_r": round(max_drawdown, 6),
        "decay_score": round(decay_score, 6),
        "recent_expectancy_r": round(recent_expectancy, 6),
        "older_expectancy_r": round(older_expectancy, 6),
    }

def _store_signal(
    self,
    db: Session,
    *,
    module: EntryModule,
    match: dict[str, Any],
    candidate: PatternCandidate,
    risk,
    execute_orders: bool,
    market_session: dict[str, Any] | None = None,
    entry_quality: dict[str, Any] | None = None,
):
    settings = self.settings
    assert settings is not None
    if module == "fox_hunter":
        status = (
            SignalStatus.LIVE_APPROVED
            if settings.live_armed and execute_orders
            else SignalStatus.PENDING_HUMAN_APPROVAL
        )
        human_approved = status == SignalStatus.LIVE_APPROVED
    else:
        status = SignalStatus.WATCHLIST if match.get("near_miss") else SignalStatus.PAPER_APPROVED
        human_approved = False if match.get("near_miss") else execute_orders
    entry_quality = entry_quality or build_entry_quality(
        match=match,
        risk=risk,
        settings=settings,
        execution_requested=execute_orders,
        market_session=market_session,
    )
    signal_snapshot = build_signal_snapshot(
        match=match,
        risk=risk,
        settings=settings,
        entry_quality=entry_quality,
        execution_requested=execute_orders,
        market_session=market_session,
    )
    evidence_type = self._signal_evidence_type(
        module,
        match=match,
        execute_orders=execute_orders,
    )
    signal = Signal(
        symbol=candidate.symbol,
        pattern=candidate.pattern,
        side=candidate.side,
        timeframe=candidate.timeframe,
        entry=candidate.entry,
        stop=candidate.stop,
        target=candidate.target,
        reward_risk=candidate.reward_risk,
        confidence=candidate.confidence,
        composite_score=candidate.composite_score,

```



```

risk_usd=risk.risk_usd,
suggested_qty=risk.suggested_qty,
strategy_version=f"{module}_pattern_{match['pattern_id']}",
status=status,
supervisor_notes=str(match.get("notes", "")),
human_approved=human_approved,
metadata_json={
    "evidence_type": evidence_type,
    "evidence_quality": EvidenceQuality.NORMAL.value,
    "entry_module": module,
    "signal_idempotency_key": self._signal_idempotency_key(module, match),
    "bar_window_end": str(match.get("window_end") or ""),
    "pattern_id": match["pattern_id"],
    "pattern_key": match["pattern_key"],
    "pattern_family_key": match.get("pattern_family_key"),
    "canonical_pattern_key": match.get("canonical_pattern_key"),
    "pattern_status": match.get("pattern_status"),
    "pattern_promotion_status": match.get("pattern_promotion_status"),
    "production_manifest": match.get("production_manifest"),
    "entry_variant_id": match.get("entry_variant_id"),
    "entry_variant": match.get("entry_variant"),
    "entry_audit": match.get("entry_audit"),
    "regime": match.get("regime"),
    "regime_fit": match.get("regime_fit"),
    "opportunity_rank": match.get("opportunity_rank"),
    "opportunity_rank_score": match.get("opportunity_rank_score"),
    "opportunity_rank_components": match.get("opportunity_rank_components"),
    "opportunity_rank_reason": match.get("opportunity_rank_reason"),
    "entry_quality_score": entry_quality["score"],
    "entry_quality": entry_quality,
    "signal_snapshot": signal_snapshot,
    "match": match,
    "entry_gate": match.get("metrics", {}).get("entry_gate"),
    "near_miss": bool(match.get("near_miss")),
    "near_miss_shadow": bool(match.get("near_miss_shadow")),
    "near_miss_type": match.get("near_miss_type"),
    "near_miss_reasons": match.get("near_miss_reasons") or [],
    "entry_gate_rejection_reasons": match.get("entry_gate_rejection_reasons") or [],
    "entry_gate_reason": match.get("entry_gate_reason"),
    "would_have_failed_entry_gate": bool(match.get("would_have_failed_entry_gate")),
    "observation_only": module == "laboratory" and not execute_orders,
    "execution_mode": self._signal_execution_mode(module, execute_orders=execute_orders),
    "paper_only": module == "laboratory" and not execute_orders,
    "no_ibkr_order": module == "laboratory" and not execute_orders,
    "execution_outcome": (
        {
            "status": "near_miss_shadow_observation",
            "reason_code": "entry_gate_volume_near_miss_shadow",
            "reason": (
                "Entry gate failed only volume confirmation; Lab opened a "
                "shadow observation with no IBKR order."
            ),
            "retryable": False,
            "next_action": "collect_shadow_outcome",
        }
        if match.get("near_miss")
        else None
    ),
    "risk": risk.model_dump(mode="json"),
    "director_audit_required": True,
},
),
db.add(signal)
db.add(
    AuditLog(
        actor=module,
        action="entry_signal_created",
        entity_type="signal",
        details_json={"signal": signal.metadata_json},
    )
)
db.commit()
db.refresh(signal)
return signal

@staticmethod
def _signal_execution_mode(module: EntryModule, *, execute_orders: bool) -> str | None:
    if module == "laboratory" and not execute_orders:
        return LAB_SHADOW_EXECUTION_MODE
    if execute_orders:
        return "ibkr"
    return None

@staticmethod
def _signal_evidence_type(
    module: EntryModule,
    *,
    match: dict[str, Any],
    execute_orders: bool,
) -> str | None:
    if module == "laboratory":
        if match.get("near_miss"):
            return EvidenceType.NEAR_MISS_SHADOW.value
        if not execute_orders:
            return EvidenceType.SHADOW_NO_ORDER.value
        return EvidenceType.IBKR_PAPER_ORDER.value

```

```

    if execute_orders:
        return EvidenceType.LIVE_ORDER.value
    return None

    @staticmethod
    def _execution_reason(module: EntryModule) -> str:
        if module == "fox_hunter":
            return "fox_hunter production live execution"
        return "laboratory IBKR paper validation"

    def _observation_service(self) -> LabPaperObservationService:
        provider = getattr(self.matcher, "provider", None)
        return LabPaperObservationService(settings=self.settings, provider=provider)

    def _close_lab_observations(self, db: Session, module: EntryModule) -> dict[str, Any]:
        if module != "laboratory":
            return {
                "open_observations": 0,
                "closed_observations": 0,
                "closed_trade_ids": [],
                "data_errors": [],
            }
        return self._observation_service().close_open_observations(db)

    def _open_lab_observation(
        self,
        db: Session,
        *,
        signal: Signal,
        match: dict[str, Any],
        risk,
    ) -> Trade | None:
        return self._observation_service().open_observation(
            db,
            signal=signal,
            match=match,
            risk=risk,
        )

```

Full Source: ``backend/tradeo/services/lab_paper_observations.py``

``backend/tradeo/services/lab_paper_observations.py`` ? 4 lines, 144 bytes, sha256 prefix ``c98e4eb498356236``

```

"""Backward-compatible imports for Laboratory paper observations."""

```

```

from tradeo.modules.laboratory.paper_observations import * # noqa: F403

```

Full Source: ``backend/tradeo/modules/laboratory/paper_observations.py``

``backend/tradeo/modules/laboratory/paper_observations.py`` ? 706 lines, 30406 bytes, sha256 prefix ``1269f020e3544623``

Audit relevance: Shadow observation lifecycle. Audit canonical exit parity, pending/closed transitions, no-order invariant, and open-gap handling.

Public surface summary before full source:

- ? class ``MarketDataFetch`` line 40 methods: ?
- ? class ``FallbackFrame`` line 46 methods: ?
- ? class ``LabPaperObservationService`` line 53 methods: `open_observation`, `close_open_observations`

Risk hints: broad ``except Exception`` present; wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any

import pandas as pd
from loguru import logger
from sqlalchemy.orm import Session, joinedload

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, Signal, Trade, TradeStatus
from tradeo.services.evidence import (
    EvidenceQuality,
    EvidenceType,
    FillProvenance,
    LAB_SHADOW_EXECUTION_MODE,
    with_evidence_metadata,
)
from tradeo.research.quant_validation import triple_barrier_outcome
from tradeo.services.data_provider import MarketDataProvider
from tradeo.services.provider_factory import get_market_data_provider
from tradeo.services.technical_indicators import normalize_ohlc

# Map canonical triple-barrier reasons to the shadow lifecycle vocabulary.
# "stop_gap" / "target_gap" pass through unchanged: implementation-shortfall

```

```

# accounting already classifies them, and renaming them to *_hit would hide
# that the fill happened at the open, not at the barrier price.
CANONICAL_EXIT_REASON_MAP = {
    "stop": "stop_hit",
    "stop_and_target_conservative": "stop_hit",
    "stop_gap": "stop_gap",
    "target": "target_hit",
    "target_gap": "target_gap",
    "time": "time_stop",
}

@dataclass(slots=True)
class MarketDataFetch:
    frame: pd.DataFrame | None
    error: str | None = None

@dataclass(slots=True)
class FallbackFrame:
    frame: pd.DataFrame
    source: str
    timestamp_source: str

@dataclass(slots=True)
class LabPaperObservationService:
    """Open and evaluate laboratory paper observations without broker orders."""

    settings: Settings | None = None
    provider: MarketDataProvider | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()
        self.provider = self.provider or get_market_data_provider()

    def open_observation(
        self,
        db: Session,
        *,
        signal: Signal,
        match: dict[str, Any],
        risk: Any,
    ) -> Trade | None:
        if signal.id is None:
            return None
        existing = (
            db.query(Trade)
            .filter(Trade.signal_id == signal.id)
            .filter(Trade.status == TradeStatus.OPEN)
            .first()
        )
        if existing is not None:
            return existing
        qty = int(getattr(risk, "suggested_qty", 0) or signal.suggested_qty or 0)
        if qty <= 0:
            return None
        now = datetime.now(timezone.utc)
        signal_metadata = signal.metadata_json or {}
        entry_gate = signal_metadata.get("entry_gate") or (match.get("metrics") or {}).get("entry_gate")
        entry_quality = signal_metadata.get("entry_quality") or {}
        near_miss = bool(signal_metadata.get("near_miss") or match.get("near_miss"))
        evidence_type = (
            EvidenceType.NEAR_MISS_SHADOW.value
            if near_miss
            else EvidenceType.SHADOW_NO_ORDER.value
        )
        metadata = {
            "evidence_type": evidence_type,
            "evidence_quality": EvidenceQuality.NORMAL.value,
            "fill_provenance": FillProvenance.SHADOW_CLOSE.value,
            "execution_mode": LAB_SHADOW_EXECUTION_MODE,
            "entry_module": "laboratory",
            "source_module": "laboratory",
            "observation_only": True,
            "no_ibkr_order": True,
            "entry_variant_id": signal_metadata.get("entry_variant_id") or match.get("entry_variant_id"),
            "entry_variant": signal_metadata.get("entry_variant") or match.get("entry_variant"),
            "pattern_id": signal_metadata.get("pattern_id") or match.get("pattern_id"),
            "pattern_key": signal_metadata.get("pattern_key") or match.get("pattern_key"),
            "regime": signal_metadata.get("regime") or match.get("regime"),
            "regime_fit": signal_metadata.get("regime_fit") or match.get("regime_fit"),
            "entry_gate": entry_gate,
            "entry_quality": entry_quality,
            "entry_audit": signal_metadata.get("entry_audit") or match.get("entry_audit"),
            "opportunity_rank": signal_metadata.get("opportunity_rank") or match.get("opportunity_rank"),
            "opportunity_rank_score": signal_metadata.get("opportunity_rank_score") or match.get("opportunity_rank_score"),
            "opportunity_rank_components": signal_metadata.get("opportunity_rank_components")
            or match.get("opportunity_rank_components"),
            "near_miss": near_miss,
            "near_miss_shadow": bool(signal_metadata.get("near_miss_shadow") or match.get("near_miss_shadow")),
            "near_miss_type": signal_metadata.get("near_miss_type") or match.get("near_miss_type"),
            "near_miss_reasons": signal_metadata.get("near_miss_reasons") or match.get("near_miss_reasons") or [],
            "entry_gate_rejection_reasons": signal_metadata.get("entry_gate_rejection_reasons")
            or match.get("entry_gate_rejection_reasons")
            or [],
        }

```

```

        "entry_gate_reason": signal_metadata.get("entry_gate_reason") or match.get("entry_gate_reason"),
        "would_have_failed_entry_gate": bool(
            signal_metadata.get("would_have_failed_entry_gate") or match.get("would_have_failed_entry_gate")
        ),
        "paper_only": True,
        "opened_reason": (
            "lab_near_miss_volume_shadow_observation"
            if near_miss
            else "lab_entry_candidate_shadow_observation"
        ),
        "entry_fill_time": now.isoformat(),
        "entry_fill_price": signal.entry,
        "entry_order_type": "shadow_observation",
        "signal_snapshot": signal_metadata.get("signal_snapshot") or match.get("signal_snapshot"),
        "match": signal_metadata.get("match") or match,
        "commission": 0.0,
        "estimated_spread_cost": 0.0,
        "estimated_slippage": 0.0,
        "other_fees": 0.0,
    }
}

trade = Trade(
    signal_id=signal.id,
    symbol=signal.symbol,
    pattern=signal.pattern,
    side=signal.side,
    qty=qty,
    entry=signal.entry,
    stop=signal.stop,
    target=signal.target,
    status=TradeStatus.OPEN,
    opened_at=now,
    evidence_type=evidence_type,
    evidence_quality=EvidenceQuality.NORMAL.value,
    metadata_json=metadata,
)

db.add(trade)
db.add(
    AuditLog(
        actor="laboratory",
        action="lab_shadow_observation_opened",
        entity_type="trade",
        entity_id=str(signal.id),
        details_json={
            "signal_id": signal.id,
            "symbol": signal.symbol,
            "pattern": signal.pattern,
            "entry_variant_id": match.get("entry_variant_id"),
            "near_miss": near_miss,
            "evidence_type": evidence_type,
            "evidence_quality": EvidenceQuality.NORMAL.value,
            "no_ibkr_order": True,
        },
    )
)

db.commit()
db.refresh(trade)
return trade

def close_open_observations(self, db: Session) -> dict[str, Any]:
    trades = (
        db.query(Trade)
        .options(joinedload(Trade.signal))
        .filter(Trade.status == TradeStatus.OPEN)
        .order_by(Trade.opened_at.asc())
        .limit(500)
        .all()
    )
    observations = [trade for trade in trades if self._is_lab_shadow_observation(trade)]
    closed_ids: list[int] = []
    diagnosed_ids: list[int] = []
    data_errors: list[dict[str, str]] = []
    frame_cache: dict[tuple[str, str], MarketDataFetch] = {}
    for trade in observations:
        signal = trade.signal
        timeframe = signal.timeframe if signal is not None else "1d"
        key = (trade.symbol.upper(), timeframe)
        if key not in frame_cache:
            frame_cache[key] = self._fetch_frame(trade.symbol, timeframe)
        fetch = frame_cache[key]
        frame = fetch.frame
        fallback: FallbackFrame | None = None
        if frame is None:
            fallback = self._fallback_frame_from_metadata(trade)
            if fallback is None:
                diagnostic = self._mark_observation_pending(
                    db,
                    trade,
                    frame=None,
                    reason=fetch.error or "market_data_unavailable",
                    status="market_data_unavailable",
                    fallback=None,
                )
                diagnosed_ids.append(trade.id)
                data_errors.append(diagnostic)
                continue
            frame = fallback.frame

```

```

outcome = self._evaluate_trade(trade, frame)
if outcome is None:
    diagnostic = self._mark_observation_pending(
        db,
        trade,
        frame=frame,
        reason=fetch.error or "awaiting_future_marketBars",
        status="market_data_unavailable" if fetch.error else "awaiting_future_marketBars",
        fallback=fallback,
    )
    diagnosed_ids.append(trade.id)
    if fetch.error:
        data_errors.append(diagnostic)
    continue
self._close_trade(db, trade, outcome, fallback=fallback)
closed_ids.append(trade.id)
if closed_ids or diagnosed_ids:
    db.commit()
return {
    "open_observations": len(observations) - len(closed_ids),
    "closed_observations": len(closed_ids),
    "closed_trade_ids": closed_ids,
    "diagnosed_observations": len(diagnosed_ids),
    "diagnosed_trade_ids": diagnosed_ids,
    "data_errors": data_errors,
}
}

@staticmethod
def _is_lab_shadow_observation(trade: Trade) -> bool:
    metadata = trade.metadata_json or {}
    if str(metadata.get("execution_mode") or "") == LAB_SHADOW_EXECUTION_MODE:
        return True
    signal = trade.signal
    if signal is None:
        return False
    signal_meta = signal.metadata_json or {}
    return (
        signal_meta.get("entry_module") == "laboratory"
        and bool(metadata.get("observation_only"))
        and bool(metadata.get("no_ibkr_order"))
    )

def _fetch_frame(self, symbol: str, timeframe: str) -> MarketDataFetch:
    assert self.provider is not None
    try:
        frame = normalize_ohlcv(self.provider.fetch_ohlcv(symbol, period="6mo", interval=timeframe))
        if frame.empty:
            return MarketDataFetch(None, "provider_returned_no_bars")
        return MarketDataFetch(frame)
    except Exception as exc: # noqa: BLE001
        logger.warning("lab shadow observation data fetch failed for {} / {}: {}".format(symbol, timeframe, exc))
        return MarketDataFetch(None, f"provider_fetch_failed: {type(exc).__name__}: {exc}")

def _evaluate_trade(self, trade: Trade, frame: pd.DataFrame) -> dict[str, Any] | None:
    """Evaluate a shadow observation with the canonical triple-barrier engine.

    Parity contract (informe §6): shadow exits use the same fill rules as
    the backtester and Research RR simulation (`triple_barrier_outcome`),
    instead of an ad-hoc loop that filled gapped stops at the stop price.

    Construction: the shadow position exists from `opened_at` at
    `trade.entry`, before the first future bar. Two synthetic bars pinned
    at the entry price (signal bar + entry bar) precede the future bars so
    every real bar is "posterior to entry" and the canonical open-gap
    rules (fill at the OPEN through a stop, at the TARGET through a
    target) apply to all of them, including the first.
    """
    opened_at = self._as_utc(trade.opened_at)
    future = frame[[self._as_utc(idx) > opened_at for idx in frame.index]]
    if future.empty:
        return None
    side = -1 if trade.side.lower().strip() == "short" else 1
    entry = float(trade.entry)
    risk = abs(entry - float(trade.stop))
    if risk <= 0:
        return None
    max_holding_bars = max(1, max(self.settings.discovery_forward_bar_list if self.settings else [20]))
    synthetic = [entry, entry]
    out = triple_barrier_outcome(
        synthetic + future["open"].astype(float).tolist(),
        synthetic + future["high"].astype(float).tolist(),
        synthetic + future["low"].astype(float).tolist(),
        synthetic + future["close"].astype(float).tolist(),
        signal_index=0,
        side=side,
        stop_price=float(trade.stop),
        target_price=float(trade.target),
        # +1: the synthetic entry bar consumes the first slot of the window
        max_bars=max_holding_bars + 1,
        entry_price=entry,
        gap_entry_policy="skip",
        conservative_both=True,
        round_trip_cost_R=0.0,
    )
    if out["status"] != "ok":
        # 'skipped'/'invalid' only occur for malformed barriers (entry at

```

```

        # or beyond stop/target); keep the observation pending so the
        # diagnostic path surfaces it instead of fabricating an exit.
        return None
    reason = str(out["reason"])
    if reason == "time" and len(future) < max_holdingBars:
        return None
    exit_pos = max(0, min(int(out["exit_index"]) - len(synthetic), len(future) - 1))
    rows_seen = [row.to_dict() for _, row in future.iloc[: exit_pos + 1].iterrows()]
    return {
        "exit_price": float(out["exit_price"]),
        "closed_at": self._as_utc(future.index[exit_pos]),
        "exit_reason": CANONICAL_EXIT_REASON_MAP.get(reason, reason),
        "bars": rows_seen,
        "canonical_outcome": {
            "engine": "triple_barrier_outcome",
            "status": str(out["status"]),
            "reason": reason,
            "r_multiple": round(float(out["R"]), 6),
            "mfe_R": round(float(out["mfe_R"]), 6),
            "mae_R": round(float(out["mae_R"]), 6),
            "bars_held": max(0, int(out["bars_held"]) - 1),
            "conservative_both": True,
            "gap_entry_policy": "skip",
            "round_trip_cost_R": 0.0,
        },
    },

def _close_trade(
    self,
    db: Session,
    trade: Trade,
    outcome: dict[str, Any],
    *,
    fallback: FallbackFrame | None = None,
) -> None:
    exit_price = float(outcome["exit_price"])
    risk = abs(float(trade.entry) - float(trade.stop))
    if trade.side.lower().strip() == "short":
        pnl = (float(trade.entry) - exit_price) * int(trade.qty)
        r_multiple = (float(trade.entry) - exit_price) / max(risk, 1e-9)
    else:
        pnl = (exit_price - float(trade.entry)) * int(trade.qty)
        r_multiple = (exit_price - float(trade.entry)) / max(risk, 1e-9)
    closed_at = outcome["closed_at"]
    metadata = with_evidence_metadata(
        trade.metadata_json or {},
        trade_status=TradeStatus.CLOSED,
        evidence_quality=EvidenceQuality.DEGRADED.value if fallback is not None else None,
        fill_provenance=FillProvenance.SHADOW_CLOSE.value,
        degradation_reason=(
            f"fallback_market_data:{fallback.source}" if fallback is not None else None
        ),
    ),
    bars = outcome.get("bars") or []
    mfe, mae = self._mfe_mae_r(trade, bars)
    metadata.update(
        {
            "evidence_quality": (
                EvidenceQuality.DEGRADED.value
                if fallback is not None
                else metadata.get("evidence_quality", EvidenceQuality.NORMAL.value)
            ),
            "fallback_used": fallback is not None,
            "fallback_source": fallback.source if fallback is not None else None,
            "fallback_timestamp_source": fallback.timestamp_source if fallback is not None else None,
            "exit_reason": outcome["exit_reason"],
            "exit_fill_time": closed_at.isoformat(),
            "exit_fill_price": round(exit_price, 4),
            "exit_order_type": "shadow_observation_exit",
            "gross_pnl": round(pnl, 4),
            "net_pnl": round(pnl, 4),
            "mfe": round(mfe, 6),
            "mae": round(mae, 6),
            "holding_period_seconds": int((closed_at - self._as_utc(trade.opened_at)).total_seconds()),
            "closed_by": "lab_shadow_observation_lifecycle",
            "canonical_outcome": outcome.get("canonical_outcome"),
        }
    )
    trade.status = TradeStatus.CLOSED
    trade.closed_at = closed_at
    trade.exit_price = round(exit_price, 4)
    trade.pnl_usd = round(pnl, 4)
    trade.r_multiple = round(r_multiple, 6)
    trade.evidence_type = metadata.get("evidence_type")
    trade.evidence_quality = metadata.get("evidence_quality")
    trade.metadata_json = metadata
    db.add(trade)
    db.add(
        AuditLog(
            actor="laboratory",
            action="lab_shadow_observation_closed",
            entity_type="trade",
            entity_id=str(trade.id),
            details_json={
                "trade_id": trade.id,
                "symbol": trade.symbol,
            },
        )
    )

```

```

        "exit_reason": outcome["exit_reason"],
        "r_multiple": trade.r_multiple,
        "evidence_type": metadata.get("evidence_type"),
        "evidence_quality": metadata.get("evidence_quality"),
        "fallback_used": fallback is not None,
        "no_ibkr_order": True,
    },
)
)

def _mark_observation_pending(
    self,
    db: Session,
    trade: Trade,
    *,
    frame: pd.DataFrame | None,
    reason: str,
    status: str,
    fallback: FallbackFrame | None,
) -> dict[str, str]:
    now = datetime.now(timezone.utc)
    metadata = with_evidence_metadata(
        trade.metadata_json or {},
        evidence_quality=EvidenceQuality.DEGRADED.value if fallback is not None else None,
        degradation_reason=(
            f"fallback_market_data:{fallback.source}" if fallback is not None else None
        ),
    )
    last_bar = self._last_bar_snapshot(frame)
    mark = self._mark_to_market(trade, last_bar)
    lifecycle = {
        "status": status,
        "reason": reason,
        "diagnosed_at": now.isoformat(),
        "fallback_used": fallback is not None,
        "fallback_source": fallback.source if fallback is not None else None,
        "fallback_timestamp_source": fallback.timestamp_source if fallback is not None else None,
        "last_bar": last_bar,
        "mark_to_market": mark,
        "no_ibkr_order": True,
        "paper_only": True,
    }
    metadata.update(
        {
            "shadow_lifecycle": lifecycle,
            "last_shadow_lifecycle_status": status,
            "market_data_unavailable": status == "market_data_unavailable",
            "market_data_unavailable_reason": reason if status == "market_data_unavailable" else None,
            "evidence_quality": (
                EvidenceQuality.DEGRADED.value
                if fallback is not None
                else metadata.get("evidence_quality", EvidenceQuality.NORMAL.value)
            ),
            "fallback_used": fallback is not None,
            "fallback_source": fallback.source if fallback is not None else None,
            "fallback_timestamp_source": fallback.timestamp_source if fallback is not None else None,
            "no_ibkr_order": True,
            "paper_only": True,
        }
    )
    trade.metadata_json = metadata
    trade.evidence_type = metadata.get("evidence_type")
    trade.evidence_quality = metadata.get("evidence_quality")
    db.add(trade)
    db.add(
        AuditLog(
            actor="laboratory",
            action=(
                "lab_shadow_observation_market_data_unavailable"
                if status == "market_data_unavailable"
                else "lab_shadow_observation_pendingBars"
            ),
            entity_type="trade",
            entity_id=str(trade.id),
            details_json={
                "trade_id": trade.id,
                "symbol": trade.symbol,
                "status": status,
                "reason": reason,
                "fallback_used": fallback is not None,
                "fallback_source": fallback.source if fallback is not None else None,
                "mark_to_market": mark,
                "no_ibkr_order": True,
            },
        )
    )
    return {
        "trade_id": str(trade.id),
        "symbol": trade.symbol,
        "reason": reason,
        "status": status,
        "diagnosed_at": now.isoformat(),
        "fallback_used": str(fallback is not None).lower(),
    }

def _fallback_frame_from_metadata(self, trade: Trade) -> FallbackFrame | None:

```

```

candidate = self._last_market_bar_candidate(trade)
if candidate is None:
    return None
price = self._safe_positive_float(candidate.get("close") or candidate.get("price"))
if price is None:
    return None
open_price = self._safe_positive_float(candidate.get("open")) or price
high = max(self._safe_positive_float(candidate.get("high")) or price, open_price, price)
low = min(self._safe_positive_float(candidate.get("low")) or price, open_price, price)
volume = self._safe_non_negative_float(candidate.get("volume")) or 0.0
timestamp = self._timestamp_or_opened_at(candidate.get("timestamp"), trade.opened_at)
frame = pd.DataFrame(
    {
        "open": [open_price],
        "high": [high],
        "low": [low],
        "close": [price],
        "volume": [volume],
    },
    index=pd.DatetimeIndex([timestamp]),
)
return FallbackFrame(
    frame=normalize_ohlc(frame),
    source=str(candidate.get("source") or "metadata"),
    timestamp_source=str(candidate.get("timestamp_source") or "opened_at"),
)

def _last_market_bar_candidate(self, trade: Trade) -> dict[str, Any] | None:
    metadata = trade.metadata_json or {}
    signal_meta = (trade.signal.metadata_json if trade.signal is not None else {}) or {}
    sources = [
        ("trade.metadata_json", metadata),
        ("signal.metadata_json", signal_meta),
        ("trade.signal_snapshot", metadata.get("signal_snapshot")),
        ("signal.signal_snapshot", signal_meta.get("signal_snapshot")),
        ("trade.match", metadata.get("match")),
        ("signal.match", signal_meta.get("match")),
    ]
    for source_name, payload in sources:
        candidate = self._bar_from_payload(payload, source_name)
        if candidate is not None:
            return candidate
    return None

def _bar_from_payload(self, payload: Any, source_name: str) -> dict[str, Any] | None:
    if not isinstance(payload, dict):
        return None
    for key in ("last_bar", "latest_bar", "market_bar", "bar", "ohlc"):
        bar = payload.get(key)
        if isinstance(bar, dict) and self._safe_positive_float(bar.get("close") or bar.get("price")) is not None:
            return {
                **bar,
                "source": f"{source_name}.{key}",
                "timestamp": bar.get("timestamp") or bar.get("time") or bar.get("date"),
                "timestamp_source": f"{source_name}.{key}",
            }
    entry_gate = payload.get("entry_gate")
    if isinstance(entry_gate, dict):
        price = self._safe_positive_float(entry_gate.get("close") or entry_gate.get("last_close"))
        if price is not None:
            return {
                "price": price,
                "open": entry_gate.get("open"),
                "high": entry_gate.get("high"),
                "low": entry_gate.get("low"),
                "volume": entry_gate.get("volume"),
                "timestamp": self._first_present(
                    entry_gate.get("timestamp"),
                    entry_gate.get("bar_close_time"),
                    entry_gate.get("decision_ts"),
                    entry_gate.get("available_data_cutoff_ts"),
                ),
                "source": f"{source_name}.entry_gate.close",
                "timestamp_source": f"{source_name}.entry_gate",
            }
    features = payload.get("features")
    if isinstance(features, dict):
        price = self._safe_positive_float(features.get("last_close") or features.get("close"))
        if price is not None:
            return {
                "price": price,
                "timestamp": self._snapshot_timestamp(payload),
                "source": f"{source_name}.features.last_close",
                "timestamp_source": f"{source_name}.captured_at",
            }
    metrics = payload.get("metrics")
    if isinstance(metrics, dict):
        return self._bar_from_payload(metrics, f"{source_name}.metrics")
    snapshot = payload.get("signal_snapshot")
    if isinstance(snapshot, dict):
        return self._bar_from_payload(snapshot, f"{source_name}.signal_snapshot")
    return None

def _last_bar_snapshot(self, frame: pd.DataFrame | None) -> dict[str, Any] | None:
    if frame is None or frame.empty:
        return None

```



```

idx = frame.index[-1]
row = frame.iloc[-1]
return {
    "timestamp": self._as_utc(idx).isoformat(),
    "open": round(float(row["open"]), 4),
    "high": round(float(row["high"]), 4),
    "low": round(float(row["low"]), 4),
    "close": round(float(row["close"]), 4),
    "volume": round(float(row["volume"]), 4),
}

def _mark_to_market(self, trade: Trade, last_bar: dict[str, Any] | None) -> dict[str, Any] | None:
    if last_bar is None:
        return None
    last_price = self._safe_positive_float(last_bar.get("close"))
    if last_price is None:
        return None
    risk = abs(float(trade.entry) - float(trade.stop))
    qty = int(trade.qty)
    if trade.side.lower().strip() == "short":
        unrealized = (float(trade.entry) - last_price) * qty
        r_multiple = (float(trade.entry) - last_price) / max(risk, 1e-9)
    else:
        unrealized = (last_price - float(trade.entry)) * qty
        r_multiple = (last_price - float(trade.entry)) / max(risk, 1e-9)
    return {
        "price": round(last_price, 4),
        "unrealized_pnl": round(unrealized, 4),
        "unrealized_r": round(r_multiple, 6),
        "entry": round(float(trade.entry), 4),
        "stop": round(float(trade.stop), 4),
        "target": round(float(trade.target), 4),
        "qty": qty,
    }

@staticmethod
def _mfe_mae_r(trade: Trade, bars: list[dict[str, Any]]) -> tuple[float, float]:
    risk = abs(float(trade.entry) - float(trade.stop))
    if risk <= 0 or not bars:
        return 0.0, 0.0
    highs = [float(row["high"]) for row in bars]
    lows = [float(row["low"]) for row in bars]
    if trade.side.lower().strip() == "short":
        mfe = (float(trade.entry) - min(lows)) / risk
        mae = (float(trade.entry) - max(highs)) / risk
    else:
        mfe = (max(highs) - float(trade.entry)) / risk
        mae = (min(lows) - float(trade.entry)) / risk
    return mfe, mae

@staticmethod
def _as_utc(value: Any) -> datetime:
    ts = pd.Timestamp(value)
    if ts.tzinfo is None:
        ts = ts.tz_localize(timezone.utc)
    else:
        ts = ts.tz_convert(timezone.utc)
    return ts.to_pydatetime()

@staticmethod
def _safe_positive_float(value: Any) -> float | None:
    try:
        number = float(value)
    except (TypeError, ValueError):
        return None
    if not pd.notna(number) or number <= 0:
        return None
    return number

@staticmethod
def _safe_non_negative_float(value: Any) -> float | None:
    try:
        number = float(value)
    except (TypeError, ValueError):
        return None
    if not pd.notna(number) or number < 0:
        return None
    return number

@staticmethod
def _timestamp_or_opened_at(value: Any, opened_at: Any) -> datetime:
    if value is None or value == "":
        return LabPaperObservationService._as_utc(opened_at)
    try:
        return LabPaperObservationService._as_utc(value)
    except Exception: # noqa: BLE001
        return LabPaperObservationService._as_utc(opened_at)

@staticmethod
def _first_present(*values: Any) -> Any:
    for value in values:
        if value not in (None, ""):
            return value
    return None

@staticmethod

```

```
def _snapshot_timestamp(payload: dict[str, Any]) -> Any:
    entry_audit = payload.get("entry_audit") if isinstance(payload.get("entry_audit"), dict) else {}
    return LabPaperObservationService._first_present(
        payload.get("captured_at"),
        entry_audit.get("available_data_cutoff_ts"),
        entry_audit.get("decision_ts"),
        payload.get("window_end"),
    )
```

Full Source: ``backend/tradeo/services/lab_diagnostics.py``

``backend/tradeo/services/lab_diagnostics.py`` ? 4 lines, 130 bytes, sha256 prefix ``9eb04be77d16540c``

```
"""Backward-compatible imports for Laboratory diagnostics."""
```

```
from tradeo.modules.laboratory.diagnostics import * # noqa: F403
```

Full Source: ``backend/tradeo/modules/laboratory/diagnostics.py``

``backend/tradeo/modules/laboratory/diagnostics.py`` ? 606 lines, 23800 bytes, sha256 prefix ``38dfb72926b894ed``

Public surface summary before full source:

? function ``laboratory_diagnostics`` line 20`

Risk hints: wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```
from __future__ import annotations

from collections import defaultdict
from datetime import datetime, timezone
from typing import Any

from sqlalchemy.orm import Session, joinedload

from tradeo.db.models import AuditLog, DiscoveredPattern, DiscoveredPatternMatch, Signal, Trade
from tradeo.services.evidence import evidence_metadata_with_stored_columns, is_paper_order_or_fill_evidence

REJECTION_ACTIONS = {
    "entry_match_rejected_by_entry_gate": "entry_gate",
    "entry_match_rejected_by_quality": "entry_quality",
    "entry_match_rejected_by_risk": "risk",
    "entry_match_skipped_by_cooldown": "cooldown",
}

def laboratory_diagnostics(db: Session, *, limit: int = 24) -> dict[str, Any]:
    """Read-only presentation payload for Lab candidate and near-miss diagnostics."""
    limit = max(1, min(limit, 100))
    rows = _dedupe_rows(
        [
            *_signal_rows(db, max_rows=max(limit * 4, 80)),
            *_rejection_rows(db, max_rows=max(limit * 6, 120)),
        ],
        _shadow_match_rows(db, max_rows=max(limit * 4, 80)),
    )
    patterns = _patterns_by_id(db, rows)
    history = _paper_history_index(db)
    enriched = [_enrich_row(row, patterns, history) for row in rows]
    enriched.sort(key=_sort_key, reverse=True)
    visible = enriched[:limit]
    return {
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "summary": {
            "candidates": sum(1 for row in enriched if row["source"] == "candidate_signal"),
            "rejected": sum(1 for row in enriched if row["source"] == "rejected"),
            "shadow_near_misses": sum(1 for row in enriched if row["source"] == "shadow_match"),
            "with_paper_history": sum(
                1 for row in enriched if row["paper_history"]["closed_trades"] > 0
            ),
        },
        "entry_gate_failures": sum(
            1
            for row in enriched
            if row.get("entry_gate", {}).get("passed") is False
            or row.get("rejection_stage") == "entry_gate"
        ),
        "returned": len(visible),
        "available": len(enriched),
    },
    "opportunities": visible,
}
```

```
def _signal_rows(db: Session, *, max_rows: int) -> list[dict[str, Any]]:
    signals = db.query(Signal).order_by(Signal.created_at.desc()).limit(max_rows * 3).all()
    rows: list[dict[str, Any]] = []
    for signal in signals:
        if len(rows) >= max_rows:
            break
        if not _is_laboratory_signal(signal):
```

```

        continue
    metadata = signal.metadata_json or {}
    if not any(
        key in metadata for key in ("match", "entry_gate", "entry_quality", "opportunity_rank")
    ):
        continue
    match = _dict(metadata.get("match"))
    snapshot = _dict(metadata.get("signal_snapshot"))
    entry_gate = _first_dict(
        metadata.get("entry_gate"),
        _dict(match.get("metrics")).get("entry_gate"),
        snapshot.get("entry_gate"),
    )
    entry_quality = _first_dict(metadata.get("entry_quality"), snapshot.get("entry_quality"))
    row = _base_match_row(
        source="candidate_signal",
        status=_status_value(signal.status),
        symbol=signal.symbol,
        pattern=signal.pattern,
        pattern_id=_safe_int(metadata.get("pattern_id") or match.get("pattern_id")),
        side=signal.side,
        timeframe=signal.timeframe,
        entry_variant_id=str(metadata.get("entry_variant_id") or match.get("entry_variant_id") or ""),
        entry_variant=_first_dict(metadata.get("entry_variant"), match.get("entry_variant")),
        regime=_first_dict(metadata.get("regime"), match.get("regime"), snapshot.get("regime")),
        regime_fit=_first_dict(
            metadata.get("regime_fit"), match.get("regime_fit"), snapshot.get("regime_fit")
        ),
        rank=metadata.get("opportunity_rank"),
        rank_score=metadata.get("opportunity_rank_score"),
        rank_components=_dict(metadata.get("opportunity_rank_components")),
        score=match.get("score", signal.composite_score),
        entry_score=entry_gate.get("entry_score", signal.composite_score),
        similarity=match.get("similarity"),
        entry_gate=entry_gate,
        entry_quality=entry_quality,
        rejection_stage=None,
        rejection_reason=_candidate_reason(signal, entry_quality),
        event_at=signal.created_at,
        window_end=str(match.get("window_end") or ""),
        source_id=signal.id,
    )
    rows.append(row)
return rows

def _rejection_rows(db: Session, *, max_rows: int) -> list[dict[str, Any]]:
    logs = (
        db.query(AuditLog)
        .filter(AuditLog.actor == "laboratory")
        .filter(AuditLog.action.in_(list(REJECTION_ACTIONS)))
        .order_by(AuditLog.timestamp.desc())
        .limit(max_rows)
        .all()
    )
    rows: list[dict[str, Any]] = []
    for log in logs:
        details = log.details_json or {}
        match = _dict(details.get("match"))
        metrics = _dict(match.get("metrics"))
        entry_gate = _first_dict(details.get("entry_gate"), metrics.get("entry_gate"))
        entry_quality = _dict(details.get("entry_quality"))
        risk = _dict(details.get("risk"))
        row = _base_match_row(
            source="rejected",
            status=log.action,
            symbol=str(match.get("symbol") or ""),
            pattern=str(match.get("pattern_name") or match.get("pattern") or ""),
            pattern_id=_safe_int(match.get("pattern_id") or log.entity_id),
            side=str(match.get("side") or ""),
            timeframe=str(match.get("timeframe") or ""),
            entry_variant_id=str(match.get("entry_variant_id") or metrics.get("entry_variant_id") or ""),
            entry_variant=_first_dict(match.get("entry_variant"), metrics.get("entry_variant")),
            regime=_first_dict(match.get("regime"), metrics.get("regime")),
            regime_fit=_first_dict(match.get("regime_fit"), metrics.get("regime_fit")),
            rank=match.get("opportunity_rank"),
            rank_score=match.get("opportunity_rank_score"),
            rank_components=_dict(match.get("opportunity_rank_components")),
            score=match.get("score"),
            entry_score=match.get("entry_score", entry_gate.get("entry_score")),
            similarity=match.get("similarity"),
            entry_gate=entry_gate,
            entry_quality=entry_quality,
            rejection_stage=REJECTION_ACTIONS.get(log.action, "unknown"),
            rejection_reason=_rejection_reason(log.action, entry_gate, entry_quality, risk, details),
            event_at=log.timestamp,
            window_end=str(match.get("window_end") or ""),
            source_id=log.id,
        )
        rows.append(row)
    return rows

def _shadow_match_rows(db: Session, *, max_rows: int) -> list[dict[str, Any]]:
    matches = (
        db.query(DiscoveredPatternMatch)

```

```

        .options(joinedload(DiscoveredPatternMatch.pattern))
        .filter(DiscoveredPatternMatch.status.in_({"lab_entry_candidate", "lab_watchlist"}))
        .order_by(DiscoveredPatternMatch.matched_at.desc(), DiscoveredPatternMatch.score.desc())
        .limit(max_rows)
        .all()
    )
rows: list[dict[str, Any]] = []
for match in matches:
    metrics = match.metrics_json or {}
    entry_gate = _dict(metrics.get("entry_gate"))
    pattern = match.pattern
    pattern_name = pattern.name if pattern is not None else f"pattern_{match.pattern_id}"
    row = _base_match_row(
        source="shadow_match",
        status=match.status,
        symbol=match.symbol,
        pattern=pattern_name,
        pattern_id=match.pattern_id,
        side=match.side,
        timeframe=match.timeframe,
        entry_variant_id=str(metrics.get("entry_variant_id") or ""),
        entry_variant=_dict(metrics.get("entry_variant")),
        regime=_dict(metrics.get("regime")),
        regime_fit=_dict(metrics.get("regime_fit")),
        rank=None,
        rank_score=None,
        rank_components={},
        score=match.score,
        entry_score=entry_gate.get("entry_score"),
        similarity=match.similarity,
        entry_gate=entry_gate,
        entry_quality={},
        rejection_stage="shadow_near_miss",
        rejection_reason=_shadow_reason(entry_gate),
        event_at=match.matched_at,
        window_end=match.window_end,
        source_id=match.id,
    )
    rows.append(row)
return rows

def _base_match_row(**values: Any) -> dict[str, Any]:
    variant = _dict(values.get("entry_variant"))
    regime = _dict(values.get("regime"))
    return {
        "source": values.get("source"),
        "status": values.get("status") or "",
        "symbol": values.get("symbol") or "",
        "pattern": values.get("pattern") or "",
        "pattern_id": values.get("pattern_id"),
        "side": values.get("side") or "",
        "timeframe": values.get("timeframe") or "",
        "entry_variant_id": values.get("entry_variant_id") or "",
        "entry_variant_label": _variant_label(variant, values.get("entry_variant_id")),
        "regime_key": str(regime.get("regime_key") or ""),
        "rank": _safe_int(values.get("rank")),
        "rank_score": _safe_float(values.get("rank_score")),
        "score": _safe_float(values.get("score")),
        "entry_score": _safe_float(values.get("entry_score")),
        "similarity": _safe_float(values.get("similarity")),
        "opportunity_rank_components": _clean_dict(values.get("rank_components")),
        "entry_gate": _clean_dict(values.get("entry_gate")),
        "entry_quality": _clean_dict(values.get("entry_quality")),
        "regime_fit": _clean_dict(values.get("regime_fit")),
        "rejection_stage": values.get("rejection_stage"),
        "rejection_reason": values.get("rejection_reason") or "",
        "created_at": _iso(values.get("event_at")),
        "window_end": values.get("window_end") or "",
        "_source_id": values.get("source_id"),
    }

def _enrich_row(
    row: dict[str, Any],
    patterns: dict[int, DiscoveredPattern],
    history_index: dict[str, Any],
) -> dict[str, Any]:
    pattern = patterns.get(int(row["pattern_id"])) if row.get("pattern_id") is not None else None
    history = _paper_history_for(row, history_index)
    missing = _missing_to_promote(pattern, history)
    return {
        **{key: value for key, value in row.items() if not key.startswith("_")},
        "entry_gate_components": _entry_gate_components(row.get("entry_gate") or {}),
        "paper_history": history,
        "promotion": _promotion_payload(pattern, missing),
        "missing_to_promote": missing,
    }

def _dedupe_rows(
    primary_rows: list[dict[str, Any]],
    shadow_rows: list[dict[str, Any]],
) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    seen: set[tuple[Any, ...]] = set()

```

```

for row in [*primary_rows, *shadow_rows]:
    key = _dedupe_key(row)
    if row["source"] == "shadow_match" and key in seen:
        continue
    seen.add(key)
    rows.append(row)
return rows

def _dedupe_key(row: dict[str, Any]) -> tuple[Any, ...]:
    return (
        row.get("symbol"),
        row.get("pattern_id"),
        row.get("entry_variant_id"),
        row.get("window_end") or row.get("_source_id"),
    )

def _patterns_by_id(db: Session, rows: list[dict[str, Any]]) -> dict[int, DiscoveredPattern]:
    pattern_ids = sorted({int(row["pattern_id"]) for row in rows if row.get("pattern_id") is not None})
    if not pattern_ids:
        return {}
    return {
        pattern.id: pattern
        for pattern in db.query(DiscoveredPattern).filter(DiscoveredPattern.id.in_(pattern_ids)).all()
    }

def _paper_history_index(db: Session) -> dict[str, Any]:
    trades = (
        db.query(Trade)
        .options(joinedload(Trade.signal))
        .order_by(Trade.opened_at.desc())
        .limit(800)
        .all()
    )
    by_id: dict[int, list[Trade]] = defaultdict(list)
    by_name: dict[str, list[Trade]] = defaultdict(list)
    for trade in trades:
        signal = trade.signal
        if signal is not None:
            if not _is_laboratory_signal(signal):
                continue
            metadata = signal.metadata_json or {}
        else:
            metadata = trade.metadata_json or {}
            if not _is_laboratory_trade_metadata(metadata):
                continue
        trade_metadata = evidence_metadata_with_stored_columns(
            trade.metadata_json or {},
            evidence_type=trade.evidence_type,
            evidence_quality=trade.evidence_quality,
        )
        if not _is_paper_order_or_fill_evidence(
            trade_metadata,
            trade_status=trade.status,
            signal_metadata=(signal.metadata_json if signal is not None else {}) or {},
            broker_order_id=trade.broker_order_id,
        ):
            continue
        pattern_id = _safe_int(metadata.get("pattern_id"))
        if pattern_id is not None:
            by_id[pattern_id].append(trade)
        by_name[str(trade.pattern)].append(trade)
    return {"by_id": by_id, "by_name": by_name}

def _paper_history_for(row: dict[str, Any], history_index: dict[str, Any]) -> dict[str, Any]:
    pattern_id = row.get("pattern_id")
    trades: list[Trade] = []
    if pattern_id is not None:
        trades = list(history_index["by_id"].get(int(pattern_id), []))
    if not trades and row.get("pattern"):
        trades = list(history_index["by_name"].get(str(row["pattern"]), []))
    closed = [trade for trade in trades if _status_value(trade.status) == "closed"]
    open_trades = [trade for trade in trades if _status_value(trade.status) == "open"]
    r_values = [float(trade.r_multiple or 0.0) for trade in closed]
    wins = [value for value in r_values if value > 0]
    losses = [abs(value) for value in r_values if value < 0]
    variant = str(row.get("entry_variant_id") or "")
    regime = str(row.get("regime_key") or "")
    return {
        "closed_trades": len(closed),
        "open_trades": len(open_trades),
        "total_r": round(sum(r_values), 4),
        "expectancy_r": round(sum(r_values) / len(r_values), 4) if r_values else 0.0,
        "win_rate": round(len(wins) / len(r_values), 4) if r_values else 0.0,
        "profit_factor": round(sum(wins) / sum(losses), 4) if losses else round(sum(wins), 4),
        "unique_symbols": len({trade.symbol for trade in closed}),
        "unique_days": len(
            {
                (trade.closed_at or trade.opened_at).date().isoformat()
                for trade in closed
                if trade.closed_at or trade.opened_at
            }
        ),
    },

```

```

        "variant_closed_trades": sum(1 for trade in closed if _trade_variant(trade) == variant),
        "regime_closed_trades": sum(1 for trade in closed if _trade_regime(trade) == regime),
        "last_trade_status": _status_value(trades[0].status) if trades else None,
    }

def _missing_to_promote(
    pattern: DiscoveredPattern | None,
    history: dict[str, Any],
) -> list[str]:
    if pattern is None:
        return ["missing_pattern_record"]
    status = _status_value(pattern.status)
    if status == "production":
        return []
    lab_execution = _dict((pattern.metrics_json or {}).get("lab_execution"))
    blockers = [str(item) for item in lab_execution.get("promotion_blockers", []) if item]
    if blockers:
        return blockers
    if lab_execution.get("eligible_for_director_review") is True:
        return ["ready_for_director_review"]

    missing: list[str] = []
    if int(history["closed_trades"]) < 10:
        missing.append("closed_lab_trades_below_10")
    if int(history["unique_symbols"]) < 3:
        missing.append("unique_lab_symbols_below_3")
    if int(history["unique_days"]) < 3:
        missing.append("unique_lab_days_below_3")
    if int(history["closed_trades"]) > 0 and float(history["expectancy_r"]) < 0:
        missing.append("lab_expectancy_below_0.0r")
    if int(history["closed_trades"]) > 0 and float(history["profit_factor"]) < 1.2:
        missing.append("lab_profit_factor_below_1.2")
    if not missing:
        missing.append("needs_director_review_refresh")
    return missing

def _promotion_payload(pattern: DiscoveredPattern | None, missing: list[str]) -> dict[str, Any]:
    if pattern is None:
        return {
            "pattern_status": "unknown",
            "promotion_status": "unknown",
            "promotion_reason": "",
            "eligible_for_director_review": False,
            "blockers": missing,
        }
    lab_execution = _dict((pattern.metrics_json or {}).get("lab_execution"))
    return {
        "pattern_status": _status_value(pattern.status),
        "promotion_status": pattern.promotion_status,
        "promotion_reason": pattern.promotion_reason,
        "eligible_for_director_review": bool(lab_execution.get("eligible_for_director_review", False)),
        "blockers": missing,
    }

def _entry_gate_components(entry_gate: dict[str, Any]) -> list[dict[str, Any]]:
    if not entry_gate:
        return []
    trigger = str(entry_gate.get("trigger") or "")
    return [
        {
            "name": "trigger",
            "value": trigger or None,
            "ok": trigger not in {"", "no_operational_trigger", "insufficient_history"},
        },
        {
            "name": "volume",
            "value": _safe_float(entry_gate.get("volume_ratio")),
            "ok": _optional_bool(entry_gate.get("volume_confirmed")),
        },
        {
            "name": "regime",
            "value": _safe_float(entry_gate.get("regime_score")),
            "ok": _optional_bool(entry_gate.get("regime_ok")),
        },
        {
            "name": "extension",
            "value": _safe_float(entry_gate.get("extension_atr")),
            "ok": _optional_bool(entry_gate.get("not_extended")),
        },
        {
            "name": "trend",
            "value": "aligned" if entry_gate.get("trend_aligned") is True else "off",
            "ok": _optional_bool(entry_gate.get("trend_aligned")),
        },
        {
            "name": "atr",
            "value": _safe_float(entry_gate.get("atr_pct")),
            "ok": _optional_bool(entry_gate.get("volatility_ok")),
        },
    ]

def _candidate_reason(signal: Signal, entry_quality: dict[str, Any]) -> str:

```

```

flags = [str(item) for item in entry_quality.get("flags", []) if item]
if flags:
    return ", ".join(flags)
if signal.human_approved:
    return "paper_execution_requested"
return "paper_candidate_not_submitted"

def _rejection_reason(
    action: str,
    entry_gate: dict[str, Any],
    entry_quality: dict[str, Any],
    risk: dict[str, Any],
    details: dict[str, Any],
) -> str:
    if action == "entry_match_rejected_by_entry_gate":
        blockers = []
        if entry_gate.get("trigger") in {"no_operational_trigger", "insufficient_history":
            blockers.append(str(entry_gate.get("trigger")))
        if entry_gate.get("volume_confirmed") is False:
            blockers.append("weak_volume_confirmation")
        if entry_gate.get("regime_ok") is False:
            blockers.append("regime_filter_failed")
        if entry_gate.get("not_extended") is False:
            blockers.append("overextended")
        return ", ".join(blockers) or str(entry_gate.get("reason") or "entry_gate_failed")
    if action == "entry_match_rejected_by_quality":
        flags = [str(item) for item in entry_quality.get("flags", []) if item]
        return ", ".join(flags) or "entry_quality_below_threshold"
    if action == "entry_match_rejected_by_risk":
        hard = [str(item) for item in risk.get("hard_rejections", []) if item]
        return ", ".join(hard) or str(risk.get("reason") or "risk_rejected")
    if action == "entry_match_skipped_by_cooldown":
        return f"cooldown_{details.get('cooldown_minutes', '?')}_minutes"
    return action

def _shadow_reason(entry_gate: dict[str, Any]) -> str:
    if entry_gate.get("passed") is False:
        return _rejection_reason("entry_match_rejected_by_entry_gate", entry_gate, {}, {}, {})
    return "stored_match_without_signal"

def _is_laboratory_signal(signal: Signal) -> bool:
    metadata = signal.metadata_json or {}
    if metadata.get("entry_module") == "laboratory":
        return True
    purpose = str(metadata.get("purpose") or "")
    return signal.strategy_version.startswith(("laboratory_", "ibkr_paper_", "ibkr_smoke_")) or (
        purpose.startswith("ibkr_paper_")
    )

def _is_laboratory_trade_metadata(metadata: dict[str, Any]) -> bool:
    return (
        metadata.get("entry_module") == "laboratory"
        or metadata.get("source_module") == "laboratory"
        or metadata.get("execution_mode") == "paper"
        or metadata.get("ibkr_mode") == "paper"
    )

def _trade_variant(trade: Trade) -> str:
    metadata = (trade.signal.metadata_json if trade.signal is not None else {}) or {}
    return str(metadata.get("entry_variant_id") or "")

def _trade_regime(trade: Trade) -> str:
    metadata = (trade.signal.metadata_json if trade.signal is not None else {}) or {}
    return str((_dict(metadata.get("regime"))).get("regime_key") or "")

def _sort_key(row: dict[str, Any]) -> tuple[float, float, float]:
    opportunity_score = row.get("rank_score")
    if opportunity_score is None:
        opportunity_score = row.get("score") or row.get("entry_score") or 0.0
    source_weight = {"candidate_signal": 0.03, "rejected": 0.02, "shadow_match": 0.01}.get(
        row["source"], 0.0
    )
    return (float(opportunity_score or 0.0), source_weight, _timestamp(row.get("created_at")))

def _variant_label(variant: dict[str, Any], variant_id: Any) -> str:
    return str(
        variant.get("label")
        or variant.get("name")
        or variant.get("order_style")
        or variant.get("id")
        or variant_id
        or ""
    )

def _first_dict(*values: Any) -> dict[str, Any]:
    for value in values:
        item = _dict(value)

```

```

        if item:
            return item
    return {}

def _dict(value: Any) -> dict[str, Any]:
    return value if isinstance(value, dict) else {}

def _clean_dict(value: Any) -> dict[str, Any]:
    item = _dict(value)
    return {str(key): _clean_value(value) for key, value in item.items()}

def _clean_value(value: Any) -> Any:
    if isinstance(value, dict):
        return {str(key): _clean_value(item) for key, item in value.items()}
    if isinstance(value, list):
        return [_clean_value(item) for item in value]
    if isinstance(value, (str, int, float, bool)) or value is None:
        return value
    return str(value)

def _status_value(value: Any) -> str:
    return str(value.value if hasattr(value, "value") else value)

def _safe_int(value: Any) -> int | None:
    try:
        if value is None or value == "":
            return None
        return int(value)
    except (TypeError, ValueError):
        return None

def _safe_float(value: Any) -> float | None:
    try:
        if value is None or value == "":
            return None
        return round(float(value), 6)
    except (TypeError, ValueError):
        return None

def _optional_bool(value: Any) -> bool | None:
    return value if isinstance(value, bool) else None

def _iso(value: Any) -> str:
    if isinstance(value, datetime):
        return value.isoformat()
    return str(value or "")

def _timestamp(value: Any) -> float:
    if not value:
        return 0.0
    try:
        return datetime.fromisoformat(str(value)).timestamp()
    except ValueError:
        return 0.0

```

Full Source: ``backend/tradeo/modules/fox_hunter/scanner.py``

``backend/tradeo/modules/fox_hunter/scanner.py`` ? 47 lines, 1414 bytes, sha256 prefix ``8027bdc9b8a288a5``

Public surface summary before full source:

? class ``FoxHunterScanner`` line 14 methods: scan, status

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Any

from sqlalchemy.orm import Session

from tradeo.core.config import Settings
from tradeo.modules.shared.entry_scanner import PatternEntryScanner
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher

@dataclass(slots=True)
class FoxHunterScanner:
    """FoxHunter facade for production-pattern entry scans."""

    settings: Settings | None = None
    matcher: NovelPatternMatcher | None = None

```



```

def _scanner(self) -> PatternEntryScanner:
    return PatternEntryScanner(settings=self.settings, matcher=self.matcher)

def scan(
    self,
    db: Session,
    *,
    symbols: list[str] | None = None,
    limit: int | None = None,
    max_patterns: int | None = None,
    similarity_threshold: float | None = None,
    store_signals: bool | None = None,
    execute_orders: bool | None = None,
) -> dict[str, Any]:
    return self._scanner().scan(
        db,
        module="fox_hunter",
        symbols=symbols,
        limit=limit,
        max_patterns=max_patterns,
        similarity_threshold=similarity_threshold,
        store_signals=store_signals,
        execute_orders=execute_orders,
    )

def status(self, db: Session) -> dict[str, Any]:
    return self._scanner().status(db)["fox_hunter"]

```

Full Source: `backend/tradeo/modules/fox\_hunter/production\_manifest.py`

`backend/tradeo/modules/fox\_hunter/production\_manifest.py` ? 254 lines, 9802 bytes, sha256 prefix `feabd8fb2aa45b49`

Public surface summary before full source:

```

? function `build_production_manifest` line 16`
? function `production_manifest_hash` line 47`
? function `production_manifest_status` line 53`
? function `production_manifest_for_pattern` line 113`
? function `production_manifest_is_active` line 117`
? function `production_manifest_summary` line 125`

```

Risk hints: wall-clock timestamp usage.

```

from __future__ import annotations

from datetime import datetime, timedelta, timezone
import hashlib
import json
from typing import Any

from tradeo.db.models import DiscoveredPattern, DiscoveredPatternStatus

PRODUCTION_MANIFEST_KEY = "production_manifest"
PRODUCTION_MANIFEST_SCHEMA = "tradeo.production_manifest.v1"
PRODUCTION_MANIFEST_HASH_ALGORITHM = "sha256_canonical_v1"
_HASH_KEYS = {"hash", "manifest_hash", "sha256"}

def build_production_manifest(
    pattern: DiscoveredPattern | None = None,
    *,
    version: str | None = None,
    reviewer: str,
    evidence_packet: dict[str, Any],
    expires_at: datetime | str | None = None,
    approved_at: datetime | str | None = None,
    approved_by: str = "director",
) -> dict[str, Any]:
    """Build the canonical manifest Fox Hunter requires for production patterns."""

    approved_time = _parse_datetime(approved_at or datetime.now(timezone.utc)) or datetime.now(timezone.utc)
    expires = _parse_datetime(expires_at) if expires_at is not None else approved_time + timedelta(days=90)
    normalized_packet = _normalized_evidence_packet(pattern, evidence_packet)
    manifest = {
        "schema": PRODUCTION_MANIFEST_SCHEMA,
        "version": str(version or _default_manifest_version(pattern, approved_time)),
        "status": "approved",
        "approved": True,
        "approved_by": approved_by,
        "reviewer": reviewer,
        "approved_at": approved_time.isoformat(),
        "expires_at": (expires or approved_time).isoformat(),
        "evidence_packet": normalized_packet,
        "hash_algorithm": PRODUCTION_MANIFEST_HASH_ALGORITHM,
    }
    manifest["manifest_hash"] = production_manifest_hash(manifest)
    return manifest

def production_manifest_hash(manifest: dict[str, Any]) -> str:
    payload = _canonical_manifest_payload(manifest)

```

```

raw = json.dumps(payload, sort_keys=True, separators="," , ":"), default=str)
return hashlib.sha256(raw.encode("utf-8")).hexdigest()

def production_manifest_status(
    pattern: DiscoveredPattern | None,
    *,
    now: datetime | None = None,
) -> dict[str, Any]:
    checked_at = now or datetime.now(timezone.utc)
    if pattern is None:
        return _status(False, "missing_pattern", checked_at)
    pattern_status = _status_value(pattern.status)
    if pattern_status != DiscoveredPatternStatus.PRODUCTION.value:
        return _status(False, "pattern_not_production", checked_at, pattern_status=pattern_status)
    manifest = _manifest_from_pattern(pattern)
    if not isinstance(manifest, dict) or not manifest:
        return _status(False, "missing_production_manifest", checked_at, pattern_status=pattern_status)

    errors: list[str] = []
    manifest_status = str(manifest.get("status") or "").lower()
    approved = bool(manifest.get("approved")) or manifest_status == "approved"
    if not approved:
        errors.append("manifest_not_approved")
    if not str(manifest.get("version") or "").strip():
        errors.append("missing_manifest_version")
    if str(manifest.get("approved_by") or "").lower() != "director":
        errors.append("missing_director_approval")
    reviewer = str(manifest.get("reviewer") or manifest.get("reviewed_by") or "").strip()
    if not reviewer:
        errors.append("missing_reviewer")
    expires_at = _parse_datetime(manifest.get("expires_at"))
    if expires_at is None:
        errors.append("missing_expiry")
    elif expires_at <= checked_at:
        errors.append("manifest_expired")
    if not _manifest_hash(manifest):
        errors.append("missing_manifest_hash")
    if not _evidence_packet_ok(manifest):
        errors.append("missing_evidence_packet")
    if (
        str(manifest.get("hash_algorithm") or "") == PRODUCTION_MANIFEST_HASH_ALGORITHM
        and _manifest_hash(manifest)
        and production_manifest_hash(manifest) != _manifest_hash(manifest)
    ):
        errors.append("manifest_hash_mismatch")

    valid = not errors
    return _status(
        valid,
        "valid" if valid else errors[0],
        checked_at,
        pattern_status=pattern_status,
        version=str(manifest.get("version") or ""),
        reviewer=reviewer,
        approved_by=str(manifest.get("approved_by") or ""),
        expires_at=expires_at.isoformat() if expires_at is not None else None,
        manifest_hash=_manifest_hash(manifest),
        evidence_packet=_evidence_packet_summary(manifest),
        hash_verified=valid and str(manifest.get("hash_algorithm") or "") == PRODUCTION_MANIFEST_HASH_ALGORITHM,
        errors=errors,
    )

def production_manifest_for_pattern(pattern: DiscoveredPattern | None) -> dict[str, Any] | None:
    return _manifest_from_pattern(pattern) if pattern is not None else None

def production_manifest_is_active(
    pattern: DiscoveredPattern | None,
    *,
    now: datetime | None = None,
) -> bool:
    return bool(production_manifest_status(pattern, now=now)["valid"])

def production_manifest_summary(patterns: list[DiscoveredPattern]) -> dict[str, Any]:
    statuses = [production_manifest_status(pattern) for pattern in patterns]
    valid = [item for item in statuses if item["valid"]]
    blocked = [item for item in statuses if not item["valid"]]
    reason_counts: dict[str, int] = {}
    for item in blocked:
        reason = str(item.get("reason_code") or "unknown")
        reason_counts[reason] = reason_counts.get(reason, 0) + 1
    return {
        "required": True,
        "eligible_patterns": len(valid),
        "blocked_patterns": len(blocked),
        "blocked_reason_counts": reason_counts,
    }

def _manifest_from_pattern(pattern: DiscoveredPattern) -> dict[str, Any] | None:
    metrics = pattern.metrics_json or {}
    manifest = metrics.get(PRODUCTION_MANIFEST_KEY)
    if isinstance(manifest, dict):

```

```

        return manifest
    params = getattr(pattern, "params_json", None) or {}
    manifest = params.get(PRODUCTION_MANIFEST_KEY) if isinstance(params, dict) else None
    return manifest if isinstance(manifest, dict) else None

def _normalized_evidence_packet(
    pattern: DiscoveredPattern | None,
    evidence_packet: dict[str, Any],
) -> dict[str, Any]:
    packet = dict(evidence_packet or {})
    if not str(packet.get("id") or packet.get("packet_id") or "").strip():
        packet["id"] = _default_evidence_packet_id(pattern)
    if not str(packet.get("hash") or packet.get("sha256") or packet.get("packet_hash") or "").strip():
        raw = json.dumps(packet, sort_keys=True, separators=(",", ":"), default=str)
        packet["hash"] = hashlib.sha256(raw.encode("utf-8")).hexdigest()
    return packet

def _default_manifest_version(pattern: DiscoveredPattern | None, approved_at: datetime) -> str:
    if pattern is None:
        return approved_at.strftime("%Y%m%dT%H%M%SZ")
    pattern_key = str(getattr(pattern, "pattern_key", "") or getattr(pattern, "id", "") or "pattern")
    return f"{pattern_key}:{approved_at.strftime('%Y%m%dT%H%M%SZ')}"

def _default_evidence_packet_id(pattern: DiscoveredPattern | None) -> str:
    if pattern is None:
        return "production-evidence-packet"
    pattern_id = str(getattr(pattern, "id", "") or getattr(pattern, "pattern_key", "") or "pattern")
    return f"production-evidence-pattern-{pattern_id}"

def _canonical_manifest_payload(manifest: dict[str, Any]) -> dict[str, Any]:
    return {key: value for key, value in manifest.items() if key not in _HASH_KEYS}

def _manifest_hash(manifest: dict[str, Any]) -> str:
    for key in ("manifest_hash", "hash", "sha256"):
        value = str(manifest.get(key) or "").strip()
        if value:
            return value
    return ""

def _evidence_packet_ok(manifest: dict[str, Any]) -> bool:
    packet = manifest.get("evidence_packet")
    packet_hash = str(
        manifest.get("evidence_packet_hash") or manifest.get("evidence_hash") or ""
    ).strip()
    if isinstance(packet, dict):
        packet_ref = any(str(packet.get(key) or "").strip() for key in ("id", "packet_id", "uri", "path"))
        packet_hash = packet_hash or next(
            (
                str(packet.get(key) or "").strip()
                for key in ("hash", "sha256", "packet_hash", "evidence_hash")
                if str(packet.get(key) or "").strip()
            ),
            "",
        )
        return packet_ref and bool(packet_hash)
    if isinstance(packet, str):
        return bool(packet.strip() and packet_hash)
    return False

def _evidence_packet_summary(manifest: dict[str, Any]) -> dict[str, Any]:
    packet = manifest.get("evidence_packet")
    if not isinstance(packet, dict):
        return {"present": bool(packet)}
    return {
        "id": packet.get("id") or packet.get("packet_id"),
        "uri": packet.get("uri") or packet.get("path"),
        "hash": packet.get("hash") or packet.get("sha256") or packet.get("packet_hash"),
    }

def _parse_datetime(value: Any) -> datetime | None:
    if isinstance(value, datetime):
        dt = value
    elif isinstance(value, str) and value.strip():
        try:
            dt = datetime.fromisoformat(value.replace("Z", "+00:00"))
        except ValueError:
            return None
    else:
        return None
    if dt.tzinfo is None:
        return dt.replace(tzinfo=timezone.utc)
    return dt.astimezone(timezone.utc)

def _isoformat(value: datetime | str) -> str:
    parsed = _parse_datetime(value)
    if parsed is not None:
        return parsed.isoformat()

```

```

    return str(value)

def _status_value(value: Any) -> str:
    return str(value.value if hasattr(value, "value") else value)

def _status(valid: bool, reason_code: str, checked_at: datetime, **extra: Any) -> dict[str, Any]:
    return {
        "valid": valid,
        "reason_code": reason_code,
        "checked_at": checked_at.isoformat(),
        **extra,
    }

```

Full Source: `backend/tradeo/services/self\_improvement.py`

`backend/tradeo/services/self\_improvement.py` ? 330 lines, 14699 bytes, sha256 prefix `1020ba1de5dbd9bc`

Audit relevance: Candidate mutation and anti-overfit gate. Audit CSCV/PBO, plateau guard, nested optimization integration, and lab candidate creation conditions.

Public surface summary before full source:

? class `SelfImprovementEngine` line 24 methods: run_lab_cycle

Risk hints: wall-clock timestamp usage.

```

from __future__ import annotations

import json
from copy import deepcopy
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import numpy as np
from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, StrategyVersion
from tradeo.research.nested_optimization import nested_optimization_report
from tradeo.research.quant_validation import pbo_cscv
from tradeo.schemas import SelfImprovementResponse
from tradeo.services.backtester import Backtester
from tradeo.services.data_provider import CachedMarketDataProvider, pick_symbols
from tradeo.services.pattern_detector import CupPatternDetector
from tradeo.services.strategy_config import load_strategy_config
from tradeo.services.provider_factory import get_market_data_provider

class SelfImprovementEngine:
    """Strategy mutation and promotion gate.

    It can propose lab candidates, but it never promotes to live automatically. The
    promotion path is: lab backtest -> walk-forward -> paper trading -> human/API review.
    """

    def __init__(self, settings: Settings | None = None) -> None:
        self.settings = settings or get_settings()

    def run_lab_cycle(self, db: Session, max_symbols: int = 25) -> SelfImprovementResponse:
        base = load_strategy_config(self.settings.strategy_config_file)
        symbols = pick_symbols(limit=max_symbols)
        candidates = self._mutations(base)
        provider = CachedMarketDataProvider(upstream=get_market_data_provider())
        records: list[dict[str, Any]] = []
        trial_budget = max(1, int(self.settings.self_improvement_max_trials))
        candidates = candidates[:trial_budget]
        trial_count = len(candidates)
        for trial_index, cfg in enumerate(candidates, start=1):
            detector = CupPatternDetector.from_config(cfg)
            metrics = Backtester(
                provider=provider,
                detector=detector,
                starting_equity=self.settings.initial_capital_usd,
            ).run(symbols, period="3y", interval="1d")
            records.append(
                {
                    "config": cfg,
                    "metrics": metrics.model_dump(),
                    "trial_accounting": {
                        "trial_index": trial_index,
                        "n_trials_this_cycle": trial_count,
                        "trial_budget": trial_budget,
                        "counts_toward_family_n_trials": True,
                    },
                },
            )
        guard_by_index, guard_summary = self._anti_overfit_guards(records)
        accepted: list[dict[str, Any]] = []
        for idx, record in enumerate(records):
            guard = guard_by_index.get(idx, {})

```

```

        record["anti_overfit"] = guard
        metrics = dict(record["metrics"])
        metrics["anti_overfit"] = guard
        metrics["trial_accounting"] = record["trial_accounting"]
        record["metrics"] = metrics
        if self._passes_lab_gate(metrics, guard):
            accepted.append(record)
            name = f"cup_lab_{datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')}_{'len(accepted)}"
            db.add(
                StrategyVersion(
                    name=name,
                    state="lab_candidate",
                    parent_name="cup_v0",
                    params_json=record["config"],
                    metrics_json=metrics,
                )
            )
        best = None
        if accepted:
            best = sorted(
                accepted,
                key=lambda x: (x["metrics"]["expectancy_r"], x["metrics"]["profit_factor"]),
                reverse=True,
            )[0]
        report_path = self._write_report(records, accepted, best, guard_summary)
        db.add(
            AuditLog(
                actor="self_improvement",
                action="lab_cycle_completed",
                entity_type="strategy",
                details_json={
                    "generated": len(records),
                    "accepted": len(accepted),
                    "report_path": str(report_path),
                    "best": best,
                    "anti_overfit": guard_summary,
                },
            )
        )
        db.commit()
        return SelfImprovementResponse(
            generated=len(records),
            accepted_lab_candidates=len(accepted),
            best_candidate=best,
            report_path=str(report_path),
        )

def _mutations(self, base: dict[str, Any]) -> list[dict[str, Any]]:
    grids = {
        "min_depth": [0.10, 0.12, 0.16],
        "max_depth": [0.34, 0.42],
        "rim_tolerance": [0.08, 0.12],
        "max_handle_depth": [0.12, 0.16, 0.18],
        "min_breakout_volume_ratio": [1.10, 1.20, 1.35],
        "min_composite_score": [0.68, 0.72, 0.76],
    }
    out: list[dict[str, Any]] = []
    for min_depth in grids["min_depth"]:
        for max_depth in grids["max_depth"]:
            for rim_tolerance in grids["rim_tolerance"]:
                for handle in grids["max_handle_depth"]:
                    for vol in grids["min_breakout_volume_ratio"]:
                        for score in grids["min_composite_score"]:
                            cfg = deepcopy(base)
                            cfg.update(
                                {
                                    "min_depth": min_depth,
                                    "max_depth": max_depth,
                                    "rim_tolerance": rim_tolerance,
                                    "max_handle_depth": handle,
                                    "min_breakout_volume_ratio": vol,
                                    "min_composite_score": score,
                                    "target_r_multiple": max(4.0, float(base.get("target_r_multiple", 4.0))),
                                }
                            )
                            out.append(cfg)
    return out[: max(1, int(self.settings.self_improvement_max_trials))]

def _passes_lab_gate(self, metrics: dict[str, Any], anti_overfit: dict[str, Any] | None = None) -> bool:
    guard = anti_overfit or {}
    return (
        metrics.get("total_trades", 0) >= 40
        and metrics.get("profit_factor", 0.0) >= 1.8
        and metrics.get("expectancy_r", 0.0) >= 0.25
        and metrics.get("max_drawdown_pct", 100.0) <= 12.0
        and bool(guard.get("pbo_passed", False))
        and bool(guard.get("plateau_passed", False))
        and bool(guard.get("nested_passed", False))
    )

def _anti_overfit_guards(self, records: list[dict[str, Any]]) -> tuple[dict[int, dict[str, Any]], dict[str, Any]]:
    pbo_report = self._pbo_report(records)
    nested_report = self._nested_report(records)
    out: dict[int, dict[str, Any]] = {}
    for idx, record in enumerate(records):
        plateau = self._plateau_report(idx, records)

```

```

        out[idx] = {
            "pbo": pbo_report,
            "pbo_passed": bool(pbo_report.get("passed", False)),
            "plateau": plateau,
            "plateau_passed": bool(plateau.get("passed", False)),
            "nested": nested_report,
            "nested_passed": bool(nested_report.get("passed", False)),
            "accepted_only_if_outer_evidence": True,
        }
    summary = dict(pbo_report)
    summary["nested"] = nested_report
    return out, summary

def _nested_report(self, records: list[dict[str, Any]]) -> dict[str, Any]:
    if not self.settings.self_improvement_nested_enabled:
        return {
            "method": "nested_outer_fold_pbo_v1",
            "passed": False,
            "blocked": True,
            "reason": "nested_optimization_disabled_fail_closed",
        }
    matrix, periods = self._performance_matrix(records)
    report = nested_optimization_report(
        matrix,
        n_outer_folds=max(4, int(self.settings.self_improvement_nested_outer_folds)),
        inner_trials=max(1, int(self.settings.self_improvement_nested_inner_trials)),
        max_pbo=float(self.settings.self_improvement_nested_max_pbo),
        seed=int(self.settings.self_improvement_nested_seed),
        use_optuna=bool(self.settings.self_improvement_nested_use_optuna),
    )
    report["period_kind"] = "monthly_realized_r_buckets"
    report["period_count_available"] = len(periods)
    return report

def _pbo_report(self, records: list[dict[str, Any]]) -> dict[str, Any]:
    matrix, periods = self._performance_matrix(records)
    min_blocks = max(4, int(self.settings.self_improvement_min_pbo_blocks))
    if matrix.shape[1] < 2 or matrix.shape[0] < min_blocks:
        return {
            "method": "cscv_pbo",
            "passed": False,
            "blocked": True,
            "reason": "insufficient_variants_or_periods_for_pbo",
            "period_count": int(matrix.shape[0]),
            "variant_count": int(matrix.shape[1]),
            "required_periods": min_blocks,
        }
    n_blocks = min(min_blocks if min_blocks % 2 == 0 else min_blocks - 1, matrix.shape[0])
    n_blocks = max(4, n_blocks)
    try:
        result = pbo_cscv(matrix, n_blocks=n_blocks, rng=17)
    except ValueError as exc:
        return {
            "method": "cscv_pbo",
            "passed": False,
            "blocked": True,
            "reason": str(exc),
            "period_count": int(matrix.shape[0]),
            "variant_count": int(matrix.shape[1]),
        }
    pbo = float(result["pbo"])
    return {
        "method": "cscv_pbo",
        "passed": pbo < float(self.settings.self_improvement_max_pbo),
        "blocked": False,
        "pbo": round(pbo, 5),
        "max_pbo": float(self.settings.self_improvement_max_pbo),
        "period_count": len(periods),
        "variant_count": int(matrix.shape[1]),
        "n_combinations": int(result["n_combinations"]),
        "best_counts": np.asarray(result["best_counts"], dtype=int).tolist(),
    }

@staticmethod
def _performance_matrix(records: list[dict[str, Any]]) -> tuple[np.ndarray, list[str]]:
    period_values: dict[str, list[float]] = {}
    for idx, record in enumerate(records):
        for trade in record.get("metrics", {}).get("trades", []):
            period = str(trade.get("exit_date", ""))[:7]
            if not period:
                continue
            period_values.setdefault(period, [0.0 for _ in records])
            period_values[period][idx] += float(trade.get("r_multiple", 0.0) or 0.0)
    periods = sorted(period_values)
    if not periods:
        return np.zeros((0, len(records)), dtype=float), []
    matrix = np.asarray([period_values[p] for p in periods], dtype=float)
    return matrix, periods

def _plateau_report(self, idx: int, records: list[dict[str, Any]]) -> dict[str, Any]:
    current = records[idx]
    cfg = current["config"]
    pf = float(current.get("metrics", {}).get("profit_factor", 0.0) or 0.0)
    if pf <= 0:
        return {"passed": False, "reason": "non_positive_profit_factor", "neighbor_count": 0}
    neighbors: list[float] = []

```

```

for j, other in enumerate(records):
    if j == idx:
        continue
    if self._within_parameter_plateau(cfg, other["config"]):
        neighbors.append(float(other.get("metrics", {}).get("profit_factor", 0.0) or 0.0))
if len(neighbors) < 2:
    return {"passed": False, "reason": "insufficient_neighbor_trials", "neighbor_count": len(neighbors)}
neighbor_median = float(np.median(neighbors))
threshold = pf * float(self.settings.self_improvement_plateau_pf_fraction)
return {
    "passed": neighbor_median >= threshold,
    "neighbor_count": len(neighbors),
    "neighbor_median_profit_factor": round(neighbor_median, 5),
    "required_profit_factor": round(threshold, 5),
    "plateau_fraction": float(self.settings.self_improvement_plateau_pf_fraction),
}

@staticmethod
def _within_parameter_plateau(left: dict[str, Any], right: dict[str, Any]) -> bool:
    keys = (
        "min_depth",
        "max_depth",
        "rim_tolerance",
        "max_handle_depth",
        "min_breakout_volume_ratio",
        "min_composite_score",
    )
    comparable = 0
    close = 0
    for key in keys:
        if key not in left or key not in right:
            continue
        a = float(left[key])
        b = float(right[key])
        comparable += 1
        if abs(a - b) <= max(abs(a), 1e-9) * 0.20:
            close += 1
    return comparable > 0 and close == comparable

def _write_report(
    self,
    generated: list[dict[str, Any]],
    accepted: list[dict[str, Any]],
    best: dict[str, Any] | None,
    guard_summary: dict[str, Any],
) -> Path:
    now = datetime.now(timezone.utc)
    path = self.settings.reports_path / f"self_improvement_{now:%Y%m%d_%H%M%S}.json"
    path.write_text(
        json.dumps(
            {
                "generated_at_utc": now.isoformat(),
                "generated_count": len(generated),
                "accepted_count": len(accepted),
                "acceptance_gate": {
                    "min_trades": 40,
                    "min_profit_factor": 1.8,
                    "min_expectancy_r": 0.25,
                    "max_drawdown_pct": 12.0,
                    "max_pbo": self.settings.self_improvement_max_pbo,
                    "plateau_pf_fraction": self.settings.self_improvement_plateau_pf_fraction,
                    "nested_max_pbo": self.settings.self_improvement_nested_max_pbo,
                    "nested_outer_folds": self.settings.self_improvement_nested_outer_folds,
                    "promotion": "requires human/API supervisor approval after paper trading",
                },
                "anti_overfit": guard_summary,
                "best": best,
                "accepted": accepted,
            },
            indent=2,
        )
    )
    return path

```

Full Source: `backend/tradeo/services/market\_regime.py`

`backend/tradeo/services/market\_regime.py` ? 270 lines, 10159 bytes, sha256 prefix `1754ae67c892c3f4`

Audit relevance: PIT benchmark regime labeling. Audit strict SMA/vol tercile construction, insufficient-history honesty, source hashes, and no future data.

Public surface summary before full source:

? class `MarketRegimeService` line 182 methods: `current_regime`, `history_table`

? function `regime\_table` line 24`

? function `regime\_at` line 70`

? function `unlabeled\_regime\_key` line 117`

? function `regime\_keys\_for\_dates` line 121`

? function `period\_to\_months` line 149`

? function `required\_benchmark\_bars` line 170`

Risk hints: broad `except Exception` present.

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import date, datetime
from hashlib import blake2b
import math
from typing import Any, Iterable

import numpy as np
import pandas as pd

from tradeo.core.config import Settings, get_settings
from tradeo.services.data_provider import MarketDataProvider
from tradeo.services.technical_indicators import normalize_ohlc

TREND_BULL = "benchmark_bull"
TREND_BEAR = "benchmark_bear"
VOL_TERCILE_LABELS = ("low_vol_tercile", "mid_vol_tercile", "high_vol_tercile")
INSUFFICIENT_HISTORY = "insufficient_history"

TRADING_DAYS_PER_YEAR = 252

def regime_table(
    df: pd.DataFrame,
    *,
    sma_window: int = 200,
    vol_window: int = 20,
    tercile_lookback: int = 252,
) -> pd.DataFrame:
    """Per-bar benchmark regime labels from closed daily bars only.

    Trend: close vs strict SMA(sma_window) (full window required, no partial
    min_periods so a young series never pretends to know its 200-day trend).
    Volatility: annualized stdev of log returns over vol_window, bucketed into
    terciles whose boundaries come exclusively from the trailing
    tercile_lookback values up to the PREVIOUS bar. Bar t never contributes to
    its own boundaries, so labels are point-in-time and never change when
    future bars are appended.
    """
    out = normalize_ohlc(df)
    close = out["close"].astype(float)
    table = pd.DataFrame(index=out.index)
    table["close"] = close
    table["sma"] = close.rolling(window=sma_window, min_periods=sma_window).mean()
    table["trend_regime"] = np.where(
        table["sma"].isna(),
        INSUFFICIENT_HISTORY,
        np.where(close >= table["sma"], TREND_BULL, TREND_BEAR),
    )
    log_returns = np.log(close / close.shift(1))
    realized = log_returns.rolling(window=vol_window, min_periods=vol_window).std(ddof=1)
    table["realized_vol"] = realized * math.sqrt(TRADING_DAYS_PER_YEAR)
    history = table["realized_vol"].shift(1)
    lower = history.rolling(window=tercile_lookback, min_periods=tercile_lookback).quantile(1 / 3)
    upper = history.rolling(window=tercile_lookback, min_periods=tercile_lookback).quantile(2 / 3)
    table["vol_tercile_lower"] = lower
    table["vol_tercile_upper"] = upper
    conditions = [
        table["realized_vol"].isna() | lower.isna() | upper.isna(),
        table["realized_vol"] <= lower,
        table["realized_vol"] <= upper,
    ]
    choices = [INSUFFICIENT_HISTORY, VOL_TERCILE_LABELS[0], VOL_TERCILE_LABELS[1]]
    table["vol_regime"] = np.select(conditions, choices, default=VOL_TERCILE_LABELS[2])
    table["regime_key"] = table["trend_regime"] + "|" + table["vol_regime"]
    return table

def regime_at(
    df: pd.DataFrame,
    as_of: date | datetime | str | None = None,
    *,
    benchmark_symbol: str = "SPY",
    sma_window: int = 200,
    vol_window: int = 20,
    tercile_lookback: int = 252,
) -> dict[str, Any]:
    """Benchmark regime snapshot at the last bar on or before as_of."""
    if df is None or df.empty:
        return _empty_regime(benchmark_symbol)
    table = regime_table(
        df,
        sma_window=sma_window,
        vol_window=vol_window,
        tercile_lookback=tercile_lookback,
    )
    if as_of is not None:
        cutoff = pd.Timestamp(as_of)
        index = pd.DatetimeIndex(table.index)
        if index.tz is not None:
            cutoff = cutoff.tz_localize(index.tz) if cutoff.tzinfo is None else cutoff.tz_convert(index.tz)
        table = table[index <= cutoff]
```



```

if table.empty:
    return _empty_regime(benchmark_symbol)
row = table.iloc[-1]
return {
    "benchmark_symbol": benchmark_symbol,
    "as_of_bar": pd.Timestamp(table.index[-1]).isoformat(),
    "trend_regime": str(row["trend_regime"]),
    "vol_regime": str(row["vol_regime"]),
    "regime_key": str(row["regime_key"]),
    "close": _round(row["close"]),
    "sma": _round(row["sma"]),
    "sma_window": int(sma_window),
    "realized_vol_annualized": _round(row["realized_vol"]),
    "vol_window": int(vol_window),
    "vol_tercile_lookback": int(tercile_lookback),
    "vol_tercile_lower": _round(row["vol_tercile_lower"]),
    "vol_tercile_upper": _round(row["vol_tercile_upper"]),
    "bars_available": int(len(table)),
    "source_bar_hash": _tail_hash(df),
    "point_in_time": True,
}

def unlabeled_regime_key() -> str:
    return f"{INSUFFICIENT_HISTORY}{INSUFFICIENT_HISTORY}"

def regime_keys_for_dates(table: pd.DataFrame | None, dates: Iterable[Any]) -> list[str]:
    """PIT regime_key at the last benchmark bar on or before each date.

    `table` is a `regime_table` output. Dates earlier than the first benchmark
    bar return the explicit unlabeled key instead of guessing, so callers can
    separate "no benchmark history yet" from real regime buckets.
    """
    materialized = list(dates)
    if table is None or table.empty or "regime_key" not in table.columns:
        return [unlabeled_regime_key() for _ in materialized]
    index = pd.DatetimeIndex(table.index)
    if index.tz is not None:
        index = index.tz_convert("UTC").tz_localize(None)
    keys = table["regime_key"].astype(str).to_numpy()
    out: list[str] = []
    for value in materialized:
        try:
            ts = pd.Timestamp(value)
        except (TypeError, ValueError):
            out.append(unlabeled_regime_key())
            continue
        if ts.tzinfo is not None:
            ts = ts.tz_convert("UTC").tz_localize(None)
        pos = int(index.searchsorted(ts, side="right")) - 1
        out.append(str(keys[pos]) if pos >= 0 else unlabeled_regime_key())
    return out

def period_to_months(period: str | None) -> int:
    """Approximate calendar months covered by a provider period string."""
    if not period:
        return 0
    text = str(period).strip().lower()
    if text == "max":
        return 120
    try:
        if text.endswith("mo"):
            return int(math.ceil(float(text[:-2])))
        if text.endswith("y"):
            return int(math.ceil(float(text[:-1]) * 12))
        if text.endswith("d"):
            return int(math.ceil(float(text[:-1]) / 30))
        if text.endswith("wk"):
            return int(math.ceil(float(text[:-2]) * 7 / 30))
    except ValueError:
        return 0
    return 0

def required_benchmark_bars(settings: Settings | None = None) -> int:
    """Bars of benchmark history needed for fully-labeled trend and terciles."""
    settings = settings or get_settings()
    return max(
        int(settings.market_regime_sma_window) + 10,
        int(settings.market_regime_vol_tercile_lookback)
        + int(settings.market_regime_vol_window)
        + 10,
    )

@dataclass(slots=True)
class MarketRegimeService:
    """Fetch the benchmark via the (cached) provider and label its regime."""

    provider: MarketDataProvider
    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()

```

```

def current_regime(self) -> dict[str, Any]:
    settings = self.settings
    assert settings is not None
    bars = required_benchmark_bars(settings)
    months = max(3, math.ceil(bars * 7 / 5 / 30) + 1)
    try:
        df = self.provider.fetch_ohlcv(
            settings.market_regime_benchmark_symbol,
            period=f"{months}mo",
            interval="1d",
        )
    except Exception: # noqa: BLE001
        return _empty_regime(settings.market_regime_benchmark_symbol)
    return regime_at(
        df,
        benchmark_symbol=settings.market_regime_benchmark_symbol,
        sma_window=settings.market_regime_sma_window,
        vol_window=settings.market_regime_vol_window,
        tercile_lookback=settings.market_regime_vol_tercile_lookback,
    )

def history_table(self, period: str | None = None) -> pd.DataFrame:
    """PIT regime table covering `period` plus the labeling warmup window.

    Raises on provider failure so callers decide whether regime-labeled
    outcomes are skippable (research warning) or fatal.
    """
    settings = self.settings
    assert settings is not None
    warmup_bars = required_benchmark_bars(settings)
    warmup_months = max(3, math.ceil(warmup_bars * 7 / 5 / 30) + 1)
    months = warmup_months + period_to_months(period)
    df = self.provider.fetch_ohlcv(
        settings.market_regime_benchmark_symbol,
        period=f"{months}mo",
        interval="1d",
    )
    table = regime_table(
        df,
        sma_window=settings.market_regime_sma_window,
        vol_window=settings.market_regime_vol_window,
        tercile_lookback=settings.market_regime_vol_tercile_lookback,
    )
    table.attrs["benchmark_symbol"] = settings.market_regime_benchmark_symbol
    table.attrs["sma_window"] = int(settings.market_regime_sma_window)
    table.attrs["vol_window"] = int(settings.market_regime_vol_window)
    table.attrs["vol_tercile_lookback"] = int(settings.market_regime_vol_tercile_lookback)
    return table

def _empty_regime(benchmark_symbol: str) -> dict[str, Any]:
    return {
        "benchmark_symbol": benchmark_symbol,
        "as_of_bar": None,
        "trend_regime": INSUFFICIENT_HISTORY,
        "vol_regime": INSUFFICIENT_HISTORY,
        "regime_key": f"{INSUFFICIENT_HISTORY}|{INSUFFICIENT_HISTORY}",
        "close": None,
        "sma": None,
        "realized_vol_annualized": None,
        "bars_available": 0,
        "source_bar_hash": "",
        "point_in_time": True,
    }

def _round(value: Any) -> float | None:
    try:
        number = float(value)
    except (TypeError, ValueError):
        return None
    if math.isnan(number):
        return None
    return round(number, 8)

def _tail_hash(df: pd.DataFrame) -> str:
    cols = [col for col in ("open", "high", "low", "close", "volume") if col in df.columns]
    payload = df.tail(3)[cols].round(8).to_csv(index=True)
    return blake2b(payload.encode("utf-8"), digest_size=12).hexdigest()

```

Full Source: ``backend/tradeo/services/pattern_health_monitor.py``

``backend/tradeo/services/pattern_health_monitor.py`` ? 216 lines, 9015 bytes, sha256 prefix ``ec8b95d40dc11c72``

Public surface summary before full source:

? class ``PatternHealthMonitor`` line 46 methods: run

Risk hints: wall-clock timestamp usage.

"""Post-promotion pattern decay detection with CUSUM (informe \$4.8).

For every pattern in PRODUCTION or DIRECTOR_REVIEW we run a two-sided CUSUM over the chronological series of realized R per trade, standardized against the Research expectancy and dispersion. A downward trigger means the live performance is drifting below what Research promised faster than chance allows: the pattern is marked ``drift\_status='decaying'`` and (with ``health\_monitor\_block\_decaying``) the matcher stops generating new signals for it until a fresh re-validation clears it.

Decay is one-way here: re-activation is an explicit human/Director decision after re-running the discovery validation on fresh data, never automatic.

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any

import numpy as np
from loguru import logger
from sqlalchemy.orm import Session, joinedload

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import (
    AuditLog,
    DiscoveredPattern,
    DiscoveredPatternStatus,
    Trade,
    TradeStatus,
)
from tradeo.research.quant_validation import cusum_drift
from tradeo.services.director_review_gate import DirectorReviewGate
from tradeo.services.implementation_shortfall import trade_slippage_r

MONITORED_STATUSES = (
    DiscoveredPatternStatus.PRODUCTION,
    DiscoveredPatternStatus.DIRECTOR_REVIEW,
)

__all__ = ["PatternHealthMonitor", "MONITORED_STATUSES"]

@dataclass(slots=True)
class PatternHealthMonitor:
    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()

    def run(self, db: Session) -> dict[str, Any]:
        settings = self.settings
        assert settings is not None
        patterns = (
            db.query(DiscoveredPattern)
            .filter(DiscoveredPattern.status.in_(list(MONITORED_STATUSES)))
            .all()
        )
        closed_trades = (
            db.query(Trade)
            .options(joinedload(Trade.signal))
            .filter(Trade.status == TradeStatus.CLOSED)
            .order_by(Trade.closed_at.asc())
            .all()
        )
        checked = 0
        skipped: list[dict[str, Any]] = []
        decaying: list[dict[str, Any]] = []
        for pattern in patterns:
            trades = [
                trade
                for trade in closed_trades
                if DirectorReviewGate._lab_pattern_id_for_trade(trade) == pattern.id
                and DirectorReviewGate._counts_as_paper_fill(trade)
            ]
            r_values = [float(trade.r_multiple or 0.0) for trade in trades]
            if len(r_values) < settings.health_monitor_min_trades:
                skipped.append(
                    {
                        "pattern_id": pattern.id,
                        "reason": "insufficient_closed_fills",
                        "closed_fills": len(r_values),
                        "min_required": settings.health_monitor_min_trades,
                    }
                )
            continue
            checked += 1
            target = float(pattern.best_expectancy_r or pattern.expectancy_r or 0.0)
            scale = self._research_r_scale(pattern, fallback=r_values)
            verdict = cusum_drift(
                r_values,
                k=settings.health_monitor_cusum_k,
                h=settings.health_monitor_cusum_h,
                target=target,
                scale=scale,
            )

```

```

shortfall_values = [
    float(row["slippage_r"])
    for row in (trade_slippage_r(trade) for trade in trades)
    if row is not None
]
shortfall_verdict = None
if len(shortfall_values) >= settings.director_min_slippage_samples:
    shortfall_verdict = cusum_drift(
        shortfall_values,
        k=settings.health_monitor_shortfall_cusum_k,
        h=settings.health_monitor_shortfall_cusum_h,
        target=settings.director_max_median_slippage_r,
        scale=max(float(np.std(np.asarray(shortfall_values), ddof=1)), 0.05)
        if len(shortfall_values) > 1
        else 0.05,
    )
health = {
    "checked_at": datetime.now(timezone.utc).isoformat(),
    "closed_fills": len(r_values),
    "shortfall_fills": len(shortfall_values),
    "target_expectancy_r": round(target, 4),
    "scale_r": round(scale, 4),
    "cusum_k": settings.health_monitor_cusum_k,
    "cusum_h": settings.health_monitor_cusum_h,
    "triggered": bool(verdict["triggered"]),
    "side": verdict["side"],
    "trigger_index": verdict["index"],
    "min_s_neg": round(float(np.min(verdict["s_neg"])), 4),
    "max_s_pos": round(float(np.max(verdict["s_pos"])), 4),
    "shortfall_cusum": self._shortfall_health(shortfall_verdict),
}
pattern.metrics_json = {*(pattern.metrics_json or {}), "pattern_health": health}
shortfall_triggered = (
    shortfall_verdict is not None
    and bool(shortfall_verdict["triggered"])
    and shortfall_verdict["side"] == "up"
)
if (verdict["triggered"] and verdict["side"] == "down") or shortfall_triggered:
    previous = str(pattern.drift_status or "stable")
    pattern.drift_status = "decaying"
    r_drift = float(abs(np.min(verdict["s_neg"])))
    shortfall_drift = (
        float(np.max(shortfall_verdict["s_pos"])) if shortfall_verdict is not None else 0.0
    )
    pattern.drift_score = round(max(r_drift, shortfall_drift), 4)
    db.add(
        AuditLog(
            actor="pattern_health_monitor",
            action="pattern_health_decay_detected",
            entity_type="discovered_pattern",
            entity_id=str(pattern.id),
            details_json={
                "pattern_id": pattern.id,
                "previous_drift_status": previous,
                "health": health,
                "decay_source": "shortfall" if shortfall_triggered else "realized_r",
                "effect": (
                    "matcher stops generating new signals for this pattern "
                    "while drift_status='decaying'; re-validation required"
                ),
            },
        ),
    )
    decaying.append({"pattern_id": pattern.id, "name": pattern.name, **health})
    logger.warning(
        "pattern {} ({} ) marked decaying by CUSUM (min_s_neg={}),"
        pattern.id,
        pattern.name,
        health["min_s_neg"],
    )
    db.add(pattern)
db.commit()
return {
    "patterns_monitored": len(patterns),
    "patterns_checked": checked,
    "patterns_skipped": len(skipped),
    "skipped": skipped,
    "decay_detected": len(decaying),
    "decaying": decaying,
}

@staticmethod
def _shortfall_health(verdict: dict[str, Any] | None) -> dict[str, Any]:
    if verdict is None:
        return {"available": False, "reason": "insufficient_real_fill_shortfall_samples"}
    return {
        "available": True,
        "triggered": bool(verdict["triggered"]),
        "side": verdict["side"],
        "trigger_index": verdict["index"],
        "min_s_neg": round(float(np.min(verdict["s_neg"])), 4),
        "max_s_pos": round(float(np.max(verdict["s_pos"])), 4),
    }

@staticmethod
def _research_r_scale(pattern: DiscoveredPattern, *, fallback: list[float]) -> float:

```

```

"""Dispersion of Research R for standardization, with honest fallbacks."""
outcomes = [
    float(example.outcome_r)
    for example in (pattern.examples or [])
    if example.outcome_r is not None
]
if len(outcomes) >= 5:
    sd = float(np.std(np.asarray(outcomes), ddof=1))
    if sd > 0:
        return sd
quant = (pattern.metrics_json or {}).get("quant_validation")
if isinstance(quant, dict):
    sd = quant.get("expectancy_sd_weighted") or quant.get("sd_weighted")
    try:
        if sd is not None and float(sd) > 0:
            return float(sd)
    except (TypeError, ValueError):
        pass
if len(fallback) > 1:
    sd = float(np.std(np.asarray(fallback), ddof=1))
    if sd > 0:
        return sd
return 1.0

```

Full Source: `backend/tradeo/services/system\_controls.py`

`backend/tradeo/services/system\_controls.py` ? 101 lines, 3005 bytes, sha256 prefix `d219006570af2a36`

Public surface summary before full source:

```

? function `runtime_kill_switch` line 33`
? function `runtime_kill_switch_active` line 37`
? function `activate_runtime_kill_switch` line 42`
? function `deactivate_runtime_kill_switch` line 77`

```

Risk hints: wall-clock timestamp usage.

```

"""Runtime kill switch persisted in the database (informe §4.5).

TRADEO_KILL_SWITCH_ENABLED is read once at startup, so an automatic safety
trigger could never make it bite without a container restart. This module adds
a DB-persisted runtime kill switch that every order path checks in addition to
the env flag. Activation is idempotent and always audited.

The env flag remains the manual, restart-surviving master switch; the runtime
switch is the automatic one (reconciliation divergence, future triggers).
Either being active blocks order submission.
"""

from __future__ import annotations

from datetime import datetime, timezone
from typing import Any

from sqlalchemy.orm import Session

from tradeo.db.models import AuditLog, SystemControl

KILL_SWITCH_KEY = "kill_switch"

__all__ = [
    "KILL_SWITCH_KEY",
    "runtime_kill_switch",
    "runtime_kill_switch_active",
    "activate_runtime_kill_switch",
    "deactivate_runtime_kill_switch",
]

def runtime_kill_switch(db: Session) -> SystemControl | None:
    return db.query(SystemControl).filter(SystemControl.key == KILL_SWITCH_KEY).first()

def runtime_kill_switch_active(db: Session) -> bool:
    control = runtime_kill_switch(db)
    return bool(control is not None and control.enabled)

def activate_runtime_kill_switch(
    db: Session,
    *,
    reason: str,
    actor: str,
    details: dict[str, Any] | None = None,
) -> SystemControl:
    """Persist the runtime kill switch as active. Idempotent, always audited."""
    control = runtime_kill_switch(db)
    already_active = bool(control is not None and control.enabled)
    if control is None:
        control = SystemControl(key=KILL_SWITCH_KEY)
        db.add(control)
    control.enabled = True

```

```

control.reason = reason
control.actor = actor
control.details_json = details or {}
control.updated_at = datetime.now(timezone.utc)
db.add(
    AuditLog(
        actor=actor,
        action="runtime_kill_switch_activated",
        entity_type="system_control",
        entity_id=KILL_SWITCH_KEY,
        details_json={
            "reason": reason,
            "already_active": already_active,
            **(details or {}),
        },
    )
)
db.commit()
return control

def deactivate_runtime_kill_switch(
    db: Session,
    *,
    reason: str,
    actor: str,
) -> SystemControl | None:
    """Deactivate the runtime kill switch. Meant for explicit human action."""
    control = runtime_kill_switch(db)
    if control is None or not control.enabled:
        return control
    control.enabled = False
    control.reason = reason
    control.actor = actor
    control.updated_at = datetime.now(timezone.utc)
    db.add(
        AuditLog(
            actor=actor,
            action="runtime_kill_switch_deactivated",
            entity_type="system_control",
            entity_id=KILL_SWITCH_KEY,
            details_json={"reason": reason},
        )
    )
    db.commit()
    return control

```

Full Source: ``backend/tradeo/services/runtime_status.py``

``backend/tradeo/services/runtime_status.py`` ? 100 lines, 4292 bytes, sha256 prefix ``1bd3c4d72f4830f1``

Public surface summary before full source:

```

? function `write_worker_heartbeat` line 14`
? function `worker_runtime_status` line 27`
? function `write_entry_scan_status` line 47`
? function `entry_scan_status` line 70`

```

Risk hints: wall-clock timestamp usage.

```

from __future__ import annotations

import json
import os
from datetime import datetime, timezone
from typing import Any

from tradeo.core.config import Settings, get_settings

WORKER_HEARTBEAT = "worker_heartbeat.json"
ENTRY_SCAN_STATUS = "entry_scan_status.json"

def write_worker_heartbeat(settings: Settings | None = None) -> None:
    settings = settings or get_settings()
    path = settings.artifacts_path / WORKER_HEARTBEAT
    payload = {
        "pid": os.getpid(),
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "scheduler_enabled": settings.scheduler_enabled,
        "laboratory_scanner_enabled": settings.laboratory_scanner_enabled,
        "fox_hunter_enabled": settings.fox_hunter_enabled,
    }
    path.write_text(json.dumps(payload), encoding="utf-8")

def worker_runtime_status(settings: Settings | None = None, *, max_age_seconds: int = 90) -> dict[str, Any]:
    settings = settings or get_settings()
    path = settings.artifacts_path / WORKER_HEARTBEAT
    if not path.exists():
        return {"ok": False, "state": "stopped", "age_seconds": None, "reason": "missing_worker_heartbeat"}
    try:

```

```

        payload = json.loads(path.read_text(encoding="utf-8"))
        timestamp = datetime.fromisoformat(str(payload["timestamp"]))
    except (OSError, ValueError, KeyError, TypeError, json.JSONDecodeError):
        return {"ok": False, "state": "stopped", "age_seconds": None, "reason": "invalid_worker_heartbeat"}
    age_seconds = (datetime.now(timezone.utc) - timestamp).total_seconds()
    ok = age_seconds <= max_age_seconds and bool(payload.get("scheduler_enabled"))
    return {
        "ok": ok,
        "state": "ok" if ok else "stopped",
        "age_seconds": round(age_seconds, 1),
        "reason": "ok" if ok else "stale_or_scheduler_disabled",
    }

def write_entry_scan_status(module: str, result: dict[str, Any], settings: Settings | None = None) -> None:
    settings = settings or get_settings()
    path = settings.artifacts_path / ENTRY_SCAN_STATUS
    try:
        payload = json.loads(path.read_text(encoding="utf-8")) if path.exists() else {}
    except (OSError, json.JSONDecodeError):
        payload = {}
    previous = payload.get(module) or {}
    last_symbols_checked = int(result.get("symbols_checked") or 0)
    cumulative_symbols_checked = int(previous.get("cumulative_symbols_checked") or 0) + last_symbols_checked
    payload[module] = {
        "symbols_checked": cumulative_symbols_checked,
        "last_symbols_checked": last_symbols_checked,
        "cumulative_symbols_checked": cumulative_symbols_checked,
        "patterns_checked": int(result.get("patterns_checked") or 0),
        "matches_found": int(result.get("matches_found") or 0),
        "skipped_reason": result.get("skipped_reason"),
        "market_session": result.get("market_session"),
        "generated_at": result.get("generated_at") or datetime.now(timezone.utc).isoformat(),
    }
    path.write_text(json.dumps(payload), encoding="utf-8")

def entry_scan_status(module: str, settings: Settings | None = None) -> dict[str, Any]:
    settings = settings or get_settings()
    path = settings.artifacts_path / ENTRY_SCAN_STATUS
    if not path.exists():
        return {
            "symbols_checked": 0,
            "last_symbols_checked": 0,
            "patterns_checked": 0,
            "matches_found": 0,
            "generated_at": None,
        }
    try:
        payload = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return {
            "symbols_checked": 0,
            "last_symbols_checked": 0,
            "patterns_checked": 0,
            "matches_found": 0,
            "generated_at": None,
        }
    data = payload.get(module) or {}
    return {
        "symbols_checked": int(data.get("cumulative_symbols_checked") or data.get("symbols_checked") or 0),
        "last_symbols_checked": int(data.get("last_symbols_checked") or data.get("symbols_checked") or 0),
        "patterns_checked": int(data.get("patterns_checked") or 0),
        "matches_found": int(data.get("matches_found") or 0),
        "skipped_reason": data.get("skipped_reason"),
        "market_session": data.get("market_session"),
        "generated_at": data.get("generated_at"),
    }

```

Full Source: `backend/tradeo/services/watchdog.py`

`backend/tradeo/services/watchdog.py` ? 110 lines, 4561 bytes, sha256 prefix `dd4d1f1ea4a6fb1c`

Public surface summary before full source:

? class `SystemWatchdog` line 14 methods: inspect, repair

Risk hints: wall-clock timestamp usage.

```

from __future__ import annotations

from datetime import datetime, timedelta, timezone
from typing import Any

from loguru import logger
from sqlalchemy import func
from sqlalchemy.orm import Session

from tradeo.core.config import get_settings
from tradeo.db.models import AuditLog, DiscoveredPatternMatch, DiscoveryRun

class SystemWatchdog:

```

```
"""Cheap deterministic health checks and local recovery.
```

```
This deliberately avoids LLM calls. It only inspects local DB state and fixes  
scheduler blockers that are safe to repair from inside the app.  
"""
```

```
actor = "system_watchdog"
```

```
def inspect(self, db: Session) -> dict[str, Any]:  
    settings = get_settings()  
    now = datetime.now(timezone.utc)  
    stale_cutoff = now - timedelta(minutes=settings.watchdog_stale_discovery_minutes)  
    running_runs = (  
        db.query(DiscoveryRun)  
        .filter(DiscoveryRun.status == "running")  
        .order_by(DiscoveryRun.started_at.asc())  
        .all()  
    )  
    stale_runs = [run for run in running_runs if self._as_utc(run.started_at) < stale_cutoff]  
    latest_run = db.query(DiscoveryRun).order_by(DiscoveryRun.started_at.desc()).first()  
    latest_match_at = db.query(func.max(DiscoveredPatternMatch.matched_at)).scalar()  
    return {  
        "ok": not stale_runs,  
        "watchdog_enabled": settings.watchdog_enabled,  
        "running_discovery_runs": [  
            {  
                "id": run.id,  
                "started_at": run.started_at.isoformat(),  
                "age_seconds": round((now - self._as_utc(run.started_at)).total_seconds(), 1),  
            }  
            for run in running_runs  
        ],  
        "stale_discovery_runs": [run.id for run in stale_runs],  
        "latest_discovery_run": self._run_summary(latest_run),  
        "latest_pattern_match_at": latest_match_at.isoformat() if latest_match_at else None,  
    }  
  
def repair(self, db: Session) -> dict[str, Any]:  
    settings = get_settings()  
    status = self.inspect(db)  
    repaired: list[dict[str, Any]] = []  
    if not settings.watchdog_enabled:  
        return {**status, "repaired": repaired}  
    if settings.watchdog_close_stale_discovery_runs:  
        for run_id in status["stale_discovery_runs"]:  
            run = db.get(DiscoveryRun, run_id)  
            if run is None or run.status != "running":  
                continue  
            now = datetime.now(timezone.utc)  
            run.status = "failed"  
            run.finished_at = now  
            run.duration_seconds = round((now - self._as_utc(run.started_at)).total_seconds(), 3)  
            run.summary_json = {  
                *(run.summary_json or {}),  
                "error": "stale running discovery closed by system watchdog",  
                "closed_by": self.actor,  
                "closed_at": now.isoformat(),  
            }  
            db.add(  
                AuditLog(  
                    actor=self.actor,  
                    action="close_stale_discovery_run",  
                    entity_type="discovery_run",  
                    entity_id=str(run.id),  
                    details_json={  
                        "started_at": run.started_at.isoformat(),  
                        "duration_seconds": run.duration_seconds,  
                    },  
                )  
            )  
            repaired.append({"type": "discovery_run", "id": run.id, "action": "marked_failed"})  
    if repaired:  
        db.commit()  
        logger.warning("watchdog repaired stale runs: {}", repaired)  
    return {**self.inspect(db), "repaired": repaired}
```

```
@staticmethod
```

```
def _run_summary(run: DiscoveryRun | None) -> dict[str, Any] | None:  
    if run is None:  
        return None  
    return {  
        "id": run.id,  
        "status": run.status,  
        "started_at": run.started_at.isoformat(),  
        "finished_at": run.finished_at.isoformat() if run.finished_at else None,  
        "symbols_scanned": run.symbols_scanned,  
        "windows_sampled": run.windows_sampled,  
        "accepted_patterns": run.accepted_patterns,  
        "rejected_patterns": run.rejected_patterns,  
    }
```

```
@staticmethod
```

```
def _as_utc(value: datetime) -> datetime:  
    if value.tzinfo is None:  
        return value.replace(tzinfo=timezone.utc)  
    return value.astimezone(timezone.utc)
```

Full Source: `backend/tradeo/services/market\_session.py`

`backend/tradeo/services/market\_session.py` ? 106 lines, 3622 bytes, sha256 prefix `9bd03d3978a31e89`

Public surface summary before full source:

```
? function `is_us_equity_regular_session_open` line 11`  
? function `market_session_status` line 24`  
? function `is_nyse_holiday` line 47`  
? function `holiday_name` line 51`
```

Risk hints: wall-clock timestamp usage.

```
from __future__ import annotations  
  
from datetime import date, datetime, time, timedelta  
from zoneinfo import ZoneInfo  
  
US_EQUITY_TZ = ZoneInfo("America/New_York")  
REGULAR_OPEN = time(9, 30)  
REGULAR_CLOSE = time(16, 0)  
  
def is_us_equity_regular_session_open(now: datetime | None = None) -> bool:  
    current = now or datetime.now(US_EQUITY_TZ)  
    if current.tzinfo is None:  
        current = current.replace(tzinfo=US_EQUITY_TZ)  
    local = current.astimezone(US_EQUITY_TZ)  
    if local.weekday() >= 5:  
        return False  
    if is_nyse_holiday(local.date()):  
        return False  
    current_time = local.time()  
    return REGULAR_OPEN <= current_time < REGULAR_CLOSE  
  
def market_session_status(now: datetime | None = None) -> dict[str, object]:  
    current = now or datetime.now(US_EQUITY_TZ)  
    local = current.astimezone(US_EQUITY_TZ) if current.tzinfo else current.replace(tzinfo=US_EQUITY_TZ)  
    is_open = is_us_equity_regular_session_open(local)  
    return {  
        "market": "us_equities",  
        "timezone": "America/New_York",  
        "regular_session_open": is_open,  
        "state": _session_state(local, is_open),  
        "checked_at": local.isoformat(),  
        "regular_hours": "09:30-16:00",  
        "holiday": holiday_name(local.date()),  
    }  
  
def _session_state(local: datetime, is_open: bool) -> str:  
    if is_open:  
        return "regular_open"  
    if is_nyse_holiday(local.date()):  
        return "market_holiday"  
    return "market_closed"  
  
def is_nyse_holiday(day: date) -> bool:  
    return holiday_name(day) is not None  
  
def holiday_name(day: date) -> str | None:  
    holidays = {  
        _observed_fixed(day.year, 1, 1): "new_years_day",  
        _observed_fixed(day.year + 1, 1, 1): "new_years_day",  
        _nth_weekday(day.year, 1, 0, 3): "martin_luther_king_jr_day",  
        _nth_weekday(day.year, 2, 0, 3): "washingtons_birthday",  
        _good_friday(day.year): "good_friday",  
        _last_weekday(day.year, 5, 0): "memorial_day",  
        _observed_fixed(day.year, 6, 19): "juneteenth",  
        _observed_fixed(day.year, 7, 4): "independence_day",  
        _nth_weekday(day.year, 9, 0, 1): "labor_day",  
        _nth_weekday(day.year, 11, 3, 4): "thanksgiving_day",  
        _observed_fixed(day.year, 12, 25): "christmas_day",  
    }  
    return holidays.get(day)  
  
def _observed_fixed(year: int, month: int, day: int) -> date:  
    actual = date(year, month, day)  
    if actual.weekday() == 5:  
        return actual - timedelta(days=1)  
    if actual.weekday() == 6:  
        return actual + timedelta(days=1)  
    return actual  
  
def _nth_weekday(year: int, month: int, weekday: int, nth: int) -> date:  
    current = date(year, month, 1)
```

```

days_until = (weekday - current.weekday()) % 7
return current + timedelta(days=days_until + (nth - 1) * 7)

def _last_weekday(year: int, month: int, weekday: int) -> date:
    current = date(year, month + 1, 1) - timedelta(days=1) if month < 12 else date(year, 12, 31)
    while current.weekday() != weekday:
        current -= timedelta(days=1)
    return current

def _good_friday(year: int) -> date:
    # Anonymous Gregorian algorithm; NYSE closes on Good Friday.
    a = year % 19
    b = year // 100
    c = year % 100
    d = b // 4
    e = b % 4
    f = (b + 8) // 25
    g = (b - f + 1) // 3
    h = (19 * a + b - d - g + 15) % 30
    i = c // 4
    k = c % 4
    weekday_offset = (32 + 2 * e + 2 * i - h - k) % 7
    m = (a + 11 * h + 22 * weekday_offset) // 451
    month = (h + weekday_offset - 7 * m + 114) // 31
    day = ((h + weekday_offset - 7 * m + 114) % 31) + 1
    return date(year, month, day) - timedelta(days=2)

```

Full Source: ``backend/tradeo/services/reports.py``

``backend/tradeo/services/reports.py`` ? 165 lines, 6996 bytes, sha256 prefix ``a4fa6a0c51fb9dca``

Public surface summary before full source:

? class ``ReportService`` line 14 methods: `generate_review_pack`, `latest_report`

Risk hints: wall-clock timestamp usage; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, EquityPoint, PatternMetric, Signal, Trade

class ReportService:
    def __init__(self, settings: Settings | None = None) -> None:
        self.settings = settings or get_settings()

    def generate_review_pack(self, db: Session) -> dict[str, Any]:
        now = datetime.now(timezone.utc)
        signals = db.query(Signal).order_by(Signal.created_at.desc()).limit(50).all()
        trades = db.query(Trade).order_by(Trade.opened_at.desc()).limit(50).all()
        metrics = db.query(PatternMetric).all()
        equity = db.query(EquityPoint).order_by(EquityPoint.timestamp.desc()).limit(200).all()
        audits = db.query(AuditLog).order_by(AuditLog.timestamp.desc()).limit(100).all()
        pack: dict[str, Any] = {
            "generated_at_utc": now.isoformat(),
            "mode": self.settings.trading_mode,
            "live_armed": self.settings.live_armed,
            "kill_switch_enabled": self.settings.kill_switch_enabled,
            "risk_policy": {
                "initial_capital_usd": self.settings.initial_capital_usd,
                "risk_per_trade_pct": self.settings.risk_per_trade_pct,
                "risk_per_trade_usd": self.settings.account_risk_usd,
                "daily_loss_limit_pct": self.settings.daily_loss_limit_pct,
                "min_reward_risk": self.settings.min_reward_risk,
                "max_open_positions": self.settings.max_open_positions,
                "allow_shorts": self.settings.allow_shorts,
                "allow_options": self.settings.allow_options,
                "allow_margin": self.settings.allow_margin,
            },
            "signals": [self._signal_to_dict(s) for s in signals],
            "trades": [self._trade_to_dict(t) for t in trades],
            "metrics": [self._metric_to_dict(m) for m in metrics],
            "equity": [
                {
                    "timestamp": e.timestamp.isoformat(),
                    "equity": e.equity,
                    "cash": e.cash,
                    "open_risk": e.open_risk,
                }
                for e in reversed(equity)
            ],
            "audit_tail": [
                {

```

```

        "timestamp": a.timestamp.isoformat(),
        "actor": a.actor,
        "action": a.action,
        "entity_type": a.entity_type,
        "entity_id": a.entity_id,
        "details": a.details_json,
    }
    for a in audits
],
"director_prompt": self._director_prompt(),
}
json_path = self._write_json(pack, now)
md_path = self._write_markdown(pack, now)
pack["paths"] = {"json": str(json_path), "markdown": str(md_path)}
return pack

def latest_report(self) -> dict[str, Any] | None:
reports = sorted(self.settings.reports_path.glob("tradeo_review_*.json"), reverse=True)
if not reports:
    return None
return json.loads(reports[0].read_text())

def _write_json(self, pack: dict[str, Any], now: datetime) -> Path:
path = self.settings.reports_path / f"tradeo_review_{now:%Y%m%d_%H%M%S}.json"
path.write_text(json.dumps(pack, indent=2, default=str))
return path

def _write_markdown(self, pack: dict[str, Any], now: datetime) -> Path:
path = self.settings.reports_path / f"tradeo_review_{now:%Y%m%d_%H%M%S}.md"
lines = [
    f"# Tradeo review pack - {pack['generated_at_utc']}",
    "",
    "## Estado",
    f"- Modo: {pack['mode']}",
    f"- Live armado: {pack['live_armed']}",
    f"- Kill switch: {pack['kill_switch_enabled']}",
    "",
    "## Riesgo",
    f"- Capital inicial: {pack['risk_policy']['initial_capital_usd']} USD",
    f"- Riesgo por operación: {pack['risk_policy']['risk_per_trade_usd']} USD",
    f"- R:R mínimo: {pack['risk_policy']['min_reward_risk']}",
    "",
    "## Señales recientes",
]
for s in pack["signals"][:15]:
    lines.append(
        f"- {s['symbol']} {s['side']} {s['pattern']} status={s['status']} "
        f"entry={s['entry']} stop={s['stop']} target={s['target']} "
        f"RR={s['reward_risk']} conf={s['confidence']}"
    )
lines.extend(["", "## Prompt para revisión externa", "", pack["director_prompt"]])
path.write_text("\n".join(lines))
return path

def _director_prompt(self) -> str:
return (
    "Actúa como Director General de Tradeo. Revisa este paquete JSON. "
    "Evalúa si cada patrón técnico tiene coherencia geométrica, riesgo controlado, "
    "R:R mínimo 1:4, liquidez suficiente y ausencia de inconsistencias. "
    "No apruebes operativa real si hay menos de 40 operaciones históricas por versión, "
    "profit factor < 1.8, expectancy <= 0.25R o drawdown máximo > 12%. "
    "Devuelve: aprobar/rechazar, razones, cambios propuestos y próximos experimentos."
)

def _signal_to_dict(self, s: Signal) -> dict[str, Any]:
return {
    "id": s.id,
    "symbol": s.symbol,
    "pattern": s.pattern,
    "side": s.side,
    "timeframe": s.timeframe,
    "entry": s.entry,
    "stop": s.stop,
    "target": s.target,
    "reward_risk": s.reward_risk,
    "confidence": s.confidence,
    "status": s.status.value,
    "human_approved": s.human_approved,
    "created_at": s.created_at.isoformat(),
    "notes": s.supervisor_notes,
    "metadata": s.metadata_json,
}

def _trade_to_dict(self, t: Trade) -> dict[str, Any]:
return {
    "id": t.id,
    "symbol": t.symbol,
    "pattern": t.pattern,
    "side": t.side,
    "qty": t.qty,
    "entry": t.entry,
    "stop": t.stop,
    "target": t.target,
    "status": t.status.value,
    "pnl_usd": t.pnl_usd,
    "r_multiple": t.r_multiple,

```

```

        "opened_at": t.opened_at.isoformat(),
        "closed_at": t.closed_at.isoformat() if t.closed_at else None,
    }

def _metric_to_dict(self, m: PatternMetric) -> dict[str, Any]:
    return {
        "pattern": m.pattern,
        "strategy_version": m.strategy_version,
        "total_trades": m.total_trades,
        "win_rate": m.win_rate,
        "profit_factor": m.profit_factor,
        "expectancy_r": m.expectancy_r,
        "max_drawdown_pct": m.max_drawdown_pct,
        "avg_r_multiple": m.avg_r_multiple,
    }

```

Full Source: `backend/tradeo/research/types.py`

`backend/tradeo/research/types.py` ? 189 lines, 6422 bytes, sha256 prefix `228a7c61177e3697`

Public surface summary before full source:

```

? class `ForwardOutcome` line 12 methods: outcome_for, mfe_for, mae_for, hit_4r_for, label_for, time_to_target_for,
time_to_stop_for, speed_label_for
? class `WindowSample` line 79 methods: ?
? class `ClusterCandidate` line 93 methods: ?
? class `HypothesisPackage` line 132 methods: to_dict

? function `freeze_json` line 112`
? function `thaw_json` line 123`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Literal

import numpy as np

Side = Literal["long", "short"]

@dataclass(slots=True)
class ForwardOutcome:
    """Forward path statistics measured in R units.

    R is calculated with a technical risk proxy at the end of the input window.
    For discovery we do not know the final execution stop yet, so this proxy is
    intentionally conservative and consistent across every window.
    """

    forward_returns: dict[int, float]
    entry_price: float
    risk_proxy: float
    forward_end: str
    long_mfe_r: float
    long_mae_r: float
    long_outcome_r: float
    long_hit_4r: bool
    short_mfe_r: float
    short_mae_r: float
    short_outcome_r: float
    short_hit_4r: bool
    forward_highs: list[float] = field(default_factory=list)
    forward_lows: list[float] = field(default_factory=list)
    forward_closes: list[float] = field(default_factory=list)
    forward_opens: list[float] = field(default_factory=list)
    execution_cost_r: float = 0.0
    long_label: str = "timeout"
    short_label: str = "timeout"
    long_time_to_target: int | None = None
    long_time_to_stop: int | None = None
    short_time_to_target: int | None = None
    short_time_to_stop: int | None = None
    long_mfe_before_mae: bool = False
    short_mfe_before_mae: bool = False
    long_gap_adverse_r: float = 0.0
    short_gap_adverse_r: float = 0.0
    long_strong_close_without_target: bool = False
    short_strong_close_without_target: bool = False
    long_speed_label: str = "unknown"
    short_speed_label: str = "unknown"
    execution: dict[str, float] = field(default_factory=dict)

    def outcome_for(self, side: Side) -> float:
        return self.long_outcome_r if side == "long" else self.short_outcome_r

    def mfe_for(self, side: Side) -> float:
        return self.long_mfe_r if side == "long" else self.short_mfe_r

```

```

def mae_for(self, side: Side) -> float:
    return self.long_mae_r if side == "long" else self.short_mae_r

def hit_4r_for(self, side: Side) -> bool:
    return self.long_hit_4r if side == "long" else self.short_hit_4r

def label_for(self, side: Side) -> str:
    return self.long_label if side == "long" else self.short_label

def time_to_target_for(self, side: Side) -> int | None:
    return self.long_time_to_target if side == "long" else self.short_time_to_target

def time_to_stop_for(self, side: Side) -> int | None:
    return self.long_time_to_stop if side == "long" else self.short_time_to_stop

def speed_label_for(self, side: Side) -> str:
    return self.long_speed_label if side == "long" else self.short_speed_label

@dataclass(slots=True)
class WindowSample:
    symbol: str
    timeframe: str
    window_size: int
    start: str
    end: str
    year: int
    vector: np.ndarray
    outcome: ForwardOutcome
    chart: dict[str, Any]
    features: dict[str, Any] = field(default_factory=dict)

@dataclass(slots=True)
class ClusterCandidate:
    pattern_key: str
    name: str
    side: Side
    timeframe: str
    window_size: int
    cluster_id: int
    centroid: list[float]
    sample_count: int
    symbol_count: int
    year_count: int
    score: float
    validation_passed: bool
    validation_reasons: list[str]
    metrics: dict[str, Any]
    feature_summary: dict[str, Any]
    examples: list[dict[str, Any]]

def freeze_json(value: Any) -> Any:
    """Return a tuple-only representation for immutable research packages."""
    if isinstance(value, dict):
        return tuple((str(key), freeze_json(value[key])) for key in sorted(value, key=str))
    if isinstance(value, list | tuple):
        return tuple(freeze_json(item) for item in value)
    if isinstance(value, np.generic):
        return value.item()
    return value

def thaw_json(value: Any) -> Any:
    if isinstance(value, tuple):
        if all(isinstance(item, tuple) and len(item) == 2 and isinstance(item[0], str) for item in value):
            return {key: thaw_json(child) for key, child in value}
        return [thaw_json(item) for item in value]
    return value

@dataclass(frozen=True, slots=True)
class HypothesisPackage:
    """Immutable research handoff for a candidate hypothesis.

    The package deliberately keeps `edge_claim` at NO_DEMOSTRADO. Any later
    paper/live promotion must be backed by separate Director evidence.
    """

    version: str
    pattern_key: str
    family_id: str
    variant_id: str
    edge_claim: Literal["NO_DEMOSTRADO"]
    falsifiable: bool
    thesis: str
    rule: str
    causal_mechanism: str
    works_when: tuple[str, ...]
    fails_when: tuple[str, ...]
    kill_conditions: tuple[str, ...]
    selection_split: Any
    fit_scope: Any
    train_metrics: Any
    out_of_sample_metrics: Any

```

```

walk_forward_metrics: Any
global_trial_count: int
event_ledger_hash: str
nested_discovery_replay: Any
evidence_accumulated: Any
falsification_tests: tuple[str, ...]
current_verdict: str

def to_dict(self) -> dict[str, Any]:
    return {
        "version": self.version,
        "pattern_key": self.pattern_key,
        "family_id": self.family_id,
        "variant_id": self.variant_id,
        "edge_claim": self.edge_claim,
        "falsifiable": self.falsifiable,
        "thesis": self.thesis,
        "rule": self.rule,
        "causal_mechanism": self.causal_mechanism,
        "works_when": list(self.works_when),
        "fails_when": list(self.fails_when),
        "kill_conditions": list(self.kill_conditions),
        "kill_criteria": list(self.kill_conditions),
        "selection_split": thaw_json(self.selection_split),
        "fit_scope": thaw_json(self.fit_scope),
        "train_metrics": thaw_json(self.train_metrics),
        "out_of_sample_metrics": thaw_json(self.out_of_sample_metrics),
        "walk_forward_metrics": thaw_json(self.walk_forward_metrics),
        "global_trial_count": self.global_trial_count,
        "event_ledger_hash": self.event_ledger_hash,
        "nested_discovery_replay": thaw_json(self.nested_discovery_replay),
        "evidence_accumulated": thaw_json(self.evidence_accumulated),
        "falsification_tests": list(self.falsification_tests),
        "current_verdict": self.current_verdict,
    }

```

Full Source: ``backend/tradeo/research/window_sampler.py``

``backend/tradeo/research/window_sampler.py`` ? 286 lines, 13940 bytes, sha256 prefix ``d1fce7c0ca5f444c``

Public surface summary before full source:

? class ``WindowSampler`` line 15 methods: `sample`

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable

import numpy as np
import pandas as pd

from tradeo.research.pattern_embedding_engine import PatternEmbeddingEngine
from tradeo.research.types import ForwardOutcome, WindowSample
from tradeo.services.technical_indicators import atr, normalize_ohlc

@dataclass(slots=True)
class WindowSampler:
    """Extract fixed-length pattern windows and label each with future path stats."""

    embedding_engine: PatternEmbeddingEngine | None = None
    target_r: float = 4.0
    stop_r: float = 1.0
    min_risk_pct: float = 0.015
    atr_multiplier: float = 1.5

    def __post_init__(self) -> None:
        self.embedding_engine = self.embedding_engine or PatternEmbeddingEngine()

    def sample(
        self,
        symbol: str,
        df: pd.DataFrame,
        timeframe: str,
        window_sizes: Iterable[int],
        forward_bars: Iterable[int],
        stride: int = 3,
        max_windows_per_symbol: int = 450,
        benchmark_frames: dict[str, pd.DataFrame] | None = None,
    ) -> list[WindowSample]:
        df = normalize_ohlc(df).dropna()
        forward_bars = sorted({int(x) for x in forward_bars if int(x) > 0})
        if not forward_bars:
            raise ValueError("at least one forward horizon is required")
        max_forward = max(forward_bars)
        if len(df) < max(window_sizes) + max_forward + 5:
            return []

        atr_series = atr(df, 14).bfill().fillna(df["close"] * self.min_risk_pct)
        samples: list[WindowSample] = []

```

```

for window_size in sorted([int(x) for x in window_sizes if int(x) >= 10]):
    if len(df) < window_size + max_forward + 2:
        continue
    # Sample from old to new. A stride keeps the system cheap enough to
    # run continuously, while still covering overlapping market states.
    for end_pos in range(window_size - 1, len(df) - max_forward - 1, max(1, stride)):
        window = df.iloc[end_pos - window_size + 1 : end_pos + 1]
        future = df.iloc[end_pos + 1 : end_pos + 1 + max_forward]
        if future.empty:
            continue
        entry = float(window["close"].iloc[-1])
        if entry <= 0:
            continue
        risk_proxy = max(
            float(atr_series.iloc[end_pos]) * self.atr_multiplier,
            entry * self.min_risk_pct,
            0.01,
        )
        execution_metrics = self._execution_cost_metrics(window, entry=entry, risk_proxy=risk_proxy)
        execution_cost_r = float(execution_metrics["execution_cost_r"])
        outcome = self._forward_outcome(
            entry,
            risk_proxy,
            future,
            forwardBars,
            execution_cost_r,
            execution_metrics,
        )
        vector, features, chart = self.embedding_engine.embed(window, benchmark_frames=benchmark_frames)
        features.update(self._data_lineage_features(window))
        features.update({f"execution_{k}": float(v) for k, v in execution_metrics.items()})
        start_idx = window.index[0]
        end_idx = window.index[-1]
        year = self._year(end_idx)
        samples.append(
            WindowSample(
                symbol=symbol.upper(),
                timeframe=timeframe,
                window_size=window_size,
                start=self._date_str(start_idx),
                end=self._date_str(end_idx),
                year=year,
                vector=vector,
                outcome=outcome,
                chart=chart,
                features=features,
            )
        )
        if len(samples) >= max_windows_per_symbol:
            return samples
    return samples

def _forward_outcome(
    self,
    entry: float,
    risk_proxy: float,
    future: pd.DataFrame,
    forwardBars: list[int],
    execution_cost_r: float = 0.0,
    execution_metrics: dict[str, float] | None = None,
) -> ForwardOutcome:
    closes = future["close"].astype(float).to_numpy()
    highs = future["high"].astype(float).to_numpy()
    lows = future["low"].astype(float).to_numpy()
    opens = future["open"].astype(float).to_numpy() if "open" in future else closes
    forward_returns = {}
    for horizon in forwardBars:
        idx = min(horizon, len(closes)) - 1
        forward_returns[horizon] = float(closes[idx] / entry - 1.0)

    long_mfe_r = float(np.max(highs - entry) / risk_proxy)
    long_mae_r = float(max(0.0, np.max(entry - lows) / risk_proxy))
    short_mfe_bar = float(np.max(entry - lows) / risk_proxy)
    short_mae_r = float(max(0.0, np.max(highs - entry) / risk_proxy))
    long_path = self._path_outcome(entry, risk_proxy, future, side="long")
    short_path = self._path_outcome(entry, risk_proxy, future, side="short")
    long_outcome_r, long_hit_4r, long_target_bar, long_stop_bar, long_label = long_path
    short_outcome_r, short_hit_4r, short_target_bar, short_stop_bar, short_label = short_path
    first_open = float(opens[0]) if len(opens) else float(entry)
    long_gap_adverse_r = max(0.0, (entry - first_open) / max(risk_proxy, 1e-9))
    short_gap_adverse_r = max(0.0, (first_open - entry) / max(risk_proxy, 1e-9))
    long_mfe_bar = int(np.argmax(highs - entry) + 1) if len(highs) else 0
    long_mae_bar = int(np.argmax(entry - lows) + 1) if len(lows) else 0
    short_mfe_bar = int(np.argmax(entry - lows) + 1) if len(lows) else 0
    short_mae_bar = int(np.argmax(highs - entry) + 1) if len(highs) else 0
    strong_close_threshold = self.target_r * 0.75
    long_final_r = float((closes[-1] - entry) / risk_proxy) if len(closes) else 0.0
    short_final_r = float((entry - closes[-1]) / risk_proxy) if len(closes) else 0.0
    return ForwardOutcome(
        forward_returns=forward_returns,
        entry_price=float(entry),
        risk_proxy=float(risk_proxy),
        forward_end=self._date_str(future.index[-1]),
        long_mfe_r=round(long_mfe_r, 4),
        long_mae_r=round(long_mae_r, 4),
        long_outcome_r=round(long_outcome_r, 4),
    )

```

```

        long_hit_4r=long_hit_4r,
        short_mfe_r=round(short_mfe_r, 4),
        short_mae_r=round(short_mae_r, 4),
        short_outcome_r=round(short_outcome_r, 4),
        short_hit_4r=short_hit_4r,
        forward_highs=np.round(highs.astype(float), 6).tolist(),
        forward_lows=np.round(lows.astype(float), 6).tolist(),
        forward_closes=np.round(closes.astype(float), 6).tolist(),
        forward_opens=np.round(opens.astype(float), 6).tolist(),
        execution_cost_r=round(float(execution_cost_r), 5),
        long_label=long_label,
        short_label=short_label,
        long_time_to_target=long_target_bar,
        long_time_to_stop=long_stop_bar,
        short_time_to_target=short_target_bar,
        short_time_to_stop=short_stop_bar,
        long_mfe_before_mae=long_mfe_bar > 0 and (long_mae_bar == 0 or long_mfe_bar <= long_mae_bar),
        short_mfe_before_mae=short_mfe_bar > 0 and (short_mae_bar == 0 or short_mfe_bar <= short_mae_bar),
        long_gap_adverse_r=round(float(long_gap_adverse_r), 5),
        short_gap_adverse_r=round(float(short_gap_adverse_r), 5),
        long_strong_close_without_target=long_target_bar is None and long_final_r >= strong_close_threshold,
        short_strong_close_without_target=short_target_bar is None and short_final_r >= strong_close_threshold,
        long_speed_label=self._speed_label(long_target_bar, long_stop_bar, len(closes)),
        short_speed_label=self._speed_label(short_target_bar, short_stop_bar, len(closes)),
        execution={k: round(float(v), 6) for k, v in (execution_metrics or {}).items()},
    )

    @staticmethod
    def _execution_cost_r(window: pd.DataFrame, *, entry: float, risk_proxy: float) -> float:
        metrics = WindowSampler._execution_cost_metrics(window, entry=entry, risk_proxy=risk_proxy)
        return float(metrics["execution_cost_r"])

    @staticmethod
    def _execution_cost_metrics(window: pd.DataFrame, *, entry: float, risk_proxy: float) -> dict[str, float]:
        latest = window.iloc[-1]
        previous_close = float(window["close"].iloc[-2]) if len(window) >= 2 else float(latest["close"])
        range_pct = float((latest["high"] - latest["low"]) / max(entry, 1e-9))
        avg_dollar_volume = float((window["close"] * window["volume"]).tail(20).mean())
        adv_shares = float(window["volume"].tail(20).mean())
        atr_pct_proxy = max(
            range_pct,
            float((window["high"] - window["low"]).tail(14).mean() / max(entry, 1e-9)),
        )
        liquidity_penalty_pct = 0.0015 if avg_dollar_volume < 5_000_000 else 0.0008
        price_penalty_pct = 0.0012 if entry < 5 else 0.0004 if entry < 15 else 0.00015
        liquidity_factor = 1.0 - min(avg_dollar_volume / 50_000_000, 1.0)
        spread_proxy_pct = min(0.025, max(0.0004, range_pct * (0.05 + 0.15 * liquidity_factor)))
        participation_rate = min(0.05, max(0.001, 250_000.0 / max(avg_dollar_volume, 1.0)))
        slippage_pct = min(0.03, atr_pct_proxy * (0.03 + np.sqrt(participation_rate) * 0.25))
        entry_gap_pct = abs(float(latest["open"]) / max(previous_close, 1e-9) - 1.0)
        entry_gap_penalty_pct = min(0.02, entry_gap_pct * 0.35)
        short_borrow_proxy_pct = min(
            0.015,
            0.00025 + max(0.0, 5_000_000.0 - avg_dollar_volume) / 5_000_000.0 * 0.004,
        )
        adverse_fill_pressure = min(
            1.0,
            spread_proxy_pct / 0.02 + slippage_pct / 0.03 + entry_gap_penalty_pct / 0.02,
        )
        fill_probability = max(0.10, min(0.99, 0.98 - adverse_fill_pressure * 0.22))
        max_size_usd = max(0.0, min(avg_dollar_volume * 0.005, adv_shares * entry * 0.02))
        roundtrip_cost_pct = (
            liquidity_penalty_pct
            + price_penalty_pct
            + spread_proxy_pct
            + slippage_pct
            + entry_gap_penalty_pct
        )
        execution_cost_r = max(0.0, float(entry * roundtrip_cost_pct / max(risk_proxy, 1e-9)))
        return {
            "execution_cost_r": execution_cost_r,
            "spread_proxy_pct": float(spread_proxy_pct),
            "slippage_proxy_pct": float(slippage_pct),
            "entry_gap_penalty_pct": float(entry_gap_penalty_pct),
            "liquidity_penalty_pct": float(liquidity_penalty_pct),
            "price_penalty_pct": float(price_penalty_pct),
            "short_borrow_proxy_pct": float(short_borrow_proxy_pct),
            "fill_probability": float(fill_probability),
            "max_size_usd": float(max_size_usd),
            "avg_dollar_volume": float(avg_dollar_volume),
            "participation_rate": float(participation_rate),
        }

    def _path_outcome(
        self,
        entry: float,
        risk: float,
        future: pd.DataFrame,
        side: str,
    ) -> tuple[float, bool, int | None, int | None, str]:
        if side == "long":
            target = entry + self.target_r * risk
            stop = entry - self.stop_r * risk
            for bar, (_, row) in enumerate(future.iterrows(), start=1):
                # Conservative path assumption: if both are touched intrabar,
                # stop is considered first. That prevents discovery from being

```



```

        # overly optimistic on daily bars.
        if float(row["low"]) <= stop:
            return -self.stop_r, False, None, bar, "stop"
        if float(row["high"]) >= target:
            return self.target_r, True, bar, None, "target"
        return float((future["close"].iloc[-1] - entry) / risk), False, None, None, "timeout"
    target = entry - self.target_r * risk
    stop = entry + self.stop_r * risk
    for bar, (_, row) in enumerate(future.iterrows(), start=1):
        if float(row["high"]) >= stop:
            return -self.stop_r, False, None, bar, "stop"
        if float(row["low"]) <= target:
            return self.target_r, True, bar, None, "target"
    return float((entry - future["close"].iloc[-1]) / risk), False, None, None, "timeout"

    @staticmethod
    def _speed_label(target_bar: int | None, stop_bar: int | None, horizon: int) -> str:
        if target_bar is None:
            return "stopped" if stop_bar is not None else "timeout"
        fast_cutoff = max(2, int(np.ceil(max(horizon, 1) * 0.25)))
        slow_cutoff = max(fast_cutoff + 1, int(np.ceil(max(horizon, 1) * 0.70)))
        if target_bar <= fast_cutoff:
            return "fast_target"
        if target_bar >= slow_cutoff:
            return "slow_target"
        return "normal_target"

    @staticmethod
    def _data_lineage_features(window: pd.DataFrame) -> dict[str, object]:
        latest = window.iloc[-1]
        lineage: dict[str, object] = {}
        if "adjusted" in window.columns:
            lineage["data_adjusted"] = bool(latest["adjusted"])
        if "what_to_show" in window.columns:
            lineage["data_what_to_show"] = str(latest["what_to_show"])
        if "bar_complete" in window.columns:
            lineage["data_bar_complete"] = bool(latest["bar_complete"])
        return lineage

    @staticmethod
    def _date_str(value: object) -> str:
        ts = pd.Timestamp(value)
        return ts.isoformat()

    @staticmethod
    def _year(value: object) -> int:
        return int(pd.Timestamp(value).year)

```

Full Source: `backend/tradeo/research/reward\_risk\_analyzer.py`

`backend/tradeo/research/reward\_risk\_analyzer.py` ? 303 lines, 14583 bytes, sha256 prefix `4d328deda65ba72f`

Audit relevance: Canonical reward/risk and skipped-signal accounting. Audit triple barrier semantics, cost model, RR metrics, and sample counts.

Public surface summary before full source:

? class `RewardRiskAnalyzer` line 12 methods: analyze, metrics_for_rr

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from tradeo.research.quant_validation import triple_barrier_outcome
from tradeo.research.types import Side, WindowSample

@dataclass(slots=True)
class RewardRiskAnalyzer:
    """Evaluate a cluster across multiple target R levels.

    The scoring is intentionally adjustable. It favors real edge quality:
    expectancy, profit factor, enough hit rate, MFE/MAE efficiency, and limited
    drawdown. Higher R is not automatically better.
    """

    rr_levels: list[float]
    min_samples: int = 100

    def analyze(self, samples: list[WindowSample], side: Side) -> dict[str, object]:
        metrics = {self._key(rr): self.metrics_for_rr(samples, side, rr) for rr in self.rr_levels}
        candidates = [
            m
            for m in metrics.values()
            if float(m["expectancy_r"]) > 0
            and float(m["profit_factor"]) > 1
            and int(m["sample_count"]) >= self.min_samples
        ]
        best = max(

```

```

        candidates,
        key=self._edge_score,
        default=max(metrics.values(), key=self._edge_score) if metrics else {},
    )
    best_rr = float(best.get("rr", 0.0)) if best else 0.0
    return {
        "rr_metrics": metrics,
        "best_rr": best_rr,
        "best_tested_rr": best_rr,
        "best_expectancy_r": float(best.get("expectancy_r", 0.0)) if best else 0.0,
        "best_profit_factor": float(best.get("profit_factor", 0.0)) if best else 0.0,
        "best_win_rate": float(best.get("win_rate", 0.0)) if best else 0.0,
        "best_max_drawdown_r": float(best.get("max_drawdown_r", 0.0)) if best else 0.0,
        "best_score": round(self._edge_score(best), 5) if best else 0.0,
    }

def metrics_for_rr(
    self,
    samples: list[WindowSample],
    side: Side,
    rr: float,
    *,
    cost_multiplier: float = 1.0,
) -> dict[str, float | int]:
    """Aggregate canonical triple-barrier outcomes for one target R level.

    Skipped signals (gap-entry policy) are true non-trades: they never enter
    the traded arrays, so win/loss/expectancy/drawdown reflect executed
    trades only. They are accounted separately via ``signal_count``,
    ``skipped_count``, ``skip_rate`` and ``skip_reason_counts``;
    ``sample_count`` counts traded samples only.
    """
    results: list[float] = []
    target_bars: list[int] = []
    stop_bars: list[int] = []
    mfe: list[float] = []
    mae: list[float] = []
    costs: list[float] = []
    gap_adverse: list[float] = []
    mfe_before_mae: list[bool] = []
    strong_close_without_target: list[bool] = []
    labels: dict[str, int] = {"target": 0, "stop": 0, "timeout": 0, "skipped": 0}
    skip_reasons: dict[str, int] = {}
    signal_count = 0
    speed_labels: dict[str, int] = {}
    fill_probabilities: list[float] = []
    max_sizes: list[float] = []
    spread_pct: list[float] = []
    slippage_pct: list[float] = []
    entry_gap_penalty_pct: list[float] = []
    short_borrow_pct: list[float] = []
    for sample in samples:
        signal_count += 1
        detail = self._simulate_sample_detail(sample, side, rr, cost_multiplier=cost_multiplier)
        status = str(detail.get("status", "ok"))
        if status not in ("ok", "fallback"):
            # skipped / invalid / no_data: a non-trade, never aggregated as 0R.
            labels["skipped"] += 1
            reason = str(detail.get("reason", "")) or status
            skip_reasons[reason] = skip_reasons.get(reason, 0) + 1
            continue
        result, target_bar, stop_bar = self._tuple_from_detail(detail)
        results.append(result)
        if target_bar is not None:
            target_bars.append(target_bar)
        if stop_bar is not None:
            stop_bars.append(stop_bar)
        mfe.append(sample.outcome.mfe_for(side))
        mae.append(sample.outcome.mae_for(side))
        costs.append(self._execution_cost_for_sample(sample, side, cost_multiplier=cost_multiplier))
        label = self._label_from_detail(detail, target_bar=target_bar, stop_bar=stop_bar)
        labels[label] = labels.get(label, 0) + 1
        speed = self._speed_label(target_bar, stop_bar, len(sample.outcome.forward_closes))
        speed_labels[speed] = speed_labels.get(speed, 0) + 1
        if side == "long":
            gap_adverse.append(sample.outcome.long_gap_adverse_r)
            mfe_before_mae.append(sample.outcome.long_mfe_before_mae)
            strong_close_without_target.append(sample.outcome.long_strong_close_without_target)
        else:
            gap_adverse.append(sample.outcome.short_gap_adverse_r)
            mfe_before_mae.append(sample.outcome.short_mfe_before_mae)
            strong_close_without_target.append(sample.outcome.short_strong_close_without_target)
        execution = sample.outcome.execution or {}
        fill_probabilities.append(float(execution.get("fill_probability", 0.0)))
        max_sizes.append(float(execution.get("max_size_usd", 0.0)))
        spread_pct.append(float(execution.get("spread_proxy_pct", 0.0)))
        slippage_pct.append(float(execution.get("slippage_proxy_pct", 0.0)))
        entry_gap_penalty_pct.append(float(execution.get("entry_gap_penalty_pct", 0.0)))
        short_borrow_pct.append(float(execution.get("short_borrow_proxy_pct", 0.0)))

    arr = np.asarray(results, dtype=float)
    wins = arr[arr > 0]
    losses = arr[arr < 0]
    gross_win = float(wins.sum()) if len(wins) else 0.0
    gross_loss = abs(float(losses.sum())) if len(losses) else 0.0
    pf = gross_win / gross_loss if gross_loss > 0 else gross_win

```

```

avg_mfe = float(np.mean(mfe)) if mfe else 0.0
avg_mae = float(np.mean(mae)) if mae else 0.0
return {
    "rr": float(rr),
    "target_r": float(target_r),
    "win_rate": round(float(np.mean(arr > 0)), 5) if len(arr) else 0.0,
    "loss_rate": round(float(np.mean(arr < 0)), 5) if len(arr) else 0.0,
    "target_hit_rate": round(len(target_bars) / len(arr), 5) if len(arr) else 0.0,
    "stop_hit_rate": round(len(stop_bars) / len(arr), 5) if len(arr) else 0.0,
    "expectancy_r": round(float(np.mean(arr)), 5) if len(arr) else 0.0,
    "profit_factor": round(float(pf), 5),
    "avg_win_r": round(float(np.mean(wins)), 5) if len(wins) else 0.0,
    "avg_loss_r": round(float(abs(np.mean(losses))), 5) if len(losses) else 0.0,
    "median_result_r": round(float(np.median(arr)), 5) if len(arr) else 0.0,
    "avg_mfe_r": round(avg_mfe, 5),
    "avg_mae_r": round(avg_mae, 5),
    "mfe_mae_ratio": round(avg_mfe / max(avg_mae, 1e-9), 5),
    "avg_execution_cost_r": round(float(np.mean(costs)), 5) if costs else 0.0,
    "max_drawdown_r": round(self._max_drawdown(arr), 5),
    "avg_bars_to_target": round(float(np.mean(target_bars)), 3) if target_bars else 0.0,
    "avg_bars_to_stop": round(float(np.mean(stop_bars)), 3) if stop_bars else 0.0,
    "triple_barrier_labels": labels,
    "avg_gap_adverse_r": round(float(np.mean(gap_adverse)), 5) if gap_adverse else 0.0,
    "mfe_before_mae_rate": round(float(np.mean(mfe_before_mae)), 5) if mfe_before_mae else 0.0,
    "strong_close_without_target_rate": round(float(np.mean(strong_close_without_target)), 5)
    if strong_close_without_target
    else 0.0,
    "speed_label_counts": speed_labels,
    "fast_target_rate": round(speed_labels.get("fast_target", 0) / len(arr), 5) if len(arr) else 0.0,
    "slow_target_rate": round(speed_labels.get("slow_target", 0) / len(arr), 5) if len(arr) else 0.0,
    "timeout_rate": round(labels.get("timeout", 0) / len(arr), 5) if len(arr) else 0.0,
    "avg_fill_probability": round(float(np.mean(fill_probabilities)), 5) if fill_probabilities else 0.0,
    "p25_max_size_usd": round(float(np.quantile(max_sizes, 0.25)), 2) if max_sizes else 0.0,
    "median_max_size_usd": round(float(np.median(max_sizes)), 2) if max_sizes else 0.0,
    "avg_spread_proxy_pct": round(float(np.mean(spread_pct)), 6) if spread_pct else 0.0,
    "avg_slippage_proxy_pct": round(float(np.mean(slippage_pct)), 6) if slippage_pct else 0.0,
    "avg_entry_gap_penalty_pct": round(float(np.mean(entry_gap_penalty_pct)), 6)
    if entry_gap_penalty_pct
    else 0.0,
    "avg_short_borrow_proxy_pct": round(float(np.mean(short_borrow_pct)), 6)
    if short_borrow_pct
    else 0.0,
    "sample_count": int(len(arr)),
    "signal_count": int(signal_count),
    "skipped_count": int(labels["skipped"]),
    "skip_rate": round(labels["skipped"] / signal_count, 5) if signal_count else 0.0,
    "skip_reason_counts": skip_reasons,
    "cost_multiplier": float(cost_multiplier),
}

@staticmethod
def _simulate_sample(
    sample: WindowSample,
    side: Side,
    rr: float,
    *,
    cost_multiplier: float = 1.0,
) -> tuple[float, int | None, int | None]:
    detail = RewardRiskAnalyzer._simulate_sample_detail(sample, side, rr, cost_multiplier=cost_multiplier)
    return RewardRiskAnalyzer._tuple_from_detail(detail)

@staticmethod
def _simulate_sample_detail(
    sample: WindowSample,
    side: Side,
    rr: float,
    *,
    cost_multiplier: float = 1.0,
) -> dict[str, object]:
    signal_entry = float(sample.outcome.entry_price)
    risk = max(float(sample.outcome.risk_proxy), 1e-9)
    highs = sample.outcome.forward_highs
    lows = sample.outcome.forward_lows
    closes = sample.outcome.forward_closes
    opens = sample.outcome.forward_opens or [signal_entry for _ in closes]
    if not highs or not lows or not closes:
        fallback = max(-1.0, min(float(rr), float(sample.outcome.outcome_for(side))))
        fallback -= RewardRiskAnalyzer._execution_cost_for_sample(sample, side, cost_multiplier=cost_multiplier)
        return {"status": "fallback", "R": float(fallback), "reason": "missing_forward_path"}

    side_int = 1 if side == "long" else -1
    stop = signal_entry - risk if side_int > 0 else signal_entry + risk
    target = signal_entry + float(rr) * risk if side_int > 0 else signal_entry - float(rr) * risk
    n = min(len(opens), len(highs), len(lows), len(closes))
    detail = triple_barrier_outcome(
        [signal_entry, *[float(x) for x in opens[:n]]],
        [signal_entry, *[float(x) for x in highs[:n]]],
        [signal_entry, *[float(x) for x in lows[:n]]],
        [signal_entry, *[float(x) for x in closes[:n]]],
        signal_index=0,
        side=side_int,
        stop_price=float(stop),
        target_price=float(target),
        max_bars=n,
        gap_entry_policy="skip",
        round_trip_cost_R=RewardRiskAnalyzer._execution_cost_for_sample(

```

```

        sample,
        side,
        cost_multiplier=cost_multiplier,
    ),
)
detail["canonical_outcome"] = True
return detail

@staticmethod
def _tuple_from_detail(detail: dict[str, object]) -> tuple[float, int | None, int | None]:
    if detail.get("status") == "skipped":
        return 0.0, None, None
    result = float(detail.get("R", 0.0) or 0.0)
    reason = str(detail.get("reason", ""))
    bars_held = int(detail.get("bars_held", 0) or 0)
    target_bar = bars_held if reason.startswith("target") else None
    stop_bar = bars_held if reason.startswith("stop") else None
    return result, target_bar, stop_bar

@staticmethod
def _label_from_detail(detail: dict[str, object], *, target_bar: int | None, stop_bar: int | None) -> str:
    if detail.get("status") == "skipped":
        return "skipped"
    if target_bar is not None:
        return "target"
    if stop_bar is not None:
        return "stop"
    return "timeout"

@staticmethod
def _execution_cost_for_sample(sample: WindowSample, side: Side, *, cost_multiplier: float = 1.0) -> float:
    cost_r = max(0.0, float(sample.outcome.execution_cost_r))
    if side == "short":
        borrow_pct = max(0.0, float((sample.outcome.execution or {}).get("short_borrow_proxy_pct", 0.0)))
        borrow_r = sample.outcome.entry_price * borrow_pct / max(sample.outcome.risk_proxy, 1e-9)
        cost_r += borrow_r
    return float(cost_r * max(0.0, float(cost_multiplier)))

@staticmethod
def _speed_label(target_bar: int | None, stop_bar: int | None, horizon: int) -> str:
    if target_bar is None:
        return "stopped" if stop_bar is not None else "timeout"
    fast_cutoff = max(2, int(np.ceil(max(horizon, 1) * 0.25)))
    slow_cutoff = max(fast_cutoff + 1, int(np.ceil(max(horizon, 1) * 0.70)))
    if target_bar <= fast_cutoff:
        return "fast_target"
    if target_bar >= slow_cutoff:
        return "slow_target"
    return "normal_target"

@staticmethod
def _max_drawdown(results: np.ndarray) -> float:
    if len(results) == 0:
        return 0.0
    equity = np.cumsum(results)
    peak = np.maximum.accumulate(equity)
    return float(np.max(peak - equity))

@staticmethod
def _edge_score(metrics: dict[str, object]) -> float:
    if not metrics:
        return -1e9
    expectancy = float(metrics.get("expectancy_r", 0.0))
    profit_factor = float(metrics.get("profit_factor", 0.0))
    win_rate = float(metrics.get("win_rate", 0.0))
    mfe_mae_ratio = float(metrics.get("mfe_mae_ratio", 0.0))
    drawdown = float(metrics.get("max_drawdown_r", 0.0))
    drawdown_penalty = min(drawdown / 12.0, 1.0)
    return (
        expectancy * 0.45
        + min(profit_factor / 3.0, 1.0) * 0.25
        + min(win_rate / 0.5, 1.0) * 0.10
        + min(mfe_mae_ratio / 4.0, 1.0) * 0.10
        - drawdown_penalty * 0.10
    )

@staticmethod
def _key(rr: float) -> str:
    return f"{float(rr):g}"

```

Full Source: ``backend/tradeo/research/cluster_research_engine.py``

``backend/tradeo/research/cluster_research_engine.py`` ? 1914 lines, 94042 bytes, sha256 prefix ``04f330e29a75cf9c``

Audit relevance: Discovery clustering engine. Audit feature construction, KMeans assumptions, null baselines, overfitting controls, regime/outcome metadata, and deterministic seeds.

Public surface summary before full source:

? class ``ClusterResearchEngine`` line 37 methods: discover

Risk hints: RNG path; verify seed/control.

```

from __future__ import annotations

from dataclasses import dataclass
from hashlib import blake2b
from itertools import combinations
import math
from typing import Iterable

import numpy as np
from sklearn.cluster import MiniBatchKMeans
from sklearn.preprocessing import StandardScaler

import pandas as pd

from tradeo.research.adversarial_research import AdversarialResearchEngine
from tradeo.research.causal_invariance import CausalInvariantTester
from tradeo.research.foundation_teacher import FoundationChartTeacher
from tradeo.research.market_replay import MarketReplayEngine
from tradeo.research.quant_validation import (
    average_uniqueness_weights,
    newey_west_tstat,
    profit_factor as weighted_profit_factor,
    sample_skew_kurt,
    select_nonoverlapping_events,
    stationary_bootstrap_ci,
)
from tradeo.research.pattern_embedding_engine import PatternEmbeddingEngine
from tradeo.research.reward_risk_analyzer import RewardRiskAnalyzer
from tradeo.research.types import ClusterCandidate, Side, WindowSample
from tradeo.services.market_regime import (
    INSUFFICIENT_HISTORY,
    regime_keys_for_dates,
)

@dataclass(slots=True)
class ClusterResearchEngine:
    """Cluster unlabeled chart windows, then measure what happened next."""

    min_cluster_size: int = 60
    max_clusters_per_window: int = 12
    random_state: int = 42
    target_r: float = 4.0
    out_of_sample_pct: float = 0.25
    rr_levels: list[float] | None = None
    min_samples: int = 100
    event_ledger_limit: int = 250
    walk_forward_folds: int = 4
    walk_forward_embargo_samples: int = 5
    cost_stress_multipliers: list[float] | None = None
    required_cost_stress_multiplier: float = 2.0
    match_tau_percentile: float = 92.5
    quant_bootstrap_draws: int = 500
    benchmark_regime_table: pd.DataFrame | None = None

    def discover(self, samples: list[WindowSample]) -> list[ClusterCandidate]:
        candidates: list[ClusterCandidate] = []
        for window_size in sorted({sample.window_size for sample in samples}):
            window_samples = [sample for sample in samples if sample.window_size == window_size]
            candidates.extend(self._cluster_window_size(window_size, window_samples))
        self._apply_novelty_diversity(candidates)
        return sorted(candidates, key=lambda c: c.score, reverse=True)

    def _cluster_window_size(self, window_size: int, samples: list[WindowSample]) -> list[ClusterCandidate]:
        if len(samples) < max(self.min_cluster_size * 2, 20):
            return []
        ordered_samples = sorted(samples, key=lambda s: s.end)
        train_samples, holdout_samples = self._train_holdout_samples(ordered_samples)
        if len(train_samples) < max(self.min_cluster_size * 2, 20):
            return []
        matrix_train = np.vstack([s.vector for s in train_samples])
        matrix_all = np.vstack([s.vector for s in ordered_samples])
        scaler = StandardScaler()
        matrix_train_scaled = scaler.fit_transform(matrix_train)
        matrix_all_scaled = scaler.transform(matrix_all)
        n_clusters = min(self.max_clusters_per_window, max(2, len(train_samples) // self.min_cluster_size))
        model = MiniBatchKMeans(
            n_clusters=n_clusters,
            random_state=self.random_state,
            n_init=10,
            batch_size=min(2048, max(128, len(train_samples))),
        )
        train_labels = model.fit_predict(matrix_train_scaled)
        all_labels = model.predict(matrix_all_scaled)
        candidates: list[ClusterCandidate] = []
        embedding_contract = PatternEmbeddingEngine().contract()
        holdout_ids = {id(sample) for sample in holdout_samples}
        for cluster_id in sorted(set(train_labels.tolist())):
            train_idxs = np.flatnonzero(train_labels == cluster_id)
            if len(train_idxs) < max(10, self.min_cluster_size // 2):
                continue
            all_idxs = np.flatnonzero(all_labels == cluster_id)
            cluster_samples = [ordered_samples[int(i)] for i in all_idxs]
            cluster_train_samples = [train_samples[int(i)] for i in train_idxs]
            cluster_holdout_samples = [sample for sample in cluster_samples if id(sample) in holdout_ids]

```

```

cluster_vectors = matrix_all_scaled[all_idx]
centroid_scaled = model.cluster_centers_[cluster_id]
rr_trial_count = len(self.rr_levels or [1.5, 2.0, 2.5, 3.0, 4.0, 5.0])
tested_trials = n_clusters * 2 * rr_trial_count
long_metrics = self._metrics_for_side(
    cluster_train_samples,
    cluster_samples,
    cluster_holdout_samples,
    train_samples,
    "long",
    multiple_testing_trials=tested_trials,
)
short_metrics = self._metrics_for_side(
    cluster_train_samples,
    cluster_samples,
    cluster_holdout_samples,
    train_samples,
    "short",
    multiple_testing_trials=tested_trials,
)
side: Side = "long" if self._side_score(long_metrics) >= self._side_score(short_metrics) else "short"
metrics = long_metrics if side == "long" else short_metrics
metrics["opposite_side"] = short_metrics if side == "long" else long_metrics
metrics["cluster_density"] = round(float(len(all_idx) / len(ordered_samples)), 5)
metrics["train_cluster_density"] = round(float(len(train_idx) / len(train_samples)), 5)
metrics["window_size"] = window_size
metrics["side"] = side
metrics["target_r"] = self.target_r
metrics["embedding_length"] = int(matrix_all.shape[1])
metrics["feature_parity_contract"] = {
    **embedding_contract,
    "vector_length": int(matrix_all.shape[1]),
    "research_path": "WindowSampler -> PatternEmbeddingEngine.embed",
    "lab_path": "NoveIPatternMatcher -> PatternEmbeddingEngine.embed",
}
metrics["scaler_mean"] = np.nan_to_num(scaler.mean_, nan=0, posinf=0, neginf=0).round(6).tolist()
metrics["scaler_scale"] = np.nan_to_num(scaler.scale_, nan=1, posinf=1, neginf=1).round(6).tolist()
metrics["validation_method"] = "train_fit_forward_holdout_walk_forward_embargo"
metrics["selection_split"] = {
    "method": metrics["validation_method"],
    "train_start": train_samples[0].end if train_samples else None,
    "train_end": train_samples[-1].end if train_samples else None,
    "holdout_start": holdout_samples[0].end if holdout_samples else None,
    "holdout_end": holdout_samples[-1].end if holdout_samples else None,
    "train_sample_count": len(train_samples),
    "holdout_sample_count": len(holdout_samples),
    "out_of_sample_pct": self.out_of_sample_pct,
}
metrics["fit_scope"] = {
    "scaler": "train_only",
    "cluster_model": "train_only",
    "rr_selection": "train_only",
    "side_selection": "train_metrics_only",
    "null_baseline": "train_population_stratified_bootstrap",
    "lab_priority_score": "train_oos_walk_forward_only",
    "promotion_score": "train_oos_walk_forward_only",
    "descriptive_all_feeds_scores": False,
}
metrics["train_cutoff"] = train_samples[-1].end if train_samples else None
metrics["holdout_start"] = holdout_samples[0].end if holdout_samples else None
metrics["holdout_end"] = holdout_samples[-1].end if holdout_samples else None
metrics["model_fit_sample_count"] = len(train_samples)
metrics["model_holdout_sample_count"] = len(holdout_samples)
metrics["real_variant_count"] = tested_trials
metrics["real_variant_dimensions"] = {
    "clusters": n_clusters,
    "sides": 2,
    "rr_levels": rr_trial_count,
}
walk_forward = self._walk_forward_metrics(cluster_samples, side)
metrics["walk_forward_folds"] = walk_forward["folds"]
metrics["walk_forward_fold_count"] = walk_forward["fold_count"]
metrics["walk_forward_positive_fold_rate"] = walk_forward["positive_fold_rate"]
metrics["walk_forward_avg_expectancy_r"] = walk_forward["avg_expectancy_r"]
metrics["walk_forward_min_expectancy_r"] = walk_forward["min_expectancy_r"]
metrics["walk_forward_pooled"] = walk_forward["pooled"]
metrics["walk_forward_embargo_samples"] = self.walk_forward_embargo_samples
metrics["walk_forward_metrics"] = {
    "folds": walk_forward["folds"],
    "fold_count": walk_forward["fold_count"],
    "positive_fold_rate": walk_forward["positive_fold_rate"],
    "avg_expectancy_r": walk_forward["avg_expectancy_r"],
    "min_expectancy_r": walk_forward["min_expectancy_r"],
    "pooled": walk_forward["pooled"],
    "embargo_samples": self.walk_forward_embargo_samples,
}
purged_cv = self._purged_combinatorial_cv(cluster_samples, side)
metrics["purged_combinatorial_cv"] = purged_cv["folds"]
metrics["purged_cv_fold_count"] = purged_cv["fold_count"]
metrics["purged_cv_positive_rate"] = purged_cv["positive_rate"]
metrics["purged_cv_avg_expectancy_r"] = purged_cv["avg_expectancy_r"]
metrics["purged_cv_min_expectancy_r"] = purged_cv["min_expectancy_r"]
metrics["purged_cv_p10_expectancy_r"] = purged_cv["p10_expectancy_r"]
metrics["purged_cv_embargo_samples"] = self.walk_forward_embargo_samples
metrics["overfit_score"] = self._overfit_score(metrics)
metrics["cost_stress"] = self._cost_stress_metrics(

```

```

        cluster_samples,
        side,
        rr=float(metrics.get("best_rr", self.target_rr)),
    )
    metrics["cost_stress_passed"] = self._cost_stress_passed(
        metrics["cost_stress"],
        required_multiplier=self.required_cost_stress_multiplier,
    )
    best_rr = float(metrics.get("best_rr", self.target_rr))
    quant = self._quant_validation_metrics(cluster_train_samples, side=side, rr=best_rr)
    metrics["quant_validation"] = quant
    metrics["effective_sample_count"] = quant.get("n_eff", 0.0)
    metrics["unique_event_count"] = quant.get("n_unique", 0)
    metrics["match_tau_percentile"] = round(float(self.match_tau_percentile), 3)
    metrics["match_tau_similarity"] = self._match_tau_similarity(
        cluster_vectors, centroid_scaled
    )
    signature = self._cluster_signature(cluster_samples, cluster_vectors, centroid_scaled, side)
    metrics["cluster_signature"] = signature
    metrics["medoid"] = signature["medoid"]
    metrics["concentration_checks"] = signature["concentration_checks"]
    metrics["cluster_stability"] = {
        "method": "outcome_diversity_concentration_proxy",
        "stability_score": metrics["stability_score"],
        "stability_year": metrics["stability_year"],
        "stability_symbol": metrics["stability_symbol"],
        "concentration_passed": signature["concentration_checks"]["passed"],
    }
    metrics["foundation_teacher"] = FoundationChartTeacher().analyze(
        cluster_samples,
        side=side,
        rr=best_rr,
        centroid=centroid_scaled,
        baseline_samples=train_samples,
    )
    metrics["market_replay"] = MarketReplayEngine().analyze(
        cluster_samples,
        side,
        best_rr,
    )
    metrics["causal_invariance"] = CausalInvariantTester().analyze(
        cluster_samples,
        side,
        best_rr,
    )
    metrics["adversarial_challenge"] = AdversarialResearchEngine().analyze(
        cluster_samples,
        baseline_samples=train_samples,
        side=side,
        rr=best_rr,
        metrics=metrics,
        causal_invariance=metrics["causal_invariance"],
        market_replay=metrics["market_replay"],
    )
    metrics["operational_trigger"] = self._operational_trigger_metrics(cluster_samples, side)
    metrics["event_ledger"] = self._event_ledger(
        cluster_samples,
        train_samples=cluster_train_samples,
        holdout_samples=cluster_holdout_samples,
        side=side,
        rr=best_rr,
    )
    metrics["event_ledger_count"] = len(metrics["event_ledger"])
    feature_summary = self._feature_summary(cluster_samples)
    regime_profile = self._regime_profile(cluster_samples)
    regime_profile["benchmark_regime_outcomes"] = self._benchmark_regime_outcomes(
        cluster_samples,
        side=side,
        rr=best_rr,
    )
    metrics["regime_profile"] = regime_profile
    metrics["human_rule"] = self._human_rule(
        cluster_samples,
        baseline_samples=train_samples,
        side=side,
        metrics=metrics,
    )
    score = self._candidate_score(metrics)
    metrics["lab_priority_score"] = round(float(score), 5)
    metrics["promotion_score"] = self._promotion_score(metrics)
    metrics["score_input_scope"] = {
        "lab_priority_score": [
            "train_metrics",
            "out_of_sample_metrics",
            "walk_forward_metrics",
            "purged_combinatorial_cv",
            "bootstrap_reality_proxy",
            "market_replay",
            "adversarial_challenge",
            "causal_invariance",
            "foundation_teacher",
        ],
        "descriptive_all_fields_used": False,
    }
    metrics["nested_discovery_replay"] = {
        "status": "required_for_finalists",
    }

```

```

        "implemented": False,
        "blocking_reason": (
            "Full nested replay must refit scaler, clustering, side selection "
            "and R:R selection inside each fold before any edge claim upgrade."
        ),
        "applies_before": ["production_gate", "edge_claim_upgrade"],
    }
    centroid = np.nan_to_num(centroid_scaled, nan=0, posinf=0, neginf=0).round(6).tolist()
    key = self._pattern_key(window_size, cluster_id, side, centroid)
    name = f"DISCOVERED_{side.upper()}_W{window_size}_C{cluster_id:02d}_{key[-8:].upper()}"
    examples = self._examples(cluster_samples, cluster_vectors, centroid_scaled, side)
    candidates.append(
        ClusterCandidate(
            pattern_key=key,
            name=name,
            side=side,
            timeframe=cluster_samples[0].timeframe if cluster_samples else "1d",
            window_size=window_size,
            cluster_id=int(cluster_id),
            centroid=centroid,
            sample_count=len(cluster_samples),
            symbol_count=len({s.symbol for s in cluster_samples}),
            year_count=len({s.year for s in cluster_samples}),
            score=round(score, 5),
            validation_passed=False,
            validation_reasons=[],
            metrics=metrics,
            feature_summary=feature_summary,
            examples=examples,
        )
    )
    return candidates

def _metrics_for_side(
    self,
    train_samples: list[WindowSample],
    all_samples: list[WindowSample],
    holdout_samples: list[WindowSample],
    baseline_samples: list[WindowSample],
    side: Side,
    multiple_testing_trials: int,
) -> dict[str, object]:
    rr_levels = self.rr_levels or [1.5, 2.0, 2.5, 3.0, 4.0, 5.0]
    rr_analysis = RewardRiskAnalyzer(
        rr_levels=rr_levels,
        min_samples=self.min_samples,
    ).analyze(train_samples, side)
    best_rr = float(rr_analysis.get("best_rr", self.target_r))
    rr_metrics = rr_analysis.get("rr_metrics", {})
    best_rr_metrics = rr_metrics.get(f"{best_rr:g}", {}) if isinstance(rr_metrics, dict) else {}
    outcomes = np.asarray(
        [RewardRiskAnalyzer._simulate_sample(s, side, best_rr)[0] for s in all_samples],
        dtype=float,
    )
    mfe = np.asarray([s.outcome.mfe_for(side) for s in all_samples], dtype=float)
    mae = np.asarray([s.outcome.mae_for(side) for s in all_samples], dtype=float)
    hits = np.asarray(
        [RewardRiskAnalyzer._simulate_sample(s, side, 4.0)[0] >= 4.0 for s in all_samples],
        dtype=bool,
    )
    train_outcomes = np.asarray(
        [RewardRiskAnalyzer._simulate_sample(s, side, best_rr)[0] for s in train_samples],
        dtype=float,
    )
    train_mfe = np.asarray([s.outcome.mfe_for(side) for s in train_samples], dtype=float)
    train_mae = np.asarray([s.outcome.mae_for(side) for s in train_samples], dtype=float)
    train_hits = np.asarray(
        [RewardRiskAnalyzer._simulate_sample(s, side, 4.0)[0] >= 4.0 for s in train_samples],
        dtype=bool,
    )
    wins = outcomes[outcomes > 0]
    losses = outcomes[outcomes < 0]
    profit_factor = float(wins.sum() / abs(losses.sum())) if len(losses) else float(wins.sum() or 0.0)
    by_year = self._group_expectancy(all_samples, outcomes, key="year")
    by_symbol = self._group_expectancy(all_samples, outcomes, key="symbol")
    train_by_year = self._group_expectancy(train_samples, train_outcomes, key="year")
    train_by_symbol = self._group_expectancy(train_samples, train_outcomes, key="symbol")
    oos = self._split_metrics(holdout_samples, side, best_rr)
    in_sample_metrics = self._split_metrics(train_samples, side, best_rr)
    null_baseline = self._null_baseline(
        baseline_samples,
        cluster_samples=train_samples,
        cluster_size=len(train_samples),
        side=side,
        rr=best_rr,
        observed_expectancy=float(in_sample_metrics["expectancy_r"]),
        multiple_testing_trials=multiple_testing_trials,
    )
    bootstrap = self._bootstrap_confidence(train_samples, side, best_rr)
    sharpe = self._sharpe_diagnostics(train_outcomes, multiple_testing_trials=multiple_testing_trials)
    edge_decay = self._edge_decay_metrics(rr_metrics, best_rr)
    stability_year = self._positive_group_fraction(train_by_year)
    stability_symbol = self._positive_group_fraction(train_by_symbol)
    diversity_score = min(1.0, len(train_by_symbol) / 20.0) * 0.55 + min(1.0, len(train_by_year) / 4.0) * 0.45
    stability_score = 0.40 * stability_year + 0.35 * stability_symbol + 0.25 * diversity_score
    avg_mae = float(np.mean(mae)) if len(mae) else 0.0

```



```

avg_mfe = float(np.mean(mfe)) if len(mfe) else 0.0
train_avg_mae = float(np.mean(train_mae)) if len(train_mae) else 0.0
train_avg_mfe = float(np.mean(train_mfe)) if len(train_mfe) else 0.0
return {
    "sample_count": len(all_samples),
    "train_sample_count": len(train_samples),
    "holdout_sample_count": len(holdout_samples),
    "symbol_count": len({s.symbol for s in all_samples}),
    "year_count": len({s.year for s in all_samples}),
    "descriptive_all_sample_count": len(all_samples),
    "descriptive_all_expectancy_r": round(float(np.mean(outcomes)), 5) if len(outcomes) else 0.0,
    "descriptive_all_median_r": round(float(np.median(outcomes)), 5) if len(outcomes) else 0.0,
    "descriptive_all_win_rate": round(float(np.mean(outcomes > 0)), 5) if len(outcomes) else 0.0,
    "descriptive_all_hit_4r_rate": round(float(np.mean(hits)), 5) if len(hits) else 0.0,
    "descriptive_all_profit_factor": round(float(profit_factor), 5),
    "descriptive_all_avg_mfe_r": round(avg_mfe, 5),
    "descriptive_all_avg_mae_r": round(avg_mae, 5),
    "expectancy_r": in_sample_metrics["expectancy_r"],
    "median_r": round(float(np.median(train_outcomes)), 5) if len(train_outcomes) else 0.0,
    "win_rate": in_sample_metrics["win_rate"],
    "hit_4r_rate": round(float(np.mean(train_hits)), 5) if len(train_hits) else 0.0,
    "profit_factor": in_sample_metrics["profit_factor"],
    "avg_mfe_r": round(train_avg_mfe, 5),
    "avg_mae_r": round(train_avg_mae, 5),
    "avg_execution_cost_r": float(best_rr_metrics.get("avg_execution_cost_r", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avgBars_to_target": float(best_rr_metrics.get("avgBars_to_target", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avgBars_to_stop": float(best_rr_metrics.get("avgBars_to_stop", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "triple_barrier_labels": best_rr_metrics.get("triple_barrier_labels", {})
    if isinstance(best_rr_metrics, dict)
    else {},
    "avg_gap_adverse_r": float(best_rr_metrics.get("avg_gap_adverse_r", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "mfe_before_mae_rate": float(best_rr_metrics.get("mfe_before_mae_rate", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "strong_close_without_target_rate": float(best_rr_metrics.get("strong_close_without_target_rate", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "speed_label_counts": best_rr_metrics.get("speed_label_counts", {})
    if isinstance(best_rr_metrics, dict)
    else {},
    "fast_target_rate": float(best_rr_metrics.get("fast_target_rate", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "slow_target_rate": float(best_rr_metrics.get("slow_target_rate", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "timeout_rate": float(best_rr_metrics.get("timeout_rate", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avg_fill_probability": float(best_rr_metrics.get("avg_fill_probability", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "p25_max_size_usd": float(best_rr_metrics.get("p25_max_size_usd", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "median_max_size_usd": float(best_rr_metrics.get("median_max_size_usd", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avg_spread_proxy_pct": float(best_rr_metrics.get("avg_spread_proxy_pct", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avg_slippage_proxy_pct": float(best_rr_metrics.get("avg_slippage_proxy_pct", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avg_entry_gap_penalty_pct": float(best_rr_metrics.get("avg_entry_gap_penalty_pct", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "avg_short_borrow_proxy_pct": float(best_rr_metrics.get("avg_short_borrow_proxy_pct", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "descriptive_all_mfe_mae_ratio": round(float(avg_mfe / max(avg_mae, 1e-9)), 5),
    "descriptive_all_reward_risk_estimate": round(
        float(np.quantile(mfe, 0.60) / max(np.quantile(mae, 0.60), 1e-9)),
        5,
    )
    if len(mfe) and len(mae)
    else 0.0,
    "mfe_mae_ratio": round(float(train_avg_mfe / max(train_avg_mae, 1e-9)), 5),
    "reward_risk_estimate": round(
        float(np.quantile(train_mfe, 0.60) / max(np.quantile(train_mae, 0.60), 1e-9)),
        5,
    )
    if len(train_mfe) and len(train_mae)
    else 0.0,
    "rr_levels": rr_levels,
    "rr_metrics": rr_metrics,
    "train_rr_metrics": rr_metrics,
    "rr_metrics_json": rr_metrics,

```

```

"best_rr": round(float(best_rr), 5),
"best_tested_rr": round(float(rr_analysis.get("best_tested_rr", best_rr)), 5),
"best_expectancy_r": round(float(rr_analysis.get("best_expectancy_r", 0.0)), 5),
"best_profit_factor": round(float(rr_analysis.get("best_profit_factor", 0.0)), 5),
"best_win_rate": round(float(rr_analysis.get("best_win_rate", 0.0)), 5),
"best_max_drawdown_r": round(float(rr_analysis.get("best_max_drawdown_r", 0.0)), 5),
"best_edge_score": round(float(rr_analysis.get("best_score", 0.0)), 5),
"target_hit_rate": float(best_rr_metrics.get("target_hit_rate", 0.0))
if isinstance(best_rr_metrics, dict)
else 0.0,
"stop_hit_rate": float(best_rr_metrics.get("stop_hit_rate", 0.0))
if isinstance(best_rr_metrics, dict)
else 0.0,
"max_drawdown_r": float(best_rr_metrics.get("max_drawdown_r", 0.0))
if isinstance(best_rr_metrics, dict)
else 0.0,
"in_sample_expectancy_r": in_sample_metrics["expectancy_r"],
"in_sample_profit_factor": in_sample_metrics["profit_factor"],
"train_metrics": {
    "sample_count": in_sample_metrics["sample_count"],
    "expectancy_r": in_sample_metrics["expectancy_r"],
    "profit_factor": in_sample_metrics["profit_factor"],
    "win_rate": in_sample_metrics["win_rate"],
    "max_drawdown_r": in_sample_metrics["max_drawdown_r"],
    "best_rr": round(float(best_rr), 5),
    "best_expectancy_r": round(float(rr_analysis.get("best_expectancy_r", 0.0)), 5),
    "best_profit_factor": round(float(rr_analysis.get("best_profit_factor", 0.0)), 5),
    "best_win_rate": round(float(rr_analysis.get("best_win_rate", 0.0)), 5),
    "target_hit_rate": float(best_rr_metrics.get("target_hit_rate", 0.0))
    if isinstance(best_rr_metrics, dict)
    else 0.0,
    "stability_year": round(stability_year, 5),
    "stability_symbol": round(stability_symbol, 5),
    "stability_score": round(float(stability_score), 5),
},
>null_expectancy_r": null_baseline["expectancy_r"],
>null_profit_factor": null_baseline["profit_factor"],
>null_win_rate": null_baseline["win_rate"],
"expectancy_lift_r": null_baseline["expectancy_lift_r"],
>null_p_value": null_baseline["p_value"],
"adjusted_p_value": null_baseline["adjusted_p_value"],
"wrc_p_value": null_baseline["wrc_p_value"],
"spa_p_value": null_baseline["spa_p_value"],
"wrc_passed": null_baseline["wrc_passed"],
"spa_passed": null_baseline["spa_passed"],
"multiple_testing_penalty": null_baseline["multiple_testing_penalty"],
"reality_check_method": null_baseline["reality_check_method"],
"reality_check_formal_test": null_baseline["reality_check_formal_test"],
"bootstrap_reality_proxy": {
    "method": null_baseline["reality_check_method"],
    "formal_test": null_baseline["reality_check_formal_test"],
    "wrc_like_p_value": null_baseline["wrc_p_value"],
    "spa_like_p_value": null_baseline["spa_p_value"],
    "wrc_like_passed": null_baseline["wrc_passed"],
    "spa_like_passed": null_baseline["spa_passed"],
    "draws": bootstrap["draws"],
    "trial_count": null_baseline["multiple_testing_trials"],
},
"multiple_testing_trials": null_baseline["multiple_testing_trials"],
"statistical_edge_passed": null_baseline["statistical_edge_passed"],
>null_method": null_baseline["method"],
>null_strata_count": null_baseline["strata_count"],
"expectancy_ci_low": bootstrap["expectancy_ci_low"],
"expectancy_ci_high": bootstrap["expectancy_ci_high"],
"profit_factor_ci_low": bootstrap["profit_factor_ci_low"],
"bootstrap_draws": bootstrap["draws"],
**Sharpe,
**edge_decay,
"out_of_sample_expectancy_r": oos["expectancy_r"],
"out_of_sample_profit_factor": oos["profit_factor"],
"out_of_sample_win_rate": oos["win_rate"],
"out_of_sample_max_drawdown_r": oos["max_drawdown_r"],
"out_of_sample_sample_count": oos["sample_count"],
"out_of_sample_metrics": {
    "sample_count": oos["sample_count"],
    "expectancy_r": oos["expectancy_r"],
    "profit_factor": oos["profit_factor"],
    "win_rate": oos["win_rate"],
    "max_drawdown_r": oos["max_drawdown_r"],
    "fit_scope": "holdout_only_no_refit",
},
"stability_year": round(stability_year, 5),
"stability_symbol": round(stability_symbol, 5),
"stability_score": round(float(stability_score), 5),
"by_year_expectancy": train_by_year,
"top_symbols_expectancy": dict(list(train_by_symbol.items())[:25]),
"descriptive_all_by_year_expectancy": by_year,
"descriptive_all_top_symbols_expectancy": dict(list(by_symbol.items())[:25]),
"descriptive_metric_policy": {
    "prefix": "descriptive_all_",
    "feeds_lab_priority_score": False,
    "feeds_promotion_score": False,
    "feeds_best_pattern_selection": False,
},
},
}

```

```

def _train_holdout_samples(self, samples: list[WindowSample]) -> tuple[list[WindowSample], list[WindowSample]]:
    if len(samples) < 8:
        return samples, []
    split = int(len(samples) * (1.0 - self.out_of_sample_pct))
    split = min(max(1, split), len(samples) - 1)
    return samples[:split], samples[split:]

def _null_baseline(
    self,
    samples: list[WindowSample],
    *,
    cluster_samples: list[WindowSample],
    cluster_size: int,
    side: Side,
    rr: float,
    observed_expectancy: float,
    multiple_testing_trials: int,
) -> dict[str, object]:
    outcomes = np.asarray([RewardRiskAnalyzer._simulate_sample(s, side, rr)[0] for s in samples], dtype=float)
    if len(outcomes) == 0 or cluster_size <= 0:
        return {
            "expectancy_r": 0.0,
            "profit_factor": 0.0,
            "win_rate": 0.0,
            "expectancy_lift_r": observed_expectancy,
            "p_value": 1.0,
            "adjusted_p_value": 1.0,
            "wrc_p_value": 1.0,
            "spa_p_value": 1.0,
            "wrc_passed": False,
            "spa_passed": False,
            "multiple_testing_penalty": 1.0,
            "multiple_testing_trials": max(1, int(multiple_testing_trials)),
            "statistical_edge_passed": False,
            "method": "stratified_regime_bootstrap",
            "reality_check_method": "bootstrap_reality_proxy",
            "reality_check_formal_test": False,
            "strata_count": 0,
        }

    baseline = self._outcome_metrics(outcomes)
    draws = min(512, max(96, len(outcomes) * 2))
    seed = self._null_seed(side, rr, cluster_size, len(outcomes))
    rng = np.random.default_rng(seed)
    stratified_groups = self._baseline_groups(samples)
    cluster_strata = self._cluster_strata(cluster_samples)
    draw_size = min(cluster_size, len(outcomes))
    random_means = []
    for _ in range(draws):
        draw = self._stratified_draw(
            rng,
            outcomes=outcomes,
            groups=stratified_groups,
            cluster_strata=cluster_strata,
            fallback_size=draw_size,
        )
        random_means.append(float(np.mean(draw)))
    p_value = (1 + sum(1 for mean in random_means if mean >= observed_expectancy)) / (draws + 1)
    trial_count = max(1, int(multiple_testing_trials))
    adjusted_p = min(1.0, p_value * trial_count)
    wrc_p = min(1.0, 1.0 - (1.0 - p_value) ** trial_count)
    spa_p = min(1.0, p_value * math.sqrt(trial_count))
    multiple_testing_penalty = min(1.0, math.sqrt(math.log1p(trial_count)) / 4.0)
    null_expectancy = float(np.mean(random_means)) if random_means else float(baseline["expectancy_r"])
    expectancy_lift = observed_expectancy - null_expectancy
    return {
        "expectancy_r": round(null_expectancy, 5),
        "profit_factor": baseline["profit_factor"],
        "win_rate": baseline["win_rate"],
        "expectancy_lift_r": round(expectancy_lift, 5),
        "p_value": round(float(p_value), 5),
        "adjusted_p_value": round(float(adjusted_p), 5),
        "wrc_p_value": round(float(wrc_p), 5),
        "spa_p_value": round(float(spa_p), 5),
        "wrc_passed": wrc_p <= 0.25 and expectancy_lift > 0,
        "spa_passed": spa_p <= 0.25 and expectancy_lift > 0,
        "multiple_testing_penalty": round(float(multiple_testing_penalty), 5),
        "multiple_testing_trials": trial_count,
        "statistical_edge_passed": adjusted_p <= 0.25 and expectancy_lift > 0,
        "method": "stratified_regime_bootstrap",
        "reality_check_method": "bootstrap_reality_proxy",
        "reality_check_formal_test": False,
        "strata_count": len(cluster_strata),
    }

def _baseline_groups(self, samples: list[WindowSample]) -> dict[tuple[object, ...], list[int]]:
    groups: dict[tuple[object, ...], list[int]] = {}
    for idx, sample in enumerate(samples):
        groups.setdefault(self._stratum(sample), []).append(idx)
    return groups

def _cluster_strata(self, samples: list[WindowSample]) -> dict[tuple[object, ...], int]:
    strata: dict[tuple[object, ...], int] = {}
    for sample in samples:
        key = self._stratum(sample)
        strata[key] = strata.get(key, 0) + 1

```

```

        return strata

def _stratified_draw(
    self,
    rng: np.random.Generator,
    *,
    outcomes: np.ndarray,
    groups: dict[tuple[object, ...], list[int]],
    cluster_strata: dict[tuple[object, ...], int],
    fallback_size: int,
) -> np.ndarray:
    selected: list[float] = []
    all_indices = np.arange(len(outcomes))
    for stratum, count in cluster_strata.items():
        group = groups.get(stratum)
        if not group:
            continue
        size = min(count, len(group))
        idxs = rng.choice(np.asarray(group), size=size, replace=False)
        selected.extend(float(outcomes[int(idx)]) for idx in idxs)
    missing = max(0, fallback_size - len(selected))
    if missing:
        idxs = rng.choice(all_indices, size=min(missing, len(outcomes)), replace=False)
        selected.extend(float(outcomes[int(idx)]) for idx in idxs)
    if not selected:
        idxs = rng.choice(all_indices, size=min(fallback_size, len(outcomes)), replace=False)
        selected.extend(float(outcomes[int(idx)]) for idx in idxs)
    return np.asarray(selected, dtype=float)

@staticmethod
def _stratum(sample: WindowSample) -> tuple[object, ...]:
    volatility = float(sample.features.get("volatility_regime", 1.0))
    trend = float(sample.features.get("trend_regime", 0.0))
    market = float(sample.features.get("market_regime_score", 0.0))
    breadth = float(sample.features.get("market_breadth_proxy", 0.5))
    sector = float(sample.features.get("sector_strength", 0.0))
    liquidity = float(sample.features.get("liquidity_score", 0.0))
    if volatility < 0.8:
        vol_bucket = "low_vol"
    elif volatility > 1.25:
        vol_bucket = "high_vol"
    else:
        vol_bucket = "normal_vol"
    if trend > 0.25:
        trend_bucket = "up"
    elif trend < -0.25:
        trend_bucket = "down"
    else:
        trend_bucket = "mixed"
    if market > 0.25:
        market_bucket = "market_up"
    elif market < -0.25:
        market_bucket = "market_down"
    else:
        market_bucket = "market_mixed"
    if breadth >= 0.67:
        breadth_bucket = "broad"
    elif breadth <= 0.33:
        breadth_bucket = "narrow"
    else:
        breadth_bucket = "neutral_breadth"
    if sector > 0.03:
        sector_bucket = "sector_strong"
    elif sector < -0.03:
        sector_bucket = "sector_weak"
    else:
        sector_bucket = "sector_neutral"
    liquidity_bucket = "liquid" if liquidity >= 0.55 else "thin"
    return (
        sample.year,
        vol_bucket,
        trend_bucket,
        market_bucket,
        breadth_bucket,
        sector_bucket,
        liquidity_bucket,
    )

def _bootstrap_confidence(self, samples: list[WindowSample], side: Side, rr: float) -> dict[str, float | int]:
    outcomes = np.asarray(
        [RewardRiskAnalyzer._simulate_sample(sample, side, rr)[0] for sample in samples],
        dtype=float,
    )
    if len(outcomes) == 0:
        return {"expectancy_ci_low": 0.0, "expectancy_ci_high": 0.0, "profit_factor_ci_low": 0.0, "draws": 0}
    draws = min(512, max(128, len(outcomes) * 3))
    rng = np.random.default_rng(self._null_seed(side, rr, len(samples), len(outcomes)) ^ 0xA5A5A5A5)
    expectancies: list[float] = []
    profit_factors: list[float] = []
    for _ in range(draws):
        idxs = rng.choice(len(outcomes), size=len(outcomes), replace=True)
        metrics = self._outcome_metrics(outcomes[idxs])
        expectancies.append(float(metrics["expectancy_r"]))
        profit_factors.append(float(metrics["profit_factor"]))
    return {
        "expectancy_ci_low": round(float(np.quantile(expectancies, 0.05)), 5),

```

```

        "expectancy_ci_high": round(float(np.quantile(expectancies, 0.95)), 5),
        "profit_factor_ci_low": round(float(np.quantile(profit_factors, 0.05)), 5),
        "draws": draws,
    }

    @staticmethod
    def _sharpe_diagnostics(outcomes: np.ndarray, *, multiple_testing_trials: int) -> dict[str, float]:
        outcomes = np.asarray(outcomes, dtype=float)
        if len(outcomes) < 3:
            return {
                "trade_sharpe": 0.0,
                "probabilistic_sharpe": 0.0,
                "deflated_sharpe": 0.0,
                "deflated_sharpe_probability": 0.0,
            }
        mean = float(np.mean(outcomes))
        std = float(np.std(outcomes, ddof=1))
        if std <= 1e-12:
            sharpe = 0.0 if mean <= 0 else 10.0
        else:
            sharpe = mean / std
        centered = outcomes - mean
        skew = float(np.mean(centered**3) / max(float(np.std(outcomes) ** 3), 1e-12))
        kurt = float(np.mean(centered**4) / max(float(np.std(outcomes) ** 4), 1e-12))
        denominator = math.sqrt(max(1e-9, 1.0 - skew * sharpe + ((kurt - 1.0) / 4.0) * sharpe * sharpe))
        psr_z = sharpe * math.sqrt(max(len(outcomes) - 1, 1)) / denominator
        trial_haircut = math.sqrt(2.0 * math.log(max(2, int(multiple_testing_trials)))) / math.sqrt(len(outcomes))
        deflated = sharpe - trial_haircut
        deflated_z = deflated * math.sqrt(max(len(outcomes) - 1, 1)) / denominator
        return {
            "trade_sharpe": round(float(sharpe), 5),
            "probabilistic_sharpe": round(ClusterResearchEngine._normal_cdf(psr_z), 5),
            "deflated_sharpe": round(float(deflated), 5),
            "deflated_sharpe_probability": round(ClusterResearchEngine._normal_cdf(deflated_z), 5),
        }

    @staticmethod
    def _edge_decay_metrics(rr_metrics: object, best_rr: float) -> dict[str, float | bool | list[dict[str, float]]]:
        if not isinstance(rr_metrics, dict) or not rr_metrics:
            return {
                "edge_decay_parameter_score": 1.0,
                "edge_decay_passed": False,
                "edge_decay_neighbors": [],
            }
        levels: list[tuple[float, float]] = []
        for key, metrics in rr_metrics.items():
            if not isinstance(metrics, dict):
                continue
            try:
                level = float(key)
            except (TypeError, ValueError):
                continue
            levels.append((level, float(metrics.get("expectancy_r", 0.0))))
        levels = sorted(levels)
        if not levels:
            return {
                "edge_decay_parameter_score": 1.0,
                "edge_decay_passed": False,
                "edge_decay_neighbors": [],
            }
        best_expectancy = next(
            (expectancy for level, expectancy in levels if abs(level - best_rr) < 1e-9),
            levels[0][1],
        )
        neighbors = sorted(
            [(level, expectancy) for level, expectancy in levels if abs(level - best_rr) >= 1e-9],
            key=lambda item: abs(item[0] - best_rr),
        )[:2]
        decays = [
            max(0.0, best_expectancy - expectancy) / max(abs(best_expectancy), 0.25)
            for _, expectancy in neighbors
        ]
        score = float(np.mean(decays)) if decays else 0.0
        return {
            "edge_decay_parameter_score": round(float(min(score, 1.0)), 5),
            "edge_decay_passed": bool(score <= 0.55 or best_expectancy <= 0.0),
            "edge_decay_neighbors": [
                {
                    "rr": round(float(level), 5),
                    "expectancy_r": round(float(expectancy), 5),
                    "decay_from_best": round(
                        float(max(0.0, best_expectancy - expectancy) / max(abs(best_expectancy), 0.25)),
                        5,
                    ),
                },
                {
                    "rr": round(float(level), 5),
                    "expectancy_r": round(float(expectancy), 5),
                    "decay_from_best": round(
                        float(max(0.0, best_expectancy - expectancy) / max(abs(best_expectancy), 0.25)),
                        5,
                    ),
                },
            ],
        }

    @staticmethod
    def _normal_cdf(value: float) -> float:
        return float(0.5 * (1.0 + math.erf(value / math.sqrt(2.0))))

    @staticmethod
    def _overfit_score(metrics: dict[str, object]) -> float:
        fold_count = int(metrics.get("walk_forward_fold_count", 0))

```

```

if fold_count == 0:
    base = 0.5
else:
    train_expectancy = float(metrics.get("in_sample_expectancy_r", metrics.get("expectancy_r", 0.0)))
    walk_forward_expectancy = float(metrics.get("walk_forward_avg_expectancy_r", 0.0))
    positive_rate = float(metrics.get("walk_forward_positive_fold_rate", 0.0))
    generalization_gap = max(0.0, train_expectancy - walk_forward_expectancy) / max(abs(train_expectancy), 0.25)
    base = (1.0 - positive_rate) * 0.55 + min(generalization_gap, 1.0) * 0.45
    purged_count = int(metrics.get("purged_cv_fold_count", 0))
    purged_penalty = 0.0
    if purged_count:
        purged_penalty = max(0.0, 0.50 - float(metrics.get("purged_cv_positive_rate", 0.0)))
    sharpe_penalty = max(0.0, 0.55 - float(metrics.get("deflated_sharpe_probability", 0.55))) * 0.5
    decay_penalty = float(metrics.get("edge_decay_parameter_score", 0.0)) * 0.25
    score = base * 0.70 + purged_penalty * 0.15 + sharpe_penalty * 0.10 + decay_penalty * 0.05
    return round(float(min(max(score, 0.0), 1.0)), 5)

def _null_seed(self, side: Side, rr: float, cluster_size: int, population_size: int) -> int:
    digest = blake2b(digest_size=4)
    digest.update(
        f"{self.random_state}|{side}|{rr:g}|cluster={cluster_size}|population={population_size}".encode()
    )
    return int.from_bytes(digest.digest(), "big", signed=False)

@staticmethod
def _outcome_metrics(outcomes: np.ndarray) -> dict[str, float]:
    if len(outcomes) == 0:
        return {"expectancy_r": 0.0, "profit_factor": 0.0, "win_rate": 0.0}
    wins = outcomes[outcomes > 0]
    losses = outcomes[outcomes < 0]
    profit_factor = float(wins.sum() / abs(losses.sum())) if len(losses) else float(wins.sum() or 0.0)
    return {
        "expectancy_r": round(float(np.mean(outcomes)), 5),
        "profit_factor": round(profit_factor, 5),
        "win_rate": round(float(np.mean(outcomes > 0)), 5),
    }

def _walk_forward_metrics(self, samples: list[WindowSample], side: Side) -> dict[str, object]:
    ordered = sorted(samples, key=lambda s: s.end)
    if len(ordered) < 12 or self.walk_forward_folds <= 0:
        return self._empty_walk_forward()

    embargo = max(0, int(self.walk_forward_embargo_samples))
    min_train = max(6, min(len(ordered) // 2, self.min_samples))
    if min_train + embargo + 3 >= len(ordered):
        min_train = max(6, len(ordered) // 2)
    remaining = len(ordered) - min_train - embargo
    if remaining < 3:
        return self._empty_walk_forward()

    fold_count = min(max(1, self.walk_forward_folds), remaining)
    validation_size = max(3, remaining // fold_count)
    folds: list[dict[str, object]] = []
    validation_samples_all: list[WindowSample] = []
    for fold_idx in range(fold_count):
        train_end = min_train + fold_idx * validation_size
        validation_start = train_end + embargo
        validation_end = min(len(ordered), validation_start + validation_size)
        train = ordered[:train_end]
        validation = ordered[validation_start:validation_end]
        if len(train) < 6 or len(validation) < 3:
            continue
        rr_analysis = RewardRiskAnalyzer(
            rr_levels=self.rr_levels or [1.5, 2.0, 2.5, 3.0, 4.0, 5.0],
            min_samples=self.min_samples,
        ).analyze(train, side)
        rr = float(rr_analysis.get("best_rr", self.target_rr))
        train_metrics = self._split_metrics(train, side, rr)
        validation_metrics = self._split_metrics(validation, side, rr)
        validation_samples_all.extend(validation)
        folds.append(
            {
                "fold": len(folds) + 1,
                "train_start": train[0].end,
                "train_end": train[-1].end,
                "validation_start": validation[0].end,
                "validation_end": validation[-1].end,
                "embargo_samples": embargo,
                "train_sample_count": len(train),
                "validation_sample_count": len(validation),
                "best_rr": round(rr, 5),
                "train_expectancy_r": train_metrics["expectancy_r"],
                "train_profit_factor": train_metrics["profit_factor"],
                "validation_expectancy_r": validation_metrics["expectancy_r"],
                "validation_profit_factor": validation_metrics["profit_factor"],
                "validation_win_rate": validation_metrics["win_rate"],
                "validation_max_drawdown_r": validation_metrics["max_drawdown_r"],
            }
        )
    )

    if not folds:
        return self._empty_walk_forward()
    validation_expectancies = [float(fold["validation_expectancy_r"]) for fold in folds]
    pooled_rr = float(folds[-1]["best_rr"])
    pooled = self._split_rr(validation_samples_all, side, pooled_rr)
    return {

```

```

        "folds": folds,
        "fold_count": len(folds),
        "positive_fold_rate": round(float(np.mean(np.asarray(validation_expectancies) > 0)), 5),
        "avg_expectancy_r": round(float(np.mean(validation_expectancies)), 5),
        "min_expectancy_r": round(float(np.min(validation_expectancies)), 5),
        "pooled": pooled,
    }

def _purged_combinatorial_cv(self, samples: list[WindowSample], side: Side) -> dict[str, object]:
    ordered = sorted(samples, key=lambda s: s.end)
    fold_target = min(max(3, self.walk_forward_folds), 6)
    if len(ordered) < fold_target * 4:
        return self._empty_purged_cv()
    folds = [list(chunk) for chunk in np.array_split(np.arange(len(ordered)), fold_target) if len(chunk)]
    validation_combos: list[tuple[int, ...]] = [(idx,) for idx in range(len(folds))]
    if len(folds) <= 5:
        validation_combos.extend(combinations(range(len(folds)), 2))
    embargo = max(0, int(self.walk_forward_embargo_samples))
    rr_levels = self.rr_levels or [1.5, 2.0, 2.5, 3.0, 4.0, 5.0]
    cv_folds: list[dict[str, object]] = []
    for combo in validation_combos[:18]:
        validation_indices = sorted(int(i) for fold_idx in combo for i in folds[fold_idx])
        purged = set(validation_indices)
        for idx in validation_indices:
            for purge_idx in range(max(0, idx - embargo), min(len(ordered), idx + embargo + 1)):
                purged.add(purge_idx)
        train = [sample for idx, sample in enumerate(ordered) if idx not in purged]
        validation = [ordered[idx] for idx in validation_indices]
        if len(train) < max(6, self.min_samples // 2) or len(validation) < 3:
            continue
        rr_analysis = RewardRiskAnalyzer(rr_levels=rr_levels, min_samples=self.min_samples).analyze(train, side)
        rr = float(rr_analysis.get("best_rr", self.target_rr))
        train_metrics = self._split_metrics(train, side, rr)
        validation_metrics = self._split_metrics(validation, side, rr)
        cv_folds.append(
            {
                "fold": len(cv_folds) + 1,
                "validation_fold_ids": list(combo),
                "train_sample_count": len(train),
                "validation_sample_count": len(validation),
                "purged_sample_count": len(purged) - len(validation_indices),
                "embargo_samples": embargo,
                "best_rr": round(rr, 5),
                "train_expectancy_r": train_metrics["expectancy_r"],
                "validation_expectancy_r": validation_metrics["expectancy_r"],
                "validation_profit_factor": validation_metrics["profit_factor"],
                "validation_win_rate": validation_metrics["win_rate"],
            }
        )
    if not cv_folds:
        return self._empty_purged_cv()
    expectancies = np.asarray([float(fold["validation_expectancy_r"]) for fold in cv_folds], dtype=float)
    return {
        "folds": cv_folds,
        "fold_count": len(cv_folds),
        "positive_rate": round(float(np.mean(expectancies > 0)), 5),
        "avg_expectancy_r": round(float(np.mean(expectancies)), 5),
        "min_expectancy_r": round(float(np.min(expectancies)), 5),
        "p10_expectancy_r": round(float(np.quantile(expectancies, 0.10)), 5),
    }

@staticmethod
def _empty_purged_cv() -> dict[str, object]:
    return {
        "folds": [],
        "fold_count": 0,
        "positive_rate": 0.0,
        "avg_expectancy_r": 0.0,
        "min_expectancy_r": 0.0,
        "p10_expectancy_r": 0.0,
    }

@staticmethod
def _empty_walk_forward() -> dict[str, object]:
    return {
        "folds": [],
        "fold_count": 0,
        "positive_fold_rate": 0.0,
        "avg_expectancy_r": 0.0,
        "min_expectancy_r": 0.0,
        "pooled": {
            "sample_count": 0,
            "expectancy_r": 0.0,
            "profit_factor": 0.0,
            "win_rate": 0.0,
            "max_drawdown_r": 0.0,
        },
    }

def _operational_trigger_metrics(self, samples: list[WindowSample], side: Side) -> dict[str, float | int]:
    key = f"{side}_entry_trigger_score"
    scores = np.asarray([float(sample.features.get(key, 0.0)) for sample in samples], dtype=float)
    if len(scores) == 0:
        return {"sample_count": 0, "avg_score": 0.0, "trigger_rate": 0.0}
    return {
        "sample_count": int(len(scores)),
    }

```

```

        "avg_score": round(float(np.mean(scores)), 5),
        "trigger_rate": round(float(np.mean(scores >= 0.58)), 5),
    }

def _event_ledger(
    self,
    samples: list[WindowSample],
    *,
    train_samples: list[WindowSample],
    holdout_samples: list[WindowSample],
    side: Side,
    rr: float,
) -> list[dict[str, object]]:
    train_ids = {id(sample) for sample in train_samples}
    holdout_ids = {id(sample) for sample in holdout_samples}
    ledger: list[dict[str, object]] = []
    ordered_samples = sorted(samples, key=lambda s: s.end)
    if self.event_ledger_limit > 0:
        ordered_samples = ordered_samples[: self.event_ledger_limit]
    for sample in ordered_samples:
        result_r, target_bar, stop_bar = RewardRiskAnalyzer._simulate_sample(sample, side, rr)
        if id(sample) in train_ids:
            split = "train"
        elif id(sample) in holdout_ids:
            split = "holdout"
        else:
            split = "assigned"
        ledger.append(
            {
                "symbol": sample.symbol,
                "window_end": sample.end,
                "forward_end": sample.outcome.forward_end,
                "year": sample.year,
                "split": split,
                "side": side,
                "rr": round(float(rr), 5),
                "result_r": round(float(result_r), 5),
                "triple_barrier_label": self._triple_barrier_label(target_bar, stop_bar),
                "target_bar": target_bar,
                "stop_bar": stop_bar,
                "sample_label": sample.outcome.label_for(side),
                "sample_speed_label": sample.outcome.speed_label_for(side),
                "sample_time_to_target": sample.outcome.time_to_target_for(side),
                "sample_time_to_stop": sample.outcome.time_to_stop_for(side),
                "gap_adverse_r": round(
                    float(self._side_attr(sample, side, "gap_adverse_r")),
                    5,
                ),
                "mfe_before_mae": bool(
                    self._side_attr(sample, side, "mfe_before_mae")
                ),
                "execution_cost_r": round(float(sample.outcome.execution_cost_r), 5),
                "fill_probability": round(float((sample.outcome.execution or {}).get("fill_probability", 0.0)), 5),
                "max_size_usd": round(float((sample.outcome.execution or {}).get("max_size_usd", 0.0)), 2),
                "entry_trigger_score": round(float(sample.features.get(f"{side}_{entry_trigger_score}", 0.0)), 5),
                "data_lineage": {
                    "adjusted": sample.features.get("data_adjusted"),
                    "what_to_show": sample.features.get("data_what_to_show"),
                    "bar_complete": sample.features.get("data_bar_complete"),
                },
            }
        )
    return ledger

@staticmethod
def _triple_barrier_label(target_bar: int | None, stop_bar: int | None) -> str:
    if target_bar is not None:
        return "target"
    if stop_bar is not None:
        return "stop"
    return "timeout"

@staticmethod
def _side_attr(sample: WindowSample, side: Side, suffix: str) -> object:
    return getattr(sample.outcome, f"{side}_{suffix}")

def _cost_stress_metrics(self, samples: list[WindowSample], side: Side, rr: float) -> dict[str, object]:
    stress: dict[str, object] = {}
    for multiplier in self.cost_stress_multipliers or [1.0, 2.0, 3.0]:
        metrics = self._split_metrics(samples, side, rr, cost_multiplier=float(multiplier))
        stress[f"{float(multiplier):g}x"] = {
            "multiplier": float(multiplier),
            "expectancy_r": metrics["expectancy_r"],
            "profit_factor": metrics["profit_factor"],
            "win_rate": metrics["win_rate"],
            "max_drawdown_r": metrics["max_drawdown_r"],
            "sample_count": metrics["sample_count"],
        }
    return stress

@staticmethod
def _cost_stress_passed(stress: object, *, required_multiplier: float) -> bool:
    if not isinstance(stress, dict):
        return False
    key = f"{float(required_multiplier):g}x"
    metrics = stress.get(key)

```



```

    if not isinstance(metrics, dict):
        return False
    return float(metrics.get("expectancy_r", 0.0)) > 0 and float(metrics.get("profit_factor", 0.0)) >= 1.0

@staticmethod
def _split_metrics(
    samples: list[WindowSample],
    side: Side,
    rr: float,
    *,
    cost_multiplier: float = 1.0,
) -> dict[str, float | int]:
    outcomes = np.asarray(
        [RewardRiskAnalyzer._simulate_sample(s, side, rr, cost_multiplier=cost_multiplier)[0] for s in samples],
        dtype=float,
    )
    if len(outcomes) == 0:
        return {
            "sample_count": 0,
            "expectancy_r": 0.0,
            "profit_factor": 0.0,
            "win_rate": 0.0,
            "max_drawdown_r": 0.0,
        }
    wins = outcomes[outcomes > 0]
    losses = outcomes[outcomes < 0]
    pf = float(wins.sum() / abs(losses.sum())) if len(losses) else float(wins.sum() or 0.0)
    return {
        "sample_count": int(len(outcomes)),
        "expectancy_r": round(float(np.mean(outcomes)), 5),
        "profit_factor": round(pf, 5),
        "win_rate": round(float(np.mean(outcomes > 0)), 5),
        "max_drawdown_r": round(RewardRiskAnalyzer._max_drawdown(outcomes), 5),
    }

@staticmethod
def _group_expectancy(samples: list[WindowSample], outcomes: np.ndarray, key: str) -> dict[str, float]:
    buckets: dict[str, list[float]] = {}
    for sample, outcome in zip(samples, outcomes, strict=True):
        value = str(getattr(sample, key))
        buckets.setdefault(value, []).append(float(outcome))
    items = sorted(
        ((k, round(float(np.mean(v)), 5)) for k, v in buckets.items()),
        key=lambda item: (item[1], item[0]),
        reverse=True,
    )
    return dict(items)

@staticmethod
def _positive_group_fraction(group_metrics: dict[str, float]) -> float:
    if not group_metrics:
        return 0.0
    return float(sum(1 for v in group_metrics.values() if v > 0) / len(group_metrics))

@staticmethod
def _side_score(metrics: dict[str, object]) -> float:
    train = metrics.get("train_metrics", {})
    expectancy = (
        float(train.get("expectancy_r", 0.0)) if isinstance(train, dict) else float(metrics.get("in_sample_expectancy_r", 0.0))
    )
    pf = min(
        float(train.get("profit_factor", 0.0)) if isinstance(train, dict) else float(metrics.get("in_sample_profit_factor", 0.0)),
        8.0,
    )
    hit_rate = float(metrics.get("target_hit_rate", metrics.get("hit_4r_rate", 0.0)))
    stability = float(metrics.get("stability_score", 0.0))
    return expectancy * 2.0 + pf * 0.15 + hit_rate * 1.5 + stability * 0.7

@staticmethod
def _candidate_score(metrics: dict[str, object]) -> float:
    train = metrics.get("train_metrics", {})
    if isinstance(train, dict):
        train_expectancy = float(train.get("best_expectancy_r", train.get("expectancy_r", 0.0)) or 0.0)
        train_pf = float(train.get("best_profit_factor", train.get("profit_factor", 0.0)) or 0.0)
        stability = float(train.get("stability_score", metrics.get("stability_score", 0.0)) or 0.0)
    else:
        train_expectancy = float(metrics.get("best_expectancy_r", metrics.get("in_sample_expectancy_r", 0.0)) or 0.0)
        train_pf = float(metrics.get("best_profit_factor", metrics.get("in_sample_profit_factor", 0.0)) or 0.0)
        stability = float(metrics.get("stability_score", 0.0))
    expectancy = max(0.0, train_expectancy)
    pf = min(train_pf, 8.0) / 8.0
    hit = float(metrics.get("target_hit_rate", metrics.get("hit_4r_rate", 0.0)))
    oos = max(0.0, float(metrics.get("out_of_sample_expectancy_r", 0.0)))
    rr = min(float(metrics.get("best_rr", 0.0)), 8.0) / 8.0
    operational = metrics.get("operational_trigger", {})
    trigger_rate = float(operational.get("trigger_rate", 0.0)) if isinstance(operational, dict) else 0.0
    walk_forward_rate = float(metrics.get("walk_forward_positive_fold_rate", 0.0))
    lift = min(max(0.0, float(metrics.get("expectancy_lift_r", 0.0))), 2.0)
    adjusted_p = min(max(float(metrics.get("adjusted_p_value", 1.0)), 0.0), 1.0)
    overfit = min(max(float(metrics.get("overfit_score", 0.5)), 0.0), 1.0)
    ci_low_bonus = min(max(0.0, float(metrics.get("expectancy_ci_low", 0.0))), 1.0)
    purged_rate = float(metrics.get("purged_cv_positive_rate", 0.0))
    deflated_psr = float(metrics.get("deflated_sharpe_probability", 0.0))
    fast_target = float(metrics.get("fast_target_rate", 0.0))
    fill_probability = float(metrics.get("avg_fill_probability", 0.0))
    replay = metrics.get("market_replay", {})

```

```

replay_expectancy = (
    max(0.0, float(replay.get("expected_expectancy_r", 0.0))) if isinstance(replay, dict) else 0.0
)
replay_quality = (
    float(replay.get("execution_quality_score", 0.0)) if isinstance(replay, dict) else 0.0
)
challenge = metrics.get("adversarial_challenge", {})
challenge_score = float(challenge.get("challenge_score", 0.0)) if isinstance(challenge, dict) else 0.0
causal = metrics.get("causal_invariance", {})
invariance_score = float(causal.get("invariance_score", 0.0)) if isinstance(causal, dict) else 0.0
teacher = metrics.get("foundation_teacher", {})
teacher_digest = teacher.get("pretraining_digest", {}) if isinstance(teacher, dict) else {}
teacher_score = (
    float(teacher_digest.get("embedding_quality_score", 0.0)) if isinstance(teacher_digest, dict) else 0.0
)
edge_decay = min(max(float(metrics.get("edge_decay_parameter_score", 0.0)), 0.0), 1.0)
testing_penalty = min(max(float(metrics.get("multiple_testing_penalty", 0.0)), 0.0), 1.0)
confidence = 1.0 - adjusted_p
raw_score = (
    expectancy * 0.24
    + pf * 0.13
    + hit * 0.11
    + stability * 0.11
    + oos * 0.08
    + rr * 0.05
    + trigger_rate * 0.04
    + walk_forward_rate * 0.03
    + purged_rate * 0.03
    + ci_low_bonus * 0.02
    + deflated_psr * 0.02
    + fast_target * 0.01
    + fill_probability * 0.01
    + min(replay_expectancy, 1.0) * 0.05
    + replay_quality * 0.03
    + challenge_score * 0.03
    + invariance_score * 0.03
    + teacher_score * 0.02
)
challenge_penalty = 0.25 if isinstance(challenge, dict) and challenge.get("rejection_recommended") else 0.0
replay_penalty = 0.18 if isinstance(replay, dict) and replay.get("passed") is False else 0.0
invariance_penalty = 0.15 if isinstance(causal, dict) and causal.get("passed") is False else 0.0
robustness = (
    (1.0 - 0.35 * overfit)
    * (1.0 - 0.12 * edge_decay)
    * (1.0 - 0.10 * testing_penalty)
    * (1.0 - challenge_penalty)
    * (1.0 - replay_penalty)
    * (1.0 - invariance_penalty)
)
return raw_score * (0.45 + 0.55 * confidence) * robustness + lift * 0.05

@staticmethod
def _promotion_score(metrics: dict[str, object]) -> float:
    train = metrics.get("train_metrics", {})
    oos = metrics.get("out_of_sample_metrics", {})
    wf = metrics.get("walk_forward_metrics", {})
    train_expectancy = (
        float(train.get("best_expectancy_r", train.get("expectancy_r", 0.0)) or 0.0)
        if isinstance(train, dict)
        else float(metrics.get("best_expectancy_r", 0.0) or 0.0)
    )
    oos_expectancy = (
        float(oos.get("expectancy_r", 0.0) or 0.0)
        if isinstance(oos, dict)
        else float(metrics.get("out_of_sample_expectancy_r", 0.0) or 0.0)
    )
    wf_rate = (
        float(wf.get("positive_fold_rate", 0.0) or 0.0)
        if isinstance(wf, dict)
        else float(metrics.get("walk_forward_positive_fold_rate", 0.0) or 0.0)
    )
    purged_rate = float(metrics.get("purged_cv_positive_rate", 0.0) or 0.0)
    adjusted_p = min(max(float(metrics.get("adjusted_p_value", 1.0) or 1.0), 0.0), 1.0)
    return round(
        max(0.0, train_expectancy) * 0.35
        + max(0.0, oos_expectancy) * 0.25
        + wf_rate * 0.20
        + purged_rate * 0.10
        + (1.0 - adjusted_p) * 0.10,
        5,
    )

@staticmethod
def _feature_summary(samples: list[WindowSample]) -> dict[str, object]:
    if not samples:
        return {}
    keys = sorted({key for s in samples for key in s.features})
    summary: dict[str, object] = {}
    for key in keys:
        values = np.asarray([s.features.get(key, 0.0) for s in samples], dtype=float)
        summary[key] = {
            "mean": round(float(np.mean(values)), 6),
            "median": round(float(np.median(values)), 6),
            "p25": round(float(np.quantile(values, 0.25)), 6),
            "p75": round(float(np.quantile(values, 0.75)), 6),
        }

```

```

        return summary

@classmethod
def _human_rule(
    cls,
    samples: list[WindowSample],
    *,
    baseline_samples: list[WindowSample],
    side: Side,
    metrics: dict[str, object],
) -> dict[str, object]:
    if not samples:
        return {"method": "monotonic_feature_rules", "rule": "", "conditions": []}
    allowed = cls._interpretable_feature_names(samples, baseline_samples)
    conditions: list[dict[str, object]] = []
    for key in allowed:
        cluster_values = np.asarray([sample.features.get(key, 0.0) for sample in samples], dtype=float)
        baseline_values = np.asarray([sample.features.get(key, 0.0) for sample in baseline_samples], dtype=float)
        if len(cluster_values) == 0 or len(baseline_values) == 0:
            continue
        cluster_median = float(np.median(cluster_values))
        baseline_median = float(np.median(baseline_values))
        iqr = float(np.quantile(baseline_values, 0.75) - np.quantile(baseline_values, 0.25))
        denominator = max(iqr, abs(baseline_median) * 0.25, 0.01)
        distinctiveness = abs(cluster_median - baseline_median) / denominator
        if distinctiveness < 0.45:
            continue
        if cluster_median >= baseline_median:
            operator = ">="
            threshold = float(np.quantile(cluster_values, 0.25))
        else:
            operator = "<="
            threshold = float(np.quantile(cluster_values, 0.75))
        conditions.append(
            {
                "feature": key,
                "label": cls._feature_label(key),
                "operator": operator,
                "threshold": round(float(threshold), 6),
                "cluster_median": round(cluster_median, 6),
                "baseline_median": round(baseline_median, 6),
                "distinctiveness": round(float(distinctiveness), 5),
                "monotonic": True,
            }
        )
    conditions = sorted(conditions, key=lambda item: float(item["distinctiveness"]), reverse=True)[:4]
    fragments = [
        f"{item['label']} {item['operator']} {item['threshold']}"
        for item in conditions
    ]
    if fragments:
        rule = f"{side} setup when " + " and ".join(fragments)
    else:
        rule = f"{side} setup defined by centroid similarity; no single feature rule dominates"
    return {
        "method": "monotonic_feature_rules",
        "side": side,
        "rule": rule,
        "conditions": conditions,
        "support": int(len(samples)),
        "quality": {
            "best_rr": metrics.get("best_rr", 0.0),
            "expectancy_r": metrics.get("expectancy_r", 0.0),
            "profit_factor": metrics.get("profit_factor", 0.0),
            "purged_cv_positive_rate": metrics.get("purged_cv_positive_rate", 0.0),
        },
    }

@staticmethod
def _interpretable_feature_names(
    samples: list[WindowSample],
    baseline_samples: list[WindowSample],
) -> list[str]:
    preferred = [
        "cumulative_return",
        "slope",
        "last_quarter_return",
        "local_drawdown",
        "local_runup",
        "range_phase_expansion",
        "volume_phase_acceleration",
        "volume_rel_last",
        "shapelet_v_reversal_score",
        "shapelet_inverted_v_score",
        "shapelet_flag_continuation_score",
        "swing_hh_rate",
        "swing_hl_rate",
        "swing_lh_rate",
        "swing_ll_rate",
        "swing_trend_score",
        "gap_up_reclaim_continuation_score",
        "gap_down_reclaim_continuation_score",
        "relative_strength_spy",
        "relative_strength_qqq",
        "relative_strength_sector",
        "relative_strength_industry",
    ]

```

```

        "market_regime_score",
        "market_breadth_proxy",
        "liquidity_score",
        "avg_dollar_volume",
        "atr_pct",
    ]
    available = {key for sample in samples + baseline_samples for key in sample.features}
    return [key for key in preferred if key in available]

@staticmethod
def _feature_label(key: str) -> str:
    labels = {
        "cumulative_return": "window return",
        "slope": "price slope",
        "last_quarter_return": "late-window return",
        "local_drawdown": "local drawdown",
        "local_runup": "runup from low",
        "range_phase_expansion": "range expansion",
        "volume_phase_acceleration": "volume acceleration",
        "volume_rel_last": "last relative volume",
        "shapelet_v_reversal_score": "V reversal shape",
        "shapelet_inverted_v_score": "inverted-V shape",
        "shapelet_flag_continuation_score": "flag continuation shape",
        "swing_hh_rate": "higher-high swing rate",
        "swing_hl_rate": "higher-low swing rate",
        "swing_lh_rate": "lower-high swing rate",
        "swing_ll_rate": "lower-low swing rate",
        "swing_trend_score": "swing trend score",
        "gap_up_reclaim_continuation_score": "gap-up reclaim continuation",
        "gap_down_reclaim_continuation_score": "gap-down reclaim continuation",
        "relative_strength_spy": "RS vs SPY",
        "relative_strength_qqq": "RS vs QQQ",
        "relative_strength_sector": "RS vs sector",
        "relative_strength_industry": "RS vs industry",
        "market_regime_score": "market regime score",
        "market_breadth_proxy": "breadth proxy",
        "liquidity_score": "liquidity score",
        "avg_dollar_volume": "avg dollar volume",
        "atr_pct": "ATR percent",
    }
    return labels.get(key, key.replace("_", " "))

@classmethod
def _regime_profile(cls, samples: list[WindowSample]) -> dict[str, object]:
    if not samples:
        return {"dominant_regime": "unknown", "buckets": {}}
    bucket_counts: dict[str, int] = {}
    for sample in samples:
        stratum = cls._stratum(sample)
        bucket = "|".join(str(part) for part in stratum[1:])
        bucket_counts[bucket] = bucket_counts.get(bucket, 0) + 1
    dominant = max(bucket_counts.items(), key=lambda item: item[1])[0] if bucket_counts else "unknown"
    feature_keys = [
        "volatility_regime",
        "trend_regime",
        "market_regime_score",
        "market_breadth_proxy",
        "sector_strength",
        "liquidity_score",
    ]
    means = {
        key: round(float(np.mean([sample.features.get(key, 0.0) for sample in samples])), 6)
        for key in feature_keys
    }
    sorted_buckets = dict(sorted(bucket_counts.items(), key=lambda item: item[1], reverse=True)[:12])
    dominant_parts = dominant.split("|")
    total = sum(bucket_counts.values()) or 1
    regime_count = len(bucket_counts)
    dominant_share = bucket_counts.get(dominant, 0) / total
    return {
        "dominant_regime": dominant,
        "preferred_regime_keys": [dominant],
        "bucket_counts": sorted_buckets,
        "buckets": sorted_buckets,
        "regime_count": regime_count,
        "dominant_share": round(float(dominant_share), 6),
        "regime_specific": regime_count < 2 or dominant_share >= 0.75,
        "research_gate": (
            "regime_specific"
            if regime_count < 2 or dominant_share >= 0.75
            else "multi_regime_observed"
        ),
        "top_trend_regime": dominant_parts[1] if len(dominant_parts) > 1 else "",
        "top_market_regime": dominant_parts[2] if len(dominant_parts) > 2 else "",
        "feature_means": means,
    }

def _benchmark_regime_outcomes(
    self,
    samples: list[WindowSample],
    *,
    side: Side,
    rr: float,
) -> dict[str, object]:
    """Labeled outcome history per benchmark-regime bucket (section 3.8).

```

Each cluster sample is labeled with the PIT benchmark regime at its window end and simulated with the canonical triple-barrier path at the pattern's side/RR, so the matcher can calibrate a regime gate against real per-bucket expectancies instead of presence heuristics.

"""

table = self.benchmark_regime_table

if table is None or table.empty:

return {

"available": False,

"reason": "benchmark_regime_table_unavailable",

"buckets": {},

}

keys = regime_keys_for_dates(table, [sample.end for sample in samples])

grouped: dict[str, list[float]] = {}

for sample, key in zip(samples, keys, strict=True):

outcome_r = float(RewardRiskAnalyzer._simulate_sample(sample, side, rr)[0])

grouped.setdefault(key, []).append(outcome_r)

buckets: dict[str, dict[str, float | int]] = {}

labeled = 0

for key in sorted(grouped):

arr = np.asarray(grouped[key], dtype=float)

stats = self._outcome_metrics(arr)

buckets[key] = {"sample_count": int(len(arr)), **stats}

if INSUFFICIENT_HISTORY not in key:

labeled += int(len(arr))

return {

"available": True,

"method": "pit_benchmark_regime_at_sample_end+canonical_triple_barrier",

"side": str(side),

"rr": round(float(rr), 4),

"benchmark_symbol": str(table.attrs.get("benchmark_symbol", "SPY")),

"labeled_sample_count": int(labeled),

"unlabeled_sample_count": int(len(samples) - labeled),

"buckets": buckets,

}

def _apply_novelty_diversity(self, candidates: list[ClusterCandidate]) -> None:

if not candidates:

return

ordered = sorted(candidates, key=lambda candidate: candidate.score, reverse=True)

prior: list[ClusterCandidate] = []

for candidate in ordered:

comparable = [

existing

for existing in prior

if existing.side == candidate.side

and existing.timeframe == candidate.timeframe

and existing.window_size == candidate.window_size

]

max_similarity = max(

(self._centroid_similarity(candidate.centroid, existing.centroid) for existing in comparable),

default=0.0,

)

novelty = max(0.0, min(1.0, 1.0 - max_similarity))

regime_profile = candidate.metrics.get("regime_profile", {})

dominant_regime = (

str(regime_profile.get("dominant_regime", "unknown"))

if isinstance(regime_profile, dict)

else "unknown"

)

bucket = f"{candidate.side}|{candidate.timeframe}|w{candidate.window_size}|{dominant_regime}"

density = float(candidate.metrics.get("cluster_density", 0.0))

rare_promising_bonus = 0.0

if density > 0.0 and density <= 0.08 and float(candidate.metrics.get("expectancy_r", 0.0)) > 0:

rare_promising_bonus = min(0.12, (0.08 - density) * 1.5)

uncertainty = 1.0 / math.sqrt(max(candidate.sample_count, 1))

quality = max(0.0, float(candidate.metrics.get("expectancy_lift_r", 0.0))) + max(

0.0,

float(candidate.metrics.get("out_of_sample_expectancy_r", 0.0)),

)

expected_information_gain = novelty * (quality + rare_promising_bonus) * min(1.0, uncertainty * 10.0)

candidate.metrics["run_max_family_similarity"] = round(float(max_similarity), 6)

candidate.metrics["novelty_score"] = round(float(novelty), 6)

candidate.metrics["diversity_bucket"] = bucket

candidate.metrics["rare_promising_bonus"] = round(float(rare_promising_bonus), 6)

candidate.metrics["expected_information_gain"] = round(float(expected_information_gain), 6)

candidate.metrics["base_score_before_novelty"] = candidate.score

prior.append(candidate)

buckets: dict[str, list[ClusterCandidate]] = {}

for candidate in candidates:

bucket = str(candidate.metrics.get("diversity_bucket", "unknown"))

buckets.setdefault(bucket, []).append(candidate)

for bucket_candidates in buckets.values():

ranked = sorted(bucket_candidates, key=lambda candidate: candidate.score, reverse=True)

for rank, candidate in enumerate(ranked, start=1):

novelty = float(candidate.metrics.get("novelty_score", 1.0))

eig = float(candidate.metrics.get("expected_information_gain", 0.0))

rare_bonus = float(candidate.metrics.get("rare_promising_bonus", 0.0))

quota_penalty = min(0.45, max(0, rank - 3) * 0.12)

candidate.metrics["diversity_quota_rank"] = rank

candidate.metrics["diversity_quota_penalty"] = round(float(quota_penalty), 6)

adjusted = (

candidate.score

* (0.78 + 0.22 * novelty)

* (1.0 - quota_penalty)

```

        + eig * 0.06
        + rare_bonus * 0.03
    )
    candidate.score = round(float(max(0.0, adjusted)), 5)

@staticmethod
def _centroid_similarity(left: list[float], right: list[float]) -> float:
    if not left or not right or len(left) != len(right):
        return 0.0
    left_arr = np.asarray(left, dtype=float)
    right_arr = np.asarray(right, dtype=float)
    if not np.isfinite(left_arr).all() or not np.isfinite(right_arr).all():
        return 0.0
    distance = float(np.linalg.norm(left_arr - right_arr) / max(1.0, np.sqrt(len(left_arr))))
    return float(1.0 / (1.0 + distance))

def _examples(
    self,
    samples: list[WindowSample],
    vectors_scaled: np.ndarray,
    centroid_scaled: np.ndarray,
    side: Side,
) -> list[dict[str, object]]:
    distances = np.linalg.norm(vectors_scaled - centroid_scaled, axis=1)
    outcomes = np.asarray([s.outcome.outcome_for(side) for s in samples], dtype=float)
    selected: list[tuple[int, str]] = []
    for idx in np.argsort(distances)[:4]:
        selected.append((int(idx), "typical"))
    for idx in np.argsort(outcomes)[-4:][::-1]:
        selected.append((int(idx), "winner"))
    for idx in np.argsort(outcomes)[:3]:
        selected.append((int(idx), "loser"))
    seen: set[int] = set()
    examples: list[dict[str, object]] = []
    for idx, kind in selected:
        if idx in seen:
            continue
        seen.add(idx)
        sample = samples[idx]
        similarity = float(1.0 / (1.0 + distances[idx]))
        examples.append(
            {
                "symbol": sample.symbol,
                "timeframe": sample.timeframe,
                "window_start": sample.start,
                "window_end": sample.end,
                "forward_end": sample.outcome.forward_end,
                "entry_price": round(sample.outcome.entry_price, 4),
                "risk_proxy": round(sample.outcome.risk_proxy, 4),
                "outcome_r": round(sample.outcome.outcome_for(side), 4),
                "mfe_r": round(sample.outcome.mfe_for(side), 4),
                "mae_r": round(sample.outcome.mae_for(side), 4),
                "triple_barrier_label": sample.outcome.label_for(side),
                "time_to_target": sample.outcome.time_to_target_for(side),
                "time_to_stop": sample.outcome.time_to_stop_for(side),
                "speed_label": sample.outcome.speed_label_for(side),
                "gap_adverse_r": round(
                    float(self._side_attr(sample, side, "gap_adverse_r")),
                    5,
                ),
                "mfe_before_mae": bool(
                    self._side_attr(sample, side, "mfe_before_mae")
                ),
                "fill_probability": round(float((sample.outcome.execution or {}).get("fill_probability", 0.0)), 5),
                "max_size_usd": round(float((sample.outcome.execution or {}).get("max_size_usd", 0.0)), 2),
                "similarity": round(similarity, 6),
                "kind": kind,
                "chart": sample.chart,
                "features": {k: round(float(v), 6) for k, v in sample.features.items()},
            }
        )
    return examples

@staticmethod
def _cluster_signature(
    samples: list[WindowSample],
    vectors_scaled: np.ndarray,
    centroid_scaled: np.ndarray,
    side: Side,
) -> dict[str, object]:
    if not samples or vectors_scaled.size == 0:
        return {
            "medoid": {},
            "similarity_distribution": {},
            "concentration_checks": {
                "passed": False,
                "reasons": ["empty_cluster"],
                "max_symbol_share": 0.0,
                "max_month_share": 0.0,
                "symbol_herfindahl": 0.0,
                "month_herfindahl": 0.0,
            },
        }
    distances = np.linalg.norm(vectors_scaled - centroid_scaled, axis=1) / max(
        1.0, math.sqrt(vectors_scaled.shape[1])
    )

```

```

similarities = 1.0 / (1.0 + distances)
medoid_idx = int(np.argmax(distances))
medoid_sample = samples[medoid_idx]

def share_counts(values: list[str]) -> tuple[dict[str, int], float, float]:
    counts: dict[str, int] = {}
    for value in values:
        counts[value] = counts.get(value, 0) + 1
    total = max(1, len(values))
    shares = [count / total for count in counts.values()]
    return counts, round(float(max(shares, default=0.0)), 6), round(float(sum(s * s for s in shares)), 6)

month_keys: list[str] = []
for sample in samples:
    try:
        month_keys.append(pd.Timestamp(sample.end).strftime("%Y-%m"))
    except (TypeError, ValueError):
        month_keys.append("unknown")
symbol_counts, max_symbol_share, symbol_hhi = share_counts([sample.symbol for sample in samples])
month_counts, max_month_share, month_hhi = share_counts(month_keys)
reasons: list[str] = []
if max_symbol_share > 0.40:
    reasons.append("symbol_concentration_gt_40pct")
if max_month_share > 0.50:
    reasons.append("month_concentration_gt_50pct")
top_symbols = sorted(symbol_counts.items(), key=lambda item: item[1], reverse=True)[:5]
top_months = sorted(month_counts.items(), key=lambda item: item[1], reverse=True)[:5]
return {
    "method": "centroid_distance_medoid_and_concentration",
    "medoid": {
        "symbol": medoid_sample.symbol,
        "timeframe": medoid_sample.timeframe,
        "window_start": medoid_sample.start,
        "window_end": medoid_sample.end,
        "outcome_r": round(float(medoid_sample.outcome.outcome_for(side)), 5),
        "similarity": round(float(similarities[medoid_idx]), 6),
        "distance": round(float(distances[medoid_idx]), 6),
        "chart": medoid_sample.chart,
        "features": {k: round(float(v), 6) for k, v in medoid_sample.features.items()},
    },
    "similarity_distribution": {
        "p05": round(float(np.quantile(similarities, 0.05)), 6),
        "p50": round(float(np.quantile(similarities, 0.50)), 6),
        "p90": round(float(np.quantile(similarities, 0.90)), 6),
        "p95": round(float(np.quantile(similarities, 0.95)), 6),
        "min": round(float(np.min(similarities)), 6),
        "max": round(float(np.max(similarities)), 6),
    },
    "concentration_checks": {
        "passed": not reasons,
        "reasons": reasons,
        "max_symbol_share": max_symbol_share,
        "max_month_share": max_month_share,
        "symbol_herfindahl": symbol_hhi,
        "month_herfindahl": month_hhi,
        "top_symbols": [{"symbol": symbol, "count": count} for symbol, count in top_symbols],
        "top_months": [{"month": month, "count": count} for month, count in top_months],
        "thresholds": {
            "max_symbol_share": 0.40,
            "max_month_share": 0.50,
        },
    },
}

def _quant_validation_metrics(
    self,
    samples: list[WindowSample],
    *,
    side: Side,
    rr: float,
) -> dict[str, object]:
    """Dedup + uniqueness-weighted inference over the cluster's train events.

    Overlapping windows from a stride sampler pseudo-replicate the same market
    episode. Here every occurrence's outcome span is placed on a shared
    calendar-day axis, occurrences of the same symbol that overlap in time are
    deduplicated, and the survivors get Lopez de Prado average-uniqueness
    weights so n_eff (= sum of weights) reflects the information actually
    available. Expectancy/PF are weight-corrected and uncertainty comes from a
    stationary block bootstrap plus a Newey-West t-stat, both of which respect
    the residual autocorrelation that an iid bootstrap ignores.
    """
    empty = {
        "n_raw": len(samples),
        "n_unique": 0,
        "n_eff": 0.0,
        "expectancy_r_weighted": 0.0,
        "profit_factor_weighted": 0.0,
        "expectancy_ci95_low": 0.0,
        "expectancy_ci95_high": 0.0,
        "newey_west_t": 0.0,
        "sharpe_per_trade": 0.0,
        "skew": 0.0,
        "kurtosis": 3.0,
        "horizonBars": 0,
        "method": "dedup_uniqueness_stationary_bootstrap_newey_west",
    }

```

```

}
if not samples:
    return empty
spans: list[tuple[int, int]] = []
horizons: list[int] = []
for sample in samples:
    entry_day = pd.Timestamp(sample.end).toordinal() + 1
    exit_day = max(entry_day, pd.Timestamp(sample.outcome.forward_end).toordinal())
    spans.append((entry_day, exit_day))
    if sample.outcome.forward_returns:
        horizons.append(max(int(h) for h in sample.outcome.forward_returns))
horizonBars = max(horizons) if horizons else 10
starts = np.asarray([s for s, _ in spans], dtype=int)
ends = np.asarray([e for _, e in spans], dtype=int)

kept: list[int] = []
by_symbol: dict[str, list[int]] = {}
for idx, sample in enumerate(samples):
    by_symbol.setdefault(sample.symbol, []).append(idx)
for indices in by_symbol.values():
    idx_arr = np.asarray(indices, dtype=int)
    local_keep = select_nonoverlapping_events(starts[idx_arr], ends[idx_arr])
    kept.extend(int(idx_arr[i]) for i in local_keep)
kept_arr = np.asarray(sorted(kept), dtype=int)
if kept_arr.size == 0:
    return empty

weights, n_eff = average_uniqueness_weights(starts[kept_arr], ends[kept_arr])
order = np.argsort(starts[kept_arr], kind="stable")
kept_sorted = kept_arr[order]
weights_sorted = weights[order]
outcomes = np.asarray(
    [RewardRiskAnalyzer._simulate_sample(samples[int(i)], side, rr)[0] for i in kept_sorted],
    dtype=float,
)
weight_sum = float(weights_sorted.sum())
mean_w = float(np.sum(weights_sorted * outcomes) / weight_sum) if weight_sum > 0 else 0.0
pf_w = weighted_profit_factor(outcomes, weights_sorted)
seed = self._null_seed(side, rr, kept_sorted.size, len(samples)) ^ 0x51A7B007
ci_lo, ci_hi, _, _ = stationary_bootstrap_ci(
    outcomes,
    np.mean,
    n_boot=max(100, int(self.quant_bootstrap_draws)),
    mean_block=horizonBars,
    rng=seed,
)
nw_t, _ = newey_west_tstat(outcomes, lags=horizonBars)
sd = float(np.std(outcomes, ddof=1)) if outcomes.size > 1 else 0.0
sharpe = float(np.mean(outcomes) / sd) if sd > 1e-12 else 0.0
skew, kurt = sample_skew_kurt(outcomes)
return {
    "n_raw": len(samples),
    "n_unique": int(kept_sorted.size),
    "n_eff": round(float(n_eff), 4),
    "expectancy_r_weighted": round(mean_w, 5),
    "profit_factor_weighted": round(float(pf_w), 5) if math.isfinite(pf_w) else pf_w,
    "expectancy_ci95_low": round(float(ci_lo), 5) if math.isfinite(ci_lo) else 0.0,
    "expectancy_ci95_high": round(float(ci_hi), 5) if math.isfinite(ci_hi) else 0.0,
    "newey_west_t": round(float(nw_t), 5) if math.isfinite(nw_t) else 0.0,
    "sharpe_per_trade": round(sharpe, 5),
    "skew": round(float(skew), 5),
    "kurtosis": round(float(kurt), 5),
    "horizonBars": int(horizonBars),
    "method": "dedup_uniqueness_stationary_bootstrap_newey_west",
}

def _match_tau_similarity(self, cluster_vectors: np.ndarray, centroid: np.ndarray) -> float:
    """Per-pattern matcher threshold from the intra-cluster similarity spread.

    Uses the same normalized-distance -> similarity mapping as
    NovelPatternMatcher so the persisted tau is directly comparable at match
    time. tau is the similarity such that match_tau_percentile% of real
    cluster members would pass; the global config threshold stays as a floor.
    """
    if cluster_vectors.size == 0:
        return 0.0
    distances = np.linalg.norm(cluster_vectors - centroid, axis=1) / max(
        1.0, math.sqrt(cluster_vectors.shape[1])
    )
    similarities = 1.0 / (1.0 + distances)
    pct = min(max(float(self.match_tau_percentile), 50.0), 100.0)
    tau = float(np.quantile(similarities, 1.0 - pct / 100.0))
    return round(tau, 6)

@staticmethod
def _pattern_key(window_size: int, cluster_id: int, side: str, centroid: Iterable[float]) -> str:
    digest = blake2b(digest_size=10)
    digest.update(f"w={window_size}|c={cluster_id}|s={side}|".encode())
    arr = np.asarray(list(centroid), dtype=np.float32)
    digest.update(np.round(arr, 3).tobytes())
    return f"novel_{side}_w{window_size}_{digest.hexdigest()}"

```

Full Source: `backend/tradeo/research/pattern\_embedding\_engine.py`

`backend/tradeo/research/pattern\_embedding\_engine.py` ? 601 lines, 26718 bytes, sha256 prefix `f45bfccceb23beee`

Audit relevance: Embedding contract used for research/lab parity. Audit contract versioning and feature availability at decision time.

Public surface summary before full source:

? class `PatternEmbeddingEngine` line 13 methods: contract, embed

Risk hints: broad `except Exception` present.

```
from __future__ import annotations

from dataclasses import dataclass
from typing import ClassVar

import numpy as np
import pandas as pd

from tradeo.services.technical_indicators import atr

@dataclass(slots=True)
class PatternEmbeddingEngine:
    """Convert arbitrary OHLCV windows into fixed-length numeric fingerprints.

    The embedding is intentionally local, deterministic and cheap. Tradeo can run
    it all day without spending API tokens because every calculation is NumPy /
    pandas based. LLM/API review only receives the final compact digest.
    """

    points_per_channel: int = 24
    CONTRACT_ID: ClassVar[str] = "tradeo.pattern_embedding.v2.legacy_prefix_stable"

    def contract(self, *, vector_length: int | None = None) -> dict[str, object]:
        return {
            "contract_id": self.CONTRACT_ID,
            "engine": "PatternEmbeddingEngine",
            "points_per_channel": int(self.points_per_channel),
            "legacy_prefix_stable": True,
            "research_lab_shared_path": True,
            "vector_length": int(vector_length) if vector_length is not None else None,
            "normalization": {
                "price": "close/first_close_minus_1",
                "returns": "winsorized_log_returns",
                "volume": "volume/median_volume_clipped",
                "range": "high_low_pct_clipped",
            },
            "matcher_scaling": "train_fit_standard_scaler_prefix",
        }

    def embed(
        self,
        window: pd.DataFrame,
        benchmark_frames: dict[str, pd.DataFrame] | None = None,
    ) -> tuple[np.ndarray, dict[str, float], dict[str, list[float]]]:
        if len(window) < 5:
            raise ValueError("window must contain at least 5 bars")
        df = window.copy()
        close = df["close"].astype(float).to_numpy()
        high = df["high"].astype(float).to_numpy()
        low = df["low"].astype(float).to_numpy()
        open_ = df["open"].astype(float).to_numpy()
        volume = df["volume"].astype(float).to_numpy()

        safe_close = np.where(close == 0, 1e-9, close)
        log_close = np.log(np.maximum(safe_close, 1e-9))
        returns = np.diff(log_close, prepend=log_close[0])
        returns = self._winsorize(returns, 0.01, 0.99)
        cumulative_return = close[-1] / close[0] - 1 if close[0] else 0.0

        price_norm = close / max(close[0], 1e-9) - 1.0
        volume_rel = volume / max(float(np.nanmedian(volume)), 1.0)
        volume_rel = np.clip(volume_rel, 0, 5) / 5.0
        range_pct = np.clip((high - low) / np.maximum(safe_close, 1e-9), 0, 0.35) / 0.35
        close_position = np.clip((close - low) / np.maximum(high - low, 1e-9), 0, 1)
        body_pct = np.clip(np.abs(close - open_) / np.maximum(high - low, 1e-9), 0, 1)
        rolling_vol = pd.Series(returns).rolling(5, min_periods=2).std().fillna(0).to_numpy()
        rolling_vol = np.clip(rolling_vol, 0, np.nanpercentile(rolling_vol, 95) + 1e-9)
        if rolling_vol.max() > 0:
            rolling_vol = rolling_vol / rolling_vol.max()

        running_max = np.maximum.accumulate(close)
        running_min = np.minimum.accumulate(close)
        drawdown = (close - running_max) / np.maximum(running_max, 1e-9)
        distance_from_low = close / np.maximum(running_min, 1e-9) - 1.0

        slope = self._slope(price_norm)
        downside_vol = float(np.std(np.minimum(returns, 0)))
        upside_vol = float(np.std(np.maximum(returns, 0)))
        local_drawdown = float(abs(np.min(drawdown)))
        local_runup = float(np.max(distance_from_low))
        last_quarter_return = float(close[-1] / close[max(0, len(close) * 3 // 4)] - 1)
        volume_trend = self._slope(volume_rel)
        atr_pct = float(atr(df, 14).iloc[-1] / close[-1]) if len(df) >= 15 and close[-1] else 0.0
        sma20 = float(np.mean(close[-min(20, len(close)) :]))
```

```

sma50 = float(np.mean(close[-min(50, len(close)) :]))
trend_regime = self._trend_regime(close[-1], sma20, sma50)
atr_series = atr(df, 14).bfill().fillna(0.0) if len(df) >= 15 else pd.Series([0.0] * len(df), index=df.index)
median_atr_pct = float(np.median(atr_series.tail(50) / np.maximum(df["close"].tail(50).astype(float), 1e-9)))
volatility_regime = float(atr_pct / max(median_atr_pct, 1e-9)) if median_atr_pct > 0 else 1.0
previous_close = float(close[-2]) if len(close) >= 2 else float(close[-1])
gap_pct = float(open[-1] / max(previous_close, 1e-9) - 1.0)
lookback = min(20, max(2, len(close) - 1))
previous_high = float(np.max(high[-lookback - 1 : -1])) if len(high) > 1 else float(high[-1])
previous_low = float(np.min(low[-lookback - 1 : -1])) if len(low) > 1 else float(low[-1])
breakout_strength = float(close[-1] / max(previous_high, 1e-9) - 1.0)
breakdown_strength = float(previous_low / max(close[-1], 1e-9) - 1.0)
avg_dollar_volume = float(np.mean(close[-lookback:] * volume[-lookback:]))
liquidity_score = float(min(1.0, np.log1p(max(avg_dollar_volume, 0.0)) / np.log1p(50_000_000.0)))
distance_sma20_atr = float((close[-1] - sma20) / max(float(atr_series.iloc[-1]), close[-1] * 0.01, 1e-9))
long_trigger_score = self._entry_trigger_score(
    side="long",
    close=float(close[-1]),
    open_=float(open[-1]),
    high=float(high[-1]),
    low=float(low[-1]),
    previous_close=previous_close,
    previous_high=previous_high,
    previous_low=previous_low,
    sma20=sma20,
)
short_trigger_score = self._entry_trigger_score(
    side="short",
    close=float(close[-1]),
    open_=float(open[-1]),
    high=float(high[-1]),
    low=float(low[-1]),
    previous_close=previous_close,
    previous_high=previous_high,
    previous_low=previous_low,
    sma20=sma20,
)
market_context = self._market_context_features(df, benchmark_frames or {})
phase_features = self._phase_features(volume_rel, range_pct, returns)
shapelet_features = self._shapelet_features(price_norm, volume_rel)
swing_features = self._swing_features(high, low, close)
gap_reclaim_features = self._gap_reclaim_features(
    close=close,
    open_=open_,
    high=high,
    low=low,
    previous_close=previous_close,
    previous_high=previous_high,
    previous_low=previous_low,
)
range_velocity = pd.Series(range_pct).diff().fillna(0.0).to_numpy()
volume_price_pressure = np.clip(np.sign(returns) * volume_rel, -1.0, 1.0)
swing_state_channel = self._swing_state_channel(high, low, close)

# Keep this legacy prefix stable. Stored pattern centroids may be shorter
# and NovelPatternMatcher intentionally compares only the saved prefix.
legacy_channels = [
    price_norm,
    returns,
    volume_rel,
    range_pct,
    close_position,
    body_pct,
    rolling_vol,
    drawdown,
    distance_from_low,
]
legacy_downsampled = np.concatenate(
    [self._resample(channel, self.points_per_channel) for channel in legacy_channels]
)
legacy_scalar_features = np.array(
    [
        cumulative_return,
        slope,
        downside_vol,
        upside_vol,
        local_drawdown,
        local_runup,
        last_quarter_return,
        volume_trend,
        atr_pct,
        trend_regime,
        min(volatility_regime, 5.0),
        gap_pct,
        breakout_strength,
        breakdown_strength,
        liquidity_score,
        distance_sma20_atr,
        long_trigger_score,
        short_trigger_score,
        market_context["weekly_return"],
        market_context["weekly_trend"],
        market_context["relative_strength_spy"],
        market_context["relative_strength_qqq"],
        market_context["benchmark_alignment"],
    ],

```

```

        dtype=float,
    )
    enhanced_channels = [
        range_velocity,
        volume_price_pressure,
        swing_state_channel,
    ]
    enhanced_downsampled = np.concatenate(
        [self._resample(channel, self.points_per_channel) for channel in enhanced_channels]
    )
    enhanced_scalar_features = np.array(
        [
            phase_features["volume_phase_early"],
            phase_features["volume_phase_mid"],
            phase_features["volume_phase_late"],
            phase_features["volume_phase_acceleration"],
            phase_features["range_phase_expansion"],
            shapelet_features["shapelet_v_reversal_score"],
            shapelet_features["shapelet_inverted_v_score"],
            shapelet_features["shapelet_flag_continuation_score"],
            shapelet_features["price_volume_impulse_corr"],
            swing_features["swing_hh_rate"],
            swing_features["swing_hl_rate"],
            swing_features["swing_lh_rate"],
            swing_features["swing_ll_rate"],
            swing_features["swing_trend_score"],
            gap_reclaim_features["gap_up_reclaim_continuation_score"],
            gap_reclaim_features["gap_down_reclaim_continuation_score"],
            market_context["market_return"],
            market_context["market_breadth_proxy"],
            market_context["sector_strength"],
            market_context["industry_strength"],
            market_context["relative_strength_sector"],
            market_context["relative_strength_industry"],
            market_context["market_regime_score"],
        ],
        dtype=float,
    )
    vector = np.concatenate(
        [legacy_downsampled, legacy_scalar_features, enhanced_downsampled, enhanced_scalar_features]
    )
    vector = np.nan_to_num(vector, nan=0.0, posinf=0.0, neginf=0.0).astype(np.float32)
    features = {
        "cumulative_return": float(cumulative_return),
        "slope": float(slope),
        "downside_vol": downside_vol,
        "upside_vol": upside_vol,
        "local_drawdown": local_drawdown,
        "local_runup": local_runup,
        "last_quarter_return": last_quarter_return,
        "volume_trend": float(volume_trend),
        "atr_pct": atr_pct,
        "trend_regime": trend_regime,
        "volatility_regime": volatility_regime,
        "gap_pct": gap_pct,
        "breakout_strength": breakout_strength,
        "breakdown_strength": breakdown_strength,
        "avg_dollar_volume": avg_dollar_volume,
        "liquidity_score": liquidity_score,
        "distance_sma20_atr": distance_sma20_atr,
        "long_entry_trigger_score": long_trigger_score,
        "short_entry_trigger_score": short_trigger_score,
        **market_context,
        **phase_features,
        **shapelet_features,
        **swing_features,
        **gap_reclaim_features,
        "range_pct_mean": float(np.mean(range_pct) * 0.35),
        "volume_rel_last": float(volume_rel[-1] * 5.0),
        "range_velocity_last": float(range_velocity[-1] * 0.35),
        "volume_price_pressure_last": float(volume_price_pressure[-1]),
    }
    chart = {
        "close_norm": self._resample(price_norm, 48).round(5).tolist(),
        "volume_rel": self._resample(volume_rel, 48).round(5).tolist(),
        "range_pct": self._resample(range_pct, 48).round(5).tolist(),
        "swing_state": self._resample(swing_state_channel, 48).round(5).tolist(),
        "volume_price_pressure": self._resample(volume_price_pressure, 48).round(5).tolist(),
    }
    return vector, features, chart

    @staticmethod
    def _resample(values: np.ndarray, length: int) -> np.ndarray:
        values = np.asarray(values, dtype=float)
        if len(values) == length:
            return values
        if len(values) == 1:
            return np.repeat(values[0], length)
        old_x = np.linspace(0, 1, len(values))
        new_x = np.linspace(0, 1, length)
        return np.interp(new_x, old_x, values)

    @staticmethod
    def _slope(values: np.ndarray) -> float:
        values = np.asarray(values, dtype=float)
        if len(values) < 2:

```

```

        return 0.0
    x = np.linspace(-1, 1, len(values))
    try:
        return float(np.polyfit(x, values, 1)[0])
    except Exception: # noqa: BLE001
        return 0.0

    @staticmethod
    def _trend_regime(close: float, sma20: float, sma50: float) -> float:
        if close >= sma20 >= sma50:
            return 1.0
        if close <= sma20 <= sma50:
            return -1.0
        return 0.0

    @staticmethod
    def _entry_trigger_score(
        *,
        side: str,
        close: float,
        open_: float,
        high: float,
        low: float,
        previous_close: float,
        previous_high: float,
        previous_low: float,
        sma20: float,
    ) -> float:
        if side == "short":
            breakout = close <= previous_low * 1.005
            reclaim = high >= previous_low and close < previous_low * 1.01 and close < open_
            momentum = close < previous_close and close < sma20
        else:
            breakout = close >= previous_high * 0.995
            reclaim = low <= previous_high and close > previous_high * 0.99 and close > open_
            momentum = close > previous_close and close > sma20
        if breakout:
            return 1.0
        if reclaim:
            return 0.78
        if momentum:
            return 0.58
        return 0.0

    @staticmethod
    def _phase_features(volume_rel: np.ndarray, range_pct: np.ndarray, returns: np.ndarray) -> dict[str, float]:
        n = len(volume_rel)
        if n == 0:
            return {
                "volume_phase_early": 0.0,
                "volume_phase_mid": 0.0,
                "volume_phase_late": 0.0,
                "volume_phase_acceleration": 0.0,
                "range_phase_expansion": 0.0,
                "return_phase_acceleration": 0.0,
            }
        thirds = np.array_split(np.arange(n), 3)
        volume_means = [float(np.mean(volume_rel[idxs])) if len(idxs) else 0.0 for idxs in thirds]
        range_means = [float(np.mean(range_pct[idxs])) if len(idxs) else 0.0 for idxs in thirds]
        return_means = [float(np.mean(returns[idxs])) if len(idxs) else 0.0 for idxs in thirds]
        return {
            "volume_phase_early": volume_means[0] * 5.0,
            "volume_phase_mid": volume_means[1] * 5.0,
            "volume_phase_late": volume_means[2] * 5.0,
            "volume_phase_acceleration": volume_means[2] - volume_means[0],
            "range_phase_expansion": range_means[2] - range_means[0],
            "return_phase_acceleration": return_means[2] - return_means[0],
        }

    @classmethod
    def _shapelet_features(cls, price_norm: np.ndarray, volume_rel: np.ndarray) -> dict[str, float]:
        price = cls._zscore(cls._resample(price_norm, 16))
        volume = cls._zscore(cls._resample(volume_rel, 16))
        x = np.linspace(-1, 1, 16)
        v_template = -np.abs(x) + np.max(np.abs(x))
        inverted_v_template = -v_template
        flag_template = np.r_[np.linspace(0.0, 1.0, 6), np.linspace(0.65, 0.45, 5), np.linspace(0.45, 1.15, 5)]
        impulse = np.r_[np.zeros(10), np.linspace(0.2, 1.0, 6)]
        return {
            "shapelet_v_reversal_score": cls._positive_corr(price, v_template),
            "shapelet_inverted_v_score": cls._positive_corr(price, inverted_v_template),
            "shapelet_flag_continuation_score": cls._positive_corr(price, flag_template),
            "shapelet_volume_impulse_score": cls._positive_corr(volume, impulse),
            "price_volume_impulse_corr": cls._corr(np.diff(price, prepend=price[0]), volume),
        }

    @staticmethod
    def _swing_features(high: np.ndarray, low: np.ndarray, close: np.ndarray) -> dict[str, float]:
        pivot_highs: list[float] = []
        pivot_lows: list[float] = []
        hh = hl = lh = ll = 0
        for idx in range(1, max(1, len(close) - 1)):
            if high[idx] >= high[idx - 1] and high[idx] >= high[idx + 1]:
                if pivot_highs:
                    if high[idx] > pivot_highs[-1]:
                        hh += 1

```

```

        else:
            lh += 1
            pivot_highs.append(float(high[idx]))
        if low[idx] <= low[idx - 1] and low[idx] <= low[idx + 1]:
            if pivot_lows:
                if low[idx] > pivot_lows[-1]:
                    hl += 1
            else:
                ll += 1
            pivot_lows.append(float(low[idx]))
    total = max(1, hh + hl + lh + ll)
    return {
        "swing_hh_rate": hh / total,
        "swing_hl_rate": hl / total,
        "swing_lh_rate": lh / total,
        "swing_ll_rate": ll / total,
        "swing_trend_score": (hh + hl - lh - ll) / total,
        "swing_pivot_count": float(len(pivot_highs) + len(pivot_lows)),
    }

@staticmethod
def _swing_state_channel(high: np.ndarray, low: np.ndarray, close: np.ndarray) -> np.ndarray:
    state = np.zeros(len(close), dtype=float)
    last_high: float | None = None
    last_low: float | None = None
    for idx in range(1, max(1, len(close) - 1)):
        if high[idx] >= high[idx - 1] and high[idx] >= high[idx + 1]:
            if last_high is not None:
                state[idx] += 1.0 if high[idx] > last_high else -1.0
            last_high = float(high[idx])
        if low[idx] <= low[idx - 1] and low[idx] <= low[idx + 1]:
            if last_low is not None:
                state[idx] += 1.0 if low[idx] > last_low else -1.0
            last_low = float(low[idx])
    if len(state) > 1:
        state = pd.Series(state).replace(0.0, np.nan).ffill().fillna(0.0).to_numpy()
    return np.clip(state, -1.0, 1.0)

@staticmethod
def _gap_reclaim_features(
    *,
    close: np.ndarray,
    open_: np.ndarray,
    high: np.ndarray,
    low: np.ndarray,
    previous_close: float,
    previous_high: float,
    previous_low: float,
) -> dict[str, float]:
    last_open = float(open_[-1])
    last_close = float(close[-1])
    last_high = float(high[-1])
    last_low = float(low[-1])
    gap_pct = last_open / max(previous_close, 1e-9) - 1.0
    gap_up = gap_pct > 0.005
    gap_down = gap_pct < -0.005
    reclaim_up = (
        gap_up
        and last_low <= previous_high
        and last_close > max(last_open, previous_high * 0.995)
    )
    reclaim_down = (
        gap_down
        and last_high >= previous_low
        and last_close < min(last_open, previous_low * 1.005)
    )
    continuation_up = (
        gap_up
        and last_close > last_open
        and last_close >= (last_low + (last_high - last_low) * 0.65)
    )
    continuation_down = (
        gap_down
        and last_close < last_open
        and last_close <= (last_low + (last_high - last_low) * 0.35)
    )
    return {
        "gap_up_reclaim_score": 1.0 if reclaim_up else 0.0,
        "gap_down_reclaim_score": 1.0 if reclaim_down else 0.0,
        "gap_up_continuation_score": 1.0 if continuation_up else 0.0,
        "gap_down_continuation_score": 1.0 if continuation_down else 0.0,
        "gap_up_reclaim_continuation_score": 1.0 if reclaim_up and continuation_up else 0.0,
        "gap_down_reclaim_continuation_score": 1.0 if reclaim_down and continuation_down else 0.0,
    }

@staticmethod
def _market_context_features(
    window: pd.DataFrame,
    benchmark_frames: dict[str, pd.DataFrame],
) -> dict[str, float]:
    close = window["close"].astype(float)
    local_return = float(close.iloc[-1] / max(close.iloc[0], 1e-9) - 1.0)
    weekly_close = close.resample("w").last() if isinstance(window.index, pd.DatetimeIndex) else close
    weekly_return = (
        float(weekly_close.iloc[-1] / max(weekly_close.iloc[0], 1e-9) - 1.0)
        if len(weekly_close) >= 2
    )

```

```

        else local_return
    )
    weekly_trend = (
        PatternEmbeddingEngine._slope(
            (weekly_close / max(float(weekly_close.iloc[0]), 1e-9) - 1.0).to_numpy()
        )
        if len(weekly_close) >= 2
        else 0.0
    )
    spy_return = PatternEmbeddingEngine._benchmark_return(window, benchmark_frames.get("SPY"))
    qqq_return = PatternEmbeddingEngine._benchmark_return(window, benchmark_frames.get("QQQ"))
    sector_return = PatternEmbeddingEngine._benchmark_return(
        window,
        PatternEmbeddingEngine._first_benchmark(
            benchmark_frames,
            ("SECTOR", "sector", "sector_etf", "SECTOR ETF"),
        ),
    )
    industry_return = PatternEmbeddingEngine._benchmark_return(
        window,
        PatternEmbeddingEngine._first_benchmark(
            benchmark_frames,
            ("INDUSTRY", "industry", "industry_etf", "INDUSTRY ETF"),
        ),
    )
    benchmark_return = spy_return if spy_return != 0.0 else qqq_return
    alignment = (
        1.0
        if local_return == 0.0
        or benchmark_return == 0.0
        or np.sign(local_return) == np.sign(benchmark_return)
        else -1.0
    )
    market_returns = [value for value in (spy_return, qqq_return) if value != 0.0]
    market_return = float(np.mean(market_returns)) if market_returns else 0.0
    breadth_inputs = [
        PatternEmbeddingEngine._benchmark_breadth_proxy(window, benchmark_frames.get("SPY")),
        PatternEmbeddingEngine._benchmark_breadth_proxy(window, benchmark_frames.get("QQQ")),
    ]
    breadth_values = [value for value in breadth_inputs if value is not None]
    breadth_proxy = float(np.mean(breadth_values)) if breadth_values else 0.5
    sector_strength = sector_return if sector_return != 0.0 else market_return
    industry_strength = industry_return if industry_return != 0.0 else sector_strength
    market_regime_score = float(np.clip(market_return * 10.0 + (breadth_proxy - 0.5) * 1.5, -1.0, 1.0))
    return {
        "weekly_return": weekly_return,
        "weekly_trend": float(weekly_trend),
        "relative_strength_spy": float(local_return - spy_return),
        "relative_strength_qqq": float(local_return - qqq_return),
        "benchmark_alignment": float(alignment),
        "market_return": market_return,
        "market_breadth_proxy": breadth_proxy,
        "market_regime_score": market_regime_score,
        "sector_strength": float(sector_strength),
        "industry_strength": float(industry_strength),
        "relative_strength_sector": float(local_return - sector_return) if sector_return != 0.0 else 0.0,
        "relative_strength_industry": float(local_return - industry_return) if industry_return != 0.0 else 0.0,
    }

    @staticmethod
    def _first_benchmark(benchmark_frames: dict[str, pd.DataFrame], keys: tuple[str, ...]) -> pd.DataFrame | None:
        lower_map = {key.lower(): value for key, value in benchmark_frames.items()}
        for key in keys:
            frame = benchmark_frames.get(key)
            if frame is not None:
                return frame
            frame = lower_map.get(key.lower())
            if frame is not None:
                return frame
        return None

    @staticmethod
    def _benchmark_return(window: pd.DataFrame, benchmark: pd.DataFrame | None) -> float:
        if benchmark is None or benchmark.empty:
            return 0.0
        if not isinstance(window.index, pd.DatetimeIndex):
            return 0.0
        bench = benchmark.copy()
        if not isinstance(bench.index, pd.DatetimeIndex):
            return 0.0
        bench = bench.sort_index()
        aligned = bench.reindex(window.index, method="ffill")
        aligned = aligned.dropna(subset=["close"])
        if len(aligned) < 2:
            return 0.0
        return float(aligned["close"].iloc[-1] / max(float(aligned["close"].iloc[0]), 1e-9) - 1.0)

    @staticmethod
    def _benchmark_breadth_proxy(window: pd.DataFrame, benchmark: pd.DataFrame | None) -> float | None:
        if benchmark is None or benchmark.empty or not isinstance(window.index, pd.DatetimeIndex):
            return None
        if not isinstance(benchmark.index, pd.DatetimeIndex):
            return None
        aligned = benchmark.sort_index().reindex(window.index, method="ffill").dropna(subset=["close"])
        if len(aligned) < 5:
            return None

```

```

close = aligned["close"].astype(float)
sma = close.rolling(min(20, len(close)), min_periods=3).mean()
above_ma = float(close.iloc[-1] >= sma.iloc[-1])
positive_return = float(close.iloc[-1] >= close.iloc[0])
return (above_ma + positive_return) / 2.0

@staticmethod
def _corr(left: np.ndarray, right: np.ndarray) -> float:
    left = np.asarray(left, dtype=float)
    right = np.asarray(right, dtype=float)
    if len(left) != len(right) or len(left) < 2:
        return 0.0
    left_std = float(np.std(left))
    right_std = float(np.std(right))
    if left_std == 0.0 or right_std == 0.0:
        return 0.0
    return float(np.corrcoef(left, right)[0, 1])

@classmethod
def _positive_corr(cls, left: np.ndarray, right: np.ndarray) -> float:
    return max(0.0, cls._corr(cls._zscore(left), cls._zscore(right)))

@staticmethod
def _zscore(values: np.ndarray) -> np.ndarray:
    arr = np.asarray(values, dtype=float)
    std = float(np.std(arr))
    if std == 0.0:
        return np.zeros_like(arr, dtype=float)
    return (arr - float(np.mean(arr))) / std

@staticmethod
def _winsorize(values: np.ndarray, low_q: float, high_q: float) -> np.ndarray:
    values = np.asarray(values, dtype=float)
    lo = np.nanquantile(values, low_q)
    hi = np.nanquantile(values, high_q)
    return np.clip(values, lo, hi)

```

Full Source: ``backend/tradeo/research/novel_pattern_matcher.py``

``backend/tradeo/research/novel_pattern_matcher.py`` ? 866 lines, 40597 bytes, sha256 prefix ``e4bbf8cb1f67d5fc``

Audit relevance: Live/current matcher for discovered patterns. Audit feature parity, threshold/ambiguity logic, regime fit, and incomplete-bar exclusions.

Public surface summary before full source:

? class ``NovelPatternMatcher`` line 35 methods: `match_current`, `match_dedupe_key`, `match_dedupe_key_from_model`

Risk hints: broad ``except Exception`` present; wall-clock timestamp usage.

```

from __future__ import annotations

from collections import defaultdict
from dataclasses import dataclass
from datetime import datetime, timezone
import math
from typing import Any, Literal

import numpy as np
import pandas as pd
from loguru import logger
from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import DiscoveredPattern, DiscoveredPatternMatch, DiscoveredPatternStatus
from tradeo.research.pattern_embedding_engine import PatternEmbeddingEngine
from tradeo.services.data_provider import MarketDataProvider, pick_symbols
from tradeo.services.entry_variants import (
    build_entry_audit_context,
    build_entry_variants,
    classify_regime,
)
from tradeo.services.market_regime import INSUFFICIENT_HISTORY, MarketRegimeService
from tradeo.services.provider_factory import get_market_data_provider
from tradeo.modules.fox_hunter.production_manifest import (
    production_manifest_for_pattern,
    production_manifest_is_active,
)
from tradeo.services.market_session import REGULAR_CLOSE, US_EQUITY_TZ
from tradeo.services.state_policy import LAB_RUNTIME_STATES, PRODUCTION_RUNTIME_STATES
from tradeo.services.technical_indicators import atr, normalize_ohlcv

@dataclass(slots=True)
class NovelPatternMatcher:
    """Find current charts that look like validated discovered patterns.

    This is still not an execution layer. Laboratory/Fox Hunter scanners decide
    whether a match becomes a signal or an IB order.
    """

    provider: MarketDataProvider | None = None
    settings: Settings | None = None

```

```

def __post_init__(self) -> None:
    self.settings = self.settings or get_settings()
    self.provider = self.provider or get_market_data_provider()

def match_current(
    self,
    db: Session,
    symbols: list[str] | None = None,
    limit: int | None = None,
    max_patterns: int | None = None,
    similarity_threshold: float | None = None,
    module: Literal["laboratory", "fox_hunter"] = "laboratory",
    store: bool = True,
) -> dict[str, Any]:
    settings = self.settings
    assert settings is not None
    statuses = self._statuses_for_module(module)
    pattern_limit = max_patterns or settings.discovery_match_max_patterns
    pattern_query = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.validation_passed.is_(True))
        .filter(DiscoveredPattern.status.in_(statuses))
        .order_by(DiscoveredPattern.score.desc())
    )
    if module == "fox_hunter":
        patterns = [
            pattern
            for pattern in pattern_query.all()
            if production_manifest_is_active(pattern)
        ][:pattern_limit]
    else:
        patterns = pattern_query.limit(pattern_limit).all()
    if symbols is None:
        symbols = pick_symbols(limit=limit or settings.discovery_match_symbol_limit)
    else:
        symbols = [s.upper().strip() for s in symbols if s.strip()]
    threshold = similarity_threshold or settings.discovery_match_similarity_threshold
    matches: list[dict[str, Any]] = []
    regime_gate_blocked = 0
    engine = PatternEmbeddingEngine()
    feature_parity_contract = {
        **engine.contract(),
        "research_path": "WindowSampler -> PatternEmbeddingEngine.embed",
        "lab_path": "NovelPatternMatcher -> PatternEmbeddingEngine.embed",
    }
    requiredBarsByTimeframe = self._requiredBarsByTimeframe(patterns)
    assert self.provider is not None
    benchmarkRegime = MarketRegimeService(
        provider=self.provider, settings=settings
    ).currentRegime()
    data_cache: dict[tuple[str, str], pd.DataFrame | None] = {}
    benchmark_cache: dict[str, dict[str, pd.DataFrame]] = {}
    embedding_cache: dict[
        tuple[str, str, int],
        tuple[pd.DataFrame, np.ndarray, dict[str, float], dict[str, list[float]]] | None,
    ] = {}
    patterns_by_timeframe: dict[str, list[DiscoveredPattern]] = defaultdict(list)
    for pattern in patterns:
        patterns_by_timeframe[pattern.timeframe].append(pattern)

    for timeframe, timeframe_patterns in patterns_by_timeframe.items():
        requiredBars = requiredBarsByTimeframe[timeframe]
        benchmark_cache[timeframe] = self._benchmarkFrames(
            timeframe,
            requiredBars=requiredBars,
            cache=data_cache,
        )
        for symbol in symbols:
            df = self._currentData(
                symbol,
                timeframe,
                requiredBars=requiredBars,
                cache=data_cache,
            )
            if df is None:
                continue
            for window_size in sorted({pattern.window_size for pattern in timeframe_patterns}):
                cache_key = (symbol.upper(), timeframe, int(window_size))
                if cache_key not in embedding_cache:
                    if len(df) < int(window_size) + 20:
                        embedding_cache[cache_key] = None
                    else:
                        try:
                            window = df.iloc[-int(window_size) :]
                            embedding_cache[cache_key] = (
                                df,
                                *engine.embed(window, benchmarkFrames=benchmark_cache[timeframe]),
                            )
                        except Exception as exc:  # noqa: BLE001
                            logger.warning(
                                "current embedding failed for {} / {} / w{: {}}",
                                symbol,
                                timeframe,
                                window_size,
                                exc,
                            )

```



```

        )
        embedding_cache[cache_key] = None

similarity_diagnostics: dict[int, dict[str, Any]] = {}
competitors_by_window: dict[tuple[str, int], list[dict[str, Any]]] = defaultdict(list)
for pattern in timeframe_patterns:
    cached = embedding_cache.get((symbol.upper(), timeframe, int(pattern.window_size)))
    diagnostic = self._similarity_diagnostic(
        pattern,
        cached,
        threshold,
    )
    if diagnostic is None:
        continue
    similarity_diagnostics[int(pattern.id)] = diagnostic
    competitors_by_window[(pattern.timeframe, int(pattern.window_size))].append(
        {
            "pattern_id": int(pattern.id),
            "pattern_name": pattern.name,
            "pattern_key": pattern.pattern_key,
            "similarity": diagnostic["similarity"],
        }
    )
ambiguity_by_pattern: dict[int, dict[str, Any]] = {}
for competitors in competitors_by_window.values():
    ranked = sorted(competitors, key=lambda item: float(item["similarity"]), reverse=True)
    for item in ranked:
        peers = [peer for peer in ranked if int(peer["pattern_id"]) != int(item["pattern_id"])]
        second = peers[0] if peers else None
        ambiguity_by_pattern[int(item["pattern_id"])] = self._ambiguity_ratio(
            similarity=float(item["similarity"]),
            second=second,
        )

for pattern in timeframe_patterns:
    try:
        cached = embedding_cache.get((symbol.upper(), timeframe, int(pattern.window_size)))
        if cached is None:
            continue
        df_for_match, vector, features, chart = cached
        diagnostic = similarity_diagnostics.get(int(pattern.id))
        if not diagnostic or not diagnostic["passed_threshold"]:
            continue
        similarity = float(diagnostic["similarity"])
        effective_threshold = float(diagnostic["similarity_threshold_used"])
        ambiguity = ambiguity_by_pattern.get(int(pattern.id)) or self._ambiguity_ratio(
            similarity=similarity,
            second=None,
        )
        features = dict(features)
        features["avg_dollar_volume"] = float(
            (df_for_match["close"] * df_for_match["volume"]).tail(20).mean()
        )
        reward_risk = float(
            pattern.best_rr or settings.unvalidated_pattern_min_reward_risk
        )
        base_score = round(
            similarity * 0.55 + pattern.score * 0.30 + pattern.stability_score * 0.15,
            6,
        )
        base_gate = self._entry_gate(pattern.side, df_for_match, score=base_score, settings=settings)
        regime = classify_regime(features, base_gate)
        regime["benchmark_regime"] = dict(benchmark_regime)
        regime_fit = self._pattern_regime_fit(pattern, regime, settings)
        if settings.market_regime_hard_gate_enabled and regime_fit.get("hard_gate_blocked"):
            regime_gate_blocked += 1
            logger.info(
                "regime hard gate blocked {} on {} ({}): {}".format(
                    pattern.name,
                    symbol,
                    regime_fit.get("label"),
                    regime_fit.get("calibration"),
                )
            )
            continue
        entry_audit = build_entry_audit_context(df_for_match, pattern.timeframe)
        variants = build_entry_variants(
            side=pattern.side,
            df=df_for_match,
            base_entry_gate=base_gate,
            score=base_score,
            reward_risk=reward_risk,
            settings=settings,
        )
        if not variants:
            continue
        match_status = (
            "production_entry_candidate"
            if module == "fox_hunter"
            else "lab_entry_candidate"
        )
        for variant in variants:
            entry_gate = variant["entry_gate"]
            score = round(
                base_score * 0.78
                + float(entry_gate.get("entry_score", 0.0)) * 0.14
                + float(regime_fit["score"]) * 0.08,
            )

```

```

        6,
    )
    match = {
        "module": module,
        "pattern_id": pattern.id,
        "pattern_name": pattern.name,
        "pattern_key": pattern.pattern_key,
        "pattern_family_key": pattern.pattern_family_key,
        "canonical_pattern_key": pattern.canonical_pattern_key,
        "pattern_status": pattern.status.value,
        "pattern_promotion_status": pattern.promotion_status,
        "production_manifest": (
            production_manifest_for_pattern(pattern)
            if module == "fox_hunter"
            else None
        ),
        "symbol": symbol,
        "timeframe": pattern.timeframe,
        "side": pattern.side,
        "similarity": round(similarity, 6),
        "similarity_threshold_used": round(effective_threshold, 6),
        "match_ambiguity": ambiguity,
        "ambiguity_ratio": ambiguity["ambiguity_ratio"],
        "score": score,
        "entry_score": entry_gate["entry_score"],
        "entry_gate_passed": entry_gate["passed"],
        "entry_trigger": entry_gate["trigger"],
        "entry_variant_id": variant["entry_variant_id"],
        "entry_variant": variant["entry_variant"],
        "entry_price": variant["entry_price"],
        "stop_price": variant["stop_price"],
        "target_price": variant["target_price"],
        "reward_risk": reward_risk,
        "window_end": str(df_for_match.index[-1]),
        "status": match_status,
        "notes": self._notes_for_module(module),
        "chart": chart,
        "entry_audit": entry_audit,
        "regime": regime,
        "regime_fit": regime_fit,
        "metrics": {
            "entry_module": module,
            "entry_variant_id": variant["entry_variant_id"],
            "entry_variant": variant["entry_variant"],
            "pattern_status": pattern.status.value,
            "pattern_promotion_status": pattern.promotion_status,
            "pattern_score": pattern.score,
            "pattern_expectancy_r": pattern.expectancy_r,
            "pattern_profit_factor": pattern.profit_factor,
            "pattern_stability_score": pattern.stability_score,
            "pattern_regime_profile": (pattern.metrics_json or {}).get("regime_profile", {}),
            "feature_parity_contract": {
                **feature_parity_contract,
                "vector_length": int(len(vector)),
            },
            "match_ambiguity": ambiguity,
            "features": features,
            "entry_gate": entry_gate,
            "base_entry_gate": base_gate,
            "entry_audit": entry_audit,
            "regime": regime,
            "regime_fit": regime_fit,
        },
    }
    matches.append(match)
except Exception as exc: # noqa: BLE001
    logger.warning(
        "novel pattern match failed for {} / {}: {}".format(
            pattern.name,
            symbol,
            exc,
        )
    )
    continue
matches = sorted(matches, key=lambda m: m["score"], reverse=True)[
    : settings.discovery_match_max_results
]
if store:
    self._store_matches(db, matches)
return {
    "patterns_checked": len(patterns),
    "symbols_checked": len(symbols),
    "matches": matches,
    "stored_matches": len(matches) if store else 0,
    "module": module,
    "similarity_threshold": threshold,
    "feature_parity_contract": feature_parity_contract,
    "benchmark_regime": benchmark_regime,
    "regime_hard_gate_enabled": bool(settings.market_regime_hard_gate_enabled),
    "regime_gate_blocked": regime_gate_blocked,
    "generated_at": datetime.now(timezone.utc).isoformat(),
}

def _benchmark_frames(
    self,
    timeframe: str,
    *,

```

```

        required_bars: int,
        cache: dict[tuple[str, str], pd.DataFrame | None],
    ) -> dict[str, pd.DataFrame]:
        frames: dict[str, pd.DataFrame] = {}
        for symbol in ("SPY", "QQQ"):
            df = self._current_data(symbol, timeframe, required_bars=required_bars, cache=cache)
            if df is not None:
                frames[symbol] = df
        return frames

    @staticmethod
    def _statuses_for_module(module: str) -> list[DiscoveredPatternStatus]:
        if module == "fox_hunter":
            return list(PRODUCTION_RUNTIME_STATES)
        return list(LAB_RUNTIME_STATES)

    @staticmethod
    def _notes_for_module(module: str) -> str:
        if module == "fox_hunter":
            return (
                "Match de patrón de Producción; requiere live_armed "
                "y auditoría continua de Director."
            )
        return (
            "Match de patrón de Laboratorio; requiere paper validation "
            "y auditoría de Director."
        )

    @staticmethod
    def _pattern_regime_fit(
        pattern: DiscoveredPattern,
        regime: dict[str, Any],
        settings: Settings | None = None,
    ) -> dict[str, Any]:
        profile = (pattern.metrics_json or {}).get("regime_profile", {})
        if not isinstance(profile, dict) or not profile:
            return {"score": 0.5, "label": "unknown_pattern_regime", "matched": False}
        calibrated = NovelPatternMatcher._calibrated_regime_fit(profile, regime, settings)
        if calibrated is not None:
            return calibrated
        regime_key = str(regime.get("regime_key") or "")
        preferred = profile.get("preferred_regime_keys")
        if isinstance(preferred, list) and preferred:
            if regime_key in {str(item) for item in preferred}:
                return {"score": 1.0, "label": "preferred_regime", "matched": True}
            return {"score": 0.35, "label": "outside_preferred_regime", "matched": False}
        buckets = profile.get("bucket_counts") if isinstance(profile.get("bucket_counts"), dict) else {}
        if not buckets and isinstance(profile.get("buckets"), dict):
            buckets = profile["buckets"]
        if regime_key and regime_key in buckets:
            return {"score": 0.85, "label": "seen_regime", "matched": True}
        dominant = str(profile.get("dominant_regime") or "")
        if dominant and regime_key and dominant in regime_key:
            return {"score": 0.75, "label": "dominant_regime_overlap", "matched": True}
        market = str(regime.get("market_regime") or "")
        trend = str(regime.get("trend_regime") or "")
        top_market = profile.get("top_market_regime")
        top_trend = profile.get("top_trend_regime")
        overlap = 0.0
        if top_market and str(top_market) == market:
            overlap += 0.25
        if top_trend and str(top_trend) == trend:
            overlap += 0.25
        return {
            "score": round(0.45 + overlap, 4),
            "label": "partial_regime_overlap" if overlap else "unseen_regime",
            "matched": bool(overlap),
        }

    @staticmethod
    def _calibrated_regime_fit(
        profile: dict[str, Any],
        regime: dict[str, Any],
        settings: Settings | None,
    ) -> dict[str, Any] | None:
        """Per-bucket calibrated fit from research-labeled benchmark outcomes (3.8).

        Only fires when Research persisted enough simulated outcomes for the
        CURRENT benchmark regime bucket; otherwise the caller falls back to the
        presence heuristics. A calibrated-negative bucket marks the match as
        hard-gate eligible, enforced in match_current behind
        market_regime_hard_gate_enabled (default off).
        """
        if settings is None:
            return None
        outcomes = profile.get("benchmark_regime_outcomes")
        if not isinstance(outcomes, dict) or not outcomes.get("available"):
            return None
        benchmark = regime.get("benchmark_regime") if isinstance(regime, dict) else None
        regime_key = str((benchmark or {}).get("regime_key") or "")
        if not regime_key or INSUFFICIENT_HISTORY in regime_key:
            return None
        buckets = outcomes.get("buckets")
        bucket = buckets.get(regime_key) if isinstance(buckets, dict) else None
        if not isinstance(bucket, dict):
            return None

```

```

sample_count = int(bucket.get("sample_count") or 0)
min_samples = int(settings.market_regime_outcome_min_samples)
if sample_count < min_samples:
    return None
expectancy_r = float(bucket.get("expectancy_r") or 0.0)
calibration = {
    "regime_key": regime_key,
    "sample_count": sample_count,
    "min_samples": min_samples,
    "expectancy_r": expectancy_r,
    "win_rate": float(bucket.get("win_rate") or 0.0),
    "profit_factor": float(bucket.get("profit_factor") or 0.0),
    "rr": outcomes.get("rr"),
    "side": outcomes.get("side"),
    "benchmark_symbol": outcomes.get("benchmark_symbol"),
    "method": outcomes.get("method"),
}
if expectancy_r > 0.0:
    return {
        "score": round(min(1.0, 0.75 + 0.25 * min(1.0, expectancy_r)), 4),
        "label": "calibrated_regime_positive",
        "matched": True,
        "hard_gate_blocked": False,
        "calibration": calibration,
    }
return {
    "score": 0.2,
    "label": "calibrated_regime_negative",
    "matched": False,
    "hard_gate_blocked": True,
    "calibration": calibration,
}

@staticmethod
def _entry_gate(side: str, df: pd.DataFrame, *, score: float, settings: Settings) -> dict[str, Any]:
    latest = df.iloc[-1]
    previous = df.iloc[-21:-1] if len(df) >= 21 else df.iloc[:-1]
    if previous.empty:
        return {
            "passed": False,
            "trigger": "insufficient_history",
            "entry_score": 0.0,
            "reason": "not enough bars for entry trigger",
        }
    close = float(latest["close"])
    open_ = float(latest["open"])
    high = float(latest["high"])
    low = float(latest["low"])
    prev_close = float(previous["close"].iloc[-1])
    prev_high = float(previous["high"].max())
    prev_low = float(previous["low"].min())
    avg_volume = float(previous["volume"].tail(20).mean())
    volume_ratio = float(latest["volume"] / max(avg_volume, 1.0))
    sma20 = float(df["close"].tail(20).mean())
    sma50 = float(df["close"].tail(50).mean()) if len(df) >= 50 else sma20
    sma20_prev = float(df["close"].iloc[-40:-20].mean()) if len(df) >= 40 else sma20
    atr_value = float(atr(df, 14).iloc[-1]) if len(df) >= 15 else max(close * 0.02, 0.01)
    atr_pct = atr_value / max(close, 0.01)
    extension_atr = abs(close - sma20) / max(atr_value, 0.01)
    extension_score = max(0.0, min(1.0, 1.0 - extension_atr / settings.entry_max_extension_atr))
    volume_score = max(0.0, min(1.0, (volume_ratio - 0.8) / 1.2))
    volatility_ok = atr_pct <= settings.max_atr_pct
    if side.lower() == "short":
        trend_aligned = close <= sma20 and sma20 <= sma50 and sma20 <= sma20_prev * 1.01
    else:
        trend_aligned = close >= sma20 and sma20 >= sma50 and sma20 >= sma20_prev * 0.99
    trend_score = 1.0 if trend_aligned else 0.35
    volatility_score = 1.0 if volatility_ok else max(0.0, 1.0 - (atr_pct / max(settings.max_atr_pct, 0.01) - 1.0))
    regime_score = round(
        trend_score * 0.45 + volatility_score * 0.30 + extension_score * 0.15 + volume_score * 0.10,
        6,
    )
    regime_ok = regime_score >= settings.entry_min_regime_score

    if side.lower() == "short":
        breakout = close <= prev_low * 1.005
        reclaim = high >= prev_low and close < prev_low * 1.01 and close < open_
        momentum = close < prev_close and close < sma20
    else:
        breakout = close >= prev_high * 0.995
        reclaim = low <= prev_high and close > prev_high * 0.99 and close > open_
        momentum = close > prev_close and close > sma20

    if breakout:
        trigger = "breakout"
        trigger_score = 1.0
    elif reclaim:
        trigger = "pullback_reclaim"
        trigger_score = 0.78
    elif momentum:
        trigger = "momentum_close"
        trigger_score = 0.58
    else:
        trigger = "no_operational_trigger"
        trigger_score = 0.0

```

```

entry_score = round(
    score * 0.45 + trigger_score * 0.25 + volume_score * 0.15 + extension_score * 0.15,
    6,
)
volume_confirmed = volume_ratio >= settings.entry_min_volume_ratio
not_extended = extension_atr <= settings.entry_max_extension_atr
rejection_reasons: list[str] = []
if trigger_score <= 0:
    rejection_reasons.append("weak_trigger")
if entry_score < settings.entry_min_score:
    rejection_reasons.append("weak_entry_score")
if not regime_ok:
    rejection_reasons.append("regime_not_aligned")
if not volume_confirmed:
    rejection_reasons.append("insufficient_volume")
if not not_extended:
    rejection_reasons.append("excessive_extension")
if not volatility_ok:
    rejection_reasons.append("excessive_volatility")
passed = (
    trigger_score > 0
    and entry_score >= settings.entry_min_score
    and regime_ok
    and volume_confirmed
    and not_extended
)
reason = "entry gate passed" if passed else ";".join(rejection_reasons) or "entry gate failed"
return {
    "passed": passed,
    "trigger": trigger,
    "entry_score": entry_score,
    "trigger_score": round(trigger_score, 4),
    "volume_ratio": round(volume_ratio, 4),
    "volume_confirmed": volume_confirmed,
    "atr_pct": round(atr_pct, 6),
    "volatility_ok": volatility_ok,
    "extension_atr": round(extension_atr, 4),
    "not_extended": not_extended,
    "regime_score": regime_score,
    "regime_ok": regime_ok,
    "trend_aligned": trend_aligned,
    "prev_high_20": round(prev_high, 4),
    "prev_low_20": round(prev_low, 4),
    "sma20": round(sma20, 4),
    "sma50": round(sma50, 4),
    "close": round(close, 4),
    "reason": reason,
    "rejection_reasons": rejection_reasons,
}

@staticmethod
def _scaler(pattern: DiscoveredPattern) -> tuple[np.ndarray | None, np.ndarray | None]:
    metrics = pattern.metrics_json or {}
    mean = metrics.get("scaler_mean")
    scale = metrics.get("scaler_scale")
    if not isinstance(mean, list) or not isinstance(scale, list):
        return None, None
    return np.asarray(mean, dtype=float), np.asarray(scale, dtype=float)

@staticmethod
def _scaled_vector_for_pattern(
    vector: np.ndarray,
    centroid: np.ndarray,
    scaler_mean: np.ndarray,
    scaler_scale: np.ndarray,
) -> np.ndarray | None:
    if len(centroid) == 0 or len(vector) < len(centroid):
        return None
    if len(scaler_mean) < len(centroid) or len(scaler_scale) < len(centroid):
        return None
    vector_prefix = vector[: len(centroid)]
    mean_prefix = scaler_mean[: len(centroid)]
    scale_prefix = scaler_scale[: len(centroid)]
    return (vector_prefix - mean_prefix) / np.where(scale_prefix == 0, 1.0, scale_prefix)

def _prices(self, pattern: DiscoveredPattern, df, *, reward_risk: float) -> dict[str, float]:
    entry = float(df["close"].iloc[-1])
    atr_value = float(atr(df, 14).iloc[-1]) if len(df) >= 15 else entry * 0.02
    risk = max(atr_value * 1.5, entry * 0.015, 0.01)
    if pattern.side == "short":
        stop = entry + risk
        target = entry - reward_risk * risk
    else:
        stop = entry - risk
        target = entry + reward_risk * risk
    return {
        "entry_price": round(entry, 4),
        "stop_price": round(stop, 4),
        "target_price": round(target, 4),
    }

def _effective_threshold(self, pattern: DiscoveredPattern, floor: float) -> float:
    """Per-pattern similarity threshold; the global config value is a floor.

    A single global threshold misfits every cluster at once: compact clusters
    accept spurious matches and loose clusters never fire (P0-4). Research

```

```

persists match_tau_similarity (the intra-cluster similarity percentile),
and the matcher requires the stricter of the two.
"""
settings = self.settings
if settings is None or not settings.discovery_match_per_pattern_threshold:
    return floor
tau = (pattern.metrics_json or {}).get("match_tau_similarity")
try:
    tau = float(tau)
except (TypeError, ValueError):
    return floor
if not math.isfinite(tau) or tau <= 0.0:
    return floor
return max(floor, tau)

def _similarity_diagnostic(
    self,
    pattern: DiscoveredPattern,
    cached: tuple[pd.DataFrame, np.ndarray, dict[str, float], dict[str, list[float]]] | None,
    floor: float,
) -> dict[str, Any] | None:
    if cached is None:
        return None
    _, vector, _, _ = cached
    scaler_mean, scaler_scale = self._scaler(pattern)
    centroid = np.asarray(pattern.centroid_json, dtype=float)
    if scaler_mean is None or scaler_scale is None or len(centroid) == 0:
        return None
    scaled = self._scaled_vector_for_pattern(vector, centroid, scaler_mean, scaler_scale)
    if scaled is None:
        return None
    normalized_distance = float(np.linalg.norm(scaled - centroid) / max(1.0, np.sqrt(len(centroid))))
    similarity = float(1.0 / (1.0 + normalized_distance))
    effective_threshold = self._effective_threshold(pattern, floor)
    return {
        "similarity": round(similarity, 6),
        "normalized_distance": round(normalized_distance, 6),
        "similarity_threshold_used": round(float(effective_threshold), 6),
        "passed_threshold": similarity >= effective_threshold,
    }

@staticmethod
def _ambiguity_ratio(
    *,
    similarity: float,
    second: dict[str, Any] | None,
) -> dict[str, Any]:
    best = max(0.0, float(similarity))
    second_similarity = max(0.0, float(second.get("similarity", 0.0))) if second else 0.0
    margin = max(0.0, best - second_similarity)
    ratio = second_similarity / best if best > 1e-12 else 0.0
    return {
        "method": "second_best_similarity_over_best_same_symbol_timeframe_window",
        "ambiguity_ratio": round(float(ratio), 6),
        "similarity_margin": round(float(margin), 6),
        "best_similarity": round(float(best), 6),
        "second_best_similarity": round(float(second_similarity), 6),
        "second_best_pattern_id": int(second["pattern_id"]) if second else None,
        "second_best_pattern_name": str(second["pattern_name"]) if second else None,
        "second_best_pattern_key": str(second["pattern_key"]) if second else None,
        "ambiguous": bool(second and ratio >= 0.95),
    }

@staticmethod
def _requiredBarsByTimeframe(patterns: list[DiscoveredPattern]) -> dict[str, int]:
    required: dict[str, int] = {}
    for pattern in patterns:
        current = required.get(pattern.timeframe, 0)
        required[pattern.timeframe] = max(current, int(pattern.window_size) + 30)
    return required

def _current_data(
    self,
    symbol: str,
    timeframe: str,
    *,
    requiredBars: int,
    cache: dict[tuple[str, str], pd.DataFrame | None],
) -> pd.DataFrame | None:
    key = (symbol.upper(), timeframe)
    if key in cache:
        return cache[key].copy() if cache[key] is not None else None
    assert self.provider is not None
    period = self._current_match_period(timeframe, requiredBars=requiredBars)
    try:
        df = normalize_ohlcv(
            self.provider.fetch_ohlcv(
                symbol,
                period=period,
                interval=timeframe,
            )
        )
    except Exception as exc:  # noqa: BLE001
        logger.warning("current data fetch failed for {} / {}: {}".format(symbol, timeframe, exc))
        cache[key] = None
        return None

```

```

        if self.settings and self.settings.discovery_match_completeBars_only:
            df = self._drop_incomplete_daily_bar(df, timeframe)
        cache[key] = df
        return df.copy()

    @staticmethod
    def _drop_incomplete_daily_bar(
        df: pd.DataFrame,
        timeframe: str,
        now: datetime | None = None,
    ) -> pd.DataFrame:
        """Drop today's daily bar while the US session has not closed (P0-3).

        Patterns were trained on completed bars; an in-session daily bar carries
        partial volume and provisional high/low/close, so matching against it
        breaks Research<->Lab feature parity. Before the 16:00 New York close
        the operative window must end on yesterday's bar.
        """
        if timeframe.lower().strip() not in {"1d", "1 day"} or df.empty:
            return df
        current = now or datetime.now(US_EQUITY_TZ)
        local = current.astimezone(US_EQUITY_TZ) if current.tzinfo else current.replace(tzinfo=US_EQUITY_TZ)
        if local.time() >= REGULAR_CLOSE:
            return df
        try:
            last_date = pd.Timestamp(df.index[-1]).date()
        except (TypeError, ValueError):
            return df
        if last_date >= local.date():
            return df.iloc[:-1]
        return df

    @staticmethod
    def _current_match_period(timeframe: str, *, required_bars: int) -> str:
        interval = timeframe.lower().strip()
        if interval in {"1d", "1 day"}:
            calendar_days = math.ceil(required_bars * 7 / 5) + 10
            months = max(3, math.ceil(calendar_days / 30))
            return f"{months}mo"
        if interval in {"1wk", "1 week"}:
            months = max(12, math.ceil(required_bars * 7 / 30))
            return f"{months}mo"
        if interval in {"1h", "30m", "30 mins", "15m", "15 mins"}:
            days = max(10, math.ceil(required_bars / 6) + 5)
            return f"{days}d"
        if interval in {"5m", "5 mins", "1m", "1 min"}:
            days = max(3, math.ceil(required_bars / 60) + 2)
            return f"{days}d"
        return "6mo"

    @classmethod
    def _store_matches(cls, db: Session, matches: list[dict[str, Any]]) -> None:
        now = datetime.now(timezone.utc)
        for match in matches:
            metrics = cls._stored_match_metrics(match, now=now)
            existing = cls._existing_match(db, match, metrics)
            if existing is None:
                db.add(
                    DiscoveredPatternMatch(
                        pattern_id=int(match["pattern_id"]),
                        symbol=str(match["symbol"]),
                        timeframe=str(match["timeframe"]),
                        side=str(match["side"]),
                        similarity=float(match["similarity"]),
                        score=float(match["score"]),
                        entry_price=float(match["entry_price"]),
                        stop_price=float(match["stop_price"]),
                        target_price=float(match["target_price"]),
                        reward_risk=float(match["reward_risk"]),
                        matched_at=now,
                        window_end=str(match["window_end"]),
                        status=str(match["status"]),
                        notes=str(match["notes"]),
                        chart_json=match.get("chart", {}),
                        metrics_json=metrics,
                    )
                )
            else:
                continue
            prior = existing.metrics_json or {}
            metrics["first_seen_at"] = prior.get("first_seen_at") or (
                existing.matched_at.isoformat() if existing.matched_at else now.isoformat()
            )
            metrics["seen_count"] = int(prior.get("seen_count") or 1) + 1
            existing.side = str(match["side"])
            existing.similarity = float(match["similarity"])
            existing.score = float(match["score"])
            existing.entry_price = float(match["entry_price"])
            existing.stop_price = float(match["stop_price"])
            existing.target_price = float(match["target_price"])
            existing.reward_risk = float(match["reward_risk"])
            existing.matched_at = now
            existing.status = str(match["status"])
            existing.notes = str(match["notes"])
            existing.chart_json = match.get("chart", {})
            existing.metrics_json = metrics
            db.add(existing)

```

```

        db.commit()

    @classmethod
    def _existing_match(
        cls,
        db: Session,
        match: dict[str, Any],
        metrics: dict[str, Any],
    ) -> DiscoveredPatternMatch | None:
        candidate_rows = (
            db.query(DiscoveredPatternMatch)
            .filter(DiscoveredPatternMatch.pattern_id == int(match["pattern_id"]))
            .filter(DiscoveredPatternMatch.symbol == str(match["symbol"]))
            .filter(DiscoveredPatternMatch.timeframe == str(match["timeframe"]))
            .filter(DiscoveredPatternMatch.window_end == str(match["window_end"]))
            .all()
        )
        wanted = cls.match_dedupe_key(match, metrics)
        for row in candidate_rows:
            if cls.match_dedupe_key_from_model(row) == wanted:
                return row
        return None

    @staticmethod
    def _stored_match_metrics(match: dict[str, Any], *, now: datetime) -> dict[str, Any]:
        metrics = dict(match.get("metrics") or {})
        metrics["entry_variant_id"] = str(match.get("entry_variant_id") or metrics.get("entry_variant_id") or "")
        metrics["entry_variant"] = match.get("entry_variant") or metrics.get("entry_variant") or {}
        metrics["match_dedupe_key"] = NovelPatternMatcher.match_dedupe_key(match, metrics)
        metrics.setdefault("first_seen_at", now.isoformat())
        metrics["last_seen_at"] = now.isoformat()
        metrics.setdefault("seen_count", 1)
        return metrics

    @staticmethod
    def match_dedupe_key(match: dict[str, Any], metrics: dict[str, Any] | None = None) -> tuple[str, ...]:
        data = metrics or match.get("metrics") or {}
        return (
            str(match.get("pattern_id") or ""),
            str(match.get("symbol") or "").upper(),
            str(match.get("timeframe") or ""),
            str(data.get("entry_variant_id") or match.get("entry_variant_id") or ""),
            str(match.get("window_end") or ""),
        )

    @staticmethod
    def match_dedupe_key_from_model(match: DiscoveredPatternMatch) -> tuple[str, ...]:
        metrics = match.metrics_json or {}
        return (
            str(match.pattern_id),
            str(match.symbol or "").upper(),
            str(match.timeframe or ""),
            str(metrics.get("entry_variant_id") or ""),
            str(match.window_end or ""),
        )

```

Full Source: `backend/tradeo/research/novel\_pattern\_registry.py`

`backend/tradeo/research/novel\_pattern\_registry.py` ? 306 lines, 14870 bytes, sha256 prefix `e0151502d1aea8a9`

Public surface summary before full source:

? class `NovelPatternRegistry` line 22 methods: store_candidates, upsert_candidate, list_patterns

Risk hints: broad `except Exception` present.

```

from __future__ import annotations

from dataclasses import dataclass
from hashlib import blake2b
from typing import Any

import numpy as np
from sqlalchemy.orm import Session

from tradeo.core.config import get_settings
from tradeo.db.models import (
    DiscoveredPattern,
    DiscoveredPatternExample,
    DiscoveredPatternMetric,
    DiscoveredPatternStatus,
)
from tradeo.services.state_policy import is_legacy_promotion_state
from tradeo.research.types import ClusterCandidate

@dataclass(slots=True)
class NovelPatternRegistry:
    """Persistence layer for newly discovered LAB patterns."""

    max_examples_per_pattern: int = 11
    similarity_threshold: float | None = None

```



```

def __post_init__(self) -> None:
    if self.similarity_threshold is None:
        self.similarity_threshold = get_settings().discovery_registry_similarity_threshold

def store_candidates(
    self,
    db: Session,
    candidates: list[ClusterCandidate],
    run_id: int | None = None,
    store_rejected: bool = True,
) -> list[DiscoveredPattern]:
    stored: list[DiscoveredPattern] = []
    for candidate in candidates:
        if not candidate.validation_passed and not store_rejected:
            continue
        stored.append(self.upsert_candidate(db, candidate, run_id=run_id))
    db.commit()
    for pattern in stored:
        db.refresh(pattern)
    return stored

def upsert_candidate(
    self,
    db: Session,
    candidate: ClusterCandidate,
    run_id: int | None = None,
) -> DiscoveredPattern:
    pattern = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.pattern_key == candidate.pattern_key)
        .one_or_none()
    )
    is_variant = False
    if pattern is None:
        pattern, similarity = self._find_similar_pattern(db, candidate)
        if pattern is not None:
            is_variant = True
            candidate.metrics["registry_deduped"] = True
            candidate.metrics["registry_similarity"] = round(similarity, 6)
            candidate.metrics["registry_canonical_pattern_key"] = pattern.pattern_key
            candidate.metrics["registry_candidate_pattern_key"] = candidate.pattern_key
        else:
            pattern = DiscoveredPattern(pattern_key=candidate.pattern_key)
            candidate.metrics["registry_deduped"] = False
            candidate.metrics["registry_similarity"] = 0.0
            candidate.metrics["registry_canonical_pattern_key"] = candidate.pattern_key
            candidate.metrics["registry_candidate_pattern_key"] = candidate.pattern_key
    metrics = candidate.metrics
    drift = self._drift(pattern, candidate, is_variant=is_variant)
    registry_similarity = float(metrics.get("registry_similarity", 0.0))
    registry_novelty = max(0.0, min(1.0, 1.0 - registry_similarity))
    registry_eig = float(metrics.get("expected_information_gain", 0.0)) * (0.35 + 0.65 * registry_novelty)
    metrics["registry_novelty_score"] = round(registry_novelty, 6)
    metrics["registry_expected_information_gain"] = round(registry_eig, 6)
    metrics["registry_family_penalty"] = round(1.0 - (0.70 + 0.30 * registry_novelty), 6)
    metrics["registry_adjusted_score"] = round(
        float(candidate.score) * (0.70 + 0.30 * registry_novelty) + registry_eig * 0.04,
        5,
    )
    metrics["registry_score_scope"] = {
        "base": "lab_priority_score",
        "uses_descriptive_all": False,
        "novelty_adjustment": "registry_novelty_score",
        "expected_information_gain_adjustment": "registry_expected_information_gain",
    }
    pattern.run_id = run_id
    pattern.name = candidate.name
    raw_status = str(metrics.get("promotion_status", "lab" if candidate.validation_passed else "rejected"))
    status = self._canonical_promotion_status(
        raw_status,
        validation_passed=candidate.validation_passed,
        confirmation_recommended=bool(metrics.get("confirmation_recommended")),
    )
    if status != raw_status:
        metrics["legacy_promotion_status_blocked"] = raw_status
        metrics["runtime_promotion_blocked"] = True
        metrics["promotion_status"] = status
    try:
        pattern.status = DiscoveredPatternStatus(status)
    except ValueError:
        pattern.status = DiscoveredPatternStatus.REJECTED
    pattern.promotion_status = status
    pattern.side = candidate.side
    pattern.timeframe = candidate.timeframe
    pattern.window_size = candidate.window_size
    pattern.cluster_id = candidate.cluster_id
    if not pattern.pattern_family_key:
        pattern.pattern_family_key = self._family_key(candidate)
    if not pattern.canonical_pattern_key:
        pattern.canonical_pattern_key = pattern.pattern_key
    pattern.variant_key = candidate.pattern_key
    pattern.variant_count = max(1, int(pattern.variant_count or 1)) + (1 if is_variant else 0)
    pattern.drift_status = str(drift["status"])
    pattern.drift_score = float(drift["score"])
    metrics["pattern_family_key"] = pattern.pattern_family_key
    metrics["canonical_pattern_key"] = pattern.canonical_pattern_key

```

```

metrics["variant_key"] = pattern.variant_key
metrics["variant_count"] = pattern.variant_count
metrics["drift_status"] = pattern.drift_status
metrics["drift_score"] = pattern.drift_score
pattern.sample_count = candidate.sample_count
pattern.symbol_count = candidate.symbol_count
pattern.year_count = candidate.year_count
pattern.score = float(metrics.get("registry_adjusted_score", candidate.score))
pattern.reward_risk_estimate = float(metrics.get("reward_risk_estimate", 0.0))
pattern.expectancy_r = float(metrics.get("expectancy_r", 0.0))
pattern.profit_factor = float(metrics.get("profit_factor", 0.0))
pattern.win_rate = float(metrics.get("win_rate", 0.0))
pattern.avg_mfe_r = float(metrics.get("avg_mfe_r", 0.0))
pattern.avg_mae_r = float(metrics.get("avg_mae_r", 0.0))
pattern.stability_score = float(metrics.get("stability_score", 0.0))
pattern.out_of_sample_expectancy_r = float(metrics.get("out_of_sample_expectancy_r", 0.0))
pattern.out_of_sample_profit_factor = float(metrics.get("out_of_sample_profit_factor", 0.0))
pattern.best_rr = float(metrics.get("best_rr", 0.0))
pattern.best_tested_rr = float(metrics.get("best_tested_rr", 0.0))
pattern.best_expectancy_r = float(metrics.get("best_expectancy_r", 0.0))
pattern.best_profit_factor = float(metrics.get("best_profit_factor", 0.0))
pattern.best_win_rate = float(metrics.get("best_win_rate", 0.0))
pattern.best_max_drawdown_r = float(metrics.get("best_max_drawdown_r", 0.0))
pattern.preferred_rr_passed = bool(metrics.get("preferred_rr_passed", False))
pattern.premium_rr_passed = bool(metrics.get("premium_rr_passed", False))
pattern.promotion_reason = str(metrics.get("promotion_reason", ""))
pattern.confirmation_status = str(metrics.get("confirmation_status", ""))
if metrics.get("confirmation_recommended") and not pattern.confirmation_status:
    pattern.confirmation_status = "needs_confirmation"
pattern.confirmation_priority_score = float(metrics.get("confirmation_priority_score", 0.0))
pattern.confirmation_reason = str(metrics.get("confirmation_reason", ""))
pattern.confirmation_next_action = str(metrics.get("confirmation_next_action", ""))
pattern.rr_metrics_json = self._json_clean(metrics.get("rr_metrics", {}))
pattern.rejection_reasons_json = self._json_clean(metrics.get("rejection_reasons", []))
pattern.in_sample_expectancy_r = float(metrics.get("in_sample_expectancy_r", 0.0))
pattern.in_sample_profit_factor = float(metrics.get("in_sample_profit_factor", 0.0))
pattern.out_of_sample_win_rate = float(metrics.get("out_of_sample_win_rate", 0.0))
pattern.out_of_sample_max_drawdown_r = float(metrics.get("out_of_sample_max_drawdown_r", 0.0))
pattern.validation_passed = candidate.validation_passed
pattern.validation_reasons_json = candidate.validation_reasons
pattern.centroid_json = candidate.centroid
pattern.metrics_json = self._json_clean(metrics)
pattern.feature_summary_json = self._json_clean(candidate.feature_summary)
db.add(pattern)
db.flush()

# Replace examples with the latest representative set.
db.query(DiscoveredPatternExample).filter(DiscoveredPatternExample.pattern_id == pattern.id).delete()
for example in candidate.examples[: self.max_examples_per_pattern]:
    db.add(
        DiscoveredPatternExample(
            pattern_id=pattern.id,
            symbol=str(example.get("symbol", "")),
            timeframe=str(example.get("timeframe", candidate.timeframe)),
            window_start=str(example.get("window_start", "")),
            window_end=str(example.get("window_end", "")),
            forward_end=str(example.get("forward_end", "")),
            entry_price=float(example.get("entry_price", 0.0)),
            risk_proxy=float(example.get("risk_proxy", 0.0)),
            outcome_r=float(example.get("outcome_r", 0.0)),
            mfe_r=float(example.get("mfe_r", 0.0)),
            mae_r=float(example.get("mae_r", 0.0)),
            similarity=float(example.get("similarity", 0.0)),
            kind=str(example.get("kind", "typical")),
            chart_json=self._json_clean(example.get("chart", {})),
            features_json=self._json_clean(example.get("features", {})),
        )
    )
db.add(
    DiscoveredPatternMetric(
        pattern_id=pattern.id,
        split="full",
        metrics_json=self._json_clean(metrics),
    )
)
return pattern

@staticmethod
def _canonical_promotion_status(
    status: str,
    *,
    validation_passed: bool,
    confirmation_recommended: bool,
) -> str:
    if is_legacy_promotion_state(status):
        return "lab_candidate" if validation_passed else "rejected"
    if confirmation_recommended and status == "rejected":
        return "needs_confirmation"
    return status

def _family_key(self, candidate: ClusterCandidate) -> str:
    digest = blake2b(digest_size=8)
    digest.update(f"{candidate.side}|{candidate.timeframe}|{candidate.window_size}".encode())
    centroid = np.asarray(candidate.centroid, dtype=np.float32)
    digest.update(np.round(centroid, 2).tobytes())
    return f"family_{candidate.side}_{candidate.timeframe}_w{candidate.window_size}_{digest.hexdigest()}"

```

```

@staticmethod
def _drift(pattern: DiscoveredPattern, candidate: ClusterCandidate, *, is_variant: bool) -> dict[str, float | str]:
    if not is_variant and not pattern.id:
        return {"status": "stable", "score": 0.0}
    old_expectancy = float(pattern.best_expectancy_r or pattern.expectancy_r or 0.0)
    new_expectancy = float(candidate.metrics.get("best_expectancy_r", candidate.metrics.get("expectancy_r", 0.0)))
    delta = new_expectancy - old_expectancy
    score = delta / max(abs(old_expectancy), 0.25)
    if score <= -0.25:
        status = "degrading"
    elif score >= 0.25:
        status = "improving"
    else:
        status = "stable"
    return {"status": status, "score": round(float(score), 5)}

def _find_similar_pattern(
    self,
    db: Session,
    candidate: ClusterCandidate,
) -> tuple[DiscoveredPattern | None, float]:
    threshold = float(self.similarity_threshold or 1.0)
    patterns = (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.side == candidate.side)
        .filter(DiscoveredPattern.timeframe == candidate.timeframe)
        .filter(DiscoveredPattern.window_size == candidate.window_size)
        .order_by(DiscoveredPattern.score.desc())
        .limit(250)
        .all()
    )
    best_pattern: DiscoveredPattern | None = None
    best_similarity = 0.0
    for pattern in patterns:
        similarity = self._centroid_similarity(candidate.centroid, pattern.centroid_json)
        if similarity > best_similarity:
            best_pattern = pattern
            best_similarity = similarity
    if best_pattern is not None and best_similarity >= threshold:
        return best_pattern, best_similarity
    return None, 0.0

@staticmethod
def _centroid_similarity(left: list[float], right: list[float]) -> float:
    if not left or not right or len(left) != len(right):
        return 0.0
    left_arr = np.asarray(left, dtype=float)
    right_arr = np.asarray(right, dtype=float)
    if not np.isfinite(left_arr).all() or not np.isfinite(right_arr).all():
        return 0.0
    distance = float(np.linalg.norm(left_arr - right_arr) / max(1.0, np.sqrt(len(left_arr))))
    return float(1.0 / (1.0 + distance))

@staticmethod
def list_patterns(
    db: Session,
    limit: int = 100,
    status: str | None = None,
) -> list[DiscoveredPattern]:
    query = db.query(DiscoveredPattern).order_by(
        DiscoveredPattern.score.desc(),
        DiscoveredPattern.created_at.desc(),
    )
    if status:
        try:
            query = query.filter(DiscoveredPattern.status == DiscoveredPatternStatus(status))
        except ValueError:
            query = query.filter(DiscoveredPattern.promotion_status == status)
    return query.limit(max(1, min(limit, 500))).all()

@staticmethod
def _json_clean(value: Any) -> Any:
    # SQLAlchemy JSON columns reject numpy scalars. Recursively convert them.
    try:
        import numpy as np
    except Exception:
        # noqa: BLE001
        np = None
    # type: ignore[assignment]
    if np is not None and isinstance(value, np.generic):
        return value.item()
    if isinstance(value, dict):
        return {str(k): NovelPatternRegistry._json_clean(v) for k, v in value.items()}
    if isinstance(value, list):
        return [NovelPatternRegistry._json_clean(v) for v in value]
    if isinstance(value, tuple):
        return [NovelPatternRegistry._json_clean(v) for v in value]
    return value

```

Full Source: ``backend/tradeo/research/global_experiment_registry.py``

``backend/tradeo/research/global_experiment_registry.py`` ? 268 lines, 10930 bytes, sha256 prefix ``6478bd182130fd5d``

Public surface summary before full source:

? class `GlobalExperimentRegistry` line 20 methods: load, register, registry_hash, hash_payload

Risk hints: wall-clock timestamp usage.

```
from __future__ import annotations

import hashlib
import json
import os
import shutil
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import numpy as np

from tradeo.research.types import ClusterCandidate

REGISTRY_HASH_ALGORITHM = "sha256_canonical_v1"

@dataclass(slots=True)
class GlobalExperimentRegistry:
    """Append-only-ish registry of discovery trials across research runs."""

    path: Path
    hash_algorithm: str = REGISTRY_HASH_ALGORITHM

    def load(self) -> dict[str, Any]:
        if not self.path.exists():
            return self._empty()
        try:
            payload = json.loads(self.path.read_text(encoding="utf-8"))
        except (OSError, json.JSONDecodeError):
            return self._empty()
        if not isinstance(payload, dict):
            return self._empty()
        payload.setdefault("schema_version", 2)
        payload.setdefault("global_trial_count", 0)
        payload.setdefault("runs", [])
        payload.setdefault("experiments", [])
        payload.setdefault("hash_algorithm", self.hash_algorithm)
        return payload

    def register(
        self,
        candidates: list[ClusterCandidate],
        *,
        run_id: int | str,
        params: dict[str, Any] | None = None,
    ) -> dict[str, Any]:
        payload = self.load()
        payload["schema_version"] = 2
        payload["hash_algorithm"] = self.hash_algorithm
        previous_registry_hash = self.registry_hash(payload) if self.path.exists() else ""
        now = datetime.now(timezone.utc).isoformat()
        payload["updated_at"] = now
        experiments = payload.setdefault("experiments", [])
        existing = {
            str(row.get("experiment_id")): row
            for row in experiments
            if isinstance(row, dict) and row.get("experiment_id")
        }
        global_trial_count = int(payload.get("global_trial_count", 0) or 0)
        added = 0
        candidate_updates: list[tuple[ClusterCandidate, str, int, int]] = []
        for rank, candidate in enumerate(sorted(candidates, key=lambda c: c.score, reverse=True), start=1):
            trial_count = self._candidate_trial_count(candidate)
            variant_id = self._variant_id(candidate)
            experiment_id = f"run={run_id}|pattern={candidate.pattern_key}|variant={variant_id}"
            row = existing.get(experiment_id)
            if row is None:
                global_trial_count += trial_count
                row = {
                    "experiment_id": experiment_id,
                    "run_id": run_id,
                    "registered_at": now,
                    "pattern_key": candidate.pattern_key,
                    "family_id": self._family_id(candidate),
                    "variant_id": variant_id,
                    "rank_by_lab_priority": rank,
                    "candidate_trial_count": trial_count,
                    "global_trial_count_after": global_trial_count,
                    "fit_scope": candidate.metrics.get("fit_scope", {}),
                    "selection_split": candidate.metrics.get("selection_split", {}),
                    "score": candidate.score,
                    "lab_priority_score": candidate.metrics.get("lab_priority_score", candidate.score),
                    "promotion_status": candidate.metrics.get("promotion_status"),
                    "edge_claim": "NO_DEMOSTRADO",
                }
                experiments.append(row)
                existing[experiment_id] = row
                added += 1
            candidate_global_count = int(row.get("global_trial_count_after", global_trial_count) or 0)
```

```

        candidate_updates.append((candidate, experiment_id, trial_count, candidate_global_count))
payload["global_trial_count"] = global_trial_count
run_manifest = self._merge_run(
    payload,
    run_id=run_id,
    now=now,
    candidates=candidates,
    params=params or {},
    added=added,
    previous_registry_hash=previous_registry_hash,
    experiment_ids=[experiment_id for _, experiment_id, _, _ in candidate_updates],
)
payload["summary"] = {
    "run_count": len(payload.get("runs", [])),
    "experiment_count": len(experiments),
    "global_trial_count": global_trial_count,
    "new_experiments": added,
    "latest_run_manifest_hash": run_manifest["run_manifest_hash"],
}
payload["latest_run_manifest"] = run_manifest
payload["registry_hash"] = self.registry_hash(payload)
backup_path = self._backup_existing()
self._atomic_write(payload)
for candidate, experiment_id, trial_count, candidate_global_count in candidate_updates:
    candidate.metrics["global_trial_count"] = candidate_global_count
    candidate.metrics["global_experiment_registry"] = {
        "path": str(self.path),
        "experiment_id": experiment_id,
        "candidate_trial_count": trial_count,
        "global_trial_count": candidate_global_count,
        "edge_claim": "NO_DEMOSTRADO",
        "hash_algorithm": self.hash_algorithm,
        "previous_registry_hash": previous_registry_hash,
        "registry_hash": payload["registry_hash"],
        "run_manifest_hash": run_manifest["run_manifest_hash"],
        "hash_chain_valid": True,
    }
return {
    "path": str(self.path),
    "new_experiments": added,
    "experiment_count": len(experiments),
    "global_trial_count": global_trial_count,
    "hash_algorithm": self.hash_algorithm,
    "previous_registry_hash": previous_registry_hash,
    "registry_hash": payload["registry_hash"],
    "run_manifest_hash": run_manifest["run_manifest_hash"],
    "backup_path": str(backup_path) if backup_path is not None else "",
}

def _merge_run(
    self,
    payload: dict[str, Any],
    *,
    run_id: int | str,
    now: str,
    candidates: list[ClusterCandidate],
    params: dict[str, Any],
    added: int,
    previous_registry_hash: str,
    experiment_ids: list[str],
) -> dict[str, Any]:
    run_manifest = {
        "run_id": run_id,
        "registered_at": now,
        "candidate_count": len(candidates),
        "new_experiments": added,
        "experiment_count": len(payload.get("experiments", [])),
        "global_trial_count": int(payload.get("global_trial_count", 0) or 0),
        "previous_registry_hash": previous_registry_hash,
        "experiment_ids": sorted(experiment_ids),
    }
    run_manifest_hash = self.hash_payload(run_manifest)
    run_manifest["run_manifest_hash"] = run_manifest_hash
    runs = [row for row in payload.setdefault("runs", []) if str(row.get("run_id")) != str(run_id)]
    runs.append(
        {
            "run_id": run_id,
            "registered_at": now,
            "candidate_count": len(candidates),
            "params": params,
            "new_experiments": added,
            "previous_registry_hash": previous_registry_hash,
            "run_manifest_hash": run_manifest_hash,
        }
    )
    payload["runs"] = runs[-500:]
    return run_manifest

@staticmethod
def _candidate_trial_count(candidate: ClusterCandidate) -> int:
    metrics = candidate.metrics
    for key in ("real_variant_count", "multiple_testing_trials"):
        value = metrics.get(key)
        if value is not None:
            return max(1, int(value))
    return 1

```

```

@staticmethod
def _variant_id(candidate: ClusterCandidate) -> str:
    for key in ("variant_id", "variant_key", "registry_candidate_pattern_key"):
        value = candidate.metrics.get(key)
        if value:
            return str(value)
    return candidate.pattern_key

@staticmethod
def _family_id(candidate: ClusterCandidate) -> str:
    for key in ("family_id", "pattern_family_key", "research_family_key"):
        value = candidate.metrics.get(key)
        if value:
            return str(value)
    from tradeo.research.research_memory_graph import ResearchMemoryGraph
    return ResearchMemoryGraph.family_key(candidate)

@staticmethod
def _empty() -> dict[str, Any]:
    return {
        "schema_version": 2,
        "updated_at": "",
        "global_trial_count": 0,
        "hash_algorithm": REGISTRY_HASH_ALGORITHM,
        "registry_hash": "",
        "runs": [],
        "experiments": [],
        "summary": {},
    }

def _backup_existing(self) -> Path | None:
    if not self.path.exists():
        return None
    backup_dir = self.path.parent / ".backups"
    backup_dir.mkdir(parents=True, exist_ok=True)
    timestamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S%fZ")
    backup = backup_dir / f"{self.path.name}.{timestamp}.bak"
    counter = 1
    while backup.exists():
        backup = backup_dir / f"{self.path.name}.{timestamp}.{counter}.bak"
        counter += 1
    shutil.copy2(self.path, backup)
    return backup

def _atomic_write(self, payload: dict[str, Any]) -> None:
    self.path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = self.path.with_name(f"{self.path.name}.{os.getpid()}.tmp")
    try:
        temp_path.write_text(
            json.dumps(self._json_clean(payload), indent=2, sort_keys=True),
            encoding="utf-8",
        )
        os.replace(temp_path, self.path)
    finally:
        if temp_path.exists():
            temp_path.unlink()

def registry_hash(self, payload: dict[str, Any]) -> str:
    clean = self._json_clean(payload)
    if isinstance(clean, dict):
        clean = {key: value for key, value in clean.items() if key != "registry_hash"}
    return self.hash_payload(clean)

@staticmethod
def hash_payload(payload: Any) -> str:
    raw = json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":"), default=str)
    return hashlib.sha256(raw.encode("utf-8")).hexdigest()

@staticmethod
def _json_clean(value: Any) -> Any:
    if isinstance(value, np.generic):
        return value.item()
    if isinstance(value, dict):
        return {str(k): GlobalExperimentRegistry._json_clean(v) for k, v in value.items()}
    if isinstance(value, list):
        return [GlobalExperimentRegistry._json_clean(v) for v in value]
    if isinstance(value, tuple):
        return [GlobalExperimentRegistry._json_clean(v) for v in value]
    return value

```

Full Source: ``backend/tradeo/research/validation_gate.py``

``backend/tradeo/research/validation_gate.py`` ? 415 lines, 22379 bytes, sha256 prefix ``224d81f313073790``

Audit relevance: Pattern validation thresholds. Audit min samples, OOS, adjusted p-values, PF/expectancy, and gate semantics.

Public surface summary before full source:

? class ``ValidationGate`` line 10 methods: `evaluate`, `evaluate_many`

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from dataclasses import dataclass

from tradeo.core.config import Settings, get_settings
from tradeo.research.types import ClusterCandidate

@dataclass(slots=True)
class ValidationGate:
    """Hard statistical gate for unknown patterns.

    A discovered pattern can be exciting, but it is never tradable merely because
    the cluster looks good. This gate rejects curve-fit artifacts early and keeps
    every discovery-stage result below any paper/live promotion state. Paper
    promotion belongs to the Director gate after auditable paper trades, fills,
    costs and clean out-of-sample evidence exist.
    """

    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()

    def evaluate(self, candidate: ClusterCandidate) -> ClusterCandidate:
        s = self.settings
        metrics = candidate.metrics
        reasons: list[str] = []
        warnings: list[str] = []
        rr_metrics = metrics.get("rr_metrics", {})
        best_rr = float(metrics.get("best_rr", 0.0))
        best_expectancy = float(metrics.get("best_expectancy_r", metrics.get("expectancy_r", 0.0)))
        best_pf = float(metrics.get("best_profit_factor", metrics.get("profit_factor", 0.0)))
        best_drawdown = float(metrics.get("best_max_drawdown_r", metrics.get("max_drawdown_r", 0.0)))
        preferred = self._metrics_at_or_above(rr_metrics, s.discovery_candidate_reward_risk)
        premium = self._metrics_at_or_above(rr_metrics, s.discovery_premium_reward_risk)
        minimum = self._metrics_at_or_above(rr_metrics, s.discovery_min_reward_risk)
        preferred_passed = self._quality_pass(preferred, s.discovery_min_expectancy_r, s.discovery_min_profit_factor)
        premium_passed = self._quality_pass(premium, s.discovery_min_expectancy_r, s.discovery_min_profit_factor)
        train_sample_count = int(metrics.get("train_sample_count", candidate.sample_count))

        quant = metrics.get("quant_validation", {})
        if not isinstance(quant, dict):
            quant = {}
        effective_samples = metrics.get("effective_sample_count")
        if candidate.sample_count < s.discovery_min_samples:
            reasons.append(f"muestras insuficientes: {candidate.sample_count} < {s.discovery_min_samples}")
        if effective_samples is not None and float(effective_samples) < s.discovery_min_effective_samples:
            reasons.append(
                f"muestras efectivas insuficientes (solapamiento/pseudo-replicacion): "
                f"n_eff={float(effective_samples):.1f} < {s.discovery_min_effective_samples:g}"
            )
        if metrics.get("fdr_passed") is False:
            reasons.append(
                f"no supera BH-FDR del run (q={float(metrics.get('fdr_q', s.discovery_fdr_q)):g}), "
                f"p={float(metrics.get('null_p_value', 1.0)):4f}, "
                f"tests={int(metrics.get('fdr_test_count', 0) or 0)}"
            )
        if train_sample_count < s.discovery_min_samples:
            reasons.append(f"muestras train insuficientes: {train_sample_count} < {s.discovery_min_samples}")
        if candidate.symbol_count < s.discovery_min_symbols:
            reasons.append(f"poca diversidad de símbolos: {candidate.symbol_count} < {s.discovery_min_symbols}")
        if candidate.year_count < s.discovery_min_years:
            reasons.append(f"poca diversidad temporal: {candidate.year_count} < {s.discovery_min_years}")
        concentration = metrics.get("concentration_checks")
        if isinstance(concentration, dict) and concentration.get("passed") is False:
            reasons.append(
                "concentracion excesiva del cluster: "
                + ", ".join(str(reason) for reason in concentration.get("reasons", [])[0:3])
            )
        if best_expectancy <= 0:
            reasons.append("sin expectancy positiva en ningún R:R evaluado")
        if best_pf <= 1:
            reasons.append("profit factor débil en todos los R:R")
        if best_drawdown > s.discovery_max_drawdown_r:
            reasons.append(f"drawdown excesivo: {best_drawdown:.2f}R > {s.discovery_max_drawdown_r:.2f}R")
        if not minimum or float(minimum.get("expectancy_r", 0.0)) <= 0:
            warnings.append(f"no demuestra edge positivo en {s.discovery_min_reward_risk:g}R")
        if float(metrics.get("profit_factor", best_pf)) < s.discovery_min_profit_factor and not preferred_passed:
            warnings.append(
                f"profit factor insuficiente: {best_pf:.2f} < {s.discovery_min_profit_factor:.2f}"
            )
        if float(metrics.get("expectancy_r", best_expectancy)) < s.discovery_min_expectancy_r and not preferred_passed:
            warnings.append(
                f"expectancy insuficiente: {best_expectancy:.2f}R < {s.discovery_min_expectancy_r:.2f}R"
            )
        if float(metrics.get("stability_score", 0.0)) < s.discovery_min_stability_score:
            reasons.append(
                f"estabilidad insuficiente: {float(metrics.get('stability_score', 0.0)):2f} < {s.discovery_min_stability_score:.2f}"
            )
        adjusted_p = metrics.get("adjusted_p_value")
        if adjusted_p is not None and float(adjusted_p) > s.discovery_max_adjusted_p_value:
            reasons.append(
                f"significancia insuficiente: p_adj={float(adjusted_p):.2f} > {s.discovery_max_adjusted_p_value:.2f}"
            )

```

```

wrc_p = metrics.get("wrc_p_value")
if wrc_p is not None and float(wrc_p) > s.discovery_max_adjusted_p_value:
    wrc_label = (
        "White Reality Check"
        if bool(metrics.get("reality_check_formal_test", False))
        else "bootstrap reality proxy WRC-like"
    )
    reasons.append(
        f"{wrc_label} insuficiente: p={float(wrc_p):.2f} > {s.discovery_max_adjusted_p_value:.2f}"
    )
spa_p = metrics.get("spa_p_value")
if spa_p is not None and float(spa_p) > s.discovery_max_adjusted_p_value:
    spa_label = (
        "SPA formal"
        if bool(metrics.get("reality_check_formal_test", False))
        else "bootstrap reality proxy SPA-like"
    )
    reasons.append(f"{spa_label} insuficiente: p={float(spa_p):.2f} > {s.discovery_max_adjusted_p_value:.2f}")
if metrics.get("statistical_edge_passed") is False:
    lift = float(metrics.get("expectancy_lift_r", 0.0))
    warnings.append(f"edge vs baseline débil: lift={lift:.2f}R")
expectancy_ci_low = metrics.get("expectancy_ci_low")
if expectancy_ci_low is not None and float(expectancy_ci_low) < s.discovery_min_expectancy_ci_low:
    reasons.append(
        f"intervalo bootstrap débil: ci_low={float(expectancy_ci_low):.2f}R < {s.discovery_min_expectancy_ci_low:.2f}R"
    )
overfit_score = metrics.get("overfit_score")
if overfit_score is not None and float(overfit_score) > s.discovery_max_overfit_score:
    reasons.append(f"riesgo de overfit alto: {float(overfit_score):.2f} > {s.discovery_max_overfit_score:.2f}")
purged_fold_count = int(metrics.get("purged_cv_fold_count", 0))
if purged_fold_count:
    purged_rate = float(metrics.get("purged_cv_positive_rate", 0.0))
    if purged_rate < s.discovery_min_walk_forward_positive_rate:
        reasons.append(
            "purged combinatorial CV inestable: "
            f"{purged_rate:.2f} < {s.discovery_min_walk_forward_positive_rate:.2f}"
        )
deflated_psr = metrics.get("deflated_sharpe_probability")
if deflated_psr is not None and float(deflated_psr) < 0.50:
    reasons.append(f"Deflated Sharpe débil: prob={float(deflated_psr):.2f} < 0.50")
probabilistic_sharpe = metrics.get("probabilistic_sharpe")
if probabilistic_sharpe is not None and float(probabilistic_sharpe) < 0.55:
    warnings.append(f"Probabilistic Sharpe bajo: {float(probabilistic_sharpe):.2f}")
if metrics.get("edge_decay_passed") is False:
    reasons.append("edge decae demasiado ante cambios cercanos de R:R")
if metrics.get("cost_stress_passed") is False:
    reasons.append(f"edge no sobrevive coste x{s.discovery_required_cost_stress_multiplier:g}")
replay = metrics.get("market_replay", {})
if isinstance(replay, dict) and replay:
    replay_expectancy = float(replay.get("expected_expectancy_r", 0.0))
    replay_fill_ratio = float(replay.get("avg_fill_ratio", 0.0))
    if replay_expectancy <= 0:
        reasons.append(f"market replay sin expectancy positiva: {replay_expectancy:.2f}R")
    if replay_fill_ratio < 0.25:
        reasons.append(f"market replay fill ratio demasiado bajo: {replay_fill_ratio:.2f}")
    elif replay_fill_ratio < 0.45:
        warnings.append(f"market replay fill ratio débil: {replay_fill_ratio:.2f}")
    for warning in replay.get("warnings", []):3:
        warnings.append(str(warning))
causal = metrics.get("causal_invariance", {})
if isinstance(causal, dict) and causal:
    invariance_score = float(causal.get("invariance_score", 0.0))
    symbol_dependency = causal.get("symbol_dependency", {})
    no_three_symbols = (
        bool(symbol_dependency.get("no_three_symbol_dependency", False))
        if isinstance(symbol_dependency, dict)
        else True
    )
    if not no_three_symbols:
        reasons.append("dependencia de <=3 simbolos/eventos en invariancia causal")
    if invariance_score < 0.25:
        reasons.append(f"invariancia causal muy débil: {invariance_score:.2f}")
    elif invariance_score < 0.45:
        warnings.append(f"invariancia causal débil: {invariance_score:.2f}")
    expected_fail_count = len(causal.get("expected_fail_buckets", []))
    if expected_fail_count:
        warnings.append(f"{expected_fail_count} buckets expected-fail detectados")
adversarial = metrics.get("adversarial_challenge", {})
if isinstance(adversarial, dict) and adversarial:
    challenge_score = float(adversarial.get("challenge_score", 0.0))
    for reason in adversarial.get("rejection_reasons", []):5:
        reasons.append(f"adversarial rejection: {reason}")
    if challenge_score < 0.45:
        reasons.append(f"challenge score bajo: {challenge_score:.2f} < 0.45")
    elif challenge_score < 0.55:
        warnings.append(f"challenge score liminal: {challenge_score:.2f} < 0.55")
    for warning in adversarial.get("warnings", []):5:
        warnings.append(str(warning))
teacher = metrics.get("foundation_teacher", {})
if isinstance(teacher, dict):
    digest = teacher.get("pretraining_digest", {})
    teacher_score = (
        float(digest.get("embedding_quality_score", 0.0))
        if isinstance(digest, dict)
        else 0.0
    )
)

```



```

        if teacher_score and teacher_score < 0.30:
            warnings.append(f"teacher embedding quality debil: {teacher_score:.2f}")
    if "avg_fill_probability" in metrics and float(metrics.get("avg_fill_probability", 1.0)) < 0.35:
        reasons.append(
            "baja probabilidad de fill estimada: "
            f"{float(metrics.get('avg_fill_probability', 0.0)):.2f}"
        )
    if (
        "p25_max_size_usd" in metrics
        and 0.0 < float(metrics.get("p25_max_size_usd", 0.0)) < s.account_risk_usd * 5
    ):
        warnings.append("tamano operable bajo para el patron; limitar size o exigir mas liquidez")
    fold_count = int(metrics.get("walk_forward_fold_count", 0))
    if fold_count:
        fold_rate = float(metrics.get("walk_forward_positive_fold_rate", 0.0))
        if fold_rate < s.discovery_min_walk_forward_positive_rate:
            reasons.append(
                f"walk-forward inestable: {fold_rate:.2f} < {s.discovery_min_walk_forward_positive_rate:.2f}"
            )
    else:
        warnings.append("walk-forward sin folds suficientes")
    oos_positive = float(metrics.get("out_of_sample_expectancy_r", 0.0)) > 0
    if not oos_positive:
        warnings.append("out-of-sample expectancy no positiva")
    if float(metrics.get("out_of_sample_profit_factor", 0.0)) < 1.2:
        warnings.append("out-of-sample profit factor demasiado débil")

    hit_rate = float(metrics.get("hit_4r_rate", 0.0))
    if hit_rate < 0.08:
        warnings.append("tasa de 4R baja; puede requerir filtro posterior de entrada")
    operational = metrics.get("operational_trigger", {})
    trigger_rate = float(operational.get("trigger_rate", 0.0)) if isinstance(operational, dict) else 0.0
    if trigger_rate < 0.25:
        warnings.append("baja tasa de trigger operativo; requiere filtro de entrada estricto")
    if candidate.symbol_count <= 3:
        warnings.append("posible dependencia de pocos símbolos")
    if candidate.year_count <= 1:
        warnings.append("posible dependencia de un solo régimen temporal")
    if premium_passed:
        warnings.append(
            "premium/paper promotion bloqueada en discovery: requiere Director gate con trades, fills, costes y OOS limpio"
        )
    confirmation = self._confirmation_candidate(
        candidate=candidate,
        hard_reasons=reasons,
        best_expectancy=best_expectancy,
        best_pf=best_pf,
        best_drawdown=best_drawdown,
        oos_positive=oos_positive,
        train_sample_count=train_sample_count,
    )
    if confirmation["recommended"]:
        warnings.append(str(confirmation["reason"]))

    promotion_status = "rejected"
    if not reasons:
        # Discovery can only create lab-stage states. It must never promote to
        # paper/live from simulated research R metrics, even when those metrics
        # look strong. The Director gate owns all execution-stage promotion.
        if (
            preferred_passed
            and oos_positive
            and float(metrics.get("stability_score", 0.0)) >= s.discovery_min_stability_score
        ):
            promotion_status = "lab_candidate"
        elif preferred_passed or (minimum and float(minimum.get("expectancy_r", 0.0)) > 0):
            promotion_status = "lab_watchlist"
        elif best_expectancy > 0:
            promotion_status = "lab"

    if promotion_status == "lab_candidate":
        # lab_candidate is the highest discovery state, so it additionally
        # requires the family-deflated Sharpe and a strictly positive
        # stationary-bootstrap CI on the deduplicated, uniqueness-weighted
        # expectancy. Failures downgrade instead of rejecting: the evidence
        # is weak, not contradicted.
        dsr_family = metrics.get("dsr_family")
        if dsr_family is None:
            promotion_status = "lab_watchlist"
            warnings.append(
                "lab_candidate bloqueado: DSR de familia no calculable (faltan trials o sr_std del ledger)"
            )
        elif float(dsr_family) < s.discovery_min_dsr:
            promotion_status = "lab_watchlist"
            warnings.append(
                f"lab_candidate bloqueado: DSR familia {float(dsr_family):.3f} < {s.discovery_min_dsr:g} "
                f"(N_trials={int(metrics.get('dsr_family_n_trials', 0) or 0)})"
            )
        stationary_ci_low = quant.get("expectancy_ci95_low")
        if stationary_ci_low is not None and float(stationary_ci_low) <= 0.0:
            promotion_status = "lab_watchlist"
            warnings.append(
                "lab_candidate bloqueado: IC95 stationary-bootstrap del expectancy ponderado "
                f"no positivo (low={float(stationary_ci_low):.3f}R)"
            )

```

```

survivorship_biased = metrics.get("survivorship_biased")
universe_pit = metrics.get("universe_point_in_time")
if promotion_status == "lab_candidate" and (
    survivorship_biased is True or universe_pit is False
):
    cap_state = str(s.survivorship_cap_state or "lab_watchlist")
    if cap_state not in {"lab", "lab_watchlist", "rejected"}:
        cap_state = "lab_watchlist"
    promotion_status = cap_state
    metrics["survivorship_cap_applied"] = True
    warnings.append(
        "lab_candidate bloqueado: universo historico no point-in-time; "
        f"cap aplicado a {promotion_status}"
    )
else:
    metrics["survivorship_cap_applied"] = False

candidate.validation_passed = promotion_status != "rejected"
candidate.validation_reasons = warnings if candidate.validation_passed else reasons + warnings
candidate.metrics["validation_warnings"] = warnings
candidate.metrics["validation_rejections"] = reasons
candidate.metrics["validation_passed"] = candidate.validation_passed
candidate.metrics["preferred_rr_passed"] = preferred_passed
candidate.metrics["premium_rr_passed"] = premium_passed
candidate.metrics["execution_promotion_blocked"] = (
    premium_passed or promotion_status in {"lab_candidate", "lab_watchlist", "lab"}
)
candidate.metrics["promotion_status"] = promotion_status
candidate.metrics["promotion_reason"] = self._promotion_reason(
    promotion_status,
    best_rr,
    best_expectancy,
    best_pf,
    oos_positive,
)
candidate.metrics["confirmation_recommended"] = bool(confirmation["recommended"])
candidate.metrics["confirmation_status"] = "needs_confirmation" if confirmation["recommended"] else ""
candidate.metrics["confirmation_reason"] = str(confirmation["reason"])
candidate.metrics["confirmation_priority_score"] = float(confirmation["priority_score"])
candidate.metrics["confirmation_next_action"] = str(confirmation["next_action"])
candidate.metrics["rejection_reasons"] = reasons
return candidate

def evaluate_many(self, candidates: list[ClusterCandidate]) -> list[ClusterCandidate]:
    return [self.evaluate(candidate) for candidate in candidates]

@staticmethod
def _metrics_at_or_above(rr_metrics: object, rr: float) -> dict[str, object] | None:
    if not isinstance(rr_metrics, dict):
        return None
    matches = []
    for key, value in rr_metrics.items():
        try:
            level = float(key)
        except (TypeError, ValueError):
            continue
        if level >= rr and isinstance(value, dict):
            matches.append(value)
    if not matches:
        return None
    return max(matches, key=lambda m: float(m.get("expectancy_r", 0.0)))

@staticmethod
def _quality_pass(metrics: dict[str, object] | None, min_expectancy: float, min_profit_factor: float) -> bool:
    if not metrics:
        return False
    return float(metrics.get("expectancy_r", 0.0)) >= min_expectancy and float(metrics.get("profit_factor", 0.0)) >=
min_profit_factor

def _confirmation_candidate(
    self,
    *,
    candidate: ClusterCandidate,
    hard_reasons: list[str],
    best_expectancy: float,
    best_pf: float,
    best_drawdown: float,
    oos_positive: bool,
    train_sample_count: int,
) -> dict[str, object]:
    s = self.settings
    assert s is not None
    sample_limited = any(reason.startswith("muestras") for reason in hard_reasons)
    if not sample_limited:
        return self._no_confirmation()
    adjusted_p = candidate.metrics.get("adjusted_p_value")
    stats_ok = adjusted_p is None or float(adjusted_p) <= s.discovery_max_adjusted_p_value
    enough_seed_samples = candidate.sample_count >= s.discovery_confirmation_min_samples
    enough_train_samples = train_sample_count >= max(20, s.discovery_confirmation_min_samples // 2)
    quality_ok = (
        best_expectancy >= s.discovery_confirmation_min_expectancy_r
        and best_pf >= s.discovery_confirmation_min_profit_factor
        and best_drawdown <= s.discovery_max_drawdown_r
        and oos_positive
        and stats_ok
    )

```

```

if not (enough_seed_samples and enough_train_samples and quality_ok):
    return self._no_confirmation()
priority = (
    max(0.0, best_expectancy) * 0.45
    + min(best_pf / 4.0, 1.0) * 0.25
    + max(0.0, float(candidate.metrics.get("expectancy_lift_r", 0.0))) * 0.20
    + min(candidate.sample_count / max(s.discovery_min_samples, 1), 1.0) * 0.10
)
return {
    "recommended": True,
    "reason": (
        "requiere confirmación ampliada: edge fuerte pero muestra train/total insuficiente "
        f"({train_sample_count}/{candidate.sample_count})"
    ),
    "priority_score": round(float(priority), 5),
    "next_action": "rerun con universo/periodo ampliado y mismos window_size/side antes de promocionar",
}

@staticmethod
def _no_confirmation() -> dict[str, object]:
    return {
        "recommended": False,
        "reason": "",
        "priority_score": 0.0,
        "next_action": "",
    }

@staticmethod
def _promotion_reason(status: str, best_rr: float, expectancy: float, profit_factor: float, oos_positive: bool) -> str:
    if status == "rejected":
        return "rechazado por filtros estadísticos duros"
    oos = "OOS positivo" if oos_positive else "OOS pendiente/debil"
    return f"{status}: best_rr={best_rr:g}, expectancy={expectancy:.2f}R, PF={profit_factor:.2f}, {oos}; paper/live requiere Director gate"

```

Full Source: `backend/tradeo/research/quant\_validation.py`

`backend/tradeo/research/quant\_validation.py` ? 798 lines, 30852 bytes, sha256 prefix `86a233012983bbec`

Audit relevance: Quant/stat validation core. Audit purged walk-forward, CSCV/PBO, DSR, WRC/SPA approximations, and multiple-testing ledger.

Public surface summary before full source:

```

? function `select_nonoverlapping_events` line 83`
? function `average_uniqueness_weights` line 118`
? function `triple_barrier_outcome` line 159`
? function `tiered_round_trip_cost_r` line 303`
? function `stationary_bootstrap_ci` line 341`
? function `newey_west_tstat` line 382`
? function `profit_factor` line 411`
? function `permutation_pvalue` line 430`
? function `benjamini_hochberg` line 459`
? function `expected_max_sharpe` line 485`
? function `probabilistic_sharpe_ratio` line 502`
? function `deflated_sharpe_ratio` line 521`
? function `sample_skew_kurt` line 534`
? function `purged_walk_forward` line 554`
? function `pbo_cscv` line 587`
? function `cusum_drift` line 652`

```

Risk hints: RNG path; verify seed/control; TODO/FIXME present.

quant_validation.py ? Núcleo de validación estadística para Tradeo Research/Lab.

Destino sugerido: backend/tradeo/research/quant_validation.py

Dependencias: numpy + biblioteca estándar (statistics.NormalDist). Sin scipy.

Implementa el pipeline de inferencia selectiva descrito en
INFORME_MEJORA_TRADEO.md, sección 3.6:

```

dedup -> n_eff -> outcome neto -> bootstrap/Newey-West -> permutación
      -> BH-FDR -> Deflated Sharpe -> walk-forward purgado -> CUSUM

```

Uso típico (por candidato de cluster, tras deduplicar POR SÍMBOLO):

```

from quant_validation import (
    select_nonoverlapping_events, triple_barrier_outcome,
    summarize_pattern_validation, benjamini_hochberg,
)

# 1) por símbolo: dedup de ocurrencias solapadas
keep = select_nonoverlapping_events(starts_sym, ends_sym)

# 2) por ocurrencia: outcome con fills pesimistas

```

```

out = triple_barrier_outcome(o, h, l, c, signal_index=i, side=+1,
                             stop_price=sl, target_price=tp, max_bars=10)

# 3) resumen de validación del candidato
rep = summarize_pattern_validation(r, starts, ends, horizon_bars=10,
                                   n_trials_family=ledger.n_trials,
                                   sr_std_across_trials=ledger.sr_std,
                                   null_means=null_baseline_means)

# 4) a nivel de run: FDR sobre los p-valores de TODOS los clusters
rechazo_h0, corte = benjamini_hochberg(p_values_del_run, q=0.05)

```

Referencias:

```

- López de Prado, "Advances in Financial Machine Learning" (2018):
  unicidad media (cap. 4), purged k-fold y embargo (cap. 7).
- Bailey & López de Prado (2012, 2014): Probabilistic / Deflated Sharpe Ratio.
- Bailey, Borwein, López de Prado, Zhu (2017): CSCV / PBO.
- Politis & Romano (1994): stationary bootstrap.
"""

```

```

from __future__ import annotations

```

```

import itertools
import math
from statistics import NormalDist
from typing import Callable, Iterator, Optional, Sequence

```

```

import numpy as np

```

```

_PHI = NormalDist().cdf # ?
_PHI_INV = NormalDist().inv_cdf # ??¹
_EULER_GAMMA = 0.5772156649015329

```

```

__all__ = [
    "select_nonoverlapping_events",
    "average_uniqueness_weights",
    "triple_barrier_outcome",
    "tiered_round_trip_cost_r",
    "stationary_bootstrap_ci",
    "newey_west_tstat",
    "profit_factor",
    "permutation_pvalue",
    "benjamini_hochberg",
    "expected_max_sharpe",
    "probabilistic_sharpe_ratio",
    "deflated_sharpe_ratio",
    "sample_skew_kurt",
    "purged_walk_forward",
    "pbo_cscv",
    "cusum_drift",
    "summarize_pattern_validation",
]

```

```

# -----
# 1. Deduplicación y tamaño muestral efectivo (P0-1 del informe)
# -----

```

```

def select_nonoverlapping_events(
    starts: Sequence[int], ends: Sequence[int]
) -> np.ndarray:
    """Selección secuencial de eventos no solapados. APLICAR POR SÍMBOLO.

```

Ordena por inicio y conserva un evento solo si su inicio es estrictamente posterior al final (inclusivo) del último conservado. Refleja la realidad operativa: no puedes estar en dos trades solapados del mismo patrón en el mismo símbolo.

Parámetros

starts, ends : índices de barra de cada evento (end inclusivo).

Devuelve

np.ndarray de índices ORIGINALES de los eventos conservados.

Nota: los descartados no se tiran; consévalos como 'shadow occurrences' para diagnósticos (MFE/MAE), sin que entren en las métricas del gate.

```

"""
starts = np.asarray(starts)
ends = np.asarray(ends)
if starts.size == 0:
    return np.asarray([], dtype=int)
order = np.argsort(starts, kind="stable")
kept: list[int] = []
last_end = -np.inf
for i in order:
    if starts[i] > last_end:
        kept.append(int(i))
        last_end = ends[i]
return np.asarray(kept, dtype=int)

```

```

def average_uniqueness_weights(
    starts: Sequence[int], ends: Sequence[int]
) -> tuple[np.ndarray, float]:

```

```

"""Pesos de unicidad media (López de Prado, AFML cap. 4) y n_eff.

Para cada barra t, c_t = n° de eventos cuyo span de outcome cubre t.
Unicidad del evento i: u_i = media_{t en [start_i, end_i]} (1 / c_t).
n_eff = ? u_i es el tamaño muestral efectivo: corrige la
pseudo-replicación por solapamiento de outcomes (incluso ENTRE símbolos,
si los spans se expresan en tiempo de calendario común).

Devuelve
-----
(weights, n_eff) con weights[i] en (0, 1].
Todos los gates de muestra del informe operan sobre n_eff.
"""
starts = np.asarray(starts, dtype=int)
ends = np.asarray(ends, dtype=int)
if starts.size == 0:
    return np.asarray([], dtype=float), 0.0
if np.any(ends < starts):
    raise ValueError("Todos los spans deben cumplir end >= start.")
t0 = int(starts.min())
t1 = int(ends.max())
# Concurrencia c_t vía difference array: O(n + rango)
diff = np.zeros(t1 - t0 + 2, dtype=float)
np.add.at(diff, starts - t0, 1.0)
np.add.at(diff, ends - t0 + 1, -1.0)
conc = np.cumsum(diff)[:t1 - t0] # c_t para t en [t0, t1]
inv = 1.0 / np.maximum(conc, 1.0) # 1/c_t (c_t >= 1 donde hay eventos)
weights = np.empty(starts.size, dtype=float)
for i in range(starts.size):
    seg = inv[starts[i] - t0: ends[i] - t0 + 1]
    weights[i] = float(seg.mean())
return weights, float(weights.sum())

# -----
# 2. Etiquetador triple-barrera con fills pesimistas (sección 3.5)
# -----

def triple_barrier_outcome(
    open_: Sequence[float],
    high: Sequence[float],
    low: Sequence[float],
    close: Sequence[float],
    signal_index: int,
    side: int,
    stop_price: float,
    target_price: float,
    maxBars: int,
    entry_price: Optional[float] = None,
    gap_entry_policy: str = "skip",
    conservative_both: bool = True,
    round_trip_cost_R: float = 0.0,
) -> dict:
    """Outcome de un evento con las reglas de fill cerradas del informe §3.5.

    Reglas
    -----
    1. Señal en el CIERRE de la barra `signal_index` => entrada en
       open[signal_index + 1] (salvo `entry_price` explícito).
    2. Gap adverso en la barra de entrada (open ya a través del stop):
       'skip' -> la señal se invalida (status='skipped',
          reason='gapped_through_stop'): near-miss auditable.
       'enter' -> se asume fill al open y stop inmediato al open; R se mide
          contra el riesgo teórico desde el cierre de señal.
       Gap a través del TARGET en la entrada -> siempre 'skip'
          (entrar ahí es perseguir; reason='gapped_past_target').
    3. En cada barra posterior, orden de comprobación:
       a) open a través del stop -> fill al OPEN (peor que el stop);
       b) open a través del target -> fill al TARGET (una limit no cobra
          el regalo del gap);
       c) intrabar toca stop Y target -> STOP si conservative_both
          (sin datos intrabar, la única suposición defendible);
       d) solo stop -> stop_price ; solo target -> target_price.
    4. Sin salida en `maxBars` -> salida al CLOSE de la última barra
       (reason='time').

    `side`: +1 largo (stop < entrada < target), -1 corto (target < entrada
    < stop). El múltiplo R se mide contra el riesgo REAL por acción
    |entrada - stop| (convención de sizing en vivo).

    `round_trip_cost_R`: coste ida+vuelta ya expresado en R (spread/2 +
    slippage + comisión, ambos lados, dividido por el riesgo por acción).
    Se resta del R bruto. Mantener 0.0 si los costes se aplican aguas abajo.

    Devuelve
    -----
    dict con: status ('ok' | 'skipped' | 'invalid' | 'no_data'), R, reason,
    entry_index, exit_index, entry_price, exit_price, bars_held, mfe_R, mae_R.
    MFE/MAE se calculan sobre barras completas (aprox.: la barra de salida
    puede incluir excursión posterior al fill).
    """
    o = np.asarray(open_, dtype=float)
    h = np.asarray(high, dtype=float)
    lo_arr = np.asarray(low, dtype=float)
    c = np.asarray(close, dtype=float)
    n = o.size

```

```

base = {
    "status": "ok", "R": float("nan"), "reason": "",
    "entry_index": None, "exit_index": None,
    "entry_price": float("nan"), "exit_price": float("nan"),
    "bars_held": 0, "mfe_R": float("nan"), "mae_R": float("nan"),
}
if side not in (+1, -1):
    return {**base, "status": "invalid", "reason": "side_must_be_pm1"}
if max_bars <= 0:
    return {**base, "status": "invalid", "reason": "max_bars_le_0"}
i0 = signal_index + 1
if i0 >= n:
    return {**base, "status": "no_data", "reason": "no_next_bar"}

entry = float(entry_price) if entry_price is not None else float(o[i0])

# --- Gaps en la barra de entrada -----
if side * (entry - target_price) >= 0:
    return {**base, "status": "skipped", "reason": "gapped_past_target",
            "entry_index": i0}
if side * (entry - stop_price) <= 0:
    if gap_entry_policy == "skip":
        return {**base, "status": "skipped",
                "reason": "gapped_through_stop", "entry_index": i0}
    # 'enter': fill al open y salida inmediata al open; R contra el
    # riesgo teórico desde el cierre de la señal.
    ref_risk = side * (float(c[signal_index]) - stop_price)
    r_val = (side * (entry - float(c[signal_index])) / ref_risk
            if ref_risk > 0 else float("nan"))
    return {**base, "R": r_val - round_trip_cost_R,
            "reason": "stop_gap_entry", "entry_index": i0,
            "exit_index": i0, "entry_price": entry, "exit_price": entry,
            "bars_held": 1, "mfe_R": 0.0, "mae_R": min(0.0, r_val)}

risk = side * (entry - stop_price) # > 0 garantizado por los checks
last = min(i0 + max_bars - 1, n - 1)

mfe = -np.inf
mae = np.inf
exit_price = float("nan")
exit_index = last
reason = "time"

for i in range(i0, last + 1):
    # excursiones en R sobre la barra completa
    e1 = side * (float(h[i]) - entry) / risk
    e2 = side * (float(lo_arr[i]) - entry) / risk
    mfe = max(mfe, e1, e2)
    mae = min(mae, e1, e2)

    if i > i0:
        oi = float(o[i])
        if side * (oi - stop_price) <= 0: # gap por el stop
            exit_price, exit_index, reason = oi, i, "stop_gap"
            break
        if side * (oi - target_price) >= 0: # gap por el target
            exit_price, exit_index, reason = float(target_price), i, "target_gap"
            break

    hit_stop = (float(lo_arr[i]) <= stop_price) if side > 0 else (float(h[i]) >= stop_price)
    hit_tgt = (float(h[i]) >= target_price) if side > 0 else (float(lo_arr[i]) <= target_price)

    if hit_stop and hit_tgt:
        if conservative_both:
            exit_price, exit_index, reason = float(stop_price), i, "stop_and_target_conservative"
        else:
            exit_price, exit_index, reason = float(target_price), i, "target"
        break
    if hit_stop:
        exit_price, exit_index, reason = float(stop_price), i, "stop"
        break
    if hit_tgt:
        exit_price, exit_index, reason = float(target_price), i, "target"
        break

if reason == "time":
    exit_price = float(c[last])

r_val = side * (exit_price - entry) / risk - round_trip_cost_R
return {**base, "R": float(r_val), "reason": reason,
        "entry_index": i0, "exit_index": int(exit_index),
        "entry_price": entry, "exit_price": float(exit_price),
        "bars_held": int(exit_index - i0 + 1),
        "mfe_R": float(mfe), "mae_R": float(mae)}

def tiered_round_trip_cost_r(
    *,
    entry_price: float,
    risk_per_share: float,
    avg_dollar_volume: float = 0.0,
    participation_rate: float = 0.005,
    multiplier: float = 1.0,
    commission_bps: float = 0.5,
) -> float:
    """Tiered spread/slippage cost model expressed in R.

```

```

The model is intentionally simple and deterministic so Research,
backtests and shadow simulations can share the same conservative cost
convention. Half-spread is applied per side, slippage grows with
participation, and the returned value is round-trip R.
"""
entry = max(float(entry_price), 1e-9)
risk = max(float(risk_per_share), 1e-9)
adv = max(float(avg_dollar_volume), 0.0)
if adv >= 25_000_000:
    half_spread_bps = 3.0
elif adv >= 5_000_000:
    half_spread_bps = 8.0
elif adv >= 1_000_000:
    half_spread_bps = 20.0
else:
    half_spread_bps = 45.0
participation_pct = max(0.0, float(participation_rate)) * 100.0
slippage_bps = 10.0 * participation_pct
per_side_bps = half_spread_bps + slippage_bps + max(0.0, float(commission_bps))
round_trip_pct = 2.0 * per_side_bps / 10_000.0
return float(entry * round_trip_pct / risk * max(0.0, float(multiplier)))

# -----
# 3. Incertidumbre honesta: bootstrap por bloques y Newey-West ($3.6 paso 4)
# -----

def stationary_bootstrap_ci(
    x: Sequence[float],
    stat_fn: Callable[[np.ndarray], float] = np.mean,
    n_boot: int = 2000,
    mean_block: Optional[int] = None,
    alpha: float = 0.05,
    rng=None,
) -> tuple[float, float, float, np.ndarray]:
    """IC por stationary bootstrap (Politis-Romano 1994), muestreo circular.

    Bloques de longitud geométrica con media `mean_block`. Respeta la
    autocorrelación de outcomes solapados. Recomendado:
    mean_block = forward_bars del patrón.

    Devuelve (lo, hi, estadístico_puntual, distribución_bootstrap).
    Para Tradeo: stat_fn = np.mean(expectancy) o profit_factor.
    Gate del informe: lo > 0 para el expectancy neto.
    """
    x = np.asarray(x, dtype=float)
    n = x.size
    if n < 3:
        return float("nan"), float("nan"), float("nan"), np.asarray([])
    gen = np.random.default_rng(rng)
    if mean_block is None:
        mean_block = max(5, int(round(n ** (1.0 / 3.0))))
    p = 1.0 / max(1, int(mean_block))
    stats = np.empty(n_boot, dtype=float)
    for b in range(n_boot):
        idx = np.empty(n, dtype=int)
        idx[0] = gen.integers(n)
        restart = gen.random(n) < p
        for t in range(1, n):
            idx[t] = gen.integers(n) if restart[t] else (idx[t - 1] + 1) % n
        stats[b] = float(stat_fn(x[idx]))
    finite = stats[np.isfinite(stats)]
    if finite.size == 0:
        return float("nan"), float("nan"), float(stat_fn(x)), stats
    lo, hi = np.quantile(finite, [alpha / 2.0, 1.0 - alpha / 2.0])
    return float(lo), float(hi), float(stat_fn(x)), stats

def newey_west_tstat(
    x: Sequence[float], lags: Optional[int] = None
) -> tuple[float, float]:
    """t-stat de la media con varianza HAC Newey-West (kernel de Bartlett).

    Robusto a la autocorrelación residual de outcomes solapados.
    Recomendado: lags = forward_bars. Si None, regla 4*(n/100)^(2/9).

    Devuelve (t, error_estándar_de_la_media).
    """
    x = np.asarray(x, dtype=float)
    n = x.size
    if n < 3:
        return float("nan"), float("nan")
    if lags is None:
        lags = int(math.floor(4.0 * (n / 100.0) ** (2.0 / 9.0)))
    lags = max(0, min(int(lags), n - 1))
    d = x - x.mean()
    s = float(d @ d) / n # gamma_0
    for lag in range(1, lags + 1):
        w = 1.0 - lag / (lags + 1.0)
        s += 2.0 * w * (float(d[lag:] @ d[:-lag]) / n)
    var_mean = s / n
    if var_mean <= 0:
        return float("nan"), float("nan")
    se = math.sqrt(var_mean)
    return float(x.mean() / se), float(se)

```

```

def profit_factor(
    r: Sequence[float], weights: Optional[Sequence[float]] = None
) -> float:
    """Profit factor (? ganancias / |? pérdidas|), opcionalmente ponderado
    por los pesos de unicidad. inf si no hay pérdidas; nan si no hay trades
    con signo."""
    r = np.asarray(r, dtype=float)
    w = np.ones_like(r) if weights is None else np.asarray(weights, dtype=float)
    wins = float(np.sum(w[r > 0] * r[r > 0]))
    losses = float(-np.sum(w[r < 0] * r[r < 0]))
    if losses == 0.0:
        return float("inf") if wins > 0 else float("nan")
    return wins / losses

# -----
# 4. Inferencia selectiva: permutación, FDR, Deflated Sharpe (§3.6 pasos 5-7)
# -----

def permutation_pvalue(
    observed: float,
    null_stats: Sequence[float],
    alternative: str = "greater",
) -> float:
    """p-valor de permutación contra los null baselines emparejados.

    `null_stats`: estadísticos (p.ej. expectancy) de K carteras nulas con
    mismas fechas+jitter, mismos símbolos/buckets, mismo etiquetador y
    mismos costes. Fórmula con corrección +1 (Phipson & Smyth):
    
$$p = (1 + \#\{\text{nulo} \geq \text{real}\}) / (K + 1)$$

    """
    null = np.asarray(null_stats, dtype=float)
    null = null[np.isfinite(null)]
    k = null.size
    if k == 0:
        return float("nan")
    if alternative == "greater":
        cnt = int(np.sum(null >= observed))
    elif alternative == "less":
        cnt = int(np.sum(null <= observed))
    elif alternative == "two-sided":
        center = float(np.median(null))
        cnt = int(np.sum(np.abs(null - center) >= abs(observed - center)))
    else:
        raise ValueError("alternative debe ser 'greater', 'less' o 'two-sided'")
    return (1.0 + cnt) / (k + 1.0)

def benjamini_hochberg(
    pvals: Sequence[float], q: float = 0.05
) -> tuple[np.ndarray, float]:
    """Control de la tasa de falsos descubrimientos (BH, 1995).

    Aplicar a nivel de RUN sobre los p-valores de TODOS los clusters
    evaluados (incluidos los rechazados: para eso está STORE_REJECTED).

    Devuelve (máscara_de_rechazo_de_H0_en_orden_original, p_corte).
    máscara[i]=True => el cluster i sobrevive al FDR q.
    """
    p = np.asarray(pvals, dtype=float)
    m = p.size
    if m == 0:
        return np.asarray([], dtype=bool), float("nan")
    order = np.argsort(p)
    ranked = p[order]
    thresh = q * np.arange(1, m + 1) / m
    below = ranked <= thresh
    if not below.any():
        return np.zeros(m, dtype=bool), float("nan")
    k = int(np.max(np.nonzero(below)[0]))
    cutoff = float(ranked[k])
    return p <= cutoff, cutoff

def expected_max_sharpe(n_trials: int, sr_std_across_trials: float) -> float:
    """E[max SR] bajo el nulo tras N pruebas (Bailey-López de Prado 2014):

    
$$E[\max] \approx \sqrt{SR} \cdot \left[ (1-\alpha) \cdot \frac{1}{\sqrt{N}} + \alpha \cdot \frac{1}{\sqrt{N \cdot e}} \right]$$


    ?_SR: desviación estándar de los SR (por-trade) ENTRE las hipótesis
    probadas de la familia ? estimada del ledger (np.std de los SR de todos
    los clusters evaluados, aprobados y rechazados). ? = Euler-Mascheroni.
    """
    n = int(n_trials)
    if n <= 1 or not (sr_std_across_trials > 0):
        return 0.0
    z1 = _PHI_INV(1.0 - 1.0 / n)
    z2 = _PHI_INV(1.0 - 1.0 / (n * math.e))
    return float(sr_std_across_trials * ((1.0 - _EULER_GAMMA) * z1 + _EULER_GAMMA * z2))

def probabilistic_sharpe_ratio(
    sr: float, sr_benchmark: float, n: float,
    skew: float = 0.0, kurt: float = 3.0,

```



```

) -> float:
    """PSR (Bailey-López de Prado 2012):  $P(SR_{verdadero} > sr_{benchmark})$ .

    
$$PSR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(SR - SR^*) \cdot \frac{1}{\sqrt{1 - skew \cdot SR + (kurt-1)/4 \cdot SR^2}}$$


    `n`: usa n_eff, no el conteo bruto. `kurt` es curtosis SIN excesos
    (normal = 3). `sr`: Sharpe por-trade (media/sd de los R).
    """
    if not (n > 1):
        return float("nan")
    denom_sq = 1.0 - skew * sr + (kurt - 1.0) / 4.0 * sr * sr
    if denom_sq <= 0:
        return float("nan")
    return float(_PHI((sr - sr_benchmark) * math.sqrt(n - 1.0) / math.sqrt(denom_sq)))

```

```

def deflated_sharpe_ratio(
    sr: float, n: float, skew: float, kurt: float,
    n_trials: int, sr_std_across_trials: float,
) -> tuple[float, float]:
    """Deflated Sharpe Ratio: PSR contra el máximo SR esperable por azar
    dadas N pruebas. Gate del informe: DSR >= 0.95 para `lab_candidate`.

    Devuelve (dsr, sr_star). El umbral sube solo con N_trials: minar más
    deja de ser gratis."""
    sr_star = expected_max_sharpe(n_trials, sr_std_across_trials)
    return probabilistic_sharpe_ratio(sr, sr_star, n, skew, kurt), sr_star

```

```

def sample_skew_kurt(r: Sequence[float]) -> tuple[float, float]:
    """Asimetría y curtosis muestrales (curtosis SIN excesos; normal = 3).
    Estimadores de momentos sesgados ? suficientes para PSR/DSR."""
    r = np.asarray(r, dtype=float)
    n = r.size
    if n < 3:
        return 0.0, 3.0
    d = r - r.mean()
    m2 = float(np.mean(d ** 2))
    if m2 <= 0:
        return 0.0, 3.0
    skew = float(np.mean(d ** 3)) / m2 ** 1.5
    kurt = float(np.mean(d ** 4)) / m2 ** 2
    return skew, kurt

```

```

# -----
# 5. Walk-forward purgado con embargo ($3.7) y PBO/CSCV ($5)
# -----

```

```

def purged_walk_forward(
    starts: Sequence[int], ends: Sequence[int],
    n_folds: int = 5, embargo: int = 0,
) -> Iterator[tuple[np.ndarray, np.ndarray]]:
    """Walk-forward purgado con embargo. Sustituye al holdout único del 25%.

    Ordena los eventos por inicio y los parte en `n_folds` grupos contiguos.
    Para k = 1..n_folds-1:
        test = grupo k
        train = eventos cuyo FINAL de etiqueta es anterior a
            (inicio del test - embargo)
    Ventana expansiva, sin futuro en train; la purga elimina los eventos
    cuyo outcome cruza la frontera y el embargo descuenta la
    autocorrelación de volatilidad. Recomendado: embargo = forwardBars.

    Genera tuplas (train_idx, test_idx) con índices ORIGINALES.
    Gate del informe: >= 3 de 4 folds OOS con E_net > 0 y ninguno < -0.10R.
    """
    starts = np.asarray(starts)
    ends = np.asarray(ends)
    order = np.argsort(starts, kind="stable")
    groups = np.array_split(order, n_folds)
    for k in range(1, n_folds):
        test_idx = np.asarray(groups[k], dtype=int)
        if test_idx.size == 0:
            continue
        test_start = starts[test_idx].min()
        train_idx = np.nonzero(ends < (test_start - embargo))[0]
        if train_idx.size == 0:
            continue
        yield train_idx, test_idx

```

```

def pbo_cscv(
    perf: np.ndarray,
    n_blocks: int = 16,
    max_combinations: int = 200,
    rng=None,
) -> dict:
    """Probabilidad de overfitting del backtest/optimizador vía CSCV
    (Bailey, Borwein, López de Prado, Zhu 2017).

    `perf`: matriz (T, M) ordenada en el tiempo ? T observaciones (R por
    trade alineados, o retornos por periodo) de M variantes de estrategia
    (las mutaciones del ImprovementAgent).

    Para cada partición simétrica de los T bloques en train/test:

```

se elige la mejor variante en train y se mira su RANGO out-of-sample.
 $\hat{?} = \text{rango}/(M+1)$; $\hat{?} = \ln(\hat{?}/(1-\hat{?}))$. PBO = fracción de particiones con
 $\hat{?} \leq 0$ (la 'ganadora' queda por debajo de la mediana fuera de muestra).

Gate del informe (§5): aceptar la variante solo si $PBO < 0.10$.

Devuelve dict: pbo, lambdas, n_combinations, best_counts (cuántas veces
ganó cada variante en train ? diagnostica inestabilidad del óptimo).

```
"""
perf = np.asarray(perf, dtype=float)
if perf.ndim != 2:
    raise ValueError("perf debe ser una matriz (T, M)")
T, M = perf.shape
if M < 2 or T < n_blocks or n_blocks % 2 != 0:
    raise ValueError("Se requiere M>=2, T>=n_blocks y n_blocks par.")
blocks = np.array_split(np.arange(T), n_blocks)
all_combos = list(itertools.combinations(range(n_blocks), n_blocks // 2))
gen = np.random.default_rng(rng)
if len(all_combos) > max_combinations:
    sel = gen.choice(len(all_combos), size=max_combinations, replace=False)
    combos = [all_combos[int(i)] for i in sel]
else:
    combos = all_combos
lambdas = []
best_counts = np.zeros(M, dtype=int)
for combo in combos:
    in_train = set(combo)
    train_rows = np.concatenate([blocks[i] for i in combo])
    test_rows = np.concatenate(
        [blocks[i] for i in range(n_blocks) if i not in in_train]
    )
    mu_train = perf[train_rows].mean(axis=0)
    j = int(np.argmax(mu_train))
    best_counts[j] += 1
    mu_test = perf[test_rows].mean(axis=0)
    rank = float(np.sum(mu_test <= mu_train[j])) # 1..M (mayor = mejor)
    omega = rank / (M + 1.0)
    lambdas.append(math.log(omega / (1.0 - omega)))
lambdas_arr = np.asarray(lambdas)
return {
    "pbo": float(np.mean(lambdas_arr <= 0.0)),
    "lambdas": lambdas_arr,
    "n_combinations": len(combos),
    "best_counts": best_counts,
}
```

6. Detector de decay post-producción (§4.8)

```
def cusum_drift(
    x: Sequence[float],
    k: float = 0.5,
    h: float = 4.0,
    target: Optional[float] = None,
    scale: Optional[float] = None,
) -> dict:
    """CUSUM de dos lados sobre z = (x - target)/scale.
```

Para el PatternHealthMonitor: x = serie de R por trade (o slippage_R)
del patrón en producción; target = E_research del patrón; scale = sd de
los R en Research. k = holgura en sd (0.5 detecta cambios ~1 sd);
h = umbral de disparo (4.0 estándar).

Disparo 'down' => deterioro => estado `decaying` => Fox Hunter deja de
operar el patrón hasta re-validación.

Devuelve dict: triggered, index (primer disparo), side ('down'/'up'),
s_pos, s_neg (trayectorias completas, para el dashboard).

```
"""
x = np.asarray(x, dtype=float)
if target is None:
    target = 0.0
if scale is None or not (scale > 0):
    scale = float(x.std(ddof=1)) if x.size > 1 else 1.0
if not (scale > 0):
    scale = 1.0
z = (x - target) / scale
s_pos = np.zeros(x.size)
s_neg = np.zeros(x.size)
sp = sn = 0.0
trig_idx: Optional[int] = None
side: Optional[str] = None
for i, zi in enumerate(z):
    sp = max(0.0, sp + zi - k)
    sn = min(0.0, sn + zi + k)
    s_pos[i] = sp
    s_neg[i] = sn
    if trig_idx is None:
        if sn <= -h:
            trig_idx, side = i, "down"
        elif sp >= h:
            trig_idx, side = i, "up"
return {"triggered": trig_idx is not None, "index": trig_idx,
        "side": side, "s_pos": s_pos, "s_neg": s_neg}
```

```

# -----
# 7. Orquestador: ValidationReport de un candidato (§3.6 completo)
# -----

def summarize_pattern_validation(
    r: Sequence[float],
    starts: Sequence[int],
    ends: Sequence[int],
    *,
    horizonBars: int,
    n_trials_family: int = 1,
    sr_std_across_trials: Optional[float] = None,
    null_means: Optional[Sequence[float]] = None,
    round_trip_cost_R: float = 0.0,
    n_boot: int = 2000,
    rng=None,
    min_n_eff: float = 60.0,
    min_dsr: float = 0.95,
) -> dict:
    """Aplica los pasos 2-7 del pipeline del informe a UN candidato y
    devuelve el dict base del ValidationReport.

    PRERREQUISITOS del llamante:
    - `r` en orden temporal y YA deduplicado por simbolo con
      select_nonoverlapping_events (paso 1).
    - `starts`/`ends`: spans de barra del OUTCOME de cada evento
      (entry_index..exit_index), en un eje temporal común a todos los
      símbolos (p.ej. índice de sesión de calendario) para que la
      unicidad capture también el solapamiento entre símbolos.
    - `round_trip_cost_R`: 0.0 si `r` ya es neto de costes.
    - `null_means`: expectativas de las K carteras nulas emparejadas
      (vuestrs null baselines), con el MISMO etiquetador y costes.

    El corte de p-valor NO se decide aquí: se decide a nivel de run con
    benjamini_hochberg sobre todos los candidatos. Aquí solo se reporta.
    """
    r = np.asarray(r, dtype=float)
    if round_trip_cost_R:
        r = r - round_trip_cost_R
    n_raw = int(r.size)

    weights, n_eff = average_uniqueness_weights(starts, ends)
    wsum = float(weights.sum()) if weights.size else 0.0

    if n_raw == 0 or wsum <= 0:
        return {"status": "empty", "n_raw": n_raw, "n_eff": 0.0}

    mean_w = float(np.sum(weights * r) / wsum)
    var_w = float(np.sum(weights * (r - mean_w) ** 2) / wsum)
    sd_w = math.sqrt(var_w) if var_w > 0 else float("nan")
    sr = mean_w / sd_w if sd_w and sd_w > 0 else float("nan")

    pf_w = profit_factor(r, weights)
    ci_lo, ci_hi, _, _ = stationary_bootstrap_ci(
        r, np.mean, n_boot=n_boot, mean_block=horizonBars, rng=rng
    )
    t_nw, se_nw = newey_west_tstat(r, lags=horizonBars)
    skew, kurt = sample_skew_kurt(r)
    psr = probabilistic_sharpe_ratio(sr, 0.0, n_eff, skew, kurt)

    if n_trials_family > 1 and sr_std_across_trials and sr_std_across_trials > 0:
        dsr, sr_star = deflated_sharpe_ratio(
            sr, n_eff, skew, kurt, n_trials_family, sr_std_across_trials
        )
    else:
        # Sin historial de trials no hay deflación posible: con 1 trial el
        # DSR colapsa al PSR; con >1 trials sin sr_std, repórtalo como nan
        # y exige al ledger ese dato antes de promocionar.
        dsr = psr if n_trials_family <= 1 else float("nan")
        sr_star = 0.0

    p_perm = (permutation_pvalue(mean_w, null_means)
               if null_means is not None else float("nan"))

    gates = {
        "n_eff_ok": bool(n_eff >= min_n_eff),
        "ci_lower_positive": bool(np.isfinite(ci_lo) and ci_lo > 0.0),
        "dsr_ok": bool(np.isfinite(dsr) and dsr >= min_dsr),
    }
    gates["all_local_gates"] = all(gates.values())

    return {
        "status": "ok",
        "n_raw": n_raw,
        "n_eff": float(n_eff),
        "expectancy_R_weighted": mean_w,
        "expectancy_ci95": (ci_lo, ci_hi),
        "profit_factor_weighted": pf_w,
        "sharpe_per_trade": sr,
        "newey_west_t": t_nw,
        "newey_west_se": se_nw,
        "skew": skew,
        "kurtosis": kurt,
        "psr_vs_0": psr,
    }

```

```

    "dsr": dsr,
    "sr_star_null_max": sr_star,
    "n_trials_family": int(n_trials_family),
    "perm_pvalue": p_perm, # corte a nivel de run via benjamini_hochberg
    "gates": gates,
}

```

Full Source: `backend/tradeo/research/nested\_optimization.py`

`backend/tradeo/research/nested\_optimization.py` ? 186 lines, 7631 bytes, sha256 prefix `942ab1c10a3006bb`

Audit relevance: Wave4-D nested optimization/PBO report. Audit pass criteria and limitations of complement train folds vs causal walk-forward.

Public surface summary before full source:

```

? function `optuna_available` line 41`
? function `nested_optimization_report` line 81`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

"""Nested optimization with an outer-fold PBO report (Wave4-D).

Closes the ImprovementAgent gap "Nested Optuna + outer-fold PBO report" without adding a hard dependency on Optuna:

- The candidate variants are already fully enumerated by the ImprovementAgent grid, so the inner optimization is a *selection* problem over M variants.
- Inner selection runs on the train periods (complement of each outer fold). When Optuna is installed and the variant count exceeds the inner trial budget, a seeded ``TPESampler`` searches the categorical variant space with pruning disabled (the objective is a cheap deterministic mean, there are no intermediate values to prune on). Otherwise a deterministic exhaustive scan (argmax with first-index tie-break) is used ? strictly better than any sampler when the budget covers the space, and bit-for-bit reproducible.
- Each outer fold then evaluates the inner winner out-of-fold: its rank among all M variants gives $\omega = \text{rank}/(M+1)$ and $\lambda = \ln(\omega/(1-\omega))$, as in CSCV (Bailey, Borwein, Lopez de Prado, Zhu 2017). The outer-fold PBO is the fraction of folds with $\lambda \leq 0$.

Honest limitations (do not overclaim):

- Folds are contiguous splits of the aggregated performance periods (monthly R buckets upstream); no embargo/purge is applied at this aggregation level.
- Train = complement of the outer fold (CSCV-style), so train may contain periods after the outer fold; this measures selection-overfitting, not a causal walk-forward forecast.
- This is an additional anti-overfit *blocker* layered on top of the existing CSCV PBO and plateau guards. It never relaxes them.

"""

```

from __future__ import annotations

```

```

import math
from collections import Counter
from typing import Any

```

```

import numpy as np

```

```

METHOD = "nested_outer_fold_pbo_v1"

```

```

def optuna_available() -> bool:

```

```

    try:
        import optuna # noqa: F401
    except ImportError:
        return False
    return True

```

```

def _blocked(reason: str, **extra: Any) -> dict[str, Any]:

```

```

    report: dict[str, Any] = {
        "method": METHOD,
        "passed": False,
        "blocked": True,
        "reason": reason,
        "optuna_available": optuna_available(),
    }
    report.update(extra)
    return report

```

```

def _inner_select_exhaustive(inner_means: np.ndarray) -> tuple[int, str]:

```

```

    return int(np.argmax(inner_means)), "exhaustive_deterministic"

```

```

def _inner_select_optuna(inner_means: np.ndarray, n_trials: int, seed: int) -> tuple[int, str]:

```

```

    import optuna

    optuna.logging.set_verbosity(optuna.logging.WARNING)
    sampler = optuna.samplers.TPESampler(seed=seed)
    study = optuna.create_study(direction="maximize", sampler=sampler)
    indices = list(range(inner_means.shape[0]))

```

```

def objective(trial: "optuna.Trial") -> float:
    idx = trial.suggest_categorical("variant_index", indices)
    return float(inner_means[int(idx)])

study.optimize(objective, n_trials=n_trials)
return int(study.best_params["variant_index"]), "optuna_tpe_seeded_no_pruning"

def nested_optimization_report(
    perf: np.ndarray,
    *,
    n_outer_folds: int = 5,
    inner_trials: int = 64,
    max_pbo: float = 0.10,
    seed: int = 17,
    min_periods_per_fold: int = 2,
    use_optuna: bool = True,
) -> dict[str, Any]:
    """Nested inner-selection / outer-evaluation report over ``perf`` (T, M).

    ``perf`` rows are time-ordered aggregation periods, columns are strategy
    variants. Returns a report dict with ``passed`` (bool) and ``blocked``
    (bool); blocked or failed reports must block lab-candidate acceptance
    (fail-closed).
    """
    perf = np.asarray(perf, dtype=float)
    if perf.ndim != 2:
        return _blocked("perf_must_be_2d_matrix")
    n_periods, n_variants = perf.shape
    if n_variants < 2:
        return _blocked(
            "insufficient_variants_for_nested_search",
            period_count=int(n_periods),
            variant_count=int(n_variants),
        )
    if n_outer_folds < 4:
        return _blocked("n_outer_folds_must_be_at_least_4", n_outer_folds=int(n_outer_folds))
    if n_periods < n_outer_folds * min_periods_per_fold:
        return _blocked(
            "insufficient_periods_for_outer_folds",
            period_count=int(n_periods),
            required_periods=int(n_outer_folds * min_periods_per_fold),
            n_outer_folds=int(n_outer_folds),
        )
    if not np.isfinite(perf).all():
        return _blocked("non_finite_performance_values")

    has_optuna = optuna_available()
    folds = np.array_split(np.arange(n_periods), n_outer_folds)
    fold_reports: list[dict[str, Any]] = []
    lambdas: list[float] = []
    selected_variants: list[int] = []
    selected_outer_scores: list[float] = []
    inner_method = "exhaustive_deterministic"
    for fold_index, test_rows in enumerate(folds):
        train_rows = np.setdiff1d(np.arange(n_periods), test_rows)
        inner_means = perf[train_rows].mean(axis=0)
        if has_optuna and use_optuna and n_variants > inner_trials:
            selected, inner_method = _inner_select_optuna(
                inner_means, n_trials=inner_trials, seed=seed + fold_index
            )
        else:
            selected, inner_method = _inner_select_exhaustive(inner_means)
        outer_means = perf[test_rows].mean(axis=0)
        rank = float(np.sum(outer_means <= outer_means[selected])) # 1..M, higher = better
        omega = rank / (n_variants + 1.0)
        lam = math.log(omega / (1.0 - omega))
        lambdas.append(lam)
        selected_variants.append(selected)
        selected_outer_scores.append(float(outer_means[selected]))
        fold_reports.append(
            {
                "fold_index": fold_index,
                "train_period_count": int(train_rows.size),
                "test_period_count": int(test_rows.size),
                "selected_variant": selected,
                "inner_score": round(float(inner_means[selected]), 6),
                "outer_score": round(float(outer_means[selected]), 6),
                "outer_rank": int(rank),
                "lambda": round(lam, 6),
                "oos_degradation": round(float(inner_means[selected] - outer_means[selected]), 6),
            }
        )

    pbo_outer = float(np.mean(np.asarray(lambdas) <= 0.0))
    outer_median_selected_score = float(np.median(selected_outer_scores))
    selection_counts = Counter(selected_variants)
    consensus_variant, consensus_count = selection_counts.most_common(1)[0]
    pbo_passed = pbo_outer < float(max_pbo)
    outer_score_passed = outer_median_selected_score > 0.0
    return {
        "method": METHOD,
        "passed": bool(pbo_passed and outer_score_passed),
        "blocked": False,
        "optuna_available": has_optuna,
    }

```

```

        "inner_method": inner_method,
        "inner_trials": int(inner_trials),
        "seed": int(seed),
        "n_outer_folds": int(n_outer_folds),
        "period_count": int(n_periods),
        "variant_count": int(n_variants),
        "pbo_outer": round(pbo_outer, 5),
        "max_pbo": float(max_pbo),
        "pbo_passed": bool(pbo_passed),
        "outer_median_selected_score": round(outer_median_selected_score, 6),
        "outer_score_passed": bool(outer_score_passed),
        "consensus_variant": int(consensus_variant),
        "selection_stability": round(consensus_count / float(n_outer_folds), 5),
        "selection_counts": {str(k): int(v) for k, v in sorted(selection_counts.items())},
        "mean_oos_degradation": round(
            float(np.mean([f["oos_degradation"] for f in fold_reports])), 6
        ),
        "outer_folds": fold_reports,
    }
}

```

Full Source: `backend/tradeo/research/rediscovery\_readiness.py`

`backend/tradeo/research/rediscovery\_readiness.py` ? 274 lines, 10079 bytes, sha256 prefix `3efbb7d03b402f75`

Audit relevance: Wave4-C readiness tool. Audit that it flags only and never fabricates metadata.

Public surface summary before full source:

```

? class `PatternReadiness` line 50 methods: needs_rediscovery, as_payload
? function `evaluate_pattern_metrics` line 73`
? function `audit_patterns` line 102`
? function `apply_rediscovery_flags` line 126`
? function `build_manifest` line 168`
? function `run_readiness` line 222`
? function `main` line 241`

```

Risk hints: wall-clock timestamp usage.

"""Rediscovery readiness audit for already-persisted discovered patterns.

Wave4 added three metadata contracts that discovery now writes into
`DiscoveredPattern.metrics\_json` but that legacy rows predate:

- embedding contract (`feature\_parity\_contract` with a `contract\_id`),
- cluster signature with concentration metadata (`cluster\_signature` /
`concentration\_checks`),
- benchmark-regime calibration buckets
(`regime\_profile.benchmark\_regime\_outcomes`).

This module audits persisted patterns against those contracts, can flag
laggards with a `rediscovery\_readiness` block inside `metrics\_json` and
emits a manifest with honest before/after counts. Flagging never populates
metadata: only a real discovery run does. This module never launches
discovery, market scans or any live/paper order path.

"""

```
from __future__ import annotations
```

```

import argparse
import json
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from sqlalchemy import select
from sqlalchemy.orm import Session
from sqlalchemy.orm.attributes import flag_modified

```

```

from tradeo.db.models import DiscoveredPattern
from tradeo.research.determinism import CONTENT_HASH_ALGO, content_hash
from tradeo.research.pattern_embedding_engine import PatternEmbeddingEngine

```

```

CHECKER_VERSION = "wave4c_rediscovery_readiness_v1"
READINESS_KEY = "rediscovery_readiness"

```

```

EMBEDDING_CONTRACT = "embedding_contract"
CLUSTER_SIGNATURE = "cluster_signature"
REGIME_CALIBRATION = "regime_calibration_buckets"
REQUIRED_FIELDS = (EMBEDDING_CONTRACT, CLUSTER_SIGNATURE, REGIME_CALIBRATION)

```

```

# Hard cap so the audit never turns into an unbounded scan; the manifest
# reports truncation explicitly when the cap is hit.
DEFAULT_AUDIT_LIMIT = 2000

```

```

@dataclass
class PatternReadiness:
    pattern_id: int
    pattern_key: str

```

```

status: str
promotion_status: str
missing: list[str] = field(default_factory=list)
stale: list[str] = field(default_factory=list)

@property
def needs_rediscovery(self) -> bool:
    return bool(self.missing or self.stale)

def as_payload(self) -> dict[str, Any]:
    return {
        "pattern_key": self.pattern_key,
        "status": self.status,
        "promotion_status": self.promotion_status,
        "missing": list(self.missing),
        "stale": list(self.stale),
        "needs_rediscovery": self.needs_rediscovery,
    }

def evaluate_pattern_metrics(metrics: dict[str, Any] | None) -> tuple[list[str], list[str]]:
    """Return (missing, stale) Wave4 metadata fields for one metrics blob."""
    metrics = metrics if isinstance(metrics, dict) else {}
    missing: list[str] = []
    stale: list[str] = []

    contract = metrics.get("feature_parity_contract")
    if not isinstance(contract, dict) or not contract.get("contract_id"):
        missing.append(EMBEDDING_CONTRACT)
    elif str(contract.get("contract_id")) != PatternEmbeddingEngine.CONTRACT_ID:
        stale.append(EMBEDDING_CONTRACT)

    signature = metrics.get("cluster_signature")
    checks = signature.get("concentration_checks") if isinstance(signature, dict) else None
    if (
        not isinstance(checks, dict)
        or "passed" not in checks
        or not isinstance(signature.get("medoid") if isinstance(signature, dict) else None, dict)
    ):
        missing.append(CLUSTER_SIGNATURE)

    profile = metrics.get("regime_profile")
    outcomes = profile.get("benchmark_regime_outcomes") if isinstance(profile, dict) else None
    if not isinstance(outcomes, dict) or "available" not in outcomes:
        missing.append(REGIME_CALIBRATION)

    return missing, stale

def audit_patterns(
    db: Session, *, limit: int = DEFAULT_AUDIT_LIMIT
) -> tuple[list[PatternReadiness], bool]:
    """Audit persisted patterns deterministically; returns (results, truncated)."""
    limit = max(1, int(limit))
    stmt = select(DiscoveredPattern).order_by(DiscoveredPattern.pattern_key.asc()).limit(limit + 1)
    rows = list(db.execute(stmt).scalars())
    truncated = len(rows) > limit
    results: list[PatternReadiness] = []
    for row in rows[:limit]:
        missing, stale = evaluate_pattern_metrics(row.metrics_json)
        results.append(
            PatternReadiness(
                pattern_id=int(row.id),
                pattern_key=str(row.pattern_key),
                status=str(getattr(row.status, "value", row.status)),
                promotion_status=str(row.promotion_status or ""),
                missing=missing,
                stale=stale,
            )
        )
    return results, truncated

def apply_rediscovery_flags(db: Session, results: list[PatternReadiness]) -> int:
    """Write a ``rediscovery_readiness`` block into metrics_json of laggards.

    Marks intent only ? it does not create, recompute or fake any metadata.
    Patterns already satisfying every contract get any stale flag cleared.
    """
    flagged = 0
    by_id = {result.pattern_id: result for result in results}
    if not by_id:
        return 0
    rows = db.execute(
        select(DiscoveredPattern).where(DiscoveredPattern.id.in_(by_id.keys()))
    ).scalars()
    now = datetime.now(timezone.utc).isoformat()
    for row in rows:
        result = by_id[int(row.id)]
        metrics = dict(row.metrics_json or {})
        if result.needs_rediscovery:
            metrics[READINESS_KEY] = {
                "needs_rediscovery": True,
                "missing": list(result.missing),
                "stale": list(result.stale),
                "checker_version": CHECKER_VERSION,
            }

```

```

        "flagged_at": now,
    }
    flagged += 1
elif READINESS_KEY in metrics:
    metrics[READINESS_KEY] = {
        "needs_rediscovery": False,
        "missing": [],
        "stale": [],
        "checker_version": CHECKER_VERSION,
        "flagged_at": now,
    }
else:
    continue
row.metrics_json = metrics
flag_modified(row, "metrics_json")
db.commit()
return flagged

def build_manifest(
    results: list[PatternReadiness],
    *,
    dry_run: bool,
    flagged_count: int,
    truncated: bool,
) -> dict[str, Any]:
    """Build an honest rediscovery manifest with before/after counts.

    ``metadata_complete_after`` always equals ``metadata_complete_before``:
    this tool only audits and flags, it never populates metadata. Counts only
    change after a real rediscovery run is executed and re-audited.
    """
    needs = [result for result in results if result.needs_rediscovery]
    missing_counts = {
        fieldname: sum(1 for result in results if fieldname in result.missing)
        for fieldname in REQUIRED_FIELDS
    }
    stale_counts = {
        fieldname: sum(1 for result in results if fieldname in result.stale)
        for fieldname in REQUIRED_FIELDS
    }
    complete_before = len(results) - len(needs)
    manifest: dict[str, Any] = {
        "checker_version": CHECKER_VERSION,
        "dry_run": dry_run,
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "truncated": truncated,
        "counts": {
            "patterns_audited": len(results),
            "metadata_complete_before": complete_before,
            "metadata_complete_after": complete_before,
            "needs_rediscovery": len(needs),
            "flagged_this_run": flagged_count,
            "missing_by_field": missing_counts,
            "stale_by_field": stale_counts,
        },
        "honesty_note": (
            "flagging marks intent only; metadata_complete_after == before because "
            "no rediscovery was executed by this tool"
        ),
        "rediscovery_plan": {
            "command": "POST /api/research/run-discovery (see docs/research/wave4_rediscovery_runbook.md)",
            "verify": "re-run this audit after discovery; needs_rediscovery must drop to 0",
        },
        "patterns_needing_rediscovery": [result.as_payload() for result in needs],
    }
    manifest["determinism"] = {
        "algo": CONTENT_HASH_ALGO,
        "content_hash": content_hash(manifest),
    }
    return manifest

def run_readiness(
    db: Session,
    *,
    apply_flags: bool = False,
    limit: int = DEFAULT_AUDIT_LIMIT,
    manifest_path: Path | None = None,
) -> dict[str, Any]:
    """Audit patterns, optionally flag laggards, and return the manifest."""
    results, truncated = audit_patterns(db, limit=limit)
    flagged = apply_rediscovery_flags(db, results) if apply_flags else 0
    manifest = build_manifest(
        results, dry_run=not apply_flags, flagged_count=flagged, truncated=truncated
    )
    if manifest_path is not None:
        manifest_path.parent.mkdir(parents=True, exist_ok=True)
        manifest_path.write_text(json.dumps(manifest, indent=2, sort_keys=True), encoding="utf-8")
    return manifest

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(
        description="Wave4-C rediscovery readiness audit (no discovery is run)"
    )

```



```

parser.add_argument(
    "--apply-flags",
    action="store_true",
    help="write rediscovery_readiness flags into metrics_json (default: dry-run)",
)
parser.add_argument(
    "--limit", type=int, default=DEFAULT_AUDIT_LIMIT, help="max patterns to audit"
)
parser.add_argument(
    "--manifest-out", type=Path, default=None, help="path for the JSON manifest"
)
args = parser.parse_args(argv)

from tradeo.db.session import SessionLocal

db = SessionLocal()
try:
    manifest = run_readiness(
        db, apply_flags=args.apply_flags, limit=args.limit, manifest_path=args.manifest_out
    )
finally:
    db.close()
print(json.dumps(manifest["counts"], indent=2, sort_keys=True))
if manifest["truncated"]:
    print(f"WARNING: audit truncated at --limit={args.limit}; rerun with a higher limit")
return 0

if __name__ == "__main__": # pragma: no cover
    raise SystemExit(main())

```

Full Source: ``backend/tradeo/research/determinism.py``

``backend/tradeo/research/determinism.py`` ? 116 lines, 4292 bytes, sha256 prefix ``67df664726130667``

Public surface summary before full source:

```

? function `canonical_payload` line 50`
? function `canonical_json` line 78`
? function `content_hash` line 88`
? function `candidate_content_payload` line 92`
? function `discovery_content_hash` line 114`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

"""Deterministic serialization and content hashing for research artifacts.

The discovery pipeline must be bit-for-bit reproducible for identical inputs, config and seeds. Wall-clock timestamps, database run ids and filesystem paths are legitimate run metadata but must never leak into the identity of a result. This module provides one canonical JSON form and a content hash that excludes those volatile keys, so two runs over the same inputs can be compared by hash regardless of when or where they executed.

"""

```

from __future__ import annotations

import json
import math
from hashlib import sha256
from typing import Any

import numpy as np

from tradeo.research.types import ClusterCandidate

CONTENT_HASH_ALGO = "sha256_canonical_json_v1"

# Run metadata that varies across re-runs of identical inputs. Excluded from
# content hashes only ? the artifact itself keeps these fields.
DEFAULT_VOLATILE_KEYS = frozenset(
    {
        "built_at",
        "generated_at",
        "created_at",
        "updated_at",
        "started_at",
        "finished_at",
        "exported_at",
        "timestamp",
        "run_id",
        "duration_seconds",
        "elapsed_seconds",
        "event_ledger_path",
        "research_paper_report_path",
        "report_path",
        "report_json_path",
        "report_md_path",
        "backup_path",
        "path",
    }
)

```

```

def canonical_payload(value: Any, *, exclude_keys: frozenset[str] = DEFAULT_VOLATILE_KEYS) -> Any:
    """Reduce a payload to plain, deterministic JSON-safe types.

    Dict keys are stringified and sorted, volatile keys dropped, sets sorted,
    numpy scalars/arrays unwrapped, and non-finite floats mapped to None so the
    output never depends on insertion order, hash seeds or NaN encodings.
    """
    if isinstance(value, dict):
        return {
            str(key): canonical_payload(item, exclude_keys=exclude_keys)
            for key, item in sorted(value.items(), key=lambda pair: str(pair[0]))
            if str(key) not in exclude_keys
        }
    if isinstance(value, (list, tuple)):
        return [canonical_payload(item, exclude_keys=exclude_keys) for item in value]
    if isinstance(value, (set, frozenset)):
        return [canonical_payload(item, exclude_keys=exclude_keys) for item in sorted(value, key=str)]
    if isinstance(value, np.ndarray):
        return [canonical_payload(item, exclude_keys=exclude_keys) for item in value.tolist()]
    if isinstance(value, np.generic):
        value = value.item()
    if isinstance(value, float):
        return value if math.isfinite(value) else None
    if isinstance(value, (str, int, bool)) or value is None:
        return value
    return str(value)

def canonical_json(value: Any, *, exclude_keys: frozenset[str] = DEFAULT_VOLATILE_KEYS) -> str:
    return json.dumps(
        canonical_payload(value, exclude_keys=exclude_keys),
        sort_keys=True,
        separators=(",", ":"),
        ensure_ascii=False,
        allow_nan=False,
    )

def content_hash(value: Any, *, exclude_keys: frozenset[str] = DEFAULT_VOLATILE_KEYS) -> str:
    return sha256(canonical_json(value, exclude_keys=exclude_keys).encode("utf-8")).hexdigest()

def candidate_content_payload(candidate: ClusterCandidate) -> dict[str, Any]:
    """Stable identity payload for one discovery candidate."""
    return {
        "pattern_key": candidate.pattern_key,
        "name": candidate.name,
        "side": candidate.side,
        "timeframe": candidate.timeframe,
        "window_size": candidate.window_size,
        "cluster_id": candidate.cluster_id,
        "centroid": candidate.centroid,
        "sample_count": candidate.sample_count,
        "symbol_count": candidate.symbol_count,
        "year_count": candidate.year_count,
        "score": candidate.score,
        "validation_passed": candidate.validation_passed,
        "validation_reasons": candidate.validation_reasons,
        "metrics": candidate.metrics,
        "feature_summary": candidate.feature_summary,
        "examples": candidate.examples,
    }

def discovery_content_hash(candidates: list[ClusterCandidate]) -> str:
    """Content hash of a full discovery result, in result order."""
    return content_hash([candidate_content_payload(candidate) for candidate in candidates])

```

Full Source: ``backend/tradeo/research/autonomous_research_director.py``

``backend/tradeo/research/autonomous_research_director.py`` ? 747 lines, 35589 bytes, sha256 prefix ``26389db9bb078daa``

Public surface summary before full source:

? class ``ResearchDirector`` line 16 methods: run

Risk hints: wall-clock timestamp usage.

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timezone
import json
from typing import Any

import numpy as np
from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import DiscoveredPattern, DiscoveredPatternMetric

```

```

@dataclass(slots=True)
class ResearchDirector:
    """Autonomous research scientist layer for discovered patterns.

    The director does not promote patterns to paper/live. It turns discovery
    output into durable research knowledge: falsifiable hypotheses, adversarial
    challenges, invariant checks, memory graph updates, active-learning agendas
    and paper-style reports.
    """

    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()

    def run(
        self,
        db: Session,
        *,
        run_id: int | None = None,
        limit: int | None = None,
        store: bool = True,
    ) -> dict[str, Any]:
        settings = self.settings
        assert settings is not None
        patterns = self._patterns(db, run_id=run_id, limit=limit or settings.research_director_pattern_limit)
        generated_at = datetime.now(timezone.utc).isoformat()
        intelligence_by_key: dict[str, dict[str, Any]] = {}
        for pattern in patterns:
            intelligence = self._pattern_intelligence(pattern, generated_at=generated_at)
            intelligence_by_key[pattern.pattern_key] = intelligence
            if store:
                metrics = dict(pattern.metrics_json or {})
                metrics["research_intelligence"] = intelligence
                metrics["research_hypothesis"] = intelligence["hypothesis"]
                metrics["research_lifecycle"] = intelligence["lifecycle"]
                metrics["research_director_updated_at"] = generated_at
                pattern.metrics_json = self._json_clean(metrics)
                pattern.updated_at = datetime.now(timezone.utc)
                db.add(pattern)
                db.add(
                    DiscoveredPatternMetric(
                        pattern_id=pattern.id,
                        split="research_director",
                        metrics_json=self._json_clean(intelligence),
                    )
                )

        memory_graph = self._memory_graph(patterns, intelligence_by_key)
        agenda = self._active_learning_agenda(patterns, intelligence_by_key, memory_graph)
        summary = {
            "generated_at": generated_at,
            "run_id": run_id,
            "patterns_reviewed": len(patterns),
            "hypotheses_created": len(intelligence_by_key),
            "memory_graph": memory_graph,
            "active_learning_agenda": agenda,
            "director_state": self._director_state(patterns, intelligence_by_key, agenda),
        }
        paths = self._write_artifacts(summary, patterns, intelligence_by_key)
        summary["artifacts"] = paths
        if store:
            db.commit()
        return self._json_clean(summary)

    def _patterns(self, db: Session, *, run_id: int | None, limit: int) -> list[DiscoveredPattern]:
        query = db.query(DiscoveredPattern).order_by(
            DiscoveredPattern.score.desc(),
            DiscoveredPattern.created_at.desc(),
            DiscoveredPattern.pattern_key.asc(),
        )
        if run_id is not None:
            query = query.filter(DiscoveredPattern.run_id == run_id)
        return query.limit(max(1, min(int(limit), 500))).all()

    def _pattern_intelligence(self, pattern: DiscoveredPattern, *, generated_at: str) -> dict[str, Any]:
        metrics = dict(pattern.metrics_json or {})
        hypothesis = self._hypothesis(pattern, metrics)
        replay = self._market_replay(pattern, metrics)
        adversarial = self._adversarial_challenge(pattern, metrics, replay)
        invariance = self._invariance(pattern, metrics)
        foundation = self._foundation_learning_digest(pattern, metrics)
        lifecycle = self._lifecycle(pattern, metrics, adversarial, invariance)
        paper = self._paper_summary(pattern, hypothesis, adversarial, invariance, replay, lifecycle)
        return {
            "generated_at": generated_at,
            "pattern_key": pattern.pattern_key,
            "hypothesis": hypothesis,
            "foundation_learning": foundation,
            "market_replay": replay,
            "adversarial_challenge": adversarial,
            "invariance": invariance,
            "lifecycle": lifecycle,
            "paper": paper,
        }

```

```

def _hypothesis(self, pattern: DiscoveredPattern, metrics: dict[str, Any]) -> dict[str, Any]:
    human_rule = metrics.get("human_rule") if isinstance(metrics.get("human_rule"), dict) else {}
    rule = str(human_rule.get("rule") or self._fallback_rule(pattern, metrics))
    regime = metrics.get("regime_profile") if isinstance(metrics.get("regime_profile"), dict) else {}
    dominant_regime = str(regime.get("dominant_regime", "unknown"))
    mechanism = self._mechanism(pattern, metrics, rule)
    works_when = self._works_when(metrics, human_rule, dominant_regime)
    fails_when = self._fails_when(metrics, dominant_regime)
    evidence = {
        "sample_count": pattern.sample_count,
        "symbol_count": pattern.symbol_count,
        "year_count": pattern.year_count,
        "best_rr": float(pattern.best_rr or metrics.get("best_rr", 0.0) or 0.0),
        "expectancy_r": float(pattern.best_expectancy_r or metrics.get("best_expectancy_r", 0.0) or 0.0),
        "profit_factor": float(pattern.best_profit_factor or metrics.get("best_profit_factor", 0.0) or 0.0),
        "oos_expectancy_r": float(pattern.out_of_sample_expectancy_r or 0.0),
        "purged_cv_positive_rate": float(metrics.get("purged_cv_positive_rate", 0.0) or 0.0),
        "deflated_sharpe_probability": float(metrics.get("deflated_sharpe_probability", 0.0) or 0.0),
    }
    return {
        "thesis": f"{pattern.side} edge candidate: {rule}",
        "edge_claim": "NO_DEMOSTRADO",
        "mechanism": mechanism,
        "works_when": works_when,
        "should_fail_when": fails_when,
        "kill_criteria": self._kill_criteria(pattern, metrics),
        "evidence": evidence,
        "confidence": self._bounded(
            0.15
            + min(pattern.sample_count / 250.0, 1.0) * 0.18
            + min(pattern.symbol_count / 20.0, 1.0) * 0.16
            + min(pattern.year_count / 5.0, 1.0) * 0.12
            + max(0.0, float(metrics.get("purged_cv_positive_rate", 0.0))) * 0.17
            + max(0.0, float(metrics.get("deflated_sharpe_probability", 0.0))) * 0.12
            + (0.10 if pattern.validation_passed else 0.0)
        ),
        "falsifiable": True,
    }

@staticmethod
def _fallback_rule(pattern: DiscoveredPattern, metrics: dict[str, Any]) -> str:
    rule_bits = [
        f"window={pattern.window_size}",
        f"side={pattern.side}",
        f"best_rr={float(metrics.get('best_rr', pattern.best_rr or 0.0)):.2f}",
        f"score={float(pattern.score or 0.0):.3f}",
    ]
    return "centroid similarity with " + " ".join(rule_bits)

@staticmethod
def _mechanism(pattern: DiscoveredPattern, metrics: dict[str, Any], rule: str) -> str:
    side_word = "demand" if pattern.side == "long" else "supply"
    if "compression" in rule or float(metrics.get("range_phase_expansion", 0.0) or 0.0) < 0:
        return f"{side_word} imbalance after compression; edge should appear quickly after range expansion."
    if float(metrics.get("mfe_before_mae_rate", 0.0) or 0.0) >= 0.55:
        return f"{side_word} imbalance appears before adverse excursion; timing quality is part of the edge."
    if float(metrics.get("relative_strength_sector", 0.0) or 0.0) > 0:
        return "sector-relative strength plus local trigger may indicate institutional rotation."
    return "recurrent chart family with forward path asymmetry; causal driver remains provisional."

@staticmethod
def _works_when(metrics: dict[str, Any], human_rule: dict[str, Any], dominant_regime: str) -> list[str]:
    conditions = []
    for condition in human_rule.get("conditions", []):
        if isinstance(condition, dict) and condition.get("label"):
            conditions.append(
                f"{condition.get('label')} {condition.get('operator')} {condition.get('threshold')}"
            )
    if dominant_regime != "unknown":
        conditions.append(f"dominant regime matches {dominant_regime}")
    if float(metrics.get("avg_fill_probability", 1.0) or 1.0) >= 0.65:
        conditions.append("estimated fill quality remains high")
    if not conditions:
        conditions.append("centroid similarity remains high and execution costs stay near research assumptions")
    return conditions[:6]

@staticmethod
def _fails_when(metrics: dict[str, Any], dominant_regime: str) -> list[str]:
    failures = [
        "out-of-sample expectancy turns negative",
        "purged CV positive fold rate drops below configured minimum",
        "cost stress x2 removes positive expectancy",
    ]
    if dominant_regime != "unknown":
        failures.append(f"market leaves preferred regime {dominant_regime}")
    if float(metrics.get("fast_target_rate", 0.0) or 0.0) > 0.35:
        failures.append("entries are delayed; fast-target edge becomes latency-fragile")
    if float(metrics.get("avg_fill_probability", 1.0) or 1.0) < 0.55:
        failures.append("fill probability deteriorates below tradable threshold")
    return failures[:6]

@staticmethod
def _kill_criteria(pattern: DiscoveredPattern, metrics: dict[str, Any]) -> list[dict[str, Any]]:
    return [
        {

```

```

        "metric": "rolling_oos_expectancy_r",
        "operator": "<=",
        "threshold": 0.0,
        "reason": "edge must stay positive outside discovery sample",
    },
    {
        "metric": "cost_stress_2x_expectancy_r",
        "operator": "<=",
        "threshold": 0.0,
        "reason": "statistical edge without cost edge is not tradable",
    },
    {
        "metric": "symbol_count",
        "operator": "<",
        "threshold": max(4, min(8, int(pattern.symbol_count or 0))),
        "reason": "avoid single-name or tiny-universe artifacts",
    },
    {
        "metric": "edge_decay_parameter_score",
        "operator": ">",
        "threshold": max(0.55, float(metrics.get("edge_decay_parameter_score", 0.55) or 0.55)),
        "reason": "nearby parameter changes should not destroy the edge",
    },
]

@staticmethod
def _market_replay(pattern: DiscoveredPattern, metrics: dict[str, Any]) -> dict[str, Any]:
    fill = float(metrics.get("avg_fill_probability", 0.0) or 0.0)
    max_size = float(metrics.get("p25_max_size_usd", 0.0) or 0.0)
    cost_r = float(metrics.get("avg_execution_cost_r", 0.0) or 0.0)
    spread = float(metrics.get("avg_spread_proxy_pct", 0.0) or 0.0)
    slippage = float(metrics.get("avg_slippage_proxy_pct", 0.0) or 0.0)
    gap = float(metrics.get("avg_entry_gap_penalty_pct", 0.0) or 0.0)
    fast = float(metrics.get("fast_target_rate", 0.0) or 0.0)
    late_entry_fragility = ResearchDirector._bounded(fast * 0.65 + gap * 8.0 + slippage * 5.0)
    tradability = ResearchDirector._bounded(
        fill * 0.35
        + min(max_size / 25_000.0, 1.0) * 0.20
        + max(0.0, 1.0 - cost_r / 0.35) * 0.25
        + max(0.0, 1.0 - late_entry_fragility) * 0.20
    )
    bottlenecks = []
    if fill < 0.55:
        bottlenecks.append("fill_probability")
    if 0 < max_size < 10_000:
        bottlenecks.append("max_size")
    if cost_r > 0.25:
        bottlenecks.append("execution_cost")
    if late_entry_fragility > 0.55:
        bottlenecks.append("entry_latency")
    return {
        "tradability_score": round(tradability, 5),
        "late_entry_fragility": round(late_entry_fragility, 5),
        "avg_fill_probability": round(fill, 5),
        "p25_max_size_usd": round(max_size, 2),
        "avg_execution_cost_r": round(cost_r, 5),
        "avg_spread_proxy_pct": round(spread, 6),
        "avg_slippage_proxy_pct": round(slippage, 6),
        "avg_entry_gap_penalty_pct": round(gap, 6),
        "bottlenecks": bottlenecks,
        "market_replay_model": "deterministic_latency_fill_cost_proxy_v1",
        "assumption": "No broker microstructure data is used; this is a conservative research replay proxy.",
    }

@staticmethod
def _adversarial_challenge(
    pattern: DiscoveredPattern,
    metrics: dict[str, Any],
    replay: dict[str, Any],
) -> dict[str, Any]:
    tests = [
        ResearchDirector._challenge_test(
            "leakage_guard",
            bool(metrics.get("train_cutoff")) and bool(metrics.get("holdout_start")),
            "train/holdout timestamps are explicit",
        ),
        ResearchDirector._challenge_test(
            "bootstrap_reality_proxy_wrc_like",
            float(metrics.get("wrc_p_value", 1.0) or 1.0) <= 0.25,
            "bootstrap reality proxy WRC-like p-value should pass; not a formal White Reality Check",
        ),
        ResearchDirector._challenge_test(
            "bootstrap_reality_proxy_spa_like",
            float(metrics.get("spa_p_value", 1.0) or 1.0) <= 0.25,
            "bootstrap reality proxy SPA-like p-value should pass; not a formal SPA test",
        ),
        ResearchDirector._challenge_test(
            "deflated_sharpe",
            float(metrics.get("deflated_sharpe_probability", 0.0) or 0.0) >= 0.50,
            "Sharpe must survive trial deflation",
        ),
        ResearchDirector._challenge_test(
            "cost_shock",
            bool(metrics.get("cost_stress_passed", False)),
            "edge must survive stressed costs",
        ),
    ],

```

```

ResearchDirector._challenge_test(
    "parameter_decay",
    bool(metrics.get("edge_decay_passed", False)),
    "nearby R:R variants should keep edge",
),
ResearchDirector._challenge_test(
    "universe_concentration",
    pattern.symbol_count >= 8,
    "edge should not depend on a tiny symbol set",
),
ResearchDirector._challenge_test(
    "time_concentration",
    pattern.year_count >= 2,
    "edge should not depend on a single year",
),
ResearchDirector._challenge_test(
    "market_replay",
    float(replay.get("tradability_score", 0.0)) >= 0.45,
    "execution replay must stay tradable",
),
]
fail_count = sum(1 for test in tests if not test["passed"])
challenge_score = ResearchDirector._bounded(1.0 - fail_count / max(len(tests), 1))
if challenge_score >= 0.78:
    verdict = "survives_adversarial_review"
elif challenge_score >= 0.50:
    verdict = "needs_targeted_confirmation"
else:
    verdict = "fragile_or_overfit"
return {
    "verdict": verdict,
    "challenge_score": round(challenge_score, 5),
    "tests": tests,
    "warnings": [test["name"] for test in tests if not test["passed"]],
}

@staticmethod
def _challenge_test(name: str, passed: bool, rationale: str) -> dict[str, Any]:
    return {"name": name, "passed": bool(passed), "rationale": rationale}

@staticmethod
def _invariance(pattern: DiscoveredPattern, metrics: dict[str, Any]) -> dict[str, Any]:
    by_year = metrics.get("by_year_expectancy") if isinstance(metrics.get("by_year_expectancy"), dict) else {}
    by_symbol = metrics.get("top_symbols_expectancy") if isinstance(metrics.get("top_symbols_expectancy"), dict) else {}
    regime = metrics.get("regime_profile") if isinstance(metrics.get("regime_profile"), dict) else {}
    positive_year_rate = ResearchDirector._positive_value_rate(by_year)
    positive_symbol_rate = ResearchDirector._positive_value_rate(by_symbol)
    purged_rate = float(metrics.get("purged_cv_positive_rate", 0.0) or 0.0)
    stability = float(metrics.get("stability_score", 0.0) or 0.0)
    invariant_score = ResearchDirector._bounded(
        min(pattern.symbol_count / 20.0, 1.0) * 0.20
        + min(pattern.year_count / 5.0, 1.0) * 0.20
        + positive_year_rate * 0.18
        + positive_symbol_rate * 0.14
        + purged_rate * 0.18
        + stability * 0.10
    )
    expected_fail_buckets = []
    if pattern.side == "long":
        expected_fail_buckets.extend(["market_down", "sector_weak"])
    else:
        expected_fail_buckets.extend(["market_up", "sector_strong"])
    if float(metrics.get("avg_fill_probability", 1.0) or 1.0) < 0.65:
        expected_fail_buckets.append("thin_liquidity")
    return {
        "invariant_score": round(invariant_score, 5),
        "positive_year_rate": round(positive_year_rate, 5),
        "positive_symbol_rate": round(positive_symbol_rate, 5),
        "purged_cv_positive_rate": round(purged_rate, 5),
        "dominant_regime": regime.get("dominant_regime", "unknown"),
        "expected_fail_buckets": expected_fail_buckets,
        "evidence": {
            "symbol_count": pattern.symbol_count,
            "year_count": pattern.year_count,
            "by_year_expectancy": by_year,
            "top_symbols_expectancy": by_symbol,
        },
    },

@staticmethod
def _foundation_learning_digest(pattern: DiscoveredPattern, metrics: dict[str, Any]) -> dict[str, Any]:
    centroid = np.asarray(pattern.centroid_json or [], dtype=float)
    finite_ratio = float(np.mean(np.isfinite(centroid))) if len(centroid) else 0.0
    norm = float(np.linalg.norm(np.nan_to_num(centroid))) if len(centroid) else 0.0
    feature_summary = pattern.feature_summary_json or {}
    feature_completeness = min(len(feature_summary) / 30.0, 1.0)
    contrastive = 1.0 - float(metrics.get("run_max_family_similarity", metrics.get("registry_similarity", 0.0)) or 0.0)
    path_alignment = ResearchDirector._bounded(
        float(metrics.get("mfe_before_mae_rate", 0.0) or 0.0) * 0.35
        + float(metrics.get("fast_target_rate", 0.0) or 0.0) * 0.25
        + max(0.0, float(metrics.get("expectancy_lift_r", 0.0) or 0.0)) * 0.20
        + max(0.0, float(metrics.get("out_of_sample_expectancy_r", 0.0) or 0.0)) * 0.20
    )
    teacher_score = ResearchDirector._bounded(
        finite_ratio * 0.20
        + feature_completeness * 0.20

```

```

        + max(0.0, min(contrastive, 1.0)) * 0.20
        + path_alignment * 0.30
        + min(norm / max(len(centroid), 1), 1.0) * 0.10
    )
    return {
        "method": "self_supervised_proxy_teacher_v1",
        "masked_reconstruction_stability": round(finite_ratio * feature_completeness, 5),
        "contrastive_family_separation": round(max(0.0, min(contrastive, 1.0)), 5),
        "future_proxy_alignment": round(path_alignment, 5),
        "embedding_teacher_score": round(teacher_score, 5),
        "note": "Lightweight local proxy; no neural training or external dependency required.",
    }

@staticmethod
def _lifecycle(
    pattern: DiscoveredPattern,
    metrics: dict[str, Any],
    adversarial: dict[str, Any],
    invariance: dict[str, Any],
) -> dict[str, Any]:
    challenge = float(adversarial.get("challenge_score", 0.0))
    invariant = float(invariance.get("invariant_score", 0.0))
    status = "discovered"
    if not pattern.validation_passed:
        status = "needs_confirmation" if metrics.get("confirmation_recommended") else "rejected_learning_memory"
    elif str(pattern.drift_status or "") == "degrading":
        status = "decaying"
    elif challenge >= 0.78 and invariant >= 0.60:
        status = "confirmed_lab_hypothesis"
    elif challenge >= 0.50:
        status = "challenged_lab_hypothesis"
    else:
        status = "fragile_lab_hypothesis"
    next_action = {
        "confirmed_lab_hypothesis": "send to Director paper-review queue; still blocked from live execution",
        "challenged_lab_hypothesis": "run targeted confirmation in weak buckets before any paper promotion",
        "fragile_lab_hypothesis": "keep as learning memory; do not allocate paper validation yet",
        "decaying": "monitor for resurrection only in original dominant regime",
        "needs_confirmation": str(metrics.get("confirmation_next_action", "expand sample and rerun")),
        "rejected_learning_memory": "retain as negative research memory and avoid near-duplicate runs",
        "discovered": "continue observation",
    }.get(status, "continue observation")
    return {
        "status": status,
        "next_action": next_action,
        "paper_live_blocked": True,
        "resurrectable_under": invariance.get("dominant_regime", "unknown"),
        "health_score": round(ResearchDirector._bounded(challenge * 0.55 + invariant * 0.45), 5),
    }

@staticmethod
def _paper_summary(
    pattern: DiscoveredPattern,
    hypothesis: dict[str, Any],
    adversarial: dict[str, Any],
    invariance: dict[str, Any],
    replay: dict[str, Any],
    lifecycle: dict[str, Any],
) -> dict[str, Any]:
    return {
        "title": f"{pattern.name} research note",
        "abstract": (
            f"{pattern.side} pattern in {pattern.timeframe}/W{pattern.window_size}: "
            f"{hypothesis['mechanism']} Current lifecycle={lifecycle['status']}."
        ),
        "rule": hypothesis["thesis"],
        "evidence": hypothesis["evidence"],
        "anti_overfit": adversarial,
        "regime": invariance,
        "execution": replay,
        "risks": adversarial["warnings"] + replay["bottlenecks"],
        "death_conditions": hypothesis["kill_criteria"],
        "next_experiment": lifecycle["next_action"],
    }

def _memory_graph(
    self,
    patterns: list[DiscoveredPattern],
    intelligence_by_key: dict[str, dict[str, Any]],
) -> dict[str, Any]:
    nodes: list[dict[str, Any]] = []
    edges: list[dict[str, Any]] = []
    families: dict[str, list[DiscoveredPattern]] = {}
    regimes: dict[str, list[str]] = {}
    for pattern in patterns:
        intelligence = intelligence_by_key.get(pattern.pattern_key, {})
        lifecycle = intelligence.get("lifecycle", {})
        invariance = intelligence.get("invariance", {})
        regime = str(invariance.get("dominant_regime", "unknown"))
        family = pattern.pattern_family_key or pattern.canonical_pattern_key or pattern.pattern_key
        families.setdefault(family, []).append(pattern)
        regimes.setdefault(regime, []).append(pattern.pattern_key)
        nodes.append(
            {
                "pattern_key": pattern.pattern_key,
                "name": pattern.name,
            }
        )

```

```

        "family": family,
        "canonical": pattern.canonical_pattern_key or pattern.pattern_key,
        "variant_key": pattern.variant_key,
        "side": pattern.side,
        "timeframe": pattern.timeframe,
        "window_size": pattern.window_size,
        "score": round(float(pattern.score or 0.0), 5),
        "status": pattern.promotion_status,
        "lifecycle": lifecycle.get("status", "unknown"),
        "dominant_regime": regime,
    }
)
for family, members in sorted(families.items()):
    if len(members) <= 1:
        continue
    canonical = sorted(members, key=lambda p: (-(p.score or 0.0), p.pattern_key))[0]
    for member in members:
        if member.pattern_key == canonical.pattern_key:
            continue
        edges.append(
            {
                "source": member.pattern_key,
                "target": canonical.pattern_key,
                "type": "variant_of",
                "family": family,
            }
        )
for regime, pattern_keys in sorted(regimes.items()):
    if regime == "unknown" or len(pattern_keys) <= 1:
        continue
    hub = f"regime::{regime}"
    nodes.append({"pattern_key": hub, "name": regime, "type": "regime"})
    for pattern_key in pattern_keys[:40]:
        edges.append({"source": pattern_key, "target": hub, "type": "works_in_regime"})
return {
    "node_count": len(nodes),
    "edge_count": len(edges),
    "nodes": nodes[:500],
    "edges": edges[:1000],
    "families": {
        family: {
            "count": len(members),
            "best_score": round(max(float(p.score or 0.0) for p in members), 5),
            "statuses": sorted([p.promotion_status for p in members]),
        }
        for family, members in sorted(families.items())
    },
}

def _active_learning_agenda(
    self,
    patterns: list[DiscoveredPattern],
    intelligence_by_key: dict[str, dict[str, Any]],
    memory_graph: dict[str, Any],
) -> list[dict[str, Any]]:
    agenda: list[dict[str, Any]] = []
    family_counts = {
        family: int(summary.get("count", 0))
        for family, summary in (memory_graph.get("families") or {}).items()
        if isinstance(summary, dict)
    }
    for pattern in patterns:
        intelligence = intelligence_by_key.get(pattern.pattern_key, {})
        lifecycle = intelligence.get("lifecycle", {})
        adversarial = intelligence.get("adversarial_challenge", {})
        foundation = intelligence.get("foundation_learning", {})
        metrics = pattern.metrics_json or {}
        family = pattern.pattern_family_key or pattern.canonical_pattern_key or pattern.pattern_key
        priority = 0.0
        reasons: list[str] = []
        experiment = "observe"
        if metrics.get("confirmation_recommended"):
            priority += 0.35
            reasons.append("underpowered_edge")
            experiment = "expand_universe_same_hypothesis"
        if float(metrics.get("expected_information_gain", 0.0) or 0.0) > 0.05:
            priority += 0.20
            reasons.append("high_information_gain")
            experiment = "sample_similar_regime_more_deeply"
        if adversarial.get("verdict") == "needs_targeted_confirmation":
            priority += 0.18
            reasons.append("adversarial_weakness")
            experiment = "target_failed_adversarial_tests"
        if lifecycle.get("status") == "decaying":
            priority += 0.12
            reasons.append("edge_decay")
            experiment = "run_regime_resurrection_check"
        if family_counts.get(family, 0) >= 4:
            priority -= 0.10
            reasons.append("family_saturated")
        if float(foundation.get("embedding_teacher_score", 0.0) or 0.0) < 0.35:
            priority += 0.08
            reasons.append("weak_embedding_teacher_signal")
        if priority <= 0 and pattern.validation_passed:
            priority = 0.05
            reasons.append("periodic_health_check")

```



```

        if priority > 0:
            agenda.append(
                {
                    "pattern_key": pattern.pattern_key,
                    "name": pattern.name,
                    "priority": round(self._bounded(priority), 5),
                    "experiment": experiment,
                    "reasons": reasons,
                    "blocked_from_execution": True,
                }
            )
        return sorted(agenda, key=lambda item: float(item["priority"]), reverse=True)[:25]

    @staticmethod
    def _director_state(
        patterns: list[DiscoveredPattern],
        intelligence_by_key: dict[str, dict[str, Any]],
        agenda: list[dict[str, Any]],
    ) -> dict[str, Any]:
        lifecycle_counts: dict[str, int] = {}
        challenge_scores = []
        invariant_scores = []
        for intelligence in intelligence_by_key.values():
            lifecycle = intelligence.get("lifecycle", {})
            status = str(lifecycle.get("status", "unknown"))
            lifecycle_counts[status] = lifecycle_counts.get(status, 0) + 1
            adversarial = intelligence.get("adversarial_challenge", {})
            invariance = intelligence.get("invariance", {})
            challenge_scores.append(float(adversarial.get("challenge_score", 0.0) or 0.0))
            invariant_scores.append(float(invariance.get("invariant_score", 0.0) or 0.0))
        return {
            "pattern_count": len(patterns),
            "lifecycle_counts": lifecycle_counts,
            "avg_challenge_score": round(float(np.mean(challenge_scores)), 5) if challenge_scores else 0.0,
            "avg_invariant_score": round(float(np.mean(invariant_scores)), 5) if invariant_scores else 0.0,
            "agenda_count": len(agenda),
            "top_agenda": agenda[:5],
        }

    def _write_artifacts(
        self,
        summary: dict[str, Any],
        patterns: list[DiscoveredPattern],
        intelligence_by_key: dict[str, dict[str, Any]],
    ) -> dict[str, str]:
        settings = self.settings
        assert settings is not None
        director_dir = settings.reports_path / "research" / "director"
        director_dir.mkdir(parents=True, exist_ok=True)
        ts = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
        payload = {
            "summary": summary,
            "patterns": [
                {
                    "pattern_key": pattern.pattern_key,
                    "name": pattern.name,
                    "intelligence": intelligence_by_key.get(pattern.pattern_key, {}),
                }
                for pattern in patterns
            ],
        }
        json_path = director_dir / f"research_director_{ts}.json"
        md_path = director_dir / f"research_director_{ts}.md"
        graph_path = director_dir / "research_memory_graph.json"
        latest_json = director_dir / "latest_research_director.json"
        latest_md = director_dir / "latest_research_director.md"
        json_text = json.dumps(self._json_clean(payload), indent=2, ensure_ascii=False, default=str)
        json_path.write_text(json_text, encoding="utf-8")
        latest_json.write_text(json_text, encoding="utf-8")
        graph_path.write_text(
            json.dumps(summary["memory_graph"], indent=2, ensure_ascii=False, default=str),
            encoding="utf-8",
        )
        md_text = self._markdown(summary, patterns, intelligence_by_key)
        md_path.write_text(md_text, encoding="utf-8")
        latest_md.write_text(md_text, encoding="utf-8")
        return {
            "json": str(json_path),
            "markdown": str(md_path),
            "latest_json": str(latest_json),
            "latest_markdown": str(latest_md),
            "memory_graph": str(graph_path),
        }

    @staticmethod
    def _markdown(
        summary: dict[str, Any],
        patterns: list[DiscoveredPattern],
        intelligence_by_key: dict[str, dict[str, Any]],
    ) -> str:
        lines = [
            "# Tradeo Autonomous Research Director",
            "",
            "## Executive State",
            f"- Generated: {summary['generated_at']}",
            f"- Patterns reviewed: {summary['patterns_reviewed']}",
        ]

```

```

f"- Hypotheses created: {summary['hypotheses_created']}",
f"- Memory graph: {summary['memory_graph']['node_count']} nodes / {summary['memory_graph']['edge_count']} edges",
f"- Agenda items: {len(summary['active_learning_agenda'])}",
"",
"## Active Learning Agenda",
]
for item in summary["active_learning_agenda"][:12]:
    lines.append(
        f"- {item['priority']}: {item['name']} -> {item['experiment']} ({', '.join(item['reasons'])})"
    )
lines.extend(["", "## Research Notes"])
for pattern in patterns[:20]:
    intelligence = intelligence_by_key.get(pattern.pattern_key, {})
    paper = intelligence.get("paper", {})
    lifecycle = intelligence.get("lifecycle", {})
    adversarial = intelligence.get("adversarial_challenge", {})
    invariance = intelligence.get("invariance", {})
    replay = intelligence.get("market_replay", {})
    lines.extend(
        [
            "",
            f"### {pattern.name}",
            f"- Lifecycle: {lifecycle.get('status')} / health {lifecycle.get('health_score')}",
            f"- Abstract: {paper.get('abstract')}",
            f"- Rule: {paper.get('rule')}",
            f"- Adversarial: {adversarial.get('verdict')} / score {adversarial.get('challenge_score')}",
            f"- Invariance: {invariance.get('invariant_score')} / regime {invariance.get('dominant_regime')}",
            f"- Replay: tradability {replay.get('tradability_score')} / bottlenecks {replay.get('bottlenecks')}",
            f"- Next: {paper.get('next_experiment')}",
        ]
    )
return "\n".join(lines) + "\n"

@staticmethod
def _positive_value_rate(values: dict[str, Any]) -> float:
    if not values:
        return 0.0
    numeric = []
    for value in values.values():
        try:
            numeric.append(float(value))
        except (TypeError, ValueError):
            continue
    if not numeric:
        return 0.0
    return float(sum(1 for value in numeric if value > 0) / len(numeric))

@staticmethod
def _bounded(value: float) -> float:
    return float(max(0.0, min(1.0, value)))

@staticmethod
def _json_clean(value: Any) -> Any:
    if isinstance(value, np.generic):
        return value.item()
    if isinstance(value, dict):
        return {str(k): ResearchDirector._json_clean(v) for k, v in value.items()}
    if isinstance(value, list):
        return [ResearchDirector._json_clean(v) for v in value]
    if isinstance(value, tuple):
        return [ResearchDirector._json_clean(v) for v in value]
    return value

```

Full Source: ``backend/tradeo/research/sequential_tests.py``

``backend/tradeo/research/sequential_tests.py`` ? 165 lines, 5979 bytes, sha256 prefix ``eebad6b9933640e2``

Public surface summary before full source:

```

? function `posterior_probability_edge` line 35`
? function `sprt_inferiority` line 77`
? function `ks_two_sample` line 137`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

"""Sequential evidence tests for the Director review loop (informe §4.7).

Pure functions, numpy + stdlib only. They turn "n=10 lab trades" from a pseudo-decision into a review trigger backed by three orthogonal checks:

- ``posterior_probability_edge``: Bayesian shrinkage of the lab expectancy toward the Research prior. With few trades the posterior leans on Research; with many, on the Lab. Promotable only if $P(E_{\text{lab}} > \text{min_edge})$ is high.
 - ``sprt_inferiority``: Wald SPRT of H_0 "the researched edge is intact" ($E = E_{\text{research}}$) against H_1 "there is no edge" ($E = 0$). A fast kill: a pattern with a run of bad trades goes back to lab without waiting for the full evidence quota.
 - ``ks_two_sample``: distribution comparison between Lab R and Research R. A large KS with similar means flags a mechanism change (e.g. gap-through stops that Research never saw) rather than mere noise.
- """

```

from __future__ import annotations

import math
from statistics import NormalDist
from typing import Sequence

import numpy as np

_PHI = NormalDist().cdf

__all__ = [
    "posterior_probability_edge",
    "sprt_inferiority",
    "ks_two_sample",
]

def posterior_probability_edge(
    lab_r: Sequence[float],
    *,
    prior_mean: float,
    prior_sd: float,
    min_edge_r: float = 0.10,
) -> dict:
    """P(E_lab > min_edge_r | datos) with a conjugate Normal prior.

    Prior:  $E \sim \text{Normal}(\text{prior\_mean}, \text{prior\_sd}^2)$  ? the Research expectancy and an
    honest uncertainty around it (informe §4.7.2). Likelihood: lab R i.i.d.
    Normal( $E, \sigma^2$ ) with  $\sigma^2$  estimated from the lab sample (fallback to
    the prior sd when  $n < 3$ ). Returns the posterior and the probability that
    the true expectancy clears ``min_edge_r``.
    """
    x = np.asarray(lab_r, dtype=float)
    n = int(x.size)
    out = {
        "n": n,
        "prior_mean": float(prior_mean),
        "prior_sd": float(prior_sd),
        "min_edge_r": float(min_edge_r),
        "posterior_mean": float("nan"),
        "posterior_sd": float("nan"),
        "probability_edge": float("nan"),
    }
    if n == 0 or not (prior_sd > 0):
        return out
    sample_sd = float(x.std(ddof=1)) if n >= 3 else float(prior_sd)
    if not (sample_sd > 0):
        sample_sd = max(float(prior_sd), 1e-6)
    prior_prec = 1.0 / (prior_sd * prior_sd)
    data_prec = n / (sample_sd * sample_sd)
    post_var = 1.0 / (prior_prec + data_prec)
    post_mean = post_var * (prior_mean * prior_prec + float(x.mean()) * data_prec)
    post_sd = math.sqrt(post_var)
    out["posterior_mean"] = float(post_mean)
    out["posterior_sd"] = float(post_sd)
    out["probability_edge"] = float(1.0 - _PHI((min_edge_r - post_mean) / post_sd))
    return out

def sprt_inferiority(
    lab_r: Sequence[float],
    *,
    research_mean: float,
    sigma: float,
    alpha: float = 0.05,
    beta: float = 0.20,
) -> dict:
    """Wald SPRT:  $H_0 E = \text{research\_mean}$  (edge intact) vs  $H_1 E = 0$  (no edge).

    Gaussian log-likelihood ratio with known sigma (use the Research R
    standard deviation; per-trade R with asymmetric payoffs is wide, which
    makes the test appropriately conservative). Decisions:

    - ``"no_edge"`` : LLR >=  $\log((1-\beta)/\alpha)$  ? kill fast (informe §4.7.3)
    - ``"edge_intact"`` : LLR <=  $\log(\beta/(1-\alpha))$ 
    - ``"continue"`` : otherwise.

    Returns the running decision plus the LLR trajectory extremes for audit.
    """
    x = np.asarray(lab_r, dtype=float)
    out = {
        "n": int(x.size),
        "research_mean": float(research_mean),
        "sigma": float(sigma),
        "alpha": float(alpha),
        "beta": float(beta),
        "llr": 0.0,
        "upper_threshold": float("nan"),
        "lower_threshold": float("nan"),
        "decision": "continue",
        "decided_at_n": None,
    }
    if x.size == 0 or not (sigma > 0) or not (research_mean > 0):
        out["decision"] = "not_applicable"
        return out
    upper = math.log((1.0 - beta) / alpha)

```

```

lower = math.log(beta / (1.0 - alpha))
out["upper_threshold"] = float(upper)
out["lower_threshold"] = float(lower)
mu0, mu1 = float(research_mean), 0.0
# log f1 - log f0 = ((x-mu0)^2 - (x-mu1)^2) / (2 sigma^2)
increments = ((x - mu0) ** 2 - (x - mu1) ** 2) / (2.0 * sigma * sigma)
llr = 0.0
for i, inc in enumerate(increments):
    llr += float(inc)
    if llr >= upper:
        out["llr"] = float(llr)
        out["decision"] = "no_edge"
        out["decided_at_n"] = i + 1
        return out
    if llr <= lower:
        out["llr"] = float(llr)
        out["decision"] = "edge_intact"
        out["decided_at_n"] = i + 1
        return out
out["llr"] = float(llr)
return out

def ks_two_sample(a: Sequence[float], b: Sequence[float]) -> dict:
    """Two-sample Kolmogorov-Smirnov statistic with asymptotic p-value.

    Compares the Lab R distribution against the Research R distribution for
    the same pattern. Uses the small-sample corrected lambda of Stephens
    (Numerical Recipes): lambda = (sqrt(ne) + 0.12 + 0.11/sqrt(ne)) * D with
    ne = n*m/(n+m), and the Kolmogorov series for the p-value.
    """
    xa = np.sort(np.asarray(a, dtype=float))
    xb = np.sort(np.asarray(b, dtype=float))
    n, m = int(xa.size), int(xb.size)
    out = {"n_a": n, "n_b": m, "statistic": float("nan"), "p_value": float("nan")}
    if n == 0 or m == 0:
        return out
    pooled = np.concatenate([xa, xb])
    cdf_a = np.searchsorted(xa, pooled, side="right") / n
    cdf_b = np.searchsorted(xb, pooled, side="right") / m
    d = float(np.max(np.abs(cdf_a - cdf_b)))
    ne = n * m / (n + m)
    lam = (math.sqrt(ne) + 0.12 + 0.11 / math.sqrt(ne)) * d
    p = 0.0
    for k in range(1, 101):
        term = 2.0 * ((-1.0) ** (k - 1)) * math.exp(-2.0 * (k * lam) ** 2)
        p += term
        if abs(term) < 1e-10:
            break
    out["statistic"] = d
    out["p_value"] = float(min(max(p, 0.0), 1.0))
    return out

```

Full Source: `backend/tradeo/research/research\_director.py`

`backend/tradeo/research/research\_director.py` ? 466 lines, 21410 bytes, sha256 prefix `21aa104056e64302`

Public surface summary before full source:

? class `ResearchDirector` line 21 methods: run

Risk hints: wall-clock timestamp usage.

```

from __future__ import annotations

import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import numpy as np
from loguru import logger

from tradeo.core.config import Settings, get_settings
from tradeo.research.active_learning import ActiveLearningAgenda
from tradeo.research.global_experiment_registry import GlobalExperimentRegistry
from tradeo.research.hypothesis_engine import HypothesisEngine
from tradeo.research.research_memory_graph import ResearchMemoryGraph
from tradeo.research.types import ClusterCandidate, WindowSample

@dataclass(slots=True)
class ResearchDirector:
    """Autonomous research director for discovery completion.

    The director enriches candidates, updates JSON memory, writes paper-style
    artifacts and suggests lifecycle states. It never promotes to paper/live.
    """

    settings: Settings | None = None

    def __post_init__(self) -> None:

```

```

self.settings = self.settings or get_settings()

def run(
    self,
    *,
    run_id: int | str,
    candidates: list[ClusterCandidate],
    samples: list[WindowSample] | None = None,
    params: dict[str, Any] | None = None,
) -> dict[str, Any]:
    settings = self.settings
    assert settings is not None
    logger.info("Research Director: iniciando cierre autonomo del run {}", run_id)
    self._attach_nested_discovery_replay_contract(
        candidates,
        finalist_limit=settings.discovery_report_top_n,
    )
    experiment_registry = GlobalExperimentRegistry(settings.reports_path / "research" / "global_experiment_registry.json")
    experiment_registry_summary = experiment_registry.register(
        candidates,
        run_id=run_id,
        params=params or {},
    )
    hypothesis_engine = HypothesisEngine()
    for candidate in candidates:
        package = hypothesis_engine.build_package(candidate)
        candidate.metrics["research_hypothesis_package"] = package.to_dict()
        candidate.metrics["research_hypothesis"] = package.to_dict()
        candidate.metrics["pattern_lifecycle"] = self._lifecycle(candidate)

    graph = ResearchMemoryGraph(settings.reports_path / "research" / "research_memory_graph.json")
    memory_summary = graph.update(candidates, run_id=run_id, params=params or {})
    logger.info(
        "Research Director: memoria actualizada con {} familias y {} patrones",
        memory_summary.get("family_count", 0),
        memory_summary.get("pattern_count", 0),
    )

    for candidate in candidates:
        candidate.metrics["pattern_lifecycle"] = self._lifecycle(candidate)

    agenda = ActiveLearningAgenda(max_items=max(5, settings.discovery_report_top_n)).build(
        candidates,
        memory_summary=memory_summary,
    )
    self._attach_agenda(candidates, agenda)
    paper_paths = self._write_papers(run_id, candidates, agenda)
    summary = self._summary(
        run_id=run_id,
        candidates=candidates,
        samples=samples or [],
        memory_summary=memory_summary,
        agenda=agenda,
        paper_paths=paper_paths,
        graph_path=graph.path,
        experiment_registry_summary=experiment_registry_summary,
    )
    artifact_paths = self._write_director_artifacts(run_id, summary, candidates, agenda)
    summary["artifact_json_path"] = str(artifact_paths["json"])
    summary["artifact_markdown_path"] = str(artifact_paths["markdown"])
    for candidate in candidates:
        candidate.metrics["research_director"] = {
            "run_id": run_id,
            "summary_path": str(artifact_paths["json"]),
            "memory_graph_path": str(graph.path),
            "agenda_rank": self._agenda_rank(candidate, agenda),
            "paper_live_auto_promotion": False,
            "director_gate_required_for_paper_or_live": True,
            "global_experiment_registry_path": experiment_registry_summary.get("path"),
        }
    logger.info(
        "Research Director: artifacts escritos {} y {}",
        artifact_paths["json"],
        artifact_paths["markdown"],
    )
    return summary

def _write_papers(
    self,
    run_id: int | str,
    candidates: list[ClusterCandidate],
    agenda: dict[str, Any],
) -> list[str]:
    settings = self.settings
    assert settings is not None
    papers_dir = settings.reports_path / "research" / "papers" / f"run_{run_id}"
    papers_dir.mkdir(parents=True, exist_ok=True)
    paths: list[str] = []
    top = sorted(candidates, key=lambda c: c.score, reverse=True)[: settings.discovery_report_top_n]
    for candidate in top:
        path = papers_dir / f"{self._safe_stem(candidate.pattern_key)}.md"
        path.write_text(self._paper_markdown(candidate, agenda), encoding="utf-8")
        candidate.metrics["research_paper_report_path"] = str(path)
        paths.append(str(path))
    return paths

```

```

def _write_director_artifacts(
    self,
    run_id: int | str,
    summary: dict[str, Any],
    candidates: list[ClusterCandidate],
    agenda: dict[str, Any],
) -> dict[str, Path]:
    settings = self.settings
    assert settings is not None
    director_dir = settings.reports_path / "research" / "director"
    director_dir.mkdir(parents=True, exist_ok=True)
    ts = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
    json_path = director_dir / f"run_{run_id}_director_{ts}.json"
    markdown_path = director_dir / f"run_{run_id}_director_{ts}.md"
    payload = {
        "summary": summary,
        "agenda": agenda,
        "patterns": [self._candidate_digest(candidate) for candidate in candidates],
    }
    json_path.write_text(json.dumps(self._json_clean(payload), indent=2, ensure_ascii=False), encoding="utf-8")
    markdown_path.write_text(self._director_markdown(summary, agenda, candidates), encoding="utf-8")
    return {"json": json_path, "markdown": markdown_path}

@staticmethod
def _summary(
    *,
    run_id: int | str,
    candidates: list[ClusterCandidate],
    samples: list[WindowSample],
    memory_summary: dict[str, Any],
    agenda: dict[str, Any],
    paper_paths: list[str],
    graph_path: Path,
    experiment_registry_summary: dict[str, Any],
) -> dict[str, Any]:
    lifecycle_counts: dict[str, int] = {}
    challenge_scores: list[float] = []
    replay_expectancies: list[float] = []
    for candidate in candidates:
        lifecycle = candidate.metrics.get("pattern_lifecycle", {})
        state = str(lifecycle.get("state", "unknown")) if isinstance(lifecycle, dict) else "unknown"
        lifecycle_counts[state] = lifecycle_counts.get(state, 0) + 1
        challenge = candidate.metrics.get("adversarial_challenge", {})
        if isinstance(challenge, dict):
            challenge_scores.append(float(challenge.get("challenge_score", 0.0)))
        replay = candidate.metrics.get("market_replay", {})
        if isinstance(replay, dict):
            replay_expectancies.append(float(replay.get("expected_expectancy_r", 0.0)))
    return {
        "run_id": run_id,
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "candidate_count": len(candidates),
        "sample_count": len(samples),
        "lifecycle_counts": lifecycle_counts,
        "avg_challenge_score": round(float(np.mean(challenge_scores)), 5) if challenge_scores else 0.0,
        "avg_replay_expectancy_r": round(float(np.mean(replay_expectancies)), 5)
        if replay_expectancies
        else 0.0,
        "memory_graph_path": str(graph_path),
        "memory_summary": memory_summary,
        "global_experiment_registry": experiment_registry_summary,
        "active_learning": {
            "experiment_count": agenda.get("experiment_count", 0),
            "kind_counts": agenda.get("kind_counts", {}),
            "top_experiments": agenda.get("experiments", [])[:10],
        },
        "paper_reports_written": len(paper_paths),
        "paper_report_paths": paper_paths,
        "paper_live_auto_promotion": False,
        "director_gate_required_for_paper_or_live": True,
    }

@staticmethod
def _attach_nested_discovery_replay_contract(
    candidates: list[ClusterCandidate],
    *,
    finalist_limit: int,
) -> None:
    finalists = {
        candidate.pattern_key: rank
        for rank, candidate in enumerate(
            sorted(candidates, key=lambda c: c.score, reverse=True)[:max(1, finalist_limit)],
            start=1,
        )
    }
    for candidate in candidates:
        rank = finalists.get(candidate.pattern_key)
        if rank is None:
            candidate.metrics["nested_discovery_replay"] = {
                "status": "not_finalist",
                "implemented": False,
                "blocking": False,
                "reason": "nested discovery replay contract applies to finalists only",
            }
            continue
        candidate.metrics["nested_discovery_replay"] = {

```

```

        "status": "blocked_contract",
        "implemented": False,
        "blocking": True,
        "finalist_rank": rank,
        "required_before": [
            "edge_claim_upgrade",
            "paper_candidate",
            "live_candidate",
        ],
        "blocking_reason": (
            "nested discovery replay for finalists is not implemented in this run; "
            "edge_claim remains NO_DEMOSTRADO"
        ),
        "minimum_contract": {
            "replay_original_discovery": "same params, frozen thresholds, fresh split",
            "compare_selection": "finalist must reappear without descriptive_all metrics in selection",
            "record": "selection_split, fit_scope, global_trial_count and event_ledger_hash",
        },
    },
}

@staticmethod
def _attach_agenda(candidates: list[ClusterCandidate], agenda: dict[str, Any]) -> None:
    by_pattern: dict[str, list[dict[str, Any]]] = {}
    for experiment in agenda.get("experiments", []):
        if not isinstance(experiment, dict):
            continue
        pattern_key = str(experiment.get("pattern_key", ""))
        by_pattern.setdefault(pattern_key, []).append(experiment)
    for candidate in candidates:
        experiments = by_pattern.get(candidate.pattern_key, [])
        candidate.metrics["active_learning"] = {
            "experiments": experiments[:5],
            "top_priority": float(experiments[0].get("priority", 0.0)) if experiments else 0.0,
            "next_action": str(experiments[0].get("action", "")) if experiments else "",
        }

@staticmethod
def _agenda_rank(candidate: ClusterCandidate, agenda: dict[str, Any]) -> int | None:
    for experiment in agenda.get("experiments", []):
        if isinstance(experiment, dict) and experiment.get("pattern_key") == candidate.pattern_key:
            rank = experiment.get("rank")
            return int(rank) if rank is not None else None
    return None

@staticmethod
def _lifecycle(candidate: ClusterCandidate) -> dict[str, Any]:
    metrics = candidate.metrics
    challenge = metrics.get("adversarial_challenge", {})
    replay = metrics.get("market_replay", {})
    causal = metrics.get("causal_invariance", {})
    memory = metrics.get("research_memory", {})
    replay_expectancy = float(replay.get("expected_expectancy_r", 0.0)) if isinstance(replay, dict) else 0.0
    challenge_score = float(challenge.get("challenge_score", 0.0)) if isinstance(challenge, dict) else 0.0
    invariance = float(causal.get("invariance_score", 0.0)) if isinstance(causal, dict) else 0.0
    decay_score = max(
        float(metrics.get("edge_decay_parameter_score", 0.0) or 0.0),
        float(memory.get("family_decay_score", 0.0) or 0.0) if isinstance(memory, dict) else 0.0,
    )
    rejection = bool(challenge.get("rejection_recommended", False)) if isinstance(challenge, dict) else False
    if decay_score >= 0.65 and replay_expectancy <= 0:
        state = "retired"
    elif decay_score >= 0.45:
        state = "decaying"
    elif not candidate.validation_passed or rejection:
        if replay_expectancy > 0.15 and challenge_score >= 0.50:
            state = "resurrectable"
        else:
            state = "challenged"
    elif replay_expectancy > 0 and challenge_score >= 0.55 and invariance >= 0.45:
        state = "confirmed"
    else:
        state = "discovered"
    return {
        "state": state,
        "allowed_states": [
            "discovered",
            "challenged",
            "confirmed",
            "decaying",
            "retired",
            "resurrectable",
        ],
        "suggestion_only": True,
        "paper_live_auto_promotion": False,
        "director_gate_required_for_paper_or_live": True,
        "reasons": {
            "validation_passed": candidate.validation_passed,
            "challenge_score": round(challenge_score, 5),
            "replay_expectancy_r": round(replay_expectancy, 5),
            "invariance_score": round(invariance, 5),
            "decay_score": round(decay_score, 5),
        },
    },

@staticmethod
def _candidate_digest(candidate: ClusterCandidate) -> dict[str, Any]:

```

```

metrics = candidate.metrics
return {
    "pattern_key": candidate.pattern_key,
    "name": candidate.name,
    "side": candidate.side,
    "score": candidate.score,
    "validation_passed": candidate.validation_passed,
    "promotion_status": metrics.get("promotion_status"),
    "lifecycle": metrics.get("pattern_lifecycle", {}),
    "hypothesis": metrics.get("research_hypothesis", {}),
    "hypothesis_package": metrics.get("research_hypothesis_package", {}),
    "nested_discovery_replay": metrics.get("nested_discovery_replay", {}),
    "global_experiment_registry": metrics.get("global_experiment_registry", {}),
    "market_replay": metrics.get("market_replay", {}),
    "adversarial_challenge": metrics.get("adversarial_challenge", {}),
    "causal_invariance": metrics.get("causal_invariance", {}),
    "foundation_teacher": metrics.get("foundation_teacher", {}),
    "research_memory": metrics.get("research_memory", {}),
    "active_learning": metrics.get("active_learning", {}),
    "paper_report_path": metrics.get("research_paper_report_path"),
}

@staticmethod
def _paper_markdown(candidate: ClusterCandidate, agenda: dict[str, Any]) -> str:
    metrics = candidate.metrics
    hypothesis = metrics.get("research_hypothesis", {})
    replay = metrics.get("market_replay", {})
    challenge = metrics.get("adversarial_challenge", {})
    causal = metrics.get("causal_invariance", {})
    lifecycle = metrics.get("pattern_lifecycle", {})
    next_experiments = [
        exp
        for exp in agenda.get("experiments", [])
        if isinstance(exp, dict) and exp.get("pattern_key") == candidate.pattern_key
    ][0:5]
    lines = [
        f"# Research Paper: {candidate.name}",
        "",
        "## Abstract",
        str(hypothesis.get("thesis", "")),
        "",
        "## Rule",
        str(hypothesis.get("rule", "")),
        "",
        "## Evidence",
        "`json`,",
        json.dumps(hypothesis.get("evidence_accumulated", {}), indent=2, ensure_ascii=False, default=str),
        "`",
        "",
        "## Hypothesis Package",
        f"- Edge claim: {hypothesis.get('edge_claim', 'NO_DEMOSTRADO')}",
        f"- Family: {hypothesis.get('family_id')}",
        f"- Variant: {hypothesis.get('variant_id')}",
        f"- Global trial count: {hypothesis.get('global_trial_count')}",
        f"- Event ledger hash: {hypothesis.get('event_ledger_hash')}",
        f"- Nested discovery replay: {hypothesis.get('nested_discovery_replay')}",
        "",
        "## Anti-Overfit",
        f"- Challenge score: {challenge.get('challenge_score') if isinstance(challenge, dict) else None}",
        f"- Challenge passed: {challenge.get('challenge_passed') if isinstance(challenge, dict) else None}",
        f"- Warnings: {challenge.get('warnings', []) if isinstance(challenge, dict) else []}",
        "",
        "## Regime",
        f"- Invariance score: {causal.get('invariance_score') if isinstance(causal, dict) else None}",
        f"- Expected fail buckets: {causal.get('expected_fail_buckets', []) if isinstance(causal, dict) else []}",
        "",
        "## Execution",
        f"- Replay expectancy: {replay.get('expected_expectancy_r') if isinstance(replay, dict) else None}R",
        f"- Fill ratio: {replay.get('avg_fill_ratio') if isinstance(replay, dict) else None}",
        f"- Latency bars: {replay.get('latencyBars') if isinstance(replay, dict) else None}",
        "",
        "## Risks",
        "\n".join(f"- {item}" for item in hypothesis.get("fails_when", [])),
        "",
        "## Death Conditions",
        "\n".join(f"- {item}" for item in hypothesis.get("kill_criteria", [])),
        "",
        "## Next Experiments",
        "\n".join(f"- {item.get('kind')}: {item.get('action')}" for item in next_experiments)
        or "- No active experiment queued",
        "",
        "## Lifecycle",
        f"- State suggestion: {lifecycle.get('state') if isinstance(lifecycle, dict) else None}",
        "- Paper/live auto-promotion: false",
        "- Director gate required: true",
        ""
    ]
    return "\n".join(lines)

@staticmethod
def _director_markdown(
    summary: dict[str, Any],
    agenda: dict[str, Any],
    candidates: list[ClusterCandidate],
) -> str:
    lines = [

```



```

f"# Tradeo Research Director Run {summary['run_id']}",
"",
"## Summary",
f"- Candidates: {summary['candidate_count']}",
f"- Lifecycle counts: {summary['lifecycle_counts']}",
f"- Avg challenge score: {summary['avg_challenge_score']}",
f"- Avg replay expectancy: {summary['avg_replay_expectancy_r']}R",
f"- Memory graph: {summary['memory_graph_path']}",
f"- Global experiment registry: {summary.get('global_experiment_registry', {})}",
f"- Paper reports: {summary['paper_reports_written']}",
f"- Paper/live auto-promotion: false",
f"- Director gate required: true",
"",
"## Active Learning Agenda",
]
for experiment in agenda.get("experiments", [])[20]:
    if not isinstance(experiment, dict):
        continue
    lines.append(
        f"- #{experiment.get('rank')} {experiment.get('kind')} "
        f"{experiment.get('pattern_key')}: {experiment.get('action')}"
    )
lines.extend(["", "## Top Patterns"])
for candidate in sorted(candidates, key=lambda c: c.score, reverse=True)[:10]:
    lifecycle = candidate.metrics.get("pattern_lifecycle", {})
    challenge = candidate.metrics.get("adversarial_challenge", {})
    replay = candidate.metrics.get("market_replay", {})
    lines.append(
        f"- {candidate.name}: state={lifecycle.get('state')} if isinstance(lifecycle, dict) else None, "
        f"challenge={challenge.get('challenge_score')} if isinstance(challenge, dict) else None, "
        f"replay={replay.get('expected_expectancy_r')} if isinstance(replay, dict) else None}R"
    )
return "\n".join(lines)

@staticmethod
def _safe_stem(value: object) -> str:
    stem = "".join(ch if ch.isalnum() or ch in {"-", "_", "."} else "_" for ch in str(value))
    return stem.strip("_")[:120] or "pattern"

@staticmethod
def _json_clean(value: Any) -> Any:
    if isinstance(value, np.generic):
        return value.item()
    if isinstance(value, Path):
        return str(value)
    if isinstance(value, dict):
        return {str(k): ResearchDirector._json_clean(v) for k, v in value.items()}
    if isinstance(value, list):
        return [ResearchDirector._json_clean(v) for v in value]
    if isinstance(value, tuple):
        return [ResearchDirector._json_clean(v) for v in value]
    return value

```

Full Source: `backend/tradeo/agents/pattern\_discovery\_lab\_agent.py`

`backend/tradeo/agents/pattern\_discovery\_lab\_agent.py` ? 869 lines, 47179 bytes, sha256 prefix `80710a4f8c03e890`

Audit relevance: Orchestrates discovery runs, persistence, global trial counts, reports, and ValidationGate. Audit end-to-end research reproducibility.

Public surface summary before full source:

? class `PatternDiscoveryLabAgent` line 39 methods: run

Risk hints: broad `except Exception` present; wall-clock timestamp usage; IBKR/broker/data dependency.

```

from __future__ import annotations

import gzip
import hashlib
import json
import time
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import numpy as np
from loguru import logger
from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, DiscoveryRun
from tradeo.research.cluster_research_engine import ClusterResearchEngine
from tradeo.research.autonomous_research_director import ResearchDirector as PersistentResearchDirector
from tradeo.research.determinism import CONTENT_HASH_ALGO, DEFAULT_VOLATILE_KEYS, content_hash
from tradeo.research.global_experiment_registry import GlobalExperimentRegistry
from tradeo.research.novel_pattern_registry import NovelPatternRegistry
from tradeo.research.quant_validation import (
    benjamini_hochberg,
    deflated_sharpe_ratio,
    probabilistic_sharpe_ratio,
)

```

```

from tradeo.research.research_director import ResearchDirector as CandidateResearchDirector
from tradeo.research.validation_gate import ValidationGate
from tradeo.research.window_sampler import WindowSampler
from tradeo.schemas import DiscoveryRunRequest, DiscoveryRunResponse
from tradeo.services.data_provider import MarketDataProvider, pick_symbols
from tradeo.services.market_regime import MarketRegimeService
from tradeo.services.provider_factory import get_market_data_provider
from tradeo.services.universe_snapshot import UniverseSnapshotService

@dataclass(slots=True)
class PatternDiscoveryLabAgent:
    """Autonomous laboratory agent for unknown technical pattern discovery.

    This agent is intentionally unable to create orders. Its output is a compact,
    token-efficient digest: the LLM supervisor receives only accepted/rejected
    cluster statistics and representative examples, never millions of raw bars.
    """

    provider: MarketDataProvider | None = None
    settings: Settings | None = None

    def __post_init__(self) -> None:
        self.settings = self.settings or get_settings()
        self.provider = self.provider or get_market_data_provider()

    def run(self, request: DiscoveryRunRequest, db: Session) -> DiscoveryRunResponse:
        settings = self.settings
        assert settings is not None
        started = time.perf_counter()
        warnings: list[str] = []
        params = self._resolve_params(request)
        run = DiscoveryRun(status="running", params_json=params)
        db.add(run)
        db.commit()
        db.refresh(run)
        logger.info("pattern discovery run {} started with params {}", run.id, params)

        try:
            symbols = self._resolve_symbols(request, params)
            universe_snapshot = self._build_universe_snapshot(warnings)
            samples = []
            sampler = WindowSampler(target_r=settings.discovery_min_reward_risk)
            benchmark_frames = self._benchmark_frames(params, warnings)
            benchmark_regime_table = self._benchmark_regime_table(params, warnings)
            for symbol in symbols:
                if len(samples) >= params["max_total_windows"]:
                    break
                try:
                    df = self.provider.fetch_ohlc(symbol, period=params["period"], interval=params["interval"])
                    symbol_samples = sampler.sample(
                        symbol=symbol,
                        df=df,
                        timeframe=params["interval"],
                        window_sizes=params["window_sizes"],
                        forward_bars=params["forward_bars"],
                        stride=params["stride"],
                        max_windows_per_symbol=params["max_windows_per_symbol"],
                        benchmark_frames=benchmark_frames,
                    )
                    remaining = params["max_total_windows"] - len(samples)
                    samples.extend(symbol_samples[:remaining])
                except Exception as exc: # noqa: BLE001
                    msg = f"{symbol}: {exc}"
                    logger.warning("discovery symbol failed: {}", msg)
                    warnings.append(msg)
                    continue

            engine = ClusterResearchEngine(
                min_cluster_size=params["min_cluster_size"],
                max_clusters_per_window=params["max_clusters_per_window"],
                target_r=settings.discovery_min_reward_risk,
                out_of_sample_pct=settings.discovery_out_of_sample_pct,
                rr_levels=params["rr_levels"],
                min_samples=settings.discovery_min_samples,
                walk_forward_folds=settings.discovery_walk_forward_folds,
                walk_forward_embargo_samples=settings.discovery_walk_forward_embargo_samples,
                cost_stress_multipliers=settings.discovery_cost_stress_multiplier_list,
                required_cost_stress_multiplier=settings.discovery_required_cost_stress_multiplier,
                event_ledger_limit=0,
                match_tau_percentile=settings.discovery_match_tau_percentile,
                benchmark_regime_table=benchmark_regime_table,
            )
            raw_candidates = engine.discover(samples)
            data_manifest = self._data_manifest(run.id, symbols, warnings)
            for candidate in raw_candidates:
                candidate.metrics["data_manifest"] = {
                    "path": data_manifest.get("path"),
                    "manifest_hash": data_manifest.get("manifest_hash"),
                    "entry_count": len((data_manifest.get("entries") or {})),
                    "artifact_format": data_manifest.get("artifact_format"),
                }
                candidate.metrics["universe_snapshot"] = universe_snapshot
                candidate.metrics["universe_point_in_time"] = bool(
                    universe_snapshot.get("point_in_time", settings.universe_point_in_time_available)
                )

```

```

        candidate.metrics["survivorship_biased"] = not bool(candidate.metrics["universe_point_in_time"])
self._apply_run_level_inference(raw_candidates)
candidates = ValidationGate(settings).evaluate_many(raw_candidates)
accepted = [c for c in candidates if c.validation_passed]
rejected = [c for c in candidates if not c.validation_passed]
ledger_artifacts = self._write_event_ledgers(run.id, candidates)
global_registry = GlobalExperimentRegistry(
    settings.reports_path / "research" / "global_experiment_registry.json"
).register(candidates, run_id=run.id, params=params)
candidate_director_summary: dict[str, Any] = {}
if settings.research_director_enabled:
    try:
        logger.info("Research Director: enriqueciendo candidatos antes del registry")
        candidate_director_summary = CandidateResearchDirector(settings=settings).run(
            run_id=run.id,
            candidates=candidates,
            samples=samples,
            params=params,
        )
    except Exception as exc: # noqa: BLE001
        msg = f"CandidateResearchDirector failed: {exc}"
        logger.exception(msg)
        warnings.append(msg)
stored = NovelPatternRegistry().store_candidates(
    db,
    candidates,
    run_id=run.id,
    store_rejected=params["store_rejected"],
)
director_result: dict[str, Any] | None = None
if settings.research_director_enabled:
    try:
        director_result = PersistentResearchDirector(settings).run(
            db,
            run_id=run.id,
            limit=settings.research_director_pattern_limit,
        )
    except Exception as exc: # noqa: BLE001
        msg = f"ResearchDirector failed: {exc}"
        logger.exception(msg)
        warnings.append(msg)
duration = round(time.perf_counter() - started, 3)
summary = self._summary(candidates, samples, warnings)
summary["data_manifest"] = {
    "path": data_manifest.get("path"),
    "manifest_hash": data_manifest.get("manifest_hash"),
    "entry_count": len((data_manifest.get("entries") or {})),
    "artifact_format": data_manifest.get("artifact_format"),
}
summary["universe_snapshot"] = universe_snapshot
summary["global_experiment_registry"] = global_registry
summary["research_director"] = {
    "candidate_completion": candidate_director_summary,
}
if director_result is not None:
    summary["research_director"]["db_director"] = {
        "patterns_reviewed": director_result.get("patterns_reviewed", 0),
        "hypotheses_created": director_result.get("hypotheses_created", 0),
        "director_state": director_result.get("director_state", {}),
        "artifacts": director_result.get("artifacts", {}),
    }
report_path = self._write_report(run.id, params, summary, candidates)
run.status = "completed"
run.finished_at = datetime.now(timezone.utc)
run.symbols_scanned = len(symbols)
run.windows_sampled = len(samples)
run.clusters_evaluated = len(candidates)
run.accepted_patterns = len(accepted)
run.rejected_patterns = len(rejected)
run.duration_seconds = duration
run.summary_json = summary
run.report_path = str(report_path) if report_path else None
db.add(
    AuditLog(
        actor="PatternDiscoveryLabAgent",
        action="discovery_run_completed",
        entity_type="discovery_run",
        entity_id=str(run.id),
        details_json={
            "symbols": len(symbols),
            "windows": len(samples),
            "clusters": len(candidates),
            "accepted": len(accepted),
            "rejected": len(rejected),
            "stored": len(stored),
            "research_director": summary.get("research_director", {}),
            "event_ledgers": ledger_artifacts,
            "data_manifest": summary.get("data_manifest", {}),
            "universe_snapshot": universe_snapshot,
            "report_path": str(report_path) if report_path else None,
        },
    ),
)
db.commit()
return DiscoveryRunResponse(
    run_id=run.id,

```

```

        status="completed",
        symbols_scanned=len(symbols),
        windows_sampled=len(samples),
        clusters_evaluated=len(candidates),
        accepted_patterns=len(accepted),
        rejected_patterns=len(rejected),
        stored_patterns=len(stored),
        duration_seconds=duration,
        report_path=str(report_path) if report_path else None,
        top_patterns=summary["top_patterns"],
        warnings=warnings[:50],
    )
except Exception as exc: # noqa: BLE001
    duration = round(time.perf_counter() - started, 3)
    run.status = "failed"
    run.finished_at = datetime.now(timezone.utc)
    run.duration_seconds = duration
    run.summary_json = {"error": str(exc), "warnings": warnings}
    db.add(run)
    db.add(
        AuditLog(
            actor="PatternDiscoveryLabAgent",
            action="discovery_run_failed",
            entity_type="discovery_run",
            entity_id=str(run.id),
            details_json={"error": str(exc), "warnings": warnings[:50]},
        )
    )
    db.commit()
    raise

def _apply_run_level_inference(self, candidates: list[Any]) -> None:
    """Run-level selective inference: BH-FDR + family-deflated Sharpe.

    Per-candidate p-values already exist (stratified permutation vs null
    baselines), but each one ignores how many sibling hypotheses this run
    tested. Benjamini-Hochberg is applied over every cluster evaluated in
    the run ? accepted and rejected alike ? and the DSR is deflated with the
    N_trials accumulated in the global experiment registry, so the bar for
    lab_candidate rises automatically as more mining happens.
    """
    settings = self.settings
    assert settings is not None
    if not candidates:
        return
    pvals: list[float] = []
    indexed: list[Any] = []
    for candidate in candidates:
        p = candidate.metrics.get("null_p_value")
        try:
            p = float(p)
        except (TypeError, ValueError):
            p = float("nan")
        if np.isfinite(p):
            pvals.append(min(max(p, 0.0), 1.0))
            indexed.append(candidate)
    if pvals:
        passed_mask, cutoff = benjamini_hochberg(pvals, q=settings.discovery_fdr_q)
        for candidate, passed in zip(indexed, passed_mask, strict=True):
            candidate.metrics["fdr_q"] = settings.discovery_fdr_q
            candidate.metrics["fdr_passed"] = bool(passed)
            candidate.metrics["fdr_cutoff_p"] = float(cutoff) if np.isfinite(cutoff) else None
            candidate.metrics["fdr_test_count"] = len(pvals)

    registry_payload = GlobalExperimentRegistry(
        settings.reports_path / "research" / "global_experiment_registry.json"
    ).load()
    prior_trials = int(registry_payload.get("global_trial_count", 0) or 0)
    trials_by_window: dict[int, int] = {}
    for candidate in candidates:
        window = int(getattr(candidate, "window_size", 0))
        count = int(candidate.metrics.get("real_variant_count", 1) or 1)
        trials_by_window[window] = max(trials_by_window.get(window, 0), count)
    run_trials = sum(trials_by_window.values())
    n_trials_total = max(1, prior_trials + run_trials)

    sharpes = np.asarray(
        [float(c.metrics.get("trade_sharpe", 0.0) or 0.0) for c in candidates], dtype=float
    )
    sharpes = sharpes[np.isfinite(sharpes)]
    sr_std = float(np.std(sharpes, ddof=1)) if sharpes.size >= 3 else 0.0

    for candidate in candidates:
        quant = candidate.metrics.get("quant_validation", {})
        if not isinstance(quant, dict):
            quant = {}
        n_eff = float(quant.get("n_eff", 0.0) or 0.0)
        sr = float(quant.get("sharpe_per_trade", candidate.metrics.get("trade_sharpe", 0.0)) or 0.0)
        skew = float(quant.get("skew", 0.0) or 0.0)
        kurt = float(quant.get("kurtosis", 3.0) or 3.0)
        if n_eff > 1 and sr_std > 0 and n_trials_total > 1:
            dsr, sr_star = deflated_sharpe_ratio(sr, n_eff, skew, kurt, n_trials_total, sr_std)
        elif n_eff > 1:
            dsr = probabilistic_sharpe_ratio(sr, 0.0, n_eff, skew, kurt)
            sr_star = 0.0
        else:

```

```

        dsr, sr_star = float("nan"), 0.0
        candidate.metrics["dsr_family"] = round(float(dsr), 5) if np.isfinite(dsr) else None
        candidate.metrics["dsr_family_sr_star"] = round(float(sr_star), 5)
        candidate.metrics["dsr_family_n_trials"] = n_trials_total
        candidate.metrics["dsr_family_sr_std"] = round(sr_std, 5)
        candidate.metrics["dsr_family_prior_registry_trials"] = prior_trials
        candidate.metrics["dsr_family_run_trials"] = run_trials

def _resolve_params(self, request: DiscoveryRunRequest) -> dict[str, Any]:
    s = self.settings
    assert s is not None
    max_total_windows = min(request.max_total_windows or s.discovery_max_total_windows, 80_000)
    return {
        "limit": request.limit or s.discovery_limit_default,
        "period": request.period or s.discovery_period,
        "interval": request.interval or s.discovery_interval,
        "window_sizes": request.window_sizes or s.discovery_window_size_list,
        "forwardBars": request.forwardBars or s.discovery_forward_bar_list,
        "stride": max(1, request.stride or s.discovery_stride),
        "max_total_windows": max(100, max_total_windows),
        "max_windows_per_symbol": max(50, request.max_windows_per_symbol or s.discovery_max_windows_per_symbol),
        "min_cluster_size": max(20, request.min_cluster_size or s.discovery_min_cluster_size),
        "max_clusters_per_window": max(2, min(request.max_clusters_per_window or s.discovery_max_clusters_per_window, 40)),
        "store_rejected": s.discovery_store_rejected if request.store_rejected is None else request.store_rejected,
        "rr_levels": s.discovery_rr_level_list,
        "min_reward_risk": s.discovery_min_reward_risk,
        "candidate_reward_risk": s.discovery_candidate_reward_risk,
        "premium_reward_risk": s.discovery_premium_reward_risk,
        "max_drawdown_r": s.discovery_max_drawdown_r,
        "walk_forward_folds": s.discovery_walk_forward_folds,
        "walk_forward_embargo_samples": s.discovery_walk_forward_embargo_samples,
        "cost_stress_multipliers": s.discovery_cost_stress_multiplier_list,
        "required_cost_stress_multiplier": s.discovery_required_cost_stress_multiplier,
    }

@staticmethod
def _resolve_symbols(request: DiscoveryRunRequest, params: dict[str, Any]) -> list[str]:
    if request.symbols:
        return [s.upper().strip() for s in request.symbols if s.strip()]
    return pick_symbols(limit=params["limit"])

def _build_universe_snapshot(self, warnings: list[str]) -> dict[str, Any]:
    settings = self.settings
    assert settings is not None
    if not settings.universe_snapshot_monthly:
        return {
            "enabled": False,
            "point_in_time": settings.universe_point_in_time_available,
            "survivorship_biased": not settings.universe_point_in_time_available,
        }
    try:
        return UniverseSnapshotService(settings).build_monthly_snapshot()
    except Exception as exc: # noqa: BLE001
        msg = f"UniverseSnapshotService failed: {exc}"
        logger.warning(msg)
        warnings.append(msg)
        return {
            "enabled": True,
            "error": str(exc),
            "point_in_time": settings.universe_point_in_time_available,
            "survivorship_biased": not settings.universe_point_in_time_available,
        }

def _data_manifest(self, run_id: int, symbols: list[str], warnings: list[str]) -> dict[str, Any]:
    provider_manifest = getattr(self.provider, "data_manifest", None)
    if not callable(provider_manifest):
        return self._write_data_manifest(
            run_id,
            {
                "schema_version": 1,
                "generated_at": datetime.now(timezone.utc).isoformat(),
                "entries": {},
                "manifest_hash": "",
                "artifact_format": "unavailable",
                "reason": "provider_does_not_expose_data_manifest",
            },
        )
    try:
        payload = provider_manifest(symbols)
    except Exception as exc: # noqa: BLE001
        warnings.append(f"data_manifest failed: {exc}")
        payload = {
            "schema_version": 1,
            "generated_at": datetime.now(timezone.utc).isoformat(),
            "entries": {},
            "manifest_hash": "",
            "artifact_format": "unavailable",
            "reason": str(exc),
        }
    return self._write_data_manifest(run_id, payload)

def _write_data_manifest(self, run_id: int, payload: dict[str, Any]) -> dict[str, Any]:
    settings = self.settings
    assert settings is not None
    manifest_dir = settings.reports_path / "research" / "data_manifests"
    manifest_dir.mkdir(parents=True, exist_ok=True)

```

```

path = manifest_dir / f"discovery_run_{run_id}_data_manifest.json"
path.write_text(json.dumps(payload, indent=2, sort_keys=True, default=str), encoding="utf-8")
payload = dict(payload)
payload["path"] = str(path)
return payload

def _benchmark_frames(self, params: dict[str, Any], warnings: list[str]) -> dict[str, Any]:
    frames: dict[str, Any] = {}
    assert self.provider is not None
    for symbol in ("SPY", "QQQ"):
        try:
            frames[symbol] = self.provider.fetch_ohlcv(
                symbol,
                period=params["period"],
                interval=params["interval"],
            )
        except Exception as exc: # noqa: BLE001
            warnings.append(f"{symbol} benchmark unavailable: {exc}")
    return frames

def _benchmark_regime_table(self, params: dict[str, Any], warnings: list[str]) -> Any:
    assert self.provider is not None
    try:
        return MarketRegimeService(provider=self.provider, settings=self.settings).history_table(
            period=str(params.get("period") or "")
        )
    except Exception as exc: # noqa: BLE001
        warnings.append(f"benchmark regime table unavailable: {exc}")
    return None

def _summary(self, candidates: list[Any], samples: list[Any], warnings: list[str]) -> dict[str, Any]:
    accepted = [c for c in candidates if c.validation_passed]
    confirmation = [c for c in candidates if c.metrics.get("confirmation_recommended")]
    status_counts: dict[str, int] = {}
    for candidate in candidates:
        status = str(candidate.metrics.get("promotion_status", "lab" if candidate.validation_passed else "rejected"))
        status_counts[status] = status_counts.get(status, 0) + 1
    top = sorted(candidates, key=lambda c: c.score, reverse=True)[: self.settings.discovery_report_top_n] # type:
ignore[union-attr]
    confirmation_top = sorted(
        confirmation,
        key=lambda c: float(c.metrics.get("confirmation_priority_score", 0.0)),
        reverse=True,
    )[: self.settings.discovery_report_top_n] # type: ignore[union-attr]
    return {
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "windows_sampled": len(samples),
        "clusters_evaluated": len(candidates),
        "accepted_patterns": len(accepted),
        "rejected_patterns": len(candidates) - len(accepted),
        "confirmation_candidates": len(confirmation),
        "status_counts": status_counts,
        "rr_levels": self.settings.discovery_rr_level_list, # type: ignore[union-attr]
        "safety": {
            "live_trading_enabled": self.settings.live_trading_enabled, # type: ignore[union-attr]
            "trading_mode": self.settings.trading_mode, # type: ignore[union-attr]
            "ibkr_readonly": self.settings.ibkr_readonly, # type: ignore[union-attr]
        },
        "token_policy": {
            "raw_windows_sent_to_llm": 0,
            "llm_review_input": "compact_digest_only",
            "reason": "Discovery is local/vectorized; API supervisor only needs top cluster metrics and examples.",
        },
        "top_patterns": [self._candidate_digest(c) for c in top],
        "confirmation_queue": [self._candidate_digest(c) for c in confirmation_top],
        "warnings": warnings[:50],
    }

@staticmethod
def _candidate_digest(candidate: Any) -> dict[str, Any]:
    metrics = candidate.metrics
    return {
        "name": candidate.name,
        "pattern_key": candidate.pattern_key,
        "status": metrics.get("promotion_status", "lab" if candidate.validation_passed else "rejected"),
        "side": candidate.side,
        "window_size": candidate.window_size,
        "sample_count": candidate.sample_count,
        "symbol_count": candidate.symbol_count,
        "year_count": candidate.year_count,
        "score": candidate.score,
        "lab_priority_score": metrics.get("lab_priority_score", candidate.score),
        "promotion_score": metrics.get("promotion_score"),
        "score_input_scope": metrics.get("score_input_scope", {}),
        "selection_split": metrics.get("selection_split", {}),
        "fit_scope": metrics.get("fit_scope", {}),
        "train_metrics": metrics.get("train_metrics", {}),
        "out_of_sample_metrics": metrics.get("out_of_sample_metrics", {}),
        "walk_forward_metrics": metrics.get("walk_forward_metrics", {}),
        "descriptive_metric_policy": metrics.get("descriptive_metric_policy", {}),
        "descriptive_all_expectancy_r": metrics.get("descriptive_all_expectancy_r"),
        "descriptive_all_profit_factor": metrics.get("descriptive_all_profit_factor"),
        "descriptive_all_win_rate": metrics.get("descriptive_all_win_rate"),
        "descriptive_all_reward_risk_estimate": metrics.get("descriptive_all_reward_risk_estimate"),
        "expectancy_r": metrics.get("expectancy_r"),
        "profit_factor": metrics.get("profit_factor"),
    }

```

```

"win_rate": metrics.get("win_rate"),
"best_rr": metrics.get("best_rr"),
"best_expectancy_r": metrics.get("best_expectancy_r"),
"best_profit_factor": metrics.get("best_profit_factor"),
"best_win_rate": metrics.get("best_win_rate"),
"best_max_drawdown_r": metrics.get("best_max_drawdown_r"),
"avg_execution_cost_r": metrics.get("avg_execution_cost_r"),
"avgBars_to_target": metrics.get("avgBars_to_target"),
"avgBars_to_stop": metrics.get("avgBars_to_stop"),
"triple_barrier_labels": metrics.get("triple_barrier_labels", {}),
"mfe_before_mae_rate": metrics.get("mfe_before_mae_rate"),
"avg_gap_adverse_r": metrics.get("avg_gap_adverse_r"),
"strong_close_without_target_rate": metrics.get("strong_close_without_target_rate"),
"speed_label_counts": metrics.get("speed_label_counts", {}),
"fast_target_rate": metrics.get("fast_target_rate"),
"slow_target_rate": metrics.get("slow_target_rate"),
"timeout_rate": metrics.get("timeout_rate"),
"avg_fill_probability": metrics.get("avg_fill_probability"),
"p25_max_size_usd": metrics.get("p25_max_size_usd"),
"median_max_size_usd": metrics.get("median_max_size_usd"),
"avg_spread_proxy_pct": metrics.get("avg_spread_proxy_pct"),
"avg_slippage_proxy_pct": metrics.get("avg_slippage_proxy_pct"),
"avg_entry_gap_penalty_pct": metrics.get("avg_entry_gap_penalty_pct"),
"avg_short_borrow_proxy_pct": metrics.get("avg_short_borrow_proxy_pct"),
"preferred_rr_passed": metrics.get("preferred_rr_passed"),
"premium_rr_passed": metrics.get("premium_rr_passed"),
"promotion_reason": metrics.get("promotion_reason"),
"rr_metrics": metrics.get("rr_metrics"),
"hit_4r_rate": metrics.get("hit_4r_rate"),
"reward_risk_estimate": metrics.get("reward_risk_estimate"),
"stability_score": metrics.get("stability_score"),
>null_expectancy_r": metrics.get("null_expectancy_r"),
"expectancy_lift_r": metrics.get("expectancy_lift_r"),
>null_p_value": metrics.get("null_p_value"),
"adjusted_p_value": metrics.get("adjusted_p_value"),
"wrc_p_value": metrics.get("wrc_p_value"),
"spa_p_value": metrics.get("spa_p_value"),
"reality_check_method": metrics.get("reality_check_method"),
"reality_check_formal_test": metrics.get("reality_check_formal_test", False),
"bootstrap_reality_proxy": metrics.get("bootstrap_reality_proxy", {}),
"probabilistic_sharpe": metrics.get("probabilistic_sharpe"),
"deflated_sharpe": metrics.get("deflated_sharpe"),
"deflated_sharpe_probability": metrics.get("deflated_sharpe_probability"),
"edge_decay_parameter_score": metrics.get("edge_decay_parameter_score"),
"edge_decay_passed": metrics.get("edge_decay_passed"),
"purged_cv_fold_count": metrics.get("purged_cv_fold_count", 0),
"purged_cv_positive_rate": metrics.get("purged_cv_positive_rate", 0.0),
"purged_cv_avg_expectancy_r": metrics.get("purged_cv_avg_expectancy_r", 0.0),
"purged_cv_min_expectancy_r": metrics.get("purged_cv_min_expectancy_r", 0.0),
"multiple_testing_trials": metrics.get("multiple_testing_trials"),
"quant_validation": metrics.get("quant_validation", {}),
"effective_sample_count": metrics.get("effective_sample_count"),
"unique_event_count": metrics.get("unique_event_count"),
"fdr_q": metrics.get("fdr_q"),
"fdr_passed": metrics.get("fdr_passed"),
"fdr_cutoff_p": metrics.get("fdr_cutoff_p"),
"fdr_test_count": metrics.get("fdr_test_count"),
"dsr_family": metrics.get("dsr_family"),
"dsr_family_n_trials": metrics.get("dsr_family_n_trials"),
"dsr_family_sr_star": metrics.get("dsr_family_sr_star"),
"match_tau_similarity": metrics.get("match_tau_similarity"),
"match_tau_percentile": metrics.get("match_tau_percentile"),
"statistical_edge_passed": metrics.get("statistical_edge_passed"),
>null_method": metrics.get("null_method"),
>null_strata_count": metrics.get("null_strata_count"),
"expectancy_ci_low": metrics.get("expectancy_ci_low"),
"expectancy_ci_high": metrics.get("expectancy_ci_high"),
"profit_factor_ci_low": metrics.get("profit_factor_ci_low"),
"overfit_score": metrics.get("overfit_score"),
"cost_stress": metrics.get("cost_stress", {}),
"cost_stress_passed": metrics.get("cost_stress_passed"),
"confirmation_recommended": metrics.get("confirmation_recommended", False),
"confirmation_status": metrics.get("confirmation_status", ""),
"confirmation_priority_score": metrics.get("confirmation_priority_score", 0.0),
"confirmation_reason": metrics.get("confirmation_reason", ""),
"confirmation_next_action": metrics.get("confirmation_next_action", ""),
"registry_deduped": metrics.get("registry_deduped", False),
"registry_similarity": metrics.get("registry_similarity", 0.0),
"registry_canonical_pattern_key": metrics.get("registry_canonical_pattern_key"),
"pattern_family_key": metrics.get("pattern_family_key"),
"canonical_pattern_key": metrics.get("canonical_pattern_key"),
"variant_key": metrics.get("variant_key"),
"variant_count": metrics.get("variant_count"),
"drift_status": metrics.get("drift_status"),
"drift_score": metrics.get("drift_score"),
"novelty_score": metrics.get("novelty_score"),
"diversity_bucket": metrics.get("diversity_bucket"),
"diversity_quota_rank": metrics.get("diversity_quota_rank"),
"expected_information_gain": metrics.get("expected_information_gain"),
"registry_novelty_score": metrics.get("registry_novelty_score"),
"registry_expected_information_gain": metrics.get("registry_expected_information_gain"),
"human_rule": metrics.get("human_rule", {}),
"regime_profile": metrics.get("regime_profile", {}),
"operational_trigger": metrics.get("operational_trigger", {}),
"event_ledger_count": metrics.get("event_ledger_count", 0),
"event_ledger_path": metrics.get("event_ledger_path"),

```

```

"event_ledger_sha256": metrics.get("event_ledger_sha256"),
"event_ledger_compressed_bytes": metrics.get("event_ledger_compressed_bytes"),
"event_ledger_uncompressed_bytes": metrics.get("event_ledger_uncompressed_bytes"),
"foundation_teacher": metrics.get("foundation_teacher", {}),
"market_replay": metrics.get("market_replay", {}),
"causal_invariance": metrics.get("causal_invariance", {}),
"adversarial_challenge": metrics.get("adversarial_challenge", {}),
"research_hypothesis": metrics.get("research_hypothesis", {}),
"research_hypothesis_package": metrics.get("research_hypothesis_package", {}),
"nested_discovery_replay": metrics.get("nested_discovery_replay", {}),
"global_experiment_registry": metrics.get("global_experiment_registry", {}),
"research_memory": metrics.get("research_memory", {}),
"active_learning": metrics.get("active_learning", {}),
"pattern_lifecycle": metrics.get("pattern_lifecycle", {}),
"research_director": metrics.get("research_director", {}),
"research_paper_report_path": metrics.get("research_paper_report_path"),
"walk_forward_fold_count": metrics.get("walk_forward_fold_count", 0),
"walk_forward_positive_fold_rate": metrics.get("walk_forward_positive_fold_rate", 0.0),
"walk_forward_avg_expectancy_r": metrics.get("walk_forward_avg_expectancy_r", 0.0),
"walk_forward_min_expectancy_r": metrics.get("walk_forward_min_expectancy_r", 0.0),
"walk_forward_folds": metrics.get("walk_forward_folds", []),
"out_of_sample_expectancy_r": metrics.get("out_of_sample_expectancy_r"),
"out_of_sample_profit_factor": metrics.get("out_of_sample_profit_factor"),
"out_of_sample_win_rate": metrics.get("out_of_sample_win_rate"),
"out_of_sample_max_max_drawdown_r": metrics.get("out_of_sample_max_drawdown_r"),
"validation_reasons": candidate.validation_reasons,
"representative_examples": [
    {
        "symbol": e.get("symbol"),
        "window_end": e.get("window_end"),
        "kind": e.get("kind"),
        "outcome_r": e.get("outcome_r"),
        "mfe_r": e.get("mfe_r"),
        "mae_r": e.get("mae_r"),
        "triple_barrier_label": e.get("triple_barrier_label"),
        "speed_label": e.get("speed_label"),
        "time_to_target": e.get("time_to_target"),
        "time_to_stop": e.get("time_to_stop"),
    }
    for e in candidate.examples[:5]
],
}

def _write_event_ledgers(self, run_id: int, candidates: list[Any]) -> int:
    settings = self.settings
    assert settings is not None
    ledger_dir = settings.reports_path / "research" / "event_ledgers" / f"run_{run_id}"
    written = 0
    for candidate in candidates:
        ledger = candidate.metrics.get("event_ledger")
        if not isinstance(ledger, list):
            continue
        ledger_dir.mkdir(parents=True, exist_ok=True)
        payload = {
            "run_id": run_id,
            "pattern_key": candidate.pattern_key,
            "name": candidate.name,
            "side": candidate.side,
            "timeframe": candidate.timeframe,
            "window_size": candidate.window_size,
            "cluster_id": candidate.cluster_id,
            "event_count": len(ledger),
            "events": ledger,
        }
        raw = json.dumps(payload, ensure_ascii=False, sort_keys=True, default=str).encode("utf-8")
        digest = hashlib.sha256(raw).hexdigest()
        compressed = gzip.compress(raw, compresslevel=9, mtime=0)
        path = ledger_dir / f"{self._safe_artifact_stem(candidate.pattern_key)}.json.gz"
        path.write_bytes(compressed)
        candidate.metrics["event_ledger_path"] = str(path)
        candidate.metrics["event_ledger_sha256"] = digest
        candidate.metrics["event_ledger_count"] = len(ledger)
        candidate.metrics["event_ledger_compressed_bytes"] = len(compressed)
        candidate.metrics["event_ledger_uncompressed_bytes"] = len(raw)
        candidate.metrics["event_ledger_persisted"] = True
        candidate.metrics["event_ledger_preview"] = ledger[:5]
        candidate.metrics.pop("event_ledger", None)
        written += 1
    return written

@staticmethod
def _safe_artifact_stem(value: object) -> str:
    stem = "".join(ch if ch.isalnum() or ch in {"-", "_", "."} else "_" for ch in str(value))
    return stem.strip("_")[:120] or "pattern"

def _write_report(
    self,
    run_id: int,
    params: dict[str, Any],
    summary: dict[str, Any],
    candidates: list[Any],
) -> Path | None:
    settings = self.settings
    assert settings is not None
    reports_dir = settings.reports_path / "research"
    reports_dir.mkdir(parents=True, exist_ok=True)

```



```

ts = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
json_path = reports_dir / f"discovery_run_{run_id}_{ts}.json"
md_path = reports_dir / f"discovery_run_{run_id}_{ts}.md"
payload = {
    "run_id": run_id,
    "params": params,
    "summary": summary,
    "patterns": [self._candidate_digest(c) for c in candidates],
    "top_by_expectancy": sorted(
        [self._candidate_digest(c) for c in candidates],
        key=lambda p: float(p.get("best_expectancy_r") or 0.0),
        reverse=True,
    )[:10],
    "top_by_stability": sorted(
        [self._candidate_digest(c) for c in candidates],
        key=lambda p: float(p.get("stability_score") or 0.0),
        reverse=True,
    )[:10],
}
payload["determinism"] = {
    "algo": CONTENT_HASH_ALGO,
    "content_hash": content_hash(payload),
    "excluded_keys": sorted(DEFAULT_VOLATILE_KEYS),
    "generated_at": datetime.now(timezone.utc).isoformat(),
}
json_path.write_text(
    json.dumps(payload, indent=2, ensure_ascii=False, sort_keys=True, default=str), encoding="utf-8"
)
md_path.write_text(self._markdown_report(run_id, params, summary), encoding="utf-8")
return json_path

@staticmethod
def _markdown_report(run_id: int, params: dict[str, Any], summary: dict[str, Any]) -> str:
    lines = [
        f"# Tradeo Research Lab · Discovery Run {run_id}",
        "",
        "## Resumen",
        f"- Ventanas analizadas: {summary['windows_sampled']}",
        f"- Clusters evaluados: {summary['clusters_evaluated']}",
        f"- Patrones LAB aceptados: {summary['accepted_patterns']}",
        f"- Patrones rechazados: {summary['rejected_patterns']}",
        f"- Candidatos para confirmación: {summary.get('confirmation_candidates', 0)}",
        f"- Estados: {summary.get('status_counts', {})}",
        f"- R:R evaluados: {summary.get('rr_levels', [])}",
        f"- Seguridad: {summary.get('safety', {})}",
        "",
        "## Política de tokens",
        "El laboratorio no envía ventanas OHLCV crudas a ningún LLM. Solo exporta este digest compacto para revisión.",
        "",
        "## Research Director",
        "```json",
        json.dumps(summary.get("research_director", {}), indent=2, ensure_ascii=False, default=str),
        "```",
        "",
        "## Parámetros",
        "```json",
        json.dumps(params, indent=2, ensure_ascii=False),
        "```",
        "",
        "## Top patrones",
    ]
    for pattern in summary.get("top_patterns", []):
        human_rule = pattern.get("human_rule")
        human_rule_text = (
            human_rule.get("rule")
            if isinstance(human_rule, dict)
            else human_rule
        )
        lines.extend(
            [
                f"### {pattern['name']}",
                f"- Estado: {pattern['status']}",
                f"- Lado: {pattern['side']} · ventana: {pattern['window_size']}",
                (
                    "- Muestras/símbolos/años: "
                    f"{pattern['sample_count']} / {pattern['symbol_count']} / {pattern['year_count']}"
                ),
                f"- Score: {pattern['score']}",
                (
                    f"- Best R:R: {pattern.get('best_rr')} · "
                    f"Expectancy: {pattern.get('best_expectancy_r')}R · "
                    f"PF: {pattern.get('best_profit_factor')} · "
                    f"Win rate: {pattern.get('best_win_rate')}"
                ),
                (
                    f"- Labels: {pattern.get('triple_barrier_labels')} · "
                    f"target bars {pattern.get('avg_bars_to_target')} · "
                    f"stop bars {pattern.get('avg_bars_to_stop')} · "
                    f"MFE antes MAE {pattern.get('mfe_before_mae_rate')}"
                ),
                (
                    f"- Coste/fill: coste {pattern.get('avg_execution_cost_r')}R · "
                    f"fill {pattern.get('avg_fill_probability')} · "
                    f"p25 max size ${pattern.get('p25_max_size_usd')} · "
                    f"spread {pattern.get('avg_spread_proxy_pct')} · "
                    f"slippage {pattern.get('avg_slippage_proxy_pct')}"
                )
            ]
        )

```

```

    ),
    f"- Trigger operativo: {pattern.get('operational_trigger')}",
    (
        "- Walk-forward: "
        f"{pattern.get('walk_forward_positive_fold_rate')} positive fold rate . "
        f"avg {pattern.get('walk_forward_avg_expectancy_r')}R . "
        f"min {pattern.get('walk_forward_min_expectancy_r')}R"
    ),
    (
        f"- Purged CV: {pattern.get('purged_cv_positive_rate')} positive . "
        f"avg {pattern.get('purged_cv_avg_expectancy_r')}R . "
        f"min {pattern.get('purged_cv_min_expectancy_r')}R"
    ),
    (
        f"- Null/CI: {pattern.get('null_method')} . "
        f"p_adj {pattern.get('adjusted_p_value')} . "
        "bootstrap_reality_proxy "
        f"wrc_like {pattern.get('wrc_p_value')} . spa_like {pattern.get('spa_p_value')} . "
        f"CI {pattern.get('expectancy_ci_low')}..{pattern.get('expectancy_ci_high')}R . "
        f"overfit {pattern.get('overfit_score')}"
    ),
    (
        f"- Sharpe/decay: PSR {pattern.get('probabilistic_sharpe')} . "
        f"DSR {pattern.get('deflated_sharpe_probability')} . "
        f"edge decay {pattern.get('edge_decay_parameter_score')}"
    ),
    (
        f"- Novelty: score {pattern.get('novelty_score')} . "
        f"EIG {pattern.get('expected_information_gain')} . "
        f"bucket {pattern.get('diversity_bucket')} . "
        f"rank {pattern.get('diversity_quota_rank')}"
    ),
    f"- Regla humana: {human_rule_text}",
    f"- Hipótesis: {(pattern.get('research_hypothesis') or {}).get('thesis') if
isinstance(pattern.get('research_hypothesis'), dict) else None}",
    f"- Edge claim: {(pattern.get('research_hypothesis') or {}).get('edge_claim') if
isinstance(pattern.get('research_hypothesis'), dict) else 'NO DEMOSTRADO'}",
    f"- Nested replay: {pattern.get('nested_discovery_replay')}",
    f"- Registry global: {pattern.get('global_experiment_registry')}",
    f"- Market replay: {pattern.get('market_replay')}",
    f"- Adversarial challenge: {pattern.get('adversarial_challenge')}",
    f"- Invariancia causal: {pattern.get('causal_invariance')}",
    f"- Lifecycle: {pattern.get('pattern_lifecycle')}",
    f"- Active learning: {pattern.get('active_learning')}",
    f"- Paper report: {pattern.get('research_paper_report_path')}",
    f"- Cost stress passed: {pattern.get('cost_stress_passed')} . {pattern.get('cost_stress')}",
    (
        f"- Ledger: {pattern.get('event_ledger_count')} eventos . "
        f"{pattern.get('event_ledger_path')} . sha256 {pattern.get('event_ledger_sha256')}"
    ),
    (
        f"- R:R estimado: {pattern['reward_risk_estimate']} . "
        f"Hit 4R: {pattern['hit_4r_rate']} . "
        f"DD: {pattern.get('best_max_drawdown_r')}R"
    ),
    (
        f"- OOS expectancy: {pattern['out_of_sample_expectancy_r']}R . "
        f"estabilidad: {pattern['stability_score']}"
    ),
    f"- Razones/avisos: {' , '.join(pattern.get('validation_reasons') or ['sin incidencias'])}",
    "",
    ]
    )
    if summary.get("confirmation_queue"):
        lines.extend(["", "## Cola de confirmación"])
        for pattern in summary.get("confirmation_queue", []):
            lines.extend(
                [
                    f"### {pattern['name']}",
                    f"- Prioridad: {pattern.get('confirmation_priority_score')}",
                    f"- Motivo: {pattern.get('confirmation_reason')}",
                    f"- Siguiente acción: {pattern.get('confirmation_next_action')}",
                    f"- Best R:R: {pattern.get('best_rr')} . Expectancy: {pattern.get('best_expectancy_r')}R . PF:
{pattern.get('best_profit_factor')}",
                    ""
                ]
            )
    return "\n".join(lines)

```

Full Source: `backend/tradeo/routers/research.py`

`backend/tradeo/routers/research.py` ? 225 lines, 8304 bytes, sha256 prefix `f36e87b7d94166bf`

Public surface summary before full source:

```

? function `run_discovery` line 35`
? function `list_discovered_patterns` line 44`
? function `get_discovered_pattern` line 54`
? function `get_discovered_pattern_examples` line 71`
? function `get_discovered_pattern_metrics` line 88`
? function `get_discovered_pattern_rr_metrics` line 106`

```

```

? function `list_confirmation_queue` line 132`
? function `match_current_patterns` line 147`
? function `list_current_matches` line 165`
? function `list_discovery_runs` line 187`
? function `run_research_director` line 196`
? function `latest_research_director` line 207`
? function `refresh_director_review_candidates` line 221`

? route run_discovery: @router.post('/run-discovery', response_model=DiscoveryRunResponse)
? route list_discovered_patterns: @router.get('/discovered-patterns', response_model=list[DiscoveredPatternOut])
? route get_discovered_pattern: @router.get('/discovered-patterns/{pattern_id}', response_model=DiscoveredPatternDetailOut)
? route get_discovered_pattern_examples: @router.get('/discovered-patterns/{pattern_id}/examples',
response_model=list[DiscoveredPatternExampleOut])
? route get_discovered_pattern_metrics: @router.get('/discovered-patterns/{pattern_id}/metrics',
response_model=list[DiscoveredPatternMetricOut])
? route get_discovered_pattern_rr_metrics: @router.get('/discovered-patterns/{pattern_id}/rr-metrics')
? route list_confirmation_queue: @router.get('/confirmation-queue', response_model=list[DiscoveredPatternOut])
? route match_current_patterns: @router.post('/match-current', response_model=NovelPatternMatchResponse)
? route list_current_matches: @router.get('/current-matches', response_model=list[NovelPatternMatchOut])
? route list_discovery_runs: @router.get('/runs', response_model=list[DiscoveryRunOut])
? route run_research_director: @router.post('/director/run', response_model=ResearchDirectorResponse)
? route latest_research_director: @router.get('/director/latest', response_model=ResearchDirectorResponse)

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import json

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session, selectinload

from tradeo.agents.pattern_discovery_lab_agent import PatternDiscoveryLabAgent
from tradeo.core.config import get_settings
from tradeo.core.security import require_admin
from tradeo.db.models import DiscoveredPattern, DiscoveredPatternExample, DiscoveredPatternMatch, DiscoveredPatternMetric, DiscoveryRun
from tradeo.db.session import get_db
from tradeo.research.autonomous_research_director import ResearchDirector
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher
from tradeo.research.novel_pattern_registry import NovelPatternRegistry
from tradeo.services.director_review_gate import DirectorReviewGate
from tradeo.schemas import (
    DiscoveredPatternDetailOut,
    DiscoveredPatternExampleOut,
    DiscoveredPatternMetricOut,
    DiscoveredPatternOut,
    DiscoveryRunOut,
    DiscoveryRunRequest,
    DiscoveryRunResponse,
    NovelPatternMatchOut,
    NovelPatternMatchRequest,
    NovelPatternMatchResponse,
    ResearchDirectorResponse,
)

router = APIRouter(prefix="/research", tags=["research"])

@router.post("/run-discovery", response_model=DiscoveryRunResponse)
def run_discovery(
    request: DiscoveryRunRequest,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> DiscoveryRunResponse:
    return PatternDiscoveryLabAgent().run(request, db)

@router.get("/discovered-patterns", response_model=list[DiscoveredPatternOut])
def list_discovered_patterns(
    status: str | None = None,
    limit: int = Query(default=100, ge=1, le=500),
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[DiscoveredPattern]:
    return NovelPatternRegistry.list_patterns(db, limit=limit, status=status)

@router.get("/discovered-patterns/{pattern_id}", response_model=DiscoveredPatternDetailOut)
def get_discovered_pattern(
    pattern_id: int,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> DiscoveredPattern:
    pattern = (
        db.query(DiscoveredPattern)
        .options(selectinload(DiscoveredPattern.examples), selectinload(DiscoveredPattern.metric_snapshots))
        .filter(DiscoveredPattern.id == pattern_id)
        .one_or_none()
    )

```

```

    )
    if not pattern:
        raise HTTPException(404, "discovered pattern not found")
    return pattern

@router.get("/discovered-patterns/{pattern_id}/examples", response_model=list[DiscoveredPatternExampleOut])
def get_discovered_pattern_examples(
    pattern_id: int,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[DiscoveredPatternExample]:
    pattern = db.get(DiscoveredPattern, pattern_id)
    if not pattern:
        raise HTTPException(404, "discovered pattern not found")
    return (
        db.query(DiscoveredPatternExample)
        .filter(DiscoveredPatternExample.pattern_id == pattern_id)
        .order_by(DiscoveredPatternExample.kind.asc(), DiscoveredPatternExample.similarity.desc())
        .all()
    )

@router.get("/discovered-patterns/{pattern_id}/metrics", response_model=list[DiscoveredPatternMetricOut])
def get_discovered_pattern_metrics(
    pattern_id: int,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[DiscoveredPatternMetric]:
    pattern = db.get(DiscoveredPattern, pattern_id)
    if not pattern:
        raise HTTPException(404, "discovered pattern not found")
    return (
        db.query(DiscoveredPatternMetric)
        .filter(DiscoveredPatternMetric.pattern_id == pattern_id)
        .order_by(DiscoveredPatternMetric.as_of.desc())
        .limit(100)
        .all()
    )

@router.get("/discovered-patterns/{pattern_id}/rr-metrics")
def get_discovered_pattern_rr_metrics(
    pattern_id: int,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> dict[str, object]:
    pattern = db.get(DiscoveredPattern, pattern_id)
    if not pattern:
        raise HTTPException(404, "discovered pattern not found")
    return {
        "id": pattern.id,
        "name": pattern.name,
        "status": pattern.status.value if hasattr(pattern.status, "value") else str(pattern.status),
        "best_rr": pattern.best_rr,
        "best_expectancy_r": pattern.best_expectancy_r,
        "best_profit_factor": pattern.best_profit_factor,
        "best_win_rate": pattern.best_win_rate,
        "best_max_drawdown_r": pattern.best_max_drawdown_r,
        "preferred_rr_passed": pattern.preferred_rr_passed,
        "premium_rr_passed": pattern.premium_rr_passed,
        "promotion_reason": pattern.promotion_reason,
        "rejection_reasons": pattern.rejection_reasons_json,
        "rr_metrics": pattern.rr_metrics_json,
    }

@router.get("/confirmation-queue", response_model=list[DiscoveredPatternOut])
def list_confirmation_queue(
    limit: int = Query(default=50, ge=1, le=200),
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[DiscoveredPattern]:
    return (
        db.query(DiscoveredPattern)
        .filter(DiscoveredPattern.confirmation_status == "needs_confirmation")
        .order_by(DiscoveredPattern.confirmation_priority_score.desc(), DiscoveredPattern.score.desc())
        .limit(limit)
        .all()
    )

@router.post("/match-current", response_model=NovelPatternMatchResponse)
def match_current_patterns(
    request: NovelPatternMatchRequest,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> NovelPatternMatchResponse:
    result = NovelPatternMatcher().match_current(
        db=db,
        symbols=request.symbols,
        limit=request.limit,
        max_patterns=request.max_patterns,
        similarity_threshold=request.similarity_threshold,
        module=request.module,
        store=request.store,
    )

```

```

    )
    return NovelPatternMatchResponse(**result)

@router.get("/current-matches", response_model=list[NovelPatternMatchOut])
def list_current_matches(
    limit: int = Query(default=100, ge=1, le=500),
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[DiscoveredPatternMatch]:
    rows = (
        db.query(DiscoveredPatternMatch)
        .order_by(DiscoveredPatternMatch.matched_at.desc(), DiscoveredPatternMatch.score.desc())
        .limit(min(5000, limit * 10))
        .all()
    )
    deduped: dict[tuple[str, ...], DiscoveredPatternMatch] = {}
    for row in rows:
        key = NovelPatternMatcher.match_dedupe_key_from_model(row)
        if key not in deduped:
            deduped[key] = row
        if len(deduped) >= limit:
            break
    return list(deduped.values())

@router.get("/runs", response_model=list[DiscoveryRunOut])
def list_discovery_runs(
    limit: int = Query(default=50, ge=1, le=200),
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> list[DiscoveryRun]:
    return db.query(DiscoveryRun).order_by(DiscoveryRun.started_at.desc()).limit(limit).all()

@router.post("/director/run", response_model=ResearchDirectorResponse)
def run_research_director(
    run_id: int | None = None,
    limit: int = Query(default=120, ge=1, le=500),
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> ResearchDirectorResponse:
    result = ResearchDirector().run(db, run_id=run_id, limit=limit)
    return ResearchDirectorResponse(**result)

@router.get("/director/latest", response_model=ResearchDirectorResponse)
def latest_research_director(
    _: str = Depends(require_admin),
) -> ResearchDirectorResponse:
    path = get_settings().reports_path / "research" / "director" / "latest_research_director.json"
    if not path.exists():
        raise HTTPException(404, "research director has not generated a report yet")
    payload = json.loads(path.read_text(encoding="utf-8"))
    summary = payload.get("summary") if isinstance(payload, dict) else None
    if not isinstance(summary, dict):
        raise HTTPException(500, "research director latest report is malformed")
    return ResearchDirectorResponse(**summary)

@router.post("/director-review/refresh")
def refresh_director_review_candidates(
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> dict[str, object]:
    return DirectorReviewGate().refresh(db)

```

Full Source: `backend/tradeo/routers/laboratory.py`

`backend/tradeo/routers/laboratory.py` ? 61 lines, 2184 bytes, sha256 prefix `c349a243be406077`

Public surface summary before full source:

```

? function `laboratory_status` line 18`
? function `laboratory_overview` line 26`
? function `laboratory_diagnostics_view` line 34`
? function `scan_laboratory` line 43`

? route laboratory_status: @router.get('/status')
? route laboratory_overview: @router.get('/overview')
? route laboratory_diagnostics_view: @router.get('/diagnostics', response_model=LabDiagnosticsResponse)
? route scan_laboratory: @router.post('/scan', response_model=PatternEntryScanResponse)

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session

from tradeo.core.security import require_admin

```

```

from tradeo.db.session import get_db
from tradeo.modules.laboratory.diagnostics import laboratory_diagnostics
from tradeo.modules.laboratory.scanner import LaboratoryScanner
from tradeo.modules.shared.entry_scanner import PatternEntryScannerSafetyError
from tradeo.schemas import LabDiagnosticsResponse, PatternEntryScanRequest, PatternEntryScanResponse
from tradeo.services.module_dashboard import module_overview

router = APIRouter(prefix="/laboratory", tags=["laboratory"])

@router.get("/status")
def laboratory_status(
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> dict[str, object]:
    return LaboratoryScanner().status(db)

@router.get("/overview")
def laboratory_overview(
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> dict[str, object]:
    return module_overview(db, "laboratory")

@router.get("/diagnostics", response_model=LabDiagnosticsResponse)
def laboratory_diagnostics_view(
    limit: int = Query(default=24, ge=1, le=100),
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> LabDiagnosticsResponse:
    return LabDiagnosticsResponse(**laboratory_diagnostics(db, limit=limit))

@router.post("/scan", response_model=PatternEntryScanResponse)
def scan_laboratory(
    request: PatternEntryScanRequest | None = None,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> PatternEntryScanResponse:
    request = request or PatternEntryScanRequest()
    try:
        result = LaboratoryScanner().scan(
            db,
            symbols=request.symbols,
            limit=request.limit,
            max_patterns=request.max_patterns,
            similarity_threshold=request.similarity_threshold,
            store_signals=request.store_signals,
            execute_orders=request.execute_orders,
        )
    except PatternEntryScannerSafetyError as exc:
        raise HTTPException(status_code=409, detail=str(exc)) from exc
    return PatternEntryScanResponse(**result)

```

Full Source: `backend/tradeo/routers/ibkr.py`

`backend/tradeo/routers/ibkr.py` ? 92 lines, 3182 bytes, sha256 prefix `828ce3cc2697f3a4`

Public surface summary before full source:

```

? class `IBKRSignalOrderRequest` line 18 methods: ?
? function `ibkr_health` line 23`
? function `ibkr_account` line 29`
? function `ibkr_positions` line 37`
? function `ibkr_open_orders` line 45`
? function `ibkr_preview_signal_order` line 53`
? function `ibkr_submit_signal_bracket` line 70`

? route ibkr_health: @router.get('/health')
? route ibkr_account: @router.get('/account')
? route ibkr_positions: @router.get('/positions')
? route ibkr_open_orders: @router.get('/open-orders')
? route ibkr_preview_signal_order: @router.post('/signals/{signal_id}/preview')
? route ibkr_submit_signal_bracket: @router.post('/signals/{signal_id}/submit-bracket', response_model=TradeOut)

```

Risk hints: broad `except Exception` present; IBKR/broker/data dependency.

```

from __future__ import annotations

from pydantic import BaseModel, Field
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from tradeo.core.security import require_admin
from tradeo.db.models import Signal
from tradeo.db.session import get_db
from tradeo.schemas import TradeOut

```

```

from tradeo.services.ibkr_broker import IBKRBroker, IBKRSafetyError
from tradeo.services.ibkr_data_provider import inspect_ibkr_connection
from tradeo.services.order_outcomes import mark_signal_order_failure

router = APIRouter(prefix="/ibkr", tags=["ibkr"])

class IBKRSignalOrderRequest(BaseModel):
    reason: str = Field(default="manual", max_length=300)

@router.get("/health")
def ibkr_health() -> dict[str, object]:
    """Read-only IBKR connectivity check."""
    return inspect_ibkr_connection()

@router.get("/account")
def ibkr_account(_: str = Depends(require_admin)) -> dict[str, object]:
    try:
        return IBKRBroker().account_snapshot()
    except Exception as exc: # noqa: BLE001
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@router.get("/positions")
def ibkr_positions(_: str = Depends(require_admin)) -> list[dict[str, object]]:
    try:
        return IBKRBroker().positions()
    except Exception as exc: # noqa: BLE001
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@router.get("/open-orders")
def ibkr_open_orders(_: str = Depends(require_admin)) -> list[dict[str, object]]:
    try:
        return IBKRBroker().open_orders()
    except Exception as exc: # noqa: BLE001
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@router.post("/signals/{signal_id}/preview")
def ibkr_preview_signal_order(
    signal_id: int,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> dict[str, object]:
    signal = db.get(Signal, signal_id)
    if not signal:
        raise HTTPException(status_code=404, detail="signal not found")
    try:
        return IBKRBroker().preview_signal_order(signal)
    except IBKRSafetyError as exc:
        raise HTTPException(status_code=409, detail=str(exc)) from exc
    except Exception as exc: # noqa: BLE001
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@router.post("/signals/{signal_id}/submit-bracket", response_model=TradeOut)
def ibkr_submit_signal_bracket(
    signal_id: int,
    request: IBKRSignalOrderRequest | None = None,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
):
    signal = db.get(Signal, signal_id)
    if not signal:
        raise HTTPException(status_code=404, detail="signal not found")
    try:
        return IBKRBroker().submit_signal_bracket(
            db,
            signal,
            reason=(request.reason if request else "manual"),
        )
    except IBKRSafetyError as exc:
        mark_signal_order_failure(signal, str(exc))
        db.commit()
        raise HTTPException(status_code=409, detail=str(exc)) from exc
    except Exception as exc: # noqa: BLE001
        mark_signal_order_failure(signal, str(exc))
        db.commit()
        raise HTTPException(status_code=502, detail=str(exc)) from exc

```

Full Source: ``backend/tradeo/routers/risk.py``

``backend/tradeo/routers/risk.py`` ? 37 lines, 1192 bytes, sha256 prefix ``d39ef52b3a23271a``

Public surface summary before full source:

? function ``set_kill_switch`` line 16`

? route `set_kill_switch`: `@router.post('/kill-switch')`

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from tradeo.core.config import get_settings
from tradeo.core.security import require_admin
from tradeo.db.models import AuditLog
from tradeo.db.session import get_db
from tradeo.schemas import KillSwitchRequest

router = APIRouter(prefix="/risk", tags=["risk"])

@router.post("/kill-switch")
def set_kill_switch(
    request: KillSwitchRequest,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> dict[str, object]:
    settings = get_settings()
    # Runtime env values are immutable in Settings cache. This endpoint logs intent for
    # audit and returns clear instructions; Docker env must be changed and restarted.
    db.add(
        AuditLog(
            actor="human",
            action="kill_switch_requested",
            entity_type="risk",
            details_json=request.model_dump(),
        )
    )
    db.commit()
    return {
        "requested": request.enabled,
        "current_env_value": settings.kill_switch_enabled,
        "message": "Set TRADEO_KILL_SWITCH_ENABLED in .env and restart containers to make it persistent.",
    }

```

Full Source: `backend/tradeo/routers/backtests.py`

`backend/tradeo/routers/backtests.py` ? 21 lines, 930 bytes, sha256 prefix `b2202d18f8eb46bf`

Public surface summary before full source:

? function `run\_backtest` line 16`

? route run_backtest: @router.post('/run', response_model=BacktestMetrics)

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from fastapi import APIRouter, Depends

from tradeo.core.config import get_settings
from tradeo.core.security import require_admin
from tradeo.schemas import BacktestMetrics, BacktestRequest
from tradeo.services.backtester import Backtester
from tradeo.services.data_provider import pick_symbols
from tradeo.services.provider_factory import get_market_data_provider

router = APIRouter(prefix="/backtests", tags=["backtests"])

@router.post("/run", response_model=BacktestMetrics)
def run_backtest(request: BacktestRequest, _: str = Depends(require_admin)) -> BacktestMetrics:
    settings = get_settings()
    symbols = request.symbols or pick_symbols(limit=request.max_symbols)
    return Backtester(provider=get_market_data_provider(), starting_equity=settings.initial_capital_usd).run(
        symbols=symbols[: request.max_symbols], period=request.period, interval=request.interval
    )

```

Full Source: `backend/tradeo/routers/scan.py`

`backend/tradeo/routers/scan.py` ? 21 lines, 609 bytes, sha256 prefix `e9f46a6e850af5c6`

Public surface summary before full source:

? function `run\_scan` line 15`

? route run_scan: @router.post("", response_model=ScanResponse)

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

```



```

from tradeo.core.security import require_admin
from tradeo.db.session import get_db
from tradeo.schemas import ScanRequest, ScanResponse
from tradeo.services.scanner import MarketScanner

router = APIRouter(prefix="/scan", tags=["scan"])

@router.post("", response_model=ScanResponse)
def run_scan(
    request: ScanRequest,
    _: str = Depends(require_admin),
    db: Session = Depends(get_db),
) -> ScanResponse:
    scanner = MarketScanner()
    return scanner.run(request, db=db, store=True)

```

Full Source: `backend/tradeo/tasks/worker.py`

`backend/tradeo/tasks/worker.py` ? 374 lines, 13673 bytes, sha256 prefix `64fa161c33db1172`

Public surface summary before full source:

```

? function `scan_job` line 30`
? function `report_job` line 45`
? function `self_improvement_job` line 56`
? function `discovery_job` line 67`
? function `novel_match_job` line 117`
? function `research_director_job` line 138`
? function `laboratory_entry_job` line 156`
? function `fox_hunter_entry_job` line 175`
? function `reconciliation_job` line 192`
? function `pattern_health_job` line 212`
? function `watchdog_job` line 232`
? function `main` line 246`

```

Risk hints: broad `except Exception` present; blocking `time.sleep` present; wall-clock timestamp usage; IBKR/broker/data dependency.

```

from __future__ import annotations

import time
from datetime import datetime, timedelta, timezone

from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.cron import CronTrigger
from loguru import logger

from tradeo.agents.pattern_discovery_lab_agent import PatternDiscoveryLabAgent
from tradeo.core.config import get_settings
from tradeo.db.init_db import init_db, seed_db
from tradeo.db.models import DiscoveryRun
from tradeo.db.session import SessionLocal
from tradeo.research.autonomous_research_director import ResearchDirector
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher
from tradeo.schemas import ScanRequest
from tradeo.modules.shared.entry_scanner import (
    PatternEntryScanner,
    PatternEntryScannerSafetyError,
)
from tradeo.services.director_review_gate import DirectorReviewGate
from tradeo.services.reports import ReportService
from tradeo.services.runtime_status import write_worker_heartbeat
from tradeo.services.scanner import MarketScanner
from tradeo.services.self_improvement import SelfImprovementEngine
from tradeo.services.watchdog import SystemWatchdog

def scan_job() -> None:
    db = SessionLocal()
    try:
        response = MarketScanner().run(
            ScanRequest(limit=get_settings().scan_limit_default),
            db=db,
            store=True,
        )
        logger.info("scan complete: {}", response.model_dump())
    except Exception as exc: # noqa: BLE001
        logger.exception("scan job failed: {}", exc)
    finally:
        db.close()

def report_job() -> None:
    db = SessionLocal()
    try:
        pack = ReportService().generate_review_pack(db)
        logger.info("report generated: {}", pack.get("paths"))
    except Exception as exc: # noqa: BLE001
        logger.exception("report job failed: {}", exc)

```

```

finally:
    db.close()

def self_improvement_job() -> None:
    db = SessionLocal()
    try:
        result = SelfImprovementEngine().run_lab_cycle(db, max_symbols=25)
        logger.info("self-improvement result: {}", result.model_dump())
    except Exception as exc: # noqa: BLE001
        logger.exception("self-improvement job failed: {}", exc)
    finally:
        db.close()

def discovery_job() -> None:
    db = SessionLocal()
    try:
        from tradeo.schemas import DiscoveryRunRequest

        settings = get_settings()
        if not settings.discovery_enabled:
            logger.info("discovery job skipped: discovery_enabled=false")
            return
        request = DiscoveryRunRequest(
            limit=settings.discovery_limit_default,
            period=settings.discovery_period,
            interval=settings.discovery_interval,
            max_total_windows=settings.discovery_max_total_windows,
            max_windows_per_symbol=settings.discovery_max_windows_per_symbol,
        )
        params = {
            "limit": request.limit,
            "period": request.period,
            "interval": request.interval,
            "max_total_windows": request.max_total_windows,
            "max_windows_per_symbol": request.max_windows_per_symbol,
            "rr_levels": settings.discovery_rr_level_list,
            "window_sizes": settings.discovery_window_size_list,
            "forwardBars": settings.discovery_forward_bar_list,
        }
        running = db.query(DiscoveryRun).filter(DiscoveryRun.status == "running").first()
        if running:
            logger.info("discovery job skipped: run {} already running", running.id)
            return
        recent_cutoff = datetime.now(timezone.utc) - timedelta(
            minutes=max(1, settings.discovery_scan_minutes)
        )
        recent = (
            db.query(DiscoveryRun)
            .filter(DiscoveryRun.status == "completed", DiscoveryRun.started_at >= recent_cutoff)
            .order_by(DiscoveryRun.started_at.desc())
            .first()
        )
        if recent and {k: recent.params_json.get(k) for k in params} == params:
            logger.info("discovery job skipped: recent equivalent run {}", recent.id)
            return
        result = PatternDiscoveryLabAgent().run(request, db)
        logger.info("pattern discovery result: {}", result.model_dump())
    except Exception as exc: # noqa: BLE001
        logger.exception("pattern discovery job failed: {}", exc)
    finally:
        db.close()

def novel_match_job() -> None:
    db = SessionLocal()
    try:
        settings = get_settings()
        if not settings.discovery_match_enabled:
            logger.info("novel pattern match job skipped: discovery_match_enabled=false")
            return
        result = NovelPatternMatcher().match_current(
            db,
            limit=settings.discovery_match_symbol_limit,
            max_patterns=settings.discovery_match_max_patterns,
            similarity_threshold=settings.discovery_match_similarity_threshold,
            store=True,
        )
        logger.info("novel pattern match result: {}", result)
    except Exception as exc: # noqa: BLE001
        logger.exception("novel pattern match job failed: {}", exc)
    finally:
        db.close()

def research_director_job() -> None:
    db = SessionLocal()
    try:
        settings = get_settings()
        if not settings.research_director_enabled:
            logger.info("research director skipped: research_director_enabled=false")
            return
        result = ResearchDirector(settings).run(
            db,
            limit=settings.research_director_pattern_limit,

```

```

    )
    logger.info("research director result: {}", result.get("director_state", {}))
except Exception as exc: # noqa: BLE001
    logger.exception("research director failed: {}", exc)
finally:
    db.close()

def laboratory_entry_job() -> None:
    db = SessionLocal()
    try:
        settings = get_settings()
        if not settings.laboratory_scanner_enabled:
            logger.info("laboratory entry scanner skipped: laboratory_scanner_enabled=false")
            return
        result = PatternEntryScanner(settings=settings).scan(db, module="laboratory")
        review_result = DirectorReviewGate.from_settings(settings).refresh(db)
        logger.info("laboratory entry scanner result: {}", result)
        logger.info("director review gate result: {}", review_result)
    except PatternEntryScannerSafetyError as exc:
        logger.warning("laboratory entry scanner blocked by safety gate: {}", exc)
    except Exception as exc: # noqa: BLE001
        logger.exception("laboratory entry scanner failed: {}", exc)
    finally:
        db.close()

def fox_hunter_entry_job() -> None:
    db = SessionLocal()
    try:
        settings = get_settings()
        if not settings.fox_hunter_enabled:
            logger.info("fox hunter skipped: fox_hunter_enabled=false")
            return
        result = PatternEntryScanner(settings=settings).scan(db, module="fox_hunter")
        logger.info("fox hunter result: {}", result)
    except PatternEntryScannerSafetyError as exc:
        logger.warning("fox hunter blocked by safety gate: {}", exc)
    except Exception as exc: # noqa: BLE001
        logger.exception("fox hunter failed: {}", exc)
    finally:
        db.close()

def reconciliation_job() -> None:
    db = SessionLocal()
    try:
        settings = get_settings()
        if not settings.reconciliation_enabled:
            logger.info("reconciliation skipped: reconciliation_enabled=false")
            return
        from tradeo.services.reconciliation import ReconciliationService

        result = ReconciliationService(settings=settings).reconcile(db)
        if result.get("divergences"):
            logger.error("reconciliation divergences: {}", result)
        else:
            logger.info("reconciliation result: {}", result)
    except Exception as exc: # noqa: BLE001
        logger.exception("reconciliation job failed: {}", exc)
    finally:
        db.close()

def pattern_health_job() -> None:
    db = SessionLocal()
    try:
        settings = get_settings()
        if not settings.health_monitor_enabled:
            logger.info("pattern health monitor skipped: health_monitor_enabled=false")
            return
        from tradeo.services.pattern_health_monitor import PatternHealthMonitor

        result = PatternHealthMonitor(settings=settings).run(db)
        if result.get("decay_detected"):
            logger.warning("pattern health monitor result: {}", result)
        else:
            logger.info("pattern health monitor result: {}", result)
    except Exception as exc: # noqa: BLE001
        logger.exception("pattern health monitor job failed: {}", exc)
    finally:
        db.close()

def watchdog_job() -> None:
    db = SessionLocal()
    try:
        result = SystemWatchdog().repair(db)
        if result.get("repaired"):
            logger.warning("watchdog result: {}", result)
        else:
            logger.debug("watchdog ok: {}", result)
    except Exception as exc: # noqa: BLE001
        logger.exception("watchdog job failed: {}", exc)
    finally:
        db.close()

```

```

def main() -> None:
    settings = get_settings()
    write_worker_heartbeat(settings)
    init_db()
    db = SessionLocal()
    try:
        seed_db(db)
    finally:
        db.close()
    scheduler = BackgroundScheduler(timezone="UTC")
    if settings.scheduler_enabled:
        if settings.watchdog_enabled:
            watchdog_job()
            scheduler.add_job(
                watchdog_job,
                "interval",
                minutes=settings.watchdog_interval_minutes,
                id="system_watchdog",
                max_instances=1,
                coalesce=True,
            )
        scheduler.add_job(
            scan_job,
            "interval",
            minutes=settings.scheduler_scan_minutes,
            id="market_scan",
        )
        scheduler.add_job(
            report_job,
            CronTrigger(hour=settings.scheduler_report_hour_utc, minute=5, day_of_week="mon-fri"),
            id="daily_report",
        )
        scheduler.add_job(
            self_improvement_job,
            CronTrigger(hour=23, minute=15, day_of_week="fri"),
            id="weekly_self_improvement",
        )
    if settings.discovery_scheduler_enabled:
        if settings.discovery_scan_minutes >= 1440:
            # Daily data only gains information once per completed session.
            # Re-running intraday multiplies N_trials and burns IBKR pacing
            # without new bars, so schedule a single post-close run instead.
            scheduler.add_job(
                discovery_job,
                CronTrigger(
                    hour=settings.discovery_post_close_hour_utc,
                    minute=settings.discovery_post_close_minute_utc,
                    day_of_week="mon-fri",
                ),
                id="pattern_discovery_lab",
                max_instances=1,
                coalesce=True,
            )
        else:
            scheduler.add_job(
                discovery_job,
                "interval",
                minutes=settings.discovery_scan_minutes,
                id="pattern_discovery_lab",
                max_instances=1,
                coalesce=True,
            )
    if settings.discovery_match_enabled:
        scheduler.add_job(
            novel_match_job,
            "interval",
            minutes=settings.discovery_match_scan_minutes,
            id="novel_pattern_matcher",
            max_instances=1,
            coalesce=True,
        )
    if settings.research_director_enabled:
        research_director_job()
        scheduler.add_job(
            research_director_job,
            "interval",
            minutes=settings.research_director_interval_minutes,
            id="research_director",
            max_instances=1,
            coalesce=True,
        )
    if settings.laboratory_scanner_enabled:
        scheduler.add_job(
            laboratory_entry_job,
            "interval",
            minutes=settings.laboratory_scan_minutes,
            id="laboratory_entry_scanner",
            max_instances=1,
            coalesce=True,
        )
    if settings.fox_hunter_enabled:
        scheduler.add_job(
            fox_hunter_entry_job,
            "interval",

```

```

        minutes=settings.fox_hunter_scan_minutes,
        id="fox_hunter_entry_scanner",
        max_instances=1,
        coalesce=True,
    )
    if settings.reconciliation_enabled:
        scheduler.add_job(
            reconciliation_job,
            "interval",
            minutes=settings.reconciliation_interval_minutes,
            id="ibkr_reconciliation",
            max_instances=1,
            coalesce=True,
        )
    if settings.health_monitor_enabled:
        scheduler.add_job(
            pattern_health_job,
            "interval",
            minutes=settings.health_monitor_interval_minutes,
            id="pattern_health_monitor",
            max_instances=1,
            coalesce=True,
        )
    scheduler.start()
    logger.info("Tradeo worker started. scheduler_enabled={}", settings.scheduler_enabled)
    try:
        while True:
            write_worker_heartbeat(settings)
            time.sleep(5)
    except KeyboardInterrupt:
        scheduler.shutdown()

if __name__ == "__main__":
    main()

```

Full Source: `backend/tradeo/services/ibkr\_broker.py`

`backend/tradeo/services/ibkr\_broker.py` ? 454 lines, 19316 bytes, sha256 prefix `f83b6b4140c5a70c`

Audit relevance: IBKR order adapter and hard safety gate for preview/submit bracket orders. Audit read-only/live mode checks, kill switch, account privacy, what-if previews, bracket validation, order acknowledgements, persistence, and whether API/entry-scanner callers can bypass these gates.

```

from __future__ import annotations

import asyncio
import random
import time
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any

from sqlalchemy.orm import Session

from tradeo.core.config import Settings, get_settings
from tradeo.db.models import AuditLog, Signal, SignalStatus, Trade, TradeStatus
from tradeo.services.evidence import EvidenceQuality, EvidenceType

class IBKRSafetyError(RuntimeError):
    """Raised when a hard safety gate blocks IBKR execution."""

def _ensure_event_loop() -> None:
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        asyncio.set_event_loop(asyncio.new_event_loop())

def _finite_price(value: float | None, field: str) -> float:
    if value is None:
        raise IBKRSafetyError(f"{field} is required")
    value = float(value)
    if value <= 0:
        raise IBKRSafetyError(f"{field} must be positive")
    return round(value, 4)

@dataclass
class IBKRBroker:
    """Interactive Brokers adapter with conservative execution gates.

    This adapter is intentionally defensive:
    - read-only mode blocks every order path;
    - live ports require the explicit live-armed gate;
    - human approval is mandatory;
    - only simple STK/SMART/USD bracket orders are supported.
    """

    settings: Settings | None = None

```

```

def __post_init__(self) -> None:
    self.settings = self.settings or get_settings()

@property
def connect_timeout(self) -> float:
    return float(getattr(self.settings, "ibkr_connect_timeout_seconds", 8.0))

@property
def order_timeout(self) -> float:
    return float(getattr(self.settings, "ibkr_order_timeout_seconds", 20.0))

def _connect(self):
    _ensure_event_loop()
    from ib_insync import IB

    client_ids = random.sample(range(1000, 10000), 4)
    if self.settings.ibkr_client_id not in client_ids:
        client_ids.append(self.settings.ibkr_client_id)
    last_exc: Exception | None = None
    for client_id in client_ids:
        ib = IB()
        try:
            ib.connect(
                self.settings.ibkr_host,
                self.settings.ibkr_port,
                clientId=client_id,
                timeout=self.connect_timeout,
            )
            return ib
        except Exception as exc: # noqa: BLE001
            last_exc = exc
            if ib.isConnected():
                ib.disconnect()
            continue
    if last_exc:
        raise last_exc
    raise IBKRSafetyError("IBKR connection failed")

def status(self) -> dict[str, Any]:
    from ib_insync import util

    ib = self._connect()
    try:
        server_time = ib.reqCurrentTime()
        accounts = ib.managedAccounts()
        return {
            "ok": True,
            "host": self.settings.ibkr_host,
            "port": self.settings.ibkr_port,
            "client_id": self.settings.ibkr_client_id,
            "readonly": self.settings.ibkr_readonly,
            "trading_mode": self.settings.trading_mode,
            "live_armed": self.settings.live_armed,
            "kill_switch_enabled": self.settings.kill_switch_enabled,
            "server_time": util.formatIBDatetime(server_time),
            "managed_accounts_count": len(accounts),
            "selected_account": self.settings.ibkr_account or (accounts[0] if accounts else None),
        }
    finally:
        if ib.isConnected():
            ib.disconnect()

def account_snapshot(self) -> dict[str, Any]:
    ib = self._connect()
    try:
        accounts = ib.managedAccounts()
        account = self.settings.ibkr_account or (accounts[0] if accounts else None)
        if not account:
            raise IBKRSafetyError("IBKR returned no managed accounts")
        wanted = {
            "NetLiquidation",
            "TotalCashValue",
            "BuyingPower",
            "AvailableFunds",
            "GrossPositionValue",
            "ExcessLiquidity",
            "MaintMarginReq",
            "InitMarginReq",
        }
        summary: dict[str, dict[str, str]] = {}
        for item in ib.accountSummary(account):
            if item.tag in wanted:
                summary[item.tag] = {"value": item.value, "currency": item.currency}
        return {
            "ok": True,
            "account": account,
            "readonly": self.settings.ibkr_readonly,
            "trading_mode": self.settings.trading_mode,
            "live_armed": self.settings.live_armed,
            "summary": summary,
        }
    finally:
        if ib.isConnected():
            ib.disconnect()

```

```

def positions(self) -> list[dict[str, Any]]:
    ib = self._connect()
    try:
        rows: list[dict[str, Any]] = []
        for p in ib.positions():
            contract = p.contract
            rows.append(
                {
                    "account": p.account,
                    "symbol": getattr(contract, "symbol", ""),
                    "sec_type": getattr(contract, "secType", ""),
                    "exchange": getattr(contract, "exchange", ""),
                    "currency": getattr(contract, "currency", ""),
                    "position": float(p.position),
                    "avg_cost": float(p.avgCost),
                }
            )
        return rows
    finally:
        if ib.isConnected():
            ib.disconnect()

def open_orders(self) -> list[dict[str, Any]]:
    ib = self._connect()
    try:
        ib.reqAllOpenOrders()
        ib.sleep(1.0)
        rows: list[dict[str, Any]] = []
        for trade in ib.openTrades():
            contract = trade.contract
            order = trade.order
            status = trade.orderStatus
            rows.append(
                {
                    "symbol": getattr(contract, "symbol", ""),
                    "sec_type": getattr(contract, "secType", ""),
                    "order_id": order.orderId,
                    "perm_id": status.permId,
                    "action": order.action,
                    "order_type": order.orderType,
                    "quantity": float(order.totalQuantity),
                    "limit_price": getattr(order, "lmtPrice", None),
                    "aux_price": getattr(order, "auxPrice", None),
                    "status": status.status,
                    "filled": float(status.filled),
                    "remaining": float(status.remaining),
                    "parent_order_id": order.parentId,
                }
            )
        return rows
    finally:
        if ib.isConnected():
            ib.disconnect()

def _stock_contract(self, symbol: str):
    from ib_insync import Stock

    return Stock(symbol.upper(), "SMART", "USD")

def _build_parent_limit_order(self, signal: Signal):
    from ib_insync import Order

    action = self._action_for_signal(signal)
    order = Order()
    order.action = action
    order.orderType = "LMT"
    order.totalQuantity = int(signal.suggested_qty)
    order.lmtPrice = _finite_price(signal.entry, "entry")
    order.tif = "DAY"
    if self.settings.ibkr_account:
        order.account = self.settings.ibkr_account
    return order

def _action_for_signal(self, signal: Signal) -> str:
    side = signal.side.lower().strip()
    if side == "long":
        return "BUY"
    if side == "short":
        if not self.settings.allow_shorts:
            raise IBKRSafetyError("short orders are disabled by TRADEO_ALLOW_SHORTS")
        return "SELL"
    raise IBKRSafetyError(f"unsupported signal side: {signal.side}")

def _validate_signal_for_order(self, signal: Signal, *, require_execution_enabled: bool) -> None:
    if self.settings.kill_switch_enabled:
        raise IBKRSafetyError("kill switch is enabled")
    if signal is None:
        raise IBKRSafetyError("signal is required")
    if signal.suggested_qty <= 0:
        raise IBKRSafetyError("signal suggested_qty must be positive")
    if signal.side.lower() not in {"long", "short"}:
        raise IBKRSafetyError(f"unsupported signal side: {signal.side}")
    if not signal.human_approved:
        raise IBKRSafetyError("human approval is required before IBKR order submission")
    if signal.status not in {SignalStatus.PAPER_APPROVED, SignalStatus.LIVE_APPROVED}:
        raise IBKRSafetyError(f"signal status is not executable through IBKR: {signal.status}")

```

```

allowed_symbols = getattr(self.settings, "ibkr_allowed_symbol_set", set())
if allowed_symbols and signal.symbol.upper() not in allowed_symbols:
    raise IBKRSafetyError(f"{signal.symbol} is not in TRADEO_IBKR_ALLOWED_SYMBOLS")

entry = _finite_price(signal.entry, "entry")
stop = _finite_price(signal.stop, "stop")
target = _finite_price(signal.target, "target")
notional = abs(entry * int(signal.suggested_qty))
max_order_value = float(getattr(self.settings, "ibkr_max_order_value_usd", 1500.0))
if notional > max_order_value:
    raise IBKRSafetyError(
        f"order notional ${notional:.2f} exceeds TRADEO_IBKR_MAX_ORDER_VALUE_USD=${max_order_value:.2f}"
    )

if signal.side.lower() == "long" and not (stop < entry < target):
    raise IBKRSafetyError("long bracket requires stop < entry < target")
if signal.side.lower() == "short" and not (target < entry < stop):
    raise IBKRSafetyError("short bracket requires target < entry < stop")

if require_execution_enabled:
    if self.settings.ibkr_readonly:
        raise IBKRSafetyError("TRADEO_IBKR_READONLY=true blocks order submission")
    if self.settings.trading_mode == "research":
        raise IBKRSafetyError("research mode blocks order submission")
    live_port = int(self.settings.ibkr_port) in {7496, 4001}
    if self.settings.trading_mode == "live" or live_port:
        if not self.settings.live_armed:
            raise IBKRSafetyError("live IBKR execution requires live_armed=true")

def preview_signal_order(self, signal: Signal) -> dict[str, Any]:
    """Run an IBKR WhatIf preview for the parent limit order without submitting it."""
    self._validate_signal_for_order(signal, require_execution_enabled=False)
    ib = self._connect()
    try:
        contract = self._stock_contract(signal.symbol)
        qualified = ib.qualifyContracts(contract)
        if not qualified:
            raise IBKRSafetyError(f"IBKR could not qualify contract for {signal.symbol}")
        contract = qualified[0]
        order = self._build_parent_limit_order(signal)
        state = ib.whatIfOrder(contract, order)
        return {
            "ok": True,
            "signal_id": signal.id,
            "symbol": signal.symbol,
            "side": signal.side,
            "qty": int(signal.suggested_qty),
            "entry": float(signal.entry),
            "stop": float(signal.stop),
            "target": float(signal.target),
            "notional_usd": round(float(signal.entry) * int(signal.suggested_qty), 2),
            "contract": {
                "con_id": contract.conId,
                "symbol": contract.symbol,
                "sec_type": contract.secType,
                "exchange": contract.exchange,
                "currency": contract.currency,
            },
            "what_if": {
                "init_margin_change": getattr(state, "initMarginChange", ""),
                "maint_margin_change": getattr(state, "maintMarginChange", ""),
                "equity_with_loan_change": getattr(state, "equityWithLoanChange", ""),
                "commission": getattr(state, "commission", None),
                "min_commission": getattr(state, "minCommission", None),
                "max_commission": getattr(state, "maxCommission", None),
                "warning_text": getattr(state, "warningText", ""),
                "status": getattr(state, "status", ""),
            },
        }
    finally:
        if ib.isConnected():
            ib.disconnect()

def submit_signal_bracket(self, db: Session, signal: Signal, *, reason: str = "manual") -> Trade:
    """Submit a bracket order to IBKR and persist the Trade record.

    This method can target a paper TWS/Gateway session when TRADEO_TRADING_MODE=paper.
    For live ports or TRADEO_TRADING_MODE=live, the live_armed gate is mandatory.
    """
    from tradeo.services.system_controls import runtime_kill_switch_active

    if runtime_kill_switch_active(db):
        raise IBKRSafetyError("runtime kill switch is active (see system_controls)")
    self._validate_signal_for_order(signal, require_execution_enabled=True)
    ib = self._connect()
    try:
        contract = self._stock_contract(signal.symbol)
        qualified = ib.qualifyContracts(contract)
        if not qualified:
            raise IBKRSafetyError(f"IBKR could not qualify contract for {signal.symbol}")
        contract = qualified[0]
        action = self._action_for_signal(signal)
        bracket = ib.bracketOrder(
            action=action,
            quantity=int(signal.suggested_qty),
            limitPrice=_finite_price(signal.entry, "entry"),

```



```

        takeProfitPrice=_finite_price(signal.target, "target"),
        stopLossPrice=_finite_price(signal.stop, "stop"),
    )
    for order in bracket:
        order.tif = "DAY"
        if self.settings.ibkr_account:
            order.account = self.settings.ibkr_account
    trades = []
    for order in bracket:
        trades.append(ib.placeOrder(contract, order))
        ib.sleep(0.25)
    deadline = time.monotonic() + self.order_timeout
    while time.monotonic() < deadline:
        ib.sleep(0.5)
        if all(t.orderStatus.permId for t in trades):
            break
    if not all(t.orderStatus.permId for t in trades):
        for trade_status in trades:
            ib.cancelOrder(trade_status.order)
        ib.sleep(1.0)
        raise IBKRSafetyError("IBKR did not acknowledge every bracket leg with a permId")

    parent_trade = trades[0]
    parent_order = parent_trade.order
    order_ids = [t.order.orderId for t in trades]
    perm_ids = [t.orderStatus.permId for t in trades]
    now = datetime.now(timezone.utc)
    signal_metadata = signal.metadata_json or {}
    evidence_type = (
        EvidenceType.LIVE_ORDER.value
        if self.settings.trading_mode == "live"
        else EvidenceType.IBKR_PAPER_ORDER.value
    )
    execution_observation = {
        "source": "ibkr_order_submission",
        "truth_status": "orders_accepted_waiting_fill_reconciliation",
        "submitted_at": now.isoformat(),
        "evidence_type": evidence_type,
        "evidence_quality": EvidenceQuality.NORMAL.value,
        "entry_variant_id": signal_metadata.get("entry_variant_id"),
        "regime_key": (signal_metadata.get("regime") or {}).get("regime_key"),
        "order_ids": order_ids,
        "perm_ids": perm_ids,
    }
    trade = Trade(
        signal_id=signal.id,
        symbol=signal.symbol,
        pattern=signal.pattern,
        side=signal.side,
        qty=signal.suggested_qty,
        entry=signal.entry,
        stop=signal.stop,
        target=signal.target,
        status=TradeStatus.OPEN,
        opened_at=now,
        broker_order_id=str(parent_order.orderId),
        evidence_type=evidence_type,
        evidence_quality=EvidenceQuality.NORMAL.value,
        metadata_json={
            "evidence_type": evidence_type,
            "evidence_quality": EvidenceQuality.NORMAL.value,
            "execution_mode": "ibkr",
            "ibkr_mode": self.settings.trading_mode,
            "source_signal": signal.id,
            "reason": reason,
            "entry_variant_id": signal_metadata.get("entry_variant_id"),
            "entry_variant": signal_metadata.get("entry_variant"),
            "entry_audit": signal_metadata.get("entry_audit"),
            "regime": signal_metadata.get("regime"),
            "regime_fit": signal_metadata.get("regime_fit"),
            "execution_observation": execution_observation,
            "contract": {
                "con_id": contract.conId,
                "symbol": contract.symbol,
                "sec_type": contract.secType,
                "exchange": contract.exchange,
                "currency": contract.currency,
            },
            "order_ids": order_ids,
            "perm_ids": perm_ids,
            "parent_order_id": parent_order.orderId,
            "submitted_at": now.isoformat(),
            "readonly": self.settings.ibkr_readonly,
            "live_armed": self.settings.live_armed,
        },
    )
    signal.status = SignalStatus.EXECUTED
    signal.metadata_json = {
        **signal_metadata,
        "evidence_type": evidence_type,
        "evidence_quality": EvidenceQuality.NORMAL.value,
        "execution_observation": execution_observation,
    }
    db.add(trade)
    db.add(
        AuditLog(

```

```

        actor="ibkr_broker",
        action="ibkr_bracket_submitted",
        entity_type="trade",
        entity_id=str(signal.id),
        details_json={
            "signal_id": signal.id,
            "symbol": signal.symbol,
            "qty": signal.suggested_qty,
            "order_ids": order_ids,
            "perm_ids": perm_ids,
            "trading_mode": self.settings.trading_mode,
            "port": self.settings.ibkr_port,
            "reason": reason,
        },
    )
)
db.commit()
db.refresh(trade)
return trade
finally:
    if ib.isConnected():
        ib.disconnect()

```

Relevant Tests Full Source Appendix

Full source for the tests most relevant to the requested audit surfaces. These tests are source evidence, not proof they were run during pack generation.

Full Source: ``backend/tradeo/tests/conftest.py``

``backend/tradeo/tests/conftest.py`` ? 23 lines, 795 bytes, sha256 prefix ``2df191924c04ff0e``

```

"""Test-session environment defaults.

Settings path properties (reports, artifacts, market data cache, universe
snapshots) create their directories on first access. The shipped defaults
live under ``/app`` which only exists inside the Docker image, so local test
runs would fail with ``PermissionError``. Point them at a temp dir unless the
environment already overrides them.
"""

from __future__ import annotations

import os
import tempfile

_TEST_DATA_ROOT = tempfile.mkdtemp(prefix="tradeo-tests-")

for _name, _sub in (
    ("TRADEO_MARKET_DATA_CACHE_DIR", "ohlcvcache"),
    ("TRADEO_UNIVERSE_SNAPSHOT_DIR", "universe_snapshots"),
    ("TRADEO_REPORTS_DIR", "reports"),
    ("TRADEO_ARTIFACTS_DIR", "artifacts"),
):
    os.environ.setdefault(_name, os.path.join(_TEST_DATA_ROOT, _sub))

```

Full Source: ``backend/tradeo/tests/fixtures.py``

``backend/tradeo/tests/fixtures.py`` ? 28 lines, 1129 bytes, sha256 prefix ``48afeb0f2ac2efa7``

Public surface summary before full source:

? function ``fixture_ohlcvc`` line 7

Risk hints: wall-clock timestamp usage; RNG path; verify seed/control.

```

from __future__ import annotations

import numpy as np
import pandas as pd

def fixture_ohlcvc(symbol: str, bars: int = 420) -> pd.DataFrame:
    seed = abs(hash(symbol)) % (2**32)
    rng = np.random.default_rng(seed)
    returns = rng.normal(0.0005, 0.025, bars)
    price = 20 * np.exp(np.cumsum(returns))
    if seed % 3 == 0 and bars > 150:
        n = 100
        x = np.linspace(-1, 1, n)
        cup = 1 - 0.22 * (1 - x**2)
        base = price[-n] * cup * (1 + np.linspace(0, 0.08, n))
        price[-n:] = base * (1 + rng.normal(0, 0.012, n))
        price[-1] = max(price[-20:]) * 1.015
    open_ = price * (1 + rng.normal(0, 0.008, bars))
    high = np.maximum(open_, price) * (1 + rng.uniform(0.005, 0.025, bars))
    low = np.minimum(open_, price) * (1 - rng.uniform(0.005, 0.025, bars))
    volume = rng.integers(300_000, 3_000_000, bars).astype(float)

```

```

if seed % 3 == 0:
    volume[-1] *= 2.5
idx = pd.date_range(end=pd.Timestamp.now(tz="UTC").date(), periods=bars, freq="B")
return pd.DataFrame(
    {"open": open_, "high": high, "low": low, "close": price, "volume": volume}, index=idx
)

```

Full Source: `backend/tradeo/tests/test\_nested\_optimization.py`

`backend/tradeo/tests/test\_nested\_optimization.py` ? 185 lines, 7235 bytes, sha256 prefix `2e3f33358a6d3de2`

Public surface summary before full source:

```

? function `test_overfit_specialists_are_blocked` line 43`
? function `test_persistent_edge_passes` line 56`
? function `test_consistent_loser_fails_outer_score_gate` line 68`
? function `test_insufficient_periods_blocks` line 76`
? function `test_insufficient_variants_blocks` line 83`
? function `test_non_finite_values_block` line 89`
? function `test_report_is_deterministic` line 97`
? function `test_fallback_contract_reports_optuna_availability` line 103`
? function `test_optuna_inner_search_is_seeded_and_deterministic` line 111`
? function `test_engine_nested_guard_blocks_specialist_overfit` line 138`
? function `test_engine_nested_guard_accepts_persistent_edge_report` line 157`
? function `test_engine_nested_disabled_fails_closed` line 164`
? function `test_lab_gate_requires_all_three_guards` line 172`

```

Risk hints: RNG path; verify seed/control.

```

from __future__ import annotations

import json

import numpy as np
import pytest

from tradeo.core.config import Settings
from tradeo.research.nested_optimization import (
    nested_optimization_report,
    optuna_available,
)
from tradeo.services.self_improvement import SelfImprovementEngine

N_PERIODS = 20
N_VARIANTS = 5
N_FOLDS = 5

def _specialist_overfit_matrix() -> np.ndarray:
    """Each variant j is great everywhere except in outer fold j, where it crashes.

    Inner selection on the complement of fold j always picks variant j (it hides
    its bad fold), and the outer fold then ranks it dead last: the canonical
    selection-overfitting shape that nested evaluation must block.
    """
    perf = np.full((N_PERIODS, N_VARIANTS), 1.0)
    folds = np.array_split(np.arange(N_PERIODS), N_FOLDS)
    for j, rows in enumerate(folds):
        perf[rows, j] = -5.0
    return perf

def _robust_matrix() -> np.ndarray:
    """Variant 0 has a persistent positive edge; the others are flat-to-negative."""
    perf = np.zeros((N_PERIODS, N_VARIANTS))
    for j in range(N_VARIANTS):
        perf[:, j] = -0.1 * j
    perf[:, 0] = 0.5 + 0.01 * np.cos(np.arange(N_PERIODS))
    return perf

def test_overfit_specialists_are_blocked() -> None:
    report = nested_optimization_report(_specialist_overfit_matrix(), n_outer_folds=N_FOLDS)
    assert report["blocked"] is False
    assert report["passed"] is False
    assert report["pbo_outer"] == 1.0
    assert report["pbo_passed"] is False
    assert report["outer_median_selected_score"] < 0.0
    assert all(f["outer_rank"] == 1 for f in report["outer_folds"])
    assert all(f["lambda"] < 0 for f in report["outer_folds"])
    # Every fold picks a different "specialist": maximal selection instability.
    assert report["selection_stability"] == pytest.approx(1.0 / N_FOLDS)

def test_persistent_edge_passes() -> None:
    report = nested_optimization_report(_robust_matrix(), n_outer_folds=N_FOLDS)
    assert report["blocked"] is False
    assert report["passed"] is True
    assert report["pbo_outer"] == 0.0

```

```

assert report["outer_median_selected_score"] > 0.0
assert report["consensus_variant"] == 0
assert report["selection_stability"] == 1.0
assert all(f["selected_variant"] == 0 for f in report["outer_folds"])
assert all(f["outer_rank"] == N_VARIANTS for f in report["outer_folds"])

def test_consistent_loser_fails_outer_score_gate() -> None:
    perf = _robust_matrix() - 2.0 # stable ranking, but everything loses money
    report = nested_optimization_report(perf, n_outer_folds=N_FOLDS)
    assert report["pbo_passed"] is True
    assert report["outer_score_passed"] is False
    assert report["passed"] is False

def test_insufficient_periods_blocks() -> None:
    report = nested_optimization_report(np.ones((4, 3)), n_outer_folds=N_FOLDS)
    assert report["blocked"] is True
    assert report["passed"] is False
    assert report["reason"] == "insufficient_periods_for_outer_folds"

def test_insufficient_variants_blocks() -> None:
    report = nested_optimization_report(np.ones((20, 1)), n_outer_folds=N_FOLDS)
    assert report["blocked"] is True
    assert report["reason"] == "insufficient_variants_for_nested_search"

def test_non_finite_values_block() -> None:
    perf = _robust_matrix()
    perf[3, 2] = np.nan
    report = nested_optimization_report(perf, n_outer_folds=N_FOLDS)
    assert report["blocked"] is True
    assert report["reason"] == "non_finite_performance_values"

def test_report_is_deterministic() -> None:
    a = nested_optimization_report(_specialist_overfit_matrix(), n_outer_folds=N_FOLDS)
    b = nested_optimization_report(_specialist_overfit_matrix(), n_outer_folds=N_FOLDS)
    assert json.dumps(a, sort_keys=True) == json.dumps(b, sort_keys=True)

def test_fallback_contract_reports_optuna_availability() -> None:
    report = nested_optimization_report(_robust_matrix(), n_outer_folds=N_FOLDS)
    assert report["optuna_available"] is optuna_available()
    if not optuna_available():
        assert report["inner_method"] == "exhaustive_deterministic"

@pytest.mark.skipif(not optuna_available(), reason="optuna not installed")
def test_optuna_inner_search_is_seeded_and_deterministic() -> None:
    rng = np.random.default_rng(7)
    perf = rng.normal(0.0, 0.2, size=(20, 96))
    perf[:, 11] += 1.0
    a = nested_optimization_report(perf, n_outer_folds=N_FOLDS, inner_trials=64)
    b = nested_optimization_report(perf, n_outer_folds=N_FOLDS, inner_trials=64)
    assert a["inner_method"] == "optuna_tpe_seeded_no_pruning"
    assert json.dumps(a, sort_keys=True) == json.dumps(b, sort_keys=True)

def _engine(tmp_path, **overrides) -> SelfImprovementEngine:
    return SelfImprovementEngine(Settings(reports_dir=str(tmp_path), **overrides))

def _records_from_matrix(perf: np.ndarray) -> list[dict]:
    months = [f"2023-{m:02d}" for m in range(1, 13)] + [f"2024-{m:02d}" for m in range(1, 9)]
    assert len(months) == perf.shape[0]
    records = []
    for j in range(perf.shape[1]):
        trades = [
            {"exit_date": f"{month}-15", "r_multiple": float(perf[t, j])}
            for t, month in enumerate(months)
        ]
        records.append({"config": {"min_depth": 0.10 + 0.01 * j}, "metrics": {"trades": trades}})
    return records

def test_engine_nested_guard_blocks_specialist_overfit(tmp_path) -> None:
    engine = _engine(tmp_path)
    guards, summary = engine._anti_overfit_guards(_records_from_matrix(_specialist_overfit_matrix()))
    nested = summary["nested"]
    assert nested["blocked"] is False
    assert nested["passed"] is False
    assert nested["pbo_outer"] == 1.0
    assert nested["period_kind"] == "monthly_realized_r_buckets"
    assert guards[0]["nested_passed"] is False
    metrics = {
        "total_trades": 50,
        "profit_factor": 2.5,
        "expectancy_r": 0.4,
        "max_drawdown_pct": 5.0,
    }
    guard = {"pbo_passed": True, "plateau_passed": True, "nested_passed": False}
    assert engine._passes_lab_gate(metrics, guard) is False

```

```

def test_engine_nested_guard_accepts_persistent_edge_report(tmp_path) -> None:
    engine = _engine(tmp_path)
    guards, summary = engine._anti_overfit_guards(_records_from_matrix(_robust_matrix()))
    assert summary["nested"]["passed"] is True
    assert guards[0]["nested_passed"] is True

def test_engine_nested_disabled_fails_closed(tmp_path) -> None:
    engine = _engine(tmp_path, self_improvement_nested_enabled=False)
    report = engine._nested_report(_records_from_matrix(_robust_matrix()))
    assert report["blocked"] is True
    assert report["passed"] is False
    assert report["reason"] == "nested_optimization_disabled_fail_closed"

def test_lab_gate_requires_all_three_guards() -> None:
    engine = SelfImprovementEngine(Settings())
    metrics = {
        "total_trades": 50,
        "profit_factor": 2.5,
        "expectancy_r": 0.4,
        "max_drawdown_pct": 5.0,
    }
    full = {"pbo_passed": True, "plateau_passed": True, "nested_passed": True}
    assert engine._passes_lab_gate(metrics, full) is True
    for key in full:
        broken = dict(full)
        broken[key] = False
        assert engine._passes_lab_gate(metrics, broken) is False

```

Full Source: `backend/tradeo/tests/test\_director\_review\_gate.py`

`backend/tradeo/tests/test\_director\_review\_gate.py` ? 661 lines, 24140 bytes, sha256 prefix `9b281f57989f371d`

Public surface summary before full source:

```

? function `session_factory` line 22`
? function `add_pattern` line 28`
? function `add_closed_lab_trade` line 46`
? function `production_contract_metrics` line 119`
? function `test_director_review_gate_waits_for_ten_closed_lab_trades` line 149`
? function `test_director_production_gate_blocks_missing_nested_replay_contract` line 169`
? function `test_director_review_gate_marks_candidate_after_ten_closed_lab_trades` line 192`
? function `test_director_review_gate_blocks_until_effective_lab_trade_minimum` line 214`
? function `test_director_review_gate_blocks_low_diversity_lab_results` line 232`
? function `test_director_review_gate_ignores_shadow_and_near_miss_no_order_observations` line 247`
? function `test_director_review_gate_ignores_degraded_fallback_evidence` line 292`
? function `test_director_review_gate_does_not_promote_closed_paper_order_without_real_fill` line 322`
? function `test_director_review_gate_counts_broker_execution_fill_with_hash` line 350`
? function `test_director_review_gate_rejects_shadow_close_provenance` line 359`
? function `test_director_review_gate_explains_missing_bucket_data_when_no_trades` line 380`
? function `test_director_review_gate_ranks_entry_variant_and_regime_buckets` line 397`

```

Risk hints: IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from datetime import datetime, timedelta, timezone

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from tradeo.db.models import (
    DiscoveredPattern,
    DiscoveredPatternStatus,
    Signal,
    SignalStatus,
    Trade,
    TradeStatus,
)
from tradeo.db.session import Base
from tradeo.services.director_review_gate import DirectorProductionGate, DirectorReviewGate
from tradeo.services.evidence import EvidenceQuality, EvidenceType, FillProvenance
from tradeo.services.pattern_health_monitor import PatternHealthMonitor

def session_factory():
    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)
    return sessionmaker(bind=engine, future=True)()

def add_pattern(db) -> DiscoveredPattern:
    pattern = DiscoveredPattern(
        pattern_key="LAB_PATTERN_1",
        name="LAB_PATTERN_1",
        status=DiscoveredPatternStatus.LAB_CANDIDATE,
        promotion_status="lab_candidate",

```

```

        validation_passed=True,
        expectancy_r=0.25,
        profit_factor=1.8,
        best_expectancy_r=0.4,
        best_profit_factor=2.0,
    )
    db.add(pattern)
    db.commit()
    db.refresh(pattern)
    return pattern

def add_closed_lab_trade(
    db,
    pattern: DiscoveredPattern,
    index: int,
    r_multiple: float = 1.0,
    *,
    symbol: str | None = None,
    entry_variant_id: str | None = None,
    regime_key: str | None = None,
    signal_metadata: dict | None = None,
    trade_metadata: dict | None = None,
) -> None:
    trade_time = datetime(2026, 1, 5, 16, 0, tzinfo=timezone.utc) + timedelta(days=index)
    metadata = {
        "entry_module": "laboratory",
        "pattern_id": pattern.id,
        "entry_variant_id": entry_variant_id,
        "regime": {"regime_key": regime_key} if regime_key else {},
    }
    metadata.update(signal_metadata or {})
    signal = Signal(
        symbol=symbol or f"T{index}",
        pattern=pattern.name,
        side="long",
        entry=10.0,
        stop=9.0,
        target=14.0,
        reward_risk=4.0,
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=1,
        strategy_version=f"laboratory_pattern_{pattern.id}",
        status=SignalStatus.EXECUTED,
        human_approved=True,
        metadata_json=metadata,
    )
    db.add(signal)
    db.flush()
    trade_meta = {
        "execution_mode": "ibkr",
        "ibkr_mode": "paper",
        "evidence_type": EvidenceType.IBKR_PAPER_FILL.value,
        "evidence_quality": EvidenceQuality.NORMAL.value,
        "fill_provenance": FillProvenance.BROKER_EXECUTION.value,
        "broker_execution_hash": f"broker-execution-hash-{index}",
        "broker_execution_time": trade_time.isoformat(),
        "commission": 0.0,
    }
    trade_meta.update(trade_metadata or {})
    db.add(
        Trade(
            signal_id=signal.id,
            symbol=signal.symbol,
            pattern=pattern.name,
            side="long",
            qty=1,
            entry=10.0,
            stop=9.0,
            target=14.0,
            status=TradeStatus.CLOSED,
            opened_at=trade_time,
            closed_at=trade_time + timedelta(minutes=30),
            pnl_usd=10.0 * r_multiple,
            r_multiple=r_multiple,
            evidence_type=trade_meta.get("evidence_type"),
            evidence_quality=trade_meta.get("evidence_quality"),
            metadata_json=trade_meta,
        )
    )
    db.commit()

def production_contract_metrics(*, nested: bool = True) -> dict:
    metrics = {
        "director_gate_status": "passed",
        "director_gate": {"status": "passed", "blockers": []},
        "event_ledger_sha256": "event-ledger-hash",
        "production_evidence_packet": {"id": "packet-1", "hash": "packet-hash"},
        "execution_provenance": {
            "costs_reconciled": True,
            "slippage_reconciled": True,
            "fills_reconciled": True,
        },
    },

```

```

        "edge_claim": "NO_DEMOSTRADO",
        "global_experiment_registry": {
            "path": "reports/research/global_experiment_registry.json",
            "registry_hash": "registry-hash",
            "previous_registry_hash": "previous-registry-hash",
            "run_manifest_hash": "run-manifest-hash",
            "hash_chain_valid": True,
        },
    },
    if nested:
        metrics["nested_discovery_replay"] = {
            "status": "passed",
            "implemented": True,
            "passed": True,
            "blocking": False,
        }
    return metrics

def test_director_review_gate_waits_for_ten_closed_lab_trades() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(9):
        add_closed_lab_trade(db, pattern, index)

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)

    assert result["marked_for_director_review"] == 0
    assert pattern.status == DiscoveredPatternStatus.LAB_CANDIDATE
    assert pattern.metrics_json["lab_execution"]["closed_lab_trades"] == 9
    assert pattern.metrics_json["lab_execution"]["trades_remaining_for_director_review"] == 1
    assert pattern.metrics_json["lab_execution"]["eligible_for_director_review"] is False
    assert "closed_lab_trades_below_10" in pattern.metrics_json["lab_execution"]["promotion_blockers"]
    assert pattern.metrics_json["lab_execution"]["by_regime"]
    assert pattern.metrics_json["lab_execution"]["by_regime_empty_reason"] == ""
    assert pattern.metrics_json["lab_execution"]["director_recommendations"][0]["action"] == "collect_closed_lab_trades"

def test_director_production_gate_blocks_missing_nested_replay_contract() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    pattern.metrics_json = production_contract_metrics(nested=False)
    db.add(pattern)
    db.commit()
    for index in range(3):
        add_closed_lab_trade(db, pattern, index)

    closed_trades = db.query(Trade).filter(Trade.status == TradeStatus.CLOSED).all()
    result = DirectorProductionGate(
        min_paper_fills=3,
        min_fill_symbols=1,
        min_fill_trading_days=1,
        min_expectancy_r=0.0,
        min_profit_factor=0.1,
    ).evaluate_pattern(pattern, closed_trades)

    assert result["approved_for_production"] is False
    assert "nested_discovery_replay_missing" in result["blockers"]
    assert result["scientific_contracts"]["director_gate_passed"] is True

def test_director_review_gate_marks_candidate_after_ten_closed_lab_trades() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(10):
        add_closed_lab_trade(db, pattern, index, r_multiple=1.0 if index < 7 else -1.0)

    result = DirectorReviewGate(min_closed_lab_trades=10, min_effective_lab_trades=10).refresh(db)
    db.refresh(pattern)

    assert result["marked_for_director_review"] == 1
    assert pattern.status == DiscoveredPatternStatus.DIRECTOR_REVIEW
    assert pattern.promotion_status == "director_review"
    assert "not production approval" in pattern.promotion_reason
    assert pattern.metrics_json["lab_execution"]["eligible_for_director_review"] is True
    assert pattern.metrics_json["lab_execution"]["lab_expectancy_r"] == 0.4
    assert pattern.metrics_json["lab_execution"]["research_expectancy_r"] == 0.4
    assert pattern.metrics_json["lab_execution"]["unique_lab_symbols"] == 10
    assert pattern.metrics_json["lab_execution"]["unique_lab_days"] == 10
    assert pattern.metrics_json["lab_execution"]["trades_remaining_for_director_review"] == 0
    assert pattern.metrics_json["lab_execution"]["director_review_trigger_trades"] == 10

def test_director_review_gate_blocks_until_effective_lab_trade_minimum() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(10):
        add_closed_lab_trade(db, pattern, index, r_multiple=1.0)

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert result["marked_for_director_review"] == 0
    assert pattern.status == DiscoveredPatternStatus.LAB_CANDIDATE

```

```

assert lab_execution["closed_lab_trades"] == 10
assert lab_execution["effective_lab_trades"] == 10
assert lab_execution["min_effective_lab_trades"] == 25
assert "effective_lab_trades_below_25" in lab_execution["promotion_blockers"]

def test_director_review_gate_blocks_low_diversity_lab_results() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(10):
        add_closed_lab_trade(db, pattern, index, symbol="SAME")

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)

    assert result["marked_for_director_review"] == 0
    assert pattern.status == DiscoveredPatternStatus.LAB_CANDIDATE
    assert pattern.metrics_json["lab_execution"]["eligible_for_director_review"] is False
    assert "unique_lab_symbols_below_3" in pattern.metrics_json["lab_execution"]["promotion_blockers"]

def test_director_review_gate_ignores_shadow_and_near_miss_no_order_observations() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(10):
        near_miss = index % 2 == 0
        add_closed_lab_trade(
            db,
            pattern,
            index,
            trade_metadata={
                "execution_mode": "lab_shadow_observation",
                "evidence_type": (
                    EvidenceType.NEAR_MISS_SHADOW.value
                    if near_miss
                    else EvidenceType.SHADOW_NO_ORDER.value
                ),
                "evidence_quality": EvidenceQuality.NORMAL.value,
                "observation_only": True,
                "no_ibkr_order": True,
                "near_miss": near_miss,
                "near_miss_shadow": near_miss,
            },
            signal_metadata={
                "paper_only": True,
                "no_ibkr_order": True,
                "near_miss": near_miss,
                "near_miss_shadow": near_miss,
            },
        )

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert result["marked_for_director_review"] == 0
    assert pattern.status == DiscoveredPatternStatus.LAB_CANDIDATE
    assert lab_execution["closed_lab_trades"] == 0
    assert lab_execution["excluded_lab_evidence_trades"] == 10
    assert lab_execution["excluded_evidence_type_counts"] == {
        EvidenceType.NEAR_MISS_SHADOW.value: 5,
        EvidenceType.SHADOW_NO_ORDER.value: 5,
    }
    assert "closed_lab_trades_below_10" in lab_execution["promotion_blockers"]

def test_director_review_gate_ignores_degraded_fallback_evidence() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(9):
        add_closed_lab_trade(db, pattern, index)
    add_closed_lab_trade(
        db,
        pattern,
        9,
        trade_metadata={
            "evidence_type": EvidenceType.IBKR_PAPER_FILL.value,
            "evidence_quality": EvidenceQuality.DEGRADED.value,
            "fallback_used": True,
            "fallback_source": "signal.metadata_json.signal_snapshot.features.last_close",
        },
    )

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert result["marked_for_director_review"] == 0
    assert lab_execution["closed_lab_trades"] == 9
    assert lab_execution["excluded_lab_evidence_trades"] == 1
    assert lab_execution["excluded_evidence_quality_counts"] == {
        EvidenceQuality.DEGRADED.value: 1,
    }
    assert lab_execution["trades_remaining_for_director_review"] == 1

```



```

def test_director_review_gate_does_not_promote_closed_paper_order_without_real_fill() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(9):
        add_closed_lab_trade(db, pattern, index)
    add_closed_lab_trade(
        db,
        pattern,
        9,
        trade_metadata={
            "evidence_type": EvidenceType.IBKR_PAPER_ORDER.value,
            "evidence_quality": EvidenceQuality.NORMAL.value,
            "fill_provenance": FillProvenance.SIMULATED_CLOSE.value,
        },
    )

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert result["marked_for_director_review"] == 0
    assert lab_execution["closed_lab_trades"] == 9
    assert lab_execution["excluded_lab_evidence_trades"] == 1
    assert lab_execution["excluded_evidence_type_counts"] == {
        EvidenceType.IBKR_PAPER_ORDER.value: 1,
    }

def test_director_review_gate_counts_broker_execution_fill_with_hash() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    add_closed_lab_trade(db, pattern, 0)
    trade = db.query(Trade).one()

    assert DirectorReviewGate._counts_as_paper_fill(trade) is True

def test_director_review_gate_rejects_shadow_close_provenance() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    add_closed_lab_trade(
        db,
        pattern,
        0,
        trade_metadata={
            "evidence_type": EvidenceType.IBKR_PAPER_FILL.value,
            "evidence_quality": EvidenceQuality.NORMAL.value,
            "fill_provenance": FillProvenance.SHADOW_CLOSE.value,
            "broker_execution_hash": "shadow-hash",
            "broker_execution_time": "2026-01-05T16:30:00+00:00",
            "commission": 0.0,
        },
    )
    trade = db.query(Trade).one()

    assert DirectorReviewGate._counts_as_paper_fill(trade) is False

def test_director_review_gate_explains_missing_bucket_data_when_no_trades() -> None:
    db = session_factory()
    pattern = add_pattern(db)

    DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert lab_execution["closed_lab_trades"] == 0
    assert lab_execution["by_entry_variant"] == {}
    assert lab_execution["by_regime"] == {}
    assert "no_closed_lab_trades" in lab_execution["by_entry_variant_empty_reason"]
    assert "no_closed_lab_trades" in lab_execution["by_regime_empty_reason"]
    assert lab_execution["director_recommendations"][0]["trades_remaining"] == 10
    assert lab_execution["director_recommendations"][1]["action"] == "keep_research_only"

def test_director_review_gate_ranks_entry_variant_and_regime_buckets() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(5):
        add_closed_lab_trade(
            db,
            pattern,
            index,
            r_multiple=1.2,
            entry_variant_id="next_bar_limit_retest",
            regime_key="market_up",
        )
    for index in range(5, 10):
        add_closed_lab_trade(
            db,
            pattern,
            index,
            r_multiple=-0.2,
            entry_variant_id="momentum_close",
            regime_key="market_down",
        )

```

```

DirectorReviewGate(
    min_closed_lab_trades=10,
    min_lab_symbols=1,
    min_lab_trading_days=1,
    min_baseline_edge_r=0.0,
    min_lab_profit_factor=0.1,
).refresh(db)
db.refresh(pattern)
lab_execution = pattern.metrics_json["lab_execution"]

assert lab_execution["best_entry_variant"]["key"] == "next_bar_limit_retest"
assert lab_execution["worst_entry_variant"]["key"] == "momentum_close"
assert lab_execution["best_regime"]["key"] == "market_up"
assert lab_execution["worst_regime"]["key"] == "market_down"
assert any(
    recommendation["action"] == "prioritize_entry_variant"
    for recommendation in lab_execution["director_recommendations"]
)
assert any(
    recommendation["action"] == "gate_by_regime_until_confirmed"
    for recommendation in lab_execution["director_recommendations"]
)

def test_director_review_gate_excludes_shadow_near_miss_and_degraded_fallback() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(9):
        add_closed_lab_trade(db, pattern, index, r_multiple=1.0)

    trade_time = datetime(2026, 2, 1, 16, 0, tzinfo=timezone.utc)
    signal = Signal(
        symbol="SHDW",
        pattern=pattern.name,
        side="long",
        entry=10.0,
        stop=9.0,
        target=14.0,
        reward_risk=4.0,
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=1,
        strategy_version=f"laboratory_pattern_{pattern.id}",
        status=SignalStatus.EXECUTED,
        human_approved=False,
        metadata_json={
            "entry_module": "laboratory",
            "pattern_id": pattern.id,
            "evidence_type": EvidenceType.NEAR_MISS_SHADOW.value,
            "near_miss": True,
            "no_ibkr_order": True,
            "observation_only": True,
        },
    )
    db.add(signal)
    db.flush()
    db.add(
        Trade(
            signal_id=signal.id,
            symbol="SHDW",
            pattern=pattern.name,
            side="long",
            qty=1,
            entry=10.0,
            stop=9.0,
            target=14.0,
            status=TradeStatus.CLOSED,
            opened_at=trade_time,
            closed_at=trade_time + timedelta(minutes=30),
            pnl_usd=100.0,
            r_multiple=10.0,
            metadata_json={
                "evidence_type": EvidenceType.NEAR_MISS_SHADOW.value,
                "evidence_quality": EvidenceQuality.NORMAL.value,
                "no_ibkr_order": True,
                "observation_only": True,
                "near_miss": True,
            },
        )
    )
    db.commit()

    result = DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)

    assert result["marked_for_director_review"] == 0
    assert pattern.metrics_json["lab_execution"]["paper_fill_trades"] == 9
    assert pattern.metrics_json["lab_execution"]["closed_lab_trades"] == 9
    assert "closed_lab_trades_below_10" in pattern.metrics_json["lab_execution"]["promotion_blockers"]
    assert pattern.status == DiscoveredPatternStatus.LAB_CANDIDATE

def add_research_skip_accounting(db, pattern: DiscoveredPattern, skip_rate: float = 0.4) -> None:
    skipped = int(round(50 * skip_rate))

```

```

pattern.best_rr = 2.0
pattern.rr_metrics_json = {
    "2": {
        "rr": 2.0,
        "expectancy_r": 0.5,
        "profit_factor": 2.2,
        "sample_count": 50 - skipped,
        "signal_count": 50,
        "skipped_count": skipped,
        "skip_rate": skip_rate,
        "skip_reason_counts": {"gap_entry_policy": skipped},
    }
}
db.add(pattern)
db.commit()

def test_director_review_gate_surfaces_high_research_skip_rate_evidence() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    add_research_skip_accounting(db, pattern, skip_rate=0.4)
    for index in range(10):
        add_closed_lab_trade(db, pattern, index, r_multiple=1.0 if index < 7 else -1.0)

    result = DirectorReviewGate(min_closed_lab_trades=10, min_effective_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]
    skip_accounting = lab_execution["research_skip_accounting"]

    # Good expectancy + high skip_rate stays visible but never blocks the gate.
    assert result["marked_for_director_review"] == 1
    assert lab_execution["promotion_blockers"] == []
    assert skip_accounting["available"] is True
    assert skip_accounting["source"] == "pattern.rr_metrics_json[2]"
    assert skip_accounting["skip_rate"] == 0.4
    assert skip_accounting["signal_count"] == 50
    assert skip_accounting["skipped_count"] == 20
    assert skip_accounting["skip_reason_counts"] == {"gap_entry_policy": 20}
    assert lab_execution["research_skip_rate_warning"] is True
    assert lab_execution["skip_rate_warning_threshold"] == 0.25
    warning = next(
        recommendation
        for recommendation in lab_execution["director_recommendations"]
        if recommendation["action"] == "review_research_skip_rate"
    )
    assert warning["priority"] == "high"
    assert warning["skip_rate"] == 0.4
    assert warning["skip_reason_counts"] == {"gap_entry_policy": 20}

def test_director_review_gate_reports_missing_skip_accounting_without_inventing_data() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(9):
        add_closed_lab_trade(db, pattern, index)

    DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]
    skip_accounting = lab_execution["research_skip_accounting"]

    assert skip_accounting["available"] is False
    assert "no skip accounting" in skip_accounting["reason"]
    assert "skip_rate" not in skip_accounting
    assert lab_execution["research_skip_rate_warning"] is False
    assert all(
        recommendation["action"] != "review_research_skip_rate"
        for recommendation in lab_execution["director_recommendations"]
    )

def test_director_review_gate_reads_backtest_shape_skip_accounting() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    pattern.metrics_json = {
        "backtest": {"total_signals": 50, "skipped_signals": 10, "skip_rate": 0.2}
    }
    db.add(pattern)
    db.commit()
    for index in range(9):
        add_closed_lab_trade(db, pattern, index)

    DirectorReviewGate(min_closed_lab_trades=10).refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]
    skip_accounting = lab_execution["research_skip_accounting"]

    assert skip_accounting["available"] is True
    assert skip_accounting["source"] == "pattern.metrics_json.backtest"
    assert skip_accounting["signal_count"] == 50
    assert skip_accounting["skipped_count"] == 10
    assert skip_accounting["skip_rate"] == 0.2
    # Below the warning threshold: evidence is visible, no warning raised.
    assert lab_execution["research_skip_rate_warning"] is False

```

```

def test_director_production_gate_report_includes_research_skip_accounting() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    add_research_skip_accounting(db, pattern, skip_rate=0.4)
    for index in range(3):
        add_closed_lab_trade(db, pattern, index)

    closed_trades = db.query(Trade).filter(Trade.status == TradeStatus.CLOSED).all()
    result = DirectorProductionGate(
        min_paper_fills=3,
        min_fill_symbols=1,
        min_fill_trading_days=1,
        min_expectancy_r=0.0,
        min_profit_factor=0.1,
    ).evaluate_pattern(pattern, closed_trades)
    skip_accounting = result["research_skip_accounting"]

    assert skip_accounting["available"] is True
    assert skip_accounting["skip_rate"] == 0.4
    assert skip_accounting["skip_reason_counts"] == {"gap_entry_policy": 20}
    # Skip accounting is evidence only: it must never appear as a blocker.
    assert all("skip" not in blocker for blocker in result["blockers"])

def test_pattern_health_monitor_marks_decaying_on_shortfall_cusum() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    pattern.status = DiscoveredPatternStatus.DIRECTOR_REVIEW
    db.add(pattern)
    db.commit()
    for index in range(8):
        add_closed_lab_trade(
            db,
            pattern,
            index,
            r_multiple=0.5,
            trade_metadata={
                "entry_fill_price": 10.35,
                "exit_fill_price": 14.0,
                "exit_reason": "target_hit",
            },
        )

    result = PatternHealthMonitor().run(db)
    db.refresh(pattern)

    assert result["decay_detected"] == 1
    assert pattern.drift_status == "decaying"
    health = pattern.metrics_json["pattern_health"]
    assert health["shortfall_fills"] == 8
    assert health["shortfall_cusum"]["available"] is True
    assert health["shortfall_cusum"]["triggered"] is True

```

Full Source: `backend/tradeo/tests/test\_discovery\_determinism.py`

`backend/tradeo/tests/test\_discovery\_determinism.py` ? 155 lines, 6240 bytes, sha256 prefix `cefd962b6c19417b`

Public surface summary before full source:

```

? function `test_canonical_payload_is_order_and_type_insensitive` line 91`
? function `test_content_hash_excludes_volatile_keys` line 97`
? function `test_canonical_json_maps_non_finite_to_null` line 109`
? function `test_discovery_is_bit_for_bit_deterministic_across_runs` line 114`
? function `test_discovery_is_independent_of_input_sample_order` line 125`
? function `test_discovery_report_content_hash_is_stable_across_run_ids` line 134`

```

Risk hints: RNG path; verify seed/control.

"""Bit-for-bit determinism contract for the discovery/research path.

Runs the real ClusterResearchEngine twice over identical synthetic fixtures and asserts the full candidate payload (metrics included) is byte-identical under canonical JSON. Scope is honest: the contract covers the engine core (clustering, validation statistics, scoring) and the discovery report artifact identity; it does not cover live market data fetches or DB state.

```

from __future__ import annotations

import json
from datetime import date, timedelta
from pathlib import Path

import numpy as np

from tradeo.agents.pattern_discovery_lab_agent import PatternDiscoveryLabAgent
from tradeo.core.config import Settings
from tradeo.research.cluster_research_engine import ClusterResearchEngine
from tradeo.research.determinism import (
    DEFAULT_VOLATILE_KEYS,
    canonical_json,

```

```

        candidate_content_payload,
        content_hash,
        discovery_content_hash,
    )
from tradeo.research.types import ForwardOutcome, WindowSample

def _sample(index: int, *, vector: np.ndarray, close: float) -> WindowSample:
    entry = 100.0
    risk = 10.0
    end = date(2024, 1, 1) + timedelta(days=index)
    highs = [max(close, entry) + 1.0]
    lows = [min(close, entry) - 1.0]
    return WindowSample(
        symbol=f"DET{index % 7}",
        timeframe="1d",
        window_size=20,
        start=(end - timedelta(days=20)).isoformat(),
        end=end.isoformat(),
        year=end.year,
        vector=vector.astype(np.float32),
        chart={},
        features={},
        outcome=ForwardOutcome(
            forward_returns={5: close / entry - 1.0},
            entry_price=entry,
            risk_proxy=risk,
            forward_end=(end + timedelta(days=1)).isoformat(),
            long_mfe_r=max((max(highs) - entry) / risk, 0.0),
            long_mae_r=max((entry - min(lows)) / risk, 0.0),
            long_outcome_r=(close - entry) / risk,
            long_hit_4r=False,
            short_mfe_r=max((entry - min(lows)) / risk, 0.0),
            short_mae_r=max((max(highs) - entry) / risk, 0.0),
            short_outcome_r=(entry - close) / risk,
            short_hit_4r=False,
            forward_highs=highs,
            forward_lows=lows,
            forward_closes=[close],
        ),
    )

def _make_samples(seed: int = 7, count: int = 48) -> list[WindowSample]:
    rng = np.random.default_rng(seed)
    samples: list[WindowSample] = []
    for i in range(count):
        blob = i % 2
        base = 0.0 if blob == 0 else 5.0
        vector = base + rng.normal(0.0, 0.05, 2)
        drift = 1.0 if blob == 0 else -1.0
        close = 100.0 + float(rng.normal(drift, 0.5))
        samples.append(_sample(i, vector=vector, close=close))
    return samples

def _engine() -> ClusterResearchEngine:
    return ClusterResearchEngine(
        min_cluster_size=5,
        max_clusters_per_window=2,
        out_of_sample_pct=0.2,
        rr_levels=[2.0],
        min_samples=1,
        quant_bootstrap_draws=100,
    )

def test_canonical_payload_is_order_and_type_insensitive() -> None:
    a = {"b": 1, "a": np.float64(2.5), "tags": {"y", "x"}, "vec": np.asarray([1.0, 2.0])}
    b = {"vec": [1.0, 2.0], "tags": ["x", "y"], "a": 2.5, "b": 1}
    assert content_hash(a) == content_hash(b)

def test_content_hash_excludes_volatile_keys() -> None:
    base = {"score": 1.25, "nested": {"win_rate": 0.5}}
    noisy = {
        **base,
        "generated_at": "2026-06-11T00:00:00Z",
        "run_id": 99,
        "nested": {"win_rate": 0.5, "built_at": "2026-06-11", "path": "/tmp/x.json"},
    }
    assert content_hash(base) == content_hash(noisy)
    assert content_hash(base) != content_hash(**base, "score": 1.26)

def test_canonical_json_maps_non_finite_to_null() -> None:
    payload = canonical_json({"a": float("nan"), "b": float("inf")})
    assert json.loads(payload) == {"a": None, "b": None}

def test_discovery_is_bit_for_bit_deterministic_across_runs() -> None:
    first = _engine().discover(_make_samples())
    second = _engine().discover(_make_samples())
    assert first, "fixture must produce candidates for the contract to be meaningful"
    assert len(first) == len(second)
    first_json = canonical_json([candidate_content_payload(c) for c in first])

```

```

second_json = canonical_json([candidate_content_payload(c) for c in second])
assert first_json == second_json
assert discovery_content_hash(first) == discovery_content_hash(second)

def test_discovery_is_independent_of_input_sample_order() -> None:
    baseline = _engine().discover(_make_samples())
    shuffled_samples = _make_samples()
    np.random.default_rng(123).shuffle(shuffled_samples) # type: ignore[arg-type]
    shuffled = _engine().discover(shuffled_samples)
    assert baseline
    assert discovery_content_hash(baseline) == discovery_content_hash(shuffled)

def test_discovery_report_content_hash_is_stable_across_run_ids(tmp_path: Path) -> None:
    agent = PatternDiscoveryLabAgent(provider=object(), settings=Settings(reports_dir=str(tmp_path)))
    candidates = _engine().discover(_make_samples())
    assert candidates
    params = {"timeframe": "1d", "window_sizes": [20], "seed": 42}
    summary = {
        "windows_sampled": 48,
        "clusters_evaluated": len(candidates),
        "accepted_patterns": len(candidates),
        "rejected_patterns": 0,
    }
    first_path = agent._write_report(1, params, dict(summary), candidates)
    second_path = agent._write_report(2, params, dict(summary), candidates)
    assert first_path is not None and second_path is not None
    first_payload = json.loads(first_path.read_text(encoding="utf-8"))
    second_payload = json.loads(second_path.read_text(encoding="utf-8"))
    assert first_payload["determinism"]["algo"] == "sha256_canonical_json_v1"
    assert first_payload["determinism"]["content_hash"] == second_payload["determinism"]["content_hash"]
    assert first_payload["determinism"]["excluded_keys"] == sorted(DEFAULT_VOLATILE_KEYS)
    stripped_first = canonical_json({k: v for k, v in first_payload.items() if k != "determinism"})
    stripped_second = canonical_json({k: v for k, v in second_payload.items() if k != "determinism"})
    assert stripped_first == stripped_second

```

Full Source: ``backend/tradeo/tests/test_rediscovery_readiness.py``

``backend/tradeo/tests/test_rediscovery_readiness.py`` ? 158 lines, 5258 bytes, sha256 prefix ``9737f7fc45ee249e``

Public surface summary before full source:

```

? function `session_factory` line 21`
? function `modern_metrics` line 27`
? function `add_pattern` line 44`
? function `test_legacy_pattern_without_fields_is_flagged` line 52`
? function `test_modern_pattern_with_all_fields_is_ok` line 66`
? function `test_outdated_embedding_contract_is_flagged_as_stale` line 78`
? function `test_dry_run_manifest_is_honest_and_mutates_nothing` line 91`
? function `test_apply_flags_marks_legacy_but_does_not_fake_metadata` line 111`
? function `test_flag_cleared_after_real_metadata_arrives` line 128`
? function `test_manifest_written_to_disk_and_truncation_reported` line 147`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import json

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from tradeo.db.models import DiscoveredPattern
from tradeo.db.session import Base
from tradeo.research.pattern_embedding_engine import PatternEmbeddingEngine
from tradeo.research.rediscovery_readiness import (
    CLUSTER_SIGNATURE,
    EMBEDDING_CONTRACT,
    READINESS_KEY,
    REGIME_CALIBRATION,
    audit_patterns,
    run_readiness,
)

def session_factory():
    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)
    return sessionmaker(bind=engine, future=True)()

def modern_metrics() -> dict:
    return {
        "feature_parity_contract": {
            "contract_id": PatternEmbeddingEngine.CONTRACT_ID,
            "vector_length": 64,
        },
        "cluster_signature": {
            "medoid": {"symbol": "AAPL", "timeframe": "1d"},

```

```

        "concentration_checks": {"passed": True, "max_symbol_share": 0.2},
    },
    "regime_profile": {
        "dominant_regime": "benchmark_bull|low_vol_tercile",
        "benchmark_regime_outcomes": {"available": True, "buckets": {}},
    },
}

def add_pattern(db, key: str, metrics: dict) -> DiscoveredPattern:
    pattern = DiscoveredPattern(pattern_key=key, name=key.upper(), metrics_json=metrics)
    db.add(pattern)
    db.commit()
    db.refresh(pattern)
    return pattern

def test_legacy_pattern_without_fields_is_flagged():
    db = session_factory()
    add_pattern(db, "legacy_cup", {"expectancy_r": 0.4, "win_rate": 0.55})

    results, truncated = audit_patterns(db)

    assert not truncated
    assert len(results) == 1
    result = results[0]
    assert result.needs_rediscovery
    assert result.missing == [EMBEDDING_CONTRACT, CLUSTER_SIGNATURE, REGIME_CALIBRATION]
    assert result.stale == []

def test_modern_pattern_with_all_fields_is_ok():
    db = session_factory()
    add_pattern(db, "modern_flag", modern_metrics())

    results, _ = audit_patterns(db)

    assert len(results) == 1
    assert not results[0].needs_rediscovery
    assert results[0].missing == []
    assert results[0].stale == []

def test_outdated_embedding_contract_is_flagged_as_stale():
    db = session_factory()
    metrics = modern_metrics()
    metrics["feature_parity_contract"]["contract_id"] = "tradeo.pattern_embedding.v1"
    add_pattern(db, "stale_contract", metrics)

    results, _ = audit_patterns(db)

    assert results[0].needs_rediscovery
    assert results[0].stale == [EMBEDDING_CONTRACT]
    assert results[0].missing == []

def test_dry_run_manifest_is_honest_and_mutates_nothing():
    db = session_factory()
    legacy = add_pattern(db, "legacy_cup", {"expectancy_r": 0.4})
    add_pattern(db, "modern_flag", modern_metrics())

    manifest = run_readiness(db, apply_flags=False)

    counts = manifest["counts"]
    assert manifest["dry_run"] is True
    assert counts["patterns_audited"] == 2
    assert counts["needs_rediscovery"] == 1
    assert counts["flagged_this_run"] == 0
    # Honest accounting: a dry-run (and even flagging) populates nothing.
    assert counts["metadata_complete_after"] == counts["metadata_complete_before"] == 1
    assert counts["missing_by_field"][EMBEDDING_CONTRACT] == 1
    db.refresh(legacy)
    assert READINESS_KEY not in (legacy.metrics_json or {})
    assert manifest["determinism"]["content_hash"]

def test_apply_flags_marks_legacy_but_does_not_fake_metadata():
    db = session_factory()
    legacy = add_pattern(db, "legacy_cup", {"expectancy_r": 0.4})

    manifest = run_readiness(db, apply_flags=True)

    assert manifest["counts"]["flagged_this_run"] == 1
    db.refresh(legacy)
    block = legacy.metrics_json[READINESS_KEY]
    assert block["needs_rediscovery"] is True
    assert block["missing"] == [EMBEDDING_CONTRACT, CLUSTER_SIGNATURE, REGIME_CALIBRATION]
    # Flagging must not make the pattern look populated on re-audit.
    results, _ = audit_patterns(db)
    assert results[0].needs_rediscovery
    assert manifest["counts"]["metadata_complete_after"] == 0

def test_flag_cleared_after_real_metadata_arrives():
    db = session_factory()
    legacy = add_pattern(db, "legacy_cup", {"expectancy_r": 0.4})

```

```

run_readiness(db, apply_flags=True)

# Simulate a real rediscovery upsert refreshing metrics_json.
db.refresh(legacy)
refreshed = {**legacy.metrics_json, **modern_metrics{}}
legacy.metrics_json = refreshed
db.commit()

manifest = run_readiness(db, apply_flags=True)

assert manifest["counts"]["needs_rediscovery"] == 0
assert manifest["counts"]["flagged_this_run"] == 0
db.refresh(legacy)
assert legacy.metrics_json[READINESS_KEY]["needs_rediscovery"] is False

def test_manifest_written_to_disk_and_truncation_reported(tmp_path):
    db = session_factory()
    for idx in range(3):
        add_pattern(db, f"legacy_{idx}", {})
    out = tmp_path / "manifest.json"

    manifest = run_readiness(db, apply_flags=False, limit=2, manifest_path=out)

    assert manifest["truncated"] is True
    assert manifest["counts"]["patterns_audited"] == 2
    on_disk = json.loads(out.read_text(encoding="utf-8"))
    assert on_disk["counts"] == manifest["counts"]

```

Full Source: `backend/tradeo/tests/test\_reward\_risk\_analyzer.py`

`backend/tradeo/tests/test\_reward\_risk\_analyzer.py` ? 142 lines, 5537 bytes, sha256 prefix `77378f96c5325e2a`

Public surface summary before full source:

```

? function `test_target_and_stop_same_bar_counts_stop_first` line 51`
? function `test_expectancy_profit_factor_and_drawdown` line 59`
? function `test_best_rr_uses_edge_score_not_highest_rr` line 71`
? function `test_simulation_subtracts_execution_cost_r` line 82`
? function `test_cost_multiplier_stresses_result_r` line 97`
? function `test_canonical_outcome_skips_entry_gap_past_target` line 103`
? function `test_skipped_signals_do_not_dilute_traded_aggregates` line 112`
? function `test_all_skipped_signals_yield_zero_sample_count` line 133`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import numpy as np

from tradeo.research.reward_risk_analyzer import RewardRiskAnalyzer
from tradeo.research.types import ForwardOutcome, WindowSample

def _sample(
    highs: list[float],
    lows: list[float],
    closes: list[float],
    cost_r: float = 0.0,
    opens: list[float] | None = None,
) -> WindowSample:
    return WindowSample(
        symbol="TST",
        timeframe="1d",
        window_size=20,
        start="2024-01-01",
        end="2024-01-20",
        year=2024,
        vector=np.zeros(64),
        chart={},
        features={},
        outcome=ForwardOutcome(
            forward_returns={5: 0.0},
            entry_price=100.0,
            risk_proxy=10.0,
            forward_end="2024-01-25",
            long_mfe_r=max(highs) / 100.0,
            long_mae_r=1.0,
            long_outcome_r=0.0,
            long_hit_4r=False,
            short_mfe_r=0.0,
            short_mae_r=0.0,
            short_outcome_r=0.0,
            short_hit_4r=False,
            forward_highs=highs,
            forward_lows=lows,
            forward_closes=closes,
            forward_opens=opens or [100.0 for _ in closes],
            execution_cost_r=cost_r,
            long_gap_adverse_r=0.2,
            long_mfe_before_mae=True,

```



```

        execution={"fill_probability": 0.8, "max_size_usd": 12_500.0, "spread_proxy_pct": 0.001},
    ),
)

def test_target_and_stop_same_bar_counts_stop_first() -> None:
    sample = _sample([130.0], [90.0], [120.0])
    result, target_bar, stop_bar = RewardRiskAnalyzer._simulate_sample(sample, "long", 3.0)
    assert result == -1.0
    assert target_bar is None
    assert stop_bar == 1

def test_expectancy_profit_factor_and_drawdown() -> None:
    samples = [
        _sample([130.0], [99.0], [130.0]),
        _sample([130.0], [99.0], [130.0]),
        _sample([104.0], [90.0], [90.0]),
    ]
    metrics = RewardRiskAnalyzer([3.0], min_samples=1).metrics_for_rr(samples, "long", 3.0)
    assert metrics["expectancy_r"] == 1.66667
    assert metrics["profit_factor"] == 6.0
    assert metrics["max_drawdown_r"] == 1.0

def test_best_rr_uses_edge_score_not_highest_rr() -> None:
    samples = [
        _sample([125.0], [99.0], [100.0]),
        _sample([125.0], [99.0], [100.0]),
        _sample([125.0], [99.0], [100.0]),
        _sample([151.0], [90.0], [151.0]),
    ]
    result = RewardRiskAnalyzer([2.0, 5.0], min_samples=1).analyze(samples, "long")
    assert result["best_rr"] == 2.0

def test_simulation_subtracts_execution_cost_r() -> None:
    sample = _sample([130.0], [99.0], [130.0], cost_r=0.15)
    result, target_bar, stop_bar = RewardRiskAnalyzer._simulate_sample(sample, "long", 3.0)
    assert result == 2.85
    assert target_bar == 1
    assert stop_bar is None
    metrics = RewardRiskAnalyzer([3.0], min_samples=1).metrics_for_rr([sample], "long", 3.0)
    assert metrics["avg_execution_cost_r"] == 0.15
    assert metrics["triple_barrier_labels"] == {"target": 1, "stop": 0, "timeout": 0, "skipped": 0}
    assert metrics["avg_gap_adverse_r"] == 0.2
    assert metrics["mfe_before_mae_rate"] == 1.0
    assert metrics["avg_fill_probability"] == 0.8
    assert metrics["p25_max_size_usd"] == 12500.0

def test_cost_multiplier_stresses_result_r() -> None:
    sample = _sample([130.0], [99.0], [130.0], cost_r=0.15)
    result, _, _ = RewardRiskAnalyzer._simulate_sample(sample, "long", 3.0, cost_multiplier=2.0)
    assert result == 2.7

def test_canonical_outcome_skips_entry_gap_past_target() -> None:
    sample = _sample([151.0], [149.0], [150.0], opens=[140.0])
    detail = RewardRiskAnalyzer._simulate_sample_detail(sample, "long", 3.0)
    result, target_bar, stop_bar = RewardRiskAnalyzer._simulate_sample(sample, "long", 3.0)
    assert detail["status"] == "skipped"
    assert detail["reason"] == "gapped_past_target"
    assert (result, target_bar, stop_bar) == (0.0, None, None)

def test_skipped_signals_do_not_dilute_traded_aggregates() -> None:
    winner = _sample([130.0], [99.0], [130.0])
    loser = _sample([104.0], [90.0], [90.0])
    skipped = _sample([151.0], [149.0], [150.0], opens=[140.0])
    metrics = RewardRiskAnalyzer([3.0], min_samples=1).metrics_for_rr([winner, loser, skipped], "long", 3.0)
    # Traded aggregates over {+3R, -1R} only; the skipped signal adds no 0R.
    assert metrics["sample_count"] == 2
    assert metrics["signal_count"] == 3
    assert metrics["skipped_count"] == 1
    assert metrics["skip_rate"] == 0.33333
    assert metrics["skip_reason_counts"] == {"gapped_past_target": 1}
    assert metrics["expectancy_r"] == 1.0
    assert metrics["win_rate"] == 0.5
    assert metrics["loss_rate"] == 0.5
    assert metrics["target_hit_rate"] == 0.5
    assert metrics["stop_hit_rate"] == 0.5
    assert metrics["triple_barrier_labels"] == {"target": 1, "stop": 1, "timeout": 0, "skipped": 1}
    # Speed labels only cover traded samples (no phantom timeout for the skip).
    assert sum(metrics["speed_label_counts"].values()) == 2

def test_all_skipped_signals_yield_zero_sample_count() -> None:
    skipped = _sample([151.0], [149.0], [150.0], opens=[140.0])
    metrics = RewardRiskAnalyzer([3.0], min_samples=1).metrics_for_rr([skipped, skipped], "long", 3.0)
    assert metrics["sample_count"] == 0
    assert metrics["signal_count"] == 2
    assert metrics["skipped_count"] == 2
    assert metrics["skip_rate"] == 1.0
    assert metrics["expectancy_r"] == 0.0
    assert metrics["win_rate"] == 0.0

```

```
assert metrics["profit_factor"] == 0.0
```

Full Source: `backend/tradeo/tests/test\_backtester\_non\_trade\_accounting.py`

`backend/tradeo/tests/test\_backtester\_non\_trade\_accounting.py` ? 76 lines, 2304 bytes, sha256 prefix `8932861b63f4aae8`

Public surface summary before full source:

```
? class `_Params` line 13 methods: ?  
? class `_StubDetector` line 17 methods: detect  
? class `_StubProvider` line 40 methods: fetch_ohlc  
  
? function `test_gap_prefiltered_signal_counts_as_skipped_not_trade` line 57`  
? function `test_executed_signal_counts_as_trade_with_zero_skips` line 68`
```

Risk hints: no obvious heuristic risk; still review logic/tests.

```
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd

from tradeo.schemas import PatternCandidate
from tradeo.services.backtester import Backtester

@dataclass
class _Params:
    max_holdingBars: int = 5

class _StubDetector:
    """Fires exactly one candidate at the first eligible bar (i=230)."""

    def __init__(self) -> None:
        self.params = _Params()

    def detect(self, symbol: str, lookback: pd.DataFrame) -> PatternCandidate | None:
        if len(lookback) != 231:
            return None
        return PatternCandidate(
            symbol=symbol,
            entry=100.0,
            stop=95.0,
            target=110.0,
            reward_risk=2.0,
            confidence=0.8,
            rule_score=0.8,
            ml_score=0.8,
            vision_score=0.8,
            composite_score=0.8,
        )

class _StubProvider:
    def __init__(self, df: pd.DataFrame) -> None:
        self.df = df

    def fetch_ohlc(self, symbol: str, period: str = "3y", interval: str = "1d") -> pd.DataFrame:
        return self.df

def _flat_df(n: int = 300) -> pd.DataFrame:
    index = pd.date_range("2024-01-01", periods=n, freq="B")
    base = np.full(n, 100.0)
    return pd.DataFrame(
        {"open": base, "high": base + 1.0, "low": base - 1.0, "close": base},
        index=index,
    )

def test_gap_prefiltered_signal_counts_as_skipped_not_trade() -> None:
    df = _flat_df()
    df.iloc[231, df.columns.get_loc("open")] = 120.0 # >8% above candidate entry
    metrics = Backtester(provider=_StubProvider(df), detector=_StubDetector()).run(["TST"])
    assert metrics.total_signals == 1
    assert metrics.skipped_signals == 1
    assert metrics.total_trades == 0
    assert metrics.skip_rate == 1.0
    assert metrics.expectancy_r == 0.0

def test_executed_signal_counts_as_trade_with_zero_skips() -> None:
    df = _flat_df()
    df.iloc[233, df.columns.get_loc("high")] = 111.0 # target 110 hit two bars in
    metrics = Backtester(provider=_StubProvider(df), detector=_StubDetector()).run(["TST"])
    assert metrics.total_signals == 1
    assert metrics.skipped_signals == 0
    assert metrics.total_trades == 1
    assert metrics.skip_rate == 0.0
```

Full Source: `backend/tradeo/tests/test\_regime\_outcome\_calibration.py`

`backend/tradeo/tests/test\_regime\_outcome\_calibration.py` ? 262 lines, 9542 bytes, sha256 prefix `c921650367fc357f`

Public surface summary before full source:

```
? function `test_regime_keys_for_dates_is_point_in_time` line 72`
? function `test_regime_keys_for_dates_without_table_is_unlabeled` line 88`
? function `test_period_to_months_pares_provider_periods` line 94`
? function `test_history_table_covers_period_plus_warmup_and_records_config` line 103`
? function `test_benchmark_regime_outcomes_buckets_by_pit_regime` line 141`
? function `test_benchmark_regime_outcomes_unavailable_without_table` line 174`
? function `test_pattern_regime_fit_calibrated_positive_bucket` line 211`
? function `test_pattern_regime_fit_calibrated_negative_bucket_is_hard_gate_eligible` line 225`
? function `test_pattern_regime_fit_falls_back_when_bucket_too_small` line 237`
? function `test_pattern_regime_fit_falls_back_for_unlabeled_benchmark_regime` line 248`
? function `test_pattern_regime_fit_without_settings_keeps_legacy_behavior` line 257`
```

Risk hints: no obvious heuristic risk; still review logic/tests.

```
from __future__ import annotations

from types import SimpleNamespace

import numpy as np
import pandas as pd

from tradeo.core.config import Settings
from tradeo.research.cluster_research_engine import ClusterResearchEngine
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher
from tradeo.research.types import ForwardOutcome, WindowSample
from tradeo.services.market_regime import (
    MarketRegimeService,
    period_to_months,
    regime_keys_for_dates,
    unlabeled_regime_key,
)

BULL_LOW = "benchmark_bull|low_vol_tercile"
BEAR_HIGH = "benchmark_bear|high_vol_tercile"

def _regime_table(keys_by_date: dict[str, str]) -> pd.DataFrame:
    index = pd.DatetimeIndex(sorted(keys_by_date))
    table = pd.DataFrame(
        {"regime_key": [keys_by_date[str(ts.date())] for ts in index]},
        index=index,
    )
    table.attrs["benchmark_symbol"] = "SPY"
    return table

def _sample(end: str, *, win: bool) -> WindowSample:
    entry = 100.0
    risk = 10.0
    if win:
        # Long target at entry + rr*risk is touched before the stop.
        opens, highs, lows, closes = [101.0], [125.0], [96.0], [120.0]
    else:
        opens, highs, lows, closes = [99.0], [101.0], [85.0], [88.0]
    return WindowSample(
        symbol="REGT",
        timeframe="1d",
        window_size=20,
        start="2024-01-01",
        end=end,
        year=int(end[:4]),
        vector=np.asarray([1.0, 0.5], dtype=np.float32),
        chart={},
        features={},
        outcome=ForwardOutcome(
            forward_returns={5: closes[-1] / entry - 1.0},
            entry_price=entry,
            risk_proxy=risk,
            forward_end=end,
            long_mfe_r=(max(highs) - entry) / risk,
            long_mae_r=max(0.0, (entry - min(lows)) / risk),
            long_outcome_r=(closes[-1] - entry) / risk,
            long_hit_4r=False,
            short_mfe_r=(entry - min(lows)) / risk,
            short_mae_r=max(0.0, (max(highs) - entry) / risk),
            short_outcome_r=(entry - closes[-1]) / risk,
            short_hit_4r=False,
            forward_opens=opens,
            forward_highs=highs,
            forward_lows=lows,
            forward_closes=closes,
```

```

    ),
)

def test_regime_keys_for_dates_is_point_in_time() -> None:
    table = _regime_table(
        {
            "2024-03-01": BULL_LOW,
            "2024-03-04": BEAR_HIGH,
        }
    )

    keys = regime_keys_for_dates(
        table,
        ["2024-02-28", "2024-03-01", "2024-03-03", "2024-03-04", "2024-03-10"],
    )

    assert keys == [unlabeled_regime_key(), BULL_LOW, BULL_LOW, BEAR_HIGH, BEAR_HIGH]

def test_regime_keys_for_dates_without_table_is_unlabeled() -> None:
    assert regime_keys_for_dates(None, ["2024-03-01"]) == [unlabeled_regime_key()]
    empty = pd.DataFrame({"regime_key": []}, index=pd.DatetimeIndex([]))
    assert regime_keys_for_dates(empty, ["2024-03-01"]) == [unlabeled_regime_key()]

def test_period_to_months_parses_provider_periods() -> None:
    assert period_to_months("2y") == 24
    assert period_to_months("6mo") == 6
    assert period_to_months("90d") == 3
    assert period_to_months("max") == 120
    assert period_to_months(None) == 0
    assert period_to_months("not-a-period") == 0

def test_history_table_covers_period_plus_warmup_and_records_config() -> None:
    requested: dict[str, str] = {}

    class Provider:
        def fetch_ohlcv(self, symbol: str, *, period: str, interval: str) -> pd.DataFrame:
            requested.update({"symbol": symbol, "period": period, "interval": interval})
            closes = np.asarray([100.0 + 0.1 * i for i in range(400)])
            idx = pd.bdate_range("2023-01-02", periods=len(closes))
            return pd.DataFrame(
                {
                    "open": closes,
                    "high": closes * 1.01,
                    "low": closes * 0.99,
                    "close": closes,
                    "volume": np.full(len(closes), 1_000_000.0),
                },
                index=idx,
            )

    settings = Settings(
        market_regime_benchmark_symbol="SPY",
        market_regime_sma_window=50,
        market_regime_vol_window=10,
        market_regime_vol_tercile_lookback=60,
    )

    table = MarketRegimeService(provider=Provider(), settings=settings).history_table(period="1y")

    assert requested["symbol"] == "SPY"
    assert requested["interval"] == "1d"
    requested_months = int(requested["period"].removesuffix("mo"))
    assert requested_months >= 12 + 3
    assert "regime_key" in table.columns
    assert table.attrs["benchmark_symbol"] == "SPY"
    assert table.attrs["sma_window"] == 50
    assert set(regime_keys_for_dates(table, [table.index[-1]])) != {unlabeled_regime_key()}

def test_benchmark_regime_outcomes_buckets_by_pit_regime() -> None:
    table = _regime_table(
        {
            "2024-03-01": BULL_LOW,
            "2024-06-03": BEAR_HIGH,
        }
    )
    engine = ClusterResearchEngine(benchmark_regime_table=table)
    samples = [
        _sample("2024-03-05", win=True),
        _sample("2024-03-06", win=True),
        _sample("2024-06-10", win=False),
        _sample("2024-02-01", win=True), # before benchmark history -> unlabeled
    ]

    outcomes = engine._benchmark_regime_outcomes(samples, side="long", rr=2.0)

    assert outcomes["available"] is True
    assert outcomes["benchmark_symbol"] == "SPY"
    assert outcomes["labeled_sample_count"] == 3
    assert outcomes["unlabeled_sample_count"] == 1
    buckets = outcomes["buckets"]
    # Canonical simulation enters at the next bar's open, so R is path-derived;

```

```

# the sign and win rate per bucket are the contract here.
assert buckets[BULL_LOW]["sample_count"] == 2
assert buckets[BULL_LOW]["expectancy_r"] > 0.0
assert buckets[BULL_LOW]["win_rate"] == 1.0
assert buckets[BEAR_HIGH]["sample_count"] == 1
assert buckets[BEAR_HIGH]["expectancy_r"] < 0.0
assert buckets[BEAR_HIGH]["win_rate"] == 0.0
assert unlabeled_regime_key() in buckets

def test_benchmark_regime_outcomes_unavailable_without_table() -> None:
    engine = ClusterResearchEngine()

    outcomes = engine._benchmark_regime_outcomes([_sample("2024-03-05", win=True)], side="long", rr=2.0)

    assert outcomes["available"] is False
    assert outcomes["buckets"] == {}

def _pattern(benchmark_regime_outcomes: dict | None) -> SimpleNamespace:
    profile: dict = {
        "preferred_regime_keys": ["calm|bull|risk_on"],
        "bucket_counts": {"calm|bull|risk_on": 10},
    }
    if benchmark_regime_outcomes is not None:
        profile["benchmark_regime_outcomes"] = benchmark_regime_outcomes
    return SimpleNamespace(metrics_json={"regime_profile": profile})

def _regime(regime_key: str) -> dict:
    return {
        "regime_key": "calm|bull|risk_on",
        "benchmark_regime": {"regime_key": regime_key},
    }

def _outcomes(bucket: dict) -> dict:
    return {
        "available": True,
        "method": "pit_benchmark_regime_at_sample_end+canonical_triple_barrier",
        "side": "long",
        "rr": 2.5,
        "benchmark_symbol": "SPY",
        "buckets": {BULL_LOW: bucket},
    }

def test_pattern_regime_fit_calibrated_positive_bucket() -> None:
    settings = Settings(market_regime_outcome_min_samples=3)
    pattern = _pattern(_outcomes({"sample_count": 8, "expectancy_r": 0.4, "win_rate": 0.5, "profit_factor": 1.8}))

    fit = NovelPatternMatcher._pattern_regime_fit(pattern, _regime(BULL_LOW), settings)

    assert fit["label"] == "calibrated_regime_positive"
    assert fit["matched"] is True
    assert fit["hard_gate_blocked"] is False
    assert fit["score"] == 0.85
    assert fit["calibration"]["sample_count"] == 8
    assert fit["calibration"]["regime_key"] == BULL_LOW

def test_pattern_regime_fit_calibrated_negative_bucket_is_hard_gate_eligible() -> None:
    settings = Settings(market_regime_outcome_min_samples=3)
    pattern = _pattern(_outcomes({"sample_count": 9, "expectancy_r": -0.3, "win_rate": 0.2, "profit_factor": 0.4}))

    fit = NovelPatternMatcher._pattern_regime_fit(pattern, _regime(BULL_LOW), settings)

    assert fit["label"] == "calibrated_regime_negative"
    assert fit["matched"] is False
    assert fit["hard_gate_blocked"] is True
    assert fit["score"] == 0.2

def test_pattern_regime_fit_falls_back_when_bucket_too_small() -> None:
    settings = Settings(market_regime_outcome_min_samples=12)
    pattern = _pattern(_outcomes({"sample_count": 5, "expectancy_r": -0.3, "win_rate": 0.2, "profit_factor": 0.4}))

    fit = NovelPatternMatcher._pattern_regime_fit(pattern, _regime(BULL_LOW), settings)

    # Falls back to the preferred-regime heuristic on the local regime key.
    assert fit["label"] == "preferred_regime"
    assert "hard_gate_blocked" not in fit

def test_pattern_regime_fit_falls_back_for_unlabeled_benchmark_regime() -> None:
    settings = Settings(market_regime_outcome_min_samples=3)
    pattern = _pattern(_outcomes({"sample_count": 9, "expectancy_r": -0.3, "win_rate": 0.2, "profit_factor": 0.4}))

    fit = NovelPatternMatcher._pattern_regime_fit(pattern, _regime(unlabeled_regime_key()), settings)

    assert fit["label"] == "preferred_regime"

def test_pattern_regime_fit_without_settings_keeps_legacy_behavior() -> None:
    pattern = _pattern(_outcomes({"sample_count": 9, "expectancy_r": -0.3, "win_rate": 0.2, "profit_factor": 0.4}))

```

```
fit = NovelPatternMatcher._pattern_regime_fit(pattern, _regime(BULL_LOW))

assert fit["label"] == "preferred_regime"
```

Full Source: `backend/tradeo/tests/test\_execution\_quality\_audit.py`

`backend/tradeo/tests/test\_execution\_quality\_audit.py` ? 255 lines, 8320 bytes, sha256 prefix `842c4aea5278e620`

Public surface summary before full source:

```
? function `make_trade` line 22`
? function `test_long_fill_above_theoretical_entry_is_positive_slippage` line 53`
? function `test_short_fill_below_theoretical_entry_is_positive_slippage` line 64`
? function `test_better_than_theoretical_fill_reports_negative_slippage` line 75`
? function `test_time_to_fill_from_submitted_at_and_fill_time` line 84`
? function `test_time_to_fill_falls_back_to_broker_execution_time` line 98`
? function `test_negative_fill_latency_is_rejected_not_reported` line 112`
? function `test_commission_bps_relative_to_fill_notional` line 126`
? function `test_estimated_vs_realized_compares_pretrade_estimates` line 137`
? function `test_data_basis_markers_declare_missing_microstructure_feed` line 151`
? function `test_shadow_fill_provenance_is_excluded` line 164`
? function `test_missing_or_nonpositive_entry_fill_price_is_excluded` line 174`
? function `test_zero_risk_reports_bps_but_not_r` line 180`
? function `test_persist_execution_quality_writes_metadata_once` line 195`
? function `test_persist_execution_quality_skips_shadow_trades` line 209`
? function `test_persist_execution_quality_rewrites_when_fill_data_changes` line 218`
```

Risk hints: JSON metadata contract; schema drift risk.

```
"""Entry-quality execution audit tests (informe §4.3 fallback path).
```

```
No real-time microstructure feed exists, so the auditable surface is the
broker fill record itself: fill price vs theoretical entry, submit->fill
latency and commissions. These tests pin down that arithmetic, the
shadow-fill exclusion, the idempotent persistence and the pattern-level
summary the Director gate stores as a diagnostic.
"""
```

```
from __future__ import annotations
```

```
from tradeo.db.models import Trade, TradeStatus
from tradeo.services.evidence import FillProvenance
from tradeo.services.execution_quality import (
    EXECUTION_QUALITY_METHOD,
    pattern_execution_quality_summary,
    persist_execution_quality,
    trade_execution_quality,
)
```

```
def make_trade(
    *,
    side: str = "long",
    entry: float = 10.0,
    stop: float = 9.0,
    qty: int = 5,
    metadata: dict | None = None,
) -> Trade:
    return Trade(
        id=1,
        symbol="AAPL",
        pattern="cup_handle",
        side=side,
        qty=qty,
        entry=entry,
        stop=stop,
        target=14.0,
        status=TradeStatus.CLOSED,
        metadata_json={
            "fill_provenance": FillProvenance.BROKER_EXECUTION.value,
            "entry_fill_price": 10.05,
            **(metadata or {}),
        },
    )
```

```
# -----
# Per-trade report arithmetic
# -----
```

```
def test_long_fill_above_theoretical_entry_is_positive_slippage() -> None:
    report = trade_execution_quality(make_trade())

    assert report is not None
    assert report["method"] == EXECUTION_QUALITY_METHOD
    # (10.05 - 10.00) / 10.00 * 10_000 = 50 bps worse than theoretical.
    assert report["entry_slippage_bps"] == 50.0
```

```

# risk per share = 1.0 -> 0.05 R
assert report["entry_slippage_r"] == 0.05

def test_short_fill_below_theoretical_entry_is_positive_slippage() -> None:
    trade = make_trade(side="short", stop=11.0, metadata={"entry_fill_price": 9.95})

    report = trade_execution_quality(trade)

    assert report is not None
    # Short filled lower than theoretical = worse entry -> positive slippage.
    assert report["entry_slippage_bps"] == 50.0
    assert report["entry_slippage_r"] == 0.05

def test_better_than_theoretical_fill_reports_negative_slippage() -> None:
    trade = make_trade(metadata={"entry_fill_price": 9.98})

    report = trade_execution_quality(trade)

    assert report is not None
    assert report["entry_slippage_bps"] == -20.0

def test_time_to_fill_from_submitted_at_and_fill_time() -> None:
    trade = make_trade(
        metadata={
            "submitted_at": "2026-06-01T15:00:00+00:00",
            "entry_fill_time": "2026-06-01T15:00:42+00:00",
        }
    )

    report = trade_execution_quality(trade)

    assert report is not None
    assert report["time_to_fill_s"] == 42.0

def test_time_to_fill_falls_back_to_broker_execution_time() -> None:
    trade = make_trade(
        metadata={
            "submitted_at": "2026-06-01T15:00:00+00:00",
            "broker_execution_time": "2026-06-01T15:01:00+00:00",
        }
    )

    report = trade_execution_quality(trade)

    assert report is not None
    assert report["time_to_fill_s"] == 60.0

def test_negative_fill_latency_is_rejected_not_reported() -> None:
    trade = make_trade(
        metadata={
            "submitted_at": "2026-06-01T15:01:00+00:00",
            "entry_fill_time": "2026-06-01T15:00:00+00:00",
        }
    )

    report = trade_execution_quality(trade)

    assert report is not None
    assert report["time_to_fill_s"] is None

def test_commission_bps_relative_to_fill_notional() -> None:
    trade = make_trade(metadata={"commission": 1.005})

    report = trade_execution_quality(trade)

    assert report is not None
    assert report["commission_usd"] == 1.005
    # notional = 10.05 * 5 = 50.25 -> 1.005 / 50.25 = 2% = 200 bps
    assert report["commission_bps"] == 200.0

def test_estimated_vs_realized_compares_pretrade_estimates() -> None:
    trade = make_trade(
        metadata={"estimated_slippage": 0.02, "estimated_spread_cost": 0.01}
    )

    report = trade_execution_quality(trade)

    assert report is not None
    comparison = report["estimated_vs_realized"]
    assert comparison is not None
    assert comparison["realized_entry_shortfall_per_share_usd"] == 0.05
    assert comparison["estimate_error_per_share_usd"] == 0.02

def test_data_basis_markers_declare_missing_microstructure_feed() -> None:
    report = trade_execution_quality(make_trade())

    assert report is not None
    assert report["data_basis"] == "broker_fill_record_eod_daily"

```

```

    assert report["microstructure_feed"] == "none_available"

# -----
# Exclusions: shadow fills and unusable inputs never produce a report
# -----

def test_shadow_fill_provenance_is_excluded() -> None:
    for provenance in (
        FillProvenance.SIMULATED_CLOSE.value,
        FillProvenance.SHADOW_CLOSE.value,
        "",
    ):
        trade = make_trade(metadata={"fill_provenance": provenance})
        assert trade_execution_quality(trade) is None

def test_missing_or_nonpositive_entry_fill_price_is_excluded() -> None:
    assert trade_execution_quality(make_trade(metadata={"entry_fill_price": None})) is None
    assert trade_execution_quality(make_trade(metadata={"entry_fill_price": 0.0})) is None
    assert trade_execution_quality(make_trade(metadata={"entry_fill_price": "n/a"})) is None

def test_zero_risk_reports_bps_but_not_r() -> None:
    trade = make_trade(stop=10.0)

    report = trade_execution_quality(trade)

    assert report is not None
    assert report["entry_slippage_bps"] == 50.0
    assert report["entry_slippage_r"] is None

# -----
# Persistence: idempotent metadata writes
# -----

def test_persist_execution_quality_writes_metadata_once() -> None:
    trade = make_trade()

    first = persist_execution_quality([trade])
    record = trade.metadata_json["execution_quality"]
    second = persist_execution_quality([trade])

    assert first == 1
    assert second == 0
    assert trade.metadata_json["execution_quality"] == record
    assert record["method"] == EXECUTION_QUALITY_METHOD
    assert "computed_at" in record

def test_persist_execution_quality_skips_shadow_trades() -> None:
    trade = make_trade(metadata={"fill_provenance": FillProvenance.SHADOW_CLOSE.value})

    touched = persist_execution_quality([trade])

    assert touched == 0
    assert "execution_quality" not in trade.metadata_json

def test_persist_execution_quality_rewrites_when_fill_data_changes() -> None:
    trade = make_trade()
    persist_execution_quality([trade])

    trade.metadata_json = {**trade.metadata_json, "entry_fill_price": 10.10}
    touched = persist_execution_quality([trade])

    assert touched == 1
    assert trade.metadata_json["execution_quality"]["entry_fill_price"] == 10.1

# -----
# Pattern-level summary
# -----

def test_pattern_summary_aggregates_real_fills_only() -> None:
    real_a = make_trade()
    real_b = make_trade(metadata={"entry_fill_price": 10.15})
    shadow = make_trade(metadata={"fill_provenance": FillProvenance.SHADOW_CLOSE.value})

    summary = pattern_execution_quality_summary([real_a, shadow, real_b])

    assert summary["count"] == 2
    assert summary["median_entry_slippage_bps"] == 100.0
    assert summary["worst_entry_slippage_bps"] == 150.0
    assert summary["microstructure_feed"] == "none_available"
    assert len(summary["per_trade"]) == 2

def test_pattern_summary_empty_when_no_real_fills() -> None:
    shadow = make_trade(metadata={"fill_provenance": FillProvenance.SHADOW_CLOSE.value})

    summary = pattern_execution_quality_summary([shadow])

```



```

assert summary["count"] == 0
assert summary["median_entry_slippage_bps"] is None
assert summary["per_trade"] == []

```

Full Source: `backend/tradeo/tests/test\_execution\_state\_transitions.py`

`backend/tradeo/tests/test\_execution\_state\_transitions.py` ? 738 lines, 24558 bytes, sha256 prefix `7ce736731141a696`

Public surface summary before full source:

```

? class `FakeBroker` line 104 methods: positions, open_orders
? class `FakeProvider` line 185 methods: fetch_ohlcv

? function `session_factory` line 46`
? function `add_signal` line 52`
? function `add_open_trade` line 75`
? function `audit_actions` line 127`
? function `test_paper_broker_opens_trade_and_marks_signal_executed` line 136`
? function `test_paper_broker_rejects_non_executable_signal_states` line 159`
? function `test_paper_broker_rejects_zero_quantity` line 170`
? function `make_frame` line 197`
? function `add_shadow_observation` line 204`
? function `observation_service` line 233`
? function `test_shadow_observation_transitions_open_to_closed_on_target_hit` line 237`
? function `test_shadow_observation_transitions_open_to_closed_on_stop_hit` line 262`
? function `test_shadow_observation_stays_open_without_future_bars` line 280`
? function `test_shadow_observation_stays_open_when_market_data_unavailable` line 297`
? function `test_open_observation_is_idempotent_per_signal` line 311`
? function `test_paper_order_promotes_to_paper_fill_only_when_closed_with_real_provenance` line 335`

```

Risk hints: IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

"""Explicit order-state transition and reconciliation tests (informe §4.5).

Covers the trade state machine end to end:

- signal -> trade transitions in the paper broker;
- lab shadow observation lifecycle (open -> closed / pending);
- evidence promotion rules (order -> fill) on status transitions;
- DB <-> broker reconciliation, including the automatic kill switch and the broker-unreachable path that must never trip it.

"""

```
from __future__ import annotations
```

```
from datetime import datetime, timedelta, timezone
```

```
import pandas as pd
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
```

```

from tradeo.core.config import Settings
from tradeo.db.models import (
    AuditLog,
    Signal,
    SignalStatus,
    Trade,
    TradeStatus,
)
from tradeo.db.session import Base
from tradeo.modules.laboratory.paper_observations import LabPaperObservationService
from tradeo.services.evidence import (
    EvidenceQuality,
    EvidenceType,
    FillProvenance,
    evidence_type_for_metadata,
)
from tradeo.services.ibkr_broker import IBKRBroker, IBKRSafetyError
from tradeo.services.paper_broker import PaperBroker
from tradeo.services.reconciliation import ReconciliationService
from tradeo.services.system_controls import (
    activate_runtime_kill_switch,
    deactivate_runtime_kill_switch,
    runtime_kill_switch_active,
)

```

```

def session_factory():
    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)
    return sessionmaker(bind=engine, future=True)()

```

```

def add_signal(db, *, status: SignalStatus, qty: int = 5, symbol: str = "AAPL") -> Signal:
    signal = Signal(
        symbol=symbol,

```

```

        pattern="cup_handle",
        side="long",
        entry=10.0,
        stop=9.0,
        target=14.0,
        reward_risk=4.0,
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=qty,
        status=status,
        human_approved=True,
        metadata_json={},
    )
    db.add(signal)
    db.commit()
    db.refresh(signal)
    return signal

def add_open_trade(
    db,
    *,
    symbol: str = "AAPL",
    execution_mode: str = "ibkr",
    broker_order_id: str | None = "101",
    qty: int = 5,
    side: str = "long",
    metadata: dict | None = None,
) -> Trade:
    trade = Trade(
        symbol=symbol,
        pattern="cup_handle",
        side=side,
        qty=qty,
        entry=10.0,
        stop=9.0,
        target=14.0,
        status=TradeStatus.OPEN,
        opened_at=datetime(2026, 6, 1, 15, 0, tzinfo=timezone.utc),
        broker_order_id=broker_order_id,
        metadata_json={"execution_mode": execution_mode, **(metadata or {})},
    )
    db.add(trade)
    db.commit()
    db.refresh(trade)
    return trade

class FakeBroker:
    def __init__(
        self,
        *,
        positions: list[dict] | None = None,
        open_orders: list[dict] | None = None,
        error: Exception | None = None,
    ) -> None:
        self._positions = positions or []
        self._open_orders = open_orders or []
        self._error = error

    def positions(self) -> list[dict]:
        if self._error is not None:
            raise self._error
        return self._positions

    def open_orders(self) -> list[dict]:
        if self._error is not None:
            raise self._error
        return self._open_orders

def audit_actions(db) -> list[str]:
    return [row.action for row in db.query(AuditLog).all()]

# -----
# Paper broker: signal -> trade transitions
# -----

def test_paper_broker_opens_trade_and_marks_signal_executed() -> None:
    db = session_factory()
    signal = add_signal(db, status=SignalStatus.PAPER_APPROVED)

    trade = PaperBroker().execute_signal(db, signal)

    assert trade.status == TradeStatus.OPEN
    assert signal.status == SignalStatus.EXECUTED
    assert trade.metadata_json["no_ibkr_order"] is True
    assert trade.evidence_type == EvidenceType.SHADOW_NO_ORDER.value
    assert "paper_trade_opened" in audit_actions(db)

@pytest.mark.parametrize(
    "status",

```

```

[
    SignalStatus.WATCHLIST,
    SignalStatus.REJECTED,
    SignalStatus.PENDING_HUMAN_APPROVAL,
    SignalStatus.EXECUTED,
    SignalStatus.EXPIRED,
],
)
def test_paper_broker_rejects_non_executable_signal_states(status: SignalStatus) -> None:
    db = session_factory()
    signal = add_signal(db, status=status)

    with pytest.raises(ValueError, match="not executable"):
        PaperBroker().execute_signal(db, signal)

    assert db.query(Trade).count() == 0
    assert signal.status == status

def test_paper_broker_rejects_zero_quantity() -> None:
    db = session_factory()
    signal = add_signal(db, status=SignalStatus.PAPER_APPROVED, qty=0)

    with pytest.raises(ValueError, match="suggested_qty"):
        PaperBroker().execute_signal(db, signal)

    assert db.query(Trade).count() == 0

# -----
# Lab shadow observation lifecycle: OPEN -> CLOSED / pending
# -----

class FakeProvider:
    def __init__(self, frame: pd.DataFrame | None = None, error: Exception | None = None) -> None:
        self._frame = frame
        self._error = error

    def fetch_ohlc(self, symbol: str, period: str = "6mo", interval: str = "1d") -> pd.DataFrame:
        if self._error is not None:
            raise self._error
        assert self._frame is not None
        return self._frame

def make_frame(rows: list[dict], start: datetime) -> pd.DataFrame:
    index = pd.DatetimeIndex(
        [pd.Timestamp(start + timedelta(days=offset)) for offset in range(len(rows))]
    )
    return pd.DataFrame(rows, index=index)

def add_shadow_observation(db, *, symbol: str = "AAPL") -> Trade:
    signal = add_signal(db, status=SignalStatus.EXECUTED, symbol=symbol)
    trade = Trade(
        signal_id=signal.id,
        symbol=symbol,
        pattern="cup_handle",
        side="long",
        qty=5,
        entry=10.0,
        stop=9.0,
        target=14.0,
        status=TradeStatus.OPEN,
        opened_at=datetime(2026, 6, 1, 15, 0, tzinfo=timezone.utc),
        evidence_type=EvidenceType.SHADOW_NO_ORDER.value,
        evidence_quality=EvidenceQuality.NORMAL.value,
        metadata_json={
            "execution_mode": "lab_shadow_observation",
            "observation_only": True,
            "no_ibkr_order": True,
            "evidence_type": EvidenceType.SHADOW_NO_ORDER.value,
            "evidence_quality": EvidenceQuality.NORMAL.value,
        },
    )
    db.add(trade)
    db.commit()
    db.refresh(trade)
    return trade

def observation_service(provider: FakeProvider) -> LabPaperObservationService:
    return LabPaperObservationService(settings=Settings(), provider=provider)

def test_shadow_observation_transitions_open_to_closed_on_target_hit() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    frame = make_frame(
        [
            {"open": 10.0, "high": 11.0, "low": 9.5, "close": 10.5, "volume": 1000.0},
            {"open": 10.5, "high": 14.5, "low": 10.0, "close": 14.0, "volume": 1000.0},
        ],
        start=datetime(2026, 6, 2, 15, 0, tzinfo=timezone.utc),
    )

```

```

result = observation_service(FakeProvider(frame)).close_open_observations(db)
db.refresh(trade)

assert result["closed_observations"] == 1
assert trade.status == TradeStatus.CLOSED
assert trade.exit_price == 14.0
assert trade.r_multiple == pytest.approx(4.0)
assert trade.metadata_json["exit_reason"] == "target_hit"
# A shadow close must never masquerade as broker fill evidence.
assert trade.evidence_type == EvidenceType.SHADOW_NO_ORDER.value
assert trade.metadata_json["fill_provenance"] == FillProvenance.SHADOW_CLOSE.value
assert "lab_shadow_observation_closed" in audit_actions(db)

def test_shadow_observation_transitions_open_to_closed_on_stop_hit() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    frame = make_frame(
        [{"open": 10.0, "high": 10.2, "low": 8.5, "close": 8.8, "volume": 1000.0}],
        start=datetime(2026, 6, 2, 15, 0, tzinfo=timezone.utc),
    )

    result = observation_service(FakeProvider(frame)).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 1
    assert trade.status == TradeStatus.CLOSED
    assert trade.exit_price == 9.0
    assert trade.r_multiple == pytest.approx(-1.0)
    assert trade.metadata_json["exit_reason"] == "stop_hit"

def test_shadow_observation_stays_open_without_future_bars() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    # Only bars at/before opened_at: nothing to evaluate yet.
    frame = make_frame(
        [{"open": 10.0, "high": 10.2, "low": 9.8, "close": 10.0, "volume": 1000.0}],
        start=datetime(2026, 5, 28, 15, 0, tzinfo=timezone.utc),
    )

    result = observation_service(FakeProvider(frame)).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 0
    assert trade.status == TradeStatus.OPEN
    assert trade.metadata_json["last_shadow_lifecycle_status"] == "awaiting_future_market_bars"

def test_shadow_observation_stays_open_when_market_data_unavailable() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    provider = FakeProvider(error=RuntimeError("provider down"))

    result = observation_service(provider).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 0
    assert result["data_errors"]
    assert trade.status == TradeStatus.OPEN
    assert trade.metadata_json["market_data_unavailable"] is True

def test_open_observation_is_idempotent_per_signal() -> None:
    db = session_factory()
    signal = add_signal(db, status=SignalStatus.EXECUTED)
    signal.metadata_json = {"entry_module": "laboratory", "pattern_id": 1}
    db.commit()
    service = observation_service(FakeProvider(pd.DataFrame()))

    class Risk:
        suggested_qty = 5

    first = service.open_observation(db, signal=signal, match={}, risk=Risk())
    second = service.open_observation(db, signal=signal, match={}, risk=Risk())

    assert first is not None
    assert second is not None
    assert first.id == second.id
    assert db.query(Trade).filter(Trade.status == TradeStatus.OPEN).count() == 1

# -----
# Evidence promotion: order -> fill only on real closed executions
# -----

def test_paper_order_promotes_to_paper_fill_only_when_closed_with_real_provenance() -> None:
    metadata = {
        "evidence_type": EvidenceType.IBKR_PAPER_ORDER.value,
        "fill_provenance": FillProvenance.BROKER_EXECUTION.value,
    }

    assert (
        evidence_type_for_metadata(metadata, trade_status=TradeStatus.OPEN)

```

```

        == EvidenceType.IBKR_PAPER_ORDER.value
    )
    assert (
        evidence_type_for_metadata(metadata, trade_status=TradeStatus.CANCELLED)
        == EvidenceType.IBKR_PAPER_ORDER.value
    )
    assert (
        evidence_type_for_metadata(metadata, trade_status=TradeStatus.CLOSED)
        == EvidenceType.IBKR_PAPER_FILL.value
    )

def test_paper_order_with_simulated_close_never_becomes_fill() -> None:
    metadata = {
        "evidence_type": EvidenceType.IBKR_PAPER_ORDER.value,
        "fill_provenance": FillProvenance.SIMULATED_CLOSE.value,
    }

    assert (
        evidence_type_for_metadata(metadata, trade_status=TradeStatus.CLOSED)
        == EvidenceType.IBKR_PAPER_ORDER.value
    )

def test_live_order_promotes_to_live_fill_on_closed_statement_import() -> None:
    metadata = {
        "evidence_type": EvidenceType.LIVE_ORDER.value,
        "fill_provenance": FillProvenance.BROKER_STATEMENT_IMPORT.value,
    }

    assert (
        evidence_type_for_metadata(metadata, trade_status=TradeStatus.CLOSED)
        == EvidenceType.LIVE_FILL.value
    )

# -----
# Reconciliation: divergences, kill switch, broker-unreachable
# -----

def reconciliation_service(broker: FakeBroker, **settings_overrides) -> ReconciliationService:
    return ReconciliationService(settings=Settings(**settings_overrides), broker=broker)

def test_reconciliation_clean_state_keeps_kill_switch_off() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL")
    broker = FakeBroker(positions=[{"symbol": "AAPL", "position": 5.0}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["ok"] is True
    assert result["divergences"] == []
    assert result["kill_switch_activated"] is False
    assert runtime_kill_switch_active(db) is False
    assert "reconciliation_completed" in audit_actions(db)

def test_reconciliation_open_order_counts_as_consistent_state() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL")
    broker = FakeBroker(open_orders=[{"symbol": "AAPL", "order_id": 101}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    assert runtime_kill_switch_active(db) is False

def test_reconciliation_db_trade_missing_at_broker_trips_kill_switch() -> None:
    db = session_factory()
    trade = add_open_trade(db, symbol="AAPL")
    broker = FakeBroker()

    result = reconciliation_service(broker).reconcile(db)

    assert result["kill_switch_activated"] is True
    assert runtime_kill_switch_active(db) is True
    assert result["divergences"] == [
        {
            "kind": "db_open_trade_missing_at_broker",
            "symbol": "AAPL",
            "trade_id": trade.id,
            "broker_order_id": "101",
        }
    ]
    actions = audit_actions(db)
    assert "reconciliation_completed" in actions
    assert "runtime_kill_switch_activated" in actions

def test_reconciliation_broker_position_not_in_db_trips_kill_switch() -> None:
    db = session_factory()
    broker = FakeBroker(positions=[{"symbol": "MSFT", "position": 3.0}])

```

```

result = reconciliation_service(broker).reconcile(db)

assert result["divergences"] == [
    {"kind": "broker_position_not_in_db", "symbol": "MSFT"}
]
assert result["kill_switch_activated"] is True
assert runtime_kill_switch_active(db) is True

def test_reconciliation_ignores_zero_quantity_broker_positions() -> None:
    db = session_factory()
    broker = FakeBroker(positions=[{"symbol": "MSFT", "position": 0.0}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    assert runtime_kill_switch_active(db) is False

def test_reconciliation_ignores_non_ibkr_open_trades() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", execution_mode="paper", broker_order_id=None)
    add_open_trade(
        db,
        symbol="MSFT",
        execution_mode="lab_shadow_observation",
        broker_order_id=None,
    )
    broker = FakeBroker()

    result = reconciliation_service(broker).reconcile(db)

    assert result["db_open_ibkr_trades"] == 0
    assert result["divergences"] == []
    assert runtime_kill_switch_active(db) is False

def test_reconciliation_broker_unreachable_never_trips_kill_switch() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL")
    broker = FakeBroker(error=ConnectionError("gateway down"))

    result = reconciliation_service(broker).reconcile(db)

    assert result["ok"] is False
    assert "broker_unreachable" in result["error"]
    assert result["kill_switch_activated"] is False
    assert runtime_kill_switch_active(db) is False
    actions = audit_actions(db)
    assert "reconciliation_broker_unreachable" in actions
    assert "runtime_kill_switch_activated" not in actions

def test_reconciliation_divergence_respects_disabled_auto_kill_switch() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL")
    broker = FakeBroker()

    result = reconciliation_service(
        broker, reconciliation_auto_kill_switch=False
    ).reconcile(db)

    assert result["divergences"]
    assert result["kill_switch_activated"] is False
    assert runtime_kill_switch_active(db) is False
    assert "reconciliation_completed" in audit_actions(db)

def test_reconciliation_reports_pre_existing_kill_switch() -> None:
    db = session_factory()
    activate_runtime_kill_switch(db, reason="manual", actor="test")
    broker = FakeBroker()

    result = reconciliation_service(broker).reconcile(db)

    assert result["kill_switch_already_active"] is True
    assert result["kill_switch_activated"] is False

# -----
# Reconciliation: explicit qty / order-id / partial-fill checks
# -----

def test_reconciliation_position_larger_than_db_trips_kill_switch() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", qty=5)
    broker = FakeBroker(positions=[{"symbol": "AAPL", "position": 8.0}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == [
        {
            "kind": "position_qty_mismatch_at_broker",
            "symbol": "AAPL",
            "db_signed_qty": 5.0,

```

```

        "broker_signed_qty": 8.0,
    }
]
assert result["kill_switch_activated"] is True
assert runtime_kill_switch_active(db) is True

def test_reconciliation_position_wrong_side_trips_kill_switch() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", qty=5, side="short")
    broker = FakeBroker(
        positions=[{"symbol": "AAPL", "position": 5.0}],
        open_orders=[{"symbol": "AAPL", "order_id": 101}],
    )

    result = reconciliation_service(broker).reconcile(db)

    assert [d["kind"] for d in result["divergences"]] == [
        "position_qty_mismatch_at_broker"
    ]
    assert result["divergences"][0]["db_signed_qty"] == -5.0
    assert runtime_kill_switch_active(db) is True

def test_reconciliation_smaller_position_with_pending_orders_is_warning_only() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", qty=5)
    broker = FakeBroker(
        positions=[{"symbol": "AAPL", "position": 3.0}],
        open_orders=[
            {
                "symbol": "AAPL",
                "order_id": 101,
                "quantity": 5.0,
                "filled": 3.0,
                "remaining": 2.0,
            }
        ],
    )

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    warning_kinds = sorted(w["kind"] for w in result["warnings"])
    assert warning_kinds == [
        "partial_fill_open_order",
        "position_qty_below_db_pending_orders",
    ]
    assert result["kill_switch_activated"] is False
    assert runtime_kill_switch_active(db) is False
    assert "runtime_kill_switch_activated" not in audit_actions(db)

def test_reconciliation_smaller_position_without_orders_trips_kill_switch() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", qty=5)
    broker = FakeBroker(positions=[{"symbol": "AAPL", "position": 3.0}])

    result = reconciliation_service(broker).reconcile(db)

    assert [d["kind"] for d in result["divergences"]] == [
        "position_qty_mismatch_at_broker"
    ]
    assert runtime_kill_switch_active(db) is True

def test_reconciliation_sums_multiple_db_trades_per_symbol() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", qty=3, broker_order_id="101")
    add_open_trade(db, symbol="AAPL", qty=2, broker_order_id="102")
    broker = FakeBroker(positions=[{"symbol": "AAPL", "position": 5.0}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    assert runtime_kill_switch_active(db) is False

def test_reconciliation_unknown_order_id_at_broker_trips_kill_switch() -> None:
    db = session_factory()
    trade = add_open_trade(db, symbol="AAPL", broker_order_id="101")
    broker = FakeBroker(open_orders=[{"symbol": "AAPL", "order_id": 999}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == [
        {
            "kind": "db_order_id_missing_at_broker",
            "symbol": "AAPL",
            "trade_id": trade.id,
            "broker_order_id": "101",
            "broker_open_order_ids": ["999"],
        }
    ]
    assert runtime_kill_switch_active(db) is True

```

```

def test_reconciliation_matches_bracket_children_via_parent_order_id() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", broker_order_id="101")
    broker = FakeBroker(
        open_orders=[
            {"symbol": "AAPL", "order_id": 102, "parent_order_id": 101},
            {"symbol": "AAPL", "order_id": 103, "parent_order_id": 101},
        ]
    )

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    assert runtime_kill_switch_active(db) is False

def test_reconciliation_partial_fill_is_audited_warning_never_divergence() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", broker_order_id="101")
    broker = FakeBroker(
        open_orders=[
            {
                "symbol": "AAPL",
                "order_id": 101,
                "quantity": 5.0,
                "filled": 2.0,
                "remaining": 3.0,
            }
        ]
    )

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    assert result["warnings"] == [
        {
            "kind": "partial_fill_open_order",
            "symbol": "AAPL",
            "order_id": 101,
            "filled": 2.0,
            "quantity": 5.0,
            "remaining": 3.0,
        }
    ]
    assert runtime_kill_switch_active(db) is False
    audit = (
        db.query(AuditLog)
        .filter(AuditLog.action == "reconciliation_completed")
        .one()
    )
    assert audit.details_json["warning_count"] == 1
    assert audit.details_json["warnings"][0]["kind"] == "partial_fill_open_order"

def test_reconciliation_trade_without_order_id_skips_order_matching() -> None:
    db = session_factory()
    add_open_trade(db, symbol="AAPL", broker_order_id=None)
    broker = FakeBroker(open_orders=[{"symbol": "AAPL", "order_id": 999}])

    result = reconciliation_service(broker).reconcile(db)

    assert result["divergences"] == []
    assert runtime_kill_switch_active(db) is False

# -----
# Kill switch state machine: activation blocks orders, deactivation restores
# -----

def test_active_kill_switch_blocks_ibkr_order_submission() -> None:
    db = session_factory()
    signal = add_signal(db, status=SignalStatus.PAPER_APPROVED)
    activate_runtime_kill_switch(db, reason="reconciliation divergence", actor="test")

    with pytest.raises(IBKRSafetyError, match="kill switch"):
        IBKRBroker(settings=Settings()).submit_signal_bracket(db, signal)

    assert db.query(Trade).count() == 0
    assert signal.status == SignalStatus.PAPER_APPROVED

def test_kill_switch_activation_is_idempotent_and_audited() -> None:
    db = session_factory()
    activate_runtime_kill_switch(db, reason="first", actor="test")
    activate_runtime_kill_switch(db, reason="second", actor="test")

    assert runtime_kill_switch_active(db) is True
    activations = [
        row
        for row in db.query(AuditLog).all()
        if row.action == "runtime_kill_switch_activated"
    ]
    assert len(activations) == 2
    assert activations[1].details_json["already_active"] is True

```



```
def test_kill_switch_deactivation_requires_explicit_actor_and_is_audited() -> None:
    db = session_factory()
    activate_runtime_kill_switch(db, reason="divergence", actor="reconciliation")

    control = deactivate_runtime_kill_switch(db, reason="human investigated", actor="asier")

    assert control is not None
    assert control.enabled is False
    assert runtime_kill_switch_active(db) is False
    assert "runtime_kill_switch_deactivated" in audit_actions(db)
```

Full Source: ``backend/tradeo/tests/test_data_provider.py``

``backend/tradeo/tests/test_data_provider.py`` ? 487 lines, 17277 bytes, sha256 prefix ``3d11e19c27302a5f``

Public surface summary before full source:

```
? class `_RecordingUpstream` line 123 methods: fetch_ohlcv
? class `FakeIB` line 440 methods: connect, reqCurrentTime, managedAccounts, accountSummary, isConnected, disconnect

? function `test_settings_reject_synthetic_market_data` line 19`
? function `test_settings_reject_non_ibkr_market_data_provider` line 24`
? function `test_market_data_factory_returns_ibkr_provider` line 29`
? function `test_market_data_factory_wraps_ibkr_with_cache` line 44`
? function `test_cached_market_data_provider_persists_manifest` line 74`
? function `test_incremental_refresh_appends_missing_tail` line 172`
? function `test_incremental_refresh_full_refetch_on_overlap_mismatch` line 202`
? function `test_incremental_refresh_disabled_keeps_cache_untouched` line 227`
? function `test_incremental_refresh_skips_fresh_cache` line 245`
? function `test_intraday_incremental_refresh_appends_missing_tail` line 263`
? function `test_intraday_incremental_refresh_skips_fresh_cache` line 291`
? function `test_intraday_incremental_refresh_full_refetch_on_large_gap` line 310`
? function `test_intraday_incremental_refresh_disabled_keeps_cache_untouched` line 332`
? function `test_incomplete_intraday_bar_is_not_served_or_persisted` line 351`
? function `test_detect_unadjusted_splits_flags_split_like_gap` line 371`
? function `test_universe_snapshot_filters_and_marks_survivorship` line 389`
```

Risk hints: wall-clock timestamp usage; IBKR/broker/data dependency.

```
from __future__ import annotations

import json
import sys
from datetime import datetime, timezone
from pathlib import Path
from types import SimpleNamespace

import pandas as pd
import pytest

from tradeo.core.config import Settings
from tradeo.services.data_provider import CachedMarketDataProvider, detect_unadjusted_splits
from tradeo.services.provider_factory import get_market_data_provider
from tradeo.services.ibkr_data_provider import inspect_ibkr_connection
from tradeo.services.universe_snapshot import UniverseSnapshotService

def test_settings_reject_synthetic_market_data() -> None:
    with pytest.raises(ValueError, match="Synthetic market data is forbidden"):
        Settings(allow_synthetic_market_data=True)

def test_settings_reject_non_ibkr_market_data_provider() -> None:
    with pytest.raises(ValueError, match="only permits IBKR market data"):
        Settings(market_data_provider="yfinance")

def test_market_data_factory_returns_ibkr_provider(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.setenv("TRADEO_MARKET_DATA_PROVIDER", "ibkr")
    monkeypatch.setenv("TRADEO_ALLOW_SYNTHETIC_MARKET_DATA", "false")
    monkeypatch.setenv("TRADEO_MARKET_DATA_CACHE_ENABLED", "false")
    from tradeo.core.config import get_settings

    get_settings.cache_clear()
    try:
        provider = get_market_data_provider()
    finally:
        get_settings.cache_clear()

    assert provider.__class__.__name__ == "IBKRHistoricalDataProvider"

def test_market_data_factory_wraps_ibkr_with_cache(monkeypatch: pytest.MonkeyPatch, tmp_path: Path) -> None:
    monkeypatch.setenv("TRADEO_MARKET_DATA_PROVIDER", "ibkr")
    monkeypatch.setenv("TRADEO_ALLOW_SYNTHETIC_MARKET_DATA", "false")
```

```

monkeypatch.setenv("TRADEO_MARKET_DATA_CACHE_ENABLED", "true")
monkeypatch.setenv("TRADEO_MARKET_DATA_CACHE_DIR", str(tmp_path / "cache"))
from tradeo.core.config import get_settings

get_settings.cache_clear()
try:
    provider = get_market_data_provider()
finally:
    get_settings.cache_clear()

assert provider.__class__.__name__ == "CachedMarketDataProvider"

def _ohlcv_frame() -> pd.DataFrame:
    idx = pd.date_range("2026-01-02", periods=3, freq="B")
    return pd.DataFrame(
        {
            "open": [10.0, 10.5, 11.0],
            "high": [10.8, 11.2, 11.8],
            "low": [9.8, 10.1, 10.7],
            "close": [10.4, 11.0, 11.4],
            "volume": [100_000, 120_000, 140_000],
        },
        index=idx,
    )

def test_cached_market_data_provider_persists_manifest(tmp_path: Path) -> None:
    class Upstream:
        calls = 0

        def fetch_ohlcv(self, symbol: str, period: str = "2y", interval: str = "1d") -> pd.DataFrame:
            self.calls += 1
            return _ohlcv_frame()

    upstream = Upstream()
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
    )

    first = provider.fetch_ohlcv("aaa", period="5y", interval="1d")
    second = provider.fetch_ohlcv("AAA", period="5y", interval="1d")
    manifest = provider.data_manifest(["AAA"])

    assert upstream.calls == 1
    assert first.equals(second)
    assert first["adjusted"].all()
    assert set(first["what_to_show"]) == {"ADJUSTED_LAST"}
    assert first["bar_complete"].all()
    assert manifest["artifact_format"] == "canonical_csv"
    assert manifest["manifest_hash"]
    entry = next(iter(manifest["entries"].values()))
    assert entry["symbol"] == "AAA"
    assert entry["adjusted"] is True
    assert entry["what_to_show"] == "ADJUSTED_LAST"

def _frame_ending(days_ago: int, *, periods: int, base: float = 10.0) -> pd.DataFrame:
    end = pd.Timestamp(datetime.now(timezone.utc).date()) - pd.Timedelta(days=days_ago)
    idx = pd.bdate_range(end=end, periods=periods)
    closes = [base + 0.1 * i for i in range(periods)]
    return pd.DataFrame(
        {
            "open": closes,
            "high": [c * 1.02 for c in closes],
            "low": [c * 0.98 for c in closes],
            "close": closes,
            "volume": [100_000 + 1_000 * i for i in range(periods)],
        },
        index=idx,
    )

class _RecordingUpstream:
    def __init__(self, df: pd.DataFrame) -> None:
        self.df = df
        self.calls: list[tuple[str, str, str]] = []

    def fetch_ohlcv(self, symbol: str, period: str = "2y", interval: str = "1d") -> pd.DataFrame:
        self.calls.append((symbol, period, interval))
        return self.df.copy()

def _seed_cache(
    tmp_path: Path,
    stale: pd.DataFrame,
    *,
    period: str = "2y",
    interval: str = "1d",
) -> None:
    seeder = CachedMarketDataProvider(
        upstream=_RecordingUpstream(stale),
        cache_dir=tmp_path,
    )

```

```

        adjusted=True,
        what_to_show="ADJUSTED_LAST",
    )
    seeder.fetch_ohlcv("AAA", period=period, interval=interval)

def _intraday_frame_ending(
    minutes_ago: int,
    *,
    periods: int,
    freq_minutes: int = 5,
    base: float = 10.0,
) -> pd.DataFrame:
    now = pd.Timestamp(datetime.now(timezone.utc))
    end = now.floor(f"{freq_minutes}min") - pd.Timedelta(minutes=minutes_ago)
    idx = pd.date_range(end=end, periods=periods, freq=f"{freq_minutes}min", tz="UTC")
    closes = [base + 0.01 * i for i in range(periods)]
    return pd.DataFrame(
        {
            "open": closes,
            "high": [c * 1.002 for c in closes],
            "low": [c * 0.998 for c in closes],
            "close": closes,
            "volume": [10_000 + 100 * i for i in range(periods)],
        },
        index=idx,
    )

def test_incremental_refresh_appends_missing_tail(tmp_path: Path) -> None:
    full = _frame_ending(1, periods=60)
    stale = full.iloc[:-7]
    _seed_cache(tmp_path, stale)

    upstream = _RecordingUpstream(full)
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=True,
        incremental_overlapBars=5,
        incremental_min_gap_days=1,
        incremental_max_gap_days=45,
    )
    refreshed = provider.fetch_ohlcv("AAA", period="2y", interval="1d")

    assert len(upstream.calls) == 1
    assert upstream.calls[0][1].endswith("d")
    assert pd.Timestamp(refreshed.index[-1]) == pd.Timestamp(full.index[-1])
    metadata = json.loads((tmp_path / "AAA_1d_2y.metadata.json").read_text())
    assert metadata["refresh_mode"] == "incremental_append"
    assert metadata["rows_appended"] == 7
    assert metadata["incremental_fetch_supported"] is True
    manifest_entry = provider.data_manifest(["AAA"])["entries"]["AAA_1d_2y"]
    assert manifest_entry["refresh_mode"] == "incremental_append"
    assert manifest_entry["incremental_fetch_supported"] is True

def test_incremental_refresh_full_refetch_on_overlap_mismatch(tmp_path: Path) -> None:
    stale = _frame_ending(1, periods=60).iloc[:-7]
    _seed_cache(tmp_path, stale)

    readjusted = _frame_ending(1, periods=60, base=9.0)
    upstream = _RecordingUpstream(readjusted)
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=True,
        incremental_overlapBars=5,
        incremental_min_gap_days=1,
        incremental_max_gap_days=45,
    )
    refreshed = provider.fetch_ohlcv("AAA", period="2y", interval="1d")

    assert [call[1] for call in upstream.calls[-1:]] == ["2y"]
    assert len(upstream.calls) == 2
    assert float(refreshed["close"].iloc[0]) == 9.0
    metadata = json.loads((tmp_path / "AAA_1d_2y.metadata.json").read_text())
    assert metadata["refresh_mode"] == "full_refetch_overlap_mismatch"

def test_incremental_refresh_disabled_keeps_cache_untouched(tmp_path: Path) -> None:
    stale = _frame_ending(10, periods=40)
    _seed_cache(tmp_path, stale)

    upstream = _RecordingUpstream(_frame_ending(1, periods=60))
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=False,
    )

```

```

cached = provider.fetch_ohlcv("AAA", period="2y", interval="1d")

assert upstream.calls == []
assert pd.Timestamp(cached.index[-1]) == pd.Timestamp(stale.index[-1])

def test_incremental_refresh_skips_fresh_cache(tmp_path: Path) -> None:
    fresh = _frame_ending(1, periods=40)
    _seed_cache(tmp_path, fresh)

    upstream = _RecordingUpstream(fresh)
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=True,
        incremental_min_gap_days=1,
    )
    provider.fetch_ohlcv("AAA", period="2y", interval="1d")

    assert upstream.calls == []

def test_intraday_incremental_refresh_appends_missing_tail(tmp_path: Path) -> None:
    full = _intraday_frame_ending(10, periods=120)
    stale = full.iloc[:-12]
    _seed_cache(tmp_path, stale, period="30d", interval="5m")

    upstream = _RecordingUpstream(full)
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=True,
        incremental_overlap_bars=5,
        incremental_intraday_enabled=True,
        incremental_intraday_min_gap_bars=2,
        incremental_intraday_max_gap_days=5,
    )
    refreshed = provider.fetch_ohlcv("AAA", period="30d", interval="5m")

    assert len(upstream.calls) == 1
    assert upstream.calls[0][1].endswith("d")
    assert pd.Timestamp(refreshed.index[-1]) == pd.Timestamp(full.index[-1])
    metadata = json.loads((tmp_path / "AAA_5m_30d.metadata.json").read_text())
    assert metadata["refresh_mode"] == "incremental_append"
    assert metadata["rows_appended"] == 12
    assert metadata["incremental_fetch_supported"] is True

def test_intraday_incremental_refresh_skips_fresh_cache(tmp_path: Path) -> None:
    fresh = _intraday_frame_ending(5, periods=40)
    _seed_cache(tmp_path, fresh, period="30d", interval="5m")

    upstream = _RecordingUpstream(fresh)
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=True,
        incremental_intraday_enabled=True,
        incremental_intraday_min_gap_bars=2,
    )
    provider.fetch_ohlcv("AAA", period="30d", interval="5m")

    assert upstream.calls == []

def test_intraday_incremental_refresh_full_refetch_on_large_gap(tmp_path: Path) -> None:
    stale = _intraday_frame_ending(60 * 24 * 10, periods=40)
    _seed_cache(tmp_path, stale, period="30d", interval="5m")

    upstream = _RecordingUpstream(_intraday_frame_ending(10, periods=120))
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
        incremental_enabled=True,
        incremental_intraday_enabled=True,
        incremental_intraday_max_gap_days=5,
    )
    refreshed = provider.fetch_ohlcv("AAA", period="30d", interval="5m")

    assert [call[1] for call in upstream.calls] == ["30d"]
    assert len(refreshed) == 120
    metadata = json.loads((tmp_path / "AAA_5m_30d.metadata.json").read_text())
    assert metadata["refresh_mode"] == "full_refetch_gap_too_large"

def test_intraday_incremental_refresh_disabled_keeps_cache_untouched(tmp_path: Path) -> None:
    stale = _intraday_frame_ending(60 * 5, periods=40)
    _seed_cache(tmp_path, stale, period="30d", interval="5m")

```

```

upstream = _RecordingUpstream(_intraday_frame_ending(10, periods=120))
provider = CachedMarketDataProvider(
    upstream=upstream,
    cache_dir=tmp_path,
    adjusted=True,
    what_to_show="ADJUSTED_LAST",
    incremental_enabled=True,
    incremental_intraday_enabled=False,
)
cached = provider.fetch_ohlcv("AAA", period="30d", interval="5m")

assert upstream.calls == []
assert pd.Timestamp(cached.index[-1]) == pd.Timestamp(stale.index[-1])

def test_incomplete_intraday_bar_is_not_served_or_persisted(tmp_path: Path) -> None:
    # minutes_ago=0: the last bar starts at floor(now, 5min), so it is still
    # in progress at fetch time while every earlier bar is complete.
    frame = _intraday_frame_ending(0, periods=20)

    upstream = _RecordingUpstream(frame)
    provider = CachedMarketDataProvider(
        upstream=upstream,
        cache_dir=tmp_path,
        adjusted=True,
        what_to_show="ADJUSTED_LAST",
    )
    served = provider.fetch_ohlcv("AAA", period="30d", interval="5m")

    assert len(served) == 19
    assert pd.Timestamp(served.index[-1]) == pd.Timestamp(frame.index[-2])
    metadata = json.loads((tmp_path / "AAA_5m_30d.metadata.json").read_text())
    assert metadata["rows"] == 19

def test_detect_unadjusted_splits_flags_split_like_gap() -> None:
    idx = pd.date_range("2026-01-02", periods=4, freq="B")
    df = pd.DataFrame(
        {
            "open": [100.0, 101.0, 50.0, 51.0],
            "high": [102.0, 103.0, 52.0, 53.0],
            "low": [99.0, 100.0, 49.0, 50.5],
            "close": [101.0, 100.0, 51.0, 52.0],
            "volume": [1_000_000, 1_050_000, 1_100_000, 1_000_000],
        },
        index=idx,
    )

    flagged = detect_unadjusted_splits(df)

    assert flagged == [pd.Timestamp(idx[2]).isoformat()]

def test_universe_snapshot_filters_and_marks_survivorship(tmp_path: Path) -> None:
    settings = Settings(
        universe_snapshot_dir=str(tmp_path / "snapshots"),
        universe_file=str(tmp_path / "universe.csv"),
        universe_point_in_time_available=False,
        min_price=2.0,
        min_avg_dollar_volume=5_000_000,
    )
    universe = pd.DataFrame(
        [
            {"symbol": "aaa", "price": 3.0, "avg_dollar_volume": 6_000_000, "exchange": "NASDAQ"},
            {"symbol": "bbb", "price": 1.0, "avg_dollar_volume": 6_000_000, "exchange": "NASDAQ"},
            {"symbol": "ccc", "price": 8.0, "avg_dollar_volume": 1_000_000, "exchange": "NYSE"},
            {"symbol": "ddd", "price": 8.0, "avg_dollar_volume": 8_000_000, "exchange": "OTC"},
        ]
    )

    snapshot = UniverseSnapshotService(settings).build_monthly_snapshot("2026-06-10", universe=universe)
    df = pd.read_csv(snapshot["path"])

    assert snapshot["snapshot_month"] == "2026-06"
    assert snapshot["row_count"] == 1
    assert snapshot["symbols"] == ["AAA"]
    assert snapshot["survivorship_biased"] is True
    assert df["symbol"].tolist() == ["AAA"]

def test_universe_snapshot_content_hash_is_deterministic(tmp_path: Path) -> None:
    settings = Settings(
        universe_snapshot_dir=str(tmp_path / "snapshots"),
        universe_file=str(tmp_path / "universe.csv"),
        universe_point_in_time_available=False,
        min_price=2.0,
        min_avg_dollar_volume=5_000_000,
    )
    universe = pd.DataFrame(
        [
            {"symbol": "aaa", "price": 3.0, "avg_dollar_volume": 6_000_000, "exchange": "NASDAQ"},
            {"symbol": "zzz", "price": 9.0, "avg_dollar_volume": 9_000_000, "exchange": "NYSE"},
        ]
    )
    service = UniverseSnapshotService(settings)

```

```

first = service.build_monthly_snapshot("2026-06-10", universe=universe)
second = service.build_monthly_snapshot("2026-06-10", universe=universe)

assert first["content_hash"] == second["content_hash"]
assert first["delisting_data_available"] is False
assert first["survivorship_biased"] is True

class FakeIB:
    account_summary_calls = 0

    def __init__(self) -> None:
        self.connected = False

    def connect(self, host: str, port: int, clientId: int, timeout: int) -> None: # noqa: N803
        self.connected = True

    def reqCurrentTime(self) -> datetime: # noqa: N802
        return datetime(2026, 6, 6, tzinfo=timezone.utc)

    def managedAccounts(self) -> list[str]: # noqa: N802
        return ["REDACTED_ACCOUNT"]

    def accountSummary(self, account: str) -> list[SimpleNamespace]: # noqa: N802
        type(self).account_summary_calls += 1
        raise AssertionError("public health check must not request account summary")

    def isConnected(self) -> bool: # noqa: N802
        return self.connected

    def disconnect(self) -> None:
        self.connected = False

def test_ibkr_health_check_omits_account_summary(monkeypatch: pytest.MonkeyPatch) -> None:
    FakeIB.account_summary_calls = 0
    monkeypatch.setitem(sys.modules, "ib_insync", SimpleNamespace(IB=FakeIB))

    status = inspect_ibkr_connection(
        Settings(
            ibkr_host="127.0.0.1",
            ibkr_port=7497,
            ibkr_client_id=17,
            ibkr_readonly=True,
            ibkr_account="REDACTED_ACCOUNT",
        )
    )

    assert status["ok"] is True
    assert status["readonly"] is True
    assert status["live_armed"] is False
    assert status["managed_accounts_count"] == 1
    assert status["selected_account_configured"] is True
    assert status["account_summary_included"] is False
    assert "account_summary" not in status
    assert FakeIB.account_summary_calls == 0

```

Full Source: ``backend/tradeo/tests/test_ibkr_data_provider.py``

``backend/tradeo/tests/test_ibkr_data_provider.py`` ? 28 lines, 839 bytes, sha256 prefix ``e01d3831fc08165f``

Public surface summary before full source:

? class ``DummyIB`` line 9 methods: disconnect

? function ``test_ibkr_provider_sets_event_loop_before_thread_import`` line 14`

Risk hints: IBKR/broker/data dependency.

```

from __future__ import annotations

from concurrent.futures import ThreadPoolExecutor

from tradeo.services import ibkr_data_provider
from tradeo.services.ibkr_data_provider import IBKRHistoricalDataProvider

class DummyIB:
    def disconnect(self) -> None:
        return None

def test_ibkr_provider_sets_event_loop_before_thread_import(monkeypatch) -> None:
    def fake_connect(settings): # noqa: ANN001
        return DummyIB(), 123

    monkeypatch setattr(ibkr_data_provider, "_connect_ibkr", fake_connect)

    def fetch_in_thread() -> None:
        provider = IBKRHistoricalDataProvider()
        try:
            provider.fetch_ohlcv("TMDX", period="3mo", interval="1d")

```

```

except AttributeError:
    pass

with ThreadPoolExecutor(max_workers=1) as executor:
    executor.submit(fetch_in_thread).result()

```

Full Source: `backend/tradeo/tests/test\_audit\_hardening.py`

`backend/tradeo/tests/test\_audit\_hardening.py` ? 1012 lines, 37530 bytes, sha256 prefix `876a224547164ea6`

Public surface summary before full source:

```

? function `test_validation_gate_never_promotes_research_metric_to_paper_candidate` line 18`
? function `test_director_gate_blocks_zero_trade_promotions_and_unproven_oos_fit` line 65`
? function `test_export_filters_fills_to_exported_paper_trade_ids` line 129`
? function `test_export_excludes_shadow_observations_from_paper_trades` line 170`
? function `test_export_marks_real_paper_fills_with_evidence_contract` line 216`
? function `test_export_rejects_closed_paper_trade_without_broker_provenance` line 263`
? function `test_export_leaves_operational_metrics_empty_without_closed_lab_trades` line 303`
? function `test_export_metrics_reconstruct_independent_sample_counts_from_event_rows` line 335`
? function `test_export_groups_closed_lab_trade_performance_by_regime_and_entry_variant` line 351`
? function `test_audit_validator_accepts_manifest_gate_buckets_and_reported_duplicates` line 389`
? function `test_audit_validator_rejects_orphan_fills_and_empty_bucket_without_reason` line 403`
? function `test_audit_validator_rejects_shadow_evidence_as_paper_trade` line 444`
? function `test_audit_validator_requires_global_experiment_scope` line 458`
? function `test_normalize_ohlcw_rejects_invalid_market_bars` line 472`
? function `test_normalize_ohlcw_rejects_duplicate_timestamps` line 488`
? function `test_audit_runner_parses_compose_ps_json_without_container_name_dependency` line 505`

```

Risk hints: subprocess usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

import csv
import importlib.util
import json
import sys
from pathlib import Path

import pandas as pd
import pytest

from tradeo.core.config import Settings
from tradeo.research.types import ClusterCandidate
from tradeo.research.validation_gate import ValidationGate
from tradeo.services.technical_indicators import normalize_ohlcw

def test_validation_gate_never_promotes_research_metric_to_paper_candidate() -> None:
    candidate = ClusterCandidate(
        pattern_key="pattern-key",
        name="DISCOVERED_LONG_W20_C01_TEST",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[0.0, 1.0],
        sample_count=250,
        symbol_count=12,
        year_count=4,
        score=1.2,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "rr_metrics": {"4": {"expectancy_r": 0.55, "profit_factor": 2.5}},
            "best_rr": 4.0,
            "best_expectancy_r": 0.55,
            "best_profit_factor": 2.5,
            "best_max_drawdown_r": 4.0,
            "expectancy_r": 0.55,
            "profit_factor": 2.5,
            "stability_score": 0.75,
            "out_of_sample_expectancy_r": 0.35,
            "out_of_sample_profit_factor": 1.8,
            "hit_4r_rate": 0.15,
            # lab_candidate exige ademas la evidencia del nucleo quant: DSR de
            # familia >= 0.95 e IC95 stationary-bootstrap positivo sobre n_eff.
            "dsr_family": 0.97,
            "dsr_family_n_trials": 500,
            "effective_sample_count": 90.0,
            "quant_validation": {"n_eff": 90.0, "expectancy_ci95_low": 0.08},
        },
        feature_summary={},
        examples=[],
    )

    result = ValidationGate(settings=Settings()).evaluate(candidate)

```

```

assert result.validation_passed is True
assert result.metrics["premium_rr_passed"] is True
assert result.metrics["promotion_status"] == "lab_candidate"
assert result.metrics["promotion_status"] not in {"premium_candidate", "paper_candidate"}
assert result.metrics["execution_promotion_blocked"] is True

def test_director_gate_blocks_zero_trade_promotions_and_unproven_oos_fit() -> None:
    director_gate = _load_director_gate()
    rows = {
        "catalog": [
            {
                "pattern_id": "PATTERN_1",
                "status": "paper_candidate",
                "entry_rule_plaintext": "Research entry is the close of the final bar in the detected window.",
            }
        ],
        "events": [
            {
                "pattern_id": "PATTERN_1",
                "duplicate_group_id": "DUP_1",
                "is_independent_sample": "true",
                "market_regime": "not_persisted",
                "sector": "not_persisted",
            }
        ],
        "trades": [],
        "fills": [],
        "experiments": [
            {
                "experiment_id": "EXP_1",
                "out_of_sample_start": "2025-01-01T00:00:00+00:00",
                "out_of_sample_end": "2025-12-31T00:00:00+00:00",
            }
        ],
        "metrics": [
            {
                "pattern_id": "PATTERN_1",
                "trade_count": "0",
                "independent_sample_count": "1",
                "profit_factor": "2.4",
                "expectancy": "0.5",
                "max_drawdown": "3.0",
            }
        ],
    }

    blockers, summary = director_gate.evaluate(
        rows,
        min_paper_trades_for_promotion=30,
        min_fills_for_promotion=30,
        max_duplicate_event_share=0.0,
    )

    assert summary["director_gate"] == "blocked"
    assert any("paper_trades.csv has zero rows" in blocker for blocker in blockers)
    assert any("promoted statuses require linked paper trades" in blocker for blocker in blockers)
    assert any("research R metrics are populated" in blocker for blocker in blockers)
    assert any("anti-lookahead cutoff columns" in blocker for blocker in blockers)
    assert any("train-only fit evidence" in blocker for blocker in blockers)
    assert summary["by_regime"]["available"] is False
    assert "metrics_by_regime.csv" in summary["by_regime"]["empty_reason"]
    assert summary["by_entry_variant"]["available"] is False
    assert summary["actionable_recommendations"][0]["action"] == "keep_all_patterns_research_only"
    assert summary["pattern_recommendations"][0]["trades_remaining_for_promotion"] == 30
    assert any(
        "entry_variant performance unavailable" in action
        for action in summary["pattern_recommendations"][0]["actions"]
    )

def test_export_filters_fills_to_exported_paper_trade_ids() -> None:
    exporter = _load_audit_exporter()
    overview = {
        "trades": [
            {
                "id": 1,
                "symbol": "AAA",
                "side": "long",
                "qty": 1,
                "entry": 10.0,
                "opened_at": "2026-06-09T10:00:00+00:00",
                "status": "open",
                "metadata_json": {"execution_mode": "paper"},
            },
            {
                "id": 2,
                "symbol": "BBB",
                "side": "long",
                "qty": 1,
                "entry": 20.0,
                "opened_at": "2026-06-09T10:00:00+00:00",
                "status": "closed",
                "metadata_json": {
                    "execution_mode": "paper",
                    "evidence_type": "ibkr_paper_fill",
                },
            },
        ],
    }

```



```

        "evidence_quality": "standard",
        "fill_provenance": "broker_execution",
        "broker_fill_id": "fill-2-entry",
        "broker_execution_time": "2026-06-09T10:01:00+00:00",
        "commission": "1",
    },
],
]
}

fills = exporter.build_ib_fills(overview, exported_trade_ids={"2"})

assert [row["trade_id"] for row in fills] == ["2"]
assert fills[0]["ticker"] == "BBB"

def test_export_excludes_shadow_observations_from_paper_trades() -> None:
    exporter = _load_audit_exporter()
    patterns = [{"id": 1, "name": "SHADOW_PATTERN"}]
    overview = {
        "signals": [
            {
                "id": 10,
                "entry": 100,
                "created_at": "2026-06-09T10:00:00+00:00",
                "risk_usd": 100,
                "metadata_json": {"pattern_id": 1},
            }
        ],
        "trades": [
            {
                "id": 20,
                "signal_id": 10,
                "symbol": "AAA",
                "pattern": "SHADOW_PATTERN",
                "side": "long",
                "qty": 1,
                "entry": 100,
                "opened_at": "2026-06-09T10:00:00+00:00",
                "closed_at": "2026-06-09T11:00:00+00:00",
                "status": "closed",
                "pnl_usd": 10,
                "r_multiple": 1.0,
                "metadata_json": {
                    "execution_mode": "lab_shadow_observation",
                    "evidence_type": "near_miss_shadow",
                    "evidence_quality": "standard",
                    "observation_only": True,
                    "no_ibkr_order": True,
                    "near_miss": True,
                },
            }
        ],
    }

    rows = exporter.build_paper_trades(patterns, overview)
    fills = exporter.build_ib_fills(overview, exported_trade_ids={"20"})

    assert rows == []
    assert fills == []

def test_export_marks_real_paper_fills_with_evidence_contract() -> None:
    exporter = _load_audit_exporter()
    patterns = [{"id": 1, "name": "PAPER_PATTERN"}]
    overview = {
        "signals": [
            {
                "id": 10,
                "entry": 100,
                "created_at": "2026-06-09T10:00:00+00:00",
                "risk_usd": 100,
                "metadata_json": {"pattern_id": 1},
            }
        ],
        "trades": [
            {
                "id": 20,
                "signal_id": 10,
                "symbol": "AAA",
                "pattern": "PAPER_PATTERN",
                "side": "long",
                "qty": 1,
                "entry": 100,
                "opened_at": "2026-06-09T10:00:00+00:00",
                "closed_at": "2026-06-09T11:00:00+00:00",
                "status": "closed",
                "pnl_usd": 10,
                "r_multiple": 1.0,
                "metadata_json": {
                    "execution_mode": "paper",
                    "evidence_type": "ibkr_paper_fill",
                    "evidence_quality": "standard",
                    "fill_provenance": "broker_execution",
                    "broker_fill_id": "fill-20-entry",
                    "broker_execution_time": "2026-06-09T10:01:00+00:00",
                },
            }
        ],
    }

```

```

        "commission": "1",
    },
    ],
}

rows = exporter.build_paper_trades(patterns, overview)

assert len(rows) == 1
assert rows[0]["evidence_type"] == "ibkr_paper_fill"
assert rows[0]["evidence_quality"] == "standard"

def test_export_rejects_closed_paper_trade_without_broker_provenance() -> None:
    exporter = _load_audit_exporter()
    patterns = [{"id": 1, "name": "PAPER_PATTERN"}]
    overview = {
        "signals": [
            {
                "id": 10,
                "entry": 100,
                "created_at": "2026-06-09T10:00:00+00:00",
                "risk_usd": 100,
                "metadata_json": {"pattern_id": 1},
            }
        ],
        "trades": [
            {
                "id": 20,
                "signal_id": 10,
                "symbol": "AAA",
                "pattern": "PAPER_PATTERN",
                "side": "long",
                "qty": 1,
                "entry": 100,
                "opened_at": "2026-06-09T10:00:00+00:00",
                "closed_at": "2026-06-09T11:00:00+00:00",
                "status": "closed",
                "pnl_usd": 10,
                "r_multiple": 1.0,
                "metadata_json": {
                    "execution_mode": "paper",
                    "evidence_type": "ibkr_paper_fill",
                    "evidence_quality": "standard",
                },
            }
        ],
    }

    assert exporter.build_paper_trades(patterns, overview) == []
    assert exporter.build_ib_fills(overview, exported_trade_ids={"20"}) == []

def test_export_leaves_operational_metrics_empty_without_closed_lab_trades() -> None:
    exporter = _load_audit_exporter()
    patterns = [
        {
            "id": 1,
            "sample_count": 120,
            "symbol_count": 10,
            "best_win_rate": 0.62,
            "best_profit_factor": 2.4,
            "best_expectancy_r": 0.5,
            "best_max_drawdown_r": 3.0,
            "rr_metrics_json": {"4": {"avg_win_r": 2.0, "avg_loss_r": 1.0, "median_result_r": 0.5}},
            "best_rr": 4.0,
        }
    ]

    rows = exporter.build_metrics_by_pattern(patterns, {1: []}, [])
    regime_rows = exporter.build_metrics_by_regime(patterns, [], [])
    variant_rows = exporter.build_metrics_by_entry_variant(patterns, [])

    assert rows[0]["trade_count"] == 0
    assert rows[0]["winrate"] == ""
    assert rows[0]["profit_factor"] == ""
    assert rows[0]["expectancy"] == ""
    assert rows[0]["max_drawdown"] == ""
    assert "No closed_lab_trades" in rows[0]["notes"]
    assert regime_rows[0]["analysis_available"] == "false"
    assert "no_closed_lab_trades" in regime_rows[0]["empty_reason"]
    assert variant_rows[0]["analysis_available"] == "false"
    assert "entry_variant_id" in variant_rows[0]["empty_reason"]

def test_export_metrics_reconstruct_independent_sample_counts_from_event_rows() -> None:
    exporter = _load_audit_exporter()
    patterns = [{"id": 1, "sample_count": 120, "symbol_count": 10}]
    events = [
        {"pattern_id": "PATTERN_000001", "is_independent_sample": "true"},
        {"pattern_id": "PATTERN_000001", "is_independent_sample": "false"},
        {"pattern_id": "PATTERN_000001", "is_independent_sample": "pending_review"},
    ]

    rows = exporter.build_metrics_by_pattern(patterns, {1: []}, [], event_rows=events)

```

```

assert rows[0]["sample_count"] == 3
assert rows[0]["independent_sample_count"] == 1
assert "raw pattern sample_count=120" in rows[0]["notes"]

def test_export_groups_closed_lab_trade_performance_by_regime_and_entry_variant() -> None:
    exporter = _load_audit_exporter()
    patterns = [{"id": 1}]
    trades = [
        {
            "pattern_id": "PATTERN_000001",
            "gross_pnl": "10",
            "net_pnl": "9",
            "r_multiple": "1.0",
            "entry_fill_time": "2026-06-09T10:00:00+00:00",
            "notes": "entry_variant=next_bar_limit_retest; regime=market_up; execution_mode=paper",
        },
        {
            "pattern_id": "PATTERN_000001",
            "gross_pnl": "-5",
            "net_pnl": "-5",
            "r_multiple": "-0.5",
            "entry_fill_time": "2026-06-10T10:00:00+00:00",
            "notes": "entry_variant=momentum_close; regime=market_down; execution_mode=paper",
        },
    ]
    events = [
        {"pattern_id": "PATTERN_000001", "market_regime": "market_up"},
        {"pattern_id": "PATTERN_000001", "market_regime": "market_down"},
    ]

    regime_rows = exporter.build_metrics_by_regime(patterns, events, trades)
    variant_rows = exporter.build_metrics_by_entry_variant(patterns, trades)

    up = next(row for row in regime_rows if row["market_regime"] == "market_up")
    retest = next(row for row in variant_rows if row["entry_variant_id"] == "next_bar_limit_retest")
    assert up["analysis_available"] == "true"
    assert up["trade_count"] == 1
    assert up["expectancy_r"] == 1.0
    assert retest["net_pnl"] == 9.0
    assert retest["profit_factor"] == 9.0

def test_audit_validator_accepts_manifest_gate_buckets_and_reported_duplicates(tmp_path: Path) -> None:
    validator = _load_audit_validator()
    package = _write_minimal_audit_package(tmp_path, validator, duplicate_repeated_rows=2)

    errors, rows = validator.validate_package(package)
    report = validator.validation_report(package, errors, rows)

    assert errors == []
    assert report["schema_valid"] is True
    assert report["package_valid"] is True
    assert report["director_gate_status"] == "blocked"
    assert report["promotion_allowed"] is False

def test_audit_validator_rejects_orphan_fills_and_empty_bucket_without_reason(tmp_path: Path) -> None:
    validator = _load_audit_validator()
    package = _write_minimal_audit_package(tmp_path, validator, duplicate_repeated_rows=2)
    _write_csv(
        package / "ib_fills.csv",
        validator.CSV_COLUMNS["ib_fills.csv"],
        [
            {
                "fill_id_hash": "fill_hash",
                "trade_id": "MISSING",
                "order_id_hash": "12345",
                "ib_execution_time": "2026-06-09T10:01:00+00:00",
                "timezone": "UTC",
                "ticker": "AAA",
                "side": "long",
                "quantity": "1",
                "price": "10",
                "commission": "1",
                "liquidity_flag": "",
                "exchange": "SMART/IBKR",
                "order_type": "limit",
                "account_id_redacted": "false",
                "raw_status": "closed",
                "notes": "",
            },
        ],
    )
    _write_csv(
        package / "metrics_by_regime.csv",
        validator.CSV_COLUMNS["metrics_by_regime.csv"],
        [{"analysis_available": "false", "empty_reason": ""}],
    )

    errors, _ = validator.validate_package(package)

    assert any("unknown trade_id MISSING" in error for error in errors)
    assert any("order_id_hash appears to contain raw numeric order id" in error for error in errors)
    assert any("account_id_redacted must be true" in error for error in errors)
    assert any("metrics_by_regime.csv: empty breakdown requires non-empty empty_reason" in error for error in errors)

```

```

def test_audit_validator_rejects_shadow_evidence_as_paper_trade(tmp_path: Path) -> None:
    validator = _load_audit_validator()
    package = _write_minimal_audit_package(tmp_path, validator, duplicate_repeated_rows=2)
    rows = list(csv.DictReader((package / "paper_trades.csv").open(newline="", encoding="utf-8")))
    rows[0]["evidence_type"] = "near_miss_shadow"
    rows[0]["notes"] = "execution_mode=lab_shadow_observation; no_ibkr_order=true"
    _write_csv(package / "paper_trades.csv", validator.CSV_COLUMNS["paper_trades.csv"], rows)

    errors, _ = validator.validate_package(package)

    assert any("evidence_type must be ibkr_paper_fill" in error for error in errors)
    assert any("shadow/no-broker evidence cannot be exported as paper trade" in error for error in errors)

def test_audit_validator_requires_global_experiment_scope(tmp_path: Path) -> None:
    validator = _load_audit_validator()
    package = _write_minimal_audit_package(tmp_path, validator, duplicate_repeated_rows=2)
    rows = list(csv.DictReader((package / "experiment_registry.csv").open(newline="", encoding="utf-8")))
    rows[0]["global_trial_count"] = ""
    rows[0]["fit_scope"] = ""
    _write_csv(package / "experiment_registry.csv", validator.CSV_COLUMNS["experiment_registry.csv"], rows)

    errors, _ = validator.validate_package(package)

    assert any("global_trial_count is required" in error for error in errors)
    assert any("fit_scope is required" in error for error in errors)

def test_normalize_ohlc rejects_invalid_market_bars() -> None:
    df = pd.DataFrame(
        {
            "open": [10.0],
            "high": [9.0],
            "low": [8.0],
            "close": [10.5],
            "volume": [1000.0],
        },
        index=pd.date_range("2026-01-01", periods=1, freq="D"),
    )

    with pytest.raises(ValueError, match="high must be greater"):
        normalize_ohlc(df)

def test_normalize_ohlc rejects_duplicate_timestamps() -> None:
    idx = pd.to_datetime(["2026-01-01", "2026-01-01"])
    df = pd.DataFrame(
        {
            "open": [10.0, 11.0],
            "high": [11.0, 12.0],
            "low": [9.0, 10.0],
            "close": [10.5, 11.5],
            "volume": [1000.0, 1200.0],
        },
        index=idx,
    )

    with pytest.raises(ValueError, match="duplicate timestamps"):
        normalize_ohlc(df)

def test_audit_runner_parses_compose_ps_json_without_container_name_dependency() -> None:
    runner = _load_audit_runner()
    stdout = "\n".join(
        [
            json.dumps(
                {
                    "Name": "tradeo-backend-1",
                    "Service": "backend",
                    "State": "running",
                    "Health": "healthy",
                }
            ),
            json.dumps(
                {
                    "Name": "tradeo-worker-1",
                    "Service": "worker",
                    "State": "running",
                    "Status": "Up 10 minutes (healthy)",
                }
            ),
        ]
    )

    services = runner.parse_compose_ps_json(stdout)

    assert [service["service"] for service in services] == ["backend", "worker"]
    assert [service["health"] for service in services] == ["healthy", "healthy"]

def test_audit_runner_ignores_nonblocking_compose_ps_failures() -> None:
    runner = _load_audit_runner()

    review = runner.deterministic_review(

```

```

        "TEST_AUDIT",
        "manual",
        {"status": "passed", "blockers": []},
        [runner.CommandRun("docker_compose_ps", ["docker", "compose", "ps"], 1, blocking=False)],
    )

    assert review["failed_commands"] == []
    assert review["priority"] == "P2"

def test_audit_runner_writes_artifacts_when_gate_command_raises(
    tmp_path: Path,
    monkeypatch: pytest.MonkeyPatch,
) -> None:
    runner = _load_audit_runner()
    requests = tmp_path / "requests"
    monkeypatch.setattr(runner, "REQUESTS", requests)
    monkeypatch.setattr(
        runner,
        "collect_compose_status",
        lambda: (
            runner.CommandRun("docker_compose_ps", ["docker", "compose", "ps"], 1, blocking=False),
            {"available": False, "reason": "docker_compose_ps_unavailable"},
        ),
    )

    def raise_from_gate(*args, **kwargs):
        raise RuntimeError("gate exploded")

    monkeypatch.setattr(runner, "run_subprocess", raise_from_gate)
    monkeypatch.setattr(
        runner.sys,
        "argv",
        [
            "run_director_audit.py",
            "--cadence",
            "manual",
            "--audit-id",
            "TEST_AUDIT",
            "--skip-export",
        ],
    )

    exit_code = runner.main()

    run_json = requests / "TEST_AUDIT" / "director_audit_run.json"
    review_json = requests / "TEST_AUDIT" / "internal_auditor_agent_review.json"
    assert exit_code == 1
    assert run_json.exists()
    assert review_json.exists()
    payload = json.loads(run_json.read_text(encoding="utf-8"))
    assert payload["director_gate_status"] == "runner_error"
    assert payload["commands"][-1]["name"] == "runner_error"
    assert payload["artifact_paths"][-1].endswith("director_audit_run.json")

def _write_minimal_audit_package(tmp_path: Path, validator, *, duplicate_repeated_rows: int) -> Path:
    bridge = tmp_path / "research" / "audit_bridge"
    package = bridge / "requests" / "TEST_AUDIT"
    (package / "config_snapshot").mkdir(parents=True)
    for rel in validator.REQUIRED_BRIDGE_FILES:
        (bridge / rel).parent.mkdir(parents=True, exist_ok=True)
        (bridge / rel).write_text("ok\n", encoding="utf-8")
    for rel in validator.REQUIRED_FILES:
        path = package / rel
        path.parent.mkdir(parents=True, exist_ok=True)
        if rel.endswith(".csv") or rel in {"manifest.json", "director_gate_result.json"}:
            continue
        if rel.endswith(".json"):
            path.write_text(json.dumps({"redacted": True}), encoding="utf-8")
        else:
            path.write_text("ok\n", encoding="utf-8")

    pattern = {
        "pattern_id": "PATTERN_000001",
        "pattern_name": "TEST_PATTERN",
        "pattern_version": "run_1",
        "description": "test",
        "hypothesis": "test",
        "detection_rule_plaintext": "test",
        "entry_rule_plaintext": "test",
        "exit_rule_plaintext": "test",
        "stop_rule_plaintext": "test",
        "take_profit_rule_plaintext": "test",
        "time_stop_rule_plaintext": "test",
        "timeframe": "1d",
        "asset_class": "US equities",
        "universe": "test",
        "session_filter": "regular",
        "min_volume_filter": "0",
        "min_price_filter": "0",
        "uses_fundamental_data": "false",
        "uses_news_data": "false",
        "uses_options_data": "false",
        "uses_future_data": "false",
        "known_risks": "",
    }

```

```

    "status": "lab_candidate",
    "created_at": "2026-06-09T00:00:00+00:00",
    "first_seen": "2026-06-09T00:00:00+00:00",
    "last_seen": "2026-06-10T00:00:00+00:00",
}
events = [
    {
        "event_id": "EVT_1",
        "pattern_id": "PATTERN_000001",
        "detected_at": "2026-06-09T00:00:00+00:00",
        "market_timestamp": "2026-06-09T00:00:00+00:00",
        "timezone": "UTC",
        "ticker": "AAA",
        "exchange": "SMART/IBKR",
        "timeframe": "1d",
        "bar_open_time": "2026-06-09T00:00:00+00:00",
        "bar_close_time": "2026-06-09T00:00:00+00:00",
        "signal_price": "10",
        "bid_at_signal": "",
        "ask_at_signal": "",
        "spread_at_signal": "",
        "volume_at_signal": "",
        "atr_at_signal": "",
        "volatility_context": "",
        "market_regime": "market_up",
        "sector": "sector_strong",
        "was_trade_triggered": "true",
        "trade_id": "T1",
        "reason_not_traded": "",
        "duplicate_group_id": "DUP_1",
        "is_independent_sample": "true",
        "data_available_at_signal": "true",
        "available_data_cutoff_ts": "2026-06-09T00:00:00+00:00",
        "decision_ts": "2026-06-09T00:00:00+00:00",
        "entry_eligible_ts": "2026-06-10T00:00:00+00:00",
        "label_generated_ts": "2026-06-10T00:00:00+00:00",
        "source_bar_hash": "source_hash_1",
        "split_id": "test_split",
        "features_used_json": "{}",
        "notes": "",
    },
    {
        "event_id": "EVT_2",
        "pattern_id": "PATTERN_000001",
        "detected_at": "2026-06-10T00:00:00+00:00",
        "market_timestamp": "2026-06-10T00:00:00+00:00",
        "timezone": "UTC",
        "ticker": "AAA",
        "exchange": "SMART/IBKR",
        "timeframe": "1d",
        "bar_open_time": "2026-06-10T00:00:00+00:00",
        "bar_close_time": "2026-06-10T00:00:00+00:00",
        "signal_price": "10",
        "bid_at_signal": "",
        "ask_at_signal": "",
        "spread_at_signal": "",
        "volume_at_signal": "",
        "atr_at_signal": "",
        "volatility_context": "",
        "market_regime": "market_up",
        "sector": "sector_strong",
        "was_trade_triggered": "false",
        "trade_id": "",
        "reason_not_traded": "",
        "duplicate_group_id": "DUP_1",
        "is_independent_sample": "true",
        "data_available_at_signal": "true",
        "available_data_cutoff_ts": "2026-06-10T00:00:00+00:00",
        "decision_ts": "2026-06-10T00:00:00+00:00",
        "entry_eligible_ts": "2026-06-11T00:00:00+00:00",
        "label_generated_ts": "2026-06-11T00:00:00+00:00",
        "source_bar_hash": "source_hash_2",
        "split_id": "test_split",
        "features_used_json": "{}",
        "notes": "",
    },
]
trade = {
    "trade_id": "T1",
    "event_id": "EVT_1",
    "pattern_id": "PATTERN_000001",
    "evidence_type": "ibkr_paper_fill",
    "evidence_quality": "standard",
    "ticker": "AAA",
    "side": "long",
    "quantity": "1",
    "entry_signal_time": "2026-06-09T10:00:00+00:00",
    "entry_order_time": "2026-06-09T10:00:00+00:00",
    "entry_fill_time": "2026-06-09T10:01:00+00:00",
    "entry_signal_price": "10",
    "entry_order_type": "limit",
    "entry_limit_price": "10",
    "entry_fill_price": "10",
    "exit_signal_time": "2026-06-09T11:00:00+00:00",
    "exit_order_time": "2026-06-09T11:00:00+00:00",
    "exit_fill_time": "2026-06-09T11:01:00+00:00",
}

```

```

"exit_order_type": "limit",
"exit_limit_price": "12",
"exit_fill_price": "12",
"exit_reason": "target",
"gross_pnl": "10",
"commission": "1",
"estimated_spread_cost": "1",
"estimated_slippage": "1",
"other_fees": "0",
"net_pnl": "7",
"return_pct": "0.7",
"mae": "",
"mfe": "",
"holding_period_seconds": "3600",
"risk_amount": "10",
"r_multiple": "1",
"account_equity_snapshot": "",
"position_size_method": "test",
"notes": "entry_variant=next_bar_limit_retest; regime=market_up; execution_mode=paper",
}
fill = {
  "fill_id_hash": "fill_hash",
  "trade_id": "T1",
  "order_id_hash": "hash_order",
  "ib_execution_time": "2026-06-09T10:01:00+00:00",
  "timezone": "UTC",
  "ticker": "AAA",
  "side": "long",
  "quantity": "1",
  "price": "10",
  "commission": "1",
  "liquidity_flag": "",
  "exchange": "SMART/IBKR",
  "order_type": "limit",
  "account_id_redacted": "true",
  "raw_status": "closed",
  "notes": "",
}
experiment = {
  "experiment_id": "EXP_1",
  "pattern_id": "PATTERN_000001",
  "variant_id": "RR_4",
  "created_at": "2026-06-09T00:00:00+00:00",
  "tested_at": "2026-06-09T00:00:00+00:00",
  "status": "active",
  "reason_status": "",
  "parameters_json": "{}",
  "dataset_start": "2026-06-09T00:00:00+00:00",
  "dataset_end": "2026-06-10T00:00:00+00:00",
  "in_sample_start": "2026-06-09T00:00:00+00:00",
  "in_sample_end": "2026-06-09T00:00:00+00:00",
  "out_of_sample_start": "2026-06-10T00:00:00+00:00",
  "out_of_sample_end": "2026-06-10T00:00:00+00:00",
  "paper_live_start": "2026-06-09T10:00:00+00:00",
  "paper_live_end": "2026-06-09T11:00:00+00:00",
  "number_of_assets_tested": "1",
  "number_of_events": "2",
  "number_of_trades": "1",
  "gross_pnl": "10",
  "net_pnl": "7",
  "winrate": "1",
  "profit_factor": "7",
  "sharpe": "",
  "sortino": "",
  "max_drawdown": "0",
  "candidate_trial_count": "12",
  "global_trial_count": "24",
  "adjusted_p_value": "0.08",
  "wrc_p_value": "0.2",
  "spa_p_value": "0.18",
  "fit_scope": "{\"scaler\":\"train_only\"}",
  "score_input_scope": "{\"descriptive_all_fields_used\":false}",
  "event_ledger_hash": "event-ledger-hash",
  "nested_discovery_replay_status": "blocked_contract",
  "nested_discovery_replay_implemented": "false",
  "nested_discovery_replay_passed": "false",
  "edge_claim": "NO_DEMONSTRADO",
  "drift_status": "stable",
  "active_blockers": "",
  "registry_path": "reports/research/global_experiment_registry.json",
  "registry_hash": "registry-hash",
  "registry_previous_hash": "previous-registry-hash",
  "registry_run_manifest_hash": "run-manifest-hash",
  "registry_hash_chain_valid": "true",
  "notes": "",
}
metric = {
  "pattern_id": "PATTERN_000001",
  "sample_count": "2",
  "independent_sample_count": "2",
  "trade_count": "1",
  "ticker_count": "1",
  "sector_count": "1",
  "first_seen": "2026-06-09T00:00:00+00:00",
  "last_seen": "2026-06-10T00:00:00+00:00",
  "gross_pnl": "10",

```

```

    "net_pnl": "7",
    "avg_trade": "7",
    "median_trade": "7",
    "std_trade": "",
    "winrate": "1",
    "avg_win": "7",
    "avg_loss": "",
    "payoff_ratio": "",
    "profit_factor": "7",
    "expectancy": "1",
    "max_drawdown": "0",
    "max_consecutive_losses": "0",
    "best_trade": "7",
    "worst_trade": "7",
    "top_5_trades_pnl": "7",
    "bottom_5_trades_pnl": "7",
    "pnl_without_best_trade": "0",
    "pnl_without_top_5_trades": "0",
    "pnl_without_worst_trade": "0",
    "sharpe": "",
    "sortino": "",
    "calmar": "",
    "avg_holding_period_seconds": "3600",
    "notes": "",
}
ticker_metric = {
    "pattern_id": "PATTERN_000001",
    "ticker": "AAA",
    "trade_count": "1",
    "event_count": "2",
    "net_pnl": "7",
    "winrate": "1",
    "profit_factor": "7",
    "avg_trade": "7",
    "max_drawdown": "0",
    "first_seen": "2026-06-09T00:00:00+00:00",
    "last_seen": "2026-06-10T00:00:00+00:00",
    "notes": "",
}
period_metric = {
    "period_start": "2026-06-09T00:00:00+00:00",
    "period_end": "2026-06-09T00:00:00+00:00",
    "period_type": "day",
    "pattern_id": "PATTERN_000001",
    "event_count": "2",
    "trade_count": "1",
    "gross_pnl": "10",
    "net_pnl": "7",
    "winrate": "1",
    "profit_factor": "7",
    "max_drawdown": "0",
    "market_regime": "market_up",
    "notes": "",
}
regime_metric = {
    "pattern_id": "PATTERN_000001",
    "market_regime": "market_up",
    "analysis_available": "true",
    "empty_reason": "",
    "trade_count": "1",
    "event_count": "2",
    "gross_pnl": "10",
    "net_pnl": "7",
    "avg_trade": "7",
    "expectancy_r": "1",
    "winrate": "1",
    "profit_factor": "7",
    "max_drawdown": "0",
    "best_trade": "7",
    "worst_trade": "7",
    "first_seen": "2026-06-09T10:00:00+00:00",
    "last_seen": "2026-06-09T11:00:00+00:00",
    "notes": "",
}
variant_metric = dict(regime_metric)
variant_metric.pop("market_regime")
variant_metric["entry_variant_id"] = "next_bar_limit_retest"

_write_csv(package / "pattern_catalog.csv", validator.CSV_COLUMNS["pattern_catalog.csv"], [pattern])
_write_csv(package / "pattern_events.csv", validator.CSV_COLUMNS["pattern_events.csv"], events)
_write_csv(package / "paper_trades.csv", validator.CSV_COLUMNS["paper_trades.csv"], [trade])
_write_csv(package / "ib_fills.csv", validator.CSV_COLUMNS["ib_fills.csv"], [fill])
_write_csv(package / "experiment_registry.csv", validator.CSV_COLUMNS["experiment_registry.csv"], [experiment])
_write_csv(package / "metrics_by_pattern.csv", validator.CSV_COLUMNS["metrics_by_pattern.csv"], [metric])
_write_csv(package / "metrics_by_ticker.csv", validator.CSV_COLUMNS["metrics_by_ticker.csv"], [ticker_metric])
_write_csv(package / "metrics_by_period.csv", validator.CSV_COLUMNS["metrics_by_period.csv"], [period_metric])
_write_csv(package / "metrics_by_regime.csv", validator.CSV_COLUMNS["metrics_by_regime.csv"], [regime_metric])
_write_csv(package / "metrics_by_entry_variant.csv", validator.CSV_COLUMNS["metrics_by_entry_variant.csv"], [variant_metric])

manifest_paths = sorted(set(validator.REQUIRED_FILES) - {"director_gate_result.json"})
manifest = {
    "audit_id": "TEST_AUDIT",
    "created_at": "2026-06-09T00:00:00+00:00",
    "created_by": "test",
    "repo_commit": "abc",
    "repo_branch": "main",

```



```

        "patterns_detected": 1,
        "total_pattern_events": 2,
        "total_paper_trades": 1,
        "total_ib_fills": 1,
        "total_experiment_variants": 1,
        "total_metrics_by_regime_rows": 1,
        "total_metrics_by_entry_variant_rows": 1,
        "duplicate_event_groups": 1 if duplicate_repeated_rows else 0,
        "duplicate_repeated_rows": duplicate_repeated_rows,
        "metric_breakdowns": {
            "by_regime": {"available": True, "rows": 1, "empty_reason": ""},
            "by_entry_variant": {"available": True, "rows": 1, "empty_reason": ""},
        },
        "director_gate_required": True,
        "contains_sensitive_data": False,
        "account_ids_redacted": True,
        "orders_anonymized": True,
        "files": [{"path": rel, "description": "test"} for rel in manifest_paths],
    }
    (package / "manifest.json").write_text(json.dumps(manifest), encoding="utf-8")
    gate = {
        "status": "blocked",
        "schema_valid": True,
        "package_valid": True,
        "director_gate_status": "blocked",
        "promotion_allowed": False,
        "summary": {
            "director_gate": "blocked",
            "director_gate_status": "blocked",
            "schema_valid": True,
            "package_valid": True,
            "promotion_allowed": False,
            "by_regime": {"available": True, "buckets": [{"pattern_id": "PATTERN_000001"}], "empty_reason": ""},
            "by_entry_variant": {"available": True, "buckets": [{"pattern_id": "PATTERN_000001"}], "empty_reason": ""},
        },
    },
    (package / "director_gate_result.json").write_text(json.dumps(gate), encoding="utf-8")
    return package

def _write_csv(path: Path, columns: list[str], rows: list[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=columns, extrasaction="ignore")
        writer.writeheader()
        for row in rows:
            writer.writerow({column: row.get(column, "") for column in columns})

def _load_director_gate():
    path = _repo_root() / "research" / "audit_bridge" / "director_gate.py"
    spec = importlib.util.spec_from_file_location("director_gate", path)
    assert spec and spec.loader
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

def _load_audit_exporter():
    path = _repo_root() / "research" / "audit_bridge" / "export_audit_package.py"
    spec = importlib.util.spec_from_file_location("export_audit_package", path)
    assert spec and spec.loader
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

def _load_audit_validator():
    path = _repo_root() / "research" / "audit_bridge" / "validate_audit_package.py"
    spec = importlib.util.spec_from_file_location("validate_audit_package", path)
    assert spec and spec.loader
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

def _load_audit_runner():
    path = _repo_root() / "research" / "audit_bridge" / "run_director_audit.py"
    spec = importlib.util.spec_from_file_location("run_director_audit", path)
    assert spec and spec.loader
    module = importlib.util.module_from_spec(spec)
    sys.modules[spec.name] = module
    spec.loader.exec_module(module)
    return module

def _repo_root() -> Path:
    for parent in Path(__file__).resolve().parents:
        if (parent / "research" / "audit_bridge").exists():
            return parent
    raise AssertionError("could not locate repo root")

```

Full Source: `backend/tradeo/tests/test\_quant\_validation.py`

`backend/tradeo/tests/test\_quant\_validation.py` ? 346 lines, 11291 bytes, sha256 prefix `86215912bff5ad99`

Public surface summary before full source:

```
? function `test_both_barriers_same_bar_resolves_to_stop` line 43`
? function `test_gap_open_through_stop_fills_at_open_worse_than_stop` line 56`
? function `test_gap_open_through_target_fills_at_target_not_open` line 68`
? function `test_entry_bar_gap_through_stop_is_skipped` line 81`
? function `test_entry_bar_gap_past_target_is_skipped` line 92`
? function `test_time_stop_exits_at_close_of_last_bar` line 103`
? function `test_short_side_is_symmetric` line 117`
? function `test_mfe_mae_are_recorded_in_r` line 134`
? function `test_round_trip_cost_reduces_r` line 145`
? function `test_select_nonoverlapping_keeps_disjoint_events` line 167`
? function `test_select_nonoverlapping_drops_overlaps` line 172`
? function `test_average_uniqueness_disjoint_events_have_full_weight` line 182`
? function `test_average_uniqueness_collapses_replicated_events` line 188`
? function `test_stationary_bootstrap_ci_brackets_the_mean` line 200`
? function `test_newey_west_tstat_detects_strong_mean` line 209`
? function `test_profit_factor_weighted` line 217`
```

Risk hints: RNG path; verify seed/control.

```
from __future__ import annotations

import numpy as np

from tradeo.research.quant_validation import (
    average_uniqueness_weights,
    benjamini_hochberg,
    cusum_drift,
    deflated_sharpe_ratio,
    expected_max_sharpe,
    newey_west_tstat,
    permutation_pvalue,
    probabilistic_sharpe_ratio,
    profit_factor,
    purged_walk_forward,
    sample_skew_kurt,
    select_nonoverlapping_events,
    stationary_bootstrap_ci,
    summarize_pattern_validation,
    triple_barrier_outcome,
)

# -----
# Golden tests del etiquetador triple-barrera (informe §3.5 / §6)
# -----

# Convención de las series: la barra 0 es la barra de señal; la entrada se
# ejecuta en el open de la barra 1.

def _long_outcome(opens, highs, lows, closes, **kwargs):
    params = {
        "signal_index": 0,
        "side": +1,
        "stop_price": 95.0,
        "target_price": 110.0,
        "max_bars": len(opens) - 1,
    }
    params.update(kwargs)
    return triple_barrier_outcome(opens, highs, lows, closes, **params)

def test_both_barriers_same_bar_resolves_to_stop() -> None:
    out = _long_outcome(
        opens=[100, 100],
        highs=[100, 112],
        lows=[100, 94],
        closes=[100, 100],
    )
    assert out["status"] == "ok"
    assert out["reason"] == "stop_and_target_conservative"
    assert out["exit_price"] == 95.0
    assert out["R"] == (95.0 - 100.0) / (100.0 - 95.0)

def test_gap_open_through_stop_fills_at_open_worse_than_stop() -> None:
    out = _long_outcome(
        opens=[100, 100, 90],
        highs=[100, 101, 91],
        lows=[100, 99, 89],
        closes=[100, 100, 90],
    )
    assert out["reason"] == "stop_gap"
    assert out["exit_price"] == 90.0
    assert out["R"] < -1.0 # peor que el stop: el gap no respeta el precio del stop
```

```
def test_gap_open_through_target_fills_at_target_not_open() -> None:
    out = _long_outcome(
        opens=[100, 100, 115],
        highs=[100, 101, 116],
        lows=[100, 99, 114],
        closes=[100, 100, 115],
    )
    assert out["reason"] == "target_gap"
    # Una limit en el target no cobra el regalo del gap.
    assert out["exit_price"] == 110.0
    assert out["R"] == (110.0 - 100.0) / (100.0 - 95.0)
```

```
def test_entry_bar_gap_through_stop_is_skipped() -> None:
    out = _long_outcome(
        opens=[100, 94],
        highs=[100, 95],
        lows=[100, 93],
        closes=[100, 94],
    )
    assert out["status"] == "skipped"
    assert out["reason"] == "gapped_through_stop"
```

```
def test_entry_bar_gap_past_target_is_skipped() -> None:
    out = _long_outcome(
        opens=[100, 111],
        highs=[100, 112],
        lows=[100, 110.5],
        closes=[100, 111],
    )
    assert out["status"] == "skipped"
    assert out["reason"] == "gapped_past_target"
```

```
def test_time_stop_exits_at_close_of_last_bar() -> None:
    out = _long_outcome(
        opens=[100, 100, 101, 102],
        highs=[100, 102, 103, 104],
        lows=[100, 99, 100, 101],
        closes=[100, 101, 102, 103],
        maxBars=3,
    )
    assert out["reason"] == "time"
    assert out["exit_price"] == 103.0
    assert out["bars_held"] == 3
    assert out["R"] == (103.0 - 100.0) / (100.0 - 95.0)
```

```
def test_short_side_is_symmetric() -> None:
    out = triple_barrier_outcome(
        open=[100, 100, 100],
        high=[100, 101, 101],
        low=[100, 99, 89],
        close=[100, 100, 90],
        signal_index=0,
        side=-1,
        stop_price=105.0,
        target_price=90.0,
        maxBars=2,
    )
    assert out["reason"] == "target"
    assert out["exit_price"] == 90.0
    assert out["R"] == (100.0 - 90.0) / (105.0 - 100.0)
```

```
def test_mfe_mae_are_recorded_in_r() -> None:
    out = _long_outcome(
        opens=[100, 100, 100],
        highs=[100, 104, 100],
        lows=[100, 97, 94],
        closes=[100, 100, 95],
    )
    assert out["mfe_R"] >= (104.0 - 100.0) / 5.0 - 1e-9
    assert out["mae_R"] <= (94.0 - 100.0) / 5.0 + 1e-9
```

```
def test_round_trip_cost_reduces_r() -> None:
    gross = _long_outcome(
        opens=[100, 100, 111],
        highs=[100, 101, 112],
        lows=[100, 99, 110],
        closes=[100, 100, 111],
    )
    net = _long_outcome(
        opens=[100, 100, 111],
        highs=[100, 101, 112],
        lows=[100, 99, 110],
        closes=[100, 100, 111],
        round_trip_cost_R=0.13,
    )
    assert net["R"] == gross["R"] - 0.13
```

```
# -----
# Dedup y n_eff (informe §2.2, §3.6 pasos 1-2)
```

```

# -----

def test_select_nonoverlapping_keeps_disjoint_events() -> None:
    keep = select_nonoverlapping_events([0, 10, 20], [5, 15, 25])
    assert keep.tolist() == [0, 1, 2]

def test_select_nonoverlapping_drops_overlaps() -> None:
    # Eventos solapados cada 3 barras con span de 10: comportamiento del stride.
    starts = list(range(0, 30, 3))
    ends = [s + 9 for s in starts]
    keep = select_nonoverlapping_events(starts, ends)
    kept_spans = [(starts[i], ends[i]) for i in keep.tolist()]
    for (s1, e1), (s2, e2) in zip(kept_spans, kept_spans[1:], strict=False):
        assert s2 > e1 # nunca dos eventos del mismo símbolo solapados

def test_average_uniqueness_disjoint_events_have_full_weight() -> None:
    weights, n_eff = average_uniqueness_weights([0, 20, 40], [9, 29, 49])
    assert np.allclose(weights, 1.0)
    assert n_eff == 3.0

def test_average_uniqueness_collapses_replicated_events() -> None:
    # 5 eventos idénticos: la información real es la de UNO.
    weights, n_eff = average_uniqueness_weights([0] * 5, [9] * 5)
    assert np.allclose(weights, 0.2)
    assert abs(n_eff - 1.0) < 1e-9

# -----
# Inferencia: bootstrap estacionario, Newey-West, permutación, FDR, DSR
# -----

def test_stationary_bootstrap_ci_brackets_the_mean() -> None:
    rng = np.random.default_rng(7)
    x = rng.normal(0.5, 1.0, 400)
    lo, hi, point, dist = stationary_bootstrap_ci(x, np.mean, n_boot=400, mean_block=5, rng=11)
    assert lo < point < hi
    assert lo > 0.0 # señal clara: media 0.5 con n=400
    assert len(dist) == 400

def test_newey_west_tstat_detects_strong_mean() -> None:
    rng = np.random.default_rng(3)
    x = rng.normal(1.0, 0.5, 200)
    t, se = newey_west_tstat(x, lags=5)
    assert t > 5.0
    assert se > 0.0

def test_profit_factor_weighted() -> None:
    r = np.asarray([1.0, 1.0, -1.0])
    assert profit_factor(r) == 2.0
    # El peso reduce la contribución de una ganancia pseudo-replicada.
    assert profit_factor(r, weights=[0.5, 1.0, 1.0]) == 1.5

def test_permutation_pvalue_with_smyth_correction() -> None:
    nulls = np.linspace(-1.0, 1.0, 99)
    p_strong = permutation_pvalue(2.0, nulls)
    p_weak = permutation_pvalue(0.0, nulls)
    assert p_strong == 1.0 / 100.0
    assert 0.4 < p_weak < 0.6

def test_benjamini_hochberg_golden() -> None:
    pvals = [0.001, 0.008, 0.039, 0.041, 0.042, 0.06, 0.074, 0.205, 0.5, 0.99]
    mask, cutoff = benjamini_hochberg(pvals, q=0.05)
    # Umbrales BH: q*i/m = 0.005, 0.010, 0.015... El mayor i con p_(i) <= q*i/m
    # es i=2 (0.008 <= 0.010); 0.039 > 0.015 corta la cadena.
    assert mask.tolist() == [True, True, False, False, False, False, False, False, False, False]
    assert abs(cutoff - 0.008) < 1e-12

def test_benjamini_hochberg_rejects_nothing_under_uniform_noise() -> None:
    mask, _ = benjamini_hochberg([0.3, 0.5, 0.7, 0.9], q=0.05)
    assert not mask.any()

def test_expected_max_sharpe_grows_with_trials() -> None:
    assert expected_max_sharpe(1, 0.5) == 0.0
    e10 = expected_max_sharpe(10, 0.5)
    e1000 = expected_max_sharpe(1000, 0.5)
    assert 0.0 < e10 < e1000

def test_deflated_sharpe_decreases_with_n_trials() -> None:
    sr, n, skew, kurt = 0.25, 80.0, 0.0, 3.0
    dsr_few, _ = deflated_sharpe_ratio(sr, n, skew, kurt, n_trials=5, sr_std_across_trials=0.2)
    dsr_many, _ = deflated_sharpe_ratio(sr, n, skew, kurt, n_trials=5000, sr_std_across_trials=0.2)
    assert dsr_many < dsr_few
    psr = probabilistic_sharpe_ratio(sr, 0.0, n, skew, kurt)
    assert dsr_few <= psr + 1e-12 # deflactar nunca mejora el PSR

```

```

# -----
# Walk-forward purgado y CUSUM
# -----

def test_purged_walk_forward_never_leaks_labels_into_train() -> None:
    starts = np.arange(0, 200, 2)
    ends = starts + 10
    embargo = 10
    folds = list(purged_walk_forward(starts, ends, n_folds=5, embargo=embargo))
    assert len(folds) >= 3
    for train_idx, test_idx in folds:
        test_start = starts[test_idx].min()
        assert np.all(ends[train_idx] < test_start - embargo)
        assert np.intersect1d(train_idx, test_idx).size == 0

def test_cusum_triggers_down_on_regime_break() -> None:
    healthy = np.full(40, 0.30)
    decayed = np.full(40, -0.50)
    result = cusum_drift(
        np.concatenate([healthy, decayed]), k=0.5, h=4.0, target=0.30, scale=0.4
    )
    assert result["triggered"] is True
    assert result["side"] == "down"
    assert result["index"] >= 40

def test_cusum_quiet_on_healthy_series() -> None:
    rng = np.random.default_rng(5)
    x = rng.normal(0.30, 0.10, 80)
    result = cusum_drift(x, k=0.5, h=4.0, target=0.30, scale=0.4)
    assert result["triggered"] is False

# -----
# Orquestador de candidato
# -----

def test_summarize_pattern_validation_gates() -> None:
    rng = np.random.default_rng(13)
    n = 120
    r = rng.normal(0.45, 0.8, n)
    starts = np.arange(0, n * 12, 12)
    ends = starts + 10
    report = summarize_pattern_validation(
        r,
        starts,
        ends,
        horizonBars=10,
        n_trials_family=50,
        sr_std_across_trials=0.15,
        null_means=rng.normal(0.0, 0.05, 500),
        rng=21,
        min_n_eff=60.0,
        min_dsr=0.95,
    )
    assert report["status"] == "ok"
    assert report["n_eff"] > 60.0
    assert report["gates"]["n_eff_ok"] is True
    assert report["gates"]["ci_lower_positive"] is True
    assert report["perm_pvalue"] < 0.05
    assert report["n_trials_family"] == 50

def test_summarize_pattern_validation_flags_thin_evidence() -> None:
    rng = np.random.default_rng(17)
    n = 30
    r = rng.normal(0.05, 1.0, n)
    starts = np.arange(0, n * 3, 3)
    ends = starts + 10 # solapamiento fuerte: n_eff << n
    report = summarize_pattern_validation(
        r, starts, ends, horizonBars=10, rng=23, min_n_eff=60.0
    )
    assert report["n_eff"] < report["n_raw"]
    assert report["gates"]["n_eff_ok"] is False
    assert report["gates"]["all_local_gates"] is False

def test_sample_skew_kurt_normal_reference() -> None:
    rng = np.random.default_rng(29)
    skew, kurt = sample_skew_kurt(rng.normal(0, 1, 5000))
    assert abs(skew) < 0.15
    assert abs(kurt - 3.0) < 0.3

```

Full Source: `backend/tradeo/tests/test\_quant\_validation\_integration.py`

`backend/tradeo/tests/test\_quant\_validation\_integration.py` ? 388 lines, 15042 bytes, sha256 prefix `2674dc2de263d603`

Public surface summary before full source:

? class `\_PatternStub` line 309 methods: ?

? function `test\_engine\_quant\_validation\_dedups\_overlapping\_occurrences` line 108`

? function `test\_engine\_quant\_validation\_keeps\_disjoint\_symbols` line 120`

? function `test\_engine\_match\_tau\_from\_cluster\_dispersion` line 130`

? function `test\_run\_level\_inference\_applies\_fdr\_and\_family\_dsr` line 159`

? function `test\_validation\_gate\_rejects\_low\_n\_eff\_and\_failed\_fdr` line 177`

? function `test\_validation\_gate\_caps\_lab\_candidate\_without\_dsr` line 197`

? function `test\_validation\_gate\_caps\_lab\_candidate\_with\_survivorship\_bias` line 226`

? function `test\_matcher\_drops\_live\_daily\_bar\_during\_session` line 282`

? function `test\_matcher\_keeps\_bar\_after\_close\_and\_yesterday\_bars` line 290`

? function `test\_matcher\_ignores\_intraday\_frames` line 303`

? function `test\_matcher\_per\_pattern\_threshold\_uses\_floor\_and\_tau` line 314`

? function `test\_signal\_idempotency\_key\_is\_stable\_per\_bar` line 347`

? function `test\_duplicate\_signal\_detected\_for\_same\_bar` line 356`

Risk hints: RNG path; verify seed/control; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from datetime import date, datetime, timedelta

import numpy as np
import pandas as pd
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from tradeo.agents.pattern_discovery_lab_agent import PatternDiscoveryLabAgent
from tradeo.core.config import Settings
from tradeo.db.models import Signal, SignalStatus
from tradeo.db.session import Base
from tradeo.modules.shared.entry_scanner import PatternEntryScanner
from tradeo.research.cluster_research_engine import ClusterResearchEngine
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher
from tradeo.research.types import ClusterCandidate, ForwardOutcome, WindowSample
from tradeo.research.validation_gate import ValidationGate
from tradeo.services.market_session import US_EQUITY_TZ

def _session():
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    return sessionmaker(bind=engine)()

def _sample(symbol: str, day_index: int, *, winner: bool) -> WindowSample:
    entry = 100.0
    risk = 5.0
    end = date(2024, 1, 2) + timedelta(days=day_index)
    if winner:
        highs = [101.0, 104.0, 121.0]
        lows = [99.5, 99.0, 103.0]
        closes = [100.5, 103.0, 120.0]
    else:
        highs = [100.5, 100.2, 100.1]
        lows = [99.0, 94.0, 93.5]
        closes = [99.5, 94.5, 94.0]
    return WindowSample(
        symbol=symbol,
        timeframe="1d",
        window_size=20,
        start=(end - timedelta(days=20)).isoformat(),
        end=end.isoformat(),
        year=end.year,
        vector=np.asarray([1.0, 0.5], dtype=np.float32),
        chart={},
        features={},
        outcome=ForwardOutcome(
            forward_returns={10: closes[-1] / entry - 1.0},
            entry_price=entry,
            risk_proxy=risk,
            forward_end=(end + timedelta(days=10)).isoformat(),
            long_mfe_r=(max(highs) - entry) / risk,
            long_mae_r=max(0.0, (entry - min(lows)) / risk),
            long_outcome_r=(closes[-1] - entry) / risk,
            long_hit_4r=winner,
            short_mfe_r=(entry - min(lows)) / risk,
            short_mae_r=max(0.0, (max(highs) - entry) / risk),
            short_outcome_r=(entry - closes[-1]) / risk,
            short_hit_4r=False,
            forward_highs=highs,
            forward_lows=lows,
            forward_closes=closes,
        ),
    )

def _candidate(name: str, *, p_value: float, sharpe: float, n_eff: float) -> ClusterCandidate:
    return ClusterCandidate(
        pattern_key=f"novel_long_w20_{name}",
        name=name,
    )

```

```

        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=0,
        centroid=[0.0, 0.0],
        sample_count=150,
        symbol_count=10,
        year_count=3,
        score=1.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "null_p_value": p_value,
            "trade_sharpe": sharpe,
            "real_variant_count": 144,
            "quant_validation": {
                "n_eff": n_eff,
                "sharpe_per_trade": sharpe,
                "skew": 0.0,
                "kurtosis": 3.0,
                "expectancy_ci95_low": 0.05,
            },
            "effective_sample_count": n_eff,
        },
        feature_summary={},
        examples=[],
    )

# -----
# Engine: n_eff y tau por patrón
# -----

def test_engine_quant_validation_dedups_overlapping_occurrences() -> None:
    engine = ClusterResearchEngine()
    # 30 ocurrencias del mismo símbolo en días consecutivos: outcomes de 10 días
    # solapados ? tras dedup + pesos, n_eff debe ser MUCHO menor que n_raw.
    samples = [_sample("AAA", i, winner=(i % 2 == 0)) for i in range(30)]
    quant = engine._quant_validation_metrics(samples, side="long", rr=4.0)
    assert quant["n_raw"] == 30
    assert quant["n_unique"] < 10
    assert quant["n_eff"] <= quant["n_unique"]
    assert quant["method"] == "dedup_uniqueness_stationary_bootstrap_newey_west"

def test_engine_quant_validation_keeps_disjoint_symbols() -> None:
    engine = ClusterResearchEngine()
    # Misma fecha en símbolos distintos: dedup por símbolo no los toca, pero los
    # pesos de unicidad sí reparten la barra compartida del calendario.
    samples = [_sample(f"SYM{i}", 0, winner=True) for i in range(5)]
    quant = engine._quant_validation_metrics(samples, side="long", rr=4.0)
    assert quant["n_unique"] == 5
    assert abs(quant["n_eff"] - 1.0) < 1e-6 # mismas fechas ? un solo episodio de mercado

def test_engine_match_tau_from_cluster_dispersion() -> None:
    engine = ClusterResearchEngine(match_tau_percentile=92.5)
    rng = np.random.default_rng(11)
    centroid = np.zeros(8)
    vectors = rng.normal(0.0, 0.3, size=(200, 8))
    tau = engine._match_tau_similarity(vectors, centroid)
    similarities = 1.0 / (1.0 + np.linalg.norm(vectors - centroid, axis=1) / np.sqrt(8))
    coverage = float(np.mean(similarities >= tau))
    assert 0.90 <= coverage <= 0.95 # ~92.5% de los miembros reales pasan tau

# -----
# Agente: BH-FDR del run + DSR de familia
# -----

def _agent(tmp_path) -> PatternDiscoveryLabAgent:
    settings = Settings(
        reports_dir=str(tmp_path / "reports"),
        artifacts_dir=str(tmp_path / "artifacts"),
    )

    class _NoProvider:
        def fetch_ohlcv(self, *args, **kwargs): # pragma: no cover
            raise AssertionError("no debe pedir datos")

    return PatternDiscoveryLabAgent(provider=_NoProvider(), settings=settings)

def test_run_level_inference_applies_fdr_and_family_dsr(tmp_path) -> None:
    agent = _agent(tmp_path)
    strong = _candidate("strong", p_value=0.001, sharpe=0.6, n_eff=90.0)
    noise = [
        _candidate(f"noise{i}", p_value=0.30 + 0.05 * i, sharpe=0.05 * (i % 3), n_eff=80.0)
        for i in range(8)
    ]
    candidates = [strong, *noise]
    agent._apply_run_level_inference(candidates)
    assert strong.metrics["fdr_passed"] is True
    assert all(c.metrics["fdr_passed"] is False for c in noise)

```

```

assert strong.metrics["fdr_test_count"] == 9
assert strong.metrics["dsr_family_n_trials"] >= 144
assert strong.metrics["dsr_family"] is not None
# El DSR de familia debe estar deflactado respecto al PSR puro.
assert 0.0 <= strong.metrics["dsr_family"] <= 1.0

def test_validation_gate_rejects_low_n_eff_and_failed_fdr(tmp_path) -> None:
    settings = Settings(
        reports_dir=str(tmp_path / "reports"), artifacts_dir=str(tmp_path / "artifacts")
    )
    gate = ValidationGate(settings)

    thin = _candidate("thin", p_value=0.001, sharpe=0.5, n_eff=12.0)
    thin.metrics["fdr_passed"] = True
    gate.evaluate(thin)
    assert thin.validation_passed is False
    assert any("muestras efectivas insuficientes" in r for r in thin.validation_reasons)

    failed_fdr = _candidate("failedfdr", p_value=0.40, sharpe=0.5, n_eff=90.0)
    failed_fdr.metrics["fdr_passed"] = False
    failed_fdr.metrics["fdr_q"] = 0.05
    gate.evaluate(failed_fdr)
    assert failed_fdr.validation_passed is False
    assert any("BH-FDR" in r for r in failed_fdr.validation_reasons)

def test_validation_gate_caps_lab_candidate_without_dsr(tmp_path) -> None:
    settings = Settings(
        reports_dir=str(tmp_path / "reports"), artifacts_dir=str(tmp_path / "artifacts")
    )
    gate = ValidationGate(settings)
    candidate = _candidate("promising", p_value=0.001, sharpe=0.6, n_eff=90.0)
    candidate.metrics.update(
        {
            "fdr_passed": True,
            "best_rr": 3.0,
            "best_expectancy_r": 0.5,
            "best_profit_factor": 2.2,
            "best_max_drawdown_r": 4.0,
            "expectancy_r": 0.5,
            "profit_factor": 2.2,
            "stability_score": 0.8,
            "out_of_sample_expectancy_r": 0.3,
            "out_of_sample_profit_factor": 1.9,
            "rr_metrics": {"3": {"expectancy_r": 0.5, "profit_factor": 2.2}},
            "train_sample_count": 150,
            "dsr_family": 0.40, # por debajo de discovery_min_dsr=0.95
            "dsr_family_n_trials": 5000,
        }
    )
    gate.evaluate(candidate)
    assert candidate.metrics["promotion_status"] != "lab_candidate"
    assert any("DSR familia" in w for w in candidate.metrics["validation_warnings"])

def test_validation_gate_caps_lab_candidate_with_survivorship_bias(tmp_path) -> None:
    settings = Settings(
        reports_dir=str(tmp_path / "reports"),
        artifacts_dir=str(tmp_path / "artifacts"),
        survivorship_cap_state="lab_watchlist",
    )
    gate = ValidationGate(settings)
    candidate = _candidate("survivor", p_value=0.001, sharpe=0.6, n_eff=90.0)
    candidate.metrics.update(
        {
            "fdr_passed": True,
            "best_rr": 3.0,
            "best_expectancy_r": 0.5,
            "best_profit_factor": 2.2,
            "best_max_drawdown_r": 4.0,
            "expectancy_r": 0.5,
            "profit_factor": 2.2,
            "stability_score": 0.8,
            "out_of_sample_expectancy_r": 0.3,
            "out_of_sample_profit_factor": 1.9,
            "rr_metrics": {"3": {"expectancy_r": 0.5, "profit_factor": 2.2}},
            "train_sample_count": 150,
            "dsr_family": 0.99,
            "dsr_family_n_trials": 5000,
            "survivorship_biased": True,
            "universe_point_in_time": False,
        }
    )
    gate.evaluate(candidate)

    assert candidate.metrics["promotion_status"] == "lab_watchlist"
    assert candidate.metrics["survivorship_cap_applied"] is True
    assert any("universo historico no point-in-time" in w for w in candidate.metrics["validation_warnings"])

# -----
# Matcher: vela viva y umbral por patrón
# -----

```



```

def _daily_df(end: date, bars: int = 30) -> pd.DataFrame:
    idx = pd.date_range(end=pd.Timestamp(end), periods=bars, freq="B")
    values = np.linspace(100.0, 110.0, bars)
    return pd.DataFrame(
        {
            "open": values,
            "high": values + 1,
            "low": values - 1,
            "close": values,
            "volume": np.full(bars, 1e6),
        },
        index=idx,
    )

def test_matcher_drops_live_daily_bar_during_session() -> None:
    in_session = datetime(2026, 6, 10, 12, 0, tzinfo=US_EQUITY_TZ) # miércoles 12:00 NY
    df = _daily_df(end=date(2026, 6, 10))
    trimmed = NovelPatternMatcher._drop_incomplete_daily_bar(df, "1d", now=in_session)
    assert len(trimmed) == len(df) - 1
    assert pd.Timestamp(trimmed.index[-1]).date() == date(2026, 6, 9)

def test_matcher_keeps_bar_after_close_and_yesterday_bars() -> None:
    after_close = datetime(2026, 6, 10, 16, 30, tzinfo=US_EQUITY_TZ)
    df_today = _daily_df(end=date(2026, 6, 10))
    assert len(NovelPatternMatcher._drop_incomplete_daily_bar(df_today, "1d", now=after_close)) == len(
        df_today
    )
    in_session = datetime(2026, 6, 10, 12, 0, tzinfo=US_EQUITY_TZ)
    df_yesterday = _daily_df(end=date(2026, 6, 9))
    assert len(
        NovelPatternMatcher._drop_incomplete_daily_bar(df_yesterday, "1d", now=in_session)
    ) == len(df_yesterday)

def test_matcher_ignores_intraday_frames() -> None:
    in_session = datetime(2026, 6, 10, 12, 0, tzinfo=US_EQUITY_TZ)
    df = _daily_df(end=date(2026, 6, 10))
    assert len(NovelPatternMatcher._drop_incomplete_daily_bar(df, "1h", now=in_session)) == len(df)

class _PatternStub:
    def __init__(self, tau: float | None) -> None:
        self.metrics_json = {} if tau is None else {"match_tau_similarity": tau}

def test_matcher_per_pattern_threshold_uses_floor_and_tau(tmp_path) -> None:
    settings = Settings(
        reports_dir=str(tmp_path / "reports"), artifacts_dir=str(tmp_path / "artifacts")
    )
    matcher = NovelPatternMatcher(provider=object(), settings=settings)
    assert matcher._effective_threshold(_PatternStub(0.80), 0.45) == 0.80
    assert matcher._effective_threshold(_PatternStub(0.20), 0.45) == 0.45 # el global es suelo
    assert matcher._effective_threshold(_PatternStub(None), 0.45) == 0.45
    settings_off = Settings(
        reports_dir=str(tmp_path / "r2"),
        artifacts_dir=str(tmp_path / "a2"),
        discovery_match_per_pattern_threshold=False,
    )
    matcher_off = NovelPatternMatcher(provider=object(), settings=settings_off)
    assert matcher_off._effective_threshold(_PatternStub(0.80), 0.45) == 0.45

# -----
# Lab: idempotencia de señal (pattern, symbol, variante, barra)
# -----

def _match_dict(window_end: str) -> dict:
    return {
        "pattern_id": 7,
        "pattern_name": "DISCOVERED_LONG_W20_C01_TEST",
        "symbol": "LABX",
        "timeframe": "1d",
        "entry_variant_id": "breakout_v1",
        "window_end": window_end,
    }

def test_signal_idempotency_key_is_stable_per_bar() -> None:
    key_a = PatternEntryScanner._signal_idempotency_key("laboratory", _match_dict("2026-06-09"))
    key_b = PatternEntryScanner._signal_idempotency_key("laboratory", _match_dict("2026-06-09"))
    key_c = PatternEntryScanner._signal_idempotency_key("laboratory", _match_dict("2026-06-10"))
    assert key_a == key_b
    assert key_a != key_c
    assert PatternEntryScanner._signal_idempotency_key("fox_hunter", _match_dict("2026-06-09")) != key_a

def test_duplicate_signal_detected_for_same_bar(tmp_path) -> None:
    db = _session()
    settings = Settings(
        reports_dir=str(tmp_path / "reports"), artifacts_dir=str(tmp_path / "artifacts")
    )
    scanner = PatternEntryScanner(settings=settings, matcher=object())

```

```

match = _match_dict("2026-06-09")
db.add(
    Signal(
        symbol="LABX",
        pattern="DISCOVERED_LONG_W20_C01_TEST",
        side="long",
        timeframe="1d",
        entry=100.0,
        stop=95.0,
        target=115.0,
        reward_risk=3.0,
        confidence=0.7,
        composite_score=0.7,
        strategy_version="laboratory_pattern_7",
        status=SignalStatus.PAPER_APPROVED,
        metadata_json={
            "entry_module": "laboratory",
            "signal_idempotency_key": PatternEntryScanner._signal_idempotency_key(
                "laboratory", match
            ),
        },
    )
)
db.commit()
assert scanner._has_duplicate_signal(db, match, module="laboratory") is True
assert scanner._has_duplicate_signal(db, _match_dict("2026-06-10"), module="laboratory") is False
assert scanner._has_duplicate_signal(db, match, module="fox_hunter") is False

```

Full Source: `backend/tradeo/tests/test\_pattern\_discovery\_lab.py`

`backend/tradeo/tests/test\_pattern\_discovery\_lab.py` ? 689 lines, 25911 bytes, sha256 prefix `891b28a2215bb009`

Public surface summary before full source:

```

? function `test_window_sampler_generates_forward_labeled_samples` line 62`
? function `test_window_sampler_adds_benchmark_relative_strength` line 94`
? function `test_cluster_engine_returns_candidates_or_empty_without_crashing` line 119`
? function `test_cluster_engine_fits_scaler_on_train_only` line 152`
? function `test_cluster_signature_records_medoid_and_concentration` line 203`
? function `test_cluster_engine_selects_best_rr_from_train_only` line 224`
? function `test_validation_gate_rejects_underpowered_candidate` line 247`
? function `test_validation_gate_allows_edge_below_4r_as_watchlist` line 264`
? function `test_validation_gate_rejects_high_adjusted_null_p_value` line 303`
? function `test_validation_gate_rejects_failed_reality_check_and_edge_decay` line 346`
? function `test_validation_gate_marks_strong_underpowered_edge_for_confirmation` line 393`
? function `test_validation_gate_rejects_unstable_walk_forward_candidate` line 438`
? function `test_validation_gate_rejects_high_overfit_score` line 481`
? function `test_validation_gate_rejects_failed_cost_stress` line 524`
? function `test_lab_agent_persists_compressed_event_ledger` line 568`
? function `test_global_experiment_registry_accumulates_trials_once_per_experiment` line 608`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import gzip
import hashlib
import json
from datetime import date, timedelta
from pathlib import Path

import numpy as np

from tradeo.agents.pattern_discovery_lab_agent import PatternDiscoveryLabAgent
from tradeo.core.config import Settings
from tradeo.research.cluster_research_engine import ClusterResearchEngine
from tradeo.research.global_experiment_registry import GlobalExperimentRegistry
from tradeo.research.types import ClusterCandidate, ForwardOutcome, WindowSample
from tradeo.research.validation_gate import ValidationGate
from tradeo.research.window_sampler import WindowSampler
from tradeo.tests.fixtures import fixture_ohlcv

def _research_sample(
    index: int,
    *,
    vector_value: float,
    highs: list[float],
    lows: list[float],
    closes: list[float],
) -> WindowSample:
    entry = 100.0
    risk = 10.0
    end = date(2024, 1, 1) + timedelta(days=index)
    return WindowSample(
        symbol="LABT",
        timeframe="1d",
        window_size=20,

```

```

start=(end - timedelta(days=20)).isoformat(),
end=end.isoformat(),
year=end.year,
vector=np.asarray([vector_value, vector_value * 0.5], dtype=np.float32),
chart={},
features={},
outcome=ForwardOutcome(
    forward_returns={5: closes[-1] / entry - 1.0},
    entry_price=entry,
    risk_proxy=risk,
    forward_end=(end + timedelta(days=len(closes))).isoformat(),
    long_mfe_r=max((max(highs) - entry) / risk, 0.0),
    long_mae_r=max((entry - min(lows)) / risk, 0.0),
    long_outcome_r=(closes[-1] - entry) / risk,
    long_hit_4r=False,
    short_mfe_r=max((entry - min(lows)) / risk, 0.0),
    short_mae_r=max((max(highs) - entry) / risk, 0.0),
    short_outcome_r=(entry - closes[-1]) / risk,
    short_hit_4r=False,
    forward_highs=highs,
    forward_lows=low,
    forward_closes=closes,
),
),
)

def test_window_sampler_generates_forward_labeled_samples() -> None:
    df = fixture_ohlcv("LABX", bars=320)
    samples = WindowSampler().sample(
        symbol="LABX",
        df=df,
        timeframe="1d",
        window_sizes=[20, 50],
        forward_bars=[5, 10, 20],
        stride=20,
        max_windows_per_symbol=30,
    )
    assert samples
    first = samples[0]
    assert first.vector.shape[0] > 50
    assert first.outcome.risk_proxy > 0
    assert first.outcome.execution_cost_r > 0
    assert first.outcome.long_label in {"target", "stop", "timeout"}
    assert first.outcome.short_label in {"target", "stop", "timeout"}
    assert first.outcome.long_speed_label in {"fast_target", "normal_target", "slow_target", "stopped", "timeout"}
    assert first.outcome.execution["fill_probability"] > 0
    assert 20 in first.outcome.forward_returns
    assert "close_norm" in first.chart
    assert "swing_state" in first.chart
    assert "long_entry_trigger_score" in first.features
    assert "liquidity_score" in first.features
    assert "shapelet_flag_continuation_score" in first.features
    assert "swing_trend_score" in first.features
    assert "execution_fill_probability" in first.features
    assert "weekly_return" in first.features
    assert "relative_strength_spy" in first.features

def test_window_sampler_adds_benchmark_relative_strength() -> None:
    df = fixture_ohlcv("LABR", bars=180)
    spy = fixture_ohlcv("SPY", bars=180)
    sector = fixture_ohlcv("SECTOR", bars=180)
    industry = fixture_ohlcv("INDUSTRY", bars=180)
    spy["close"] = spy["close"].iloc[0]
    sector["close"] = sector["close"].iloc[0] * 0.98
    industry["close"] = industry["close"].iloc[0] * 1.02
    samples = WindowSampler().sample(
        symbol="LABR",
        df=df,
        timeframe="1d",
        window_sizes=[20],
        forward_bars=[5],
        stride=20,
        max_windows_per_symbol=5,
        benchmark_frames={"SPY": spy, "SECTOR": sector, "INDUSTRY": industry},
    )
    assert samples
    assert samples[0].features["relative_strength_spy"] != 0.0
    assert "relative_strength_sector" in samples[0].features
    assert "relative_strength_industry" in samples[0].features
    assert "market_breadth_proxy" in samples[0].features

def test_cluster_engine_returns_candidates_or_empty_without_crashing() -> None:
    df = fixture_ohlcv("LABY", bars=500)
    samples = WindowSampler().sample(
        symbol="LABY",
        df=df,
        timeframe="1d",
        window_sizes=[20],
        forward_bars=[5, 10, 20],
        stride=2,
        max_windows_per_symbol=180,
    )
    engine = ClusterResearchEngine(min_cluster_size=20, max_clusters_per_window=4)
    candidates = engine.discover(samples)

```

```

assert isinstance(candidates, list)
if candidates:
    candidate = candidates[0]
    assert candidate.sample_count > 0
    assert candidate.metrics["sample_count"] == candidate.sample_count
    assert candidate.side in {"long", "short"}
    assert "operational_trigger" in candidate.metrics
    assert "event_ledger_count" in candidate.metrics
    assert "cost_stress" in candidate.metrics
    assert "human_rule" in candidate.metrics
    assert "regime_profile" in candidate.metrics
    assert "novelty_score" in candidate.metrics
    assert "expected_information_gain" in candidate.metrics
    assert candidate.metrics["feature_parity_contract"]["research_lab_shared_path"] is True
    assert candidate.metrics["feature_parity_contract"]["contract_id"].startswith("tradeo.pattern_embedding")
    assert "medoid" in candidate.metrics["cluster_signature"]
    assert "similarity_distribution" in candidate.metrics["cluster_signature"]
    assert "concentration_checks" in candidate.metrics

def test_cluster_engine_fits_scaler_on_train_only() -> None:
    samples = [
        _research_sample(i, vector_value=0.01 * (i % 2), highs=[101.0], lows=[99.0], closes=[100.0])
        for i in range(24)
    ]
    samples.extend(
        _research_sample(24 + i, vector_value=50.0, highs=[101.0], lows=[99.0], closes=[100.0])
        for i in range(6)
    )
    candidates = ClusterResearchEngine(
        min_cluster_size=5,
        max_clusters_per_window=2,
        out_of_sample_pct=0.2,
        rr_levels=[2.0],
        min_samples=1,
    ).discover(samples)
    assert candidates
    assert candidates[0].metrics["validation_method"] == "train_fit_forward_holdout_walk_forward_embargo"
    assert candidates[0].metrics["model_fit_sample_count"] == 24
    assert candidates[0].metrics["model_holdout_sample_count"] == 6
    assert "walk_forward_folds" in candidates[0].metrics
    assert candidates[0].metrics["multiple_testing_trials"] >= 4
    assert "adjusted_p_value" in candidates[0].metrics
    assert "wrc_p_value" in candidates[0].metrics
    assert "spa_p_value" in candidates[0].metrics
    assert candidates[0].metrics["null_method"] == "stratified_regime_bootstrap"
    assert candidates[0].metrics["reality_check_method"] == "bootstrap_reality_proxy"
    assert candidates[0].metrics["reality_check_formal_test"] is False
    assert candidates[0].metrics["bootstrap_reality_proxy"]["formal_test"] is False
    assert "expectancy_ci_low" in candidates[0].metrics
    assert "deflated_sharpe_probability" in candidates[0].metrics
    assert "purged_cv_fold_count" in candidates[0].metrics
    assert "edge_decay_parameter_score" in candidates[0].metrics
    assert "overfit_score" in candidates[0].metrics
    assert "selection_split" in candidates[0].metrics
    assert candidates[0].metrics["fit_scope"]["descriptive_all_feeds_scores"] is False
    assert candidates[0].metrics["feature_parity_contract"]["research_path"].startswith("WindowSampler")
    assert candidates[0].metrics["cluster_stability"]["concentration_passed"] in {True, False}
    assert "train_metrics" in candidates[0].metrics
    assert "out_of_sample_metrics" in candidates[0].metrics
    assert "walk_forward_metrics" in candidates[0].metrics
    assert "descriptive_all_expectancy_r" in candidates[0].metrics
    assert candidates[0].metrics["descriptive_metric_policy"]["feeds_lab_priority_score"] is False
    assert candidates[0].metrics["score_input_scope"]["descriptive_all_fields_used"] is False
    mutated = dict(candidates[0].metrics)
    mutated["descriptive_all_expectancy_r"] = 999.0
    mutated["descriptive_all_profit_factor"] = 999.0
    assert ClusterResearchEngine._candidate_score(mutated) == ClusterResearchEngine._candidate_score(candidates[0].metrics)
    assert abs(float(candidates[0].metrics["scaler_mean"][0])) < 0.01

def test_cluster_signature_records_medoid_and_concentration() -> None:
    samples = [
        _research_sample(i, vector_value=0.01 * i, highs=[104.0], lows=[99.0], closes=[103.0])
        for i in range(6)
    ]
    vectors = np.asarray([[0.0, 0.0], [0.1, 0.0], [0.2, 0.0], [0.3, 0.0], [0.4, 0.0], [0.5, 0.0]])
    signature = ClusterResearchEngine._cluster_signature(
        samples,
        vectors,
        np.asarray([0.2, 0.0]),
        "long",
    )

    assert signature["medoid"]["window_end"] == samples[2].end
    assert signature["similarity_distribution"]["p50"] > 0
    checks = signature["concentration_checks"]
    assert checks["passed"] is False
    assert checks["max_symbol_share"] == 1.0
    assert "symbol_concentration_gt_40pct" in checks["reasons"]

def test_cluster_engine_selects_best_rr_from_train_only() -> None:
    train = [
        _research_sample(i, vector_value=0.0, highs=[121.0], lows=[99.0], closes=[100.0])
        for i in range(20)
    ]

```

```

]
holdout = [
    _research_sample(20 + i, vector_value=0.0, highs=[151.0], lows=[99.0], closes=[151.0])
    for i in range(20)
]
candidates = ClusterResearchEngine(
    min_cluster_size=5,
    max_clusters_per_window=2,
    out_of_sample_pct=0.5,
    rr_levels=[2.0, 5.0],
    min_samples=1,
).discover(train + holdout)
assert candidates
candidate = max(candidates, key=lambda c: c.sample_count)
assert candidate.metrics["best_rr"] == 2.0
assert candidate.metrics["train_sample_count"] == 20
assert candidate.metrics["holdout_sample_count"] == 20

def test_validation_gate_rejects_underpowered_candidate() -> None:
    df = fixture_ohlcv("LABZ", bars=320)
    samples = WindowSampler().sample(
        symbol="LABZ",
        df=df,
        timeframe="1d",
        window_sizes=[20],
        forward_bars=[5, 10, 20],
        stride=3,
        max_windows_per_symbol=90,
    )
    candidates = ClusterResearchEngine(min_cluster_size=15, max_clusters_per_window=3).discover(samples)
    if candidates:
        candidate = ValidationGate().evaluate(candidates[0])
        assert candidate.validation_reasons

def test_validation_gate_allows_edge_below_4r_as_watchlist() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=120,
        symbol_count=10,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "best_rr": 2.5,
            "best_expectancy_r": 0.12,
            "best_profit_factor": 1.3,
            "best_max_drawdown_r": 4.0,
            "expectancy_r": 0.12,
            "profit_factor": 1.3,
            "stability_score": 0.5,
            "out_of_sample_expectancy_r": 0.05,
            "out_of_sample_profit_factor": 1.3,
            "walk_forward_fold_count": 2,
            "walk_forward_positive_fold_rate": 1.0,
            "expectancy_ci_low": 0.01,
            "overfit_score": 0.1,
            "rr_metrics": {"2.5": {"expectancy_r": 0.12, "profit_factor": 1.3, "sample_count": 120}},
        },
        feature_summary={},
        examples=[],
    )
    evaluated = ValidationGate().evaluate(candidate)
    assert evaluated.validation_passed
    assert evaluated.metrics["promotion_status"] == "lab_watchlist"

def test_validation_gate_rejects_high_adjusted_null_p_value() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=140,
        symbol_count=12,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "train_sample_count": 110,
            "best_rr": 3.0,
            "best_expectancy_r": 0.35,
            "best_profit_factor": 2.1,
            "best_max_drawdown_r": 4.0,
        },
    )
    evaluated = ValidationGate().evaluate(candidate)
    assert not evaluated.validation_passed
    assert evaluated.validation_reasons == ["High adjusted null p-value"]

```

```

        "expectancy_r": 0.35,
        "profit_factor": 2.1,
        "stability_score": 0.6,
        "out_of_sample_expectancy_r": 0.12,
        "out_of_sample_profit_factor": 1.5,
        "expectancy_lift_r": 0.04,
        "adjusted_p_value": 0.7,
        "statistical_edge_passed": False,
        "walk_forward_fold_count": 2,
        "walk_forward_positive_fold_rate": 1.0,
        "expectancy_ci_low": 0.05,
        "overfit_score": 0.1,
        "rr_metrics": {"3": {"expectancy_r": 0.35, "profit_factor": 2.1, "sample_count": 110}},
    },
    feature_summary={},
    examples=[],
)
evaluated = ValidationGate().evaluate(candidate)
assert not evaluated.validation_passed
assert any("significancia insuficiente" in reason for reason in evaluated.validation_reasons)

def test_validation_gate_rejects_failed_reality_check_and_edge_decay() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=140,
        symbol_count=12,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "train_sample_count": 120,
            "best_rr": 3.0,
            "best_expectancy_r": 0.45,
            "best_profit_factor": 2.4,
            "best_max_drawdown_r": 5.0,
            "expectancy_r": 0.45,
            "profit_factor": 2.4,
            "stability_score": 0.62,
            "out_of_sample_expectancy_r": 0.22,
            "out_of_sample_profit_factor": 1.8,
            "expectancy_lift_r": 0.18,
            "adjusted_p_value": 0.08,
            "wrc_p_value": 0.7,
            "spa_p_value": 0.08,
            "statistical_edge_passed": True,
            "walk_forward_fold_count": 4,
            "walk_forward_positive_fold_rate": 1.0,
            "expectancy_ci_low": 0.05,
            "overfit_score": 0.1,
            "edge_decay_passed": False,
            "rr_metrics": {"3": {"expectancy_r": 0.45, "profit_factor": 2.4, "sample_count": 120}},
        },
        feature_summary={},
        examples=[],
    )
    evaluated = ValidationGate().evaluate(candidate)
    assert not evaluated.validation_passed
    assert any("bootstrap reality proxy WRC-like" in reason for reason in evaluated.validation_reasons)
    assert any("edge decae" in reason for reason in evaluated.validation_reasons)

def test_validation_gate_marks_strong_underpowered_edge_for_confirmation() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=72,
        symbol_count=12,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "train_sample_count": 58,
            "best_rr": 3.0,
            "best_expectancy_r": 0.45,
            "best_profit_factor": 2.4,
            "best_max_drawdown_r": 5.0,
            "expectancy_r": 0.45,
            "profit_factor": 2.4,
            "stability_score": 0.62,
            "out_of_sample_expectancy_r": 0.22,
            "out_of_sample_profit_factor": 1.8,
            "expectancy_lift_r": 0.18,

```

```

        "adjusted_p_value": 0.08,
        "statistical_edge_passed": True,
        "walk_forward_fold_count": 2,
        "walk_forward_positive_fold_rate": 1.0,
        "expectancy_ci_low": 0.05,
        "overfit_score": 0.1,
        "rr_metrics": {"3": {"expectancy_r": 0.45, "profit_factor": 2.4, "sample_count": 58}},
    },
    feature_summary={},
    examples=[],
)
evaluated = ValidationGate().evaluate(candidate)
assert not evaluated.validation_passed
assert evaluated.metrics["confirmation_recommended"] is True
assert evaluated.metrics["confirmation_priority_score"] > 0
assert any("confirmación ampliada" in reason for reason in evaluated.validation_reasons)

def test_validation_gate_rejects_unstable_walk_forward_candidate() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=140,
        symbol_count=12,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "train_sample_count": 120,
            "best_rr": 3.0,
            "best_expectancy_r": 0.45,
            "best_profit_factor": 2.4,
            "best_max_drawdown_r": 5.0,
            "expectancy_r": 0.45,
            "profit_factor": 2.4,
            "stability_score": 0.62,
            "out_of_sample_expectancy_r": 0.22,
            "out_of_sample_profit_factor": 1.8,
            "expectancy_lift_r": 0.18,
            "adjusted_p_value": 0.08,
            "statistical_edge_passed": True,
            "walk_forward_fold_count": 4,
            "walk_forward_positive_fold_rate": 0.25,
            "expectancy_ci_low": 0.05,
            "overfit_score": 0.1,
            "rr_metrics": {"3": {"expectancy_r": 0.45, "profit_factor": 2.4, "sample_count": 120}},
        },
        feature_summary={},
        examples=[],
    )
    evaluated = ValidationGate().evaluate(candidate)
    assert not evaluated.validation_passed
    assert any("walk-forward inestable" in reason for reason in evaluated.validation_reasons)

def test_validation_gate_rejects_high_overfit_score() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=140,
        symbol_count=12,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "train_sample_count": 120,
            "best_rr": 3.0,
            "best_expectancy_r": 0.45,
            "best_profit_factor": 2.4,
            "best_max_drawdown_r": 5.0,
            "expectancy_r": 0.45,
            "profit_factor": 2.4,
            "stability_score": 0.62,
            "out_of_sample_expectancy_r": 0.22,
            "out_of_sample_profit_factor": 1.8,
            "expectancy_lift_r": 0.18,
            "adjusted_p_value": 0.08,
            "statistical_edge_passed": True,
            "walk_forward_fold_count": 4,
            "walk_forward_positive_fold_rate": 1.0,
            "expectancy_ci_low": 0.05,
            "overfit_score": 0.9,
            "rr_metrics": {"3": {"expectancy_r": 0.45, "profit_factor": 2.4, "sample_count": 120}},
        },
    ),

```

```

        feature_summary={},
        examples=[],
    )
    evaluated = ValidationGate().evaluate(candidate)
    assert not evaluated.validation_passed
    assert any("overfit alto" in reason for reason in evaluated.validation_reasons)

def test_validation_gate_rejects_failed_cost_stress() -> None:
    candidate = ClusterCandidate(
        pattern_key="x",
        name="x",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=140,
        symbol_count=12,
        year_count=3,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "train_sample_count": 120,
            "best_rr": 3.0,
            "best_expectancy_r": 0.45,
            "best_profit_factor": 2.4,
            "best_max_drawdown_r": 5.0,
            "expectancy_r": 0.45,
            "profit_factor": 2.4,
            "stability_score": 0.62,
            "out_of_sample_expectancy_r": 0.22,
            "out_of_sample_profit_factor": 1.8,
            "expectancy_lift_r": 0.18,
            "adjusted_p_value": 0.08,
            "statistical_edge_passed": True,
            "walk_forward_fold_count": 4,
            "walk_forward_positive_fold_rate": 1.0,
            "expectancy_ci_low": 0.05,
            "overfit_score": 0.1,
            "cost_stress_passed": False,
            "rr_metrics": {"3": {"expectancy_r": 0.45, "profit_factor": 2.4, "sample_count": 120}},
        },
        feature_summary={},
        examples=[],
    )
    evaluated = ValidationGate().evaluate(candidate)
    assert not evaluated.validation_passed
    assert any("coste x2" in reason for reason in evaluated.validation_reasons)

def test_lab_agent_persists_compressed_event_ledger(tmp_path: Path) -> None:
    candidate = ClusterCandidate(
        pattern_key="family/unsafe pattern",
        name="auditable",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=2,
        symbol_count=1,
        year_count=1,
        score=0.0,
        validation_passed=False,
        validation_reasons=[],
        metrics={
            "event_ledger": [
                {"symbol": "AAA", "window_end": "2024-01-01", "result_r": 1.2},
                {"symbol": "BBB", "window_end": "2024-01-02", "result_r": -1.0},
            ],
        },
        feature_summary={},
        examples=[],
    )
    agent = PatternDiscoveryLabAgent(provider=object(), settings=Settings(reports_dir=str(tmp_path)))

    assert agent._write_event_ledgers(42, [candidate]) == 1

    path = Path(str(candidate.metrics["event_ledger_path"]))
    raw = gzip.decompress(path.read_bytes())
    payload = json.loads(raw)
    assert path.name == "family_unsafe_pattern.json.gz"
    assert payload["event_count"] == 2
    assert payload["events"][0]["symbol"] == "AAA"
    assert candidate.metrics["event_ledger_sha256"] == hashlib.sha256(raw).hexdigest()
    assert candidate.metrics["event_ledger_persisted"] is True
    assert "event_ledger" not in candidate.metrics
    assert len(candidate.metrics["event_ledger_preview"]) == 2

def test_global_experiment_registry_accumulates_trials_once_per_experiment(tmp_path: Path) -> None:
    candidate = ClusterCandidate(
        pattern_key="pattern-a",
        name="pattern-a",

```



```

        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=10,
        symbol_count=3,
        year_count=2,
        score=0.8,
        validation_passed=True,
        validation_reasons=[],
        metrics={"real_variant_count": 12, "fit_scope": {"scaler": "train_only"}},
        feature_summary={},
        examples=[],
    )
    registry = GlobalExperimentRegistry(tmp_path / "global_experiment_registry.json")

    first = registry.register([candidate], run_id=1, params={"interval": "1d"})
    duplicate = registry.register([candidate], run_id=1, params={"interval": "1d"})
    second = registry.register([candidate], run_id=2, params={"interval": "1d"})

    assert first["global_trial_count"] == 12
    assert duplicate["global_trial_count"] == 12
    assert second["global_trial_count"] == 24
    assert candidate.metrics["global_experiment_registry"]["edge_claim"] == "NO_DEMOSTRADO"

def test_global_experiment_registry_hash_chain_atomic_write_and_backup(tmp_path: Path) -> None:
    first_candidate = ClusterCandidate(
        pattern_key="pattern-a",
        name="pattern-a",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=1,
        centroid=[],
        sample_count=10,
        symbol_count=3,
        year_count=2,
        score=0.8,
        validation_passed=True,
        validation_reasons=[],
        metrics={"real_variant_count": 2, "fit_scope": {"scaler": "train_only"}},
        feature_summary={},
        examples=[],
    )
    second_candidate = ClusterCandidate(
        pattern_key="pattern-b",
        name="pattern-b",
        side="long",
        timeframe="1d",
        window_size=20,
        cluster_id=2,
        centroid=[],
        sample_count=12,
        symbol_count=4,
        year_count=2,
        score=0.9,
        validation_passed=True,
        validation_reasons=[],
        metrics={"real_variant_count": 3, "fit_scope": {"scaler": "train_only"}},
        feature_summary={},
        examples=[],
    )
    registry = GlobalExperimentRegistry(tmp_path / "global_experiment_registry.json")

    first = registry.register([first_candidate], run_id=1, params={"interval": "1d"})
    first_payload = registry.load()
    second = registry.register([second_candidate], run_id=2, params={"interval": "1d"})
    second_payload = registry.load()

    assert first_payload["registry_hash"] == first["registry_hash"]
    assert second_payload["latest_run_manifest"]["previous_registry_hash"] == first["registry_hash"]
    assert second_payload["latest_run_manifest"]["run_manifest_hash"] == second["run_manifest_hash"]
    assert second_payload["registry_hash"] == registry.registry_hash(second_payload)
    assert second_candidate.metrics["global_experiment_registry"]["registry_hash"] == second["registry_hash"]
    assert list((tmp_path / ".backups").glob("global_experiment_registry.json.*.bak"))
    assert list(tmp_path.glob("global_experiment_registry.json.*.tmp")) == []

```

Full Source: ``backend/tradeo/tests/test_pattern_entry_scanner.py``

``backend/tradeo/tests/test_pattern_entry_scanner.py`` ? 1307 lines, 50359 bytes, sha256 prefix ``9756c4432df307a7``

Public surface summary before full source:

```

? class `FixtureProvider` line 32 methods: fetch_ohlc
? class `StaticProvider` line 43 methods: fetch_ohlc
? class `NoBarsProvider` line 51 methods: fetch_ohlc
? class `StaticMatcher` line 56 methods: match_current

? function `session_factory` line 71`
? function `add_pattern` line 77`

```

? function `add\_shadow\_observation` line 115`
 ? function `match\_payload` line 164`
 ? function `near\_miss\_volume\_payload` line 199`
 ? function `scanner` line 244`
 ? function `test\_laboratory\_scanner\_creates\_paper\_signal\_for\_validated\_lab\_pattern` line 266`
 ? function `test\_laboratory\_matcher\_records\_feature\_parity\_and\_ambiguity\_ratio` line 324`
 ? function `test\_laboratory\_volume\_near\_miss\_opens\_shadow\_observation\_without\_ibkr` line 381`
 ? function `test\_laboratory\_near\_miss\_hard\_blocker\_does\_not\_open\_shadow\_observation` line 462`
 ? function `test\_fox\_hunter\_does\_not\_open\_lab\_near\_miss\_shadow\_observation` line 499`
 ? function `test\_current\_match\_storage\_upserts\_equivalent\_entry\_variant` line 528`
 ? function `test\_current\_match\_storage\_keeps\_distinct\_entry\_variants` line 558`
 ? function `test\_lab\_shadow\_observation\_closes\_on\_target\_without\_ibkr\_order` line 566`
 ? function `test\_lab\_shadow\_observation\_records\_market\_data\_unavailable\_without\_fallback` line 595`
 ? function `test\_lab\_shadow\_observation\_marks\_to\_market\_from\_signal\_snapshot\_after\_no\_bars` line 618`

Risk hints: IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

from __future__ import annotations

from datetime import datetime, timezone

import pandas as pd
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from tradeo.core.config import Settings
from tradeo.db.models import (
    DiscoveredPattern,
    DiscoveredPatternMatch,
    DiscoveredPatternStatus,
    Signal,
    SignalStatus,
    Trade,
    TradeStatus,
)
from tradeo.db.session import Base
from tradeo.research.novel_pattern_matcher import NovelPatternMatcher
from tradeo.research.pattern_embedding_engine import PatternEmbeddingEngine
from tradeo.services.evidence import EvidenceQuality, EvidenceType, FillProvenance
from tradeo.modules.fox_hunter.production_manifest import build_production_manifest
from tradeo.modules.laboratory.paper_observations import LAB_SHADOW_EXECUTION_MODE, LabPaperObservationService
from tradeo.modules.shared.entry_scanner import (
    PatternEntryScanner,
    PatternEntryScannerSafetyError,
)
from tradeo.tests.fixtures import fixture_ohlcv

class FixtureProvider:
    def __init__(self, symbol: str = "LABX") -> None:
        self.symbol = symbol
        self.df = fixture_ohlcv(symbol, bars=260)
        self.fetch_calls: list[tuple[str, str, str]] = []

    def fetch_ohlcv(self, symbol: str, period: str = "2y", interval: str = "1d"):
        self.fetch_calls.append((symbol.upper(), period, interval))
        return self.df.copy()

class StaticProvider:
    def __init__(self, df: pd.DataFrame) -> None:
        self.df = df

    def fetch_ohlcv(self, symbol: str, period: str = "2y", interval: str = "1d"):
        return self.df.copy()

class NoBarsProvider:
    def fetch_ohlcv(self, symbol: str, period: str = "2y", interval: str = "1d"):
        raise ValueError(f"IBKR returned no bars for {symbol}")

class StaticMatcher:
    def __init__(self, provider: FixtureProvider, matches: list[dict]) -> None:
        self.provider = provider
        self.matches = matches

    def match_current(self, *args, **kwargs):
        return {
            "patterns_checked": len({match.get("pattern_id") for match in self.matches}),
            "symbols_checked": len({match.get("symbol") for match in self.matches}),
            "matches": self.matches,
            "stored_matches": len(self.matches),
            "similarity_threshold": 0.45,
        }

def session_factory():
    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)

```

```

return sessionmaker(bind=engine, future=True)()

def add_pattern(
    db,
    provider: FixtureProvider,
    *,
    status: DiscoveredPatternStatus,
) -> DiscoveredPattern:
    window = provider.df.iloc[-20:]
    vector, _, _ = PatternEmbeddingEngine().embed(window)
    pattern = DiscoveredPattern(
        pattern_key=f"{status.value}_key",
        name=f"{status.value}_pattern",
        status=status,
        side="long",
        timeframe="1d",
        window_size=20,
        sample_count=120,
        symbol_count=12,
        year_count=3,
        score=0.9,
        reward_risk_estimate=4.0,
        expectancy_r=0.4,
        profit_factor=2.1,
        win_rate=0.55,
        stability_score=0.8,
        out_of_sample_expectancy_r=0.3,
        out_of_sample_profit_factor=1.9,
        best_rr=4.0,
        validation_passed=True,
        promotion_status=status.value,
        centroid_json=vector.tolist(),
        metrics_json={"scaler_mean": [0.0] * len(vector), "scaler_scale": [1.0] * len(vector)},
    )
    db.add(pattern)
    db.commit()
    db.refresh(pattern)
    return pattern

def add_shadow_observation(
    db,
    *,
    opened_at: datetime,
    signal_metadata: dict | None = None,
    trade_metadata: dict | None = None,
) -> tuple[Signal, Trade]:
    signal = Signal(
        symbol="LABX",
        pattern="shadow_pattern",
        side="long",
        entry=10.0,
        stop=9.0,
        target=12.0,
        reward_risk=2.0,
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=3,
        strategy_version="laboratory_pattern_1",
        status=SignalStatus.PAPER_APPROVED,
        metadata_json=signal_metadata or {"entry_module": "laboratory", "pattern_id": 1},
    )
    db.add(signal)
    db.flush()
    metadata = {
        "execution_mode": LAB_SHADOW_EXECUTION_MODE,
        "observation_only": True,
        "no_ibkr_order": True,
    }
    metadata.update(trade_metadata or {})
    trade = Trade(
        signal_id=signal.id,
        symbol="LABX",
        pattern="shadow_pattern",
        side="long",
        qty=3,
        entry=10.0,
        stop=9.0,
        target=12.0,
        status=TradeStatus.OPEN,
        opened_at=opened_at,
        metadata_json=metadata,
    )
    db.add(trade)
    db.commit()
    return signal, trade

def match_payload(**overrides):
    payload = {
        "module": "laboratory",
        "pattern_id": 1,
        "pattern_name": "dedupe_pattern",
        "pattern_key": "dedupe_key",
    }

```

```

        "pattern_status": "lab_candidate",
        "pattern_promotion_status": "lab_candidate",
        "symbol": "LABX",
        "timeframe": "1d",
        "side": "long",
        "similarity": 0.7,
        "score": 0.7,
        "entry_score": 0.7,
        "entry_gate_passed": True,
        "entry_trigger": "breakout",
        "entry_variant_id": "next_bar_limit_retest",
        "entry_variant": {"id": "next_bar_limit_retest"},
        "entry_price": 10.0,
        "stop_price": 9.0,
        "target_price": 14.0,
        "reward_risk": 4.0,
        "window_end": "2026-06-09T00:00:00+00:00",
        "status": "lab_entry_candidate",
        "notes": "test",
        "chart": {},
        "metrics": {
            "entry_variant_id": "next_bar_limit_retest",
            "entry_gate": {"passed": True, "reason": "entry gate passed"},
        },
    }
}
payload.update(overrides)
return payload

def near_miss_volume_payload(**overrides):
    payload = match_payload(
        pattern_name="near_miss_volume_pattern",
        pattern_key="near_miss_volume_key",
        similarity=0.86,
        score=0.86,
        entry_score=0.68,
        entry_gate_passed=False,
        entry_trigger="breakout",
        entry_variant_id="volume_confirmed_close",
        entry_variant={"id": "volume_confirmed_close", "order_style": "shadow_observation"},
        entry_audit={
            "available_data_cutoff_ts": "2026-06-09T00:00:00+00:00",
            "entry_eligible_ts": "2026-06-10T00:00:00+00:00",
            "source_bar_hash": "near-miss-hash",
        },
        regime={"regime_key": "market_up|uptrend|normal_vol|liquid|rs_leader"},
        regime_fit={"score": 0.74},
        metrics={
            "pattern_score": 0.86,
            "pattern_expectancy_r": 0.4,
            "pattern_profit_factor": 2.2,
            "pattern_stability_score": 0.85,
            "features": {"avg_dollar_volume": 10_000_000, "atr_pct": 0.04},
            "entry_gate": {
                "passed": False,
                "trigger": "breakout",
                "trigger_score": 1.0,
                "entry_score": 0.68,
                "volume_ratio": 0.92,
                "volume_confirmed": False,
                "atr_pct": 0.04,
                "volatility_ok": True,
                "extension_atr": 0.6,
                "not_extended": True,
                "regime_ok": True,
                "reason": "insufficient_volume",
                "rejection_reasons": ["insufficient_volume"],
            },
        },
    )
    payload.update(overrides)
    return payload

def scanner(provider: FixtureProvider, **settings_overrides) -> PatternEntryScanner:
    defaults = {
        "min_avg_dollar_volume": 0,
        "max_atr_pct": 1.0,
        "laboratory_auto_submit_paper_orders": False,
        "laboratory_market_hours_only": False,
        "fox_hunter_enabled": False,
        "fox_hunter_market_hours_only": False,
        "entry_gate_enabled": False,
        "entry_min_quality_score": 0.0,
        "entry_cooldown_minutes": 0,
        # Fixtures terminan en la barra de "hoy": sin esto el resultado del
        # matcher dependeria de si el test corre antes o despues del cierre NY.
        "discovery_match_completeBars_only": False,
        "artifacts_dir": "/tmp/tradeo-test-artifacts",
    }
    defaults.update(settings_overrides)
    settings = Settings(**defaults)
    matcher = NovelPatternMatcher(provider=provider, settings=settings)
    return PatternEntryScanner(settings=settings, matcher=matcher)

```

```

def test_laboratory_scanner_creates_paper_signal_for_validated_lab_pattern() -> None:
    db = session_factory()
    provider = FixtureProvider()
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

    result = scanner(provider).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )

    assert result["matches_found"] == 1
    assert result["entry_variants_considered"] >= 1
    assert result["signals_created"] == 1
    assert result["paper_observations_opened"] == 1
    assert result["paper_observation_trade_ids"]
    signal_id = result["signal_ids"][0]
    signal = db.get(Signal, signal_id)
    assert signal.status == SignalStatus.PAPER_APPROVED
    assert signal.human_approved is False
    assert signal.metadata_json["entry_module"] == "laboratory"
    assert signal.metadata_json["evidence_type"] == EvidenceType.SHADOW_NO_ORDER.value
    assert signal.metadata_json["evidence_quality"] == EvidenceQuality.NORMAL.value
    assert signal.metadata_json["entry_variant_id"]
    assert signal.metadata_json["entry_audit"]["available_data_cutoff_ts"]
    assert signal.metadata_json["entry_audit"]["entry_eligible_ts"]
    assert signal.metadata_json["entry_audit"]["source_bar_hash"]
    assert signal.metadata_json["regime"]["regime_key"]
    assert signal.metadata_json["entry_quality_score"] > 0
    assert signal.metadata_json["entry_quality"]["label"] in {"blocked", "weak", "watch", "actionable", "high"}
    assert isinstance(signal.metadata_json["entry_quality"]["flags"], list)
    assert signal.metadata_json["opportunity_rank"] == 1
    assert signal.metadata_json["opportunity_rank_score"] > 0
    assert signal.metadata_json["signal_snapshot"]["symbol"] == provider.symbol
    assert signal.metadata_json["signal_snapshot"]["entry_variant_id"] == signal.metadata_json["entry_variant_id"]
    assert signal.metadata_json["signal_snapshot"]["entry_audit"]["source_bar_hash"]
    assert signal.metadata_json["signal_snapshot"]["risk"]["approved"] is True
    stored_match = (
        db.query(DiscoveredPatternMatch)
        .filter(
            DiscoveredPatternMatch.metrics_json["entry_variant_id"].as_string()
            == signal.metadata_json["entry_variant_id"]
        )
        .one()
    )
    assert stored_match.metrics_json["entry_variant_id"] == signal.metadata_json["entry_variant_id"]
    assert stored_match.metrics_json["entry_variant"]["id"] == signal.metadata_json["entry_variant_id"]
    observation = db.get(Trade, result["paper_observation_trade_ids"][0])
    assert observation.status == TradeStatus.OPEN
    assert observation.signal_id == signal.id
    assert observation.metadata_json["execution_mode"] == LAB_SHADOW_EXECUTION_MODE
    assert observation.metadata_json["evidence_type"] == EvidenceType.SHADOW_NO_ORDER.value
    assert observation.metadata_json["evidence_quality"] == EvidenceQuality.NORMAL.value
    assert observation.metadata_json["no_ibkr_order"] is True

def test_laboratory_matcher_records_feature_parity_and_ambiguity_ratio() -> None:
    db = session_factory()
    provider = FixtureProvider()
    first = add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)
    vector, _, _ = PatternEmbeddingEngine().embed(provider.df.iloc[-20:])
    second = DiscoveredPattern(
        pattern_key="lab_candidate_key_second",
        name="lab_candidate_pattern_second",
        status=DiscoveredPatternStatus.LAB_CANDIDATE,
        side="long",
        timeframe="1d",
        window_size=20,
        sample_count=120,
        symbol_count=12,
        year_count=3,
        score=0.88,
        reward_risk_estimate=4.0,
        expectancy_r=0.35,
        profit_factor=2.0,
        win_rate=0.54,
        stability_score=0.78,
        out_of_sample_expectancy_r=0.25,
        out_of_sample_profit_factor=1.8,
        best_rr=4.0,
        validation_passed=True,
        promotion_status=DiscoveredPatternStatus.LAB_CANDIDATE.value,
        centroid_json=[float(value) + 0.005 for value in vector.tolist()],
        metrics_json={"scaler_mean": [0.0] * len(vector), "scaler_scale": [1.0] * len(vector)},
    )
    db.add(second)
    db.commit()
    db.refresh(second)

    settings = Settings(
        discovery_match_completeBars_only=False,
        discovery_match_max_patterns=10,
        discovery_match_max_results=10,
        entry_gate_enabled=False,
    )

```

```

)
result = NovelPatternMatcher(provider=provider, settings=settings).match_current(
    db,
    symbols=[provider.symbol],
    store=False,
)

assert result["feature_parity_contract"]["contract_id"] == PatternEmbeddingEngine.CONTRACT_ID
matches = {int(match["pattern_id"]): match for match in result["matches"]}
assert first.id in matches
ambiguity = matches[first.id]["match_ambiguity"]
assert ambiguity["second_best_pattern_id"] == second.id
assert ambiguity["ambiguity_ratio"] > 0.95
assert matches[first.id]["metrics"]["match_ambiguity"] == ambiguity
contract = matches[first.id]["metrics"]["feature_parity_contract"]
assert contract["research_lab_shared_path"] is True
assert contract["contract_id"] == PatternEmbeddingEngine.CONTRACT_ID

def test_laboratory_volume_near_miss_opens_shadow_observation_without_ibkr(monkeypatch) -> None:
    db = session_factory()
    provider = FixtureProvider()
    pattern = add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

    class FailBroker:
        def __init__(self, settings) -> None:
            self.settings = settings

        def submit_signal_bracket(self, db, signal, *, reason: str = "manual"):
            raise AssertionError("near-miss shadow observations must not call IBKR")

    monkeypatch.setattr("tradeo.modules.shared.entry_scanner.IBKRBroker", FailBroker)
    match = near_miss_volume_payload(
        pattern_id=pattern.id,
        pattern_name=pattern.name,
        pattern_key=pattern.pattern_key,
    )
    cfg = scanner(provider, entry_gate_enabled=True).settings

    result = PatternEntryScanner(settings=cfg, matcher=StaticMatcher(provider, [match])).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )

    assert cfg.laboratory_auto_submit_paper_orders is False
    assert result["rejected_by_entry_gate"] == 1
    assert result["signals_created"] == 1
    assert result["orders_submitted"] == 0
    assert result["paper_observations_opened"] == 1
    assert result["near_miss_shadow_observations_opened"] == 1
    assert result["near_miss_signal_ids"] == result["signal_ids"]
    assert result["near_miss_trade_ids"] == result["paper_observation_trade_ids"]

    signal = db.get(Signal, result["signal_ids"][0])
    assert signal.status == SignalStatus.WATCHLIST
    assert signal.human_approved is False
    signal_meta = signal.metadata_json
    required_keys = {
        "entry_variant_id",
        "regime",
        "entry_gate",
        "entry_quality",
        "opportunity_rank",
        "opportunity_rank_score",
        "near_miss",
        "near_miss_reasons",
        "entry_gate_rejection_reasons",
        "would_have_failed_entry_gate",
        "paper_only",
        "no_ibkr_order",
    }
    assert required_keys.issubset(signal_meta)
    assert signal_meta["near_miss"] is True
    assert signal_meta["near_miss_shadow"] is True
    assert signal_meta["near_miss_reasons"] == ["insufficient_volume"]
    assert signal_meta["entry_gate_rejection_reasons"] == ["insufficient_volume"]
    assert signal_meta["would_have_failed_entry_gate"] is True
    assert signal_meta["paper_only"] is True
    assert signal_meta["no_ibkr_order"] is True
    assert signal_meta["evidence_type"] == EvidenceType.NEAR_MISS_SHADOW.value
    assert signal_meta["evidence_quality"] == EvidenceQuality.NORMAL.value
    assert signal_meta["entry_quality"]["actionable"] is False
    assert signal_meta["entry_module"] == "laboratory"

    observation = db.get(Trade, result["paper_observation_trade_ids"][0])
    trade_meta = observation.metadata_json
    assert observation.status == TradeStatus.OPEN
    assert trade_meta["execution_mode"] == LAB_SHADOW_EXECUTION_MODE
    assert trade_meta["opened_reason"] == "lab_near_miss_volume_shadow_observation"
    assert required_keys.issubset(trade_meta)
    assert trade_meta["near_miss"] is True
    assert trade_meta["paper_only"] is True
    assert trade_meta["no_ibkr_order"] is True

```

```

assert trade_meta["evidence_type"] == EvidenceType.NEAR_MISS_SHADOW.value
assert trade_meta["evidence_quality"] == EvidenceQuality.NORMAL.value

def test_laboratory_near_miss_hard_blocker_does_not_open_shadow_observation() -> None:
    db = session_factory()
    provider = FixtureProvider()
    pattern = add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)
    match = near_miss_volume_payload(
        pattern_id=pattern.id,
        pattern_name=pattern.name,
        pattern_key=pattern.pattern_key,
    )
    entry_gate = dict(match["metrics"]["entry_gate"])
    entry_gate.update(
        {
            "extension_atr": 3.4,
            "not_extended": False,
            "reason": "insufficient_volume;excessive_extension",
            "rejection_reasons": ["insufficient_volume", "excessive_extension"],
        }
    )
    match["metrics"] = {**match["metrics"], "entry_gate": entry_gate}
    cfg = scanner(provider, entry_gate_enabled=True).settings

    result = PatternEntryScanner(settings=cfg, matcher=StaticMatcher(provider, [match])).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )

    assert result["rejected_by_entry_gate"] == 1
    assert result["signals_created"] == 0
    assert result["paper_observations_opened"] == 0
    assert result["near_miss_shadow_observations_opened"] == 0
    assert db.query(Signal).count() == 0
    assert db.query(Trade).count() == 0

def test_fox_hunter_does_not_open_lab_near_miss_shadow_observation() -> None:
    db = session_factory()
    provider = FixtureProvider("FOXX")
    match = near_miss_volume_payload(
        module="fox_hunter",
        pattern_status="production",
        pattern_promotion_status="production",
        symbol=provider.symbol,
        status="production_entry_candidate",
    )
    cfg = scanner(provider, entry_gate_enabled=True).settings

    result = PatternEntryScanner(settings=cfg, matcher=StaticMatcher(provider, [match])).scan(
        db,
        module="fox_hunter",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )

    assert result["rejected_by_entry_gate"] == 0
    assert result["rejected_by_production_manifest"] == 1
    assert result["signals_created"] == 0
    assert result["paper_observations_opened"] == 0
    assert result["near_miss_shadow_observations_opened"] == 0
    assert db.query(Signal).count() == 0
    assert db.query(Trade).count() == 0

def test_current_match_storage_upserts_equivalent_entry_variant() -> None:
    db = session_factory()
    NovelPatternMatcher._store_matches(db, [match_payload(score=0.51, similarity=0.61)])
    NovelPatternMatcher._store_matches(
        db,
        [
            match_payload(
                score=0.88,
                similarity=0.91,
                metrics={
                    "entry_variant_id": "next_bar_limit_retest",
                    "entry_gate": {
                        "passed": False,
                        "reason": "insufficient_volume",
                        "rejection_reasons": ["insufficient_volume"],
                    },
                },
            ),
        ],
    )

    rows = db.query(DiscoveredPatternMatch).all()

    assert len(rows) == 1
    assert rows[0].score == 0.88
    assert rows[0].similarity == 0.91

```

```

assert rows[0].metrics_json["seen_count"] == 2
assert rows[0].metrics_json["entry_gate"]["reason"] == "insufficient_volume"

def test_current_match_storage_keeps_distinct_entry_variants() -> None:
    db = session_factory()
    NovelPatternMatcher._store_matches(db, [match_payload(entry_variant_id="variant_a", metrics={"entry_variant_id": "variant_a"})])
    NovelPatternMatcher._store_matches(db, [match_payload(entry_variant_id="variant_b", metrics={"entry_variant_id": "variant_b"})])

    assert db.query(DiscoveredPatternMatch).count() == 2

def test_lab_shadow_observation_closes_on_target_without_ibkr_order() -> None:
    db = session_factory()
    opened_at = datetime(2026, 1, 2, 14, 30, tzinfo=timezone.utc)
    _, trade = add_shadow_observation(db, opened_at=opened_at)
    df = pd.DataFrame(
        {
            "open": [10.1, 10.3],
            "high": [11.0, 12.4],
            "low": [9.8, 10.2],
            "close": [10.5, 12.1],
            "volume": [1000.0, 1100.0],
        },
        index=pd.to_datetime(["2026-01-03T00:00:00Z", "2026-01-04T00:00:00Z"]),
    )

    result = LabPaperObservationService(settings=Settings(), provider=StaticProvider(df)).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 1
    assert trade.status == TradeStatus.CLOSED
    assert trade.exit_price == 12.0
    assert trade.r_multiple == 2.0
    assert trade.pnl_usd == 6.0
    assert trade.metadata_json["exit_reason"] == "target_hit"
    assert trade.metadata_json["evidence_type"] == EvidenceType.SHADOW_NO_ORDER.value
    assert trade.metadata_json["evidence_quality"] == EvidenceQuality.NORMAL.value
    assert trade.metadata_json["no_ibkr_order"] is True

def test_lab_shadow_observation_records_market_data_unavailable_without_fallback() -> None:
    db = session_factory()
    _, trade = add_shadow_observation(
        db,
        opened_at=datetime(2026, 1, 2, 14, 30, tzinfo=timezone.utc),
    )

    result = LabPaperObservationService(settings=Settings(), provider=NoBarsProvider()).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 0
    assert result["diagnosed_observations"] == 1
    assert result["data_errors"][0]["status"] == "market_data_unavailable"
    assert "IBKR returned no bars" in result["data_errors"][0]["reason"]
    assert trade.status == TradeStatus.OPEN
    assert trade.metadata_json["market_data_unavailable"] is True
    assert trade.metadata_json["shadow_lifecycle"]["last_bar"] is None
    assert trade.metadata_json["shadow_lifecycle"]["mark_to_market"] is None
    assert trade.metadata_json["evidence_type"] == EvidenceType.SHADOW_NO_ORDER.value
    assert trade.metadata_json["evidence_quality"] == EvidenceQuality.NORMAL.value
    assert trade.metadata_json["no_ibkr_order"] is True

def test_lab_shadow_observation_marks_to_market_from_signal_snapshot_after_no_bars() -> None:
    db = session_factory()
    _, trade = add_shadow_observation(
        db,
        opened_at=datetime(2026, 1, 2, 14, 30, tzinfo=timezone.utc),
        signal_metadata={
            "entry_module": "laboratory",
            "pattern_id": 1,
            "signal_snapshot": {
                "captured_at": "2026-01-02T14:29:00+00:00",
                "features": {"last_close": 10.75},
            },
        },
    )

    result = LabPaperObservationService(settings=Settings(), provider=NoBarsProvider()).close_open_observations(db)
    db.refresh(trade)

    lifecycle = trade.metadata_json["shadow_lifecycle"]
    assert result["closed_observations"] == 0
    assert result["diagnosed_observations"] == 1
    assert trade.status == TradeStatus.OPEN
    assert lifecycle["status"] == "market_data_unavailable"
    assert lifecycle["fallback_used"] is True
    assert lifecycle["fallback_source"] == "signal.metadata_json.signal_snapshot.features.last_close"
    assert lifecycle["mark_to_market"]["price"] == 10.75
    assert lifecycle["mark_to_market"]["unrealized_pnl"] == 2.25
    assert lifecycle["mark_to_market"]["entry"] == 10.0
    assert trade.metadata_json["evidence_type"] == EvidenceType.SHADOW_NO_ORDER.value
    assert trade.metadata_json["evidence_quality"] == EvidenceQuality.DEGRADED.value
    assert trade.metadata_json["no_ibkr_order"] is True

```



```

def test_lab_shadow_observation_closes_from_metadata_bar_after_no_bars() -> None:
    db = session_factory()
    _, trade = add_shadow_observation(
        db,
        opened_at=datetime(2026, 1, 2, 14, 30, tzinfo=timezone.utc),
        trade_metadata={
            "latest_bar": {
                "timestamp": "2026-01-02T14:35:00+00:00",
                "open": 10.2,
                "high": 12.4,
                "low": 10.1,
                "close": 12.1,
                "volume": 1200,
            },
        },
    )

    result = LabPaperObservationService(settings=Settings(), provider=NoBarsProvider()).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 1
    assert result["diagnosed_observations"] == 0
    assert trade.status == TradeStatus.CLOSED
    assert trade.exit_price == 12.0
    assert trade.metadata_json["exit_reason"] == "target_hit"
    assert trade.metadata_json["evidence_type"] == EvidenceType.SHADOW_NO_ORDER.value
    assert trade.metadata_json["evidence_quality"] == EvidenceQuality.DEGRADED.value
    assert trade.metadata_json["fallback_used"] is True
    assert trade.metadata_json["no_ibkr_order"] is True


def test_laboratory_scanner_marks_ibkr_bracket_failure_retryable(monkeypatch) -> None:
    db = session_factory()
    provider = FixtureProvider()
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

    class BrokenBroker:
        def __init__(self, settings) -> None:
            self.settings = settings

        def submit_signal_bracket(self, db, signal, *, reason: str = "manual"):
            raise RuntimeError("IBKR did not acknowledge every bracket leg with a permId")

    monkeypatch.setattr("tradeo.modules.shared.entry_scanner.IBKRBroker", BrokenBroker)

    result = scanner(provider, ibkr_readonly=False).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=True,
    )

    signal = db.get(Signal, result["signal_ids"][0])
    outcome = signal.metadata_json["execution_outcome"]
    assert result["orders_submitted"] == 0
    assert result["order_errors"][0]["reason_code"] == "ibkr_bracket_not_accepted"
    assert result["order_errors"][0]["retryable"] is True
    assert outcome["status"] == "retry_order_submission"
    assert outcome["next_action"] == "retry_order_submission"


def test_laboratory_scanner_rejects_match_without_entry_trigger() -> None:
    db = session_factory()
    provider = FixtureProvider()
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

    class NoTriggerMatcher:
        def match_current(self, *args, **kwargs):
            return {
                "patterns_checked": 1,
                "symbols_checked": 1,
                "matches": [
                    {
                        "module": "laboratory",
                        "pattern_id": 1,
                        "pattern_name": "no_trigger_pattern",
                        "pattern_key": "no_trigger_key",
                        "pattern_status": "lab_candidate",
                        "pattern_promotion_status": "lab_candidate",
                        "symbol": provider.symbol,
                        "timeframe": "1d",
                        "side": "long",
                        "similarity": 0.9,
                        "score": 0.9,
                        "entry_score": 0.2,
                        "entry_gate_passed": False,
                        "entry_trigger": "no_operational_trigger",
                        "entry_price": 10.0,
                        "stop_price": 9.0,
                        "target_price": 14.0,
                        "reward_risk": 4.0,
                        "metrics": {
                            "features": {"avg_dollar_volume": 10_000_000, "atr_pct": 0.04},
                            "entry_gate": {

```

```

        "passed": False,
        "trigger": "no_operational_trigger",
        "entry_score": 0.2,
    },
},
    },
    ],
    "stored_matches": 1,
    "similarity_threshold": 0.45,
}

result = PatternEntryScanner(
    settings=scanner(provider, entry_gate_enabled=True).settings,
    matcher=NoTriggerMatcher(),
).scan(db, module="laboratory", symbols=[provider.symbol], store_signals=True)

assert result["rejected_by_entry_gate"] == 1
assert result["signals_created"] == 0
assert db.query(Signal).count() == 0

def test_laboratory_scanner_rejects_low_quality_match() -> None:
    db = session_factory()
    provider = FixtureProvider()
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

    class LowQualityMatcher:
        def match_current(self, *args, **kwargs):
            return {
                "patterns_checked": 1,
                "symbols_checked": 1,
                "matches": [
                    {
                        "module": "laboratory",
                        "pattern_id": 1,
                        "pattern_name": "low_quality_pattern",
                        "pattern_key": "low_quality_key",
                        "pattern_status": "lab_candidate",
                        "pattern_promotion_status": "lab_candidate",
                        "symbol": provider.symbol,
                        "timeframe": "1d",
                        "side": "long",
                        "similarity": 0.1,
                        "score": 0.1,
                        "entry_score": 0.1,
                        "entry_gate_passed": True,
                        "entry_trigger": "momentum_close",
                        "entry_price": 10.0,
                        "stop_price": 9.0,
                        "target_price": 14.0,
                        "reward_risk": 4.0,
                        "metrics": {
                            "pattern_score": 0.1,
                            "pattern_expectancy_r": 0.0,
                            "pattern_profit_factor": 0.0,
                            "pattern_stability_score": 0.1,
                            "features": {"avg_dollar_volume": 10_000_000, "atr_pct": 0.04},
                            "entry_gate": {
                                "passed": True,
                                "trigger": "momentum_close",
                                "entry_score": 0.1,
                                "volume_ratio": 1.5,
                                "extension_atr": 0.5,
                                "regime_ok": True,
                            },
                        },
                    },
                ],
            },
            ],
            "stored_matches": 1,
            "similarity_threshold": 0.45,
        }

    cfg = scanner(provider, entry_min_quality_score=0.8).settings
    result = PatternEntryScanner(settings=cfg, matcher=LowQualityMatcher()).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=True,
    )

    assert result["rejected_by_entry_quality"] == 1
    assert result["signals_created"] == 0
    assert db.query(Signal).count() == 0

```

```

def test_laboratory_scanner_processes_best_ranked_opportunity_first() -> None:
    db = session_factory()
    provider = FixtureProvider()

    class RankingMatcher:
        def match_current(self, *args, **kwargs):
            def match(name: str, score: float, entry_score: float, expectancy: float):
                return {
                    "module": "laboratory",
                    "pattern_id": 1 if name == "weak_pattern" else 2,
                    "pattern_name": name,
                }

```

```

        "pattern_key": f"{name}_key",
        "pattern_status": "lab_candidate",
        "pattern_promotion_status": "lab_candidate",
        "symbol": provider.symbol,
        "timeframe": "1d",
        "side": "long",
        "similarity": score,
        "score": score,
        "entry_score": entry_score,
        "entry_gate_passed": True,
        "entry_trigger": "breakout",
        "entry_price": 10.0,
        "stop_price": 9.0,
        "target_price": 14.0,
        "reward_risk": 4.0,
        "metrics": {
            "pattern_score": score,
            "pattern_expectancy_r": expectancy,
            "pattern_profit_factor": 2.0,
            "pattern_stability_score": score,
            "features": {"avg_dollar_volume": 10_000_000, "atr_pct": 0.04},
            "entry_gate": {
                "passed": True,
                "trigger": "breakout",
                "entry_score": entry_score,
                "volume_ratio": 2.0,
                "extension_atr": 0.5,
                "regime_ok": True,
            },
        },
    },
}

return {
    "patterns_checked": 2,
    "symbols_checked": 1,
    "matches": [
        match("weak_pattern", 0.2, 0.2, 0.0),
        match("strong_pattern", 0.9, 0.9, 0.5),
    ],
    "stored_matches": 2,
    "similarity_threshold": 0.45,
}

result = PatternEntryScanner(
    settings=scanner(provider, entry_min_quality_score=0.0).settings,
    matcher=RankingMatcher(),
).scan(db, module="laboratory", symbols=[provider.symbol], store_signals=True)

assert result["signals_created"] == 2
first_signal = db.get(Signal, result["signal_ids"][0])
second_signal = db.get(Signal, result["signal_ids"][1])
assert first_signal.pattern == "strong_pattern"
assert first_signal.metadata_json["opportunity_rank"] == 1
assert second_signal.metadata_json["opportunity_rank"] == 2

def test_laboratory_scanner_prefers_entry_variant_with_paper_history() -> None:
    db = session_factory()
    provider = FixtureProvider()
    historical_signal = Signal(
        symbol=provider.symbol,
        pattern="adaptive_pattern",
        side="long",
        entry=10.0,
        stop=9.0,
        target=14.0,
        reward_risk=4.0,
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=1,
        strategy_version="laboratory_pattern_1",
        status=SignalStatus.EXECUTED,
        metadata_json={
            "entry_module": "laboratory",
            "entry_variant_id": "next_bar_limit_retest",
            "regime": {"regime_key": "market_up|uptrend|normal_vol|liquid|rs_leader"},
        },
    )
    db.add(historical_signal)
    db.flush()
    for index in range(10):
        db.add(
            Trade(
                signal_id=historical_signal.id,
                symbol=provider.symbol,
                pattern="adaptive_pattern",
                side="long",
                qty=1,
                entry=10.0,
                stop=9.0,
                target=14.0,
                status=TradeStatus.CLOSED,
                pnl_usd=20.0,
                r_multiple=2.0,
                evidence_type=EvidenceType.IBKR_PAPER_FILL.value,
            )
        )

```

```

        evidence_quality=EvidenceQuality.NORMAL.value,
        metadata_json={
            "execution_mode": "ibkr",
            "ibkr_mode": "paper",
            "evidence_type": EvidenceType.IBKR_PAPER_FILL.value,
            "evidence_quality": EvidenceQuality.NORMAL.value,
            "fill_provenance": FillProvenance.BROKER_EXECUTION.value,
            "broker_execution_hash": f"scanner-history-fill-{index}",
            "broker_execution_time": f"2026-01-{index + 1:02d}T16:00:00+00:00",
            "commission": 0.0,
        },
    )
)
db.commit()

class VariantMatcher:
    def match_current(self, *args, **kwargs):
    def match(variant: str):
        return {
            "module": "laboratory",
            "pattern_id": 1,
            "pattern_name": "adaptive_pattern",
            "pattern_key": "adaptive_key",
            "pattern_status": "lab_candidate",
            "pattern_promotion_status": "lab_candidate",
            "symbol": provider.symbol,
            "timeframe": "1d",
            "side": "long",
            "similarity": 0.8,
            "score": 0.8,
            "entry_score": 0.8,
            "entry_gate_passed": True,
            "entry_trigger": variant,
            "entry_variant_id": variant,
            "entry_variant": {"id": variant, "order_style": "next_bar_limit"},
            "entry_audit": {"source_bar_hash": "abc", "entry_eligible_ts": "2026-06-10T00:00:00+00:00"},
            "regime": {"regime_key": "market_up|uptrend|normal_vol|liquid|rs_leader"},
            "regime_fit": {"score": 0.8},
            "entry_price": 10.0,
            "stop_price": 9.0,
            "target_price": 14.0,
            "reward_risk": 4.0,
            "metrics": {
                "pattern_score": 0.8,
                "pattern_expectancy_r": 0.2,
                "pattern_profit_factor": 2.0,
                "pattern_stability_score": 0.8,
                "features": {"avg_dollar_volume": 10_000_000, "atr_pct": 0.04},
                "entry_gate": {
                    "passed": True,
                    "trigger": variant,
                    "entry_score": 0.8,
                    "volume_ratio": 2.0,
                    "extension_atr": 0.5,
                    "regime_ok": True,
                },
            },
        },
    }

    return {
        "patterns_checked": 1,
        "symbols_checked": 1,
        "matches": [match("momentum_close_next_bar_limit"), match("next_bar_limit_retest")],
        "stored_matches": 2,
        "similarity_threshold": 0.45,
    }

result = PatternEntryScanner(
    settings=scanner(provider, entry_min_quality_score=0.0).settings,
    matcher=VariantMatcher(),
).scan(db, module="laboratory", symbols=[provider.symbol], store_signals=True)

assert result["entry_variants_considered"] == 2
signal = db.get(Signal, result["signal_ids"][0])
assert signal.metadata_json["entry_variant_id"] == "next_bar_limit_retest"
assert signal.metadata_json["opportunity_rank_components"]["history_count"] >= 1
assert signal.metadata_json["opportunity_rank_components"]["history_profit_factor"] > 0
assert signal.metadata_json["opportunity_rank_components"]["history_decay"] >= 0
assert signal.metadata_json["opportunity_rank_reason"] == "paper_history_weighted"

def test_laboratory_scanner_respects_symbol_pattern_cooldown() -> None:
    db = session_factory()
    provider = FixtureProvider()
    pattern = add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)
    db.add(
        Signal(
            symbol=provider.symbol,
            pattern=pattern.name,
            side="long",
            entry=10.0,
            stop=9.0,
            target=14.0,
            reward_risk=4.0,
            confidence=0.7,
            composite_score=0.7,

```

```

        risk_usd=10.0,
        suggested_qty=1,
        strategy_version=f"laboratory_pattern_{pattern.id}",
        status=SignalStatus.EXPIRED,
        metadata_json={"entry_module": "laboratory", "pattern_id": pattern.id},
    )
)
db.commit()

result = scanner(provider, entry_cooldown_minutes=60).scan(
    db,
    module="laboratory",
    symbols=[provider.symbol],
    store_signals=True,
)

assert result["skipped_cooldown"] == 1
assert result["signals_created"] == 0

def test_laboratory_scanner_skips_signal_creation_when_market_closed(monkeypatch) -> None:
    db = session_factory()
    provider = FixtureProvider()
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

    monkeypatch.setattr(
        "tradeo.modules.shared.entry_scanner.market_session_status",
        lambda: {
            "market": "us_equities",
            "timezone": "America/New_York",
            "regular_session_open": False,
            "state": "market_closed",
            "checked_at": "2026-06-09T01:30:00-04:00",
            "regular_hours": "09:30-16:00",
        },
    )

    result = scanner(provider, laboratory_market_hours_only=True).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )

    assert result["skipped_reason"] == "market_closed"
    assert result["symbols_checked"] == 0
    assert result["signals_created"] == 0
    assert db.query(Signal).count() == 0

def test_fox_hunter_ignores_lab_patterns_and_uses_production_only() -> None:
    db = session_factory()
    provider = FixtureProvider("FOXX")
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)
    add_pattern(db, provider, status=DiscoveredPatternStatus.DIRECTOR_REVIEW)
    production = add_pattern(db, provider, status=DiscoveredPatternStatus.PRODUCTION)

    blocked = scanner(provider).scan(
        db,
        module="fox_hunter",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )
    assert blocked["patterns_checked"] == 0
    assert blocked["signals_created"] == 0

    production.metrics_json = {
        **(production.metrics_json or {}),
        "production_manifest": build_production_manifest(
            production,
            reviewer="test_director",
            evidence_packet={"approved_for_production": True, "ibkr_paper_fills": 30},
        ),
    }
    db.add(production)
    db.commit()

    result = scanner(provider).scan(
        db,
        module="fox_hunter",
        symbols=[provider.symbol],
        store_signals=True,
        execute_orders=False,
    )

    assert result["patterns_checked"] == 1
    assert result["signals_created"] == 1
    signal = db.get(Signal, result["signal_ids"][0])
    assert signal.pattern == production.name
    assert signal.status == SignalStatus.PENDING_HUMAN_APPROVAL
    assert signal.metadata_json["entry_module"] == "fox_hunter"

def test_laboratory_refuses_live_port_execution() -> None:

```

```

db = session_factory()
provider = FixtureProvider()
add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)

live_port_scanner = scanner(
    provider,
    trading_mode="paper",
    ibkr_port=4001,
)

try:
    live_port_scanner.scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        execute_orders=True,
    )
except PatternEntryScannerSafetyError as exc:
    assert "refuses live IBKR ports" in str(exc)
else:
    raise AssertionError("Laboratory must not execute through live IBKR ports")

def test_laboratory_matcher_reuses_symbol_data_across_patterns() -> None:
    db = session_factory()
    provider = FixtureProvider()
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_CANDIDATE)
    add_pattern(db, provider, status=DiscoveredPatternStatus.LAB_WATCHLIST)

    result = scanner(provider).scan(
        db,
        module="laboratory",
        symbols=[provider.symbol],
        store_signals=False,
        execute_orders=False,
    )

    assert result["patterns_checked"] == 2
    assert result["matches_found"] == 2
    assert provider.fetch_calls.count(("SPY", "3mo", "1d")) == 1
    assert provider.fetch_calls.count(("QQQ", "3mo", "1d")) == 1
    assert provider.fetch_calls.count((provider.symbol, "3mo", "1d")) == 1
    benchmark_regime_calls = [
        call
        for call in provider.fetch_calls
        if call[0] == "SPY" and call[2] == "1d" and call[1] != "3mo"
    ]
    assert len(benchmark_regime_calls) == 1
    assert len(provider.fetch_calls) == 4

def near_miss_match(provider: FixtureProvider, **overrides):
    payload = {
        "module": "laboratory",
        "pattern_id": 42,
        "pattern_name": "near_miss_pattern",
        "pattern_key": "near_miss_key",
        "pattern_status": "lab_candidate",
        "pattern_promotion_status": "lab_candidate",
        "symbol": provider.symbol,
        "timeframe": "1d",
        "side": "long",
        "similarity": 0.7,
        "score": 0.72,
        "entry_score": 0.56,
        "entry_gate_passed": False,
        "entry_trigger": "next_bar_stop_confirmation",
        "entry_variant_id": "next_bar_stop_confirmation",
        "entry_variant": {"id": "next_bar_stop_confirmation", "order_style": "next_bar_stop"},
        "entry_audit": {
            "source_bar_hash": "near-miss-hash",
            "entry_eligible_ts": "2026-06-10T00:00:00+00:00",
        },
    },
    "regime": {"regime_key": "market_mixed|uptrend|normal_vol|liquid|rs_leader"},
    "regime_fit": {"score": 0.7},
    "entry_price": 10.0,
    "stop_price": 9.0,
    "target_price": 14.0,
    "reward_risk": 4.0,
    "metrics": {
        "pattern_score": 0.8,
        "pattern_expectancy_r": 0.3,
        "pattern_profit_factor": 2.0,
        "pattern_stability_score": 0.8,
        "features": {"avg_dollar_volume": 10_000_000, "atr_pct": 0.04},
        "entry_gate": {
            "passed": False,
            "reason": "insufficient_volume",
            "rejection_reasons": ["insufficient_volume"],
            "trigger": "next_bar_stop_confirmation",
            "entry_score": 0.56,
            "volume_ratio": 0.45,
            "extension_atr": 0.4,
            "regime_ok": True,
        },
    },
}

```

```

    }
    payload.update(overrides)
    return payload

def test_laboratory_opens_near_miss_shadow_observation_for_soft_volume_failure() -> None:
    db = session_factory()
    provider = FixtureProvider("MISS")

    class NearMissMatcher:
        def match_current(self, *args, **kwargs):
            return {
                "patterns_checked": 1,
                "symbols_checked": 1,
                "matches": [near_miss_match(provider)],
                "stored_matches": 1,
                "similarity_threshold": 0.45,
            }

    result = PatternEntryScanner(
        settings=scanner(provider, entry_gate_enabled=True).settings,
        matcher=NearMissMatcher(),
    ).scan(db, module="laboratory", symbols=[provider.symbol], store_signals=True, execute_orders=False)

    assert result["signals_created"] == 1
    assert result["near_miss_shadow_observations_opened"] == 1
    assert result["orders_submitted"] == 0
    assert result["rejected_by_entry_gate"] == 1
    signal = db.get(Signal, result["signal_ids"][0])
    assert signal.metadata_json["near_miss"] is True
    assert signal.metadata_json["evidence_type"] == EvidenceType.NEAR_MISS_SHADOW.value
    assert signal.metadata_json["near_miss_shadow"] is True
    assert signal.metadata_json["would_have_failed_entry_gate"] is True
    assert signal.metadata_json["paper_only"] is True
    assert signal.metadata_json["no_ibkr_order"] is True
    assert signal.metadata_json["entry_variant_id"] == "next_bar_stop_confirmation"
    assert signal.metadata_json["regime"]["regime_key"]
    assert signal.metadata_json["entry_gate"]["near_miss_shadow"] is True
    assert signal.metadata_json["entry_quality"]["near_miss_shadow"] is True
    observation = db.get(Trade, result["paper_observation_trade_ids"][0])
    assert observation.metadata_json["execution_mode"] == LAB_SHADOW_EXECUTION_MODE
    assert observation.metadata_json["evidence_type"] == EvidenceType.NEAR_MISS_SHADOW.value
    assert observation.metadata_json["near_miss_shadow"] is True
    assert observation.metadata_json["no_ibkr_order"] is True

def test_laboratory_does_not_open_near_miss_shadow_for_hard_entry_blocker() -> None:
    db = session_factory()
    provider = FixtureProvider("HARD")
    hard_match = near_miss_match(
        provider,
        entry_score=0.2,
        metrics={
            **near_miss_match(provider)["metrics"],
            "entry_gate": {
                **near_miss_match(provider)["metrics"]["entry_gate"],
                "reason": "weak_entry_score;insufficient_volume",
                "rejection_reasons": ["weak_entry_score", "insufficient_volume"],
                "entry_score": 0.2,
            },
        },
    )

    class HardBlockerMatcher:
        def match_current(self, *args, **kwargs):
            return {
                "patterns_checked": 1,
                "symbols_checked": 1,
                "matches": [hard_match],
                "stored_matches": 1,
                "similarity_threshold": 0.45,
            }

    result = PatternEntryScanner(
        settings=scanner(provider, entry_gate_enabled=True).settings,
        matcher=HardBlockerMatcher(),
    ).scan(db, module="laboratory", symbols=[provider.symbol], store_signals=True, execute_orders=False)

    assert result["signals_created"] == 0
    assert result["near_miss_shadow_observations_opened"] == 0
    assert result["rejected_by_entry_gate"] == 1
    assert db.query(Trade).count() == 0

```

Full Source: ``backend/tradeo/tests/test_shadow_canonical_outcome_parity.py``

``backend/tradeo/tests/test_shadow_canonical_outcome_parity.py`` ? 301 lines, 9913 bytes, sha256 prefix ``9cbbbeef6cdb475c``

Public surface summary before full source:

? class ``FakeProvider`` line 46 methods: `fetch_ohlcv`

? function ``session_factory`` line 40`

? function ``make_frame`` line 54`

```

? function `bar` line 61`
? function `add_shadow_observation` line 65`
? function `service` line 119`
? function `test_gap_through_stop_on_first_bar_fills_at_open` line 126`
? function `test_gap_through_stop_on_later_bar_fills_at_open` line 142`
? function `test_gap_past_target_fills_at_target_not_open` line 160`
? function `test_intrabar_stop_and_target_resolves_to_stop` line 178`
? function `test_short_side_gap_through_stop_fills_at_open` line 193`
? function `test_time_stop_waits_for_full_holding_window` line 211`
? function `test_canonical_outcome_block_is_persisted` line 234`
? function `test_shadow_exit_matches_canonical_engine_directly` line 260`

```

Risk hints: IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

"""ShadowTracker canonical-outcome parity tests (informe §6).

```

```

Shadow observation exits must use the same triple-barrier fill rules as the
backtester and Research RR simulation (`triple_barrier_outcome`):

```

- a bar that opens through the stop fills at the OPEN (worse than the stop), never at the stop price;
- a bar that opens through the target fills at the TARGET (a limit order does not collect the gap);
- intrabar stop+target in the same bar resolves to the STOP (conservative);
- time stops wait for the full holding window before closing at the close.

```

The closed trade also persists a `canonical_outcome` metadata block so any
gate decision can be reproduced against the canonical engine output.
"""

```

```

from __future__ import annotations

from datetime import datetime, timedelta, timezone

import pandas as pd
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from tradeo.core.config import Settings
from tradeo.db.models import Signal, SignalStatus, Trade, TradeStatus
from tradeo.db.session import Base
from tradeo.modules.laboratory.paper_observations import (
    CANONICAL_EXIT_REASON_MAP,
    LabPaperObservationService,
)
from tradeo.research.quant_validation import triple_barrier_outcome
from tradeo.services.evidence import EvidenceQuality, EvidenceType

OPENED_AT = datetime(2026, 6, 1, 15, 0, tzinfo=timezone.utc)
FIRST_BAR_AT = datetime(2026, 6, 2, 15, 0, tzinfo=timezone.utc)

def session_factory():
    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)
    return sessionmaker(bind=engine, future=True)()

class FakeProvider:
    def __init__(self, frame: pd.DataFrame) -> None:
        self._frame = frame

    def fetch_ohlc(self, symbol: str, period: str = "6mo", interval: str = "1d") -> pd.DataFrame:
        return self._frame

def make_frame(rows: list[dict], start: datetime = FIRST_BAR_AT) -> pd.DataFrame:
    index = pd.DatetimeIndex(
        [pd.Timestamp(start + timedelta(days=offset)) for offset in range(len(rows))]
    )
    return pd.DataFrame(rows, index=index)

def bar(open_: float, high: float, low: float, close: float) -> dict:
    return {"open": open_, "high": high, "low": low, "close": close, "volume": 1000.0}

def add_shadow_observation(
    db,
    *,
    side: str = "long",
    entry: float = 10.0,
    stop: float = 9.0,
    target: float = 14.0,
) -> Trade:
    signal = Signal(
        symbol="AAPL",
        pattern="cup_handle",
        side=side,
        entry=entry,

```



```

        stop=stop,
        target=target,
        reward_risk=abs(target - entry) / max(abs(entry - stop), 1e-9),
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=5,
        status=SignalStatus.EXECUTED,
        human_approved=True,
        metadata_json={},
    )
    db.add(signal)
    db.commit()
    db.refresh(signal)
    trade = Trade(
        signal_id=signal.id,
        symbol="AAPL",
        pattern="cup_handle",
        side=side,
        qty=5,
        entry=entry,
        stop=stop,
        target=target,
        status=TradeStatus.OPEN,
        opened_at=OPENED_AT,
        evidence_type=EvidenceType.SHADOW_NO_ORDER.value,
        evidence_quality=EvidenceQuality.NORMAL.value,
        metadata_json={
            "execution_mode": "lab_shadow_observation",
            "observation_only": True,
            "no_ibkr_order": True,
            "evidence_type": EvidenceType.SHADOW_NO_ORDER.value,
            "evidence_quality": EvidenceQuality.NORMAL.value,
        },
    )
    db.add(trade)
    db.commit()
    db.refresh(trade)
    return trade

def service(frame: pd.DataFrame, *, forwardBars: str = "5,10,20") -> LabPaperObservationService:
    return LabPaperObservationService(
        settings=Settings(discovery_forwardBars=forwardBars),
        provider=FakeProvider(frame),
    )

def test_gap_through_stop_on_first_bar_fills_at_open() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    # First bar after opened_at opens at 8.4, through the 9.0 stop.
    frame = make_frame([bar(8.4, 8.6, 8.2, 8.5)])

    result = service(frame).close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 1
    assert trade.status == TradeStatus.CLOSED
    assert trade.exit_price == pytest.approx(8.4)
    assert trade.r_multiple == pytest.approx(-1.6)
    assert trade.metadata_json["exit_reason"] == "stop_gap"

def test_gap_through_stop_on_later_bar_fills_at_open() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    frame = make_frame(
        [
            bar(10.0, 10.5, 9.8, 10.2),
            bar(8.5, 8.7, 8.3, 8.6),
        ]
    )

    service(frame).close_open_observations(db)
    db.refresh(trade)

    assert trade.exit_price == pytest.approx(8.5)
    assert trade.r_multiple == pytest.approx(-1.5)
    assert trade.metadata_json["exit_reason"] == "stop_gap"

def test_gap_past_target_fills_at_target_not_open() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    frame = make_frame(
        [
            bar(10.0, 10.5, 9.8, 10.2),
            bar(15.0, 15.5, 14.8, 15.2),
        ]
    )

    service(frame).close_open_observations(db)
    db.refresh(trade)

    assert trade.exit_price == pytest.approx(14.0)

```

```

assert trade.r_multiple == pytest.approx(4.0)
assert trade.metadata_json["exit_reason"] == "target_gap"

def test_intrabar_stop_and_target_resolves_to_stop() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    # Single bar touches both barriers without opening through either.
    frame = make_frame([bar(10.0, 14.5, 8.5, 10.0)])

    service(frame).close_open_observations(db)
    db.refresh(trade)

    assert trade.exit_price == pytest.approx(9.0)
    assert trade.r_multiple == pytest.approx(-1.0)
    assert trade.metadata_json["exit_reason"] == "stop_hit"
    assert trade.metadata_json["canonical_outcome"]["reason"] == "stop_and_target_conservative"

def test_short_side_gap_through_stop_fills_at_open() -> None:
    db = session_factory()
    trade = add_shadow_observation(db, side="short", entry=10.0, stop=11.0, target=6.0)
    frame = make_frame(
        [
            bar(10.0, 10.5, 9.8, 10.2),
            bar(11.6, 11.8, 11.4, 11.7),
        ]
    )

    service(frame).close_open_observations(db)
    db.refresh(trade)

    assert trade.exit_price == pytest.approx(11.6)
    assert trade.r_multiple == pytest.approx(-1.6)
    assert trade.metadata_json["exit_reason"] == "stop_gap"

def test_time_stop_waits_for_full_holding_window() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    quiet = bar(10.0, 10.4, 9.8, 10.1)
    # Window is 3 bars; only 2 available -> must stay open.
    frame = make_frame([quiet, quiet])

    result = service(frame, forward_bars="3").close_open_observations(db)
    db.refresh(trade)

    assert result["closed_observations"] == 0
    assert trade.status == TradeStatus.OPEN

    # With the full window available it closes at the last close.
    frame = make_frame([quiet, quiet, bar(10.0, 10.4, 9.8, 10.3)])
    service(frame, forward_bars="3").close_open_observations(db)
    db.refresh(trade)

    assert trade.status == TradeStatus.CLOSED
    assert trade.exit_price == pytest.approx(10.3)
    assert trade.metadata_json["exit_reason"] == "time_stop"

def test_canonical_outcome_block_is_persisted() -> None:
    db = session_factory()
    trade = add_shadow_observation(db)
    frame = make_frame(
        [
            bar(10.0, 11.0, 9.5, 10.5),
            bar(10.5, 14.5, 10.0, 14.0),
        ]
    )

    service(frame).close_open_observations(db)
    db.refresh(trade)

    block = trade.metadata_json["canonical_outcome"]
    assert block["engine"] == "triple_barrier_outcome"
    assert block["status"] == "ok"
    assert block["reason"] == "target"
    assert block["conservative_both"] is True
    assert block["round_trip_cost_R"] == 0.0
    assert block["bars_held"] == 2
    assert block["r_multiple"] == pytest.approx(4.0)
    assert trade.metadata_json["exit_reason"] == "target_hit"
    assert trade.r_multiple == pytest.approx(block["r_multiple"])

@pytest.mark.parametrize("side", ["long", "short"])
def test_shadow_exit_matches_canonical_engine_directly(side: str) -> None:
    """Engine-level parity: the persisted exit equals triple_barrier_outcome
    run over the same synthetic-entry construction."""
    db = session_factory()
    if side == "long":
        trade = add_shadow_observation(db, side="long", entry=10.0, stop=9.0, target=14.0)
        rows = [
            bar(10.0, 10.8, 9.4, 10.2),
            bar(8.6, 9.2, 8.4, 9.0),
        ]

```

```

else:
    trade = add_shadow_observation(db, side="short", entry=10.0, stop=11.0, target=6.0)
    rows = [
        bar(10.0, 10.6, 9.2, 9.8),
        bar(9.5, 9.7, 5.8, 6.1),
    ]
    frame = make_frame(rows)

    service(frame).close_open_observations(db)
    db.refresh(trade)

    entry = float(trade.entry)
    expected = triple_barrier_outcome(
        [entry, entry, *(r["open"] for r in rows)],
        [entry, entry, *(r["high"] for r in rows)],
        [entry, entry, *(r["low"] for r in rows)],
        [entry, entry, *(r["close"] for r in rows)],
        signal_index=0,
        side=-1 if side == "short" else 1,
        stop_price=float(trade.stop),
        target_price=float(trade.target),
        max_bars=21,
        entry_price=entry,
        gap_entry_policy="skip",
        conservative_both=True,
        round_trip_cost_R=0.0,
    )

    assert trade.status == TradeStatus.CLOSED
    assert trade.exit_price == pytest.approx(float(expected["exit_price"]))
    assert trade.r_multiple == pytest.approx(float(expected["R"]))
    assert trade.metadata_json["exit_reason"] == CANONICAL_EXIT_REASON_MAP[str(expected["reason"])]

```

Full Source: `backend/tradeo/tests/test\_effective\_sample\_weights.py`

`backend/tradeo/tests/test\_effective\_sample\_weights.py` ? 228 lines, 7503 bytes, sha256 prefix `4ac65f9636e7c0c1`

Public surface summary before full source:

```

? function `session_factory` line 28`
? function `add_pattern` line 34`
? function `add_paper_fill` line 52`
? function `test_same_symbol_same_day_cluster_counts_as_one_effective_sample` line 115`
? function `test_mixed_clusters_report_expected_n_eff_and_kish` line 134`
? function `test_independent_fills_keep_full_effective_sample` line 150`
? function `test_gate_blocks_concentrated_fills_even_above_raw_count_minimum` line 169`
? function `test_gate_persists_per_trade_weights_in_trade_metadata` line 189`
? function `test_gate_accepts_diversified_effective_sample` line 210`

```

Risk hints: IBKR/broker/data dependency; JSON metadata contract; schema drift risk.

```

"""Persisted effective-sample weights for Director paper fills (informe §4.7)."""

from __future__ import annotations

from datetime import datetime, timedelta, timezone

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from tradeo.db.models import (
    DiscoveredPattern,
    DiscoveredPatternStatus,
    Signal,
    SignalStatus,
    Trade,
    TradeStatus,
)
from tradeo.db.session import Base
from tradeo.services.director_review_gate import DirectorReviewGate
from tradeo.services.effective_sample import (
    EFFECTIVE_SAMPLE_METHOD,
    effective_sample_summary,
)
from tradeo.services.evidence import EvidenceQuality, EvidenceType, FillProvenance

def session_factory():
    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)
    return sessionmaker(bind=engine, future=True)()

def add_pattern(db) -> DiscoveredPattern:
    pattern = DiscoveredPattern(
        pattern_key="LAB_PATTERN_1",
        name="LAB_PATTERN_1",
        status=DiscoveredPatternStatus.LAB_CANDIDATE,
        promotion_status="lab_candidate",
    )

```

```

        validation_passed=True,
        expectancy_r=0.25,
        profit_factor=1.8,
        best_expectancy_r=0.4,
        best_profit_factor=2.0,
    )
    db.add(pattern)
    db.commit()
    db.refresh(pattern)
    return pattern

def add_paper_fill(
    db,
    pattern: DiscoveredPattern,
    index: int,
    *,
    symbol: str,
    closed_at: datetime,
    r_multiple: float = 1.0,
) -> None:
    signal = Signal(
        symbol=symbol,
        pattern=pattern.name,
        side="long",
        entry=10.0,
        stop=9.0,
        target=14.0,
        reward_risk=4.0,
        confidence=0.7,
        composite_score=0.7,
        risk_usd=10.0,
        suggested_qty=1,
        strategy_version=f"laboratory_pattern_{pattern.id}",
        status=SignalStatus.EXECUTED,
        human_approved=True,
        metadata_json={"entry_module": "laboratory", "pattern_id": pattern.id},
    )
    db.add(signal)
    db.flush()
    db.add(
        Trade(
            signal_id=signal.id,
            symbol=symbol,
            pattern=pattern.name,
            side="long",
            qty=1,
            entry=10.0,
            stop=9.0,
            target=14.0,
            status=TradeStatus.CLOSED,
            opened_at=closed_at - timedelta(minutes=30),
            closed_at=closed_at,
            pnl_usd=10.0 * r_multiple,
            r_multiple=r_multiple,
            evidence_type=EvidenceType.IBKR_PAPER_FILL.value,
            evidence_quality=EvidenceQuality.NORMAL.value,
            metadata_json={
                "execution_mode": "ibkr",
                "ibkr_mode": "paper",
                "evidence_type": EvidenceType.IBKR_PAPER_FILL.value,
                "evidence_quality": EvidenceQuality.NORMAL.value,
                "fill_provenance": FillProvenance.BROKER_EXECUTION.value,
                "broker_execution_hash": f"hash-{index}",
                "broker_execution_time": closed_at.isoformat(),
                "commission": 0.0,
            },
        ),
    )
    db.commit()

BASE_TIME = datetime(2026, 1, 5, 16, 0, tzinfo=timezone.utc)

def test_same_symbol_same_day_cluster_counts_as_one_effective_sample() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(4):
        add_paper_fill(db, pattern, index, symbol="AAPL", closed_at=BASE_TIME)
    trades = db.query(Trade).all()

    summary = effective_sample_summary(trades)

    assert summary["method"] == EFFECTIVE_SAMPLE_METHOD
    assert summary["n_trades"] == 4
    assert summary["n_eff"] == pytest.approx(1.0)
    # Kish ESS only reflects weight inequality (equal weights -> n); the
    # binding gate number is n_eff.
    assert summary["kish_n_eff"] == pytest.approx(4.0)
    assert summary["cluster_sizes"] == {"AAPL|2026-01-05": 4}
    assert all(entry["weight"] == pytest.approx(0.25) for entry in summary["weights"])

def test_mixed_clusters_report_expected_n_eff_and_kish() -> None:
    db = session_factory()

```

```

pattern = add_pattern(db)
add_paper_fill(db, pattern, 0, symbol="MSFT", closed_at=BASE_TIME)
for index in range(1, 4):
    add_paper_fill(db, pattern, index, symbol="AAPL", closed_at=BASE_TIME)
trades = db.query(Trade).all()

summary = effective_sample_summary(trades)

# Weights are [1, 1/3, 1/3, 1/3]: n_eff = 2 clusters, Kish = 4/(4/3) = 3.
assert summary["n_eff"] == pytest.approx(2.0)
assert summary["kish_n_eff"] == pytest.approx(3.0)
assert summary["cluster_count"] == 2

def test_independent_fills_keep_full_effective_sample() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(5):
        add_paper_fill(
            db,
            pattern,
            index,
            symbol=f"SYM{index}",
            closed_at=BASE_TIME + timedelta(days=index),
        )
    trades = db.query(Trade).all()

    summary = effective_sample_summary(trades)

    assert summary["n_eff"] == pytest.approx(5.0)
    assert summary["kish_n_eff"] == pytest.approx(5.0)

def test_gate_blocks_concentrated_fills_even_above_raw_count_minimum() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    # 25 raw fills, but all on the same symbol and day: one effective sample.
    for index in range(25):
        add_paper_fill(db, pattern, index, symbol="AAPL", closed_at=BASE_TIME)

    gate = DirectorReviewGate(min_closed_lab_trades=10, min_effective_lab_trades=25)
    result = gate.refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert result["marked_for_director_review"] == 0
    assert lab_execution["closed_lab_trades"] == 25
    assert lab_execution["effective_lab_trades"] == pytest.approx(1.0)
    assert "effective_lab_trades_below_25" in lab_execution["promotion_blockers"]
    assert lab_execution["effective_sample"]["method"] == EFFECTIVE_SAMPLE_METHOD
    assert lab_execution["effective_sample"]["n_eff"] == pytest.approx(1.0)

def test_gate_persists_per_trade_weights_in_trade_metadata() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    add_paper_fill(db, pattern, 0, symbol="MSFT", closed_at=BASE_TIME)
    for index in range(1, 3):
        add_paper_fill(db, pattern, index, symbol="AAPL", closed_at=BASE_TIME)

    DirectorReviewGate().refresh(db)

    trades = db.query(Trade).all()
    for trade in trades:
        record = trade.metadata_json["effective_sample"]
        assert record["method"] == EFFECTIVE_SAMPLE_METHOD
        assert record["computed_at"]
    weights = {
        trade.symbol: trade.metadata_json["effective_sample"]["weight"] for trade in trades
    }
    assert weights["MSFT"] == pytest.approx(1.0)
    assert weights["AAPL"] == pytest.approx(0.5)

def test_gate_accepts_diversified_effective_sample() -> None:
    db = session_factory()
    pattern = add_pattern(db)
    for index in range(25):
        add_paper_fill(
            db,
            pattern,
            index,
            symbol=f"SYM{index}",
            closed_at=BASE_TIME + timedelta(days=index),
        )

    gate = DirectorReviewGate(min_closed_lab_trades=10, min_effective_lab_trades=25)
    gate.refresh(db)
    db.refresh(pattern)
    lab_execution = pattern.metrics_json["lab_execution"]

    assert lab_execution["effective_lab_trades"] == pytest.approx(25.0)
    assert "effective_lab_trades_below_25" not in lab_execution["promotion_blockers"]

```

Full Source: `backend/tradeo/tests/test\_remediation\_docs\_traceability.py`

`backend/tradeo/tests/test\_remediation\_docs\_traceability.py` ? 129 lines, 4885 bytes, sha256 prefix `04562a11a122eff2`

Public surface summary before full source:

```
? function `test_remediation_docs_directory_present_or_skipped` line 50`
? function `test_files_changed_references_resolve_to_real_files` line 56`
? function `test_compliance_matrix_covers_all_agents` line 86`
? function `test_every_agent_report_is_reflected_in_matrix` line 96`
? function `test_audit_export_exposes_independent_sample_fields` line 122`
```

Risk hints: no obvious heuristic risk; still review logic/tests.

```
"""Traceability checks for the 12-phase remediation documentation.

Every remediation report under ``docs/remediation`` lists the files it
changed. These tests assert that those references still resolve to real
files in the repository, so the audit package cannot silently drift away
from the code it claims to describe.

The docs tree is not copied into the Docker image (only ``backend/tradeo``
and ``research`` are), so the tests skip when ``docs/remediation`` is not
present next to the backend tree.
"""

from __future__ import annotations

import re
from pathlib import Path

import pytest

_REPO_ROOT = Path(__file__).resolve().parents[3]
_REMEDIATION_DIR = _REPO_ROOT / "docs" / "remediation"

_FILES_CHANGED_HEADING = re.compile(r"^#\s+Files Changed\s*$", re.MULTILINE)
_BULLET_PATH = re.compile(r"^\s+`([^\s]+)`\s*$")

def _files_changed_entries(doc: Path) -> list[str]:
    text = doc.read_text(encoding="utf-8")
    match = _FILES_CHANGED_HEADING.search(text)
    if match is None:
        return []
    section = text[match.end():]
    next_heading = re.search(r"^#\s+", section, re.MULTILINE)
    if next_heading is not None:
        section = section[: next_heading.start()]
    entries: list[str] = []
    for line in section.splitlines():
        bullet = _BULLET_PATH.match(line.strip())
        if bullet is not None:
            entries.append(bullet.group(1))
    return entries

def _remediation_docs() -> list[Path]:
    if not _REMEDIATION_DIR.is_dir():
        return []
    return sorted(_REMEDIATION_DIR.glob("*.md"))

def test_remediation_docs_directory_present_or_skipped() -> None:
    if not _REMEDIATION_DIR.is_dir():
        pytest.skip("docs/remediation not available in this environment")
    assert _remediation_docs(), "docs/remediation exists but holds no reports"

def test_files_changed_references_resolve_to_real_files() -> None:
    docs = _remediation_docs()
    if not docs:
        pytest.skip("docs/remediation not available in this environment")
    checked = 0
    missing: list[str] = []
    for doc in docs:
        for entry in _files_changed_entries(doc):
            checked += 1
            if not (_REPO_ROOT / entry).is_file():
                missing.append(f"{doc.name}: {entry}")
    assert checked > 0, "no Files Changed entries found in any remediation doc"
    assert not missing, f"remediation docs reference missing files: {missing}"

_MATRIX_PATH = (
    _REMEDIATION_DIR / "tradeo_12_phase_compliance_matrix_2026_06_10.md"
)
_AGENT_DOC_NAME = re.compile(r"^agent_([a-z0-9]+)_")

def _matrix_agents_with_rows(text: str) -> set[str]:
    rows = [line for line in text.splitlines() if line.startswith("|")]
```

```

    return {
        cells[2]
        for line in rows
        if len(cells := [cell.strip() for cell in line.split("|")]) > 3
    }

def test_compliance_matrix_covers_all_agents() -> None:
    if not _MATRIX_PATH.is_file():
        pytest.skip("docs/remediation not available in this environment")
    agents_with_rows = _matrix_agents_with_rows(
        _MATRIX_PATH.read_text(encoding="utf-8")
    )
    for agent in ("A", "B", "C", "D", "E", "E2", "G", "H"):
        assert agent in agents_with_rows, f"matrix lost agent {agent} rows"

def test_every_agent_report_is_reflected_in_matrix() -> None:
    """Each agent_<id>_*.md report must leave a trace in the matrix.

    Either a table row (Agent column) or an explicit "Agent <ID>" status note
    counts; what failed before is an agent merging to main with no matrix
    record at all (as Agent E2 initially did).
    """
    if not _MATRIX_PATH.is_file():
        pytest.skip("docs/remediation not available in this environment")
    text = _MATRIX_PATH.read_text(encoding="utf-8")
    agents_with_rows = _matrix_agents_with_rows(text)
    unrecorded: list[str] = []
    for doc in _remediation_docs():
        match = _AGENT_DOC_NAME.match(doc.name)
        if match is None:
            continue
        agent_id = match.group(1).upper()
        in_rows = agent_id in agents_with_rows
        in_notes = re.search(rf"\bAgent {agent_id}\b", text) is not None
        if not (in_rows or in_notes):
            unrecorded.append(f"{doc.name} (agent {agent_id})")
    assert not unrecorded, (
        f"remediation reports with no compliance-matrix record: {unrecorded}"
    )

def test_audit_export_exposes_independent_sample_fields() -> None:
    """The audit export contract keeps the honest-count fields added by D."""
    export_script = _REPO_ROOT / "research" / "audit_bridge" / "export_audit_package.py"
    if not export_script.is_file():
        pytest.skip("research/audit_bridge not available in this environment")
    text = export_script.read_text(encoding="utf-8")
    for field in ("is_independent_sample", "independent_sample_count", "event_count"):
        assert field in text, f"audit export lost honest-count field {field!r}"

```

Full Source: ``backend/tradeo/tests/test_data_quality.py``

``backend/tradeo/tests/test_data_quality.py`` ? 124 lines, 4365 bytes, sha256 prefix ``59573ff7b23b7e99``

Public surface summary before full source:

```

? function `test_clean_fixture_is_research_grade` line 17`
? function `test_empty_frame_is_rejected` line 25`
? function `test_insufficient_bars_flagged` line 31`
? function `test_zero_volume_run_flagged` line 37`
? function `test_stale_close_run_flagged` line 45`
? function `test_calendar_gap_flagged_for_daily_bars` line 53`
? function `test_calendar_gap_ignored_for_intraday` line 62`
? function `test_suspect_split_jump_flagged` line 69`
? function `test_downward_split_jump_flagged_symmetrically` line 77`
? function `test_report_to_dict_is_json_friendly` line 84`
? function `test_scanner_skips_low_quality_symbols` line 93`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import numpy as np
import pandas as pd

from tradeo.services.data_quality import (
    ISSUE_CALENDAR_GAP,
    ISSUE_INSUFFICIENT_BARS,
    ISSUE_STALE_CLOSSES,
    ISSUE_SUSPECT_BAR_RETURN,
    ISSUE_ZERO_VOLUME,
    assess_ohlcw_quality,
)
from tradeo.tests.fixtures import fixture_ohlcw

def test_clean_fixture_is_research_grade() -> None:

```

```

df = fixture_ohlc("AAPL", bars=300)
report = assess_ohlc_quality(df, "AAPL")
assert report.research_grade
assert report.issues == []
assert report.bars == 300

def test_empty_frame_is_rejected() -> None:
    report = assess_ohlc_quality(pd.DataFrame(columns=["open", "high", "low", "close", "volume"]))
    assert not report.research_grade
    assert ISSUE_INSUFFICIENT_BARS in report.issues

def test_insufficient_bars_flagged() -> None:
    df = fixture_ohlc("AAPL", bars=30)
    report = assess_ohlc_quality(df, "AAPL", min_bars=60)
    assert ISSUE_INSUFFICIENT_BARS in report.issues

def test_zero_volume_run_flagged() -> None:
    df = fixture_ohlc("HALT", bars=200)
    df.loc[df.index[-80:], "volume"] = 0.0
    report = assess_ohlc_quality(df, "HALT")
    assert ISSUE_ZERO_VOLUME in report.issues
    assert report.zero_volume_pct >= 0.15

def test_stale_close_run_flagged() -> None:
    df = fixture_ohlc("FROZEN", bars=200)
    df.loc[df.index[-20:], "close"] = 12.34
    report = assess_ohlc_quality(df, "FROZEN")
    assert ISSUE_STALE_CLOSES in report.issues
    assert report.longest_stale_close_run >= 20

def test_calendar_gap_flagged_for_daily_bars() -> None:
    df = fixture_ohlc("GAPPY", bars=200)
    # Remove 15 business days from the middle of the series.
    df = pd.concat([df.iloc[:100], df.iloc[115:]])
    report = assess_ohlc_quality(df, "GAPPY", interval="1d")
    assert ISSUE_CALENDAR_GAP in report.issues
    assert report.max_single_gap_business_days >= 15

def test_calendar_gap_ignored_for_intraday() -> None:
    df = fixture_ohlc("GAPPY", bars=200)
    df = pd.concat([df.iloc[:100], df.iloc[115:]])
    report = assess_ohlc_quality(df, "GAPPY", interval="5m")
    assert ISSUE_CALENDAR_GAP not in report.issues

def test_suspect_split_jump_flagged() -> None:
    df = fixture_ohlc("SPLIT", bars=200)
    df.loc[df.index[-50:], ["open", "high", "low", "close"]] *= 10.0
    report = assess_ohlc_quality(df, "SPLIT")
    assert ISSUE_SUSPECT_BAR_RETURN in report.issues
    assert report.max_bar_return_ratio > 4.0

def test_downward_split_jump_flagged_symmetrically() -> None:
    df = fixture_ohlc("RSPLIT", bars=200)
    df.loc[df.index[-50:], ["open", "high", "low", "close"]] /= 10.0
    report = assess_ohlc_quality(df, "RSPLIT")
    assert ISSUE_SUSPECT_BAR_RETURN in report.issues

def test_report_to_dict_is_json_friendly() -> None:
    df = fixture_ohlc("AAPL", bars=120)
    payload = assess_ohlc_quality(df, "AAPL").to_dict()
    assert payload["symbol"] == "AAPL"
    assert isinstance(payload["issues"], list)
    assert payload["research_grade"] is True
    assert all(np.isfinite(payload[k]) for k in ("zero_volume_pct", "max_bar_return_ratio"))

def test_scanner_skips_low_quality_symbols(monkeypatch) -> None:
    from sqlalchemy import create_engine, select
    from sqlalchemy.orm import sessionmaker

    from tradeo.db.models import AuditLog
    from tradeo.db.session import Base
    from tradeo.schemas import ScanRequest
    from tradeo.services.scanner import MarketScanner

    engine = create_engine("sqlite:///memory:", future=True)
    Base.metadata.create_all(bind=engine)
    session = sessionmaker(bind=engine, future=True)()

    bad = fixture_ohlc("BAD", bars=200)
    bad.loc[bad.index[-80:], "volume"] = 0.0

    class _Provider:
        def fetch_ohlc(self, symbol: str, period: str, interval: str) -> pd.DataFrame:
            return bad

    scanner = MarketScanner(provider=_Provider())

```



```

response = scanner.run(ScanRequest(force_symbols=["BAD"]), session, store=True)

assert response.data_quality_skips == 1
assert response.candidates == 0
session.commit()
audit_rows = session.execute(
    select(AuditLog).where(AuditLog.action == "market_data_quality_reject")
).scalars().all()
assert len(audit_rows) == 1
assert audit_rows[0].entity_id == "BAD"
assert ISSUE_ZERO_VOLUME in audit_rows[0].details_json["issues"]

```

Full Source: `backend/tradeo/tests/test\_agent\_c\_remediation.py`

`backend/tradeo/tests/test\_agent\_c\_remediation.py` ? 70 lines, 2286 bytes, sha256 prefix `dd2284b9b595cf39`

Public surface summary before full source:

```

? function `test_backtester_uses_canonical_both_touch_stop_first` line 27`
? function `test_backtester_gap_past_target_entry_is_skipped` line 39`
? function `test_self_improvement_blocks_when_pbo_unavailable` line 48`

```

Risk hints: no obvious heuristic risk; still review logic/tests.

```

from __future__ import annotations

import pandas as pd

from tradeo.core.config import Settings
from tradeo.schemas import PatternCandidate
from tradeo.services.backtester import Backtester
from tradeo.services.self_improvement import SelfImprovementEngine

def _candidate() -> PatternCandidate:
    return PatternCandidate(
        symbol="TST",
        entry=100.0,
        stop=95.0,
        target=110.0,
        reward_risk=2.0,
        confidence=0.9,
        rule_score=0.9,
        ml_score=0.9,
        vision_score=0.9,
        composite_score=0.9,
        features={"avg_dollar_volume": 50_000_000.0},
    )

def test_backtester_uses_canonical_both_touch_stop_first() -> None:
    future = pd.DataFrame(
        {"open": [100.0], "high": [112.0], "low": [94.0], "close": [108.0]},
        index=pd.date_range("2024-01-02", periods=1, freq="B"),
    )
    trade = Backtester(provider=object())._simulate_exit("TST", future, _candidate(), entry=100.0)
    assert trade is not None
    assert trade.outcome == "stop_and_target_conservative"
    assert trade.exit == 95.0
    assert trade.r_multiple < -1.0

def test_backtester_gap_past_target_entry_is_skipped() -> None:
    future = pd.DataFrame(
        {"open": [111.0], "high": [112.0], "low": [110.0], "close": [111.0]},
        index=pd.date_range("2024-01-02", periods=1, freq="B"),
    )
    trade = Backtester(provider=object())._simulate_exit("TST", future, _candidate(), entry=111.0)
    assert trade is None

def test_self_improvement_blocks_when_pbo_unavailable(tmp_path) -> None:
    engine = SelfImprovementEngine(
        Settings(
            reports_dir=str(tmp_path),
            self_improvement_min_pbo_blocks=16,
        )
    )
    records = [
        {
            "config": {"min_depth": 0.10},
            "metrics": {
                "total_trades": 40,
                "profit_factor": 2.0,
                "expectancy_r": 0.3,
                "max_drawdown_pct": 5.0,
                "trades": [{"exit_date": "2024-01-01", "r_multiple": 1.0}],
            },
        }
    ]
    guards, summary = engine._anti_overfit_guards(records)
    assert summary["blocked"] is True

```

```
assert guards[0]["pbo_passed"] is False
assert engine._passes_lab_gate(records[0]["metrics"], guards[0]) is False
```

Key Docs And Remediation Full Text Appendix

Key project documentation and recent remediation notes, included verbatim with high-confidence secret redaction. Historical reports may be stale by design; the remediation ledger above explains current interpretation.

Full Document: `README.md`

`README.md` ? 284 lines, 10077 bytes, sha256 prefix `4c8347e24c7d9f5f`

Heading summary: Tradeo; Estado operativo por defecto; Arquitectura; Instalación rápida en Ubuntu; Comandos útiles; Flujo de decisión; Flujo exacto hasta operar en live; 1. Research descubre patrones; 2. Lab valida entradas reales/paper; 3. Gate de 10 casos Lab cerrados; 4. Director decide Producción; 5. Fox Hunter opera solo Producción

Tradeo

Tradeo es una aplicación privada para investigar, validar y operar en modo controlado patrones técnicos en acciones USA mid cap / small cap.

La v0 está diseñada con una prioridad clara: ****paper trading y validación estadística primero; live trading bloqueado por defecto****. Incluye backend FastAPI, worker programado, PostgreSQL, Redis, frontend Next.js moderno, detector de patrones tipo cup/base, gestor de riesgo, backtesting, automejora en laboratorio, reportes y un punto de integración para supervisor vía API.

Estado operativo por defecto

- Capital inicial: 3.000 USD.
- Riesgo máximo por operación: 1% = 30 USD.
- Beneficio/riesgo mínimo: 1:4.
- Modo: `paper`.
- Live trading: desactivado.
- Opciones y margen: desactivados por defecto.
- Cortos: permitidos en la política interna, pero solo se ejecutarían en real si IBKR y los gates lo permiten.
- Dashboard: <http://localhost:3000>
- API: <http://localhost:8000/api>

Arquitectura

Tradeo

??? frontend/	Next.js + Recharts
??? backend/	FastAPI + SQLAlchemy
? ??? tradeo/	
? ??? agents/	Agentes especializados
? ??? modules/	Research, Laboratorio, FoxHunter y shared
? ??? services/	Detector, riesgo, backtest, broker, reportes
? ??? routers/	API privada
? ??? tasks/	Worker programado
? ??? db/	Modelos y bootstrap de PostgreSQL
??? data/	Universo inicial mid/small cap editable
??? config/	Parámetros de estrategia
??? docs/	Diseño, riesgo y operación
??? ops/	Scripts e instrucciones OpenClaw
??? docker-compose.yml	

Instalación rápida en Ubuntu

```
cd tradeo
make setup
nano .env
make up
```

Abre:

<http://localhost:3000>

La primera vez, cambia en `.env`:

```
TRADEO_ADMIN_PASSWORD=una-contrasena-larga
TRADEO_SECRET_KEY=un-secreto-largo
```

Luego reinicia:

docker compose up -d --build

Comandos útiles

make logs	# ver logs
make scan	# ejecutar escaneo manual
make report	# generar paquete de revisión
make self-improve	# lanzar ciclo de automejora en laboratorio
make test	# ejecutar tests
make down	# parar

Flujo de decisión

1. `DataCollectorAgent` obtiene OHLCV.
2. `PatternHunterAgent` busca cup/base breakout en acciones mid/small cap.
3. `TechnicalContextAgent` evalúa contexto técnico simple: tendencia, SMA, volatilidad.
4. `FeatureScorer` aporta scoring tipo ML auditable.
5. `VisionGeometryScorer` puntúa la geometría visual del patrón.
6. `RiskAgent` aplica límites duros: 30 USD por operación, liquidez, volatilidad, posiciones máximas, R:R mínimo 1:4.
7. `SupervisorAgent` decide si pasa a watchlist, paper, revisión humana o rechazo.
8. `AuditAgent` genera paquetes de revisión para supervisión externa.
9. `ImprovementAgent` muta parámetros en laboratorio y solo propone candidatos si pasan gates.

Flujo exacto hasta operar en live

Tradeo usa una cadena de promoción cerrada. Ningún patrón descubierto por Research puede llegar a live directamente.

Research -> Lab -> Director review -> Producción -> Fox Hunter -> Live

1. Research descubre patrones

`PatternDiscoveryLabAgent` trabaja con barras diarias (`1d`) de IBKR, ventanas OHLCV y clustering local. Para que un patrón salga de Research debe superar gates estadísticos de descubrimiento:

- muestras históricas suficientes;
- diversidad de símbolos y años;
- expectancy y profit factor positivos;
- validación out-of-sample;
- estabilidad mínima;
- drawdown dentro de límites;
- R:R compatible con la política del sistema.

Aunque las métricas de Research sean buenas, el patrón solo puede quedar en estados de laboratorio como:

- `lab`
- `lab\_watchlist`
- `lab\_candidate`

Research no puede marcar nada como `production`.

2. Lab valida entradas reales/paper

El módulo Laboratorio (`tradeo.modules.laboratory`) toma patrones validados por Research y busca señales de entrada actuales. Lab no debe crear señales fuera de la sesión regular USA; si el mercado está cerrado queda en `market\_closed`.

Cuando Lab encuentra una señal:

- crea una señal paper/auditable;
- si la ejecución paper está activada, intenta enviar bracket a IBKR Paper;
- separa los motivos exactos de no ejecución: no enviado, fallo IBKR, bracket no aceptado, orden enviada esperando fill, señal expirada, etc.;
- no promueve el patrón por sí mismo.

Lab es el entorno donde se comprueba si el patrón funciona con entradas y salidas operables, no solo con simulación Research.

3. Gate de 10 casos Lab cerrados

Cuando un patrón acumula al menos 10 operaciones Lab cerradas (`entrada + salida`), `DirectorReviewGate` calcula su rendimiento real/paper:

- `closed\_lab\_trades`
- `lab\_expectancy\_r`
- `lab\_profit\_factor`
- `lab\_win\_rate`
- `research\_expectancy\_r`
- `research\_profit\_factor`
- delta de expectancy Lab vs Research;
- delta de profit factor Lab vs Research.

Si llega a ese mínimo, el patrón se marca como:

director_review

Esto significa: "listo para que Director lo audite como posible candidato a Producción". No significa que pueda operar live.

4. Director decide Producción

Solo Director puede pasar un patrón de `director\_review` a:

production

Director debe revisar el paquete auditado, comprobar que las operaciones Lab son suficientes y comparar el rendimiento observado contra lo que prometía Research. Si no hay trades/fills suficientes, costes realistas, OOS limpio o evidencia operable, el patrón no debe promoverse.

5. Fox Hunter opera solo Producción

Fox Hunter (`tradeo.modules.fox\_hunter`) es el clon operativo de Lab para patrones ya aprobados. Usa el mismo motor compartido de entrada, pero filtra exclusivamente patrones:

production

Fox no mira `lab\_candidate`, `director\_review`, `paper\_candidate` ni otros estados intermedios para operar. Si encuentra una entrada en un patrón `production`, puede crear la señal live/paper según configuración, pero live sigue bloqueado por las gates globales.

6. Live sigue bloqueado por seguridad

Aunque un patrón esté en `production`, live requiere además:

- `TRADEO\_TRADING\_MODE=live`;
- `TRADEO\_LIVE\_TRADING\_ENABLED=true`;
- frase de confirmación exacta;
- `TRADEO\_IBKR\_READONLY=false`;
- kill switch apagado;
- puerto IBKR live correcto;
- límites de riesgo aprobados;
- aprobación humana por señal cuando aplique.

Si cualquiera de esas condiciones falla, no hay operación live.

Gates de automejora

Una variante de estrategia solo puede pasar a candidata de laboratorio si cumple:

- mínimo 40 operaciones históricas;
- profit factor ≥ 1.8 ;
- expectancy $\geq 0.25R$;
- drawdown máximo $\leq 12\%$;
- target mínimo 4R;
- revisión posterior en paper trading;
- aprobación humana/API antes de live.

Integración con Interactive Brokers

La v0 puede usar IBKR para datos reales mediante TWS o IB Gateway. No guarda credenciales de IBKR: TWS/Gateway debe estar abierto y con API socket habilitada. En TWS: `Global Configuration` -> `API` -> `Settings` -> activar `Enable ActiveX and Socket Clients`.

Puertos habituales:

- TWS paper: `7497`
- TWS live: `7496`
- IB Gateway paper: `4002`
- IB Gateway live: `4001`

Configuración inicial recomendada:

```
TRADEO_TRADING_MODE=paper
TRADEO_MARKET_DATA_PROVIDER=ibkr
TRADEO_ALLOW_SYNTHETIC_MARKET_DATA=false
TRADEO_LIVE_TRADING_ENABLED=false
TRADEO_LIVE_TRADING_CONFIRMATION_VALUE=
TRADEO_IBKR_HOST=host.docker.internal
TRADEO_IBKR_PORT=7497
TRADEO_IBKR_CLIENT_ID=17
TRADEO_IBKR_READONLY=true
```

Comprueba la conexión con:

```
curl http://localhost:8000/api/health/ibkr
```

El adaptador `IBKRBroker` para órdenes bracket de acciones existe, pero queda bloqueado por defecto. Para la fase inicial usa IBKR Paper Trading o el broker simulado interno. Live requiere `TRADEO\_TRADING\_MODE=live`, `TRADEO\_LIVE\_TRADING\_ENABLED=true`, frase de confirmación exacta, `TRADEO\_IBKR\_READONLY=false`, kill switch apagado y aprobación humana por señal.

Integración con supervisor API

Para activar revisión API:

```
TRADEO_OPENAI_SUPERVISOR_ENABLED=true
TRADEO_OPENAI_API_KEY=sk-...
TRADEO_OPENAI_SUPERVISOR_MODEL=gpt-5.5-pro
```

El supervisor API nunca sustituye los límites duros de riesgo. Solo añade revisión estructurada y notas de viabilidad.

Universo de activos

`data/universe\_us\_mid\_small.csv` es una lista inicial editable. No es una recomendación de inversión. Sustitúyela por un universo actualizado mediante IBKR Scanner, Polygon, Nasdaq Data Link u otro proveedor.

Seguridad mínima

- Despliegue pensado para `localhost` o VPN.
- Auth básica para API/dashboard.
- Live trading bloqueado por defecto.
- Variables sensibles en `.env`.
- Reportes auditables en `reports/`.
- Kill switch persistente vía `.env`.
- No instales skills OpenClaw no auditadas en la misma máquina de trading.

Director externo

Genera un paquete de revisión:

make report

El JSON/Markdown resultante queda en `reports/`. Pégalo en ChatGPT o envíalo a tu supervisor API para evaluación final.

Research Lab

Tradeo incluye una ampliación de descubrimiento de patrones no predefinidos. El `PatternDiscoveryLabAgent` extrae ventanas OHLCV, genera embeddings locales, agrupa formas similares con clustering y valida si esos grupos precedieron movimientos favorables con R:R mínimo 1:4.

Este laboratorio no opera dinero real. Todo candidato queda en estados de laboratorio o revisión (`lab`, `lab\_watchlist`, `lab\_candidate`, `director\_review`) hasta que Director lo promueva explícitamente a `production`.

Comandos:

```
make discover-patterns
make match-discovered-patterns
make current-matches
make research-runs
```

Documentación detallada: `docs/pattern\_discovery\_lab.md`.

Full Document: `CLAW\_IMPLEMENTATION\_PROMPT.md`

`CLAW\_IMPLEMENTATION\_PROMPT.md` ? 88 lines, 1944 bytes, sha256 prefix `3b4edfd0d0be408a`

Heading summary: Prompt maestro para OpenClaw; Reglas innegociables; Tarea de despliegue; Tareas periódicas recomendadas; Formato de reporte para el usuario/director; Prohibido

Prompt maestro para OpenClaw

Eres OpenClaw operando en una VM Ubuntu local/VPN. Tu objetivo es desplegar y mantener Tradeo con seguridad, sin activar live trading salvo instrucción explícita y documentada del usuario.

Reglas innegociables

1. No actives `TRADEO\_TRADING\_MODE=live`.
2. No pongas `TRADEO\_LIVE\_TRADING\_ENABLED=true`.
3. No pongas `TRADEO\_IBKR\_READONLY=false`.
4. No instales skills, paquetes o scripts de terceros no auditados.
5. No copies claves API ni credenciales en logs, commits o mensajes.
6. Antes de cualquier cambio, lee `README.md`, `docs/risk\_policy.md` y `.env.example`.
7. Toda modificación de estrategia debe quedar registrada en `reports/` o en un commit local con resumen.
8. Si el sistema falla, activa/respetas kill switch y prioriza no ejecutar órdenes.

Tarea de despliegue

Ejecuta:

```
cd /ruta/a/tradeo
make setup
```

Edita `.env` con:

```
TRADEO_ADMIN_PASSWORD=<password-fuerte>
```

```
TRADEO_SECRET_KEY=<secreto-largo>
TRADEO_TRADING_MODE=paper
TRADEO_LIVE_TRADING_ENABLED=false
TRADEO_IBKR_READONLY=true
```

Después:

```
make up
make logs
```

Comprueba:

```
curl http://localhost:8000/api/health
```

Abre dashboard:

```
http://localhost:3000
```

Tareas periódicas recomendadas

Cada día de mercado:

```
make scan
make report
```

Cada viernes tras cierre de mercado:

```
make self-improve
make report
```

Formato de reporte para el usuario/director

Cuando generes reporte, responde con:

- estado de contenedores;
- última hora de escaneo;
- número de señales;
- operaciones abiertas;
- resumen de riesgo;
- ruta del último JSON/Markdown en `reports/`;
- errores relevantes de logs, si los hay.

Prohibido

- Activar operativa real.
- Cambiar límites de riesgo para hacer pasar una señal.
- Modificar `.env` con credenciales reales en texto visible.
- Desactivar autenticación.
- Exponer la web públicamente sin VPN/HTTPS/auth fuerte.

Full Document: `CLAW\_RESEARCH\_LAB\_INTEGRATION\_PROMPT.md`

`CLAW\_RESEARCH\_LAB\_INTEGRATION\_PROMPT.md` ? 218 lines, 5466 bytes, sha256 prefix `d6170d0668861a10`

Heading summary: OpenClaw task · Integrar Tradeo Research Lab; Restricciones no negociables; Tareas; 1. Comprobar estructura; 2. Comprobar Docker; 3. Revisar `.env`; 4. Construir y arrancar; 5. Verificar salud; 6. Ejecutar tests; 7. Lanzar primera búsqueda controlada; 8. Ver resultados; 9. Modo autónomo todo el día

OpenClaw task · Integrar Tradeo Research Lab

Estás trabajando dentro del proyecto Tradeo en Ubuntu.

Ruta esperada:

```
cd ~/tradeo
```

Objetivo: integrar y arrancar la ampliación `PatternDiscoveryLabAgent`, que descubre patrones técnicos no predefinidos en acciones USA mid/small cap usando embeddings OHLCV, clustering y validación estadística.

Restricciones no negociables

1. No actives live trading.
2. No cambies `TRADEO\_LIVE\_TRADING\_ENABLED` a `true`.
3. No conectes órdenes reales a Interactive Brokers.
4. No permitas que patrones descubiertos operen dinero real.

5. No reduzcas `TRADEO\_MIN\_REWARD\_RISK=4`.
6. No reduzcas los límites de validación salvo aprobación humana explícita.
7. No borres base de datos ni volúmenes Docker sin pedir confirmación.
8. No instales skills/extensiones de OpenClaw ajenas al proyecto.
9. Si cambias código, deja resumen exacto de archivos modificados.

Tareas

1. Comprobar estructura

```
cd ~/tradeo
```

```
ls -la
```

```
ls -la backend/tradeo/research backend/tradeo/agents/pattern_discovery_lab_agent.py docs/pattern_discovery_lab.md
```

Si esos archivos no existen, detente y avisa: falta aplicar el paquete de ampliación.

2. Comprobar Docker

```
docker --version
```

```
docker compose version
```

Si Docker no existe, instalar:

```
sudo apt update
```

```
sudo apt install -y docker.io docker-compose-plugin make curl
```

```
sudo systemctl enable --now docker
```

```
sudo usermod -aG docker $USER
```

Usa `newgrp docker` si hace falta para aplicar permisos sin cerrar sesión. Si no funciona, usa `sudo docker compose` temporalmente.

3. Revisar `.env`

No sobrescribas `.env` si existe.

```
test -f .env || cp .env.example .env
```

Asegura que existen estas variables en `.env`:

```
TRADEO_DISCOVERY_ENABLED=true
TRADEO_DISCOVERY_SCHEDULER_ENABLED=true
TRADEO_DISCOVERY_SCAN_MINUTES=90
TRADEO_DISCOVERY_PERIOD=5y
TRADEO_DISCOVERY_INTERVAL=1d
TRADEO_DISCOVERY_LIMIT_DEFAULT=80
TRADEO_DISCOVERY_WINDOW_SIZES=20,50,100,200
TRADEO_DISCOVERY_FORWARD_BARS=5,10,20
TRADEO_DISCOVERY_STRIDE=3
TRADEO_DISCOVERY_MAX_TOTAL_WINDOWS=12000
TRADEO_DISCOVERY_MAX_WINDOWS_PER_SYMBOL=450
TRADEO_DISCOVERY_MIN_CLUSTER_SIZE=60
TRADEO_DISCOVERY_MAX_CLUSTERS_PER_WINDOW=12
TRADEO_DISCOVERY_MIN_SAMPLES=100
TRADEO_DISCOVERY_MIN_SYMBOLS=8
TRADEO_DISCOVERY_MIN_YEARS=2
TRADEO_DISCOVERY_MIN_PROFIT_FACTOR=1.8
TRADEO_DISCOVERY_MIN_EXPECTANCY_R=0.25
TRADEO_DISCOVERY_MIN_STABILITY_SCORE=0.45
TRADEO_DISCOVERY_STORE_REJECTED=true
TRADEO_DISCOVERY_MATCH_ENABLED=true
TRADEO_DISCOVERY_MATCH_SCAN_MINUTES=30
TRADEO_DISCOVERY_MATCH_SYMBOL_LIMIT=80
TRADEO_DISCOVERY_MATCH_MAX_PATTERNS=25
TRADEO_DISCOVERY_MATCH_SIMILARITY_THRESHOLD=0.45
TRADEO_DISCOVERY_MATCH_MAX_RESULTS=100
```

También confirma que live sigue bloqueado:

```
TRADEO_TRADING_MODE=paper
TRADEO_LIVE_TRADING_ENABLED=false
TRADEO_ALLOW_OPTIONS=false
TRADEO_ALLOW_MARGIN=false
```

4. Construir y arrancar

make up

Si falla por permisos:

sudo docker compose up -d --build

5. Verificar salud

docker compose ps
curl http://localhost:8000/api/health

Si has usado `sudo`, usa:

sudo docker compose ps

6. Ejecutar tests

make test

Si el test falla, recoge logs y explica el error antes de cambiar código.

7. Lanzar primera búsqueda controlada

Ejecuta una búsqueda pequeña para verificar integración:

make discover-patterns

0 equivalente:

```
source .env
curl -u "$TRADEO_ADMIN_USERNAME:$TRADEO_ADMIN_PASSWORD" \
  -X POST http://localhost:8000/api/research/run-discovery \
  -H 'Content-Type: application/json' \
  -d '{"limit":20,"period":"3y","interval":"1d","max_total_windows":3000,"max_windows_per_symbol":180,"store_rejected":true}'
```

8. Ver resultados

```
source .env
curl -u "$TRADEO_ADMIN_USERNAME:$TRADEO_ADMIN_PASSWORD" \
  http://localhost:8000/api/research/discovered-patterns?limit=20
curl -u "$TRADEO_ADMIN_USERNAME:$TRADEO_ADMIN_PASSWORD" http://localhost:8000/api/research/runs
ls -lah reports/research || true
```

Si existen patrones `lab`, prueba coincidencias actuales en modo laboratorio:

make match-discovered-patterns
make current-matches

Dashboard:

http://localhost:3000

Debe aparecer la sección `Research Lab · patrones descubiertos desde cero`.

9. Modo autónomo todo el día

El worker ejecuta el laboratorio cada `TRADEO\_DISCOVERY\_SCAN\_MINUTES` minutos si los contenedores están arriba.

Comprueba logs:

docker compose logs -f worker --tail=120

Busca mensajes como:

Criterio de aceptación

Devuélveme un resumen con:

- Docker instalado o ya disponible.
- Contenedores en marcha.
- Resultado de `curl /api/health`.
- Resultado resumido de `make test`.
- Resultado de `make discover-patterns`.
- Número de ventanas muestreadas.
- Número de clusters evaluados.
- Número de patrones `lab` y `rejected`.
- Si hay patrones `lab`, número de coincidencias actuales `lab\_watchlist`.
- Ruta de reporte generada en `reports/research`.
- Confirmación explícita de que live trading sigue bloqueado.

Optimización de tokens

No envíes a ningún LLM ni pegues en chats históricos OHLCV completos, logs enormes o JSON completo de miles de patrones. Para revisión externa usa solo:

- los 20 patrones top;
- métricas resumidas;
- razones de rechazo;
- ejemplos representativos comprimidos;
- rutas de reportes generados.

Full Document: `TRADEO\_INTRADAY\_SMALLCAP\_EXPANSION.md`

`TRADEO\_INTRADAY\_SMALLCAP\_EXPANSION.md` ? 498 lines, 14624 bytes, sha256 prefix `f5d951f48a299c06`

Heading summary: Tradeo Intraday Small-Cap Expansion; Resumen; Principios; Arquitectura Propuesta; Universo Intradia; Small-Cap First; Universo Dinamico; Datos E IBKR; Research Intradia; Carriles; Parametros Iniciales; Features Nuevas

Tradeo Intraday Small-Cap Expansion

Fecha: 2026-06-09

Estado: propuesta de ampliacion, no implementada.

Resumen

Tradeo puede ampliarse a operativa intradia aprovechando la arquitectura actual: Research descubre patrones, Laboratorio valida en paper/shadow, Director audita y Fox Hunter solo opera patrones de Produccion. La ampliacion no debe crear un bot separado ni usar LLMs mirando velas minuto a minuto. Debe ser un carril multi-timeframe determinista, con small caps activas como universo principal, paper/shadow-first y gates mas duros que en daily.

La tesis: para intradia, las small caps activas son mas interesantes que mid/large caps porque concentran volatilidad, gaps, volumen relativo explosivo e ineficiencias. Pero no se debe operar "small caps" de forma ciega. El universo debe ser small-cap first, dinamico y filtrado por liquidez, spread, volumen, catalyst y riesgo operativo.

Principios

1. No bajar seguridad por buscar mas actividad.
2. No usar datos sinteticos ni providers no IBKR.
3. No activar live trading automaticamente.
4. No promocionar patrones intradia sin Director.
5. No mezclar estadisticas daily con intradia.
6. No usar LLMs para tomar decisiones por vela.
7. Usar scanners deterministas y auditoria compacta.
8. Recolectar primero shadow observations, despues paper IBKR limitado.
9. Tratar intradia como edge de vida corta: confirmar, monitorizar y retirar rapido.

Arquitectura Propuesta

La ampliacion debe ser un "intraday lane" dentro del sistema existente:

- Research daily/1h: contexto y preseleccion de simbolos.
- Research 15m: descubrimiento de setups intradia operables.
- Research 5m: timing y entrada.
- 1m: confirmacion/gestion solo para candidatos armados, no discovery masivo.
- Laboratorio intradia: shadow observations y luego paper IBKR limitado.
- Director intradia: auditoria de edge, costes, decay, sesgos de hora y regimen.
- Fox Hunter: sin cambios de principio; solo Produccion y con live_armed.

El flujo recomendado:

1. Daily/1h decide que simbolos tienen contexto interesante.
2. 15m detecta setup operable.
3. 5m afina entrada.
4. 1m confirma/gestiona solo si el candidato ya esta armado.
5. Laboratorio crea observaciones shadow sin IBKR.
6. Tras evidencia suficiente, Laboratorio puede probar paper IBKR limitado.
7. Director decide si el patron pasa a revision de Produccion.

Universo Intradia

El universo intradia no debe ser fijo ni igual al universo daily.

Small-Cap First

Priorizar small caps activas:

- Acciones USA listadas, no OTC.
- Precio minimo preferible: > 3 USD o > 5 USD.
- Gap premarket o intradia relevante: 3-5%+.
- Volumen relativo fuerte.
- Dollar volume suficiente.
- Spread controlado.
- Rango intradia suficiente.
- Liquidez real en Level 1/prints si esta disponible.
- Catalyst detectado o comportamiento anormal verificable.

Excluir o penalizar:

- Penny stocks sin liquidez.
- Microcaps con spread enorme.
- Tickers OTC.
- Tickers con volumen aparente pero mala salida.
- Acciones recién halted o con halt risk excesivo.
- Acciones con float/estructura demasiado peligrosa si tenemos datos.
- Tickers con patrones de pump sin ejecucion limpia.

Universo Dinamico

Crear un selector por sesion:

- Pre-market builder: gap, volumen, RVOL, precio, dollar volume, spread.
- Market-open builder: primeros 15-30 minutos, rango, VWAP, liquidity.
- Midday refresh: descartar simbolos muertos, anadir nuevos movers.
- Power-hour refresh: solo si hay estrategia especifica para cierre.

Objetivo operativo:

- 1h/daily puede mirar 40-80 simbolos.
- 15m puede mirar 20-40 simbolos.
- 5m debe mirar 5-25 simbolos.
- 1m solo debe mirar candidatos armados, no universo completo.

Datos E IBKR

IBKR soporta barras intradia como 1h, 30m, 15m, 5m y 1m, pero no conviene refetch bruto de todo el universo por pacing, latencia y coste operativo.

Mejoras necesarias:

- Cache local de OHLCV intradia por `(symbol, interval, timestamp)`.
- Fetch incremental en vez de descargar ventanas completas cada ciclo.
- Rate limiter por interval/simbolo.
- Backfill fuera de mercado para 15m/5m.
- Durante mercado, fetch solo de simbolos candidatos.
- Estado de frescura por simbolo/timeframe.
- Auditoria de gaps de datos, barras duplicadas y barras incompletas.
- Nunca usar vela viva para etiquetar patrones o decidir entrada.

Research Intradia

La maquinaria actual de ventanas, embeddings, clustering, walk-forward, null baselines, cost stress, familias y ledgers puede reutilizarse. Lo que cambia son los parametros, features y controles.

Carriles

- `daily\_context`: patrones estructurales y filtro de contexto.
- `intraday\_1h\_context`: tendencia intradia amplia, gap continuation, reversal.
- `intraday\_15m\_research`: setup principal.
- `intraday\_5m\_research`: entrada y confirmacion.
- `intraday\_1m\_execution`: solo ejecucion/gestion, no discovery masivo.

Parametros Iniciales

Para 15m:

- Window sizes: 20, 40, 80, 160.
- Forward bars: 3, 6, 12, 24.
- Stride: mayor fuera de mercado, menor en investigacion focalizada.
- Min samples: mas alto que daily.
- Min symbols: mantener diversidad, pero aceptar cohortes pequenas solo en confirmation queue, no en Produccion.

Para 5m:

- Window sizes: 20, 40, 80.
- Forward bars: 3, 6, 12.
- Uso principal: timing/entrada y no tesis principal.

Para 1m:

- Solo candidatos armados.
- Horizon corto.

- Prohibir discovery masivo hasta tener cache y rate limits robustos.

Features Nuevas

Añadir features intradia:

- Distancia a VWAP.
- Reclaim/loss de VWAP.
- Opening range high/low.
- Posicion dentro del rango del dia.
- Distancia a high/low del dia.
- Gap vs cierre anterior.
- Gap fade o gap continuation.
- Volumen relativo contra la misma hora historica.
- Dollar volume intradia.
- Spread proxy.
- Range expansion/compression.
- ATR intradia.
- Fuerza relativa vs SPY/QQQ en la misma ventana.
- Regimen de mercado intradia.
- Session bucket: premarket, open, mid-day, power hour.
- Hour bucket.
- Halt proximity/risk si se puede medir.

Null Baseline Intradia

El baseline no debe ser generico. Comparar contra:

- Mismo simbolo.
- Misma hora o bucket de sesion.
- Mismo regimen de volatilidad.
- Similar RVOL.
- Similar gap bucket.
- Similar liquidez/spread bucket.

Esto evita aprobar patrones que solo capturan "a las 9:35 todo se mueve mas" o "este ticker tuvo una noticia unica".

Deteccion De Patrones

La deteccion intradia debe guardar el timeframe como parte esencial del patron. Un patron 15m no es el mismo patron que uno daily aunque el chart se parezca.

Cada patron intradia debe persistir:

- Timeframe.
- Window size.
- Session bucket preferido.
- Hour bucket preferido.
- Regime profile.
- Liquidity profile.
- Spread/slippage profile.
- Gap/RVOL profile.
- Cost stress results.
- Fresh OOS results.
- Decay status.
- Kill conditions.

Rechazar o mandar a confirmation queue si:

- Edge concentrado en una sola hora.
- Edge concentrado en un solo ticker.
- Edge desaparece con 2x costes.
- Edge depende de eventos no repetibles.
- Edge ocurre solo con datos train.
- Muestra insuficiente para intradia.

Senales De Entrada

La entrada cambia mas que el discovery. Daily puede tolerar decision lenta; intradia no. Cada senal debe ser auditable y expirar rapido.

Variantes Intradia

Añadir variantes especificas:

- `opening\_range\_breakout\_retest`
- `opening\_range\_breakdown\_retest`
- `vwap\_reclaim\_followthrough`
- `vwap\_loss\_followthrough`
- `failed\_breakdown\_reversal`
- `failed\_breakout\_reversal`
- `range\_compression\_break`
- `pullback\_to\_vwap`
- `pullback\_to\_ema`
- `high\_of\_day\_reclaim`
- `low\_of\_day\_reject`
- `momentum\_bar\_continuation`
- `liquidity\_sweep\_reclaim`

Reglas Duras

- Decidir solo despues de vela cerrada.
- Entry eligibile despues del cutoff de datos.
- Expirar si no entra en 1-3 velas.
- No duplicar exposicion por simbolo/patron.

- Cooldown tras stop.
- Max senales por simbolo/dia.
- No operar si spread supera limite.
- No operar si dollar volume cae por debajo del minimo.
- No operar si el patron esta fuera de su session bucket validado.

Filtros De Tiempo

Por defecto:

- Evitar primeros 5-15 minutos salvo estrategia dedicada.
- Evitar ultimos 10-15 minutos salvo estrategia dedicada.
- Separar open, mid-day y power hour.
- No mezclar resultados entre buckets sin auditoria.

Laboratorio Intradia

Laboratorio debe validar mas rapido, pero con mas escepticismo.

Shadow First

Antes de enviar ordenes IBKR paper:

- Crear shadow observations para candidatos top-ranked.
- Cerrar shadow por stop, target o time stop usando barras reales posteriores.
- Guardar MFE/MAE, time-to-target, time-to-stop, slippage proxy, spread proxy.
- Separar near-miss por causa: volumen, trigger, regimen, extension, liquidez.

Paper IBKR Limitado

Solo cuando shadow tenga evidencia:

- Activar paper IBKR por patron/variante concreto.
- Tamanos pequenos.
- Max trades/dia.
- Max open intraday positions.
- Sin live.
- Reconciliacion estricta de fills y open orders.

Metrics

No basta con win rate. Guardar:

- Expectancy R.
- Profit factor.
- MFE/MAE.
- Time in trade.
- Hit rate por target.
- Stop speed.
- Slippage estimado vs real si hay fill.
- Resultado por variante.
- Resultado por session bucket.
- Resultado por simbolo.
- Resultado por regimen.
- Resultado por liquidity bucket.

Para intradia, el minimo para Director deberia ser mucho mayor que daily:

- Shadow observations: 50-200 por patron/variante.
- Paper IBKR fills: suficientes para verificar ejecucion, no solo teoria.
- Diversidad de dias y simbolos.

Risk Management Intradia

Usar politica especifica:

- Cerrar todo intradia.
- Max perdidas/dia mas bajo que en swing/daily.
- Max trades por dia.
- Max trades por simbolo.
- Max open positions intradia.
- Position sizing menor.
- Stop rapido.
- Time stop obligatorio.
- No margin por defecto.
- No opciones.
- No market orders salvo aprobacion explicita.
- No operar si IBKR health no esta OK.
- Kill switch si hay desincronizacion entre DB e IBKR.

Small caps requieren filtros mas duros:

- Spread maximo.
- Dollar volume minimo.
- Price minimo.
- Borrow/short availability si es short.
- Halt risk.
- Liquidez suficiente para salir.

Director Intradia

El Director debe auditar intradia como una categoria separada.

Gates

Bloquear promocion si:

- No hay shadow/paper cerrado suficiente.
- Edge concentrado en un solo día.
- Edge concentrado en una sola hora.
- Edge concentrado en un solo ticker.
- Cost stress 2x/3x destruye expectancy.
- Paper real diverge demasiado de shadow.
- Slippage real supera proxy.
- Pattern decay activo.
- Hay gaps de datos o barras incompletas.
- Hay duplicate events o leakage.

Decay

Intradia debe decaer rapido:

- Cada patron tiene freshness window.
- Si N sesiones recientes fallan, pasa a decaying/retired.
- Si un patron depende de un regimen especifico, queda disabled fuera de regimen.
- Si la liquidez cambia, se recalcula sizing o se bloquea.

Agentes Y Jobs

No usar agentes LLM leyendo velas minuto a minuto. Usar jobs deterministas:

- `IntradayUniverseBuilder`: crea universo small-cap activo.
- `IntradayDataSync`: sincroniza OHLCV intradia incremental.
- `IntradayResearchScheduler`: lanza discovery/backfill por timeframe.
- `IntradayCandidateBuilder`: arma candidatos por contexto.
- `IntradayEntryScanner`: procesa velas cerradas y genera matches.
- `IntradayObservationCloser`: cierra shadow observations.
- `IntradayDirector`: resume, audita y propone kill/confirm/retire.

LLM/Director solo recibe:

- Resumen de patrones.
- Metricas agregadas.
- Casos top.
- Fallos/rechazos.
- Drift/decay.
- Audit ledger compacto.

Dashboard

Añadir vista intradia sin mezclar con daily:

- Universo activo.
- Timeframe lane.
- Candidatos armados.
- Entradas rechazadas y motivo.
- Shadow observations abiertas/cerradas.
- Paper IBKR fills.
- Funnel por variante.
- Funnel por session bucket.
- Cost/slippage health.
- IBKR pacing/data freshness.
- Director blockers.
- Patterns decaying/retired.

Fases De Implementacion

Fase 0: Diseno Y Config

- Crear config `TRADEO\_INTRADAY\_\*`.
- No cambiar live.
- No activar ordenes.
- Documentar invariantes.

Fase 1: Universo Small-Cap Intradia

- Builder de universo dinamico.
- Filtros: precio, gap, RVOL, dollar volume, spread.
- Baseline mid/large opcional para comparar.

Fase 2: Cache Intradia IBKR

- OHLCV incremental por timeframe.
- Rate limiter.
- Freshness checks.
- Audit de barras incompletas.

Fase 3: Research 15m/5m

- Carriles separados.
- Features intradia.
- Baseline por hora/regimen/liquidez.
- Cost stress agresivo.
- Confirmation queue.

Fase 4: Lab Shadow Intradia

- Entry variants intradia.
- Shadow observations.
- Cierre por stop/target/time stop.
- Dashboard y diagnostics.

Fase 5: Director Intradia

- Gates intradia.
- Decay/freshness.
- Reportes por session bucket.
- Kill conditions.

Fase 6: Paper IBKR Limitado

- Activar solo patrones/variantes con evidencia.
- Tamanos pequenos.
- Reconciliacion estricta.
- No live.

Fase 7: 1m Como Confirmacion

- Solo candidatos armados.
- Sin discovery masivo.
- Validar que mejora entrada y no solo aumenta ruido.

Riesgos

- Overfitting por hora del dia.
- Overfitting a noticias unicas.
- Pacing/throttling IBKR.
- Barras incompletas o duplicadas.
- Slippage mayor que lo simulado.
- Spreads que destruyen el edge.
- Halts.
- False liquidity.
- Edge que desaparece rapido.
- Demasiadas senales de baja calidad.

Decision Recomendada

Empezar por small caps activas en 15m/5m, con shadow observations y sin ordenes. Mantener mid/large caps solo como baseline de control. Si la evidencia muestra que small caps tienen edge superior despues de costes, concentrar el carril intradia en small caps. No empezar con 1m universal.

La primera version util deberia demostrar:

- Universo intradia small-cap limpio.
- Datos intradia reales IBKR cacheados.
- Research 15m/5m separado de daily.
- Entradas shadow auditables.
- Director intradia bloqueando promociones sin evidencia.

Solo despues de eso tiene sentido hablar de paper IBKR intradia, y mucho despues de Produccion.

Referencias Operativas

- IBKR historical data pacing/limitations:
https://interactivebrokers.github.io/tws-api/historical_limitations.html
- IBKR historical bars:
https://interactivebrokers.github.io/tws-api/historical_bars.html
- SEC microcap stock risks:
<https://www.sec.gov/about/reports-publications/investorpubsmicrocapstock>
- FINRA low-priced stocks:
<https://www.finra.org/investors/insights/low-priced-stocks-big-problems>

Full Document: `docs/architecture.md`

`docs/architecture.md` ? 119 lines, 3647 bytes, sha256 prefix `ff5a336a47ca43b8`

Heading summary: Arquitectura de Tradeo; Objetivo; Componentes; Frontend; Backend; Worker; Base de datos; Motor de patrones; Scoring combinado; Broker; Decisiones deliberadas; Por qué Docker

Arquitectura de Tradeo

Objetivo

Tradeo busca patrones técnicos repetibles en acciones USA mid cap / small cap y valida si el movimiento esperado tiene una relación beneficio/riesgo mínima de 1:4.

El sistema no depende de noticias, fundamentales ni sentimiento en la v0. La capa de contexto es técnica: precio, volumen, volatilidad, liquidez, estructura de tendencia y geometría visual del patrón.

Componentes

Frontend

Next.js con Recharts. Muestra:

- equity curve;
- rendimiento por patrón;
- señales recientes;
- operaciones abiertas;
- estado del supervisor;
- botones para escaneo, backtest y reportes.

Backend

FastAPI expone una API privada bajo `/api`:

- `/api/health` estado;
- `/api/dashboard/summary` datos de dashboard;
- `/api/scan` escaneo manual;
- `/api/backtests/run` backtesting;
- `/api/signals` señales;
- `/api/reports/generate` paquetes de revisión;
- `/api/self-improvement/run` laboratorio de automejora;
- `/api/risk/kill-switch` auditoria de kill switch.

Los dominios principales viven bajo `backend/tradeo/modules/`:

- `research`: descubrimiento, hipótesis y validación científica;
- `laboratory`: validación paper de patrones de Research;
- `fox\_hunter`: vigilancia de patrones en producción y ejecución live gated;
- `shared`: mecánicas comunes de entrada que deben ser idénticas entre Lab y FoxHunter.

Detalle de fronteras: `docs/module\_boundaries.md`.

Worker

Proceso separado con APScheduler:

- escaneo periódico;
- reporte diario;
- automejora semanal.

Base de datos

PostgreSQL almacena:

- señales;
- operaciones;
- equity;
- métricas por patrón;
- versiones de estrategia;
- auditoría;
- ledger de riesgo.

Motor de patrones

`CupPatternDetector` evalúa ventanas de 45 a 210 barras y exige:

- profundidad controlada de base;
- simetría entre rims;
- handle no demasiado profundo;
- precio cerca de pivot o ruptura;
- volumen de ruptura;
- liquidez mínima;
- volatilidad acotada;
- entrada, stop y target explícitos;
- R:R mínimo 1:4.

Scoring combinado

El supervisor combina tres fuentes:

1. reglas cuantitativas OHLCV;
2. scoring tipo ML auditable basado en features;
3. geometría visual del gráfico mediante `VisionGeometryScorer`.

Broker

- `PaperBroker`: ejecución simulada interna.
- `IBKRBroker`: adaptador preparado para bracket orders de acciones, bloqueado por defecto.

Decisiones deliberadas

Por qué Docker

Docker separa backend, worker, frontend, PostgreSQL y Redis. Facilita escalar funcionalidades futuras sin contaminar la VM.

Por qué paper-first

El sistema debe demostrar estabilidad antes de ejecutar dinero real. El usuario aporta 3.000 USD, por lo que un error de sizing, slippage o opciones/margen puede destruir una parte significativa del capital.

Por qué no root total

Aunque la VM sea dedicada, el proceso de trading no necesita privilegios de root para operar. Las credenciales y la ejecución de órdenes son más sensibles que la instalación del sistema.

Research Lab

Tradeo incluye una ampliación de descubrimiento de patrones no predefinidos. El `PatternDiscoveryLabAgent` extrae ventanas OHLCV, genera embeddings locales, agrupa formas similares con clustering y valida si esos grupos precedieron movimientos favorables con R:R mínimo 1:4.

Este laboratorio no opera dinero real. Todo candidato queda en `lab` o `rejected` y requiere revisión posterior, backtest dedicado, paper trading y aprobación antes de tocar el motor operativo.

Comandos:

make discover-patterns
make research-runs

Documentación detallada: `docs/pattern\_discovery\_lab.md`.

Full Document: `docs/module\_boundaries.md`

`docs/module\_boundaries.md` ? 78 lines, 1829 bytes, sha256 prefix `689846347a47a69c`

Heading summary: Tradeo Module Boundaries; Research; Laboratory; FoxHunter; Shared

Tradeo Module Boundaries

Tradeo is split into three product domains plus shared primitives.

Research

Purpose: discover and validate pattern hypotheses. Research never submits broker orders.

Primary package:

- `backend/tradeo/research/`
- `backend/tradeo/modules/research/`

Key responsibilities:

- Window sampling, embeddings and clustering.
- Novel pattern registry and matching contracts.
- Research Director, hypothesis packages and experiment memory.
- Validation gates before any pattern can move toward paper validation.

Laboratory

Purpose: validate Research patterns through paper execution and shadow observations.

Primary package:

- `backend/tradeo/modules/laboratory/`

Key responsibilities:

- Laboratory API facade.
- Paper validation scans.
- Paper observation lifecycle.
- Laboratory diagnostics.

Laboratory may submit paper orders only when paper-mode safety gates allow it. It must not submit live orders.

FoxHunter

Purpose: monitor production-approved patterns and, only after explicit live arming, route live execution.

Primary package:

- `backend/tradeo/modules/fox\_hunter/`

Key responsibilities:

- FoxHunter API facade.
- Production manifest validation.
- Production-pattern scans.
- Live execution gating.

FoxHunter requires an active production manifest and live safety gates before any live order path can open.

Shared

Purpose: host mechanics that must stay identical across Laboratory and FoxHunter.

Primary package:

- `backend/tradeo/modules/shared/`

Key responsibilities:

- Entry matching orchestration.
- Common signal creation flow.
- Risk/quality gate integration.
- Runtime status updates for entry scans.

Compatibility wrappers remain under `backend/tradeo/services/` so older imports continue to work, but new code should enter through the domain packages.

Full Document: `docs/api\_review\_contract.md`

`docs/api\_review\_contract.md` ? 46 lines, 1125 bytes, sha256 prefix `69061ea38d89f746`

Heading summary: Contrato de revisión API / ChatGPT; Objetivo; Entrada; Respuesta esperada del supervisor externo; Reglas

```
# Contrato de revisión API / ChatGPT

## Objetivo

Permitir que un supervisor externo revise paquetes de auditoría generados por Tradeo.

## Entrada

`POST /api/reports/generate` produce un JSON con:

- estado del sistema;
- riesgo;
- señales recientes;
- operaciones;
- métricas;
- auditoría;
- prompt recomendado para revisión.

## Respuesta esperada del supervisor externo

{
  "approve_live": false,
  "approve_paper_continuation": true,
  "patterns": [
    {
      "symbol": "EXAMPLE",
      "signal_id": 1,
      "decision": "approve_paper",
      "reason": "Pattern geometry and risk are coherent; live not approved because sample is insufficient."
    }
  ],
  "required_changes": [
    "Increase minimum sample to 60 trades before promotion."
  ],
  "next_experiments": [
    "Test breakout volume threshold 1.2 vs 1.35."
  ]
}

## Reglas

- El supervisor externo no puede aprobar live si `TRADEO_LIVE_TRADING_ENABLED=false`.
- El supervisor externo no puede reducir `TRADEO_MIN_REWARD_RISK` por debajo de 4.
- Toda aprobación live requiere aprobación humana local y cambio explícito de `.env`.
```

Full Document: `docs/agent\_design.md`

`docs/agent\_design.md` ? 47 lines, 1171 bytes, sha256 prefix `f5d524456f157f3e`

Heading summary: Diseño de agentes; DataCollectorAgent; PatternHunterAgent; TechnicalContextAgent; RiskAgent; SupervisorAgent; ImprovementAgent; AuditAgent; Nivel de razonamiento recomendado

```
# Diseño de agentes

## DataCollectorAgent

Obtiene OHLCV y normaliza columnas. En v0 usa yfinance para prototipo y puede ampliarse a IBKR/Polygon.

## PatternHunterAgent

Ejecuta `CupPatternDetector`. Busca estructuras cup/base en ventanas multi-longitud.

## TechnicalContextAgent

Evalúa contexto puramente técnico:

- precio vs SMA50/SMA200;
- tendencia reciente;
- volatilidad;
- liquidez.

## RiskAgent

Aplica límites duros. Ningún broker recibe una orden si este agente rechaza.

## SupervisorAgent
```

Combina reglas, ML scorer, visión geométrica y riesgo. Decide:

- reject;
- watchlist;
- paper_approved;
- pending_human_approval;
- live_approved.

ImprovementAgent

Crea mutaciones de parámetros y lanza backtests. Solo guarda candidatos de laboratorio si pasan gates.

AuditAgent

Genera reportes JSON/Markdown para revisión externa. Incluye prompt para director.

Nivel de razonamiento recomendado

- Data/Risk/Audit: determinista, sin creatividad.
- Pattern/Supervisor: razonamiento medio-alto, pero auditable.
- Improvement: alto, pero limitado a laboratorio.
- API supervisor: alto, solo revisión estructurada; sin poder de saltar gates.

Full Document: `docs/autonomous\_research\_lab.md`

`docs/autonomous\_research\_lab.md` ? 57 lines, 2190 bytes, sha256 prefix `51d0eb0e1fd1dc49`

Heading summary: Autonomous Research Lab; Flow; Artifacts; Lifecycle; Kill Conditions; Tests

Autonomous Research Lab

Tradeo Research now adds a deterministic autonomous-science layer to discovery. It does not train neural networks, install dependencies, migrate the database or promote patterns to paper/live. All durable state is JSON/Markdown under `reports/research` and candidate `metrics\_json`.

Flow

1. `PatternDiscoveryLabAgent` samples windows and calls `ClusterResearchEngine`.
2. Each cluster receives:
 - `foundation\_teacher`: masked reconstruction, contrastive embedding and future-proxy diagnostics.
 - `market\_replay`: latency, late entry, partial fill, fill probability, size cap, spread/slippage and gap penalties.
 - `causal\_invariance`: regime/year/symbol/liquidity/sector proxy invariance and expected-fail buckets.
 - `adversarial\_challenge`: leakage probe, placebo, shuffled assignment, universe/date/cost/regime/parameter shocks.
3. `ValidationGate` consumes those metrics and can reject fragile candidates.
4. `ResearchDirector` runs at discovery completion before registry persistence.
5. The existing DB director/scheduler can still run separately when enabled.

Artifacts

The discovery-completion director writes:

- `reports/research/research\_memory\_graph.json`
- `reports/research/director/run\_{id}\_director\_{ts}.json`
- `reports/research/director/run\_{id}\_director\_{ts}.md`
- `reports/research/papers/run\_{id}/{pattern\_key}.md`

The memory graph stores families, variants, relations, regimes, decay and family state. It is intentionally JSON so existing databases stay compatible.

Lifecycle

Lifecycle states are suggestions only:

- `discovered`
- `challenged`
- `confirmed`
- `decaying`
- `retired`
- `resurrectable`

Every lifecycle payload includes:

- `paper\_live\_auto\_promotion: false`
- `director\_gate\_required\_for\_paper\_or\_live: true`

Kill Conditions

Generated hypotheses include explicit death conditions such as failed fresh OOS, negative replay expectancy, adversarial hard failure, causal concentration, cost-stress failure and parameter decay.

Tests

Focused tests live in `backend/tradeo/tests/test\_autonomous\_research\_lab.py`. They cover replay penalties, adversarial gate rejection and director artifacts.

Full Document: `docs/autonomous\_research\_director.md`

`docs/autonomous\_research\_director.md` ? 95 lines, 3455 bytes, sha256 prefix `49cc3d2f68effae0`

Heading summary: Tradeo Autonomous Research Director; Objetivo; Que hace; Automatizacion; Endpoints; Artifacts; Filosofia de seguridad; Interpretacion

Tradeo Autonomous Research Director

Objetivo

El `ResearchDirector` convierte Research en un laboratorio cientifico autonomo: no solo encuentra clusters, sino que los transforma en hipotesis falsables, los intenta romper, guarda memoria acumulativa y decide que estudiar despues.

No opera, no promociona a paper/live y no cambia permisos de ejecucion. Todo lo que produce queda en `metrics\_json`, snapshots de metricas y artifacts en `reports/research/director/`.

Que hace

- **Hypothesis Engine:** cada patron recibe tesis, mecanismo, condiciones donde deberia funcionar/fallar, kill criteria y evidencia.
- **Research Memory Graph:** guarda nodos de patrones, familias, variantes y regimenes en `research\_memory\_graph.json`.
- **Foundation-learning proxy:** calcula un teacher local y determinista para medir estabilidad de embedding, separacion contrastiva y alineacion con path futuro sin dependencias nuevas ni entrenamiento pesado.
- **Market Replay:** evalua tradability con fill probability, spread, slippage, gap, coste en R, fragilidad por entrada tarde y size cap.
- **Adversarial Research:** ejecuta checks de leakage, WRC/SPA, deflated Sharpe, cost shock, parameter decay, concentracion de simbolos/tiempo y replay.
- **Causal/Invariant Testing:** mide invariancia por anos, simbolos, purged CV y regimen dominante; declara buckets donde espera que falle.
- **Active Learning:** genera agenda priorizada con experimentos siguientes.
- **Pattern Lifecycle:** sugiere estados de investigacion como `confirmed\_lab\_hypothesis`, `challenged\_lab\_hypothesis`, `decaying`, `needs\_confirmation` o `rejected\_learning\_memory`.
- **Auto paper report:** crea mini papers por patron con abstract, regla, evidencia, anti-overfit, ejecucion, riesgos, death conditions y next action.

Automatizacion

El director corre de dos formas:

1. Al terminar cada `PatternDiscoveryLabAgent.run`, sobre los patrones del run.
2. En el worker cada `TRADEO\_RESEARCH\_DIRECTOR\_INTERVAL\_MINUTES` minutos.

Settings:

TRADEO_RESEARCH_DIRECTOR_ENABLED=true
TRADEO_RESEARCH_DIRECTOR_INTERVAL_MINUTES=180
TRADEO_RESEARCH_DIRECTOR_PATTERN_LIMIT=120

Endpoints

- `POST /api/research/director/run`
- `GET /api/research/director/latest`

Ejemplo:

```
curl -u "$TRADEO_ADMIN_USERNAME:$TRADEO_ADMIN_PASSWORD" \  
-X POST "http://localhost:8000/api/research/director/run?limit=120"
```

Artifacts

El director escribe:

- `reports/research/director/latest\_research\_director.json`
- `reports/research/director/latest\_research\_director.md`
- `reports/research/director/research\_memory\_graph.json`
- `reports/research/director/research\_director\_YYYYMMDD\_HHMMSS.json`
- `reports/research/director/research\_director\_YYYYMMDD\_HHMMSS.md`

Filosofia de seguridad

El director es un cientifico, no un trader:

- no envia ordenes;
- no cambia estados a paper/live;
- no habilita IBKR;
- no usa datos externos nuevos;
- no depende de LLM para tomar decisiones de ejecucion;
- deja trazabilidad completa para Director gate.

Interpretacion

Un patron bueno no es el que tiene mejor score bruto. Es el que:

- tiene hipotesis clara;
- sobrevive al abogado del diablo;
- mantiene edge fuera de muestra;
- no depende de pocos simbolos;

- se puede ejecutar con costes/fill realistas;
- sabe decir donde deberia fallar.

Ese es el salto: Tradeo deja de ser un scanner y empieza a acumular conocimiento científico sobre familias de mercado.

Full Document: `docs/lab\_research\_operating\_loop.md`

`docs/lab\_research\_operating\_loop.md` ? 238 lines, 7995 bytes, sha256 prefix `ef79275af30db727`

Heading summary: Lab and Research Operating Loop; Objetivo; Research; Lab Scanner; Paper/Shadow Observations; Near-Miss Shadow; Fallback Post-Close; Ranking; Director; Audit Bridge; Dashboard; Invariantes De Seguridad

Lab and Research Operating Loop

Fecha: 2026-06-09

Objetivo

Tradeo separa descubrimiento, validacion paper y produccion. Research puede encontrar familias visuales y proponer hipotesis, pero Lab debe acumular verdad operativa antes de que Director permita cualquier promocion.

El flujo esperado es:

1. Research descubre o revisa patrones.
2. Lab escanea el mercado actual y genera variantes de entrada.
3. Cada oportunidad queda deduplicada y auditada.
4. Las oportunidades de Lab crean observaciones paper/shadow, sin enviar ordenes.
5. Las observaciones cerradas alimentan ranking, regimenes, variantes y Director.
6. Director decide que estudiar, congelar o mantener como research-only.
7. El audit bridge exporta un paquete reproducible para revisar el estado.

Research

Research trabaja con labels path-dependent, embeddings enriquecidos, discovery por regimen y validacion anti-overfit. Sus resultados se guardan como patrones descubiertos y memoria de investigacion.

Research no puede promocionar directamente a ejecucion. Un patron con buen score solo pasa a ser candidato para Lab si sobrevive a filtros estadisticos y al Director gate.

Lab Scanner

Lab convierte cada match de Research en oportunidades de entrada concretas. Para cada oportunidad calcula:

- `entry\_variant\_id` y descripcion de la variante;
- precio de entrada, stop, target y reward/risk;
- `entry\_gate` y `base\_entry\_gate`;
- `entry\_audit` anti-lookahead;
- `regime` y `regime\_fit`;
- componentes de ranking;
- razones exactas de rechazo.

Los matches se deduplican por:

pattern_id + symbol + timeframe + entry_variant_id + window_end

Si llega el mismo match otra vez, Tradeo actualiza metadata y score en vez de crear filas repetidas.

Paper/Shadow Observations

Lab puede crear observaciones internas de tipo `lab\_shadow\_observation`.

Estas observaciones:

- no llaman a IBKR;
- no mandan ordenes;
- usan el precio/stop/target calculado por la variante;
- se cierran por target, stop o timeout cuando llegan nuevas barras;
- alimentan historial por patron, variante y regimen.

El objetivo es generar datos para decidir, no simular ejecucion live perfecta.

Near-Miss Shadow

Lab tambien puede abrir observaciones shadow para near-misses de alta prioridad aunque `entry\_gate` falle por confirmacion de volumen moderada. Esto solo aplica a `laboratory`; Fox Hunter y produccion no cambian.

La observacion near-miss exige:

- `entry\_variant\_id`;
- `regime.regime\_key`;
- ranking alto o `opportunity\_rank\_score` suficiente;
- fallo blando por `insufficient\_volume` o `weak\_volume\_confirmation`;
- sin blockers duros como trigger inexistente, regimen invalido, exceso de ATR,

sobreextension, score de entrada debil o riesgo rechazado.

Estas senales/observaciones quedan marcadas con:

- `entry\_variant\_id`;
- `regime`;
- `entry\_gate`;
- `entry\_quality`;
- `opportunity\_rank`;
- `opportunity\_rank\_score`;
- `near\_miss=true`;
- `near\_miss\_shadow=true`;
- `would\_have\_failed\_entry\_gate=true`;
- `paper\_only=true`;
- `no\_ibkr\_order=true`;
- razones exactas en `near\_miss\_reasons`.

Cuando cierran, cuentan como historial shadow de Lab para ranking y Director.
No relajan promociones: Director sigue bloqueando hasta que su gate apruebe evidencia suficiente.

El objetivo es generar historia Lab cerrada para Director sin relajar ejecucion.
Director sigue bloqueando promociones hasta que haya evidencia paper suficiente.

Fallback Post-Close

Tras el cierre de mercado, IBKR puede responder `no bars` para una observacion recién abierta. Ese caso no debe dejar el lifecycle mudo ni intentar ordenes.

`LabPaperObservationService` aplica este orden conservador:

1. Si hay barras nuevas de mercado, evalua target, stop y time stop como antes.
2. Si el provider no devuelve barras, busca una ultima vela o precio guardado en metadata (`latest\_bar`, `last\_bar`, `entry\_gate.close`, `signal\_snapshot.features.last\_close` o `match.metrics.entry\_gate`).
3. Si esa vela guardada es posterior a `opened\_at` y toca stop/target, cierra la observacion shadow de forma determinista con `no\_ibkr\_order=true`.
4. Si solo sirve para valorar, deja la trade `open` y escribe `shadow\_lifecycle.status=market\_data\_unavailable` con `mark\_to\_market`, razon, timestamp, fuente del fallback y `paper\_only=true`.
5. Si no hay fallback usable, deja la trade `open` y registra `market\_data\_unavailable` con razon y timestamp para auditoria.

Este fallback solo afecta observaciones `lab\_shadow\_observation`; no activa live, no envia ordenes a IBKR y no convierte near-miss en promocion.

Ranking

El ranking combina:

- calidad de entrada;
- similitud contra el patron;
- edge de Research;
- reward/risk;
- liquidez;
- fit de regimen;
- historial paper cerrado cuando existe;
- bonus pequeno de exploracion para variantes con poca muestra.

Si no hay historial paper, el ranking cae a score Research + entry score. La metadata debe explicar que componentes se usaron.

Director

Director agrega Lab execution dentro de `metrics\_json["lab\_execution"]`.
Cuando hay trades cerrados, separa:

- rendimiento por `entry\_variant\_id`;
- rendimiento por `regime\_key`;
- drawdown, expectancy, winrate y profit factor;
- diferencia contra baseline y contra Research.

Cuando no hay datos suficientes, Director debe decirlo de forma accionable:

- cuantos trades faltan;
- que buckets siguen vacios;
- que patron debe permanecer congelado;
- que evidencia falta para revisar promocion.

Audit Bridge

El audit bridge genera paquetes reproducibles bajo:

research/audit_bridge/requests/<AUDIT_ID>/

El contrato actual exige:

- manifest y conteos coherentes;
- configuracion redacted;
- `paper\_trades.csv` y `ib\_fills.csv` consistentes;
- sin fills huérfanos;
- duplicados reportados;
- Director gate presente;
- `metrics\_by\_regime.csv`;

- `metrics\_by\_entry\_variant.csv`;
- buckets vacios con `empty\_reason` cuando no hay closed Lab trades.

Un paquete puede validar tecnicamente y aun asi quedar `blocked`. Eso es normal: blocked significa que no hay suficiente evidencia para promocionar, no que el export este roto.

Dashboard

El dashboard de Laboratorio muestra diagnostico de solo lectura desde:

GET /api/laboratory/diagnostics?limit=24

Debe ayudar a responder:

- que oportunidades se estan viendo;
- que variante y regimen corresponden;
- por que entry gate rechazo;
- que historial paper existe;
- que falta para promover;
- que oportunidades son near-miss.

Invariantes De Seguridad

- `live\_armed=false` salvo decision humana explicita.
- Lab no envia ordenes live.
- Shadow observations no llaman a IBKR.
- Fox Hunter solo usa patrones de produccion.
- Director no promociona automaticamente.
- Los exports no deben contener secretos ni account IDs reales.

Estado Esperado Tras Redeploy

Un estado sano durante mercado abierto puede ser:

- backend y worker healthy;
- IBKR health OK;
- Lab encuentra matches;
- entry gate rechaza oportunidades debiles;
- shadow observations empiezan a acumular evidencia;
- Director sigue bloqueando promociones hasta tener trades/fills suficientes;
- audit export valida contrato pero puede quedar `blocked` por falta de evidencia.

Operacion 2026-06-09

Tras el redeploy del ciclo Lab/Research se detectaron duplicados historicos en `discovered\_pattern\_matches`. Se dejaron 877 matches unicos vivos por la clave:

pattern_id, symbol, timeframe, entry_variant_id, window_end

Las 53.381 filas duplicadas se conservaron en la tabla local de backup `discovered\_pattern\_matches\_dedupe\_backup\_20260609`. Despues de repetir un scan manual limitado, la DB siguio con `duplicate\_rows=0`, confirmando que el upsert actual no vuelve a crear duplicados exactos.

El paquete

`TRADEO-AUDIT-20260609-1601\_post\_dedupe\_health` queda como referencia post-fix: el export es usable y genera los buckets nuevos, pero Director lo bloquea porque aun no existen closed Lab trades ni fills IB Paper enlazados.

Full Document: `docs/laboratory\_entry\_signal\_engine.md`

`docs/laboratory\_entry\_signal\_engine.md` ? 98 lines, 3848 bytes, sha256 prefix `8a726137a4e06c5d`

Heading summary: Laboratory Entry Signal Engine; Objetivo; Cambios; 1. Scanner mas rapido; 2. Variantes de entrada; 3. Anti-lookahead operativo; 4. Regime router; 5. Ranking adaptativo; 6. Paper truth y Director; 7. Diagnostico UI/API; Invariantes

Laboratory Entry Signal Engine

Fecha: 2026-06-09

Objetivo

Laboratorio ya no trata un match de Research como una entrada unica. Ahora convierte cada match en candidatos de entrada auditables, aprende de paper trades cerrados y prioriza solo la mejor oportunidad por patron/simbolo en cada scan.

Cambios

1. Scanner mas rapido

- `NovelPatternMatcher.match\_current` agrupa patrones por `timeframe`.
- Descarga cada `(symbol, timeframe)` una sola vez por scan.
- Calcula cada embedding `(symbol, timeframe, window\_size)` una sola vez.
- Compara ese embedding contra todos los centroides compatibles.
- SPY/QQQ se cachean por timeframe para contexto de fuerza relativa.

2. Variantes de entrada

Cada match puede generar variantes:

- ``*_next_bar_limit``: entrada tras breakout/reclaim/momentum confirmado.
- ``next_bar_stop_confirmation``: exige continuacion por encima/debajo del extremo de la vela.
- ``next_bar_limit_retest``: espera retest cerca del nivel de ruptura/reclaim.
- ``volume_confirmed_close``: variante con confirmacion de volumen.
- ``gap_open_follow_through``: gap con continuacion hasta cierre.

Laboratorio considera varias variantes, pero procesa solo la mejor por ``(symbol, pattern)`` en cada scan para evitar exposicion duplicada.

3. Anti-lookahead operativo

Cada match/senal guarda:

- ``available_data_cutoff_ts``
- ``decision_ts``
- ``entry_eligible_ts``
- ``label_generated_ts``
- ``source_bar_hash``
- ``split_id``

La regla es simple: una entrada solo es elegible despues del cutoff de datos usados para decidir.

4. Regime router

Cada match conserva ``regime_key`` con:

- regimen de mercado;
- tendencia;
- volatilidad;
- liquidez;
- fuerza relativa vs SPY.

Si Research guardo ``regime_profile``, el matcher calcula ``regime_fit`` para favorecer setups en regimenes ya vistos por el patron.

5. Ranking adaptativo

``opportunity_ranking`` ahora usa:

- calidad de entrada;
- similitud;
- edge de Research;
- reward/risk;
- liquidez;
- fit de regimen;
- historial de ejecucion real por patron, simbolo, variante y regimen;
- bonus pequeno de exploracion cuando una variante tiene poca muestra.

El historial sale de trades cerrados de Laboratorio, no de metricas teoricas de Research.

6. Paper truth y Director

- Las senales guardan ``entry_variant_id``, ``entry_variant``, ``entry_audit``, ``regime`` y ``regime_fit``.
- ``IBKRBroker.submit_signal_bracket`` copia esos campos al trade.
- ``DirectorReviewGate`` agrega performance por variante y regimen dentro de ``pattern.metrics_json["lab_execution"]``.
- El audit exporter ya puede exportar ``paper_trades.csv`` e ``ib_fills.csv`` desde el overview de Laboratorio cuando existan.

Lab puede abrir ``lab_shadow_observation`` near-miss para oportunidades top-ranked que fallan ``entry_gate`` solo por volumen (``insufficient_volume`` o ``weak_volume_confirmation``). Estas senales quedan ``near_miss=true``, ``would_have_failed_entry_gate=true``, ``paper_only=true`` y ``no_ibkr_order=true``; Fox Hunter no usa este modo y no se envia ninguna orden IBKR.

7. Diagnostico UI/API

- ``GET /api/laboratory/diagnostics?limit=24`` es un endpoint de solo lectura para presentacion.
- Combina senales paper candidatas, rechazos en ``audit_logs`` y ``discovered_pattern_matches`` recientes.
- Cada fila resume simbolo, patron, variante, regimen, rank/score, razon de rechazo, componentes de ``entry_gate``, historial paper por patron/variante/regimen y blockers para promocion.
- No ejecuta scanner, no crea senales y no envia ordenes.
- El dashboard muestra esta informacion en ``Laboratorio -> Diagnostico Lab``.

Invariantes

- Laboratorio sigue siendo paper-first.
- Fox Hunter sigue usando solo patrones ``production``.
- No hay promocion automatica a Produccion.
- Director sigue exigiendo trades/fills reales antes de aprobar.
- La UI solo muestra diagnosticos de presentacion; no cambia ejecucion.

Full Document: ``docs/pattern_discovery_lab.md``

``docs/pattern_discovery_lab.md`` ? 137 lines, 5753 bytes, sha256 prefix ``26fc2d5213971221``

Heading summary: Tradeo Pattern Discovery Lab; Objetivo; Pipeline; Política de tokens; Endpoints; Comandos; Scheduler; Estados; Qué todavía no hace

Tradeo Pattern Discovery Lab

Objetivo

El ``PatternDiscoveryLabAgent`` amplía Tradeo para buscar patrones no definidos previamente en acciones USA mid/small cap. No intenta

reconocer una figura conocida como cup/base; extrae millones de ventanas OHLCV, las transforma en fingerprints numéricos, agrupa formas parecidas y mide qué ocurrió después de cada grupo.

El laboratorio es pionero, pero está aislado del motor operativo. Su salida máxima es un patrón en estado `LAB`. Ningún patrón descubierto puede operar dinero real automáticamente.

Pipeline

- 1. **WindowSampler****
 - Extrae ventanas de 20, 50, 100 y 200 velas.
 - Calcula forward returns a 5, 10 y 20 velas.
 - Calcula MFE/MAE y resultado en R para long y short con objetivo 4R y stop 1R.
 - Usa una hipótesis conservadora: si en una vela diaria se toca stop y target, cuenta primero el stop.
- 2. **PatternEmbeddingEngine****
 - Convierte cada ventana OHLCV en un vector fijo.
 - Features: retornos, volumen relativo, rango, posición del cierre, cuerpo de vela, volatilidad rolling, drawdown local, distancia a mínimos, pendiente y ATR relativo.
 - Todo se ejecuta localmente con NumPy/pandas. No se gastan tokens en esta fase.
- 3. **ClusterResearchEngine****
 - Agrupa ventanas similares por tamaño de ventana con `MiniBatchKMeans`.
 - Para cada cluster decide si la ventaja histórica fue long o short.
 - Calcula expectancy, profit factor, win rate, hit rate de 4R, MFE/MAE, estabilidad por símbolo, estabilidad por año y split out-of-sample.
- 4. **ValidationGate****

Rechaza patrones si no pasan mínimos duros:

 - muestras mínimas;
 - diversidad mínima de símbolos;
 - diversidad temporal mínima;
 - R:R estimado mínimo 1:4;
 - profit factor mínimo;
 - expectancy mínima;
 - estabilidad mínima;
 - out-of-sample positivo.
- 5. **NovelPatternRegistry****
 - Guarda patrones aceptados como `lab`.
 - Guarda rechazados si `TRADEO\_DISCOVERY\_STORE\_REJECTED=true` para auditoría.
 - Guarda ejemplos típicos, ganadores y perdedores.
- 6. **Research digest****
 - Exporta JSON y Markdown en `reports/research/`.
 - El digest es compacto para revisión humana/API: no incluye millones de velas crudas.
- 7. **Autonomous Research Director****
 - Convierte cada patrón en hipótesis falsable.
 - Ejecuta challenge adversarial, invariance testing, replay de ejecución y lifecycle.
 - Mantiene un memory graph de familias, variantes y regímenes.
 - Genera agenda activa de próximos experimentos y mini papers por patrón.
 - Ver: `docs/autonomous\_research\_director.md`.

Política de tokens

El agente está diseñado para trabajar todo el día sin consumir tokens de modelo:

- No usa LLM para muestrear ventanas.
- No usa LLM para clustering.
- No usa LLM para calcular métricas.
- No envía gráficos crudos ni histórico completo a la API.
- Solo genera un digest de los mejores patrones con métricas, razones de validación y ejemplos comprimidos.

El supervisor API debe recibir únicamente el archivo JSON/Markdown de `reports/research/` o los endpoints de resumen.

Endpoints

- `POST /api/research/run-discovery`
- `GET /api/research/discovered-patterns`
- `GET /api/research/discovered-patterns/{id}`
- `GET /api/research/discovered-patterns/{id}/examples`
- `GET /api/research/discovered-patterns/{id}/metrics`
- `POST /api/research/match-current`
- `GET /api/research/current-matches`
- `GET /api/research/runs`
- `POST /api/research/director/run`
- `GET /api/research/director/latest`

Comandos

```
make discover-patterns
make match-discovered-patterns
make current-matches
make research-runs
```

Para una ejecución manual más grande:

```
curl -u "$TRADEO_ADMIN_USERNAME:$TRADEO_ADMIN_PASSWORD" \
-X POST http://localhost:8000/api/research/run-discovery \
-H 'Content-Type: application/json' \
```



```
-d '{"limit":120,"period":"5y","interval":"1d","max_total_windows":20000,"max_windows_per_symbol":500}'
```

Scheduler

El worker ejecuta el laboratorio cada `TRADEO\_DISCOVERY\_SCAN\_MINUTES` minutos si:

```
TRADEO_SCHEDULER_ENABLED=true
TRADEO_DISCOVERY_ENABLED=true
TRADEO_DISCOVERY_SCHEDULER_ENABLED=true
```

Por defecto son 90 minutos. Se usa `max\_instances=1` para evitar solapamientos.

El Research Director también corre automáticamente:

```
TRADEO_RESEARCH_DIRECTOR_ENABLED=true
TRADEO_RESEARCH_DIRECTOR_INTERVAL_MINUTES=180
TRADEO_RESEARCH_DIRECTOR_PATTERN_LIMIT=120
```

Además, cada discovery run lo invoca al terminar para que los patrones nuevos queden enriquecidos con hipótesis, challenge, lifecycle, memory graph y agenda.

Estados

- `lab`: patrón aceptado por el gate estadístico. Requiere revisión humana/API, backtest dedicado y paper trading antes de uso operativo.
- `rejected`: patrón interesante pero insuficiente. Se guarda para aprender qué no funciona y evitar repetir falsos positivos.
- `paper\_watchlist`: reservado para futuras promociones manuales.
- `retired`: patrón retirado.

Qué todavía no hace

- No convierte automáticamente un patrón descubierto en estrategia operativa.
- No opera con dinero real.
- No promociona a paper/live sin Director gate.
- No entrena modelos profundos todavía; usa un proxy self-supervised local para medir calidad de embedding y aprendizaje.

La ampliación incluye `NovelPatternMatcher`, que compara gráficos actuales contra centroides validados y guarda coincidencias en `lab\_watchlist`. Esas coincidencias no son señales operativas; sirven para paper validation y revisión.

Ver también: `docs/laboratory\_entry\_signal\_engine.md` para el motor operativo de Laboratorio: variantes de entrada, anti-lookahead, regime router, ranking adaptativo y aprendizaje desde paper trades.

Full Document: `docs/risk\_policy.md`

`docs/risk\_policy.md` ? 63 lines, 1893 bytes, sha256 prefix `5a44930978604f25`

Heading summary: Política de riesgo de Tradeo; Parámetros iniciales; Fórmula de tamaño de posición; Rechazos duros; Opciones y margen; Gates antes de live

Política de riesgo de Tradeo

Parámetros iniciales

- Capital inicial: 3.000 USD.
- Riesgo máximo por operación: 1% = 30 USD.
- Pérdida máxima diaria: 2% = 60 USD.
- Pérdida máxima mensual: 8% = 240 USD.
- Máximo de posiciones abiertas: 4.
- R:R mínimo: 1:4.
- Exposición máxima por posición: 45% del equity.
- Exposición bruta máxima sin margen: 95% del equity.

Fórmula de tamaño de posición

```
riesgo_por_accion = abs(entry - stop)
qty_por_riesgo = floor((equity * 0.01) / riesgo_por_accion)
qty_por_notional = floor((equity * max_position_value_pct) / entry)
qty = min(qty_por_riesgo, qty_por_notional)
```

La operación se rechaza si `qty <= 0` o si el riesgo excede el presupuesto.

Rechazos duros

Una señal no puede avanzar si ocurre cualquiera de estas condiciones:

- kill switch activo;
- R:R menor a 1:4;
- stop inválido;
- liquidez media insuficiente;
- ATR porcentual excesivo;
- posiciones máximas alcanzadas;

- pérdida diaria alcanzada;
- dirección deshabilitada;
- live trading no armado;
- falta aprobación humana para live.

Opciones y margen

Aunque el usuario permite opciones y margen, la v0 los mantiene desactivados por defecto. Motivo: con 3.000 USD, opciones, assignment, spreads mal modelados, margen y short borrow pueden convertir una pérdida controlada en una pérdida no lineal.

La ruta correcta es:

1. validar acciones en paper;
2. añadir módulo de opciones separado;
3. modelar griegas, liquidez, spread, assignment y vencimiento;
4. backtest específico;
5. paper trading específico;
6. aprobación externa.

Gates antes de live

Una estrategia no pasa a revisión live si no cumple:

- 40 operaciones históricas mínimas;
- profit factor ≥ 1.8 ;
- expectancy $\geq 0.25R$;
- drawdown máximo $\leq 12\%$;
- al menos una fase de paper trading estable;
- revisión del paquete de auditoria;
- aprobación humana.

Full Document: `docs/libkr\_operational\_guide.md`

`docs/libkr\_operational\_guide.md` ? 76 lines, 1818 bytes, sha256 prefix `46c9ba903f231d10`

Heading summary: Tradeo IBKR operational guide; Default posture; Endpoints; Suggested setup steps; Live mode gate

Tradeo IBKR operational guide

This integration is designed for controlled IBKR paper/live execution through TWS or IB Gateway.

Default posture

Keep the first phase read-only:

```
TRADEO_TRADING_MODE=paper
TRADEO_IBKR_HOST=host.docker.internal
TRADEO_IBKR_PORT=7497
TRADEO_IBKR_CLIENT_ID=17
TRADEO_IBKR_READONLY=true
TRADEO_LIVE_TRADING_ENABLED=false
TRADEO_LIVE_TRADING_CONFIRMATION_VALUE=
TRADEO_KILL_SWITCH_ENABLED=false
```

Endpoints

All endpoints are under `/api`:

```
GET /api/libkr/health
GET /api/libkr/account
GET /api/libkr/positions
GET /api/libkr/open-orders
POST /api/libkr/signals/{signal_id}/preview
POST /api/libkr/signals/{signal_id}/submit-bracket
```

The submit endpoint is blocked unless all hard gates pass:

- `TRADEO\_IBKR\_READONLY=false`;
- kill switch is off;
- signal has `human\_approved=true`;
- signal status is `paper\_approved` or `live\_approved`;
- order notional is below `TRADEO\_IBKR\_MAX\_ORDER\_VALUE\_USD`;
- live ports or live mode require `live\_armed=true`.

Suggested setup steps

1. Start TWS or IB Gateway in paper mode.
2. Enable API socket clients in TWS/Gateway.
3. Verify the socket port.
4. Run:

```
curl http://localhost:8000/api/libkr/health
```

5. Keep orders blocked until health/account/positions are stable.
6. For IBKR paper order testing only, set:

```
TRADEO_IBKR_READONLY=false
TRADEO_TRADING_MODE=paper
TRADEO_IBKR_PORT=7497
```

7. Never point to live ports (`7496` or `4001`) unless the live gate is deliberately armed.

Live mode gate

Live execution requires all of this:

```
TRADEO_TRADING_MODE=live
TRADEO_LIVE_TRADING_ENABLED=true
TRADEO_LIVE_TRADING_CONFIRMATION_VALUE=I_ACCEPT_LIVE_MARKET_RISK
TRADEO_IBKR_READONLY=false
TRADEO_KILL_SWITCH_ENABLED=false
```

This still does not bypass human approval on the signal.

Full Document: `docs/openclaw.md`

`docs/openclaw.md` ? 45 lines, 905 bytes, sha256 prefix `c4f1c915c697d33f`

Heading summary: OpenClaw como operador de despliegue; Comandos permitidos; Comandos peligrosos; Tareas periódicas

OpenClaw como operador de despliegue

OpenClaw debe tratar Tradeo como software financiero sensible. Su rol recomendado es:

- desplegar;
- comprobar logs;
- ejecutar escaneos;
- generar reportes;
- avisar al usuario;
- nunca activar live trading por iniciativa propia.

Comandos permitidos

```
make setup
make up
make logs
make scan
make report
make self-improve
make test
```

Comandos peligrosos

Cualquier comando que cambie estas variables requiere aprobación explícita del usuario:

```
TRADEO_TRADING_MODE=<non-paper-value>
TRADEO_LIVE_TRADING_ENABLED=<non-false-value>
TRADEO_IBKR_READONLY=<non-true-value>
TRADEO_ALLOW_OPTIONS=true
TRADEO_ALLOW_MARGIN=true
```

Tareas periódicas

El worker ya ejecuta escaneos y reportes. OpenClaw puede hacer una comprobación externa diaria:

```
docker compose ps
make report
```

Y devolver al usuario la ruta del último reporte.

Full Document: `docs/deployment.md`

`docs/deployment.md` ? 47 lines, 794 bytes, sha256 prefix `8df3a6931992c4f9`

Heading summary: Despliegue en Ubuntu local/VPN; Requisitos; Instalación de Docker si falta; Despliegue; Comprobación; Backup mínimo; Actualización

```
# Despliegue en Ubuntu local/VPN

## Requisitos

- Ubuntu moderno.
- Docker y Docker Compose Plugin.
- Conectividad a internet para datos y dependencias.
- IBKR TWS o IB Gateway solo si quieres integrar broker.

## Instalación de Docker si falta

sudo apt-get update
sudo apt-get install -y docker.io docker-compose-plugin
sudo usermod -aG docker "$USER"

Cierra sesión y vuelve a entrar si acabas de añadir tu usuario al grupo docker.

## Despliegue

cd tradeo
make setup
nano .env
make up

## Comprobación

docker compose ps
curl http://localhost:8000/api/health

## Backup mínimo

tar -czf tradeo_reports_backup_$(date +%Y%m%d).tar.gz reports data config .env

## Actualización

git pull || true
docker compose up -d --build
```

Full Document: `docs/audit\_2026\_06\_10\_implementation\_report.md`

`docs/audit\_2026\_06\_10\_implementation\_report.md` ? 68 lines, 3377 bytes, sha256 prefix `df974ab63c4d0503`

Heading summary: Tradeo Audit Implementation Report - 2026-06-10; Implemented Improvements; Deliberate Non-Activation; Verification

```
# Tradeo Audit Implementation Report - 2026-06-10

Source audit: critical internal audit dated 2026-06-10.

Main verdict after implementation: Tradeo remains blocked from live trading and from any "edge demonstrated" claim, but the runtime, dashboard, research layer and audit package now enforce the scientific safety contracts requested by the audit.

## Implemented Improvements

- Evidence taxonomy added: `near_miss_shadow`, `shadow_no_order`, `ibkr_paper_order`, `ibkr_paper_fill`, `live_order`, `live_fill`.
- Director Review now counts only normal `ibkr_paper_fill` evidence.
- Shadow, near-miss, `no_ibkr_order`, `observation_only` and degraded fallback evidence are excluded from promotion metrics.
- The 10-trade Director threshold is a review trigger only, not production approval.
- `DirectorProductionGate` is separate from `DirectorReviewGate`.
- Dashboard separates operational fills from shadow and near-miss observations.
- Dashboard PnL, win rate and closed-trade execution stats use only paper/live fill evidence.
- Lab fallback evidence is marked `evidence_quality=degraded`.
- Fox Hunter requires `production` status plus a valid `ProductionManifest`.
- Production patterns without a valid manifest are ignored before signal/order creation.
- Legacy states `paper_candidate` and `premium_candidate` are excluded from runtime eligibility.
- `NovelPatternRegistry` canonicalizes legacy promotion states back to lab-safe
```

```

states and marks them as blocked.
- Research full-sample metrics are labeled `descriptive_all_*`.
- Research scoring/ranking no longer consumes `descriptive_all_*` metrics.
- Hypothesis packages are immutable and include split, fit scope, global trial
  count, ledger hash, kill conditions and `edge_claim=NO_DEMOSTRADO`.
- `global_experiment_registry` accumulates multiple-testing trial counts.
- WRC/SPA-like checks are labeled as `bootstrap_reality_proxy`, not formal
  tests.
- Audit bridge `paper_trades.csv` requires `evidence_type` and
  `evidence_quality`.
- Audit bridge exports only normal `ibkr_paper_fill` rows as paper trades.
- Audit validator rejects shadow/no-broker evidence inside `paper_trades.csv`.
- Audit `experiment_registry.csv` exports global trial count, p-values, fit
  scope and score input scope.

## Deliberate Non-Activation

- No live trading was enabled.
- No orders were sent.
- Real production manifests still require human/Director approval before use.
- Nested discovery replay remains an explicit blocking contract; it was not
  upgraded to a full replay implementation in this change.

## Verification

- `./venv/bin/python -m pytest tradeo/tests/test_audit_hardening.py tradeo/tests/test_novel_pattern_registry.py
tradeo/tests/test_pattern_discovery_lab.py tradeo/tests/test_autonomous_research_lab.py tradeo/tests/test_director_review_gate.py
tradeo/tests/test_lab_diagnostics.py tradeo/tests/test_module_dashboard.py tradeo/tests/test_pattern_entry_scanner.py
tradeo/tests/test_research_lab_fox_lifecycle.py -q`
  - Result: 79 passed, 24 warnings.
- `./venv/bin/ruff check ...`
  - Result: passed.
- `./venv/bin/python -m compileall ...`
  - Result: passed.
- `git diff --check`
  - Result: clean.
- `npm run build` in `frontend/`
  - Result: passed.
- `npm run lint` in `frontend/`
  - Result: not validated; Next entered ESLint setup because no ESLint config is
    present.

```

Full Document: `docs/fable5\_tradeo\_upgrade\_2026\_06\_10.md`

`docs/fable5\_tradeo\_upgrade\_2026\_06\_10.md` ? 136 lines, 8558 bytes, sha256 prefix `3bb1da852451f991`

Heading summary: Auditoría y mejora de Tradeo ? Fable5, 2026-06-10; 1. Veredicto general; 2. Hallazgos por área; Filosofía e intenciones ? sólido; Seguridad paper/live ? sólido; Calidad de datos ? hueco corregido en esta rama; Automatización y observabilidad ? correcto con huecos; Tests y CI ? hueco principal, corregido; Frontend / UX ? funcional, mejorable; Código backend ? bueno, con matices; 3. Cambios implementados en esta rama; 4. Verificación

Auditoría y mejora de Tradeo ? Fable5, 2026-06-10

Rama: `fable5/tradeo-research-upgrade-20260610` ? merge a `main`.
 Alcance: auditoría completa (filosofía, arquitectura, research, datos, seguridad paper/live, automatización, observabilidad, tests, docs) + mejoras de alto impacto y bajo riesgo implementadas y verificadas.

1. Veredicto general

Tradeo está en un estado notablemente mejor de lo que su tamaño sugiere. La filosofía ("paper y validación estadística primero; live bloqueado por defecto") está implementada de verdad, no solo declarada: la cadena Research ? Lab ? Director review ? Producción ? Fox Hunter ? Live tiene gates reales en código, con provenance de fills, manifiestos firmados con caducidad, kill switch, límites duros de riesgo y triple confirmación para armar live. Las auditorías previas (R1?R6) cerraron los huecos graves de evidencia.

Los puntos débiles reales no estaban en la seguridad de trading sino en la ingeniería alrededor: **no había CI que ejecutara tests ni lint**, el frontend no tenía lockfile (builds no reproducibles), y el pipeline de research consumía OHLCV sin un filtro de calidad por símbolo (halts, feeds congelados, splits sin ajustar, huecos de calendario), que es justo el tipo de ruido que envenena estadísticas de patrones en small caps.

2. Hallazgos por área

Filosofía e intenciones ? sólido

- Separación estricta de módulos: Research (descubre, nunca ejecuta), Laboratory (paper/shadow, nunca manda órdenes), Fox Hunter (solo patrones `production` con manifiesto válido). Research no puede marcar nada como `production`.
- Promoción cerrada con evidencia: ValidationGate estadístico (muestras, diversidad, expectancy, profit factor, p-value ajustado, walk-forward) ? DirectorReviewGate (?10 fills paper, ?3 símbolos, PF ?1.2) ? manifiesto de producción firmado y con caducidad (90 días) ? gate `live\_armed` en tiempo de ejecución.

Seguridad paper/live ? sólido

- Live exige simultáneamente: `trading\_mode=live`, `live\_trading\_enabled=true`, frase de confirmación correcta, kill switch apagado, `ibkr\_readonly=false`, aprobación humana por señal y solo órdenes bracket con tope de valor (1.500 USD).
- Datos sintéticos prohibidos por validator (`allow\_synthetic\_market\_data=False`).
- `.env` con credenciales **no está ni ha estado nunca en el historial de git** (verificado con `git log --all -- .env`); solo existe `.env.example`. El barrido inicial lo marcó como crítico por error: la contraseña vive solo en disco local.

```

#### Calidad de datos ? hueco corregido en esta rama
- `normalize_ohlcv`/`validate_ohlcv` ya rechazaban barras estructuralmente rotas
  (NaN, precios ?0, high<low, índice desordenado/duplicado). Bien.
- Faltaba la capa siguiente: datos bien formados pero estadísticamente tóxicos.
  **Añadido `services/data_quality.py`** (ver §3).

#### Automatización y observabilidad ? correcto con huecos
- Scheduler APScheduler con 6+ jobs (scan, discovery, matching, lab, fox, director,
  reportes), watchdog que cierra discovery runs zombies, heartbeat del worker,
  endpoints `/health`, `/health/deep`, `/health/ibkr`.
- Huecos: el watchdog solo loguea (sin alerting), sin backup específico de PostgreSQL
  (solo tar de ficheros), sin rotación de logs/reportes. Ver roadmap §6.

#### Tests y CI ? hueco principal, corregido
- 109 tests pasaban en local, ruff limpio? pero **ningún workflow de CI los ejecutaba**:
  `director-audit.yml` solo valida paquetes de auditoría. Un push podía romper el
  backend sin que nada avisara. **Añadido `.github/workflows/ci.yml`** (ver §3).

#### Frontend / UX ? funcional, mejorable
- Dashboard único (`page.tsx`, ~990 líneas) con los tres módulos, embudo de señales,
  diagnósticos del Lab y near-misses. Cumple para operación personal.
- Sin lockfile ? builds no reproducibles. **Corregido**: `package-lock.json` generado
  (fija Next 14.2.35, que además incorpora los fixes de seguridad de la serie 14.2.x).
- Pendiente (no bloqueante): ESLint sin configurar, sin vista de resultados de
  backtest, componente monolítico difícil de mantener. Ver roadmap.

#### Código backend ? bueno, con matices
- Tipado consistente, AuditLog en todas las acciones relevantes, config con 250+
  settings validados. Dos señalamientos del barrido inicial resultaron falsos
  positivos al revisarlos: el doble `research_hypothesis`/`research_hypothesis_package`
  es un alias intencional consumido por director_review_gate y el lab agent, y el
  `time.sleep(5)` del worker es el bucle de heartbeat.
- Matices reales que quedan: ~32 `except Exception` # noqa: BLE001 (enmascaran bugs),
  llamadas IBKR síncronas dentro de FastAPI (bloquean el event loop hasta 8 s), y
  umbrales mágicos sin justificar (p. ej. similitud 96% en dedupe de patrones).

## 3. Cambios implementados en esta rama

1. **Filtro de calidad de datos por símbolo** (`backend/tradeo/services/data_quality.py`):
  `assess_ohlcv_quality()` produce un `DataQualityReport` con métricas e issues:
  - `insufficient_bars` (menos de 60 barras por defecto)
  - `excess_zero_volume_bars` (>15% de barras sin volumen ? halts/iliquidez)
  - `stale_close_run` (>8 cierres idénticos seguidos ? feed congelado)
  - `calendar_gap` (>5 días hábiles sin barra, solo en `1d` ? delisting/halt/datos rotos)
  - `suspect_split_or_bad_tick` (ratio close/close >4x o <0.25x ? split sin ajustar)
  Cableado en `MarketScanner`: el símbolo se salta, se cuenta en el nuevo campo
  `ScanResponse.data_quality_skips` y queda registrado en `AuditLog` con acción
  `market_data_quality_reject` y el informe completo. Configurable vía settings
  `TRADEO_DATA_QUALITY_*` (documentados en `.env.example`); se puede desactivar con
  `TRADEO_DATA_QUALITY_FILTER_ENABLED=false`. El módulo es reutilizable desde el
  discovery lab y el entry scanner (siguiente paso natural, ver §6).
2. **CI real** (`.github/workflows/ci.yml`): en cada push a main y cada PR ejecuta
  ruff + pytest del backend (Python 3.12, `pip install -e ".[dev]"`) y build del
  frontend (Node 20, `npm ci` + `next build`).
3. **Lockfile del frontend** (`frontend/package-lock.json`): builds reproducibles y
  `npm ci` posible en CI; resuelve Next a 14.2.35.
4. **Tests**: `test_data_quality.py` con 11 tests (unitarios de cada issue + integración
  del scanner con AuditLog sobre SQLite en memoria).
5. **Higiene**: `pd.Timestamp.utcnow()` deprecado sustituido en fixtures (los avisos
  del suite bajan de 26 a 1).

## 4. Verificación

- `pytest`: **120 passed** (109 previos + 11 nuevos), 1 warning (externo, menor).
- `ruff check .`: limpio.
- `npm run build` (frontend): compila, mismas rutas y tamaños esperados.
- No ejecutado: mypy (no estaba en verde antes de esta rama y no es gate actual),
  tests E2E contra IBKR real (requieren TWS/Gateway activo).

## 5. Decisiones y supuestos

- No toqué umbrales de estrategia ni gates de promoción: cualquier cambio ahí debe
  salir de evidencia del Lab, no de una auditoría de código.
- Umbrales del filtro de calidad deliberadamente conservadores (ratio 4x no dispara
  con gaps reales de small caps de ±50%±100%; solo splits/ticks rotos). Mejor pocos
  falsos positivos: cada rechazo queda auditado y es revisable.
- No añadí el filtro al discovery lab en esta pasada para mantener el diff revisable;
  el scanner era el punto de mayor exposición (alimenta señales operativas).
- El directorio untracked `research/audit_bridge/requests/TRADEO-AUDIT-...` pertenece
  al flujo de auditoría diaria de otro proceso: intacto y sin commitear.

## 6. Roadmap recomendado (orden de valor/esfuerzo)

1. Aplicar `assess_ohlcv_quality` también en `PatternDiscoveryLabAgent` y
  `PatternEntryScanner` (mismo módulo, ~20 líneas por sitio + tests).
2. Alerting del watchdog: un webhook/notificación cuando `repair()` actúe o un job
  falle N veces seguidas (hoy solo queda en logs).
3. Backup de PostgreSQL con `pg_dump` en `ops/scripts/backup_tradeo.sh` (hoy el tar
  no captura el volumen de la BD).
4. Sustituir los `except Exception` genéricos por jerarquía de excepciones propia,
  empezando por data provider y broker (donde más duele un error enmascarado).
5. Ejecutar llamadas IBKR en threadpool (`run_in_executor`) para no bloquear FastAPI.
6. Frontend: configurar ESLint, trocear `page.tsx` por módulo, vista de backtests.
7. mypy en verde e incorporarlo al CI como segundo gate.

? Fable5

```

Full Document: `docs/post\_audit\_r1\_r6\_tracking\_2026\_06\_10.md`

`docs/post\_audit\_r1\_r6\_tracking\_2026\_06\_10.md` ? 43 lines, 5339 bytes, sha256 prefix `07fd2778f9731876`

Heading summary: Post-Audit Remediation Report - 2026-06-10; Corrections Applied; Problems Found While Fixing; Remaining Blockers; Verification

Post-Audit Remediation Report - 2026-06-10

Source: post-implementation audit dated 2026-06-10.

Verdict after remediation: live/production remains blocked by design until real IBKR paper fills are reconciled and a Director audit package passes. The code now blocks the false-positive paths identified by the audit instead of treating closed internal trades, shadow observations or dashboard summaries as proof of edge.

Corrections Applied

Audit item	Exact correction	Main files
R1 - `ibkr\_paper\_fill` inferred from closed status	Removed closed-status promotion as proof of fill. `ibkr\_paper\_order` becomes `ibkr\_paper\_fill` only with real fill provenance: `broker\_execution` or `broker\_statement\_import`, fill/execution id or hash, broker timestamp and commission. Simulated and shadow closes never count as strong fills.	`backend/tradeo/services/evidence.py`; `research/audit\_bridge/export\_audit\_package.py`; `backend/tradeo/services/director\_review\_gate.py`; `backend/tradeo/services/module\_dashboard.py`
R1 - reconstructed audit fills	`paper\_trades.csv` and `ib\_fills.csv` export only broker-provenance fills. The exporter no longer reconstructs IBKR fills from a closed paper trade record. Fill rows now derive hashes from broker fill/execution metadata and mark the provenance source.	`research/audit\_bridge/export\_audit\_package.py`; `backend/tradeo/tests/test\_audit\_hardening.py`
R2 - incomplete production gate	`DirectorProductionGate` now requires scientific contracts in addition to 30 fills: nested discovery replay implemented/passed, Director audit passed, no active blockers, event ledger hash, evidence packet reference/hash, reconciled cost/slippage/fill provenance, non-degrading drift, `edge\_claim=NO\_DEMOSTRADO`, and registry hash-chain metadata.	`backend/tradeo/services/director\_review\_gate.py`; `backend/tradeo/tests/test\_director\_review\_gate.py`; `backend/tradeo/tests/test\_research\_lab\_fox\_lifecycle.py`
R3 - evidence stored only in JSON	Added typed `Trade.evidence\_type` and `Trade.evidence\_quality` columns plus init-time backfill from `metadata\_json`. Runtime helpers merge typed columns with legacy metadata for compatibility.	`backend/tradeo/db/models.py`; `backend/tradeo/db/init\_db.py`; write paths in `ibkr\_broker.py`, `paper\_broker.py`, `lab\_paper\_observations.py`
R4 - validator/gate semantics	Validator output now separates `schema\_valid` / `package\_valid`, `director\_gate\_status`, and `promotion\_allowed`. A schema-valid package can be explicitly blocked for promotion. Anti-lookahead columns and scientific experiment columns are part of the expected export contract.	`research/audit\_bridge/validate\_audit\_package.py`; `research/audit\_bridge/director\_gate.py`; `research/audit\_bridge/export\_audit\_package.py`
R5 - global experiment registry too soft	Registry writes are now temp+rename atomic, existing registries are backed up, each write stores a canonical registry hash, previous hash and run manifest hash, and candidates receive registry hash-chain metadata.	`backend/tradeo/research/global\_experiment\_registry.py`; `backend/tradeo/tests/test\_pattern\_discovery\_lab.py`
R6 - dashboard partial scope unclear	Dashboard overview now exposes `data\_scope`, `query\_limit`, `summary\_limit`, `pnl\_basis`, `director\_source=false`, and a scope note. Frontend shows the limited scope beside fill KPIs.	`backend/tradeo/services/module\_dashboard.py`; `frontend/app/page.tsx`; `backend/tradeo/tests/test\_module\_dashboard.py`

Problems Found While Fixing

Problem	Resolution
The backend evidence helper was fixed by agents, but the audit exporter still had its own closed-status to fill inference.	Aligned `export\_audit\_package.py` with the stricter provenance contract.
`active\_blockers=[]` could become the literal CSV text `[ ]` and be interpreted as a blocker.	Export now writes an empty string when there are no blockers; Director gate ignores empty list sentinels.
Legacy tests encoded the old assumption that closed paper trades were fills.	Updated tests to require broker provenance and added a regression test rejecting closed paper trades without broker fill metadata.
Dashboard fill counts could still look like full-history evidence.	Dashboard now labels the scope as latest-query summary and declares it is not a Director source.

Remaining Blockers

- Real edge is still `NO\_DEMOSTRADO`.
- Production remains blocked until real IBKR paper executions are reconciled into trade metadata or imported broker statements with fill IDs/hashes, broker timestamps, commissions, slippage/cost provenance and audit package hashes.
- A process still needs to persist the real Director audit result and evidence packet metadata into `pattern.metrics\_json` before `DirectorProductionGate.approve\_pattern()` can ever pass.
- Existing audit packages may remain schema-valid but promotion-blocked until they include the new scientific contract fields, anti-lookahead values and broker-provenance fills.

Verification

- Focused backend remediation tests passed for audit hardening, Director gates and dashboard scope.
- Ruff passed on the modified audit/evidence/gate files before final full verification.
- Full-suite verification is recorded in the commit summary for this branch.

Full Document: `docs/research/quant\_validation\_core\_2026\_06\_10.md`

`docs/research/quant\_validation\_core\_2026\_06\_10.md` ? 125 lines, 6498 bytes, sha256 prefix `c97ac98bc54335b5`

Heading summary: Núcleo de validación cuantitativa (Fase 1-2 del informe de mejora); Qué se implementó; 1. `tradeo/research/quant\_validation.py` (nuevo); 2. Pseudo-replicación / n_{eff} (P0-1); 3. Multiplicidad a nivel de run (P0-2); 4. Scheduler post-cierre (P0-8); 5. Vela viva en el matcher (P0-3); 6. Umbral por patrón (P0-4); 7. Idempotencia de señal (P0-3/\$4.1); Decisiones y desviaciones respecto al informe; Pendientes (orden sugerido); Aceptación

Núcleo de validación cuantitativa (Fase 1-2 del informe de mejora)

Fecha: 2026-06-10 · Rama: `feature/quant-validation-core`
Origen: `INFORME\_MEJORA\_TRADEO.md` (informe externo de revisión cuantitativa) +
módulo entregado `quant\_validation.py`.

Qué se implementó

1. `tradeo/research/quant\_validation.py` (nuevo)

Módulo autocontenido (numpy + stdlib) con el pipeline de inferencia selectiva: `select\_nonoverlapping\_events`, `average\_uniqueness\_weights` (n_eff, López de Prado), `triple\_barrier\_outcome` (fills pesimistas: entrada en open(t+1), gaps por stop al OPEN, gaps por target al TARGET, both-touch ? stop, time stop, MFE/MAE en R), `stationary\_bootstrap\_ci`, `newey\_west\_tstat`, `permutation\_pvalue`, `benjamini\_hochberg`, `expected\_max\_sharpe` / `probabilistic\_sharpe\_ratio` / `deflated\_sharpe\_ratio`, `purged\_walk\_forward`, `pbo\_cscv`, `cusum\_drift`, `summarize\_pattern\_validation`.

Golden tests en `tradeo/tests/test\_quant\_validation.py` (los 6 casos del contrato de simulación \$6 del informe, propiedad de no-solapamiento, n_eff, BH-FDR, monotonía del DSR, no-leakage del walk-forward purgado, CUSUM).

2. Pseudo-replicación / n_eff (P0-1)

`ClusterResearchEngine.quant\_validation\_metrics()` : por candidato, los spans de outcome se colocan en un eje de días de calendario común, se deduplica por símbolo (no puedes estar en dos trades solapados del mismo patrón) y los supervivientes reciben pesos de unicidad media. Se persiste en `metrics.quant\_validation` (n_raw / n_unique / n_eff, expectancy y PF ponderados, IC95 stationary bootstrap, t Newey-West, skew/kurtosis) y en `metrics.effective\_sample\_count`.

Gate duro en `ValidationGate`: `n\_eff >= TRADEO\_DISCOVERY\_MIN\_EFFECTIVE\_SAMPLES` (60 por defecto). El conteo bruto (`discovery\_min\_samples`) se mantiene.

3. Multiplicidad a nivel de run (P0-2)

`PatternDiscoveryLabAgent.\_apply\_run\_level\_inference()` :

- ****BH-FDR**** (`q = TRADEO\_DISCOVERY\_FDR\_Q = 0.05`) sobre los `null\_p\_value` (permutación estratificada ya existente) de TODOS los clusters del run, aceptados y rechazados. `fdr\_passed=false` ? rechazo duro en el gate.
- ****DSR de familia****: `deflated\_sharpe\_ratio` con `n = n\_eff`, `N\_trials = global\_trial\_count` previo del `GlobalExperimentRegistry` + los trials de este run (max `real\_variant\_count` por window-size), y `sr\_std` estimada entre los Sharpe de los candidatos del run. `lab\_candidate` exige `dsr\_family >= TRADEO\_DISCOVERY\_MIN\_DSR (0.95)` ****y**** IC95 inferior > 0 del expectancy ponderado; si falla, se degrada a `lab\_watchlist` (no se rechaza: evidencia débil ? evidencia en contra).

Esto complementa (no sustituye) el `adjusted\_p\_value` Bonferroni-like, los proxies WRC/SPA y el DSR per-cluster que ya existían.

4. Scheduler post-cierre (P0-8)

`TRADEO\_DISCOVERY\_SCAN\_MINUTES` por defecto 90 ? ****1440****. En el worker, `>= 1440` cambia el trigger a cron post-cierre
(`TRADEO\_DISCOVERY\_POST\_CLOSE\_HOUR\_UTC=22:15`, lun-vie). Valores < 1440 conservan el comportamiento de intervalo (escape hatch).

5. Vela viva en el matcher (P0-3)

`NovelPatternMatcher.\_drop\_incomplete\_daily\_bar()` : con
`TRADEO\_DISCOVERY\_MATCH\_COMPLETE\_BARS\_ONLY=true` (default), antes de las 16:00 NY se descarta la barra diaria de hoy. La ventana operativa en sesión termina en la barra de ayer ? misma definición de barra que usó Research.

6. Umbral por patrón (P0-4)

Research persiste `match\_tau\_similarity` (percentil
`TRADEO\_DISCOVERY\_MATCH\_TAU\_PERCENTILE=92.5` de la distancia intra-cluster al centroide, mapeada con la MSMA fórmula de similitud del matcher). El matcher usa `max(suelo\_global, tau\_patron)`; `TRADEO\_DISCOVERY\_MATCH\_SIMILARITY\_THRESHOLD` pasa a ser solo suelo. Cada match registra `similarity\_threshold\_used`.

7. Idempotencia de señal (P0-3/\$4.1)

`PatternEntryScanner.\_signal\_idempotency\_key` =
`module|pattern\_id|symbol|timeframe|entry\_variant\_id|window\_end(bar\_ts)`.
Se persiste en `Signal.metadata\_json.signal\_idempotency\_key` y se comprueba antes de crear señal (camino normal y near-miss shadow). Un re-scan de la misma barra o un reinicio del worker no duplica señales (AuditLog
`entry\_match\_skipped\_idempotent`).

Decisiones y desviaciones respecto al informe

- El repo ya tenía mucho de la Fase 1 (null baselines estratificados, purged CV combinatorio, cost stress como gate, WRC/SPA proxies, DSR per-cluster). Se integró el módulo ****encima**** de eso, sin sustituir métricas existentes: los números nuevos viven en `quant\_validation.\*`, `fdr.\*` y `dsr\_family.\*`.
- El etiquetador de outcomes de discovery (`WindowSampler.\_path\_outcome`, entrada al cierre de la ventana) NO se reemplazó en este cambio: cambiar la definición de entrada invalida la comparabilidad con todos los ledgers y patrones existentes. `triple\_barrier\_outcome` queda como contrato canónico (\$6) listo para ShadowTracker/backtester y para una migración controlada del etiquetador (pendiente, ver abajo).
- RR levels: se mantiene la rejilla actual porque cada nivel ya se carga como


```
trial en `real_variant_count` ? el DSR de familia la penaliza. Reducir a
2 niveles a priori (`2.5,4.0`) es un cambio de un parámetro en `.env` cuando
Asier lo decida.
- El ratio de ambigüedad del matcher (margen vs 2º mejor patrón) no se
implementó: las distancias entre patrones con scalers distintos no son
directamente comparables; requiere diseño propio.
```

```
## Pendientes (orden sugerido)
```

1. Migrar el etiquetador de Research a `triple\_barrier\_outcome` (entrada en `open(t+1)`) con re-run completo y comparación antes/después.
2. ShadowTracker + implementation shortfall (\$4.6) reutilizando `triple\_barrier\_outcome` como única implementación.
3. Reconciliación DB?IBKR con kill switch automático (\$4.5).
4. DirectorReviewGate secuencial (posterior bayesiano + SPRT + KS) (\$4.7).
5. PatternHealthMonitor con `cusum\_drift` sobre R por trade en producción (\$4.8) ? la función ya está disponible y testeada.
6. Universo point-in-time / snapshots mensuales (P0-5) y `survivorship\_biased=true` en el ledger mientras tanto.
7. PBO/CSCV (`pbo\_cscv`) en el ciclo del ImprovementAgent (\$5).

```
## Aceptación
```

- `pytest`: 161 tests (39 nuevos) en verde; `ruff` limpio.
- Tras desplegar: el nº de `lab\_candidate` por run debe `**caer**` (si no cae, revisar que ``_apply_run_level_inference`` corre antes del gate). Cada candidato lleva ``fdr_*``, ``dsr_family*`` y ``quant_validation`` en su digest.

Full Document: `docs/research/research\_hardening\_p1\_p2\_2026\_06\_10.md`

`docs/research/research\_hardening\_p1\_p2\_2026\_06\_10.md` ? 39 lines, 1707 bytes, sha256 prefix `289cf4552de67999`

Heading summary: Research Hardening P1/P2 - 2026-06-10; Invariants; Hypothesis Package; Global Experiment Registry; Nested Discovery Replay

```
# Research Hardening P1/P2 - 2026-06-10
```

```
## Invariants
```

- Full-sample descriptive metrics use the `descriptive\_all\_\*` prefix.
- `descriptive\_all\_\*` metrics are not score inputs for `lab\_priority\_score`, `promotion\_score`, registry-adjusted score, or best-pattern selection.
- Legacy `expectancy\_r`, `profit\_factor`, `win\_rate`, `avg\_mfe\_r`, and `avg\_mae\_r` now mirror train/in-sample scope for compatibility.
- Candidate ranking uses `lab\_priority\_score`, which is scoped to train, OOS, walk-forward, purged CV, bootstrap reality proxy, replay, adversarial, invariance, and teacher evidence.
- WRC/SPA-like values are displayed as `bootstrap\_reality\_proxy` unless `reality\_check\_formal\_test=true`.

```
## Hypothesis Package
```

Research Director emits `research\_hypothesis\_package` from an immutable `HypothesisPackage` object. The serialized package includes:

- `selection\_split`
- `fit\_scope`
- `train\_metrics`
- `out\_of\_sample\_metrics`
- `walk\_forward\_metrics`
- `global\_trial\_count`
- `event\_ledger\_hash`
- `kill\_conditions`
- `family\_id`
- `variant\_id`
- `edge\_claim=NO\_DEMOSTRADO`
- `nested\_discovery\_replay`

```
## Global Experiment Registry
```

`reports/research/global\_experiment\_registry.json` accumulates discovery experiments by `run\_id`, `pattern\_key`, and `variant\_id`. Re-registering the same run/pattern/variant is idempotent and does not inflate `global\_trial\_count`.

```
## Nested Discovery Replay
```

Full nested discovery replay is not implemented yet. Finalists receive a blocking contract:

- `nested\_discovery\_replay.status=blocked\_contract`
- `nested\_discovery\_replay.blocking=true`
- edge claim remains `NO\_DEMOSTRADO`
- paper/live promotion remains blocked until replay is implemented and recorded

Full Document: `docs/research/director\_pattern\_search\_hardening.md`

`docs/research/director\_pattern\_search\_hardening.md` ? 212 lines, 8064 bytes, sha256 prefix `6bb409f2fb33577d`

Heading summary: Director Research Pattern Search Hardening; Objetivo; 1. Walk-Forward Rolling Con Embargo; 2. Null/Baseline Estratificado, Bootstrap Y Overfit; 3. Lifecycle Real De Confirmacion; 4. Familias, Variantes Y Drift; 5. Multi-Timeframe Y Relative Strength; 6. Stress Gates De Costes Y Slippage; 7. Event Ledger Auditable

```
# Director Research Pattern Search Hardening
```

Fecha: 2026-06-09

Objetivo

Endurecer la búsqueda de patrones de Research para que el Director reciba evidencia auditable, reproducible y resistente a overfitting antes de permitir cualquier avance hacia Lab/produccion.

1. Walk-Forward Rolling Con Embargo

Estado: implementado.

Cambios:

- Cada patron descubierto conserva el fit train-only ya existente para scaler/KMeans/R:R.
- Ademas calcula folds walk-forward cronologicos dentro del cluster.
- Cada fold elige `best\_rr` solo con train del fold.
- La validacion del fold empieza despues de `discovery\_walk\_forward\_embargo\_samples`.
- Se publican `walk\_forward\_folds`, `walk\_forward\_positive\_fold\_rate`, `walk\_forward\_avg\_expectancy\_r`, `walk\_forward\_min\_expectancy\_r` y `walk\_forward\_pooled`.
- `ValidationGate` rechaza candidatos con folds suficientes si `walk\_forward\_positive\_fold\_rate < discovery\_min\_walk\_forward\_positive\_rate`.

Lectura para Director:

- `walk\_forward\_positive\_fold\_rate` bajo indica edge concentrado en pocos periodos.
- `walk\_forward\_min\_expectancy\_r` negativo indica al menos un regimen donde el patron falla.
- Candidatos sin folds suficientes no se promocionan por esta evidencia; quedan con warning.

Checklist de auditoria:

- Confirmar que `validation\_method` es `train\_fit\_forward\_holdout\_walk\_forward\_embargo`.
- Revisar que fold train/validation no se solapan.
- Revisar `embargo\_samples`.
- Comparar OOS agregado contra folds individuales.

2. Null/Baseline Estratificado, Bootstrap Y Overfit

Estado: implementado.

Cambios:

- El null baseline ya no compara contra una poblacion cruda.
- Cada draw preserva estratos de `year`, regimen de volatilidad y regimen de tendencia.
- Se publican `null\_method`, `null\_strata\_count`, `null\_expectancy\_r`, `expectancy\_lift\_r`, `null\_p\_value` y `adjusted\_p\_value`.
- Se calcula bootstrap determinista sobre resultados train del patron:
 - `expectancy\_ci\_low`
 - `expectancy\_ci\_high`
 - `profit\_factor\_ci\_low`
- Se calcula `overfit\_score` usando gap entre train y walk-forward junto con tasa de folds positivos.
- `ValidationGate` rechaza si:
 - `expectancy\_ci\_low < discovery\_min\_expectancy\_ci\_low`
 - `overfit\_score > discovery\_max\_overfit\_score`

Lectura para Director:

- `expectancy\_lift\_r` mide cuanto gana el patron frente a una entrada aleatoria comparable por regimen.
- `expectancy\_ci\_low <= 0` indica que el edge no es robusto en bootstrap.
- `overfit\_score` cerca de 1 indica fuerte degradacion fuera de train o folds inconsistentes.

Checklist de auditoria:

- Revisar que `null\_method` sea `stratified\_regime\_bootstrap`.
- Confirmar que `adjusted\_p\_value` incluye penalizacion por numero de variantes evaluadas.
- Exigir CI positivo antes de cualquier escalado serio.
- Tratar `overfit\_score` alto como veto salvo explicacion de regimen.

3. Lifecycle Real De Confirmacion

Estado: implementado.

Cambios:

- Se anaden estados de patron:
 - `needs\_confirmation`
 - `confirmed\_candidate`
 - `failed\_confirmation`
- `discovered\_patterns` guarda columnas dedicadas:
 - `confirmation\_status`
 - `confirmation\_priority\_score`
 - `confirmation\_reason`
 - `confirmation\_next\_action`
 - `confirmation\_attempts`
- Los candidatos fuertes pero underpowered ya no son solo `rejected` con metadata JSON.
- Si `ValidationGate` recomienda confirmacion, `NovelPatternRegistry` persiste el patron como `needs\_confirmation`.
- `/api/research/confirmation-queue` filtra por columna DB y ordena por prioridad.

Lectura para Director:

- `needs\_confirmation` no es aprobacion ni promocion.
- Es una cola formal para re-run ampliado con mismo setup/side/window antes de volver a evaluar.
- `confirmation\_priority\_score` ayuda a ordenar coste computacional vs promesa estadistica.

Checklist de auditoria:

- Revisar que ningun `needs\_confirmation` pueda alimentar Fox/produccion.
- Exigir nuevo run ampliado antes de cambiar a `confirmed\_candidate`.
- Si el re-run falla, marcar `failed\_confirmation` y conservar evidencia.

4. Familias, Variantes Y Drift

Estado: implementado.

Cambios:

- `NovelPatternRegistry` mantiene:
 - `pattern\_family\_key`
 - `canonical\_pattern\_key`
 - `variant\_key`
 - `variant\_count`
 - `drift\_status`
 - `drift\_score`
- Patrones con centroides similares se fusionan en una familia canonica.
- Cada nuevo candidato similar queda registrado como variante del canonico.
- Si la nueva evidencia baja materialmente la expectancy frente al canonico, se marca `degrading`.
- Si mejora materialmente, se marca `improving`; si no, `stable`.

Lectura para Director:

- Una familia acumula evidencia de multiples runs sin inflar el numero de patrones.
- `variant\_count` alto con `stable/improving` aumenta confianza.
- `degrading` indica que nuevos datos erosionan el edge y debe bloquear escalado.

Checklist de auditoria:

- No contar variantes similares como patrones independientes.
- Revisar familias con `drift\_status=degrading`.
- Confirmar que el patron canonico conserva lineage de variantes.

5. Multi-Timeframe Y Relative Strength

Estado: implementado.

Cambios:

- El embedding incorpora contexto weekly calculado desde la ventana:
 - `weekly\_return`
 - `weekly\_trend`
- Discovery intenta descargar SPY y QQQ una vez por run.
- El embedding incorpora relative strength:
 - `relative\_strength\_spy`
 - `relative\_strength\_qqq`
 - `benchmark\_alignment`
- El matcher actual tambien calcula SPY/QQQ para poder comparar patrones nuevos.
- Patrones antiguos siguen funcionando por compatibilidad de prefijo en el matcher.

Lectura para Director:

- Un patron con edge solo cuando la fuerza relativa es positiva puede requerir filtro operativo.
- `benchmark\_alignment=-1` indica setup contrarian vs mercado; revisar por separado.
- Relative strength ayuda a separar setup local de simple beta de mercado.

Checklist de auditoria:

- Confirmar si el edge depende de SPY/QQQ.
- Separar patrones momentum de patrones contrarian.
- Revisar familias con mismo setup pero distinto contexto benchmark.

6. Stress Gates De Costes Y Slippage

Estado: implementado.

Cambios:

- `RewardRiskAnalyzer` soporta `cost\_multiplier`.
- Research calcula `cost\_stress` para multiplicadores configurados, por defecto:
 - `1x`
 - `2x`
 - `3x`
- `cost\_stress\_passed` exige expectancy positiva y $PF \geq 1$ en el multiplicador requerido.
- `ValidationGate` rechaza si el edge no sobrevive `discovery\_required\_cost\_stress\_multiplier`, por defecto x2.

Lectura para Director:

- Si el patron falla con coste x2, el edge es demasiado fragil para promocion.
- `cost\_stress` permite ver si el patron soporta spreads/slippage peores en small/mid caps.
- El coste base sigue viniendo de proxy de rango/liquidez por ventana.

Checklist de auditoria:

- Revisar `avg\_execution\_cost\_r`.
- Exigir `cost\_stress\_passed=true` antes de Lab candidate serio.
- Comparar expectancy neta 1x vs 2x vs 3x.

7. Event Ledger Auditable

Estado: implementado.

Cambios:

- El agente genera el ledger completo por candidato en los runs de discovery.
- Cada ledger se persiste como artefacto comprimido `.json.gz` en:
 - `reports/research/event\_ledgers/run\_<run\_id>/`
- El digest del patron conserva:

- `event\_ledger\_path`
- `event\_ledger\_sha256`
- `event\_ledger\_count`
- `event\_ledger\_compressed\_bytes`
- `event\_ledger\_uncompressed\_bytes`
- El ledger raw se retira de `metrics\_json` tras persistirlo para no inflar DB/reportes.
- Queda una preview compacta de los primeros eventos en `event\_ledger\_preview`.

Lectura para Director:

- El hash SHA-256 valida el contenido canonico sin comprimir.
- Si el patron se reevalua, se puede comparar ledger previo vs ledger nuevo por path/hash/count.
- El reporte de discovery ya muestra ruta y hash junto al patron.

Checklist de auditoria:

- Abrir el `.json.gz` del patron revisado.
- Verificar que `event\_count` coincide con `event\_ledger\_count`.
- Validar que el hash del JSON descomprimido coincide con `event\_ledger\_sha256`.
- Revisar dispersion por simbolo, fecha, split y resultado R antes de aprobar promocion.

Full Document: `docs/research/wave4\_rediscovery\_runbook.md`

`docs/research/wave4\_rediscovery\_runbook.md` ? 106 lines, 4280 bytes, sha256 prefix `73216efae13fc5d`

Heading summary: Wave4-C Rediscovery Runbook (2026-06-11); 0. Preconditions; 1. Audit (dry-run, safe, read-only); 2. Flag laggards (optional, writes `metrics\_json.rediscovery\_readiness`); 3. Execute real rediscovery (the only step that populates metadata); or: make discover-patterns; 4. Verify success; 5. Rollback / safety

Wave4-C Rediscovery Runbook (2026-06-11)

How to repopulate Wave4 metadata on legacy discovered patterns: embedding contract (`feature\_parity\_contract`), cluster signature + concentration metadata (`cluster\_signature` / `concentration\_checks`), and benchmark-regime calibration buckets (`regime\_profile.benchmark\_regime\_outcomes`).

Honesty contract: the readiness tool below only *audits and flags*. It never populates metadata. Patterns only become complete after a real discovery run re-upserts them via `NovelPatternRegistry.upsert\_candidate`, which overwrites `metrics\_json` with engine-fresh metrics.

0. Preconditions

- Backend stack up (`make up`) and DB reachable.
- No live/paper trading dependency: discovery is research-only, but avoid running it while a director cycle is mid-flight to keep run accounting clean.
- Recent market data cache present (discovery re-downloads otherwise).

1. Audit (dry-run, safe, read-only)

```
docker compose run --rm backend \
python -m tradeo.research.rediscovery_readiness \
--manifest-out /app/artifacts/rediscovery_manifest_$(date +%Y%m%d).json
```

- Default `--limit 2000`; if the manifest reports `"truncated": true`, rerun with a higher `--limit`.
- The manifest lists every `pattern\_key` needing rediscovery, with per-field `missing`/`stale` reasons and a `determinism.content\_hash` for comparison across runs.

2. Flag laggards (optional, writes `metrics\_json.rediscovery\_readiness`)

```
docker compose run --rm backend \
python -m tradeo.research.rediscovery_readiness --apply-flags \
--manifest-out /app/artifacts/rediscovery_manifest_flagged.json
```

- Adds a `rediscovery\_readiness` block (`needs\_rediscovery`, `missing`, `stale`, `checker\_version`, `flagged\_at`) inside each laggard's `metrics\_json`. No schema migration needed (JSONB is extensible).
- Re-flagging after metadata arrives flips the block to `needs\_rediscovery: false`; it is never deleted, preserving audit trail.

3. Execute real rediscovery (the only step that populates metadata)

Discovery upserts by `pattern\_key` family/centroid similarity, so re-running discovery over the same universe/timeframes refreshes legacy rows in place with the new metadata.

Bounded run (recommended first):

```
curl -s -X POST http://localhost:8000/api/research/run-discovery \
-H 'Content-Type: application/json' \
-d '{"limit": 40, "max_total_windows": 4000}' | jq .
```

or: make discover-patterns

For full coverage use the existing loop with explicit bounds
(`scripts/research\_forever.sh` honors `DISCOVERY\_LIMIT`, `MAX\_TOTAL\_WINDOWS`,
`MAX\_WINDOWS\_PER\_SYMBOL`). Do **not** launch unbounded loops as part of this
runbook; run discrete bounded passes and re-audit between them.

Caveat: a legacy pattern whose cluster no longer re-emerges from current data
will **not** be refreshed by rediscovery. Decide per pattern: retire it
(status/promotion demotion) or accept it stays flagged. Do not hand-edit its
metadata to look populated.

4. Verify success

```
docker compose run --rm backend \
python -m tradeo.research.rediscovery_readiness \
--manifest-out /app/artifacts/rediscovery_manifest_after.json
```

Success criteria:

1. `counts.needs\_rediscovery` drops to 0 (or only to patterns explicitly
accepted as retired/non-reemerging, each with a written disposition).
2. `counts.missing\_by\_field` and `counts.stale\_by\_field` are all 0 for
non-retired patterns.
3. Embedding contract on refreshed rows equals
`PatternEmbeddingEngine.CONTRACT\_ID`
(`tradeo.pattern\_embedding.v2.legacy\_prefix\_stable`).
4. Discovery report carries a `determinism.content\_hash` block (Wave4-B) and
targeted tests stay green:

```
docker compose run --rm backend pytest \
tradeo/tests/test_rediscovery_readiness.py \
tradeo/tests/test_novel_pattern_registry.py \
tradeo/tests/test_discovery_determinism.py -q
```

5. Rollback / safety

- The readiness tool itself only writes the `rediscovery\_readiness` JSON
block; nothing else. Worst case it can be ignored by all consumers.
- Discovery upserts are the standard production path (same as the director
loop) ? no special rollback beyond normal DB backups.
- Never run this against live trading sessions' hot path; it is DB-only.

Full Document: `docs/remediation/agent\_a\_data\_universe\_repro\_2026\_06\_10.md`

`docs/remediation/agent\_a\_data\_universe\_repro\_2026\_06\_10.md` ? 56 lines, 4090 bytes, sha256 prefix `39ddfa1d193d619d`

Heading summary: Agent A Fallback - Data, Universe, Reproducibility; Scope; External Report Sections Addressed; Files Changed;
Tests Run; Remaining Risks / Known Non-Compliance; Merge Notes

Agent A Fallback - Data, Universe, Reproducibility

Date: 2026-06-10

Branch/worktree: `agent-a-data-universe-repro-fallback` at `/home/vboxuser/tradeo-worktrees/agent-a-data-universe-repro-fallback`

Scope

Implemented bounded remediation for external report sections 3.1, 3.2, 3.8, 7, and related config. No live trading, no order routing, no secrets.

External Report Sections Addressed

- 3.1 Data layer: added deterministic local OHLCV cache with canonical CSV artifacts, sidecar metadata, `adjusted`, `what\_to\_show`,
`bar\_complete`, split-like jump heuristic, and per-run data manifest export.
- 3.2 Universe: added monthly forward snapshot service with eligibility filters, metadata/hash, explicit `point\_in\_time` and
`survivorship\_biased` flags.
- 3.8 Regimes: enriched regime profile with `regime\_count`, `dominant\_share`, and `regime\_specific`/`multi\_regime\_observed` research
gate metadata; event ledgers now include data lineage.
- 7 Reproducibility: discovery runs now attach a data manifest summary and universe snapshot metadata to run summaries and candidates.
- 8 Config: added explicit env/config knobs for market-data cache, adjusted feed metadata, universe snapshots, PIT availability, and
survivorship cap.

Files Changed

- `.env.example`
- `backend/tradeo/agents/pattern\_discovery\_lab\_agent.py`
- `backend/tradeo/core/config.py`
- `backend/tradeo/research/cluster\_research\_engine.py`
- `backend/tradeo/research/types.py`
- `backend/tradeo/research/validation\_gate.py`
- `backend/tradeo/research/window\_sampler.py`
- `backend/tradeo/services/data\_provider.py`
- `backend/tradeo/services/ibkr\_data\_provider.py`

```
- `backend/tradeo/services/provider_factory.py`  
- `backend/tradeo/services/universe_snapshot.py`  
- `backend/tradeo/tests/test_data_provider.py`  
- `backend/tradeo/tests/test_quant_validation_integration.py`  
- `docs/remediation/agent_a_data_universe_repro_2026_06_10.md`  
- `docs/remediation/tradeo_12_phase_compliance_matrix_2026_06_10.md`
```

Tests Run

```
- `python3 -m compileall tradeo/core/config.py tradeo/services/data_provider.py tradeo/services/provider_factory.py  
tradeo/services/ibkr_data_provider.py tradeo/services/universe_snapshot.py tradeo/research/window_sampler.py tradeo/research/types.py  
tradeo/research/cluster_research_engine.py tradeo/research/validation_gate.py tradeo/agents/pattern_discovery_lab_agent.py`  
- `docker run --rm -v "$PWD/backend/tradeo:/app/tradeo:ro" d407ca6eb516 pytest tradeo/tests/test_data_provider.py  
tradeo/tests/test_quant_validation_integration.py -q` -> 21 passed  
- `docker run --rm -v "$PWD/backend/tradeo:/app/tradeo:ro" d407ca6eb516 ruff check ...touched files...` -> passed
```

Note: host Python lacked `pytest` and `pandas`. Full Docker build failed only at `COPY data /app/data` because this isolated worktree has no `data/` directory. Tests were run in the successfully built dependency layer with current code mounted read-only.

Remaining Risks / Known Non-Compliance

- Cache is deterministic and manifest-backed, but not true incremental-by-date because the existing provider boundary accepts only `period`/`interval`; metadata marks `incremental\_fetch\_supported=false`.
- Artifact format is canonical CSV, not Parquet, to avoid adding heavy unpinned dependencies. The manifest makes the format explicit.
- IBKR `ADJUSTED\_LAST` is now configurable/defaulted for research data, but live verification against the gateway was not run.
- Historical point-in-time/delisting coverage is still unavailable. The system now honestly caps `lab\_candidate` to `lab\_watchlist` when discovery metadata declares `survivorship\_biased=true`.
- Regime profile is metadata/gating support, not a full SPY SMA200 tercile service yet.

Merge Notes

- This branch is isolated from `/home/vboxuser/tradeo` and does not revert other agents' work.
- Safe merge order: after any branches touching the same validation/config/provider files, review conflicts around `ValidationGate`, `PatternDiscoveryLabAgent`, and `.env.example`.
- No live trading settings were enabled; paper-first/read-only defaults remain intact.

Full Document:

`docs/remediation/agent\_b\_representation\_matching\_parity\_2026\_06\_10.md`

`docs/remediation/agent\_b\_representation\_matching\_parity\_2026\_06\_10.md` ? 47 lines, 2969 bytes, sha256 prefix
`e20da591f7b7a17b`

Heading summary: Agent B - Representation, Clustering, Matching Parity; Scope; External Report Sections Addressed; Files Changed; Tests Run; Remaining Risks / Known Non-Compliance; Merge Notes

Agent B - Representation, Clustering, Matching Parity

Date: 2026-06-10
Branch: `agent-b-representation-parity`
Worktree: `/home/vboxuser/tradeo-worktrees/agent-b-representation-parity`

Scope

Disjoint fallback scope for external report sections 3.3, 3.4, 4.1, 4.2 and 7.
No live trading behavior was enabled or changed.

External Report Sections Addressed

- 3.3 Representation: added an explicit `PatternEmbeddingEngine` contract and persisted it from Research and Lab so audit packages can prove both paths use the same embedding implementation.
- 3.4 Clustering: added medoid/signature metadata, intra-cluster similarity distribution and concentration diagnostics. Validation now rejects new candidates when the new concentration block says a cluster is dominated by one symbol or month.
- 4.1 Research/Lab parity: matcher scan results and match metrics now include the same feature parity contract used by Research.
- 4.2 Matcher threshold/ambiguity: matcher now computes second-best competitor diagnostics per symbol/timeframe/window and stores `ambiguity\_ratio`, margin and second-best pattern metadata for each match.
- 7 Reproducibility/audit: new metrics are deterministic, stored in JSON fields and covered by targeted tests.

Files Changed

```
- `backend/tradeo/research/pattern_embedding_engine.py`  
- `backend/tradeo/research/cluster_research_engine.py`  
- `backend/tradeo/research/novel_pattern_matcher.py`  
- `backend/tradeo/research/validation_gate.py`  
- `backend/tradeo/tests/test_pattern_discovery_lab.py`  
- `backend/tradeo/tests/test_pattern_entry_scanner.py`  
- `docs/remediation/agent_b_representation_matching_parity_2026_06_10.md`  
- `docs/remediation/tradeo_12_phase_compliance_matrix_2026_06_10.md`
```

Tests Run

```
- `./venv/bin/python -m pytest backend/tradeo/tests/test_pattern_discovery_lab.py backend/tradeo/tests/test_pattern_entry_scanner.py  
-q`  
- Result: 40 passed.
```

Remaining Risks / Known Non-Compliance

- Matrix Profile, DTW refinement and HDBSCAN were not added to avoid new heavy dependencies and broader behavior changes in a shared remediation window.
- Existing KMeans clustering remains; new medoid/signature/concentration metadata makes its output more auditable but does not replace the clustering algorithm.
- `ambiguity\_ratio` is recorded, not used as a hard block. A later calibration pass should decide whether a high second-best ratio

should downgrade or block entries.

- Historical patterns created before this patch will not have `feature\_parity\_contract`, `cluster\_signature`, `medoid` or `concentration\_checks` until rediscovered/re-registered.

Merge Notes

- The concentration gate is backward-compatible: it only fires when `metrics.concentration\_checks` exists and explicitly has `passed=False`.
- Matcher storage already persists arbitrary `metrics\_json`, so no DB migration is required.
- The matcher precomputes similarities once per symbol/timeframe/window and reuses them for match generation.

Full Document:

`docs/remediation/agent\_c\_outcomes\_costs\_backtester\_2026\_06\_10.md`

`docs/remediation/agent\_c\_outcomes\_costs\_backtester\_2026\_06\_10.md` ? 61 lines, 3723 bytes, sha256 prefix `87038d25b9f10468`

Heading summary: Agent C - Outcomes, Costs, Backtester, ImprovementAgent; Scope; External Report Sections Addressed; Files Changed; Tests Run; Remaining Risks / Known Non-Compliance; Merge Notes

Agent C - Outcomes, Costs, Backtester, ImprovementAgent

Date: 2026-06-10
Branch: `agent-c-outcomes-costs-backtester`
Base: `origin/main` at `9cc7ccb`

Scope

Implemented Agent C fallback remediation for outcomes, cost modeling, backtester parity, RR trial accounting defaults, and ImprovementAgent anti-overfitting guards.

No live trading was enabled. The change stays in Research/Lab/backtest code paths.

External Report Sections Addressed

- 3.5 Outcomes: Research RR simulation now delegates to canonical `triple\_barrier\_outcome`, including next-open entry, pessimistic both-touch handling, gap skip policy, time exits, MFE/MAE, and net R costs.
- 3.6 Validation/RR accounting: RR defaults reduced to the two a priori levels `2.5,4.0`; existing `real\_variant\_count`/`multiple\_testing\_trials` now counts the smaller grid by default.
- 5 ImprovementAgent: lab mutation cycles now record fixed trial budgets, block acceptance without CSCV/PBO evidence, require `PBO < 0.10`, and require a local parameter plateau check.
- 6 Backtester: simulated exits now reuse `triple\_barrier\_outcome` and the shared tiered round-trip cost model instead of a separate optimistic high/low loop.
- 8 Config: `.env.example` and `Settings` now expose Agent C defaults for RR levels and self-improvement anti-overfit thresholds.

Files Changed

- `.env.example`
- `backend/tradeo/core/config.py`
- `backend/tradeo/research/quant\_validation.py`
- `backend/tradeo/research/reward\_risk\_analyzer.py`
- `backend/tradeo/research/types.py`
- `backend/tradeo/research/window\_sampler.py`
- `backend/tradeo/services/backtester.py`
- `backend/tradeo/services/self\_improvement.py`
- `backend/tradeo/tests/test\_agent\_c\_remediation.py`
- `backend/tradeo/tests/test\_reward\_risk\_analyzer.py`
- `docs/remediation/agent\_c\_outcomes\_costs\_backtester\_2026\_06\_10.md`
- `docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`

Tests Run

From `backend/` in a local `.venv`:

- `.venv/bin/pytest -q tradeo/tests/test\_quant\_validation.py tradeo/tests/test\_reward\_risk\_analyzer.py tradeo/tests/test\_agent\_c\_remediation.py`
 - 36 passed
- `.venv/bin/pytest -q tradeo/tests/test\_pattern\_discovery\_lab.py tradeo/tests/test\_quant\_validation\_integration.py tradeo/tests/test\_autonomous\_research\_director.py`
 - 30 passed
- `.venv/bin/ruff check .`
 - passed
- `.venv/bin/pytest -q`
 - 165 passed, 1 warning from `eventkit` Python 3.14 deprecation

Remaining Risks / Known Non-Compliance

- `RewardRiskAnalyzer` still represents skipped entry gaps as `0.0R` for compatibility with existing aggregate arrays. The label now records `skipped`, but future work should propagate true non-trade counts through every metric.
- Research uses the canonical outcome through RR simulation, but `WindowSampler` still precomputes legacy descriptive outcome fields for compatibility.
- ImprovementAgent now blocks without PBO/plateau evidence, but it is still a grid search, not full Optuna nested optimization.
- PBO is computed from monthly realized backtest R buckets. A richer outer-fold/nested report should be added when the ImprovementAgent gets a full optimizer.
- ShadowTracker parity is not in this scope; this branch only covers Research RR simulation and the backtester.

Merge Notes

- This branch adds `ForwardOutcome.forward\_opens` with a default empty list, so existing serialized objects remain compatible.
- `TRADEO\_DISCOVERY\_RR\_LEVELS` default changes from six levels to `2.5,4.0`. If another branch assumes six levels, keep their explicit env override or reconcile docs.
- `SelfImprovementEngine` may accept fewer or zero lab candidates because PBO/plateau evidence is now mandatory. This is intentional anti-overfitting behavior.

Full Document:

`docs/remediation/agent\_d\_execution\_director\_monitoring\_2026\_06\_10.md`

`docs/remediation/agent\_d\_execution\_director\_monitoring\_2026\_06\_10.md` ? 55 lines, 3628 bytes, sha256 prefix `8d454ec384248c38`

Heading summary: Agent D - Execution, Director, Monitoring Remediation; Scope; External Report Sections Addressed; Files Changed; Tests Run; Remaining Risks / Known Non-Compliance; Merge Notes

Agent D - Execution, Director, Monitoring Remediation

Date: 2026-06-10
Branch: `remediation/agent-d-fallback`
Worktree: `/home/vboxuser/tradeo-worktrees/agent-d-fallback`

Scope

Agent D fallback scope for the 12-phase remediation: Execution Loop, Director, Monitoring, and Audit Package. No live trading was enabled.

External Report Sections Addressed

- 4.3: strengthened signal-time microstructure inputs by making liquidity/ATR/entry-gate outputs part of risk and signal snapshots; no live-bar/matcher behavior was loosened.
- 4.4: added ADV participation sizing cap and max open positions per pattern family.
- 4.5: preserved existing reconciliation auto-kill-switch path and kept tests green.
- 4.6: integrated implementation shortfall into PatternHealthMonitor, not only DirectorReviewGate.
- 4.7: made Director review use stricter configured sequential evidence minima: 25 effective paper fills, 8 symbols, 10 days by default; 10 fills remains only a local/test override trigger.
- 4.8: health monitor now runs CUSUM over both realized R and real-fill `slippage\_R`.
- 7: audit export now separates exported event count from verified independent sample count and tests reconstructability.

Files Changed

- `.env.example`
- `backend/tradeo/core/config.py`
- `backend/tradeo/modules/shared/entry\_scanner.py`
- `backend/tradeo/research/novel\_pattern\_matcher.py`
- `backend/tradeo/services/risk\_manager.py`
- `backend/tradeo/services/director\_review\_gate.py`
- `backend/tradeo/services/pattern\_health\_monitor.py`
- `research/audit\_bridge/export\_audit\_package.py`
- `backend/tradeo/tests/test\_risk.py`
- `backend/tradeo/tests/test\_director\_review\_gate.py`
- `backend/tradeo/tests/test\_research\_lab\_fox\_lifecycle.py`
- `backend/tradeo/tests/test\_audit\_hardening.py`

Tests Run

- `.venv/bin/ruff check backend/tradeo/services/risk_manager.py backend/tradeo/modules/shared/entry_scanner.py backend/tradeo/research/novel_pattern_matcher.py backend/tradeo/services/director_review_gate.py backend/tradeo/services/pattern_health_monitor.py backend/tradeo/tests/test_risk.py backend/tradeo/tests/test_director_review_gate.py backend/tradeo/tests/test_audit_hardening.py research/audit_bridge/export_audit_package.py`
- `.venv/bin/pytest -q backend/tradeo/tests/test\_risk.py backend/tradeo/tests/test\_director\_review\_gate.py backend/tradeo/tests/test\_audit\_hardening.py` -> 36 passed
- `.venv/bin/ruff check backend/tradeo/tests/test\_research\_lab\_fox\_lifecycle.py`
- `.venv/bin/pytest -q backend/tradeo/tests` -> 166 passed, 1 warning from `eventkit` on Python 3.14 deprecation

Remaining Risks / Known Non-Compliance

- `effective\_lab\_trades` is a conservative real-fill count proxy until per-trade uniqueness weights are persisted for paper fills.
- ADV cap uses `avg\_dollar\_volume / entry` from available features; real-time venue volume or float-adjusted ADV is not integrated.
- Pattern-family cap depends on matcher/signal metadata carrying `pattern\_family\_key` or `canonical\_pattern\_key`; legacy signals without those fields can only fall back to `pattern\_key`/`pattern\_id`.
- Health monitor shortfall CUSUM only uses trades with real fill provenance and fill prices; shadow/no-order observations remain excluded by design.
- Order state machine/reconciliation service existed before this patch; this patch did not add a new order-state enum or migrations.

Merge Notes

- Uses only backend Python and audit-bridge changes; no frontend or live broker configuration changes.
- Local `.venv` was created in this worktree for verification and is untracked.
- Defaults remain paper-first and live disabled.

Full Document: `docs/remediation/agent\_e\_data\_regime\_gap\_2026\_06\_11.md`

`docs/remediation/agent\_e\_data\_regime\_gap\_2026\_06\_11.md` ? 125 lines, 6701 bytes, sha256 prefix `2d6e8c7659cb259f`

Heading summary: Agent E - Data / Universe / Regime / Reproducibility Gap Closure; Scope; Sections Addressed; 3.8 Market regimes ? SPY SMA200 + volatility tercile service (new); 3.1 Data layer ? true incremental fetch (unblocked within existing signature); 3.2 / 7 Universe + reproducibility; Files Changed; Tests Run; Remaining Risks / Known Non-Compliance; Merge Notes

Agent E - Data / Universe / Regime / Reproducibility Gap Closure

Date: 2026-06-11
Branch/worktree: `feat/tradeo12-data-regime-gap-20260611` at `/home/vboxuser/tradeo-worktrees/data-regime-gap`
Base: local `main` @ `df418fb` (agents A-D already merged).

Scope

Single bounded package closing the remaining 3.1 / 3.2 / 3.8 / 7 gaps from the compliance matrix without touching execution/matcher beyond the minimum integration point. No live trading paths, no order routing, no secrets.

Sections Addressed

3.8 Market regimes ? SPY SMA200 + volatility tercile service (new)

- New ``backend/tradeo/services/market_regime.py``:
 - ``regime_table(df)``: per-bar benchmark labels. Trend = close vs strict SMA200 (full window required; young series report ``insufficient_history`` instead of a fake trend). Volatility = annualized stdev of 20d log returns, bucketed into terciles whose boundaries come exclusively from the trailing 252 values up to the ``previous`` bar (shifted), so labels are point-in-time and never change when future bars are appended (covered by a dedicated stability test).
 - ``regime_at(df, as_of)``: snapshot dict with ``trend_regime``, ``vol_regime``, ``regime_key`` (``benchmark_bull|mid_vol_tercile`` style), numeric values, tercile bounds, ``source_bar_hash`` lineage, and honest ``insufficient_history`` labels.
 - ``MarketRegimeService``: fetches the benchmark through the (cached) provider and degrades to an honest empty regime on provider failure (never blocks a scan).
- Minimal matcher integration (``novel_pattern_matcher.py``): one benchmark regime snapshot per ``match_current`` run, attached additively as ``regime["benchmark_regime"]`` on each match and ``benchmark_regime`` on the run result. The per-symbol ``regime_key`` consumed by throttles/entry scanner is untouched, so no behavior change in matching/gating; the SPY regime is audit/lineage metadata until calibrated as a gate.
- New settings: ``market_regime_benchmark_symbol`` (SPY), ``market_regime_sma_window`` (200), ``market_regime_vol_window`` (20), ``market_regime_vol_tercile_lookback`` (252).

3.1 Data layer ? true incremental fetch (unblocked within existing signature)

- ``CachedMarketDataProvider`` now performs an overlap-verified tail merge for daily artifacts instead of serving a stale disk cache forever:
 - If the cached last bar is older than ``min_gap_days``, it fetches only the missing tail (``gap+overlap``d period through the ``existing`` ``period/interval`` provider boundary ? no provider signature change needed).
 - The refetched overlap window (default 5 bars) must match cached bars (open/close, 1e-4 relative tolerance). Any mismatch ? e.g. dividend/split re-adjustment of an ``ADJUSTED_LAST`` feed ? forces an honest full refetch (``full_refetch_overlap_mismatch``).
 - Gaps beyond ``max_gap_days`` (45) skip the merge and full-refetch.
 - Metadata now records ``incremental_fetch_supported=true`` (daily), ``refresh_mode`` (``full_fetch`` / ``incremental_append`` / ``full_refetch_overlap_mismatch`` / ``full_refetch_gap_too_large``) and ``rows_appended``; the data manifest exposes ``refresh_mode``, ``incremental_fetch_supported`` and ``last_timestamp`` per entry.
- New settings: ``market_data_incremental_enabled`` (true), ``market_data_incremental_overlap_bars`` (5), ``market_data_incremental_min_gap_days`` (1), ``market_data_incremental_max_gap_days`` (45). Knobs are also constructor fields so tests/operators can override per instance.

3.2 / 7 Universe + reproducibility

- Universe snapshots now carry a deterministic ``content_hash`` (month, as-of, symbols, eligibility rules ? excludes ``built_at`` and absolute paths) so bit-for-bit snapshot identity can be asserted across rebuilds.
- Explicit ``delisting_data_available`` flag (settings-backed, default false) in snapshot metadata: PIT/delisting remains ``**honestly blocked**`` ? there is still no licensed delisting/PIT membership source; survivorship flags and the ``lab_watchlist`` cap from Agent A remain the operative mitigation.

Files Changed

- ``backend/tradeo/services/market_regime.py`` (new)
- ``backend/tradeo/services/data_provider.py``
- ``backend/tradeo/services/universe_snapshot.py``
- ``backend/tradeo/research/novel_pattern_matcher.py``
- ``backend/tradeo/core/config.py``
- ``.env.example``
- ``backend/tradeo/tests/test_market_regime.py`` (new, 11 tests)
- ``backend/tradeo/tests/test_data_provider.py`` (+5 tests)
- ``backend/tradeo/tests/test_pattern_entry_scanner.py`` (fetch-count assertion updated for the one extra SPY regime fetch per scan)
- ``docs/remediation/agent_e_data_regime_gap_2026_06_11.md``
- ``docs/remediation/tradeo_i2_phase_compliance_matrix_2026_06_10.md`` (Agent E rows)

Tests Run

- ``docker run --rm -v "$PWD/backend/tradeo:/app/tradeo:ro" tradeo-backend:latest pytest tradeo/tests -q`` ? ``**193 passed**`` (was 178 before this phase; +15 new, 1 updated).
- ``ruff check`` on all touched files ? passed.
- Host Python lacks pytest/pandas; all verification ran in the ``tradeo-backend:latest`` image with the worktree mounted read-only.

Remaining Risks / Known Non-Compliance

- ``**PIT/delisting (3.2): still blocked.**`` Requires a licensed vendor; flagged honestly via ``survivorship_biased`` + ``delisting_data_available=false``.
- The SPY regime snapshot is attached as audit metadata; it is ``not`` yet a

hard gate in ``_pattern_regime_fit``. Calibrating it as a gate needs labeled outcome history per regime bucket (deliberate, to avoid an uncalibrated behavior change in match scoring).

- Incremental merge assumes tz-consistent daily indexes (IBKR daily bars are date-indexed). Intraday intervals keep the previous cache-forever behavior.
- The regime fetch adds one benchmark request per matcher run (cached thereafter); on IBKR outage it degrades to ``insufficient_history`` rather than failing the scan.
- Incremental refresh changes runtime behavior: stale daily caches now refresh on first access per process. This is the intended fix (previously a disk cache was never refreshed), but operators can disable it via ``TRADEO_MARKET_DATA_INCREMENTAL_ENABLED=false``.

Merge Notes

- Conflict surface vs other agents: ``core/config.py`` (new settings block), ``env.example``, ``novel_pattern_matcher.py`` (3 small additive hunks), compliance matrix. No changes to validation gate, risk manager, broker or execution paths.
- ``test_pattern_entry_scanner.py`` change is a single assertion block update; if another branch touches the same test, keep both by summing expected fetch counts.
- No live-trading defaults changed; provider factory still IBKR-only with synthetic data forbidden.

Full Document:

``docs/remediation/agent_e2_intraday_incremental_cache_2026_06_11.md``

``docs/remediation/agent_e2_intraday_incremental_cache_2026_06_11.md`` ? 80 lines, 4207 bytes, sha256 prefix ``1824b8ede6e682c1``

Heading summary: Agent E2 - Intraday Incremental OHLCV Cache Refresh (gap 3.1 follow-up); Scope; Sections Addressed; 3.1 Intraday incremental cache refresh (was deferred); Files Changed; Tests Run; Remaining Risks / Honest Notes

Agent E2 - Intraday Incremental OHLCV Cache Refresh (gap 3.1 follow-up)

Date: 2026-06-11
Branch/worktree: ``main`` (direct, working tree) @ base ``8ba0348``.
Role: data/regime phase agent.

Scope

Closes the "intraday incremental cache refresh" item explicitly deferred in ``agent_e_data_regime_gap_2026_06_11.md`` (gap 3.1): until now only ``id`` artifacts got the overlap-verified tail append; every intraday interval (``1m/5m/15m/30m/1h``) was cache-forever, so an intraday CSV written once never refreshed again. No matcher/execution behavior touched.

Sections Addressed

3.1 Intraday incremental cache refresh (was deferred)

- ``CachedMarketDataProvider._maybe_refresh`` now also refreshes intraday intervals using the same overlap-verified tail append as daily:
 - Gap thresholds are interval-aware: minimum gap is bar-relative (``market_data_incremental_intraday_min_gap_bars``, default 2 bars) so a 5m cache refreshes on a 10-minute gap while a 1h cache waits 2 hours; maximum gap is wall-clock (``market_data_incremental_intraday_max_gap_days``, default 5 days) beyond which a full refetch is cheaper and safer than stitching.
 - The tail fetch period covers the gap plus a 3-day buffer so overlap bars survive weekends/holidays; any overlap mismatch still forces an honest ``full_refetch_overlap_mismatch``.
 - Master switch ``market_data_incremental_intraday_enabled`` (default true) on top of the existing ``market_data_incremental_enabled``; disabling it restores the previous cache-forever behavior exactly.
- Honest intraday bar completeness: ``_bar_complete_mask`` previously marked every intraday bar complete, including the in-progress one. It now marks an intraday bar complete only when ``bar_start + bar_width <= now (UTC)``; naive timestamps are treated as UTC (IBKR intraday bars arrive tz-aware).
- Complete-bars-only persistence: the cache CSV now stores only complete bars (serving already filtered them, so served data is unchanged for complete bars). This also fixes a latent daily defect: a partial bar written mid-day was cemented with ``bar_complete=false`` and stale OHLC values, was never replaced by the strictly-after-last-timestamp append, and only self-healed via a costly overlap-mismatch full refetch. Partial bars are no longer persisted at all.
- Metadata/manifest honesty: ``incremental_fetch_supported`` now reflects intraday support per interval and flag state; ``refresh_mode`` values are shared with the daily path (``incremental_append``, ``full_refetch_gap_too_large``, ``full_refetch_overlap_mismatch``).

Files Changed

- ``backend/tradeo/services/data_provider.py``
- ``backend/tradeo/core/config.py``
- ``backend/tradeo/tests/test_data_provider.py``
- ``env.example``
- ``docs/remediation/agent_e2_intraday_incremental_cache_2026_06_11.md``

Tests Run

- ``docker compose run --rm -v "$PWD/backend/tradeo:/app/tradeo:ro" backend pytest``

```
? 258 passed, 3 skipped (skips are the docs-traceability tests, absent from
the image by design). Baseline before this change collected 253 tests; the
5 new tests cover intraday append, fresh-cache skip, gap-too-large full
refetch, disabled-flag cache-forever, and in-progress-bar exclusion.
- `docker compose run --rm -v "$PWD/backend/tradeo:/app/tradeo:ro" backend ruff check ...`
? clean on the three touched Python files.
```

Remaining Risks / Honest Notes

- Intraday completeness assumes regular bar widths from the known interval map; unknown intervals (e.g. `1wk`) keep the permissive all-complete mask.
- RTH session boundaries are not modeled: the last RTH bar of the day is considered complete once its wall-clock width elapses, which is correct, but the gap measured across overnight/weekend periods is wall-clock, so the min-gap check effectively always passes overnight ? harmless (the append is a no-op when no new bars exist).
- Naive intraday timestamps are interpreted as UTC for gap/completeness math; IBKR intraday data is tz-aware so this path is defensive only.
- The SPY regime hard-gate calibration and rediscovery items from the final report remain open (they require labeled outcome history, out of scope for this phase).

Full Document: `docs/remediation/agent\_f\_execution\_state\_gap\_2026\_06\_11.md`

`docs/remediation/agent\_f\_execution\_state\_gap\_2026\_06\_11.md` ? 142 lines, 7479 bytes, sha256 prefix `e46ad84dd09f8b91`

Heading summary: Agent F - Execution State-Machine Tests & Effective-Sample Weights; Scope; External Report Sections Addressed; 4.5 ? Explicit order-state transition tests; 4.7 ? Persisted effective-sample weights for paper fills; Files Changed; Tests Run; Risks; Merge Notes

Agent F - Execution State-Machine Tests & Effective-Sample Weights

Date: 2026-06-11
Branch: `feat/tradeo12-execution-state-gap-20260611`
Worktree: `/home/vboxuser/tradeo-worktrees/execution-state-gap`
Base: local `main` at `df418fb` (phases A-D already merged)

Scope

Close the two remaining Execution/Director gaps from the 12-phase compliance matrix (`tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`):

- 4.5 Execution/reconciliation: the auto kill switch existed but had no explicit order-state transition tests.
- 4.7 Director sequential gate: `effective\_lab\_trades` was a raw-count proxy ("until per-trade uniqueness weights are persisted"); weights are now computed and persisted.

4.3 (richer real-time microstructure feeds) remains *future* ? no provider is available; nothing in this phase touches it. No data/research code was modified. No live trading paths were loosened.

External Report Sections Addressed

4.5 ? Explicit order-state transition tests

New test module `backend/tradeo/tests/test\_execution\_state\_transitions.py` (22 tests) covering the full trade state machine:

- **Paper broker**: `PAPER\_APPROVED/LIVE\_APPROVED` signal ? `OPEN` trade + signal `EXECUTED`; every non-executable signal status is rejected without creating a trade; zero quantity is rejected.
- **Lab shadow observations**: `OPEN ? CLOSED` on target hit, stop hit; stays `OPEN` (with `awaiting\_future\_market\_bars` lifecycle metadata) when no future bars exist; stays `OPEN` with `market\_data\_unavailable` diagnostics when the provider fails; `open\_observation` is idempotent per signal (no duplicate `OPEN` rows). Shadow closes keep `shadow\_no\_order` evidence and `shadow\_close` provenance ? they never masquerade as broker fills.
- **Evidence promotion transitions**: `ibkr\_paper\_order ? ibkr\_paper\_fill` happens only on `CLOSED` status with real broker provenance (`broker\_execution` / `broker\_statement\_import`); `OPEN` and `CANCELLED` never promote; `simulated\_close` provenance never promotes; `live\_order ? live\_fill` follows the same rule.
- **Reconciliation**: clean DB?broker state leaves the kill switch off (open orders count as consistent); `db\_open\_trade\_missing\_at\_broker` and `broker\_position\_not\_in\_db` divergences both activate the runtime kill switch and audit it; zero-quantity broker positions are ignored; non-IBKR open trades (paper / lab shadow) are excluded from divergence checks; broker-unreachable is audited but **never** trips the kill switch; `reconciliation\_auto\_kill\_switch=false` records the divergence without activation; a pre-existing kill switch is reported via `kill\_switch\_already\_active`.
- **Kill switch state machine**: an active runtime kill switch blocks `IBKRBroker.submit\_signal\_bracket` before any broker connection; activation is idempotent and audited (with `already\_active` flag); deactivation requires an explicit actor and is audited.

4.7 ? Persisted effective-sample weights for paper fills

New service `backend/tradeo/services/effective\_sample.py` (method id `inverse\_symbol\_day\_cluster\_size\_v1`):

- Each closed normal IBKR paper fill belongs to a `(symbol, trading day)` cluster (day from `closed\_at`, falling back to `opened\_at`).
- Each fill weighs `1 / cluster\_size`, so a correlated same-symbol same-day burst contributes exactly one effective sample.
- `n\_eff = ? weights = number of distinct clusters` is the binding number; Kish ESS is reported as a secondary diagnostic of weight inequality only.
- Weights are **persisted twice** for auditability:
 - per trade in `trade.metadata\_json.effective\_sample` (method, cluster key, cluster size, weight, computed_at), idempotent across refreshes;
 - per pattern in `metrics\_json.lab\_execution.effective\_sample` (full per-trade breakdown + cluster sizes), so any gate decision can be reproduced from stored rows alone.

`DirectorReviewGate` changes (`director\_review\_gate.py`):

- `effective\_lab\_trades` is now the weighted `n\_eff` (was raw fill count) and the `effective\_lab\_trades\_below\_{min}` blocker compares `n\_eff` against `director\_min\_eff\_trades` (default 25, unchanged).
- The old "conservative lower-bound proxy" note was replaced by the weighted definition; `closed\_lab\_trades` / `paper\_fill\_trades` remain raw counts.
- `\_promotion\_blockers` takes `effective\_trades` (defaults to the raw count when not supplied, preserving behavior for any external caller).

This is intentionally stricter: 25 fills concentrated on one symbol-day now count as 1 effective sample and stay blocked, which is the failure mode the matrix gap described. `DirectorProductionGate` thresholds were not touched.

Files Changed

- `backend/tradeo/services/effective\_sample.py` (new)
- `backend/tradeo/services/director\_review\_gate.py`
- `backend/tradeo/tests/test\_execution\_state\_transitions.py` (new, 22 tests)
- `backend/tradeo/tests/test\_effective\_sample\_weights.py` (new, 6 tests)
- `backend/tradeo/tests/test\_research\_lab\_fox\_lifecycle.py` (fixture only: trades now carry `opened\_at`/`closed\_at` consistent with their declared per-day `broker\_execution\_time`; previously all 10 fills defaulted to the same day and the new weighting correctly blocked them as 1 effective sample)
- `docs/remediation/agent\_f\_execution\_state\_gap\_2026\_06\_11.md` (this file)

Tests Run

- `pytest tradeo/tests/test\_execution\_state\_transitions.py tradeo/tests/test\_effective\_sample\_weights.py` ? 33 passed
- `pytest tradeo/tests` (full backend suite) ? **210 passed**, 1 warning
- `ruff check` on all changed files ? clean

(venv: `/home/vboxuser/tradeo/backend/.venv`, package resolved from this worktree via cwd/PYTHONPATH.)

Risks

- **Semantic tightening**: any pattern whose paper evidence is concentrated by symbol-day will now report a lower `effective\_lab\_trades` and may regain the `effective\_lab\_trades\_below\_25` blocker. This is the intended fix, but operators reviewing dashboards should expect the number to drop for concentrated patterns.
- The gate now writes `effective\_sample` into closed trades' `metadata\_json` during `refresh()`. Writes are idempotent (skipped when method/cluster/weight are unchanged) and only touch fills of patterns under review, but it is a new write path on evidence rows; the original evidence fields are never modified.
- Weights depend on cluster composition at evaluation time: adding a fill to an existing cluster re-weights its siblings on the next refresh. The persisted `computed\_at`/`method` fields make each snapshot traceable.
- Kish ESS is informational only; if someone later wires it into a gate they should note it does not capture clustering (equal weights ? raw n).

Merge Notes

- Single-commit branch over `df418fb`; touches only Execution/Director service + tests + this doc, so conflicts with other phase branches are unlikely unless they also edit `director\_review\_gate.py` (`\_store\_lab\_execution\_metrics` / `\_promotion\_blockers`) or the lifecycle test fixture.
- Compliance matrix follow-up after merge: 4.5 gap "explicit order-state transition tests" ? closed; 4.7 gap "persisted effective-sample weights for paper fills" ? closed; 4.3 richer real-time feeds remains *future* (no provider).
- No config/env changes; `director\_min\_eff\_trades=25`, `director\_min\_symbols=8`, `director\_min\_days=10`, `reconciliation\_auto\_kill\_switch=true` defaults untouched.

Full Document: `docs/remediation/agent\_g\_audit\_final\_gap\_2026\_06\_11.md`

`docs/remediation/agent\_g\_audit\_final\_gap\_2026\_06\_11.md` ? 85 lines, 3984 bytes, sha256 prefix `6fb8b1c2a97a286e`

Heading summary: Agent G - Audit Final Package & Compliance Traceability; Scope; Deliverables; Verification performed against code (not just docs); Files Changed; Tests Run; Remaining Risks / Known Non-Compliance; Merge Notes

Agent G - Audit Final Package & Compliance Traceability

Date: 2026-06-11
 Branch: `feat/tradeo12-audit-final-gap-20260611`
 Worktree: `/home/vboxuser/tradeo-worktrees/audit-final-gap`

Base: local `main` @ `df418fb` (agents A?D merged)

Scope

Final-package phase: consolidate the 12-section status across both remediation waves (A?D merged on 2026-06-10; E merged / F pending on 2026-06-11), verify documentation claims against the code, document the merge order, and add light traceability tests. No production logic was changed.

Deliverables

- `docs/remediation/tradeo\_12\_phase\_final\_report\_2026\_06\_11.md` ? final consolidated audit package (12-section status, test-evidence ledger, honest-claims review, remaining gaps, merge order). Supersedes the 06-10 final report on `main`.
- `docs/remediation/agent\_g\_audit\_final\_gap\_2026\_06\_11.md` ? this report.
- `backend/tradeo/tests/test\_remediation\_docs\_traceability.py` ? 4 tests:
 - every path under a "Files Changed" heading in `docs/remediation/\*.md` resolves to a real file;
 - the compliance matrix keeps rows for agents A?D;
 - `research/audit\_bridge/export\_audit\_package.py` retains the honest-count fields (`event\_count`, `independent\_sample\_count`, `is\_independent\_sample`) added by Agent D;
 - tests skip gracefully where `docs/` is absent (the Docker image only copies `backend/tradeo` and `research`).
- Appended an Agent G consolidation section to `tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md` (append-only; no other agent's rows were edited).

Verification performed against code (not just docs)

- Confirmed all "Files Changed" paths in agent A?D reports exist at this branch's tree (also enforced by the new test).
- Confirmed `export\_audit\_package.py` exposes `sample\_count`, `independent\_sample\_count`, `event\_count`, `is\_independent\_sample`, and that `test\_audit\_hardening.py` includes `test\_export\_metrics\_reconstruct\_independent\_sample\_counts\_from\_event\_rows`.
- Inspected Agent E (`83ccd97`) and Agent F (`12f2157`) commits to confirm their reports' claims match their diffs (regime service + incremental cache + snapshot hash; order-state tests + effective-sample weights).

Files Changed

- `backend/tradeo/tests/test\_remediation\_docs\_traceability.py`
- `docs/remediation/tradeo\_12\_phase\_final\_report\_2026\_06\_11.md`
- `docs/remediation/agent\_g\_audit\_final\_gap\_2026\_06\_11.md`
- `docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`

Tests Run

- `docker run --rm -v "\$PWD:/repo:ro" tradeo-backend:latest pytest /repo/backend/tradeo/tests/test\_remediation\_docs\_traceability.py -q` (repo mounted read-only so the docs tree is visible) ? 4 passed.
- `docker run --rm -v "\$PWD/backend/tradeo:/app/tradeo:ro" tradeo-backend:latest pytest tradeo/tests -q` (image layout, no docs) ? 178 passed, 3 skipped (the docs-dependent tests skip as designed; 177 was the A?D baseline, +1 from the audit-export field test).
- `ruff check tradeo/tests/test\_remediation\_docs\_traceability.py` in the image ? passed.

Remaining Risks / Known Non-Compliance

- The traceability test validates path existence, not content accuracy; prose claims were reviewed manually in this phase.
- The matrix file accumulates per-wave rows; statuses for 4.5/4.7 in the Agent D rows describe wave-1 state and are superseded by the Agent G consolidation note (and by Agent F's report) rather than edited in place.
- This branch is based at `df418fb`, so it does not contain Agent E's merge or the 06-10 final report; both new docs are additive and the matrix edit is an end-of-file append to keep the merge surface minimal.

Merge Notes

- Recommended order: F first, then G (details in the final report).
- Only expected conflict: the compliance matrix (E appended rows on `main`); resolve by keeping both E's rows and G's trailing section.
- No live trading settings touched; docs and one additive test module only.

Full Document:

`docs/remediation/agent\_h\_shadow\_canonical\_outcome\_parity\_2026\_06\_11.md`

`docs/remediation/agent\_h\_shadow\_canonical\_outcome\_parity\_2026\_06\_11.md` ? 112 lines, 5935 bytes, sha256 prefix `8dc3fa9e4133ac47`

Heading summary: Agent H - ShadowTracker Canonical-Outcome Parity; Problem; Change; Behavior change (intentional tightening); Files Changed; Tests Run; Risks; Next Step Recommended

Agent H - ShadowTracker Canonical-Outcome Parity

Date: 2026-06-11
Branch: `main` (direct, single-phase execution/director task)
Scope: external report section 6 (backtester/shadow parity), gap 6 of the
2026-06-11 final report: "ShadowTracker canonical-outcome parity (6)".

Problem

The lab shadow-observation lifecycle
(`LabPaperObservationService.\_evaluate\_trade`) closed observations with an
ad-hoc stop/target loop while the backtester (Agent C) and Research RR
simulation already shared the canonical triple-barrier engine
(`triple\_barrier\_outcome` in `quant\_validation.py`). The divergences were
all optimistic for the shadow path:

- a bar that **opened through the stop** was filled at the stop price; the canonical rule fills at the OPEN (worse than the stop);
- a bar that **opened through the target** happened to fill at the target, but only because stop/target were checked intrabar ? the open-gap ordering (stop gap before target gap) was not applied;
- intrabar stop+target resolution matched by code-order accident, not by an explicit `conservative\_both` contract;
- no canonical outcome record was persisted, so a gate decision based on a shadow close could not be reproduced against the shared engine.

Because Director review evidence includes shadow/near-miss observations, optimistic shadow fills inflate apparent R for gapped stops ? exactly the class of bias section 6 flags.

Change

`\_evaluate\_trade` now delegates exit math to `triple\_barrier\_outcome`:

- The shadow position exists from `opened\_at` at `trade.entry`, before the first future bar. Two synthetic bars pinned at the entry price (signal bar + entry bar) precede the future bars, so every real bar is "posterior to entry" and the canonical open-gap rules apply to **all** future bars, including the first. `maxBars` is passed as `max\_holding\_bars + 1` to account for the synthetic entry-bar slot.
- `entry\_price=trade.entry`, `side=±1` from `trade.side`, `gap\_entry\_policy="skip"`, `conservative\_both=True`, `round\_trip\_cost\_R=0.0` (shadow observations carry zero execution costs by design; cost modelling for shadow evidence stays downstream).
- Pending semantics preserved: a `time` outcome is only accepted once `len(future) >= max\_holding\_bars`; otherwise the observation stays open and the existing diagnostic path (`awaiting\_future\_market\_bars`) runs. Non-`ok` engine statuses (malformed barriers, e.g. entry at/through a barrier) also keep the observation pending instead of fabricating an exit.
- `exit\_reason` keeps the shadow vocabulary via `CANONICAL\_EXIT\_REASON\_MAP` (`stop`/`stop\_and\_target\_conservative` ? `stop\_hit`, `target` ? `target\_hit`, `time` ? `time\_stop`); `stop\_gap`/`target\_gap` pass through unchanged because `implementation\_shortfall.py` already classifies them and renaming would hide that the fill was at the open.
- Closed trades persist a `canonical\_outcome` metadata block (engine, status, canonical reason, R, mfe_R, mae_R, bars_held, conservative_both, gap_entry_policy, round_trip_cost_R) so any gate decision can be reproduced from stored rows against the shared engine.

`r\_multiple`/`pnl` continue to be computed in `\_close\_trade` from the exit price; with zero costs they equal the canonical `R` (asserted in tests).

Behavior change (intentional tightening)

Shadow observations whose stop is gapped at the open now record the open as the exit price (e.g. entry 10, stop 9, next open 8.4 ? R ?1.6, was ?1.0), with `exit\_reason="stop\_gap"`. Dashboards and gate evidence will show worse R for gapped shadow stops; this is the parity fix, not a regression.

Files Changed

- `backend/tradeo/modules/laboratory/paper\_observations.py`
- `backend/tradeo/tests/test\_shadow\_canonical\_outcome\_parity.py`
- `docs/remediation/agent\_h\_shadow\_canonical\_outcome\_parity\_2026\_06\_11.md`
- `docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`

Tests Run

- `.venv/bin/pytest -q tradeo/tests/test\_shadow\_canonical\_outcome\_parity.py tradeo/tests/test\_execution\_state\_transitions.py tradeo/tests/test\_pattern\_entry\_scanner.py` ? 68 passed
- `.venv/bin/pytest -q tradeo/tests` (full backend suite) ? **270 passed**, 1 warning
- `.venv/bin/ruff check` on both changed Python files ? clean

New tests (8) cover: first-bar and later-bar stop gaps fill at the open (long and short), target gaps fill at the target (not the open), intrabar stop+target resolves to the stop with the canonical reason recorded, time-stop waits for the full holding window, the persisted `canonical\_outcome` block, and a direct engine-parity assertion comparing the persisted exit against `triple\_barrier\_outcome` on the same arrays.

Risks

- **Evidence numbers shift down** for any pattern whose shadow history contains gapped stops; `effective\_lab\_trades` is unaffected (counts, not R), but health/SQN-style metrics fed by shadow R will be more conservative. Intended.
- Malformed observations (entry at or beyond a barrier) now stay pending

with diagnostics instead of being force-closed at a barrier price. They surface via the existing `lab\_shadow\_observation\_pending\_bars` audit path.

- MFE/MAE: both the legacy `mfe`/`mae` fields (from observed bars) and the canonical `mfe\_R`/`mae\_R` (which include the zero-excursion synthetic entry bar) are persisted; consumers should prefer the canonical block for engine-comparable numbers.
- No order routing, broker configuration, gate thresholds or live-trading flags were touched; shadow observations remain `no\_ibkr\_order=true`.

Next Step Recommended

Gap 3.5 (true non-trade/skipped-signal accounting through aggregate metrics): the engine's `skipped` statuses (`gapped\_through\_stop`, `gapped\_past\_target`) are now reachable from the shadow path's pending diagnostics, so wiring them into the aggregate non-trade metrics is the natural follow-on execution/director phase.

Full Document:

`docs/remediation/agent\_i\_regime\_outcome\_calibration\_2026\_06\_11.md`

`docs/remediation/agent\_i\_regime\_outcome\_calibration\_2026\_06\_11.md` ? 122 lines, 6422 bytes, sha256 prefix `59b39ad715b02f85`

Heading summary: Agent I - Calibrated Benchmark-Regime Outcome History (3.8); Problem; Change; Honest limits; Files Changed; Tests Run; Risks; Next Step Recommended

Agent I - Calibrated Benchmark-Regime Outcome History (3.8)

Date: 2026-06-11
Branch: `feat/trade012-regime-outcome-calibration-20260611`
Scope: external report section 3.8 (market regimes), the Agent E remaining gap: "Not yet a hard gate in `\_pattern\_regime\_fit`; needs labeled outcome history per regime bucket to calibrate."

Problem

Agent E shipped the PIT SPY SMA200/vol-tercile regime service and attached `benchmark\_regime` to every match, but `\_pattern\_regime\_fit` could only use presence heuristics (was this regime *seen* during research?) because no outcome history existed per benchmark-regime bucket. A pattern that was merely *observed* in `benchmark\_bear|high\_vol\_tercile` scored 0.85 even if every such sample lost money, and there was no data to ever justify a hard gate.

Change

Research now persists labeled outcome history per benchmark-regime bucket, and the matcher consumes it as a calibrated fit with an optional hard gate.

1. `**services/market_regime.py**`
 - `regime\_keys\_for\_dates(table, dates)`: PIT regime_key at the last benchmark bar on or before each date; dates before benchmark history return an explicit `insufficient\_history|insufficient\_history` key (`unlabeled\_regime\_key()`) instead of guessing.
 - `MarketRegimeService.history\_table(period)`: full PIT regime table covering the research period plus the labeling warmup (`required\_benchmark\_bars`), with the regime config recorded in `DataFrame.attrs`. `period\_to\_months` converts provider period strings.
2. `**research/cluster_research_engine.py**`
 - New optional field `benchmark\_regime\_table`.
 - `\_benchmark\_regime\_outcomes(samples, side, rr)`: labels each cluster sample's window end with the PIT benchmark regime and simulates it with the canonical triple-barrier path (`RewardRiskAnalyzer.\_simulate\_sample`, same engine as backtester/shadow) at the pattern's side and best RR. Per-bucket `sample\_count`/`expectancy\_r`/`win\_rate`/`profit\_factor` are persisted inside `regime\_profile.benchmark\_regime\_outcomes`, with honest `labeled\_sample\_count`/`unlabeled\_sample\_count` and an `available:false` record when the benchmark table is missing.
3. `**agents/pattern_discovery_lab_agent.py**`: builds the regime table via `MarketRegimeService.history\_table(params["period"])` and passes it to the engine; failure degrades to a run warning, never a failed discovery run.
4. `**research/novel_pattern_matcher.py**`
 - `\_pattern\_regime\_fit(pattern, regime, settings)` now tries `\_calibrated\_regime\_fit` first: when the CURRENT `benchmark\_regime` bucket has at least `market\_regime\_outcome\_min\_samples` labeled outcomes, the fit is calibrated ? positive expectancy ? `calibrated\_regime\_positive` (score 0.75?1.0, scaled by expectancy), non-positive ? `calibrated\_regime\_negative` (score 0.2, `hard\_gate\_blocked: true`). The full bucket evidence is attached as `regime\_fit.calibration` for audit.
 - Insufficient bucket samples, unlabeled benchmark regime, or missing calibration data fall back to the existing presence heuristics unchanged (legacy patterns keep their previous behavior bit-for-bit).
 - `match\_current` drops calibrated-negative matches before variant creation only when `market\_regime\_hard\_gate\_enabled` is on (default OFF), logging each block; run output now reports `regime\_hard\_gate\_enabled` and `regime\_gate\_blocked` counts.
5. `**Config**`: `market\_regime\_outcome\_min\_samples` (default 12) and `market\_regime\_hard\_gate\_enabled` (default false) in `Settings` and `.env.example`.

Honest limits

- The calibration data only exists for patterns (re)discovered after this change; existing DB patterns carry no `benchmark\_regime\_outcomes` and keep heuristic scoring. No backfill is fabricated.
- Buckets are calibrated from research sample simulations (canonical triple-barrier at next-bar open, zero-cost variant via best-RR), not from real fills; real-fill regime attribution stays future work once enough labeled paper fills exist per bucket.
- The hard gate ships disabled. Enabling it is a Director-level decision once rediscovered patterns populate calibrated buckets.

Files Changed

- `backend/tradeo/services/market\_regime.py`
- `backend/tradeo/research/cluster\_research\_engine.py`
- `backend/tradeo/agents/pattern\_discovery\_lab\_agent.py`
- `backend/tradeo/research/novel\_pattern\_matcher.py`
- `backend/tradeo/core/config.py`
- `.env.example`
- `backend/tradeo/tests/test\_regime\_outcome\_calibration.py`
- `docs/remediation/agent\_i\_regime\_outcome\_calibration\_2026\_06\_11.md`
- `docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`

Tests Run

- `pytest /app/tradeo/tests/test\_regime\_outcome\_calibration.py -q` in `tradeo-backend:latest` (worktree mounted read-only) ? ****11 passed****
- Full backend suite in the same image ? ****278 passed, 4 skipped****
- `ruff check /app/tradeo` ? clean

New tests (11) cover: PIT date?regime_key mapping including the before-history unlabeled key, empty/missing-table degradation, period-string parsing, warmup-inclusive `history\_table` fetch with config attrs, per-bucket outcome aggregation (sign + win rate + labeled/unlabeled counts), `available:false` without a table, calibrated positive/negative fits with calibration evidence, fallback when the bucket is too small, when the benchmark regime is unlabeled, and when no settings are passed (legacy call shape).

Risks

- Match scores can change for newly discovered patterns once their buckets reach `min\_samples`: a calibrated-positive bucket scores ≥ 0.75 where the heuristic gave 0.85 for mere presence; a calibrated-negative bucket drops to 0.2. Regime fit is 8% of the final match score, so drift is bounded; the hard filter itself stays off by default.
- `history\_table` adds one daily benchmark fetch per discovery run, served by the existing OHLCV cache (incremental tail refresh on the daily feed).
- No execution-path, sizing, gate-threshold, or live-trading flags touched.

Next Step Recommended

Once rediscovered patterns populate calibrated buckets, attribute real paper-fill outcomes (`benchmark\_regime` is already on every stored match) into the same bucket schema and let the Director compare research-simulated vs realized per-regime expectancy before enabling the hard gate.

Full Document: `docs/remediation/agent\_j\_non\_trade\_accounting\_2026\_06\_11.md`

`docs/remediation/agent\_j\_non\_trade\_accounting\_2026\_06\_11.md` ? 103 lines, 4717 bytes, sha256 prefix `46c259fd5efc5cf6`

Heading summary: Agent J - True Non-Trade / Skipped-Signal Accounting in Aggregate Metrics; Problem; Change; Behavior change (intentional tightening); Files Changed; Tests Run; Risks; Next Step Recommended

Agent J - True Non-Trade / Skipped-Signal Accounting in Aggregate Metrics

Date: 2026-06-11
 Branch: `feat/non-trade-accounting-aggregates`
 Scope: gap 3.5 remainder from Agent C ("True non-trade/skipped signal accounting should propagate beyond labels"), flagged again by Agent H as "3.5 non-trade accounting still open".

Problem

The canonical triple-barrier engine (`triple\_barrier\_outcome`, gap-entry policy `skip`) already labeled gap-skipped signals, but the skip never propagated past the label:

- `RewardRiskAnalyzer.metrics\_for\_rr` appended `0.0R` to the results array for every skipped signal (via `\_tuple\_from\_detail`). That phantom `0R` diluted `win\_rate`/`loss\_rate`/`expectancy\_r`/`median\_result\_r`, inflated `sample\_count` (the `min\_samples` candidate gate counted non-trades as evidence), shrank `timeout\_rate`/`target\_hit\_rate`/`stop\_hit\_rate` denominators, and pushed the per-sample MFE/MAE/cost/fill arrays and a phantom `timeout` speed label for signals that never traded.
- Non-`ok` statuses other than `skipped` (`invalid`, `no\_data`) carry `R=NaN`; had one ever occurred it would have poisoned every aggregate. The backtester had the same latent NaN path (`simulate\_exit` only filtered `skipped`).
- `Backtester.run` silently dropped skipped signals: `BacktestMetrics` had no record of how many detected signals never became trades, so backtest expectancy could not be reconciled against signal frequency.

Change


```

`backend/tradeo/research/reward_risk_analyzer.py`:

- `metrics_for_rr` treats any non-(`ok`/`fallback`) status as a true
  non-trade: it increments `triple_barrier_labels["skipped"]` and a new
  per-reason counter, then skips the sample before any traded array is
  touched. Traded aggregates now cover executed trades only.
- New aggregate keys (additive; all consumers read defensively via
  `.get`): `signal_count` (all evaluated samples), `skipped_count`,
  `skip_rate`, `skip_reason_counts` (e.g. `gapped_past_target`,
  `gapped_to_stop_zone`). `sample_count` now counts traded samples only,
  which makes the `min_samples` candidate gate and downstream
  `validation_gate`/`active_learning` sample thresholds conservative
  (non-trades no longer count as evidence).

`backend/tradeo/services/backtester.py`:

- `_run_symbol` returns `(trades, signals, skipped)`; a detected candidate
  that fails the entry-gap pre-filter or the canonical gap-entry skip is
  counted instead of vanishing.
- `_simulate_exit` filters on `status != "ok"` (was `== "skipped"`),
  closing the latent `invalid`/`no_data` NaN path.
- `run` aggregates `total_signals`/`skipped_signals`/`skip_rate` into
  `BacktestMetrics`.

`backend/tradeo/schemas.py`:

- `BacktestMetrics` gains `total_signals: int = 0`,
  `skipped_signals: int = 0`, `skip_rate: float = 0.0` (defaulted, so
  persisted/serialized payloads remain compatible).

## Behavior change (intentional tightening)

Patterns whose edge previously leaned on phantom `OR` fills will now show
honest (usually wider) win/loss rates and an explicit `skip_rate`;
`sample_count` drops by the number of skipped signals, so clusters near the
`min_samples` boundary may stop qualifying. That is the point of gap 3.5:
non-trades are not evidence.

## Files Changed

- `backend/tradeo/research/reward_risk_analyzer.py`
- `backend/tradeo/services/backtester.py`
- `backend/tradeo/schemas.py`
- `backend/tradeo/tests/test_reward_risk_analyzer.py`
- `backend/tradeo/tests/test_backtester_non_trade_accounting.py`
- `docs/remediation/tradeo_12_phase_compliance_matrix_2026_06_10.md`
- `docs/remediation/agent_j_non_trade_accounting_2026_06_11.md`

## Tests Run

- New: `test_skipped_signals_do_not_dilute_traded_aggregates`,
  `test_all_skipped_signals_yield_zero_sample_count` (analyzer);
  `test_gap_prefiltered_signal_counts_as_skipped_not_trade`,
  `test_executed_signal_counts_as_trade_with_zero_skips` (backtester, stub
  provider/detector).
- Full backend suite: 286 passed (includes Agent I's regime calibration
  work merged to `main` during this task). `ruff check`: clean.

## Risks

- `sample_count` semantics changed from "signals evaluated" to "trades
  simulated". Reviewed consumers (`validation_gate`, `active_learning`,
  `cluster_research_engine`) treat it as a minimum-evidence gate, so the
  change only tightens; nothing computes derived rates from it.
- Shadow/paper observation path is untouched (its skip semantics differ by
  design: malformed barriers only, per Agent H).

## Next Step Recommended

Surface `skip_rate` in Director review evidence: a pattern with strong
traded expectancy but high gap-skip frequency has lower deployable
capacity, and the Director gate currently does not see it.

```

Full Document: `docs/remediation/agent\_k\_discovery\_determinism\_2026\_06\_11.md`

`docs/remediation/agent\_k\_discovery\_determinism\_2026\_06\_11.md` ? 80 lines, 5374 bytes, sha256 prefix `c0c6aa40abb71a0e`

Heading summary: Agent K ? Discovery Bit-for-Bit Determinism (Wave4-B, 2026-06-11); Objective; Audit findings; Implemented; Harness; Honest limits

Agent K ? Discovery Bit-for-Bit Determinism (Wave4-B, 2026-06-11)

Branch: `feat/wave4-discovery-determinism-20260611`

Objective

Make discovery/research output reproducible bit-for-bit for identical inputs, config and seed ? or close the scope honestly with a harness plus fixes for the identified nondeterminism sources.

Audit findings

```
| Source | Location | Verdict |
|---|---|---|
| Clustering RNG | `cluster_research_engine.py` `MiniBatchKMeans(random_state=42, n_init=10)` | Already seeded; deterministic in-process for identical input matrices. |
| Null/bootstrap RNG | `cluster_research_engine.py` `_null_seed()` (blake2b of side/rr/sizes), `adversarial_research.py` `_seed_index()`, `quant_validation.py` explicit `rng` params | Already content-addressed/seeded; no wall-clock or global RNG. |
| Sample ordering | `discover()` sorts window sizes; `_cluster_window_size` sorts samples by `end` | Deterministic given unique window-end dates; verified by shuffle test. |
| Engine-core timestamps | `cluster_research_engine.py`, `window_sampler.py`, `adversarial_research.py`, `quant_validation.py` | None present ? engine core is wall-clock free. |
| Report JSON ordering | `pattern_discovery_lab_agent._write_report` dumped without `sort_keys` | Fixed: report JSON now dumps with `sort_keys=True`. |
| Report identity vs run metadata | Report filename embeds wall-clock `ts`; payload embeds `run_id`, paths, `generated_at` | Fixed via content hash*: payload now carries `determinism.content_hash` (sha256 over canonical JSON excluding volatile keys), so two runs over identical inputs are comparable by hash regardless of run_id/timestamp/paths. |
| Memory-graph dict ordering | `autonomous_research_director.py` iterated `families.items()` / `regimes.items()` in DB-row insertion order | Fixed: iterations sorted; canonical-member tie now breaks on `pattern_key`. |
| Pattern query tie order | `_patterns` ordered by `(score desc, created_at desc)` only | Fixed: added `pattern_key asc` tiebreaker so equal-score/timestamp rows enumerate deterministically. |
| Test fixture hash salt | `tests/fixtures.py` `fixture_ohlcv` seeds from `abs(hash(symbol))` (PYTHONHASHSEED-salted) and `pd.Timestamp.now()` index | Not changed (out of scope; would silently reshape data under existing tests). New harness builds its own seeded fixtures. Noted as a known cross-process variance in test fixture data, not in production discovery. |
```

Implemented

1. `backend/tradeo/research/determinism.py` (new):
 - `canonical\_payload` / `canonical\_json`: sorted stringified keys, volatile keys dropped, sets sorted, numpy unwrapped, non-finite floats ? `null`.
 - `content\_hash` (`sha256.canonical\_json\_v1`) with `DEFAULT\_VOLATILE\_KEYS` = timestamps (`built\_at`, `generated\_at`, ?), `run\_id`, durations, and artifact/filesystem path keys.
 - `candidate\_content\_payload` / `discovery\_content\_hash` for full candidate-list identity (metrics included).
2. `pattern\_discovery\_lab\_agent.\_write\_report`: appends a `determinism` block (`algo`, `content\_hash`, `excluded\_keys`, `generated\_at`) computed before the block itself is added, and writes the JSON with `sort\_keys=True`.
3. `autonomous\_research\_director.py`: deterministic enumeration (sorted families/regimes, pattern_key tiebreakers in query and canonical pick).

No statistics, thresholds or gates were changed; all edits are ordering, serialization and metadata-identity only.

Harness

`backend/tradeo/tests/test\_discovery\_determinism.py` (6 tests):

- Canonical-payload unit contracts: key-order/type insensitivity, volatile-key exclusion, non-finite ? null.
- **Double-run engine contract**: real `ClusterResearchEngine.discover` runs twice over freshly rebuilt seeded synthetic fixtures (two separated vector blobs, 48 windows, unique end dates); full candidate payloads compared as canonical JSON strings ? bit-for-bit equality, plus equal `discovery\_content\_hash`.
- **Input-order independence**: shuffled sample order produces an identical content hash.
- **Report artifact identity**: `\_write\_report` called with different `run\_id`'s yields equal `determinism.content\_hash` and byte-identical canonical payloads outside the `determinism` block.

Evidence: targeted file 6 passed; image suite 288 passed, 4 skipped; ruff clean on touched files.

Honest limits

- The contract is **same machine, same image, same library versions, in-process**. sklearn/BLAS may vary float reductions across versions, hardware or thread configurations; cross-environment bit-equality is not claimed and not tested.
- Full real discovery (`PatternDiscoveryLabAgent.run`) hits live market data, DB autoincrement ids and the accumulating global experiment registry; those are run-context, excluded from the identity hash, and a full end-to-end double-run is intentionally out of scope. The deterministic contract covers the engine core (clustering, validation statistics, scoring) and the report payload identity.
- Registry hash-chain values depend on prior registry state by design; they are deterministic given the same starting state but differ across accumulating runs.
- `tests/fixtures.py` `fixture\_ohlcv` remains PYTHONHASHSEED-sensitive across processes (pre-existing; affects only fixture-generated test data).

Full Document: `docs/remediation/agent\_l\_rediscovery\_readiness\_2026\_06\_11.md`

`docs/remediation/agent\_l\_rediscovery\_readiness\_2026\_06\_11.md` ? 70 lines, 4693 bytes, sha256 prefix `8e9c68e185694a6c`

Heading summary: Agent L ? Rediscovery/Backfill Readiness (Wave4-C, 2026-06-11); Objective; Remediation matrix; Implemented; Explicitly NOT done in this phase; Tests run; Risks / assumptions

Agent L ? Rediscovery/Backfill Readiness (Wave4-C, 2026-06-11)

Branch: `feat/wave4-rediscovery-readiness-20260611`

Objective

Legacy `DiscoveredPattern` rows predate the Wave4 metadata contracts and lack the embedding contract, cluster signature/concentration metadata and benchmark-regime calibration buckets. Deliver a safe path to identify them, mark them and verify a real rediscovery repopulated them ? without running a heavy unbounded job in this phase and without pretending production rows were populated.

Remediation matrix

```
| Gap | Where it lives | Remediation delivered | Populated in prod? |
|----|----|----|----|
| Legacy patterns lack `feature_parity_contract` (embedding contract) | `metrics_json` on `discovered_patterns` rows written before Wave4 | Audit check `embedding_contract`: missing if no `contract_id`; **stale** if `contract_id != PatternEmbeddingEngine.CONTRACT_ID` (v2 legacy-prefix-stable) | No ? readiness only; runbook §3 populates |
| Legacy patterns lack `cluster_signature` / `concentration_checks` / `medoid` | same | Audit check `cluster_signature`: requires dict signature with `medoid` dict and `concentration_checks.passed` present | No ? readiness only |
| Legacy patterns lack `regime_profile.benchmark_regime_outcomes` (calibration buckets) | same | Audit check `regime_calibration_buckets`: requires outcomes dict with `available` key | No ? readiness only |
| No way to mark laggards | n/a | `--apply-flags` writes `metrics_json.rediscovery_readiness` block (`needs_rediscovery`, `missing`, `stale`, `checker_version`, `flagged_at`); JSONB-extensible, no schema migration, flips honestly to `false` once real metadata arrives | Flag only ? never fakes metadata |
| No rediscovery plan/manifest | n/a | `run_readiness()` emits manifest: per-pattern reasons, per-field counts, `metadata_complete_before == metadata_complete_after` by construction (tool populates nothing), truncation reported, `determinism.content_hash` (Wave4-B algo) | Manifest is dry-run honest |
| Unbounded scan risk | n/a | `DEFAULT_AUDIT_LIMIT = 2000` hard cap + explicit `truncated` flag in manifest and CLI warning | n/a |
| No execution/verification procedure | n/a | `docs/research/wave4_rediscovery_runbook.md`: bounded discovery commands, re-audit verification, success criteria, non-reemerging-pattern disposition rule | Pending operator run |
```

Implemented

1. `backend/tradeo/research/rediscovery\_readiness.py` (new):
 - `evaluate\_pattern\_metrics` ? pure per-blob check returning (`missing`, `stale`) against the three Wave4 contracts.
 - `audit\_patterns` ? deterministic scan (ordered by `pattern\_key`), bounded by `limit`, reports truncation.
 - `apply\_rediscovery\_flags` ? opt-in flagging into `metrics\_json`; clears to `needs\_rediscovery: false` when metadata is later present.
 - `build\_manifest` / `run\_readiness` ? honest manifest with before/after counts, content hash, and rediscovery plan pointer; optional JSON dump.
 - CLI: `python -m tradeo.research.rediscovery\_readiness [--apply-flags] [--limit N] [--manifest-out PATH]` ? dry-run by default.
2. `backend/tradeo/tests/test\_rediscovery\_readiness.py` (new, 7 tests):
legacy-pattern flagged with all three fields missing; modern pattern clean; stale v1 embedding contract detected; dry-run mutates nothing and `after == before`; `--apply-flags` marks but re-audit still flags (flag ? metadata); flag flips to false after real metadata upsert; manifest written to disk and truncation honest at `limit`.
3. `docs/research/wave4\_rediscovery\_runbook.md` (new): operator runbook.

Explicitly NOT done in this phase

- No discovery/rediscovery job was executed; production rows remain unpopulated until an operator follows runbook §3.
- No `--apply-flags` run against the production DB (operator decision).
- No schema migration (JSONB block chosen deliberately; no Alembic in repo).
- No live/paper order paths touched.

Tests run

```
backend/.venv pytest tradeo/tests/test_rediscovery_readiness.py -> 7 passed
pytest tradeo/tests/test_novel_pattern_registry.py tradeo/tests/test_discovery_determinism.py -> 10 passed
ruff check + format on new files -> clean
```

Risks / assumptions

- Assumes rediscovery re-emerges the same clusters so upserts refresh legacy rows; patterns that never re-emerge stay flagged and need an explicit retire/accept disposition (runbook §3 caveat).
- Stale-contract detection keys off exact `CONTRACT\_ID` equality; a future contract bump automatically marks all rows stale (intended behavior).
- Audit reads whole rows; at current pattern counts (?2000) this is trivial.

Full Document:

`docs/remediation/wave4\_a\_director\_skip\_evidence\_2026\_06\_11.md`

`docs/remediation/wave4\_a\_director\_skip\_evidence\_2026\_06\_11.md` ? 78 lines, 3832 bytes, sha256 prefix `4ccd50dfacad61ed`

Heading summary: Wave4-A - Director Review Evidence: skip_rate / Non-Trade Accounting; Problem; Change; Files Changed; Tests Run; Risks; Next Step Recommended

Wave4-A - Director Review Evidence: skip_rate / Non-Trade Accounting

Date: 2026-06-11
Branch: `feat/wave4-director-skip-evidence-20260611`
Scope: gap left open by Agent J ("skip_rate" not yet surfaced in Director review evidence"). Evidence/reporting only ? no trading behavior change, no gate relaxed or tightened.

Problem

Agent J made non-trade accounting real in Research and Backtest aggregates: ``RewardRiskAnalyzer.metrics_for_rr`` reports ``signal_count`/`skipped_count`/`skip_rate`/`skip_reason_counts`` (persisted per RR level in ``pattern.rr_metrics_json``), and ``Backtester`` reports ``total_signals`/`skipped_signals`/`skip_rate``. But the Director never saw it: a pattern with strong traded expectancy could reach ``DIRECTOR_REVIEW`` (and the ``DirectorProductionGate`` packet) while 40% of its research signals were non-trades ? lower deployable capacity, invisible in the evidence.

Change

``backend/tradeo/services/director_review_gate.py`` (additive only):

- ``DirectorReviewGate._research_skip_accounting(pattern)`` locates stored skip accounting, preferring ``pattern.rr_metrics_json[best_rr]``, then other RR levels, then ``pattern.metrics_json.rr_metrics``, then backtest-shaped blocks (``metrics_json.backtest`` / ``lab_backtest`` / ``backtest_summary``, normalizing ``total_signals`/`skipped_signals`` to ``signal_count`/`skipped_count``). When nothing is stored it returns ``{"available": false, "reason": ...}`` ? it never derives or invents numbers for runs that predate non-trade accounting.
- ``lab_execution`` evidence gains ``research_skip_accounting`` (with ``source`` provenance), ``skip_rate_warning_threshold`` (default 0.25), ``research_skip_rate_warning`` and an explanatory note.
- When the warning fires, ``director_recommendations`` gains a high-priority ``review_research_skip_rate`` entry carrying ``skip_rate`/`signal_count`/`skipped_count`/`skip_reason_counts``.
- ``DirectorProductionGate.evaluate_pattern`` report includes the same ``research_skip_accounting`` block for the Director packet.
- Explicitly NOT a blocker anywhere: ``promotion_blockers`` and production ``blockers`` are unchanged for any input.

Files Changed

- ``backend/tradeo/services/director_review_gate.py``
- ``backend/tradeo/tests/test_director_review_gate.py``
- ``docs/remediation/tradeo_12_phase_compliance_matrix_2026_06_10.md``
- ``docs/remediation/wave4_a_director_skip_evidence_2026_06_11.md``

Tests Run

- New: ``test_director_review_gate_surfaces_high_research_skip_rate_evidence`` (good expectancy + 0.4 skip_rate: pattern still marked, zero blockers, evidence + warning + recommendation visible), ``test_director_review_gate_reports_missing_skip_accounting_without_inventing_data``, ``test_director_review_gate_reads_backtest_shape_skip_accounting``, ``test_director_production_gate_report_includes_research_skip_accounting``.
- ``test_director_review_gate.py``: 18 passed. Related suites (``test_effective_sample_weights.py``, ``test_research_lab_fox_lifecycle.py``): 7 passed. ``ruff check`` on touched files: clean.

Risks

- Warning threshold (0.25) is a reporting default, not calibrated; the Director can tune ``skip_rate_warning_threshold`` per instance. No runtime setting added on purpose (evidence-only phase).
- Patterns discovered before Agent J's change report ``available: false`` until rediscovery refreshes ``rr_metrics_json``; that is honest, not a regression.
- Research ``skip_rate`` (gap-entry policy at signal level) and lab paper skip semantics differ by design (Agent H); the evidence block labels its ``source`` so the Director does not conflate them.

Next Step Recommended

Once calibrated evidence accumulates, the Director may decide whether a hard ``skip_rate`` ceiling belongs in ``DirectorProductionGate`` ? out of scope here (would tighten the gate; needs Asier's sign-off).

Full Document: ``docs/remediation/wave4_d_nested_optuna_pbo_2026_06_11.md``

``docs/remediation/wave4_d_nested_optuna_pbo_2026_06_11.md`` ? 126 lines, 6578 bytes, sha256 prefix ``fd4bebf8f049fb5d``

Heading summary: Wave4-D ? Nested Optimization + Outer-Fold PBO Report (2026-06-11); Scope; What Was Built; Optuna Decision (honest); Gate Wiring; Config Knobs (defaults); Honest Limitations; Tests; Files Changed

Wave4-D ? Nested Optimization + Outer-Fold PBO Report (2026-06-11)

Scope

Closes the section-5 ImprovementAgent gap "Nested Optuna workflow + outer-fold PBO report" from ``tradeo_12_phase_final_report_2026_06_11.md``. This is a pure anti-overfit hardening: it adds a third mandatory blocker to the lab-candidate gate. It does not relax any existing threshold and cannot improve any claim.

What Was Built

``backend/tradeo/research/nested_optimization.py`` implements ``nested_optimization_report(perf)`` over the same (T periods x M variants) performance matrix the ImprovementAgent already builds from monthly realized backtest R buckets:

- **Outer folds.** Periods are split into ``n_outer_folds`` (default 5) contiguous folds.
- **Inner optimization.** For each outer fold, the inner search selects the best variant on the complement (train) periods. The ImprovementAgent's candidate space is fully enumerated by its grid, so inner optimization is a selection problem over M variants.
- **Outer evaluation + PBO.** The inner winner is ranked out-of-fold among all M variants: $\text{omega} = \text{rank}/(M+1)$, $\text{lambda} = \ln(\text{omega}/(1-\text{omega}))$ (CSCV convention, Bailey/Borwein/Lopez de Prado/Zhu 2017). ``pbo_outer`` is the fraction of outer folds with $\text{lambda} \leq 0$.
- **Pass criteria (both required).** ``pbo_outer < max_pbo`` (default 0.10, same ceiling as the existing CSCV guard) **and** median out-of-fold score of the selected variants > 0 . A stable-but-losing family cannot pass.
- **Report.** Per-fold detail (selected variant, inner/outer score, rank, lambda, OOS degradation), ``consensus_variant``, ``selection_stability``, ``selection_counts``, ``mean_oos_degradation``. Persisted via the existing self-improvement report JSON (``anti_overfit.nested``), per-candidate ``metrics_json.anti_overfit.nested``, and the lab-cycle audit log.

Optuna Decision (honest)

Optuna is **not installed** in the backend venv and was **not added** as a dependency. Rationale:

- The inner search space is a fully enumerated categorical set (grid mutations). A deterministic exhaustive scan (argmax, first-index tie-break) is strictly better than any sampler whenever the trial budget covers the space, and is bit-for-bit reproducible ? which Wave4-B made a contract.
- Adding a heavy optimizer dependency to "look more rigorous" without a larger-than-budget search space would be cargo cult, not hardening.

The Optuna interface ships behind the same contract anyway: if ``optuna`` is importable, ``self_improvement_nested_use_optuna=true``, and the variant count exceeds ``self_improvement_nested_inner_trials``, inner selection uses a seeded ``TPESampler`` (per-fold derived seed) with **pruning off** ? the objective is a cheap deterministic mean with no intermediate values, so pruning could only add nondeterminism for zero compute savings. Reports always carry ``optuna_available`` and ``inner_method`` (``exhaustive_deterministic`` or ``optuna_tpe_seeded_no_pruning``), so evidence never overstates what ran. If Optuna is installed later it must be version-pinned in ``backend/pyproject.toml`` and the skipped seeded-determinism test (``test_optuna_inner_search_is_seeded_and_deterministic``) becomes active.

Gate Wiring

- ``SelfImprovementEngine._anti_overfit_guards`` now computes the nested report once per cycle and attaches ``nested`` / ``nested_passed`` to every candidate guard; the guard summary embeds it under ``nested``.
- ``_passes_lab_gate`` now requires ``nested_passed`` **in addition to** the existing CSCV ``pbo_passed`` and ``plateau_passed``. Nothing was removed or loosened.
- Fail-closed: a blocked report (insufficient periods/variants, non-finite values) or ``self_improvement_nested_enabled=false`` yields ``nested_passed=false`` and blocks lab-candidate acceptance. Disabling the knob disables candidate creation, not the guard.

Config Knobs (defaults)

Knob	Default	Meaning
<code>`self_improvement_nested_enabled`</code>	<code>`true`</code>	Fail-closed master switch (off = block acceptance).
<code>`self_improvement_nested_outer_folds`</code>	<code>`5`</code>	Contiguous outer folds (engine floors at 4).
<code>`self_improvement_nested_inner_trials`</code>	<code>`64`</code>	Inner budget; exhaustive scan when it covers M.
<code>`self_improvement_nested_max_pbo`</code>	<code>`0.10`</code>	Outer-fold PBO ceiling (same as CSCV guard).
<code>`self_improvement_nested_seed`</code>	<code>`17`</code>	Base seed for the optional Optuna sampler.
<code>`self_improvement_nested_use_optuna`</code>	<code>`true`</code>	Use Optuna only if importable and M > budget.

Honest Limitations

- Folds are contiguous splits of monthly aggregated R buckets; no embargo/purge at this aggregation level (trade-level purging lives in ``purged_walk_forward``).
- Train is the complement of the outer fold (CSCV-style), so train can contain periods after the fold: this measures selection overfitting, not a causal walk-forward forecast, and is documented as such in the module.
- PBO is still computed from monthly realized R buckets of the lab backtest; no production claim of any kind follows from a pass.

Tests

``backend/tradeo/tests/test_nested_optimization.py`` (13 tests, 1 skipped without Optuna):

- Synthetic "specialist" overfit matrix ? each variant is best in-sample for exactly the fold where it crashes ? yields ``pbo_outer = 1.0``, worst outer rank in every fold, and is blocked.
- Persistent-edge matrix passes with ``pbo_outer = 0.0``, full selection stability, consensus variant 0.
- Stable-but-losing matrix fails the outer-median-score criterion despite ``pbo_passed``.
- Insufficient periods/variants and non-finite values block (fail-closed).
- Byte-identical reports on repeated runs (determinism contract).
- Engine integration: monthly trade records reproducing the specialist matrix block the gate; ``_passes_lab_gate`` requires all three guards; disabled knob fails closed; seeded Optuna determinism test guarded by ``importorskip``.

Full backend suite after this change: 315 passed, 1 skipped. Before the matrix append the suite carried 1 pre-existing failure ('test_remediation_docs_traceability') inherited from Wave4-C shipping 'agent_l_rediscovery_readiness_2026_06_11.md' without a compliance-matrix record; the Wave4-D matrix append records that trace, restoring green.

Files Changed

- `backend/tradeo/research/nested\_optimization.py`
- `backend/tradeo/services/self\_improvement.py`
- `backend/tradeo/core/config.py`
- `backend/tradeo/tests/test\_nested\_optimization.py`
- `docs/remediation/wave4\_d\_nested\_optuna\_pbo\_2026\_06\_11.md`
- `docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`

Full Document:

`docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md`

`docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md` ? 145 lines, 17500 bytes, sha256 prefix `cb4f70d13860bb4f`

Heading summary: Tradeo 12-Phase Compliance Matrix; Agent E Test Evidence; Agent C Test Evidence; Cross-Agent Notes; Agent G Consolidation (2026-06-11); Agent H Update (2026-06-11); Agent E2 Update (2026-06-11, recorded by audit/traceability agent); Agent I Update (2026-06-11); Agent J Update (2026-06-11); Wave4-A Update (2026-06-11); Agent K Update (2026-06-11, Wave4-B); Wave4-D Update (2026-06-11)

Tradeo 12-Phase Compliance Matrix

Date: 2026-06-10

Each agent appends its own rows without removing other branches' work.

Section	Agent	Status	Evidence	Remaining gap
3.1 Data layer	A	Partial implemented	Deterministic OHLVCV cache, metadata columns, split heuristic, provider manifest.	True incremental fetch remains blocked by current provider signature.
3.2 Universe	A	Partial implemented	Monthly snapshot service, eligibility filters, hash metadata, explicit survivorship flags, validation cap to `lab\_watchlist` when non-PIT.	Delisting/PIT vendor data still absent.
3.8 Market regimes	A	Partial implemented	Candidate regime profile exposes regime count/share and `regime\_specific` metadata; matcher consumes `preferred\_regime\_keys`.	Full SPY SMA200/vol tercile service remains pending.
7 Reproducibility (data lineage)	A	Partial implemented	Discovery summaries/candidates attach data manifest and universe snapshot lineage.	Full bit-for-bit discovery determinism still depends on broader clustering/research paths.
8 Config (data scope)	A	Implemented for this scope	Env settings for cache, adjusted feed, whatToShow, snapshots, PIT availability, and survivorship cap.	?
3.3 Representation	B	Partial implemented	Shared `PatternEmbeddingEngine.contract()` persisted from Research and Lab; targeted parity test added.	Matrix Profile/DTW/learned embeddings deferred to avoid heavy unpinned dependencies.
3.4 Clustering	B	Partial implemented	Cluster signature now records medoid, similarity distribution and concentration checks; gate rejects new concentrated clusters when diagnostics exist.	KMeans remains; HDBSCAN/noise labeling deferred.
4.1 Matching parity / no live daily bar	B	Mostly implemented	Prior patch already drops incomplete daily bars; this patch adds explicit feature parity contract to matcher output and match metrics.	Existing historical patterns need rediscovery to carry the contract.
4.2 Matcher threshold / ambiguity	B	Partial implemented	Prior patch uses per-pattern tau; this patch adds `ambiguity\_ratio`, similarity margin and second-best pattern metadata.	Ratio is audit-only until calibrated as a hard gate.
7 Audit / reproducibility	B	Partial implemented	New JSON metrics are deterministic and covered by tests; remediation report created.	Coordinator should merge all agents' reports into the final consolidated package.
3.5 Canonical outcomes	C	Partial implemented	`RewardRiskAnalyzer.\_simulate\_sample\_detail` and `Backtester.\_simulate\_exit` delegate to `triple\_barrier\_outcome`; `WindowSampler` persists `forward\_opens`.	True non-trade/skipped signal accounting should propagate beyond labels.
3.5 Cost model	C	Partial implemented	`tiered\_round\_trip\_cost\_r` added and used by backtester; Research continues to use existing execution cost metrics through canonical net R.	?
3.6 RR/N_trials accounting	C	Partial implemented	Default RR grid reduced to `2.5,4.0`; existing `real\_variant\_count`/`multiple\_testing\_trials` counts configured RR levels.	?
5 ImprovementAgent anti-overfitting	C	Partial implemented	Fixed trial budget, mandatory CSCV/PBO guard, PBO threshold, and plateau check added before lab candidate creation.	Guarded but not yet a nested Optuna workflow.
6 Backtester parity	C	Partial implemented	Backtester exit path now uses canonical `triple\_barrier\_outcome` and shared tiered costs.	ShadowTracker canonical outcome parity remains for Agent D/future work.
8 Config (outcomes scope)	C	Partial implemented	`.env.example` and `Settings` expose RR and self-improvement guard defaults.	?
4.3 Microstructure filters	D	Partial implemented	Signal snapshots and entry quality keep liquidity, ATR, volume, extension, regime, and entry-gate rejection reasons auditable.	Richer real-time microstructure feeds where available.
4.4 Sizing	D	Improved	`RiskManager` caps quantity by ADV participation and blocks excess open positions in the same pattern family.	?
4.5 Execution/reconciliation	D	Existing, verified	Reconciliation auto-kill-switch path remains enabled by default and backend suite is green.	Explicit order-state transition tests.
4.6 Shortfall	D	Improved	Director shortfall gate existed; PatternHealthMonitor now also monitors real-fill `slippage\_R` CUSUM.	?
4.7 Director sequential gate	D	Improved	Defaults now require 25 effective real paper fills, 8 symbols, and 10 days before Director review eligibility.	Persisted effective-sample weights for paper fills.
4.8 Health monitor	D	Improved	Health monitor tracks realized R decay and shortfall deterioration.	?
7 Audit reproducibility	D	Improved	Audit export separates exported event count from verified independent sample count and tests reconstruction from event rows.	?

3.1 Data layer (incremental fetch)	E	Implemented for daily artifacts	Overlap-verified tail merge inside the existing `period/interval` boundary; `refresh\_mode`/`rows\_appended` metadata; manifest exposes refresh lineage. Mismatched overlap (re-adjusted feed) forces full refetch.	Intraday intervals keep cache-forever behavior.
3.2 Universe	E	Partial implemented	Snapshot metadata adds deterministic `content\_hash` and explicit `delisting\_data\_available=false`.	Delisting/PIT vendor data still absent (honestly blocked).
3.8 Market regimes	E	Implemented (audit-grade)	`market\_regime.py`: SPY strict-SMA200 trend + trailing vol-tercile labels, PIT-stable by construction (test-covered); attached per match as `benchmark\_regime` and on matcher run output.	Not yet a hard gate in `\_pattern\_regime\_fit`; needs labeled outcome history per regime bucket to calibrate.
7 Reproducibility (data lineage)	E	Improved	Snapshot `content\_hash` excludes timestamps/paths for bit-for-bit identity; regime	

snapshots carry `source\_bar\_hash`; manifest entries record `last\_timestamp` + refresh mode. | Bit-for-bit determinism of full discovery runs still depends on clustering/research paths. |
| 8 Config (regime/incremental scope) | E | Implemented for this scope | Env knobs for incremental cache refresh (enabled/overlap/min/max gap), regime windows, benchmark symbol, delisting availability. | ? |

Agent E Test Evidence

- Full backend suite in `tradeo-backend:latest` (worktree mounted read-only): 193 passed.
- Ruff on touched files: passed.

Agent C Test Evidence

- Full backend suite: 165 passed (in Agent C worktree).
- Ruff: passed.

Cross-Agent Notes

- Agent D did not change live trading enablement.
- Remaining full compliance needs persisted effective-sample weights for paper fills, explicit order-state transition tests, and richer real-time microstructure feeds where available.

Agent G Consolidation (2026-06-11)

Section	Agent	Status	Evidence	Remaining gap
7 Audit package consolidation	G	Implemented	Final consolidated report `tradeo\_12\_phase\_final\_report\_2026\_06\_11.md` (12-section status, test ledger, merge order, honest-claims review); supersedes the 06-10 final report.	?
7 Docs traceability	G	Implemented	`test\_remediation\_docs\_traceability.py`: Files Changed references must resolve to real files; matrix must keep A?D rows; audit export must keep `event\_count`/`independent\_sample\_count`/`is\_independent\_sample`.	Validates path existence, not prose accuracy.

Status updates recorded by Agent G (rows above are wave-1/wave-2 history and were not edited in place):

- 4.5 Execution/reconciliation: "explicit order-state transition tests" gap closed by Agent F (`feat/tradeo12-execution-state-gap-20260611`, 22 tests in `test\_execution\_state\_transitions.py`), pending merge at the time of writing.
- 4.7 Director sequential gate: "persisted effective-sample weights" gap closed by Agent F (`effective\_sample.py`, weights persisted in trade metadata and pattern `lab\_execution` metrics; `n\_eff` now binding), pending merge at the time of writing.
- The Cross-Agent Notes list above predates Agent F; of its three items only "richer real-time microstructure feeds" remains open (no provider available).

Agent H Update (2026-06-11)

Section	Agent	Status	Evidence	Remaining gap
6 Backtester/Shadow parity	H	Implemented	Shadow observation exits delegate to canonical `triple\_barrier\_outcome` (open-gap stop fills at the OPEN, target gaps at the TARGET, conservative intrabar both-hit); closed trades persist a `canonical\_outcome` block. 8 parity tests in `test\_shadow\_canonical\_outcome\_parity.py`; full suite 270 passed. See `agent\_h\_shadow\_canonical\_outcome\_parity\_2026\_06\_11.md`.	Shadow costs remain 0R by design (observation-only); 3.5 non-trade accounting still open.

Status update recorded by Agent H (earlier rows unedited): the section-6 row's "ShadowTracker canonical outcome parity remains for Agent D/future work" gap is now closed on `main`.

Agent E2 Update (2026-06-11, recorded by audit/traceability agent)

Agent E2's intraday incremental cache work merged to `main` as `b83c71a` without appending its matrix rows; this section records them after the fact so the matrix matches the code on `main`. Earlier rows are unedited.

Section	Agent	Status	Evidence	Remaining gap
3.1 Data layer (intraday incremental fetch)	E2	Implemented	Intraday intervals (`1m/5m/15m/30m/1h`) now use the same overlap-verified tail append as daily, with bar-relative min-gap and wall-clock max-gap thresholds, master switch `market\_data\_incremental\_intraday\_enabled`, honest in-progress-bar exclusion in `\_bar\_complete\_mask`, and complete-bars-only persistence (also fixes the latent daily partial-bar cementing defect). 5 new tests in `test\_data\_provider.py`; image suite 258 passed. See `agent\_e2\_intraday\_incremental\_cache\_2026\_06\_11.md`.	RTH session boundaries not modeled (overnight gap math is wall-clock, harmless no-op); unknown intervals keep the permissive all-complete mask.

Status updates recorded by the audit/traceability agent (earlier rows unedited):

- 3.1 Data layer (incremental fetch): the E row's remaining gap "Intraday intervals keep cache-forever behavior" is now closed on `main` (`b83c71a`).
- Agent F merge status: the Agent G notes above describe F's work as "pending merge at the time of writing"; F is now merged on `main` (`12f2157`, via merge `646b995`), so both F items (order-state transition tests, persisted effective-sample weights) are live.

Agent I Update (2026-06-11)

Section	Agent	Status	Evidence	Remaining gap
3.8 Market regimes (outcome calibration)	I	Implemented (gate ships disabled)	Research persists `regime\_profile.benchmark\_regime\_outcomes`: per-PIT-benchmark-bucket sample_count/expectancy_r/win_rate/profit_factor from canonical triple-barrier simulation at the pattern's side/best RR. `\_pattern\_regime\_fit` uses the calibrated bucket when it has ? `market\_regime\_outcome\_min\_samples` (default 12) labeled outcomes, attaching the evidence as `regime\_fit.calibration`; calibrated-negative buckets are droppable via `market\_regime\_hard\_gate\_enabled` (default OFF), with `regime\_gate\_blocked` counts on matcher output. 11 tests in `test\_regime\_outcome\_calibration.py`; image suite 278 passed. See `agent\_i\_regime\_outcome\_calibration\_2026\_06\_11.md`.	Existing DB patterns lack calibration until rediscovery; real-fill per-regime attribution and the enable-gate decision remain Director follow-ups.

Status update recorded by Agent I (earlier rows unedited): the E row's section-3.8 remaining gap ("Not yet a hard gate in `\_pattern\_regime\_fit`; needs labeled outcome history per regime bucket to calibrate") now has the labeled outcome history and a config-gated hard gate on this branch; the gate default stays off pending calibrated real-fill evidence.

Agent J Update (2026-06-11)

```
| Section | Agent | Status | Evidence | Remaining gap |
|---|---|---|---|---|
| 3.5 Canonical outcomes (non-trade accounting) | J | Implemented | Skipped/invalid/no-data signals no longer enter Research RR aggregates as phantom `0R`: `metrics_for_rr` excludes them from all traded arrays and reports `signal_count`/'`skipped_count`/'`skip_rate`/'`skip_reason_counts`, with `sample_count` now traded-only (conservative for `min_samples` and validation-gate thresholds). `Backtester` counts detected-vs-skipped signals and `BacktestMetrics` exposes `total_signals`/'`skipped_signals`/'`skip_rate`; `_simulate_exit` filters every non-`ok` status, closing the latent `invalid`/'`no_data` NaN path. 4 new tests; full suite 286 passed. See `agent_j_non_trade_accounting_2026_06_11.md`. | `skip_rate` not yet surfaced in Director review evidence; shadow path intentionally untouched. |
```

Status update recorded by Agent J (earlier rows unedited): the C row's section-3.5 remaining gap ("True non-trade/skipped signal accounting should propagate beyond labels") and the H row's note ("3.5 non-trade accounting still open") are closed on this branch.

Wave4-A Update (2026-06-11)

```
| Section | Agent | Status | Evidence | Remaining gap |
|---|---|---|---|---|
| 3.5 Canonical outcomes (Director skip-rate evidence) | Wave4-A | Implemented (evidence only) | `DirectorReviewGate` now surfaces stored non-trade accounting as Director evidence: `lab_execution.research_skip_accounting` (with `source` provenance from `pattern_rr_metrics_json[best_rr]`, other RR levels, `metrics_json.rr_metrics`, or backtest-shaped `total_signals`/'`skipped_signals` blocks), `research_skip_rate_warning` at threshold 0.25, and a high-priority `review_research_skip_rate` director recommendation carrying `skip_rate`/'`signal_count`/'`skipped_count`/'`skip_reason_counts`. `DirectorProductionGate.evaluate_pattern` includes the same block in its packet. Never a blocker; returns `available: false` (no invented numbers) when the research run predates non-trade accounting. 4 new tests; gate suite 18 passed. See `wave4_a_director_skip_evidence_2026_06_11.md`. | Whether a hard `skip_rate` ceiling belongs in `DirectorProductionGate` is a Director/Asier decision once calibrated evidence accumulates. |
```

Status update recorded by Wave4-A (earlier rows unedited): the J row's remaining gap ("`skip\_rate` not yet surfaced in Director review evidence") is closed on this branch.

Agent K Update (2026-06-11, Wave4-B)

```
| Section | Agent | Status | Evidence | Remaining gap |
|---|---|---|---|---|
| 3.4 Clustering / reproducibility (discovery determinism) | K | Implemented (in-process contract) | Discovery output is bit-for-bit reproducible for identical inputs/config/seed on the same image: new `research/determinism.py` provides canonical JSON + `sha256_canonical_json_v1` content hash excluding volatile run metadata (timestamps, `run_id`, artifact paths); discovery report JSON now dumps `sort_keys=True` and carries a `determinism.content_hash` block; `autonomous_research_director` memory-graph enumeration and pattern query get deterministic sort/tiebreakers. Harness `test_discovery_determinism.py` runs the real engine twice over seeded synthetic fixtures and asserts byte-identical canonical payloads, input-order independence, and run_id-stable report hashes. 6 new tests; image suite 288 passed, 4 skipped. See `agent_k_discovery_determinism_2026_06_11.md`. | Cross-environment (sklearn/BLAS/hardware) bit-equality not claimed; full live-data end-to-end double-run out of scope (run-context excluded from identity hash); `tests/fixtures.py` `fixture_ohlcw` stays PYTHONHASHSEED-sensitive (pre-existing, test-data only). |
```

Wave4-D Update (2026-06-11)

```
| Section | Agent | Status | Evidence | Remaining gap |
|---|---|---|---|---|
| 5 ImprovementAgent anti-overfitting (nested optimization + outer-fold PBO) | Wave4-D | Implemented (hardening only) | New `research/nested_optimization.py`: contiguous outer folds over the monthly realized-R performance matrix, inner selection on the train complement (deterministic exhaustive scan; seeded Optuna TPE with pruning off only if installed and M > inner budget ? Optuna is NOT installed and was not added), outer-fold ranks give omega/lambda and `pbo_outer`; pass requires `pbo_outer < 0.10` AND median out-of-fold selected score > 0. `_passes_lab_gate` now requires `nested_passed` in addition to CSCV `pbo_passed` and `plateau_passed` (fail-closed on blocked reports or disabled knob); report persisted in `anti_overfit.nested` (report JSON, candidate `metrics_json`, audit log) with `optuna_available`/'`inner_method` so evidence never overstates what ran. 13 new tests (1 skipped without Optuna) incl. synthetic specialist-overfit matrix blocked at `pbo_outer=1.0`; full suite 315 passed, 1 skipped. See `wave4_d_nested_optuna_pbo_2026_06_11.md`. | True Optuna path inactive until a justified pinned dependency lands (skipped seeded test ready); folds are monthly-bucket contiguous, no embargo at this aggregation level; train is CSCV-style complement, not causal walk-forward. |
```

Traceability note recorded by Wave4-D (earlier rows unedited): Agent L (Wave4-C, `agent\_l\_rediscovery\_readiness\_2026\_06\_11.md`, rediscovery readiness audit/flags/manifest) merged to main without a matrix record; this note records that trace so the docs-traceability test holds.

Full Document: `docs/remediation/tradeo\_12\_phase\_final\_report\_2026\_06\_10.md`

`docs/remediation/tradeo\_12\_phase\_final\_report\_2026\_06\_10.md` ? 41 lines, 2929 bytes, sha256 prefix `ca4163a875e0d711`

Heading summary: Tradeo 12-Phase Remediation ? Final Consolidated Report; Merged branches (in order); Coordinator fixes; Verification; Section status; Remaining gaps (next wave)

Tradeo 12-Phase Remediation ? Final Consolidated Report

Date: 2026-06-11 (consolidation of work dated 2026-06-10)
Coordinator: Fable director session (tradeo12-fable-high-director)
Base: `main` @ `9cc7ccb` ? final `main` @ merge of all four agent branches.

Merged branches (in order)

1. `agent-a-data-universe-repro-fallback` (`4a33dfc`, merge `b81d4bc`) ? data layer cache/manifest, universe snapshots, survivorship flags, regime metadata, lineage gates.
2. `agent-b-representation-parity` (`39f11a5`, merge `04c1ee1`) ? embedding contract parity Research?Lab, medoid/concentration diagnostics, matcher ambiguity ratio.
3. `agent-c-outcomes-costs-backtester` (`8d98c11`, merge `e8bfc64`) ? canonical `triple\_barrier\_outcome` in RewardRiskAnalyzer and Backtester, tiered cost model, RR grid `2.5,4.0`, ImprovementAgent CSCV/PBO guards.
4. `remediation/agent-d-fallback` (`24bf0de`, merge `df418fb`) ? ADV participation cap, family caps, shortfall CUSUM in PatternHealthMonitor, stricter Director sequential gate (25 fills/8 symbols/10 days), honest audit export counts.


```
All conflicts were limited to `docs/remediation/tradeo_12_phase_compliance_matrix_2026_06_10.md` (add/add) and were resolved by combining every agent's rows into one table; no code was dropped.

## Coordinator fixes

- `backend/tradeo/tests/conftest.py` (new): points `TRADEO_MARKET_DATA_CACHE_DIR`, `TRADEO_UNIVERSE_SNAPSHOT_DIR`, `TRADEO_REPORTS_DIR`, `TRADEO_ARTIFACTS_DIR` at a temp dir during tests. Agent A's `market_data_cache_path` property makedirs `/app/...` on access, which fails outside Docker and broke 8 tests locally; defaults still apply inside the image.

## Verification

- `pytest`: 177 passed (was 161 before this remediation wave).
- `ruff check .`: clean.
- Frontend: untouched by all four branches; no rebuild needed.
- Docker: backend + worker images rebuilt and restarted; both healthy.
- API health: `{ "ok": true, "mode": "paper", "live_armed": false, "kill_switch_enabled": false }`.
- `env`: `TRADEO_LIVE_TRADING_ENABLED=false`, lab scanner paper-only with auto-submit paper orders.

## Section status

See `tradeo_12_phase_compliance_matrix_2026_06_10.md` for the per-section matrix. Summary: sections 3.1?3.6, 3.8, 4.1?4.8, 5, 6, 7, 8 are now partially-to-mostly implemented with tests; nothing claims full compliance where vendor data (PIT/delistings) or heavy deps (Matrix Profile/HDBSCAN) are missing.

## Remaining gaps (next wave)

- PIT/delisting vendor data ? lift survivorship cap from `lab_watchlist`.
- Full SPY SMA200/vol-tercile regime service.
- Rediscovery run so historical patterns carry the new embedding contract.
- Calibrate `ambiguity_ratio` into a hard gate.
- Nested Optuna workflow in ImprovementAgent (currently budget+PBO guarded only).
- Persisted effective-sample weights for paper fills; explicit order-state transition tests.
- Non-trade/skipped signal accounting beyond labels.
```

Full Document: `docs/remediation/tradeo\_12\_phase\_final\_report\_2026\_06\_11.md`

`docs/remediation/tradeo\_12\_phase\_final\_report\_2026\_06\_11.md` ? 224 lines, 14689 bytes, sha256 prefix `e94054d61a408bbe`

Heading summary: Tradeo 12-Phase Remediation ? Final Audit Package (2026-06-11); Branch state at time of writing; Recommended merge order (for the coordinator); The 12 sections ? consolidated status; Test evidence ledger; Honest-claims review; Remaining real gaps (next wave); Audit-package traceability artifacts; Safety invariants (unchanged across all waves); Wave-3 Addendum (2026-06-11, Agents I/J); Merge state at time of writing; Section status changes vs the table above

```
# Tradeo 12-Phase Remediation ? Final Audit Package (2026-06-11)

Date: 2026-06-11
Author: Agent G (audit/final-package/compliance-traceability phase)
Branch: `feat/tradeo12-audit-final-gap-20260611`
Base: local `main` @ `df418fb` (agents A?D merged)

This package supersedes `tradeo_12_phase_final_report_2026_06_10.md` (commit `f648475`, on `main`), which consolidated only the first wave (agents A?D).
This report consolidates both waves:

- Wave 1 (2026-06-10): agents A?D ? merged into local `main` at `df418fb`.
- Wave 2 (2026-06-11): agent E (merged into `main` at `2f0f604`), agent F (pending merge), agent G (this branch, docs + traceability tests only).

## Branch state at time of writing

| Branch | Tip | Content | Merge state |
|---|---|---|---|
| `main` (local) | `2f0f604` | A?D + 06-10 final report + Agent E | ahead of `origin/main`, not pushed |
| `feat/tradeo12-data-regime-gap-20260611` | `83ccd97` | Agent E (SPY regime service, incremental cache, snapshot hash) | merged into `main` |
| `feat/tradeo12-execution-state-gap-20260611` | `12f2157` | Agent F (order-state tests, effective-sample weights) | NOT merged |
| `feat/tradeo12-audit-final-gap-20260611` | this branch | Agent G (this package, traceability tests) | NOT merged |

## Recommended merge order (for the coordinator)

1. Merge `feat/tradeo12-execution-state-gap-20260611` (Agent F). Code + tests; its branch reports 210 passed, ruff clean. It modifies `director_review_gate.py`, which no other unmerged branch touches.
2. Merge `feat/tradeo12-audit-final-gap-20260611` (Agent G). Docs plus one additive test module. Expected conflict surface: only `tradeo_12_phase_compliance_matrix_2026_06_10.md` (Agent E appended rows on `main`; Agent G appends a separate section at end of file ? an add/add region that should auto-merge or resolve by keeping both).
3. Run the full backend suite and ruff on merged `main`; rebuild backend/worker images, restart, verify API health, then push.

## The 12 sections ? consolidated status

Section numbering follows the external report as used by the compliance matrix: seven data/research subsections (3.1?3.6, 3.8), the execution loop as one section (4.1?4.8), then 5, 6, 7, 8 ? twelve sections total. Detailed per-agent rows live in `tradeo_12_phase_compliance_matrix_2026_06_10.md`.

| # | Section | Final status | Key evidence (code / tests) | Real remaining gap |
|---|---|---|---|---|
| 1 | 3.1 Data layer | Implemented for daily artifacts | Deterministic OHLCV cache + manifest (A); overlap-verified incremental tail
```

merge, `refresh\_mode`/`rows\_appended` lineage (E). `test\_data\_provider.py`. | Intraday intervals keep cache-forever; CSV (not Parquet) is the canonical artifact; IBKR `ADJUSTED\_LAST` not verified against a live gateway. |

| 2 | 3.2 Universe | Partial | Monthly snapshots, eligibility filters, hash + `content\_hash`, explicit `point\_in\_time`/`survivorship\_biased`/`delisting\_data\_available` flags; survivorship cap to `lab\_watchlist` (A, E). | PIT/delisting vendor data absent ? honestly blocked; cap stays until vendor data exists. |

| 3 | 3.3 Representation | Partial | Shared `PatternEmbeddingEngine.contract()` persisted from Research and Lab; parity test (B). | Matrix Profile/DTW/learned embeddings deferred; historical patterns need rediscovery to carry the contract. |

| 4 | 3.4 Clustering | Partial | Medoid/signature, similarity distribution, concentration diagnostics; gate rejects concentrated new clusters (B). | KMeans remains; HDBSCAN/noise labeling deferred. |

| 5 | 3.5 Outcomes & costs | Partial | Canonical `triple\_barrier\_outcome` in `RewardRiskAnalyzer` and `Backtester`; `tiered\_round\_trip\_cost\_r` shared cost model (C). `test\_agent\_c\_remediation.py`, `test\_reward\_risk\_analyzer.py`. | Skipped entries still appear as `0.0R` in aggregate arrays; true non-trade accounting beyond labels pending. |

| 6 | 3.6 RR / N-trials | Implemented for scope | RR grid reduced to a priori `2.5,4.0`; `real\_variant\_count`/`multiple\_testing\_trials` count the configured grid (C). | ? |

| 7 | 3.8 Market regimes | Implemented (audit-grade) | `market\_regime.py`: SPY strict-SMA200 trend + trailing vol terciles, PIT-stable labels, `insufficient\_history` honesty, `source\_bar\_hash` lineage; attached additionally to matcher output (E). `test\_market\_regime.py`. | Not yet a hard gate; calibration needs labeled outcome history per regime bucket. |

| 8 | 4.1?4.8 Execution loop & Director | Mostly implemented | No live daily bar + parity contract (B); microstructure auditability (D); ADV participation cap + pattern-family cap (D); order-state transition tests, 22 tests (F); shortfall CUSUM in `PatternHealthMonitor` (D); sequential gate 25 eff. fills / 8 symbols / 10 days with persisted per-fill effective-sample weights, `n\_eff` binding (D, F). | Richer real-time microstructure feeds (4.3) have no available provider; `ambiguity\_ratio` (4.2) recorded but not calibrated as a hard gate. |

| 9 | 5 ImprovementAgent | Partial | Fixed trial budget, mandatory CSCV/PBO (`PBO < 0.10`), plateau check before lab candidates (C). | Still grid search, not nested Optuna; PBO from monthly realized R buckets, no outer-fold report. |

| 10 | 6 Backtester parity | Partial | Backtester exits reuse canonical outcome + shared tiered costs (C). | ShadowTracker canonical-outcome parity not done (F covered shadow `state transitions`, not outcome math). |

| 11 | 7 Reproducibility / audit | Improved | Data manifest + universe snapshot lineage on discovery runs (A); deterministic JSON metrics (B); audit export separates `event\_count` from `independent\_sample\_count` and tests reconstruction from event rows (D); effective-sample weights persisted twice ? `trade.metadata.json.effective\_sample` and pattern `metrics.json.lab\_execution` (F); docs-traceability tests (G). | Bit-for-bit determinism of full discovery runs still depends on clustering/research paths. |

| 12 | 8 Config | Implemented for touched scopes | Env/Settings knobs for cache, snapshots, PIT/survivorship, RR levels, anti-overfit guards, regime windows, incremental refresh (A, C, E). | ? |

Test evidence ledger

Counts are full-backend-suite results in each branch's verification environment, as recorded in the per-agent reports:

```
| Wave | Where | Result |
|---|---|---|
| A | targeted tests in Docker dependency layer | 21 passed (targeted) |
| B | targeted tests in worktree venv | 40 passed (targeted) |
| C | full suite in worktree venv | 165 passed |
| D | full suite in worktree venv | 166 passed |
| A?D merged | `main` after coordinator conf test fix | 177 passed, ruff clean |
| E | full suite in `tradeo-backend:latest`, worktree mounted R0 | 193 passed |
| F | full suite in worktree | 210 passed, ruff clean |
| G | traceability module (this branch) | see `agent_g_audit_final_gap_2026_06_11.md` |
```

Final authoritative number must be re-established by the coordinator on merged `main` (expected 2210 + 4 new traceability tests when docs are visible; the traceability tests skip inside the Docker image because `docs/` is not copied into it).

Honest-claims review

Checked all remediation docs for overstated claims. Findings:

- The 06-10 final report's "Remaining gaps (next wave)" list is now stale, not wrong: full SPY regime service, incremental fetch (daily), persisted effective-sample weights, and order-state transition tests are closed by E and F. This report supersedes that list; the 06-10 file is kept as a historical record of wave 1.
- Compliance-matrix "Cross-Agent Notes" cite the same now-closed gaps; an Agent G consolidation note in the matrix records the closure instead of rewriting other agents' rows.
- Agent E's "3.8 Implemented (audit-grade)" is accurate as scoped: the regime service is lineage/audit metadata, explicitly not a trading gate.
- No document claims demonstrated edge, live-trading readiness, or full compliance for sections blocked on vendor data (PIT/delisting) or heavy dependencies (Matrix Profile, HDBSCAN) ? claims are consistent with code.

Remaining real gaps (next wave)

1. PIT/delisting vendor data ? lift the survivorship cap from `lab\_watchlist` (3.2). Blocked on data, not code.
2. Intraday incremental cache refresh (3.1; daily-only today).
3. Rediscovery run so historical patterns carry the embedding contract, cluster signature and concentration metadata (3.3/3.4).
4. Calibrate `ambiguity\_ratio` and `benchmark\_regime` into hard gates once labeled outcome history per bucket exists (4.2, 3.8).
5. Nested Optuna workflow + outer-fold PBO report in ImprovementAgent (5).
6. ShadowTracker canonical-outcome parity (6).
7. True non-trade/skipped-signal accounting through all aggregate metrics (3.5).
8. Richer real-time microstructure feeds where a provider becomes available (4.3).
9. Bit-for-bit discovery determinism across clustering/research paths (7).

Audit-package traceability artifacts

- `docs/remediation/agent\_{a.g}\_\*.md` ? per-phase reports with Files Changed and Tests Run sections.
- `docs/remediation/tradeo\_12\_phase\_compliance\_matrix\_2026\_06\_10.md` ? per-section, per-agent status rows (append-only by convention).
- `backend/tradeo/tests/test\_remediation\_docs\_traceability.py` (G) ?

```

asserts every "Files Changed" reference resolves to a real file, the
matrix keeps rows for agents A?D, and the audit export retains the
honest-count fields (`event_count`, `independent_sample_count`,
`is_independent_sample`).
- `research/audit_bridge/` ? export/validate contract unchanged in wave 2
except Agent D's honest-count separation, covered by
`test_audit_hardening.py`.

## Safety invariants (unchanged across all waves)

- `TRADEO_LIVE_TRADING_ENABLED=false`; paper-first defaults intact.
- No order routing or broker configuration changed by E, F or G.
- Director production gate remains separate from review gate; no document
or code claims production approval from the 10-trade review trigger.

## Wave-3 Addendum (2026-06-11, Agents I/J)

Recorded after the wave-3 merges; the report body above is kept verbatim as
the wave-1/wave-2 historical record and this addendum supersedes its stale
claims. Earlier sections were not edited in place, per the append-only
convention used in the compliance matrix.

### Merge state at time of writing

Wave-3 implementation branches reviewed at local `main` tip `b46a684`;
Agent J then closes the non-trade/skipped-signal accounting gap on top of
that base.

| Agent | Content | Merged at |
|---|---|---|
| F | Order-state transition tests, persisted effective-sample weights | `12f2157` via merge `646b995` |
| G | Final report 06-11, docs-traceability test module | merged on `main` (test module live, extended at `0b1e258`) |
| H | Shadow observation exits via canonical `triple_barrier_outcome` | `7375a95` |
| E2 | Intraday incremental OHLCV cache refresh | `b83c71a` |
| I | Benchmark-regime outcome calibration + optional hard gate | `9044fb6` via merge `b46a684` |
| J | True non-trade/skipped-signal accounting in RR and backtest aggregates | `9acafdc` |

### Section status changes vs the table above

- Section 1 (3.1 Data layer): "Intraday intervals keep cache-forever" is
closed by Agent E2 ? intraday intervals (`1m/5m/15m/30m/1h`) reuse the
overlap-verified tail append, persist complete bars only, and the latent
daily partial-bar cementing defect is fixed. Status: Implemented for
daily and intraday cache artifacts. Remaining there: RTH session
boundaries not modeled; CSV canonical artifact and live IBKR
`ADJUSTED_LAST` verification unchanged.
- Section 7 (3.8 Market regimes): Agent I adds per-PIT-bucket labeled
outcome history (`benchmark_regime_outcomes`) from canonical
triple-barrier simulation and a config-gated hard gate
(`market_regime_hard_gate_enabled`, default OFF; min-samples default 12).
Status: Implemented (gate ships disabled). Remaining: rediscovery for
existing DB patterns, real-fill per-regime attribution, Director
enable-gate decision.
- Section 10 (6 Backtester parity): "ShadowTracker canonical-outcome
parity not done" is closed by Agent H ? shadow observation exits
delegate to canonical `triple_barrier_outcome`; closed trades persist a
`canonical_outcome` block. Status: Implemented. Remaining: shadow costs
stay 0R by design (observation-only).
- Gap 3.5 (canonical outcomes / non-trade accounting): Agent J removes
phantom `0R` aggregation for skipped/invalid/no-data signals in Research
RR metrics, adds explicit `signal_count`/`skipped_count`/`skip_rate` and
`skip_reason_counts`, and exposes detected-vs-skipped signal counts in
`BacktestMetrics`. Status: Implemented. Remaining: surface `skip_rate`
in Director review evidence.

### "Remaining real gaps (next wave)" list ? wave-3 status

| # | Gap | Status after wave 3 |
|---|---|---|
| 1 | PIT/delisting vendor data | Open (blocked on data) |
| 2 | Intraday incremental cache refresh | Closed (E2, `b83c71a`) |
| 3 | Rediscovery for embedding contract/cluster metadata | Open (now also needed for regime calibration) |
| 4 | Calibrate `ambiguity_ratio` + `benchmark_regime` as hard gates | Half closed: regime gate calibrated and shipped disabled (I,
`9044fb6`); `ambiguity_ratio` still audit-only |
| 5 | Nested Optuna + outer-fold PBO | Open |
| 6 | ShadowTracker canonical-outcome parity | Closed (H, `7375a95`) |
| 7 | Non-trade/skipped-signal accounting | Closed (J, `9acafdc`) |
| 8 | Real-time microstructure feeds | Open (no provider) |
| 9 | Bit-for-bit discovery determinism | Open |

### Test evidence ledger ? wave-3 additions

| Wave | Where | Result | |
|---|---|---|---|
| H | full suite in `tradeo-backend:latest` | 270 passed |
| E2 | full suite in `tradeo-backend:latest` | 258 passed |
| I | full suite in `tradeo-backend:latest` | 278 passed |
| J | Fable full backend suite | 286 passed; `ruff check` clean |
| J | local review | rebuilt `tradeo-backend:latest`; targeted analyzer/backtester tests + ruff | 10 passed; `ruff check tradeo` clean |

### Traceability hardening in this phase

The compliance matrix now carries explicit Agent I and Agent J rows, and
this addendum mirrors those rows so the consolidated final report no
longer lags the merged implementation waves.

```

Safety invariants (re-checked for wave 3)

- `TRADEO\_LIVE\_TRADING\_ENABLED=false` unchanged; H and E2 touch no order routing; Agent I's only gate ships disabled by default (`market\_regime\_hard\_gate\_enabled=false`).
- Agent J changes aggregate accounting and backtest metrics only; no order routing, broker configuration, sizing, or live-trading flags changed.

Audit Bridge Full Text Appendix

Audit bridge contracts, skills, exporter, validator, Director gate, schedules and pending tasks.

Full Document: `research/audit\_bridge/README.md`

`research/audit\_bridge/README.md` ? 116 lines, 4900 bytes, sha256 prefix `424e21edebc6d9d2`

Heading summary: Research Audit Bridge; Crear un nuevo paquete; Auditoria automatizada robusta; Datos obligatorios; Datos que nunca deben subirse; Reproducibilidad; Costes

Research Audit Bridge

`research/audit\_bridge/` es el nexo comun entre Claw y ChatGPT para auditar el laboratorio de research de trading de Tradeo.

La carpeta define un contrato estable: Claw exporta paquetes reproducibles con patrones, eventos, trades, fills, configuracion y linaje de datos; ChatGPT los revisa con el mismo formato cada vez y puede comparar calidad estadistica, sesgos, bugs del laboratorio, data leakage, lookahead bias, costes, ejecucion y robustez.

Crear un nuevo paquete

Cada auditoria vive en:

research/audit_bridge/requests/<audit_id>/

El `audit\_id` debe ser estable y legible, por ejemplo:

2026-06-07_ib_paper_patterns

Flujo recomendado:

1. Crear una rama dedicada, nunca trabajar directo sobre `main`.
2. Instalar dependencias del puente si hiciera falta: `python3 -m pip install -r research/audit\_bridge/requirements.txt`.
3. Regenerar los CSV y documentos con `export\_audit\_package.py` o un exportador equivalente documentado.
4. Ejecutar `validate\_audit\_package.py`.
5. Revisar que no haya datos sensibles.
6. Commit del paquete completo o, si es demasiado grande, commit de muestra representativa, manifest con hashes e instrucciones exactas de regeneracion.

Auditoria automatizada robusta

El entrypoint recomendado para auditorias manuales, diarias y semanales es:

python research/audit_bridge/run_director_audit.py --cadence manual --audit-id <audit_id>

El runner siempre intenta escribir artefactos locales en:

research/audit_bridge/requests/<audit_id>/

Como minimo deja `director\_audit\_run.json`, `director\_audit\_run.md`, `internal\_auditor\_agent\_review.json` e `internal\_auditor\_agent\_review.md` incluso si export, gate, validacion o entrega externa fallan antes de completar el ciclo.

La comprobacion de runtime Docker es informativa y no bloqueante. Usa `docker compose ps --format json` y cae a `docker compose ps` si la salida JSON no esta disponible. No depende de nombres fijos como `tradeo-backend` o `tradeo-worker`; el campo estable es el servicio Compose (`backend`, `worker`, `db`, `redis`, `frontend`). Si hay que usar compose files no estandar, definir:

TRADEO_AUDIT_COMPOSE_FILES=docker-compose.yml:docker-compose.audit.yml \
python research/audit_bridge/run_director_audit.py --cadence daily

Datos obligatorios

Todo paquete debe incluir, como minimo:

- `manifest.json` con metadatos legibles por maquina.
- `AUDIT\_REQUEST.md` con resumen humano.
- `pattern\_catalog.csv`, una fila por patron detectado.
- `pattern\_events.csv`, una fila por ocurrencia independiente o por evento representativo exportado si el laboratorio aun no conserva

todos los eventos.

- `paper\_trades.csv`, una fila por trade paper generado por patrones.
- `ib\_fills.csv`, una fila por fill de IB Paper, anonimizado.
- `experiment\_registry.csv`, todas las variantes probadas, incluidas las descartadas.
- `metrics\_by\_pattern.csv`, `metrics\_by\_ticker.csv`, `metrics\_by\_period.csv`.
- `metrics\_by\_regime.csv` y `metrics\_by\_entry\_variant.csv`, con rendimiento por bucket si hay closed paper trades o `empty\_reason` si no hay datos.
- `director\_gate\_result.json`, generado antes de la validacion estricta del paquete.
- `code\_references.md`.
- `config\_snapshot/` con configuracion redacted usada para generar el paquete.
- `data\_lineage.md`, `known\_issues.md`, `audit\_checklist.md`, `chatgpt\_questions.md`, `reproducibility.md`.

Datos que nunca deben subirse

No subir nunca:

- Claves, tokens, secrets, cookies, session IDs, passwords o credenciales.
- Account IDs reales de IB o identificadores de cuenta equivalentes.
- Informacion personal.
- Rutas locales sensibles.
- Hostnames privados si revelan infraestructura sensible.
- Order IDs o execution IDs sin hash.

Los tickers, fechas de mercado, resultados agregados y reglas de patron pueden mantenerse si no revelan datos sensibles, porque ChatGPT los necesita para auditar robustez.

Reproducibilidad

Los exports deben ser reproducibles:

- Documentar comando exacto, rama, commit y entorno.
- Registrar fuente de datos y timeframes.
- Mantener hashes de archivos de datos cuando aplique.
- Indicar si el paquete es snapshot transaccional o export de API en caliente.

Todos los timestamps deben indicar zona horaria. Si el sistema solo guarda fechas diarias sin zona, el paquete debe normalizarlas y documentarlo en `data\_lineage.md` o `known\_issues.md`.

Costes

Los costes deben mantenerse separados:

- `commission`
- `estimated\_spread\_cost`
- `estimated\_slippage`
- `other\_fees`

`net\_pnl` debe calcularse como:

gross_pnl - commission - estimated_spread_cost - estimated_slippage - other_fees

Si no hay trades paper o fills, los CSV correspondientes deben existir con cabecera y sin filas. Los breakdowns por regimen y entry variant deben seguir existiendo y explicar la ausencia de `closed\_lab\_trades`.

Full Document: `research/audit\_bridge/AUDIT\_CONTRACT.md`

`research/audit\_bridge/AUDIT\_CONTRACT.md` ? 214 lines, 7807 bytes, sha256 prefix `75c401506df75319`

Heading summary: Audit Contract; A. Objetivo De Auditoria; B. Preguntas Que Debe Responder La Auditoria; C. Requisitos Minimos De Datos; D. Estructura De Archivos Esperada; E. Definicion De Patron; F. Definicion De Muestra Independiente; G. Reglas De Entrada Y Salida; H. Costes Y Ejecucion; I. Control De Sesgos; J. Separacion In-Sample / Out-Of-Sample / Paper-Live; K. Metricas Obligatorias

Audit Contract

A. Objetivo De Auditoria

Validar si los patrones detectados por el laboratorio de research de Tradeo tienen una ventaja reproducible y operable con datos reales de Interactive Brokers Paper, sin sesgos estadisticos, sin data leakage y con costes de ejecucion separados.

La auditoria no aprueba trading live. Solo clasifica patrones como aptos para seguir investigando, refinar, congelar o descartar.

B. Preguntas Que Debe Responder La Auditoria

- El patron esta definido de forma objetiva y reproducible?
- La entrada usa solo datos disponibles antes o en el momento de la senal?
- La salida, stop, take profit y time stop estan definidos antes del trade?
- El EV neto sigue siendo positivo tras costes?
- El resultado se mantiene fuera de muestra?
- El resultado aparece en varios tickers, fechas y regimenes?
- El resultado depende de pocos trades extremos?
- Hay duplicacion accidental de muestras?
- Hay lookahead bias, survivorship bias o data leakage?
- El patron merece pasar a mas paper-live controlado, requiere refinamiento o debe descartarse?

C. Requisitos Minimos De Datos

Cada paquete debe contener:

- Catalogo de patrones con reglas humanas.
- Eventos o muestras independientes con timestamps y features usadas.

- Trades paper, incluidos ganadores, perdedores, flat, cancelados, parciales y abiertos.
- Fills IB Paper anonimizados o archivo vacio con cabecera si no existen.
- Registro de experimentos con variantes descartadas.
- Metricas por patron, ticker, periodo, regimen y entry variant.
- Configuracion usada para detectar, evaluar y ejecutar.
- Linaje de datos y problemas conocidos.

D. Estructura De Archivos Esperada

```

research/audit_bridge/
  README.md
  AUDIT_CONTRACT.md
  validate_audit_package.py
  requirements.txt
  export_audit_package.py
  requests/<audit_id>/
    AUDIT_REQUEST.md
    manifest.json
    pattern_catalog.csv
    pattern_events.csv
    paper_trades.csv
    ib_fills.csv
    experiment_registry.csv
    metrics_by_pattern.csv
    metrics_by_ticker.csv
    metrics_by_period.csv
    metrics_by_regime.csv
    metrics_by_entry_variant.csv
    director_gate_result.json
    code_references.md
    config_snapshot/
      detector_config.json
      universe_config.json
      risk_config.json
      execution_config.json
      ib_paper_config.redacted.json
      data_config.json
    data_lineage.md
    known_issues.md
    audit_checklist.md
    chatgpt_questions.md
    reproducibility.md

```

E. Definicion De Patron

Un patron es una regla reproducible que transforma datos de mercado disponibles en una senal potencial. Debe tener:

- Identificador estable.
- Version.
- Descripcion.
- Hipotesis.
- Regla de deteccion.
- Regla de entrada.
- Regla de salida.
- Stop.
- Take profit.
- Time stop.
- Timeframe, universo y filtros.
- Referencias al codigo que lo genera.

Un cluster estadistico no es suficiente si no puede describirse y regenerarse con reglas claras.

F. Definicion De Muestra Independiente

Una muestra independiente es una ocurrencia que puede contar para estadistica sin duplicar informacion casi identica. Debe indicar:

- `event\_id` unico.
- `duplicate\_group\_id` para agrupar senales repetidas o muy cercanas.
- `is\_independent\_sample`.
- Ticker, fecha, timeframe y features disponibles en el momento.

Ventanas solapadas, senales repetidas del mismo ticker o multiples variantes del mismo detector deben tratarse con cuidado y agruparse antes de contar edge estadistico.

G. Reglas De Entrada Y Salida

La auditoria debe verificar que:

- La entrada se define antes del resultado futuro.
- El stop se define antes o en el momento de la entrada.
- El take profit se define antes o en el momento de la entrada.
- El time stop se define antes o en el momento de la entrada.

- La salida no depende de datos futuros no disponibles.
- Las ordenes paper reflejan el tipo de orden y precio realista.

H. Costes Y Ejecucion

Los costes deben auditarse separados:

- Comision.
- Spread estimado.
- Slippage estimado.
- Otros fees.

No se acepta mezclar costes en un unico ajuste opaco. La auditoria debe recalcular `net\_pnl` y revisar si el patron sigue positivo con costes mas conservadores.

I. Control De Sesgos

La auditoria debe comprobar:

- No aceptar patrones solo por winrate.
- Verificar EV neto positivo despues de costes.
- Verificar profit factor.
- Verificar drawdown.
- Verificar si el resultado depende de pocos trades extremos.
- Verificar distribucion por ticker.
- Verificar distribucion por fecha.
- Verificar regimen de mercado.
- Verificar numero de variantes probadas, incluidas las descartadas.
- Verificar que no hay lookahead bias.
- Verificar que no hay survivorship bias.
- Verificar que no hay duplicacion accidental de muestras.
- Verificar que los datos usados para detectar el patron no son posteriores a la entrada.
- Verificar que los datos de IB Paper estan separados de backtests historicos.

J. Separacion In-Sample / Out-Of-Sample / Paper-Live

Cada experimento debe distinguir:

- `in\_sample\_start` y `in\_sample\_end`.
- `out\_of\_sample\_start` y `out\_of\_sample\_end`.
- `paper\_live\_start` y `paper\_live\_end`.

La deteccion historica y el paper-live no deben mezclarse. Un patron puede tener buen backtest y aun asi fallar en paper-live.

K. Metricas Obligatorias

Metricas minimas:

- Sample count e independent sample count.
- Trade count.
- Ticker count y sector count.
- Gross PnL y net PnL.
- Avg trade, median trade y std trade.
- Winrate, avg win, avg loss, payoff ratio.
- Profit factor.
- Expectancy.
- Max drawdown y max consecutive losses.
- Best/worst trade.
- PnL sin mejor trade y sin top 5 trades.
- Sharpe, Sortino y Calmar si hay suficientes trades.
- Distribucion por ticker.
- Distribucion por periodo.
- Distribucion por regimen con rendimiento separado cuando haya `closed\_lab\_trades`; si no, razon explicita.
- Distribucion por `entry\_variant` con rendimiento separado cuando haya `closed\_lab\_trades`; si no, razon explicita.
- Separacion de costes.

L. Criterios De Aprobacion, Refinamiento O Descarte

Un patron no debe aprobarse si solo gana por winrate. Para pasar a paper controlado debe, como minimo:

- Tener EV neto positivo despues de costes realistas.
- Tener profit factor suficiente y estable.
- Tener drawdown aceptable.
- No depender de uno o pocos trades extremos.
- No concentrarse en pocos tickers, fechas o un unico regimen.
- Tener suficientes muestras independientes.
- Tener variantes descartadas documentadas.
- Pasar controles de lookahead, survivorship y duplicacion.
- Tener reglas de entrada/salida reproducibles.
- Tener datos IB Paper separados de backtests historicos.

Estados esperados:

- `approved\_for\_more\_paper`: puede seguir en paper controlado, no live.
- `refine`: requiere mas datos, filtros o correcciones.
- `freeze`: no operar ni ampliar hasta resolver dudas.
- `discard`: edge insuficiente o sesgado.

M. Checklist Final Antes De Pedir Revision A ChatGPT

- No hay claves, tokens ni credenciales.
- No hay account IDs reales.
- Todos los timestamps indican zona horaria o estan documentados.
- Cada patron tiene regla de deteccion, entrada, salida, stop, take profit y time stop.
- Costes separados: comision, spread, slippage y otros fees.
- Hay variantes descartadas y numero total de experimentos probados.

- Hay datos por ticker y periodo.
- Hay fills IB Paper anonimizados o archivo vacio con cabecera.
- Hay `metrics\_by\_regime.csv` y `metrics\_by\_entry\_variant.csv`, o filas explicitamente vacias con `empty\_reason`.
- Hay `director\_gate\_result.json`.
- Hay referencias de codigo y configuracion.
- Cada muestra indica independencia o limitacion conocida.
- Hay separacion in-sample, out-of-sample y paper-live.
- Hay known issues honestos.
- El validador del paquete pasa.

Full Document: `research/audit\_bridge/SKILL.md`

`research/audit\_bridge/SKILL.md` ? 1721 lines, 39426 bytes, sha256 prefix `47f55c5c7abe3614`

Heading summary: SKILL.md ? Tradeo Director Audit; 1. Propósito; 2. Principios obligatorios; 2.1. Trazabilidad total; 2.2. Rentabilidad positiva no implica validez; 2.3. Separación estricta entre descubrir y validar; 2.4. No se ocultan experimentos fallidos; 2.5. Antes de tocar trading real, bloquear por defecto; 3. Estructura de directorios recomendada; 4. Identificadores de auditoría; 5. Flujo operativo; 5.1. Flujo normal

```
---
name: tradeo-director-audit
version: 1.0.0
last_updated: 2026-06-07
owner_role: ChatGPT Director / Quant Auditor
description: >
  Skill operativo para auditar los resultados generados por Claw/Codex/Researcher
  dentro del proyecto Tradeo, detectar errores estadísticos o de proceso, y emitir
  nuevas instrucciones para mejorar el laboratorio de detección de patrones.
---
```

SKILL.md ? Tradeo Director Audit

1. Propósito

Este skill define cómo debe trabajar el sistema de auditoría del proyecto **Tradeo**.

El objetivo no es simplemente comprobar si una estrategia gana dinero en un backtest. El objetivo es determinar si un patrón detectado por el Researcher es:

- real o ruido estadístico;
- reproducible o accidental;
- robusto o sobreoptimizado;
- operable o solo teórico;
- apto para más investigación, paper trading o rechazo.

La responsabilidad del **Researcher / Claw / Codex** es producir experimentos, datos, métricas, código y paquetes de auditoría.

La responsabilidad del **ChatGPT Director / Quant Auditor** es auditar esos resultados, detectar fallos, exigir nuevas pruebas, cambiar el proceso matemático si hace falta y emitir la siguiente tarea de investigación.

Ningún patrón debe ser promovido a producción, paper trading serio o ejecución real sin pasar por esta auditoría.

```
---
```

2. Principios obligatorios

2.1. Trazabilidad total

Todo resultado debe poder reconstruirse desde:

1. commit de código;
2. versión de datos;
3. configuración;
4. parámetros;
5. costes aplicados;
6. filtros usados;
7. universo de símbolos;
8. periodo temporal;
9. scripts ejecutados;
10. salidas generadas.

Si un resultado no es reproducible, no es auditable.

2.2. Rentabilidad positiva no implica validez

Un patrón con PnL positivo puede ser inválido por:

- lookahead bias;
- data leakage;
- survivorship bias;
- overfitting;
- pocos trades;
- dependencia de outliers;
- mala simulación de costes;
- selección posterior de parámetros;
- múltiples pruebas no corregidas;
- falta de estabilidad temporal;
- falta de estabilidad por ticker;
- imposibilidad de ejecución real.

2.3. Separación estricta entre descubrir y validar

El Researcher puede descubrir patrones.

El Director debe validar si esos patrones sobreviven fuera del proceso que los descubrió.

Siempre que sea posible, separar:

- in-sample;
- validation;
- out-of-sample;
- walk-forward;
- ticker holdout;
- periodo holdout;
- régimen holdout.

2.4. No se ocultan experimentos fallidos

El Researcher debe registrar también variantes descartadas. Ocultar pruebas fallidas distorsiona la probabilidad real de falso positivo.

2.5. Antes de tocar trading real, bloquear por defecto

La auditoría puede aprobar investigación o paper trading controlado, pero no debe autorizar trading real salvo instrucción humana explícita.

3. Estructura de directorios recomendada

Crear esta estructura dentro del repositorio:

```
research/
audit_bridge/
  SKILL.md
  BRIDGE_CONTRACT.md
  README.md

task_queue/
  pending/
  running/
  done/
  blocked/

requests/
  TRADEO-AUDIT-YYYYMMDD-NNN_slug/
    AUDIT_REQUEST.md
    manifest.json
    config_snapshot.json
    data_lineage.json
    costs_model.json
    experiment_registry.csv
    pattern_catalog.csv
    pattern_events.csv
    trades.csv
    fills.csv
    metrics_summary.json
    metrics_by_pattern.csv
    metrics_by_ticker.csv
    metrics_by_period.csv
    metrics_by_regime.csv
    validation_log.md
    known_limitations.md
    plots/

responses/
  TRADEO-AUDIT-YYYYMMDD-NNN_RESPONSE.md
  TRADEO-AUDIT-YYYYMMDD-NNN_RESPONSE.json

decisions/
  DECISIONS.md
  TRADEO-AUDIT-YYYYMMDD-NNN_DECISION.md

state/
  current_context.md
  open_questions.md
  rejected_patterns.md
  approved_candidates.md
  process_improvements.md
  risk_register.md

templates/
  AUDIT_REQUEST_TEMPLATE.md
  AUDIT_RESPONSE_TEMPLATE.md
```

```
TASK_TEMPLATE.md
DECISION_TEMPLATE.md
MANIFEST_TEMPLATE.json
COSTS_MODEL_TEMPLATE.json
```

```
schemas/
  manifest.schema.json
  audit_response.schema.json
  metrics_summary.schema.json
```

```
validators/
  validate_audit_package.py
  validate_no_leakage_basic.py
  validate_metrics_consistency.py
```

```
tools/
  bridge_status.py
  package_audit_request.py
  summarize_audit_package.py
```

Si el repositorio ya tiene otra estructura para research, adaptar estos directorios, pero mantener los nombres internos del `audit\_bridge`.

4. Identificadores de auditoría

Todo paquete de auditoría debe tener un identificador único.

Formato:

TRADEO-AUDIT-YYYYMMDD-NNN_slug

Ejemplos:

TRADEO-AUDIT-20260607-001_ib_paper_patterns
TRADEO-AUDIT-20260607-002_opening_gap_reversal
TRADEO-AUDIT-20260608-001_volume_regime_filter

Reglas:

- `YYYYMMDD` debe ser la fecha de creación del paquete.
- `NNN` debe ser incremental dentro del día.
- `slug` debe ser corto, en minúsculas y con guiones bajos.
- El mismo `audit\_id` debe aparecer en todos los ficheros del paquete.

5. Flujo operativo

5.1. Flujo normal

-
1. Researcher ejecuta experimento.
 2. Researcher genera paquete en `research/audit_bridge/requests/<audit_id>/.`
 3. Researcher ejecuta validadores básicos.
 4. Researcher crea tarea en `task_queue/pending/<audit_id>.md`.
 5. Director audita el paquete.
 6. Director emite respuesta en `responses/<audit_id>_RESPONSE.md` y `.json`.
 7. Director registra decisión en `decisions/`.
 8. Director genera nueva tarea para Claw/Codex si hace falta.
-

5.2. Flujo con Pull Request

Cuando se use Codex o Claw con GitHub:

-
1. Crear rama: `research/<audit_id>` o `feature/<slug>`.
 2. Generar o modificar ficheros.
 3. Ejecutar validaciones.
 4. Crear commit.
 5. Abrir PR.
 6. Incluir enlace o resumen del paquete de auditoría.
 7. Director revisa diff + paquete.
 8. Humano aprueba merge si procede.
-

No se debe hacer push directo a `main` salvo autorización humana explícita.

6. Paquete mínimo obligatorio para auditoría

Un paquete incompleto debe recibir veredicto `BLOCKED\_INCOMPLETE\_PACKAGE`.

6.1. Ficheros obligatorios

Cada request debe incluir:

AUDIT_REQUEST.md
manifest.json
config_snapshot.json
data_lineage.json
costs_model.json
experiment_registry.csv
pattern_catalog.csv
pattern_events.csv
trades.csv
metrics_summary.json
metrics_by_pattern.csv
metrics_by_ticker.csv
metrics_by_period.csv
validation_log.md
known_limitations.md

`fills.csv`, `metrics\_by\_regime.csv` y `plots/` son recomendados. Si no existen, debe explicarse por qué.

6.2. Reglas generales de formato

- Markdown en UTF-8.
- JSON válido y parseable.
- CSV con cabecera.
- Separador CSV: coma.
- Decimales: punto.
- Sin separadores de miles.
- Timestamps en ISO-8601.
- Zona horaria explícita; preferiblemente UTC.
- No incluir secretos, tokens, claves API ni credenciales.
- No incluir datos privados innecesarios.
- Si un fichero es demasiado grande, entregar:
 - path relativo;
 - hash SHA-256;
 - número de filas;
 - columnas;
 - muestra representativa;
 - resumen agregado.

7. `AUDIT\_REQUEST.md`

Debe explicar qué quiere que audite el Director.

Plantilla:

Audit Request: <audit_id>

1. Objetivo

Describe el experimento y qué patrón se pretende validar.

2. Hipótesis

Ejemplo:

Cuando ocurre <condición>, el precio tiende a <comportamiento> durante <horizonte>, con EV neto positivo después de costes.

3. Resumen ejecutivo del Researcher

- ? Resultado principal:
- ? Universo:
- ? Periodo:
- ? Número de eventos:
- ? Número de trades:
- ? EV neto:
- ? Profit factor:
- ? Max drawdown:
- ? Principal riesgo conocido:

4. Qué ha cambiado respecto al experimento anterior

- ? Código:
- ? Datos:
- ? Parámetros:
- ? Costes:
- ? Filtros:

5. Decisión que solicita el Researcher

- Una de:
- ? revisar si el patrón es válido;
 - ? aprobar más investigación;
 - ? pasar a paper trading controlado;
 - ? rechazar;
 - ? mejorar framework de research;
 - ? detectar fallos estadísticos.

6. Limitaciones conocidas

Lista honesta de carencias, dudas, supuestos y posibles sesgos.

7. Ficheros incluidos

Lista de ficheros del paquete y breve descripción.

```
---

## 8. `manifest.json`

Debe permitir verificar trazabilidad y reproducibilidad.

Campos obligatorios:
```

```
{
  "schema_version": "1.0.0",
  "audit_id": "TRADEO-AUDIT-YYYYMMDD-NNN_slug",
  "created_at": "2026-06-07T00:00:00Z",
  "created_by": "Claw/Researcher/Codex",
  "repo": "tradeo",
  "branch": "research/example",
  "commit_sha": "REQUIRED",
  "parent_commit_sha": "REQUIRED",
  "python_version": "REQUIRED_IF_APPLICABLE",
  "environment": "local/codex/cloud/ci",
  "data_sources": [
    {
      "name": "source_name",
      "version": "data_version_or_date",
      "path": "relative/path",
      "sha256": "REQUIRED",
      "rows": 0,
      "columns": [],
      "start_ts": "YYYY-MM-DDTHH:MM:SSZ",
      "end_ts": "YYYY-MM-DDTHH:MM:SSZ",
      "timezone": "UTC"
    }
  ],
  "generated_files": [
    {
      "path": "research/audit_bridge/requests/.../trades.csv",
      "sha256": "REQUIRED",
      "rows": 0,
      "description": "Trade-level results"
    }
  ],
  "commands_run": [
    {
      "command": "python ...",
      "started_at": "2026-06-07T00:00:00Z",
      "finished_at": "2026-06-07T00:00:00Z",
      "exit_code": 0
    }
  ]
}
```

```
],
"notes": []
}
```

Bloquear auditoria si faltan:

- `audit\_id`;
- `commit\_sha`;
- fuentes de datos;
- hashes;
- lista de comandos ejecutados;
- rango temporal de datos.

9. `data\_lineage.json`

Debe explicar de dónde vienen los datos y cómo se transformaron.

Campos mínimos:

```
{
  "audit_id": "TRADEO-AUDIT-YYYYMMDD-NNN_slug",
  "raw_inputs": [],
  "intermediate_outputs": [],
  "final_outputs": [],
  "transformations": [
    {
      "step": "resample_1min",
      "input": "path/input.csv",
      "output": "path/output.csv",
      "code_reference": "module:function",
      "parameters": {},
      "known_risks": []
    }
  ],
  "excluded_data": [
    {
      "reason": "missing volume",
      "rows": 0,
      "symbols": []
    }
  ]
}
```

El Director debe revisar especialmente:

- si hay filtrados posteriores al resultado;
- si se eliminan símbolos perdedores;
- si se usan datos futuros para construir features;
- si se mezclan timestamps de señal, decisión y ejecución;
- si los datos ajustados introducen sesgos.

10. `config\_snapshot.json`

Debe contener todos los parámetros usados.

Campos recomendados:

```
{
  "audit_id": "TRADEO-AUDIT-YYYYMMDD-NNN_slug",
  "strategy_or_pattern_name": "name",
  "universe": {
    "symbols": [],
    "selection_rule": "explicit rule",
    "selection_time": "before_test/after_test/unknown"
  },
  "time_range": {
    "start": "YYYY-MM-DD",
    "end": "YYYY-MM-DD",
    "timezone": "UTC"
  },
  "bar_size": "1m/5m/1d/etc",
  "features": [],
  "entry_rules": [],
}
```

```
"exit_rules": [],
"risk_rules": [],
"filters": [],
"parameters": {},
"random_seed": null,
"split_method": "in_sample/out_of_sample/walk_forward/none",
"split_details": {}
}
```

11. `costs\_model.json`

Debe describir el modelo de costes usado.

Campos mínimos:

```
{
  "audit_id": "TRADEO-AUDIT-YYYYMMDD-NNN_slug",
  "currency": "USD",
  "commission_model": {
    "type": "fixed/per_share/bps/custom",
    "value": 0.0,
    "notes": ""
  },
  "slippage_model": {
    "type": "fixed_ticks/bps/spread_fraction/volume_impact/custom",
    "value": 0.0,
    "notes": ""
  },
  "spread_model": {
    "source": "actual/estimated/constant/not_used",
    "value": 0.0
  },
  "borrow_cost_model": {
    "applies_to_shorts": true,
    "value": 0.0,
    "notes": ""
  },
  "latency_assumption_ms": 0,
  "market_impact_assumption": "none/low/medium/high/custom",
  "sensitivity_tests": [
    {
      "name": "double_slippage",
      "net_ev": 0.0,
      "profit_factor": 0.0
    }
  ]
}
```

Auditoría debe bloquear o pedir mejora si:

- no hay costes;
- el slippage es cero sin justificación;
- no se evalúa sensibilidad a costes;
- el patrón solo gana con costes irreales;
- se ignora liquidez o spread cuando la estrategia lo requiere.

12. `experiment\_registry.csv`

Registra todos los experimentos y variantes, incluidos fallidos.

Columnas obligatorias:

experiment_id	audit_id	parent_experiment_id	created_at	description	status	code_ref	config_ref	data_ref	parameters_json	num_patterns_tested	num_variants_tested	primary_metric	primary_metric_value	notes
---------------	----------	----------------------	------------	-------------	--------	----------	------------	----------	-----------------	---------------------	---------------------	----------------	----------------------	-------

Valores recomendados para `status`:

completed,failed,discarded,blocked,running,reproduced,not_reproduced

El Director debe penalizar paquetes donde solo aparezca la variante ganadora.

13. `pattern\_catalog.csv`

Catálogo de patrones detectados.

Columnas obligatorias:

pattern_id,audit_id,pattern_name,hypothesis,feature_set,entry_rule,exit_rule,holding_period,side,universe_rule,created_by,created_at,discovery_method,parameters_json,notes

Reglas:

- `entry\_rule` y `exit\_rule` deben ser suficientemente explícitas para reproducir la señal.
- `feature\_set` debe indicar solo features disponibles en el momento de decisión.
- `discovery\_method` debe indicar si fue búsqueda manual, grid search, ML, clustering, heurística, etc.

14. `pattern\_events.csv`

Eventos donde el patrón se activa.

Columnas obligatorias:

event_id,audit_id,pattern_id,symbol,event_ts,signal_ts,decision_ts,available_data_cutoff_ts,side,feature_values_json,regime_tags_json,source_row_id,notes

Definiciones:

- `event\_ts`: momento del evento de mercado.
- `signal\_ts`: momento en que la señal queda formada.
- `decision\_ts`: momento en que la estrategia podría decidir.
- `available\_data\_cutoff\_ts`: último dato que la señal tenía permitido usar.

Regla crítica:

$available_data_cutoff_ts \leq decision_ts \leq entry_ts$

Si esto no se cumple, posible lookahead.

15. `trades.csv`

Resultados trade a trade.

Columnas obligatorias:

trade_id,audit_id,pattern_id,event_id,symbol,side,entry_ts,exit_ts,entry_price,exit_price,quantity,gross_pnl,commission,slippage,spread_cost,borrow_cost,other_costs,total_costs,net_pnl,return_pct,mae,mfe,holding_minutes,entry_reason,exit_reason,execution_model,notes

Reglas:

- $net_pnl = gross_pnl - total_costs$.
- $total_costs = commission + slippage + spread_cost + borrow_cost + other_costs$.
- `entry\_ts` debe ser posterior o igual a `decision\_ts`.
- No se permite usar precio de entrada imposible para el timeframe usado.
- Debe poder enlazarse cada trade a un `event\_id`.

16. `fills.csv`

Recomendado cuando haya simulación de ejecución o paper trading.

Columnas:

fill_id,trade_id,audit_id,symbol,side,fill_ts,price,quantity,liquidity_flag,order_type,fees,slippage_estimate,venue,notes

Si no existe, el Researcher debe explicar el modelo de ejecución usado.

17. `metrics\_summary.json`

Resumen agregado.

Campos mínimos:

```
{
  "audit_id": "TRADEO-AUDIT-YYYYMMDD-NNN_slug",
  "num_patterns": 0,
  "num_events": 0,
  "num_trades": 0,
  "gross_pnl": 0.0,
  "net_pnl": 0.0,
  "total_costs": 0.0,
  "expectancy_per_trade": 0.0,
  "expectancy_per_dollar": 0.0,
  "win_rate": 0.0,
  "avg_win": 0.0,
  "avg_loss": 0.0,
  "profit_factor": 0.0,
  "max_drawdown": 0.0,
  "sharpe": null,
  "sortino": null,
  "calmar": null,
  "median_trade_pnl": 0.0,
  "p05_trade_pnl": 0.0,
  "p95_trade_pnl": 0.0,
  "largest_win": 0.0,
  "largest_loss": 0.0,
  "top_5pct_trades_pnl_share": 0.0,
  "oos_net_pnl": null,
  "oos_profit_factor": null,
  "permutation_p_value": null,
  "bootstrap_expectancy_ci_low": null,
  "bootstrap_expectancy_ci_high": null,
  "multiple_testing_adjusted_p_value": null,
  "researcher_claim": "",
  "researcher_confidence": "low/medium/high"
}
```

18. `metrics\_by\_pattern.csv`

Métricas por patrón.

Columnas obligatorias:

pattern_id,audit_id,num_events,num_trades,gross_pnl,net_pnl,total_costs,expectancy_per_trade,win_rate,avg_win,avg_loss,profit_factor,max_drawdown,sharpe,sortino,median_trade_pnl,p05_trade_pnl,p95_trade_pnl,largest_win,largest_loss,top_5pct_trades_pnl_share,bootstrap_ci_low,bootstrap_ci_high,permutation_p_value,multiple_testing_adjusted_p_value,oos_net_pnl,oos_profit_factor,stability_score,concentration_score,researcher_decision,notes

19. `metrics\_by\_ticker.csv`

Métricas por símbolo.

Columnas obligatorias:

symbol,audit_id,pattern_id,num_trades,net_pnl,expectancy_per_trade,win_rate,profit_factor,max_drawdown,largest_win,largest_loss,pnl_share,notes

El Director debe revisar:

- si un solo ticker explica la mayoría del beneficio;
- si los tickers perdedores fueron excluidos;
- si el patrón solo funciona en símbolos poco líquidos;
- si el universo fue definido antes del test.

20. `metrics\_by\_period.csv`

Métricas por tramo temporal.

Columnas obligatorias:

period_start,period_end,audit_id,pattern_id,num_trades,net_pnl,expectancy_per_trade,win_rate,profit_factor,max_drawdown,regime_label,notes

Tramos recomendados:

- mes;
- trimestre;
- año;
- ventanas walk-forward;
- periodos de volatilidad alta/baja;
- mercado alcista/bajista/lateral.

21. `metrics\_by\_regime.csv`

Recomendado para estrategias sensibles al contexto.

Columnas:

regime_label	audit_id	pattern_id	num_trades	net_pnl	expectancy_per_trade	win_rate	profit_factor	max_drawdown	definition_json	notes
--------------	----------	------------	------------	---------	----------------------	----------	---------------	--------------	-----------------	-------

Regímenes posibles:

- SPY positivo/negativo;
- QQQ positivo/negativo;
- VIX alto/bajo si existe ese dato;
- volumen relativo alto/bajo;
- tendencia intradía;
- gap premarket;
- volatilidad realizada;
- sesión regular/premarket/after-hours.

22. `validation\_log.md`

Debe registrar pruebas ejecutadas antes de pedir auditoría.

Plantilla:

Validation Log: <audit_id>

Commands executed

```
python research/audit_bridge/validators/validate_audit_package.py <path>
python research/audit_bridge/validators/validate_metrics_consistency.py <path>
```

Results

- ? Package completeness: PASS/FAIL
- ? JSON parse: PASS/FAIL
- ? CSV schema: PASS/FAIL
- ? Row counts: PASS/FAIL
- ? Trade/event links: PASS/FAIL
- ? Net PnL consistency: PASS/FAIL
- ? Timestamp ordering: PASS/FAIL
- ? Duplicate event check: PASS/FAIL
- ? Missing values check: PASS/FAIL

Known failures

Listar cualquier fallo no resuelto.

23. `known\_limitations.md`

Debe ser honesto y explícito.

Preguntas que debe responder:

Known Limitations: <audit_id>

Data limitations

- ? ¿Faltan símbolos?
- ? ¿Faltan sesiones?

- ? ¿Hay huecos de datos?
- ? ¿Hay ajustes corporativos?

Statistical limitations

- ? ¿Muestra pequeña?
- ? ¿Muchas variantes probadas?
- ? ¿Sin out-of-sample?
- ? ¿Sin bootstrap?
- ? ¿Sin test de permutación?

Execution limitations

- ? ¿Slippage estimado?
- ? ¿No hay spread real?
- ? ¿No hay profundidad de mercado?
- ? ¿Ordenes simuladas demasiado optimistas?

Researcher concerns

- ? ¿Qué sospecha el Researcher que puede estar mal?

Un paquete sin limitaciones conocidas es sospechoso, no mejor.

24. Checks obligatorios del Director

El Director debe revisar como mínimo:

24.1. Integridad del paquete

- ¿Están todos los ficheros obligatorios?
- ¿Los JSON parsean?
- ¿Los CSV tienen columnas requeridas?
- ¿Los hashes y row counts existen?
- ¿Los IDs enlazan correctamente?

24.2. Reproducibilidad

- ¿Hay commit SHA?
- ¿Hay comandos ejecutados?
- ¿Hay configuración completa?
- ¿Hay data lineage?
- ¿Puede repetirse el experimento?

24.3. Timestamps y lookahead

- ¿La señal usa solo datos disponibles antes de la decisión?
- ¿`available\_data\_cutoff\_ts <= decision\_ts <= entry\_ts`?
- ¿Se usan máximos/mínimos de la vela antes de poder conocerlos?
- ¿Se usan indicadores calculados con datos futuros?
- ¿Se mezclan timestamps de evento y ejecución?

24.4. Data leakage

- ¿El target entra en features?
- ¿El filtro se calculó con todo el dataset?
- ¿La normalización usa datos futuros?
- ¿La selección de símbolos se hizo después del resultado?
- ¿Hay información post-evento en features?

24.5. Survivorship bias

- ¿El universo excluye símbolos desaparecidos?
- ¿Solo se usan tickers actuales?
- ¿Se ignoran días sin datos porque fueron malos?

24.6. Múltiples pruebas

- ¿Cuántos patrones y variantes se probaron?
- ¿Se reportan descartados?
- ¿Hay corrección por multiple testing?
- ¿El p-value se interpreta después de muchas búsquedas?

24.7. Overfitting

- ¿Hay demasiados parámetros?
- ¿La estrategia fue ajustada contra el mismo periodo evaluado?
- ¿El resultado cambia mucho ante pequeñas variaciones?
- ¿Funciona fuera de la ventana donde fue descubierta?

24.8. Robustez temporal

- ¿Funciona por años/meses/trimestres?
- ¿Se rompe en OOS?
- ¿Depende de un único periodo excepcional?
- ¿Sobrevive walk-forward?

24.9. Robustez por ticker

- ¿Funciona en varios símbolos?
- ¿Un solo ticker explica todo?
- ¿Los perdedores fueron excluidos?
- ¿Hay estabilidad por sectores/tipos de activo?

24.10. Dependencia de outliers

- ¿El top 1%, 5% o 10% de trades explica casi todo?
- ¿La mediana es positiva?
- ¿El resultado sobrevive winsorization?
- ¿Qué ocurre quitando los mejores N trades?

24.11. Costes y ejecución

- ¿Incluye comisiones?
- ¿Incluye slippage?
- ¿Incluye spread?
- ¿Incluye borrow para cortos?
- ¿La latencia asumida es razonable?
- ¿Las entradas y salidas son ejecutables?
- ¿Hay sensibilidad a costes duplicados/triplicados?

24.12. Lógica económica

- ¿Existe una explicación plausible?
- ¿La señal captura comportamiento real del mercado?
- ¿O es solo una coincidencia de parámetros?

24.13. Riesgo operativo

- ¿Requiere datos no disponibles en vivo?
- ¿Requiere ejecución imposible?
- ¿Puede fallar por horarios, liquidez o latencia?
- ¿Tiene exposición concentrada?

25. Tests estadísticos recomendados

El Researcher debe implementar progresivamente estos tests. No todos son obligatorios para paquetes tempranos, pero si faltan debe indicarse.

25.1. Baseline aleatorio

Comparar contra entradas aleatorias con mismo universo, horarios y número de trades.

Resultado esperado:

El patrón debe superar claramente al baseline aleatorio después de costes.

25.2. Permutation test

Permutar señales, retornos o labels según corresponda.

Guardar:

- número de permutaciones;
- métrica observada;
- distribución nula;
- p-value;
- seed.

25.3. Bootstrap de trades

Estimar intervalo de confianza de expectancy.

Guardar:

- número de muestras bootstrap;
- CI 5%-95% o 2.5%-97.5%;
- probabilidad de EV ≤ 0 .

25.4. Walk-forward

Evaluar por ventanas temporales:

train -> validate -> test

Registrar parámetros elegidos en train y resultado en test.

25.5. Ticker holdout

Descubrir en un grupo de símbolos y validar en otros.

25.6. Period holdout

Descubrir en un periodo y validar en otro no usado.

25.7. Sensibilidad a parámetros

Pequeños cambios de parámetros no deberían destruir completamente el edge.

25.8. Sensibilidad a costes

Recalcular con:

- costes normales;
- slippage x2;
- slippage x3;
- spread x2;
- comisión x2;
- delay de entrada;
- peor fill razonable.

25.9. Outlier removal

Recalcular quitando:

- mejor trade;
- mejores 5 trades;
- top 1%;
- top 5%;
- winsorization 1%/99%.

25.10. Multiple testing adjustment

Aplicar corrección cuando haya muchas variantes:

- Bonferroni;
- Benjamini-Hochberg;
- deflated Sharpe ratio si procede;
- registro explícito de `num\_variants\_tested`.

26. Criterios de scoring del Director

El Director debe emitir un score de 0 a 100.

Propuesta de pesos:

Data integrity / reproducibility	15
Leakage & timestamp safety	20
Statistical robustness	20
Out-of-sample / walk-forward evidence	15
Execution realism & costs	15
Cross-ticker / cross-regime stability	10
Economic plausibility	5

Interpretación:

-
- | | |
|--------|---|
| 0-29 | Rechazo fuerte o paquete inválido |
| 30-49 | Débil, requiere rediseño |
| 50-64 | Prometedor pero insuficiente |
| 65-79 | Buen candidato para más validación / paper limitado |
| 80-89 | Candidato fuerte para paper extendido |
| 90-100 | Excepcional, aun así requiere aprobación humana para real |
-

Este score no sustituye el juicio del Director. Sirve para estandarizar auditorías.

27. Veredictos posibles

El Director debe usar uno de estos veredictos:

-
- | |
|--------------------------------|
| APPROVED_FOR_NEXT_RESEARCH |
| PAPER_TEST_CANDIDATE |
| READY_FOR_PAPER_EXTENDED |
| REJECTED_NO_EDGE |
| REJECTED_DATA_LEAKAGE |
| REJECTED_LOOKAHEAD |
| REJECTED_OVERFITTING |
| REJECTED_UNREALISTIC_EXECUTION |
| REJECTED_INSUFFICIENT_SAMPLE |
| PROMISING_BUT_WEAK |
| NEEDS_MORE_DATA |
| NEEDS_OOS_VALIDATION |
| NEEDS_COST_MODEL_FIX |
| NEEDS_PROCESS_IMPROVEMENT |
| BLOCKED_INCOMPLETE_PACKAGE |

BLOCKED_REPRODUCIBILITY_FAILURE
BLOCKED_SCHEMA_FAILURE
HUMAN_REVIEW_REQUIRED

Regla:

- Un veredicto `READY\_FOR\_PAPER\_EXTENDED` no significa autorización para trading real.
- Un veredicto `PAPER\_TEST\_CANDIDATE` requiere paper trading controlado y monitorizado.
- Cualquier sospecha seria de leakage/lookahead debe rechazar o bloquear el patrón.

28. Umbrales orientativos de aprobación

Estos umbrales son guías, no leyes. Pueden cambiar según mercado, timeframe y tipo de estrategia.

Para considerar un patrón como candidato serio:

-
- ? Net expectancy > 0 después de costes.
 - ? Profit factor OOS preferiblemente > 1.15-1.25.
 - ? Muestra suficiente o justificación explícita si es pequeña.
 - ? Resultado no explicado por 1-5 trades extremos.
 - ? Mediana de trade no fuertemente negativa.
 - ? Costes razonables incluidos.
 - ? Sin señales de lookahead/leakage.
 - ? Alguna forma de validación fuera de muestra.
 - ? Estabilidad por periodo o explicación clara de régimen.
 - ? No depender de un único ticker salvo que el patrón sea específico de ese ticker.
-

Señales de rechazo o bloqueo:

-
- ? Slippage cero sin justificación.
 - ? Entry price imposible.
 - ? Uso de high/low de vela antes de cierre.
 - ? Selección de símbolos posterior al resultado.
 - ? Solo se reporta la mejor variante.
 - ? Top 5% de trades explica casi todo el PnL.
 - ? OOS negativo o inexistente sin justificación.
 - ? PnL positivo bruto pero negativo neto.
 - ? P-value sin corrección tras miles de pruebas.
 - ? Falta de commit SHA o data lineage.
-

29. Formato de respuesta del Director

El Director debe generar dos ficheros:

responses/<audit_id>_RESPONSE.md
responses/<audit_id>_RESPONSE.json

29.1. `AUDIT\_RESPONSE.md`

Plantilla:

Director Audit Response: <audit_id>

Verdict

<VERDICT>

Director score

<0-100>

Confidence

low / medium / high

Executive summary

Resumen claro de si el patrón parece válido, débil, contaminado o no auditable.

What was reviewed

- ? Ficheros revisados:
- ? Métricas revisadas:
- ? Código/diff revisado si aplica:

Positive evidence

- ? Evidencia a favor del patrón.

Negative evidence / risks

- ? Riesgos encontrados.
- ? Posibles sesgos.
- ? Debilidades estadísticas.

Blocking issues

Problemas que impiden aprobación.

Statistical audit

- ? Sample size:
- ? EV neto:
- ? Profit factor:
- ? Outlier dependence:
- ? Bootstrap:
- ? Permutation test:
- ? Multiple testing:
- ? OOS / walk-forward:

Data audit

- ? Data lineage:
- ? Survivorship:
- ? Missing data:
- ? Symbol universe:
- ? Timestamp safety:

Execution audit

- ? Costes:
- ? Slippage:
- ? Spread:
- ? Liquidez:
- ? Latencia:
- ? Realismo de entrada/salida:

Required next actions for Claw/Codex

Lista numerada de acciones concretas.

Process improvements ordered

Cambios al framework de research, métricas, validadores o datos.

Promotion decision

- Una de:
- ? stay_in_research
 - ? repeat_with_fixes
 - ? expand_dataset
 - ? run_oos
 - ? run_paper_limited
 - ? run_paper_extended
 - ? reject_and_archive

Human notes

Cualquier decisión que requiera intervención humana.

29.2. `AUDIT\_RESPONSE.json`

Formato:

{

```
"audit_id": "TRADEO-AUDIT-YYYYMMDD-NNN_slug",
"verdict": "PROMISING_BUT_WEAK",
"director_score": 0,
"confidence": "low/medium/high",
"promotion_decision": "stay_in_research",
"blocking_issues": [],
"positive_evidence": [],
"negative_evidence": [],
"required_next_actions": [
  {
    "priority": "P0/P1/P2/P3",
    "owner": "Claw/Codex/Human",
    "action": "",
    "expected_output": "",
    "acceptance_criteria": ""
  }
],
"process_improvements": [],
"created_at": "2026-06-07T00:00:00Z"
}
```

30. Formato de tarea para Claw/Codex

Cada tarea debe ir en:

task_queue/pending/<audit_id>_<task_slug>.md

Plantilla:

Task: <task_id>

From

ChatGPT Director

To

Claw/Codex Researcher

Related audit

<audit_id>

Priority

P0 / P1 / P2 / P3

Goal

Objetivo concreto.

Context

Resumen de por qué se pide esta tarea.

Required inputs

? Ficheros o módulos necesarios.

Actions

1. Acción concreta.
2. Acción concreta.
3. Acción concreta.

Expected outputs

- ? Ficheros nuevos/modificados.
- ? Métricas esperadas.
- ? Logs.

Acceptance criteria

? Condiciones para considerar completada la tarea.

Constraints

- ? No tocar producción.
- ? No borrar datos.
- ? No meter secretos.
- ? No hacer push a main.

Response format

Al terminar, responder con:

- ? Estado: OK / BLOCKED / NEEDS_HUMAN
- ? Rama:
- ? Commits:
- ? Ficheros modificados:
- ? Validaciones ejecutadas:
- ? Resultados:
- ? Riesgos:
- ? Siguiendo paso recomendado:

31. Reglas para Claw/Codex antes de pedir auditoría

Claw/Codex debe hacer lo siguiente antes de enviar un paquete:

1. Ejecutar validadores básicos.
2. Confirmar que no hay secretos en los ficheros.
3. Confirmar que no ha modificado datos originales.
4. Confirmar que no ha hecho push a `main`.
5. Confirmar que el paquete tiene `manifest.json` completo.
6. Confirmar que todas las métricas netas incluyen costes.
7. Confirmar que los timestamps cumplen orden lógico.
8. Confirmar que las variantes descartadas están registradas.
9. Confirmar que las limitaciones conocidas están escritas.
10. Incluir comandos ejecutados y exit codes.

32. Reglas de seguridad

Prohibido:

- ? Incluir claves API, tokens, contraseñas o credenciales.
- ? Modificar configuración de trading real sin autorización humana.
- ? Activar órdenes reales.
- ? Borrar datasets originales.
- ? Sobrescribir resultados históricos sin backup.
- ? Hacer push directo a main sin permiso.
- ? Ocultar experimentos fallidos.
- ? Cambiar métricas para maquillar resultados.
- ? Reportar PnL bruto como si fuera neto.

Obligatorio:

- ? Trabajar en rama.
- ? Dejar trazabilidad.
- ? Usar commits descriptivos.
- ? Registrar supuestos.
- ? Registrar limitaciones.
- ? Mantener datos originales inmutables.

33. Mejoras de proceso que el Director puede ordenar

El Director puede ordenar cambios como:

- ? Añadir validadores de leakage.
- ? Añadir tests de permutación.
- ? Añadir bootstrap.
- ? Añadir walk-forward.
- ? Añadir ticker holdout.

- ? Añadir régimen de mercado.
- ? Añadir sensibilidad a costes.
- ? Añadir análisis de outliers.
- ? Añadir corrección por multiple testing.
- ? Añadir score de robustez.
- ? Añadir score de concentración.
- ? Añadir score de explicabilidad económica.
- ? Mejorar data lineage.
- ? Mejorar tracking de experimentos fallidos.
- ? Cambiar fórmula de scoring.
- ? Cambiar definición de patrón.
- ? Interrelacionar más datos: volumen, volatilidad, gaps, SPY/QQQ, calendario, sesión, liquidez, spreads.

34. Fórmulas y métricas recomendadas

34.1. Expectancy neta por trade

expectancy_per_trade = mean(net_pnl)

34.2. Profit factor

profit_factor = sum(net_pnl where net_pnl > 0) / abs(sum(net_pnl where net_pnl < 0))

Si no hay pérdidas, reportar como `null` o `inf` y explicar. No usarlo solo para aprobar.

34.3. Costes totales

total_costs = commission + slippage + spread_cost + borrow_cost + other_costs
net_pnl = gross_pnl - total_costs

34.4. Dependencia de outliers

top_5pct_trades_pnl_share = pnl_top_5pct_trades / total_net_pnl

Si este valor es muy alto, el edge puede ser frágil.

34.5. Concentración por ticker

symbol_pnl_share = net_pnl_symbol / total_net_pnl

Revisar si un único símbolo domina el resultado.

34.6. Robustez simple

Una posible métrica inicial:

stability_score = porcentaje_de_segmentos_con_ev_netto_positivo

Segmentos pueden ser meses, trimestres, tickers o regímenes.

35. Modo de auditoría rápida

Si el Director no puede revisar todo, debe hacer como mínimo:

1. Verificar paquete completo.
2. Revisar `AUDIT\_REQUEST.md`.
3. Revisar `metrics\_summary.json`.
4. Revisar `metrics\_by\_pattern.csv`.
5. Revisar `metrics\_by\_ticker.csv`.
6. Revisar `metrics\_by\_period.csv`.
7. Revisar `known\_limitations.md`.
8. Buscar señales de leakage/lookahead.
9. Revisar costes.
10. Emitir veredicto conservador.

Si falta información crítica, usar `BLOCKED\_INCOMPLETE\_PACKAGE` o `NEEDS\_MORE\_DATA`.

36. Modo de auditoría profunda

Para patrones prometedores:

1. Revisar código o diff.
2. Recalcular métricas si es posible.
3. Verificar consistencia trade/event.
4. Analizar distribución de PnL.
5. Analizar outliers.
6. Analizar periodos.
7. Analizar tickers.
8. Analizar regímenes.
9. Analizar costes extremos.
10. Analizar sensibilidad a parámetros.
11. Comparar contra baseline aleatorio.
12. Evaluar lógica económica.
13. Decidir siguiente experimento.

37. Promoción de patrones

Estados recomendados:

DISCOVERED
UNDER_AUDIT
REJECTED
NEEDS_FIX
NEEDS_MORE_DATA
RESEARCH_CANDIDATE
OOS_CANDIDATE
PAPER_LIMITED_CANDIDATE
PAPER_EXTENDED_CANDIDATE
REAL_TRADING_REQUIRES_HUMAN_APPROVAL

Reglas:

- `DISCOVERED` no significa válido.
- `RESEARCH\_CANDIDATE` solo permite más investigación.
- `PAPER\_LIMITED\_CANDIDATE` requiere tamaño pequeño y controlado.
- `PAPER\_EXTENDED\_CANDIDATE` requiere monitorización.
- Trading real siempre requiere aprobación humana explícita.

38. Registro de decisiones

Cada decisión debe añadirse a:

research/audit_bridge/decisions/DECISIONS.md

Formato:

<YYYY-MM-DD> ? <audit_id>

- ? Verdict: <VERDICT>
 - ? Director score: <0-100>
 - ? Promotion decision: <decision>
 - ? Main reason:
 - ? Required next action:
 - ? Related files:
-

39. Estado global del sistema

Mantener estos ficheros:

`state/current\_context.md`

Resumen del estado actual del research.

`state/open\_questions.md`

Preguntas abiertas que el Researcher debe resolver.

`state/rejected\_patterns.md`

Patrones rechazados y razón.

`state/approved\_candidates.md`

Patrones candidatos y estado.

`state/process\_improvements.md`

Mejoras pendientes del framework.

`state/risk\_register.md`

Riesgos conocidos del sistema.

40. Comandos recomendados

Ejemplos:

```
python research/audit_bridge/tools/bridge_status.py
python research/audit_bridge/validators/validate_audit_package.py research/audit_bridge/requests/<audit_id>
python research/audit_bridge/tools/summarize_audit_package.py research/audit_bridge/requests/<audit_id>
```

Los scripts deben devolver exit code distinto de cero si detectan errores bloqueantes.

41. Criterios para bloquear inmediatamente

Bloquear sin seguir auditando si ocurre cualquiera de estos casos:

-
- ? Falta manifest.json.
 - ? Falta commit SHA.
 - ? No hay data lineage.
 - ? No hay costes netos.
 - ? Los trades no enlazan con eventos.
 - ? entry_ts < decision_ts.
 - ? Se usan datos futuros en features.
 - ? No se puede parsear JSON/CSV.
 - ? El paquete no indica universo o periodo.
 - ? Hay secretos o credenciales.
 - ? Se modificó producción sin autorización.
-

42. Información que el Director necesita recibir en cada auditoría

Resumen mínimo que debe estar explícito:

-
- ? Qué patrón se quiere validar.
 - ? Por qué debería existir ese edge.
 - ? Qué datos se usaron.
 - ? Qué periodo se usó.
 - ? Qué universo se usó.
 - ? Cuántas variantes se probaron.
 - ? Cuántas fallaron.
 - ? Qué filtros se aplicaron.
 - ? Qué costes se aplicaron.
 - ? Qué resultados netos se obtuvieron.
 - ? Cómo se comporta por ticker.
 - ? Cómo se comporta por periodo.
 - ? Cómo se comporta fuera de muestra.
 - ? Qué riesgos sospecha el propio Researcher.
 - ? Qué decisión solicita el Researcher.
-

43. Prompt base para Claw/Codex

Usar este prompt cuando se pida generar un paquete de auditoría:

Actúa como Researcher del proyecto Tradeo.

Debes preparar un paquete de auditoría para ChatGPT Director siguiendo:

research/audit_bridge/SKILL.md

No puedes declarar un patrón como válido. Solo puedes presentar evidencia.

Crea un audit_id con formato:

TRADEO-AUDIT-YYYYMMDD-NNN_slug

Genera el paquete en:

research/audit_bridge/requests/<audit_id>/

Incluye como mínimo:

- ? AUDIT_REQUEST.md
- ? manifest.json
- ? config_snapshot.json
- ? data_lineage.json
- ? costs_model.json
- ? experiment_registry.csv
- ? pattern_catalog.csv
- ? pattern_events.csv
- ? trades.csv
- ? metrics_summary.json
- ? metrics_by_pattern.csv
- ? metrics_by_ticker.csv
- ? metrics_by_period.csv
- ? validation_log.md
- ? known_limitations.md

Ejecuta validadores básicos si existen.

No modifiques trading real.

No borres datos originales.

No incluyas secretos.

No hagas push a main.

Al terminar, responde con:

- ? audit_id
- ? rama
- ? commits
- ? ficheros creados/modificados
- ? comandos ejecutados
- ? validaciones
- ? resultados principales
- ? limitaciones conocidas
- ? riesgos
- ? pregunta concreta para el Director

44. Prompt base para el Director

Usar este prompt cuando se entregue un paquete al Director:

Actúa como ChatGPT Director / Quant Auditor de Tradeo.

Audita el paquete:

research/audit_bridge/requests/<audit_id>/

Usa las reglas de:

research/audit_bridge/SKILL.md

Debes determinar si el resultado del Researcher es correcto, insuficiente, sesgado, sobreoptimizado o prometedor.

Revisa especialmente:

- ? lookahead;
- ? leakage;
- ? costes;
- ? robustez temporal;
- ? robustez por ticker;
- ? outliers;
- ? multiple testing;
- ? sample size;
- ? OOS/walk-forward;
- ? lógica económica;
- ? reproducibilidad;
- ? calidad del paquete.

Entrega:

1. Verdict.
2. Director score 0-100.
3. Evidencia a favor.
4. Evidencia en contra.
5. Problemas bloqueantes.
6. Acciones obligatorias para Claw/Codex.
7. Mejoras de proceso.
8. Decisión de promoción.
9. Fichero de respuesta Markdown.

10. Fichero de respuesta JSON.

45. Definición de éxito de este skill

Este skill funciona si consigue que el sistema Tradeo:

- deje de aceptar backtests bonitos sin auditoría;
- detecte leakage y lookahead antes de paper trading;
- registre experimentos fallidos;
- mejore la calidad matemática del research;
- haga comparaciones fuera de muestra;
- cuantifique costes y ejecución;
- aprenda de patrones rechazados;
- convierta cada auditoría en una mejora del proceso.

46. Regla final

El Researcher descubre.

Claw/Codex ejecutan.

El Director audita, interpreta y mejora el sistema.

Ningún patrón avanza por rentabilidad aparente. Solo avanza por evidencia robusta, reproducible y operable.

Full Document: `research/audit\_bridge/director\_gate.py`

`research/audit\_bridge/director\_gate.py` ? 921 lines, 36587 bytes, sha256 prefix `44a2f9806614fc13`

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import csv
import json
import sys
from collections import Counter
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

EXECUTION_PROMOTION_STATUSES = {
    "paper_candidate",
    "premium_candidate",
    "paper_limited_candidate",
    "paper_extended_candidate",
    "ready_for_paper_extended",
    "approved",
    "live_candidate",
    "production_candidate",
    "production",
}

RESEARCH_ONLY_STATUSES = {
    "lab",
    "lab_watchlist",
    "lab_candidate",
    "director_review",
    "needs_confirmation",
    "confirmed_candidate",
    "failed_confirmation",
    "rejected",
    "discard",
    "freeze",
    "refine",
}

REQUIRED_LOOKAHEAD_COLUMNS = {
    "available_data_cutoff_ts",
    "decision_ts",
    "entry_eligible_ts",
    "label_generated_ts",
    "source_bar_hash",
    "split_id",
}

SCIENTIFIC_EXPERIMENT_COLUMNS = {
    "event_ledger_hash",
    "nested_discovery_replay_status",
    "nested_discovery_replay_implemented",
    "nested_discovery_replay_passed",
    "edge_claim",
    "drift_status",
    "active_blockers",
    "registry_hash",
    "registry_run_manifest_hash",
    "registry_hash_chain_valid",
}
```

```

TRAIN_ONLY_EVIDENCE_COLUMNS = {
    "fit_scope",
    "fit_on_train_only",
    "split_protocol",
    "purged_embargo_applied",
    "selection_split",
}

RESEARCH_METRIC_COLUMNS = {
    "winrate",
    "avg_win",
    "avg_loss",
    "payoff_ratio",
    "profit_factor",
    "expectancy",
    "max_drawdown",
    "median_trade",
}

class GateInputsError(RuntimeError):
    pass

def main() -> int:
    parser = argparse.ArgumentParser(description="Conservative Director gate for Tradeo audit packages.")
    parser.add_argument("package", type=Path)
    parser.add_argument("--min-paper-trades-for-promotion", type=int, default=30)
    parser.add_argument("--min-fills-for-promotion", type=int, default=30)
    parser.add_argument("--max-duplicate-event-share", type=float, default=0.0)
    parser.add_argument("--json-output", type=Path, default=None)
    parser.add_argument("--markdown-output", type=Path, default=None)
    parser.add_argument(
        "--allow-blocked-exit-zero",
        action="store_true",
        help="Write BLOCKED status but exit 0. Use this for scheduled audits that must not stop the worker.",
    )
    args = parser.parse_args()

    try:
        rows = load_rows(args.package)
    except (FileNotFoundError, GateInputsError) as exc:
        result = result_payload(
            package=args.package,
            status="invalid",
            blockers=[str(exc)],
            summary={"director_gate": "invalid"},
        )
        write_outputs(result, args.json_output, args.markdown_output)
        print("DIRECTOR GATE INVALID")
        for blocker in result["blockers"]:
            print(f"- {blocker}")
        return 1

    blockers, summary = evaluate(
        rows,
        min_paper_trades_for_promotion=args.min_paper_trades_for_promotion,
        min_fills_for_promotion=args.min_fills_for_promotion,
        max_duplicate_event_share=args.max_duplicate_event_share,
    )
    status = "blocked" if blockers else "passed"
    result = result_payload(package=args.package, status=status, blockers=blockers, summary=summary)
    write_outputs(result, args.json_output, args.markdown_output)

    if blockers:
        print("DIRECTOR GATE BLOCKED")
        for blocker in blockers:
            print(f"- {blocker}")
        print(json.dumps(summary, indent=2, ensure_ascii=False))
        return 0 if args.allow_blocked_exit_zero else 2

    print("DIRECTOR GATE PASSED")
    print(json.dumps(summary, indent=2, ensure_ascii=False))
    return 0

def load_rows(package: Path) -> dict[str, list[dict[str, str]]]:
    rows = {
        "catalog": read_csv(package / "pattern_catalog.csv"),
        "events": read_csv(package / "pattern_events.csv"),
        "trades": read_csv(package / "paper_trades.csv"),
        "fills": read_csv(package / "ib_fills.csv"),
        "experiments": read_csv(package / "experiment_registry.csv"),
        "metrics": read_csv(package / "metrics_by_pattern.csv"),
        "metrics_by_regime": read_csv_optional(package / "metrics_by_regime.csv"),
        "metrics_by_entry_variant": read_csv_optional(package / "metrics_by_entry_variant.csv"),
    }
    return rows

def read_csv(path: Path) -> list[dict[str, str]]:
    with path.open(newline="", encoding="utf-8") as handle:
        reader = csv.DictReader(handle)
        if reader.fieldnames is None:
            raise GateInputsError(f"{path.name} has no CSV header")

```

```

        return list(reader)

def read_csv_optional(path: Path) -> list[dict[str, str]]:
    if not path.exists():
        return []
    return read_csv(path)

def evaluate(
    rows: dict[str, list[dict[str, str]]],
    *,
    min_paper_trades_for_promotion: int,
    min_fills_for_promotion: int,
    max_duplicate_event_share: float,
) -> tuple[list[str], dict[str, Any]]:
    catalog = rows["catalog"]
    events = rows["events"]
    trades = rows["trades"]
    fills = rows["fills"]
    experiments = rows["experiments"]
    metrics = rows["metrics"]
    metrics_by_regime = rows.get("metrics_by_regime", [])
    metrics_by_entry_variant = rows.get("metrics_by_entry_variant", [])

    blockers: list[str] = []
    statuses = {row.get("pattern_id", ""): (row.get("status") or "").strip().lower() for row in catalog}
    promoted = sorted(pid for pid, status in statuses.items() if status in EXECUTION_PROMOTION_STATUSES)
    unknown_statuses = sorted(
        {status for status in statuses.values() if status and status not in EXECUTION_PROMOTION_STATUSES | RESEARCH_ONLY_STATUSES}
    )

    if not catalog:
        blockers.append("pattern_catalog.csv has zero rows; no pattern inventory is auditable.")
    if not events:
        blockers.append("pattern_events.csv has zero rows; sample counts cannot be reconstructed from an event ledger.")
    if not trades:
        blockers.append("paper_trades.csv has zero rows; no pattern can be approved beyond research/watchlist.")
    if not fills:
        blockers.append("ib_fills.csv has zero rows; execution, commission, spread and slippage validation are unavailable.")
    if unknown_statuses:
        blockers.append("unknown pattern statuses require explicit Director mapping: " + preview(unknown_statuses))

    zero_trade_promotions = []
    for row in metrics:
        pid = row.get("pattern_id", "")
        if statuses.get(pid) not in EXECUTION_PROMOTION_STATUSES:
            continue
        if as_int(row.get("trade_count")) < min_paper_trades_for_promotion:
            zero_trade_promotions.append(pid)
    if zero_trade_promotions:
        blockers.append("promoted statuses require linked paper trades; offenders: " + preview(zero_trade_promotions))

    if promoted and len(fills) < min_fills_for_promotion:
        blockers.append(
            f"promoted statuses require at least {min_fills_for_promotion} IB Paper fills; package has {len(fills)}."
        )

    research_metric_offenders = research_metrics_in_operational_columns(metrics)
    if research_metric_offenders:
        blockers.append(
            "research R metrics are populated in operational metric columns while trade_count is zero; offenders: "
            + preview(research_metric_offenders)
        )

    event_columns = set().union(*(row.keys() for row in events)) if events else set()
    missing_lookahead_columns = sorted(REQUIRED_LOOKAHEAD_COLUMNS - event_columns)
    if missing_lookahead_columns:
        blockers.append("event ledger lacks anti-lookahead cutoff columns: " + ", ".join(missing_lookahead_columns))
    else:
        blank_lookahead_rows = count_rows_missing_any(events, REQUIRED_LOOKAHEAD_COLUMNS)
        if blank_lookahead_rows:
            blockers.append(
                f"{blank_lookahead_rows} event rows have blank anti-lookahead contract values."
            )

    close_entry_patterns = [
        row.get("pattern_id", "")
        for row in catalog
        if "close of the final bar" in (row.get("entry_rule_plaintext") or "").lower()
        or "latest close as signal/entry" in (row.get("entry_rule_plaintext") or "").lower()
    ]
    if close_entry_patterns and "entry_eligible_ts" not in event_columns:
        blockers.append(
            "same-bar close entry model lacks entry_eligible_ts proof; offenders: " + preview(close_entry_patterns)
        )

    duplicate_counts = Counter(
        (row.get("duplicate_group_id") or "").strip()
        for row in events
        if (row.get("duplicate_group_id") or "").strip()
    )
    duplicate_repeated_rows = sum(count for count in duplicate_counts.values() if count > 1)
    duplicate_share = duplicate_repeated_rows / len(events) if events else 0.0
    if duplicate_share > max_duplicate_event_share:
        blockers.append(

```

```

        f"duplicate_group_id repeats exceed gate: {duplicate_repeated_rows}/{len(events)} rows ({duplicate_share:.2%})."
    )

independent_events_by_pattern: Counter[str] = Counter()
non_independent_rows = 0
pending_independence_rows = 0
for row in events:
    value = (row.get("is_independent_sample") or "").strip().lower()
    if value == "true":
        independent_events_by_pattern[row.get("pattern_id", "")] += 1
    else:
        non_independent_rows += 1
        if value not in {"false", "0", "no"}:
            pending_independence_rows += 1
if non_independent_rows:
    blockers.append(f"{non_independent_rows} event rows are not verified independent samples.")
if pending_independence_rows:
    blockers.append(f"{pending_independence_rows} event rows have pending/unknown independence labels.")

sample_mismatch_patterns = []
for row in metrics:
    pid = row.get("pattern_id", "")
    reported = as_int(row.get("independent_sample_count"))
    observed = independent_events_by_pattern.get(pid, 0)
    if reported and observed and reported > observed:
        sample_mismatch_patterns.append(pid)
if sample_mismatch_patterns:
    blockers.append(
        f"independent_sample_count is not reconstructable from exported event rows for "
        f"{len(sample_mismatch_patterns)} patterns."
    )

regimes = {(row.get("market_regime") or "").strip().lower() for row in events if row.get("market_regime")}
if not regimes or regimes <= {"not_persisted", "unknown", "none"}:
    blockers.append("market_regime is not persisted; cross-regime robustness cannot be audited.")
sectors = {(row.get("sector") or "").strip().lower() for row in events if row.get("sector")}
if not sectors or sectors <= {"not_persisted", "unknown", "none"}:
    blockers.append("sector is not persisted; cross-sector concentration cannot be audited.")

missing_oos = [
    row.get("experiment_id", "")
    for row in experiments
    if not (row.get("out_of_sample_start") or "").strip()
    or not (row.get("out_of_sample_end") or "").strip()
]
if experiments and len(missing_oos) == len(experiments):
    blockers.append("no experiment has explicit out_of_sample_start/out_of_sample_end boundaries.")

train_only_evidence_missing = False
if experiments:
    experiment_columns = set().union(*(row.keys() for row in experiments))
    train_only_evidence_missing = not bool(experiment_columns & TRAIN_ONLY_EVIDENCE_COLUMNS)
    if train_only_evidence_missing:
        blockers.append(
            "no train-only fit evidence fields found; cannot prove scaler/clustering/R:R selection avoided OOS contamination."
        )

if len(experiments) >= 20 and not has_multiple_testing_columns(experiments):
    blockers.append(
        f"{len(experiments)} experiment variants were exported without multiple-testing adjusted evidence."
    )

scientific_contracts = scientific_contract_status(
    experiments=experiments,
    trades=trades,
    fills=fills,
)
blockers.extend(scientific_contracts["blockers"])

by_regime = bucket_breakdown(
    metrics_by_regime,
    bucket_column="market_regime",
    missing_reason=(
        "metrics_by_regime.csv is missing or empty; regenerate audit export with closed_lab_trades "
        "or an explicit no_closed_lab_trades reason."
    ),
)

by_entry_variant = bucket_breakdown(
    metrics_by_entry_variant,
    bucket_column="entry_variant_id",
    missing_reason=(
        "metrics_by_entry_variant.csv is missing or empty; regenerate audit export with "
        "signal.metadata.json.entry_variant_id on closed_lab_trades or an explicit no_closed_lab_trades reason."
    ),
)

pattern_recommendations = build_pattern_recommendations(
    catalog=catalog,
    events=events,
    metrics=metrics,
    metrics_by_regime=metrics_by_regime,
    metrics_by_entry_variant=metrics_by_entry_variant,
    min_paper_trades_for_promotion=min_paper_trades_for_promotion,
    statuses=statuses,
)

actionable_recommendations = package_recommendations(
    blockers=blockers,

```



```

        pattern_recommendations=pattern_recommendations,
        min_paper_trades_for_promotion=min_paper_trades_for_promotion,
        min_fills_for_promotion=min_fills_for_promotion,
    )

    summary = {
        "patterns": len(catalog),
        "events": len(events),
        "paper_trades": len(trades),
        "fills": len(fills),
        "experiments": len(experiments),
        "promoted_patterns": len(promoted),
        "promoted_pattern_ids_preview": promoted[:20],
        "unknown_statuses": unknown_statuses,
        "duplicate_repeated_rows": duplicate_repeated_rows,
        "duplicate_repeated_row_share": round(duplicate_share, 6),
        "non_independent_event_rows": non_independent_rows,
        "pending_independence_rows": pending_independence_rows,
        "sample_count_mismatch_patterns": len(sample_mismatch_patterns),
        "missing_oos_experiments": len(missing_oos),
        "missing_lookahead_columns": missing_lookahead_columns,
        "research_metric_column_offenders": len(research_metric_offenders),
        "train_only_evidence_missing": train_only_evidence_missing,
        "scientific_contracts": scientific_contracts,
        "by_regime": by_regime,
        "by_entry_variant": by_entry_variant,
        "actionable_recommendations": actionable_recommendations,
        "pattern_recommendation_count": len(pattern_recommendations),
        "pattern_recommendations": pattern_recommendations,
        "director_gate": "blocked" if blockers else "passed",
        "director_gate_status": "blocked" if blockers else "passed",
        "schema_valid": True,
        "package_valid": True,
        "promotion_allowed": not blockers,
        "active_blocker_count": len(blockers),
    }
    return blockers, summary

def scientific_contract_status(
    *,
    experiments: list[dict[str, str]],
    trades: list[dict[str, str]],
    fills: list[dict[str, str]],
) -> dict[str, Any]:
    blockers: list[str] = []
    experiment_columns = set().union(*(row.keys() for row in experiments)) if experiments else set()
    missing_columns = sorted(SCIENTIFIC_EXPERIMENT_COLUMNS - experiment_columns)
    if experiments and missing_columns:
        blockers.append("experiment_registry.csv lacks scientific contract columns: " + ", ".join(missing_columns))

    event_ledger_missing = count_blank(experiments, "event_ledger_hash")
    if experiments and event_ledger_missing:
        blockers.append(f"{event_ledger_missing} experiment rows lack event_ledger_hash.")

    nested_missing = 0
    nested_not_passed = 0
    for row in experiments:
        implemented = truthy(row.get("nested_discovery_replay_implemented"))
        passed = truthy(row.get("nested_discovery_replay_passed")) or (
            str(row.get("nested_discovery_replay_status") or "").strip().lower() == "passed"
        )
        if not any(
            str(row.get(column) or "").strip()
            for column in (
                "nested_discovery_replay_status",
                "nested_discovery_replay_implemented",
                "nested_discovery_replay_passed",
            )
        ):
            nested_missing += 1
        elif not implemented or not passed:
            nested_not_passed += 1
    if nested_missing:
        blockers.append(f"{nested_missing} experiment rows lack nested_discovery_replay evidence.")
    if nested_not_passed:
        blockers.append(f"{nested_not_passed} experiment rows have nested_discovery_replay not implemented/passed.")

    inflated_edge_claims = [
        row.get("experiment_id", "")
        for row in experiments
        if str(row.get("edge_claim") or "").strip()
        and str(row.get("edge_claim") or "").strip() != "NO_DEMOSTRADO"
    ]
    if inflated_edge_claims:
        blockers.append("edge_claim is inflated before paper/live proof; offenders: " + preview(inflated_edge_claims))
    edge_claim_missing = count_blank(experiments, "edge_claim")
    if experiments and edge_claim_missing:
        blockers.append(f"{edge_claim_missing} experiment rows lack edge_claim=NO_DEMOSTRADO.")

    degrading = [
        row.get("experiment_id", "")
        for row in experiments
        if str(row.get("drift_status") or "").strip().lower() in {"degrading", "regressing", "deteriorating"}
    ]
    if degrading:

```

```

        blockers.append("drift_status is degrading; offenders: " + preview(degrading))

active_blocker_rows = [
    row.get("experiment_id", "")
    for row in experiments
    if has_active_blocker_text(row.get("active_blockers"))
]
if active_blocker_rows:
    blockers.append("experiment rows report active blockers: " + preview(active_blocker_rows))

registry_hash_missing = count_blank(experiments, "registry_hash")
registry_run_manifest_missing = count_blank(experiments, "registry_run_manifest_hash")
registry_chain_invalid = [
    row.get("experiment_id", "")
    for row in experiments
    if row.get("registry_hash_chain_valid") and not truthy(row.get("registry_hash_chain_valid"))
]
if experiments and registry_hash_missing:
    blockers.append(f"{registry_hash_missing} experiment rows lack registry_hash.")
if experiments and registry_run_manifest_missing:
    blockers.append(f"{registry_run_manifest_missing} experiment rows lack registry_run_manifest_hash.")
if registry_chain_invalid:
    blockers.append("registry hash chain is invalid; offenders: " + preview(registry_chain_invalid))

fill_trade_ids = {str(row.get("trade_id") or "").strip() for row in fills if row.get("trade_id")}
trade_ids = [str(row.get("trade_id") or "").strip() for row in trades if row.get("trade_id")]
missing_fill_rows = [trade_id for trade_id in trade_ids if trade_id and trade_id not in fill_trade_ids]
cost_missing_rows = [
    row.get("trade_id", "")
    for row in trades
    if any(not str(row.get(field) or "").strip() for field in ("commission", "estimated_spread_cost"))
]
slippage_missing_rows = [
    row.get("trade_id", "")
    for row in trades
    if not str(row.get("estimated_slippage") or "").strip()
]
if missing_fill_rows:
    blockers.append("paper trades lack reconciled fill rows; offenders: " + preview(missing_fill_rows))
if cost_missing_rows:
    blockers.append("paper trades lack commission/spread cost provenance; offenders: " + preview(cost_missing_rows))
if slippage_missing_rows:
    blockers.append("paper trades lack slippage provenance; offenders: " + preview(slippage_missing_rows))

return {
    "blockers": blockers,
    "event_ledger_hash_rows": len(experiments) - event_ledger_missing if experiments else 0,
    "nested_replay_passed_rows": len(experiments) - nested_missing - nested_not_passed if experiments else 0,
    "registry_hash_rows": len(experiments) - registry_hash_missing if experiments else 0,
    "registry_run_manifest_hash_rows": (
        len(experiments) - registry_run_manifest_missing if experiments else 0
    ),
    "paper_trades_with_fill_rows": len(trade_ids) - len(missing_fill_rows),
    "paper_trades": len(trade_ids),
    "promotion_allowed": not blockers,
}

def count_blank(rows: list[dict[str, str]], column: str) -> int:
    return sum(1 for row in rows if not str(row.get(column) or "").strip())

def count_rows_missing_any(rows: list[dict[str, str]], columns: set[str]) -> int:
    return sum(
        1
        for row in rows
        if any(not str(row.get(column) or "").strip() for column in columns)
    )

def has_active_blocker_text(value: str | None) -> bool:
    text = str(value or "").strip()
    return bool(text and text not in {"[]", "{}", "null", "None", "none"})

def research_metrics_in_operational_columns(metrics: list[dict[str, str]]) -> list[str]:
    offenders = []
    for row in metrics:
        if as_int(row.get("trade_count")) != 0:
            continue
        if any(nonzero_or_nonblank(row.get(column)) for column in RESEARCH_METRIC_COLUMNS):
            offenders.append(row.get("pattern_id", "unknown"))
    return offenders

def bucket_breakdown(
    rows: list[dict[str, str]],
    *,
    bucket_column: str,
    missing_reason: str,
) -> dict[str, Any]:
    if not rows:
        return {"available": False, "buckets": [], "empty_reason": missing_reason}
    available_rows = [
        row
        for row in rows
    ]

```

```

        if truthy(row.get("analysis_available")) and as_int(row.get("trade_count")) > 0
    ]
    if not available_rows:
        reason = next((row.get("empty_reason", "").strip() for row in rows if row.get("empty_reason")), "")
        return {
            "available": False,
            "buckets": [],
            "empty_reason": reason or missing_reason,
        }
    ranked = sorted(
        available_rows,
        key=lambda row: (
            as_float(row.get("expectancy_r")),
            as_float(row.get("net_pnl")),
            as_int(row.get("trade_count")),
        ),
        reverse=True,
    )
    buckets = [bucket_summary_row(row, bucket_column) for row in ranked[:20]]
    return {
        "available": True,
        "bucket_count": len(available_rows),
        "best": bucket_summary_row(ranked[0], bucket_column),
        "worst": bucket_summary_row(ranked[-1], bucket_column),
        "buckets": buckets,
        "empty_reason": "",
    }

def bucket_summary_row(row: dict[str, str], bucket_column: str) -> dict[str, Any]:
    return {
        "pattern_id": row.get("pattern_id", ""),
        "bucket": row.get(bucket_column, "") or "unknown",
        "trade_count": as_int(row.get("trade_count")),
        "net_pnl": as_float(row.get("net_pnl")),
        "expectancy_r": as_float(row.get("expectancy_r")),
        "winrate": as_float(row.get("winrate")),
        "profit_factor": as_float(row.get("profit_factor")),
        "max_drawdown": as_float(row.get("max_drawdown")),
    }

def build_pattern_recommendations(
    *,
    catalog: list[dict[str, str]],
    events: list[dict[str, str]],
    metrics: list[dict[str, str]],
    metrics_by_regime: list[dict[str, str]],
    metrics_by_entry_variant: list[dict[str, str]],
    min_paper_trades_for_promotion: int,
    statuses: dict[str, str],
) -> list[dict[str, Any]]:
    metrics_by_pattern = {row.get("pattern_id", ""): row for row in metrics}
    events_by_pattern: dict[str, list[dict[str, str]]] = {}
    for row in events:
        events_by_pattern.setdefault(row.get("pattern_id", ""), []).append(row)
    best_regime_by_pattern, worst_regime_by_pattern = pattern_bucket_extremes(
        metrics_by_regime,
        bucket_column="market_regime",
    )
    best_variant_by_pattern, worst_variant_by_pattern = pattern_bucket_extremes(
        metrics_by_entry_variant,
        bucket_column="entry_variant_id",
    )

    recommendations: list[dict[str, Any]] = []
    for row in catalog:
        pid = row.get("pattern_id", "")
        metric = metrics_by_pattern.get(pid, {})
        trade_count = as_int(metric.get("trade_count"))
        independent_samples = as_int(metric.get("independent_sample_count"))
        ticker_count = as_int(metric.get("ticker_count"))
        status = statuses.get(pid, "")
        trades_remaining = max(0, min_paper_trades_for_promotion - trade_count)
        event_blockers = event_gate_blockers(events_by_pattern.get(pid, []))
        actions: list[str] = []

        active_research_statuses = {
            "lab",
            "lab_watchlist",
            "lab_candidate",
            "director_review",
            "needs_confirmation",
            "confirmed_candidate",
        }
        rejected_statuses = {"rejected", "discard", "freeze", "failed_confirmation"}
        if status in EXECUTION_PROMOTION_STATUSES and trades_remaining:
            actions.append(
                f"freeze promoted status; needs {trades_remaining} more linked paper trades before promotion evidence exists"
            )
        elif (
            status in active_research_statuses
            and trade_count == 0
            and independent_samples >= 100
            and ticker_count >= 8
        ):

```

```

        actions.append(
            "promising research candidate without confirmation; collect controlled paper trades before ranking by PnL"
        )
    elif status in rejected_statuses and trade_count == 0 and independent_samples >= 100:
        actions.append(
            "broad rejected research memory; keep frozen unless a new hypothesis explains the prior rejection"
        )
    elif trades_remaining:
        actions.append(f"collect {trades_remaining} more closed paper trades before Director review")

    if event_blockers["entry_gate"]:
        actions.append(
            f"debug entry gate before more paper allocation; {event_blockers['entry_gate']} events mention entry-gate blockage"
        )
    if event_blockers["volume"]:
        actions.append(
            f"review volume/liquidity filter; {event_blockers['volume']} events mention volume blockage"
        )
    if not best_variant_by_pattern.get(pid):
        actions.append("entry_variant performance unavailable; missing closed_lab_trades with entry_variant_id")
    if not best_regime_by_pattern.get(pid):
        actions.append("regime performance unavailable; missing closed_lab_trades with regime_key")

    if not actions:
        actions.append("review Director packet; no automatic promotion")

    recommendations.append(
        {
            "pattern_id": pid,
            "pattern_name": row.get("pattern_name", ""),
            "status": status,
            "independent_sample_count": independent_samples,
            "ticker_count": ticker_count,
            "trade_count": trade_count,
            "trades_remaining_for_promotion": trades_remaining,
            "best_entry_variant": best_variant_by_pattern.get(pid, {}),
            "worst_entry_variant": worst_variant_by_pattern.get(pid, {}),
            "best_regime": best_regime_by_pattern.get(pid, {}),
            "worst_regime": worst_regime_by_pattern.get(pid, {}),
            "actions": actions,
        }
    )
    return sorted(
        recommendations,
        key=lambda item: (
            int(item["trades_remaining_for_promotion"] == 0),
            int(item["status"] in EXECUTION_PROMOTION_STATUSES),
            int(item["independent_sample_count"]),
            int(item["ticker_count"]),
        ),
        reverse=True,
    )
)

def pattern_bucket_extremes(
    rows: list[dict[str, str]],
    *,
    bucket_column: str,
) -> tuple[dict[str, dict[str, Any]], dict[str, dict[str, Any]]]:
    grouped: dict[str, list[dict[str, str]]] = {}
    for row in rows:
        if not truthy(row.get("analysis_available")) or as_int(row.get("trade_count")) <= 0:
            continue
        grouped.setdefault(row.get("pattern_id", ""), []).append(row)
    best: dict[str, dict[str, Any]] = {}
    worst: dict[str, dict[str, Any]] = {}
    for pid, pattern_rows in grouped.items():
        ranked = sorted(
            pattern_rows,
            key=lambda row: (
                as_float(row.get("expectancy_r")),
                as_float(row.get("net_pnl")),
                as_int(row.get("trade_count")),
            ),
            reverse=True,
        )
        best[pid] = bucket_summary_row(ranked[0], bucket_column)
        worst[pid] = bucket_summary_row(ranked[-1], bucket_column)
    return best, worst

def event_gate_blockers(rows: list[dict[str, str]]) -> dict[str, int]:
    entry_gate = 0
    volume = 0
    for row in rows:
        text = " ".join(
            str(row.get(column, "")).lower()
            for column in ("reason_not_traded", "notes")
        )
        if "entry gate" in text or "entry_gate" in text or "no_operational_trigger" in text:
            entry_gate += 1
        if "volume" in text or "liquidity" in text:
            volume += 1
    return {"entry_gate": entry_gate, "volume": volume}

```

```

def package_recommendations(
    *,
    blockers: list[str],
    pattern_recommendations: list[dict[str, Any]],
    min_paper_trades_for_promotion: int,
    min_fills_for_promotion: int,
) -> list[dict[str, Any]]:
    recommendations: list[dict[str, Any]] = []
    if any("paper_trades.csv has zero rows" in blocker for blocker in blockers):
        recommendations.append(
            {
                "action": "keep_all_patterns_research_only",
                "priority": "P0",
                "reason": "paper_trades.csv has zero rows; no closed_lab_trades can confirm any pattern",
                "required_data": f"at least {min_paper_trades_for_promotion} linked paper trades for any promoted pattern",
            }
        )
    if any("ib_fills.csv has zero rows" in blocker for blocker in blockers):
        recommendations.append(
            {
                "action": "ingest_ib_paper_fills",
                "priority": "P0",
                "reason": "fills, commissions, spread and slippage are unavailable",
                "required_data": f"at least {min_fills_for_promotion} anonymized IB Paper fills for promoted patterns",
            }
        )
    if any("duplicate_group_id repeats" in blocker for blocker in blockers):
        recommendations.append(
            {
                "action": "deduplicate_or_explain_events",
                "priority": "P0",
                "reason": "duplicate event rows can inflate sample counts and apparent edge",
            }
        )
    if any("out_of_sample" in blocker or "train-only" in blocker for blocker in blockers):
        recommendations.append(
            {
                "action": "add_oos_and_train_only_evidence",
                "priority": "P0",
                "reason": "Director cannot separate discovery from validation without split evidence",
            }
        )
    promising = [
        item
        for item in pattern_recommendations
        if any("promising research candidate" in action for action in item.get("actions", []))
    ][:10]
    if promising:
        recommendations.append(
            {
                "action": "prioritize_controlled_confirmation",
                "priority": "P1",
                "patterns": [item["pattern_id"] for item in promising],
                "reason": "these patterns have samples/tickers but no paper confirmation",
            }
        )
    if not recommendations:
        recommendations.append(
            {
                "action": "prepare_director_review_packet",
                "priority": "P2",
                "reason": "gate did not find blocking package-level issues; Director still decides manually",
            }
        )
    return recommendations

def nonzero_or_nonblank(value: str | None) -> bool:
    text = (value or "").strip()
    if not text:
        return False
    try:
        return float(text) != 0.0
    except ValueError:
        return True

def result_payload(package: Path, status: str, blockers: list[str], summary: dict[str, Any]) -> dict[str, Any]:
    package_valid = status != "invalid"
    promotion_allowed = bool(package_valid and status == "passed" and not blockers)
    summary.setdefault("schema_valid", package_valid)
    summary.setdefault("package_valid", package_valid)
    summary.setdefault("director_gate_status", status)
    summary.setdefault("promotion_allowed", promotion_allowed)
    payload = {
        "created_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "package": str(package),
        "status": status,
        "schema_valid": package_valid,
        "package_valid": package_valid,
        "director_gate_status": status,
        "promotion_allowed": promotion_allowed,
        "blockers": blockers,
        "summary": summary,
    }
    if summary.get("actionable_recommendations"):

```

```

        payload["recommendations"] = summary["actionable_recommendations"]
    if summary.get("pattern_recommendations"):
        payload["pattern_recommendations"] = summary["pattern_recommendations"]
    return payload

def write_outputs(result: dict[str, Any], json_output: Path | None, markdown_output: Path | None) -> None:
    if json_output:
        json_output.parent.mkdir(parents=True, exist_ok=True)
        json_output.write_text(json.dumps(result, indent=2, ensure_ascii=False), encoding="utf-8")
    if markdown_output:
        markdown_output.parent.mkdir(parents=True, exist_ok=True)
        lines = [
            "# Director Gate Result",
            "",
            f"- Created at: `{result['created_at']}`",
            f"- Package: `{result['package']}`",
            f"- Status: `{result['status']}`",
            "",
            "## Blockers",
            ""
        ]
        blockers = result.get("blockers") or []
        if blockers:
            lines.extend(f"- {blocker}" for blocker in blockers)
        else:
            lines.append("- None")
        recommendations = result.get("recommendations") or []
        lines.extend(["", "## Recommendations", ""])
        if recommendations:
            for recommendation in recommendations:
                action = recommendation.get("action", "review")
                priority = recommendation.get("priority", "")
                reason = recommendation.get("reason", "")
                lines.append(f"- `{priority}` `{action}`: {reason}".strip())
        else:
            lines.append("- None")
        pattern_recommendations = result.get("pattern_recommendations") or []
        lines.extend(["", "## Pattern Actions", ""])
        if pattern_recommendations:
            for item in pattern_recommendations[:40]:
                actions = "; ".join(str(action) for action in item.get("actions", [])[:3])
                lines.append(
                    f"- `{item.get('pattern_id')}` trades={item.get('trade_count')} "
                    f"remaining={item.get('trades_remaining_for_promotion')}: {actions}"
                )
        else:
            lines.append("- None")
        lines.extend(["", "## Summary", "", "```json", json.dumps(result.get("summary", {}), indent=2, ensure_ascii=False), "```", ""])
        markdown_output.write_text("\n".join(lines), encoding="utf-8")

def as_int(value: str | None) -> int:
    try:
        text = (value or "").strip()
        return int(float(text)) if text else 0
    except ValueError:
        return 0

def as_float(value: str | None) -> float:
    try:
        text = (value or "").strip()
        return float(text) if text else 0.0
    except ValueError:
        return 0.0

def truthy(value: str | None) -> bool:
    return (value or "").strip().lower() in {"true", "1", "yes", "y"}

def has_multiple_testing_columns(rows: list[dict[str, str]]) -> bool:
    if not rows:
        return False
    columns = set().union(*(row.keys() for row in rows))
    wanted = {
        "multiple_testing_adjusted_p_value",
        "adjusted_p_value",
        "deflated_sharpe",
        "permutation_p_value",
    }
    return bool(columns & wanted)

def preview(values: list[str], limit: int = 10) -> str:
    shown = ", ".join(values[:limit])
    if len(values) > limit:
        return f"{shown} ({len(values) - limit} more)"
    return shown

if __name__ == "__main__":
    sys.exit(main())

```

Full Document: `research/audit\_bridge/export\_audit\_package.py`

`research/audit\_bridge/export\_audit\_package.py` ? 2373 lines, 103922 bytes, sha256 prefix `a291a0c77fc90d5b`

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import base64
import csv
import hashlib
import json
import re
import statistics
import subprocess
import sys
from collections import defaultdict
from datetime import datetime, timezone
from pathlib import Path
from typing import Any
from urllib.request import Request, urlopen
from zoneinfo import ZoneInfo

AUDIT_ID = "2026-06-07_ib_paper_patterns"
ROOT = Path(__file__).resolve().parents[2]
BRIDGE = ROOT / "research" / "audit_bridge"
REQUEST_ROOT = BRIDGE / "requests" / AUDIT_ID
CONFIG_ROOT = REQUEST_ROOT / "config_snapshot"
LOCAL_TZ = ZoneInfo("Europe/Madrid")
EVIDENCE_NEAR_MISS_SHADOW = "near_miss_shadow"
EVIDENCE_SHADOW_NO_ORDER = "shadow_no_order"
EVIDENCE_IBKR_PAPER_ORDER = "ibkr_paper_order"
EVIDENCE_IBKR_PAPER_FILL = "ibkr_paper_fill"
EVIDENCE_QUALITY_STANDARD = "standard"
EVIDENCE_QUALITY_DEGRADED = "degraded"
FILL_PROVENANCE_BROKER_EXECUTION = "broker_execution"
FILL_PROVENANCE_BROKER_STATEMENT_IMPORT = "broker_statement_import"
LAB_SHADOW_EXECUTION_MODE = "lab_shadow_observation"
REAL_FILL_PROVENANCE = {
    FILL_PROVENANCE_BROKER_EXECUTION,
    FILL_PROVENANCE_BROKER_STATEMENT_IMPORT,
}
FILL_ID_KEYS = (
    "fill_id",
    "broker_fill_id",
    "ib_fill_id",
    "execution_id",
    "exec_id",
    "ib_exec_id",
    "broker_execution_id",
    "execution_hash",
    "broker_execution_hash",
    "ib_execution_hash",
    "fill_id_hash",
)
ENTRY_FILL_ID_KEYS = tuple(f"entry_{key}" for key in FILL_ID_KEYS) + FILL_ID_KEYS
EXIT_FILL_ID_KEYS = tuple(f"exit_{key}" for key in FILL_ID_KEYS) + FILL_ID_KEYS
BROKER_TIMESTAMP_KEYS = (
    "broker_execution_time",
    "ib_execution_time",
    "broker_executed_at",
    "execution_time",
    "executed_at",
    "fill_time",
    "broker_fill_time",
    "statement_execution_time",
)
ENTRY_BROKER_TIMESTAMP_KEYS = tuple(f"entry_{key}" for key in BROKER_TIMESTAMP_KEYS) + (
    "entry_fill_time",
) + BROKER_TIMESTAMP_KEYS
EXIT_BROKER_TIMESTAMP_KEYS = tuple(f"exit_{key}" for key in BROKER_TIMESTAMP_KEYS) + (
    "exit_fill_time",
) + BROKER_TIMESTAMP_KEYS
COMMISSION_KEYS = (
    "commission",
    "commission_usd",
    "broker_commission",
    "ib_commission",
    "fees",
    "other_fees",
)
EVIDENCE_NESTED_METADATA_KEYS = (
    "broker_execution",
    "execution",
    "execution_report",
    "execution_observation",
    "broker_statement",
    "statement_row",
)
PATTERN_CATALOG_COLUMNS = [
    "pattern_id",
    "pattern_name",

```

```

"pattern_version",
"description",
"hypothesis",
"detection_rule_plaintext",
"entry_rule_plaintext",
"exit_rule_plaintext",
"stop_rule_plaintext",
"take_profit_rule_plaintext",
"time_stop_rule_plaintext",
"timeframe",
"asset_class",
"universe",
"session_filter",
"min_volume_filter",
"min_price_filter",
"uses_fundamental_data",
"uses_news_data",
"uses_options_data",
"uses_future_data",
"known_risks",
"status",
"created_at",
"first_seen",
"last_seen",
]

```

```

PATTERN_EVENTS_COLUMNS = [
"event_id",
"pattern_id",
"detected_at",
"market_timestamp",
"timezone",
"ticker",
"exchange",
"timeframe",
"bar_open_time",
"bar_close_time",
"signal_price",
"bid_at_signal",
"ask_at_signal",
"spread_at_signal",
"volume_at_signal",
"atr_at_signal",
"volatility_context",
"market_regime",
"sector",
"was_trade_triggered",
"trade_id",
"reason_not_traded",
"duplicate_group_id",
"is_independent_sample",
"data_available_at_signal",
"available_data_cutoff_ts",
"decision_ts",
"entry_eligible_ts",
"label_generated_ts",
"source_bar_hash",
"split_id",
"features_used_json",
"notes",
]

```

```

PAPER_TRADES_COLUMNS = [
"trade_id",
"event_id",
"pattern_id",
"evidence_type",
"evidence_quality",
"ticker",
"side",
"quantity",
"entry_signal_time",
"entry_order_time",
"entry_fill_time",
"entry_signal_price",
"entry_order_type",
"entry_limit_price",
"entry_fill_price",
"exit_signal_time",
"exit_order_time",
"exit_fill_time",
"exit_order_type",
"exit_limit_price",
"exit_fill_price",
"exit_reason",
"gross_pnl",
"commission",
"estimated_spread_cost",
"estimated_slippage",
"other_fees",
"net_pnl",
"return_pct",
"mae",
"mfe",
"holding_period_seconds",
"risk_amount",
]

```



```

        "r_multiple",
        "account_equity_snapshot",
        "position_size_method",
        "notes",
    ]

IB_FILLS_COLUMNS = [
    "fill_id_hash",
    "trade_id",
    "order_id_hash",
    "ib_execution_time",
    "timezone",
    "ticker",
    "side",
    "quantity",
    "price",
    "commission",
    "liquidity_flag",
    "exchange",
    "order_type",
    "account_id_redacted",
    "raw_status",
    "notes",
]

EXPERIMENT_COLUMNS = [
    "experiment_id",
    "pattern_id",
    "variant_id",
    "created_at",
    "tested_at",
    "status",
    "reason_status",
    "parameters_json",
    "dataset_start",
    "dataset_end",
    "in_sample_start",
    "in_sample_end",
    "out_of_sample_start",
    "out_of_sample_end",
    "paper_live_start",
    "paper_live_end",
    "number_of_assets_tested",
    "number_of_events",
    "number_of_trades",
    "gross_pnl",
    "net_pnl",
    "winrate",
    "profit_factor",
    "sharpe",
    "sortino",
    "max_drawdown",
    "candidate_trial_count",
    "global_trial_count",
    "adjusted_p_value",
    "wrc_p_value",
    "spa_p_value",
    "fit_scope",
    "score_input_scope",
    "event_ledger_hash",
    "nested_discovery_replay_status",
    "nested_discovery_replay_implemented",
    "nested_discovery_replay_passed",
    "edge_claim",
    "drift_status",
    "active_blockers",
    "registry_path",
    "registry_hash",
    "registry_previous_hash",
    "registry_run_manifest_hash",
    "registry_hash_chain_valid",
    "notes",
]

METRICS_BY_PATTERN_COLUMNS = [
    "pattern_id",
    "sample_count",
    "independent_sample_count",
    "trade_count",
    "ticker_count",
    "sector_count",
    "first_seen",
    "last_seen",
    "gross_pnl",
    "net_pnl",
    "avg_trade",
    "median_trade",
    "std_trade",
    "winrate",
    "avg_win",
    "avg_loss",
    "payoff_ratio",
    "profit_factor",
    "expectancy",
    "max_drawdown",
    "max_consecutive_losses",

```

```

    "best_trade",
    "worst_trade",
    "top_5_trades_pnl",
    "bottom_5_trades_pnl",
    "pnl_without_best_trade",
    "pnl_without_top_5_trades",
    "pnl_without_worst_trade",
    "sharpe",
    "sortino",
    "calmar",
    "avg_holding_period_seconds",
    "notes",
]

METRICS_BY_TICKER_COLUMNS = [
    "pattern_id",
    "ticker",
    "trade_count",
    "event_count",
    "net_pnl",
    "winrate",
    "profit_factor",
    "avg_trade",
    "max_drawdown",
    "first_seen",
    "last_seen",
    "notes",
]

METRICS_BY_PERIOD_COLUMNS = [
    "period_start",
    "period_end",
    "period_type",
    "pattern_id",
    "event_count",
    "trade_count",
    "gross_pnl",
    "net_pnl",
    "winrate",
    "profit_factor",
    "max_drawdown",
    "market_regime",
    "notes",
]

METRICS_BY_REGIME_COLUMNS = [
    "pattern_id",
    "market_regime",
    "analysis_available",
    "empty_reason",
    "trade_count",
    "event_count",
    "gross_pnl",
    "net_pnl",
    "avg_trade",
    "expectancy_r",
    "winrate",
    "profit_factor",
    "max_drawdown",
    "best_trade",
    "worst_trade",
    "first_seen",
    "last_seen",
    "notes",
]

METRICS_BY_ENTRY_VARIANT_COLUMNS = [
    "pattern_id",
    "entry_variant_id",
    "analysis_available",
    "empty_reason",
    "trade_count",
    "event_count",
    "gross_pnl",
    "net_pnl",
    "avg_trade",
    "expectancy_r",
    "winrate",
    "profit_factor",
    "max_drawdown",
    "best_trade",
    "worst_trade",
    "first_seen",
    "last_seen",
    "notes",
]

def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--audit-id", default=AUDIT_ID)
    parser.add_argument("--api-url", default=None)
    parser.add_argument("--pattern-limit", type=int, default=500)
    parser.add_argument("--match-limit", type=int, default=500)
    args = parser.parse_args()

```

```

env = read_env(ROOT / ".env")
api_url = (args.api_url or env.get("TRADEO_API_URL") or "http://localhost:8000/api").rstrip("/")
user = env.get("TRADEO_ADMIN_USERNAME", "admin")
password = env.get("TRADEO_ADMIN_PASSWORD", "change-me")
package = BRIDGE / "requests" / args.audit_id
package.mkdir(parents=True, exist_ok=True)
(package / "config_snapshot").mkdir(parents=True, exist_ok=True)

patterns = api_get(api_url, f"/research/discovered-patterns?limit={args.pattern_limit}", user, password)
runs = api_get(api_url, "/research/runs?limit=200", user, password)
matches = api_get(api_url, f"/research/current-matches?limit={args.match_limit}", user, password)
laboratory_overview = api_get_optional(api_url, "/laboratory/overview", user, password, default={})

examples_by_pattern: dict[int, list[dict[str, Any]]] = {}
for pattern in patterns:
    examples_by_pattern[int(pattern["id"])] = api_get(
        api_url, f"/research/discovered-patterns/{pattern['id']}/examples", user, password
    )

universe_symbols = read_universe_symbols(ROOT / "data" / "universe_us_mid_small.csv")
git_commit = run_git("rev-parse", "HEAD")
git_branch = run_git("branch", "--show-current")
created_at = datetime.now(LOCAL_TZ).isoformat(timespec="seconds")
generated_utc = datetime.now(timezone.utc).isoformat(timespec="seconds")

context = {
    "audit_id": args.audit_id,
    "created_at": created_at,
    "generated_utc": generated_utc,
    "git_commit": git_commit,
    "git_branch": git_branch,
    "env": env,
    "api_url": api_url,
    "universe_symbols": universe_symbols,
}

pattern_catalog_rows = build_pattern_catalog(patterns, examples_by_pattern, context)
event_rows = build_pattern_events(patterns, examples_by_pattern, matches, context)
paper_trade_rows = build_paper_trades(patterns, laboratory_overview)
exported_trade_ids = {
    str(row.get("trade_id") or "")
    for row in paper_trade_rows
    if str(row.get("trade_id") or "").strip()
}
fill_rows = build_ib_fills(laboratory_overview, exported_trade_ids=exported_trade_ids)
experiment_rows = build_experiment_registry(patterns, examples_by_pattern, runs, context)
metrics_pattern_rows = build_metrics_by_pattern(
    patterns,
    examples_by_pattern,
    paper_trade_rows,
    event_rows=event_rows,
)
metrics_ticker_rows = build_metrics_by_ticker(examples_by_pattern)
metrics_period_rows = build_metrics_by_period(examples_by_pattern)
metrics_regime_rows = build_metrics_by_regime(patterns, event_rows, paper_trade_rows)
metrics_entry_variant_rows = build_metrics_by_entry_variant(patterns, paper_trade_rows)

write_csv(package / "pattern_catalog.csv", PATTERN_CATALOG_COLUMNS, pattern_catalog_rows)
write_csv(package / "pattern_events.csv", PATTERN_EVENTS_COLUMNS, event_rows)
write_csv(package / "paper_trades.csv", PAPER_TRADES_COLUMNS, paper_trade_rows)
write_csv(package / "ib_fills.csv", IB_FILLS_COLUMNS, fill_rows)
write_csv(package / "experiment_registry.csv", EXPERIMENT_COLUMNS, experiment_rows)
write_csv(package / "metrics_by_pattern.csv", METRICS_BY_PATTERN_COLUMNS, metrics_pattern_rows)
write_csv(package / "metrics_by_ticker.csv", METRICS_BY_TICKER_COLUMNS, metrics_ticker_rows)
write_csv(package / "metrics_by_period.csv", METRICS_BY_PERIOD_COLUMNS, metrics_period_rows)
write_csv(package / "metrics_by_regime.csv", METRICS_BY_REGIME_COLUMNS, metrics_regime_rows)
write_csv(package / "metrics_by_entry_variant.csv", METRICS_BY_ENTRY_VARIANT_COLUMNS, metrics_entry_variant_rows)

write_audit_request(package, context, patterns, pattern_catalog_rows, metrics_pattern_rows, experiment_rows, event_rows)
write_audit_summary_for_chatgpt(
    package,
    pattern_catalog_rows,
    event_rows,
    experiment_rows,
    metrics_pattern_rows,
    metrics_ticker_rows,
    metrics_period_rows,
    metrics_regime_rows,
    metrics_entry_variant_rows,
    paper_trade_rows,
    fill_rows,
)
write_manifest(
    package,
    context,
    patterns,
    event_rows,
    experiment_rows,
    paper_trade_rows,
    fill_rows,
    metrics_regime_rows,
    metrics_entry_variant_rows,
)
write_code_references(package, patterns)
write_config_snapshot(package / "config_snapshot", context)
write_supporting_docs(package, context, patterns, event_rows, experiment_rows, paper_trade_rows, fill_rows)

```

```

write_hashes(package)

print(json.dumps({
    "audit_id": args.audit_id,
    "patterns_exported": len(patterns),
    "events_exported": len(event_rows),
    "paper_trades_exported": len(paper_trade_rows),
    "fills_exported": len(fill_rows),
    "experiments_exported": len(experiment_rows),
    "metrics_by_regime_rows": len(metrics_regime_rows),
    "metrics_by_entry_variant_rows": len(metrics_entry_variant_rows),
    "package": str(package.relative_to(ROOT)),
}, indent=2))
return 0

def read_env(path: Path) -> dict[str, str]:
    env: dict[str, str] = {}
    if not path.exists():
        return env
    for line in path.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        value = value.strip().strip('"').strip("'")
        env[key.strip()] = value
    return env

def api_get(api_url: str, path: str, user: str, password: str) -> Any:
    token = base64.b64encode(f"{user}:{password}".encode("utf-8")).decode("ascii")
    request = Request(f"{api_url}{path}", headers={"Authorization": f"Basic {token}"})
    with urlopen(request, timeout=120) as response: # noqa: S310 - local admin API export
        return json.loads(response.read().decode("utf-8"))

def api_get_optional(api_url: str, path: str, user: str, password: str, *, default: Any) -> Any:
    try:
        return api_get(api_url, path, user, password)
    except Exception: # noqa: BLE001
        return default

def run_git(*args: str) -> str:
    return subprocess.check_output(["git", *args], cwd=ROOT, text=True).strip()

def read_universe_symbols(path: Path) -> list[str]:
    if not path.exists():
        return []
    with path.open(newline="", encoding="utf-8") as handle:
        return [row["symbol"].strip().upper() for row in csv.DictReader(handle) if row.get("symbol")]

def write_csv(path: Path, columns: list[str], rows: list[dict[str, Any]]) -> None:
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=columns, extrasaction="ignore")
        writer.writeheader()
        for row in rows:
            writer.writerow({column: csv_value(row.get(column, "")) for column in columns})

def csv_value(value: Any) -> str:
    if value is None:
        return ""
    if isinstance(value, bool):
        return "true" if value else "false"
    if isinstance(value, (dict, list)):
        return json.dumps(value, ensure_ascii=False, sort_keys=True)
    return str(value)

def pattern_id(pattern: dict[str, Any]) -> str:
    return f"PATTERN_{int(pattern['id']):06d}"

def normalize_ts(value: Any) -> str:
    if value is None or value == "":
        return ""
    text = str(value)
    if re.search(r"Z|[-+]\d\d:\d\d$", text):
        return text.replace("Z", "+00:00")
    if re.fullmatch(r"\d{4}-\d{2}-\d{2}", text):
        return f"{text}T00:00:00+00:00"
    if "T" in text:
        return f"{text}+00:00"
    return f"{text}T00:00:00+00:00"

def next_eligible_ts(value: Any, timeframe: Any) -> str:
    normalized = normalize_ts(value)
    if not normalized:
        return ""
    try:
        base = datetime.fromisoformat(normalized)

```

```

except ValueError:
    return normalized
tf = str(timeframe or "1d").lower().strip()
if tf in {"1d", "1 day"}:
    delta_seconds = 24 * 60 * 60
elif tf in {"1wk", "1 week"}:
    delta_seconds = 7 * 24 * 60 * 60
elif tf.endswith("m") and tf[:-1].isdigit():
    delta_seconds = int(tf[:-1]) * 60
elif tf.endswith("h") and tf[:-1].isdigit():
    delta_seconds = int(tf[:-1]) * 60 * 60
else:
    delta_seconds = 24 * 60 * 60
return datetime.fromtimestamp(base.timestamp() + delta_seconds, tz=timezone.utc).isoformat()

def stable_hash(payload: Any) -> str:
    raw = json.dumps(payload, ensure_ascii=False, sort_keys=True, default=str)
    return hashlib.sha256(raw.encode("utf-8")).hexdigest()

def regime_label_from_features(features: Any) -> str:
    if not isinstance(features, dict):
        return "not_persisted"
    score = safe_float(features.get("market_regime_score"), 0.0)
    if score > 0.25:
        return "market_up"
    if score < -0.25:
        return "market_down"
    return "market_mixed"

def sector_label_from_features(features: Any) -> str:
    if not isinstance(features, dict):
        return "not_persisted"
    strength = safe_float(features.get("sector_strength"), 0.0)
    if strength > 0.03:
        return "sector_strong"
    if strength < -0.03:
        return "sector_weak"
    return "sector_neutral"

def safe_float(value: Any, default: float = 0.0) -> float:
    try:
        return float(value)
    except (TypeError, ValueError):
        return default

def truthy(value: Any) -> bool:
    if isinstance(value, bool):
        return value
    return str(value or "").strip().lower() in {"1", "true", "yes", "y", "on"}

def trade_is_closed(trade: dict[str, Any], metadata: dict[str, Any]) -> bool:
    status = str(trade.get("status") or metadata.get("status") or "").strip().lower()
    if status in {"closed", "filled", "done", "executed"}:
        return True
    return bool(trade.get("closed_at") or metadata.get("exit_fill_time"))

def trade_evidence_type(trade: dict[str, Any], metadata: dict[str, Any]) -> str:
    explicit = trade.get("evidence_type") or metadata.get("evidence_type")
    if explicit:
        explicit_type = str(explicit)
        if (
            explicit_type == EVIDENCE_IBKR_PAPER_ORDER
            and trade_is_closed(trade, metadata)
            and fill_provenance(metadata) in REAL_FILL_PROVENANCE
        ):
            return EVIDENCE_IBKR_PAPER_FILL
        return explicit_type
    execution_mode = str(metadata.get("execution_mode") or trade.get("execution_mode") or "").strip().lower()
    if execution_mode == LAB_SHADOW_EXECUTION_MODE or truthy(metadata.get("observation_only")) or truthy(metadata.get("no_ibkr_order")):
        return EVIDENCE_NEAR_MISS_SHADOW if truthy(metadata.get("near_miss")) else EVIDENCE_SHADOW_NO_ORDER
    if execution_mode in {"ibkr", "ibkr_paper", "paper"}:
        return (
            EVIDENCE_IBKR_PAPER_FILL
            if trade_is_closed(trade, metadata) and fill_provenance(metadata) in REAL_FILL_PROVENANCE
            else EVIDENCE_IBKR_PAPER_ORDER
        )
    if trade.get("broker_order_id") or metadata.get("broker_order_id") or metadata.get("parent_order_id"):
        return (
            EVIDENCE_IBKR_PAPER_FILL
            if trade_is_closed(trade, metadata) and fill_provenance(metadata) in REAL_FILL_PROVENANCE
            else EVIDENCE_IBKR_PAPER_ORDER
        )
    return ""

def trade_evidence_quality(metadata: dict[str, Any]) -> str:
    explicit = metadata.get("evidence_quality")
    if explicit:
        return str(explicit)

```

```

    if truthy(metadata.get("fallback_used")) or metadata.get("fallback_source"):
        return EVIDENCE_QUALITY_DEGRADED
    return EVIDENCE_QUALITY_STANDARD

def is_exportable_paper_fill_trade(trade: dict[str, Any], metadata: dict[str, Any]) -> bool:
    evidence_type = trade_evidence_type(trade, metadata)
    evidence_quality = trade_evidence_quality(metadata)
    if evidence_type != EVIDENCE_IBKR_PAPER_FILL:
        return False
    if evidence_quality not in {EVIDENCE_QUALITY_STANDARD, "normal"}:
        return False
    if not trade_is_closed(trade, metadata):
        return False
    if (
        truthy(metadata.get("observation_only"))
        or truthy(metadata.get("no_ibkr_order"))
        or truthy(metadata.get("near_miss"))
    ):
        return False
    return (
        fill_provenance(metadata) in REAL_FILL_PROVENANCE
        and has_any_metadata_value(metadata, FILL_ID_KEYS)
        and has_any_metadata_value(metadata, BROKER_TIMESTAMP_KEYS)
        and has_any_metadata_value(metadata, COMMISSION_KEYS)
    )

def fill_provenance(metadata: dict[str, Any]) -> str:
    return str(metadata.get("fill_provenance") or "").strip()

def has_any_metadata_value(metadata: dict[str, Any], keys: tuple[str, ...]) -> bool:
    for source in metadata_sources(metadata):
        for key in keys:
            value = source.get(key)
            if value is not None and str(value).strip() != "":
                return True
    return False

def first_metadata_value(metadata: dict[str, Any], keys: tuple[str, ...]) -> Any:
    for source in metadata_sources(metadata):
        for key in keys:
            value = source.get(key)
            if value is not None and str(value).strip() != "":
                return value
    return ""

def metadata_sources(metadata: dict[str, Any]) -> list[dict[str, Any]]:
    sources: list[dict[str, Any]] = [metadata]
    for key in EVIDENCE_NESTED_METADATA_KEYS:
        value = metadata.get(key)
        if isinstance(value, dict):
            sources.append(value)
    return sources

def min_max_times(examples: list[dict[str, Any]]) -> tuple[str, str]:
    values = sorted(normalize_ts(e.get("window_end")) for e in examples if e.get("window_end"))
    if not values:
        return "", ""
    return values[0], values[-1]

def best_rr_metrics(pattern: dict[str, Any]) -> dict[str, Any]:
    rr = pattern.get("best_rr") or pattern.get("best_tested_rr") or 0
    metrics = pattern.get("rr_metrics_json") or {}
    key = f"{float(rr):g}" if rr else ""
    if key in metrics:
        return metrics[key]
    return {}

def pattern_research_metrics(pattern: dict[str, Any]) -> dict[str, Any]:
    metrics = pattern.get("metrics_json")
    return metrics if isinstance(metrics, dict) else {}

def metric_value(pattern: dict[str, Any], metrics: dict[str, Any], key: str, default: Any = "") -> Any:
    if key in metrics:
        return metrics.get(key)
    return pattern.get(key, default)

def status_for_experiment(pattern: dict[str, Any]) -> str:
    if pattern.get("validation_passed"):
        return "active" if pattern.get("status") in {"lab", "lab_watchlist", "lab_candidate", "director_review"} else "detected"
    if pattern.get("status") == "rejected":
        return "discarded"
    return "pending_review"

def reason_for(pattern: dict[str, Any]) -> str:
    reasons = pattern.get("validation_reasons_json") or pattern.get("rejection_reasons_json") or []

```

```

if reasons:
    return "; ".join(str(r) for r in reasons)
return str(pattern.get("promotion_reason") or "")

def build_pattern_catalog(
    patterns: list[dict[str, Any]],
    examples_by_pattern: dict[int, list[dict[str, Any]]],
    context: dict[str, Any],
) -> list[dict[str, Any]]:
    env = context["env"]
    universe_count = len(context["universe_symbols"])
    rows = []
    for pattern in patterns:
        examples = examples_by_pattern.get(int(pattern["id"]), [])
        first_seen, last_seen = min_max_times(examples)
        rr = pattern.get("best_rr") or pattern.get("best_tested_rr") or env.get("TRADEO_DISCOVERY_MIN_REWARD_RISK", "2.5")
        rows.append({
            "pattern_id": pattern_id(pattern),
            "pattern_name": pattern.get("name", ""),
            "pattern_version": f"run_{pattern.get('run_id')} or 'unknown'",
            "description": (
                f"Clustered normalized OHLCV window pattern. side={pattern.get('side')}, "
                f"window_size={pattern.get('window_size')}, cluster_id={pattern.get('cluster_id')}."
            ),
            "hypothesis": (
                "Charts close to this cluster centroid may have asymmetric forward R distribution "
                f"for {pattern.get('side')} setups at the tested reward/risk levels."
            ),
            "detection_rule_plaintext": (
                "Normalize OHLCV, compute local deterministic embedding for the configured daily window, "
                "standardize features, cluster historical windows with MiniBatchKMeans, and identify windows "
                f"assigned to cluster {pattern.get('cluster_id')} for window_size={pattern.get('window_size')}. "
                "Current matches compare scaled embedding distance to the stored centroid."
            ),
            "entry_rule_plaintext": (
                "Research entry is the close of the final bar in the detected window. Current lab matches use "
                "latest close as signal/entry reference; no order is submitted by the research lab."
            ),
            "exit_rule_plaintext": (
                "Historical simulation exits at the first stop/target touch in the forward path, or at the final "
                "forward close if neither is touched. If stop and target are both touched intrabar, stop is assumed first."
            ),
            "stop_rule_plaintext": "1R stop based on max(1.5*ATR14, 1.5% of entry, 0.01).",
            "take_profit_rule_plaintext": f"Targets tested at R levels from config; best_rr currently {rr}.",
            "time_stop_rule_plaintext": "Forward bars tested from discovery config; final forward close is used as time-stop fallback.",
            "timeframe": pattern.get("timeframe", "1d"),
            "asset_class": "US equities",
            "universe": f"data/universe_us_mid_small.csv ({universe_count} configured symbols)",
            "session_filter": "Regular daily bars from IBKR historical data; intraday session filter not persisted.",
            "min_volume_filter": env.get("TRADEO_MIN_AVG_DOLLAR_VOLUME", "5000000"),
            "min_price_filter": env.get("TRADEO_MIN_PRICE", "2.0"),
            "uses_fundamental_data": "false",
            "uses_news_data": "false",
            "uses_options_data": "false",
            "uses_future_data": "false",
            "known_risks": "; ".join([
                reason_for(pattern),
                "stored examples are representative, not a full raw event ledger",
                "daily timestamps normalized to UTC when original bars lacked explicit timezone",
            ]).strip("; "),
            "status": pattern.get("status", ""),
            "created_at": normalize_ts(pattern.get("created_at")),
            "first_seen": first_seen,
            "last_seen": last_seen,
        })
    return rows

def build_pattern_events(
    patterns: list[dict[str, Any]],
    examples_by_pattern: dict[int, list[dict[str, Any]]],
    matches: list[dict[str, Any]],
    context: dict[str, Any],
) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    pattern_by_id = {int(p["id"]): p for p in patterns}
    for pid, examples in examples_by_pattern.items():
        pattern = pattern_by_id.get(pid)
        if not pattern:
            continue
        for example in examples:
            event_id = f"EVT_{pattern_id(pattern)}_EX_{int(example['id']):06d}"
            window_end = normalize_ts(example.get("window_end"))
            features = example.get("features_json") or {}
            rows.append({
                "event_id": event_id,
                "pattern_id": pattern_id(pattern),
                "detected_at": normalize_ts(example.get("created_at")),
                "market_timestamp": window_end,
                "timezone": "UTC",
                "ticker": example.get("symbol", ""),
                "exchange": "SMART/IBKR",
                "timeframe": example.get("timeframe", pattern.get("timeframe", "1d")),
                "bar_open_time": normalize_ts(example.get("window_start")),
                "bar_close_time": window_end,
            })

```

```

        "signal_price": example.get("entry_price", ""),
        "bid_at_signal": "",
        "ask_at_signal": "",
        "spread_at_signal": "",
        "volume_at_signal": "",
        "atr_at_signal": example.get("risk_proxy", ""),
        "volatility_context": f"risk_proxy={example.get('risk_proxy', '')}; mfe_r={example.get('mfe_r', '')};
mae_r={example.get('mae_r', '')}",
        "market_regime": regime_label_from_features(features),
        "sector": sector_label_from_features(features),
        "was_trade_triggered": "false",
        "trade_id": "",
        "reason_not_traded": "Historical representative research sample; not routed to paper execution.",
        "duplicate_group_id": f"DUP_{pattern_id(pattern)}_{example.get('symbol', '')}_{window_end}",
        "is_independent_sample": "true",
        "data_available_at_signal": "true",
        "available_data_cutoff_ts": window_end,
        "decision_ts": window_end,
        "entry_eligible_ts": next_eligible_ts(window_end, example.get("timeframe", pattern.get("timeframe", "1d"))),
        "label_generated_ts": normalize_ts(example.get("forward_end")),
        "source_bar_hash": stable_hash({"example": example.get("id"), "window_end": window_end, "features": features}),
        "split_id": "research_example",
        "features_used_json": features,
        "notes": f"stored_example_kind={example.get('kind', '')}; outcome_r={example.get('outcome_r', '')};
similarity={example.get('similarity', '')}",
    })
    for match in matches:
        pattern = pattern_by_id.get(int(match["pattern_id"]))
        if not pattern:
            continue
        matched_at = normalize_ts(match.get("matched_at"))
        event_id = f"EVT_{pattern_id(pattern)}_MATCH_{int(match['id']):06d}"
        metrics = match.get("metrics_json") or {}
        features = metrics.get("features") or {}
        audit = metrics.get("entry_audit") or {}
        regime = metrics.get("regime") or {}
        rows.append({
            "event_id": event_id,
            "pattern_id": pattern_id(pattern),
            "detected_at": matched_at,
            "market_timestamp": normalize_ts(match.get("window_end")),
            "timezone": "UTC",
            "ticker": match.get("symbol", ""),
            "exchange": "SMART/IBKR",
            "timeframe": match.get("timeframe", pattern.get("timeframe", "1d")),
            "bar_open_time": "",
            "bar_close_time": normalize_ts(match.get("window_end")),
            "signal_price": match.get("entry_price", ""),
            "bid_at_signal": "",
            "ask_at_signal": "",
            "spread_at_signal": "",
            "volume_at_signal": "",
            "atr_at_signal": "",
            "volatility_context": json.dumps(features, ensure_ascii=False, sort_keys=True),
            "market_regime": regime.get("regime_key") or regime_label_from_features(features),
            "sector": regime.get("sector_regime") or sector_label_from_features(features),
            "was_trade_triggered": "false",
            "trade_id": "",
            "reason_not_traded": "Current lab watchlist match; execution disabled/read-only and requires paper validation.",
            "duplicate_group_id": f"DUP_{pattern_id(pattern)}_{match.get('symbol', '')}_{normalize_ts(match.get('window_end'))}",
            "is_independent_sample": "pending_review",
            "data_available_at_signal": "true",
            "available_data_cutoff_ts": audit.get("available_data_cutoff_ts") or normalize_ts(match.get("window_end")),
            "decision_ts": audit.get("decision_ts") or matched_at,
            "entry_eligible_ts": audit.get("entry_eligible_ts")
            or next_eligible_ts(match.get("window_end"), match.get("timeframe", pattern.get("timeframe", "1d"))),
            "label_generated_ts": audit.get("label_generated_ts") or "",
            "source_bar_hash": audit.get("source_bar_hash")
            or stable_hash({"match": match.get("id"), "window_end": match.get("window_end"), "features": features}),
            "split_id": audit.get("split_id") or "live_forward_scan",
            "features_used_json": features,
            "notes": f"similarity={match.get('similarity')}; score={match.get('score')}; status={match.get('status')}",
        })
    return rows

def build_paper_trades(patterns: list[dict[str, Any]], laboratory_overview: dict[str, Any]) -> list[dict[str, Any]]:
    signals = {
        int(signal["id"]): signal
        for signal in laboratory_overview.get("signals", [])
        if isinstance(signal, dict) and str(signal.get("id", "")).isdigit()
    }
    pattern_by_name = {str(pattern.get("name")): pattern for pattern in patterns}
    rows: list[dict[str, Any]] = []
    for trade in laboratory_overview.get("trades", []):
        if not isinstance(trade, dict):
            continue
        signal = signals.get(int(trade.get("signal_id") or 0), {})
        metadata = trade.get("metadata_json") or {}
        signal_metadata = signal.get("metadata_json") or {}
        evidence_type = trade_evidence_type(trade, metadata)
        evidence_quality = trade_evidence_quality(metadata)
        if not is_exportable_paper_fill_trade(trade, metadata):
            continue
        pattern_ref = audit_pattern_id(signal_metadata, trade, pattern_by_name)
        if not pattern_ref:

```



```

        continue
    event_id = linked_event_id(pattern_ref, signal, trade)
    entry = safe_float(trade.get("entry"), safe_float(signal.get("entry"), 0.0))
    qty = safe_float(trade.get("qty"), 0.0)
    gross_pnl = safe_float(trade.get("pnl_usd"), 0.0)
    commission = safe_float(first_metadata_value(metadata, COMMISSION_KEYS), 0.0)
    estimated_spread = safe_float(metadata.get("estimated_spread_cost"), 0.0)
    estimated_slippage = safe_float(metadata.get("estimated_slippage"), 0.0)
    other_fees = safe_float(metadata.get("other_fees"), 0.0)
    net_pnl = gross_pnl - commission - estimated_spread - estimated_slippage - other_fees
    entry_fill_time = first_metadata_value(metadata, ENTRY_BROKER_TIMESTAMP_KEYS)
    exit_fill_time = first_metadata_value(metadata, EXIT_BROKER_TIMESTAMP_KEYS)
    rows.append(
        {
            "trade_id": trade.get("id"),
            "event_id": event_id,
            "pattern_id": pattern_ref,
            "evidence_type": evidence_type,
            "evidence_quality": evidence_quality,
            "ticker": trade.get("symbol", ""),
            "side": trade.get("side", ""),
            "quantity": trade.get("qty", ""),
            "entry_signal_time": signal.get("created_at", ""),
            "entry_order_time": metadata.get("submitted_at") or trade.get("opened_at", ""),
            "entry_fill_time": entry_fill_time,
            "entry_signal_price": signal.get("entry", entry),
            "entry_order_type": ((signal_metadata.get("entry_variant") or {}).get("order_style")) or "limit",
            "entry_limit_price": signal.get("entry", entry),
            "entry_fill_price": metadata.get("entry_fill_price") or entry,
            "exit_signal_time": trade.get("closed_at", ""),
            "exit_order_time": metadata.get("exit_order_time") or trade.get("closed_at", ""),
            "exit_fill_time": exit_fill_time,
            "exit_order_type": metadata.get("exit_order_type") or "bracket_exit",
            "exit_limit_price": trade.get("target", ""),
            "exit_fill_price": trade.get("exit_price", ""),
            "exit_reason": metadata.get("exit_reason") or trade.get("status", ""),
            "gross_pnl": round(gross_pnl, 4),
            "commission": round(commission, 4),
            "estimated_spread_cost": round(estimated_spread, 4),
            "estimated_slippage": round(estimated_slippage, 4),
            "other_fees": round(other_fees, 4),
            "net_pnl": round(net_pnl, 4),
            "return_pct": round(net_pnl / max(abs(entry * qty), 1e-9), 6) if qty else "",
            "mae": metadata.get("mae"),
            "mfe": metadata.get("mfe"),
            "holding_period_seconds": metadata.get("holding_period_seconds"),
            "risk_amount": signal.get("risk_usd", ""),
            "r_multiple": trade.get("r_multiple", ""),
            "account_equity_snapshot": metadata.get("account_equity_snapshot", ""),
            "position_size_method": "risk_manager_suggested_qty",
            "notes": "; ".join(
                part
                for part in (
                    f"entry_variant={signal_metadata.get('entry_variant_id', '')}",
                    f"regime={((signal_metadata.get('regime') or {}).get('regime_key') or '')}",
                    f"execution_mode={metadata.get('execution_mode', '')}"
                )
            ),
            if part
        )
    )
}
return rows

```

```

def build_ib_fills(laboratory_overview: dict[str, Any], *, exported_trade_ids: set[str] | None = None) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    allowed_trade_ids = exported_trade_ids if exported_trade_ids is not None else None
    for trade in laboratory_overview.get("trades", []):
        if not isinstance(trade, dict):
            continue
        metadata = trade.get("metadata_json") or {}
        if not is_exportable_paper_fill_trade(trade, metadata):
            continue
        trade_id = str(trade.get("id") or "")
        if allowed_trade_ids is not None and trade_id not in allowed_trade_ids:
            continue
        entry_price = safe_float(metadata.get("entry_fill_price"), safe_float(trade.get("entry"), 0.0))
        opened_at = normalize_ts(first_metadata_value(metadata, ENTRY_BROKER_TIMESTAMP_KEYS))
        if entry_price > 0 and opened_at:
            rows.append(fill_row(trade, metadata, trade_id=trade_id, suffix="ENTRY", timestamp=opened_at, price=entry_price))
        exit_price = safe_float(metadata.get("exit_fill_price"), safe_float(trade.get("exit_price"), 0.0))
        closed_at = normalize_ts(first_metadata_value(metadata, EXIT_BROKER_TIMESTAMP_KEYS))
        if exit_price > 0 and closed_at:
            rows.append(fill_row(trade, metadata, trade_id=trade_id, suffix="EXIT", timestamp=closed_at, price=exit_price))
    return rows

```

```

def fill_row(
    trade: dict[str, Any],
    metadata: dict[str, Any],
    *,
    trade_id: str,
    suffix: str,
    timestamp: str,
    price: float,
) -> dict[str, Any]:

```

```

order_ids = metadata.get("order_ids") if isinstance(metadata.get("order_ids"), list) else []
raw_order_id = order_ids[0] if order_ids else metadata.get("parent_order_id", "")
fill_keys = ENTRY_FILL_ID_KEYS if suffix == "ENTRY" else EXIT_FILL_ID_KEYS
fill_id = first_metadata_value(metadata, fill_keys)
return {
    "fill_id_hash": stable_hash({"fill_id": fill_id, "trade_id": trade_id, "suffix": suffix})[:24],
    "trade_id": trade_id,
    "order_id_hash": stable_hash({"order_id": raw_order_id})[:24] if raw_order_id else "",
    "ib_execution_time": timestamp,
    "timezone": "UTC",
    "ticker": trade.get("symbol", ""),
    "side": trade.get("side", ""),
    "quantity": trade.get("qty", ""),
    "price": round(price, 4),
    "commission": first_metadata_value(metadata, COMMISSION_KEYS),
    "liquidity_flag": metadata.get("liquidity_flag", ""),
    "exchange": "SMART/IBKR",
    "order_type": metadata.get("entry_order_type") or "bracket",
    "account_id_redacted": "true",
    "raw_status": trade.get("status", ""),
    "notes": f"{suffix.lower()} fill source={fill_provenance(metadata)}",
}

def audit_pattern_id(
    signal_metadata: dict[str, Any],
    trade: dict[str, Any],
    pattern_by_name: dict[str, dict[str, Any]],
) -> str:
    raw_id = signal_metadata.get("pattern_id")
    try:
        if raw_id:
            return f"PATTERN_{int(raw_id):06d}"
    except (TypeError, ValueError):
        pass
    pattern = pattern_by_name.get(str(trade.get("pattern") or ""))
    return pattern_id(pattern) if pattern else ""

def linked_event_id(pattern_ref: str, signal: dict[str, Any], trade: dict[str, Any]) -> str:
    signal_id = signal.get("id") or trade.get("signal_id") or "0"
    return f"EVT_{pattern_ref}_TRADE_SIGNAL_{int(signal_id):06d}" if str(signal_id).isdigit() else f"EVT_{pattern_ref}_TRADE"

def build_experiment_registry(
    patterns: list[dict[str, Any]],
    examples_by_pattern: dict[int, list[dict[str, Any]]],
    runs: list[dict[str, Any]],
    context: dict[str, Any],
) -> list[dict[str, Any]]:
    run_by_id = {r.get("id"): r for r in runs}
    rows: list[dict[str, Any]] = []
    for pattern in patterns:
        examples = examples_by_pattern.get(int(pattern["id"]), [])
        first_seen, last_seen = min_max_times(examples)
        run = run_by_id.get(pattern.get("run_id")) or {}
        params = run.get("params_json") or {}
        research_metrics = pattern_research_metrics(pattern)
        rr_metrics = pattern.get("rr_metrics_json") or {}
        if not rr_metrics:
            rr_metrics = {"best": best_rr_metrics(pattern)}
        nested_replay = nested_discovery_replay_metadata(research_metrics)
        registry = registry_metadata(research_metrics)
        edge_claim = edge_claim_metadata(research_metrics)
        event_ledger_hash = event_ledger_hash_metadata(research_metrics)
        active_blockers = active_blockers_metadata(research_metrics)
        for rr_key, metrics in sorted(rr_metrics.items(), key=lambda item: str(item[0])):
            variant_id = f"RR_{str(rr_key).replace('.', '_')}"
            rows.append({
                "experiment_id": f"EXP_{pattern_id(pattern)}_{variant_id}",
                "pattern_id": pattern_id(pattern),
                "variant_id": variant_id,
                "created_at": normalize_ts(pattern.get("created_at")),
                "tested_at": normalize_ts(pattern.get("updated_at") or pattern.get("created_at")),
                "status": status_for_experiment(pattern),
                "reason_status": reason_for(pattern),
                "parameters_json": {
                    "rr": metrics.get("rr", rr_key) if isinstance(metrics, dict) else rr_key,
                    "window_size": pattern.get("window_size"),
                    "cluster_id": pattern.get("cluster_id"),
                    "side": pattern.get("side"),
                    "run_params": params,
                },
                "dataset_start": first_seen,
                "dataset_end": last_seen,
                "in_sample_start": first_seen,
                "in_sample_end": "",
                "out_of_sample_start": "",
                "out_of_sample_end": last_seen,
                "paper_live_start": "",
                "paper_live_end": "",
                "number_of_assets_tested": pattern.get("symbol_count", ""),
                "number_of_events": pattern.get("sample_count", ""),
                "number_of_trades": "0",
                "gross_pnl": "",
                "net_pnl": "",
            })

```

```

        "winrate": metrics.get("win_rate", pattern.get("best_win_rate", "")) if isinstance(metrics, dict) else "",
        "profit_factor": metrics.get("profit_factor", pattern.get("best_profit_factor", "")) if isinstance(metrics, dict) else
    "",
    "sharpe": "",
    "sortino": "",
    "max_drawdown": metrics.get("max_drawdown_r", pattern.get("best_max_drawdown_r", "")) if isinstance(metrics, dict) else
    "",
    "candidate_trial_count": metric_value(pattern, research_metrics, "candidate_trial_count"),
    "global_trial_count": metric_value(pattern, research_metrics, "global_trial_count"),
    "adjusted_p_value": metric_value(pattern, research_metrics, "adjusted_p_value"),
    "wrc_p_value": metric_value(pattern, research_metrics, "wrc_p_value"),
    "spa_p_value": metric_value(pattern, research_metrics, "spa_p_value"),
    "fit_scope": metric_value(pattern, research_metrics, "fit_scope", {}),
    "score_input_scope": metric_value(pattern, research_metrics, "score_input_scope", {}),
    "event_ledger_hash": event_ledger_hash,
    "nested_discovery_replay_status": nested_replay.get("status", ""),
    "nested_discovery_replay_implemented": nested_replay.get("implemented", ""),
    "nested_discovery_replay_passed": nested_replay.get("passed", ""),
    "edge_claim": edge_claim,
    "drift_status": metric_value(pattern, research_metrics, "drift_status", pattern.get("drift_status", "")),
    "active_blockers": "; ".join(active_blockers),
    "registry_path": registry.get("path", ""),
    "registry_hash": registry.get("registry_hash") or registry.get("hash") or registry.get("sha256") or "",
    "registry_previous_hash": registry.get("previous_registry_hash", ""),
    "registry_run_manifest_hash": (
        registry.get("run_manifest_hash") or registry.get("latest_run_manifest_hash") or ""
    ),
    "registry_hash_chain_valid": registry.get("hash_chain_valid", ""),
    "notes": "Research variant metrics are in R units; no paper trade PnL exists yet.",
    })
return rows

def nested_discovery_replay_metadata(metrics: dict[str, Any]) -> dict[str, Any]:
    nested = metrics.get("nested_discovery_replay")
    if not isinstance(nested, dict):
        hypothesis = metrics.get("research_hypothesis")
        if isinstance(hypothesis, dict):
            nested = hypothesis.get("nested_discovery_replay")
    if not isinstance(nested, dict):
        hypothesis = metrics.get("research_hypothesis_package")
        if isinstance(hypothesis, dict):
            nested = hypothesis.get("nested_discovery_replay")
    if not isinstance(nested, dict):
        return {}
    status = str(nested.get("status") or "").strip().lower()
    passed = truthy(nested.get("passed")) or status in {"passed", "pass", "ok", "complete", "completed"}
    return {
        "status": status,
        "implemented": truthy(nested.get("implemented")),
        "passed": passed,
    }

def registry_metadata(metrics: dict[str, Any]) -> dict[str, Any]:
    registry = metrics.get("global_experiment_registry")
    return registry if isinstance(registry, dict) else {}

def edge_claim_metadata(metrics: dict[str, Any]) -> str:
    for key in ("edge_claim",):
        value = str(metrics.get(key) or "").strip()
        if value:
            return value
    for key in ("research_hypothesis", "research_hypothesis_package", "global_experiment_registry"):
        payload = metrics.get(key)
        if isinstance(payload, dict):
            value = str(payload.get("edge_claim") or "").strip()
            if value:
                return value
    return ""

def event_ledger_hash_metadata(metrics: dict[str, Any]) -> str:
    for key in ("event_ledger_hash", "event_ledger_sha256"):
        value = str(metrics.get(key) or "").strip()
        if value:
            return value
    for key in ("research_hypothesis", "research_hypothesis_package"):
        payload = metrics.get(key)
        if isinstance(payload, dict):
            value = str(payload.get("event_ledger_hash") or "").strip()
            if value:
                return value
    return ""

def active_blockers_metadata(metrics: dict[str, Any]) -> list[str]:
    blockers: list[str] = []
    for key in ("active_blockers", "promotion_blockers", "director_blockers"):
        value = metrics.get(key)
        if isinstance(value, list):
            blockers.extend(str(item) for item in value if str(item).strip())
        elif isinstance(value, str) and value.strip():
            blockers.append(value.strip())
    lab_execution = metrics.get("lab_execution")

```

```

if isinstance(lab_execution, dict):
    value = lab_execution.get("promotion_blockers")
    if isinstance(value, list):
        blockers.extend(str(item) for item in value if str(item).strip())
return blockers

def build_metrics_by_pattern(
    patterns: list[dict[str, Any]],
    examples_by_pattern: dict[int, list[dict[str, Any]]],
    paper_trade_rows: list[dict[str, Any]],
    *,
    event_rows: list[dict[str, Any]] | None = None,
) -> list[dict[str, Any]]:
    trades_by_pattern: dict[str, list[dict[str, Any]]] = defaultdict(list)
    for trade in paper_trade_rows:
        trades_by_pattern[str(trade.get("pattern_id") or "")].append(trade)
    sample_counts, independent_counts = event_sample_counts(event_rows or [])
    rows = []
    for pattern in patterns:
        examples = examples_by_pattern.get(int(pattern["id"]), [])
        first_seen, last_seen = min_max_times(examples)
        metrics = best_rr_metrics(pattern)
        audit_id = pattern_id(pattern)
        trades = trades_by_pattern.get(audit_id, [])
        net_values = [safe_float(trade.get("net_pnl"), 0.0) for trade in trades]
        gross_values = [safe_float(trade.get("gross_pnl"), 0.0) for trade in trades]
        r_values = [safe_float(trade.get("r_multiple"), 0.0) for trade in trades]
        wins = [value for value in net_values if value > 0]
        losses = [abs(value) for value in net_values if value < 0]
        trade_count = len(trades)
        avg_win = metrics.get("avg_win_r", "")
        avg_loss = metrics.get("avg_loss_r", "")
        payoff = round(float(avg_win) / float(avg_loss), 6) if trades and avg_win not in ("", 0) and avg_loss not in ("", 0) else ""
        exported_event_count = sample_counts.get(audit_id, len(examples))
        verified_independent_count = independent_counts.get(audit_id, 0)
        rows.append({
            "pattern_id": audit_id,
            "sample_count": exported_event_count,
            "independent_sample_count": verified_independent_count,
            "trade_count": trade_count,
            "ticker_count": pattern.get("symbol_count", ""),
            "sector_count": "",
            "first_seen": first_seen,
            "last_seen": last_seen,
            "gross_pnl": round(sum(gross_values), 4) if trades else "0",
            "net_pnl": round(sum(net_values), 4) if trades else "0",
            "avg_trade": round(sum(net_values) / trade_count, 4) if trades else "",
            "median_trade": metrics.get("median_result_r", "") if trades else "",
            "std_trade": round(float(statistics.pstdev(net_values)), 4) if len(net_values) > 1 else "",
            "winrate": round(len(wins) / trade_count, 6) if trades else "",
            "avg_win": avg_win if trades else "",
            "avg_loss": avg_loss if trades else "",
            "payoff_ratio": payoff,
            "profit_factor": round(sum(wins) / sum(losses), 6) if losses else (round(sum(wins), 6) if wins else ""),
            "expectancy": round(sum(r_values) / trade_count, 6) if trades else "",
            "max_drawdown": max_drawdown(r_values) if trades else "",
            "max_consecutive_losses": "",
            "best_trade": max(net_values) if net_values else "",
            "worst_trade": min(net_values) if net_values else "",
            "top_5_trades_pnl": round(sum(sorted(net_values, reverse=True)[:5]), 4) if net_values else "",
            "bottom_5_trades_pnl": round(sum(sorted(net_values)[:5]), 4) if net_values else "",
            "pnl_without_best_trade": round(sum(net_values) - max(net_values), 4) if net_values else "",
            "pnl_without_top_5_trades": round(sum(net_values) - sum(sorted(net_values, reverse=True)[:5]), 4) if net_values else "",
            "pnl_without_worst_trade": round(sum(net_values) - min(net_values), 4) if net_values else "",
            "sharpe": "",
            "sortino": "",
            "calmar": "",
            "avg_holding_period_seconds": "",
            "notes": (
                "Operational columns use closed paper trades only; research R metrics remain in experiment_registry.csv."
                if trades
                else (
                    "No closed_lab_trades/paper trades; operational performance columns intentionally blank. "
                    f"Exported events={exported_event_count}; verified independent events={verified_independent_count}; "
                    f"raw pattern sample_count={pattern.get('sample_count', '')}."
                )
            ),
        })
    return rows

def event_sample_counts(event_rows: list[dict[str, Any]]) -> tuple[dict[str, int], dict[str, int]]:
    sample_counts: dict[str, int] = defaultdict(int)
    independent_counts: dict[str, int] = defaultdict(int)
    for row in event_rows:
        pid = str(row.get("pattern_id") or "")
        if not pid:
            continue
        sample_counts[pid] += 1
        if str(row.get("is_independent_sample") or "").strip().lower() == "true":
            independent_counts[pid] += 1
    return dict(sample_counts), dict(independent_counts)

def build_metrics_by_regime(

```

```

patterns: list[dict[str, Any]],
event_rows: list[dict[str, Any]],
paper_trade_rows: list[dict[str, Any]],
) -> list[dict[str, Any]]:
    if not paper_trade_rows:
        return [empty_bucket_row("market_regime")]
    events_by_pattern_regime: dict[tuple[str, str], int] = defaultdict(int)
    for row in event_rows:
        key = (str(row.get("pattern_id", "")), str(row.get("market_regime") or "unknown"))
        events_by_pattern_regime[key] += 1
    buckets: dict[tuple[str, str], list[dict[str, Any]]] = defaultdict(list)
    for trade in paper_trade_rows:
        regime = note_value(trade.get("notes"), "regime") or "unknown"
        buckets[(str(trade.get("pattern_id", "")), regime)].append(trade)
    rows = []
    known_patterns = {pattern_id(pattern) for pattern in patterns}
    for (pid, regime), trades in sorted(buckets.items()):
        row = performance_bucket_row(
            pattern_id_value=pid,
            bucket_value=regime,
            bucket_column="market_regime",
            trades=trades,
            event_count=events_by_pattern_regime.get((pid, regime), len(trades)),
            notes="Closed paper-trade performance grouped by persisted regime metadata.",
        )
        rows.append(row)
    missing_patterns = known_patterns - {str(row.get("pattern_id", "")) for row in rows}
    for pid in sorted(missing_patterns):
        rows.append(empty_bucket_row("market_regime", pattern_id_value=pid))
    return rows

def build_metrics_by_entry_variant(
    patterns: list[dict[str, Any]],
    paper_trade_rows: list[dict[str, Any]],
) -> list[dict[str, Any]]:
    if not paper_trade_rows:
        return [empty_bucket_row("entry_variant_id")]
    buckets: dict[tuple[str, str], list[dict[str, Any]]] = defaultdict(list)
    for trade in paper_trade_rows:
        variant = note_value(trade.get("notes"), "entry_variant") or "unknown"
        buckets[(str(trade.get("pattern_id", "")), variant)].append(trade)
    rows = []
    known_patterns = {pattern_id(pattern) for pattern in patterns}
    for (pid, variant), trades in sorted(buckets.items()):
        row = performance_bucket_row(
            pattern_id_value=pid,
            bucket_value=variant,
            bucket_column="entry_variant_id",
            trades=trades,
            event_count=len(trades),
            notes="Closed paper-trade performance grouped by entry variant metadata.",
        )
        rows.append(row)
    missing_patterns = known_patterns - {str(row.get("pattern_id", "")) for row in rows}
    for pid in sorted(missing_patterns):
        rows.append(empty_bucket_row("entry_variant_id", pattern_id_value=pid))
    return rows

def empty_bucket_row(bucket_column: str, *, pattern_id_value: str = "") -> dict[str, Any]:
    missing = (
        "closed_lab_trades with signal.metadata_json.regime.regime_key"
        if bucket_column == "market_regime"
        else "closed_lab_trades with signal.metadata_json.entry_variant_id"
    )
    return {
        "pattern_id": pattern_id_value,
        "bucket_column": "",
        "analysis_available": "false",
        "empty_reason": f"no_closed_lab_trades: missing {missing}; paper_trades.csv has no matching closed trades.",
        "trade_count": "0",
        "event_count": "0",
        "gross_pnl": "0",
        "net_pnl": "0",
        "avg_trade": "",
        "expectancy_r": "",
        "winrate": "",
        "profit_factor": "",
        "max_drawdown": "",
        "best_trade": "",
        "worst_trade": "",
        "first_seen": "",
        "last_seen": "",
        "notes": "Explicit empty bucket for Director audit contract.",
    }

def performance_bucket_row(
    *,
    pattern_id_value: str,
    bucket_value: str,
    bucket_column: str,
    trades: list[dict[str, Any]],
    event_count: int,
    notes: str,

```

```

) -> dict[str, Any]:
    net_values = [safe_float(trade.get("net_pnl"), 0.0) for trade in trades]
    gross_values = [safe_float(trade.get("gross_pnl"), 0.0) for trade in trades]
    r_values = [safe_float(trade.get("r_multiple"), 0.0) for trade in trades]
    wins = [value for value in net_values if value > 0]
    losses = [abs(value) for value in net_values if value < 0]
    first_seen, last_seen = trade_time_bounds(trades)
    trade_count = len(trades)
    return {
        "pattern_id": pattern_id_value,
        "bucket_column": bucket_value or "unknown",
        "analysis_available": "true",
        "empty_reason": "",
        "trade_count": trade_count,
        "event_count": event_count,
        "gross_pnl": round(sum(gross_values), 4),
        "net_pnl": round(sum(net_values), 4),
        "avg_trade": round(sum(net_values) / trade_count, 4) if trade_count else "",
        "expectancy_r": round(sum(r_values) / trade_count, 6) if trade_count else "",
        "winrate": round(len(wins) / trade_count, 6) if trade_count else "",
        "profit_factor": round(sum(wins) / sum(losses), 6) if losses else (round(sum(wins), 6) if wins else ""),
        "max_drawdown": max_drawdown(r_values),
        "best_trade": max(net_values) if net_values else "",
        "worst_trade": min(net_values) if net_values else "",
        "first_seen": first_seen,
        "last_seen": last_seen,
        "notes": notes,
    }

}

def note_value(notes: Any, key: str) -> str:
    for part in str(notes or "").split(";"):
        if "=" not in part:
            continue
        raw_key, value = part.split("=", 1)
        if raw_key.strip() == key:
            return value.strip()
    return ""

def trade_time_bounds(trades: list[dict[str, Any]]) -> tuple[str, str]:
    values = sorted(
        normalize_ts(
            trade.get("exit_fill_time")
            or trade.get("exit_order_time")
            or trade.get("entry_fill_time")
            or trade.get("entry_order_time")
        )
        for trade in trades
        if trade.get("exit_fill_time")
        or trade.get("exit_order_time")
        or trade.get("entry_fill_time")
        or trade.get("entry_order_time")
    )
    if not values:
        return "", ""
    return values[0], values[-1]

def build_metrics_by_ticker(examples_by_pattern: dict[int, list[dict[str, Any]]]) -> list[dict[str, Any]]:
    buckets: dict[tuple[str, str], list[dict[str, Any]]] = defaultdict(list)
    for pid, examples in examples_by_pattern.items():
        fake_pattern = {"id": pid}
        for example in examples:
            buckets[(pattern_id(fake_pattern), str(example.get("symbol", "")))].append(example)
    rows = []
    for (pid, ticker), examples in sorted(buckets.items()):
        outcomes = [float(e.get("outcome_r") or 0.0) for e in examples]
        first_seen, last_seen = min_max_times(examples)
        rows.append({
            "pattern_id": pid,
            "ticker": ticker,
            "trade_count": "0",
            "event_count": len(examples),
            "net_pnl": "0",
            "winrate": ratio(sum(1 for value in outcomes if value > 0), len(outcomes)),
            "profit_factor": profit_factor(outcomes),
            "avg_trade": mean(outcomes),
            "max_drawdown": max_drawdown(outcomes),
            "first_seen": first_seen,
            "last_seen": last_seen,
            "notes": "Aggregated from stored representative examples, not full event ledger.",
        })
    return rows

def build_metrics_by_period(examples_by_pattern: dict[int, list[dict[str, Any]]]) -> list[dict[str, Any]]:
    buckets: dict[tuple[str, str], list[dict[str, Any]]] = defaultdict(list)
    for pid, examples in examples_by_pattern.items():
        fake_pattern = {"id": pid}
        for example in examples:
            ts = normalize_ts(example.get("window_end"))
            if not ts:
                continue
            week_start = datetime.fromisoformat(ts).date().isoformat()
            buckets[(week_start, pattern_id(fake_pattern))].append(example)

```

```

rows = []
for (period_start, pid), examples in sorted(buckets.items()):
    outcomes = [float(e.get("outcome_r") or 0.0) for e in examples]
    rows.append({
        "period_start": normalize_ts(period_start),
        "period_end": normalize_ts(period_start),
        "period_type": "day",
        "pattern_id": pid,
        "event_count": len(examples),
        "trade_count": "0",
        "gross_pnl": "0",
        "net_pnl": "0",
        "winrate": ratio(sum(1 for value in outcomes if value > 0), len(outcomes)),
        "profit_factor": profit_factor(outcomes),
        "max_drawdown": max_drawdown(outcomes),
        "market_regime": "not_persisted",
        "notes": "Representative example outcome in R units only; no realized paper PnL.",
    })
return rows

def ratio(num: int, den: int) -> str:
    return "" if den == 0 else f"{num / den:.6f}"

def mean(values: list[float]) -> str:
    return "" if not values else f"{sum(values) / len(values):.6f}"

def profit_factor(values: list[float]) -> str:
    wins = sum(v for v in values if v > 0)
    losses = abs(sum(v for v in values if v < 0))
    if losses == 0:
        return f"{wins:.6f}" if wins else ""
    return f"{wins / losses:.6f}"

def max_drawdown(values: list[float]) -> str:
    equity = 0.0
    peak = 0.0
    max_dd = 0.0
    for value in values:
        equity += value
        peak = max(peak, equity)
        max_dd = max(max_dd, peak - equity)
    return f"{max_dd:.6f}" if values else ""

def write_manifest(
    package: Path,
    context: dict[str, Any],
    patterns: list[dict[str, Any]],
    event_rows: list[dict[str, Any]],
    experiment_rows: list[dict[str, Any]],
    paper_trade_rows: list[dict[str, Any]],
    fill_rows: list[dict[str, Any]],
    metrics_regime_rows: list[dict[str, Any]],
    metrics_entry_variant_rows: list[dict[str, Any]],
) -> None:
    duplicate_groups = duplicate_event_groups(event_rows)
    manifest = {
        "audit_id": context["audit_id"],
        "created_at": context["created_at"],
        "created_by": "Claw",
        "repo_commit": context["git_commit"],
        "repo_branch": context["git_branch"],
        "data_source": "Interactive Brokers Paper",
        "environment": context["env"].get("TRADEO_ENVIRONMENT", "local"),
        "timezone": "Europe/Madrid",
        "market_timezone": "America/New_York",
        "asset_class": "US equities",
        "patterns_detected": len(patterns),
        "total_pattern_events": len(event_rows),
        "total_paper_trades": len(paper_trade_rows),
        "total_ib_fills": len(fill_rows),
        "total_experiment_variants": len(experiment_rows),
        "total_metrics_by_regime_rows": len(metrics_regime_rows),
        "total_metrics_by_entry_variant_rows": len(metrics_entry_variant_rows),
        "duplicate_event_groups": duplicate_groups["groups"],
        "duplicate_repeated_rows": duplicate_groups["repeated_rows"],
        "metric_breakdowns": {
            "by_regime": metric_breakdown_state(metrics_regime_rows),
            "by_entry_variant": metric_breakdown_state(metrics_entry_variant_rows),
        },
        "director_gate_required": True,
        "contains_sensitive_data": False,
        "account_ids_redacted": True,
        "orders_anonymized": True,
        "files": [
            {"path": "AUDIT_REQUEST.md", "description": "Human-readable request and pattern summary"},
            {"path": "AUDIT_SUMMARY_FOR_CHATGPT.md", "description": "Computed audit summary, rankings, concentration checks, and preliminary Claw recommendations"},
            {"path": "manifest.json", "description": "Machine-readable package metadata"},
            {"path": "pattern_catalog.csv", "description": "One row per detected pattern"},
            {"path": "pattern_events.csv", "description": "One row per exported independent/representative pattern occurrence"},
            {"path": "paper_trades.csv", "description": "One row per paper trade generated by a pattern"},
        ],
    }

```

```

        {"path": "ib_fills.csv", "description": "Anonymized IB Paper fills"},
        {"path": "experiment_registry.csv", "description": "All tested variants, including discarded ones"},
        {"path": "metrics_by_pattern.csv", "description": "Aggregated metrics by pattern"},
        {"path": "metrics_by_ticker.csv", "description": "Aggregated metrics by pattern/ticker"},
        {"path": "metrics_by_period.csv", "description": "Aggregated metrics by period"},
        {"path": "metrics_by_regime.csv", "description": "Closed paper-trade performance by market regime, or explicit empty
reason"},
        {"path": "metrics_by_entry_variant.csv", "description": "Closed paper-trade performance by entry variant, or explicit empty
reason"},
        {"path": "code_references.md", "description": "Code paths and functions relevant to detection, validation, execution, and
risk"},
        {"path": "data_lineage.md", "description": "Data origin, transformations, and known data-quality limits"},
        {"path": "known_issues.md", "description": "Known lab limitations and audit warnings"},
        {"path": "audit_checklist.md", "description": "Pre-review checklist for Claw and ChatGPT"},
        {"path": "chatgpt_questions.md", "description": "Explicit questions for the ChatGPT audit"},
        {"path": "reproducibility.md", "description": "Commands and environment needed to reproduce the export"},
        {"path": "file_hashes.sha256", "description": "SHA-256 hashes for package files"},
        {"path": "config_snapshot/detector_config.json", "description": "Detector and discovery settings, redacted"},
        {"path": "config_snapshot/universe_config.json", "description": "Universe settings used for the export"},
        {"path": "config_snapshot/risk_config.json", "description": "Risk settings used by the lab/bot"},
        {"path": "config_snapshot/execution_config.json", "description": "Execution and safety settings, redacted"},
        {"path": "config_snapshot/ib_paper_config.redacted.json", "description": "IB Paper connectivity metadata without account IDs
or secrets"},
        {"path": "config_snapshot/data_config.json", "description": "Market data provider settings, redacted"},
    ],
}
(package / "manifest.json").write_text(json.dumps(manifest, indent=2, ensure_ascii=False), encoding="utf-8")

def duplicate_event_groups(event_rows: list[dict[str, Any]]) -> dict[str, int]:
    counts: dict[str, int] = defaultdict(int)
    for row in event_rows:
        group = str(row.get("duplicate_group_id") or "").strip()
        if group:
            counts[group] += 1
    repeated = [count for count in counts.values() if count > 1]
    return {"groups": len(repeated), "repeated_rows": sum(repeated)}

def metric_breakdown_state(rows: list[dict[str, Any]]) -> dict[str, Any]:
    available_rows = [
        row
        for row in rows
        if str(row.get("analysis_available", "")).strip().lower() == "true"
        and int(float(str(row.get("trade_count") or "0"))) > 0
    ]
    if available_rows:
        return {"available": True, "rows": len(available_rows), "empty_reason": ""}
    reason = next((str(row.get("empty_reason", "")).strip() for row in rows if row.get("empty_reason")), "")
    return {"available": False, "rows": len(rows), "empty_reason": reason}

def write_audit_request(
    package: Path,
    context: dict[str, Any],
    patterns: list[dict[str, Any]],
    pattern_rows: list[dict[str, Any]],
    metrics_rows: list[dict[str, Any]],
    experiment_rows: list[dict[str, Any]],
    event_rows: list[dict[str, Any]],
) -> None:
    metric_by_pattern = {row["pattern_id"]: row for row in metrics_rows}
    timeframes = sorted({str(p.get("timeframe", "")) for p in patterns if p.get("timeframe")})
    discarded = sum(1 for row in experiment_rows if row["status"] == "discarded")
    total_paper_trades = sum(to_int(row.get("trade_count")) or 0 for row in metrics_rows)
    table_lines = [
        "| pattern_id | pattern_name | status | sample_count | trade_count | ticker_count | first_seen | last_seen | gross_pnl | net_pnl |
| winrate | avg_win | avg_loss | profit_factor | max_drawdown | notes |",
        "|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|",
    ]
    for row in pattern_rows:
        metrics = metric_by_pattern.get(row["pattern_id"], {})
        table_lines.append(
            "| {pattern_id} | {pattern_name} | {status} | {sample_count} | {trade_count} | {ticker_count} | {first_seen} | {last_seen} |
{gross_pnl} | {net_pnl} | {winrate} | {avg_win} | {avg_loss} | {profit_factor} | {max_drawdown} | {notes} |".format(
                pattern_id=md(row["pattern_id"]),
                pattern_name=md(row["pattern_name"]),
                status=md(row["status"]),
                sample_count=md(metrics.get("sample_count", "")),
                trade_count=md(metrics.get("trade_count", "")),
                ticker_count=md(metrics.get("ticker_count", "")),
                first_seen=md(metrics.get("first_seen", "")),
                last_seen=md(metrics.get("last_seen", "")),
                gross_pnl=md(metrics.get("gross_pnl", "")),
                net_pnl=md(metrics.get("net_pnl", "")),
                winrate=md(metrics.get("winrate", "")),
                avg_win=md(metrics.get("avg_win", "")),
                avg_loss=md(metrics.get("avg_loss", "")),
                profit_factor=md(metrics.get("profit_factor", "")),
                max_drawdown=md(metrics.get("max_drawdown", "")),
                notes=md(metrics.get("notes", "")),
            )
        )
    text = f"""# Audit Request: {context['audit_id']}

```


Resumen

```
- Fecha de generacion: {context['created_at']}
- Commit hash actual del sistema: `{context['git_commit']}`
- Rama actual: `{context['git_branch']}`
- Entorno usado: `{context['env'].get('TRADEO_ENVIRONMENT', 'local')}`
- Version del bot/laboratorio: Tradeo research lab at commit `{context['git_commit']}`
- Fuente de datos: IB Paper / Interactive Brokers historical market data provider
- Timezone principal: Europe/Madrid
- Timezone de mercado: America/New_York
- Mercado: US equities
- Timeframes usados: {'', '.join(timeframes) or 'not_available'}`
- Universo de activos analizado: `data/universe_us_mid_small.csv`, {len(context['universe_symbols'])} simbolos configurados
- Numero total de patrones detectados/exportados: {len(patterns)}
- Numero total de variantes probadas: {len(experiment_rows)}
- Numero total de variantes descartadas: {discarded}
- Numero de eventos exportados: {len(event_rows)}
- Numero de trades paper cerrados exportados: {total_paper_trades}. Si es 0, falta evidencia `closed_lab_trades` y no se puede rankear por PnL, regimen o entry_variant.
```

Que esperamos de ChatGPT

Auditar si los patrones exportados son estadisticamente creibles, reproducibles y operables. En especial: sesgos, lookahead, survivorship, independencia de muestras, concentracion por ticker/fecha/regimen, robustez ante costes, dependencia de outliers, bugs del laboratorio y tests automaticos que faltan.

Tabla Resumen

```
{chr(10).join(table_lines)}
"""
    (package / "AUDIT_REQUEST.md").write_text(text, encoding="utf-8")
```

```
def md(value: Any) -> str:
    return str(value).replace("|", "\\|").replace("\n", " ")
```

```
def write_code_references(package: Path, patterns: list[dict[str, Any]]) -> None:
    shared = ""
    Detection:
    - path: backend/tradeo/research/window_sampler.py
    - function: WindowSampler.sample, WindowSampler._forward_outcome
    - path: backend/tradeo/research/cluster_research_engine.py
    - function: ClusterResearchEngine.discover, ClusterResearchEngine._cluster_window_size
```

```
Validation:
- path: backend/tradeo/research/reward_risk_analyzer.py
- function: RewardRiskAnalyzer.analyze, RewardRiskAnalyzer.metrics_for_rr
- path: backend/tradeo/research/validation_gate.py
- function: ValidationGate.evaluate_many
```

```
Registry:
- path: backend/tradeo/research/novel_pattern_registry.py
- function: NovelPatternRegistry.store_candidates, NovelPatternRegistry.upsert_candidate
```

```
Current Matching:
- path: backend/tradeo/research/novel_pattern_matcher.py
- function: NovelPatternMatcher.match_current
```

```
Execution:
- path: backend/tradeo/services/paper_broker.py
- function: PaperBroker.execute_signal
- path: backend/tradeo/services/ibkr_broker.py
- function: IBKRBroker.preview_signal_order, IBKRBroker.submit_signal_bracket
```

```
Risk:
- path: backend/tradeo/services/risk_manager.py
- function: RiskManager.check_signal, RiskManager.approve_signal_for_live
```

```
Configuration:
- path: backend/tradeo/core/config.py
- class: Settings
- path: config/strategy_cup_v0.json
```

```
Tests:
- path: backend/tradeo/tests/test_pattern_discovery_lab.py
- path: backend/tradeo/tests/test_reward_risk_analyzer.py
- path: backend/tradeo/tests/test_data_provider.py
```

```
TODOs conocidos:
- Persistir todos los eventos crudos, no solo ejemplos representativos.
- Persistir bid/ask/spread en el momento de senal.
- Persistir sector y regimen de mercado.
- Persistir fills IB anonimizados cuando se active paper execution.
```

```
"""
    lines = ["# Code References", ""]
    for pattern in patterns:
        lines.extend([f"## {pattern_id(pattern)}", "", f"Pattern name: `{pattern.get('name', '')}`", "", shared, ""])
    (package / "code_references.md").write_text("\n".join(lines), encoding="utf-8")
```

```
def write_config_snapshot(config_dir: Path, context: dict[str, Any]) -> None:
    env = context["env"]
    config_dir.mkdir(parents=True, exist_ok=True)
    strategy = read_json(ROOT / "config" / "strategy_cup_v0.json")
    snapshots = {
        "detector_config.json": {
```

```

        "discovery_period": env.get("TRADEO_DISCOVERY_PERIOD", "5y"),
        "discovery_interval": env.get("TRADEO_DISCOVERY_INTERVAL", "1d"),
        "discovery_window_sizes": env.get("TRADEO_DISCOVERY_WINDOW_SIZES", "20,50,100,200"),
        "discovery_forward_bars": env.get("TRADEO_DISCOVERY_FORWARD_BARS", "5,10,20"),
        "discovery_rr_levels": env.get("TRADEO_DISCOVERY_RR_LEVELS", "1.5,2.0,2.5,3.0,4.0,5.0"),
        "discovery_stride": env.get("TRADEO_DISCOVERY_STRIDE", "3"),
        "discovery_min_cluster_size": env.get("TRADEO_DISCOVERY_MIN_CLUSTER_SIZE", "60"),
        "discovery_max_clusters_per_window": env.get("TRADEO_DISCOVERY_MAX_CLUSTERS_PER_WINDOW", "12"),
        "strategy_config": strategy,
    },
    "universe_config.json": {
        "universe_file": "data/universe_us_mid_small.csv",
        "asset_class": "US equities",
        "symbol_count": len(context["universe_symbols"]),
        "symbols": context["universe_symbols"],
    },
    "risk_config.json": {
        "trading_mode": env.get("TRADEO_TRADING_MODE", "paper"),
        "live_trading_enabled": env.get("TRADEO_LIVE_TRADING_ENABLED", "false"),
        "kill_switch_enabled": env.get("TRADEO_KILL_SWITCH_ENABLED", "false"),
        "risk_per_trade_pct": env.get("TRADEO_RISK_PER_TRADE_PCT", "0.01"),
        "daily_loss_limit_pct": env.get("TRADEO_DAILY_LOSS_LIMIT_PCT", "0.02"),
        "monthly_loss_limit_pct": env.get("TRADEO_MONTHLY_LOSS_LIMIT_PCT", "0.08"),
        "max_open_positions": env.get("TRADEO_MAX_OPEN_POSITIONS", "4"),
        "max_position_value_pct": env.get("TRADEO_MAX_POSITION_VALUE_PCT", "0.45"),
    },
    "execution_config.json": {
        "allow_long": env.get("TRADEO_ALLOW_LONGS", "true"),
        "allow_shorts": env.get("TRADEO_ALLOW_SHORTS", "true"),
        "allow_options": env.get("TRADEO_ALLOW_OPTIONS", "false"),
        "allow_margin": env.get("TRADEO_ALLOW_MARGIN", "false"),
        "ibkr_readonly": env.get("TRADEO_IBKR_READONLY", "true"),
        "ibkr_allow_market_orders": env.get("TRADEO_IBKR_ALLOW_MARKET_ORDERS", "false"),
        "ibkr_max_order_value_usd": env.get("TRADEO_IBKR_MAX_ORDER_VALUE_USD", "1500"),
    },
    "ib_paper_config.redacted.json": {
        "data_source": "Interactive Brokers Paper",
        "ibkr_host": env.get("TRADEO_IBKR_HOST", "host.docker.internal"),
        "ibkr_port": env.get("TRADEO_IBKR_PORT", "7497"),
        "ibkr_client_id": env.get("TRADEO_IBKR_CLIENT_ID", "17"),
        "ibkr_account": "REDACTED_OR_EMPTY",
        "account_id_redacted": True,
        "readonly": env.get("TRADEO_IBKR_READONLY", "true"),
    },
    "data_config.json": {
        "market_data_provider": env.get("TRADEO_MARKET_DATA_PROVIDER", "ibkr"),
        "allow_synthetic_market_data": env.get("TRADEO_ALLOW_SYNTHETIC_MARKET_DATA", "false"),
        "scan_period": env.get("TRADEO_SCAN_PERIOD", "2y"),
        "scan_interval": env.get("TRADEO_SCAN_INTERVAL", "1d"),
        "min_avg_dollar_volume": env.get("TRADEO_MIN_AVG_DOLLAR_VOLUME", "5000000"),
        "min_price": env.get("TRADEO_MIN_PRICE", "2.0"),
        "max_atr_pct": env.get("TRADEO_MAX_ATR_PCT", "0.14"),
    },
}
for name, payload in snapshots.items():
    (config_dir / name).write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")

def read_json(path: Path) -> Any:
    if not path.exists():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))

def write_supporting_docs(
    package: Path,
    context: dict[str, Any],
    patterns: list[dict[str, Any]],
    event_rows: list[dict[str, Any]],
    experiment_rows: list[dict[str, Any]],
    paper_trade_rows: list[dict[str, Any]],
    fill_rows: list[dict[str, Any]],
) -> None:
    (package / "data_lineage.md").write_text(f"""# Data Lineage

- Fuente de barras/precios: Interactive Brokers Paper historical data through `IBKRHistoricalDataProvider`.
- Fuente de bid/ask: no persistida en este paquete; campos bid/ask/spread quedan vacios.
- Fuente de paper trades: `paper_trades.csv` exporta {len(paper_trade_rows)} filas desde Laboratorio cuando existen.
- Fuente de fills: `ib_fills.csv` exporta {len(fill_rows)} fills paper reconstruidos desde trades cerrados/registrados cuando existen.
- Fuente de breakdown por regimen: `metrics_by_regime.csv` usa closed paper trades si existen; si no, contiene `empty_reason`.
- Fuente de breakdown por entry variant: `metrics_by_entry_variant.csv` usa `entry_variant_id` de metadata de senal si existen closed paper trades; si no, contiene `empty_reason`.
- Fuente de comisiones/slippage: se exportan desde metadata de trade si estan persistidas; si no, quedan vacias o en cero.
- Fuente de volumen: barras OHLCV de IBKR cuando estan disponibles; el export actual solo conserva volumen en features agregadas si el detector lo genero.
- Ajustes por splits/dividendos: no documentados por barra en el laboratorio actual; requiere auditoria adicional del proveedor IBKR.
- Acciones deslistadas: el universo actual viene de `data/universe_us_mid_small.csv`; no hay control completo de survivorship bias.
- Control survivorship bias: pendiente; el paquete documenta universo usado pero no contiene delistings.
- Control lookahead bias: `WindowSampler` usa ventana hasta `window_end` y forward path posterior solo para label/metricas; revisar codigo citado.
- Sincronizacion timestamps: timestamps diarios sin zona se normalizan a `+00:00`; timezone de mercado declarada `America/NewYork`.
- Datos disponibles en el momento de senal: features de embedding sobre ventana historica hasta `window_end`; forward outcomes se calculan despues y no deben usarse en deteccion.
- Datos calculados despues: `outcome_r`, `mfe_r`, `mae_r`, profit factor, expectancy, drawdown, in/out-of-sample metrics.
- Snapshot: generado {context['created_at']} desde API local en caliente; el laboratorio continuo puede seguir insertando runs despues del export.

```

```

""" , encoding="utf-8")

(package / "known_issues.md").write_text(f"""# Known Issues

- El laboratorio guarda ejemplos representativos por patron, no todos los eventos crudos. Eventos exportados: {len(event_rows)}.
- Puede haber muestras solapadas por `stride`; independencia estadística requiere auditoria adicional.
- Bid/ask/spread no estan persistidos en algunos eventos.
- Slippage estimado solo es auditable si cada trade lo persiste en metadata.
- Universo limitado a `data/universe_us_mid_small.csv`.
- Datos actuales vienen de IB Paper/historicos; no equivalen a ejecucion live.
- Posible survivorship bias: no hay archivo de tickers deslistados.
- Posible concentracion por pocos tickers o regimenes: revisar `metrics_by_ticker.csv` y `metrics_by_period.csv`.
- Regimen de mercado y sector se exportan desde features/metadata cuando estan disponibles.
- `metrics_by_regime.csv` y `metrics_by_entry_variant.csv` deben tener filas accionables solo cuando haya `closed_lab_trades`; si no, documentan la razon de vacio.
- Los timestamps diarios se normalizan a UTC cuando el dato original no trae timezone explicita.
- Fills IB Paper exportados en este paquete: {len(fill_rows)}.
- El paquete contiene {len(patterns)} patrones y {len(experiment_rows)} variantes RR; ChatGPT debe revisar tambien variantes descartadas.
""" , encoding="utf-8")

(package / "audit_checklist.md").write_text(f"""# Audit Checklist

- [ ] No hay claves ni tokens.
- [ ] No hay account IDs reales.
- [ ] Todos los timestamps tienen timezone.
- [ ] Cada patron tiene regla de deteccion clara.
- [ ] Cada patron tiene regla de entrada clara.
- [ ] Cada patron tiene regla de salida clara.
- [ ] Costes separados: comision, spread, slippage, fees.
- [ ] Hay lista de variantes descartadas.
- [ ] Hay numero total de experimentos probados.
- [ ] Hay datos por ticker.
- [ ] Hay datos por periodo.
- [ ] Hay fills de IB Paper o archivo vacio con cabecera.
- [ ] Hay code references.
- [ ] Hay config snapshot.
- [ ] Se indica si cada muestra es independiente.
- [ ] Se indica si hay datos in-sample, out-of-sample y paper-live.
- [ ] Se documentan known issues.
""" , encoding="utf-8")

(package / "chatgpt_questions.md").write_text(f"""# ChatGPT Questions

1. ¿Los patrones exportados estan definidos de forma objetiva y reproducible?
2. ¿Hay riesgo de lookahead bias?
3. ¿Hay riesgo de survivorship bias?
4. ¿Las muestras son realmente independientes?
5. ¿Hay concentración excesiva en pocos tickers?
6. ¿Hay concentración excesiva en pocos días o régimen de mercado?
7. ¿Los resultados sobreviven a costes realistas?
8. ¿El EV neto es positivo?
9. ¿El patrón depende de pocos trades extremos?
10. ¿Qué patrón debería priorizarse?
11. ¿Qué patrón debería descartarse o congelarse?
12. ¿Qué datos faltan para una validación exhaustiva?
13. ¿Qué cambios necesita el laboratorio de research?
14. ¿Qué tests automáticos deberían añadirse?
15. ¿Qué umbrales mínimos deberían exigirse antes de pasar a paper controlado o live?

Nota: esta exportacion contiene todos los patrones almacenados en el snapshot, no solo cuatro. Si se desea una auditoria limitada a cuatro candidatos, filtrar `pattern_catalog.csv` y mantener el manifest actualizado.
""" , encoding="utf-8")

(package / "reproducibility.md").write_text(f"""# Reproducibility

```

```
## Comando usado para exportar patrones
```

```
python3 research/audit_bridge/export_audit_package.py --audit-id {context['audit_id']}
```

```
## Comando usado para exportar trades
```

El mismo exportador crea `paper\_trades.csv`. Si no hay trades paper en DB, genera cabecera vacia.

```
## Comando usado para exportar fills
```

El mismo exportador crea `ib\_fills.csv`. Si no hay fills IB Paper anonimizados en DB, genera cabecera vacia.

```
## Variables de entorno necesarias
```

```

- `TRADEO_ADMIN_USERNAME`
- `TRADEO_ADMIN_PASSWORD`
- `TRADEO_MARKET_DATA_PROVIDER`
- `TRADEO_ALLOW_SYNTHETIC_MARKET_DATA`
- `TRADEO_IBKR_HOST`
- `TRADEO_IBKR_PORT`
- `TRADEO_IBKR_CLIENT_ID`
- `TRADEO_IBKR_READONLY`

```

No incluir valores secretos en el paquete.

```
## Dependencias principales
```

```
- Python standard library para export/validation.
```

- Tradeo backend local en `http://localhost:8000/api`.
- Docker Compose stack de Tradeo si se quiere regenerar DB/API.
- Backend: Python 3.12 en contenedor.
- Frontend: Node/Next.js segun `frontend/Dockerfile`.

Validar paquete completo despues del Director gate

```
python3 research/audit_bridge/validate_audit_package.py research/audit_bridge/requests/{context['audit_id']}
```

Ejecutar Director gate antes del validator estricto

```
python3 research/audit_bridge/director_gate.py research/audit_bridge/requests/{context['audit_id']} \
--json-output research/audit_bridge/requests/{context['audit_id']}/director_gate_result.json \
--markdown-output research/audit_bridge/requests/{context['audit_id']}/director_gate_result.md \
--allow-blocked-exit-zero
```

Hashes

```
Ver `file_hashes.sha256`.
""", encoding="utf-8")
```

```
def write_audit_summary_for_chatgpt(
    package: Path,
    pattern_rows: list[dict[str, Any]],
    event_rows: list[dict[str, Any]],
    experiment_rows: list[dict[str, Any]],
    metrics_rows: list[dict[str, Any]],
    metrics_ticker_rows: list[dict[str, Any]],
    metrics_period_rows: list[dict[str, Any]],
    metrics_regime_rows: list[dict[str, Any]],
    metrics_entry_variant_rows: list[dict[str, Any]],
    paper_trade_rows: list[dict[str, Any]],
    fill_rows: list[dict[str, Any]],
) -> None:
    catalog_by_pattern = {row["pattern_id"]: row for row in pattern_rows}
    sample_counts = sorted((to_int(row.get("independent_sample_count")) for row in metrics_rows)
                           if value is not None]
    total_patterns = len(pattern_rows)

    events_by_pattern: dict[str, list[dict[str, Any]]] = defaultdict(list)
    for row in event_rows:
        events_by_pattern[str(row.get("pattern_id", ""))].append(row)

    summary_rows = [
        ("total patterns", total_patterns),
        ("min independent_sample_count", sample_counts[0] if sample_counts else ""),
        ("p25", percentile(sample_counts, 0.25)),
        ("median", percentile(sample_counts, 0.50)),
        ("p75", percentile(sample_counts, 0.75)),
        ("p90", percentile(sample_counts, 0.90)),
        ("max", sample_counts[-1] if sample_counts else ""),
        ("patterns >= 30 samples", count_ge(sample_counts, 30)),
        ("patterns >= 50 samples", count_ge(sample_counts, 50)),
        ("patterns >= 100 samples", count_ge(sample_counts, 100)),
        ("patterns >= 300 samples", count_ge(sample_counts, 300)),
        ("patterns >= 500 samples", count_ge(sample_counts, 500)),
    ]

    top_by_samples = sorted(
        metrics_rows,
        key=lambda row: (to_int(row.get("independent_sample_count")) or 0, to_int(row.get("sample_count")) or 0),
        reverse=True,
    )[:30]

    pnl_rows = [
        row for row in metrics_rows
        if (to_int(row.get("trade_count")) or 0) > 0 and to_float(row.get("net_pnl")) is not None
    ]
    top_by_pnl = sorted(pnl_rows, key=lambda row: to_float(row.get("net_pnl")) or 0.0, reverse=True)[:30]

    pnl_best_trade = [
        row for row in pnl_rows
        if materially_depends_on(row.get("net_pnl"), row.get("pnl_without_best_trade"))
    ]
    pnl_top5 = [
        row for row in pnl_rows
        if materially_depends_on(row.get("net_pnl"), row.get("pnl_without_top_5_trades"))
    ]
    few_tickers = [row for row in metrics_rows if (to_int(row.get("ticker_count")) or 0) < 5]
    few_days = [
        {"pattern_id": pid, "days": len(unique_event_days(rows))}
        for pid, rows in events_by_pattern.items()
        if len(unique_event_days(rows)) < 5
    ]
    single_regime = [
        {"pattern_id": pid, "regime": next(iter(regimes)) if regimes else ""}
        for pid, rows in events_by_pattern.items()
        for regimes in [str(row.get("market_regime", "")).strip() for row in rows if str(row.get("market_regime", "")).strip()]
        if len(regimes) == 1
    ]
```

```

]

duplicate_groups: dict[str, int] = defaultdict(int)
for row in event_rows:
    group = str(row.get("duplicate_group_id", "")).strip()
    if group:
        duplicate_groups[group] += 1
possible_duplicates = sum(count - 1 for count in duplicate_groups.values() if count > 1)
non_independent = sum(1 for row in event_rows if str(row.get("is_independent_sample", "")).lower() != "true")
timestamps_without_timezone = count_timestamps_without_timezone(pattern_rows, event_rows, experiment_rows, metrics_rows,
metrics_ticker_rows, metrics_period_rows)
leakage_features = leakage_feature_rows(event_rows)
uses_future = [row for row in pattern_rows if str(row.get("uses_future_data", "")).lower() == "true"]
duplicate_variants = count_duplicate_variants(experiment_rows)
missing_exit = [row for row in pattern_rows if not str(row.get("exit_rule_plaintext", "")).strip()]
missing_entry = [row for row in pattern_rows if not str(row.get("entry_rule_plaintext", "")).strip()]

candidates_for_audit = [
    row for row in top_by_samples
    if (to_int(row.get("independent_sample_count")) or 0) >= 100 and (to_int(row.get("ticker_count")) or 0) >= 8
][:30]
needs_more_samples = [row for row in metrics_rows if (to_int(row.get("independent_sample_count")) or 0) < 100]
freeze_until_more_data = [row for row in metrics_rows if (to_int(row.get("trade_count")) or 0) == 0]
discard_candidates = [
    row for row in metrics_rows
    if (to_int(row.get("independent_sample_count")) or 0) < 30 or (to_int(row.get("ticker_count")) or 0) < 5
]

validation_stance = (
    f"""Validation stance:** no pattern is approved automatically. "
    f"This package includes {len(paper_trade_rows)} paper trades and {len(fill_rows)} reconstructed fills; "
    f"Director must still verify execution quality before promotion."
    if paper_trade_rows or fill_rows
    else """Validation stance:** no pattern is approved for operation in this package. Paper/live execution validation is
    unavailable because `paper_trades.csv` and `ib_fills.csv` have zero rows; there are no `closed_lab_trades` from which to rank by regime
    or entry variant."
)
lines = [
    "# Audit Summary For ChatGPT",
    "",
    "Generated from the package CSV exports. This is a lab-audit and candidate-ranking aid only.",
    "",
    validation_stance,
    "",
    "## A. Sample Distribution By Pattern",
    "",
    "| metric | value |",
    "|---|---:|",
]
lines.extend(f"| {label} | {value} |" for label, value in summary_rows)

lines.extend([
    "",
    "## B. Top 30 Patterns By Independent Sample Count",
    "",
    "| pattern_id | pattern_name | independent_sample_count | event_count | ticker_count | first_seen | last_seen | net_pnl_estimado | profit_factor | max_drawdown | status |",
    "|---|---|---:|---:|---:|---:|---:|---:|---:|---:|",
])
for row in top_by_samples:
    catalog = catalog_by_pattern.get(row["pattern_id"], {})
    lines.append(
        f"| {md(row['pattern_id'])} | {md(catalog.get('pattern_name', ''))} | {cell(row.get('independent_sample_count'))} | {cell(row.get('sample_count'))} | {cell(row.get('ticker_count'))} | {cell(row.get('first_seen'))} | {cell(row.get('last_seen'))} | {cell(row.get('net_pnl'))} | {cell(row.get('profit_factor'))} | {cell(row.get('max_drawdown'))} | {md(catalog.get('status', ''))} |"
    )

lines.extend([
    "",
    "## C. Top 30 Patterns By Estimated Net PnL",
    "",
])
if top_by_pnl:
    lines.extend([
        "| pattern_id | pattern_name | independent_sample_count | trade_count | ticker_count | net_pnl | profit_factor | max_drawdown | pnl_without_best_trade | pnl_without_top_5_trades | status |",
        "|---|---|---:|---:|---:|---:|---:|---:|---:|---:|",
    ])
    for row in top_by_pnl:
        catalog = catalog_by_pattern.get(row["pattern_id"], {})
        lines.append(
            f"| {md(row['pattern_id'])} | {md(catalog.get('pattern_name', ''))} | {cell(row.get('independent_sample_count'))} | {cell(row.get('trade_count'))} | {cell(row.get('ticker_count'))} | {cell(row.get('net_pnl'))} | {cell(row.get('profit_factor'))} | {cell(row.get('max_drawdown'))} | {cell(row.get('pnl_without_best_trade'))} | {cell(row.get('pnl_without_top_5_trades'))} | {md(catalog.get('status', ''))} |"
        )
    else:
        lines.append("No available `net_pnl` values. Current package has zero paper trades/fills, so PnL ranking is not actionable.")

lines.extend([
    "",
    "## D. Concentration",
    "",
    "| check | count | notes |",
    "|---|---:|---:|",
    f"| patterns whose PnL depends on best trade | {len(pnl_best_trade)} | Requires non-empty PnL fields. |",
])

```

```

f"| patterns whose PnL depends on top 5 trades | {len(pnl_top5)} | Requires non-empty PnL fields. |",
f"| patterns concentrated in fewer than 5 tickers | {len(few_tickers)} | Based on `metrics_by_pattern.ticker_count`. |",
f"| patterns concentrated in fewer than 5 days | {len(few_days)} | Based on unique event dates in `pattern_events.csv`. |",
f"| patterns concentrated in one market regime | {len(single_regime)} | Current export mostly uses `not_persisted`; regime audit
is limited. |",
f"| regime performance buckets available | {count_available_bucket_rows(metrics_regime_rows)} | See `metrics_by_regime.csv`;
empty rows include `empty_reason`. |",
f"| entry variant performance buckets available | {count_available_bucket_rows(metrics_entry_variant_rows)} | See
`metrics_by_entry_variant.csv`; empty rows include `empty_reason`. |",
"""
### E. Automatically Detected Risks
"""
f"| risk | count | notes |",
f"|---|---|---|",
f"| possible duplicated event rows | {possible_duplicates} | Counted from repeated `duplicate_group_id`. |",
f"| signals not marked independent | {non_independent} | Any `is_independent_sample` value other than `true`. |",
f"| timestamps without timezone | {timestamps_without_timezone} | Checked exported timestamp-like fields. |",
f"| features with possible leakage keywords | {len(leakage_features)} | Keywords: future, forward, outcome, target, label, mfe,
mae. Review manually. |",
f"| patterns with uses_future_data = true | {len(uses_future)} | Must be zero unless explicitly justified. |",
f"| duplicated or nearly duplicated variants | {duplicate_variants} | Exact duplicate `(pattern_id, parameters_json)` pairs. |",
f"| patterns without clear exit rule | {len(missing_exit)} | Empty `exit_rule_plaintext`. |",
f"| patterns without clear entry rule | {len(missing_entry)} | Empty `entry_rule_plaintext`. |",
"""
### F. Preliminary Claw Recommendation
"""
f"| bucket | count | recommendation |",
f"|---|---|---|",
f"| candidates_for_audit | {len(candidates_for_audit)} | Review first for lab quality and statistical robustness; do not approve
for operation. |",
f"| needs_more_samples | {len(needs_more_samples)} | Gather more independent samples before statistical claims. |",
f"| likely_duplicates | {possible_duplicates + duplicate_variants} | Deduplicate or explain before ranking. |",
f"| freeze_until_more_data | {len(freeze_until_more_data)} | No paper trades yet; execution validation unavailable. |",
f"| discard_candidates | {len(discard_candidates)} | Low sample or low ticker diversity; keep rejected unless new evidence
appears. |",
f"| no_closed_lab_trades_bucket_gap | {int(not paper_trade_rows)} | Fill `paper_trades.csv` with closed lab trades carrying
regime and entry_variant metadata before bucket ranking. |",
"""
### First Candidates For Audit
"""
f"| pattern_id | pattern_name | independent_sample_count | ticker_count | status |",
f"|---|---|---|---|---|",
])
for row in candidates_for_audit[:30]:
    catalog = catalog_by_pattern.get(row["pattern_id"], {})
    lines.append(
        f"| {md(row['pattern_id'])} | {md(catalog.get('pattern_name', ''))} | {cell(row.get('independent_sample_count'))} |
{cell(row.get('ticker_count'))} | {md(catalog.get('status', ''))} |"
    )

lines.extend(["", "### Regime Bucket Snapshot", ""])
available_regime = [row for row in metrics_regime_rows if str(row.get("analysis_available", "")).lower() == "true"]
if available_regime:
    lines.extend([
        "| pattern_id | market_regime | trade_count | net_pnl | expectancy_r | profit_factor |",
        "|---|---|---|---|---|---|",
    ])
    for row in sorted(available_regime, key=lambda item: to_float(item.get("expectancy_r")) or 0.0, reverse=True)[:20]:
        lines.append(
            f"| {md(row.get('pattern_id', ''))} | {md(row.get('market_regime', ''))} | {cell(row.get('trade_count'))} |
{cell(row.get('net_pnl'))} | {cell(row.get('expectancy_r'))} | {cell(row.get('profit_factor'))} |"
        )
else:
    reason = next((str(row.get("empty_reason", "")).strip() for row in metrics_regime_rows if row.get("empty_reason")), "")
    lines.append(reason or "No regime bucket performance is available.")

lines.extend(["", "### Entry Variant Bucket Snapshot", ""])
available_variants = [row for row in metrics_entry_variant_rows if str(row.get("analysis_available", "")).lower() == "true"]
if available_variants:
    lines.extend([
        "| pattern_id | entry_variant_id | trade_count | net_pnl | expectancy_r | profit_factor |",
        "|---|---|---|---|---|---|",
    ])
    for row in sorted(available_variants, key=lambda item: to_float(item.get("expectancy_r")) or 0.0, reverse=True)[:20]:
        lines.append(
            f"| {md(row.get('pattern_id', ''))} | {md(row.get('entry_variant_id', ''))} | {cell(row.get('trade_count'))} |
{cell(row.get('net_pnl'))} | {cell(row.get('expectancy_r'))} | {cell(row.get('profit_factor'))} |"
        )
else:
    reason = next((str(row.get("empty_reason", "")).strip() for row in metrics_entry_variant_rows if row.get("empty_reason")), "")
    lines.append(reason or "No entry-variant bucket performance is available.")

lines.extend([
    "",
    "### Bottom-Line Instruction",
    "",
    "Do not approve any pattern from this package. Use it to audit the research lab, detect data/logic issues, and rank candidates
for future paper validation.",
    "",
])
(package / "AUDIT_SUMMARY_FOR_CHATGPT.md").write_text("\n".join(lines), encoding="utf-8")

def count_available_bucket_rows(rows: list[dict[str, Any]]) -> int:
    return sum(1 for row in rows if str(row.get("analysis_available", "")).strip().lower() == "true")

```

```

def to_int(value: Any) -> int | None:
    try:
        text = str(value).strip()
        if not text:
            return None
        return int(float(text))
    except (TypeError, ValueError):
        return None

def to_float(value: Any) -> float | None:
    try:
        text = str(value).strip()
        if not text:
            return None
        return float(text)
    except (TypeError, ValueError):
        return None

def percentile(values: list[int], q: float) -> str:
    if not values:
        return ""
    if len(values) == 1:
        return str(values[0])
    pos = (len(values) - 1) * q
    lo = int(pos)
    hi = min(lo + 1, len(values) - 1)
    weight = pos - lo
    return f"{values[lo] * (1 - weight) + values[hi] * weight:.2f}"

def count_ge(values: list[int], threshold: int) -> int:
    return sum(1 for value in values if value >= threshold)

def cell(value: Any) -> str:
    text = str(value if value is not None else "").strip()
    return md(text) if text else ""

def materially_depends_on(total_value: Any, reduced_value: Any) -> bool:
    total = to_float(total_value)
    reduced = to_float(reduced_value)
    if total is None or reduced is None or abs(total) < 1e-9:
        return False
    return abs(total - reduced) / abs(total) >= 0.5

def unique_event_days(rows: list[dict[str, Any]]) -> set[str]:
    days = set()
    for row in rows:
        timestamp = str(row.get("market_timestamp") or row.get("detected_at") or "").strip()
        if len(timestamp) >= 10:
            days.add(timestamp[:10])
    return days

def count_timestamps_without_timezone(*row_groups: list[dict[str, Any]]) -> int:
    timestamp_columns = {
        "created_at",
        "first_seen",
        "last_seen",
        "detected_at",
        "market_timestamp",
        "bar_open_time",
        "bar_close_time",
        "tested_at",
        "dataset_start",
        "dataset_end",
        "in_sample_start",
        "in_sample_end",
        "out_of_sample_start",
        "out_of_sample_end",
        "paper_live_start",
        "paper_live_end",
        "period_start",
        "period_end",
    }
    count = 0
    for rows in row_groups:
        for row in rows:
            for key, value in row.items():
                if key not in timestamp_columns:
                    continue
                text = str(value).strip()
                if text and not re.search(r"(?:Z|[-+]\d\d:\d\d)$", text):
                    count += 1
    return count

def leakage_feature_rows(event_rows: list[dict[str, Any]]) -> list[dict[str, Any]]:
    pattern = re.compile(r"\b(future|forward|outcome|target|label|mfe|mae)\b", re.I)
    return [row for row in event_rows if pattern.search(str(row.get("features_used_json", "")))]

```

```

def count_duplicate_variants(experiment_rows: list[dict[str, Any]]) -> int:
    seen: set[tuple[str, str]] = set()
    duplicates = 0
    for row in experiment_rows:
        key = (str(row.get("pattern_id", "")), str(row.get("parameters_json", "")))
        if key in seen:
            duplicates += 1
        seen.add(key)
    return duplicates

def write_hashes(package: Path) -> None:
    lines = []
    for path in sorted(package.rglob("**")):
        if not path.is_file() or path.name == "file_hashes.sha256":
            continue
        digest = hashlib.sha256(path.read_bytes()).hexdigest()
        lines.append(f"{digest} {path.relative_to(package)}")
    (package / "file_hashes.sha256").write_text("\n".join(lines) + "\n", encoding="utf-8")

if __name__ == "__main__":
    sys.exit(main())

```

Full Document: `research/audit\_bridge/validate\_audit\_package.py`

`research/audit\_bridge/validate\_audit\_package.py` ? 599 lines, 28065 bytes, sha256 prefix `576e18692d1b3cdd`

```

#!/usr/bin/env python3
from __future__ import annotations

import argparse
import csv
import json
import re
import sys
from decimal import Decimal, InvalidOperation
from pathlib import Path
from typing import Any

REQUIRED_BRIDGE_FILES = [
    "README.md",
    "AUDIT_CONTRACT.md",
    "validate_audit_package.py",
    "requirements.txt",
]

REQUIRED_FILES = [
    "AUDIT_REQUEST.md",
    "AUDIT_SUMMARY_FOR_CHATGPT.md",
    "manifest.json",
    "pattern_catalog.csv",
    "pattern_events.csv",
    "paper_trades.csv",
    "ib_fills.csv",
    "experiment_registry.csv",
    "metrics_by_pattern.csv",
    "metrics_by_ticker.csv",
    "metrics_by_period.csv",
    "metrics_by_regime.csv",
    "metrics_by_entry_variant.csv",
    "code_references.md",
    "config_snapshot/detector_config.json",
    "config_snapshot/universe_config.json",
    "config_snapshot/risk_config.json",
    "config_snapshot/execution_config.json",
    "config_snapshot/ib_paper_config.redacted.json",
    "config_snapshot/data_config.json",
    "data_lineage.md",
    "known_issues.md",
    "audit_checklist.md",
    "chatgpt_questions.md",
    "reproducibility.md",
    "director_gate_result.json",
]

CSV_COLUMNS = {
    "pattern_catalog.csv": [
        "pattern_id", "pattern_name", "pattern_version", "description", "hypothesis",
        "detection_rule_plaintext", "entry_rule_plaintext", "exit_rule_plaintext",
        "stop_rule_plaintext", "take_profit_rule_plaintext", "time_stop_rule_plaintext",
        "timeframe", "asset_class", "universe", "session_filter", "min_volume_filter",
        "min_price_filter", "uses_fundamental_data", "uses_news_data", "uses_options_data",
        "uses_future_data", "known_risks", "status", "created_at", "first_seen", "last_seen",
    ],
    "pattern_events.csv": [
        "event_id", "pattern_id", "detected_at", "market_timestamp", "timezone", "ticker",
        "exchange", "timeframe", "bar_open_time", "bar_close_time", "signal_price",
        "bid_at_signal", "ask_at_signal", "spread_at_signal", "volume_at_signal",
        "atr_at_signal", "volatility_context", "market_regime", "sector",
        "was_trade_triggered", "trade_id", "reason_not_traded", "duplicate_group_id",
    ],
}

```



```

        "is_independent_sample", "data_available_at_signal", "available_data_cutoff_ts",
        "decision_ts", "entry_eligible_ts", "label_generated_ts", "source_bar_hash",
        "split_id", "features_used_json", "notes",
    ],
    "paper_trades.csv": [
        "trade_id", "event_id", "pattern_id", "ticker", "side", "quantity", "entry_signal_time",
        "evidence_type", "evidence_quality", "entry_order_time", "entry_fill_time",
        "entry_signal_price", "entry_order_type", "entry_limit_price", "entry_fill_price",
        "exit_signal_time", "exit_order_time",
        "exit_fill_time", "exit_order_type", "exit_limit_price", "exit_fill_price",
        "exit_reason", "gross_pnl", "commission", "estimated_spread_cost",
        "estimated_slippage", "other_fees", "net_pnl", "return_pct", "mae", "mfe",
        "holding_period_seconds", "risk_amount", "r_multiple", "account_equity_snapshot",
        "position_size_method", "notes",
    ],
    "ib_fills.csv": [
        "fill_id_hash", "trade_id", "order_id_hash", "ib_execution_time", "timezone", "ticker",
        "side", "quantity", "price", "commission", "liquidity_flag", "exchange", "order_type",
        "account_id_redacted", "raw_status", "notes",
    ],
    "experiment_registry.csv": [
        "experiment_id", "pattern_id", "variant_id", "created_at", "tested_at", "status",
        "reason_status", "parameters_json", "dataset_start", "dataset_end", "in_sample_start",
        "in_sample_end", "out_of_sample_start", "out_of_sample_end", "paper_live_start",
        "paper_live_end", "number_of_assets_tested", "number_of_events", "number_of_trades",
        "gross_pnl", "net_pnl", "winrate", "profit_factor", "sharpe", "sortino",
        "max_drawdown", "candidate_trial_count", "global_trial_count", "adjusted_p_value",
        "wrc_p_value", "spa_p_value", "fit_scope", "score_input_scope",
        "event_ledger_hash", "nested_discovery_replay_status",
        "nested_discovery_replay_implemented", "nested_discovery_replay_passed",
        "edge_claim", "drift_status", "active_blockers", "registry_path",
        "registry_hash", "registry_previous_hash", "registry_run_manifest_hash",
        "registry_hash_chain_valid", "notes",
    ],
    "metrics_by_pattern.csv": [
        "pattern_id", "sample_count", "independent_sample_count", "trade_count", "ticker_count",
        "sector_count", "first_seen", "last_seen", "gross_pnl", "net_pnl", "avg_trade",
        "median_trade", "std_trade", "winrate", "avg_win", "avg_loss", "payoff_ratio",
        "profit_factor", "expectancy", "max_drawdown", "max_consecutive_losses", "best_trade",
        "worst_trade", "top_5_trades_pnl", "bottom_5_trades_pnl", "pnl_without_best_trade",
        "pnl_without_top_5_trades", "pnl_without_worst_trade", "sharpe", "sortino", "calmar",
        "avg_holding_period_seconds", "notes",
    ],
    "metrics_by_ticker.csv": [
        "pattern_id", "ticker", "trade_count", "event_count", "net_pnl", "winrate",
        "profit_factor", "avg_trade", "max_drawdown", "first_seen", "last_seen", "notes",
    ],
    "metrics_by_period.csv": [
        "period_start", "period_end", "period_type", "pattern_id", "event_count", "trade_count",
        "gross_pnl", "net_pnl", "winrate", "profit_factor", "max_drawdown", "market_regime", "notes",
    ],
    "metrics_by_regime.csv": [
        "pattern_id", "market_regime", "analysis_available", "empty_reason", "trade_count",
        "event_count", "gross_pnl", "net_pnl", "avg_trade", "expectancy_r", "winrate",
        "profit_factor", "max_drawdown", "best_trade", "worst_trade", "first_seen",
        "last_seen", "notes",
    ],
    "metrics_by_entry_variant.csv": [
        "pattern_id", "entry_variant_id", "analysis_available", "empty_reason", "trade_count",
        "event_count", "gross_pnl", "net_pnl", "avg_trade", "expectancy_r", "winrate",
        "profit_factor", "max_drawdown", "best_trade", "worst_trade", "first_seen",
        "last_seen", "notes",
    ],
    ],
}

TIMESTAMP_COLUMNS = {
    "pattern_catalog.csv": ["created_at", "first_seen", "last_seen"],
    "pattern_events.csv": [
        "detected_at", "market_timestamp", "bar_open_time", "bar_close_time",
        "available_data_cutoff_ts", "decision_ts", "entry_eligible_ts", "label_generated_ts",
    ],
    "paper_trades.csv": [
        "entry_signal_time", "entry_order_time", "entry_fill_time", "exit_signal_time",
        "exit_order_time", "exit_fill_time",
    ],
    "ib_fills.csv": ["ib_execution_time"],
    "experiment_registry.csv": [
        "created_at", "tested_at", "dataset_start", "dataset_end", "in_sample_start",
        "in_sample_end", "out_of_sample_start", "out_of_sample_end", "paper_live_start", "paper_live_end",
    ],
    "metrics_by_pattern.csv": ["first_seen", "last_seen"],
    "metrics_by_ticker.csv": ["first_seen", "last_seen"],
    "metrics_by_period.csv": ["period_start", "period_end"],
    "metrics_by_regime.csv": ["first_seen", "last_seen"],
    "metrics_by_entry_variant.csv": ["first_seen", "last_seen"],
}

ACCOUNT_RE = re.compile(r"(?:DU|DUN|U|F)\d{5,}\b")
TZ_RE = re.compile(r"(?:Z|[-+]\d\d:\d\d)$")
SENSITIVE_KEY_RE = re.compile(
    r"^(?!(token|secret|password|apikey|api_key|credential|session_id|cookie))(_|$)",
    re.I,
)

def main() -> int:

```

```

parser = argparse.ArgumentParser()
parser.add_argument("package", type=Path)
args = parser.parse_args()
package = args.package
errors, csv_rows = validate_package(package)
status = validation_report(package, errors, csv_rows)

if errors:
    print("AUDIT PACKAGE INVALID")
    for error in errors:
        print(f"- {error}")
    print(json.dumps(status, indent=2))
    return 1
print("AUDIT PACKAGE OK")
print(json.dumps(status, indent=2))
return 0

def validate_package(package: Path) -> tuple[list[str], dict[str, list[dict[str, str]]]]:
    errors: list[str] = []
    bridge_root = package.parent.parent

    for rel in REQUIRED_BRIDGE_FILES:
        if not (bridge_root / rel).exists():
            errors.append(f"missing required bridge file: {rel}")

    for rel in REQUIRED_FILES:
        if not (package / rel).exists():
            errors.append(f"missing required file: {rel}")

    manifest = load_json(package / "manifest.json", errors)
    director_gate = load_json(package / "director_gate_result.json", errors)
    if manifest:
        check_manifest(package, manifest, errors)
    if director_gate:
        check_director_gate_result(director_gate, errors)

    csv_rows: dict[str, list[dict[str, str]]] = {}
    for rel, expected_columns in CSV_COLUMNS.items():
        path = package / rel
        if not path.exists():
            continue
        rows, columns = read_csv(path, errors)
        csv_rows[rel] = rows
        missing = [column for column in expected_columns if column not in columns]
        if missing:
            errors.append(f"{rel}: missing columns: {' '.join(missing)}")
        check_timestamps(rel, rows, errors)

    check_sensitive_values(package, errors)
    check_references(csv_rows, errors)
    check_evidence_contract(csv_rows, errors)
    check_experiment_contract(csv_rows, errors)
    check_pnl(csv_rows.get("paper_trades.csv", []), errors)
    check_manifest_counts(manifest, csv_rows, errors)
    check_duplicate_reporting(manifest, csv_rows.get("pattern_events.csv", []), errors)
    check_metric_coherence(csv_rows, errors)
    check_bucket_breakdowns(csv_rows, errors)
    return errors, csv_rows

def validation_report(
    package: Path,
    errors: list[str],
    csv_rows: dict[str, list[dict[str, str]]],
) -> dict[str, Any]:
    director_gate = load_json_silent(package / "director_gate_result.json")
    director_gate_status = director_gate_status_value(director_gate)
    schema_valid = not errors
    gate_allows_promotion = director_gate_promotion_allowed(director_gate, director_gate_status)
    return {
        "schema_valid": schema_valid,
        "package_valid": schema_valid,
        "director_gate_status": director_gate_status,
        "promotion_allowed": bool(schema_valid and gate_allows_promotion),
        "patterns": len(csv_rows.get("pattern_catalog.csv", [])),
        "events": len(csv_rows.get("pattern_events.csv", [])),
        "paper_trades": len(csv_rows.get("paper_trades.csv", [])),
        "fills": len(csv_rows.get("ib_fills.csv", [])),
        "experiments": len(csv_rows.get("experiment_registry.csv", [])),
        "error_count": len(errors),
    }

def load_json_silent(path: Path) -> Any:
    if not path.exists():
        return None
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception: # noqa: BLE001
        return None

def director_gate_status_value(result: Any) -> str:
    if not isinstance(result, dict):
        return "missing"

```

```

status = str(result.get("director_gate_status") or result.get("status") or "").strip().lower()
if status:
    return status
summary = result.get("summary")
if isinstance(summary, dict):
    return (
        str(summary.get("director_gate_status") or summary.get("director_gate") or "")
        .strip()
        .lower()
        or "missing"
    )
return "missing"

def director_gate_promotion_allowed(result: Any, status: str) -> bool:
    if not isinstance(result, dict):
        return False
    if "promotion_allowed" in result:
        return bool(result.get("promotion_allowed"))
    summary = result.get("summary")
    if isinstance(summary, dict) and "promotion_allowed" in summary:
        return bool(summary.get("promotion_allowed"))
    return status == "passed"

def load_json(path: Path, errors: list[str]) -> Any:
    if not path.exists():
        return None
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception as exc: # noqa: BLE001
        errors.append(f"{path.name}: invalid JSON: {exc}")
        return None

def check_manifest(package: Path, manifest: dict[str, Any], errors: list[str]) -> None:
    required_fields = [
        "audit_id",
        "created_at",
        "repo_commit",
        "repo_branch",
        "patterns_detected",
        "total_pattern_events",
        "total_paper_trades",
        "total_ib_fills",
        "total_experiment_variants",
        "duplicate_event_groups",
        "duplicate_repeated_rows",
        "metric_breakdowns",
        "files",
    ]
    for field in required_fields:
        if field not in manifest:
            errors.append(f"manifest.{field} is required")
    if manifest.get("contains_sensitive_data") is not False:
        errors.append("manifest.contains_sensitive_data must be false")
    if manifest.get("account_ids_redacted") is not True:
        errors.append("manifest.account_ids_redacted must be true")
    if manifest.get("orders_anonymized") is not True:
        errors.append("manifest.orders_anonymized must be true")
    if manifest.get("director_gate_required") is not True:
        errors.append("manifest.director_gate_required must be true")
    files = manifest.get("files")
    if not isinstance(files, list):
        errors.append("manifest.files must be a list")
        return
    listed_paths = {str(item.get("path", "")) for item in files if isinstance(item, dict)}
    required_manifest_paths = set(REQUIRED_FILES) - {"director_gate_result.json"}
    missing_listed = sorted(required_manifest_paths - listed_paths)
    if missing_listed:
        errors.append("manifest.files missing required paths: " + ", ".join(missing_listed))
    for rel in listed_paths:
        if rel and not (package / rel).exists():
            errors.append(f"manifest.files lists missing file: {rel}")

def check_director_gate_result(result: dict[str, Any], errors: list[str]) -> None:
    status = result.get("status")
    if status not in {"passed", "blocked", "invalid"}:
        errors.append("director_gate_result.json status must be passed, blocked or invalid")
    director_gate_status = result.get("director_gate_status")
    if director_gate_status is not None and director_gate_status != status:
        errors.append("director_gate_result.json director_gate_status must match status")
    if "schema_valid" in result and not isinstance(result.get("schema_valid"), bool):
        errors.append("director_gate_result.json schema_valid must be boolean when present")
    if "package_valid" in result and not isinstance(result.get("package_valid"), bool):
        errors.append("director_gate_result.json package_valid must be boolean when present")
    if "promotion_allowed" in result and not isinstance(result.get("promotion_allowed"), bool):
        errors.append("director_gate_result.json promotion_allowed must be boolean when present")
    if result.get("promotion_allowed") is True and status != "passed":
        errors.append("director_gate_result.json promotion_allowed cannot be true unless status is passed")
    summary = result.get("summary")
    if not isinstance(summary, dict):
        errors.append("director_gate_result.json summary must be an object")
        return
    if summary.get("director_gate") not in {"passed", "blocked", "invalid"}:

```

```

        errors.append("director_gate_result.json summary.director_gate must be present")
    if "promotion_allowed" in summary and not isinstance(summary.get("promotion_allowed"), bool):
        errors.append("director_gate_result.json summary.promotion_allowed must be boolean when present")
    for field in ("by_regime", "by_entry_variant"):
        value = summary.get(field)
        if not isinstance(value, dict):
            errors.append(f"director_gate_result.json summary.{field} must be present")
            continue
        available = value.get("available")
        buckets = value.get("buckets") if isinstance(value.get("buckets"), list) else []
        reason = str(value.get("empty_reason", "")).strip()
        if available is False and not reason:
            errors.append(f"director_gate_result.json summary.{field} is empty without empty_reason")
        if available is True and not buckets:
            errors.append(f"director_gate_result.json summary.{field} is available but has no buckets")

def read_csv(path: Path, errors: list[str]) -> tuple[list[dict[str, str]], list[str]]:
    try:
        with path.open(newline="", encoding="utf-8") as handle:
            reader = csv.DictReader(handle)
            columns = reader.fieldnames or []
            return list(reader), columns
    except Exception as exc: # noqa: BLE001
        errors.append(f"{path.name}: invalid CSV: {exc}")
        return [], []

def check_timestamps(rel: str, rows: list[dict[str, str]], errors: list[str]) -> None:
    for column in TIMESTAMP_COLUMNS.get(rel, []):
        for idx, row in enumerate(rows, start=2):
            value = (row.get(column) or "").strip()
            if not value:
                continue
            if not TZ_RE.search(value):
                errors.append(f"{rel}:{idx}: timestamp lacks timezone in {column}: {value}")

def check_sensitive_values(package: Path, errors: list[str]) -> None:
    for path in package.rglob("*"):
        if not path.is_file():
            continue
        text = path.read_text(encoding="utf-8", errors="ignore")
        if ACCOUNT_RE.search(text):
            errors.append(f"{path.relative_to(package)}: apparent IB account id present")
        if path.suffix.lower() == ".json":
            try:
                scan_json(json.loads(text), str(path.relative_to(package)), errors)
            except json.JSONDecodeError:
                pass
        if path.suffix.lower() == ".csv":
            with path.open(newline="", encoding="utf-8") as handle:
                for row_num, row in enumerate(csv.DictReader(handle), start=2):
                    for key, value in row.items():
                        if SENSITIVE_KEY_RE.search(key or "") and value and value.upper() not in {"REDACTED", "REDACTED_OR_EMPTY"}:
                            errors.append(f"{path.relative_to(package)}:{row_num}: sensitive field {key} has non-redacted value")

def scan_json(value: Any, location: str, errors: list[str], prefix: str = "") -> None:
    if isinstance(value, dict):
        for key, child in value.items():
            full = f"{prefix}.{key}" if prefix else str(key)
            if SENSITIVE_KEY_RE.search(str(key)) and child not in ("", None, "REDACTED", "REDACTED_OR_EMPTY"):
                errors.append(f"{location}: sensitive field {full} has non-redacted value")
            scan_json(child, location, errors, full)
    elif isinstance(value, list):
        for idx, child in enumerate(value):
            scan_json(child, location, errors, f"{prefix}[{idx}]")

def check_references(csv_rows: dict[str, list[dict[str, str]]], errors: list[str]) -> None:
    pattern_ids = {row["pattern_id"] for row in csv_rows.get("pattern_catalog.csv", []) if row.get("pattern_id")}
    event_ids = {row["event_id"] for row in csv_rows.get("pattern_events.csv", []) if row.get("event_id")}
    trade_ids = {row["trade_id"] for row in csv_rows.get("paper_trades.csv", []) if row.get("trade_id")}
    for rel in ["pattern_events.csv", "paper_trades.csv", "experiment_registry.csv", "metrics_by_pattern.csv", "metrics_by_ticker.csv"]:
        for idx, row in enumerate(csv_rows.get(rel, []), start=2):
            pid = row.get("pattern_id")
            if pid and pid not in pattern_ids:
                errors.append(f"{rel}:{idx}: unknown pattern_id {pid}")
    for idx, row in enumerate(csv_rows.get("paper_trades.csv", []), start=2):
        event_id = row.get("event_id")
        if event_id and event_id not in event_ids:
            errors.append(f"paper_trades.csv:{idx}: unknown event_id {event_id}")
    for idx, row in enumerate(csv_rows.get("ib_fills.csv", []), start=2):
        trade_id = row.get("trade_id")
        if trade_id and trade_id not in trade_ids:
            errors.append(f"ib_fills.csv:{idx}: unknown trade_id {trade_id}")
        if str(row.get("account_id_redacted", "")).strip().lower() != "true":
            errors.append(f"ib_fills.csv:{idx}: account_id_redacted must be true")
        order_id_hash = str(row.get("order_id_hash", "")).strip()
        if order_id_hash and order_id_hash.isdigit():
            errors.append(f"ib_fills.csv:{idx}: order_id_hash appears to contain raw numeric order id")

def check_evidence_contract(csv_rows: dict[str, list[dict[str, str]]], errors: list[str]) -> None:
    for idx, row in enumerate(csv_rows.get("paper_trades.csv", []), start=2):

```

```

evidence_type = str(row.get("evidence_type", "")).strip()
evidence_quality = str(row.get("evidence_quality", "")).strip()
if evidence_type != "ibkr_paper_fill":
    errors.append(f"paper_trades.csv:{idx}: evidence_type must be ibkr_paper_fill")
if evidence_quality not in {"standard", "normal"}:
    errors.append(f"paper_trades.csv:{idx}: evidence_quality must be standard")
notes = str(row.get("notes", "")).lower()
if "lab_shadow_observation" in notes or "near_miss" in notes or "no_ibkr_order" in notes:
    errors.append(f"paper_trades.csv:{idx}: shadow/no-broker evidence cannot be exported as paper trade")

def check_experiment_contract(csv_rows: dict[str, list[dict[str, str]]], errors: list[str]) -> None:
    required_fields = ("global_trial_count", "fit_scope", "score_input_scope")
    for idx, row in enumerate(csv_rows.get("experiment_registry.csv", []), start=2):
        for field in required_fields:
            if not str(row.get(field, "")).strip():
                errors.append(f"experiment_registry.csv:{idx}: {field} is required")

def check_pnl(rows: list[dict[str, str]], errors: list[str]) -> None:
    for idx, row in enumerate(rows, start=2):
        fields = ["gross_pnl", "commission", "estimated_spread_cost", "estimated_slippage", "other_fees", "net_pnl"]
        if not all((row.get(field) or "").strip() for field in fields):
            continue
        try:
            gross, commission, spread, slippage, fees, net = [Decimal(row[field]) for field in fields]
        except InvalidOperation:
            errors.append(f"paper_trades.csv:{idx}: invalid PnL decimal")
            continue
        expected = gross - commission - spread - slippage - fees
        if abs(expected - net) > Decimal("0.01"):
            errors.append(f"paper_trades.csv:{idx}: net_pnl formula mismatch expected {expected} got {net}")

def check_manifest_counts(
    manifest: Any,
    csv_rows: dict[str, list[dict[str, str]]],
    errors: list[str],
) -> None:
    if not isinstance(manifest, dict):
        return
    expected = {
        "patterns_detected": len(csv_rows.get("pattern_catalog.csv", [])),
        "total_pattern_events": len(csv_rows.get("pattern_events.csv", [])),
        "total_paper_trades": len(csv_rows.get("paper_trades.csv", [])),
        "total_ib_fills": len(csv_rows.get("ib_fills.csv", [])),
        "total_experiment_variants": len(csv_rows.get("experiment_registry.csv", [])),
        "total_metrics_by_regime_rows": len(csv_rows.get("metrics_by_regime.csv", [])),
        "total_metrics_by_entry_variant_rows": len(csv_rows.get("metrics_by_entry_variant.csv", [])),
    }
    for field, observed in expected.items():
        if field not in manifest:
            errors.append(f"manifest.{field} is required")
            continue
        if as_int(manifest.get(field)) != observed:
            errors.append(f"manifest.{field} mismatch expected {observed} got {manifest.get(field)}")

def check_duplicate_reporting(
    manifest: Any,
    event_rows: list[dict[str, str]],
    errors: list[str],
) -> None:
    if not isinstance(manifest, dict):
        return
    counts: dict[str, int] = {}
    for row in event_rows:
        group = str(row.get("duplicate_group_id", "")).strip()
        if group:
            counts[group] = counts.get(group, 0) + 1
    repeated_counts = [count for count in counts.values() if count > 1]
    duplicate_groups = len(repeated_counts)
    duplicate_repeated_rows = sum(repeated_counts)
    if as_int(manifest.get("duplicate_event_groups")) != duplicate_groups:
        errors.append(
            f"manifest.duplicate_event_groups mismatch expected {duplicate_groups} got {manifest.get('duplicate_event_groups')}"
        )
    if as_int(manifest.get("duplicate_repeated_rows")) != duplicate_repeated_rows:
        errors.append(
            f"manifest.duplicate_repeated_rows mismatch expected {duplicate_repeated_rows} got {manifest.get('duplicate_repeated_rows')}"
        )

def check_metric_coherence(csv_rows: dict[str, list[dict[str, str]]], errors: list[str]) -> None:
    trades_by_pattern: dict[str, list[dict[str, str]]] = {}
    for trade in csv_rows.get("paper_trades.csv", []):
        trades_by_pattern.setdefault(trade.get("pattern_id", ""), []).append(trade)
    for idx, row in enumerate(csv_rows.get("metrics_by_pattern.csv", []), start=2):
        pid = row.get("pattern_id", "")
        trades = trades_by_pattern.get(pid, [])
        trade_count = as_int(row.get("trade_count"))
        if trade_count != len(trades):
            errors.append(f"metrics_by_pattern.csv:{idx}: trade_count expected {len(trades)} got {trade_count}")
        if trade_count == 0:
            for field in ("avg_trade", "median_trade", "winrate", "profit_factor", "expectancy", "max_drawdown"):

```

```

        if str(row.get(field, "")).strip():
            errors.append(f"metrics_by_pattern.csv:{idx}: {field} must be empty when trade_count is zero")
            continue
        gross = sum(decimal_or_zero(trade.get("gross_pnl")) for trade in trades)
        net = sum(decimal_or_zero(trade.get("net_pnl")) for trade in trades)
        if abs(gross - decimal_or_zero(row.get("gross_pnl"))) > Decimal("0.01"):
            errors.append(f"metrics_by_pattern.csv:{idx}: gross_pnl does not match paper_trades.csv")
        if abs(net - decimal_or_zero(row.get("net_pnl"))) > Decimal("0.01"):
            errors.append(f"metrics_by_pattern.csv:{idx}: net_pnl does not match paper_trades.csv")

def check_bucket_breakdowns(csv_rows: dict[str, list[dict[str, str]]], errors: list[str]) -> None:
    for rel, bucket_column in (
        ("metrics_by_regime.csv", "market_regime"),
        ("metrics_by_entry_variant.csv", "entry_variant_id"),
    ):
        rows = csv_rows.get(rel, [])
        if not rows:
            errors.append(f"{rel}: must contain bucket rows or one explicit empty_reason row")
            continue
        available_rows = [row for row in rows if truthy(row.get("analysis_available"))]
        if not available_rows:
            if not any(str(row.get("empty_reason", "")).strip() for row in rows):
                errors.append(f"{rel}: empty breakdown requires non-empty empty_reason")
            continue
        for idx, row in enumerate(rows, start=2):
            if not truthy(row.get("analysis_available")):
                continue
            if not str(row.get(bucket_column, "")).strip():
                errors.append(f"{rel}:{idx}: {bucket_column} is required when analysis_available=true")
            if as_int(row.get("trade_count")) <= 0:
                errors.append(f"{rel}:{idx}: trade_count must be positive when analysis_available=true")

def as_int(value: Any) -> int:
    try:
        text = str(value or "").strip()
        return int(float(text)) if text else 0
    except (TypeError, ValueError):
        return 0

def decimal_or_zero(value: Any) -> Decimal:
    try:
        text = str(value or "").strip()
        return Decimal(text) if text else Decimal("0")
    except InvalidOperation:
        return Decimal("0")

def truthy(value: Any) -> bool:
    return str(value or "").strip().lower() in {"true", "1", "yes", "y"}

if __name__ == "__main__":
    sys.exit(main())

```

Full Document: `research/audit\_bridge/run\_director\_audit.py`

`research/audit\_bridge/run\_director\_audit.py` ? 409 lines, 15636 bytes, sha256 prefix `2f4c5cdc41009747`

```

#!/usr/bin/env python3
from __future__ import annotations

import argparse
import contextlib
import importlib.util
import io
import json
import os
import subprocess
import sys
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

ROOT = Path(__file__).resolve().parents[2]
BRIDGE = ROOT / "research" / "audit_bridge"
REQUESTS = BRIDGE / "requests"

@dataclass
class CommandRun:
    name: str
    command: list[str]
    exit_code: int
    stdout: str = ""
    stderr: str = ""
    blocking: bool = True

def main() -> int:

```

```

args = parse_args()
audit_id = args.audit_id or build_audit_id(args.cadence)
package = REQUESTS / audit_id
package.mkdir(parents=True, exist_ok=True)
api_url = args.api_url or os.environ.get("TRADEO_API_URL") or default_api_url()
created_at = datetime.now(timezone.utc).isoformat(timespec="seconds")

runs: List[CommandRun] = []
gate_result: dict[str, Any] = {}
agent_review: dict[str, Any] = {}
runtime_status: dict[str, Any] = {"available": False, "reason": "not_collected"}
runner_error: dict[str, str] | None = None
exit_code = 1

try:
    runtime_run, runtime_status = collect_compose_status()
    runs.append(runtime_run)

    if not args.skip_export:
        runs.append(run_exporter(audit_id, api_url, args.pattern_limit, args.match_limit))

    gate_json = package / "director_gate_result.json"
    gate_md = package / "director_gate_result.md"
    runs.append(
        run_subprocess(
            "director_gate",
            [
                sys.executable,
                str(BRIDGE / "director_gate.py"),
                str(package),
                "--json-output",
                str(gate_json),
                "--markdown-output",
                str(gate_md),
                "--allow-blocked-exit-zero",
            ],
        )
    )
    runs.append(run_subprocess("validate", [sys.executable, str(BRIDGE / "validate_audit_package.py"), str(package)]))

    gate_result = read_json(gate_json)
    agent_review = deterministic_review(audit_id, args.cadence, gate_result, runs)

    export_failed = any(run.name == "export" and run.exit_code != 0 for run in runs)
    validation_failed = any(run.name == "validate" and run.exit_code != 0 for run in runs)
    blocked = gate_result.get("status") == "blocked" if isinstance(gate_result, dict) else False
    if export_failed or validation_failed:
        exit_code = 1
    elif blocked and args.fail_on_blocked:
        exit_code = 2
    else:
        exit_code = 0
except Exception as exc: # noqa: BLE001
    runner_error = {"type": type(exc).__name__, "message": str(exc)}
    runs.append(CommandRun("runner_error", [], 1, stderr=f"{type(exc).__name__}: {exc}"))
    gate_result = {"status": "runner_error", "blockers": [f"runner_error: {type(exc).__name__}: {exc}"]}
    agent_review = deterministic_review(audit_id, args.cadence, gate_result, runs)
    exit_code = 1

result = {
    "audit_id": audit_id,
    "cadence": args.cadence,
    "created_at": created_at,
    "package": display_path(package),
    "api_url": api_url,
    "director_gate_status": gate_result.get("status", "unknown") if isinstance(gate_result, dict) else "unknown",
    "runtime_status": runtime_status,
    "commands": [asdict(run) for run in runs],
    "agent_review": agent_review,
    "artifact_paths": {
        "run_json": display_path(package / "director_audit_run.json"),
        "run_markdown": display_path(package / "director_audit_run.md"),
        "agent_review_json": display_path(package / "internal_auditor_agent_review.json"),
        "agent_review_markdown": display_path(package / "internal_auditor_agent_review.md"),
    },
}

if runner_error is not None:
    result["runner_error"] = runner_error

try:
    write_json(package / "internal_auditor_agent_review.json", agent_review)
    write_agent_md(package / "internal_auditor_agent_review.md", agent_review)
    write_json(package / "director_audit_run.json", result)
    write_md(package / "director_audit_run.md", result)
except Exception as exc: # noqa: BLE001
    print(f"failed to write audit artifacts in {package}: {type(exc).__name__}: {exc}", file=sys.stderr)
    return 1

return exit_code

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run the automated Tradeo Director audit loop.")
    parser.add_argument("--audit-id", default=None)
    parser.add_argument("--cadence", choices=["daily", "weekly", "manual"], default="daily")
    parser.add_argument("--api-url", default=None)

```

```

parser.add_argument("--pattern-limit", type=int, default=500)
parser.add_argument("--match-limit", type=int, default=500)
parser.add_argument("--skip-export", action="store_true")
parser.add_argument("--fail-on-blocked", action="store_true")
return parser.parse_args()

def build_audit_id(cadence: str) -> str:
    now = datetime.now(timezone.utc)
    return f"TRADEO-AUDIT-{now:%Y%m%d-%H%M%S}_{cadence}_internal"

def default_api_url() -> str:
    if Path("/.dockerenv").exists() or Path("/app").exists():
        return "http://backend:8000/api"
    return "http://localhost:8000/api"

def collect_compose_status() -> tuple[CommandRun, dict[str, Any]]:
    command = compose_ps_command(json_format=True)
    run = run_subprocess("docker_compose_ps", command, blocking=False)
    if run.exit_code == 0:
        services = parse_compose_ps_json(run.stdout)
        if not services and run.stdout.strip():
            fallback = run_subprocess("docker_compose_ps", compose_ps_command(json_format=False), blocking=False)
            return fallback, {
                "available": fallback.exit_code == 0,
                "source": "docker compose ps",
                "service_count": None,
                "services": [],
                "reason": "json_output_unparseable",
                "json_stdout": run.stdout,
                "json_stderr": run.stderr,
            }
        return run, {
            "available": True,
            "source": "docker compose ps --format json",
            "service_count": len(services),
            "services": services,
        }
    fallback = run_subprocess("docker_compose_ps", compose_ps_command(json_format=False), blocking=False)
    return fallback, {
        "available": fallback.exit_code == 0,
        "source": "docker compose ps",
        "service_count": None,
        "services": [],
        "reason": "json_format_failed" if fallback.exit_code == 0 else "docker_compose_ps_unavailable",
        "json_stderr": run.stderr,
    }

def compose_ps_command(*, json_format: bool) -> list[str]:
    command = ["docker", "compose"]
    files = [value for value in os.environ.get("TRADEO_AUDIT_COMPOSE_FILES", "").split(os.pathsep) if value]
    for file_name in files:
        command.extend(["-f", file_name])
    command.append("ps")
    if json_format:
        command.extend(["--format", "json"])
    return command

def parse_compose_ps_json(stdout: str) -> list[dict[str, Any]]:
    text = stdout.strip()
    if not text:
        return []
    try:
        raw = json.loads(text)
        rows = raw if isinstance(raw, list) else [raw]
    except json.JSONDecodeError:
        rows = []
        for line in text.splitlines():
            line = line.strip()
            if not line:
                continue
            try:
                rows.append(json.loads(line))
            except json.JSONDecodeError:
                return []

    services: list[dict[str, Any]] = []
    for row in rows:
        if not isinstance(row, dict):
            continue
        name = row.get("Name") or row.get("name") or ""
        service = row.get("Service") or row.get("service") or name
        state = row.get("State") or row.get("state") or ""
        health = row.get("Health") or row.get("health") or health_from_status(row.get("Status") or row.get("status"))
        services.append(
            {
                "service": service,
                "name": name,
                "state": state,
                "health": health,
                "exit_code": row.get("ExitCode") or row.get("exit_code"),
            }
        )

```



```

    }
)
return services

def health_from_status(status: object) -> str:
    text = str(status or "").lower()
    if "unhealthy" in text:
        return "unhealthy"
    if "(healthy)" in text or "healthy" in text:
        return "healthy"
    return ""

def run_exporter(audit_id: str, api_url: str, pattern_limit: int, match_limit: int) -> CommandRun:
    exporter_path = BRIDGE / "export_audit_package.py"
    command = [
        sys.executable,
        str(exporter_path),
        "--audit-id",
        audit_id,
        "--api-url",
        api_url,
        "--pattern-limit",
        str(pattern_limit),
        "--match-limit",
        str(match_limit),
    ]
    stdout = io.StringIO()
    stderr = io.StringIO()
    exit_code = 1
    old_argv = sys.argv[:]
    try:
        spec = importlib.util.spec_from_file_location("tradeo_export_audit_package", exporter_path)
        if not spec or not spec.loader:
            raise RuntimeError(f"cannot import exporter from {exporter_path}")
        exporter = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(exporter)
        original_read_env = exporter.read_env

        def read_env_with_process_env(path: Path) -> dict[str, str]:
            env = original_read_env(path)
            for key, value in os.environ.items():
                if key.startswith("TRADEO_"):
                    env[key] = value
            env["TRADEO_API_URL"] = api_url
            return env

        exporter.read_env = read_env_with_process_env
        sys.argv = command[1:]
        with contextlib.redirect_stdout(stdout), contextlib.redirect_stderr(stderr):
            try:
                exit_code = int(exporter.main())
            except SystemExit as exc:
                exit_code = exc.code if isinstance(exc.code, int) else 1
    except Exception as exc: # noqa: BLE001
        stderr.write(f"{type(exc).__name__}: {exc}\n")
        exit_code = 1
    finally:
        sys.argv = old_argv
    return CommandRun("export", command, exit_code, stdout.getvalue(), stderr.getvalue())

def run_subprocess(name: str, command: list[str], *, blocking: bool = True) -> CommandRun:
    try:
        completed = subprocess.run(command, cwd=ROOT, capture_output=True, text=True, check=False) # noqa: S603
    except FileNotFoundError as exc:
        return CommandRun(name, command, 127, stderr=f"{type(exc).__name__}: {exc}", blocking=blocking)
    return CommandRun(name, command, completed.returncode, completed.stdout, completed.stderr, blocking)

def deterministic_review(
    audit_id: str,
    cadence: str,
    gate_result: dict[str, Any],
    command_runs: list[CommandRun],
) -> dict[str, Any]:
    blockers = gate_result.get("blockers", []) if isinstance(gate_result, dict) else []
    failed_commands = [run.name for run in command_runs if run.blocking and run.exit_code != 0]
    status = gate_result.get("status", "unknown") if isinstance(gate_result, dict) else "unknown"
    priority = "P0" if failed_commands or blockers else "P2"
    return {
        "audit_id": audit_id,
        "cadence": cadence,
        "agent": "tradeo-internal-daily-auditor",
        "model_profile": "gpt-5.5-xhigh-specified-in-skill; deterministic fallback used by runner",
        "status": status,
        "priority": priority,
        "failed_commands": failed_commands,
        "blocker_count": len(blockers),
        "top_blockers": blockers[:10],
        "director_handoff": "ChatGPT Director must review weekly packs or any P0 blocker before promotion.",
        "promotion_decision": "stay_in_research" if blockers else "eligible_for_director_review",
        "required_next_actions": required_actions(blockers, failed_commands),
    }
}

```

```

def required_actions(blockers: list[str], failed_commands: list[str]) -> list[str]:
    actions: list[str] = []
    if failed_commands:
        actions.append("Fix failed audit commands before trusting the package.")
    if any("paper_trades.csv has zero rows" in blocker for blocker in blockers):
        actions.append("Keep all patterns in research/watchlist until paper trades are exported.")
    if any("ib_fills.csv has zero rows" in blocker for blocker in blockers):
        actions.append("Ingest IB Paper fills with commissions, spread and slippage before promotion.")
    if any("out_of_sample" in blocker for blocker in blockers):
        actions.append("Add explicit OOS/walk-forward boundaries and metrics.")
    if any("market_regime" in blocker or "sector" in blocker for blocker in blockers):
        actions.append("Persist market regime and sector, then regenerate metrics_by_regime.csv.")
    if not actions:
        actions.append("Prepare the package for ChatGPT Director review; do not auto-promote.")
    return actions

def read_json(path: Path) -> dict[str, Any]:
    if not path.exists():
        return {}
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return {}

def display_path(path: Path) -> str:
    try:
        return str(path.relative_to(ROOT))
    except ValueError:
        return str(path)

def write_json(path: Path, payload: dict[str, Any]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")

def write_md(path: Path, result: dict[str, Any]) -> None:
    commands = result.get("commands", [])
    lines = [
        "# Director Audit Run",
        "",
        f"- Audit ID: `{result['audit_id']}`",
        f"- Cadence: `{result['cadence']}`",
        f"- Created at: `{result['created_at']}`",
        f"- Package: `{result['package']}`",
        f"- Director gate status: `{result['director_gate_status']}`",
        f"- Runtime status: `{result.get('runtime_status', {}).get('source', 'unknown')}`",
        "",
        "## Commands",
        "",
        "| name | exit_code | blocking |",
        "| --- | --- | --- |",
    ]
    lines.extend(f"| {run['name']} | {run['exit_code']} | {run.get('blocking', True)} |" for run in commands)
    lines.extend(["", "## Agent review", "", "```json", json.dumps(result.get("agent_review", {}), indent=2, ensure_ascii=False), "```",
    ""])
    path.write_text("\n".join(lines), encoding="utf-8")

def write_agent_md(path: Path, review: dict[str, Any]) -> None:
    lines = [
        "# Internal Auditor Agent Review",
        "",
        f"- Audit ID: `{review.get('audit_id')}`",
        f"- Status: `{review.get('status')}`",
        f"- Priority: `{review.get('priority')}`",
        f"- Promotion decision: `{review.get('promotion_decision')}`",
        "",
        "## Top blockers",
        "",
    ]
    blockers = review.get("top_blockers") or []
    if blockers:
        lines.extend(f"- {blocker}" for blocker in blockers)
    else:
        lines.append("- None")
    lines.extend(["", "## Required next actions", ""])
    actions = review.get("required_next_actions") or []
    if actions:
        lines.extend(f"- {action}" for action in actions)
    else:
        lines.append("- None")
    path.write_text("\n".join(lines), encoding="utf-8")

if __name__ == "__main__":
    sys.exit(main())

```

Full Document: ``research/audit_bridge/agents/internal_daily_auditor/SKILL.md``

``research/audit_bridge/agents/internal_daily_auditor/SKILL.md`` ? 402 lines, 11415 bytes, sha256 prefix ``fb0bb3af26c76221``

Heading summary: SKILL.md ? Tradeo Internal Daily Auditor; 1. Mission; 2. Model profile; 3. Daily cadence; 4. Weekly cadence; 5. Director hardening rules introduced on 2026-06-08; 5.1. Discovery-stage promotion ceiling; 5.2. Director Gate is the promotion authority; 5.3. Research R metrics are not operational PnL; 5.4. Anti-lookahead event ledger columns; 5.5. Same-bar close entry rule; 5.6. Train-only fit evidence

```
---
name: tradeo-internal-daily-auditor
version: 1.1.1
last_updated: 2026-06-09
owner_role: Internal Audit Agent
model_profile: gpt-5.5-pro / reasoning effort xhigh
reports_to: ChatGPT Director
---
```

SKILL.md ? Tradeo Internal Daily Auditor

1. Mission

The Internal Daily Auditor is the first-line audit agent for Tradeo. Its job is to inspect every newly exported research package, run deterministic gates, summarize blockers, and prepare clean handoff material for the ChatGPT Director.

It does not approve patterns. It does not change live trading settings. It does not submit orders. It exists to reduce noise before Director review and to catch methodological failures before any pattern reaches paper/live promotion language.

The daily auditor must be stricter than the Researcher. A package can be schema-valid and still blocked for promotion.

2. Model profile

Preferred profile:

model: gpt-5.5-pro
reasoning.effort: xhigh
visible output: concise, structured, audit-ready

`xhigh` is the internal equivalent of ?extra high? reasoning. Use it only for daily/weekly audit summaries, security-sensitive review, code review, and high-value research judgment. Do not use it on raw window-by-window discovery data.

3. Daily cadence

Every daily run must:

1. Export the latest audit package.
2. Run `validate\_audit\_package.py`.
3. Run `director\_gate.py`.
4. Write:
 - `director\_gate\_result.json`
 - `director\_gate\_result.md`
 - `internal\_auditor\_agent\_review.json`
 - `internal\_auditor\_agent\_review.md`
 - `director\_audit\_run.json`
 - `director\_audit\_run.md`
5. Mark every blocker as P0/P1/P2/P3.
6. Keep all patterns in research if paper trades, fills, OOS, cost, anti-lookahead or train-only-fit evidence is missing.
7. Never treat schema validation as promotion approval.
8. Escalate any P0 blocker to the ChatGPT Director packet.

Use `python research/audit\_bridge/run\_director\_audit.py --cadence daily` as the preferred local runner. It creates the package directory before checks and writes `director\_audit\_run.\*` plus `internal\_auditor\_agent\_review.\*` even when export, gate or validation fails. Runtime Docker status is informational: use `docker compose ps --format json` or the runner snapshot, never `docker inspect` against fixed container names.

4. Weekly cadence

Every weekly run must prepare a Director packet:

- ? packages reviewed this week
- ? repeated blockers
- ? new blockers
- ? candidate patterns worth deeper Director review
- ? patterns that should be frozen or archived
- ? recommended math/model/code changes
- ? required Claw/Codex tasks

The weekly packet is what ChatGPT Director uses to decide larger mathematical or code changes.

5. Director hardening rules introduced on 2026-06-08

The application now enforces stricter promotion hygiene. The daily auditor must apply and report these rules explicitly.

5.1. Discovery-stage promotion ceiling

The discovery `ValidationGate` must not promote a pattern to:

premium_candidate
paper_candidate
paper_limited_candidate
paper_extended_candidate
approved
live_candidate
production_candidate

from research-only R metrics.

Without auditable paper trades, fills, costs and clean out-of-sample evidence, the maximum acceptable discovery-stage states are:

lab
lab_watchlist
lab_candidate
rejected

If a discovery package contains `premium\_candidate`, `paper\_candidate` or anything stronger while `trade\_count = 0` or `fills = 0`, classify the package as `blocked` with priority `P0`.

5.2. Director Gate is the promotion authority

`director\_gate.py` is the deterministic promotion authority for audit packages.

The daily auditor must run it and preserve both:

schema_validation_status
promotion_gate_status

A package can be:

schema_validation_status = passed
promotion_gate_status = blocked

That is a valid outcome and must not be softened.

5.3. Research R metrics are not operational PnL

If `trade\_count = 0`, the following columns must not be interpreted as operational trading results:

winrate
avg_win
avg_loss
payoff_ratio
profit_factor
expectancy
max_drawdown
median_trade

These are research/simulation metrics unless backed by paper trades and fills.

The daily auditor must flag a P0 blocker when research R metrics populate operational-looking columns while `trade\_count = 0`, especially if any status implies paper readiness.

Recommended wording:

BLOCK_RESEARCH_METRICS_AS_OPERATIONAL_PNL:

Research R metrics are populated in operational metric columns while trade_count is zero. These values cannot support promotion.

5.4. Anti-lookahead event ledger columns

Every promotable package must include event-level evidence fields:

available_data_cutoff_ts
decision_ts
entry_eligible_ts
label_generated_ts
source_bar_hash
split_id

If any are missing, block promotion.

The daily auditor must verify:

available_data_cutoff_ts <= decision_ts <= entry_eligible_ts
label_generated_ts >= decision_ts
source_bar_hash is present
split_id is present

If the package does not provide row-level timestamps, mark the check as `not\_verifiable` and block promotion.

5.5. Same-bar close entry rule

If `entry\_rule\_plaintext` says the research entry uses the close of the final bar or latest close, promotion is blocked unless the package proves the real execution time through `entry\_eligible\_ts`.

Required rule:

If signal uses close[t], execution cannot be assumed at close[t] unless there is an explicit MOC/auction model. Otherwise, earliest eligible execution is next tradable bar or later.

P0 blocker:

BLOCK_SAME_BAR_CLOSE_ENTRY_UNPROVEN:

Signal uses the same close as the entry reference but the package lacks entry_eligible_ts or a realistic execution model.

5.6. Train-only fit evidence

The daily auditor must block promotion if the package cannot prove that scaler, clustering, side selection, R:R selection and scoring were fit/selected without using OOS/test data.

Acceptable evidence fields include one or more of:

fit_scope
fit_on_train_only
split_protocol
purged_embargo_applied
selection_split

The daily auditor must treat any OOS metric as weak if:

StandardScaler.fit
MiniBatchKMeans.fit
side selection
best_rr selection
candidate scoring

were performed before the train/validation/test split.

P0 blocker:

BLOCK_OOS_CONTAMINATION_RISK:

No train-only fit evidence fields found; cannot prove scaler/clustering/R:R selection avoided OOS contamination.

5.7. Multiple-testing threshold

If the package has 20 or more experiment variants, it must provide at least one multiple-testing adjustment field, such as:

multiple_testing_adjusted_p_value
adjusted_p_value
deflated_sharpe
permutation_p_value

Without this, the package may remain valid for research logging, but promotion must be blocked.

5.8. Regime and sector persistence

Promotion requires non-empty, non-placeholder regime and sector fields.

Blocked placeholder values:

not_persisted
unknown

none

If all rows have placeholder regime/sector, mark:

BLOCK_REGIME_OR_SECTOR_NOT_PERSISTED

5.9. OHLCV data quality

The application now validates OHLCV invariants before research use. The daily auditor must treat these as hard data-quality expectations:

-
- ? no duplicate timestamps
 - ? timestamps sorted ascending
 - ? finite open/high/low/close/volume
 - ? open/high/low/close strictly positive
 - ? volume non-negative
 - ? high >= low
 - ? high >= open and close
 - ? low <= open and close
-

Any OHLCV validation failure is at least P1. If it affects exported research metrics or candidate patterns, it becomes P0.

6. Hard rules

The agent must block or warn if any of these is true:

-
- ? pattern_catalog.csv has zero rows
 - ? pattern_events.csv has zero rows
 - ? paper_trades.csv has zero rows
 - ? ib_fills.csv has zero rows
 - ? no explicit OOS/walk-forward boundaries
 - ? no train-only fit evidence
 - ? market_regime or sector is not persisted
 - ? duplicate_group_id repeats without explanation
 - ? sample_count is not reconstructable from exported events
 - ? many variants were tested without multiple-testing adjustment
 - ? research R metrics are being treated as realized net PnL
 - ? any pattern status implies paper/live readiness without execution evidence
 - ? anti-lookahead event fields are missing
 - ? same-bar close entry lacks entry_eligible_ts or explicit execution model
 - ? OHLCV validation failed upstream
-

7. Promotion policy

Allowed states without paper/fill evidence:

lab
lab_watchlist
lab_candidate
rejected
DISCOVERED
UNDER_RESEARCH
WATCHLIST
RESEARCH_CANDIDATE_PENDING_DIRECTOR

Blocked states without paper/fill evidence:

paper_candidate
premium_candidate
paper_limited_candidate
paper_extended_candidate
ready_for_paper_extended
approved
live_candidate
production_candidate

Minimum promotion evidence before any execution-stage state:

-
- ? sufficient paper_trades.csv rows
-

- ? sufficient ib_fills.csv rows
- ? net_pnl reconstructed from gross_pnl - commission - spread - slippage - fees
- ? bid/ask/spread/slippage evidence or conservative stress model
- ? clean OOS/walk-forward evidence
- ? train-only fit/selection proof
- ? anti-lookahead timestamps
- ? regime/sector persistence
- ? multiple-testing adjustment where applicable
- ? sample_count reconstructed from event ledger

8. Output schema

The JSON review must contain:

```
{
  "audit_id": "",
  "cadence": "daily|weekly|manual",
  "agent": "tradeo-internal-daily-auditor",
  "status": "passed|blocked|invalid|unknown",
  "schema_validation_status": "passed|failed|not_run",
  "promotion_gate_status": "passed|blocked|invalid|not_run",
  "priority": "P0|P1|P2|P3",
  "blocker_count": 0,
  "top_blockers": [],
  "promotion_decision": "stay_in_research|eligible_for_director_review",
  "required_next_actions": [],
  "director_handoff": ""
}
```

The Markdown review must include:

-
- ? Executive status
 - ? Schema validation result
 - ? Director gate result
 - ? P0 blockers
 - ? Repeated blockers
 - ? New blockers
 - ? Whether any pattern is incorrectly promoted
 - ? Whether research metrics are being presented as operational metrics
 - ? Whether OOS is clean or contaminated/not verifiable
 - ? Whether anti-lookahead columns exist
 - ? Whether OHLCV validation is clean
 - ? Required next actions
-

9. Director handoff

The agent must escalate to ChatGPT Director when:

-
- ? priority is P0
 - ? any pattern appears promotable
 - ? a blocker repeats for three consecutive daily audits
 - ? OOS results disagree with in-sample results
 - ? OOS cleanliness is not verifiable
 - ? paper fills appear for the first time
 - ? the detector, universe, risk model, or data provider changed
 - ? the Director Gate behavior changed
 - ? OHLCV validation starts rejecting data used by previous research packages
-

10. Safety

Never include secrets, account IDs, credentials, raw tokens, session cookies, or unredacted broker account data. Never modify live trading configuration. Never authorize execution. Human approval remains mandatory for any real trading path.

11. Final rule

The daily auditor does not decide that a pattern is good. It decides whether a package is clean enough for the Director to spend high-quality judgment on it.

When uncertain, block promotion and escalate with evidence.

Full Document: `research/audit\_bridge/agents/director\_weekly\_auditor/SKILL.md`

`research/audit\_bridge/agents/director\_weekly\_auditor/SKILL.md` ? 426 lines, 23005 bytes, sha256 prefix `b39915a93ffac3bd`

Heading summary: Research Director Weekly Tradeo Audit; Scope; Rule Zero; Required Inputs To Locate; Mandatory Process; 1. Review Internal Daily Auditor Work; 2. Fix Audit Context; 3. Inventory Full Pipeline; 4. Audit Data Quality; 5. Audit Temporal Alignment; 6. Formalize Pattern; 7. Audit Mathematical Metrics

```
---
name: "research-director-weekly-tradeo-audit"
description: "Director semanal audita Tradeo cada domingo 22:00 Europe/Madrid."
---
```

Research Director Weekly Tradeo Audit

Scope

Use this skill for the Director's weekly Tradeo audit, scheduled each Sunday at 22:00 Europe/Madrid, normally launched from web chat/GTP.

Reference repository: `https://github.com/AsierIP/tradeo`.

Role: act as Tradeo's Director General, mathematical auditor, technical auditor, and validation auditor. Be critical, not complacent. The goal is to discover quickly whether the Researcher is wrong, not to prove them right.

Do not modify the Tradeo repository, create commits, push, open PRs, or write inside the repo unless Asier explicitly asks later. Produce an external audit report and actionable recommendations.

If the runtime allows a stronger reasoning mode, use the strongest available mode, preferably Pro Extended or equivalent.

Rule Zero

Do not approve any pattern unless it has passed checks for:

- Data integrity.
- Correct temporal alignment.
- No lookahead bias.
- No direct or indirect leakage.
- Out-of-sample validation.
- Robustness to costs, slippage, and spreads.
- Robustness by market regime.
- Robustness by ticker, date, and trade cluster.
- Reasonable independence from a few extreme trades.
- Reproducibility.
- Mathematical metric coherence.
- Sufficient code quality in the generation pipeline.

If evidence is missing, write `NO VERIFICADO` and state exactly what evidence is missing. Do not invent.

Required Inputs To Locate

Try to find and review:

- Current Tradeo repo state: branch, commit hash, last change date, recent modified files.
- Researcher results since the previous audit.
- Experiments, backtests, metrics, logs, notebooks, CSV, JSON, databases, reports, and generated artifacts.
- Experiment configs, ticker universe, date range, data source, timezone, costs, spreads, slippage, commissions.
- Exact entry, exit, stop, take-profit, sizing, and risk rules.
- Features, labels/targets, train/validation/test/out-of-sample splits.
- Variants tried and discarded.
- Previous audits, if available.

If an artifact is unavailable, record it clearly.

Mandatory Process

Follow these steps in order.

1. Review Internal Daily Auditor Work

Before starting the Director audit, review the weekly packet and recent work produced by `research/audit\_bridge/agents/internal\_daily\_auditor/SKILL.md`: daily audit outputs, repeated blockers, new blockers, candidate patterns escalated for Director review, patterns recommended for freeze/archive, and proposed math/model/code changes. Treat this as the first evidence layer, not as validation; independently verify every claim before accepting it.

2. Fix Audit Context

Record exact audit date, time, timezone, repo, branch, commit, recent changes, available result artifacts, analyzed data period, and whether a comparable prior audit exists.

If branch, commit, or artifacts cannot be identified, the maximum overall status is `APROBADO CON RESERVAS`.

3. Inventory Full Pipeline

Reconstruct the path from raw data to final pattern:

- Ingestion.
- Cleaning.
- Normalization.
- Feature engineering.
- Candidate generation.
- Labeling.
- Training or rule search.
- Backtesting.
- Pattern ranking.
- Export.

- Final report.

For each stage identify input, output, responsible code, data format, existing validations, missing validations, and silent-failure risks.

4. Audit Data Quality

Check ordered timestamps, unexpected duplicates, explicit timezone, correct market calendar, explained temporal gaps, OHLCV consistency, justified extreme returns, split/dividend handling, delistings, survivorship bias, adjusted vs unadjusted data, signal/execution frequency consistency, illegitimate forward fill, revised data used as realtime data, and future events joined into past prices.

Recommended tests: row counts by ticker/date, NaN percentage, gap distribution, primary-key duplicates, date range per ticker, return outliers, schema/types, join cardinality.

Data faults that can alter a signal are at least `ALTA`; if leakage is possible, `CRÍTICA`.

5. Audit Temporal Alignment

For each feature ask when it is known, whether it is available before entry, whether it uses future prices, whether rolling/expanding/ranking accidentally includes execution or future bars, whether `shift` is correct, whether labels are temporally separated, whether global transforms were calculated before split, and whether split precedes contamination-prone transforms.

Search especially for close-to-close same-bar execution, current-bar indicators used for same-close entry, future max/min features, global normalization, full-period feature selection, future-performance universe filters, removal of dead assets, wrong event publication time, label-based sample filtering, and date joins without publication time.

Any reasonable suspicion of lookahead or leakage puts the pattern in quarantine or rejection.

6. Formalize Pattern

Express each pattern with:

- Asset universe `U`.
- Decision time `t`.
- Available information vector `I\_t`.
- Signal `S\_t` computed only from `I\_t`.
- Entry rule `E(S\_t)`.
- Theoretical and realistic entry price.
- Exit rule and horizon `H`.
- Cost `C`.
- Gross return `R`.
- Net return $R_{net} = R - C$.
- Expected `R\_net` distribution.
- Capital and liquidity constraints.

If a pattern cannot be formalized precisely, do not approve it.

Also evaluate whether it has a plausible hypothesis: behavioral inefficiency, microstructure, momentum, reversal, seasonality, volatility, liquidity, event, regime, or compensated risk. Patterns without hypotheses need stricter statistical validation.

7. Audit Mathematical Metrics

For each pattern calculate or require:

- Trade count, active days, ticker count.
- Gross/net EV per trade and annualized net EV if applicable.
- Profit factor, win rate, average win/loss, payoff ratio.
- Max drawdown and drawdown duration.
- Sharpe, Sortino, Calmar when meaningful.
- Exposure time, turnover, average cost, estimated slippage, approximate capacity.
- Return distribution, skewness, kurtosis, worst trade, best trade.
- PnL weight of top 1%, 5%, and 10% trades.
- PnL by month, year, and regime.

Treat these as suspicious: high profit factor with few trades, EV only positive before costs, EV driven by one or two trades, high Sharpe with asymmetric distribution, high win rate with severe left tail, single ticker/period concentration, collapse under small slippage, non-executable prices, missing drawdown, unknown number of variants.

8. Audit Costs, Slippage, And Execution

Every pattern must be assessed net of realistic friction: commission, spread, slippage, market impact, latency, liquidity, tradable volume, sizing, short restrictions, borrow fees, overnight fees, gaps, realistic stop/take-profit execution, same-bar stop/take ambiguity, and bar-based vs live execution.

Stress tests: base cost, cost x2, cost x3, adverse slippage, worst plausible fill, entry one bar later, exit one bar later, and reduced liquidity. If it only works with optimistic costs, classify it `C` or `D`.

9. Audit Backtest Realism

Look for same-close signal and entry, impossible same-bar exits, unknown intrabar order, optimistic stop/take handling, costless rebalances, infinite capital, unrealistic capital reuse, missing exposure limits, missing per-ticker/sector/correlation limits, missing liquidity/gap/suspension/delisting controls, and mismatch between signal frequency and execution frequency.

Mandatory questions: does the signal exist before execution, is entry reachable, is exit reachable, is capital realistic, is sizing known before outcome, are impossible trades allowed, are overlapping trades handled correctly, are open trades at the end accounted for, and does the equity curve reflect cash, exposure, and simultaneous operations.

10. Audit Out-Of-Sample And Anti-Overfitting

Verify strict temporal separation, train before validation before test, test not used for design, clean OOS, walk-forward if optimized, purging for overlapping labels, embargo when needed, fixed parameters before test, total experiments/variants, discarded variants, baselines, label permutation, shifted signals, random entries with same exposure, buy-and-hold when relevant, and no-trade baseline.

Lower confidence automatically when the number of variants tried is unknown.

11. Audit Robustness

Recommended perturbations: small threshold changes, indicator window changes, start/end changes, excluding best/worst ticker, best month/year, top 1% and 5% trades, doubling/tripling costs, delayed execution, adverse slippage, segmentation by volatility/trend/liquidity/sector/company size/session.

A robust pattern need not be excellent in every scenario, but it must not collapse under small reasonable changes.

12. Audit Data Interrelationships

For each join check join key, expected vs actual cardinality, duplicates before/after, row loss, artificial row creation, temporal order, merge direction, tolerance, timezone, whether the joined datum was known at that time, forward/backward fill legitimacy, mixed frequencies/sessions/tickers, recycled symbols, and wrong event dates.

Danger cases: date-only joins, natural calendar instead of market calendar, fiscal date instead of publication date, backward fill from future data, global universe rankings with future data, post-period ticker aggregations, ex-post liquidity filters, and global normalization before split.

13. Audit Code

Review for long functions, ambiguous names, duplicated logic, hardcodes, magic constants, missing tests/asserts/schema validation/timezone controls/sorting/input-output validation, dangerous implicit indexes, accidental row order dependency, uncontrolled in-place mutation, global state, absent seeds, nondeterminism, bad NaN/inf handling, divide-by-zero, risky rounding/type coercion/float precision, vectorized index misalignment, wrong `shift`/`rolling`/`expanding`/`groupby`, accidental ticker mixing, shallow tests, poor logging, and swallowed exceptions.

Require tests for features, labels, train/test split, simple trade backtest, gap backtest, same-bar stop/take, costs, slippage, sizing, temporal joins, duplicates, unsorted data, NaN, sparse tickers, delisted or terminal-missing tickers, and fixed-seed reproducibility.

14. Audit Researcher Process

Ask whether there was an initial hypothesis, which variants were tried, which failed, whether failures were logged, whether success criteria changed, whether bad results were dropped, whether thresholds were tuned after test, how many candidates were generated/reported, whether experiment-config-result traceability exists, whether the winner used a predefined metric, whether cherry-picking risk exists, whether baseline comparison exists, and whether costs/assumptions/limitations were documented.

A good result with opaque process is at most `B` or `C`, never `A`.

15. Apply Blocking Criteria

Quarantine or reject any pattern with confirmed or likely lookahead/leakage, negative realistic net EV, non-executable prices, dependence on few extreme trades, collapse under moderately higher costs, unjustified concentration in ticker/period, no clean OOS, unknown variant count, missing config traceability, non-reproducible data, duplications that inflate results, suspicious temporal joins, feature/label mixing, split after contaminating transforms, no evidence signal precedes execution, no commissions/spread/slippage, unrealistic liquidity, or aggregate metrics that do not reconcile with trades.

16. Candidate-A Approval Criteria

Only classify a pattern as `A` if it reasonably has: clear hypothesis, traceable data, demonstrated no leakage/lookahead, realistic backtest, costs included, positive net EV, tolerable drawdown, sufficient sample, independence from few trades, OOS survival, no collapse under doubled costs or small parameter changes, reasonable behavior by ticker/period/regime, reproducible config, sufficiently clear code, minimum tests present or clearly defined, and known documented risks.

Even `A` means strong candidate for the next validation phase, not production-ready.

17. Severity Scale

Use:

- `CRÍTICA`: can invalidate results: leakage, lookahead, impossible backtest, future data, wrong net EV.
- `ALTA`: can materially change conclusion: incomplete costs, insufficient sample, extreme concentration, doubtful joins, weak OOS.
- `MEDIA`: relevant weakness reducing confidence: missing tests, insufficient logging, incomplete segmentation.
- `BAJA`: technical/documentation improvement.
- `OBSERVACIÓN`: useful comment without immediate impact.

18. General Verdict Matrix

- `APROBADO`: no critical/high findings, sufficient robustness, reproducible, net of costs.
- `APROBADO CON RESERVAS`: no critical findings, but medium doubts or evidence limitations.
- `EN CUARENTENA`: serious suspicions, missing key evidence, or high risk of methodological artifact.
- `RECHAZADO`: leakage, lookahead, invalid backtest, negative net EV, grave reproducibility failure, or extreme fragility.

19. Special Mathematical Checks

When enough data exists, evaluate bootstrap of trades, block bootstrap, EV confidence interval, probability net EV ≤ 0 , cost sensitivity, outlier sensitivity of profit factor, parameter stability, in-sample vs OOS degradation, negative streak distribution, approximate ruin risk, simultaneous-trade correlation, temporal loss/gain clustering, autocorrelation, gross/net exposure, tail risk, worst historical scenario, removing best trades, delayed execution, and equivalent random signals.

20. Machine Learning Checks

If ML is used, also audit temporal split before preprocessing, realtime feature availability, shifted target, leakage in scaling/imputation/feature selection/dimensionality reduction/class balancing, purging, embargo, financial metrics beyond predictive metrics, calibration, feature-importance stability, feature/label/performance drift, regime robustness, simple-model baseline, trivial-rule baseline, and minimum interpretability.

Never accept accuracy, precision, recall, or F1 as sufficient proof of profitability. The decisive metric is financial, net of costs, and out-of-sample.

21. Production Checks

If a pattern approaches production, require realtime data availability, latency, API robustness, provider failures, retries, logs, alerts, duplicate-order control, open-position control, broker reconciliation, circuit breakers, daily loss limits, exposure limits, per-ticker/correlation limits, disconnection handling, gap handling, kill switch, paper trading, and drift monitoring.

Research validation does not imply production readiness.

Mandatory Baselines And Null Tests

When possible, compare every pattern against no-trade, buy-and-hold or relevant benchmark, random entry with same frequency, random entry with same holding period, random entry with same exposure, shifted signal, permuted labels, permuted dates, permuted tickers, inverted pattern when sensible, neighboring parameters, period excluding best segment, and period excluding worst segment.

If the pattern does not clearly beat these baselines, do not classify it as strong.

Reconcile Reported Results

Do not trust aggregate metrics alone. Reconcile individual trades, trade PnL, cumulative PnL, equity curve, drawdown, aggregate metrics, costs, trade count, entry/exit dates, and entry/exit prices.

Trades must explain the equity curve. The equity curve must explain the metrics. If not, quarantine.

Priority Red Flags

Treat as especially suspicious: results too good, abnormal Sharpe, very high profit factor with few trades, almost no drawdown, too-smooth equity curve, metrics without trades, no-cost backtest, large train/test gap, repeated test use, missing logs of failed experiments, rolling features without clear shift, global normalization, ex-post universe filters, unjustified deletion of bad data, close signal and same-close execution, identical signal/trade timestamp without explanation, correlated patterns counted as independent, joins increasing rows unexpectedly, and untracked manual changes.

Recommendation Priority

Prioritize recommendations in this order: eliminate leakage/lookahead, make experiments reproducible, validate data and schemas, improve realistic backtest, include costs/slippage, separate train/validation/test, log failed variants, add null baselines, add automated tests, improve robustness metrics, improve regime segmentation, improve traceability, refactor fragile code, and optimize performance only after correctness.

Each recommendation must be concrete. Avoid vague phrases like `mejorar validación` or `revisar datos`. Write actions such as `Añadir test que verifique que todas las features tienen timestamp anterior al timestamp de entrada`.

Required Final Report Format

Every audit report must use this exact structure:

A. RESUMEN EJECUTIVO

- Fecha de auditoría.
- Estado general: APROBADO / APROBADO CON RESERVAS / EN CUARENTENA / RECHAZADO.
- Principal conclusión.
- Mayor riesgo detectado.
- Mejor oportunidad de mejora.
- Decisión recomendada para el Researcher.

B. CONTEXTO AUDITADO

- Repositorio, rama y commit revisado.
- Periodo de resultados revisado.
- Artefactos revisados.
- Artefactos no encontrados.
- Supuestos usados.
- Limitaciones de la auditoría.

C. MAPA DEL SISTEMA

- Componentes identificados.
- Flujo de datos.
- Flujo de generación de patrones.
- Flujo de backtesting.
- Flujo de validación.
- Dependencias críticas entre módulos.
- Puntos donde puede aparecer error silencioso.

D. VALIDACIÓN DE DATOS

- Integridad.
- Duplicados.
- Huecos temporales.
- Timestamps y timezones.
- Corporate actions.
- Survivorship bias.
- Delistings.
- Ajustes OHLCV.
- Coherencia entre fuentes.
- Calidad de joins.
- Riesgo de datos futuros filtrados al pasado.

E. VALIDACIÓN MATEMÁTICA DEL PATRÓN

- Definición formal del patrón.
- Hipótesis económica o conductual.
- Features usadas.
- Label usado.
- Horizonte temporal.
- Regla de entrada.

- Regla de salida.
- EV bruto.
- EV neto tras costes.
- Profit factor.
- Win rate.
- Payoff ratio.
- Max drawdown.
- Expectancy por trade.
- Distribución de retornos.
- Intervalos de confianza o estimación de incertidumbre.
- Sensibilidad a outliers.
- Dependencia de pocos trades extremos.
- Robustez por régimen.

F. VALIDACIÓN DE BACKTEST

- Realismo de fills.
- Uso correcto de open/high/low/close.
- Ambigüedad intrabar.
- Latencia.
- Spread.
- Slippage.
- Comisiones.
- Liquidez.
- Volumen disponible.
- Position sizing.
- Gestión de capital.
- Reglas de simultaneidad.
- Exposición total.
- Reutilización de capital.
- Riesgo de ejecución imposible.

G. VALIDACIÓN OUT-OF-SAMPLE Y ANTI-OVERFITTING

- Separación temporal train/test.
- Walk-forward.
- Purged cross-validation, si aplica.
- Embargo temporal, si aplica.
- Número de variantes probadas.
- Riesgo de p-hacking.
- Corrección por múltiples comparaciones.
- Comparación contra modelos nulos.
- Degradación in-sample vs out-of-sample.
- Estabilidad de parámetros.
- Robustez ante pequeños cambios de umbral.

H. VALIDACIÓN DE INTERRELACIÓN DE DATOS

- Joins entre datasets.
- Cardinalidad esperada vs real.
- Posibles explosiones one-to-many.
- Alineación temporal entre señales y precios.
- Alineación entre eventos y velas.
- Uso correcto de datos publicados con retraso.
- Riesgo de forward fill indebido.
- Riesgo de join_asof mal direccionado.
- Riesgo de mezclar sesiones, mercados o timezones.
- Riesgo de duplicar muestras correlacionadas.

I. AUDITORÍA DE CÓDIGO

- Claridad.
- Modularidad.
- Determinismo.
- Reproducibilidad.
- Tests existentes.
- Tests faltantes.
- Validaciones de schema.
- Manejo de NaN.
- Manejo de infinitos.
- Manejo de datos vacíos.
- Manejo de errores.
- Logging.
- Hardcodes peligrosos.
- Estado mutable oculto.
- Dependencias frágiles.
- Riesgo de cálculos vectorizados mal alineados.

- Riesgo de índices desordenados.
- Riesgo de mezclar datos ajustados y no ajustados.

J. HALLAZGOS

Para cada hallazgo usa este formato:

- ID:
- Severidad: CRÍTICA / ALTA / MEDIA / BAJA / OBSERVACIÓN
- Categoría: Datos / Matemáticas / Backtest / Código / Validación / Riesgo / Proceso
- Descripción:
- Evidencia:
- Impacto:
- Cómo reproducirlo:
- Recomendación:
- Prioridad:
- Estado sugerido: Bloqueante / Corregir pronto / Monitorizar

K. DECISIÓN SOBRE CADA PATRÓN

Clasifica cada patrón auditado como:

- A: Candidato fuerte para seguir investigando.
- B: Prometedor, pero necesita validación adicional.
- C: En cuarentena por riesgo metodológico.
- D: Rechazado por evidencia insuficiente, leakage, fragilidad o EV neto negativo.

L. PLAN DE ACCIÓN PARA EL RESEARCHER

- Acciones inmediatas.
- Experimentos que deben repetirse.
- Experimentos que deben descartarse.
- Tests que deben añadirse.
- Métricas que deben empezar a registrarse.
- Datos que deben corregirse.
- Refactors recomendados.
- Validaciones matemáticas adicionales.
- Prioridad semanal.

M. MEJORAS PROPUESTAS AL INSTRUCTION

- Qué parte de este instruction funcionó bien.
- Qué parte fue insuficiente.
- Qué nuevas comprobaciones deben añadirse.
- Qué checks deben convertirse en obligatorios.
- Nueva versión propuesta del instruction, si procede.

N. CONCLUSIÓN FINAL

- Veredicto.
- Riesgo principal.
- Próxima acción más importante.
- Nivel de confianza en la auditoría: Alto / Medio / Bajo.
- Qué información faltaría para aumentar la confianza.

Also include a section titled `Mejoras propuestas para estas Instructions` at the end of each audit, covering useful controls, missing controls, new permanent checks, redundancies, hardening opportunities, and proposed wording for future versions.

Final Principle

Prefer quarantining or rejecting a promising pattern over approving a contaminated one. Protect Tradeo from misaligned data, deceptive math, unrealistic backtests, silently wrong code, statistical bias, false correlations, overfitting, leakage, lookahead, and non-reproducible conclusions.

Post-Report Remediation Pull Request

After creating the weekly audit report, create a separate remediation branch, improve the application so the same blockers should not appear in the next weekly audit, and leave a Pull Request ready for human review. The PR must also improve the daily audit flow by updating `research/audit\_bridge/agents/internal\_daily\_auditor/SKILL.md` so new code, checks, and functions are actually used by the daily auditor. The PR must be scoped to audit findings, include tests or validation evidence when possible, and must not modify live trading configuration or authorize execution.

Full Document: `research/audit\_bridge/schedules/PERIODIC\_TASKS.md`

`research/audit\_bridge/schedules/PERIODIC\_TASKS.md` ? 89 lines, 3995 bytes, sha256 prefix `09f7fa74920001df`

Heading summary: Tradeo Periodic Audit Tasks; Purpose; Always-on checks; Daily tasks; Weekly tasks; ChatGPT Director recurring task; Claw/Codex recurring task; Manual emergency audit triggers; Commands; Full automated run; Full automated run with explicit compose files; Weekly Director packet

Tradeo Periodic Audit Tasks

Purpose

This file defines the standing audit cadence for Tradeo. It separates automatic checks, Claw/Codex maintenance, the Internal Daily

Auditor, and ChatGPT Director review.

Always-on checks

```
| Cadence | Owner | Task | Command / Output | Blocking rule |
|---|---|---|---|---|
| Every audit export | Claw/Codex or worker | Validate package schema | `python research/audit_bridge/validate_audit_package.py`
research/audit_bridge/requests/<audit_id>` | Invalid packages block review. |
| Every audit export | Internal Daily Auditor | Run Director gate | `python research/audit_bridge/director_gate.py`
research/audit_bridge/requests/<audit_id>` | Gate BLOCKED means no promotion. |
| Every audit export | Internal Daily Auditor | Write machine-readable result | `director_gate_result.json`, `director_gate_result.md` |
Missing result blocks promotion. |
| Every automated audit | Internal Daily Auditor | Capture Compose runtime snapshot | `docker compose ps --format json` via
`run_director_audit.py` | Informational only; do not block on container names. |
```

Daily tasks

```
| Time UTC | Owner | Task | Expected output |
|---|---|---|---|
| 21:35 | Internal Daily Auditor | Export latest package and run deterministic audit | `director_audit_run.json`,
`internal_auditor_agent_review.md` |
| 21:45 | Claw/Codex | Review P0/P1 blockers from daily run | Task update or fix PR if needed |
| 22:00 | ChatGPT Director | Review daily summary only if P0 or new execution evidence exists | Direction for Claw/Codex |
```

Weekly tasks

```
| Time UTC | Owner | Task | Expected output |
|---|---|---|---|
| Sunday 22:15 | Internal Daily Auditor | Generate weekly Director packet | weekly audit package / summary |
| Sunday after packet | ChatGPT Director | Perform large audit and decide math/code changes | Director response + tasks + PR review |
| Monday 08:00 | Claw/Codex | Implement Director-ordered fixes | PR or status report |
```

ChatGPT Director recurring task

Every Sunday after the weekly audit packet exists:

1. Review the week's packages, daily summaries, Director gate results and blockers.
2. Decide whether the mathematical model, clustering, scoring, OOS split, cost model, or code must change.
3. Create or review PRs for process improvements.
4. Do not approve paper/live promotion unless the gate passes and paper/fill evidence exists.

Claw/Codex recurring task

Every day after the audit run:

1. Check `director_gate_result.json`.
2. Fix P0/P1 blockers first.
3. Do not promote patterns when the package is Director-gate-blocked.
4. Keep discovery running only as research/watchlist.
5. Prepare PRs for export, validator, and research-model improvements.

Manual emergency audit triggers

Run a Director audit immediately if any of these happens:

- ? a pattern appears exceptionally strong
- ? any pattern is proposed for paper_candidate or stronger
- ? paper fills are ingested
- ? detector code changes
- ? universe file changes
- ? IBKR provider or execution logic changes
- ? data errors spike
- ? duplicate samples spike
- ? OOS collapses relative to in-sample

Commands

Full automated run

```
python research/audit_bridge/run_director_audit.py --cadence daily
```

Full automated run with explicit compose files

```
TRADEO_AUDIT_COMPOSE_FILES=docker-compose.yml:docker-compose.audit.yml \
python research/audit_bridge/run_director_audit.py --cadence daily
```

Weekly Director packet

```
python research/audit_bridge/run_director_audit.py --cadence weekly
```

Gate an existing package

python research/audit_bridge/run_director_audit.py --cadence manual --audit-id <audit_id> --skip-export

`run\_director\_audit.py` creates the package directory before running checks and always writes `director\_audit\_run.\*` plus `internal\_auditor\_agent\_review.\*` there when the local filesystem is writable. Do not use `docker inspect` with fixed container names in cron pre-checks; Compose service names are the stable source.

Full Document: `research/audit\_bridge/decisions/DECISIONS.md`

`research/audit\_bridge/decisions/DECISIONS.md` ? 20 lines, 1164 bytes, sha256 prefix `df5f20987d71586b`

Heading summary: Tradeo Director Decisions; 2026-06-07 ? 2026-06-07_ib_paper_patterns; Decision notes

Tradeo Director Decisions

2026-06-07 ? 2026-06-07_ib_paper_patterns

- Verdict: `NEEDS\_PROCESS\_IMPROVEMENT`
- Director score: `30 / 100`
- Confidence: `medium`
- Promotion decision: `repeat\_with\_fixes`
- Pattern approval: none
- Main reason: the package is useful for process audit but has zero paper trades, zero IB fills, no observed costs/slippage/spread, no explicit OOS/walk-forward proof, and no persisted market regime/sector.
- Required next action: run the new Director gate and fix the audit/export process before ranking or promoting any pattern.
- Related files:
 - `research/audit\_bridge/director\_gate.py`
 - `research/audit\_bridge/responses/2026-06-07\_ib\_paper\_patterns\_RESPONSE.md`
 - `research/audit\_bridge/responses/2026-06-07\_ib\_paper\_patterns\_RESPONSE.json`
 - `research/audit\_bridge/task\_queue/pending/2026-06-07\_ib\_paper\_patterns\_director\_gate\_and\_research\_fixes.md`

Decision notes

The package may remain as a historical audit artifact, but it must not be interpreted as pattern approval. Research R-metrics are not net realized PnL. Future packages must distinguish base schema validity from Director promotion eligibility.

Full Document: `research/audit\_bridge/state/process\_improvements.md`

`research/audit\_bridge/state/process\_improvements.md` ? 24 lines, 1240 bytes, sha256 prefix `04476d06834eaf0c`

Heading summary: Tradeo Research Process Improvements; 2026-06-07 ? Director audit gate and promotion hygiene; Required improvements; Non-negotiable rule

Tradeo Research Process Improvements

2026-06-07 ? Director audit gate and promotion hygiene

Status: `ordered`
Related audit: `2026-06-07\_ib\_paper\_patterns`
Priority: `P0`

Required improvements

1. Separate package schema validity from Director promotion eligibility.
2. Add a strict Director gate to block promotion without sufficient paper trades/fills.
3. Separate upstream `sample\_count` from exported event count and verified independent sample count.
4. Export full event ledgers, not only representative examples.
5. Persist timestamp cutoff fields that prove no lookahead: `available\_data\_cutoff\_ts`, `decision\_ts`, `entry\_eligible\_ts`, `label\_generated\_ts`.
6. Add OOS/walk-forward, ticker holdout and period holdout outputs.
7. Add bootstrap, permutation and multiple-testing adjusted evidence.
8. Persist sector/regime and add regime metrics.
9. Add paper trade/fill cost realism: commission, spread, slippage, borrow, order type and fill quality.
10. Keep PR descriptions aligned with actual code and data changes.

Non-negotiable rule

A pattern can be interesting for research with zero trades, but it cannot be approved, extended, or treated as operationally validated without paper trade and fill evidence.

Full Document:

`research/audit\_bridge/task\_queue/pending/2026-06-07\_claw\_merge\_redeploy\_director\_audit\_automation.md`

`research/audit\_bridge/task\_queue/pending/2026-06-07\_claw\_merge\_redeploy\_director\_audit\_automation.md` ? 90 lines, 2908 bytes, sha256 prefix `3c9737d808da2569`

Heading summary: Task: 2026-06-07_claw_merge_redeploy_director_audit_automation; From; To; Priority; Goal; Context; Actions; Acceptance criteria; Response format

Task: 2026-06-07_claw_merge_redeploy_director_audit_automation

From

ChatGPT Director

To

Claw/Codex Researcher

Priority

`P0`

Goal

Resolve conflicts, merge the Director audit automation PR, update the local deployment, and redeploy all services so the new audit loop is available as soon as possible.

Context

This PR adds the automated Director audit runner, machine-readable Director gate outputs, an internal daily auditor skill, standing periodic task definitions, and a Docker Compose audit overlay that mounts `./research` into backend/worker containers.

Actions

1. Fetch the PR branch and check for conflicts:

```
```bash
git fetch origin
git checkout main
git pull --ff-only origin main
git checkout automation/director-audit-loop
git rebase main
```
```

2. Resolve conflicts if any.

3. Run validation:

```
```bash
python research/audit_bridge/run_director_audit.py --cadence manual --audit-id 2026-06-07_ib_paper_patterns --skip-export
python research/audit_bridge/validate_audit_package.py research/audit_bridge/requests/2026-06-07_ib_paper_patterns
python research/audit_bridge/director_gate.py research/audit_bridge/requests/2026-06-07_ib_paper_patterns
```
```

The current package is expected to be Director-gate-blocked, not approved.

4. Merge the PR after checks are acceptable.

5. Update local:

```
```bash
git checkout main
git pull --ff-only origin main
```
```

6. Rebuild and redeploy with the audit overlay:

```
```bash
docker compose -f docker-compose.yml -f docker-compose.audit.yml build --no-cache backend worker frontend
docker compose -f docker-compose.yml -f docker-compose.audit.yml up -d --remove-orphans
docker compose -f docker-compose.yml -f docker-compose.audit.yml ps
```
```

7. Smoke-test services:

```
```bash
curl -fsS http://localhost:8000/api/health
curl -fsS http://localhost:8000/api/health/deep
curl -fsS http://localhost:8000/api/health/ibkr || true
```
```

8. Run one audit manually after redeploy:

```
```bash
docker compose -f docker-compose.yml -f docker-compose.audit.yml exec worker python research/audit_bridge/run_director_audit.py
--cadence daily
```
```

Acceptance criteria

- PR is merged or explicitly blocked with reason.
- Local `main` contains the merged changes.
- Docker services are rebuilt and running with `./research:/app/research:ro` mounted.
- Manual Director audit run produces:
 - `director\_gate\_result.json`
 - `director\_gate\_result.md`
 - `internal\_auditor\_agent\_review.json`
 - `internal\_auditor\_agent\_review.md`
 - `director\_audit\_run.json`
 - `director\_audit\_run.md`
- No live trading configuration is changed.
- No pattern is promoted from a blocked package.

Response format

- Estado: OK / BLOCKED / NEEDS_HUMAN
- Rama:
- PR:
- Commits:
- Conflictos:
- Validaciones ejecutadas:
- Redeploy:
- Health checks:
- Riesgos:
- Siguiente paso recomendado:

Full Document:

`research/audit\_bridge/task\_queue/pending/2026-06-07\_ib\_paper\_patterns\_director\_gate\_and\_research\_fixes.md`

`research/audit\_bridge/task\_queue/pending/2026-06-07\_ib\_paper\_patterns\_director\_gate\_and\_research\_fixes.md` ? 87 lines, 3800 bytes, sha256 prefix `3b3f55d7d6991623`

Heading summary: Task: 2026-06-07_ib_paper_patterns_director_gate_and_research_fixes; From; To; Related audit; Priority; Goal; Context; Required inputs; Actions; Expected outputs; Acceptance criteria; Constraints

Task: 2026-06-07_ib_paper_patterns_director_gate_and_research_fixes

From

ChatGPT Director

To

Claw/Codex Researcher

Related audit

`2026-06-07\_ib\_paper\_patterns`

Priority

`P0`

Goal

Improve the Tradeo research audit process so detected patterns cannot be promoted from discovery metrics alone.

Context

The current package exports 297 patterns, 3,685 representative events and 1,737 RR variants, but zero paper trades and zero IB fills. The Director verdict is `NEEDS\_PROCESS\_IMPROVEMENT`; no pattern is approved. The next work must improve the process before trying to select winners.

Required inputs

- `research/audit\_bridge/director\_gate.py`
- `research/audit\_bridge/validate\_audit\_package.py`
- `research/audit\_bridge/export\_audit\_package.py`
- `research/audit\_bridge/requests/2026-06-07\_ib\_paper\_patterns/`
- Research DB models/endpoints that produce discovered patterns, examples, matches, runs, paper trades and fills
- IBKR paper execution logs/fills if available

Actions

1. Run:
``bash
python research/audit_bridge/director_gate.py research/audit_bridge/requests/2026-06-07_ib_paper_patterns
``
Treat a Director-gate failure as expected for the current package.
2. Update the exporter so `sample\_count`, `exported\_event\_count`, and `verified\_independent\_sample\_count` are separate and auditable.
3. Export a full immutable event ledger rather than representative examples only. Include source row IDs, window hashes, raw data hashes, `available\_data\_cutoff\_ts`, `decision\_ts`, `entry\_eligible\_ts`, and `label\_generated\_ts`.
4. Block or demote statuses such as `paper\_candidate`, `premium\_candidate`, `paper\_limited\_candidate`, `paper\_extended\_candidate`, `approved`, or stronger whenever paper trades/fills are missing or below threshold.
5. Add OOS/walk-forward boundaries and outputs. Include ticker holdout and period holdout.
6. Add bootstrap, permutation test, and multiple-testing correction outputs. Preserve the total number of variants tested.
7. Persist market regime and sector, then add `metrics\_by\_regime.csv`.
8. Add paper trade/fill ingestion to future packages, including commission, spread, slippage, borrow costs for shorts, order type, fill time, and realized vs expected entry/exit.
9. Keep live trading disabled and do not modify production execution gates.
10. Keep PR metadata/scope aligned with actual code and data changes.

Expected outputs

- Updated exporter and/or validators.
- A rerun audit package, or a follow-up package, with explicit Director gate result.
- A report explaining which checks are still blocked and why.
- Tests or fixture package proving the Director gate blocks zero-trade promotion.

Acceptance criteria

- Base package validation can pass while Director promotion eligibility can fail with a distinct message/exit code.
- Current package is blocked by Director gate because it has zero paper trades/fills and missing OOS/regime evidence.
- No pattern with zero paper trades/fills appears as `paper\_candidate`, `premium\_candidate`, `approved`, or stronger in future packages.
- For every promoted pattern, sample counts can be reconstructed from exported events or are explicitly separated as upstream counts with hashes.
- Future ranking uses net-cost evidence, OOS evidence, adjusted multiple-testing evidence, and concentration checks.

Constraints

- No live trading.
- No direct push to `main`.
- No secrets or account IDs in exported audit artifacts.
- Do not delete historical audit packages.
- Do not report gross or research R-metrics as net realized PnL.

Response format

At completion, respond with:

- Estado: OK / BLOCKED / NEEDS_HUMAN
- Rama:
- Commits:
- Ficheros modificados:
- Validaciones ejecutadas:
- Resultados:
- Riesgos:
- Siguiente paso recomendado:

Secondary File Summaries And Risks

These files are not embedded in full unless listed elsewhere. They are summarized so Fable can decide whether to request or inspect them if needed. Risk flags are heuristic.

Secondary Python Public API Inventory

```
path	lines	role / docstring	public classes	public functions	routes	risk heuristics
backend/tradeo/__init__.py	1	No module docstring; infer role from symbols/imports.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/audit_agent.py	14	No module docstring; infer role from symbols/imports.	AuditAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/base.py	19	No module docstring; infer role from symbols/imports.	AgentResult:8, Agent:15	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/data_collector.py	16	No module docstring; infer role from symbols/imports.	DataCollectorAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/improvement_agent.py	14	No module docstring; infer role from symbols/imports.	ImprovementAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/pattern_hunter.py	21	No module docstring; infer role from symbols/imports.	PatternHunterAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/risk_agent.py	14	No module docstring; infer role from symbols/imports.	RiskAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/supervisor_agent.py	19	No module docstring; infer role from symbols/imports.	SupervisorAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/agents/technical_context.py	22	No module docstring; infer role from symbols/imports.	TechnicalContextAgent:9	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/__init__.py	2	Domain modules for Tradeo research, paper validation and production hunting.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/fox_hunter/__init__.py	2	FoxHunter domain: production pattern monitoring and gated live execution.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/laboratory/__init__.py	2	Laboratory domain: paper validation of Research patterns.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/laboratory/scanner.py	47	No module docstring; infer role from symbols/imports.	LaboratoryScanner:14	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/research/__init__.py	7	Research domain facade. The implementation remains in ``tradeo.research`` because it is already a cohesive package. This namespace marks it as one of the three top-level Tradeo domains alongside Laboratory and FoxHunter.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/modules/shared/__init__.py	2	Shared operational primitives used by Tradeo domain modules.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/__init__.py	5	Research laboratory modules for discovering novel market patterns. The research package is deliberately isolated from order execution. It can create LAB candidates and review artifacts, but it cannot route trades to brok	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/active_learning.py	146	No module docstring; infer role from symbols/imports.	ActiveLearningAgenda:10	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/adversarial_research.py	332	No module docstring; infer role from symbols/imports.	AdversarialResearchEngine:14	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/causal_invariance.py	237	No module docstring; infer role from symbols/imports.	CausalInvariantTester:16	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/foundation_teacher.py	262	No module docstring; infer role from symbols/imports.	FoundationChartTeacher:13	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/hypothesis_engine.py	296	No module docstring; infer role from symbols/imports.	HypothesisEngine:12	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/market_replay.py	306	No module docstring; infer role from symbols/imports.	MarketReplayConfig:13, MarketReplayEngine:20	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/research/research_memory_graph.py	320	No module docstring; infer role from symbols/imports.	ResearchMemoryGraph:16	?	?	wall-clock timestamp usage
backend/tradeo/routers/__init__.py	1	No module docstring; infer role from symbols/imports.	?	?	?	no obvious heuristic risk; still review logic/tests
backend/tradeo/routers/dashboard.py	40	No module docstring; infer role from symbols/imports.	?	?	summary:16	summary:
```

@router.get('/summary', response_model=DashboardSummary) | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/routers/fox_hunter.py | 51 | No module docstring; infer role from symbols/imports. | ? | fox_hunter_status:17, fox_hunter_overview:25, scan_fox_hunter:33 | fox_hunter_status: @router.get('/status')
fox_hunter_overview: @router.get('/overview')
scan_fox_hunter: @router.post('/scan', response_model=PatternEntryScanResponse) | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/routers/health.py | 65 | No module docstring; infer role from symbols/imports. | ? | health:16, health_notice:28, deep_health:51, ibkr_health:64 | health: @router.get('/health')
health_notice: @router.get('/health/notice')
deep_health: @router.get('/health/deep')
ibkr_health: @router.get('/health/ibkr') | IBKR/broker/data dependency |
| backend/tradeo/routers/reports.py | 23 | No module docstring; infer role from symbols/imports. | ? | generate_report:14, latest_report:19 | generate_report: @router.post('/generate')
latest_report: @router.get('/latest') | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/routers/self_improvement.py | 20 | No module docstring; infer role from symbols/imports. | ? | run_self_improvement:15 | run_self_improvement: @router.post('/run', response_model=SelfImprovementResponse) | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/routers/signals.py | 47 | No module docstring; infer role from symbols/imports. | ? | list_signals:16, human_approve_signal:21, paper_execute_signal:39 | list_signals: @router.get("", response_model=list[SignalOut])
human_approve_signal: @router.post('/{signal_id}/human-approve', response_model=SignalOut)
paper_execute_signal: @router.post('/{signal_id}/paper-execute', response_model=TradeOut) | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/entry_variants.py | 297 | No module docstring; infer role from symbols/imports. | ? | build_entry_audit_context:13, classify_regime:29, build_entry_variants:61 | ? | wall-clock timestamp usage |
| backend/tradeo/services/ibkr_broker.py | 454 | No module docstring; infer role from symbols/imports. | IBKRSafetyError:17, IBKRBroker:38 | ? | ? | broad `except Exception` present; wall-clock timestamp usage; RNG path; verify seed/control; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/services/metrics.py | 35 | No module docstring; infer role from symbols/imports. | ? | refresh_pattern_metrics:9 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/ml_scorer.py | 50 | No module docstring; infer role from symbols/imports. | FeatureScorer:10 | ? | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/module_dashboard.py | 535 | No module docstring; infer role from symbols/imports. | ? | module_overview:24 | ? | wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/services/openai_supervisor.py | 84 | No module docstring; infer role from symbols/imports. | OpenAISupervisor:11 | ? | ? | broad `except Exception` present |
| backend/tradeo/services/opportunity_ranking.py | 164 | No module docstring; infer role from symbols/imports. | ? | rank_entry_matches:8 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/paper_broker.py | 58 | No module docstring; infer role from symbols/imports. | PaperBroker:11 | ? | ? | wall-clock timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/services/pattern_detector.py | 270 | No module docstring; infer role from symbols/imports. | CupDetectorParams:17, CupPatternDetector:32 | ? | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/production_manifest.py | 4 | Backward-compatible imports for FoxHunter production manifests. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/scanner.py | 90 | No module docstring; infer role from symbols/imports. | MarketScanner:17 | ? | ? | broad `except Exception` present |
| backend/tradeo/services/signal_quality.py | 227 | No module docstring; infer role from symbols/imports. | ? | build_entry_quality:9, build_signal_snapshot:99 | ? | wall-clock timestamp usage |
| backend/tradeo/services/state_policy.py | 26 | No module docstring; infer role from symbols/imports. | ? | is_legacy_promotion_state:23 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/strategy_config.py | 19 | No module docstring; infer role from symbols/imports. | ? | load_strategy_config:8 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/supervisor.py | 102 | No module docstring; infer role from symbols/imports. | TradeSupervisor:12 | ? | ? | JSON metadata contract; schema drift risk |
| backend/tradeo/services/technical_indicators.py | 89 | No module docstring; infer role from symbols/imports. | ? | sma:7, atr:11, max_drawdown_pct:22, normalize_ohlc:31, validate_ohlc:55 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/services/vision_scorer.py | 75 | No module docstring; infer role from symbols/imports. | VisionGeometryScorer:11 | render_pattern_chart:46 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tasks/__init__.py | 1 | No module docstring; infer role from symbols/imports. | ? | ? | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_autonomous_research_director.py | 133 | No module docstring; infer role from symbols/imports. | ? | session_factory:14, test_research_director_enriches_patterns_and_writes_artifacts:98, test_research_director_latest_artifact_is_reusable_json:123 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_autonomous_research_lab.py | 281 | No module docstring; infer role from symbols/imports. | ? | test_market_replay_penalizes_latency_and_partial_fills:118, test_validation_gate_rejects_adversarial_hard_failure:134, test_research_director_writes_memory_graph_papers_and_agenda:179, test_hypothesis_engine_emits_immutable_package:252 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_domain_module_facades.py | 53 | No module docstring; infer role from symbols/imports. | ? | test_laboratory_scanner_uses_laboratory_module:7, test_fox_hunter_scanner_uses_fox_hunter_module:31 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_lab_diagnostics.py | 278 | No module docstring; infer role from symbols/imports. | ? | session_factory:23, add_pattern:29, add_closed_paper_trade:55, add_closed_shadow_observation:112, test_laboratory_diagnostics_combines_candidates_rejections_and_paper_history:165 | ? | IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/tests/test_market_regime.py | 169 | No module docstring; infer role from symbols/imports. | ? | test_trend_regime_bull_and_bear:37, test_trend_requires_full_sma_window:50, test_vol_tercile_high_after_calm_history:59,

test_vol_tercile_low_after_turbulent_history:67, test_regime_labels_are_point_in_time_stable:74,
test_regime_at_respects_as_of_cutoff:94, test_regime_at_empty_frame_is_honest:104,
test_regime_key_combines_trend_and_vol:112, test_market_regime_service_uses_provider_and_settings:127,
test_market_regime_service_survives_provider_failure:151 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_market_session.py | 32 | No module docstring; infer role from symbols/imports. | ? |
test_us_equity_regular_session_is_open_during_weekday_rth:12, test_us_equity_regular_session_is_closed_after_hours:16,
test_market_session_status_reports_closed_state:20, test_market_session_closes_on_nyse_holiday:27 | ? | no obvious heuristic risk;
still review logic/tests |
| backend/tradeo/tests/test_module_dashboard.py | 414 | No module docstring; infer role from symbols/imports. | ? | session_factory:13,
test_legacy_ibkr_paper_research_trades_are_laboratory_history:19, test_module_overview_exposes_dashboard_data_scope:67,
test_laboratory_overview_api_exposes_default_dashboard_data_scope:81,
test_legacy_ibkr_paper_smoke_order_is_laboratory_history:93, test_cancelled_paper_trade_is_signal_status_not_operation:139,
test_closed_trade_keeps_signal_executed_and_operation_closed:186,
test_unsubmitted_signal_keeps_approval_status_for_module_dashboard:241,
test_approved_signal_without_trade_is_retryable_for_module_dashboard:290,
test_expired_signal_is_discarded_not_retried_for_module_dashboard:321 | ? | IBKR/broker/data dependency; JSON metadata contract;
schema drift risk |
| backend/tradeo/tests/test_novel_pattern_registry.py | 123 | No module docstring; infer role from symbols/imports. | ? |
session_factory:14, test_registry_dedupes_similar_centroids:52, test_matcher_scales_legacy_centroid_prefix:77,
test_registry_persists_confirmation_lifecycle:89, test_registry_blocks_legacy_promotion_states:113 | ? | no obvious heuristic risk; still
review logic/tests |
| backend/tradeo/tests/test_pattern_detector.py | 11 | No module docstring; infer role from symbols/imports. | ? |
test_detector_runs_without_crash:8 | ? | no obvious heuristic risk; still review logic/tests |
| backend/tradeo/tests/test_research_lab_fox_lifecycle.py | 285 | No module docstring; infer role from symbols/imports. |
FixtureProvider:26 | session_factory:43, settings:49, add_research_approved_pattern:71, close_lab_trade:127,
test_research_to_lab_to_director_to_fox_lifecycle:174 | ? | IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/tests/test_risk.py | 134 | No module docstring; infer role from symbols/imports. | ? |
test_position_size_respects_30_usd_risk_budget:15, test_rejects_reward_risk_below_four:34,
test_position_size_respects_adv_participation_cap:53, test_rejects_when_pattern_family_position_cap_reached:76 | ? | wall-clock
timestamp usage; IBKR/broker/data dependency; JSON metadata contract; schema drift risk |
| backend/tradeo/tests/test_runtime_status.py | 36 | No module docstring; infer role from symbols/imports. | ? |
test_entry_scan_status_accumulates_symbols:7, test_entry_scan_status_keeps_market_closed_reason:19 | ? | no obvious heuristic
risk; still review logic/tests |
| backend/tradeo/tests/test_watchdog.py | 35 | No module docstring; infer role from symbols/imports. | ? |
test_watchdog_closes_stale_running_discovery_run:13 | ? | wall-clock timestamp usage |

Secondary Non-Python Inventory

| path | kind | lines | summary / first lines | risk |
| --- | --- | --- | --- | --- |
|.dockerignore | text | 11 |
.git
.env
reports
artifacts
backups
frontend/node_modules
frontend/.next
backend/.venv | secondary
frontend/config asset |
|.env.example | text | 201 | # Tradeo local/VPN
configuration
TRADEO_APP_NAME=Tradeo
TRADEO_ENVIRONMENT=local
TRADEO_DATABASE_URL=postgresql+ps
ycpg://tradeo:tradeo@db:5432/tradeo
TRADEO_REDIS_URL=redis://redis:6379/0
TRADEO_SECRET_KEY=replace-with-a-lo
ng-random-string
TRADEO_ADMIN_USERNAME=admin
TRADEO_ADMIN_PASSWORD=replace-this-password
| configuration/ops surface; verify defaults and secret handling |
|.github/workflows/ci.yml | yaml | 45 | name: ci

on:
 push:
 branches: [main]
 pull_request:

workflow_dispatch:
 | configuration/ops surface; verify defaults and secret handling |
|.github/workflows/director-audit.yml | yaml | 42 | name: Director audit bridge

on:
 pull_request:
 paths:
 -
"research/audit_bridge/*"
 - ".github/workflows/director-audit.yml"
 schedule: | configuration/ops surface; verify defaults and
secret handling |
|.gitignore | text | 63 | .env
.env.*
!.env.example
*.key
*.pem
*.crt
*.p12
*.pfx | secondary frontend/config
asset |
| Makefile | text | 52 | -include .env
export

.PHONY: setup up down logs ps test scan report self-improve discover-patterns
match-discovered-patterns current-matches research-runs research-director research-director-latest

setup:
 cp -n
.env.example .env \\| true
 mkdir -p reports artifacts | configuration/ops surface; verify defaults and secret handling |
| backend/Dockerfile | text | 24 | FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1

PYTHONUNBUFFERED=1
 MPLBACKEND=Agg

WORKDIR /app
 | secondary frontend/config asset |
| backend/pyproject.toml | toml | 45 | [project]
name = "tradeo-backend"
version = "0.1.0"
description = "Tradeo automated
pattern research, paper trading and supervised execution backend"
requires-python = ">=3.11"
dependencies = [

"fastapi">=0.115.0,
 "uvicorn[standard]>=0.30.0", | configuration/ops surface; verify defaults and secret handling |
| config/strategy_cup_v0.json | json | 25 | {
 "name": "cup_v0",
 "pattern": "mid_small_cap_cup_breakout",
 "description":
"Technical cup/base breakout detector for USA mid/small-cap equities. Paper-first; 4R minimum target."
 "target_r_multiple":
4.0,
 "minBars": 45,
 "maxBars": 210,
 "minDepth": 0.10, | configuration/ops surface; verify defaults and secret handling
|
| docker-compose.audit.yml | yaml | 8 | services:
 backend:
 volumes:
 - ./research:/app/research:ro

worker:
 volumes:
 - ./research:/app/research:ro | configuration/ops surface; verify defaults and secret handling |
| docker-compose.yml | yaml | 94 | services:
 db:
 image: postgres:16-alpine
 container_name: tradeo-db
 restart:
unless-stopped
 environment:
 POSTGRES_USER: tradeo
 POSTGRES_PASSWORD: tradeo | configuration/ops

surface; verify defaults and secret handling |
 | frontend/Dockerfile | text | 21 | FROM node:22-alpine AS deps
WORKDIR /app
COPY frontend/package.json
 ./package.json
RUN npm install

FROM node:22-alpine AS builder
WORKDIR /app
COPY --from=deps
 /app/node_modules ./node_modules | secondary frontend/config asset |
 | frontend/app/api/tradeo/[...path]/route.ts | typescript | 47 | import { NextRequest, NextResponse } from 'next/server'

const
 apiBase = process.env.TRADEO_API_INTERNAL_URL \\| 'http://backend:8000/api'
const username =
 process.env.TRADEO_ADMIN_USERNAME \\| 'admin'
const password = process.env.TRADEO_ADMIN_PASSWORD \\|
 'change-me'

function authHeader() {
 return `Basic \${Buffer.from(`\${username}:\${password}`).toString('base64')}` |
 secondary frontend/config asset |
 | frontend/app/globals.css | css | 282 | :root {
 --bg: #070b12;
 --panel: rgba(255, 255, 255, 0.065);
 --panel-strong:
 rgba(255, 255, 255, 0.11);
 --border: rgba(255, 255, 255, 0.14);
 --text: #edf3ff;
 --muted: #9ca9bd;
 --accent: #7dd3fc;
 | secondary frontend/config asset |
 | frontend/app/layout.tsx | tsx | 15 | import './globals.css'
import type { Metadata } from 'next'

export const metadata:
 Metadata = {
 title: 'Tradeo',
 description: 'Private automated pattern research dashboard'
}
 | secondary frontend/config
 asset |
 | frontend/app/page.tsx | tsx | 1031 | Path included in tree only. | secondary frontend/config asset |
 | frontend/lib/api.ts | typescript | 19 | export const API_URL = '/api/tradeo'

export async function fetcher(path: string) {
 const
 clean = path.replace(/^(\/|"/>

Final Instructions For Fable Max

Return a direct external audit, not a rewritten version of this dossier. Use the source blocks and docs above as evidence. Prefer precise findings over broad advice.

Mandatory final output sections:

1. Verdict.
2. Findings table with `id`, `severity`, `confidence`, `area`, `file/path`, `line\_or\_block`, `evidence`, `impact`, `recommended\_fix`, `tests\_to\_add`.
3. End-to-end trace risks.
4. Overclaim/stale documentation review.
5. Honest open gaps that should remain documented.
6. Patch plan ordered by risk reduction.
7. Questions for Asier only where a policy or product decision is required.

Do not assume the pending PIT/Nasdaq target spec is implemented. Treat it as design input for review, not completed evidence. Do not assume live trading is safe. Do not assume any edge is proven. If a claim cannot be supported by code/tests/docs in this pack, mark it as unsupported.